C

K19G

2026-11-06

Table of contents

C Programming **76**
Who this is for . . . 76
How to use this book . . . 76
How to compile examples . . . 77
Workbook (`content/99-workbook`) . . . 77
Related books . . . 77
Modules (15 + workbook + appendices) . . . 78
Prerequisites . . . 78
Getting started . . . 79

Intro **80**

Comprehensive C Programming Syllabus **81**
Course Overview . . . 81
Learning Objectives . . . 81
Prerequisites . . . 81
Course Structure . . . 81
Module 1: Foundations and Environment Setup . . . 81
1.1 Introduction to C Programming . . . 82
1.2 Development Environment Setup . . . 82
1.3 First C Program . . . 82
1.4 Basic I/O . . . 82
Module 2: Data Types and Variables . . . 83
2.1 Fundamental Data Types . . . 83
2.2 Variables and Constants . . . 83
2.3 Type Conversions . . . 83
2.4 Operators and Expressions . . . 83
Module 3: Control Flow . . . 83
3.1 Conditional Statements . . . 84
3.2 Loops . . . 84
3.3 Program Flow Control . . . 84
Module 4: Functions and Modular Programming . . . 84
4.1 Function Fundamentals . . . 84
4.2 Advanced Function Concepts . . . 85
4.3 Modular Programming . . . 85
4.4 Standard Library Functions . . . 85
Module 5: Arrays and Strings . . . 85
5.1 Arrays . . . 85
5.2 Strings . . . 86

5.3 Advanced Array Concepts . . . 86
Module 6: Pointers and Memory Management . . . 86
6.1 Pointer Fundamentals . . . 86
6.2 Pointers and Arrays . . . 86
6.3 Dynamic Memory Management . . . 87
6.4 Advanced Pointer Concepts . . . 87
6.5 Common Pointer Pitfalls . . . 87
Module 7: Structures and User-Defined Types . . . 87
7.1 Structures . . . 87
7.2 Unions and Enumerations . . . 87
7.3 Advanced Structure Concepts . . . 88
7.4 Abstract Data Types . . . 88
Module 8: File I/O and System Programming . . . 88
8.1 File Operations . . . 88
8.2 File I/O Functions . . . 88
8.3 Advanced File Operations . . . 88
8.4 System-Level Programming . . . 89
Module 9: Advanced C Features and Modern Standards . . . 89
9.1 C99 Features . . . 89
9.2 C11 Features . . . 89
9.3 C17 and C23 Features . . . 89
9.4 Modern C Development Practices (2024) . . . 90
9.5 Advanced Preprocessor . . . 90
Module 10: Data Structures and Algorithms . . . 91
10.1 Linear Data Structures . . . 91
10.2 Tree Structures . . . 91
10.3 Hash Tables . . . 91
10.4 Graph Algorithms . . . 91
10.5 Sorting and Searching . . . 91
Module 11: Network Programming . . . 92
11.1 Network Fundamentals . . . 92
11.2 Socket Programming . . . 92
11.3 Advanced Network Programming . . . 92
11.4 Network Protocols . . . 92
11.5 Security Considerations . . . 92
Module 12: Embedded Systems Programming . . . 93
12.1 Embedded Systems Basics . . . 93
12.2 Low-Level Programming . . . 93
12.3 Embedded C Techniques . . . 93
12.4 Real-Time Programming . . . 93
12.5 Hardware Interfaces . . . 93
Module 13: Performance Optimization and Profiling . . . 94
13.1 Performance Analysis . . . 94
13.2 Optimization Techniques . . . 94
13.3 Memory Optimization . . . 94
13.4 Advanced Optimization . . . 94
Module 14: Testing and Debugging . . . 94
14.1 Testing Methodologies . . . 95

14.2 Debugging Techniques . . . 95
14.3 Error Handling . . . 95
14.4 Code Quality . . . 95
Module 15: Advanced Topics and Capstone . . . 95
15.1 Multi-threading and Concurrency . . . 95
15.2 Advanced System Programming . . . 96
15.3 Domain-Specific Applications . . . 96
15.4 Capstone Project . . . 96
Assessment Methods . . . 96
Continuous Assessment (60%) . . . 96
Major Projects (25%) . . . 96
Examinations (15%) . . . 97
Recommended Resources . . . 97
Essential Books . . . 97
Advanced and Specialized References . . . 97
Current Standards and Specifications . . . 97
Modern Online Resources (2024) . . . 97
Development Tools (2024 Current Versions) . . . 98
Embedded and IoT Development (Current) . . . 98
Security and Best Practices . . . 98
Course Policies . . . 98
Prerequisites Reinforcement . . . 98
Academic Integrity . . . 98
Accessibility and Support . . . 99
Learning Outcomes Verification . . . 99

Foundations **100**

Introduction to C Programming **101**
What is C? . . . 101
A short history (enough to orient you) . . . 101
Why learn C? . . . 101
From source to process: how a C program runs . . . 102
What each stage does . . . 103
Watching the stages with GCC . . . 103
Memory model intro: text, data, stack, heap . . . 104
Hello, World — anatomy line by line . . . 105
Small but important details . . . 106
Compile and run . . . 106
Full mini-programs . . . 106
1. Temperature converter (°C °F) . . . 106
2. Maximum of three integers . . . 107
3. Simple calculator CLI . . . 108
4. Exit status practice . . . 109
Common misconceptions . . . 110
“C is basically the same as C++” . . . 110
“If it compiles, it is correct” . . . 110

"The compiler is just a translator; any compiler is fine" . . . 111
"I need to master every standard library function first" . . . 111
"`void main()` is fine" . . . 111
How this book is organized . . . 111
What to practice first . . . 112
Applications of C (why the language stays relevant) . . . 112
Summary . . . 113
Exercises . . . 113

Development Environment Setup **115**
Introduction . . . 115
Installing Compilers . . . 115
GCC (GNU Compiler Collection) . . . 115
Clang . . . 116
Microsoft Visual C++ (MSVC) . . . 116
Modern IDEs and Editors . . . 117
Visual Studio Code . . . 117
CLion . . . 118
Code::Blocks . . . 119
Command-Line Tools . . . 119
Make . . . 119
CMake . . . 119
Git . . . 120
Debugging Tools . . . 120
GDB (GNU Debugger) . . . 120
Valgrind . . . 121
AddressSanitizer . . . 121
Package Management . . . 122
vcpkg . . . 122
Conan . . . 122
Cloud Development Environments . . . 122
GitHub Codespaces . . . 122
Gitpod . . . 123
Cross-Platform Development . . . 123
Windows Subsystem for Linux (WSL) . . . 123
Docker . . . 123
Real Workflows: From One File to a Small Project . . . 124
Single-file recipe . . . 124
Multi-file build recipes . . . 125
Full program: tiny CLI calculator (single file) . . . 126
GCC vs Clang (What Actually Differs) . . . 128
Side-by-side on the same bug . . . 128
Useful shared flags . . . 128
macOS note . . . 129
Make: Multi-File Builds Without Typing Everything . . . 129
Minimal `Makefile` for the util/main project . . . 129
Full program + Make: word count of argv strings . . . 130

CMake Mini-Project (Practical One-Pager) . . . 130
Configure, build, run . . . 131
Optional: compile_commands.json for editors . . . 131
Editor + Debugger One-Pager . . . 132
Terminal editor + GDB (always works) . . . 132
LLDB (macOS default) . . . 132
VS Code (minimum useful setup) . . . 133
GDB against a real bug (complete mini-lab) . . . 133
Sanitizers and Memory Tools (Environment Checklist) . . . 134
Modern Development Practices . . . 134
Version Control Workflow . . . 134
Project Structure . . . 135
Continuous Integration . . . 135
Verify Your Setup (Smoke Tests) . . . 135
Exercises . . . 136
Summary . . . 137

Your First C Program **138**
Introduction . . . 138
The classic "Hello, World!" . . . 138
Line-by-line . . . 138
Create, compile, run . . . 139
What the compiler does (short version) . . . 139
Structure of a C program . . . 140
Comments . . . 140
More complete single-file examples . . . 141
Greeting with command-line arguments . . . 141
Sum of two integers (with validation) . . . 141
Multiple helpers in one file . . . 142
Print a small banner . . . 142
Multi-file tiny program . . . 142
Build it . . . 143
Why the include guard? . . . 144
Header vs source rules . . . 144
Common compile errors (with explanations) . . . 144
Missing semicolon . . . 144
Missing `#include <stdio.h>` . . . 144
Wrong main spelling or signature . . . 145
Case sensitivity . . . 145
Undeclared function (multi-file) . . . 145
Multiple definition of a function . . . 145
Forgetting to link a file . . . 145
Format / type mismatch (warning) . . . 145
Using `=` instead of `==` in a condition (logic bug; may still compile) . . . 146
Unterminated string . . . 146
Tips for reading diagnostics . . . 146
Lab A — Personal info printer . . . 146
Lab B — Unit converter . . . 147

Variations worth knowing . . . 149
Implicit `return 0` in `main` (C99+) . . . 149
Prototype before `main` . . . 149
Using `stderr` for errors . . . 149
Beginner mistakes checklist . . . 150
Summary . . . 150
Exercises . . . 150

Basic Input and Output **152**
Introduction . . . 152
The `<stdio.h>` library . . . 152
Output with `printf` . . . 153
Prototype (conceptual) . . . 153
Minimal example . . . 153
Format specifier table . . . 153
Runnable demo — mixed types . . . 154
Width, precision, flags . . . 154
`fprintf` and streams . . . 155
`snprintf` — safe string formatting . . . 155
Input with `scanf` . . . 156
Prototype (conceptual) . . . 156
Address operator . . . 156
Specifier table for `scanf` (common cases) . . . 156
Full demo — reading several fields . . . 157
Buffer caveats with `scanf` . . . 158
Leftover newlines . . . 158
`%s` stops at whitespace . . . 158
Always bound string width . . . 158
Failed conversions leave bad data . . . 159
Clearing the rest of a line . . . 159
Character I/O: `getchar` and `putchar` . . . 159
Echo a character . . . 160
Copy stdin to stdout until EOF . . . 160
Count characters and lines . . . 160
Reading lines safely with `fgets` . . . 161
Prototype . . . 161
Basic pattern . . . 161
Why prefer `fgets` for text lines? . . . 162
Mixing `fgets` and `scanf` . . . 162
Detecting truncated lines . . . 163
`sscanf` — parse from a string . . . 164
Checking return values . . . 164
`printf` . . . 164
`scanf` . . . 165
Mini-project 1 — Menu-driven CLI . . . 165
Mini-project 2 — Simple invoice printer . . . 167
Practical calculator (formatted I/O review) . . . 169
Best practices (quick reference) . . . 170

Summary . . . 170
Exercises . . . 170

Module 1: Foundations and Environment Setup **172**
Overview . . . 172
Learning goals . . . 172
Chapters (map) . . . 172
Suggested path . . . 173
Tooling cheat sheet . . . 173
Prerequisites . . . 173
Self-check before Module 2 . . . 174
Estimated time . . . 174
Next module . . . 174
Resources . . . 174

Data Types 175

Fundamental Data Types **176**
Introduction . . . 176
Basic Data Types in C . . . 176
Integer Types . . . 176
Typical Sizes (LP64 Linux, e.g. x86_64) . . . 176
Unsigned Variants . . . 177
Worked Demo: sizeof of Major Types . . . 177
Limits from `limits.h` . . . 178
Floating-Point Types and `float.h` . . . 179
Classic Precision Surprise . . . 180
Character Types . . . 181
Boolean Type (C99+) . . . 181
Fixed-Width Types (`stdint.h`) — Prefer These When Width Matters . . . 182
Exact-Width . . . 182
Printing Portably: `inttypes.h` . . . 182
When to Use What . . . 183
Example: Portable Binary Header . . . 183
Signed Overflow Is Undefined Behavior . . . 185
Safer Patterns Summary . . . 187
Type Specifiers and Qualifiers . . . 187
Type Promotion (Preview) . . . 188
Practical Combined Explorer . . . 189
Best Practices . . . 189
Exercises . . . 190
Exercise 1 — ABI report . . . 190
Exercise 2 — limits-driven clamp . . . 190
Exercise 3 — safe multiply . . . 190
Exercise 4 — protocol fields . . . 190
Exercise 5 — float.h scavenger . . . 190
Exercise 6 — UB sanitizer lab . . . 191

Summary . . . 191

Variables and Constants **192**
Introduction . . . 192
Variables . . . 192
What is a Variable? . . . 192
Variable Declaration . . . 192
Variable Initialization . . . 192
Variable Assignment . . . 193
Variable Naming Rules . . . 193
Variable Naming Conventions . . . 194
Constants . . . 194
Literal Constants . . . 194
#define Preprocessor Directive . . . 195
const Qualifier . . . 195
enum (Enumeration) . . . 196
Storage Classes . . . 196
auto . . . 196
register . . . 197
static . . . 197
extern . . . 198
Scope and Lifetime . . . 199
Scope . . . 199
Lifetime . . . 200
Type Qualifiers . . . 200
const . . . 200
volatile . . . 201
restrict (C99+) . . . 201
Practical Examples . . . 202
Variable Declaration and Usage . . . 202
Constants Comparison . . . 202
Storage Class Demonstration . . . 203
Best Practices . . . 204
1. Initialize Variables . . . 204
2. Use Meaningful Names . . . 204
3. Use const for Fixed Values . . . 204
4. Limit Global Variables . . . 205
5. Use Appropriate Storage Classes . . . 205
Summary . . . 205

Type Conversions **206**
Introduction . . . 206
Implicit Type Conversions (Coercion) . . . 206
Usual Arithmetic Conversions . . . 206
Integer Promotion . . . 207
Conversion Hierarchy . . . 207
Boolean Context Conversions . . . 208

Explicit Type Conversions (Casting) . . . 209
Cast Operator Syntax . . . 209
Common Casting Scenarios . . . 209
Pointer Casting . . . 210
Potential Pitfalls and Data Loss . . . 211
Truncation . . . 211
Overflow . . . 211
Precision Loss . . . 212
Safe Conversion Practices . . . 212
Checking for Overflow . . . 212
Using Standard Library Functions . . . 213
Conversion Functions . . . 213
Mathematical Conversions . . . 213
String Conversions . . . 214
Practical Examples . . . 214
Temperature Converter with Type Safety . . . 214
Data Type Range Checker . . . 215
Mixed Type Calculator . . . 216
Best Practices . . . 217
1. Be Explicit About Conversions . . . 217
2. Check for Data Loss . . . 217
3. Use Appropriate Functions for String Conversion . . . 217
4. Understand Truncation vs Rounding . . . 218
5. Be Careful with Signed/Unsigned Conversions . . . 218
Summary . . . 218

Hexadecimal Floating Literals and %a: Exact Floats in C 219

Hexadecimal Floating Literals and %a: Exact Floats in C 220
Why Decimal Literals Lie . . . 220
The Hexadecimal Floating Literal Syntax . . . 220
Printing Exact Values with `%a` . . . 221
Round-Tripping Floats Losslessly . . . 221
Reading IEEE 754 Binary64 Through Hexadecimal . . . 222
Common Floating-Point Constants in Hex . . . 222
Subnormal Numbers . . . 223
Deconstructing Floats with `frexp` and `ldexp` . . . 223
Complete Lab Walk-Through . . . 223
Pitfalls and Common Mistakes . . . 225
1. Missing the Binary Exponent `p` . . . 225
2. Assuming `p` Represents Base-16 Power . . . 226
3. Floating-Point Promotion in `printf` . . . 226
4. Mismatched Specifiers for `long double` . . . 226
5. Locale Specificity with Decimal Points . . . 226
Key Takeaways . . . 226
Sources . . . 227

Operators and Expressions **228**
Introduction . . . 228
Categories of Operators . . . 228
Arithmetic Operators . . . 228
Basic Arithmetic Operations . . . 229
Division and Modulus Behavior . . . 229
Relational Operators . . . 230
Relational Operations . . . 230
Logical Operators . . . 230
Logical Operations . . . 231
Bitwise Operators . . . 231
Bitwise Operations . . . 232
Assignment Operators . . . 233
Assignment Operations . . . 233
Increment and Decrement Operators . . . 234
Increment/Decrement Examples . . . 234
Conditional (Ternary) Operator . . . 235
Conditional Operator Examples . . . 235
Special Operators . . . 236
sizeof Operator . . . 236
Address-of (&) and Dereference (*) Operators . . . 236
Operator Precedence and Associativity . . . 237
Precedence Levels (Highest to Lowest) . . . 237
Precedence Examples . . . 237
Practical Examples . . . 238
Mathematical Expression Evaluator . . . 238
Bit Manipulation Utility . . . 239
Conditional Logic Demonstrator . . . 240
Best Practices . . . 241
1. Use Parentheses for Clarity . . . 241
2. Be Careful with Side Effects . . . 241
3. Use Meaningful Variable Names in Complex Expressions . . . 241
4. Be Aware of Integer Division . . . 242
5. Use Bitwise Operations for Bit Manipulation . . . 242
Summary . . . 242

Module 2: Data Types and Variables **244**
Overview . . . 244
Learning Objectives . . . 244
Chapters . . . 244
Key Concepts Covered . . . 244
Prerequisites . . . 245
Tools and Technologies . . . 245
Estimated Time to Complete . . . 245
Assessment . . . 245
Next Module . . . 245
Additional Resources . . . 246

Control Flow **247**

Conditional Statements **248**
Introduction . . . 248
The if Statement . . . 248
Basic Syntax . . . 248
Simple if Statement . . . 248
if-else Statement . . . 249
if-else if-else Chain . . . 249
The switch Statement . . . 250
Basic Syntax . . . 250
Simple switch Example . . . 250
Intentional Fall-Through . . . 251
Nested Conditionals . . . 252
Nested Condition Pitfalls . . . 252
The Dangling else (Else-Bind Ambiguity) . . . 253
Conditional (Ternary) Expressions . . . 254
Ternary Best Practices . . . 255
Best Practices for Conditionals . . . 256
1. Use Braces Consistently . . . 256
2. Handle All Cases in switch . . . 256
3. Prefer Early Returns Over Deep Nesting . . . 257
4. Name Complex Conditions . . . 257
Common Pitfalls . . . 257
1. Assignment vs Comparison . . . 257
2. Floating-Point Comparisons . . . 258
3. Missing break in switch . . . 258
Worked Example 1: Full Grading System . . . 258
Worked Example 2: Leap Year (Rules + Nested Form) . . . 260
Worked Example 3: Simple ATM Menu with switch . . . 262
Short-Circuit Evaluation . . . 264
Exercises . . . 265
Exercise 1 — Triangle classifier . . . 265
Exercise 2 — Rock–paper–scissors . . . 266
Exercise 3 — BMI category . . . 266
Exercise 4 — Rewrite dangling-else trap . . . 266
Exercise 5 — ATM extension . . . 266
Summary . . . 266

Loops **268**
Introduction . . . 268
The while Loop . . . 268
Basic Syntax . . . 268
Simple while Loop . . . 268
while with User Input . . . 269
The do-while Loop . . . 269
Basic Syntax . . . 270
Simple do-while . . . 270

Menu Pattern (classic do-while use) . . . 270
The for Loop . . . 271
Basic Syntax . . . 271
Simple for Loop . . . 271
for with Arrays . . . 271
Decision Guide: for vs while vs do-while . . . 272
Same Task Three Ways . . . 272
When while Beats for . . . 273
Loop Control: break and continue . . . 274
break . . . 274
continue . . . 274
Nested Loops . . . 275
Multiplication Table Pattern . . . 275
Nested break/continue Scope . . . 275
Nested Loop Patterns Cheatsheet . . . 276
Infinite Loops (Servers, Embedded, Tools) . . . 277
Forms . . . 277
Pattern: Event Loop with break Conditions . . . 277
Pattern: Embedded-Style Superloop . . . 278
Pattern: Retry with Cap (avoid true infinite hang) . . . 279
Worked Example 1: Prime Check . . . 280
Worked Example 2: FizzBuzz . . . 281
Worked Example 3: Multiplication Table . . . 282
Worked Example 4: Running Average from Input . . . 283
Performance Notes . . . 284
Best Practices . . . 285
Common Pitfalls . . . 285
Infinite loop by forgetting the update . . . 285
Off-by-one . . . 285
Floating loop variable . . . 286
More Practical Examples . . . 286
Number Patterns . . . 286
Array Stats and Reverse . . . 287
Exercises . . . 288
Exercise 1 — Collatz steps . . . 288
Exercise 2 — Nested loop: prime pairs . . . 288
Exercise 3 — Choose the right loop . . . 288
Exercise 4 — FizzBuzz customization . . . 288
Exercise 5 — Server-style loop . . . 288
Exercise 6 — Running variance (optional stretch) . . . 289
Summary . . . 289

Advanced Control Flow **290**
Introduction . . . 290
The goto Statement . . . 290
Basic Syntax . . . 290
Simple goto Example . . . 290
goto for Error Handling . . . 291

goto in Switch Statements . . . 292
Function Calls as Control Flow . . . 293
Basic Function Calls . . . 293
Recursive Function Calls . . . 293
Function Pointers for Dynamic Control Flow . . . 294
Error Handling Patterns . . . 295
Return Code Pattern . . . 295
errno and perror for System Errors . . . 296
Setjmp and Longjmp for Non-Local Jumps . . . 297
Advanced Loop Control Techniques . . . 298
Loop Labels and Break to Outer Loops . . . 298
State Machines . . . 298
Best Practices for Advanced Control Flow . . . 300
1. Use goto Sparingly and Structured . . . 300
2. Avoid Deep Nesting . . . 300
3. Use Function Pointers for Strategy Pattern . . . 301
Common Pitfalls and How to Avoid Them . . . 302
1. goto Leading to Spaghetti Code . . . 302
2. Infinite Recursion . . . 302
3. Longjmp Across Function Boundaries . . . 303
Practical Examples . . . 304
Resource Management with goto . . . 304
Command-Line Parser with State Machine . . . 305
Complex Error Handling System . . . 307
Summary . . . 309

Module 3: Control Flow **310**
Overview . . . 310
Learning Objectives . . . 310
Chapters . . . 310
Key Concepts Covered . . . 310
Prerequisites . . . 311
Tools and Technologies . . . 311
Estimated Time to Complete . . . 311
Assessment . . . 311
Next Module . . . 311
Additional Resources . . . 311

Functions **312**

Function Fundamentals **313**
Introduction . . . 313
What is a Function? . . . 313
Function Definition . . . 313
Basic Syntax . . . 313
Simple Function Example . . . 314
Function with No Parameters . . . 314

Function with No Return Value (void) . . . 314
Function Declaration (Prototypes) . . . 315
Syntax . . . 315
Function Prototype Example . . . 315
Function Parameters . . . 316
Parameter Passing by Value . . . 316
Multiple Parameters . . . 316
Return Statements . . . 317
Returning Values . . . 317
Early Returns . . . 318
Function Scope and Lifetime . . . 318
Local Variables . . . 319
Static Local Variables . . . 319
Function Naming Conventions . . . 320
Common Naming Styles . . . 320
Best Practices for Functions . . . 320
1. Single Responsibility Principle . . . 320
2. Appropriate Function Size . . . 321
3. Meaningful Parameter Names . . . 322
Common Pitfalls and How to Avoid Them . . . 323
1. Missing Return Statements . . . 323
2. Using Uninitialized Variables . . . 323
3. Function Declaration vs Definition Mismatch . . . 324
Practical Examples . . . 324
Temperature Converter Functions . . . 324
Mathematical Utility Functions . . . 325
String Processing Functions . . . 327
Summary . . . 328

Advanced Functions **329**
Introduction . . . 329
Recursive Functions . . . 329
Basic Recursion Concepts . . . 329
Simple Recursion Example . . . 329
Fibonacci Sequence . . . 330
Recursive Array Processing . . . 330
Variable Argument Functions . . . 331
Basic Variable Argument Functions . . . 331
Formatted Print Function . . . 333
Inline Functions (C99) . . . 334
Basic Inline Functions . . . 334
Performance Considerations . . . 334
Function Pointers . . . 335
Basic Function Pointers . . . 335
Function Pointer Arrays . . . 336
Callback Functions . . . 337
Higher-Order Functions . . . 338
Function Factory Pattern . . . 338

Best Practices for Advanced Functions . . . 339
1. Recursion Best Practices . . . 339
2. Variable Argument Functions Best Practices . . . 340
3. Function Pointer Best Practices . . . 341
Common Pitfalls and How to Avoid Them . . . 342
1. Infinite Recursion . . . 342
2. Variable Argument Function Errors . . . 342
3. Function Pointer Errors . . . 343
Practical Examples . . . 344
Recursive Directory Traversal . . . 344
Mathematical Expression Evaluator . . . 345
Event Handler System . . . 346
Summary . . . 347

Modular Programming **348**
Introduction . . . 348
Header Files and Source Files . . . 348
Header Files (`.h`) . . . 348
Source Files (`.c`) . . . 348
Example Layout . . . 349
Include Guards . . . 349
Traditional Include Guards . . . 349
`#pragma once` (Non-standard, Widely Supported) . . . 349
Compilation Units and Linking . . . 349
Linkage: `extern`, `static`, and No Linkage . . . 350
External Linkage (Default for File-Scope Functions) . . . 350
Internal Linkage (`static` at File Scope) . . . 351
`static` vs `extern` at a Glance . . . 352
No Linkage . . . 352
Complete Multi-Module Project: `string_stats` . . . 352
Project Tree . . . 352
`strstats.h` — Public Interface . . . 353
`strstats.c` — Implementation . . . 353
`main.c` — Driver . . . 355
Manual Build . . . 356
Full Makefile: Debug, Release, and `-MMD` Dependencies . . . 356
Variant with `build/` Directory . . . 357
Creating Libraries . . . 358
Static Library (`.a`) . . . 358
Shared Library (`.so` on Linux / macOS) . . . 358
Static vs Shared . . . 359
Header Design Rules . . . 359
Recommended Include Order . . . 360
Forward Declarations . . . 360
Opaque Pointers / Incomplete Types . . . 361
`vector.h` . . . 361
`vector.c` . . . 361
`main_vector.c` . . . 363

Alternate Multi-Module Example: 2D Vector Math . . . 363
`vec2.h` . . . 364
`vec2.c` . . . 364
`main_vec2.c` . . . 365
Callback Interfaces (Flexible Modules) . . . 365
Best Practices . . . 366
Exercises . . . 366
Summary . . . 367

Standard Library Functions **368**
Introduction . . . 368
String Functions (<string.h>) . . . 368
String Copying . . . 368
String Concatenation . . . 368
String Comparison . . . 369
String Searching . . . 369
String Tokenization . . . 369
String Length . . . 369
Memory Functions (<string.h>) . . . 369
Memory Copying . . . 369
Memory Setting . . . 369
Memory Comparison . . . 370
Memory Searching . . . 370
Mathematical Functions (<math.h>) . . . 370
Basic Mathematical Functions . . . 370
Hyperbolic Functions . . . 371
Floating-Point Functions . . . 371
Classification and Comparison Functions . . . 371
Character Classification (<ctype.h>) . . . 371
Character Testing . . . 371
Character Conversion . . . 372
Time Functions (<time.h>) . . . 372
Time Types . . . 372
Time Functions . . . 372
Input/Output Functions (<stdio.h>) . . . 373
File Operations . . . 373
Formatted I/O . . . 373
Character I/O . . . 373
Direct I/O . . . 373
File Positioning . . . 374
Error Handling . . . 374
Memory Management (<stdlib.h>) . . . 374
Dynamic Memory Allocation . . . 374
Utility Functions . . . 374
Random Number Generation . . . 375
Environment Functions . . . 375
Searching and Sorting . . . 375
Integer Arithmetic . . . 375

Advanced Standard Library Features (C23) . . . 376
New String Functions . . . 376
Enhanced Memory Functions . . . 376
Practical Examples . . . 376
String Manipulation Example . . . 376
Mathematical Functions Example . . . 377
Time Functions Example . . . 378
Memory Management Example . . . 378
Full Programs: Standard Library in Practice . . . 379
Program 1 — Safe string toolbox (`strtool.c`) . . . 379
Program 2 — Math lab: vectors and roots (`mathlab.c`) . . . 382
Program 3 — Character classifier report (`ctype_report.c`) . . . 383
Program 4 — Time stamps and elapsed work (`timelab.c`) . . . 384
Program 5 — `qsort` + `bsearch` + `stdlib` utilities (`sortlab.c`) . . . 385
Program 6 — Mini word frequency with `string.h` + heap (`wfreq.c`) . . . 387
Best Practices for Using Standard Library Functions . . . 389
1. Error Checking . . . 389
2. Buffer Safety . . . 389
3. Memory Management . . . 389
4. Locale Awareness . . . 389
5. Thread Safety . . . 390
6. Link math explicitly . . . 390
Exercises . . . 390
Summary . . . 391

Module 4: Functions and Modular Programming **392**
Overview . . . 392
Learning Objectives . . . 392
Chapters . . . 392
Key Concepts Covered . . . 392
Prerequisites . . . 393
Tools and Technologies . . . 393
Estimated Time to Complete . . . 393
Assessment . . . 393
Next Module . . . 393
Additional Resources . . . 394

Arrays and Strings **395**

Arrays **396**
Introduction . . . 396
Array Declaration and Initialization . . . 396
Basic Declaration . . . 396
Initialization . . . 396
Constant Arrays . . . 397
Access and Indexing . . . 397
No Automatic Bounds Checking . . . 397

Traversal . . . 397
`sizeof` and Array Length . . . 398
Passing Arrays to Functions . . . 398
Equivalent Parameter Declarations . . . 398
Full Program: Size Parameter Pattern . . . 399
Optional: Static Array Size Hints (C99+) . . . 399
Multidimensional Arrays and Row-Major Layout . . . 400
2D Declaration . . . 400
Row-Major Storage . . . 400
Passing 2D Arrays . . . 400
Worked Algorithms (Full Programs) . . . 401
1. Reverse an Array In Place . . . 401
2. Find Minimum and Maximum . . . 402
3. Frequency Count . . . 403
4. Small Matrix Multiply ($2\times3 \cdot 3\times2 \to 2\times2$) . . . 404
5. Histogram of Grades . . . 405
6. Linear Search and Selection Sort (Together) . . . 406
Common Bugs: Wrong vs Right . . . 407
Off-by-One in Loops . . . 407
Forgetting That Arrays Decay . . . 408
Using Uninitialized Local Arrays . . . 408
Copying with `=` . . . 409
Returning a Pointer to a Local Array . . . 409
Practical Examples . . . 409
Temperature Analysis . . . 409
Student Scores (Parallel Arrays) . . . 410
Mini-Lab: Array Toolkit . . . 410
Exercises . . . 411
Summary . . . 412

Strings **413**
Introduction . . . 413
String Representation . . . 413
Character Arrays and Literals . . . 413
Memory Layout . . . 413
Literals Must Not Be Modified . . . 414
Declaration Patterns . . . 414
Standard Library Overview . . . 414
Safe String Handling Patterns . . . 414
Prefer `snprintf` over bare `strcpy` / `strcat` . . . 414
Building Paths or Messages Safely . . . 415
`strncpy` vs `snprintf` — Know the Trap . . . 416
Bounded Copy Helper (Recommended Pattern) . . . 416
Safe Concatenation . . . 417
Input: Prefer `fgets` + Strip Newline . . . 418
Implement `strlen` / `strcpy` / `strcmp` Yourself . . . 418
Full Learning Program . . . 418

Tokenizing and CSV Parsing . . . 420
`strtok` vs `strtok_r` . . . 420
Full Program: CSV Line with `strtok_r` . . . 420
Full Program: Manual CSV Parse (No `strtok`) . . . 421
Simple Key=Value Line . . . 423
Buffer Overflows: Awareness and Safe Alternatives . . . 424
Fragile Pattern (Do Not Ship) . . . 424
Safe Alternatives Checklist . . . 424
Full Program: Safe Line Reader and Copy . . . 425
Compiler Help . . . 426
More String Techniques . . . 426
Remove Newline, Reverse, Case . . . 426
Palindrome Check . . . 427
Word Count . . . 428
Character Class Statistics . . . 428
Pitfalls Recap . . . 429
Exercises . . . 429
Summary . . . 430

Bounded String Copies: strncpy, strlcpy, memccpy and strscpy Compared 431

Bounded String Copies: strncpy, strlcpy, memccpy and strscpy Compared 432
The problem: truncation, termination, and detection . . . 432
The lab . . . 432
Reading the results byte by byte . . . 435
`strncpy` . . . 435
`strlcpy` . . . 436
`memccpy` . . . 436
`snprintf` . . . 436
`strscpy` . . . 436
Function comparison table . . . 436
Choosing the right copy function . . . 437
Compiler warnings help detect misuse . . . 438
Common mistakes . . . 438
1. Passing total buffer size to `strncat` instead of remaining space . . . 438
2. Calling `sizeof` on a pointer parameter . . . 438
3. Ignoring the return value of `strlcpy` or `snprintf` . . . 439
4. Off-by-one errors with `size` versus `size - 1` . . . 439
Key takeaways . . . 439
Sources . . . 440

Advanced Arrays 441
Introduction . . . 441
Multi-dimensional Arrays . . . 441
Two-dimensional Arrays . . . 441
Accessing Multi-dimensional Arrays . . . 441
Multi-dimensional Array Functions . . . 442

Three-dimensional and Higher-dimensional Arrays . . . 443
Three-dimensional Arrays . . . 443
Higher-dimensional Arrays . . . 443
Variable-length Arrays (VLAs) - C99 . . . 443
Basic VLA Declaration . . . 443
Multi-dimensional VLAs . . . 444
VLA Function Parameters . . . 445
Array of Arrays vs. Multi-dimensional Arrays . . . 445
Array of Arrays . . . 445
Multi-dimensional Arrays . . . 446
Jagged Arrays . . . 446
Creating Jagged Arrays . . . 446
Jagged Array Functions . . . 447
Array Algorithms and Operations . . . 448
Matrix Operations . . . 448
Dynamic Array Resizing . . . 449
Array Performance Considerations . . . 449
Memory Layout . . . 449
Cache Performance . . . 450
Advanced Array Techniques . . . 451
Array Pointers . . . 451
Array of Pointers . . . 451
Compound Literals (C99) . . . 451
Practical Examples . . . 452
Sudoku Validator . . . 452
Image Processing Example . . . 453
Summary . . . 455

Length-Prefixed Strings: an sds-Style String Library in Under 60 Lines **456**

Length-Prefixed Strings: an sds-Style String Library in Under 60 Lines **457**
Thesis . . . 457
The whole library . . . 457
It is still a C string, and also binary safe . . . 459
The one rule: always keep the returned pointer . . . 462
When to use which . . . 462
Takeaways . . . 463
Sources . . . 463

Module 5: Arrays and Strings **464**
Overview . . . 464
Learning Objectives . . . 464
Chapters . . . 464
Key Concepts Covered . . . 464
Prerequisites . . . 465
Tools and Technologies . . . 465
Estimated Time to Complete . . . 465
Assessment . . . 465

Next Module . . . 465
Additional Resources . . . 466

Pointers **467**

Pointer Fundamentals **468**
Introduction . . . 468
What Is a Pointer? . . . 468
Visual Step-by-Step: `&` and `*` . . . 468
Step 1 — Create an `int` . . . 468
Step 2 — Point at it . . . 469
Step 3 — Read through the pointer . . . 469
Step 4 — Write through the pointer . . . 469
Full Demo Program . . . 469
Declaration and Initialization . . . 470
Null Pointers . . . 470
Swap via Pointers (Full Program) . . . 471
Why Values Alone Fail . . . 471
Correct Swap with Pointers . . . 471
`const` and Pointers: Three Combinations . . . 472
Full Program . . . 472
Why `const int *` Appears in APIs . . . 473
Note on `const int *` vs `int const *` . . . 473
`void *` with Careful Casting . . . 473
Full Program: Generic Swap of Bytes . . . 473
Rules of Thumb for `void *` . . . 474
Pointer Arithmetic (Brief) . . . 475
Pointers vs Arrays: Equivalence and Differences . . . 475
Where They Look the Same . . . 475
Where They Differ . . . 475
Full Comparison Program . . . 476
Out-Parameters: Return Status Separately from Data . . . 476
Full Program . . . 477
Design Tips . . . 478
Pointers and Functions (More) . . . 479
Returning a Pointer into a Caller-Owned Array . . . 479
Unsafe: Returning Address of Local . . . 479
Pointer Comparison and Size . . . 480
Best Practices . . . 480
Practical Examples . . . 480
Pointer-Based String Reverse . . . 480
Sum with Pointer Walking . . . 481
Exercises . . . 482
Summary . . . 482

Pointers and Arrays **483**
Introduction . . . 483

Array-Pointer Equivalence . . . 483
Arrays as Pointers . . . 483
Important Distinction . . . 483
Pointer Subscripting . . . 484
Using Pointers as Arrays . . . 484
Array Access Methods . . . 484
Pointer Arithmetic with Arrays . . . 485
Incrementing Pointers . . . 485
Pointer Difference . . . 485
Multi-dimensional Arrays and Pointers . . . 485
Two-dimensional Arrays . . . 485
Row-wise Traversal . . . 486
Arrays as Function Parameters . . . 487
Array Decay to Pointers . . . 487
Multi-dimensional Array Parameters . . . 487
Dynamic Arrays with Pointers . . . 488
Allocating Dynamic Arrays . . . 488
Resizing Dynamic Arrays . . . 489
Pointer Arrays vs. Arrays of Pointers . . . 490
Array of Pointers . . . 490
Pointer to Array . . . 491
Advanced Array-Pointer Techniques . . . 491
Pointer Arithmetic for Multi-dimensional Arrays . . . 491
Function Pointers for Array Operations . . . 492
Practical Examples . . . 493
Matrix Transpose Using Pointers . . . 493
String Array Sorting . . . 494
Dynamic 2D Array . . . 494
Summary . . . 496

Dynamic Memory Management **497**
Introduction . . . 497
Memory Layout in C Programs . . . 497
Program Memory Sections . . . 497
Stack vs. Heap . . . 497
Dynamic Memory Allocation Functions . . . 498
malloc() . . . 498
calloc() . . . 498
realloc() . . . 499
free() . . . 500
Memory Allocation Strategies . . . 501
Single Element Allocation . . . 501
Array Allocation . . . 501
2D Array Allocation . . . 502
Memory Management Best Practices . . . 504
Error Checking . . . 504
Preventing Dangling Pointers . . . 504
Avoiding Memory Leaks . . . 504

Memory Reallocation Safety . . . 504
Common Memory Management Patterns . . . 505
Dynamic String Handling . . . 505
Dynamic Array with Resizing . . . 506
Memory Debugging Tools . . . 507
Valgrind (Linux/Unix) . . . 507
AddressSanitizer (GCC/Clang) . . . 508
Static Analysis Tools . . . 508
Common Memory Errors and Solutions . . . 508
Memory Leaks . . . 508
Dangling Pointers . . . 509
Buffer Overflows . . . 509
Advanced Memory Management (C11 and Later) . . . 509
aligned_alloc() . . . 509
Practical Examples . . . 510
Memory Pool Implementation . . . 510
Smart Pointer-like Implementation . . . 513
Summary . . . 514

Advanced Pointer Concepts **515**
Introduction . . . 515
Pointers to Pointers . . . 515
Basic Concept . . . 515
Practical Applications . . . 515
Function Pointers . . . 517
Basic Declaration and Usage . . . 517
Function Pointer Arrays . . . 518
Callback Functions . . . 518
Function Pointers as Parameters . . . 519
Void Pointers . . . 520
Basic Concept . . . 520
Generic Functions with Void Pointers . . . 521
Void Pointer Arrays . . . 522
Const Correctness with Pointers . . . 523
Pointer to Constant . . . 523
Constant Pointer . . . 523
Constant Pointer to Constant . . . 524
Const Correctness in Function Parameters . . . 524
Advanced Pointer Techniques . . . 525
Pointer Arithmetic with Structs . . . 525
Flexible Array Members (C99) . . . 526
Restricted Pointers (C99) . . . 527
Practical Examples . . . 528
Generic Linked List Implementation . . . 528
Event System with Function Pointers . . . 530
Summary . . . 532

Common Pointer Pitfalls **533**
Introduction . . . 533
Dangling Pointers . . . 533
What Are Dangling Pointers? . . . 533
Common Causes . . . 533
Prevention Strategies . . . 534
Memory Leaks . . . 535
What Are Memory Leaks? . . . 535
Common Causes . . . 536
Prevention Strategies . . . 537
Buffer Overflows . . . 538
What Are Buffer Overflows? . . . 538
Common Causes . . . 538
Prevention Strategies . . . 539
Uninitialized Pointers . . . 540
What Are Uninitialized Pointers? . . . 540
Common Causes . . . 540
Prevention Strategies . . . 541
Pointer Arithmetic Errors . . . 543
Common Mistakes . . . 543
Prevention Strategies . . . 544
NULL Pointer Dereferencing . . . 544
What Is NULL Pointer Dereferencing? . . . 544
Common Causes . . . 544
Prevention Strategies . . . 545
Type Safety Issues . . . 545
Common Problems . . . 545
Prevention Strategies . . . 546
Debugging Pointer Issues . . . 547
Using Debugging Tools . . . 547
Static Analysis Tools . . . 547
Best Practices for Pointer Safety . . . 547
1. Initialize Pointers . . . 547
2. Check for NULL . . . 548
3. Validate Array Bounds . . . 548
4. Use Safe String Functions . . . 548
5. Match Allocation with Deallocation . . . 548
Practical Example: Safe Dynamic Array . . . 549
Summary . . . 551

When the Optimizer Deletes Your NULL Check **552**

When the Optimizer Deletes Your NULL Check **553**
Thesis . . . 553
Mental model . . . 553
Worked examples . . . 554
Case 1: Dereference first, check second . . . 555
Case 2: A zero-length `memcpy` counts as a dereference . . . 557

Case 3: Your own `nonnull` attribute, and the flag that does not help . . . 558
Case 4: Finding them before the optimizer does . . . 559
Case 5: The fix is ordering, not flags . . . 561
The trap . . . 563
The boring rule . . . 563
Try this . . . 563
Sources . . . 564

Module 6: Pointers and Memory Management **565**
Overview . . . 565
Learning Objectives . . . 565
Chapters . . . 565
Key Concepts Covered . . . 565
Prerequisites . . . 566
Tools and Technologies . . . 566
Estimated Time to Complete . . . 566
Assessment . . . 566
Next Module . . . 567
Additional Resources . . . 567

Structures **568**

Structures **569**
Introduction . . . 569
Structure Declaration and Definition . . . 569
Basic Structure Declaration . . . 569
Structure Variable Declaration . . . 569
Structure Initialization . . . 570
Designated Initializers (C99) . . . 570
Nested Structure Initialization . . . 571
Accessing Structure Members . . . 571
Dot Operator (.) . . . 571
Arrow Operator (->) . . . 572
Arrays of Structures . . . 573
Declaration and Initialization . . . 573
Dynamic Array of Structures . . . 573
Nested Structures . . . 574
Basic Nested Structures . . . 574
Self-Referential Structures . . . 575
Structure Assignment and Comparison . . . 576
Structure Assignment . . . 576
Structure Comparison . . . 577
Passing Structures to Functions . . . 577
Pass by Value . . . 577
Pass by Reference (Pointer) . . . 578
Structure Size and Memory Layout . . . 579
Calculating Structure Size . . . 579

Memory Alignment . . . 580
Practical Examples . . . 580
Student Management System . . . 580
Point Structure with Operations . . . 582
Summary . . . 583

Unions and Enumerations **584**
Introduction . . . 584
Unions . . . 584
Union Declaration and Basics . . . 584
Union Memory Layout . . . 585
Practical Union Usage . . . 585
Anonymous Unions (C11) . . . 587
Enumerations . . . 588
Basic Enumeration Declaration . . . 588
Explicit Enumeration Values . . . 589
Enumeration with Negative Values . . . 590
Enumeration Size and Type Specification (C23) . . . 591
Combining Unions and Enumerations . . . 592
Tagged Union Pattern . . . 592
Configuration Structure with Unions . . . 594
Practical Examples . . . 596
Network Packet Header . . . 596
Mathematical Expression Evaluator . . . 598
Summary . . . 601

Advanced Structure Concepts **602**
Introduction . . . 602
Bit Fields . . . 602
Basic Bit Field Declaration . . . 602
Practical Bit Field Usage . . . 603
Bit Field Limitations and Considerations . . . 605
Structure Packing and Alignment . . . 606
Default Structure Alignment . . . 606
Controlling Alignment with Pragma Pack . . . 607
Using _Alignas (C11) . . . 608
Flexible Array Members (C99) . . . 608
Basic Flexible Array Member . . . 608
Flexible Array in String Structure . . . 609
Self-Referential Structures . . . 611
Linked List Implementation . . . 611
Binary Tree Implementation . . . 612
Advanced Structure Techniques . . . 614
Structure with Function Pointers (C Object-Oriented Approach) . . . 614
Structure with Nested Anonymous Structures (C11) . . . 616
Practical Examples . . . 617
Configuration Manager with Advanced Structures . . . 617
Memory Pool with Flexible Array Members . . . 620

Summary . . . 622

Abstract Data Types **624**
Introduction . . . 624
Information Hiding Principles . . . 624
Separation of Interface and Implementation . . . 624
Opaque Pointers . . . 627
Creating Opaque Pointers . . . 627
Constructor and Destructor Patterns . . . 629
Resource Acquisition Is Initialization (RAII) in C . . . 629
Generic Abstract Data Types . . . 631
Using Void Pointers for Generic ADTs . . . 631
Interface Design in C . . . 636
Designing Clean APIs . . . 636
Practical Examples . . . 641
Complete ADT Implementation: Dynamic Array . . . 641
Summary . . . 646

Bit-Field Traps: Signedness, Promotion, Layout and Shared Words **647**

Bit-Field Traps: Signedness, Promotion, Layout and Shared Words **648**
Thesis . . . 648
Mental model . . . 648
Worked examples . . . 649
Case 1: A 1-bit `int` flag that can't equal 1 . . . 649
Case 2: The width of the field picks its type . . . 652
Case 3: A bit-field struct is not a byte layout . . . 653
Case 4: Two threads, two fields, one memory location . . . 654
Case 5: The boring versions . . . 656
The trap . . . 658
The boring rule . . . 658
Try this . . . 658
Sources . . . 659

Opaque Handles and Library APIs Other Languages Can Bind **660**

Opaque Handles and Library APIs Other Languages Can Bind **661**
The ABI Contract: Why Naive C APIs Break Other Languages . . . 661
Anti-Patterns and Reliable Solutions . . . 662
Designing the Tally Library . . . 662
Opaque Handles and Lifecycle Management . . . 663
Return Codes and Out-Parameters . . . 663
Caller-Owned Buffers and Two-Pass Size Queries . . . 663
Versioned Options Structs . . . 664
Symbol Visibility and Dynamic Linker Sonames . . . 664
Library Implementation . . . 665
Consuming the Library in C . . . 669
Binding from Python via ctypes . . . 672

Pitfalls . . . 675
1. Ownership of Returned Strings Across Language runtimes . . . 675
2. Callbacks Reentering the Library . . . 675
3. Struct Size Versioning Rules . . . 676
Key Takeaways . . . 676
Sources . . . 676

Module 7: Structures and User-Defined Types **678**
Overview . . . 678
Learning Objectives . . . 678
Chapters . . . 678
Key Concepts Covered . . . 678
Prerequisites . . . 679
Tools and Technologies . . . 679
Estimated Time to Complete . . . 679
Assessment . . . 679
Next Module . . . 680
Additional Resources . . . 680

File I/O 681

File Operations **682**
File Pointers and the `FILE` Structure . . . 682
Opening Files: `fopen` . . . 682
Modes . . . 682
Minimal open / close . . . 683
Text vs Binary Mode (Caveats) . . . 683
Text mode (`"r"`, `"w"`, …) . . . 683
Binary mode (`"rb"`, `"wb"`, …) . . . 684
Closing and Buffering . . . 684
Error Handling: `fopen`, `ferror`, `errno` . . . 685
Check every open . . . 685
Distinguish EOF from error . . . 685
Pattern: cleanup on partial failure . . . 686
Full Program 1: Copy a File . . . 687
Full Program 2: Simplified `wc` (Lines, Words, Bytes) . . . 688
Full Program 3: Append Log Lines with Timestamps . . . 691
File Size and Status Helpers . . . 693
Best Practices . . . 694
Exercises . . . 695
Exercise 1 — Safe truncate-free write . . . 695
Exercise 2 — Byte-at-a-time vs block copy . . . 695
Exercise 3 — Extend `mywc` . . . 695
Exercise 4 — Rotating log . . . 695
Exercise 5 — Error path drill . . . 695
Exercise 6 — Binary round-trip . . . 695
Summary . . . 696

File I/O Functions **697**
Introduction . . . 697
Character I/O Functions . . . 697
fgetc() and getc() . . . 697
fputc() and putc() . . . 698
ungetc() . . . 698
Examples . . . 698
Line I/O Functions . . . 699
fgets() . . . 699
fputs() . . . 699
Examples . . . 700
Formatted I/O Functions . . . 700
fprintf() . . . 701
fscanf() . . . 701
Examples . . . 701
Binary I/O Functions . . . 702
fwrite() . . . 702
fread() . . . 702
Examples . . . 703
Practical Examples . . . 704
CSV File Processor . . . 704
Binary File Analyzer . . . 705
Error Handling and Best Practices . . . 707
Comprehensive Error Checking . . . 707
Full Programs: Character, Line, Formatted, Binary . . . 708
Program 1 — ROT13 filter (`rot13.c`) . . . 708
Program 2 — Line numberer with long-line handling (`nlnum.c`) . . . 709
Program 3 — CSV-ish records with `fprintf` / `fscanf` (`scores_io.c`) . . . 710
Program 4 — Binary record dump / load (`binrec.c`) . . . 712
Program 5 — `ungetc` simple number peek (`peek_num.c`) . . . 714
Program 6 — Block copy with progress (`bcopy.c`) . . . 716
Error Handling Patterns (Quick Reference) . . . 717
Summary . . . 718
Exercises . . . 718

Advanced File Operations **720**
Introduction . . . 720
File Positioning . . . 720
fseek() . . . 720
ftell() . . . 720
rewind() . . . 721
Examples . . . 721
Random Access Files . . . 722
Structured Data Example . . . 722
Temporary Files . . . 725
tmpfile() . . . 725
tmpnam() . . . 725
mkstemp() (POSIX) . . . 725

Examples . . . 726
File Locking and Concurrent Access . . . 727
flockfile() and funlockfile() . . . 727
ftrylockfile() . . . 727
Example with File Locking . . . 727
File Status and Information . . . 728
fileno() . . . 729
fstat() (POSIX) . . . 729
Example with File Information . . . 729
Buffering Strategies . . . 730
setvbuf() . . . 730
Example with Custom Buffering . . . 730
Practical Examples . . . 731
File Merger Utility . . . 731
File Splitter Utility . . . 733
Summary . . . 734

System-Level Programming **735**
Introduction . . . 735
Command-Line Arguments . . . 735
argc and argv . . . 735
Examples . . . 736
Practical Command-Line Processor . . . 736
Environment Variables . . . 737
getenv() . . . 738
Examples . . . 738
Setting Environment Variables . . . 738
Process Management . . . 739
exit() . . . 739
atexit() . . . 739
Examples . . . 740
Process Exit Status . . . 741
Signal Handling . . . 741
signal() . . . 741
Common Signals . . . 742
Examples . . . 742
Advanced Signal Handling . . . 743
System Calls and Process Creation . . . 744
system() . . . 744
Examples . . . 745
Practical Examples . . . 745
Configuration File Processor . . . 745
Process Monitor . . . 748
Full Programs: CLI, Env, Signals, Processes . . . 749
Program 1 — Robust argument parser (`opts.c`) . . . 749
Program 2 — Environment-driven config (`envcfg.c`) . . . 751
Program 3 — `atexit` cleanup and `exit` status (`cleanup_demo.c`) . . . 752
Program 4 — Graceful SIGINT shutdown (`sigint_loop.c`) . . . 753

Program 5 — `fork` + `exec` + `wait` mini runner (`run_cmd.c`) . . . 754
Program 6 — Echo lines with PID and env prefix (`line_echo.c`) . . . 756
Exercises . . . 756
Summary . . . 757

Module 8: File I/O and System Programming **758**
Overview . . . 758
Learning Objectives . . . 758
Chapters . . . 758
Key Concepts Covered . . . 758
Prerequisites . . . 759
Tools and Technologies . . . 759
Estimated Time to Complete . . . 759
Assessment . . . 759
Next Module . . . 760
Additional Resources . . . 760

Modern C **761**

C99 Features **762**
Introduction . . . 762
Variable-Length Arrays (VLAs) . . . 762
Basic VLA Declaration . . . 762
Multi-dimensional VLAs . . . 763
VLA Function Parameters . . . 763
Designated Initializers . . . 764
Array Designated Initializers . . . 764
Structure Designated Initializers . . . 765
Compound Literals . . . 766
Array Compound Literals . . . 766
Structure Compound Literals . . . 767
New Data Types . . . 768
long long Type . . . 768
_Bool Type . . . 768
Inline Functions . . . 769
restrict Keyword . . . 770
Flexible Array Members . . . 771
Practical Examples . . . 772
Matrix Library Using C99 Features . . . 772
Summary . . . 775

C11 Features **776**
Introduction . . . 776
Static Assertions . . . 776
_Static_assert . . . 776
Practical Static Assertion Examples . . . 777

Anonymous Structures and Unions . . . 778
Anonymous Structures . . . 778
Anonymous Unions . . . 778
Thread-Local Storage . . . 779
Basic Thread-Local Storage . . . 780
Thread-Local Storage with Initialization . . . 781
Bounds-Checking Functions (Annex K) . . . 782
Safe String Functions . . . 782
Safe Memory Functions . . . 783
Generic Selection . . . 784
Basic Generic Selection . . . 784
Advanced Generic Selection . . . 785
Alignment Support . . . 786
Alignment Specifier . . . 787
Practical Examples . . . 787
Thread-Safe Logger with Static Assertions . . . 787
Summary . . . 789

C17 and C23 Features **791**
Introduction . . . 791
C17: Technical Corrigendum . . . 791
Defect Fixes . . . 791
Example of C17 Improvements . . . 791
C23: typeof Operators . . . 792
Basic typeof Usage . . . 792
typeof with Complex Expressions . . . 793
typeof_unqual for Unqualified Types . . . 793
Binary Literals . . . 794
Basic Binary Literals . . . 794
Bit Manipulation with Binary Literals . . . 794
Enhanced Preprocessor Directives . . . 795
#elifdef and #elifndef . . . 796
#warning Directive . . . 796
#embed Directive (Conceptual) . . . 797
Attribute Syntax . . . 797
Standard Attributes . . . 797
Extended Integer Types . . . 798
_BitInt Example (Conceptual) . . . 798
Enhanced _Static_assert . . . 799
Enhanced Static Assertions . . . 799
New Standard Library Functions . . . 800
strdup and strndup . . . 800
Enhanced aligned_alloc . . . 801
Practical Examples . . . 801
Modern Configuration System . . . 801
Summary . . . 803

What C23 Removed: Trigraphs, K&R Definitions and () Meaning "Anything" **805**

What C23 Removed: Trigraphs, K&R Definitions and `()` Meaning "Anything" **806**
Thesis . . . 806
The test files . . . 807
1. Trigraphs are gone (N2940) . . . 809
2. K&R function definitions are removed (N2432) . . . 810
3. `()` now means `(void)` (N2841) . . . 810
4. `bool`, `true`, `false` are keywords (N2934) . . . 810
Also changed: representation and `realloc(p, 0)` . . . 811
A migration checklist . . . 811
Takeaways . . . 811
Sources . . . 811

Modern C Development Practices **812**
Introduction . . . 812
DevOps and C: CI/CD Pipelines . . . 812
GitHub Actions for C Projects . . . 812
GitLab CI for C Projects . . . 813
Containerized Development . . . 814
Docker for C Development . . . 814
Multi-stage Docker Build . . . 815
Docker Compose for Development . . . 816
Cross-compilation . . . 817
CMake Cross-compilation . . . 817
Cross-compilation Toolchain File . . . 818
WebAssembly (WASM) . . . 819
Emscripten Setup . . . 819
Simple C to WASM Example . . . 819
HTML Integration . . . 820
Cloud-native C Applications . . . 820
Simple HTTP Server in C . . . 820
Dockerfile for Cloud Deployment . . . 823
Supply Chain Security . . . 823
Conan Package Manager . . . 824
Vulnerability Scanning . . . 824
Modern Testing Practices . . . 825
Unity Testing Framework . . . 825
Property-Based Testing with CMocka . . . 826
Fuzzing with AFL . . . 827
Documentation as Code . . . 829
Doxygen Configuration . . . 829
Example Documentation . . . 830
Practical Examples . . . 831
Modern C Project Structure . . . 831
Complete CMakeLists.txt . . . 831
Summary . . . 832

Advanced Preprocessor **834**
Introduction . . . 834

Macro Definitions and Function-like Macros . . . 834
Advanced Macro Techniques . . . 834
Generic Programming with Macros . . . 835
Conditional Compilation . . . 837
Platform-Specific Compilation . . . 837
Version-Specific Code . . . 839
Pragma Directives . . . 840
Common Pragma Directives . . . 841
Compiler-Specific Pragmas . . . 842
Predefined Macros . . . 844
System and Compiler Information . . . 844
Token Manipulation and Advanced Techniques . . . 846
Deferred Macro Expansion . . . 846
X-Macros Pattern . . . 847
C23 Enhanced Preprocessor Features . . . 849
#elifdef and #elifndef . . . 849
#warning Directive . . . 850
Digit Separators in Preprocessor . . . 850
Practical Examples . . . 851
Configuration System Using Preprocessor . . . 851
Advanced Error Handling with Preprocessor . . . 853
Summary . . . 855

C23 `#embed`, `nullptr`, and `constexpr` 857
Overview . . . 857
`#embed` — ship bytes without `xxd` hacks . . . 857
Basic embed into an array . . . 857
Resource limits and empty files . . . 858
When not to use `#embed` . . . 858
`nullptr` — a real null pointer constant . . . 859
`constexpr` objects . . . 859
Combined sketch . . . 860
Portability checklist . . . 860
Safety . . . 861
Try this . . . 861
Sources . . . 861

Checked Integer Arithmetic with C23 `<stdckdint.h>` 862
Thesis . . . 862
Mental model . . . 862
Worked examples . . . 863
Case 1: the check the optimizer is allowed to delete . . . 863
Case 2: what the macros actually report . . . 864
Case 3: sizing an allocation from an untrusted count . . . 866
Case 4: parsing a bounded number digit by digit . . . 868
Case 5: one header for C23 and older toolchains . . . 869
The trap . . . 870
The boring rule . . . 872

Try this . . . 872
Sources . . . 872

C23 <stdbit.h>: Bit and Byte Utilities **874**
Thesis . . . 874
Mental model . . . 874
Worked examples . . . 875
Case 1: a tour on one value . . . 875
Case 2: sizing a ring buffer with stdc_bit_ceil . . . 877
Case 3: a slot/ID allocator on a bitmap . . . 878
Case 4: a power-of-two latency histogram . . . 880
Case 5: endianness — detect with <stdbit.h>, decode with shifts . . . 881
Case 6: one shim for older libcs . . . 882
The trap . . . 883
The boring rule . . . 885
Try this . . . 885
Sources . . . 885

defer: Scope-Based Cleanup (TS 25755) **887**

defer: Scope-Based Cleanup with TS 25755 **888**
Thesis . . . 888
Mental model . . . 889
Worked examples . . . 889
Case 0: is it there? . . . 890
Case 1: when deferred blocks run . . . 890
Case 2: replacing a cleanup ladder . . . 892
Case 3: release with defer, commit explicitly . . . 895
Case 4: the same shape on today's GCC . . . 896
The trap . . . 898
Jumps the compiler rejects . . . 898
Leaving without leaving the block . . . 899
A homegrown defer macro meets <stddefer.h> . . . 900
Mutating the result from a deferred block . . . 901
The boring rule . . . 901
Try this . . . 902
Sources . . . 902

GCC -ftrivial-auto-var-init: Zero and Pattern Init **903**

-ftrivial-auto-var-init: Making Uninitialized Stack Boring **904**
Thesis . . . 904
Where things stand (2026-10) . . . 904
Mental model . . . 905
Worked examples . . . 905
Case 0: does the flag exist? . . . 906
Case 1: garbage vs zero vs pattern . . . 906
Case 2: partial writes still surprise people . . . 907
Case 3: unions . . . 908

Case 4: warnings do not go away . . . 908
The trap . . . 908
The boring rule . . . 909
Try this . . . 909
Sources . . . 909

GCC 16: What Changed for C **910**

GCC 16: What Changed for C **911**
Thesis . . . 911
Where things stand (2026-10) . . . 911
Mental model . . . 911
What actually changed for C in GCC 16 . . . 912
GNU extensions . . . 912
C2y alignment . . . 912
Diagnostics and tooling (C and C++ share these) . . . 913
Warnings: unused-but-set levels . . . 913
Worked examples . . . 913
Case 0: what default dialect do we actually get? . . . 913
Case 1: pin the standard so GCC 16 cannot surprise you . . . 914
Case 2: `_Maxof` / `_Minof` (GCC 16 — docs only on this lab) . . . 914
Case 3: `counted_by` on a pointer member (shape for GCC 16) . . . 915
Case 4: SARIF instead of `json` . . . 915
Case 5: unused-but-set under `-Wall` . . . 916
The trap . . . 916
The boring rule . . . 916
Try this . . . 916
Sources . . . 917

The XOR-Swap Trap and a Type-Safe C23 SWAP Macro **918**

The XOR-Swap Trap and a Type-Safe C23 `SWAP` Macro **919**
Thesis . . . 919
Trap 1: swapping an element with itself . . . 920
Trap 2: the one-liner is undefined . . . 920
Trap 3: it is not faster . . . 921
Doing it right: a C23 `SWAP` macro . . . 922
Takeaways . . . 923

C23 typeof and typeof_unqual: type of an expression **924**

C23 typeof and typeof_unqual: type of an expression **925**
Overview . . . 925
Basic declaration from expressions . . . 925
`typeof` vs `typeof_unqual` . . . 926
Structuring type-safe macros with `typeof_unqual` . . . 926
Difference vs GCC historical `__typeof__` . . . 927
Common mistakes . . . 928
Summary . . . 928

Verified output . . . 928
Sources . . . 928

Module 9: Advanced C Features and Modern Standards **929**
Overview . . . 929
Learning Objectives . . . 929
Chapters . . . 929
Key Concepts Covered . . . 930
Prerequisites . . . 930
Tools and Technologies . . . 930
Estimated Time to Complete . . . 931
Assessment . . . 931
Next Module . . . 931
Additional Resources . . . 931

Data Structures 932

Linear Data Structures **933**
Introduction . . . 933
Dynamic Arrays (Vectors) . . . 933
Basic Dynamic Array Implementation . . . 933
Generic Dynamic Array . . . 936
Linked Lists . . . 938
Singly Linked List . . . 939
Doubly Linked List . . . 943
Stacks . . . 946
Array-based Stack . . . 946
Linked List-based Stack . . . 948
Queues . . . 951
Circular Buffer Queue . . . 951
Linked List-based Queue . . . 954
Practical Examples . . . 957
Expression Evaluation Using Stacks . . . 957
Summary . . . 960

Tree Structures **961**
Introduction . . . 961
Binary Trees . . . 961
Basic Binary Tree Implementation . . . 961
Binary Search Trees (BST) . . . 965
BST Implementation . . . 965
AVL Trees . . . 969
AVL Tree Implementation . . . 970
Red-Black Trees . . . 976
Red-Black Tree Properties . . . 976
B-Trees . . . 981
B-Tree Implementation (Simplified) . . . 981

Practical Examples . . . 985
File System Directory Tree . . . 985
Summary . . . 988

Hash Tables **990**
Introduction . . . 990
Hash Function Design . . . 990
Basic Hash Functions . . . 990
Collision Resolution Strategies . . . 992
Chaining . . . 992
Open Addressing . . . 996
Dynamic Hash Tables . . . 1001
Resizable Hash Table with Chaining . . . 1001
Hash Table Performance Analysis . . . 1007
Load Factor and Performance . . . 1007
Practical Examples . . . 1011
Symbol Table Implementation . . . 1011
Cache Implementation . . . 1016
Summary . . . 1021

Roaring Bitmaps from Scratch: Array, Bitmap and Run Containers **1022**

Roaring Bitmaps from Scratch: Array, Bitmap and Run Containers **1023**
Why Compressed Bitmaps? . . . 1023
The Two-Level Layout . . . 1024
Containers and the 4096 Rule . . . 1024
The Break-Even Calculation . . . 1024
Run Container Trade-off . . . 1025
Implementation Walk-through . . . 1025
Function Breakdowns . . . 1028
Reading the Results . . . 1030
What Real Roaring Adds . . . 1030
Run Optimization Pass . . . 1030
Specialized Container-Pair Operations . . . 1031
Serialization Specification . . . 1031
64-bit Variants . . . 1031
Production Usage with CRoaring . . . 1031
Common Mistakes . . . 1032
1. Assuming Bitmaps Are Always Smaller Than Arrays . . . 1032
2. Ignoring Per-Container Overhead on Sparse Data . . . 1032
3. Inserting Unsorted Data into Array Containers . . . 1032
4. Integer Overflow during High/Low Bit Splitting . . . 1032
Exercises . . . 1033
Summary . . . 1033
Sources . . . 1033

Graph Algorithms **1034**
Introduction . . . 1034

Graph Representations . . . 1034
Adjacency Matrix . . . 1034
Adjacency List . . . 1036
Graph Traversal Algorithms . . . 1040
Breadth-First Search (BFS) . . . 1040
Depth-First Search (DFS) . . . 1044
Shortest Path Algorithms . . . 1048
Dijkstra's Algorithm . . . 1048
Minimum Spanning Tree Algorithms . . . 1055
Prim's Algorithm . . . 1055
Practical Examples . . . 1062
Social Network Analysis . . . 1062
Summary . . . 1067

Sorting and Searching **1069**
Introduction . . . 1069
Comparison-Based Sorting Algorithms . . . 1069
Bubble Sort . . . 1069
Selection Sort . . . 1070
Insertion Sort . . . 1071
Merge Sort . . . 1072
Quick Sort . . . 1074
Heap Sort . . . 1076
Non-Comparison Sorting Algorithms . . . 1078
Counting Sort . . . 1078
Radix Sort . . . 1079
Bucket Sort . . . 1081
Searching Algorithms . . . 1083
Linear Search . . . 1083
Binary Search . . . 1084
Interpolation Search . . . 1086
Advanced Searching Techniques . . . 1087
Exponential Search . . . 1087
Jump Search . . . 1088
Selection Algorithms . . . 1090
Quickselect (Finding kth Smallest Element) . . . 1090
Performance Comparison and Analysis . . . 1092
Sorting Algorithm Performance Test . . . 1092
Practical Examples . . . 1098
Phone Book Implementation . . . 1098
Summary . . . 1101

Module 10: Data Structures and Algorithms **1103**
Overview . . . 1103
Learning Objectives . . . 1103
Chapters . . . 1103
Key Concepts Covered . . . 1103
Prerequisites . . . 1104

Tools and Technologies . . . 1104
Estimated Time to Complete . . . 1105
Assessment . . . 1105
Next Module . . . 1105
Additional Resources . . . 1105

Network **1106**

Network Fundamentals **1107**
Computer Network Basics . . . 1107
Topologies (conceptual) . . . 1107
Network types by scope . . . 1107
OSI and TCP/IP Models . . . 1108
OSI (7 layers) . . . 1108
TCP/IP (practical 4 layers) . . . 1108
IP Addressing and Ports . . . 1108
IPv4 . . . 1108
IPv6 (awareness) . . . 1109
Ports . . . 1109
TCP vs UDP (what sockets expose) . . . 1109
Sockets and Address Structures . . . 1109
Headers . . . 1109
Structures . . . 1110
Endianness: Why Byte Order Matters . . . 1110
Program: inspect endianness and multi-byte layout . . . 1110
`htons` / `htonl` / `ntohs` / `ntohl` . . . 1112
Demo program . . . 1112
`inet_pton` and `inet_ntop` . . . 1113
Full conversion + `sockaddr_in` setup demo . . . 1113
Filling `sockaddr_in` checklist . . . 1115
Minimal UDP Echo: Server and Client . . . 1115
Server (`udp_echo_server.c`) . . . 1115
Client (`udp_echo_client.c`) . . . 1117
Build and run (two terminals) . . . 1118
Optional: see the packets . . . 1119
Network Errors (practical) . . . 1119
Hostname Lookup (brief, modern) . . . 1120
Protocol Quick Reference . . . 1120
Exercises . . . 1120
Exercise 1 — Endianness report . . . 1120
Exercise 2 — Port round-trip . . . 1120
Exercise 3 — `inet_pton` validation . . . 1121
Exercise 4 — Bind failure . . . 1121
Exercise 5 — Echo timeout . . . 1121
Exercise 6 — Multi-message server log . . . 1121
Exercise 7 — Binary payload . . . 1121
Summary . . . 1121

Socket Programming **1122**
Introduction . . . 1122
Socket Basics . . . 1122
Socket Creation . . . 1122
Example Socket Creation . . . 1122
Socket Address Structures . . . 1123
IPv4 Address Structure . . . 1123
Generic Socket Address Structure . . . 1123
Server-Side Socket Programming . . . 1123
Binding a Socket . . . 1124
Listening for Connections . . . 1124
Accepting Connections . . . 1124
Complete TCP Server Example . . . 1124
Client-Side Socket Programming . . . 1126
Connecting to a Server . . . 1126
Complete TCP Client Example . . . 1126
UDP Socket Programming . . . 1127
UDP Server Example . . . 1127
UDP Client Example . . . 1129
Advanced Socket Options . . . 1130
Socket Options . . . 1130
Common Socket Options . . . 1130
Example: Setting Socket Options . . . 1130
Error Handling in Socket Programming . . . 1131
Common Socket Errors . . . 1131
Robust Error Handling Example . . . 1131
Practical Examples . . . 1132
Simple Echo Server . . . 1132
Simple Echo Client . . . 1134
Summary . . . 1136

Advanced Networking **1137**
Introduction . . . 1137
Non-blocking I/O . . . 1137
Setting Non-blocking Mode . . . 1137
Non-blocking Socket Example . . . 1137
I/O Multiplexing . . . 1139
select() . . . 1139
Example: Using select() for Multiple Clients . . . 1139
poll() . . . 1142
Example: Using poll() . . . 1142
Concurrent Server Architectures . . . 1145
Threading Approach . . . 1145
Forking Approach . . . 1147
Advanced Socket Options . . . 1150
Socket Buffer Sizes . . . 1150
Timeout Options . . . 1151
Signal-Driven I/O . . . 1152

Practical Examples . . . 1153
High-Performance Echo Server . . . 1153
Summary . . . 1157

Network Protocols **1158**
Introduction . . . 1158
HTTP Protocol Implementation . . . 1158
HTTP Request Structure . . . 1158
HTTP Response Structure . . . 1158
Simple HTTP Client . . . 1159
Simple HTTP Server . . . 1161
FTP Protocol Implementation . . . 1164
FTP Client Implementation . . . 1164
DNS Protocol Implementation . . . 1167
Simple DNS Resolver . . . 1167
SMTP Protocol Implementation . . . 1170
Practical Examples . . . 1174
Protocol Analyzer . . . 1174
Summary . . . 1175

Network Security **1177**
Introduction . . . 1177
Security Fundamentals . . . 1177
CIA Triad . . . 1177
Common Security Threats . . . 1177
Cryptography Basics . . . 1177
Symmetric Encryption . . . 1178
Hash Functions . . . 1178
Secure Socket Layer (SSL/TLS) . . . 1179
Installing OpenSSL . . . 1179
Simple HTTPS Client with OpenSSL . . . 1180
Authentication and Authorization . . . 1183
Simple Authentication System . . . 1183
Secure Coding Practices . . . 1185
Input Validation . . . 1185
Buffer Overflow Prevention . . . 1187
Firewall and Access Control . . . 1188
Simple Packet Filter . . . 1188
Secure Communication Patterns . . . 1190
Mutual Authentication . . . 1190
Practical Examples . . . 1192
Secure File Transfer . . . 1192
Summary . . . 1195

Module 11: Network Programming **1197**
Overview . . . 1197
Learning Objectives . . . 1197
Chapters . . . 1197

Prerequisites . . . 1197
Key Concepts . . . 1198

Embedded **1199**

Embedded Basics **1200**
Introduction . . . 1200
What Are Embedded Systems? . . . 1200
Characteristics . . . 1200
Types (rough scale) . . . 1201
Microcontroller vs Microprocessor . . . 1201
Typical Components . . . 1201
Development Environments and Cross-Compilation . . . 1201
High-Level Toolchain Notes (`arm-none-eabi-gcc`) . . . 1201
Other Tools You Will Meet . . . 1202
Memory Organization . . . 1203
Startup Sketch (bare metal idea) . . . 1203
Section Placement Reminders . . . 1203
Bit Manipulation Helpers . . . 1204
`volatile` — Why Embedded Needs It . . . 1205
Packed Structs . . . 1207
Memory-Mapped Register Pattern as Struct Overlay . . . 1208
Software Model: Button Debounce State Machine . . . 1210
Classic Patterns (Still Useful) . . . 1212
LED Blink Superloop (conceptual MMIO) . . . 1212
Interrupt Sketch . . . 1213
Timing . . . 1213
Practical Host Lab: Mini "Firmware" Module . . . 1213
Exercises . . . 1215
Exercise 1 — Bit library . . . 1215
Exercise 2 — Packed vs unpacked . . . 1215
Exercise 3 — Overlay UART . . . 1215
Exercise 4 — Debounce FSM extension . . . 1215
Exercise 5 — Cross-compile dry run . . . 1215
Exercise 6 — Volatile experiment . . . 1216
Summary . . . 1216

Low-Level Programming **1217**
Introduction . . . 1217
Memory-Mapped I/O . . . 1217
Direct Register Access . . . 1217
Bit Manipulation Techniques . . . 1218
Advanced Bit Operations . . . 1219
Bit Fields . . . 1219
Inline Assembly . . . 1220
Basic Inline Assembly . . . 1220
Advanced Inline Assembly . . . 1221

Memory Management . . . 1222
Custom Memory Allocator . . . 1222
Memory Sections and Linker Scripts . . . 1223
Hardware Abstraction Layer (HAL) . . . 1223
GPIO HAL Implementation . . . 1223
Interrupt Management . . . 1227
Nested Vectored Interrupt Controller (NVIC) . . . 1227
Practical Examples . . . 1228
UART Driver Implementation . . . 1228
Summary . . . 1230

Embedded C Techniques **1231**
Introduction . . . 1231
Compiler Extensions and Attributes . . . 1231
GCC Attributes . . . 1231
IAR and Keil Extensions . . . 1232
Optimization Techniques . . . 1233
Compiler Optimization Levels . . . 1233
Manual Optimization Techniques . . . 1234
Fixed-Point Arithmetic . . . 1235
Volatile Keyword and Memory-Mapped I/O . . . 1236
Interrupt Service Routines . . . 1236
Memory-Constrained Programming . . . 1237
Stack Management . . . 1238
Code Size Optimization . . . 1239
Real-Time Considerations . . . 1240
Timing Analysis . . . 1240
Hardware-Specific Features . . . 1241
ARM Cortex-M Specific Features . . . 1241
Practical Examples . . . 1242
Efficient Sensor Data Processing . . . 1242
Power Management . . . 1244
Summary . . . 1245

Real-time Programming **1246**
Introduction . . . 1246
Real-time System Characteristics . . . 1246
Types of Real-time Systems . . . 1246
Real-time Requirements . . . 1246
Task Scheduling . . . 1247
Rate Monotonic Scheduling (RMS) . . . 1247
Earliest Deadline First (EDF) Scheduling . . . 1248
Timing and Synchronization . . . 1249
Timer Management . . . 1249
Semaphore Implementation . . . 1251
Interrupt Handling in Real-time Systems . . . 1253
Interrupt Latency Management . . . 1254

Resource Management . . . 1256
Memory Pool Management . . . 1256
Priority Inheritance . . . 1257
Timing Analysis and Deadline Monitoring . . . 1259
Deadline Monitoring . . . 1259
Practical Examples . . . 1262
Real-time Control System . . . 1262
Real-time Data Acquisition System . . . 1264
Summary . . . 1267

Hardware Interfaces **1268**
Introduction . . . 1268
GPIO (General Purpose Input/Output) . . . 1268
GPIO Configuration and Control . . . 1268
SPI (Serial Peripheral Interface) . . . 1272
SPI Driver Implementation . . . 1272
I2C (Inter-Integrated Circuit) . . . 1276
I2C Driver Implementation . . . 1276
UART (Universal Asynchronous Receiver-Transmitter) . . . 1284
UART Driver Implementation . . . 1284
Practical Examples . . . 1288
Sensor Interface Example . . . 1288
Summary . . . 1293

Module 12: Embedded Systems Programming **1294**
Overview . . . 1294
Learning Objectives . . . 1294
Chapters . . . 1294
Prerequisites . . . 1294
Key Concepts . . . 1295

Performance **1296**

Performance Analysis **1297**
Introduction . . . 1297
Performance Metrics . . . 1297
Time Complexity Metrics . . . 1297
Memory Usage Metrics . . . 1298
CPU Usage Metrics . . . 1299
Profiling Tools . . . 1300
GNU gprof . . . 1300
Valgrind Callgrind . . . 1301
perf (Linux Performance Events) . . . 1302
Custom Profiling Framework . . . 1302
Benchmarking Framework . . . 1305
Cache Performance Analysis . . . 1308
Memory Access Pattern Analysis . . . 1309

Practical Examples . . . 1311
Comprehensive Performance Analysis Tool . . . 1311
Summary . . . 1314

Optimization Techniques **1316**
Introduction . . . 1316
Algorithmic Optimization . . . 1316
Time Complexity Analysis . . . 1316
Search Algorithm Optimization . . . 1319
Loop Optimization . . . 1322
Loop Unrolling . . . 1322
Loop Fusion and Fission . . . 1324
Data Structure Optimization . . . 1326
Array vs Linked List . . . 1326
Compiler Optimization Techniques . . . 1329
Compiler Flags and Pragmas . . . 1329
Mathematical Optimization . . . 1331
Fast Mathematical Operations . . . 1331
Practical Examples . . . 1333
Image Processing Optimization . . . 1333
Summary . . . 1336

Memory Optimization **1337**
Introduction . . . 1337
Memory Hierarchy and Performance . . . 1337
Memory Latency Comparison . . . 1337
Cache Behavior Impact . . . 1337
Memory Allocation Optimization . . . 1339
Custom Memory Allocators . . . 1339
Stack Allocation vs Heap Allocation . . . 1342
Data Structure Optimization . . . 1343
Structure of Arrays vs Array of Structures . . . 1343
Memory Alignment . . . 1345
Memory Layout Optimization . . . 1346
Data Locality . . . 1346
Memory Pooling . . . 1348
Memory Access Patterns . . . 1351
Sequential vs Random Access . . . 1351
Practical Memory Optimization Techniques . . . 1353
Reducing Memory Fragmentation . . . 1353
Memory Prefetching . . . 1354
Memory Debugging and Analysis . . . 1355
Simple Memory Tracker . . . 1356
Summary . . . 1359

Advanced Optimization **1360**
Introduction . . . 1360

Compiler Optimization . . . 1360
Optimization Levels . . . 1360
Compiler Hints and Pragmas . . . 1361
Profile-Guided Optimization (PGO) . . . 1362
Vectorization . . . 1362
Manual Vectorization with Intrinsics . . . 1362
Auto-Vectorization . . . 1365
Parallel Processing . . . 1366
OpenMP for Shared-Memory Parallelism . . . 1366
Thread-Level Parallelism with Pthreads . . . 1368
Architecture-Specific Optimizations . . . 1370
Cache Optimization Techniques . . . 1370
Branch Prediction Optimization . . . 1372
Memory-Level Parallelism . . . 1373
Advanced Profiling and Analysis . . . 1374
Optimization Decision Making . . . 1375
Practical Optimization Workflow . . . 1376
Summary . . . 1377

ISPC 1.32: AVX10.2, APX, and Zen 6 Targets 1378

ISPC 1.32: AVX10.2, APX, and Zen 6 Targets 1379
Learning goals . . . 1379
Thesis . . . 1379
Mental model — SPMD, gangs, packing . . . 1379
What AVX10.2 and APX enable . . . 1380
The trap: CPUID OS support . . . 1380
Worked example — SPMD add (docs-faithful) . . . 1381
Case — picking packing width . . . 1382
The trap . . . 1382
The boring rule . . . 1382
Try this . . . 1383
Sources . . . 1383

Arena Allocators: Bump Pointers, Alignment, and Lifetimes Instead of free() 1384

Arena Allocators: Bump Pointers, Alignment, and Lifetimes Instead of `free()` 1385
Thesis . . . 1385
Mental model . . . 1385
Worked examples . . . 1385
The whole allocator . . . 1386
Case 1: Sizes, padding and the alignment `malloc` doesn't promise . . . 1388
Case 2: Two lifetimes, and a scratch arena passed by value . . . 1389
Case 3: What it buys, measured . . . 1391
Case 4: The trap, a pointer that outlives its reset . . . 1394
The trap . . . 1395
The boring rule . . . 1396
Try this . . . 1396

Sources . . . 1397

Double-Ended Stack Allocators: Two Lifetimes, One Buffer, No Fragmentation 1398

Double-Ended Stack Allocators: Two Lifetimes, One Buffer, No Fragmentation 1399
Thesis . . . 1399
Mental model . . . 1399
Worked examples . . . 1399
The allocator . . . 1399
Case 1: two lifetimes without fragmentation . . . 1401
Case 2: variable-sized records plus an index in a fixed packet . . . 1402
Case 3: sharing it between threads . . . 1404
Things that bite . . . 1407
Try this . . . 1407
Sources . . . 1407

How Redis wraps malloc: zmalloc accounting, used_memory vs RSS, and fragmentation 1408
Two ways to know a block's size . . . 1408
Real output (gcc 14.2 -O2, glibc 2.41, x86_64) . . . 1410
What the numbers say . . . 1411
What Redis does about it: active defrag . . . 1411
Checklist . . . 1412

restrict and Pointer Aliasing 1413

restrict and Pointer Aliasing 1414
Thesis . . . 1414
Mental model . . . 1414
Worked example . . . 1414
Trap . . . 1415
Boring rule . . . 1415
Try this . . . 1415
Sources . . . 1415

Timing Code with RDTSC and RDTSCP: Fences, Overhead, and Converting Ticks 1416

Timing Code with RDTSC and RDTSCP: Fences, Overhead, and Converting Ticks 1417
Thesis . . . 1417
Step 0: is the TSC usable at all? . . . 1418
Step 1: order the reads . . . 1418
Step 2: measure the measurement . . . 1419
Step 3: ticks are not core cycles . . . 1419
When to use which . . . 1420
Pitfalls . . . 1420
Takeaways . . . 1420
Sources . . . 1421
Appendix: the full harness . . . 1421

Module 13: Performance Optimization **1423**
Overview . . . 1423
Learning Objectives . . . 1423
Chapters . . . 1423
Prerequisites . . . 1423
Key Concepts . . . 1424

Testing **1425**

Testing Methodologies **1426**
Why Testing Matters . . . 1426
Types of Testing (C-focused) . . . 1426
Unit testing characteristics . . . 1426
Style 1: `assert.h` Unit Tests . . . 1427
Style 2: Minimal Custom `TEST` Macro Harness . . . 1427
Style 3: Unity-Style Suite That Actually Builds . . . 1430
Option A — Self-contained "unity-like" runner (no download) . . . 1430
Option B — Real Unity (vendored) . . . 1432
Test-Driven Development (TDD) Walkthrough: `is_leap_year()` . . . 1433
Step 0 — Scaffold (no real logic yet) . . . 1433
Step 1 — RED: first failing test . . . 1433
Step 2 — GREEN: minimal code that passes . . . 1434
Step 3 — RED: century rule . . . 1434
Step 4 — GREEN: handle centuries . . . 1434
Step 5 — RED: year 2000 must be leap . . . 1435
Step 6 — GREEN: full Gregorian rule . . . 1435
Step 7 — REFACTOR: keep tests green . . . 1435
Alternate TDD target: `clamp()` . . . 1435
Boundary Value Testing . . . 1436
Integers . . . 1436
Strings . . . 1437
Table-Driven Tests in C . . . 1438
Code Coverage with `gcov` / `lcov` . . . 1439
Instrument and run . . . 1439
Sanitizers: Address and Undefined Behavior . . . 1440
AddressSanitizer (ASan) . . . 1440
UndefinedBehaviorSanitizer (UBSan) . . . 1440
Combined (common for tests) . . . 1441
Integration and System Testing (brief) . . . 1441
Black-Box vs White-Box (quick map) . . . 1441
FIRST and AAA . . . 1442
Best Practices . . . 1442
Exercises . . . 1442
Exercise 1 — Custom harness for `clamp` . . . 1442
Exercise 2 — TDD `int max3(int a, int b, int c)` . . . 1443
Exercise 3 — Table-driven string trim . . . 1443
Exercise 4 — Coverage gap hunt . . . 1443

Exercise 5 — Sanitizer lab . . . 1443
Exercise 6 — Boundary design . . . 1443
Summary . . . 1443

Debugging Techniques **1445**
The Debugging Process . . . 1445
GDB: Session-Style Walkthroughs . . . 1445
Program under study . . . 1446
Transcript 1 — break, run, print, next, continue . . . 1446
Transcript 2 — backtrace on a crash . . . 1447
Transcript 3 — watchpoints . . . 1448
Handy GDB extras . . . 1449
`printf` Debugging and `DEBUG_LOG` Macros . . . 1449
Rubber duck and binary-search debugging . . . 1451
Valgrind Memcheck: Intentional Leak and Reading the Report . . . 1452
Example program with a real leak and a bad read . . . 1452
How to read typical output . . . 1453
Common Memcheck flags . . . 1453
`strace` and `ltrace` (brief) . . . 1454
`strace` — system calls . . . 1454
`ltrace` — library calls . . . 1454
Core Dumps: Capture and Analyze . . . 1454
Enable dumps . . . 1454
Produce a core . . . 1455
Analyze with GDB . . . 1455
Common Failure Modes . . . 1456
Segmentation faults . . . 1456
Logic errors . . . 1456
Memory leaks and corruption . . . 1456
Race conditions (threads) . . . 1456
Static Analysis (quick) . . . 1457
Debugging Best Practices . . . 1457
Exercises . . . 1457
Exercise 1 — GDB fix of `sum_range` . . . 1457
Exercise 2 — `DEBUG_LOG` toggle . . . 1457
Exercise 3 — Valgrind leak . . . 1458
Exercise 4 — Invalid write . . . 1458
Exercise 5 — Core dump . . . 1458
Exercise 6 — `strace` detective . . . 1458
Summary . . . 1458

Error Handling **1460**
Introduction to Error Handling . . . 1460
Why Error Handling is Important . . . 1460
Error Handling Strategies in C . . . 1460
Return Codes . . . 1460
Global Error Variables . . . 1462

Advanced Error Handling Techniques . . . 1464
Setjmp and Longjmp . . . 1464
Resource Acquisition Is Initialization (RAII) . . . 1465
Error Handling Best Practices . . . 1467
Check All Return Values . . . 1467
Use Assertions for Debugging . . . 1467
Provide Meaningful Error Messages . . . 1468
Implement Error Logging . . . 1469
Error Handling in Different Contexts . . . 1470
File I/O Error Handling . . . 1470
Memory Management Error Handling . . . 1472
Conclusion . . . 1473

Code Quality **1474**
Introduction to Code Quality . . . 1474
Dimensions of Code Quality . . . 1474
Coding Standards and Style Guidelines . . . 1474
Naming Conventions . . . 1475
Code Formatting . . . 1475
Commenting Best Practices . . . 1475
Code Review Practices . . . 1476
Benefits of Code Reviews . . . 1476
Code Review Checklist . . . 1476
Example Code Review . . . 1476
Static Analysis Tools . . . 1477
GCC with Warnings . . . 1477
Cppcheck . . . 1477
Clang Static Analyzer . . . 1478
Code Formatting and Style Tools . . . 1478
Clang-Format . . . 1478
AStyle (Artistic Style) . . . 1478
Test Coverage Analysis . . . 1478
GCC with Coverage . . . 1479
Coverage Metrics . . . 1479
Performance Analysis . . . 1479
Profiling with gprof . . . 1479
Profiling with perf . . . 1480
Security Best Practices . . . 1480
Input Validation . . . 1480
Memory Safety . . . 1481
Secure Coding Practices . . . 1481
Documentation and Code Comments . . . 1481
Self-Documenting Code . . . 1482
API Documentation . . . 1482
Refactoring and Code Improvement . . . 1483
Code Smells to Avoid . . . 1483
Refactoring Example . . . 1483

Full Programs: Quality Gates You Can Run Locally . . . 1485
Program 1 — Strict-warning "clean" module (`clamp_lib`) . . . 1485
Program 2 — Warning archaeology (`smell.c` → `clean.c`) . . . 1486
Program 3 — Complexity reduction: nested ifs → early return . . . 1487
Program 4 — Static assertions and size contracts . . . 1489
Program 5 — Local quality script (`check.sh` + target) . . . 1489
Program 6 — Review checklist applied to a PR-sized change . . . 1490
Continuous Integration and Quality Gates . . . 1492
CI Pipeline Components . . . 1492
Example CI Configuration . . . 1492
Local gate CI gate . . . 1493
Exercises . . . 1493
Conclusion . . . 1494

Compiler Sanitizers (ASan, UBSan, TSan, MSan) 1495
Overview . . . 1495
Mental model . . . 1496
AddressSanitizer (ASan) . . . 1496
Planted use-after-free . . . 1496
Heap buffer overflow . . . 1497
Useful ASan runtime knobs . . . 1497
UndefinedBehaviorSanitizer (UBSan) . . . 1498
ThreadSanitizer (TSan) . . . 1498
MemorySanitizer (MSan) — brief . . . 1499
Interpreting reports (operator checklist) . . . 1500
CI gates (practical matrix) . . . 1500
Common false friends . . . 1501
Try this . . . 1501
Sources . . . 1501

Memory-Safe C in Practice: Fil-C, counted_by, and -fbounds-safety 1503

Memory-Safe C in Practice: Fil-C, `counted_by`, and `-fbounds-safety` 1504
Thesis . . . 1504
Mental model . . . 1505
Worked examples . . . 1505
Case 1: an overflow into the next allocation . . . 1506
Case 2: use after free . . . 1507
Case 3: teach the compiler how long an array is . . . 1509
Case 4: what Fil-C asks you to change . . . 1511
Case 5: what it costs . . . 1512
The trap . . . 1514
The boring rule . . . 1516
Try this . . . 1517
Sources . . . 1517

GCC Diagnostics: JSON and SARIF for CI 1518

GCC Diagnostics: JSON and SARIF for CI **1519**
Thesis . . . 1519
Mental model . . . 1519
Worked examples . . . 1519
Case 0: which formats exist? . . . 1520
Case 1: text baseline . . . 1520
Case 2: SARIF file for dashboards . . . 1520
Case 3: JSON for a tiny gate . . . 1521
Case 4: plain text without carets (log-friendly) . . . 1521
Case 5: analyzer events still flow through text (and into SARIF when enabled) . . . 1522
The trap . . . 1523
The boring rule . . . 1523
Try this . . . 1523
Sources . . . 1523

Testing the Error Paths: Deterministic Failure Injection **1524**

Testing the Error Paths: Deterministic Failure Injection **1525**
Thesis . . . 1525
Mental model . . . 1525
Worked examples . . . 1526
Case 1: A loader whose happy-path test passes under ASan . . . 1526
Case 2: The tripwire header and its zero-cost release build . . . 1529
Case 3: The sweep finds all three bugs . . . 1531
Case 4: One cleanup block, and a clean sweep . . . 1534
The trap . . . 1536
The boring rule . . . 1537
Try this . . . 1537
Sources . . . 1538

setjmp/longjmp for Error Unwinding: The Rules That Bite **1539**

setjmp/longjmp for Error Unwinding: The Rules That Bite **1540**
Thesis . . . 1540
Mental model . . . 1540
Worked example 1: the volatile rule, made visible . . . 1541
Worked example 2: a parser that unwinds and doesn't leak . . . 1542
Worked example 3: leaving a signal handler . . . 1545
Things that bite . . . 1546
When to choose it . . . 1547
Try this . . . 1547
Sources . . . 1547

Module 14: Testing **1548**
Learning goals . . . 1548
Chapters (map) . . . 1548
Suggested order . . . 1549
Tooling cheat sheet . . . 1550
How this module connects . . . 1550

Module self-check . . . 1550
Estimated time . . . 1551
Next . . . 1551

Advanced **1552**

Module 15.1: Multithreading in C **1553**
Introduction . . . 1553
Learning Objectives . . . 1553
1. Fundamentals of Multithreading . . . 1553
1.1 Processes vs Threads . . . 1553
1.2 Thread Lifecycle . . . 1553
1.3 Thread Attributes . . . 1554
2. Creating and Managing Threads . . . 1554
2.1 Basic Thread Creation . . . 1554
2.2 Thread Arguments and Return Values . . . 1555
3. Thread Synchronization . . . 1556
3.1 Race Conditions . . . 1556
3.2 Mutexes . . . 1556
3.3 Condition Variables . . . 1557
4. Advanced Threading Concepts . . . 1559
4.1 Thread-Specific Data . . . 1559
4.2 Thread Cancellation . . . 1560
5. Performance Considerations . . . 1561
5.1 Thread Pool Implementation . . . 1561
6. Debugging Multithreaded Applications . . . 1564
6.1 Common Issues . . . 1564
6.2 Debugging Tools . . . 1564
7. Best Practices . . . 1564
7.1 Design Principles . . . 1564
7.2 Performance Optimization . . . 1565
Summary . . . 1565

Module 15.2: System Programming in C **1566**
Introduction . . . 1566
Learning Objectives . . . 1566
1. System Calls and APIs . . . 1566
1.1 What are System Calls? . . . 1566
1.2 Common System Call Categories . . . 1566
1.3 Error Handling in System Calls . . . 1566
2. Process Management . . . 1567
2.1 Process Creation with fork() . . . 1567
2.2 Process Execution with exec() . . . 1568
2.3 Process Groups and Sessions . . . 1568
3. Advanced File System Operations . . . 1569
3.1 File Metadata with stat() . . . 1569
3.2 Directory Operations . . . 1570

3.3 Hard Links and Symbolic Links . . . 1571
4. Inter-Process Communication (IPC) . . . 1572
4.1 Pipes . . . 1572
4.2 Named Pipes (FIFOs) . . . 1574
4.3 Shared Memory . . . 1575
5. Signal Handling . . . 1576
5.1 Basic Signal Handling . . . 1576
5.2 Advanced Signal Handling with sigaction() . . . 1577
6. Memory Management . . . 1578
6.1 Memory Mapping with mmap() . . . 1578
6.2 Memory Protection . . . 1579
7. Security Considerations . . . 1581
7.1 Secure Programming Practices . . . 1581
7.2 Example of Secure File Handling . . . 1581
8. Best Practices . . . 1582
8.1 Error Handling . . . 1582
8.2 Resource Management . . . 1582
8.3 Portability . . . 1582
Summary . . . 1583

Module 15.3: Domain-Specific Applications in C **1584**
Introduction . . . 1584
Learning Objectives . . . 1584
1. Characteristics of Domain-Specific Applications . . . 1584
1.1 Domain Requirements . . . 1584
1.2 Design Considerations . . . 1584
2. Embedded Systems Applications . . . 1585
2.1 Microcontroller Programming . . . 1585
2.2 Real-Time Operating System Integration . . . 1586
3. Scientific Computing Applications . . . 1589
3.1 Numerical Libraries Integration . . . 1589
3.2 Fast Fourier Transform (FFT) . . . 1593
4. Networking Applications . . . 1596
4.1 Protocol Implementation . . . 1596
4.2 HTTP Client Implementation . . . 1600
5. Best Practices for Domain-Specific Applications . . . 1606
5.1 Domain Analysis . . . 1606
5.2 Design Patterns . . . 1606
5.3 Testing Strategies . . . 1606
Summary . . . 1607

Module 15.4: Capstone Project **1608**
Introduction . . . 1608
Learning Objectives . . . 1608
Project Requirements . . . 1608
1. Technical Requirements . . . 1608
2. Project Scope . . . 1609
3. Deliverables . . . 1609

Project Ideas . . . 1609
1. System Monitoring Tool . . . 1609
2. Custom Database Engine . . . 1614
3. Network File Transfer System . . . 1618
Project Development Process . . . 1626
1. Planning Phase . . . 1626
2. Implementation Phase . . . 1627
3. Testing and Debugging Phase . . . 1627
4. Deployment and Presentation Phase . . . 1627
Evaluation Criteria . . . 1627
Technical Excellence (40%) . . . 1627
Design and Architecture (25%) . . . 1628
Documentation (20%) . . . 1628
Testing (10%) . . . 1628
Presentation (5%) . . . 1628
Best Practices . . . 1628
1. Code Quality . . . 1628
2. Project Management . . . 1628
3. Collaboration . . . 1629
Resources and Support . . . 1629
Tools and Libraries . . . 1629
Learning Resources . . . 1629
Conclusion . . . 1629

Static Linking: Why glibc Binaries Are Big, and When musl Is the Answer 1630

Static Linking: Why glibc Binaries Are Big, and When musl Is the Answer 1631
Thesis . . . 1631
Mental model . . . 1631
Worked examples . . . 1632
Case 1: Measure the three builds . . . 1632
Case 2: Ask the linker why . . . 1633
Case 3: Users and hosts . . . 1634
The trap . . . 1635
The boring rule . . . 1636
Try this . . . 1636
Sources . . . 1636

Fil-C: Memory-Safe C with Invisible Capabilities 1637

Fil-C: Memory-Safe C with Invisible Capabilities 1638
Thesis . . . 1638
Mental model . . . 1638
Worked examples . . . 1639
Case 1: The bug class Fil-C turns into panics . . . 1639
Case 2: InvisiCaps at rest (why unions and atomics still work) . . . 1640
Case 3: Fil-xz and the overhead story . . . 1640
Case 4: Anthropic byval / tail-padding fix . . . 1640

Case 5: What v0.686 shipped . . . 1641
The trap . . . 1641
The boring rule . . . 1641
Try this . . . 1641
Sources . . . 1642

File-Descriptor Hygiene: CLOEXEC, close_range, and O_CLOFORK **1643**

File-Descriptor Hygiene: CLOEXEC, close_range, and O_CLOFORK **1644**
Thesis . . . 1644
Mental model . . . 1644
Worked examples . . . 1645
Case 0: What this libc actually defines . . . 1648
Case 1: A leaked pipe write end hides EOF . . . 1650
Case 2: The child inherits the listening socket . . . 1652
Case 3: Audit with `/proc/self/fd` . . . 1655
Case 4: Atomic constructors . . . 1656
Case 5: `close_range`, including `CLOSE_RANGE_CLOEXEC` . . . 1659
Case 6: The threads + `fork` race, shown as a window . . . 1661
Case 7: POSIX.1-2024 `O_CLOFORK`, and who actually has it . . . 1663
The trap . . . 1664
The boring rule . . . 1664
Try this . . . 1664
Sources . . . 1665

fork(), vfork() or posix_spawn(): Starting Programs in Modern C **1666**

`fork()`, `vfork()` or `posix_spawn()`: Starting Programs in Modern C **1667**
Thesis . . . 1667
The three APIs . . . 1667
Measurement: cost versus parent memory . . . 1668
Errors and child setup . . . 1670
When to use which . . . 1671
Pitfalls . . . 1672
Takeaways . . . 1672
Sources . . . 1672

Work Stealing in C: a Chase–Lev Deque with C11 Atomics **1673**

Work Stealing in C: a Chase–Lev Deque with C11 Atomics **1674**
Thesis . . . 1674
Mental model . . . 1674
Worked examples . . . 1676
Case 1: One thread, both ends . . . 1679
Case 2: One owner, seven thieves, two million items . . . 1680
Case 3: Delete one fence and the deque duplicates and loses work . . . 1682
Case 4: Unsigned indices turn “empty” into “an item” . . . 1684
Case 5: ThreadSanitizer does not see fences . . . 1684
Case 6: A scheduler in 100 lines . . . 1686

The trap . . . 1689
The boring rule . . . 1689
Try this . . . 1689
Sources . . . 1690

Open-Addressing Hash Table from Scratch **1691**

Open-Addressing Hash Table from Scratch **1692**
Thesis . . . 1692
Mental model . . . 1692
Worked examples . . . 1692
Case 1: FNV-1a + linear probe + grow . . . 1693
Trap . . . 1694
Boring rule . . . 1695
Try this . . . 1695
Sources . . . 1695

Cuckoo Hashing: Two Tables, Two Probes, and the 50% Wall **1696**

Cuckoo Hashing: Two Tables, Two Probes, and the 50% Wall **1697**
Thesis . . . 1697
The structure . . . 1698
Insert: kick until something is empty . . . 1698
Rehash on failure . . . 1699
Measurement 1: where is the wall? . . . 1699
Measurement 2: what rehashing costs near the wall . . . 1700
Measurement 3: lookups against linear probing at 0.45 . . . 1700
When to choose cuckoo . . . 1700
Takeaways . . . 1701
Appendix: linear-probing baseline and benchmark driver . . . 1701

Catching Heap Buffer Overflows with AddressSanitizer **1703**

Catching Heap Buffer Overflows with AddressSanitizer **1704**
Thesis . . . 1704
Mental model . . . 1704
Worked examples . . . 1704
Case 1: heap buffer overflow . . . 1704
Trap . . . 1705
Boring rule . . . 1705
Try this . . . 1706
Sources . . . 1706

A Toy Mark-and-Sweep Garbage Collector **1707**

A Toy Mark-and-Sweep Garbage Collector **1708**
Thesis . . . 1708
Mental model . . . 1708

Worked examples . . . 1708
Case 1: root keeps a child; garbage dies . . . 1708
Trap . . . 1709
Boring rule . . . 1709
Try this . . . 1709
Sources . . . 1709

NaN Boxing: A Dynamic Value in One 64-bit Word **1710**

NaN Boxing: A Dynamic Value in One 64-bit Word **1711**
Thesis . . . 1711
The bit budget . . . 1711
The encoding, without undefined behaviour . . . 1712
What the words look like . . . 1713
Bug 1: a mask the hardware's own NaN matches . . . 1713
Bug 2: NaN bits from outside . . . 1714
What it buys: half the memory, fewer cache misses . . . 1714
Portability limits . . . 1715
Takeaways . . . 1715
Sources . . . 1716

C11 Atomics and Memory Order **1717**

C11 Atomics and Memory Order **1718**
Thesis . . . 1718
Mental model . . . 1718
Worked example . . . 1718
Trap . . . 1719
Boring rule . . . 1719
Try this . . . 1719
Sources . . . 1720

A Lock-Free SPSC Ring Buffer in C11 **1721**

A Lock-Free SPSC Ring Buffer in C11 **1722**
Thesis . . . 1722
The design in four rules . . . 1722
The code . . . 1723
Test 1: order and integrity under load . . . 1724
Test 2: wraparound . . . 1726
Test 3: what a relaxed publish breaks . . . 1726
Test 4: padding and cached indices, measured . . . 1727
Pitfalls . . . 1728
Takeaways . . . 1729
Sources . . . 1729

Flexible Array Members Done Right **1730**

Flexible Array Members Done Right **1731**
Thesis . . . 1731
Mental model . . . 1731
Worked example . . . 1731
Trap . . . 1732
Boring rule . . . 1732
Try this . . . 1732
Sources . . . 1732

Function Interposition with LD_PRELOAD **1733**

Function Interposition with LD_PRELOAD **1734**
Thesis . . . 1734
Mental model . . . 1734
Worked example . . . 1735
Proving the binding with LD_DEBUG . . . 1737
errno has to survive the wrapper . . . 1737
Where interposition silently does nothing . . . 1738
1. Static binaries . . . 1738
2. Calls made inside libc . . . 1738
3. Raw system calls . . . 1738
4. Secure-execution mode . . . 1739
Writing a wrapper that doesn't bite . . . 1739
Try this . . . 1739
Sources . . . 1739

GCC attribute((cleanup)): scope-based free without C23 defer **1740**

GCC attribute((cleanup)): scope-based free without C23 defer **1741**
Thesis . . . 1741
Mental model . . . 1741
Worked example . . . 1742
Breakdown of execution flow . . . 1742
Advanced usage: Generic auto-free macros and FILE handlers . . . 1743
Helper macros for pointers and files . . . 1743
Cleanup ordering rules . . . 1744
GCC cleanup vs. C23 defer . . . 1744
Common mistakes . . . 1745
1. Wrong indirection level in the cleanup function . . . 1745
2. Using `longjmp` or `exit` . . . 1745
3. Transferring ownership without resetting pointers . . . 1746
Summary . . . 1746
Sources . . . 1746

memfd_create and file seals: anonymous files peers cannot rewrite **1747**

memfd_create and file seals: anonymous files peers cannot rewrite **1748**
Thesis . . . 1748
Flags that matter . . . 1748

Lab: seal, then fail a write . . . 1749
Gotcha we hit first . . . 1750
Producer / consumer sketch . . . 1750
Security note (short) . . . 1750
Pitfalls . . . 1751
Relation to `shm_open` and `mmap(MAP_ANONYMOUS)` . . . 1751
Checklist for a sealed handoff . . . 1751
Summary . . . 1751
Sources . . . 1751

Workbook **1752**

Examples **1753**

Foundations **1754**

Data Types **1755**

Control Flow **1756**

Functions **1757**

Arrays and Strings **1758**

Pointers **1759**

Structures **1760**

File I/O **1761**

Modern C **1762**

Data Structures **1763**

Network **1764**

Embedded **1765**

Performance **1766**

Testing **1767**

Advanced **1768**

Exercises **1769**

Foundations **1770**

Module 1: Foundations Exercises **1771**
Exercise 1: Basic Program Structure . . . 1771
Exercise 2: Personal Information Display . . . 1771
Exercise 3: Simple Calculator Input . . . 1771
Exercise 4: Character Analysis . . . 1771
Exercise 5: Temperature Converter . . . 1772
Exercise 6: String Formatting Practice . . . 1772
Exercise 7: Input Buffer Management . . . 1772
Exercise 8: Comprehensive Demo Program . . . 1772
Solutions and Tips . . . 1773
Exercise 1 Solution Example: . . . 1773
Exercise 3 Solution Example: . . . 1773
Common Pitfalls to Avoid: . . . 1774
Compilation Tips: . . . 1774

Data Types **1775**

Module 2: Data Types and Variables Exercises **1776**
Exercise 1: Data Type Explorer . . . 1776
Exercise 2: Variable Declaration Challenge . . . 1776
Exercise 3: Constants Comparison . . . 1776
Exercise 4: Storage Class Demonstrator . . . 1777
Exercise 5: Type Conversion Safety . . . 1777
Exercise 6: Operator Expression Builder . . . 1777
Exercise 7: Bit Manipulation Toolkit . . . 1777
Exercise 8: Data Type Selection Advisor . . . 1778
Exercise 9: Comprehensive Calculator . . . 1778
Exercise 10: Real-World Application . . . 1778
Solutions and Tips . . . 1778
Exercise 1 Solution Example: . . . 1778
Exercise 3 Solution Example: . . . 1779

Exercise 6 Solution Example: . . . 1780
Exercise 7 Solution Example: . . . 1780
Common Pitfalls to Avoid . . . 1781
Compilation Tips . . . 1782

Control Flow **1783**

Module 3: Control Flow Exercises **1784**
Exercise 1: Grade Calculator with Multiple Conditions . . . 1784
Exercise 2: Pattern Generator . . . 1784
Exercise 3: Menu-Driven Calculator . . . 1784
Exercise 4: Prime Number Analyzer . . . 1784
Exercise 5: Resource Manager with goto . . . 1785
Exercise 6: Function Pointer Calculator . . . 1785
Exercise 7: State Machine Implementation . . . 1785
Exercise 8: Error Handling Framework . . . 1785
Exercise 9: Complex Loop Optimization . . . 1785
Exercise 10: Comprehensive Control Flow Application . . . 1786
Solutions and Tips . . . 1786
Exercise 1 Solution Example: . . . 1786
Exercise 2 Solution Example: . . . 1788
Exercise 3 Solution Example: . . . 1789
Exercise 5 Solution Example: . . . 1792
Common Pitfalls to Avoid . . . 1794
Compilation Tips . . . 1794

Functions **1795**

Module 4: Functions and Modular Programming Exercises **1796**
Exercise 1: Basic Function Implementation . . . 1796
Exercise 2: Recursive Functions . . . 1796
Exercise 3: Variable Argument Functions . . . 1796
Exercise 4: Function Pointers and Callbacks . . . 1797
Exercise 5: Modular Programming with Multiple Files . . . 1797
Exercise 6: Inline Functions and Macro Functions . . . 1797
Exercise 7: Standard Library Function Practice . . . 1797
Exercise 8: Advanced Modular Programming . . . 1798
Solutions and Tips . . . 1798
Exercise 1 Solution Example: . . . 1798
Exercise 2 Solution Example: . . . 1800
Exercise 3 Solution Example: . . . 1802
Exercise 4 Solution Example: . . . 1804
Common Pitfalls to Avoid . . . 1807
Compilation Tips . . . 1807

Arrays and Strings **1808**

Module 5: Arrays and Strings Exercises **1809**
Exercise 1: Array Manipulation Functions . . . 1809
Exercise 2: Matrix Operations . . . 1809
Exercise 3: String Processing Library . . . 1809
Exercise 4: Dynamic Array Implementation . . . 1810
Exercise 5: Multi-dimensional Array Challenges . . . 1810
Exercise 6: String Search and Replace . . . 1810
Exercise 7: Command Line Argument Processing . . . 1810
Exercise 8: Advanced Array and String Integration . . . 1811
Solutions and Tips . . . 1811
Exercise 1 Solution Example: . . . 1811
Exercise 3 Solution Example: . . . 1812
Common Pitfalls to Avoid: . . . 1813
Performance Tips: . . . 1813

Pointers **1815**

Module 6: Pointers Exercises **1816**
Exercise 1: Basic Pointer Operations . . . 1816
Exercise 2: Pointers and Arrays . . . 1816
Exercise 3: Dynamic Memory Allocation . . . 1816
Exercise 4: Pointer to Pointers . . . 1817
Exercise 5: Function Pointers . . . 1817
Exercise 6: Advanced Pointer Techniques . . . 1817
Exercise 7: Pointer Debugging and Safety . . . 1817
Exercise 8: Comprehensive Pointer Application . . . 1818
Solutions and Tips . . . 1818
Exercise 1 Solution Example: . . . 1818
Exercise 3 Solution Example: . . . 1819
Common Pitfalls to Avoid: . . . 1820
Best Practices: . . . 1821

Structures **1822**

Module 7: Structures, Unions, and Enums Exercises **1823**
Exercise 1: Basic Structure Implementation . . . 1823
Exercise 2: Array of Structures . . . 1823
Exercise 3: Linked List Implementation . . . 1823
Exercise 4: Unions and Bit Fields . . . 1824
Exercise 5: Enumerations and Type Definitions . . . 1824
Exercise 6: Advanced Structure Techniques . . . 1824
Exercise 7: Structure Memory Management . . . 1824
Exercise 8: Comprehensive Structure Application . . . 1825
Solutions and Tips . . . 1825
Exercise 1 Solution Example: . . . 1825

Exercise 3 Solution Example: . . . 1826
Common Pitfalls to Avoid: . . . 1828
Best Practices: . . . 1828

File I/O **1829**

Module 8: File I/O Exercises **1830**
Exercise 1: Basic File Operations . . . 1830
Exercise 2: Text File Processing . . . 1830
Exercise 3: Binary File Operations . . . 1830
Exercise 4: File System Operations . . . 1830
Exercise 5: Random Access Files . . . 1831
Exercise 6: Advanced File I/O Techniques . . . 1831
Exercise 7: Error Handling and Recovery . . . 1831
Exercise 8: Comprehensive File Management System . . . 1831
Solutions and Tips . . . 1832
Exercise 1 Solution Example: . . . 1832
Exercise 3 Solution Example: . . . 1834
Common Pitfalls to Avoid: . . . 1836
Best Practices: . . . 1836

Modern C **1837**

Module 9: Modern C Features Exercises **1838**
Exercise 1: C99 Features Implementation . . . 1838
Exercise 2: C11 Standard Features . . . 1838
Exercise 3: C17/C23 New Features . . . 1838
Exercise 4: Type Safety and Modern Practices . . . 1839
Exercise 5: Advanced Preprocessor Techniques . . . 1839
Exercise 6: Memory Management and Allocation . . . 1839
Exercise 7: Error Handling and Diagnostics . . . 1839
Exercise 8: Comprehensive Modern C Application . . . 1840
Solutions and Tips . . . 1840
Exercise 1 Solution Example: . . . 1840
Exercise 2 Solution Example: . . . 1841
Common Pitfalls to Avoid: . . . 1843
Best Practices: . . . 1843

Data Structures **1844**

Module 10: Data Structures Exercises **1845**
Exercise 1: Linear Data Structures Implementation . . . 1845
Exercise 2: Linked List Variations . . . 1845
Exercise 3: Tree Data Structures . . . 1845
Exercise 4: Hash Tables and Associative Arrays . . . 1846
Exercise 5: Graph Representations and Algorithms . . . 1846

Exercise 6: Advanced Data Structures . . . 1846
Exercise 7: Data Structure Performance Analysis . . . 1846
Exercise 8: Comprehensive Data Structure Application . . . 1847
Solutions and Tips . . . 1847
Exercise 1 Solution Example: . . . 1847
Exercise 3 Solution Example: . . . 1850
Common Pitfalls to Avoid: . . . 1853
Best Practices: . . . 1853

Network **1854**

Module 11: Network Programming Exercises **1855**
Exercise 1: Basic Socket Programming . . . 1855
Exercise 2: HTTP Client Implementation . . . 1855
Exercise 3: Multi-client Server . . . 1855
Exercise 4: Network Protocol Implementation . . . 1856
Exercise 5: Advanced Networking Concepts . . . 1856
Exercise 6: Network Security Implementation . . . 1856
Exercise 7: Network Debugging and Monitoring . . . 1856
Exercise 8: Comprehensive Network Application . . . 1857
Solutions and Tips . . . 1857
Exercise 1 Solution Example: . . . 1857
Exercise 2 Solution Example: . . . 1861
Common Pitfalls to Avoid: . . . 1864
Best Practices: . . . 1864

Embedded **1865**

Module 12: Embedded Systems Programming Exercises **1866**
Exercise 1: Bit Manipulation and Register Operations . . . 1866
Exercise 2: Memory-Mapped I/O Implementation . . . 1866
Exercise 3: Interrupt Service Routines . . . 1866
Exercise 4: Real-Time Constraints Implementation . . . 1867
Exercise 5: Low-Power Programming Techniques . . . 1867
Exercise 6: Hardware Abstraction Layer (HAL) . . . 1867
Exercise 7: Embedded Debugging and Monitoring . . . 1867
Exercise 8: Comprehensive Embedded Application . . . 1868
Solutions and Tips . . . 1868
Exercise 1 Solution Example: . . . 1868
Exercise 2 Solution Example: . . . 1870
Common Pitfalls to Avoid: . . . 1872
Best Practices: . . . 1872

Performance **1873**

Module 13: Performance Optimization Exercises **1874**
Exercise 1: Algorithm Complexity Analysis . . . 1874
Exercise 2: Memory Access Patterns Optimization . . . 1874
Exercise 3: Compiler Optimization Techniques . . . 1874
Exercise 4: Data Structure Optimization . . . 1875
Exercise 5: Parallel Processing Implementation . . . 1875
Exercise 6: SIMD and Vectorization . . . 1875
Exercise 7: Memory Management Optimization . . . 1875
Exercise 8: Comprehensive Performance Optimization . . . 1876
Solutions and Tips . . . 1876
Exercise 1 Solution Example: . . . 1876
Exercise 2 Solution Example: . . . 1879
Common Pitfalls to Avoid: . . . 1881
Best Practices: . . . 1881

Testing **1883**

Module 14: Testing and Quality Assurance Exercises **1884**
Exercise 1: Unit Testing Framework Implementation . . . 1884
Exercise 2: Mocking and Stubbing Techniques . . . 1884
Exercise 3: Integration Testing Implementation . . . 1884
Exercise 4: Code Coverage Analysis . . . 1885
Exercise 5: Static Analysis and Linting . . . 1885
Exercise 6: Fuzz Testing Implementation . . . 1885
Exercise 7: Performance and Stress Testing . . . 1885
Exercise 8: Comprehensive Testing Strategy . . . 1886
Solutions and Tips . . . 1886
Exercise 1 Solution Example: . . . 1886
Exercise 2 Solution Example: . . . 1889
Common Pitfalls to Avoid: . . . 1893
Best Practices: . . . 1893

Advanced **1894**

Module 15: Advanced Topics Exercises **1895**
Exercise 1: Metaprogramming with Macros . . . 1895
Exercise 2: Generic Programming Techniques . . . 1895
Exercise 3: Coroutines and Cooperative Multitasking . . . 1895
Exercise 4: Reflection and Introspection . . . 1896
Exercise 5: Template Metaprogramming Simulation . . . 1896
Exercise 6: Memory Model and Concurrency . . . 1896
Exercise 7: Language Interoperability . . . 1896
Exercise 8: Comprehensive Advanced Application . . . 1897
Solutions and Tips . . . 1897
Exercise 1 Solution Example: . . . 1897

Exercise 2 Solution Example: . . . 1899
Common Pitfalls to Avoid: . . . 1902
Best Practices: . . . 1903

Projects **1904**

Advanced **1905**

Advanced Projects **1906**
Project 1: Concurrent Web Server . . . 1906
Description . . . 1906
Learning Objectives . . . 1906
Requirements . . . 1906
Implementation Steps . . . 1906
Sample Code Structure . . . 1907
Common Pitfalls to Avoid . . . 1909
Best Practices . . . 1909
Project 2: Memory Allocator . . . 1910
Description . . . 1910
Learning Objectives . . . 1910
Requirements . . . 1910
Implementation Steps . . . 1910
Sample Code Structure . . . 1910
Common Pitfalls to Avoid . . . 1912
Best Practices . . . 1913
Project 3: Real-time Data Processing System . . . 1913
Description . . . 1913
Learning Objectives . . . 1913
Requirements . . . 1913
Implementation Steps . . . 1914
Sample Code Structure . . . 1914
Common Pitfalls to Avoid . . . 1916
Best Practices . . . 1917
Project 4: Embedded System Simulator . . . 1917
Description . . . 1917
Learning Objectives . . . 1917
Requirements . . . 1917
Implementation Steps . . . 1917
Sample Code Structure . . . 1918
Common Pitfalls to Avoid . . . 1920
Best Practices . . . 1921
Tips for Success . . . 1921

Beginner **1922**

Beginner Projects **1923**
Project 1: Personal Information Manager . . . 1923
Description . . . 1923
Learning Objectives . . . 1923
Requirements . . . 1923
Implementation Steps . . . 1923
Sample Code Structure . . . 1924
Common Pitfalls to Avoid . . . 1925
Best Practices . . . 1925
Project 2: Simple Calculator . . . 1925
Description . . . 1925
Learning Objectives . . . 1926
Requirements . . . 1926
Implementation Steps . . . 1926
Sample Code Structure . . . 1926
Common Pitfalls to Avoid . . . 1927
Best Practices . . . 1927
Project 3: Number Guessing Game . . . 1927
Description . . . 1927
Learning Objectives . . . 1927
Requirements . . . 1928
Implementation Steps . . . 1928
Sample Code Structure . . . 1928
Common Pitfalls to Avoid . . . 1929
Best Practices . . . 1929
Project 4: Temperature Converter . . . 1930
Description . . . 1930
Learning Objectives . . . 1930
Requirements . . . 1930
Implementation Steps . . . 1930
Sample Code Structure . . . 1930
Common Pitfalls to Avoid . . . 1931
Best Practices . . . 1932
Tips for Success . . . 1932

Capstone **1933**

Capstone Projects **1934**
Project 1: Distributed Chat Application . . . 1934
Description . . . 1934
Learning Objectives . . . 1934
Requirements . . . 1934
Implementation Steps . . . 1935
Sample Code Structure . . . 1935
Key Challenges . . . 1939

Evaluation Criteria . . . 1939
Project 2: Operating System Kernel . . . 1939
Description . . . 1939
Learning Objectives . . . 1940
Requirements . . . 1940
Implementation Steps . . . 1941
Sample Code Structure . . . 1941
Key Challenges . . . 1945
Evaluation Criteria . . . 1945
Project 3: Database Management System . . . 1946
Description . . . 1946
Learning Objectives . . . 1946
Requirements . . . 1946
Implementation Steps . . . 1947
Sample Code Structure . . . 1947
Key Challenges . . . 1952
Evaluation Criteria . . . 1952
Project 4: Game Engine . . . 1952
Description . . . 1952
Learning Objectives . . . 1952
Requirements . . . 1953
Implementation Steps . . . 1953
Sample Code Structure . . . 1954
Key Challenges . . . 1960
Evaluation Criteria . . . 1960
Tips for Capstone Success . . . 1960

Intermediate **1961**

Intermediate Projects **1962**
Project 1: Student Grade Management System . . . 1962
Description . . . 1962
Learning Objectives . . . 1962
Requirements . . . 1962
Implementation Steps . . . 1962
Sample Code Structure . . . 1963
Common Pitfalls to Avoid . . . 1965
Best Practices . . . 1965
Project 2: Simple Text Editor . . . 1965
Description . . . 1965
Learning Objectives . . . 1965
Requirements . . . 1965
Implementation Steps . . . 1966
Sample Code Structure . . . 1966
Common Pitfalls to Avoid . . . 1967
Best Practices . . . 1968

Project 3: Library Management System 1968
Description 1968
Learning Objectives 1968
Requirements 1968
Implementation Steps 1968
Sample Code Structure 1969
Common Pitfalls to Avoid 1971
Best Practices 1971
Project 4: Simple Database Engine 1971
Description 1971
Learning Objectives 1972
Requirements 1972
Implementation Steps 1972
Sample Code Structure 1972
Common Pitfalls to Avoid 1974
Best Practices 1975
Tips for Success 1975

Appendices **1976**

Appendix A: Additional Resources **1977**
Books and Publications 1977
Core C Programming 1977
Systems Programming 1977
Data Structures and Algorithms 1977
Online Resources 1978
Official Documentation 1978
Tutorials and Learning Platforms 1978
Reference Sites 1979
Development Tools 1979
Compilers 1979
Debugging Tools 1979
Profiling Tools 1980
Static Analysis Tools 1980
Libraries and Frameworks 1980
Standard Libraries 1980
Networking Libraries 1981
Data Structure Libraries 1981
Communities and Forums 1981
Online Communities 1981
Mailing Lists 1982
Conferences and Events 1982
Major Conferences 1982
Open Source Projects 1982
Learning from Real Code 1982
Practice Platforms 1983
Coding Challenges 1983

Standards Organizations . . . 1983
Key Organizations . . . 1983
Additional Learning Resources . . . 1983
Video Courses . . . 1983
Podcasts . . . 1984

Appendix B: C Language Reference **1985**
C Keywords . . . 1985
Standard C Keywords (C17/C18) . . . 1985
Deprecated Keywords . . . 1985
Data Types and Sizes . . . 1985
Basic Data Types . . . 1985
Type Specifiers . . . 1986
Type Qualifiers . . . 1986
Operators . . . 1986
Arithmetic Operators . . . 1986
Relational Operators . . . 1986
Logical Operators . . . 1987
Bitwise Operators . . . 1987
Assignment Operators . . . 1987
Other Operators . . . 1988
Standard Library Functions . . . 1988
Input/Output Functions (<stdio.h>) . . . 1988
String Functions (<string.h>) . . . 1989
Memory Management Functions (<stdlib.h>) . . . 1989
Mathematical Functions (<math.h>) . . . 1989
Utility Functions (<stdlib.h>) . . . 1990
Time Functions (<time.h>) . . . 1991
Preprocessor Directives . . . 1991
Common Directives . . . 1991
Standard Macros . . . 1991
Format Specifiers . . . 1991
printf Format Specifiers . . . 1991
scanf Format Specifiers . . . 1992
Format Modifiers . . . 1992
Escape Sequences . . . 1993
Standard Escape Sequences . . . 1993
Octal and Hexadecimal Escapes . . . 1993
Storage Classes . . . 1993
Storage Class Specifiers . . . 1993
Error Handling . . . 1994
errno Values (<errno.h>) . . . 1994
Assertions (<assert.h>) . . . 1994
Signal Handling (<signal.h>) . . . 1994
Signal Types . . . 1994
Signal Functions . . . 1994
File Operations Modes . . . 1994
fopen Modes . . . 1994

Binary Mode Suffix . . . 1995
Standard Streams . . . 1995
Predefined Streams . . . 1995
Limits and Constants . . . 1995
Implementation Limits (<limits.h>) . . . 1995
Floating-Point Limits (<float.h>) . . . 1996
Thread Support (C11) (<threads.h>) . . . 1996
Thread Functions . . . 1996
Mutex Functions . . . 1996
Condition Variable Functions . . . 1996
Atomic Operations (C11) (<stdatomic.h>) . . . 1997
Atomic Types . . . 1997
Atomic Operations . . . 1997
Generic Selection (C11) (<stdnoreturn.h>) . . . 1997
_Generic Keyword . . . 1997
Alignment (C11) (<stdalign.h>) . . . 1998
Alignment Specifiers . . . 1998

Appendix C: Exercise Solutions **1999**
Module 1: Foundations . . . 1999
Exercise 1.1: Hello World Variations . . . 1999
Exercise 1.2: Basic Calculator . . . 1999
Module 2: Data Types . . . 2000
Exercise 2.1: Size and Range Calculator . . . 2000
Exercise 2.2: Temperature Converter . . . 2001
Module 3: Control Flow . . . 2002
Exercise 3.1: Prime Number Checker . . . 2002
Exercise 3.2: Pattern Printer . . . 2003
Module 4: Functions . . . 2004
Exercise 4.1: Mathematical Functions Library . . . 2004
Exercise 4.2: Recursive Functions . . . 2007
Module 5: Arrays and Strings . . . 2008
Exercise 5.1: Array Operations . . . 2008
Exercise 5.2: String Manipulation . . . 2010
Module 6: Pointers . . . 2012
Exercise 6.1: Pointer Arithmetic . . . 2012
Exercise 6.2: Dynamic Memory Allocation . . . 2013
Module 7: Structures . . . 2015
Exercise 7.1: Student Database . . . 2015
Exercise 7.2: Complex Number Operations . . . 2018
Module 8: File I/O . . . 2021
Exercise 8.1: Text File Processor . . . 2021
Exercise 8.2: Binary File Operations . . . 2024
Module 9: Modern C Features . . . 2027
Exercise 9.1: C99 Features Demonstration . . . 2027
Exercise 9.2: C11 Features Demonstration . . . 2028
Module 10: Data Structures . . . 2030
Exercise 10.1: Linked List Implementation . . . 2030

Exercise 10.2: Binary Search Tree Implementation 2034
Module 11: Network Programming 2038
Exercise 11.1: Simple TCP Client-Server 2038
Exercise 11.2: HTTP Client 2041
Module 12: Embedded Systems Programming 2043
Exercise 12.1: Bit Manipulation Library 2043
Exercise 12.2: Memory-Mapped I/O Simulation 2044
Module 13: Performance Optimization 2047
Exercise 13.1: Algorithm Complexity Analysis 2047
Exercise 13.2: Memory Access Patterns 2049
Module 14: Testing and Quality Assurance 2052
Exercise 14.1: Unit Testing Framework 2052
Exercise 14.2: Mocking Framework 2055
Module 15: Advanced Topics 2059
Exercise 15.1: Multithreading Example 2059
Exercise 15.2: System Programming Example 2060

C Programming

A modular C library: foundations through systems topics, with a separate workbook for extra drills and projects.

This book uses **C17** as the default dialect. Examples prefer:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
./prog
```

Clang is fine as a drop-in for learning:

```
clang -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
```

Who this is for

Path A — Beginner (language first)
Start at Module 1 and move in order through Module 5 before jumping ahead. Type every full program. Treat warnings as errors in practice (`-Werror` once you are comfortable). Goal: confident control flow, functions, arrays/strings, and a first honest encounter with pointers.

Path B — Systems-oriented (already know some C syntax)
Skim Module 1 for the compile/link mental model and warning flags, then prioritize Modules 6–8 (pointers, structures, file/system I/O). Use Modules 11–15 when you need sockets, embedded patterns, performance, testing, or threads. Still run Module 1's multi-file and I/O labs if those habits are rusty.

Both paths should enable warnings early and avoid treating "it compiled" as "it is correct." Undefined behavior is introduced early for a reason.

How to use this book

1. Read a section, then **type** the programs (do not only read them).
2. Compile with `-std=c17 -Wall -Wextra -Wpedantic`.
3. Break one thing on purpose; read the diagnostic; fix it.
4. Do the chapter exercises before moving on.
5. Use the workbook when you want more reps or a larger project—not as a substitute for chapter work.

How to compile examples

From any directory where you saved a single-file example:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o hello hello.c
./hello
```

Multi-file:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o app main.c util.c
# or
gcc -std=c17 -Wall -Wextra -Wpedantic -c main.c
gcc -std=c17 -Wall -Wextra -Wpedantic -c util.c
gcc -o app main.o util.o
```

Useful extras:

Flags	Purpose
`-g -O0`	Debug builds
`-O2`	Optimized builds
`-fsanitize=address,undefined`	Runtime checks (GCC/Clang, host-dependent)

Workbook demos live under `content/99-workbook/examples/` (per module). Some multi-file folders include a `Makefile`—read it, then still know how to issue the raw `gcc` line yourself.

Workbook (`content/99-workbook`)

Area	Path	Use for
Examples	`content/99-workbook/examples/`	Runnable demos aligned to modules
Exercises	`content/99-workbook/exercises/`	Extra problem sets
Projects	`content/99-workbook/projects/`	Beginner → capstone project prompts

Suggested rhythm: finish the module chapters → attempt workbook exercises for that module → pick a project only when the drills feel mechanical.

Appendices under `content/100-appendices/` hold resources, quick reference material, and solution notes.

Related books

Book	Overlap
Linux	Toolchain, make, and systems environment
Go	Contrasting memory and concurrency models
Maths	Discrete math and complexity behind algorithms chapters

Modules (15 + workbook + appendices)

1. **Foundations and Environment** — history and process model, toolchain, first programs, console I/O
2. **Data Types and Variables** — types, storage, conversions, operators
3. **Control Flow** — decisions, loops, non-local flow
4. **Functions and Modularity** — design, linkage, library overview
5. **Arrays and Strings** — contiguous data and text
6. **Pointers and Memory** — addresses, dynamic allocation, pitfalls
7. **Structures and Types** — `struct`, `union`, `enum`, simple ADTs
8. **File I/O and System Programming** — streams and system-facing I/O
9. **Modern C Standards** — C99 through C23 and modern practice
10. **Data Structures and Algorithms** — lists, trees, tables, graphs, sorting
11. **Network Programming** — sockets and protocols
12. **Embedded Systems** — constrained and hardware-oriented C
13. **Performance Optimization** — measure, then improve
14. **Testing and Debugging** — quality, debug, error handling
15. **Advanced Topics and Capstone** — concurrency, systems topics, capstone

Plus **99-workbook** (examples, exercises, projects) and **appendices**.

Prerequisites

- Comfort using a text editor and a terminal

- Ability to install a C compiler (GCC, Clang, or MSVC/MinGW on Windows)
- No prior programming required for Path A; Path B assumes basic syntax familiarity

Getting started

1. Introduction to C — mental model and first mini-programs
2. Development Environment — install and verify the toolchain
3. Your First C Program — compile, multi-file, labs
4. Basic I/O — `printf` / `scanf` / `fgets`

Then continue through the modules in order (Path A) or jump to pointers and systems modules (Path B) once foundations are solid.

Intro

Comprehensive C Programming Syllabus

Course Overview

This comprehensive C programming course is designed to take students from complete beginners to advanced practitioners capable of writing efficient, maintainable, and professional C code. The syllabus draws from classic texts like Kernighan & Ritchie's "The C Programming Language," modern best practices, and real-world applications in systems programming, embedded development, and network programming.

Learning Objectives

By the end of this course, students will be able to: - Write efficient and maintainable C programs - Understand memory management and pointer manipulation - Develop system-level applications - Create network applications and embedded systems - Apply modern C development practices and tooling - Debug and optimize C programs - Work with various C standards (C89, C99, C11, C17, C23)

Prerequisites

- Basic understanding of computer systems
- Familiarity with command-line interfaces
- Mathematical reasoning skills
- No prior programming experience required

Course Structure

The course is divided into 12 comprehensive modules, progressing from fundamentals to specialized applications.

Module 1: Foundations and Environment Setup

Duration: 1-2 weeks

1.1 Introduction to C Programming

- History and evolution of C (1972-present)
- Why C remains crucial in modern programming (2024 perspective)
- **Current C standards: C89/C90, C99, C11, C17, C23 (latest - ISO/IEC 9899:2024)**
- Relationship to other languages (C++, Rust, Go, Zig influence)
- Modern applications: Operating Systems, Embedded Systems, IoT, WebAssembly, System Tools
- C in cloud computing and microservices
- Performance-critical applications and real-time systems

1.2 Development Environment Setup

- **Modern compilers supporting C23**: GCC 14+, Clang 18+, MSVC 2022+
- Cross-platform development: MinGW-w64, WSL2, Docker containers
- **Modern IDEs and editors**: VS Code with C/C++ extensions, CLion 2024, Neovim/Vim with LSP
- **Cloud development**: GitHub Codespaces, Gitpod, online compilers
- Command-line tools and build systems: Make, CMake 3.25+, Ninja, Meson
- **Modern version control**: Git workflows, GitHub/GitLab CI/CD
- **Advanced debugging tools**: GDB 13+, LLDB, Valgrind, AddressSanitizer, ThreadSanitizer
- **Package management**: vcpkg, Conan 2.0, CPM.cmake
- **Static analysis**: Clang Static Analyzer, PVS-Studio, Cppcheck, SonarQube

1.3 First C Program

- Structure of a C program
- The `main()` function
- Comments and code style
- Compilation process: preprocessing, compilation, linking
- Understanding object files and executables

1.4 Basic I/O

- `printf()` and format specifiers
- `scanf()` and input handling
- Character I/O with `getchar()` and `putchar()`
- Introduction to streams

Projects: - Hello World variations - Simple calculator - Number guessing game

---

Module 2: Data Types and Variables

Duration: 2 weeks

2.1 Fundamental Data Types

- Integer types: `char`, `short`, `int`, `long`, `long long`
- Floating-point types: `float`, `double`, `long double`
- Boolean type (`_Bool`, `<stdbool.h>`)
- Size and range considerations
- Fixed-width integer types (`<stdint.h>`)

2.2 Variables and Constants

- Variable declaration and initialization
- Scope and lifetime
- Storage classes: `auto`, `static`, `extern`, `register`
- Constants: `const` keyword, `#define`, `enum`
- Volatile qualifier

2.3 Type Conversions

- Implicit conversions and promotion rules
- Explicit casting
- Common pitfalls and warnings

2.4 Operators and Expressions

- Arithmetic operators and precedence
- Relational and logical operators
- Bitwise operators
- Assignment operators
- Increment/decrement operators
- Conditional operator
- `sizeof` operator

Projects: - Temperature conversion utility - Bitwise operations calculator - Simple arithmetic expression evaluator

---

Module 3: Control Flow

Duration: 2 weeks

3.1 Conditional Statements

- `if`, `else if`, `else` statements
- Nested conditionals
- `switch` statements and `break`
- Best practices for conditional logic

3.2 Loops

- `while` and `do-while` loops
- `for` loops and variants
- Nested loops
- Loop control: `break`, `continue`
- Infinite loops and when to use them

3.3 Program Flow Control

- `goto` statement (when and why to avoid)
- Function calls as control flow
- Error handling patterns

Projects: - Menu-driven program - Pattern printing programs - Prime number generator - Simple text-based games (Tic-tac-toe, Hangman)

Module 4: Functions and Modular Programming

Duration: 2-3 weeks

4.1 Function Fundamentals

- Function declaration and definition
- Parameter passing (by value)
- Return values and `return` statement
- Function prototypes and headers
- Scope rules and name hiding

4.2 Advanced Function Concepts

- Recursive functions
- Variable argument functions (`<stdarg.h>`)
- Inline functions (C99)
- Function pointers and callbacks
- Higher-order functions

4.3 Modular Programming

- Header files and source files
- Include guards and `#pragma once`
- Compilation units and linking
- Static vs. external linkage
- Creating and using libraries

4.4 Standard Library Functions

- String functions (`<string.h>`)
- Mathematical functions (`<math.h>`)
- Character classification (`<ctype.h>`)
- Time functions (`<time.h>`)
- Random number generation (`<stdlib.h>`)

Projects: - Mathematical library implementation - Text processing utilities - Recursive algorithms (factorial, Fibonacci, tree traversal) - Function pointer demonstration programs

---

Module 5: Arrays and Strings

Duration: 2-3 weeks

5.1 Arrays

- Array declaration and initialization
- Multi-dimensional arrays
- Array bounds and buffer overflows
- Variable-length arrays (C99)
- Arrays as function parameters

5.2 Strings

- Character arrays and null termination
- String literals and string constants
- String input/output functions
- String manipulation functions
- Common string operations

5.3 Advanced Array Concepts

- Array of arrays vs. multi-dimensional arrays
- Jagged arrays using pointers
- Dynamic arrays (preview to pointers)
- Array algorithms: searching, sorting

Projects: - Text analyzer (word count, character frequency) - Simple sorting algorithms implementation - String manipulation library - Command-line argument processor

---

Module 6: Pointers and Memory Management

Duration: 3-4 weeks

6.1 Pointer Fundamentals

- Understanding memory addresses
- Pointer declaration and initialization
- Dereferencing and address-of operators
- Null pointers and pointer validation
- Pointer arithmetic

6.2 Pointers and Arrays

- Array-pointer equivalence
- Pointer subscripting
- Multi-dimensional arrays via pointers
- Function parameters: arrays vs. pointers

6.3 Dynamic Memory Management

- `malloc()`, `calloc()`, `realloc()`, `free()`
- Memory leaks and double-free errors
- Memory debugging tools
- Best practices for dynamic allocation

6.4 Advanced Pointer Concepts

- Pointers to pointers
- Function pointers advanced usage
- Void pointers and generic programming
- Const correctness with pointers

6.5 Common Pointer Pitfalls

- Dangling pointers
- Buffer overflows
- Segmentation faults
- Memory alignment issues

Projects: - Dynamic array implementation - Linked list data structure - Memory allocator simulation - Generic sorting function using function pointers

---

Module 7: Structures and User-Defined Types

Duration: 2-3 weeks

7.1 Structures

- Structure declaration and definition
- Structure initialization and assignment
- Nested structures
- Arrays of structures
- Pointers to structures

7.2 Unions and Enumerations

- Union declaration and usage
- Anonymous unions (C11)
- Enumeration types
- Enumeration with explicit values

7.3 Advanced Structure Concepts

- Bit fields
- Structure packing and alignment
- Flexible array members (C99)
- Self-referential structures

7.4 Abstract Data Types

- Information hiding principles
- Opaque pointers
- Constructor/destructor patterns
- Interface design in C

Projects: - Student record management system - Binary tree implementation - Stack and queue data structures - Simple database using structures

---

Module 8: File I/O and System Programming

Duration: 2-3 weeks

8.1 File Operations

- File pointers and `FILE` structure
- Opening and closing files
- Text vs. binary modes
- Error handling in file operations

8.2 File I/O Functions

- Character I/O: `fgetc()`, `fputc()`, `ungetc()`
- Line I/O: `fgets()`, `fputs()`
- Formatted I/O: `fprintf()`, `fscanf()`
- Binary I/O: `fread()`, `fwrite()`

8.3 Advanced File Operations

- File positioning: `fseek()`, `ftell()`, `rewind()`
- Random access files
- Temporary files
- File locking and concurrent access

8.4 System-Level Programming

- Command-line arguments
- Environment variables
- Exit status and `atexit()`
- Signal handling basics
- Process creation (system calls overview)

Projects: - File encryption/decryption utility - Log file analyzer - Configuration file parser - Simple text editor

---

Module 9: Advanced C Features and Modern Standards

Duration: 2-3 weeks

9.1 C99 Features

- Variable-length arrays
- Designated initializers
- Compound literals
- `restrict` keyword
- `inline` functions
- New data types (`long long`, `_Bool`)

9.2 C11 Features

- Anonymous structures and unions
- Static assertions (`_Static_assert`)
- Improved Unicode support
- Thread-local storage
- Bounds-checking functions (Annex K)

9.3 C17 and C23 Features

- C17 (ISO/IEC 9899:2018): Technical corrigendum with bug fixes and clarifications
- **C23 (ISO/IEC 9899:2024): Latest Standard - Official Release**
  - `typeof` and `typeof_unqual` operators
  - `_BitInt(N)` extended integer types
  - Binary integer constants (e.g., `0b1010`)
  - `#elifdef` and `#elifndef` preprocessor directives
  - `#warning` directive
  - `#embed` directive for binary resource inclusion

- `[[]]` attribute syntax support
- `[[noreturn]]`, `[[maybe_unused]]`, `[[deprecated]]` attributes
- `[[nodiscard]]`, `[[fallthrough]]` attributes
- `_Decimal128`, `_Decimal32`, `_Decimal64` decimal floating types
- Improved `_Static_assert` (message now optional)
- Enhanced integer constant expressions
- `auto` type inference (limited scope)
- Improved Unicode support (char8_t via `__STDC_UTF_8__`)
- New standard library functions in `<stdbit.h>`
- Memory alignment improvements
- `strdup()`, `strndup()` functions standardized
- `aligned_alloc()` improvements
- Enhanced time utilities
- Better support for embedded systems

9.4 Modern C Development Practices (2024)

- **DevOps and C**: CI/CD pipelines for C projects
- **Containerized development**: Docker for C development environments
- **Cross-compilation**: Modern toolchains and target support
- **WebAssembly (WASM)**: Compiling C to WebAssembly with Emscripten
- **Cloud-native C**: Microservices and serverless functions in C
- **Supply chain security**: Dependency management and vulnerability scanning
- **Documentation as code**: Doxygen, Sphinx integration
- **Modern testing**: Fuzzing, property-based testing, mutation testing

9.5 Advanced Preprocessor

- Macro definitions and function-like macros
- Conditional compilation
- Pragma directives
- Predefined macros
- Token pasting and stringification

Projects: - Generic data structure library using macros - Configuration system with conditional compilation - Cross-platform compatibility layer - C23 feature detection and compatibility testing - Modern C development workflow setup

Note on C23 Compiler Support (2024): - **GCC 14+**: Partial C23 support, `typeof`, binary literals, `#elifdef` - **Clang 18+**: Growing C23 support, attributes, enhanced preprocessor - **MSVC 2022**: Limited C23 features, focus on C11/C17 compliance - **Industry adoption**: Gradual migration, embedded systems leading adoption - **Compatibility strategies**: Feature detection macros, fallback implementations

---

Module 10: Data Structures and Algorithms

Duration: 3-4 weeks

10.1 Linear Data Structures

- Dynamic arrays (vectors)
- Linked lists (singly, doubly, circular)
- Stacks and queues
- Deques and priority queues

10.2 Tree Structures

- Binary trees and binary search trees
- AVL trees and red-black trees
- B-trees and B+ trees
- Tries and suffix trees

10.3 Hash Tables

- Hash function design
- Collision resolution strategies
- Dynamic resizing
- Performance analysis

10.4 Graph Algorithms

- Graph representations
- Breadth-first and depth-first search
- Shortest path algorithms
- Minimum spanning trees

10.5 Sorting and Searching

- Comparison-based sorting algorithms
- Non-comparison sorting (radix, counting)
- Binary search variations
- Selection algorithms

Projects: - Complete data structure library - Graph algorithms visualization - Sorting algorithm performance comparison - Hash table implementation with benchmarks

---

Module 11: Network Programming

Duration: 3-4 weeks

11.1 Network Fundamentals

- OSI and TCP/IP models
- Sockets concept
- IPv4 and IPv6 addressing
- Ports and protocols

11.2 Socket Programming

- Socket creation and configuration
- TCP client-server programming
- UDP programming
- Address structures and conversion functions

11.3 Advanced Network Programming

- Non-blocking I/O
- I/O multiplexing (`select()`, `poll()`, `epoll()`)
- Multi-threading in network programs
- Signal handling in network applications

11.4 Network Protocols

- HTTP client implementation
- FTP basics
- DNS resolution
- Custom protocol design

11.5 Security Considerations

- Buffer overflow prevention
- Input validation
- Encryption basics (with OpenSSL)
- Secure coding practices

Projects: - Echo server and client - HTTP web server - Chat application - File transfer utility - Network monitoring tools

---

Module 12: Embedded Systems Programming

Duration: 3-4 weeks

12.1 Embedded Systems Basics

- Microcontroller architecture
- Memory models (Harvard vs. Von Neumann)
- Real-time constraints
- Resource limitations

12.2 Low-Level Programming

- Direct memory access
- Hardware registers manipulation
- Interrupt service routines
- Device drivers basics

12.3 Embedded C Techniques

- Volatile variables usage
- Bit manipulation for hardware control
- Fixed-point arithmetic
- Code optimization for space and speed

12.4 Real-Time Programming

- Task scheduling concepts
- Inter-task communication
- Synchronization primitives
- Real-time operating systems (RTOS) basics

12.5 Hardware Interfaces

- GPIO programming
- Serial communication (UART, SPI, I2C)
- Analog-to-digital conversion
- PWM and timing control

Projects: - LED control system - Sensor data acquisition - Motor control application - Communication protocol implementation - Embedded web server (if hardware permits)

Module 13: Performance Optimization and Profiling

Duration: 2 weeks

13.1 Performance Analysis

- Profiling tools (gprof, perf, Instruments)
- Time and space complexity analysis
- Benchmarking methodologies
- Performance bottleneck identification

13.2 Optimization Techniques

- Algorithmic optimizations
- Data structure selection
- Cache-friendly programming
- Compiler optimizations

13.3 Memory Optimization

- Memory alignment and padding
- Memory pool allocation
- Stack vs. heap usage
- Memory-mapped files

13.4 Advanced Optimization

- SIMD instructions
- Loop optimizations
- Function inlining strategies
- Profile-guided optimization

Projects: - Performance comparison suite - Memory usage analyzer - Optimized mathematical library - Real-world application optimization

---

Module 14: Testing and Debugging

Duration: 2 weeks

14.1 Testing Methodologies

- Unit testing in C
- Integration testing
- Test-driven development
- Testing frameworks (Unity, CUnit, Check)

14.2 Debugging Techniques

- GDB usage and advanced features
- Static analysis tools (lint, cppcheck, clang-analyzer)
- Dynamic analysis (Valgrind, AddressSanitizer)
- Debugging embedded systems

14.3 Error Handling

- Error codes and return values
- Exception simulation in C
- Logging and tracing
- Graceful degradation

14.4 Code Quality

- Code review practices
- Static analysis integration
- Continuous integration
- Documentation standards

Projects: - Comprehensive test suite for previous projects - Debugging exercise with intentional bugs - Code quality assessment tools

---

Module 15: Advanced Topics and Capstone

Duration: 3-4 weeks

15.1 Multi-threading and Concurrency

- POSIX threads (pthreads)
- Thread synchronization (mutexes, semaphores)
- Thread-safe programming
- Lock-free data structures

15.2 Advanced System Programming

- Inter-process communication
- Shared memory
- Memory mapping
- System call wrappers

15.3 Domain-Specific Applications

- Compiler construction basics
- Operating system components
- Database engine components
- Graphics programming

15.4 Capstone Project

- Project planning and design
- Implementation phases
- Testing and documentation
- Code review and presentation

Capstone Project Ideas: - Mini operating system kernel - Database management system - Network protocol implementation - Embedded control system - Compiler for a simple language - Real-time data processing system

---

Assessment Methods

Continuous Assessment (60%)

- Weekly programming assignments (30%)
- Module projects (20%)
- Code review participation (10%)

Major Projects (25%)

- Mid-term project (Module 6-8)
- Capstone project (Module 15)

Examinations (15%)

- Practical coding exams
- Theoretical understanding tests

Recommended Resources

Essential Books

1. **Kernighan, Brian W. & Ritchie, Dennis M.** - "The C Programming Language" (2nd Edition) - Still the definitive classic
2. **King, K.N.** - "C Programming: A Modern Approach" (2nd Edition)
3. **Harbison, Samuel P. & Steele, Guy L.** - "C: A Reference Manual" (5th Edition)
4. **Gustedt, Jens** - "Modern C" (2019) - Covers C11/C17 with modern practices

Advanced and Specialized References

5. **Stevens, W. Richard & Rago, Stephen A.** - "Advanced Programming in the UNIX Environment" (3rd Edition, 2013)
6. **Stevens, W. Richard** - "UNIX Network Programming" (3rd Edition)
7. **Barr, Michael & Massa, Anthony** - "Programming Embedded Systems" (2nd Edition)
8. **Seacord, Robert C.** - "Effective C" (2020) - Modern secure coding practices
9. **van der Linden, Peter** - "Expert C Programming" - Classic advanced techniques

Current Standards and Specifications

10. **ISO/IEC 9899:2024** - C23 Standard (Latest Official Release)
11. **ISO/IEC 9899:2018** - C17 Standard
12. **ISO/IEC 9899:2011** - C11 Standard
13. **POSIX.1-2017** - IEEE Std 1003.1-2017

Modern Online Resources (2024)

- **C23 Documentation**: Official ISO documentation and compiler support matrices
- **cppreference.com/c** - Comprehensive C reference with C23 updates
- **GNU C Library Documentation** - Latest glibc documentation
- **Clang/LLVM Documentation** - Modern compiler features and C23 support
- **GCC Documentation** - C23 implementation status and extensions
- **CERT C Coding Standard** - Updated security guidelines
- **Linux man pages** - System programming references

Development Tools (2024 Current Versions)

- **Compilers:** GCC 14.x, Clang 18.x, MSVC 2022 (17.8+)
- **IDEs:** CLion 2024.x, VS Code with C/C++ Extension Pack
- **Build Systems:** CMake 3.28+, Meson 1.3+, Bazel for large projects
- **Debugging:** GDB 14.x, LLDB 18.x, Intel Inspector, ARM Development Studio
- **Static Analysis:** Clang Static Analyzer, PVS-Studio 2024, PC-lint Plus
- **Dynamic Analysis:** Valgrind 3.22+, AddressSanitizer, Dr. Memory
- **Profiling:** Intel VTune 2024, AMD Prof, Google Performance Tools
- **Version Control:** Git 2.43+, GitHub Advanced Security features
- **CI/CD:** GitHub Actions, GitLab CI, Azure DevOps with C/C++ support

Embedded and IoT Development (Current)

- **ARM Cortex Development:** ARM Keil MDK, STM32CubeIDE 1.14+
- **RISC-V Tools:** SiFive Freedom Studio, PlatformIO
- **IoT Platforms:** ESP-IDF 5.x, Zephyr RTOS 3.x, FreeRTOS 11.x
- **Real-time Systems:** QNX 8.0, VxWorks 23.09

Security and Best Practices

- **OWASP C/C++ Security Guidelines** (2023 updates)
- **NIST Cybersecurity Framework** - C development guidelines
- **MISRA C:2023** - Latest automotive safety standard
- **CERT C Secure Coding Standard** - Updated for modern threats

---

Course Policies

Prerequisites Reinforcement

- Mathematical foundation review sessions
- Command-line tutorial for beginners
- Basic computer architecture overview

Academic Integrity

- Code plagiarism detection
- Proper attribution for reference code
- Collaboration guidelines

Accessibility and Support

- Office hours schedule
- Peer programming sessions
- Online forum for questions
- Additional resources for struggling students

Learning Outcomes Verification

Upon completion, students should demonstrate:

1. **Technical Proficiency**
   - Write efficient, readable, and maintainable C code
   - Debug complex programs using various tools
   - Optimize code for performance and memory usage
2. **System Understanding**
   - Understand memory management and pointer manipulation
   - Implement system-level functionality
   - Work with hardware interfaces in embedded systems
3. **Professional Skills**
   - Follow coding standards and best practices
   - Conduct code reviews effectively
   - Document code appropriately
   - Work collaboratively on coding projects
4. **Problem-Solving Abilities**
   - Analyze complex problems and design solutions
   - Choose appropriate data structures and algorithms
   - Handle edge cases and error conditions
5. **Specialized Knowledge**
   - Network programming capabilities
   - Embedded systems development
   - Multi-threaded programming
   - Cross-platform development awareness

This comprehensive syllabus provides a thorough foundation in C programming while preparing students for real-world development challenges across various domains including systems programming, embedded development, and network applications.

Foundations

Introduction to C Programming

What is C?

C is a general-purpose, procedural programming language developed in the early 1970s by Dennis Ritchie at Bell Labs. It was created to rewrite the UNIX operating system in a portable high-level language, and it still sits at the foundation of modern systems software.

C is often called a "middle-level" language: it has high-level control structures and types, yet it exposes memory addresses, integer sizes, and hardware-friendly representations. That combination is why kernels, drivers, embedded firmware, databases, and language runtimes are still written in C (or in languages that interoperate closely with C).

You will write source files ending in `.c`, compile them into machine code, and run the resulting process. This chapter explains what that pipeline does, how a running C program is laid out in memory, and how to read a simple program line by line.

A short history (enough to orient you)

Era	Milestone
1969–1972	C evolves from B (and BCPL) on early UNIX at Bell Labs
1973	UNIX kernel rewritten in C
1978	Kernighan & Ritchie publish *The C Programming Language* (K&R C)
1989 / 1990	ANSI C (C89), then ISO C (C90)
1999	C99: `//` comments, `long long`, VLAs, mixed declarations
2011	C11: atomics, threads API, better Unicode support
2018	C17: mostly clarifications and defect fixes of C11
2024	C23: binary literals, `typeof`, attributes, more library work

This book defaults to **C17** for portable examples (`-std=c17`). When a feature needs C23 or is C99-only, the text says so.

Why learn C?

- **Control**: you decide layout, lifetimes, and how memory is used.
- **Performance**: little runtime magic between your code and the CPU.
- **Systems literacy**: APIs for operating systems and libraries are often C (or C-shaped).

- **Transferable mental model**: C++, Rust, Go, Zig, and many VMs make more sense after you understand C's model of memory, linking, and calling conventions.
- **Career relevance**: kernels, embedded, networking, game engines, databases, security tooling, and high-performance services still need C-fluent engineers.

C is not the only systems language. It is still the lingua franca of low-level interfaces.

From source to process: how a C program runs

A `.c` file is not what the CPU executes. The build tools turn text into a binary image; the operating system loads that image as a **process**.

```
source.c

Preprocess     #include, #define, conditional compilation

   .i (optional intermediate)

Compile        C → assembly (or straight to object on some toolchains)

   .s

Assemble       assembly → machine code in a relocatable object

   .o

Link           resolve symbols, pull in libc, produce executable

a.out / hello

Run            OS loads program → process with its own address space
```

What each stage does

1. **Preprocess** — text-level expansion. `#include <stdio.h>` is replaced by the contents of that header (plus nested includes). Macros like `#define N 10` become substitutions.
2. **Compile** — the compiler checks types and syntax, then generates assembly (or an internal IR that becomes object code).
3. **Assemble** — assembly becomes a **relocatable object file** (`.o`): machine instructions and data with unresolved external references (for example, `printf`).
4. **Link** — the linker stitches object files and libraries together, assigns final addresses, and produces an **executable**.
5. **Run** — the OS creates a process, maps the executable (and shared libraries) into memory, sets up the stack, and jumps to the program entry point (which eventually calls `main`).

Watching the stages with GCC

Save this as `hello.c`:

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Recommended compile line (use this habit throughout the book):

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o hello hello.c
./hello
```

Inspect stages individually:

```
# Preprocessed source (huge; useful when macros go wrong)
gcc -std=c17 -E hello.c -o hello.i

# Assembly listing
gcc -std=c17 -S hello.c -o hello.s

# Object file without linking
gcc -std=c17 -c hello.c -o hello.o

# Link the object into an executable
gcc -o hello hello.o
```

You do not need to memorize assembly. You do need the mental model: **source → translation unit → object → executable → process**.

Memory model intro: text, data, stack, heap

When your program runs, the process address space is divided into regions. Details differ by OS and architecture, but this classic picture is enough for foundations:

```
High addresses

          stack                  grows downward ← local vars, call frames


          (free)


          heap                   grows upward ← malloc / free

   BSS (uninitialized data)      global/static zeros

   data (initialized)            global/static with values

   text (code)                   machine instructions (often read-only)

Low addresses
```

Region	Typical contents	Lifetime
text	Compiled instructions	Whole program
data	Initialized globals / statics	Whole program
BSS	Zero-initialized globals / statics	Whole program
heap	`malloc` / `calloc` / `realloc` memory	Until `free` (or exit)
stack	Locals, parameters, return addresses	Function call

Early chapters use **stack** locals and string literals. Dynamic **heap** allocation appears later with pointers. The important habit now: every object has a **storage duration** (automatic, static, or allocated) and a **scope** (where its name is visible). Confusing those two is a common source of bugs.

Minimal illustration of where things live (you are not expected to print addresses yet—this is orientation):

```c
#include <stdio.h>
#include <stdlib.h>

int g_count = 1;          /* data (initialized global) */
int g_zero;               /* BSS (zero-initialized global) */
```

```c
int main(void) {
    int local = 42;        /* stack */
    static int sticky = 7;/* data/BSS, survives across calls */
    int *p = malloc(sizeof *p); /* heap (if malloc succeeds) */

    if (p == NULL) {
        fprintf(stderr, "malloc failed\n");
        return 1;
    }
    *p = 99;

    printf("local=%d sticky=%d *p=%d g_count=%d g_zero=%d\n",
           local, sticky, *p, g_count, g_zero);

    free(p);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o mem_regions mem_regions.c
./mem_regions
```

Hello, World — anatomy line by line

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Line	Role
`#include <stdio.h>`	Preprocessor pulls in declarations for standard I/O, including `printf`. Angle brackets mean "search system include paths."
`int main(void)`	Program entry from the C language's point of view. `int` is the return type. `(void)` means "no parameters" (preferred over empty `()` in C).
`{ ... }`	Function body. Execution of `main` starts at the first statement.
`printf("Hello, World!\n");`	Call into the C library to write to standard output. `\n` is a newline. Statement ends with `;`.

Line	Role
`return 0;`	Exit status for the process. By convention, `0` means success; non-zero means failure.

Small but important details

- **C is case-sensitive**: `Printf` is not `printf`.
- **Headers declare; libraries define**: including `<stdio.h>` is not optional if you call `printf`—without a declaration, modern compilers reject or warn about implicit declarations.
- **`main` forms**: `int main(void)` or `int main(int argc, char *argv[])` are the portable forms. Returning from `main` is equivalent to calling `exit` with that status.
- **In C99+**, falling off the end of `main` without a `return` is treated as returning `0`. Still write `return 0;` while you are learning—clarity beats cleverness.

Compile and run

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o hello hello.c
./hello
echo $?    # shell shows exit status: 0 on success
```

Full mini-programs

These programs are complete and compilable. Type them yourself; do not only read them.

1. Temperature converter (°C ↔ °F)

```
#include <stdio.h>

int main(void) {
    int choice;
    double value;
    double result;

    printf("Temperature converter\n");
    printf("  1) Celsius to Fahrenheit\n");
    printf("  2) Fahrenheit to Celsius\n");
    printf("Choice: ");

    if (scanf("%d", &choice) != 1) {
```

```
        fprintf(stderr, "Invalid choice input\n");
        return 1;
    }

    printf("Value: ");
    if (scanf("%lf", &value) != 1) {
        fprintf(stderr, "Invalid number\n");
        return 1;
    }

    if (choice == 1) {
        result = value * 9.0 / 5.0 + 32.0;
        printf("%.2f C = %.2f F\n", value, result);
    } else if (choice == 2) {
        result = (value - 32.0) * 5.0 / 9.0;
        printf("%.2f F = %.2f C\n", value, result);
    } else {
        fprintf(stderr, "Choice must be 1 or 2\n");
        return 1;
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o temp_convert temp_convert.c
./temp_convert
```

Notes:

- `scanf("%lf", &value)` is required for `double` (`%f` is for `float` with `scanf`).
- Always check that `scanf` returned the number of items you expected.

2. Maximum of three integers

```
#include <stdio.h>

int main(void) {
    int a, b, c;
    int max;

    printf("Enter three integers: ");
    if (scanf("%d %d %d", &a, &b, &c) != 3) {
        fprintf(stderr, "Expected three integers\n");
        return 1;
    }
```

```c
    max = a;
    if (b > max) {
        max = b;
    }
    if (c > max) {
        max = c;
    }

    printf("max(%d, %d, %d) = %d\n", a, b, c, max);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o max3 max3.c
./max3
# sample input: 9 -2 7
```

This is deliberately free of arrays and functions so you can focus on comparison logic. Later modules rewrite the same idea with loops and helpers.

3. Simple calculator CLI

```c
#include <stdio.h>

int main(void) {
    double left, right, result;
    char op;

    printf("Enter expression (e.g. 3.5 * 2): ");
    if (scanf("%lf %c %lf", &left, &op, &right) != 3) {
        fprintf(stderr, "Could not parse expression\n");
        return 1;
    }

    switch (op) {
    case '+':
        result = left + right;
        break;
    case '-':
        result = left - right;
        break;
    case '*':
        result = left * right;
        break;
    case '/':
        if (right == 0.0) {
```

```
            fprintf(stderr, "Division by zero\n");
            return 1;
        }
        result = left / right;
        break;
    default:
        fprintf(stderr, "Unsupported operator '%c'\n", op);
        return 1;
    }

    printf("%.10g %c %.10g = %.10g\n", left, op, right, result);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o calc calc.c
./calc
# sample input: 12.5 / 2.5
```

`%.10g` prints a readable floating-point form without forcing fixed decimals. Division by zero is checked for `/` only—floating-point edge cases (`NaN`, overflow) come later.

4. Exit status practice

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int code;

    printf("Enter exit code (0-255 recommended): ");
    if (scanf("%d", &code) != 1) {
        return EXIT_FAILURE;
    }

    if (code == 0) {
        printf("Reporting success\n");
        return EXIT_SUCCESS;
    }

    fprintf(stderr, "Reporting failure with code %d\n", code);
    return code;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o exitdemo exitdemo.c
./exitdemo
echo $?
```

Common misconceptions

"C is basically the same as C++"

No. C++ accepts a large amount of C-looking code, but they are different languages:

- C has no classes, references, templates, or RAII.
- Valid C is not always valid C++ (`void *` assignments, designated initializers historically differed, `restrict`, compound literals, etc.).
- Compile C with a **C** compiler and `-std=c17`. Do not learn C by pasting into a C++ project and "fixing until it builds."

"If it compiles, it is correct"

C allows **undefined behavior** (UB): the standard places no requirements on what the program may do. Classic examples include signed integer overflow, using an uninitialized value, buffer overruns, and dangling pointers. The program may crash, appear to work, or corrupt data later—and the behavior can change with optimization level.

```c
/* Illustrating a class of bug - do NOT treat the result as meaningful */
#include <stdio.h>

int main(void) {
    int x;
    /* x is indeterminate; reading it is undefined behavior */
    /* printf("%d\n", x); */
    printf("Uncommenting the printf above would be undefined behavior.\n");
    return 0;
}
```

Enable warnings and treat them seriously:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -o prog prog.c
```

Later: AddressSanitizer (`-fsanitize=address`), undefined-behavior sanitizer (`-fsanitize=undefined`), and tools like Valgrind.

"The compiler is just a translator; any compiler is fine"

Compilers differ in diagnostics, optimization, and default dialects. A program that "works" with loose defaults may fail under stricter flags or another vendor. Standardize on:

```
gcc -std=c17 -Wall -Wextra -Wpedantic
```

Add `-Werror` in CI when you want warnings to fail the build. Clang is an excellent alternative with often clearer messages; the same flags apply in spirit (`clang -std=c17 -Wall -Wextra -Wpedantic`).

"I need to master every standard library function first"

You need a core loop: edit → compile with warnings → run → fix. Learn I/O, types, control flow, functions, then pointers and memory. The library is large; this book introduces pieces when you have a use for them.

"`void main()` is fine"

Not for hosted (normal OS) programs. Use `int main(void)` or `int main(int argc, char *argv[])`.

How this book is organized

The C book under `books/C/` is a modular library, not a 90-day calendar:

Module	Path	Focus
1 Foundations	`content/01-foundations/`	Tooling, first programs, console I/O
2 Data types	`content/02-data-types/`	Types, variables, operators, conversions
3 Control flow	`content/03-control-flow/`	`if`, loops, `switch`, jumps
4 Functions	`content/04-functions/`	Design, scope, modularity, libc overview
5 Arrays & strings	`content/05-arrays-strings/`	Contiguous data and text
6 Pointers	`content/06-pointers/`	Addresses, dynamic memory, pitfalls
7 Structures	`content/07-structures/`	`struct`, `union`, `enum`, simple ADTs
8 File I/O	`content/08-file-io/`	Streams and system-facing I/O
9 Modern C	`content/09-modern-c/`	C99–C23 and modern practice

Module	Path	Focus
10 Data structures	`content/10-data-structures/`	Lists, trees, tables, graphs, sorting
11 Network	`content/11-network/`	Sockets and protocols
12 Embedded	`content/12-embedded/`	Bare-metal-oriented techniques
13 Performance	`content/13-performance/`	Measurement and optimization
14 Testing	`content/14-testing/`	Debug, test, harden
15 Advanced	`content/15-advanced/`	Threads, systems topics, capstone
Workbook	`content/99-workbook/`	Extra demos, exercises, projects
Appendices	`content/100-appendices/`	Resources, reference, solution notes

What to practice first

1. Install a compiler and confirm `gcc --version` (or `clang --version`).
2. Compile Hello World with `-std=c17 -Wall -Wextra -Wpedantic`.
3. Type the temperature converter and calculator by hand; break them on purpose and read the diagnostics.
4. Complete the exercises at the end of this chapter before racing ahead to pointers.
5. Use the workbook (`content/99-workbook/examples/01-foundations/`) as optional extra drills—not a substitute for typing chapter programs.

Recommended daily loop for Module 1:

```
read a section → type one full program → compile with warnings → run →
change one thing → explain the new behavior out loud or in a comment
```

Applications of C (why the language stays relevant)

- **Operating systems and runtimes**: kernels, process loaders, language VMs.
- **Embedded and IoT**: microcontrollers, sensors, firmware with tight RAM/flash budgets.
- **Infrastructure**: databases, web servers, container tooling, networking stacks.
- **Performance-critical domains**: games, HFT-adjacent tooling, scientific kernels, codecs.
- **Interoperability**: foreign-function interfaces almost always speak C ABIs.

Relationship sketch (not a full language survey):

Language	Relation to C
C++	Historically grew from C; different language and type system
Rust / Go / Zig	Modern systems languages; often call C libraries via FFI
Java / C#	Syntax influenced by C-family; managed runtimes

Language	Relation to C
Python / scripting	Many interpreters and extension modules are C under the hood

---

Summary

- C is a systems-oriented language with a small core and a large ecosystem of libraries and platforms.
- A program is **preprocessed, compiled, assembled, linked**, then loaded as a **process**.
- Running processes have **text, data/BSS, heap, and stack** regions; know what lives where at a high level.
- Hello World teaches includes, `main`, library calls, and exit status.
- Prefer `gcc -std=c17 -Wall -Wextra -Wpedantic`, check I/O return values, and treat undefined behavior as a first-class hazard.
- Work module by module; use the **workbook** for extra practice after the chapter programs.

Next: set up (or verify) your toolchain in *Development Environment Setup*, then deepen compilation and multi-file builds in *Your First C Program*.

Exercises

Do these with `-std=c17 -Wall -Wextra -Wpedantic`. Fix every warning.

1. **Hello variants** — Print three lines: your name, today's focus topic, and the value of `sizeof(int)`.
2. **Stage tour** — For `hello.c`, produce `.i`, `.s`, and `.o` artifacts. Open the `.s` file and find the string `Hello` (or a reference to it). You do not need to understand every instruction.
3. **Exit codes** — Write a program that returns `0` if the user enters an even integer and `1` otherwise. Inspect `$?` in the shell.
4. **Swap print** — Read two integers and print them in reverse order without using a third variable *for output only* is optional; using a temporary is fine. Focus on correct `scanf` checks.
5. **Fahrenheit table** — Print a table of Celsius from 0 to 100 in steps of 10 with the Fahrenheit equivalent (no user input required).
6. **Max of three — self-test** — Extend the max program to also print the minimum.
7. **Calculator harden** — Reject invalid operators and division by zero; print errors to `stderr`.
8. **Comment audit** — Take any mini-program above and add brief comments describing each stage of execution in `main` (not line-by-line noise).
9. **Broken on purpose** — Remove `#include <stdio.h>` and compile. Record the diagnostic. Restore it. Then misspell `main` as `mian` and record what happens.

10. **Mental model** — In 5–8 sentences, explain the difference between an object file (`.o`) and an executable, and between stack storage and heap storage.

Optional stretch: time yourself rewriting the temperature converter from memory after a short break. Accuracy and clean compile matter more than speed.

Development Environment Setup

Introduction

Setting up a proper development environment is crucial for effective C programming. This chapter walks through installing compilers, choosing editors, configuring debuggers, and — most importantly — **real day-to-day workflows**: multi-file builds, Make, a mini CMake project, and GCC vs Clang diagnostics.

Default compile line used throughout this book:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
```

Clang is a fine drop-in for learning:

```
clang -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
```

Installing Compilers

GCC (GNU Compiler Collection)

GCC is the most widely used C compiler, especially on Linux and macOS systems.

Linux Installation

```
# Ubuntu/Debian
sudo apt update
sudo apt install build-essential

# Fedora/RHEL/CentOS
sudo dnf install gcc gcc-c++ make

# Arch Linux
sudo pacman -S gcc make
```

macOS Installation

Install Xcode Command Line Tools:

```
xcode-select --install
```

Or use Homebrew:

```
brew install gcc
```

Windows Installation

1. Install MinGW-w64: https://www.mingw-w64.org/
2. Or use MSYS2: https://www.msys2.org/
3. Or install Visual Studio with C/C++ support

Clang

Clang is a modern compiler known for excellent error messages and fast compilation.

Installation

```
# Ubuntu/Debian
sudo apt install clang

# macOS (via Homebrew)
brew install llvm

# Windows (via MSYS2)
pacman -S mingw-w64-x86_64-clang
```

Microsoft Visual C++ (MSVC)

For Windows development, MSVC is Microsoft's official C/C++ compiler.

Installation

1. Download Visual Studio Community (free): https://visualstudio.microsoft.com/
2. During installation, select "Desktop development with C++" workload

Modern IDEs and Editors

Visual Studio Code

A lightweight but powerful editor with excellent C/C++ support.

Setup Steps

1. Download and install VS Code: https://code.visualstudio.com/
2. Install the C/C++ extension by Microsoft
3. Install additional helpful extensions:
    - C/C++ Extension Pack
    - Code Runner
    - GitLens

Configuration

Create a `.vscode` directory in your project with these files:

tasks.json (for building):

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "type": "cppbuild",
            "label": "C/C++: gcc build active file",
            "command": "/usr/bin/gcc",
            "args": [
                "-fdiagnostics-color=always",
                "-g",
                "${file}",
                "-o",
                "${fileDirname}/${fileBasenameNoExtension}",
                "-lm"
            ],
            "options": {
                "cwd": "${fileDirname}"
            },
            "problemMatcher": ["$gcc"],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "detail": "Compiler: /usr/bin/gcc"
        }
```

```
    ]
}
```

launch.json (for debugging):

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "C/C++: gcc build and debug active file",
            "type": "cppdbg",
            "request": "launch",
            "program": "${fileDirname}/${fileBasenameNoExtension}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${fileDirname}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ],
            "preLaunchTask": "C/C++: gcc build active file",
            "miDebuggerPath": "/usr/bin/gdb"
        }
    ]
}
```

CLion

A full-featured IDE specifically designed for C/C++ development.

Features

- Intelligent code completion
- Integrated debugger
- Built-in terminal
- Version control integration
- CMake support

Installation

1. Download from JetBrains: https://www.jetbrains.com/clion/
2. Install with a free student license if you're a student
3. Configure toolchain to point to your compiler

Code::Blocks

A free, open-source C/C++ IDE.

Installation

```
# Ubuntu/Debian
sudo apt install codeblocks

# macOS (via Homebrew)
brew install --cask codeblocks

# Windows
Download from http://www.codeblocks.org/
```

Command-Line Tools

Make

A build automation tool that automatically determines which pieces of a program need to be recompiled.

Installation

```
# Usually comes with build-essential on Linux
sudo apt install make

# macOS (via Xcode Command Line Tools)
xcode-select --install

# Windows (via MSYS2)
pacman -S make
```

CMake

A cross-platform build system generator.

Installation

```
# Ubuntu/Debian
sudo apt install cmake

# macOS (via Homebrew)
brew install cmake

# Windows
Download from https://cmake.org/download/
```

Git

Version control system essential for modern development.

Installation

```
# Ubuntu/Debian
sudo apt install git

# macOS
xcode-select --install

# Windows
Download from https://git-scm.com/
```

Debugging Tools

GDB (GNU Debugger)

The standard debugger for GCC.

Installation

```
# Ubuntu/Debian
sudo apt install gdb

# macOS
# Comes with Xcode Command Line Tools

# Windows (via MSYS2)
pacman -S mingw-w64-x86_64-gdb
```

Basic Usage

```
# Compile with debug symbols
gcc -g -o program program.c

# Start debugging
gdb program

# Common GDB commands
(gdb) run                    # Run the program
(gdb) break main             # Set breakpoint at main function
(gdb) step                   # Step into function
(gdb) next                   # Step over function
(gdb) print variable_name    # Print variable value
(gdb) continue               # Continue execution
(gdb) quit                   # Exit GDB
```

Valgrind

A powerful tool for detecting memory leaks and errors.

Installation

```
# Ubuntu/Debian
sudo apt install valgrind

# macOS
brew install valgrind

# Note: Valgrind has limited support on macOS
```

Usage

```
# Compile with debug symbols
gcc -g -o program program.c

# Run with Valgrind
valgrind --leak-check=full ./program
```

AddressSanitizer

A fast memory error detector built into GCC and Clang.

Usage

```
# Compile with AddressSanitizer
gcc -fsanitize=address -g -o program program.c

# Run normally - errors will be reported automatically
./program
```

Package Management

vcpkg

Microsoft's C++ package manager that works with C as well.

Installation

```
git clone https://github.com/Microsoft/vcpkg.git
cd vcpkg
./bootstrap-vcpkg.sh  # Linux/macOS
./bootstrap-vcpkg.bat # Windows
```

Conan

A decentralized C/C++ package manager.

Installation

```
pip install conan
```

Cloud Development Environments

GitHub Codespaces

Browser-based development environment with full VS Code features.

Setup

1. Create a GitHub repository
2. Go to repository settings
3. Enable Codespaces
4. Create a dev container configuration

Gitpod

Ready-to-code development environments in the cloud.

Setup

1. Create a Gitpod account
2. Add a `.gitpod.yml` file to your repository:

```
image: gitpod/workspace-full
tasks:
  - init: echo "Initializing project"
    command: echo "Starting development environment"
vscode:
  extensions:
    - ms-vscode.cpptools
```

Cross-Platform Development

Windows Subsystem for Linux (WSL)

Run Linux development tools on Windows.

Installation

1. Open PowerShell as Administrator
2. Run: `wsl --install`
3. Restart your computer
4. Install Ubuntu from Microsoft Store

Docker

Containerize your development environment.

Example Dockerfile

```
FROM gcc:latest

WORKDIR /app
COPY . .

RUN gcc -o program program.c

CMD ["./program"]
```

Usage

```
# Build the container
docker build -t c-program .

# Run the program
docker run c-program
```

Real Workflows: From One File to a Small Project

Installation is only half the job. This section is the **daily loop** you will use for the rest of the book.

Single-file recipe

```
# write hello.c, then:
gcc -std=c17 -Wall -Wextra -Wpedantic -o hello hello.c
./hello
echo $?          # exit status: 0 means success
```

Debug-friendly variant:

```
gcc -std=c17 -Wall -Wextra -g -O0 -o hello hello.c
gdb ./hello      # or lldb on macOS
```

Release-ish variant:

```
gcc -std=c17 -Wall -Wextra -O2 -o hello hello.c
```

Multi-file build recipes

Suppose you have:

```
util.h
util.c
main.c
```

util.h

```
#ifndef UTIL_H
#define UTIL_H

int add(int a, int b);
void greet(const char *name);

#endif
```

util.c

```
#include "util.h"
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

void greet(const char *name) {
    if (name == NULL) {
        name = "(null)";
    }
    printf("Hello, %s!\n", name);
}
```

main.c

```
#include <stdio.h>
#include "util.h"

int main(void) {
    greet("C developer");
    printf("2 + 3 = %d\n", add(2, 3));
    return 0;
}
```

Recipe A — one-shot link

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o app main.c util.c
./app
```

Recipe B — separate compile, then link (scales better)

```
gcc -std=c17 -Wall -Wextra -Wpedantic -c main.c -o main.o
gcc -std=c17 -Wall -Wextra -Wpedantic -c util.c -o util.o
gcc -o app main.o util.o
./app
```

When you change only `util.c`, recompile that object and re-link:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -c util.c -o util.o
gcc -o app main.o util.o
```

Recipe C — show each stage (learning aid)

```
gcc -std=c17 -E util.c -o util.i     # preprocess
gcc -std=c17 -S util.c -o util.s     # assembly
gcc -std=c17 -c util.c -o util.o     # object
nm util.o                            # symbols defined/referenced
```

Full program: tiny CLI calculator (single file)

Type this, compile it, break it on purpose, fix warnings.

```
/* file: calc.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void usage(const char *prog) {
    fprintf(stderr, "Usage: %s <add|sub|mul|div> <a> <b>\n", prog);
}

int main(int argc, char *argv[]) {
    double a, b, result;
    char *end;

    if (argc != 4) {
```

```
        usage(argv[0]);
        return EXIT_FAILURE;
    }

    a = strtod(argv[2], &end);
    if (end == argv[2] || *end != '\0') {
        fprintf(stderr, "invalid number: %s\n", argv[2]);
        return EXIT_FAILURE;
    }
    b = strtod(argv[3], &end);
    if (end == argv[3] || *end != '\0') {
        fprintf(stderr, "invalid number: %s\n", argv[3]);
        return EXIT_FAILURE;
    }

    if (strcmp(argv[1], "add") == 0) {
        result = a + b;
    } else if (strcmp(argv[1], "sub") == 0) {
        result = a - b;
    } else if (strcmp(argv[1], "mul") == 0) {
        result = a * b;
    } else if (strcmp(argv[1], "div") == 0) {
        if (b == 0.0) {
            fprintf(stderr, "division by zero\n");
            return EXIT_FAILURE;
        }
        result = a / b;
    } else {
        usage(argv[0]);
        return EXIT_FAILURE;
    }

    printf("%.10g\n", result);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o calc calc.c
./calc add 2 3
./calc div 10 4
./calc div 1 0    # should fail cleanly
```

GCC vs Clang (What Actually Differs)

Both implement the C language and understand the same core flags. Prefer **either** for learning; use both occasionally so diagnostics look familiar.

Topic	GCC	Clang
Typical binary	`gcc`	`clang`
Error style	Compact, classic	Often more "caret" detail / notes
Sanitizers	Excellent (`-fsanitize=…`)	Excellent (often slightly clearer reports)
Static analysis extras	`-fanalyzer` (GCC)	`clang --analyze` / clang-tidy ecosystem
Default on macOS	Homebrew `gcc` or Xcode's `clang` as `gcc` alias	Xcode CLT is Clang under the hood

Side-by-side on the same bug

```c
/* file: warn_me.c - deliberately sloppy */
#include <stdio.h>

int main(void) {
    int x;
    printf("%d\n", x);   /* uninitialized use */
    return 0;
}
```

```
gcc   -std=c17 -Wall -Wextra -Wpedantic -o warn_me warn_me.c
clang -std=c17 -Wall -Wextra -Wpedantic -o warn_me warn_me.c
```

Compare messages. Fix:

```c
int x = 0;
```

Useful shared flags

```
# Treat warnings as errors once you are comfortable
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o prog prog.c

# Extra useful warnings (optional)
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -o prog prog.c

# Sanitizers (Linux/macOS; great for learning memory bugs)
```

```
gcc -std=c17 -Wall -Wextra -g -O1 -fsanitize=address,undefined -o prog prog.c
```

macOS note

`gcc --version` on stock macOS often prints **Apple Clang**. For real GCC:

```
brew install gcc
# then use gcc-14 / g++-14 (version may differ)
```

Make: Multi-File Builds Without Typing Everything

Minimal `Makefile` for the util/main project

```
# Makefile
CC      = gcc
CFLAGS  = -std=c17 -Wall -Wextra -Wpedantic
LDFLAGS =
TARGET  = app
OBJS    = main.o util.o

.PHONY: all clean run

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $(OBJS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# Rebuild if headers change (simple approach)
main.o: main.c util.h
util.o: util.c util.h

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(OBJS) $(TARGET)
```

```
make
make run
make clean
```

Full program + Make: word count of argv strings

wcargs.c

```c
/* file: wcargs.c */
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    int i;
    size_t total = 0;

    for (i = 1; i < argc; i++) {
        size_t n = strlen(argv[i]);
        printf("%zu\t%s\n", n, argv[i]);
        total += n;
    }
    printf("%zu\ttotal chars (no spaces between args)\n", total);
    return 0;
}
```

```
# one-file Makefile variant
CC = gcc
CFLAGS = -std=c17 -Wall -Wextra -Wpedantic
wcargs: wcargs.c
    $(CC) $(CFLAGS) -o $@ $<
```

```
make wcargs
./wcargs hello world
```

CMake Mini-Project (Practical One-Pager)

CMake generates native build files (Make, Ninja, Xcode, VS). For a tiny C project:

```
mini_cmake/
  CMakeLists.txt
  include/
```

```
    util.h
src/
    util.c
    main.c
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(mini_app C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

add_compile_options(-Wall -Wextra -Wpedantic)

add_executable(mini_app
    src/main.c
    src/util.c
)

target_include_directories(mini_app PRIVATE include)
```

Copy the same `util.h` / `util.c` / `main.c` content from the multi-file section (put `util.h` under `include/`, sources under `src/`, and use `#include "util.h"`).

Configure, build, run

```
cd mini_cmake
cmake -S . -B build
cmake --build build
./build/mini_app
```

Debug build:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
```

Out-of-source `build/` keeps sources clean. Delete `build/` anytime and reconfigure.

Optional: compile_commands.json for editors

```
cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
# many editors read build/compile_commands.json (or a symlink at repo root)
```

Editor + Debugger One-Pager

You do not need a heavy IDE. You need: **edit → build → run under a debugger**.

Terminal editor + GDB (always works)

```
# 1. edit files in vim/nano/emacs/VS Code
# 2. build with symbols
gcc -std=c17 -Wall -Wextra -g -O0 -o app main.c util.c

# 3. debug
gdb ./app
```

Essential GDB session:

```
(gdb) break main
(gdb) run
(gdb) next          # step over
(gdb) step          # step into
(gdb) print a
(gdb) backtrace     # or: bt
(gdb) continue
(gdb) quit
```

LLDB (macOS default)

```
clang -std=c17 -Wall -Wextra -g -O0 -o app main.c util.c
lldb ./app
```

```
(lldb) b main
(lldb) run
(lldb) n
(lldb) p a
(lldb) bt
(lldb) c
(lldb) q
```

VS Code (minimum useful setup)

1. Install **C/C++** extension (Microsoft) or clangd-based setup.
2. Ensure `gcc`/`clang` and `gdb`/`lldb` are on `PATH`.
3. Use the earlier `tasks.json` / `launch.json`, **or** just open an integrated terminal and run `make` + `gdb`.
4. Prefer a multi-file task that builds the whole project, not only the active file, once you leave single-file labs.

GDB against a real bug (complete mini-lab)

```c
/* file: off_by_one.c */
#include <stdio.h>

static int sum_first_n(const int *a, int n) {
    int total = 0;
    int i;
    /* BUG: i <= n reads one past the end when n is the count */
    for (i = 0; i <= n; i++) {
        total += a[i];
    }
    return total;
}

int main(void) {
    int data[5] = {1, 2, 3, 4, 5};
    printf("sum = %d\n", sum_first_n(data, 5));
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -O0 -o off_by_one off_by_one.c
gdb ./off_by_one
```

```
(gdb) break sum_first_n
(gdb) run
(gdb) print n
(gdb) print i
(gdb) next
# watch i go to 5 while valid indices are 0..4
```

Fix the loop to `i < n`, rebuild, confirm `sum = 15`.

Sanitizers and Memory Tools (Environment Checklist)

```
# Address + undefined behavior (recommended learning default)
gcc -std=c17 -Wall -Wextra -g -O1 -fsanitize=address,undefined -o app main.c util.c
./app

# Valgrind Memcheck (Linux; limited/absent on modern macOS)
gcc -std=c17 -Wall -Wextra -g -O0 -o app main.c util.c
valgrind --leak-check=full ./app
```

Keep a personal alias if you like:

```
alias c17='gcc -std=c17 -Wall -Wextra -Wpedantic'
alias c17san='gcc -std=c17 -Wall -Wextra -g -O1 -fsanitize=address,undefined'
```

Modern Development Practices

Version Control Workflow

1. Initialize a Git repository: `git init`
2. Create a `.gitignore` for C projects:

```
# Compiled objects
*.o
*.obj

# Executables
*.exe
*.out
a.out
app
calc

# Debug files
*.dSYM/
core
core.*

# Build directories
build/
cmake-build-*/
```

```
compile_commands.json

# Editor
.vscode/
.idea/
```

3. Commit early, commit small: “compiles with -Wall” is a good checkpoint.

Project Structure

```
project/
   src/           # Source files
   include/       # Header files
   tests/         # Test files
   docs/          # Documentation
   build/         # Build output (gitignored)
   CMakeLists.txt # or Makefile
   README.md
```

Continuous Integration

```yaml
# .github/workflows/c-ci.yml
name: C CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Compile
        run: gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o app src/*.c -I include
      - name: Run
        run: ./app
```

Verify Your Setup (Smoke Tests)

Run these once after installing tools:

```
# Compilers
gcc --version
```

```
clang --version   # if installed

# Build tools
make --version
cmake --version

# Debuggers
gdb --version     # or: lldb --version

# Sanitizer smoke
printf '%s\n' '#include <stdio.h>' 'int main(void){puts("ok");}' > t.c
gcc -std=c17 -Wall -Wextra -fsanitize=address,undefined -o t t.c && ./t
```

Expected: versions print without error; `./t` prints `ok`.

Exercises

Compile every program with:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o <name> <sources...>
```

1. **Hello toolchain** — Compile the same `hello.c` with both `gcc` and `clang` (if available). Note any diagnostic differences when you introduce an unused variable.
2. **Multi-file link** — Create `add.h` / `add.c` / `main.c`. Build with Recipe A and Recipe B. Delete `add.o` and rebuild only what is needed.
3. **Makefile** — Write a `Makefile` that builds the multi-file app, supports `make clean` and `make run`, and rebuilds when a header changes.
4. **CMake mini** — Recreate the `mini_cmake` layout. Configure an out-of-source `build/`, build, and run. Then change `util.c`, rebuild, and confirm only the needed objects recompile (watch timestamps / build log).
5. **Debugger lab** — Use GDB or LLDB on `off_by_one.c`. Set a breakpoint, print `i` and `a[i]`, fix the bug, and show the correct sum.
6. **Sanitizer lab** — Write a program that does `int *p = NULL; *p = 1;`. Run it normally (crash), then under AddressSanitizer. Paste the first few lines of the ASan report into a comment in your fixed file.
7. **calc CLI** — Extend `calc.c` with a `mod` operation using integers (`strtol`) and reject non-integers cleanly.
8. **gitignore** — Initialize git in a throwaway folder, build an app, confirm artifacts are ignored, and make one commit of sources only.

Summary

A solid C environment is:

1. **Compiler** — GCC and/or Clang with `-std=c17 -Wall -Wextra`
2. **Editor** — anything that edits text reliably; IDE optional
3. **Debugger** — GDB/LLDB with `-g -O0` builds
4. **Build tools** — one-shot `gcc`, then Make, then CMake as projects grow
5. **Sanitizers / Valgrind** — treat memory and UB as first-class
6. **Git** — ignore build products; commit sources

Master the multi-file and debugger loops now; every later module assumes you can edit, build, run, and inspect without friction.

Your First C Program

Introduction

This chapter moves from "what C is" to a working edit–compile–run loop. You will write complete programs, read compiler diagnostics, split a tiny program across multiple files, and finish a short lab (personal info printer + unit converter).

Default compile line for every example:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
```

The classic "Hello, World!"

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Line-by-line

Piece	Meaning
`#include <stdio.h>`	Bring in declarations for standard I/O (`printf`, `scanf`, …).
`int main(void)`	Entry point; returns an `int` status; no parameters.
`printf(...)`	Library call writing to standard output.
`\n`	Newline character in the string.
`return 0;`	Report success to the operating system / shell.

Create, compile, run

```
# create hello.c with your editor, then:
gcc -std=c17 -Wall -Wextra -Wpedantic -o hello hello.c
./hello
```

On Windows (MinGW / similar), the executable may be `hello.exe`; run `hello` or `.\hello.exe` depending on your shell.

What the compiler does (short version)

`hello.c → preprocess → compile → assemble → link → hello`

Stage	Flag to stop here	Output
Preprocess	`-E`	Preprocessed source (`.i`)
Compile to assembly	`-S`	Assembly (`.s`)
Assemble only	`-c`	Object file (`.o`)
Full link	(default)	Executable

```
gcc -std=c17 -E hello.c -o hello.i
gcc -std=c17 -S hello.c -o hello.s
gcc -std=c17 -c hello.c -o hello.o
gcc -o hello hello.o
```

Useful flags you will reuse:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -g -O0 -o hello hello.c   # debug-friendly
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -o hello hello.c      # optimized release-ish
```

Flag	Role
`-std=c17`	Language dialect
`-Wall -Wextra -Wpedantic`	Strong warnings
`-g`	Debug symbols (GDB/LLDB)
`-O0` / `-O2`	Optimization level
`-o name`	Output file name

Structure of a C program

```c
/* 1. Preprocessor directives */
#include <stdio.h>
#define APP_NAME "demo"

/* 2. Optional global declarations / prototypes */
void greet(const char *who);

/* 3. main - required for hosted programs */
int main(void) {
    greet("World");
    return 0;
}

/* 4. Other function definitions */
void greet(const char *who) {
    printf("Hello, %s!\n", who);
}
```

Order rules of thumb:

1. Includes and macros at the top.
2. Prototypes before first use (or define functions before `main`).
3. One clear `main`.
4. Keep globals rare; prefer locals and parameters.

Comments

```c
// Single-line comment (C99+)

/*
 * Multi-line comment
 * useful for file headers or longer notes
 */
```

Prefer comments that explain **why** or non-obvious constraints. Restating `i++` as "increment i" adds noise.

More complete single-file examples

Greeting with command-line arguments

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <name>\n", argv[0]);
        return 1;
    }

    printf("Hello, %s!\n", argv[1]);
    printf("Argument count: %d\n", argc);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o greeter greeter.c
./greeter Ada
./greeter          # should print usage and exit non-zero
```

`argv[0]` is usually the program name as invoked. `argc` is the count of elements in `argv`.

Sum of two integers (with validation)

```c
#include <stdio.h>

int main(void) {
    int a, b;

    printf("Enter two integers: ");
    if (scanf("%d %d", &a, &b) != 2) {
        fprintf(stderr, "error: expected two integers\n");
        return 1;
    }

    printf("%d + %d = %d\n", a, b, a + b);
    return 0;
}
```

Multiple helpers in one file

```c
#include <stdio.h>

static int square(int x) {
    return x * x;
}

static int max2(int a, int b) {
    return (a > b) ? a : b;
}

int main(void) {
    int x = 5;
    int y = 12;

    printf("square(%d) = %d\n", x, square(x));
    printf("max(%d, %d) = %d\n", x, y, max2(x, y));
    return 0;
}
```

`static` on a function at file scope means "internal linkage": the name is not exported to the linker for other `.c` files. That keeps helpers private to the translation unit.

Print a small banner

```c
#include <stdio.h>

int main(void) {
    puts("**************************");
    puts("*   My First C Program   *");
    puts("**************************");
    return 0;
}
```

`puts` appends a newline for you and is convenient for simple strings. Prefer `printf` when you need formatting.

Multi-file tiny program

Real projects split declarations (headers) from definitions (source files). Minimal pattern:

`util.h`

```c
#ifndef UTIL_H
#define UTIL_H

int add(int a, int b);
void print_sum(int a, int b);

#endif /* UTIL_H */
```

util.c

```c
#include <stdio.h>
#include "util.h"

int add(int a, int b) {
    return a + b;
}

void print_sum(int a, int b) {
    printf("%d + %d = %d\n", a, b, add(a, b));
}
```

main.c

```c
#include "util.h"

int main(void) {
    print_sum(3, 4);
    return 0;
}
```

Build it

```
# Separate compile + link
gcc -std=c17 -Wall -Wextra -Wpedantic -c util.c -o util.o
gcc -std=c17 -Wall -Wextra -Wpedantic -c main.c -o main.o
gcc -o multifile main.o util.o

# Or one shot
gcc -std=c17 -Wall -Wextra -Wpedantic -o multifile main.c util.c

./multifile
```

Why the include guard?

```
#ifndef UTIL_H
#define UTIL_H
/* declarations */
#endif
```

If two files include `util.h` and a third includes both, guards prevent duplicate declarations in one translation unit.

Header vs source rules

Put in `.h`	Put in `.c`
Function prototypes	Function bodies
Shared `struct` types (later)	`static` helpers
`extern` data declarations (rare)	Definitions of globals

Include your own headers with quotes: `#include "util.h"`. System headers use angle brackets: `#include <stdio.h>`.

Common compile errors (with explanations)

Missing semicolon

```
printf("hi\n")   /* error: expected ';' */
```

Fix: statements end with `;`. The diagnostic often points at the *next* line—check the previous statement.

Missing `#include <stdio.h>`

```
int main(void) {
    printf("hi\n");
    return 0;
}
```

Symptom: implicit declaration / unknown identifier `printf`.
Fix: include the proper header.

Wrong main spelling or signature

```
int mian(void) { return 0; }
```

Symptom: linker error `undefined reference to main`.
Fix: the entry point must be named `main`.

Case sensitivity

```
Printf("hi\n");  /* wrong */
```

Fix: `printf`, not `Printf`.

Undeclared function (multi-file)

`main.c` calls `add` but never includes `util.h` and has no prototype:

Symptom: implicit declaration warning/error, sometimes wrong assumptions about types.
Fix: `#include "util.h"` or declare `int add(int, int);` before use.

Multiple definition of a function

Defining `int add(int a, int b) { ... }` in a header that two `.c` files include.

Symptom: linker `multiple definition of add`.
Fix: declare in the header; define in exactly one `.c` file.

Forgetting to link a file

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o multifile main.c
# undefined reference to print_sum
```

Fix: include every translation unit: `main.c util.c`.

Format / type mismatch (warning)

```
int x = 3;
printf("%f\n", x);  /* wrong specifier for int */
```

Fix: use `%d` for `int`. Enable `-Wall`; modern GCC/Clang catch many of these.

Using = instead of == in a condition (logic bug; may still compile)

```c
int x = 0;
if (x = 1) {  /* assignment; always true here */
    printf("oops\n");
}
```

Mitigation: `-Wall` often warns about suspicious assignments in conditions. Prefer clear comparisons.

Unterminated string

```c
printf("hello\n);
```

Symptom: missing terminating " / stray errors cascading.
Fix: close the string literal; recompile—cascading errors often shrink after the first fix.

Tips for reading diagnostics

1. Start with the **first** error, not the last.
2. Fix includes and syntax before chasing linker issues.
3. Recompile after each fix; noise often disappears.
4. If the message is opaque, reduce to a 10-line file that still fails.

Lab A — Personal info printer

Goal: print a short personal profile using variables and formatted output.

```c
#include <stdio.h>

int main(void) {
    char name[] = "Ada Lovelace";
    int age = 36;
    char city[] = "London";
    char language[] = "C";
    double years_experience = 2.5;

    printf("==============================\n");
    printf("       PERSONAL PROFILE\n");
    printf("==============================\n");
    printf("Name:       %s\n", name);
    printf("Age:        %d\n", age);
```

```
    printf("City:       %s\n", city);
    printf("Language:   %s\n", language);
    printf("Experience: %.1f years\n", years_experience);
    printf("==============================\n");

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o profile profile.c
./profile
```

Lab stretch:

1. Change the data to your own details.
2. Add a second `printf` block that re-prints the same data as a single comma-separated line.
3. Replace hard-coded values with `scanf` / `fgets` input (after the next chapter if needed).

Lab B — Unit converter

Goal: convert between a few everyday units with a tiny menu.

```
#include <stdio.h>

int main(void) {
    int choice;
    double input;
    double output;

    printf("Unit converter\n");
    printf("  1) kilometers to miles\n");
    printf("  2) miles to kilometers\n");
    printf("  3) kilograms to pounds\n");
    printf("  4) pounds to kilograms\n");
    printf("Select: ");

    if (scanf("%d", &choice) != 1) {
        fprintf(stderr, "Invalid menu selection\n");
        return 1;
    }

    printf("Value: ");
    if (scanf("%lf", &input) != 1) {
        fprintf(stderr, "Invalid numeric value\n");
```

```
        return 1;
    }

    switch (choice) {
    case 1:
        output = input * 0.621371;
        printf("%.4f km = %.4f mi\n", input, output);
        break;
    case 2:
        output = input * 1.60934;
        printf("%.4f mi = %.4f km\n", input, output);
        break;
    case 3:
        output = input * 2.20462;
        printf("%.4f kg = %.4f lb\n", input, output);
        break;
    case 4:
        output = input * 0.453592;
        printf("%.4f lb = %.4f kg\n", input, output);
        break;
    default:
        fprintf(stderr, "Unknown option %d\n", choice);
        return 1;
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o units units.c
./units
```

Lab stretch:

1. Add Celsius Fahrenheit (reuse logic from the intro chapter).
2. Split conversion formulas into functions in a second file (`convert.c` / `convert.h`) and link them.
3. Loop the menu until the user chooses `0` to quit (preview of control flow).

Variations worth knowing

Implicit `return 0` in `main` (C99+)

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
}
```

Legal in C99 and later for `main` only. Prefer an explicit `return` while learning.

Prototype before `main`

```c
#include <stdio.h>

void print_greeting(void);

int main(void) {
    print_greeting();
    return 0;
}

void print_greeting(void) {
    printf("Hello, World!\n");
}
```

Using `stderr` for errors

```c
#include <stdio.h>

int main(void) {
    fprintf(stderr, "This goes to standard error\n");
    printf("This goes to standard output\n");
    return 1;
}
```

```
./prog >out.txt 2>err.txt
```

Beginner mistakes checklist

Mistake	Fix
Missing `;`	Terminate statements
Missing header	`#include <stdio.h>` etc.
`void main()`	Use `int main(void)`
Calling without prototype	Declare or include header
Ignoring warnings	Compile with `-Wall -Wextra -Wpedantic`
Wrong file extension / dialect	Save as `.c`, pass `-std=c17`
Editing the wrong directory	Confirm path before compile
Running old binary	Rebuild after every edit

Summary

- Every hosted C program needs a `main` and usually standard library headers for I/O.
- Compilation is a pipeline; object files link into executables.
- Multi-file programs separate **declarations** (headers) from **definitions** (sources).
- Learn to read the **first** compiler/linker diagnostic.
- Labs: personal profile printer and unit converter reinforce the full cycle.

Next: *Basic Input and Output* — format strings, `scanf` pitfalls, and safer line input with `fgets`.

Exercises

Compile each with:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o exN exN.c
```

1. **Banner** — Print a boxed title using only `printf` or `puts` (no user input).
2. **Three-file arithmetic** — Create `mathx.h`, `mathx.c` (`mul`, `div_safe`), and `main.c`. `div_safe` should return `0` and print an error to `stderr` when dividing by zero (or use an `int` status code—your design, document it).
3. **Argc printer** — Print all command-line arguments, one per line, including `argv[0]`.
4. **Warning hunt** — Write a file that triggers at least two different warnings under `-Wall -Wextra`, then fix them. Save both versions in comments at the bottom of the file.
5. **Profile lab upgrade** — Personal info printer that reads age with `scanf` and name with a fixed string or `scanf("%49s", ...)`.
6. **Converter lab upgrade** — Add an option `0` to exit; use a loop if you already know `while`/`for`, otherwise document that the stretch is deferred.

7. **Explain a log** — Deliberately omit `util.o` from a multi-file link command and paste the linker error into a comment; explain it in one sentence.
8. **Style pass** — Reformat one lab with consistent indentation (4 spaces), braces on the same style throughout, and a one-line file header comment with your name and purpose.

Basic Input and Output

Introduction

Almost every useful program talks to the outside world. In C, console I/O goes through the **standard streams**:

Stream	Pointer	Typical use
Standard input	`stdin`	Keyboard (or redirected file)
Standard output	`stdout`	Normal program output
Standard error	`stderr`	Diagnostics and errors

The declarations live in `<stdio.h>`. This chapter covers formatted I/O (`printf` / `scanf`), character I/O, safer line input with `fgets`, buffer pitfalls, and two mini-projects: a menu-driven CLI and a simple invoice printer.

Compile examples with:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
```

The `<stdio.h>` library

```
#include <stdio.h>
```

You get:

- Formatted I/O: `printf`, `fprintf`, `sprintf`, `snprintf`, `scanf`, `fscanf`, `sscanf`
- Character I/O: `getchar`, `putchar`, `fgetc`, `fputc`
- Line I/O: `fgets`, `fputs`
- File operations (later module): `fopen`, `fclose`, …

Output with `printf`

Prototype (conceptual)

```
int printf(const char *format, ...);
```

Returns the number of characters written, or a negative value on error.

Minimal example

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Format specifier table

Specifier	Typical argument	Meaning
`%d` / `%i`	`int`	Signed decimal
`%u`	`unsigned int`	Unsigned decimal
`%ld` / `%lu`	`long` / `unsigned long`	Long variants
`%lld` / `%llu`	`long long` / `unsigned long long`	Long long variants
`%f`	`double` (for `printf`)	Fixed-point floating
`%e` / `%E`	`double`	Scientific notation
`%g` / `%G`	`double`	Compact float format
`%c`	`int` (promoted `char`)	Character
`%s`	`char *`	String (must be null-terminated)
`%x` / `%X`	`unsigned`	Hexadecimal
`%o`	`unsigned`	Octal
`%p`	`void *`	Pointer address
`%zu`	`size_t`	Size type (C99+)
`%%`	—	Literal `%`

Important: for `printf`, `float` is promoted to `double`, so `%f` is used for both. For `scanf`, `%f` is `float *` and `%lf` is `double *`.

Runnable demo — mixed types

```c
#include <stdio.h>

int main(void) {
    int age = 25;
    unsigned int score = 9001u;
    double height = 1.75;
    char initial = 'J';
    char name[] = "Jordan";
    void *self = &age;

    printf("Name:    %s\n", name);
    printf("Initial: %c\n", initial);
    printf("Age:     %d\n", age);
    printf("Score:   %u\n", score);
    printf("Height:  %.2f m\n", height);
    printf("Hex age: 0x%x\n", (unsigned)age);
    printf("Pointer: %p\n", self);
    printf("Percent: 100%% done\n");

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o print_types print_types.c
./print_types
```

Width, precision, flags

```c
#include <stdio.h>

int main(void) {
    int n = 42;
    double pi = 3.1415926535;
    char word[] = "HelloWorld";

    printf("|%10d|\n", n);      /* right-align in width 10 */
    printf("|%-10d|\n", n);     /* left-align */
    printf("|%010d|\n", n);     /* zero-pad */
    printf("|%.2f|\n", pi);     /* 2 digits after decimal */
    printf("|%8.2f|\n", pi);    /* width 8, 2 decimals */
    printf("|%.5s|\n", word);   /* at most 5 chars from string */

    return 0;
}
```

Pattern	Effect
`%10d`	Minimum width 10, right-aligned
`%-10d`	Left-aligned
`%010d`	Zero-filled to width 10
`%.2f`	Two digits after decimal point
`%.5s`	Print at most 5 characters of a string
`%#x`	Hex with `0x` prefix (alternate form)

`fprintf` and streams

```c
#include <stdio.h>

int main(void) {
    printf("normal message\n");
    fprintf(stdout, "also normal\n");
    fprintf(stderr, "something went wrong\n");
    return 1;
}
```

Redirect separately:

```
./prog >out.txt 2>err.txt
```

`snprintf` — safe string formatting

Prefer `snprintf` over `sprintf` to avoid buffer overruns:

```c
#include <stdio.h>

int main(void) {
    char buf[32];
    int age = 30;
    int n = snprintf(buf, sizeof buf, "age=%d", age);

    if (n < 0) {
        fprintf(stderr, "encoding error\n");
        return 1;
    }
    if ((size_t)n >= sizeof buf) {
        fprintf(stderr, "output truncated\n");
    }

    printf("%s\n", buf);
    return 0;
```

```
}
```

Input with `scanf`

Prototype (conceptual)

```
int scanf(const char *format, ...);
```

Returns the number of input **items** assigned, or `EOF` on input failure before any assignment.

Address operator

`scanf` needs **pointers** to the objects it will write:

```
int number;
scanf("%d", &number);   /* correct */
/* scanf("%d", number); // wrong: passes value, not address */
```

Arrays (and string buffers) already decay to pointers:

```
char name[50];
scanf("%49s", name);    /* no & for the array */
```

Specifier table for `scanf` (common cases)

Specifier	Argument type	Notes
`%d`	`int *`	Signed int
`%u`	`unsigned int *`	Unsigned
`%ld`	`long *`	Long
`%f`	`float *`	Float
`%lf`	`double *`	Double — easy to forget
`%c`	`char *`	Does **not** skip whitespace unless you force it
`%s`	`char *`	Stops at whitespace; always set a max width
`%x`	`unsigned int *`	Hex input

Full demo — reading several fields

```c
#include <stdio.h>

int main(void) {
    int age;
    double height;
    char initial;
    char name[50];

    printf("Enter age: ");
    if (scanf("%d", &age) != 1) {
        fprintf(stderr, "bad age\n");
        return 1;
    }

    printf("Enter height (meters): ");
    if (scanf("%lf", &height) != 1) {
        fprintf(stderr, "bad height\n");
        return 1;
    }

    printf("Enter middle initial: ");
    if (scanf(" %c", &initial) != 1) { /* leading space skips whitespace */
        fprintf(stderr, "bad initial\n");
        return 1;
    }

    printf("Enter first name (no spaces): ");
    if (scanf("%49s", name) != 1) {
        fprintf(stderr, "bad name\n");
        return 1;
    }

    printf("\n--- Profile ---\n");
    printf("%s %c. age=%d height=%.2f\n", name, initial, age, height);
    return 0;
}
```

Buffer caveats with `scanf`

Leftover newlines

`scanf("%d", &n)` reads the number but leaves the terminating newline in the input buffer. A following `%c` can read that newline instead of the character you meant.

```c
#include <stdio.h>

int main(void) {
    int n;
    char ch;

    printf("Enter an integer: ");
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    printf("Enter a character: ");
    /* Without the leading space, ch often becomes '\n' */
    if (scanf(" %c", &ch) != 1) {
        return 1;
    }

    printf("n=%d ch='%c' (code %d)\n", n, ch, ch);
    return 0;
}
```

Rule of thumb: use a leading space in the format for `%c` when you want to skip leftover whitespace: `scanf(" %c", &ch)`.

`%s` stops at whitespace

`scanf("%s", buf)` cannot read `"Ada Lovelace"` as one field—it stops at the space. Use `fgets` for full lines (below).

Always bound string width

```c
char name[20];
scanf("%19s", name);   /* leave room for '\0' */
```

Unbounded `%s` is a classic buffer overflow.

Failed conversions leave bad data

If the user types `abc` for `%d`, `scanf` returns `0` and does not update the `int` reliably for your purposes—check the return value before using the variable.

Clearing the rest of a line

After a failed or partial read, discard through newline:

```c
#include <stdio.h>

static void clear_line(void) {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        /* discard */
    }
}

int main(void) {
    int n;

    printf("Enter integer: ");
    while (scanf("%d", &n) != 1) {
        fprintf(stderr, "Not an integer. Try again: ");
        clear_line();
    }
    clear_line(); /* eat trailing newline after a good read if needed later */

    printf("Got %d\n", n);
    return 0;
}
```

Character I/O: `getchar` and `putchar`

```c
int getchar(void);   /* returns unsigned char cast to int, or EOF */
int putchar(int c);
```

Echo a character

```c
#include <stdio.h>

int main(void) {
    int ch;

    printf("Press a key then Enter: ");
    ch = getchar();
    if (ch == EOF) {
        return 1;
    }

    printf("You typed: ");
    putchar(ch);
    putchar('\n');
    return 0;
}
```

Copy stdin to stdout until EOF

```c
#include <stdio.h>

int main(void) {
    int ch;

    while ((ch = getchar()) != EOF) {
        putchar(ch);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o catish catish.c
./catish < some_file.txt
# interactive: type text, end with Ctrl+D (Unix) or Ctrl+Z then Enter (Windows consoles)
```

Count characters and lines

```c
#include <stdio.h>

int main(void) {
    int ch;
    unsigned long chars = 0;
```

```
    unsigned long lines = 0;

    while ((ch = getchar()) != EOF) {
        chars++;
        if (ch == '\n') {
            lines++;
        }
    }

    printf("chars=%lu lines=%lu\n", chars, lines);
    return 0;
}
```

Use `int` for the variable that holds `getchar()` so `EOF` (-1) is distinguishable from a byte value `0xFF`.

Reading lines safely with `fgets`

Prototype

```
char *fgets(char *s, int size, FILE *stream);
```

Reads at most `size - 1` characters into `s`, stops early on newline, always null-terminates on success. Returns `s` on success, `NULL` on EOF/error.

Basic pattern

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char line[128];

    printf("Enter a line: ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        fprintf(stderr, "no input\n");
        return 1;
    }

    /* strip trailing newline if present */
    line[strcspn(line, "\n")] = '\0';
```

```
    printf("You said: \"%s\"\n", line);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o line_echo line_echo.c
./line_echo
```

Why prefer `fgets` for text lines?

Approach	Risk
`gets`	**Removed from C11** — never use; unbounded
`scanf("%s")`	Stops at space; easy to overflow without width
`scanf("%[^\\n]")`	Awkward; still easy to get wrong
`fgets`	Bounded; keeps spaces; predictable

Mixing `fgets` and `scanf`

Leftover newlines still bite. Patterns that work:

Option A — prefer `fgets` for everything, then parse with `sscanf`:

```
#include <stdio.h>

int main(void) {
    char line[64];
    int age;

    printf("Age: ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 1;
    }
    if (sscanf(line, "%d", &age) != 1) {
        fprintf(stderr, "invalid age\n");
        return 1;
    }

    printf("age=%d\n", age);
    return 0;
}
```

Option B — after `scanf`, drain the line before the next `fgets`:

```
#include <stdio.h>
#include <string.h>
```

```c
static void clear_line(void) {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) { }
}

int main(void) {
    int id;
    char name[64];

    printf("id: ");
    if (scanf("%d", &id) != 1) {
        return 1;
    }
    clear_line();

    printf("full name: ");
    if (fgets(name, sizeof name, stdin) == NULL) {
        return 1;
    }
    name[strcspn(name, "\n")] = '\0';

    printf("id=%d name=\"%s\"\n", id, name);
    return 0;
}
```

Detecting truncated lines

If `fgets` fills the buffer without seeing `\n`, the line was longer than the buffer:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char line[16];

    printf("Short buffer demo (type a long line):\n");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 1;
    }

    if (strchr(line, '\n') == NULL) {
        fprintf(stderr, "warning: line truncated\n");
        /* optional: drain the rest of the line */
        int c;
        while ((c = getchar()) != '\n' && c != EOF) { }
```

```
    } else {
        line[strcspn(line, "\n")] = '\0';
    }

    printf("stored: \"%s\"\n", line);
    return 0;
}
```

`sscanf` — parse from a string

```
#include <stdio.h>

int main(void) {
    const char *input = "Jordan 25 1.75";
    char name[32];
    int age;
    double height;

    if (sscanf(input, "%31s %d %lf", name, &age, &height) != 3) {
        fprintf(stderr, "parse failed\n");
        return 1;
    }

    printf("%s is %d years old and %.2f m\n", name, age, height);
    return 0;
}
```

Useful when you already have a full line from `fgets` and want conversion with error checking.

Checking return values

`printf`

```
int n = printf("Hello\n");
if (n < 0) {
    /* rare on console; more relevant with broken streams */
}
```

scanf

```
int a, b;
int got = scanf("%d %d", &a, &b);
if (got != 2) {
    fprintf(stderr, "expected two integers, got %d\n", got);
    return 1;
}
```

Treat return-value checks as part of correct I/O, not optional polish.

Mini-project 1 — Menu-driven CLI

A small loop that presents options until the user quits. Uses `fgets` + `sscanf` to avoid buffer traps.

```
#include <stdio.h>
#include <string.h>

static int read_int(const char *prompt, int *out) {
    char line[64];

    printf("%s", prompt);
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;
    }
    return sscanf(line, "%d", out) == 1;
}

int main(void) {
    char line[64];
    int choice;
    int running = 1;

    while (running) {
        printf("\n==== Mini CLI ====\n");
        printf("1) Echo a line\n");
        printf("2) Add two integers\n");
        printf("3) Show ASCII code of a character\n");
        printf("0) Quit\n");
        printf("Select: ");

        if (fgets(line, sizeof line, stdin) == NULL) {
            break;
```

```
    }
    if (sscanf(line, "%d", &choice) != 1) {
        printf("Please enter a number from the menu.\n");
        continue;
    }

    switch (choice) {
    case 0:
        running = 0;
        printf("Bye.\n");
        break;
    case 1: {
        char text[128];
        printf("Text: ");
        if (fgets(text, sizeof text, stdin) == NULL) {
            running = 0;
            break;
        }
        text[strcspn(text, "\n")] = '\0';
        printf("Echo: %s\n", text);
        break;
    }
    case 2: {
        int a, b;
        if (!read_int("a = ", &a) || !read_int("b = ", &b)) {
            printf("Invalid integer input.\n");
            break;
        }
        printf("%d + %d = %d\n", a, b, a + b);
        break;
    }
    case 3: {
        char raw[16];
        printf("Character: ");
        if (fgets(raw, sizeof raw, stdin) == NULL) {
            running = 0;
            break;
        }
        if (raw[0] == '\0' || raw[0] == '\n') {
            printf("No character entered.\n");
            break;
        }
        printf("'%c' has code %d\n", raw[0], (unsigned char)raw[0]);
        break;
    }
    default:
        printf("Unknown option.\n");
```

```
            break;
        }
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o menu_cli menu_cli.c
./menu_cli
```

Mini-project 2 — Simple invoice printer

Reads a few line items and prints a formatted invoice on `stdout`.

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 5
#define NAME_LEN  48

int main(void) {
    char customer[64];
    char names[MAX_ITEMS][NAME_LEN];
    int quantities[MAX_ITEMS];
    double prices[MAX_ITEMS];
    int count = 0;
    double subtotal = 0.0;
    const double tax_rate = 0.08;
    char line[128];

    printf("Customer name: ");
    if (fgets(customer, sizeof customer, stdin) == NULL) {
        return 1;
    }
    customer[strcspn(customer, "\n")] = '\0';

    printf("How many line items (1-%d)? ", MAX_ITEMS);
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 1;
    }
    if (sscanf(line, "%d", &count) != 1 || count < 1 || count > MAX_ITEMS) {
        fprintf(stderr, "Invalid item count\n");
        return 1;
```

```
    }

    for (int i = 0; i < count; i++) {
        printf("\nItem %d name: ", i + 1);
        if (fgets(names[i], NAME_LEN, stdin) == NULL) {
            return 1;
        }
        names[i][strcspn(names[i], "\n")] = '\0';

        printf("Quantity: ");
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 1;
        }
        if (sscanf(line, "%d", &quantities[i]) != 1 || quantities[i] < 0) {
            fprintf(stderr, "Invalid quantity\n");
            return 1;
        }

        printf("Unit price: ");
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 1;
        }
        if (sscanf(line, "%lf", &prices[i]) != 1 || prices[i] < 0.0) {
            fprintf(stderr, "Invalid price\n");
            return 1;
        }
    }

    printf("\n========================================\n");
    printf("              INVOICE\n");
    printf("========================================\n");
    printf("Customer: %s\n\n", customer);
    printf("%-20s %5s %10s %10s\n", "Item", "Qty", "Unit", "Line");
    printf("----------------------------------------\n");

    for (int i = 0; i < count; i++) {
        double line_total = quantities[i] * prices[i];
        subtotal += line_total;
        printf("%-20.20s %5d %10.2f %10.2f\n",
               names[i], quantities[i], prices[i], line_total);
    }

    double tax = subtotal * tax_rate;
    double total = subtotal + tax;

    printf("----------------------------------------\n");
    printf("%-28s %10.2f\n", "Subtotal", subtotal);
```

```
    printf("%-28s %10.2f\n", "Tax (8%)", tax);
    printf("%-28s %10.2f\n", "TOTAL", total);
    printf("========================================\n");

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o invoice invoice.c
./invoice
```

This uses a C99-style `for (int i = 0; ...)` loop (fine under `-std=c17`). Arrays show up formally in a later module; here they are only storage for the demo.

Practical calculator (formatted I/O review)

```
#include <stdio.h>

int main(void) {
    double num1, num2, result;
    char op;
    char line[128];

    printf("Enter expression (e.g. 3.5 * 2): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 1;
    }
    if (sscanf(line, "%lf %c %lf", &num1, &op, &num2) != 3) {
        fprintf(stderr, "Could not parse expression\n");
        return 1;
    }

    switch (op) {
    case '+': result = num1 + num2; break;
    case '-': result = num1 - num2; break;
    case '*': result = num1 * num2; break;
    case '/':
        if (num2 == 0.0) {
            fprintf(stderr, "Division by zero\n");
            return 1;
        }
        result = num1 / num2;
        break;
    default:
```

```c
        fprintf(stderr, "Invalid operator '%c'\n", op);
        return 1;
    }

    printf("%.10g %c %.10g = %.10g\n", num1, op, num2, result);
    return 0;
}
```

Best practices (quick reference)

1. **Check return values** of `scanf`, `fgets`, and parsing helpers.
2. **Bound every string read** (`%49s`, `fgets(..., sizeof buf, ...)`).
3. **Prefer `fgets` + `sscanf`** for interactive line-oriented tools.
4. **Never use `gets`.**
5. **Send errors to `stderr`** with `fprintf(stderr, ...)`.
6. **Mind whitespace** when mixing `%d` and `%c`, or `scanf` and `fgets`.
7. **Use `snprintf`**, not unbounded `sprintf`, when building strings.
8. **Compile with** `-std=c17 -Wall -Wextra -Wpedantic`.

Summary

- `printf` formats output; width and precision control alignment and detail.
- `scanf` reads tokens; always pass addresses (except arrays), check the return count, and bound `%s`.
- Character I/O with `getchar` / `putchar` is simple and stream-friendly; store results in `int`.
- `fgets` is the default safe way to read a line; strip `\n` with `strcspn` or equivalent.
- Menu CLIs and small printers are good practice for real interactive programs.

Next module: data types and variables—so the values you print and scan have precise sizes, ranges, and conversion rules.

Exercises

Use `-std=c17 -Wall -Wextra -Wpedantic` for all of these.

1. **Specifier drill** — Print the same integer as decimal, octal, and hex on one line.

2. **Column report** — Print three rows of "name / quantity / price" using width specifiers so columns align.
3. **Scanf trap lab** — Demonstrate the leftover-newline bug with `%d` followed by `%c` *without* a leading space, then show the fixed version.
4. **Line reverser (partial)** — Read a line with `fgets` and print its length (excluding newline). Full reverse can wait until arrays/strings if needed; do it now if you already can.
5. **Retry loop** — Keep asking for an integer until `sscanf` after `fgets` succeeds.
6. **Menu extension** — Add a fourth menu option that converts Celsius to Fahrenheit.
7. **Invoice tax flag** — Modify the invoice program to ask whether tax applies (`y`/`n`) before printing totals.
8. **stderr routing** — Write a program that prints `OK` to stdout and `BAD` to stderr; run with redirects and show both files.
9. **`snprintf` card** — Build a string `"ID=###;NAME=..."` into a 64-byte buffer and report truncation if it occurs.
10. **Char histogram (lite)** — Read characters until EOF and count how many were digits (`'0'`–`'9'`). Print the count.

Module 1: Foundations and Environment Setup

Overview

This module is the **on-ramp** for the C book: what C is, how to install and drive a toolchain, the edit–compile–run loop, and console I/O. Finish it before jumping to data types and control flow unless you already ship C daily.

Default compile line (memorize it):

```
gcc -std=c17 -Wall -Wextra -Wpedantic -o prog prog.c
```

Clang is fine: replace `gcc` with `clang`.

Learning goals

By the end of this module you should be able to:

- Explain where C sits (systems language, standards C17/C23) and when it is the right tool
- Install **GCC** and/or **Clang**, a debugger (**GDB**/**LLDB**), and optional **CMake** / Make
- Write, compile, and run small programs; read common compiler diagnostics
- Split a program across `.c` / `.h` files and link object files
- Use `printf` / `scanf` / `fgets` carefully (check return values, avoid unbounded `%s`)
- Prefer `-g -O0` when debugging and treat warnings as signal, not noise

Chapters (map)

#	Chapter	What you practice
1	Introduction to C	History, standards, “Hello”, mini converters, mental model of compile/link
2	Development Environment	Install compilers; **multi-file recipes**; Make; **mini CMake**; GCC vs Clang; GDB one-pager; sanitizers

#	Chapter	What you practice
3	First C Program	Program structure, flags, multi-file lab, personal-info / converter labs
4	Basic Input/Output	`printf`/`scanf`, format specs, `fgets`, mixing line and token input safely

Suggested path

1. Skim **Intro** so the compile pipeline and `main` form are clear.
2. Do **Environment** *for real* — multi-file build + one debugger session, not only install notes.
3. Type every program in **First Program**.
4. Complete **Basic I/O** exercises before Module 2; I/O bugs dominate early frustration.

Tooling cheat sheet

```
# single file
gcc -std=c17 -Wall -Wextra -Wpedantic -o hello hello.c && ./hello

# multi-file
gcc -std=c17 -Wall -Wextra -Wpedantic -c main.c util.c
gcc -o app main.o util.o

# debug
gcc -std=c17 -Wall -Wextra -g -O0 -o app main.c util.c
gdb ./app    # or: lldb ./app

# memory / UB (Linux/macOS)
gcc -std=c17 -Wall -Wextra -g -O1 -fsanitize=address,undefined -o app app.c
```

Prerequisites

- Basic computer literacy and a terminal
- Ability to install packages (apt, brew, MSYS2, or Xcode CLT)
- No prior C required for Path A (see book landing page)

Self-check before Module 2

- ☐ `hello.c` builds with zero warnings under `-Wall -Wextra`
- ☐ Multi-file program links (`main.c` + `util.c`)
- ☐ One GDB/LLDB session: breakpoint, `print`, continue
- ☐ Program reads an integer and a line of text without leftover-newline confusion
- ☐ You know how to show the compiler's version and the executable's exit status (`echo $?`)

Estimated time

- Reading: 4–6 hours
- Typing examples + exercises: 6–10 hours
- Optional: CMake mini-project and sanitizer lab from chapter 2

Next module

Module 2: Data Types and Variables — types, sizes, constants, conversions, operators.

Resources

- K&R *The C Programming Language* (classic reference, older dialect)
- GCC / Clang manuals for flag details
- Book appendices for reference tables and solutions pointers

Data Types

Fundamental Data Types

Introduction

Data types define what values you can store, how much memory they use, which operations apply, and how the compiler interprets bits. In C, sizes of some types are **implementation-defined** (within standard minimums), so portable code measures with `sizeof` and uses limits headers or fixed-width types when exact width matters.

This chapter covers integer and floating types, `char` and `bool`, full `sizeof` / `limits.h` / `float.h` demos, signed overflow undefined behavior and safer patterns, and a strong emphasis on `stdint.h` fixed-width types—with full programs you can run on Linux.

Compile convention:

```
gcc -std=c17 -Wall -Wextra -o program program.c
./program
```

Basic Data Types in C

1. **Integer types** — whole numbers
2. **Floating-point types** — real numbers
3. **Character types** — `char` (integer family)
4. **Boolean** — `_Bool` / `bool` (C99+)

Integer Types

Typical Sizes (LP64 Linux, e.g. x86_64)

These are **common** on 64-bit Linux with GCC—not guarantees for every platform.

Type	Size (typical)	Range (typical, signed)
`char`	1 byte	−128..127 or 0..255 if unsigned `char`

Type	Size (typical)	Range (typical, signed)
`short`	2 bytes	−32 768..32 767
`int`	4 bytes	−2 147 483 648..2 147 483 647
`long`	8 bytes (LP64)	full 64-bit signed range
`long long`	8 bytes	full 64-bit signed range

Unsigned Variants

Type	Typical max
`unsigned char`	255
`unsigned short`	65 535
`unsigned int`	4 294 967 295
`unsigned long`	$2^{}-1$ on LP64
`unsigned long long`	$2^{}-1$

Unsigned arithmetic wraps modulo $2^{}$ (well-defined). Signed overflow is **not** well-defined (see below).

Worked Demo: sizeof of Major Types

Run this on your machine and keep the output—it is your ABI cheatsheet.

```c
/* sizeof_tour.c */
#include <stdio.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

int main(void) {
    printf("=== sizeof tour (this machine) ===\n");
    printf("char:          %zu\n", sizeof(char));
    printf("signed char:   %zu\n", sizeof(signed char));
    printf("unsigned char: %zu\n", sizeof(unsigned char));
    printf("short:         %zu\n", sizeof(short));
    printf("unsigned short:%zu\n", sizeof(unsigned short));
    printf("int:           %zu\n", sizeof(int));
    printf("unsigned int:  %zu\n", sizeof(unsigned int));
    printf("long:          %zu\n", sizeof(long));
    printf("unsigned long: %zu\n", sizeof(unsigned long));
    printf("long long:     %zu\n", sizeof(long long));
    printf("unsigned long long: %zu\n", sizeof(unsigned long long));
```

```
    printf("float:          %zu\n", sizeof(float));
    printf("double:         %zu\n", sizeof(double));
    printf("long double:    %zu\n", sizeof(long double));
    printf("bool / _Bool:   %zu\n", sizeof(bool));
    printf("void *:         %zu\n", sizeof(void *));
    printf("size_t:         %zu\n", sizeof(size_t));
    printf("ptrdiff_t:      %zu\n", sizeof(ptrdiff_t));
    printf("int8_t:         %zu\n", sizeof(int8_t));
    printf("int16_t:        %zu\n", sizeof(int16_t));
    printf("int32_t:        %zu\n", sizeof(int32_t));
    printf("int64_t:        %zu\n", sizeof(int64_t));
    printf("uint64_t:       %zu\n", sizeof(uint64_t));
    printf("intptr_t:       %zu\n", sizeof(intptr_t));
    printf("uintptr_t:      %zu\n", sizeof(uintptr_t));
    printf("intmax_t:       %zu\n", sizeof(intmax_t));

    int x = 0;
    printf("sizeof x:       %zu (parens optional for expressions)\n", sizeof x);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o sizeof_tour sizeof_tour.c
./sizeof_tour
```

Sample-shaped output on a typical x86_64 Linux GCC (yours may match):

```
char: 1, short: 2, int: 4, long: 8, long long: 8
float: 4, double: 8, long double: 16
void *: 8
```

On 32-bit ILP32 Linux, `long` and pointers are often 4 bytes—another reason not to assume.

Limits from `limits.h`

```
/* limits_demo.c */
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("=== limits.h (signed) ===\n");
    printf("CHAR_BIT   = %d bits in a byte\n", CHAR_BIT);
    printf("CHAR_MIN   = %d\n", CHAR_MIN);
```

```c
    printf("CHAR_MAX   = %d\n", CHAR_MAX);
    printf("SCHAR_MIN  = %d\n", SCHAR_MIN);
    printf("SCHAR_MAX  = %d\n", SCHAR_MAX);
    printf("SHRT_MIN   = %d\n", SHRT_MIN);
    printf("SHRT_MAX   = %d\n", SHRT_MAX);
    printf("INT_MIN    = %d\n", INT_MIN);
    printf("INT_MAX    = %d\n", INT_MAX);
    printf("LONG_MIN   = %ld\n", LONG_MIN);
    printf("LONG_MAX   = %ld\n", LONG_MAX);
    printf("LLONG_MIN  = %lld\n", LLONG_MIN);
    printf("LLONG_MAX  = %lld\n", LLONG_MAX);

    printf("\n=== limits.h (unsigned maxima) ===\n");
    printf("UCHAR_MAX  = %u\n", UCHAR_MAX);
    printf("USHRT_MAX  = %u\n", USHRT_MAX);
    printf("UINT_MAX   = %u\n", UINT_MAX);
    printf("ULONG_MAX  = %lu\n", ULONG_MAX);
    printf("ULLONG_MAX = %llu\n", ULLONG_MAX);

    return 0;
}
```

```c
gcc -std=c17 -Wall -Wextra -o limits_demo limits_demo.c
./limits_demo
```

Use these macros for **portable** range checks instead of hard-coded `2147483647`.

Floating-Point Types and `float.h`

Type	Typical size	Approx. decimal digits
`float`	4	~6–7 (`FLT_DIG`)
`double`	8	~15–16 (`DBL_DIG`)
`long double`	8–16	extended (`LDBL_DIG`)

```c
/* float_limits_demo.c */
#include <stdio.h>
#include <float.h>

int main(void) {
    printf("=== sizes ===\n");
    printf("float:       %zu\n", sizeof(float));
```

```
    printf("double:      %zu\n", sizeof(double));
    printf("long double: %zu\n", sizeof(long double));

    printf("\n=== decimal digits of precision ===\n");
    printf("FLT_DIG  = %d\n", FLT_DIG);
    printf("DBL_DIG  = %d\n", DBL_DIG);
    printf("LDBL_DIG = %d\n", LDBL_DIG);

    printf("\n=== min positive normalized / max finite ===\n");
    printf("FLT_MIN  = %e  FLT_MAX  = %e\n", FLT_MIN, FLT_MAX);
    printf("DBL_MIN  = %e  DBL_MAX  = %e\n", DBL_MIN, DBL_MAX);
    printf("LDBL_MIN = %Le LDBL_MAX = %Le\n", LDBL_MIN, LDBL_MAX);

    printf("\n=== epsilon (1 + eps != 1) ===\n");
    printf("FLT_EPSILON  = %e\n", FLT_EPSILON);
    printf("DBL_EPSILON  = %e\n", DBL_EPSILON);
    printf("LDBL_EPSILON = %Le\n", LDBL_EPSILON);

    printf("\n=== radix and mantissa bits ===\n");
    printf("FLT_RADIX = %d\n", FLT_RADIX);
    printf("FLT_MANT_DIG = %d  DBL_MANT_DIG = %d  LDBL_MANT_DIG = %d\n",
           FLT_MANT_DIG, DBL_MANT_DIG, LDBL_MANT_DIG);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o float_limits_demo float_limits_demo.c
./float_limits_demo
```

Classic Precision Surprise

```
#include <stdio.h>

int main(void) {
    double a = 0.1 + 0.2;
    printf("0.1 + 0.2 = %.20f\n", a);
    printf("equals 0.3? %s\n", (a == 0.3) ? "yes" : "no");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o fp_surprise fp_surprise.c
./fp_surprise
```

Character Types

`char` is an integer type holding a character code (often ASCII/UTF-8 unit). Whether plain `char` is signed is **implementation-defined**.

```c
#include <stdio.h>

int main(void) {
    char letter = 'A';
    char from_code = 65;

    printf("Letter: %c\n", letter);
    printf("As int: %d\n", letter);
    printf("from_code: %c\n", from_code);

    signed char sc = -1;
    unsigned char uc = 255;
    printf("signed char -1 -> %d\n", sc);
    printf("unsigned char 255 -> %u\n", uc);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o char_demo char_demo.c
./char_demo
```

Prefer `unsigned char` for raw byte buffers; prefer `signed char` when you need negative small integers explicitly.

Boolean Type (C99+)

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    _Bool flag = 1;
    bool ok = true;
    bool bad = false;

    printf("flag=%d ok=%d bad=%d\n", flag, ok, bad);
    if (ok) {
        printf("Task complete path\n");
    }
    return 0;
```

```
}
```

Any scalar converts to `_Bool`: zero → 0, nonzero → 1.

Fixed-Width Types (`stdint.h`) — Prefer These When Width Matters

`int` and `long` change meaning across ABIs. For wire formats, file headers, embedded registers, cryptography, and cross-platform protocols, use **exact-width** types when the implementation provides them (all modern hosted Linux toolchains do for 8/16/32/64).

Exact-Width

Type	Width	Signed range (two's complement)
`int8_t` / `uint8_t`	8	$-128..127$ / $0..255$
`int16_t` / `uint16_t`	16	$\pm 2^{1}$ / $0..2^{1}-1$
`int32_t` / `uint32_t`	32	$\pm 2^{31}$ / $0..2^{32}-1$
`int64_t` / `uint64_t`	64	$\pm 2^{3}$ / $0..2$ -1

Also: `int_leastN_t`, `int_fastN_t`, `intmax_t`, `intptr_t`, `uintptr_t`.

Printing Portably: `inttypes.h`

```c
/* stdint_demo.c */
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <stddef.h>

int main(void) {
    int8_t   a = -5;
    uint8_t  b = 200;
    int16_t  c = -1000;
    uint16_t d = 50000;
    int32_t  e = 123456789;
    uint32_t f = 4000000000u;
    int64_t  g = -123456789012345LL;
    uint64_t h = 123456789012345ULL;

    printf("int8:   %" PRId8 "\n", a);
    printf("uint8:  %" PRIu8 "\n", b);
```

```
    printf("int16:  %" PRId16 "\n", c);
    printf("uint16: %" PRIu16 "\n", d);
    printf("int32:  %" PRId32 "\n", e);
    printf("uint32: %" PRIu32 "\n", f);
    printf("int64:  %" PRId64 "\n", g);
    printf("uint64: %" PRIu64 "\n", h);

    /* Hex */
    printf("uint32 hex: 0x%08" PRIX32 "\n", f);

    /* Pointer-sized integers */
    void *p = &e;
    uintptr_t raw = (uintptr_t)p;
    printf("pointer as uintptr_t: 0x%" PRIxPTR "\n", raw);

    /* Limits from stdint.h */
    printf("INT32_MIN=%" PRId32 " INT32_MAX=%" PRId32 "\n",
           INT32_MIN, INT32_MAX);
    printf("UINT64_MAX=%" PRIu64 "\n", UINT64_MAX);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o stdint_demo stdint_demo.c
./stdint_demo
```

When to Use What

Need	Choose
Exact layout / protocol field	`intN_t` / `uintN_t`
"At least N bits", speed OK	`int_fastN_t`
"At least N bits", small size	`int_leastN_t`
Object sizes, array indices	`size_t`
Pointer difference	`ptrdiff_t`
Generic local counter on host	`int` is fine
File offsets (POSIX)	often `off_t` (with feature macros)

Example: Portable Binary Header

```
/* packet_header.c */
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
```

```c
#include <string.h>

struct PacketHeader {
    uint32_t magic;    /* 0x4B313947 'K19G' style demo */
    uint16_t version;
    uint16_t length;
    uint32_t crc32;
};

int main(void) {
    struct PacketHeader h = {
        .magic = 0x4B313947u,
        .version = 1,
        .length = 42,
        .crc32 = 0xDEADBEEFu
    };

    printf("sizeof header = %zu (expect 12 if no padding)\n", sizeof h);
    printf("magic=%08" PRIX32 " ver=%" PRIu16 " len=%" PRIu16 " crc=%08" PRIX32 "\n",
           h.magic, h.version, h.length, h.crc32);

    /* Serialize explicitly (endianness still your responsibility) */
    uint8_t buf[12];
    memcpy(buf + 0, &h.magic, 4);
    memcpy(buf + 4, &h.version, 2);
    memcpy(buf + 6, &h.length, 2);
    memcpy(buf + 8, &h.crc32, 4);

    printf("bytes:");
    for (size_t i = 0; i < sizeof buf; i++) {
        printf(" %02X", buf[i]);
    }
    printf("\n");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o packet_header packet_header.c
./packet_header
```

Note: `memcpy` of multi-byte integers uses **host endianness**. Network protocols often need explicit big-endian packing.

Signed Overflow Is Undefined Behavior

For **signed** integer types, overflow (and some underflows) are **undefined behavior (UB)** in C. The compiler may assume signed overflow never happens and optimize accordingly—including deleting "impossible" checks.

```c
/* overflow_ub_note.c - educational; do not rely on wraparound of signed int */
#include <stdio.h>
#include <limits.h>
#include <stdint.h>
#include <inttypes.h>

int main(void) {
    int x = INT_MAX;
    printf("INT_MAX = %d\n", x);

    /* The expression (x + 1) is UB if it overflows.
     * In practice you might see wrap, a trap, or surprising optimization.
     * Do NOT write production code that expects wrap on signed overflow.
     */
    printf("Naive signed x+1 is UB if evaluated at INT_MAX.\n");

    /* Well-defined wrap: use unsigned */
    unsigned int ux = (unsigned int)INT_MAX;
    printf("unsigned wrap: %u + 1 = %u\n", ux, ux + 1u);

    /* Or use wider type for the math */
    int64_t wide = (int64_t)x + 1;
    printf("widened: %" PRId64 "\n", wide);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o overflow_ub_note overflow_ub_note.c
./overflow_ub_note
```

Safer patterns:

```c
/* overflow_safe_patterns.c */
#include <stdio.h>
#include <limits.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdbool.h>

/* Return false if a + b would overflow int */
```

```c
static bool safe_add_int(int a, int b, int *out) {
    if (b > 0 && a > INT_MAX - b) {
        return false;
    }
    if (b < 0 && a < INT_MIN - b) {
        return false;
    }
    *out = a + b;
    return true;
}

/* Unsigned: wrap is defined; detect wrap if you need saturation/error */
static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t *out) {
    if (a > UINT32_MAX - b) {
        return false; /* would wrap */
    }
    *out = a + b;
    return true;
}

static bool safe_mul_int(int a, int b, int *out) {
    if (a == 0 || b == 0) {
        *out = 0;
        return true;
    }
    /* Use int64_t intermediate when int is 32-bit - common on Linux */
    int64_t r = (int64_t)a * (int64_t)b;
    if (r > INT_MAX || r < INT_MIN) {
        return false;
    }
    *out = (int)r;
    return true;
}

int main(void) {
    int sum;
    if (!safe_add_int(INT_MAX, 1, &sum)) {
        printf("add INT_MAX+1: overflow detected (good)\n");
    }

    if (safe_add_int(100, 23, &sum)) {
        printf("100+23 = %d\n", sum);
    }

    uint32_t u;
    if (!safe_add_u32(UINT32_MAX, 1u, &u)) {
        printf("u32 MAX+1: overflow detected\n");
```

```
    }

    int prod;
    if (!safe_mul_int(100000, 100000, &prod)) {
        printf("100000*100000 does not fit in int\n");
    }

    /* Compiler helpers (GCC/Clang) - practical on Linux */
    int r;
    if (__builtin_add_overflow(INT_MAX, 1, &r)) {
        printf("__builtin_add_overflow caught INT_MAX+1\n");
    }
    if (!__builtin_add_overflow(20, 22, &r)) {
        printf("__builtin_add_overflow 20+22 = %d\n", r);
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o overflow_safe_patterns overflow_safe_patterns.c
./overflow_safe_patterns
```

Safer Patterns Summary

Pattern	Notes
Check before op	`if (a > INT_MAX - b)` style for add
Wider intermediate	`int64_t` for 32-bit `int` multiply/add
Unsigned modular math	Defined wrap; still check if wrap is an error
`__builtin_*_overflow`	GCC/Clang; clean and efficient
Sanitizers	`gcc -fsanitize=undefined` catches overflow at runtime in tests

```
gcc -std=c17 -Wall -Wextra -fsanitize=undefined -o ov_san ov.c
./ov_san
```

Type Specifiers and Qualifiers

Specifier / qualifier	Role
`signed` / `unsigned`	Signedness
`short` / `long` / `long long`	Rank / width hints
`const`	Not assignable through this lvalue
`volatile`	May change outside program's knowledge
`restrict`	Pointer aliasing promise (C99)

```c
#include <stdio.h>

int main(void) {
    const int max_size = 100;
    long long big = 123456789012345LL;
    unsigned short port = 8080;

    printf("max_size=%d big=%lld port=%hu\n", max_size, big, port);
    return 0;
}
```

Type Promotion (Preview)

`char` and `short` promote to `int` (or `unsigned int`) in expressions. Mixed `int`/`float`/`double` follow usual arithmetic conversions. Details belong in the type-conversion chapter; for now, know that `sizeof(char + char)` is often `sizeof(int)`.

```c
#include <stdio.h>

int main(void) {
    char a = 1, b = 2;
    printf("sizeof(char)=%zu sizeof(a+b)=%zu\n", sizeof(char), sizeof(a + b));
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o promote promote.c
./promote
```

Practical Combined Explorer

```c
/* type_explorer.c */
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>
#include <inttypes.h>
#include <stddef.h>

int main(void) {
    printf("=== Memory Requirements ===\n");
    printf("char %zu | int %zu | long %zu | float %zu | double %zu | void* %zu\n",
           sizeof(char), sizeof(int), sizeof(long),
           sizeof(float), sizeof(double), sizeof(void *));

    int n = 1000;
    printf("\nArray of %d ints: %zu bytes\n", n, (size_t)n * sizeof(int));
    printf("Array of %d int64_t: %zu bytes\n", n, (size_t)n * sizeof(int64_t));

    printf("\n=== Critical limits ===\n");
    printf("INT_MAX=%d UINT_MAX=%u\n", INT_MAX, UINT_MAX);
    printf("LONG_MAX=%ld\n", LONG_MAX);
    printf("SIZE_MAX=%zu\n", (size_t)SIZE_MAX);
    printf("DBL_DIG=%d DBL_EPSILON=%e\n", DBL_DIG, DBL_EPSILON);

    printf("\n=== Fixed width sanity ===\n");
    printf("sizeof(int32_t)=%zu sizeof(int64_t)=%zu\n",
           sizeof(int32_t), sizeof(int64_t));

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o type_explorer type_explorer.c
./type_explorer
```

Best Practices

1. **Measure** with `sizeof`; don't hard-code type sizes.

2. Use **`stdint.h`** for on-disk, on-wire, and register-facing fields.

3. Print fixed-width values with **`inttypes.h`** macros (`PRId32`, …).
4. Treat **signed overflow as a bug**, not as wraparound.
5. Prefer **`size_t`** for sizes and indices into objects.
6. Compare floats with **tolerances**, not `==`, unless you need exact bit identity.
7. Enable **`-Wall -Wextra`**; add **`-fsanitize=undefined`** in debug builds.

Exercises

Exercise 1 — ABI report

Extend `sizeof_tour.c` to also print `CHAR_BIT`, whether plain `char` is signed (`(char)-1 < 0`), and `sizeof(struct { char c; int i; })` vs sum of members (padding).

```
gcc -std=c17 -Wall -Wextra -o abi_report abi_report.c
./abi_report
```

Exercise 2 — limits-driven clamp

Write `int clamp_to_int(long long x)` that returns `INT_MIN`/`INT_MAX` if out of range, else `(int)x`. Test with values from `limits_demo` output.

Exercise 3 — safe multiply

Implement `bool safe_mul_i64(int64_t a, int64_t b, int64_t *out)` without invoking UB. Compare against `__builtin_mul_overflow`.

Exercise 4 — protocol fields

Define a message with `uint8_t type`, `uint16_t id`, `uint32_t payload_len` using fixed-width types. Pack into a byte buffer in **little-endian** explicitly (shifts and masks), then unpack and verify round-trip.

Exercise 5 — float.h scavenger

Print `FLT_ROUNDS` and explain what each value means (look up in the standard or man pages). Measure how many times you can add `FLT_EPSILON` to `1.0f` before the sum changes (loop with a counter).

Exercise 6 — UB sanitizer lab

Intentionally evaluate `int x = INT_MAX; x = x + 1;` in a tiny program under:

```
gcc -std=c17 -Wall -Wextra -fsanitize=undefined -O0 -o ub ub.c
./ub
```

Then rewrite using a safe helper and confirm the sanitizer is silent.

Summary

1. **Integer and floating types** — families, typical Linux sizes
2. **`sizeof` tour** — measure your ABI
3. **`limits.h` / `float.h`** — portable ranges and FP characteristics
4. **`char` / `bool`** — integer nature and boolean conversion
5. **`stdint.h` + `inttypes.h`** — fixed-width types and portable I/O
6. **Signed overflow UB** — detection patterns and builtins
7. **Qualifiers** — `const`, `volatile`, `restrict` (preview)
8. **Exercises** — ABI report, clamp, safe mul, packing, sanitizers

Next: variables, constants, and storage duration—how these types appear in real programs.

Variables and Constants

Introduction

Variables and constants are fundamental building blocks in C programming. Variables are named storage locations in memory that can hold values which may change during program execution, while constants are fixed values that cannot be altered after they are defined.

Understanding how to properly declare, initialize, and use variables and constants is crucial for writing effective C programs. This chapter will cover all aspects of variables and constants, including their declaration, scope, lifetime, and best practices.

Variables

What is a Variable?

A variable is a named memory location that stores data which can be modified during program execution. Every variable in C has: - A name (identifier) - A type (determines what kind of data it can store) - A value (the actual data stored) - A memory address (location in memory)

Variable Declaration

Before using a variable, you must declare it by specifying its type and name:

```
int age;           // Declares an integer variable named 'age'
float height;      // Declares a float variable named 'height'
char initial;      // Declares a character variable named 'initial'
double salary;     // Declares a double variable named 'salary'
```

Variable Initialization

Variables can be initialized (given an initial value) at the time of declaration:

```
int age = 25;                  // Initialize with value 25
float height = 5.9f;           // Initialize with value 5.9
char initial = 'J';            // Initialize with character 'J'
double salary = 75000.50;      // Initialize with value 75000.50
```

You can also declare multiple variables of the same type in a single statement:

```c
int x = 10, y = 20, z = 30;      // Multiple integers with initial values
float a, b, c;                   // Multiple floats without initialization
```

Variable Assignment

After declaration, you can assign new values to variables using the assignment operator (=):

```c
#include <stdio.h>

int main() {
    int number;         // Declaration
    number = 42;        // Assignment

    printf("Number: %d\n", number);

    number = 100;       // New assignment
    printf("Number: %d\n", number);

    return 0;
}
```

Variable Naming Rules

C has specific rules for naming variables:

1. **Must begin with a letter (a-z, A-Z) or underscore (_)**
2. **Can contain letters, digits (0-9), and underscores**
3. **Cannot be C keywords (int, float, if, etc.)**
4. **Case-sensitive (age and Age are different variables)**
5. **No spaces or special characters allowed**

Valid Variable Names

```c
int age;            // Valid
int _temp;          // Valid (starts with underscore)
int student1;       // Valid (contains digit)
int myVariable;     // Valid (camelCase)
int MAX_SIZE;       // Valid (uppercase convention for constants)
```

Invalid Variable Names

```
int 1age;          // Invalid (starts with digit)
int my-age;        // Invalid (contains hyphen)
int int;           // Invalid (keyword)
int my age;        // Invalid (contains space)
int $amount;       // Invalid (contains special character)
```

Variable Naming Conventions

While C allows various naming styles, following consistent conventions improves code readability:

```
// Snake case (recommended for C)
int student_age;
float account_balance;
char first_name[50];

// Camel case (also acceptable)
int studentAge;
float accountBalance;
char firstName[50];

// Upper case for constants (convention)
const int MAX_STUDENTS = 100;
const float PI = 3.14159f;
```

Constants

Constants are fixed values that cannot be changed during program execution. C provides several ways to define constants.

Literal Constants

Literal constants are values written directly in the code:

```
int main() {
    int age = 25;           // 25 is an integer literal
    float price = 19.99f;   // 19.99 is a floating-point literal
    char grade = 'A';       // 'A' is a character literal
    char name[] = "John";   // "John" is a string literal

    return 0;
}
```

#define Preprocessor Directive

The `#define` directive creates symbolic constants:

```c
#include <stdio.h>

#define MAX_SIZE 100
#define PI 3.14159
#define COMPANY_NAME "TechCorp"
#define IS_ACTIVE 1

int main() {
    int array[MAX_SIZE];
    float area = PI * 10 * 10;

    printf("Company: %s\n", COMPANY_NAME);
    printf("Area of circle: %.2f\n", area);

    if (IS_ACTIVE) {
        printf("System is active\n");
    }

    return 0;
}
```

const Qualifier

The `const` keyword creates typed constants:

```c
#include <stdio.h>

int main() {
    const int max_size = 100;
    const float pi = 3.14159f;
    const char company_name[] = "TechCorp";

    printf("Max size: %d\n", max_size);
    printf("PI: %.5f\n", pi);
    printf("Company: %s\n", company_name);

    // max_size = 200;  // Error: cannot modify const variable

    return 0;
}
```

enum (Enumeration)

Enumerations create named integer constants:

```c
#include <stdio.h>

enum Weekday {
    MONDAY = 1,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};

enum Status {
    INACTIVE = 0,
    ACTIVE = 1,
    PENDING = 2
};

int main() {
    enum Weekday today = WEDNESDAY;
    enum Status user_status = ACTIVE;

    printf("Today is day %d of the week\n", today);
    printf("User status: %d\n", user_status);

    if (user_status == ACTIVE) {
        printf("User is active\n");
    }

    return 0;
}
```

Storage Classes

Storage classes define the scope, visibility, and lifetime of variables and functions.

auto

The `auto` storage class is the default for local variables:

```c
#include <stdio.h>

int main() {
    auto int count = 0;  // auto is optional
    int number = 10;     // implicitly auto

    printf("Count: %d\n", count);
    printf("Number: %d\n", number);

    return 0;
}
```

register

The `register` storage class suggests that the variable be stored in a CPU register for faster access:

```c
#include <stdio.h>

int main() {
    register int counter = 0;
    register int i;

    for (i = 0; i < 1000000; i++) {
        counter += i;
    }

    printf("Counter: %d\n", counter);

    return 0;
}
```

static

The `static` storage class has different meanings depending on context:

Static Local Variables

```c
#include <stdio.h>

void counter() {
    static int count = 0;  // Retains value between function calls
    count++;
    printf("Count: %d\n", count);
}
```

```c
int main() {
    counter();  // Output: Count: 1
    counter();  // Output: Count: 2
    counter();  // Output: Count: 3

    return 0;
}
```

Static Global Variables

```c
#include <stdio.h>

static int internal_count = 0;  // Only accessible within this file

void increment_count() {
    internal_count++;
}

int get_count() {
    return internal_count;
}

int main() {
    increment_count();
    increment_count();
    printf("Internal count: %d\n", get_count());

    return 0;
}
```

extern

The `extern` storage class declares a variable that is defined in another file:

```c
// file1.c
int global_counter = 0;  // Definition

// file2.c
#include <stdio.h>

extern int global_counter;  // Declaration (defined elsewhere)

void print_counter() {
    printf("Global counter: %d\n", global_counter);
```

```
}
```

Scope and Lifetime

Scope

Scope determines where a variable can be accessed in the code:

Local Scope

```c
#include <stdio.h>

int main() {
    int local_var = 10;  // Local to main function

    if (local_var > 5) {
        int block_var = 20;  // Local to this block
        printf("Local var: %d, Block var: %d\n", local_var, block_var);
    }

    // printf("Block var: %d\n", block_var);  // Error: out of scope

    return 0;
}
```

Global Scope

```c
#include <stdio.h>

int global_var = 100;  // Global scope

void function1() {
    printf("Global var in function1: %d\n", global_var);
}

int main() {
    printf("Global var in main: %d\n", global_var);
    function1();

    return 0;
}
```

Lifetime

Lifetime determines how long a variable exists in memory:

```c
#include <stdio.h>

int global_var = 100;  // Exists for entire program duration

void function() {
    static int static_var = 0;  // Initialized only once, exists for program duration
    int local_var = 10;         // Created and destroyed each function call

    static_var++;
    printf("Static var: %d, Local var: %d\n", static_var, local_var);
}

int main() {
    function();  // Static var: 1, Local var: 10
    function();  // Static var: 2, Local var: 10
    function();  // Static var: 3, Local var: 10

    return 0;
}
```

Type Qualifiers

const

The `const` qualifier makes a variable read-only:

```c
#include <stdio.h>

int main() {
    const int max_items = 50;
    const float tax_rate = 0.08f;

    // max_items = 100;  // Error: cannot modify const variable

    printf("Max items: %d\n", max_items);
    printf("Tax rate: %.2f\n", tax_rate);

    return 0;
}
```

volatile

The `volatile` qualifier tells the compiler that a variable's value may change unexpectedly:

```c
#include <stdio.h>

int main() {
    volatile int sensor_value = 0;  // May be changed by hardware

    // Compiler won't optimize away reads/writes to volatile variables
    while (sensor_value < 100) {
        // Read sensor value (might be updated by hardware)
        printf("Sensor value: %d\n", sensor_value);
    }

    return 0;
}
```

restrict (C99+)

The `restrict` qualifier is used with pointers to indicate that no other pointer will access the same memory location:

```c
#include <stdio.h>

void add_arrays(int n, int *restrict a, int *restrict b, int *restrict c) {
    // Compiler can optimize knowing a, b, c don't overlap
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    int array1[5] = {1, 2, 3, 4, 5};
    int array2[5] = {10, 20, 30, 40, 50};
    int result[5];

    add_arrays(5, array1, array2, result);

    for (int i = 0; i < 5; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    return 0;
}
```

Practical Examples

Variable Declaration and Usage

```c
#include <stdio.h>

int main() {
    // Different ways to declare and initialize variables
    int age = 25;
    float height = 5.9f;
    double weight = 150.5;
    char grade = 'A';
    char name[] = "John Doe";

    // Display initial values
    printf("=== Initial Values ===\n");
    printf("Age: %d\n", age);
    printf("Height: %.1f feet\n", height);
    printf("Weight: %.1f pounds\n", weight);
    printf("Grade: %c\n", grade);
    printf("Name: %s\n", name);

    // Modify variables
    age = 26;
    height = 6.0f;
    weight = 155.0;
    grade = 'B';

    // Display modified values
    printf("\n=== Modified Values ===\n");
    printf("Age: %d\n", age);
    printf("Height: %.1f feet\n", height);
    printf("Weight: %.1f pounds\n", weight);
    printf("Grade: %c\n", grade);

    return 0;
}
```

Constants Comparison

```c
#include <stdio.h>

#define PI_DEFINE 3.14159
const float PI_CONST = 3.14159f;
```

```c
enum Colors {
    RED = 1,
    GREEN = 2,
    BLUE = 4
};

int main() {
    // Using #define constant
    float area1 = PI_DEFINE * 10 * 10;

    // Using const variable
    float area2 = PI_CONST * 15 * 15;

    // Using enum constants
    int selected_colors = RED | BLUE;

    printf("Area using #define: %.2f\n", area1);
    printf("Area using const: %.2f\n", area2);
    printf("Selected colors: %d\n", selected_colors);

    return 0;
}
```

Storage Class Demonstration

```c
#include <stdio.h>

int global_counter = 0;  // Global variable

void demonstrate_storage_classes() {
    auto int auto_var = 10;        // Automatic storage (default)
    register int reg_var = 20;     // Register storage
    static int static_var = 0;     // Static storage

    global_counter++;
    static_var++;

    printf("Auto var: %d\n", auto_var);
    printf("Register var: %d\n", reg_var);
    printf("Static var: %d\n", static_var);
    printf("Global counter: %d\n", global_counter);
    printf("---\n");
}

int main() {
```

```
    printf("First call:\n");
    demonstrate_storage_classes();

    printf("Second call:\n");
    demonstrate_storage_classes();

    printf("Third call:\n");
    demonstrate_storage_classes();

    return 0;
}
```

Best Practices

1. Initialize Variables

```
// Good practice
int count = 0;
float price = 0.0f;

// Avoid (uninitialized variables)
int count;  // Contains garbage value
```

2. Use Meaningful Names

```
// Good
int student_age;
float account_balance;
char first_name[50];

// Avoid
int x;
float y;
char z[50];
```

3. Use const for Fixed Values

```
// Good
const int MAX_STUDENTS = 100;
const float TAX_RATE = 0.08f;

// Avoid
```

```c
#define MAX_STUDENTS 100
#define TAX_RATE 0.08
```

4. Limit Global Variables

```c
// Prefer local variables when possible
void calculate_area(float radius) {
    const float PI = 3.14159f;  // Local constant
    float area = PI * radius * radius;
    printf("Area: %.2f\n", area);
}
```

5. Use Appropriate Storage Classes

```c
// Use static for persistent local state
void counter() {
    static int count = 0;
    count++;
    printf("Call #%d\n", count);
}
```

Summary

In this chapter, you've learned about:

1. **Variables**: Declaration, initialization, naming rules, and assignment
2. **Constants**: #define, const, and enum for creating fixed values
3. **Storage Classes**: auto, register, static, and extern
4. **Scope and Lifetime**: Local vs global scope, variable lifetime
5. **Type Qualifiers**: const, volatile, and restrict
6. **Best Practices**: Proper variable naming, initialization, and usage

Understanding variables and constants is essential for effective C programming. In the next chapter, we'll explore type conversions and how C handles different data types in expressions.

Type Conversions

Introduction

Type conversion, also known as type casting, is the process of converting a value from one data type to another. C provides both implicit (automatic) and explicit (manual) type conversion mechanisms. Understanding how type conversions work is crucial for writing correct and efficient C programs, as improper conversions can lead to unexpected behavior, data loss, or undefined results.

This chapter will explore both implicit and explicit type conversions, their rules, potential pitfalls, and best practices for safe conversions.

Implicit Type Conversions (Coercion)

Implicit type conversions occur automatically when the compiler converts one data type to another without explicit programmer intervention. These conversions follow specific rules to ensure data integrity while maintaining program functionality.

Usual Arithmetic Conversions

When operators are applied to operands of different types, C performs usual arithmetic conversions to bring them to a common type:

```c
#include <stdio.h>

int main() {
    int i = 10;
    float f = 3.14f;
    double d = 2.5;

    // int + float -> float
    float result1 = i + f;
    printf("int + float = %.2f (float)\n", result1);

    // float + double -> double
    double result2 = f + d;
    printf("float + double = %.2f (double)\n", result2);

    // int + double -> double
```

```
    double result3 = i + d;
    printf("int + double = %.2f (double)\n", result3);

    return 0;
}
```

Integer Promotion

Integer promotion is the automatic conversion of smaller integer types to `int` or `unsigned int`:

```
#include <stdio.h>

int main() {
    char c = 'A';
    short s = 100;

    // char and short are promoted to int in expressions
    int result = c + s;
    printf("char + short = %d (int)\n", result);

    // Promotion also happens with unary operators
    int negated = -c;
    printf("Negated char: %d\n", negated);

    return 0;
}
```

Conversion Hierarchy

C follows a specific hierarchy for implicit conversions:

1. **long double** (highest)
2. **double**
3. **float**
4. **unsigned long long**
5. **long long**
6. **unsigned long**
7. **long**
8. **unsigned int**
9. **int** (lowest)

When operands of different types are used together, the lower type is converted to the higher type:

```
#include <stdio.h>

int main() {
```

```
    char c = 'X';
    short s = 1000;
    int i = 50000;
    float f = 3.14f;
    double d = 2.71828;

    // All converted to double
    double result = c + s + i + f + d;
    printf("Mixed arithmetic result: %.5f\n", result);

    return 0;
}
```

Boolean Context Conversions

In conditional expressions, all scalar types are implicitly converted to boolean values:

```
#include <stdio.h>

int main() {
    int number = 42;
    float zero_float = 0.0f;
    char letter = 'A';
    char null_char = '\0';

    // Non-zero values are true, zero values are false
    if (number) {
        printf("Number %d is true\n", number);
    }

    if (!zero_float) {
        printf("Zero float is false\n");
    }

    if (letter) {
        printf("Character '%c' is true\n", letter);
    }

    if (!null_char) {
        printf("Null character is false\n");
    }

    return 0;
}
```

Explicit Type Conversions (Casting)

Explicit type conversions, or casting, allow programmers to manually convert values from one type to another using cast operators.

Cast Operator Syntax

The cast operator has the form `(type)` where `type` is the target data type:

```c
#include <stdio.h>

int main() {
    double pi = 3.14159;
    int integer_pi = (int)pi;  // Cast double to int

    printf("Original: %.5f\n", pi);
    printf("Casted: %d\n", integer_pi);

    return 0;
}
```

Common Casting Scenarios

Floating-Point to Integer

```c
#include <stdio.h>

int main() {
    float temperature = 98.6f;
    int temp_int = (int)temperature;

    printf("Float temperature: %.1f\n", temperature);
    printf("Integer temperature: %d\n", temp_int);

    // Note: Truncation, not rounding
    float negative = -3.7f;
    int neg_int = (int)negative;
    printf("Negative float: %.1f -> int: %d\n", negative, neg_int);

    return 0;
}
```

Integer to Floating-Point

```c
#include <stdio.h>

int main() {
    int total = 100;
    int count = 3;

    // Without casting, integer division occurs
    float average_int_div = total / count;
    printf("Integer division: %d / %d = %.1f\n", total, count, average_int_div);

    // With casting, floating-point division occurs
    float average_float_div = (float)total / count;
    printf("Float division: %.1f / %d = %.2f\n", (float)total, count, average_float_div);

    return 0;
}
```

Character to Integer and Vice Versa

```c
#include <stdio.h>

int main() {
    char letter = 'A';
    int ascii_value = (int)letter;

    printf("Character: %c\n", letter);
    printf("ASCII value: %d\n", ascii_value);

    int number = 66;
    char character = (char)number;
    printf("Number: %d\n", number);
    printf("Character: %c\n", character);

    return 0;
}
```

Pointer Casting

C allows casting between different pointer types:

```c
#include <stdio.h>
```

```c
int main() {
    int number = 42;
    int *int_ptr = &number;

    // Cast int pointer to void pointer
    void *void_ptr = (void*)int_ptr;

    // Cast void pointer back to int pointer
    int *new_int_ptr = (int*)void_ptr;

    printf("Original number: %d\n", number);
    printf("Via pointer: %d\n", *new_int_ptr);

    return 0;
}
```

Potential Pitfalls and Data Loss

Truncation

```c
#include <stdio.h>

int main() {
    double large_number = 12345.6789;
    int truncated = (int)large_number;

    printf("Original: %.4f\n", large_number);
    printf("Truncated: %d\n", truncated);

    return 0;
}
```

Overflow

```c
#include <stdio.h>
#include <limits.h>

int main() {
    long long big_number = 3000000000LL;  // Larger than INT_MAX
    int overflowed = (int)big_number;

    printf("Big number: %lld\n", big_number);
    printf("Overflowed: %d\n", overflowed);  // Undefined behavior
```

```
    return 0;
}
```

Precision Loss

```
#include <stdio.h>

int main() {
    float precise = 0.1f;
    double more_precise = 0.1;

    printf("Float: %.20f\n", precise);
    printf("Double: %.20f\n", more_precise);

    // Converting double to float loses precision
    float less_precise = (float)more_precise;
    printf("Casted double: %.20f\n", less_precise);

    return 0;
}
```

Safe Conversion Practices

Checking for Overflow

```
#include <stdio.h>
#include <limits.h>

int safe_int_cast(long long value) {
    if (value > INT_MAX || value < INT_MIN) {
        printf("Warning: Value %lld cannot fit in int\n", value);
        return (value > 0) ? INT_MAX : INT_MIN;
    }
    return (int)value;
}

int main() {
    long long big_value = 3000000000LL;
    int safe_value = safe_int_cast(big_value);

    printf("Safe value: %d\n", safe_value);

    return 0;
```

```
}
```

Using Standard Library Functions

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char number_str[] = "12345";
    long parsed_number = strtol(number_str, NULL, 10);

    printf("String: %s\n", number_str);
    printf("Parsed long: %ld\n", parsed_number);

    // Safe conversion with range checking
    if (parsed_number <= INT_MAX && parsed_number >= INT_MIN) {
        int converted = (int)parsed_number;
        printf("Converted to int: %d\n", converted);
    } else {
        printf("Number too large for int\n");
    }

    return 0;
}
```

Conversion Functions

Mathematical Conversions

```c
#include <stdio.h>
#include <math.h>

int main() {
    double value = -3.7;

    printf("Original: %.1f\n", value);
    printf("Floor: %.1f\n", floor(value));    // Round down
    printf("Ceil: %.1f\n", ceil(value));     // Round up
    printf("Truncate: %.1f\n", trunc(value)); // Truncate towards zero
    printf("Round: %.1f\n", round(value));   // Round to nearest

    return 0;
}
```

String Conversions

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // String to integer
    char int_str[] = "123";
    int int_val = atoi(int_str);
    printf("atoi(\"%s\") = %d\n", int_str, int_val);

    // String to long
    char long_str[] = "123456789";
    long long_val = atol(long_str);
    printf("atol(\"%s\") = %ld\n", long_str, long_val);

    // String to double
    char double_str[] = "3.14159";
    double double_val = atof(double_str);
    printf("atof(\"%s\") = %.5f\n", double_str, double_val);

    // More robust conversions with error checking
    char *endptr;
    long strtol_val = strtol("456abc", &endptr, 10);
    printf("strtol(\"456abc\") = %ld, stopped at: %s\n", strtol_val, endptr);

    return 0;
}
```

Practical Examples

Temperature Converter with Type Safety

```c
#include <stdio.h>

// Safe conversion from Fahrenheit to Celsius
double fahrenheit_to_celsius(double fahrenheit) {
    return (fahrenheit - 32.0) * 5.0 / 9.0;
}

// Safe conversion from Celsius to Fahrenheit
double celsius_to_fahrenheit(double celsius) {
    return (celsius * 9.0 / 5.0) + 32.0;
}
```

```c
int main() {
    float user_input;
    int choice;

    printf("Temperature Converter\n");
    printf("1. Fahrenheit to Celsius\n");
    printf("2. Celsius to Fahrenheit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Enter temperature in Fahrenheit: ");
        scanf("%f", &user_input);
        double result = fahrenheit_to_celsius((double)user_input);
        printf("%.2f°F = %.2f°C\n", user_input, result);
    } else if (choice == 2) {
        printf("Enter temperature in Celsius: ");
        scanf("%f", &user_input);
        double result = celsius_to_fahrenheit((double)user_input);
        printf("%.2f°C = %.2f°F\n", user_input, result);
    } else {
        printf("Invalid choice\n");
    }

    return 0;
}
```

Data Type Range Checker

```c
#include <stdio.h>
#include <limits.h>
#include <float.h>

void check_int_range(long long value) {
    printf("Checking value: %lld\n", value);

    if (value > INT_MAX) {
        printf("  Too large for int (max: %d)\n", INT_MAX);
    } else if (value < INT_MIN) {
        printf("  Too small for int (min: %d)\n", INT_MIN);
    } else {
        printf("  Fits in int range\n");
        int converted = (int)value;
        printf("  Converted value: %d\n", converted);
    }
```

```
    printf("\n");
}

int main() {
    long long test_values[] = {
        100LL,
        3000000000LL,
        -3000000000LL,
        INT_MAX,
        INT_MIN,
        (long long)INT_MAX + 1,
        (long long)INT_MIN - 1
    };

    int num_tests = sizeof(test_values) / sizeof(test_values[0]);

    for (int i = 0; i < num_tests; i++) {
        check_int_range(test_values[i]);
    }

    return 0;
}
```

Mixed Type Calculator

```
#include <stdio.h>

int main() {
    int int_val = 10;
    float float_val = 3.14f;
    double double_val = 2.71828;
    char char_val = 'A';  // ASCII 65

    printf("=== Mixed Type Arithmetic ===\n");
    printf("int: %d\n", int_val);
    printf("float: %.2f\n", float_val);
    printf("double: %.5f\n", double_val);
    printf("char: %c (ASCII: %d)\n", char_val, char_val);
    printf("\n");

    // Various combinations
    printf("int + char: %d + %d = %d\n", int_val, char_val, int_val + char_val);
    printf("float + int: %.2f + %d = %.2f\n", float_val, int_val, float_val + int_val);
    printf("double + float: %.5f + %.2f = %.5f\n", double_val, float_val, double_val + float
    printf("char + double: %d + %.5f = %.5f\n", char_val, double_val, char_val + double_val)
```

```
    // Explicit casting examples
    printf("\n=== Explicit Casting ===\n");
    printf("double to int: (int)%.5f = %d\n", double_val, (int)double_val);
    printf("int to float: (float)%d = %.1f\n", int_val, (float)int_val);
    printf("float to char: (char)%.2f = %c\n", float_val, (char)float_val);

    return 0;
}
```

Best Practices

1. Be Explicit About Conversions

```
// Good: Clear intent
float average = (float)total / count;

// Less clear: Relies on implicit conversion
float average = total / (float)count;
```

2. Check for Data Loss

```
// Good: Range checking
if (big_value <= INT_MAX && big_value >= INT_MIN) {
    int small_value = (int)big_value;
}
```

3. Use Appropriate Functions for String Conversion

```
// Good: Error checking with strtol
char *endptr;
long value = strtol(string, &endptr, 10);
if (*endptr != '\0') {
    printf("Invalid number format\n");
}
```

4. Understand Truncation vs Rounding

```c
#include <math.h>
// For truncation (default casting)
int truncated = (int)3.7;  // Results in 3

// For proper rounding
int rounded = (int)round(3.7);  // Results in 4
```

5. Be Careful with Signed/Unsigned Conversions

```c
// Dangerous conversion
unsigned int u_value = 4000000000U;
int s_value = (int)u_value;  // Implementation-defined behavior
```

Summary

In this chapter, you've learned about:

1. **Implicit Conversions**: Automatic type conversions following C's hierarchy
2. **Explicit Conversions**: Manual casting using the cast operator
3. **Integer Promotion**: Automatic promotion of smaller types to `int`
4. **Arithmetic Conversions**: Rules for mixed-type arithmetic operations
5. **Potential Pitfalls**: Truncation, overflow, and precision loss
6. **Safe Practices**: Range checking and proper conversion techniques
7. **Standard Library Functions**: atoi, atof, strtol for string conversions
8. **Best Practices**: Writing safe and clear conversion code

Understanding type conversions is essential for writing robust C programs. In the next chapter, we'll explore operators and expressions, which heavily rely on type conversion rules.

Hexadecimal Floating Literals and %a: Exact Floats in C

Hexadecimal Floating Literals and %a: Exact Floats in C

After this page you will be able to write floating-point constants that need no rounding, print the exact value a `double` holds with `%a`, pass floats through text and get the same bits back, and read the IEEE 754 binary64 layout from hex.

Why Decimal Literals Lie

When you write a base-10 decimal floating-point literal in C, such as `0.1`, the compiler must convert that decimal fraction into a binary floating-point representation (typically IEEE 754 `double`). Because 0.1 has no finite representation in binary—much like 1/3 cannot be represented as a finite decimal—the value stored in memory is an approximation rather than the exact decimal fraction.

```c
double tenth = 0.1;
```

In standard IEEE 754 double precision (64-bit), the exact value stored for `0.1` is actually `0.0001100110011001100110011001100110011001100110011001101...` in binary, which evaluates to approximately `0.1000000000000000055511151231257827021181583404541015625`.

The rounding happens at compile time, silently. Usually that is fine. When you write the constants of a math library, a bit-exact test vector, or a tolerance such as 2^{-2}, you want to say exactly which bits you mean. Hexadecimal floating literals (C99) let you do that: they are written in base 2 to begin with, so there is nothing to round.

The Hexadecimal Floating Literal Syntax

A hexadecimal floating-point literal consists of a prefix (`0x` or `0X`), a hexadecimal significand (mantissa) with an optional radix point (`.`), a required binary exponent indicator (`p` or `P`), an optional sign (`+` or `-`), and a decimal integer representing a power of two.

The binary exponent `p` or `P` is **mandatory**. `0x1.8` on its own is not a floating literal and does not compile. Without the `p`, `0x1e5` would be an integer, since `e` is a hex digit; that is why hex floats use `p` rather than `e`.

The general form is:

$$0\text{x}\ [\text{hex-digits}]\ [.\text{hex-digits}]\ \text{p}\ [\pm]\ \text{decimal-exponent}\ [\text{suffix}]$$

The value of the literal is computed as:

$$\text{significand}_{16} \times 2^{\text{decimal-exponent}}$$

Hexadecimal Literal	Equivalent Mathematical Expression	Decimal Value	Type
`0x1p0`	$1.0_{16} \times 2^0$	1.0	`double`
`0x1.8p+4`	$(1 + 8/16) \times 2^4 = 1.5 \times 16$	24.0	`double`
`0x0.8p0`	$(0+8/16) \times 2^0 = 0.5 \times 1$	0.5	`double`
`0x1.ap-2`	$(1 + 10/16) \times 2^{-2} = 1.625 \times 0.25$	0.40625	`double`
`0x1.8p+4f`	1.5×2^4	24.0	`float`
`0x1.8p+4L`	1.5×2^4	24.0	`long double`

Suffixes function identically to standard decimal floating literals: `f` or `F` specifies `float`, while `l` or `L` specifies `long double`. Unsuffixed literals default to `double`.

A hexadecimal literal containing at most 13 hex fraction digits after a normalized leading `1` occupies at most $13 \times 4 = 52$ bits in the significand fraction. A `double` stores exactly 52 fraction bits, so any such literal is stored with no rounding at all, provided its exponent is in the normal range.

Printing Exact Values with %a

C99 introduced the `%a` and `%A` conversion specifiers for `printf` family functions. While `%f` prints fixed-point decimal notation and `%e` prints decimal scientific notation, `%a` prints the exact floating-point value in hexadecimal scientific format.

The default format produced by `%a` for a normalized `double` in glibc is:

$$[-]0\text{x}h.hhhh\ldots\text{p} \pm d$$

where h represents hexadecimal digits and d represents the decimal exponent power of two. `%A` uses uppercase characters (`0X`, hex digits `A-F`, and `P`).

Round-Tripping Floats Losslessly

`%.6g` throws digits away, so the value you read back is a different `double`. `%.17g` prints enough significant digits (17) that any binary64 `double` reads back exactly with a correctly rounding `strtod`, as glibc's is, but the text is long and does not show you the bits.

`%a` is exact by construction: when `FLT_RADIX` is 2, as on every mainstream platform, it prints the binary value digit for digit, and `strtod` (or `strtof`, `strtold`) accepts the same notation. The round trip gives back the same bits:

```c
char buf[64];
double original = 1.0 / 3.0;

snprintf(buf, sizeof(buf), "%a", original);
double restored = strtod(buf, NULL);

assert(original == restored); /* needs <assert.h>, <stdio.h>, <stdlib.h> */
```

Reading IEEE 754 Binary64 Through Hexadecimal

An IEEE 754 binary64 (`double`) value consists of 64 bits structured into three distinct fields:

1. **Sign bit**: 1 bit (s)
2. **Biased exponent**: 11 bits (e), with a bias of 1023
3. **Significand fraction**: 52 bits (f)

```
 1 bit   11 bits                      52 bits
+------+-----------+-------------------------------------------------+
| Sign | Exponent  |                    Fraction                     |
+------+-----------+-------------------------------------------------+
  63    62       52 51                                              0
```

52 bits are exactly 13 hex digits ($52/4 = 13$). So for a normal number, the 13 digits after `0x1.` in `%a` output *are* the fraction field, and the `p` exponent plus 1023 is the exponent field. For example, `0x1.8p+4` (24.0) has sign 0, exponent field $4+1023 = 1027 =$ `0x403`, and fraction `0x8000000000000`. Packed together, the 64 bits are `0x4038000000000000`.

Common Floating-Point Constants in Hex

The standard `<float.h>` header defines limits for floating-point types. The table below illustrates how these limits map directly to binary64 hex values:

Constant	Hexadecimal Notation	Binary Exponent	Description
`DBL_MAX`	`0x1.fffffffffffffp+1023`	1023	Maximum finite `double` value
`DBL_MIN`	`0x1p-1022`	2^{-1022}	Smallest normalized positive `double`
`DBL_EPSILON`	`0x1p-52`	2^{-52}	Difference between 1.0 and next representable `double`
Subnormal min	`0x0.0000000000001p-1022`	2^{1074}	Smallest positive subnormal `double`, as glibc prints it

Subnormal Numbers

When a value falls below `DBL_MIN` (2^{-1022}), the implicit leading bit becomes `0` rather than `1`. In glibc implementations, formatting the minimum denormalized value produces `0x0.0000000000001p-1022`, showing a zero before the hexadecimal point and a single 1 in the 13th fractional position ($2^{-52} \times 2^{-1022} = 2^{-1074}$).

Deconstructing Floats with `frexp` and `ldexp`

The standard C library provides `frexp` and `ldexp` in `<math.h>` to inspect and construct floating-point values programmatically.

The `frexp` function breaks a floating-point number x into a normalized fraction m and an integer exponent e such that:

$$x = m \times 2^e$$

where $|m| \in [0.5, 1.0)$ or $m = 0$.

Notice that `frexp` normalizes the mantissa m to the range $[0.5, 1.0)$, whereas glibc's `%a` prints a normal `double` with a leading `1` before the point, which is the range $[1.0, 2.0)$.

For example, when evaluating `24.0` (`0x1.8p+4`): * `%a` represents 24.0 as 1.5×2^4 (`0x1.8p+4`). * `frexp` breaks 24.0 into 0.75×2^5 (`0x1.8p-1` $\times 2^5$).

`ldexp(m, e)` performs the inverse calculation, computing $m \times 2^e$.

Complete Lab Walk-Through

The following program demonstrates decimal vs. hexadecimal literal precision, formatting with `%a`, exact round-tripping, `float` promotion rules, and decomposition with `frexp` and `ldexp`.

```c
#include <float.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    /* 1. Decimal literals are rounded; hex literals are exact. */
    double tenth = 0.1;
    double one_p = 0x1p0;          /* 1.0 */
    double max_hex = 0x1.fffffffffffffp+1023;
    printf("0.1        = %a  (%.17g)\n", tenth, tenth);
    printf("0x1p0      = %a  (%g)\n", one_p, one_p);
    printf("DBL_MAX    : %s\n", max_hex == DBL_MAX ? "equal to 0x1.fffffffffffffp+1023" : "d
    printf("DBL_MIN    = %a\n", DBL_MIN);
    printf("DBL_EPSILON= %a\n", DBL_EPSILON);
```

```
    printf("denorm min = %a\n", nextafter(0.0, 1.0));

    /* 2. The doubles just below and just above 1.0. */
    printf("below 1.0  = %a\n", nextafter(1.0, 0.0));
    printf("above 1.0  = %a\n", nextafter(1.0, 2.0));

    /* 3. Round trip through text: %a is exact, %.6g is not. */
    double x = 1.0 / 3.0;
    char a[64], g[64];
    snprintf(a, sizeof a, "%a", x);
    snprintf(g, sizeof g, "%.6g", x);
    printf("1/3 as %%a  = %s -> round trip %s\n", a, strtod(a, NULL) == x ? "exact" : "LOST"
    printf("1/3 as %%.6g= %s -> round trip %s\n", g, strtod(g, NULL) == x ? "exact" : "LOST"

    /* 4. Float and long double use the same notation. */
    printf("0.1f       = %a\n", (double)0.1f);
    printf("0.1L       = %La\n", 0.1L);

    /* 5. Pulling the exponent and mantissa apart. */
    int e;
    double m = frexp(0x1.8p+4, &e); /* 1.5 * 2^4 = 24 */
    printf("frexp(24)  = %a * 2^%d\n", m, e);
    printf("ldexp      = %g\n", ldexp(0x1.8p0, 4));
    return 0;
}
```

This was tested with gcc 14.2 and glibc 2.41:

```
gcc --version | head -1
```

```
gcc (Debian 14.2.0-19) 14.2.0
```

Compile with warnings on. This produced no warnings:

```
gcc -std=c17 -Wall -Wextra -O2 -o hexfloat hexfloat.c -lm
```

Execute the compiled binary:

```
./hexfloat
```

It printed:

```
0.1        = 0x1.999999999999ap-4  (0.10000000000000001)
0x1p0      = 0x1p+0  (1)
DBL_MAX    : equal to 0x1.fffffffffffffp+1023
DBL_MIN    = 0x1p-1022
```

```
DBL_EPSILON= 0x1p-52
denorm min = 0x0.0000000000001p-1022
below 1.0  = 0x1.fffffffffffffp-1
above 1.0  = 0x1.0000000000001p+0
1/3 as %a  = 0x1.5555555555555p-2 -> round trip exact
1/3 as %.6g= 0.333333 -> round trip LOST
0.1f       = 0x1.99999ap-4
0.1L       = 0xc.cccccccccccccdp-7
frexp(24)  = 0x1.8p-1 * 2^5
ldexp      = 24
```

Reading the output line by line:

- `0.1` is really `0x1.999999999999ap-4`. The repeating `9`s are binary `1001` repeating, and the final `a` is where the compiler rounded up. `%.17g` shows the same rounding in decimal: `0.10000000000000001`.
- `0x1p0` is exactly 1, and `DBL_MAX` compares equal to the literal `0x1.fffffffffffffp+1023`.
- `DBL_MIN`, `DBL_EPSILON` and the smallest subnormal are powers of two, so in hex they are just `0x1p-1022`, `0x1p-52` and a single `1` in the last fraction digit.
- Just above 1.0, only the last hex digit changes (`0x1.0000000000001p+0`): the gap is `0x1p-52`, which is `DBL_EPSILON`. Just below 1.0 the exponent drops to `-1` and every fraction digit is `f`. The gap there is half as big, 2^{-53}.
- `%a` text reads back exactly with `strtod`. `%.6g` text (`0.333333`) does not.
- `0.1f` shows only 6 fraction digits ending in `a`: a `float` has 23 fraction bits, so it is rounded much earlier than the `double`.
- `0.1L` on x86-64 is the 80-bit x87 format, which stores 64 significand bits with an explicit leading bit. glibc prints it with a leading hex digit `c` (binary `1100`) and adjusts the exponent: `0xc.cccccccccccccdp-7`.
- `frexp` reports 24 as `0x1.8p-1 * 2^5`, that is 0.75×2^5. `ldexp(0x1.8p0, 4)` goes the other way and gives 24.

Pitfalls and Common Mistakes

1. Missing the Binary Exponent `p`

Leaving out the `p` exponent is a compile error.

```c
/* Incorrect: Missing exponent */
double bad = 0x1.8;

/* Correct */
double good = 0x1.8p0;
```

2. Assuming `p` Represents Base-16 Power

In hexadecimal floating literals, `p` stands for "power of two", not power of sixteen.

```
/* 0x1.0p4 means 1.0 * (2^4) = 16.0, NOT 1.0 * (16^4) */
double val = 0x1.0p4;
```

3. Floating-Point Promotion in `printf`

Variadic functions such as `printf` promote a `float` argument to `double`. So `%a` with a `float` prints the `double` that has the same value as that `float`. The conversion is exact, so what you see is the `float`'s own rounding: it ends after 6 hex digits, much earlier than `0.1` as a `double`.

```
float f = 0.1f;
printf("%a\n", f); /* glibc prints 0x1.99999ap-4 (trailing zeros dropped) */
```

4. Mismatched Specifiers for `long double`

`%a` expects a `double` (or a promoted `float`). Passing a `long double` to `%a` is undefined behaviour, and `-Wall` warns about it. Use `%La`.

```
long double ld = 0.1L;
printf("%La\n", ld); /* Correct format specifier for long double */
```

5. Locale Specificity with Decimal Points

Source-code literals always use `.`. At run time, though, `printf("%a")` and `strtod` use the radix character of the current `LC_NUMERIC` locale, and that applies to hex floats too. A program that calls `setlocale(LC_ALL, "")` under a locale that uses `,` will write `0x1,8p+4`, and `strtod` stops reading at the `.` in `0x1.8p+4`. Keep `LC_NUMERIC` as `"C"` for data files, or use a locale-independent parser.

Key Takeaways

- **Exact Representation**: Hexadecimal floating literals (C99+) allow writing binary floating-point numbers without decimal conversion errors.
- **Syntax Rule**: A hex float literal requires `0x`/`0X`, a hex significand, and a mandatory binary exponent `p`/`P` followed by a decimal power of two.
- **Bit-Exact I/O**: The `%a` specifier formats floats in exact hex notation. Text written with `%a` reads back with `strtod` to the same bits. `%.17g` also round-trips a `double`; `%.6g` does not.
- **Direct Bit Mapping**: 13 hexadecimal fractional digits map directly to the 52 significand fraction bits of an IEEE 754 binary64 `double`.
- **Standard Functions**: `frexp` decomposes numbers into normalized mantissa ($[0.5, 1.0)$) and integer exponent, while `ldexp` combines them.

Sources

- cppreference: Floating constants
- cppreference: fprintf, printf, sprintf
- cppreference: strtof, strtod, strtold
- cppreference: frexp, frexpf, frexpl
- WG14 N3220 working draft (C23 text plus fixes)
- GNU C Library Manual: Floating-Point Conversions

Operators and Expressions

Introduction

Operators are symbols that perform specific operations on operands (variables, constants, or expressions). Expressions are combinations of operators and operands that evaluate to a single value. Understanding operators and how to construct expressions is fundamental to C programming, as they form the building blocks of all program logic.

This chapter will cover all major categories of operators in C, their precedence and associativity rules, and how to write effective expressions.

Categories of Operators

C provides a rich set of operators that can be categorized as follows:

1. **Arithmetic Operators**: Perform mathematical operations
2. **Relational Operators**: Compare values
3. **Logical Operators**: Combine boolean expressions
4. **Bitwise Operators**: Manipulate individual bits
5. **Assignment Operators**: Assign values to variables
6. **Increment/Decrement Operators**: Modify values by one
7. **Conditional Operator**: Ternary selection operator
8. **Special Operators**: sizeof, address-of, dereference, etc.

Arithmetic Operators

Arithmetic operators perform basic mathematical operations:

Operator	Description	Example	Result
`+`	Addition	`5 + 3`	8
`-`	Subtraction	`5 - 3`	2
`*`	Multiplication	`5 * 3`	15
`/`	Division	`5 / 3`	1 (integer division)
`%`	Modulus (remainder)	`5 % 3`	2

Basic Arithmetic Operations

```c
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("a = %d, b = %d\n", a, b);
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);  // Integer division
    printf("a %% b = %d\n", a % b); // Modulus

    // Floating-point arithmetic
    float x = 10.0f, y = 3.0f;
    printf("\nFloat arithmetic:\n");
    printf("x = %.1f, y = %.1f\n", x, y);
    printf("x / y = %.2f\n", x / y);  // Floating-point division

    return 0;
}
```

Division and Modulus Behavior

```c
#include <stdio.h>

int main() {
    // Integer division truncates toward zero
    printf("Integer division:\n");
    printf("7 / 3 = %d\n", 7 / 3);     // 2
    printf("-7 / 3 = %d\n", -7 / 3);   // -2
    printf("7 / -3 = %d\n", 7 / -3);   // -2
    printf("-7 / -3 = %d\n", -7 / -3); // 2

    // Modulus result has the same sign as the dividend
    printf("\nModulus operations:\n");
    printf("7 %% 3 = %d\n", 7 % 3);     // 1
    printf("-7 %% 3 = %d\n", -7 % 3);   // -1
    printf("7 %% -3 = %d\n", 7 % -3);   // 1
    printf("-7 %% -3 = %d\n", -7 % -3); // -1

    return 0;
}
```

Relational Operators

Relational operators compare two values and return a boolean result (1 for true, 0 for false):

Operator	Description	Example	Result
`==`	Equal to	`5 == 3`	0 (false)
`!=`	Not equal to	`5 != 3`	1 (true)
`>`	Greater than	`5 > 3`	1 (true)
`<`	Less than	`5 < 3`	0 (false)
`>=`	Greater than or equal to	`5 >= 5`	1 (true)
`<=`	Less than or equal to	`5 <= 3`	0 (false)

Relational Operations

```c
#include <stdio.h>

int main() {
    int x = 10, y = 20;

    printf("x = %d, y = %d\n", x, y);
    printf("x == y: %d\n", x == y);
    printf("x != y: %d\n", x != y);
    printf("x > y: %d\n", x > y);
    printf("x < y: %d\n", x < y);
    printf("x >= y: %d\n", x >= y);
    printf("x <= y: %d\n", x <= y);

    // Floating-point comparison (be careful!)
    float a = 0.1f + 0.2f;
    float b = 0.3f;
    printf("\nFloating-point comparison:\n");
    printf("a = %.20f\n", a);
    printf("b = %.20f\n", b);
    printf("a == b: %d\n", a == b);

    return 0;
}
```

Logical Operators

Logical operators work with boolean values and are used to combine or invert conditions:

Operator	Description	Example	Result
`&&`	Logical AND	`1 && 0`	0 (false)
`||`	Logical OR	`1 || 0`	1 (true)
`!`	Logical NOT	`!1`	0 (false)

Logical Operations

```c
#include <stdio.h>

int main() {
    int a = 1, b = 0, c = 1;

    printf("a = %d, b = %d, c = %d\n", a, b, c);
    printf("a && b: %d\n", a && b);  // 1 AND 0 = 0
    printf("a || b: %d\n", a || b);  // 1 OR 0 = 1
    printf("!a: %d\n", !a);          // NOT 1 = 0
    printf("a && c: %d\n", a && c);  // 1 AND 1 = 1
    printf("a || (b && c): %d\n", a || (b && c));  // 1 OR (0 AND 1) = 1

    // Short-circuit evaluation
    printf("\nShort-circuit evaluation:\n");
    int x = 5, y = 0;
    if (y != 0 && x / y > 1) {  // Second condition not evaluated
        printf("This won't print\n");
    } else {
        printf("Short-circuit prevented division by zero\n");
    }

    return 0;
}
```

Bitwise Operators

Bitwise operators work at the bit level and are particularly useful in systems programming and embedded development:

Operator	Description	Example	Result
`&`	Bitwise AND	`5 & 3`	1
`|`	Bitwise OR	`5 | 3`	7
`^`	Bitwise XOR	`5 ^ 3`	6
`~`	Bitwise NOT	`~5`	-6
`<<`	Left shift	`5 << 1`	10
`>>`	Right shift	`5 >> 1`	2

Bitwise Operations

```c
#include <stdio.h>

void print_binary(unsigned int num) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (num >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }
    printf("\n");
}

int main() {
    unsigned int a = 12;  // Binary: 1100
    unsigned int b = 10;  // Binary: 1010

    printf("a = %u, b = %u\n", a, b);
    printf("a in binary: ");
    print_binary(a);
    printf("b in binary: ");
    print_binary(b);

    printf("\nBitwise operations:\n");
    printf("a & b = %u (AND)\n", a & b);
    printf("a | b = %u (OR)\n", a | b);
    printf("a ^ b = %u (XOR)\n", a ^ b);
    printf("~a = %u (NOT)\n", ~a);
    printf("a << 1 = %u (Left shift)\n", a << 1);
    printf("a >> 1 = %u (Right shift)\n", a >> 1);

    // Practical example: Setting, clearing, and toggling bits
    unsigned char flags = 0b00001010;  // 10 in decimal
    printf("\nFlag operations:\n");
    printf("Original flags: ");
    print_binary(flags);

    // Set bit 2 (make it 1)
    flags |= (1 << 2);
    printf("After setting bit 2: ");
    print_binary(flags);

    // Clear bit 3 (make it 0)
    flags &= ~(1 << 3);
    printf("After clearing bit 3: ");
    print_binary(flags);
```

```
    // Toggle bit 1
    flags ^= (1 << 1);
    printf("After toggling bit 1: ");
    print_binary(flags);

    return 0;
}
```

Assignment Operators

Assignment operators assign values to variables. C provides several compound assignment operators for convenience:

Operator	Description	Equivalent To
`=`	Simple assignment	`a = b`
`+=`	Add and assign	`a = a + b`
`-=`	Subtract and assign	`a = a - b`
`*=`	Multiply and assign	`a = a * b`
`/=`	Divide and assign	`a = a / b`
`%=`	Modulus and assign	`a = a % b`
`&=`	Bitwise AND and assign	`a = a & b`
`|=`	Bitwise OR and assign	`a = a | b`
`^=`	Bitwise XOR and assign	`a = a ^ b`
`<<=`	Left shift and assign	`a = a << b`
`>>=`	Right shift and assign	`a = a >> b`

Assignment Operations

```
#include <stdio.h>

int main() {
    int a = 10;
    printf("Initial value: a = %d\n", a);

    a += 5;    // a = a + 5
    printf("After a += 5: a = %d\n", a);

    a -= 3;    // a = a - 3
    printf("After a -= 3: a = %d\n", a);

    a *= 2;    // a = a * 2
    printf("After a *= 2: a = %d\n", a);
```

```
    a /= 4;    // a = a / 4
    printf("After a /= 4: a = %d\n", a);

    a %= 3;    // a = a % 3
    printf("After a %%= 3: a = %d\n", a);

    // Multiple assignments
    int x, y, z;
    x = y = z = 5;  // All variables get value 5
    printf("\nMultiple assignment: x = %d, y = %d, z = %d\n", x, y, z);

    return 0;
}
```

Increment and Decrement Operators

These operators increase or decrease a variable's value by 1:

Operator	Description	Example	Result
`++`	Pre-increment	`++a`	Increment a, then use new value
`++`	Post-increment	`a++`	Use current value, then increment a
`--`	Pre-decrement	`--a`	Decrement a, then use new value
`--`	Post-decrement	`a--`	Use current value, then decrement a

Increment/Decrement Examples

```
#include <stdio.h>

int main() {
    int a = 5, b = 5;
    int result1, result2;

    printf("Initial: a = %d, b = %d\n", a, b);

    // Pre-increment
    result1 = ++a;
    printf("After ++a: a = %d, result1 = %d\n", a, result1);

    // Post-increment
    result2 = b++;
    printf("After b++: b = %d, result2 = %d\n", b, result2);

    // Practical example in loops
```

```
    printf("\nLoop examples:\n");

    // Using pre-increment
    printf("Pre-increment loop:\n");
    for (int i = 0; i < 3; ++i) {
        printf("i = %d\n", i);
    }

    // Using post-increment
    printf("Post-increment loop:\n");
    for (int i = 0; i < 3; i++) {
        printf("i = %d\n", i);
    }

    // Be careful with complex expressions
    int x = 10;
    int y = ++x + x++;  // Undefined behavior!
    printf("\nUndefined behavior example: y = %d\n", y);

    return 0;
}
```

Conditional (Ternary) Operator

The conditional operator is C's only ternary operator, taking three operands:

```
condition ? value_if_true : value_if_false
```

Conditional Operator Examples

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    // Simple conditional assignment
    int max = (a > b) ? a : b;
    printf("Maximum of %d and %d is %d\n", a, b, max);

    // Conditional string selection
    char *message = (a % 2 == 0) ? "even" : "odd";
    printf("%d is %s\n", a, message);
```

```
    // Nested conditional operators
    int score = 85;
    char *grade = (score >= 90) ? "A" :
                  (score >= 80) ? "B" :
                  (score >= 70) ? "C" :
                  (score >= 60) ? "D" : "F";
    printf("Score %d gets grade %s\n", score, grade);

    // Conditional with function calls (be careful with side effects)
    int x = 5, y = 10;
    int result = (x > y) ? (x += 5) : (y += 5);
    printf("After conditional: x = %d, y = %d, result = %d\n", x, y, result);

    return 0;
}
```

Special Operators

sizeof Operator

Returns the size (in bytes) of a data type or variable:

```
#include <stdio.h>

int main() {
    printf("Size of char: %zu bytes\n", sizeof(char));
    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of float: %zu bytes\n", sizeof(float));
    printf("Size of double: %zu bytes\n", sizeof(double));
    printf("Size of pointer: %zu bytes\n", sizeof(int*));

    int array[10];
    printf("Size of array[10]: %zu bytes\n", sizeof(array));
    printf("Size of array element: %zu bytes\n", sizeof(array[0]));

    return 0;
}
```

Address-of (&) and Dereference (*) Operators

Used with pointers (covered in detail in Module 6):

```
#include <stdio.h>
```

```c
int main() {
    int number = 42;
    int *ptr = &number;  // Address-of operator

    printf("Value of number: %d\n", number);
    printf("Address of number: %p\n", (void*)&number);
    printf("Value of ptr: %p\n", (void*)ptr);
    printf("Value pointed to by ptr: %d\n", *ptr);  // Dereference operator

    return 0;
}
```

Operator Precedence and Associativity

Operators in C have specific precedence levels and associativity rules that determine the order of evaluation in expressions:

Precedence Levels (Highest to Lowest)

1. **Postfix**: `()`, `[]`, `->`, `.`, `++`, `--` (Left to Right)
2. **Unary**: `++`, `--`, `+`, `-`, `!`, `~`, `(type)`, `*`, `&`, `sizeof` (Right to Left)
3. **Multiplicative**: `*`, `/`, `%` (Left to Right)
4. **Additive**: `+`, `-` (Left to Right)
5. **Shift**: `<<`, `>>` (Left to Right)
6. **Relational**: `<`, `<=`, `>`, `>=` (Left to Right)
7. **Equality**: `==`, `!=` (Left to Right)
8. **Bitwise AND**: `&` (Left to Right)
9. **Bitwise XOR**: `^` (Left to Right)
10. **Bitwise OR**: `|` (Left to Right)
11. **Logical AND**: `&&` (Left to Right)
12. **Logical OR**: `||` (Left to Right)
13. **Conditional**: `?:` (Right to Left)
14. **Assignment**: `=`, `+=`, `-=`, etc. (Right to Left)
15. **Comma**: `,` (Left to Right)

Precedence Examples

```c
#include <stdio.h>

int main() {
    int a = 5, b = 3, c = 2;

    // Multiplication has higher precedence than addition
```

```
    int result1 = a + b * c;        // 5 + (3 * 2) = 11
    int result2 = (a + b) * c;      // (5 + 3) * 2 = 16

    printf("a + b * c = %d\n", result1);
    printf("(a + b) * c = %d\n", result2);

    // Relational operators have lower precedence than arithmetic
    int result3 = a > b + c;        // a > (b + c) = 5 > 5 = 0
    int result4 = (a > b) + c;      // (a > b) + c = 1 + 2 = 3

    printf("a > b + c = %d\n", result3);
    printf("(a > b) + c = %d\n", result4);

    // Logical operators have lower precedence than relational
    int x = 10, y = 5, z = 15;
    int result5 = x > y && y < z;   // (x > y) && (y < z) = 1 && 1 = 1
    int result6 = x > y & y < z;    // x > (y & y) < z = 10 > 1 < 15 = 1

    printf("x > y && y < z = %d\n", result5);
    printf("x > y & y < z = %d\n", result6);

    return 0;
}
```

Practical Examples

Mathematical Expression Evaluator

```
#include <stdio.h>

int main() {
    double x, y, z;

    printf("Enter three numbers: ");
    scanf("%lf %lf %lf", &x, &y, &z);

    // Complex mathematical expression
    double result = (x + y) * z / (x - y) + (x * y - z);

    printf("Result of (x + y) * z / (x - y) + (x * y - z):\n");
    printf("= (%.2f + %.2f) * %.2f / (%.2f - %.2f) + (%.2f * %.2f - %.2f)\n",
           x, y, z, x, y, x, y, z);
    printf("= %.2f\n", result);
```

```
    return 0;
}
```

Bit Manipulation Utility

```c
#include <stdio.h>

// Function to check if a bit is set
int is_bit_set(unsigned int num, int position) {
    return (num >> position) & 1;
}

// Function to set a bit
unsigned int set_bit(unsigned int num, int position) {
    return num | (1 << position);
}

// Function to clear a bit
unsigned int clear_bit(unsigned int num, int position) {
    return num & ~(1 << position);
}

// Function to toggle a bit
unsigned int toggle_bit(unsigned int num, int position) {
    return num ^ (1 << position);
}

void print_bits(unsigned int num) {
    printf("Binary: ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (num >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }
    printf("\n");
}

int main() {
    unsigned int number = 0b10101010;

    printf("Original number: %u\n", number);
    print_bits(number);

    // Check bit at position 3
    printf("Bit at position 3: %d\n", is_bit_set(number, 3));
```

```
    // Set bit at position 2
    number = set_bit(number, 2);
    printf("After setting bit 2: %u\n", number);
    print_bits(number);

    // Clear bit at position 7
    number = clear_bit(number, 7);
    printf("After clearing bit 7: %u\n", number);
    print_bits(number);

    // Toggle bit at position 5
    number = toggle_bit(number, 5);
    printf("After toggling bit 5: %u\n", number);
    print_bits(number);

    return 0;
}
```

Conditional Logic Demonstrator

```
#include <stdio.h>

int main() {
    int score;
    printf("Enter student score (0-100): ");
    scanf("%d", &score);

    // Validate input
    if (score < 0 || score > 100) {
        printf("Invalid score!\n");
        return 1;
    }

    // Determine letter grade using conditional operators
    char *letter_grade = (score >= 90) ? "A" :
                         (score >= 80) ? "B" :
                         (score >= 70) ? "C" :
                         (score >= 60) ? "D" : "F";

    // Determine pass/fail status
    char *status = (score >= 60) ? "PASS" : "FAIL";

    // Provide feedback
    char *feedback = (score >= 90) ? "Excellent!" :
                     (score >= 80) ? "Good job!" :
```

```
                    (score >= 70) ? "Satisfactory" :
                    (score >= 60) ? "Needs improvement" : "Failed";

    printf("\nResults:\n");
    printf("Score: %d\n", score);
    printf("Grade: %s\n", letter_grade);
    printf("Status: %s\n", status);
    printf("Feedback: %s\n", feedback);

    return 0;
}
```

Best Practices

1. Use Parentheses for Clarity

```
// Good: Clear intent
int result = (a + b) * c;

// Less clear: Relies on precedence knowledge
int result = a + b * c;
```

2. Be Careful with Side Effects

```
// Dangerous: Undefined behavior
int a = 5;
int result = a++ + ++a;

// Safe: Clear separation of operations
int a = 5;
int temp1 = a++;
int temp2 = ++a;
int result = temp1 + temp2;
```

3. Use Meaningful Variable Names in Complex Expressions

```
// Good: Self-documenting
int base_salary = 50000;
int bonus_percentage = 10;
int years_of_service = 5;
int total_compensation = base_salary + (base_salary * bonus_percentage / 100) * years_of_ser
```

```c
// Less clear
int a = 50000;
int b = 10;
int c = 5;
int d = a + (a * b / 100) * c;
```

4. Be Aware of Integer Division

```c
// If you want floating-point result
float average = (float)total / count;

// If you want integer result
int average = total / count;
```

5. Use Bitwise Operations for Bit Manipulation

```c
// Good for setting flags
flags |= FLAG_READ;

// Good for clearing flags
flags &= ~FLAG_WRITE;

// Good for checking flags
if (flags & FLAG_EXECUTE) {
    // Execute code
}
```

Summary

In this chapter, you've learned about:

1. **Arithmetic Operators**: `+`, `-`, `*`, `/`, `%` for mathematical operations
2. **Relational Operators**: `==`, `!=`, `>`, `<`, `>=`, `<=` for comparisons
3. **Logical Operators**: `&&`, `||`, `!` for boolean logic
4. **Bitwise Operators**: `&`, `|`, `^`, `~`, `<<`, `>>` for bit manipulation
5. **Assignment Operators**: `=`, `+=`, `-=`, etc. for value assignment
6. **Increment/Decrement Operators**: `++`, `--` for modifying values by one
7. **Conditional Operator**: `?:` for ternary selection
8. **Special Operators**: `sizeof`, `&`, `*` for specific operations
9. **Operator Precedence**: Rules determining evaluation order
10. **Best Practices**: Writing clear and safe expressions

Understanding operators and expressions is crucial for C programming, as they form the foundation for all program logic. With this knowledge, you're now ready to tackle more complex programming constructs like control flow statements in the next module.

Module 2: Data Types and Variables

Overview

This module delves into the fundamental building blocks of C programming: data types and variables. You'll learn about C's rich type system, from basic integer and floating-point types to advanced fixed-width types introduced in modern C standards. The module also covers variable declaration, initialization, scope, constants, and type conversions.

Learning Objectives

By the end of this module, you will be able to: - Understand and use all fundamental C data types - Declare, initialize, and manipulate variables effectively - Work with constants using various methods (#define, const, enum) - Apply storage classes (auto, static, register, extern) appropriately - Perform safe type conversions and understand implicit/explicit casting - Use operators effectively to create complex expressions - Understand operator precedence and associativity - Apply best practices for data type selection and variable management

Chapters

1. **Fundamental Data Types** - Integer, floating-point, character, and boolean types
2. **Variables and Constants** - Declaration, initialization, scope, and storage classes
3. **Type Conversions** - Implicit and explicit conversions, potential pitfalls
4. **Operators and Expressions** - Arithmetic, relational, logical, bitwise, and assignment operators

Key Concepts Covered

- Integer types (char, short, int, long, long long) and their unsigned variants
- Floating-point types (float, double, long double)
- Fixed-width integer types (int8_t, int16_t, int32_t, int64_t)
- Boolean types (_Bool, bool with stdbool.h)
- Variable declaration, initialization, and naming conventions
- Constants using #define, const, and enum
- Storage classes (auto, static, register, extern)
- Scope (local, global) and lifetime of variables
- Type qualifiers (const, volatile, restrict)

- Implicit and explicit type conversions
- Arithmetic, relational, logical, bitwise, and assignment operators
- Operator precedence and associativity
- Bit manipulation techniques
- Safe conversion practices

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Basic understanding of C program structure
- Familiarity with basic I/O operations

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Understanding of number systems (binary, hexadecimal)
- Basic mathematical concepts

Estimated Time to Complete

- Reading: 6-8 hours
- Exercises: 8-10 hours
- Projects: 3-5 hours

Assessment

- Complete all chapter exercises
- Successfully implement variable manipulation programs
- Demonstrate understanding of type conversions
- Pass operators and expressions quiz
- Submit a comprehensive data type demonstration program

Next Module

After completing this module, proceed to **Module 3: Control Flow** to learn about decision-making and looping constructs in C.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 2)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Online references for limits.h and stdint.h
- Bit manipulation tutorials and resources
- Operator precedence tables

Control Flow

Conditional Statements

Introduction

Conditional statements are fundamental control structures that allow programs to make decisions and execute different code paths based on specific conditions. They enable programs to respond dynamically to varying inputs, user interactions, and runtime conditions.

In C, conditional statements provide the foundation for implementing logic, validation, error handling, and complex decision-making processes. This chapter explores `if` / `else`, `switch`, nested conditions, the ternary operator, common pitfalls, and several full worked programs you can compile on Linux.

Compile flag convention used throughout this book:

```
gcc -std=c17 -Wall -Wextra -o program program.c
./program
```

The if Statement

The `if` statement is the most basic conditional construct in C. It executes a block of code only if a specified condition evaluates to true (non-zero).

Basic Syntax

```
if (condition) {
    // Code to execute if condition is true
}
```

Simple if Statement

```
#include <stdio.h>

int main(void) {
    int age = 20;

    if (age >= 18) {
```

```
        printf("You are eligible to vote.\n");
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o if_simple if_simple.c
./if_simple
```

if-else Statement

The `if-else` construct allows you to specify alternative code paths for when the condition is true or false:

```
#include <stdio.h>

int main(void) {
    int number = 15;

    if (number % 2 == 0) {
        printf("%d is even.\n", number);
    } else {
        printf("%d is odd.\n", number);
    }

    return 0;
}
```

if-else if-else Chain

For multiple conditions, chain `if-else if-else` statements. Order matters: the first true branch wins.

```
#include <stdio.h>

int main(void) {
    int score = 85;

    if (score >= 90) {
        printf("Grade: A\n");
    } else if (score >= 80) {
        printf("Grade: B\n");
    } else if (score >= 70) {
        printf("Grade: C\n");
```

```
    } else if (score >= 60) {
        printf("Grade: D\n");
    } else {
        printf("Grade: F\n");
    }

    return 0;
}
```

The switch Statement

The `switch` statement provides an alternative to long `if-else if` chains when comparing a single expression against multiple **constant integer** values (including character constants).

Basic Syntax

```
switch (expression) {
    case constant1:
        // Code for constant1
        break;
    case constant2:
        // Code for constant2
        break;
    default:
        // Code for no match
        break;
}
```

Simple switch Example

```
#include <stdio.h>

int main(void) {
    int day = 3;

    switch (day) {
        case 1:  printf("Monday\n");    break;
        case 2:  printf("Tuesday\n");   break;
        case 3:  printf("Wednesday\n"); break;
        case 4:  printf("Thursday\n");  break;
        case 5:  printf("Friday\n");    break;
        case 6:  printf("Saturday\n");  break;
```

```
        case 7:  printf("Sunday\n");    break;
        default: printf("Invalid day\n"); break;
    }

    return 0;
}
```

Intentional Fall-Through

Without `break`, control falls into the next case. Use this only deliberately, and document it:

```
#include <stdio.h>

int main(void) {
    int month = 2;

    switch (month) {
        case 12:
        case 1:
        case 2:
            printf("Winter\n");
            break;
        case 3:
        case 4:
        case 5:
            printf("Spring\n");
            break;
        case 6:
        case 7:
        case 8:
            printf("Summer\n");
            break;
        case 9:
        case 10:
        case 11:
            printf("Autumn\n");
            break;
        default:
            printf("Invalid month\n");
            break;
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o seasons seasons.c
./seasons
```

Nested Conditionals

Conditionals can be nested to handle multi-factor decisions:

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int age = 25;
    bool has_license = true;

    if (age >= 18) {
        if (has_license) {
            printf("You can drive legally.\n");
        } else {
            printf("You need to get a license.\n");
        }
    } else {
        printf("You are not old enough to drive.\n");
    }

    return 0;
}
```

Nested Condition Pitfalls

Deep nesting is hard to read and easy to get wrong. Prefer early returns or combined boolean expressions when the logic allows.

```c
#include <stdio.h>
#include <stdbool.h>

/* Nested - correct but noisy */
static void check_access_nested(int age, bool licensed, bool insured) {
    if (age >= 18) {
        if (licensed) {
            if (insured) {
                printf("Drive allowed.\n");
            } else {
                printf("Need insurance.\n");
            }
```

```
        } else {
            printf("Need license.\n");
        }
    } else {
        printf("Too young.\n");
    }
}

/* Flattened with early exits - usually clearer */
static void check_access_flat(int age, bool licensed, bool insured) {
    if (age < 18) {
        printf("Too young.\n");
        return;
    }
    if (!licensed) {
        printf("Need license.\n");
        return;
    }
    if (!insured) {
        printf("Need insurance.\n");
        return;
    }
    printf("Drive allowed.\n");
}

int main(void) {
    check_access_nested(20, true, false);
    check_access_flat(20, true, false);
    return 0;
}
```

The Dangling else (Else-Bind Ambiguity)

In C, an `else` always binds to the **nearest unmatched `if`**. Without braces this can surprise you.

```
#include <stdio.h>

int main(void) {
    int x = 0;
    int y = 1;

    /* DANGEROUS style (shown only to illustrate binding):
     *
     * if (x)
```

```
     *     if (y)
     *         printf("both\n");
     * else
     *     printf("x was false?\n");   // actually binds to if (y)!
     *
     * Always use braces:
     */

    if (x) {
        if (y) {
            printf("both\n");
        }
    } else {
        printf("x was false (this else belongs to if (x))\n");
    }

    /* Explicit binding for the inner else */
    if (x) {
        if (y) {
            printf("both\n");
        } else {
            printf("x true, y false\n");
        }
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o dangling_else dangling_else.c
./dangling_else
```

Rule: always brace `if` / `else` bodies, even one-liners. It eliminates the ambiguity.

Conditional (Ternary) Expressions

C supports the conditional operator as a shorthand for simple value selection:

```
#include <stdio.h>

int main(void) {
    int a = 10, b = 20;

    int max_if;
    if (a > b) {
        max_if = a;
```

```
    } else {
        max_if = b;
    }

    int max_ternary = (a > b) ? a : b;

    printf("Max (if-else): %d\n", max_if);
    printf("Max (ternary): %d\n", max_ternary);

    int age = 17;
    const char *status = (age >= 18) ? "adult" : "minor";
    printf("Status: %s\n", status);

    return 0;
}
```

Ternary Best Practices

Prefer ternary when…	Prefer if/else when…
Selecting a **value** for assignment/return	The branches have **side effects** (I/O, mutations)
Expression stays on one readable line	Nested ternaries would appear (`a ? b : c ? d : e`)
Types of both arms match cleanly	Branches need multiple statements

```
#include <stdio.h>

/* Good: pure value selection */
static int clamp(int v, int lo, int hi) {
    return (v < lo) ? lo : (v > hi) ? hi : v;  /* nested: still OK if tiny */
}

/* Better readability for nested selection */
static int clamp_clear(int v, int lo, int hi) {
    if (v < lo) {
        return lo;
    }
    if (v > hi) {
        return hi;
    }
    return v;
}

/* Bad: side effects buried in a ternary */
```

```c
static void bad_style(int flag) {
    /* avoid: flag ? printf("yes\n") : printf("no\n"); */
    if (flag) {
        printf("yes\n");
    } else {
        printf("no\n");
    }
}

int main(void) {
    printf("clamp(150, 0, 100) = %d\n", clamp(150, 0, 100));
    printf("clamp_clear(-5, 0, 100) = %d\n", clamp_clear(-5, 0, 100));
    bad_style(1);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o ternary_style ternary_style.c
./ternary_style
```

Best Practices for Conditionals

1. Use Braces Consistently

```c
/* Good */
if (condition) {
    do_something();
}

/* Avoid: inconsistent / brace-less style */
if (condition)
    do_something();
else
    do_something_else();
```

2. Handle All Cases in switch

Always include `default` for unexpected values (or document why it is impossible). Always `break` unless fall-through is intentional and commented.

3. Prefer Early Returns Over Deep Nesting

```
int process_data(int data) {
    if (data < 0) {
        return -1;
    }
    if (data > 100) {
        return -2;
    }
    return data * 2;
}
```

4. Name Complex Conditions

```
int is_adult = (age >= 18);
int has_permission = (permission_level > 0);

if (is_adult && has_permission) {
    grant_access();
}
```

Common Pitfalls

1. Assignment vs Comparison

```
#include <stdio.h>

int main(void) {
    int x = 5;

    /* Wrong: assignment; condition is always true here after assign */
    if (x = 10) {
        printf("Assigned and truthy: x = %d\n", x);
    }

    /* Correct comparison */
    if (x == 10) {
        printf("x equals 10\n");
    }

    /* Yoda style some teams use to catch typos: if (10 == x) */
    if (10 == x) {
        printf("Yoda comparison OK\n");
```

```
    }

    return 0;
}
```

With `-Wall`, `gcc` warns about assignment used as a condition unless you add extra parentheses: `if ((x = 10))`.

2. Floating-Point Comparisons

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.1 + 0.2;
    double b = 0.3;

    if (a == b) {
        printf("Equal\n");
    } else {
        printf("Not bit-equal (a = %.20f, b = %.20f)\n", a, b);
    }

    const double epsilon = 1e-9;
    if (fabs(a - b) < epsilon) {
        printf("Approximately equal\n");
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o fp_cmp fp_cmp.c -lm
./fp_cmp
```

3. Missing break in switch

Fall-through without intent is a classic bug. Enable and respect warnings; some projects use `/* fallthrough */` comments for intentional cases (GCC recognizes them).

Worked Example 1: Full Grading System

Letter grade, pass/fail, GPA-style points, and input validation.

```c
/* grade_system.c - interactive grading helper */
#include <stdio.h>

static char letter_grade(float score) {
    if (score >= 90.0f) {
        return 'A';
    }
    if (score >= 80.0f) {
        return 'B';
    }
    if (score >= 70.0f) {
        return 'C';
    }
    if (score >= 60.0f) {
        return 'D';
    }
    return 'F';
}

static float grade_points(char letter) {
    switch (letter) {
        case 'A': return 4.0f;
        case 'B': return 3.0f;
        case 'C': return 2.0f;
        case 'D': return 1.0f;
        default:  return 0.0f;
    }
}

static const char *feedback(char letter) {
    switch (letter) {
        case 'A': return "Excellent";
        case 'B': return "Good";
        case 'C': return "Satisfactory";
        case 'D': return "Needs improvement";
        default:  return "Failing - retake recommended";
    }
}

int main(void) {
    float score;

    printf("Enter student score (0-100): ");
    if (scanf("%f", &score) != 1) {
        printf("Invalid input (not a number).\n");
        return 1;
    }
```

```
    if (score < 0.0f || score > 100.0f) {
        printf("Invalid score! Enter a value between 0 and 100.\n");
        return 1;
    }

    char letter = letter_grade(score);
    float gpa = grade_points(letter);

    printf("Score : %.1f\n", score);
    printf("Grade : %c (%s)\n", letter, feedback(letter));
    printf("Points: %.1f / 4.0\n", gpa);

    if (score >= 60.0f) {
        printf("Result: PASS\n");
    } else {
        printf("Result: FAIL\n");
    }

    /* Plus/minus band using nested conditions */
    if (letter != 'F') {
        int band = (int)score % 10;
        if (band >= 7 || score >= 97.0f) {
            printf("Band  : %c+\n", letter);
        } else if (band <= 2 && score < 90.0f) {
            printf("Band  : %c-\n", letter);
        } else {
            printf("Band  : %c\n", letter);
        }
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o grade_system grade_system.c
echo "87.5" | ./grade_system
```

Worked Example 2: Leap Year (Rules + Nested Form)

Gregorian rule: divisible by 4, except centuries unless also divisible by 400.

```
/* leap_year.c */
#include <stdio.h>
```

```c
#include <stdbool.h>

static bool is_leap_flat(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

static bool is_leap_nested(int year) {
    if (year % 4 == 0) {
        if (year % 100 == 0) {
            if (year % 400 == 0) {
                return true;
            }
            return false;
        }
        return true;
    }
    return false;
}

int main(void) {
    int year;

    printf("Enter a year: ");
    if (scanf("%d", &year) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    if (year <= 0) {
        printf("Use a positive year (Gregorian-style check).\n");
        return 1;
    }

    bool flat = is_leap_flat(year);
    bool nest = is_leap_nested(year);

    printf("%d is %sa leap year (flat).\n", year, flat ? "" : "not ");
    printf("%d is %sa leap year (nested).\n", year, nest ? "" : "not ");

    /* Spot-check known cases without user input */
    int samples[] = {1900, 2000, 2024, 2023};
    printf("\nSelf-check:\n");
    for (int i = 0; i < 4; i++) {
        int y = samples[i];
        printf("  %d -> %s\n", y, is_leap_flat(y) ? "leap" : "common");
    }
```

```
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o leap_year leap_year.c
echo "2000" | ./leap_year
```

Expected self-check: 1900 common, 2000 leap, 2024 leap, 2023 common.

Worked Example 3: Simple ATM Menu with switch

Balance, deposit, withdraw, and PIN check — menu loop with `switch`.

```
/* atm_menu.c - console ATM simulation */
#include <stdio.h>
#include <stdbool.h>

#define PIN_CODE 1234
#define MAX_TRIES 3

int main(void) {
    double balance = 1000.00;
    int pin;
    int tries = 0;
    bool authenticated = false;

    printf("=== Mini ATM ===\n");
    while (tries < MAX_TRIES) {
        printf("Enter PIN: ");
        if (scanf("%d", &pin) != 1) {
            printf("Invalid input.\n");
            /* drain line */
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
            }
            tries++;
            continue;
        }
        if (pin == PIN_CODE) {
            authenticated = true;
            break;
        }
        tries++;
        printf("Wrong PIN (%d/%d).\n", tries, MAX_TRIES);
```

```
    }

    if (!authenticated) {
        printf("Card locked. Goodbye.\n");
        return 1;
    }

    int choice = -1;
    do {
        printf("\n--- Menu ---\n");
        printf("1. Check balance\n");
        printf("2. Deposit\n");
        printf("3. Withdraw\n");
        printf("4. Quick $50 withdraw\n");
        printf("0. Exit\n");
        printf("Choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Please enter a number.\n");
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
            }
            choice = -1;
            continue;
        }

        switch (choice) {
            case 1:
                printf("Balance: $%.2f\n", balance);
                break;

            case 2: {
                double amount;
                printf("Deposit amount: ");
                if (scanf("%lf", &amount) != 1 || amount <= 0.0) {
                    printf("Invalid amount.\n");
                    break;
                }
                balance += amount;
                printf("Deposited. New balance: $%.2f\n", balance);
                break;
            }

            case 3: {
                double amount;
                printf("Withdraw amount: ");
                if (scanf("%lf", &amount) != 1 || amount <= 0.0) {
```

```
                    printf("Invalid amount.\n");
                    break;
                }
                if (amount > balance) {
                    printf("Insufficient funds.\n");
                } else {
                    balance -= amount;
                    printf("Withdrawn. New balance: $%.2f\n", balance);
                }
                break;
            }

            case 4:
                if (balance < 50.0) {
                    printf("Insufficient funds for quick withdraw.\n");
                } else {
                    balance -= 50.0;
                    printf("Dispensed $50.00. Balance: $%.2f\n", balance);
                }
                break;

            case 0:
                printf("Thank you. Session ended.\n");
                break;

            default:
                printf("Unknown option.\n");
                break;
        }
    } while (choice != 0);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o atm_menu atm_menu.c
# Interactive: enter PIN 1234, then try menu options
./atm_menu
```

Short-Circuit Evaluation

`&&` and `||` stop evaluating once the result is known. Use this to guard expensive or unsafe operations:

```c
#include <stdio.h>

int main(void) {
    int a = 5, b = 0;

    /* Safe: a / b never runs when b == 0 */
    if (b != 0 && a / b > 1) {
        printf("Division is safe and result > 1\n");
    } else {
        printf("Skipped unsafe division or result not > 1\n");
    }

    int calls = 0;
    if (a > 10 && (++calls > 0)) {
        printf("Won't print\n");
    }
    printf("Side-effect calls: %d (short-circuit skipped ++)\n", calls);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o short_circuit short_circuit.c
./short_circuit
```

Exercises

Exercise 1 — Triangle classifier

Read three side lengths. Print whether they form a valid triangle, and if so whether it is equilateral, isosceles, or scalene. Reject non-positive sides.

Solution sketch:

```c
/* Valid if a+b>c, a+c>b, b+c>a for all permutations.
 * Then: all equal → equilateral; any two equal → isosceles; else scalene.
 * Use if / else if chain; avoid floating equality issues by using int sides first.
 */
```

```
gcc -std=c17 -Wall -Wextra -o triangle triangle.c
```

Exercise 2 — Rock–paper–scissors

Map chars `r/p/s` (or integers 0/1/2) for player and computer. Use `switch` for the winner matrix. Print win/lose/draw.

Solution sketch:

```
/* switch (player) {
 *   case 'r': if (cpu=='s') win; else if (cpu=='p') lose; else draw; break;
 *   ...
 * }
 * Or encode both as ints and use (player - cpu + 3) % 3.
 */
```

Exercise 3 — BMI category

Read weight (kg) and height (m). Compute BMI = kg / m². Categories: underweight <18.5, normal <25, overweight <30, obese 30. Guard height 0.

Solution sketch:

```
/* if (height <= 0) error;
 * double bmi = weight / (height * height);
 * else-if ladder on thresholds; print bmi with %.1f
 */
```

Exercise 4 — Rewrite dangling-else trap

Given brace-less nested `if/else` that misclassifies, rewrite with braces so the `else` binds to the outer `if`. Add a unit-style table of `(x,y)` inputs and expected messages.

Exercise 5 — ATM extension

Extend `atm_menu.c` with a transfer option (case 5) that subtracts from balance only if funds suffice, and logs a fake destination account id. Add a daily withdraw limit with a nested check.

Summary

In this chapter you covered:

1. **if / if-else / else-if** — ordered multi-way branching

2. **switch** — integer/character multi-way with break and intentional fall-through
3. **Nested conditions** — when needed, and how to flatten
4. **Dangling else** — else binds to nearest unmatched if; always use braces
5. **Ternary** — value selection; avoid side effects and deep nesting
6. **Worked programs** — grading system, leap year, ATM menu
7. **Pitfalls** — `=` vs `==`, float compares, missing `break`, short-circuit
8. **Exercises** — triangle, RPS, BMI, else-binding, ATM extensions

Next: loops (`while`, `do-while`, `for`) for repetition and iterative algorithms.

Loops

Introduction

Loops are control structures that allow programs to repeat a block of code multiple times. They are essential for processing collections of data, implementing algorithms, and automating repetitive tasks. C provides three main loop kinds: `while`, `do-while`, and `for`.

This chapter covers syntax, a **for vs while vs do-while** decision guide, nested patterns, infinite loops for servers/embedded-style code, worked examples (primes, FizzBuzz, tables, running average), and exercises.

Compile convention:

```
gcc -std=c17 -Wall -Wextra -o program program.c
./program
```

The while Loop

The `while` loop is a **pre-test** loop: the condition is evaluated before each iteration. The body may run zero times.

Basic Syntax

```
while (condition) {
    // Code to execute repeatedly
}
```

Simple while Loop

```
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 5) {
        printf("Count: %d\n", count);
```

```
        count++;  /* must update the control variable */
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o while_simple while_simple.c
./while_simple
```

while with User Input

```
#include <stdio.h>

int main(void) {
    int number;

    printf("Enter positive numbers (0 to stop):\n");
    if (scanf("%d", &number) != 1) {
        return 1;
    }

    while (number > 0) {
        printf("You entered: %d\n", number);
        if (scanf("%d", &number) != 1) {
            break;
        }
    }

    printf("Loop ended.\n");
    return 0;
}
```

The do-while Loop

The `do-while` loop is a **post-test** loop: the body runs **at least once**, then the condition is checked.

Basic Syntax

```
do {
    // Code to execute
} while (condition);  /* note the trailing semicolon */
```

Simple do-while

```
#include <stdio.h>

int main(void) {
    int count = 1;

    do {
        printf("Count: %d\n", count);
        count++;
    } while (count <= 5);

    return 0;
}
```

Menu Pattern (classic do-while use)

```
#include <stdio.h>

int main(void) {
    int choice;

    do {
        printf("\n=== Menu ===\n");
        printf("1. Option 1\n");
        printf("2. Option 2\n");
        printf("0. Exit\n");
        printf("Choice: ");
        if (scanf("%d", &choice) != 1) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
            }
            choice = -1;
            continue;
        }

        switch (choice) {
```

```
            case 1: printf("Option 1\n"); break;
            case 2: printf("Option 2\n"); break;
            case 0: printf("Exiting...\n"); break;
            default: printf("Invalid choice.\n"); break;
        }
    } while (choice != 0);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o menu_do menu_do.c
./menu_do
```

The for Loop

The `for` loop packs initialization, condition, and update in one place. Ideal when iteration count or index range is clear.

Basic Syntax

```
for (initialization; condition; update) {
    // body
}
```

Simple for Loop

```
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("Count: %d\n", i);
    }
    return 0;
}
```

for with Arrays

```
#include <stdio.h>

int main(void) {
    int numbers[] = {10, 20, 30, 40, 50};
```

```
    int size = (int)(sizeof numbers / sizeof numbers[0]);

    for (int i = 0; i < size; i++) {
        printf("numbers[%d] = %d\n", i, numbers[i]);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o for_array for_array.c
./for_array
```

Any of the three `for` clauses may be empty:

```
int i = 0;
for (; i < 10; ) {
    printf("%d\n", i);
    i++;
}
```

Decision Guide: for vs while vs do-while

Situation	Prefer	Why
Known count / index over a range	`for`	Init, test, and step live together; hard to forget the step
Iterate until a **runtime condition** (file ends, sentinel input)	`while`	Condition is the story; body may not run
Must show a menu / prompt **at least once**	`do-while`	Post-test guarantees one execution
Event loop / server / firmware "forever"	`while (1)` or `for (;;)`	Exit only via `break`, return, or signal
Multiple control variables advanced together	`for` (multi-update)	e.g. `for (i = 0, j = n-1; i < j; i++, j--)`

Same Task Three Ways

Sum integers from 1 to `n`:

```c
#include <stdio.h>

int main(void) {
    int n = 10;
    long sum;

    /* for - clearest for counted range */
    sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    printf("for:      %ld\n", sum);

    /* while - same logic, control variable outside */
    sum = 0;
    int i = 1;
    while (i <= n) {
        sum += i;
        i++;
    }
    printf("while:    %ld\n", sum);

    /* do-while - awkward if n could be 0; body always runs once */
    sum = 0;
    i = 1;
    if (n >= 1) {
        do {
            sum += i;
            i++;
        } while (i <= n);
    }
    printf("do-while: %ld\n", sum);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o sum_three sum_three.c
./sum_three
```

When while Beats for

Reading until failure — the “next value” is also the loop condition:

```c
#include <stdio.h>
```

```c
int main(void) {
    int x;
    long sum = 0;
    int count = 0;

    printf("Enter integers; non-numeric or EOF ends input:\n");
    while (scanf("%d", &x) == 1) {
        sum += x;
        count++;
    }

    if (count == 0) {
        printf("No numbers.\n");
    } else {
        printf("count=%d sum=%ld avg=%.2f\n",
               count, sum, (double)sum / count);
    }
    return 0;
}
```

Loop Control: break and continue

break

Leaves the **innermost** loop immediately.

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break;
        }
        printf("%d ", i);
    }
    printf("\nDone.\n");
    return 0;
}
```

continue

Skips the rest of the current iteration and proceeds to the next test/update.

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;  /* skip evens */
        }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Nested Loops

Multiplication Table Pattern

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

Nested break/continue Scope

`break` and `continue` affect only the **inner** loop unless you use flags or `goto` (rare) for multi-level exit.

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("outer %d\n", i);
        for (int j = 1; j <= 5; j++) {
            if (j == 3) {
                continue;  /* next j */
```

```
            }
            if (i == 2 && j == 4) {
                break;      /* leave inner only */
            }
            printf("  inner %d\n", j);
        }
    }
    return 0;
}
```

Nested Loop Patterns Cheatsheet

Pattern	Structure	Use
Rectangle	outer rows × inner cols	grids, matrices, tables
Triangle	inner runs to `i` or `n-i`	patterns, combinations prefix
Pairwise	`for i` / `for j = i+1`	unique pairs without double count
2D walk	`for r` / `for c` over `a[r][c]`	images, boards

```
#include <stdio.h>

int main(void) {
    int n = 4;

    printf("Triangle:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    printf("\nUnique pairs (i < j):\n");
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            printf("(%d,%d) ", i, j);
        }
    }
    printf("\n");
    return 0;
```

```
}
```

Infinite Loops (Servers, Embedded, Tools)

Infinite loops are normal when the process should run until an explicit stop condition.

Forms

```
while (1) { /* ... */ }
while (true) { /* need stdbool.h */ }
for (;;) { /* idiomatic "forever" */ }
```

Pattern: Event Loop with break Conditions

```
/* event_loop_demo.c - desktop stand-in for a service loop */
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

int main(void) {
    char line[128];
    bool running = true;
    unsigned long ticks = 0;

    printf("Commands: status | quit | panic\n");

    while (running) {
        ticks++;
        printf("> ");
        if (!fgets(line, sizeof line, stdin)) {
            /* EOF: clean shutdown */
            break;
        }

        /* strip newline */
        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "quit") == 0) {
            running = false;          /* graceful flag */
        } else if (strcmp(line, "panic") == 0) {
```

```
            printf("Fatal: breaking out immediately.\n");
            break;                      /* hard exit from loop */
        } else if (strcmp(line, "status") == 0) {
            printf("ticks=%lu running=%d\n", ticks, running);
        } else if (line[0] != '\0') {
            printf("Unknown command: %s\n", line);
        }

        /* In a real server: poll sockets, handle timers, etc. here */
    }

    printf("Shutdown after %lu loop iterations.\n", ticks);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o event_loop_demo event_loop_demo.c
printf 'status\nquit\n' | ./event_loop_demo
```

Pattern: Embedded-Style Superloop

```
#include <stdio.h>
#include <stdbool.h>

/* Simulated "hardware" flags */
static volatile bool g_button_irq;
static volatile bool g_shutdown;

static void poll_inputs(void) {
    /* On bare metal: read GPIO. Here we fake nothing. */
}

static void service_button(void) {
    if (g_button_irq) {
        g_button_irq = false;
        printf("button handled\n");
    }
}

/* Call this many times; break when shutdown requested */
static void superloop_iteration(void) {
    poll_inputs();
    service_button();
    /* feed watchdog, sleep until interrupt, etc. */
}
```

```c
int main(void) {
    int simulated_cycles = 0;

    for (;;) {
        superloop_iteration();
        simulated_cycles++;

        /* Host demo stop conditions */
        if (simulated_cycles >= 3) {
            g_shutdown = true;
        }
        if (g_shutdown) {
            break;
        }
    }

    printf("MCU superloop exited cleanly.\n");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o superloop superloop.c
./superloop
```

Pattern: Retry with Cap (avoid true infinite hang)

```c
#include <stdio.h>
#include <stdbool.h>

static bool try_connect(int attempt) {
    printf("connect attempt %d\n", attempt);
    return attempt >= 3;  /* succeed on 3rd try in this demo */
}

int main(void) {
    const int max_attempts = 5;
    int attempt = 0;
    bool ok = false;

    while (attempt < max_attempts) {
        attempt++;
        if (try_connect(attempt)) {
            ok = true;
            break;
        }
    }
```

```
    printf(ok ? "Connected.\n" : "Gave up.\n");
    return ok ? 0 : 1;
}
```

Worked Example 1: Prime Check

```
/* prime_check.c */
#include <stdio.h>
#include <stdbool.h>
#include <math.h>

static bool is_prime(long n) {
    if (n <= 1) {
        return false;
    }
    if (n <= 3) {
        return true;
    }
    if (n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    /* check 6k ± 1 up to sqrt(n) */
    for (long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}

int main(void) {
    long n;
    printf("Enter integer: ");
    if (scanf("%ld", &n) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%ld is prime.\n", n);
    } else {
        printf("%ld is not prime.\n", n);
```

```
    }

    printf("Primes up to 50: ");
    for (long i = 2; i <= 50; i++) {
        if (is_prime(i)) {
            printf("%ld ", i);
        }
    }
    printf("\n");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o prime_check prime_check.c
echo 97 | ./prime_check
```

Worked Example 2: FizzBuzz

```
/* fizzbuzz.c */
#include <stdio.h>

int main(void) {
    int limit = 30;

    for (int i = 1; i <= limit; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
        }
    }

    /* Compact variant with continue-style layering */
    printf("--- compact ---\n");
    for (int i = 1; i <= 15; i++) {
        int printed = 0;
        if (i % 3 == 0) {
            printf("Fizz");
            printed = 1;
```

```
        }
        if (i % 5 == 0) {
            printf("Buzz");
            printed = 1;
        }
        if (!printed) {
            printf("%d", i);
        }
        printf("\n");
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o fizzbuzz fizzbuzz.c
./fizzbuzz
```

Worked Example 3: Multiplication Table

```
/* mult_table.c */
#include <stdio.h>

int main(void) {
    int n;

    printf("Table size (1-12): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 12) {
        printf("Use an integer from 1 to 12.\n");
        return 1;
    }

    printf("    ");
    for (int c = 1; c <= n; c++) {
        printf("%4d", c);
    }
    printf("\n");

    for (int r = 1; r <= n; r++) {
        printf("%4d", r);
        for (int c = 1; c <= n; c++) {
            printf("%4d", r * c);
        }
```

```
        printf("\n");
    }

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o mult_table mult_table.c
echo 10 | ./mult_table
```

Worked Example 4: Running Average from Input

Read numbers until a sentinel (or EOF); maintain count and sum for a live average.

```
/* running_avg.c */
#include <stdio.h>

int main(void) {
    double x;
    double sum = 0.0;
    int count = 0;

    printf("Enter numbers (non-numeric or Ctrl-D to finish):\n");

    while (scanf("%lf", &x) == 1) {
        count++;
        sum += x;
        printf("  n=%d  last=%.4f  sum=%.4f  avg=%.4f\n",
               count, x, sum, sum / count);
    }

    if (count == 0) {
        printf("No data.\n");
        return 1;
    }

    printf("Final average of %d values: %.4f\n", count, sum / count);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o running_avg running_avg.c
printf '10 20 30 40\n' | ./running_avg
```

Sentinel variant (stop on a magic value without relying on EOF):

```c
/* running_avg_sentinel.c */
#include <stdio.h>

int main(void) {
    double x, sum = 0.0;
    int count = 0;

    printf("Enter numbers; enter -999 to stop:\n");
    while (scanf("%lf", &x) == 1) {
        if (x == -999.0) {
            break;
        }
        count++;
        sum += x;
        printf("  running avg = %.4f\n", sum / count);
    }
    if (count > 0) {
        printf("Final avg = %.4f\n", sum / count);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o running_avg_sentinel running_avg_sentinel.c
printf '2 4 6 -999\n' | ./running_avg_sentinel
```

Performance Notes

```c
#include <stdio.h>

int main(void) {
    int array[1000];
    const int n = (int)(sizeof array / sizeof array[0]);

    /* Hoist invariant size (here already a constant) */
    for (int i = 0; i < n; i++) {
        array[i] = i;
    }

    long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += array[i];
    }
```

```
    printf("sum=%ld\n", sum);
    return 0;
}
```

Avoid `float`/`double` as the primary loop counter (precision can skip or overshoot the end test). Use integer counters and derive floats inside the body.

Best Practices

1. **Initialize** control variables clearly; prefer `for (int i = 0; ...)` (C99+).
2. **Name** indices meaningfully in complex loops (`row`, `col`, `student_i`).
3. Prefer **`break` / flags** over mutating the loop index mid-body.
4. Match **loop kind** to the problem (see decision guide).
5. Always ensure a **progress condition** (or intentional infinite loop with external exit).

Common Pitfalls

Infinite loop by forgetting the update

```
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    /* missing i++ */
}
```

Off-by-one

```
int a[5];
for (int i = 0; i <= 5; i++) {  /* BAD: i==5 is out of bounds */
    a[i] = 0;
}
/* Correct: i < 5 or i < n */
```

Floating loop variable

```
/* Fragile */
for (float x = 0.0f; x != 1.0f; x += 0.1f) { }

/* Robust */
for (int i = 0; i < 10; i++) {
    float x = (float)i * 0.1f;
}
```

More Practical Examples

Number Patterns

```
#include <stdio.h>

int main(void) {
    int rows = 5;

    printf("Right triangle:\n");
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    printf("\nPyramid:\n");
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= rows - i; j++) {
            printf(" ");
        }
        for (int k = 1; k <= 2 * i - 1; k++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o patterns patterns.c
./patterns
```

Array Stats and Reverse

```c
#include <stdio.h>

int main(void) {
    int numbers[] = {12, 7, 23, 8, 15, 3, 19, 11, 5, 17};
    int size = (int)(sizeof numbers / sizeof numbers[0]);

    int max = numbers[0];
    for (int i = 1; i < size; i++) {
        if (numbers[i] > max) {
            max = numbers[i];
        }
    }
    printf("Max: %d\n", max);

    long sum = 0;
    for (int i = 0; i < size; i++) {
        sum += numbers[i];
    }
    printf("Average: %.2f\n", (double)sum / size);

    for (int i = 0, j = size - 1; i < j; i++, j--) {
        int tmp = numbers[i];
        numbers[i] = numbers[j];
        numbers[j] = tmp;
    }

    printf("Reversed: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o array_stats array_stats.c
./array_stats
```

Exercises

Exercise 1 — Collatz steps

Read a positive integer `n`. While `n != 1`, if even set `n = n/2`, else `n = 3n+1`. Count steps. Print the sequence.

Sketch: `while (n != 1) { print; if (n%2==0) n/=2; else n=3*n+1; steps++; }` Guard overflow with `long`.

```
gcc -std=c17 -Wall -Wextra -o collatz collatz.c
```

Exercise 2 — Nested loop: prime pairs

Find all pairs `(p, q)` with `2 p q 50` where both are prime and `p + q == 50` (Goldbach-style for 50).

Sketch: outer `p`, inner not needed if `q = 50-p` and `q >= p`; reuse `is_prime`.

Exercise 3 — Choose the right loop

Implement three programs that read a password until correct:

1. `while` — re-prompt only if first read failed validation
2. `do-while` — always prompt at least once
3. `for` — at most 5 attempts

Document which feels most natural and why.

Exercise 4 — FizzBuzz customization

Accept `n`, `a`, `b` from the command line or stdin. Print 1..`n` with Fizz for multiples of `a`, Buzz for `b`, FizzBuzz for both.

Exercise 5 — Server-style loop

Write a `for (;;)` loop that reads lines; commands `inc`, `dec`, `show`, `quit`. Maintain a counter. Reject unknown commands without exiting. Exit only on `quit` or EOF.

Exercise 6 — Running variance (optional stretch)

Extend running average to track sum of squares and print population variance each step: `var = (sumsq / n) - mean²` (careful with floating error; fine for a learning demo).

Summary

1. **while** — pre-test; zero or more iterations
2. **do-while** — post-test; at least once (menus)
3. **for** — counted ranges and clear index updates
4. **Decision guide** — pick the loop that matches the story
5. **break / continue** — innermost loop only
6. **Nested loops** — tables, patterns, pairs
7. **Infinite loops** — servers/superloops with explicit exit conditions
8. **Worked examples** — primes, FizzBuzz, mult table, running average
9. **Pitfalls** — forgotten updates, off-by-one, float counters

Next: advanced flow control (`goto` rarity, multi-level exit patterns, structured alternatives).

Advanced Control Flow

Introduction

Advanced control flow mechanisms in C go beyond basic conditionals and loops to provide more sophisticated ways to manage program execution. This chapter explores the `goto` statement, function calls as control flow mechanisms, error handling patterns, and advanced techniques for structuring complex programs.

While some of these mechanisms are powerful, they must be used judiciously to maintain code readability and maintainability. Understanding when and how to use these advanced features is crucial for writing professional C code.

The goto Statement

The `goto` statement allows unconditional jumps to labeled statements within the same function. While generally discouraged in modern programming, there are specific scenarios where `goto` can improve code clarity and efficiency.

Basic Syntax

```
goto label;
...
label: statement;
```

Simple goto Example

```
#include <stdio.h>

int main() {
    int choice;

    printf("Enter 1 to continue, 0 to exit: ");
    scanf("%d", &choice);

    if (choice == 0) {
        goto end;  // Jump to the 'end' label
```

```
    }

    printf("Continuing with the program...\n");
    // More program logic here

end:
    printf("Program ended.\n");
    return 0;
}
```

goto for Error Handling

One of the most legitimate uses of `goto` is in error handling, particularly when cleaning up resources:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file1 = NULL;
    FILE *file2 = NULL;
    int *array = NULL;

    // Allocate resources
    file1 = fopen("file1.txt", "r");
    if (file1 == NULL) {
        printf("Error opening file1.txt\n");
        goto cleanup;
    }

    file2 = fopen("file2.txt", "w");
    if (file2 == NULL) {
        printf("Error opening file2.txt\n");
        goto cleanup;
    }

    array = (int*)malloc(100 * sizeof(int));
    if (array == NULL) {
        printf("Error allocating memory\n");
        goto cleanup;
    }

    // Main program logic here
    printf("All resources allocated successfully!\n");

    // Normal cleanup (reverse order of allocation)
```

```
    free(array);
    fclose(file2);
    fclose(file1);
    return 0;

cleanup:
    // Error cleanup (free allocated resources)
    if (array != NULL) {
        free(array);
    }
    if (file2 != NULL) {
        fclose(file2);
    }
    if (file1 != NULL) {
        fclose(file1);
    }
    return 1;  // Error exit code
}
```

goto in Switch Statements

`goto` can be used to jump between cases in switch statements:

```
#include <stdio.h>

int main() {
    int state = 1;

    switch (state) {
        case 1:
            printf("State 1\n");
            goto state_3;  // Jump to case 3
        case 2:
            printf("State 2\n");
            break;
        case 3:
        state_3:
            printf("State 3\n");
            break;
        default:
            printf("Unknown state\n");
    }

    return 0;
}
```

Function Calls as Control Flow

Function calls are a fundamental control flow mechanism that transfer execution to another part of the program. They enable code reuse, modularity, and structured programming.

Basic Function Calls

```c
#include <stdio.h>

void print_hello() {
    printf("Hello from function!\n");
}

int add(int a, int b) {
    return a + b;
}

int main() {
    print_hello();  // Function call transfers control

    int result = add(5, 3);  // Function call with return value
    printf("5 + 3 = %d\n", result);

    return 0;
}
```

Recursive Function Calls

Recursive functions call themselves to solve problems that can be broken down into smaller, similar subproblems:

```c
#include <stdio.h>

// Factorial calculation using recursion
int factorial(int n) {
    if (n <= 1) {
        return 1;  // Base case
    }
    return n * factorial(n - 1);  // Recursive call
}

// Fibonacci sequence using recursion
int fibonacci(int n) {
    if (n <= 1) {
        return n;  // Base cases
```

```
    }
    return fibonacci(n - 1) + fibonacci(n - 2);  // Recursive calls
}

int main() {
    int num = 5;

    printf("Factorial of %d: %d\n", num, factorial(num));
    printf("Fibonacci of %d: %d\n", num, fibonacci(num));

    return 0;
}
```

Function Pointers for Dynamic Control Flow

Function pointers allow dynamic selection of which function to call at runtime:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int multiply(int a, int b) {
    return a * b;
}

int divide(int a, int b) {
    if (b != 0) {
        return a / b;
    }
    return 0;
}

int main() {
    int x = 10, y = 5;
    int (*operation)(int, int);  // Function pointer

    // Dynamic function selection
    char op;
    printf("Enter operation (+, -, *, /): ");
    scanf(" %c", &op);
```

```
    switch (op) {
        case '+':
            operation = add;
            break;
        case '-':
            operation = subtract;
            break;
        case '*':
            operation = multiply;
            break;
        case '/':
            operation = divide;
            break;
        default:
            printf("Invalid operation\n");
            return 1;
    }

    int result = operation(x, y);  // Call selected function
    printf("Result: %d\n", result);

    return 0;
}
```

Error Handling Patterns

C doesn't have built-in exception handling, so error handling is typically done through return codes and careful checking.

Return Code Pattern

```
#include <stdio.h>
#include <stdlib.h>

#define SUCCESS 0
#define ERROR_INVALID_INPUT 1
#define ERROR_MEMORY_ALLOCATION 2
#define ERROR_FILE_OPERATION 3

int process_data(int *data, int size) {
    if (data == NULL || size <= 0) {
        return ERROR_INVALID_INPUT;
    }
```

```
    // Process data
    for (int i = 0; i < size; i++) {
        data[i] *= 2;
    }

    return SUCCESS;
}

int main() {
    int data[] = {1, 2, 3, 4, 5};
    int size = sizeof(data) / sizeof(data[0]);

    int result = process_data(data, size);

    switch (result) {
        case SUCCESS:
            printf("Data processed successfully!\n");
            for (int i = 0; i < size; i++) {
                printf("%d ", data[i]);
            }
            printf("\n");
            break;
        case ERROR_INVALID_INPUT:
            printf("Error: Invalid input data\n");
            break;
        case ERROR_MEMORY_ALLOCATION:
            printf("Error: Memory allocation failed\n");
            break;
        case ERROR_FILE_OPERATION:
            printf("Error: File operation failed\n");
            break;
        default:
            printf("Unknown error occurred\n");
    }

    return result;
}
```

errno and perror for System Errors

```
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main() {
```

```
    FILE *file = fopen("nonexistent_file.txt", "r");

    if (file == NULL) {
        printf("Error opening file: %s\n", strerror(errno));
        perror("fopen");  // Alternative way to print error
        return errno;
    }

    fclose(file);
    return 0;
}
```

Setjmp and Longjmp for Non-Local Jumps

The `setjmp` and `longjmp` functions provide a way to perform non-local jumps, similar to exceptions in other languages:

```
#include <stdio.h>
#include <setjmp.h>
#include <stdlib.h>

jmp_buf jump_buffer;

void risky_function() {
    printf("Entering risky function\n");

    // Simulate an error condition
    int error_condition = 1;
    if (error_condition) {
        printf("Error occurred, jumping back!\n");
        longjmp(jump_buffer, 1);  // Jump back to setjmp
    }

    printf("This won't be printed\n");
}

int main() {
    int result = setjmp(jump_buffer);

    if (result == 0) {
        printf("First time through setjmp\n");
        risky_function();
    } else {
        printf("Returned from longjmp with value: %d\n", result);
    }
```

```
    return 0;
}
```

Advanced Loop Control Techniques

Loop Labels and Break to Outer Loops

While C doesn't support loop labels like some languages, you can achieve similar functionality using flags or goto:

```
#include <stdio.h>

int main() {
    int found = 0;

    // Search for a value in a 2D array
    int matrix[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int target = 5;

    for (int i = 0; i < 3 && !found; i++) {
        for (int j = 0; j < 3; j++) {
            if (matrix[i][j] == target) {
                printf("Found %d at position (%d, %d)\n", target, i, j);
                found = 1;
                break;  // Breaks inner loop
            }
        }
        // Outer loop will exit due to !found condition
    }

    return 0;
}
```

State Machines

State machines are a powerful pattern for managing complex control flow:

```
#include <stdio.h>

typedef enum {
    STATE_IDLE,
    STATE_PROCESSING,
    STATE_COMPLETE,
    STATE_ERROR
```

```
} State;

int main() {
    State current_state = STATE_IDLE;
    int input;

    while (current_state != STATE_COMPLETE && current_state != STATE_ERROR) {
        switch (current_state) {
            case STATE_IDLE:
                printf("Enter command (1=start, 2=quit): ");
                scanf("%d", &input);
                if (input == 1) {
                    current_state = STATE_PROCESSING;
                    printf("Processing started...\n");
                } else if (input == 2) {
                    current_state = STATE_COMPLETE;
                }
                break;

            case STATE_PROCESSING:
                printf("Enter data (0=complete, -1=error): ");
                scanf("%d", &input);
                if (input == 0) {
                    current_state = STATE_COMPLETE;
                    printf("Processing completed!\n");
                } else if (input == -1) {
                    current_state = STATE_ERROR;
                    printf("Processing error!\n");
                }
                break;

            case STATE_COMPLETE:
                printf("Process complete.\n");
                break;

            case STATE_ERROR:
                printf("Process error.\n");
                break;
        }
    }

    return 0;
}
```

Best Practices for Advanced Control Flow

1. Use goto Sparingly and Structured

```c
// Good: Structured error handling with goto
int process_files() {
    FILE *file1 = NULL;
    FILE *file2 = NULL;
    int *buffer = NULL;
    int result = -1;  // Default error

    file1 = fopen("input.txt", "r");
    if (!file1) goto cleanup;

    file2 = fopen("output.txt", "w");
    if (!file2) goto cleanup;

    buffer = malloc(1024);
    if (!buffer) goto cleanup;

    // Main processing logic
    result = 0;  // Success

cleanup:
    if (buffer) free(buffer);
    if (file2) fclose(file2);
    if (file1) fclose(file1);
    return result;
}
```

2. Avoid Deep Nesting

```c
// Good: Early returns to reduce nesting
int validate_input(int value) {
    if (value < 0) {
        return -1;  // Invalid
    }

    if (value > 100) {
        return -1;  // Invalid
    }

    if (value % 2 != 0) {
        return -1;  // Invalid
    }
```

```
    return 0;  // Valid
}

// Avoid: Deep nesting
int validate_input_nested(int value) {
    if (value >= 0) {
        if (value <= 100) {
            if (value % 2 == 0) {
                return 0;  // Valid
            } else {
                return -1;  // Invalid
            }
        } else {
            return -1;  // Invalid
        }
    } else {
        return -1;  // Invalid
    }
}
```

3. Use Function Pointers for Strategy Pattern

```
#include <stdio.h>

typedef int (*operation_func)(int, int);

int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }

int calculate(int a, int b, operation_func op) {
    return op(a, b);
}

int main() {
    int x = 10, y = 5;

    printf("Add: %d\n", calculate(x, y, add));
    printf("Subtract: %d\n", calculate(x, y, subtract));
    printf("Multiply: %d\n", calculate(x, y, multiply));

    return 0;
}
```

Common Pitfalls and How to Avoid Them

1. goto Leading to Spaghetti Code

```c
#include <stdio.h>

int main() {
    // Wrong: Complex goto usage leading to unreadable code
    int i = 0;
    goto start;

loop:
    printf("In loop: %d\n", i);
    i++;
    if (i < 5) goto loop;
    goto end;

start:
    printf("Starting...\n");
    goto loop;

end:
    printf("Ending...\n");

    // Correct: Use proper loop constructs
    printf("Starting...\n");
    for (int j = 0; j < 5; j++) {
        printf("In loop: %d\n", j);
    }
    printf("Ending...\n");

    return 0;
}
```

2. Infinite Recursion

```c
#include <stdio.h>

// Wrong: No base case
int factorial_wrong(int n) {
    return n * factorial_wrong(n - 1);  // Infinite recursion
}

// Correct: With proper base case
int factorial_correct(int n) {
```

```
    if (n <= 1) return 1;  // Base case
    return n * factorial_correct(n - 1);
}

int main() {
    printf("Factorial of 5: %d\n", factorial_correct(5));
    return 0;
}
```

3. Longjmp Across Function Boundaries

```
#include <stdio.h>
#include <setjmp.h>

jmp_buf env;

void function_a() {
    printf("In function A\n");
    longjmp(env, 1);  // This is safe within the same function context
}

void function_b() {
    function_a();  // This is also safe
}

int main() {
    if (setjmp(env) == 0) {
        printf("Setting jump point\n");
        function_b();
    } else {
        printf("Jumped back!\n");
    }

    return 0;
}
```

Practical Examples

Resource Management with goto

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *name;
    int *data;
    FILE *file;
} ResourceContainer;

int initialize_resources(ResourceContainer *container) {
    // Allocate name
    container->name = malloc(50);
    if (!container->name) {
        goto cleanup;
    }
    strcpy(container->name, "Resource");

    // Allocate data
    container->data = malloc(100 * sizeof(int));
    if (!container->data) {
        goto cleanup;
    }

    // Open file
    container->file = fopen("data.txt", "w");
    if (!container->file) {
        goto cleanup;
    }

    return 0;  // Success

cleanup:
    // Clean up in reverse order
    if (container->file) {
        fclose(container->file);
    }
    if (container->data) {
        free(container->data);
    }
    if (container->name) {
        free(container->name);
```

```
    }
    return -1;  // Error
}

void cleanup_resources(ResourceContainer *container) {
    if (container->file) {
        fclose(container->file);
    }
    if (container->data) {
        free(container->data);
    }
    if (container->name) {
        free(container->name);
    }
    memset(container, 0, sizeof(ResourceContainer));
}

int main() {
    ResourceContainer container = {0};

    if (initialize_resources(&container) == 0) {
        printf("Resources initialized successfully\n");
        printf("Name: %s\n", container.name);
        // Use resources...
        cleanup_resources(&container);
    } else {
        printf("Failed to initialize resources\n");
        return 1;
    }

    return 0;
}
```

Command-Line Parser with State Machine

```
#include <stdio.h>
#include <string.h>

typedef enum {
    STATE_START,
    STATE_OPTION,
    STATE_ARGUMENT,
    STATE_DONE,
    STATE_ERROR
} ParseState;
```

```
typedef struct {
    int verbose;
    char *output_file;
    int count;
} Options;

int parse_arguments(int argc, char *argv[], Options *options) {
    ParseState state = STATE_START;

    for (int i = 1; i < argc && state != STATE_ERROR; i++) {
        switch (state) {
            case STATE_START:
                if (argv[i][0] == '-') {
                    state = STATE_OPTION;
                    i--;  // Re-process this argument
                } else {
                    state = STATE_DONE;
                }
                break;

            case STATE_OPTION:
                if (strcmp(argv[i], "-v") == 0) {
                    options->verbose = 1;
                } else if (strcmp(argv[i], "-o") == 0) {
                    state = STATE_ARGUMENT;
                } else if (strcmp(argv[i], "-c") == 0) {
                    state = STATE_ARGUMENT;
                } else {
                    printf("Unknown option: %s\n", argv[i]);
                    state = STATE_ERROR;
                }
                break;

            case STATE_ARGUMENT:
                if (strcmp(argv[i-1], "-o") == 0) {
                    options->output_file = argv[i];
                } else if (strcmp(argv[i-1], "-c") == 0) {
                    options->count = atoi(argv[i]);
                }
                state = STATE_START;
                break;

            case STATE_DONE:
            case STATE_ERROR:
                break;
        }
```

```
    }

    return (state == STATE_DONE || state == STATE_START) ? 0 : -1;
}

int main(int argc, char *argv[]) {
    Options options = {0};

    if (parse_arguments(argc, argv, &options) == 0) {
        printf("Parsing successful:\n");
        printf("  Verbose: %s\n", options.verbose ? "yes" : "no");
        printf("  Output file: %s\n", options.output_file ? options.output_file : "not speci
        printf("  Count: %d\n", options.count);
    } else {
        printf("Parsing failed\n");
        return 1;
    }

    return 0;
}
```

Complex Error Handling System

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

typedef enum {
    ERR_SUCCESS = 0,
    ERR_MEMORY,
    ERR_FILE_OPEN,
    ERR_FILE_READ,
    ERR_INVALID_INPUT,
    ERR_UNKNOWN
} ErrorCode;

const char* error_string(ErrorCode code) {
    switch (code) {
        case ERR_SUCCESS: return "Success";
        case ERR_MEMORY: return "Memory allocation failed";
        case ERR_FILE_OPEN: return "File open failed";
        case ERR_FILE_READ: return "File read failed";
        case ERR_INVALID_INPUT: return "Invalid input";
        case ERR_UNKNOWN: return "Unknown error";
```

```
        default: return "Unrecognized error";
    }
}

ErrorCode process_file(const char *filename) {
    FILE *file = NULL;
    char *buffer = NULL;
    ErrorCode result = ERR_SUCCESS;

    // Open file
    file = fopen(filename, "r");
    if (!file) {
        result = ERR_FILE_OPEN;
        goto cleanup;
    }

    // Allocate buffer
    buffer = malloc(1024);
    if (!buffer) {
        result = ERR_MEMORY;
        goto cleanup;
    }

    // Read file
    if (fread(buffer, 1, 1024, file) == 0) {
        if (feof(file)) {
            printf("File is empty\n");
        } else {
            result = ERR_FILE_READ;
            goto cleanup;
        }
    }

    printf("File processed successfully\n");

cleanup:
    if (buffer) free(buffer);
    if (file) fclose(file);
    return result;
}

int main() {
    const char *filename = "test.txt";
    ErrorCode result = process_file(filename);

    if (result == ERR_SUCCESS) {
        printf("Operation completed successfully\n");
```

```
    } else {
        printf("Error: %s\n", error_string(result));
        if (result == ERR_FILE_OPEN) {
            printf("System error: %s\n", strerror(errno));
        }
    }

    return result;
}
```

Summary

In this chapter, you've learned about:

1. **goto Statement**: When and how to use unconditional jumps
2. **Function Calls**: Basic and recursive function calls as control flow
3. **Function Pointers**: Dynamic function selection and callback mechanisms
4. **Error Handling**: Return codes, errno, and advanced error management
5. **setjmp/longjmp**: Non-local jumps for exceptional control flow
6. **Advanced Loop Control**: Techniques for managing complex iterations
7. **State Machines**: Pattern for managing complex program states
8. **Best Practices**: Writing clean, maintainable advanced control flow
9. **Common Pitfalls**: Avoiding typical mistakes with advanced control structures

Advanced control flow mechanisms provide powerful tools for managing complex program execution, but they must be used carefully to maintain code quality. With these techniques, you can implement sophisticated control logic while maintaining readability and reliability.

Module 3: Control Flow

Overview

This module explores the control structures that govern program execution flow in C. You'll learn how to implement decision-making with conditionals, repetition with loops, and advanced control mechanisms. Mastering control flow is essential for creating programs that respond dynamically to inputs and execute complex logic.

Learning Objectives

By the end of this module, you will be able to: - Implement decision-making using if, if-else, and switch statements - Create repetitive code execution using while, do-while, and for loops - Apply loop control statements (break, continue) effectively - Use advanced control flow mechanisms including goto and function calls - Implement error handling patterns and state machines - Write efficient and maintainable control flow code - Avoid common pitfalls in control structure implementation

Chapters

1. **Conditional Statements** - if, if-else, switch, and nested conditionals
2. **Loops** - while, do-while, for loops and nested iterations
3. **Advanced Control Flow** - goto, function pointers, error handling, and state machines

Key Concepts Covered

- Conditional execution with if, if-else, and switch statements
- Loop constructs: while, do-while, and for loops
- Loop control with break and continue statements
- Nested loops and complex iteration patterns
- Infinite loops and loop performance considerations
- The goto statement and when to use it appropriately
- Function calls as control flow mechanisms
- Error handling patterns and return codes
- State machines for complex program logic
- Advanced techniques like setjmp/longjmp
- Best practices for writing clean control flow code

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Basic understanding of operators and expressions
- Familiarity with variable declaration and initialization

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Understanding of basic programming concepts
- Familiarity with debugging tools

Estimated Time to Complete

- Reading: 6-8 hours
- Exercises: 8-12 hours
- Projects: 4-6 hours

Assessment

- Complete all chapter exercises
- Successfully implement various control flow patterns
- Demonstrate understanding of loop optimization techniques
- Pass control flow programming quiz
- Submit a comprehensive program using multiple control structures

Next Module

After completing this module, proceed to **Module 4: Functions and Modular Programming** to learn about code organization, function design, and modular programming techniques.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 3)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Online tutorials on control flow patterns
- Debugging tools documentation (GDB, Valgrind)
- Best practices guides for structured programming

Functions

Function Fundamentals

Introduction

Functions are fundamental building blocks in C programming that allow you to break down complex problems into smaller, manageable pieces. They promote code reusability, modularity, and maintainability by encapsulating specific tasks into named, callable units.

A function in C is a self-contained block of code that performs a specific task and can optionally return a value to the caller. Functions help organize code logically, reduce redundancy, and make programs easier to understand and debug.

This chapter will explore the fundamentals of functions in C, including their definition, declaration, calling mechanisms, and parameter passing.

What is a Function?

A function is a named block of code that performs a specific task and can be called (invoked) from other parts of the program. Functions have several key characteristics:

1. **Name**: A unique identifier used to call the function
2. **Parameters**: Input values passed to the function
3. **Return Type**: The type of value the function returns (or void if none)
4. **Body**: The code that executes when the function is called

Function Definition

A function definition specifies what the function does and how it does it. It includes the function's return type, name, parameters, and body.

Basic Syntax

```
return_type function_name(parameter_list) {
    // Function body
    // Local variable declarations
    // Statements
    // Return statement (if not void)
}
```

Simple Function Example

```c
#include <stdio.h>

// Function definition
int add(int a, int b) {
    int sum = a + b;
    return sum;
}

int main() {
    int result = add(5, 3);  // Function call
    printf("5 + 3 = %d\n", result);
    return 0;
}
```

Function with No Parameters

```c
#include <stdio.h>

void print_greeting() {
    printf("Hello, welcome to C programming!\n");
}

int main() {
    print_greeting();  // Function call with no arguments
    return 0;
}
```

Function with No Return Value (void)

```c
#include <stdio.h>

void print_line() {
    for (int i = 0; i < 20; i++) {
        printf("-");
    }
    printf("\n");
}

int main() {
    print_line();
    printf("Main Content\n");
    print_line();
```

```
    return 0;
}
```

Function Declaration (Prototypes)

A function declaration (also called a prototype) tells the compiler about a function's name, return type, and parameters without providing the implementation. This allows you to call a function before its definition appears in the code.

Syntax

```
return_type function_name(parameter_types);
```

Function Prototype Example

```
#include <stdio.h>

// Function prototypes
int multiply(int x, int y);
void print_result(int value);
double calculate_average(double a, double b);

int main() {
    int product = multiply(4, 5);
    print_result(product);

    double avg = calculate_average(10.5, 20.3);
    printf("Average: %.2f\n", avg);

    return 0;
}

// Function definitions (can appear after main)
int multiply(int x, int y) {
    return x * y;
}

void print_result(int value) {
    printf("Result: %d\n", value);
}

double calculate_average(double a, double b) {
```

```
    return (a + b) / 2.0;
}
```

Function Parameters

Function parameters are variables that receive values when the function is called. C uses "pass by value" for parameter passing, meaning that copies of the actual arguments are passed to the function.

Parameter Passing by Value

```
#include <stdio.h>

void modify_value(int x) {
    x = 100;  // Only modifies the local copy
    printf("Inside function: x = %d\n", x);
}

int main() {
    int num = 5;
    printf("Before function call: num = %d\n", num);
    modify_value(num);
    printf("After function call: num = %d\n", num);  // Unchanged
    return 0;
}
```

Multiple Parameters

```
#include <stdio.h>

int find_maximum(int a, int b, int c) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}

void print_student_info(char name[], int age, float gpa) {
    printf("Student: %s\n", name);
    printf("Age: %d\n", age);
    printf("GPA: %.2f\n", gpa);
}
```

```c
int main() {
    int max_value = find_maximum(10, 25, 15);
    printf("Maximum value: %d\n", max_value);

    print_student_info("John Doe", 20, 3.75);

    return 0;
}
```

Return Statements

The `return` statement terminates function execution and optionally returns a value to the caller. For functions with a return type other than `void`, a return statement with a value is required.

Returning Values

```c
#include <stdio.h>

int square(int n) {
    return n * n;
}

float calculate_area(float radius) {
    const float PI = 3.14159f;
    return PI * radius * radius;
}

char get_grade(int score) {
    if (score >= 90) return 'A';
    else if (score >= 80) return 'B';
    else if (score >= 70) return 'C';
    else if (score >= 60) return 'D';
    else return 'F';
}

int main() {
    printf("Square of 5: %d\n", square(5));
    printf("Area of circle (radius 3): %.2f\n", calculate_area(3.0f));
    printf("Grade for score 85: %c\n", get_grade(85));

    return 0;
}
```

Early Returns

```c
#include <stdio.h>

int absolute_value(int n) {
    if (n < 0) {
        return -n;  // Early return for negative numbers
    }
    return n;  // Return positive numbers as-is
}

int find_factorial(int n) {
    if (n < 0) {
        return -1;  // Error case
    }
    if (n == 0 || n == 1) {
        return 1;  // Base cases
    }

    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    printf("Absolute value of -5: %d\n", absolute_value(-5));
    printf("Absolute value of 5: %d\n", absolute_value(5));

    printf("Factorial of 5: %d\n", find_factorial(5));
    printf("Factorial of -3: %d\n", find_factorial(-3));  // Error case

    return 0;
}
```

Function Scope and Lifetime

Variables declared inside a function have local scope and exist only during the function's execution.

Local Variables

```c
#include <stdio.h>

void function_a() {
    int local_var = 10;  // Local to function_a
    printf("In function_a: local_var = %d\n", local_var);
}

void function_b() {
    int local_var = 20;  // Different variable, local to function_b
    printf("In function_b: local_var = %d\n", local_var);
}

int main() {
    function_a();
    function_b();

    // printf("%d\n", local_var);  // Error: local_var not accessible here

    return 0;
}
```

Static Local Variables

Static local variables retain their values between function calls:

```c
#include <stdio.h>

void counter() {
    static int count = 0;  // Initialized only once
    count++;
    printf("Function called %d time(s)\n", count);
}

int main() {
    counter();  // Output: Function called 1 time(s)
    counter();  // Output: Function called 2 time(s)
    counter();  // Output: Function called 3 time(s)

    return 0;
}
```

Function Naming Conventions

Following consistent naming conventions improves code readability and maintainability:

Common Naming Styles

```c
#include <stdio.h>

// Snake case (recommended for C)
int calculate_area_rectangle(int width, int height) {
    return width * height;
}

// Camel case (also acceptable)
int calculateAreaRectangle(int width, int height) {
    return width * height;
}

// Descriptive names
void print_student_information(char student_name[], int student_age) {
    printf("Name: %s, Age: %d\n", student_name, student_age);
}

int main() {
    int area1 = calculate_area_rectangle(5, 10);
    int area2 = calculateAreaRectangle(3, 7);

    printf("Area 1: %d\n", area1);
    printf("Area 2: %d\n", area2);

    print_student_information("Alice Smith", 22);

    return 0;
}
```

Best Practices for Functions

1. Single Responsibility Principle

```c
#include <stdio.h>

// Good: Each function has a single, clear purpose
int calculate_rectangle_area(int width, int height) {
```

```
    return width * height;
}

int calculate_rectangle_perimeter(int width, int height) {
    return 2 * (width + height);
}

void print_rectangle_info(int width, int height) {
    printf("Rectangle: %d x %d\n", width, height);
    printf("Area: %d\n", calculate_rectangle_area(width, height));
    printf("Perimeter: %d\n", calculate_rectangle_perimeter(width, height));
}

// Avoid: Function doing multiple unrelated things
/*
int calculate_and_print_rectangle(int width, int height) {
    int area = width * height;
    printf("Area: %d\n", area);
    return area;
}
*/

int main() {
    print_rectangle_info(5, 3);
    return 0;
}
```

2. Appropriate Function Size

```
#include <stdio.h>

// Good: Functions are small and focused
int is_even(int number) {
    return number % 2 == 0;
}

int is_positive(int number) {
    return number > 0;
}

int is_valid_input(int number) {
    return is_positive(number) && is_even(number);
}

// Avoid: Functions that are too long
```

```c
/*
int process_complex_logic(int a, int b, int c, int d, int e) {
    // 100+ lines of code doing many different things
}
*/

int main() {
    int test_number = 10;
    if (is_valid_input(test_number)) {
        printf("%d is valid input\n", test_number);
    }

    return 0;
}
```

3. Meaningful Parameter Names

```c
#include <stdio.h>

// Good: Descriptive parameter names
double calculate_triangle_area(double base, double height) {
    return 0.5 * base * height;
}

// Avoid: Generic parameter names
/*
double calculate_triangle_area(double b, double h) {
    return 0.5 * b * h;
}
*/

int main() {
    double area = calculate_triangle_area(10.0, 5.0);
    printf("Triangle area: %.2f\n", area);

    return 0;
}
```

Common Pitfalls and How to Avoid Them

1. Missing Return Statements

```c
#include <stdio.h>

// Wrong: Missing return statement in non-void function
/*
int calculate_square(int n) {
    int result = n * n;
    // Missing return statement
}
*/

// Correct: Always include return statement
int calculate_square(int n) {
    int result = n * n;
    return result;
}

int main() {
    int square = calculate_square(5);
    printf("Square of 5: %d\n", square);

    return 0;
}
```

2. Using Uninitialized Variables

```c
#include <stdio.h>

// Wrong: Using uninitialized local variable
/*
int add_numbers(int a, int b) {
    int sum;  // Not initialized
    sum = sum + a + b;  // Using uninitialized variable
    return sum;
}
*/

// Correct: Initialize variables properly
int add_numbers(int a, int b) {
    int sum = 0;  // Properly initialized
    sum = sum + a + b;
    return sum;
```

```
}

int main() {
    int result = add_numbers(3, 4);
    printf("3 + 4 = %d\n", result);

    return 0;
}
```

3. Function Declaration vs Definition Mismatch

```
#include <stdio.h>

// Function prototype
int multiply(int x, int y);

int main() {
    int result = multiply(5, 3);
    printf("5 * 3 = %d\n", result);
    return 0;
}

// Correct: Definition matches prototype
int multiply(int x, int y) {
    return x * y;
}

// Wrong: Definition doesn't match prototype
/*
int multiply(int a, int b) {  // Different parameter names are OK
    return a * b;
}
// But different types or number of parameters would be wrong
*/
```

Practical Examples

Temperature Converter Functions

```
#include <stdio.h>

// Function prototypes
double celsius_to_fahrenheit(double celsius);
```

```c
double fahrenheit_to_celsius(double fahrenheit);
void print_temperature_conversion(double celsius, double fahrenheit);

// Convert Celsius to Fahrenheit
double celsius_to_fahrenheit(double celsius) {
    return (celsius * 9.0 / 5.0) + 32.0;
}

// Convert Fahrenheit to Celsius
double fahrenheit_to_celsius(double fahrenheit) {
    return (fahrenheit - 32.0) * 5.0 / 9.0;
}

// Print temperature conversion
void print_temperature_conversion(double celsius, double fahrenheit) {
    printf("%.2f°C = %.2f°F\n", celsius, fahrenheit);
    printf("%.2f°F = %.2f°C\n", fahrenheit, celsius_to_fahrenheit(fahrenheit));
}

int main() {
    double c_temp = 25.0;
    double f_temp = 77.0;

    double c_to_f = celsius_to_fahrenheit(c_temp);
    double f_to_c = fahrenheit_to_celsius(f_temp);

    printf("Temperature Conversions:\n");
    print_temperature_conversion(c_temp, f_temp);

    return 0;
}
```

Mathematical Utility Functions

```c
#include <stdio.h>

// Function prototypes
int power(int base, int exponent);
int factorial(int n);
int gcd(int a, int b);
int lcm(int a, int b);

// Calculate power (base^exponent)
int power(int base, int exponent) {
    if (exponent < 0) return 0;  // Handle negative exponents
```

```
    if (exponent == 0) return 1;

    int result = 1;
    for (int i = 0; i < exponent; i++) {
        result *= base;
    }
    return result;
}

// Calculate factorial
int factorial(int n) {
    if (n < 0) return -1;  // Error for negative numbers
    if (n == 0 || n == 1) return 1;

    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

// Calculate Greatest Common Divisor
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Calculate Least Common Multiple
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    printf("Mathematical Utilities:\n");
    printf("2^8 = %d\n", power(2, 8));
    printf("5! = %d\n", factorial(5));
    printf("GCD of 48 and 18 = %d\n", gcd(48, 18));
    printf("LCM of 12 and 18 = %d\n", lcm(12, 18));

    return 0;
}
```

String Processing Functions

```c
#include <stdio.h>
#include <string.h>

// Function prototypes
int string_length(char str[]);
void string_copy(char dest[], char src[]);
int string_compare(char str1[], char str2[]);
void string_reverse(char str[]);

// Calculate string length
int string_length(char str[]) {
    int length = 0;
    while (str[length] != '\0') {
        length++;
    }
    return length;
}

// Copy string
void string_copy(char dest[], char src[]) {
    int i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';  // Null terminate
}

// Compare strings
int string_compare(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] != '\0' && str2[i] != '\0') {
        if (str1[i] != str2[i]) {
            return str1[i] - str2[i];
        }
        i++;
    }
    return str1[i] - str2[i];
}

// Reverse string
void string_reverse(char str[]) {
    int length = string_length(str);
    for (int i = 0; i < length / 2; i++) {
```

```
        char temp = str[i];
        str[i] = str[length - 1 - i];
        str[length - 1 - i] = temp;
    }
}

int main() {
    char str1[100] = "Hello";
    char str2[100] = "World";
    char str3[100];

    printf("String Processing Examples:\n");
    printf("Length of '%s': %d\n", str1, string_length(str1));

    string_copy(str3, str1);
    printf("Copied string: %s\n", str3);

    printf("Comparison '%s' vs '%s': %d\n", str1, str2, string_compare(str1, str2));

    printf("Original: %s\n", str1);
    string_reverse(str1);
    printf("Reversed: %s\n", str1);

    return 0;
}
```

Summary

In this chapter, you've learned about:

1. **Function Basics**: What functions are and why they're important
2. **Function Definition**: How to define functions with proper syntax
3. **Function Declaration**: Using prototypes to declare functions before use
4. **Parameter Passing**: How C passes parameters by value
5. **Return Statements**: Using return to send values back to callers
6. **Scope and Lifetime**: Understanding variable scope in functions
7. **Best Practices**: Writing clean, maintainable functions
8. **Common Pitfalls**: Avoiding typical function-related errors

Functions are essential for organizing code into logical, reusable components. In the next chapter, we'll explore advanced function concepts including recursion, variable arguments, and function pointers.

Advanced Functions

Introduction

Building upon the fundamental concepts of functions, this chapter explores advanced function features in C that provide greater flexibility and power. These advanced features include recursive functions, variable argument functions, inline functions, and function pointers. Understanding these concepts is crucial for writing sophisticated C programs and leveraging the full capabilities of the language.

Advanced functions enable you to implement complex algorithms, create flexible interfaces, and build more efficient code. This chapter will dive deep into each of these advanced topics with practical examples and best practices.

Recursive Functions

Recursion is a powerful programming technique where a function calls itself to solve a problem. Recursive functions are particularly useful for problems that can be broken down into smaller, similar subproblems.

Basic Recursion Concepts

A recursive function must have: 1. **Base Case**: A condition that stops the recursion 2. **Recursive Case**: The function calling itself with modified parameters

Simple Recursion Example

```c
#include <stdio.h>

// Calculate factorial using recursion
int factorial(int n) {
    // Base case
    if (n <= 1) {
        return 1;
    }

    // Recursive case
    return n * factorial(n - 1);
```

```
}

int main() {
    for (int i = 0; i <= 6; i++) {
        printf("%d! = %d\n", i, factorial(i));
    }

    return 0;
}
```

Fibonacci Sequence

```
#include <stdio.h>

// Calculate Fibonacci number using recursion
int fibonacci(int n) {
    // Base cases
    if (n <= 1) {
        return n;
    }

    // Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    printf("Fibonacci sequence (first 10 numbers):\n");
    for (int i = 0; i < 10; i++) {
        printf("F(%d) = %d\n", i, fibonacci(i));
    }

    return 0;
}
```

Recursive Array Processing

```
#include <stdio.h>

// Calculate sum of array elements using recursion
int array_sum(int arr[], int size) {
    // Base case
    if (size <= 0) {
        return 0;
    }
```

```c
    // Recursive case
    return arr[size - 1] + array_sum(arr, size - 1);
}

// Find maximum element in array using recursion
int array_max(int arr[], int size) {
    // Base case
    if (size == 1) {
        return arr[0];
    }

    // Recursive case
    int max_of_rest = array_max(arr + 1, size - 1);
    return (arr[0] > max_of_rest) ? arr[0] : max_of_rest;
}

int main() {
    int numbers[] = {3, 7, 2, 9, 1, 8, 4};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    printf("Sum: %d\n", array_sum(numbers, size));
    printf("Maximum: %d\n", array_max(numbers, size));

    return 0;
}
```

Variable Argument Functions

C provides the `<stdarg.h>` header for creating functions that can accept a variable number of arguments. This is useful for functions like `printf()` that can handle different numbers of parameters.

Basic Variable Argument Functions

```c
#include <stdio.h>
#include <stdarg.h>
```

```c
// Function that calculates the sum of variable arguments
int sum(int count, ...) {
    va_list args;
    va_start(args, count);

    int total = 0;
    for (int i = 0; i < count; i++) {
        int value = va_arg(args, int);
        total += value;
    }

    va_end(args);
    return total;
}

// Function that finds maximum among variable arguments
int maximum(int count, ...) {
    if (count <= 0) return 0;

    va_list args;
    va_start(args, count);

    int max = va_arg(args, int);
    for (int i = 1; i < count; i++) {
        int value = va_arg(args, int);
        if (value > max) {
            max = value;
        }
    }

    va_end(args);
    return max;
}

int main() {
    printf("Sum of 3 numbers: %d\n", sum(3, 10, 20, 30));
    printf("Sum of 5 numbers: %d\n", sum(5, 1, 2, 3, 4, 5));

    printf("Maximum of 4 numbers: %d\n", maximum(4, 15, 8, 23, 42));
    printf("Maximum of 3 numbers: %d\n", maximum(3, 100, 50, 75));

    return 0;
}
```

Formatted Print Function

```c
#include <stdio.h>
#include <stdarg.h>

// Custom printf-like function
void my_printf(const char *format, ...) {
    va_list args;
    va_start(args, format);

    while (*format != '\0') {
        if (*format == '%' && *(format + 1) != '\0') {
            format++;
            switch (*format) {
                case 'd':
                    printf("%d", va_arg(args, int));
                    break;
                case 'f':
                    printf("%f", va_arg(args, double));
                    break;
                case 'c':
                    printf("%c", va_arg(args, int));  // char is promoted to int
                    break;
                case 's':
                    printf("%s", va_arg(args, char*));
                    break;
                default:
                    printf("%%");
                    printf("%c", *format);
                    break;
            }
        } else {
            printf("%c", *format);
        }
        format++;
    }

    va_end(args);
}

int main() {
    my_printf("Hello, %s! You are %d years old.\n", "World", 25);
    my_printf("Pi is approximately %f\n", 3.14159);
    my_printf("Character: %c\n", 'A');

    return 0;
}
```

Inline Functions (C99)

Inline functions are a C99 feature that suggests to the compiler that the function code should be inserted directly at the call site, potentially reducing function call overhead.

Basic Inline Functions

```c
#include <stdio.h>

// Inline function suggestion (C99)
inline int square(int x) {
    return x * x;
}

inline int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int result1 = square(5);
    int result2 = max(10, 20);

    printf("Square of 5: %d\n", result1);
    printf("Maximum of 10 and 20: %d\n", result2);

    return 0;
}
```

Performance Considerations

```c
#include <stdio.h>
#include <time.h>

// Regular function
int regular_square(int x) {
    return x * x;
}

// Inline function
inline int inline_square(int x) {
    return x * x;
}

int main() {
    const int iterations = 100000000;
```

```
    clock_t start, end;

    // Test regular function
    start = clock();
    volatile int sum1 = 0;
    for (int i = 0; i < iterations; i++) {
        sum1 += regular_square(i % 1000);
    }
    end = clock();
    double regular_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Test inline function
    start = clock();
    volatile int sum2 = 0;
    for (int i = 0; i < iterations; i++) {
        sum2 += inline_square(i % 1000);
    }
    end = clock();
    double inline_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Regular function time: %.4f seconds\n", regular_time);
    printf("Inline function time: %.4f seconds\n", inline_time);
    printf("Results: %d, %d\n", sum1, sum2);

    return 0;
}
```

Function Pointers

Function pointers are variables that store the address of a function. They enable dynamic function selection, callbacks, and more flexible program design.

Basic Function Pointers

```
#include <stdio.h>

// Function definitions
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}
```

```c
int multiply(int a, int b) {
    return a * b;
}

int divide(int a, int b) {
    if (b != 0) {
        return a / b;
    }
    return 0;
}

int main() {
    // Function pointer declaration
    int (*operation)(int, int);

    // Assign function addresses to pointer
    operation = add;
    printf("5 + 3 = %d\n", operation(5, 3));

    operation = subtract;
    printf("5 - 3 = %d\n", operation(5, 3));

    operation = multiply;
    printf("5 * 3 = %d\n", operation(5, 3));

    operation = divide;
    printf("6 / 3 = %d\n", operation(6, 3));

    return 0;
}
```

Function Pointer Arrays

```c
#include <stdio.h>

// Mathematical operations
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

int main() {
    // Array of function pointers
    int (*operations[])(int, int) = {add, subtract, multiply, divide};
    char *operation_names[] = {"Add", "Subtract", "Multiply", "Divide"};
```

```
    int x = 10, y = 5;

    printf("Operations on %d and %d:\n", x, y);
    for (int i = 0; i < 4; i++) {
        int result = operations[i](x, y);
        printf("%s: %d\n", operation_names[i], result);
    }

    return 0;
}
```

Callback Functions

```
#include <stdio.h>

// Function that takes a function pointer as parameter
void process_array(int arr[], int size, void (*callback)(int)) {
    for (int i = 0; i < size; i++) {
        callback(arr[i]);
    }
}

// Callback functions
void print_square(int n) {
    printf("%d^2 = %d\n", n, n * n);
}

void print_double(int n) {
    printf("%d * 2 = %d\n", n, n * 2);
}

void print_negative(int n) {
    printf("-%d = %d\n", n, -n);
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Squares:\n");
    process_array(numbers, size, print_square);

    printf("\nDoubles:\n");
    process_array(numbers, size, print_double);
```

```
    printf("\nNegatives:\n");
    process_array(numbers, size, print_negative);

    return 0;
}
```

Higher-Order Functions

Higher-order functions are functions that take other functions as parameters or return functions as results. While C doesn't support returning functions directly, it can work with function pointers to achieve similar functionality.

Function Factory Pattern

```
#include <stdio.h>
#include <stdlib.h>

// Function pointer type for mathematical operations
typedef int (*math_operation)(int, int);

// Function that returns a function pointer
math_operation get_operation(char op) {
    switch (op) {
        case '+': return (math_operation)add;
        case '-': return (math_operation)subtract;
        case '*': return (math_operation)multiply;
        case '/': return (math_operation)divide;
        default: return NULL;
    }
}

// Mathematical operations (defined earlier)
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

int main() {
    char operators[] = {'+', '-', '*', '/'};
    int x = 15, y = 3;

    for (int i = 0; i < 4; i++) {
        math_operation op = get_operation(operators[i]);
        if (op != NULL) {
```

```
            int result = op(x, y);
            printf("%d %c %d = %d\n", x, operators[i], y, result);
        }
    }

    return 0;
}
```

Best Practices for Advanced Functions

1. Recursion Best Practices

```
#include <stdio.h>

// Good: Tail recursion optimization possible
int factorial_tail(int n, int accumulator) {
    if (n <= 1) {
        return accumulator;
    }
    return factorial_tail(n - 1, n * accumulator);
}

int factorial(int n) {
    return factorial_tail(n, 1);
}

// Avoid: Deep recursion that may cause stack overflow
/*
int fibonacci_slow(int n) {
    if (n <= 1) return n;
    return fibonacci_slow(n - 1) + fibonacci_slow(n - 2);  // Exponential time!
}
*/

// Better: Iterative approach or memoization
int fibonacci_fast(int n) {
    if (n <= 1) return n;

    int a = 0, b = 1, result;
    for (int i = 2; i <= n; i++) {
        result = a + b;
        a = b;
        b = result;
    }
    return result;
```

```
}

int main() {
    printf("Factorial of 5: %d\n", factorial(5));
    printf("Fibonacci of 10: %d\n", fibonacci_fast(10));

    return 0;
}
```

2. Variable Argument Functions Best Practices

```
#include <stdio.h>
#include <stdarg.h>

// Good: Clear count parameter
int sum_with_count(int count, ...) {
    if (count <= 0) return 0;

    va_list args;
    va_start(args, count);

    int total = 0;
    for (int i = 0; i < count; i++) {
        total += va_arg(args, int);
    }

    va_end(args);
    return total;
}

// Better: Sentinel-based approach (like printf)
int sum_with_sentinel(int first, ...) {
    if (first == 0) return 0;

    va_list args;
    va_start(args, first);

    int total = first;
    int value;
    while ((value = va_arg(args, int)) != 0) {
        total += value;
    }

    va_end(args);
    return total;
```

```
}

int main() {
    printf("Sum with count: %d\n", sum_with_count(4, 1, 2, 3, 4));
    printf("Sum with sentinel: %d\n", sum_with_sentinel(1, 2, 3, 4, 0));

    return 0;
}
```

3. Function Pointer Best Practices

```
#include <stdio.h>

// Good: Type-safe function pointers
typedef int (*binary_operation)(int, int);

int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }

// Good: Function pointer validation
int safe_calculate(binary_operation op, int x, int y) {
    if (op == NULL) {
        printf("Error: Invalid operation\n");
        return 0;
    }
    return op(x, y);
}

int main() {
    binary_operation operations[] = {add, subtract, NULL};
    int x = 10, y = 5;

    for (int i = 0; operations[i] != NULL; i++) {
        int result = safe_calculate(operations[i], x, y);
        printf("Result: %d\n", result);
    }

    // Test with NULL pointer
    int result = safe_calculate(NULL, x, y);
    printf("NULL operation result: %d\n", result);

    return 0;
}
```

Common Pitfalls and How to Avoid Them

1. Infinite Recursion

```c
#include <stdio.h>

// Wrong: Missing or incorrect base case
/*
int factorial_wrong(int n) {
    return n * factorial_wrong(n - 1);  // Infinite recursion!
}
*/

// Correct: Proper base case
int factorial_correct(int n) {
    if (n < 0) return -1;  // Handle negative input
    if (n <= 1) return 1;  // Base case
    return n * factorial_correct(n - 1);
}

int main() {
    printf("Factorial of 5: %d\n", factorial_correct(5));
    printf("Factorial of -3: %d\n", factorial_correct(-3));

    return 0;
}
```

2. Variable Argument Function Errors

```c
#include <stdio.h>
#include <stdarg.h>

// Wrong: Incorrect argument type retrieval
/*
void print_numbers_wrong(int count, ...) {
    va_list args;
    va_start(args, count);

    for (int i = 0; i < count; i++) {
        double value = va_arg(args, double);  // Wrong if ints were passed
        printf("%.2f ", value);
    }

    va_end(args);
    printf("\n");
```

```
}
*/

// Correct: Match argument types
void print_numbers_correct(int count, ...) {
    va_list args;
    va_start(args, count);

    for (int i = 0; i < count; i++) {
        int value = va_arg(args, int);  // Correct type
        printf("%d ", value);
    }

    va_end(args);
    printf("\n");
}

int main() {
    print_numbers_correct(4, 10, 20, 30, 40);

    return 0;
}
```

3. Function Pointer Errors

```
#include <stdio.h>

int add(int a, int b) { return a + b; }

int main() {
    // Wrong: Incorrect function pointer type
    /*
    int (*wrong_ptr)(int) = add;  // Mismatch: add takes 2 params
    */

    // Correct: Matching function pointer type
    int (*correct_ptr)(int, int) = add;

    int result = correct_ptr(5, 3);
    printf("5 + 3 = %d\n", result);

    return 0;
}
```

Practical Examples

Recursive Directory Traversal

```c
#include <stdio.h>
#include <string.h>

// Simulate directory structure
typedef struct {
    char name[50];
    int is_directory;
    struct {
        char name[50];
        int is_directory;
    } contents[10];
    int content_count;
} Directory;

// Recursive function to print directory structure
void print_directory(Directory *dir, int depth) {
    // Print indentation
    for (int i = 0; i < depth; i++) {
        printf("  ");
    }

    // Print directory name
    printf("%s%s\n", dir->name, dir->is_directory ? "/" : "");

    // Recursively print contents
    if (dir->is_directory) {
        for (int i = 0; i < dir->content_count; i++) {
            Directory sub_dir;
            strcpy(sub_dir.name, dir->contents[i].name);
            sub_dir.is_directory = dir->contents[i].is_directory;
            // In real implementation, would load actual directory data
            print_directory(&sub_dir, depth + 1);
        }
    }
}

int main() {
    Directory root = {"root", 1, {{"documents", 1}, {"file1.txt", 0}}, 2};
    print_directory(&root, 0);

    return 0;
}
```

Mathematical Expression Evaluator

```c
#include <stdio.h>
#include <stdarg.h>
#include <string.h>

// Function pointer type for operations
typedef double (*operation_func)(double, double);

// Mathematical operations
double add(double a, double b) { return a + b; }
double subtract(double a, double b) { return a - b; }
double multiply(double a, double b) { return a * b; }
double divide(double a, double b) { return b != 0 ? a / b : 0; }

// Calculator with variable arguments
double calculate(const char *operations, double first, ...) {
    va_list args;
    va_start(args, first);

    double result = first;
    int op_index = 0;

    while (operations[op_index] != '\0') {
        double operand = va_arg(args, double);
        operation_func op;

        switch (operations[op_index]) {
            case '+': op = add; break;
            case '-': op = subtract; break;
            case '*': op = multiply; break;
            case '/': op = divide; break;
            default: op = NULL; break;
        }

        if (op != NULL) {
            result = op(result, operand);
        }

        op_index++;
    }

    va_end(args);
    return result;
}

int main() {
```

```
    // Calculate: 10 + 5 * 2 - 3 / 2
    double result = calculate("+-*/", 10.0, 5.0, 2.0, 3.0, 2.0);
    printf("Result: %.2f\n", result);

    return 0;
}
```

Event Handler System

```
#include <stdio.h>
#include <string.h>

#define MAX_HANDLERS 10

// Event handler function pointer type
typedef void (*event_handler)(const char *event_data);

// Event handler registry
typedef struct {
    char event_name[50];
    event_handler handlers[MAX_HANDLERS];
    int handler_count;
} EventHandlerRegistry;

EventHandlerRegistry registry = {0};

// Register event handler
void register_handler(const char *event_name, event_handler handler) {
    if (registry.handler_count < MAX_HANDLERS) {
        registry.handlers[registry.handler_count] = handler;
        registry.handler_count++;
        strcpy(registry.event_name, event_name);
        printf("Handler registered for event: %s\n", event_name);
    }
}

// Trigger event
void trigger_event(const char *event_name, const char *event_data) {
    printf("Triggering event: %s\n", event_name);
    for (int i = 0; i < registry.handler_count; i++) {
        if (registry.handlers[i] != NULL) {
            registry.handlers[i](event_data);
        }
    }
}
```

```c
// Sample event handlers
void on_user_login(const char *user_data) {
    printf("User logged in: %s\n", user_data);
}

void on_user_logout(const char *user_data) {
    printf("User logged out: %s\n", user_data);
}

void on_data_update(const char *data) {
    printf("Data updated: %s\n", data);
}

int main() {
    // Register handlers
    register_handler("user_login", on_user_login);
    register_handler("user_logout", on_user_logout);
    register_handler("data_update", on_data_update);

    // Trigger events
    trigger_event("user_login", "Alice");
    trigger_event("data_update", "Profile information");
    trigger_event("user_logout", "Alice");

    return 0;
}
```

Summary

In this chapter, you've learned about:

1. **Recursive Functions**: Functions that call themselves to solve problems
2. **Variable Argument Functions**: Functions that accept a variable number of parameters
3. **Inline Functions**: C99 feature for potential performance optimization
4. **Function Pointers**: Variables that store function addresses
5. **Higher-Order Functions**: Functions that work with other functions
6. **Callback Functions**: Functions passed as parameters to other functions
7. **Best Practices**: Writing safe and efficient advanced functions
8. **Common Pitfalls**: Avoiding typical errors with advanced functions

Advanced functions provide powerful mechanisms for creating flexible, efficient, and maintainable C programs. These concepts are essential for implementing complex algorithms, building extensible systems, and leveraging the full power of the C language. In the next chapter, we'll explore modular programming techniques that build upon these advanced function concepts.

Modular Programming

Introduction

Modular programming organizes code into separate, independent modules that combine into a complete program. In C, you achieve this with header files, source files, and careful linkage. The payoff is reusability, maintainability, and clear separation of concerns.

This chapter walks through a full multi-module project, a production-style Makefile, `static` vs `extern`, static and shared libraries, header design, opaque types, and include hygiene — all with code that compiles under:

```
gcc -std=c17 -Wall -Wextra
```

Header Files and Source Files

Header Files (`.h`)

Headers declare the **interface** the rest of the program can see:

- Function prototypes
- Type definitions (`struct`, `union`, `enum`, `typedef`)
- Macro definitions
- Global variable **declarations** (`extern`)
- Includes of other headers the interface needs

Source Files (`.c`)

Source files provide the **implementation**:

- Function definitions
- Global variable **definitions**
- File-scope helpers marked `static`
- Algorithms and data

Example Layout

```
project/
  include/
    strstats.h
  src/
    strstats.c
    main.c
  Makefile
```

Include Guards

Include guards prevent a header from being processed more than once in a translation unit (which would cause redefinition errors).

Traditional Include Guards

```c
#ifndef STRSTATS_H
#define STRSTATS_H

// Header content goes here

#endif /* STRSTATS_H */
```

`#pragma once` (Non-standard, Widely Supported)

```c
#pragma once

// Header content goes here
```

Prefer traditional guards for maximum portability; many codebases use both.

Compilation Units and Linking

Each `.c` file is a **translation unit**. The compiler turns it into an object file (`.o`). The linker then combines object files, resolves symbols, and produces the executable (or library).

```
# Compile each unit separately
gcc -std=c17 -Wall -Wextra -c strstats.c -o strstats.o
gcc -std=c17 -Wall -Wextra -c main.c -o main.o
```

```
# Link
gcc strstats.o main.o -o program
```

Pipeline summary:

1. **Compile** — syntax check, generate machine code, emit undefined external references
2. **Link** — match calls to definitions, bind addresses, pull in libraries
3. **Load / run** — OS maps the binary and starts `main`

Linkage: `extern`, `static`, and No Linkage

External Linkage (Default for File-Scope Functions)

Symbols with external linkage are visible across translation units. Function names at file scope have external linkage by default. Global variables need an `extern` declaration in a header and exactly one definition in one `.c` file.

```
/* config.h */
#ifndef CONFIG_H
#define CONFIG_H

extern int g_verbose;          /* declaration - many TUs may include this */
int parse_args(int argc, char **argv);

#endif
```

```
/* config.c */
#include "config.h"

int g_verbose = 0;             /* definition - exactly once in the program */

int parse_args(int argc, char **argv) {
    for (int i = 1; i < argc; i++) {
        if (argv[i][0] == '-' && argv[i][1] == 'v') {
            g_verbose = 1;
        }
    }
    return 0;
}
```

```
/* main.c */
#include <stdio.h>
#include "config.h"

int main(int argc, char **argv) {
```

```
    parse_args(argc, argv);
    if (g_verbose) {
        printf("verbose mode on\n");
    }
    return 0;
}
```

Build:

```
gcc -std=c17 -Wall -Wextra -c config.c -o config.o
gcc -std=c17 -Wall -Wextra -c main.c -o main.o
gcc config.o main.o -o config_demo
./config_demo -v
```

Internal Linkage (`static` at File Scope)

`static` on a file-scope function or variable restricts it to that translation unit. Other `.c` files cannot call or see it — even if they somehow declare the same name.

```
/* helpers.c */
#include <stdio.h>

/* Only visible inside helpers.c */
static int clamp(int x, int lo, int hi) {
    if (x < lo) return lo;
    if (x > hi) return hi;
    return x;
}

static int call_count = 0;   /* private counter */

int public_scale(int x) {
    call_count++;
    return clamp(x, 0, 100);
}

void public_report(void) {
    printf("public_scale called %d times\n", call_count);
}
```

```
/* main.c - cannot call clamp(); link would fail if it tried */
#include <stdio.h>

int public_scale(int x);
void public_report(void);
```

```c
int main(void) {
    printf("%d\n", public_scale(150));  /* 100 */
    printf("%d\n", public_scale(-5));   /* 0 */
    public_report();
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra helpers.c main.c -o static_demo
./static_demo
```

`static` vs `extern` at a Glance

Keyword / form	Scope	Linkage	Typical use
`int foo(void);` in header	program	external	public API
`static int helper(void)` in `.c`	one `.c` file	internal	private helper
`extern int g;` in header	program	external	shared global (use sparingly)
`static int g;` in `.c`	one `.c` file	internal	module-private state
local `int x;` in a function	block	none	ordinary local

No Linkage

Parameters and locals have **no linkage**. Each function invocation gets its own stack storage; names are not visible outside the block.

Complete Multi-Module Project: `string_stats`

A small library that analyzes C strings: length, word count, letter/digit counts, and case-folded copy. Public API lives in a header; implementation and private helpers stay in the `.c` file.

Project Tree

```
string_stats/
  strstats.h
  strstats.c
  main.c
  Makefile
```

`strstats.h` — Public Interface

```c
#ifndef STRSTATS_H
#define STRSTATS_H

#include <stddef.h>  /* size_t */

typedef struct {
    size_t length;       /* characters, excluding '\0' */
    size_t words;        /* whitespace-separated tokens */
    size_t letters;
    size_t digits;
    size_t spaces;
    size_t others;
} StrStats;

/* Analyze src; returns 0 on success, -1 if src is NULL. */
int strstats_analyze(const char *src, StrStats *out);

/* Print a human-readable report to stdout. */
void strstats_print(const StrStats *s);

/*
 * Copy src into dest, converting A-Z to a-z.
 * dest_size is the full buffer size (including room for '\0').
 * Returns 0 on success, -1 on NULL args or if dest is too small.
 */
int strstats_to_lower(char *dest, size_t dest_size, const char *src);

#endif /* STRSTATS_H */
```

`strstats.c` — Implementation

```c
#include "strstats.h"

#include <ctype.h>
#include <stdio.h>
#include <string.h>

/* File-private: not declared in the header. */
static int is_word_char(unsigned char c) {
    return !isspace(c);
}

int strstats_analyze(const char *src, StrStats *out) {
```

```
    if (src == NULL || out == NULL) {
        return -1;
    }

    memset(out, 0, sizeof(*out));

    int in_word = 0;
    for (size_t i = 0; src[i] != '\0'; i++) {
        unsigned char c = (unsigned char)src[i];
        out->length++;

        if (isalpha(c)) {
            out->letters++;
        } else if (isdigit(c)) {
            out->digits++;
        } else if (isspace(c)) {
            out->spaces++;
        } else {
            out->others++;
        }

        if (is_word_char(c)) {
            if (!in_word) {
                out->words++;
                in_word = 1;
            }
        } else {
            in_word = 0;
        }
    }
    return 0;
}

void strstats_print(const StrStats *s) {
    if (s == NULL) {
        return;
    }
    printf("length  : %zu\n", s->length);
    printf("words   : %zu\n", s->words);
    printf("letters : %zu\n", s->letters);
    printf("digits  : %zu\n", s->digits);
    printf("spaces  : %zu\n", s->spaces);
    printf("others  : %zu\n", s->others);
}

int strstats_to_lower(char *dest, size_t dest_size, const char *src) {
    if (dest == NULL || src == NULL || dest_size == 0) {
```

```
        return -1;
    }

    size_t need = strlen(src) + 1; /* include '\0' */
    if (need > dest_size) {
        return -1;
    }

    for (size_t i = 0; src[i] != '\0'; i++) {
        dest[i] = (char)tolower((unsigned char)src[i]);
    }
    dest[need - 1] = '\0';
    return 0;
}
```

`main.c` — Driver

```
#include <stdio.h>

#include "strstats.h"

int main(void) {
    const char *sample = "Hello, C17 World! 42 cats.";
    StrStats stats;

    if (strstats_analyze(sample, &stats) != 0) {
        fprintf(stderr, "analyze failed\n");
        return 1;
    }

    printf("Input: \"%s\"\n\n", sample);
    strstats_print(&stats);

    char lower[128];
    if (strstats_to_lower(lower, sizeof lower, sample) != 0) {
        fprintf(stderr, "to_lower failed\n");
        return 1;
    }
    printf("\nlower: \"%s\"\n", lower);
    return 0;
}
```

Manual Build

```
gcc -std=c17 -Wall -Wextra -c strstats.c -o strstats.o
gcc -std=c17 -Wall -Wextra -c main.c -o main.o
gcc strstats.o main.o -o string_stats
./string_stats
```

Expected output (abbreviated):

```
Input: "Hello, C17 World! 42 cats."

length  : 26
words   : 5
letters : 15
digits  : 2
...
lower: "hello, c17 world! 42 cats."
```

Full Makefile: Debug, Release, and -MMD Dependencies

A Makefile that scales beyond one file: separate build directories, automatic header dependency tracking, and clean targets.

```
# Makefile for string_stats
CC      := gcc
STD     := -std=c17
WARN    := -Wall -Wextra -Wpedantic
SRCS    := strstats.c main.c
OBJS    := $(SRCS:.c=.o)
DEPS    := $(OBJS:.o=.d)

TARGET  := string_stats

# Default = debug
BUILD   ?= debug

ifeq ($(BUILD),release)
  CFLAGS  := $(STD) $(WARN) -O2 -DNDEBUG
else
  CFLAGS  := $(STD) $(WARN) -g -O0
endif

# -MMD: write .d dependency files; -MP: phony headers so deletes don't break make
DEPFLAGS := -MMD -MP
```

```
.PHONY: all debug release clean run

all: $(TARGET)

debug:
    $(MAKE) BUILD=debug

release:
    $(MAKE) BUILD=release

$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $@

%.o: %.c
    $(CC) $(CFLAGS) $(DEPFLAGS) -c $< -o $@

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(OBJS) $(DEPS) $(TARGET)

# Pull in generated dependency fragments (if present)
-include $(DEPS)
```

Usage:

```
make              # debug build
make release      # optimized
make run
make clean
```

Why `-MMD -MP` matters: if you edit `strstats.h`, Make recompiles every `.c` that includes it — without you listing headers by hand.

Variant with `build/` Directory

```
CC      := gcc
CFLAGS  := -std=c17 -Wall -Wextra -g -O0 -MMD -MP
SRCS    := strstats.c main.c
BUILD   := build
OBJS    := $(addprefix $(BUILD)/,$(SRCS:.c=.o))
DEPS    := $(OBJS:.o=.d)
TARGET  := $(BUILD)/string_stats

.PHONY: all clean
```

```
all: $(TARGET)

$(TARGET): $(OBJS) | $(BUILD)
    $(CC) $(OBJS) -o $@

$(BUILD)/%.o: %.c | $(BUILD)
    $(CC) $(CFLAGS) -c $< -o $@

$(BUILD):
    mkdir -p $(BUILD)

clean:
    rm -rf $(BUILD)

-include $(DEPS)
```

Creating Libraries

Static Library (`.a`)

Object code is copied into the final executable at link time. No runtime dependency on the `.a` file.

```
# Compile objects
gcc -std=c17 -Wall -Wextra -c strstats.c -o strstats.o

# Archive into libstrstats.a
#   r = replace/insert, c = create if needed, s = write symbol index
ar rcs libstrstats.a strstats.o

# Link: -L. looks in current dir; -lstrstats finds libstrstats.a
gcc -std=c17 -Wall -Wextra main.c -L. -lstrstats -o string_stats_static

./string_stats_static
```

Inspect the archive:

```
ar t libstrstats.a          # list members
nm libstrstats.a | head     # symbols
```

Shared Library (`.so` on Linux / macOS)

Shared libraries are loaded at runtime and can be shared by multiple processes.

```
# Position-independent code is required for shared objects
gcc -std=c17 -Wall -Wextra -fPIC -c strstats.c -o strstats.o

# Build the shared library
gcc -shared -o libstrstats.so strstats.o

# Link the program against it
gcc -std=c17 -Wall -Wextra main.c -L. -lstrstats -o string_stats_shared

# Runtime: loader must find libstrstats.so
# Linux:
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
# macOS:
export DYLD_LIBRARY_PATH=.:$DYLD_LIBRARY_PATH

./string_stats_shared
```

Check linkage:

```
# Linux
ldd ./string_stats_shared
# macOS
otool -L ./string_stats_shared
```

Static vs Shared

	Static (`.a`)	Shared (`.so`)
Linked when	Build time	Load / run time
Binary size	Larger (code copied in)	Smaller
Updates	Rebuild app to pick up lib fix	Replace `.so`, restart app
Deploy	Single executable (simpler)	Ship binary + libraries

Header Design Rules

1. **Headers declare; sources define.** Do not put non-`inline` function bodies or non-`static` data definitions in headers (except `static inline` helpers when appropriate).
2. **Self-contained.** A header should compile after `#include "foo.h"` alone — include what *it* needs (`stddef.h` for `size_t`, etc.).
3. **Include what you use** in `.c` files; do not rely on transitive includes from other headers.
4. **Minimize public surface.** Prefer incomplete types / opaque pointers for private structs.
5. **One module, one header** as a default; split only when the API is large.
6. **Document contracts** in comments: preconditions, ownership, error returns.

7. **Avoid `using` the preprocessor for "namespaces"** beyond a clear prefix (`strstats_`, `vec_`).
8. **Never `#include` a `.c` file** in normal modular code.

Recommended Include Order

A common, readable order inside a `.c` file:

```c
/* 1. Related header first - proves the header is self-contained */
#include "strstats.h"

/* 2. Other project headers */
#include "config.h"

/* 3. Standard library headers (alphabetical is fine) */
#include <ctype.h>
#include <stdio.h>
#include <string.h>

/* 4. System / third-party headers */
/* #include <unistd.h> */
```

Why related header first: if `strstats.h` forgets a needed include, the compiler fails when compiling `strstats.c` — not later, deep in some other unit.

Forward Declarations

When you only need a pointer or reference to a type, forward-declare instead of including a heavy header:

```c
/* log.h - does not need the full definition of StrStats */
#ifndef LOG_H
#define LOG_H

struct StrStats;   /* forward declaration (incomplete type) */

void log_stats(const struct StrStats *s);

#endif
```

```c
/* log.c - full type only needed where members are accessed */
#include "log.h"
#include "strstats.h"
#include <stdio.h>
```

```c
void log_stats(const struct StrStats *s) {
    if (s == NULL) {
        return;
    }
    printf("[log] length=%zu words=%zu\n", s->length, s->words);
}
```

Forward declarations break circular header dependencies and speed up compiles.

Opaque Pointers / Incomplete Types

Hide the real struct layout so callers cannot poke private fields. The header only names the type; the `.c` file owns the definition.

vector.h

```c
#ifndef VECTOR_H
#define VECTOR_H

#include <stddef.h>

typedef struct Vector Vector;   /* incomplete - no members visible */

Vector *vector_create(void);
void    vector_destroy(Vector *v);

int     vector_push(Vector *v, double x);   /* 0 ok, -1 OOM / NULL */
size_t  vector_size(const Vector *v);
int     vector_get(const Vector *v, size_t i, double *out);  /* 0 ok, -1 bad index */
void    vector_clear(Vector *v);

#endif /* VECTOR_H */
```

vector.c

```c
#include "vector.h"

#include <stdlib.h>
#include <string.h>

struct Vector {
    double *data;
    size_t  size;
```

```
    size_t  capacity;
};

Vector *vector_create(void) {
    Vector *v = calloc(1, sizeof *v);
    return v;  /* NULL if OOM */
}

void vector_destroy(Vector *v) {
    if (v == NULL) {
        return;
    }
    free(v->data);
    free(v);
}

int vector_push(Vector *v, double x) {
    if (v == NULL) {
        return -1;
    }
    if (v->size == v->capacity) {
        size_t new_cap = (v->capacity == 0) ? 8 : v->capacity * 2;
        double *p = realloc(v->data, new_cap * sizeof *p);
        if (p == NULL) {
            return -1;
        }
        v->data = p;
        v->capacity = new_cap;
    }
    v->data[v->size++] = x;
    return 0;
}

size_t vector_size(const Vector *v) {
    return v ? v->size : 0;
}

int vector_get(const Vector *v, size_t i, double *out) {
    if (v == NULL || out == NULL || i >= v->size) {
        return -1;
    }
    *out = v->data[i];
    return 0;
}

void vector_clear(Vector *v) {
    if (v) {
```

```
        v->size = 0;
    }
}
```

main_vector.c

```
#include <stdio.h>
#include "vector.h"

int main(void) {
    Vector *v = vector_create();
    if (v == NULL) {
        return 1;
    }

    vector_push(v, 1.5);
    vector_push(v, 2.5);
    vector_push(v, 3.0);

    for (size_t i = 0; i < vector_size(v); i++) {
        double x;
        if (vector_get(v, i, &x) == 0) {
            printf("v[%zu] = %.1f\n", i, x);
        }
    }

    /* This would NOT compile - incomplete type:
       printf("%zu\n", v->size);
    */

    vector_destroy(v);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra vector.c main_vector.c -o vector_demo
./vector_demo
```

Benefits: you can change `struct Vector` internals without breaking callers, and you force all access through a documented API.

Alternate Multi-Module Example: 2D Vector Math

Same patterns, different domain — useful if you prefer numeric code over string stats.

vec2.h

```c
#ifndef VEC2_H
#define VEC2_H

typedef struct {
    double x;
    double y;
} Vec2;

Vec2   vec2_add(Vec2 a, Vec2 b);
Vec2   vec2_sub(Vec2 a, Vec2 b);
Vec2   vec2_scale(Vec2 a, double s);
double vec2_dot(Vec2 a, Vec2 b);
double vec2_length(Vec2 a);
void   vec2_print(const char *label, Vec2 a);

#endif
```

vec2.c

```c
#include "vec2.h"

#include <math.h>
#include <stdio.h>

Vec2 vec2_add(Vec2 a, Vec2 b) {
    return (Vec2){a.x + b.x, a.y + b.y};
}

Vec2 vec2_sub(Vec2 a, Vec2 b) {
    return (Vec2){a.x - b.x, a.y - b.y};
}

Vec2 vec2_scale(Vec2 a, double s) {
    return (Vec2){a.x * s, a.y * s};
}

double vec2_dot(Vec2 a, Vec2 b) {
    return a.x * b.x + a.y * b.y;
}

double vec2_length(Vec2 a) {
    return sqrt(vec2_dot(a, a));
}
```

```c
void vec2_print(const char *label, Vec2 a) {
    printf("%s: (%.3f, %.3f)\n", label ? label : "v", a.x, a.y);
}
```

main_vec2.c

```c
#include "vec2.h"
#include <stdio.h>

int main(void) {
    Vec2 a = {3.0, 4.0};
    Vec2 b = {1.0, 0.0};

    vec2_print("a", a);
    vec2_print("b", b);
    vec2_print("a+b", vec2_add(a, b));
    printf("|a| = %.3f\n", vec2_length(a));
    printf("a·b = %.3f\n", vec2_dot(a, b));
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra vec2.c main_vec2.c -lm -o vec2_demo
./vec2_demo
```

Note: `-lm` links the math library for `sqrt`.

Callback Interfaces (Flexible Modules)

Function pointers let a module stay generic while callers plug in behavior:

```c
/* map.h */
#ifndef MAP_H
#define MAP_H

typedef int (*map_fn)(int value, void *ctx);

void map_apply(int *arr, int n, map_fn fn, void *ctx);

#endif
```

```c
/* map.c */
#include "map.h"
```

```c
void map_apply(int *arr, int n, map_fn fn, void *ctx) {
    if (arr == NULL || fn == NULL || n <= 0) {
        return;
    }
    for (int i = 0; i < n; i++) {
        arr[i] = fn(arr[i], ctx);
    }
}
```

```c
/* main_map.c */
#include "map.h"
#include <stdio.h>

static int add_k(int value, void *ctx) {
    int k = *(const int *)ctx;
    return value + k;
}

int main(void) {
    int a[] = {1, 2, 3, 4};
    int k = 10;
    map_apply(a, 4, add_k, &k);
    for (int i = 0; i < 4; i++) {
        printf("%d ", a[i]);  /* 11 12 13 14 */
    }
    printf("\n");
    return 0;
}
```

Best Practices

1. **Naming** — Prefix public symbols with the module name (`strstats_`, `vector_`).
2. **Error handling** — Return error codes; document them; let callers decide how to report.
3. **Documentation** — Comment the header: parameters, returns, ownership, thread-safety.
4. **Testing** — Prefer unit-testable modules with minimal I/O in the library layer.
5. **Globals** — Prefer parameters and opaque state over `extern` globals.
6. **Versioning** — Keep header and library in lockstep; document API breaks.
7. **Compiler flags** — Always develop with `-std=c17 -Wall -Wextra` (add `-Wpedantic` when practical).

Exercises

1. **Split a program.** Take any single-file program you have written and split it into `utils.h` / `utils.c` / `main.c`. Add a Makefile with debug/release and `-MMD`.

2. **Extend `string_stats`.** Add `strstats_to_upper` and a function that counts vowels. Keep helpers `static` in the `.c` file.
3. **Static library.** Build `libstrstats.a` and link a second small program that only calls `strstats_analyze`. Confirm with `nm` that symbols come from the archive.
4. **Shared library.** Build `libstrstats.so`, link, and run with the appropriate library path environment variable on your OS.
5. **Opaque counter.** Implement `counter.h` / `counter.c` with `counter_create`, `counter_inc`, `counter_get`, `counter_destroy`. Do not expose the struct layout in the header.
6. **Include audit.** Deliberately break a header's self-containment (remove a needed `#include`), put the related `#include "mod.h"` first in `mod.c`, and observe the compile error. Fix it correctly.
7. **Forward declarations.** Create two modules that would circularly include each other if you `#include` both headers. Resolve the cycle with a forward declaration and move the full include into a `.c` file.
8. **Vector math library.** Add `vec2_normalize` and unit tests (or a small `assert`-based driver). Link with `-lm`.

Summary

Modular C rests on a few durable ideas:

1. **Headers declare interfaces; sources define implementations.**
2. **Include guards** keep headers safe to include repeatedly.
3. **Linkage** (`extern` / `static`) controls who can see a symbol.
4. **Makefiles** with `-MMD` keep multi-file projects rebuild-correct.
5. **Static (`.a`) and shared (`.so`) libraries** package modules for reuse.
6. **Opaque pointers** hide representation and stabilize APIs.
7. **Include order and forward declarations** keep dependencies honest and compiles fast.

Master these patterns and you can grow from single-file experiments to maintainable multi-module programs and libraries without fighting the toolchain.

Standard Library Functions

Introduction

The C standard library is a collection of functions, macros, and type definitions that are part of the C programming language specification. These functions provide essential functionality for input/output operations, string manipulation, mathematical computations, memory management, and more. Understanding and effectively using the standard library is crucial for C programmers as it provides a foundation for building robust applications.

This chapter mixes **API maps** with **complete, compilable programs**. Prefer the programs: type them, break them, and re-run.

```
gcc -std=c17 -Wall -Wextra -o prog prog.c
# math examples need the math library on many systems:
gcc -std=c17 -Wall -Wextra -o prog prog.c -lm
```

String Functions (<string.h>)

String Copying

```
#include <string.h>

char *strcpy(char *dest, const char *src);
char *strncpy(char *dest, const char *src, size_t n);
char *strdup(const char *s);        // C23 standard
char *strndup(const char *s, size_t n); // C23 standard
```

String Concatenation

```
char *strcat(char *dest, const char *src);
char *strncat(char *dest, const char *src, size_t n);
```

String Comparison

```
int strcmp(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);
int strcoll(const char *s1, const char *s2);  // Locale-specific
```

String Searching

```
char *strchr(const char *s, int c);     // Find first occurrence
char *strrchr(const char *s, int c);    // Find last occurrence
char *strstr(const char *haystack, const char *needle);  // Find substring
size_t strcspn(const char *s, const char *reject);       // Span of non-matching chars
size_t strspn(const char *s, const char *accept);        // Span of matching chars
char *strpbrk(const char *s, const char *accept);        // Find any of a set of chars
```

String Tokenization

```
char *strtok(char *str, const char *delim);
```

String Length

```
size_t strlen(const char *s);
size_t strnlen(const char *s, size_t maxlen);  // C23 standard
```

Memory Functions (<string.h>)

Memory Copying

```
void *memcpy(void *dest, const void *src, size_t n);
void *memmove(void *dest, const void *src, size_t n);
```

Memory Setting

```
void *memset(void *s, int c, size_t n);
```

Memory Comparison

```
int memcmp(const void *s1, const void *s2, size_t n);
```

Memory Searching

```
void *memchr(const void *s, int c, size_t n);
```

Mathematical Functions (<math.h>)

Basic Mathematical Functions

```
#include <math.h>

double fabs(double x);          // Absolute value
double ceil(double x);          // Ceiling function
double floor(double x);         // Floor function
double round(double x);         // Round to nearest integer
double trunc(double x);         // Truncate towards zero

double sqrt(double x);          // Square root
double cbrt(double x);          // Cube root
double pow(double x, double y); // Power function
double exp(double x);           // Exponential function
double log(double x);           // Natural logarithm
double log10(double x);         // Base-10 logarithm
double log2(double x);          // Base-2 logarithm

double sin(double x);           // Sine function
double cos(double x);           // Cosine function
double tan(double x);           // Tangent function
double asin(double x);          // Arc sine
double acos(double x);          // Arc cosine
double atan(double x);          // Arc tangent
double atan2(double y, double x); // Arc tangent with two arguments
```

Hyperbolic Functions

```
double sinh(double x);          // Hyperbolic sine
double cosh(double x);          // Hyperbolic cosine
double tanh(double x);          // Hyperbolic tangent
```

Floating-Point Functions

```
double fmod(double x, double y);    // Floating-point remainder
double remainder(double x, double y); // IEEE remainder
double copysign(double x, double y);  // Copy sign
double nextafter(double x, double y); // Next representable value
```

Classification and Comparison Functions

```
int isfinite(double x);         // Check if finite
int isinf(double x);            // Check if infinite
int isnan(double x);            // Check if NaN
int isnormal(double x);         // Check if normal
int signbit(double x);          // Check sign
```

Character Classification (<ctype.h>)

Character Testing

```
#include <ctype.h>

int isalnum(int c);             // Alphanumeric character
int isalpha(int c);             // Alphabetic character
int isdigit(int c);             // Decimal digit
int isxdigit(int c);            // Hexadecimal digit
int islower(int c);             // Lowercase letter
int isupper(int c);             // Uppercase letter
int isblank(int c);             // Blank character (space or tab)
int isspace(int c);             // Whitespace character
int iscntrl(int c);             // Control character
int isprint(int c);             // Printable character
int isgraph(int c);             // Graphic character
int ispunct(int c);             // Punctuation character
```

Character Conversion

```c
int tolower(int c);             // Convert to lowercase
int toupper(int c);             // Convert to uppercase
```

Time Functions (<time.h>)

Time Types

```c
#include <time.h>

typedef long time_t;            // Calendar time
typedef long clock_t;           // Processor time
struct tm {                     // Broken-down time
    int tm_sec;                 // Seconds (0-60)
    int tm_min;                 // Minutes (0-59)
    int tm_hour;                // Hours (0-23)
    int tm_mday;                // Day of month (1-31)
    int tm_mon;                 // Month (0-11)
    int tm_year;                // Year since 1900
    int tm_wday;                // Day of week (0-6, Sunday = 0)
    int tm_yday;                // Day of year (0-365)
    int tm_isdst;               // Daylight saving time flag
};
```

Time Functions

```c
time_t time(time_t *tloc);      // Get current time
double difftime(time_t time1, time_t time0);  // Difference between two times
time_t mktime(struct tm *tm);   // Convert broken-down time to calendar time
char *asctime(const struct tm *tm);  // Convert to string
char *ctime(const time_t *timep);    // Convert to string
struct tm *gmtime(const time_t *timep);  // Convert to UTC
struct tm *localtime(const time_t *timep);  // Convert to local time
size_t strftime(char *s, size_t max, const char *format, const struct tm *tm);
```

Input/Output Functions (<stdio.h>)

File Operations

```c
#include <stdio.h>

FILE *fopen(const char *pathname, const char *mode);
int fclose(FILE *stream);
int fflush(FILE *stream);
FILE *freopen(const char *pathname, const char *mode, FILE *stream);
FILE *fdopen(int fd, const char *mode);
FILE *fmemopen(void *buf, size_t size, const char *mode);  // C23 standard
```

Formatted I/O

```c
int printf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
int sprintf(char *str, const char *format, ...);
int snprintf(char *str, size_t size, const char *format, ...);  // C99

int scanf(const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
int sscanf(const char *str, const char *format, ...);
```

Character I/O

```c
int fgetc(FILE *stream);
char *fgets(char *s, int size, FILE *stream);
int fputc(int c, FILE *stream);
int fputs(const char *s, FILE *stream);
int getchar(void);
char *gets(char *s);            // Deprecated, unsafe
int putchar(int c);
int puts(const char *s);
int ungetc(int c, FILE *stream);
```

Direct I/O

```c
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

File Positioning

```
int fseek(FILE *stream, long offset, int whence);
long ftell(FILE *stream);
void rewind(FILE *stream);
int fgetpos(FILE *stream, fpos_t *pos);
int fsetpos(FILE *stream, const fpos_t *pos);
```

Error Handling

```
void clearerr(FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
void perror(const char *s);
```

Memory Management (<stdlib.h>)

Dynamic Memory Allocation

```
#include <stdlib.h>

void *malloc(size_t size);
void *calloc(size_t nmemb, size_t size);
void *realloc(void *ptr, size_t size);
void free(void *ptr);
void *aligned_alloc(size_t alignment, size_t size);  // C11 standard
```

Utility Functions

```
double atof(const char *nptr);          // Convert string to double
int atoi(const char *nptr);             // Convert string to int
long atol(const char *nptr);            // Convert string to long
long long atoll(const char *nptr);      // Convert string to long long (C99)

double strtod(const char *nptr, char **endptr);     // Convert string to double
float strtof(const char *nptr, char **endptr);      // Convert string to float (C99)
long double strtold(const char *nptr, char **endptr); // Convert string to long double (C99)
long strtol(const char *nptr, char **endptr, int base);
long long strtoll(const char *nptr, char **endptr, int base);  // C99
unsigned long strtoul(const char *nptr, char **endptr, int base);
unsigned long long strtoull(const char *nptr, char **endptr, int base);  // C99
```

Random Number Generation

```
int rand(void);                 // Generate pseudo-random number
void srand(unsigned int seed);  // Seed the random number generator
```

Environment Functions

```
void abort(void);               // Cause abnormal program termination
int atexit(void (*func)(void)); // Register function to be called at exit
int at_quick_exit(void (*func)(void));  // Register function for quick exit (C11)
void exit(int status);          // Terminate program normally
void _Exit(int status);         // Terminate program immediately (C99)
void quick_exit(int status);    // Quick program termination (C11)
char *getenv(const char *name); // Get environment variable
int system(const char *command); // Execute system command
```

Searching and Sorting

```
void *bsearch(const void *key, const void *base,
              size_t nmemb, size_t size,
              int (*compar)(const void *, const void *));
void qsort(void *base, size_t nmemb, size_t size,
           int (*compar)(const void *, const void *));
```

Integer Arithmetic

```
int abs(int j);                 // Absolute value of int
long labs(long j);              // Absolute value of long
long long llabs(long long j);   // Absolute value of long long (C99)
div_t div(int numer, int denom);  // Integer division
ldiv_t ldiv(long numer, long denom);  // Long integer division
lldiv_t lldiv(long long numer, long long denom);  // Long long integer division (C99)
```

Advanced Standard Library Features (C23)

New String Functions

```
// C23 additions
char *strdup(const char *s);
char *strndup(const char *s, size_t n);
size_t strnlen(const char *s, size_t maxlen);
```

Enhanced Memory Functions

```
// Bounds-checking functions (Annex K, optional in C23)
errno_t memcpy_s(void *dest, rsize_t destsz,
                 const void *src, rsize_t count);
errno_t strcpy_s(char *dest, rsize_t destsz,
                 const char *src);
```

Practical Examples

String Manipulation Example

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char str1[50] = "Hello, ";
    char str2[] = "World!";
    char str3[100];

    // String concatenation
    strcpy(str3, str1);
    strcat(str3, str2);
    printf("Concatenated: %s\n", str3);

    // String comparison
    if (strcmp(str1, str2) == 0) {
        printf("Strings are equal\n");
    } else {
        printf("Strings are different\n");
    }
```

```
    // String searching
    char *found = strstr(str3, "World");
    if (found) {
        printf("Found 'World' at position: %ld\n", found - str3);
    }

    // String tokenization
    char sentence[] = "This is a sample sentence";
    char *token = strtok(sentence, " ");
    while (token != NULL) {
        printf("Token: %s\n", token);
        token = strtok(NULL, " ");
    }

    return 0;
}
```

Mathematical Functions Example

```
#include <stdio.h>
#include <math.h>

int main() {
    double x = 4.0;
    double y = 3.0;

    printf("Square root of %.2f: %.2f\n", x, sqrt(x));
    printf("%.2f raised to the power of %.2f: %.2f\n", x, y, pow(x, y));
    printf("Sine of %.2f radians: %.2f\n", x, sin(x));
    printf("Natural log of %.2f: %.2f\n", x, log(x));

    // Rounding functions
    double value = 3.7;
    printf("Value: %.2f\n", value);
    printf("Ceiling: %.2f\n", ceil(value));
    printf("Floor: %.2f\n", floor(value));
    printf("Round: %.2f\n", round(value));

    return 0;
}
```

Time Functions Example

```c
#include <stdio.h>
#include <time.h>

int main() {
    // Get current time
    time_t rawtime;
    struct tm *timeinfo;
    char buffer[80];

    time(&rawtime);
    timeinfo = localtime(&rawtime);

    printf("Current local time: %s", asctime(timeinfo));

    // Format time
    strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", timeinfo);
    printf("Formatted time: %s\n", buffer);

    // Calculate time difference
    time_t start_time = time(NULL);

    // Simulate some work
    for (int i = 0; i < 1000000; i++) {
        // Busy loop
    }

    time_t end_time = time(NULL);
    double elapsed = difftime(end_time, start_time);
    printf("Elapsed time: %.0f seconds\n", elapsed);

    return 0;
}
```

Memory Management Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // Dynamic memory allocation
    int n = 5;
    int *arr = (int*)malloc(n * sizeof(int));
```

```
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Initialize array
    for (int i = 0; i < n; i++) {
        arr[i] = i + 1;
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    // Resize array
    n = 10;
    arr = (int*)realloc(arr, n * sizeof(int));

    if (arr == NULL) {
        printf("Memory reallocation failed\n");
        return 1;
    }

    // Initialize new elements
    for (int i = 5; i < n; i++) {
        arr[i] = i + 1;
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    // Free memory
    free(arr);
    arr = NULL;  // Good practice to avoid dangling pointers

    return 0;
}
```

Full Programs: Standard Library in Practice

The snippets above are reference maps. The programs below are meant to be **typed, compiled, and run**.

Program 1 — Safe string toolbox (`strtool.c`)

Demonstrates length, bounded copy, search, tokenize, and compare without buffer overflows.

```
/* file: strtool.c */
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* Copy src into dest; always NUL-terminates if dest_sz > 0.
 * Returns 0 on full copy, -1 if truncated or bad args. */
static int str_copy(char *dest, size_t dest_sz, const char *src) {
    size_t i;

    if (dest == NULL || src == NULL || dest_sz == 0) {
        return -1;
    }
    for (i = 0; i + 1 < dest_sz && src[i] != '\0'; i++) {
        dest[i] = src[i];
    }
    dest[i] = '\0';
    return (src[i] == '\0') ? 0 : -1;
}

static void trim_inplace(char *s) {
    char *start, *end;
    size_t len;

    if (s == NULL) {
        return;
    }
    start = s;
    while (*start != '\0' && isspace((unsigned char)*start)) {
        start++;
    }
    if (*start == '\0') {
        s[0] = '\0';
        return;
    }
    end = start + strlen(start) - 1;
    while (end > start && isspace((unsigned char)*end)) {
        end--;
    }
    len = (size_t)(end - start + 1);
    memmove(s, start, len);
    s[len] = '\0';
}

int main(void) {
    char line[128];
    char key[32];
    char *tok;
```

```
    const char *hay = "the quick brown fox jumps over the lazy dog";
    const char *needle = "fox";
    char *found;

    printf("strlen(hay) = %zu\n", strlen(hay));
    found = strstr(hay, needle);
    if (found != NULL) {
        printf("strstr: \"%s\" at offset %td\n", needle, found - hay);
    }

    if (str_copy(key, sizeof key, "port=8080#comment") != 0) {
        fprintf(stderr, "key truncated\n");
    }
    printf("copy: \"%s\"\n", key);

    /* strtok is standard C (not reentrant - fine for this demo) */
    {
        char mutable[] = "alice,bob,carol";
        printf("tokens:");
        for (tok = strtok(mutable, ","); tok != NULL; tok = strtok(NULL, ",")) {
            printf(" [%s]", tok);
        }
        printf("\n");
    }

    printf("Enter a line to trim: ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return EXIT_FAILURE;
    }
    /* strip newline then trim spaces */
    line[strcspn(line, "\n")] = '\0';
    trim_inplace(line);
    printf("trimmed: \"%s\"\n", line);

    if (strcmp(line, "quit") == 0) {
        printf("matched quit\n");
    } else {
        printf("strcmp vs \"quit\": %d\n", strcmp(line, "quit"));
    }

    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o strtool strtool.c
./strtool
```

Notes

- Prefer `strcspn` / manual copies / `snprintf` over unbounded `strcpy`/`strcat`.
- Cast to `unsigned char` before `isspace` / `ctype.h` macros.
- `strtok` modifies its buffer and keeps hidden state — fine for simple tools; avoid in threaded code.

Program 2 — Math lab: vectors and roots (`mathlab.c`)

```c
/* file: mathlab.c */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <errno.h>

struct Vec2 {
    double x;
    double y;
};

static double vec_len(struct Vec2 v) {
    return hypot(v.x, v.y);  /* safer than sqrt(x*x + y*y) for large values */
}

static struct Vec2 vec_norm(struct Vec2 v) {
    double len = vec_len(v);
    struct Vec2 out = {0.0, 0.0};
    if (len > 0.0) {
        out.x = v.x / len;
        out.y = v.y / len;
    }
    return out;
}

static int safe_sqrt(double x, double *out) {
    if (out == NULL) {
        return -1;
    }
    if (x < 0.0) {
        errno = EDOM;
        return -1;
    }
    *out = sqrt(x);
    return 0;
}
```

```
int main(void) {
    struct Vec2 a = {3.0, 4.0};
    struct Vec2 n = vec_norm(a);
    double root;
    double angle;

    printf("|a| = %.6f\n", vec_len(a));
    printf("unit(a) = (%.6f, %.6f)\n", n.x, n.y);

    angle = atan2(a.y, a.x);
    printf("atan2(y,x) = %.6f rad (%.2f deg)\n",
           angle, angle * 180.0 / 3.14159265358979323846);

    if (safe_sqrt(2.0, &root) == 0) {
        printf("sqrt(2) = %.10f\n", root);
    }
    if (safe_sqrt(-1.0, &root) != 0) {
        printf("sqrt(-1) rejected (errno may be EDOM)\n");
    }

    printf("pow(2,10) = %.0f\n", pow(2.0, 10.0));
    printf("floor(3.7)=%.1f ceil(3.2)=%.1f round(3.5)=%.1f\n",
           floor(3.7), ceil(3.2), round(3.5));

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o mathlab mathlab.c -lm
./mathlab
```

Program 3 — Character classifier report (`ctype_report.c`)

```
/* file: ctype_report.c */
#include <stdio.h>
#include <ctype.h>
#include <string.h>

static void classify(int ch) {
    unsigned char c = (unsigned char)ch;
    printf("'%c' (0x%02x):", (c >= 32 && c < 127) ? c : '?', c);
    if (isdigit(c))  printf(" digit");
    if (isalpha(c))  printf(" alpha");
    if (isalnum(c))  printf(" alnum");
    if (isspace(c))  printf(" space");
    if (ispunct(c))  printf(" punct");
```

```
    if (isupper(c))  printf(" upper");
    if (islower(c))  printf(" lower");
    printf(" -> upper='%c' lower='%c'\n",
           toupper(c), tolower(c));
}

int main(int argc, char *argv[]) {
    const char *s;
    size_t i;

    if (argc < 2) {
        fprintf(stderr, "Usage: %s <string>\n", argv[0]);
        return 1;
    }
    s = argv[1];
    for (i = 0; s[i] != '\0'; i++) {
        classify((unsigned char)s[i]);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o ctype_report ctype_report.c
./ctype_report 'Hi 42!'
```

Program 4 — Time stamps and elapsed work (`timelab.c`)

```
/* file: timelab.c */
#include <stdio.h>
#include <time.h>
#include <string.h>

int main(void) {
    time_t now;
    struct tm local_tm;
    char buf[64];
    clock_t t0, t1;
    volatile double sink = 0.0;
    long i;

    now = time(NULL);
    if (now == (time_t)-1) {
        perror("time");
        return 1;
    }
```

```
#if defined(_POSIX_VERSION)
    if (localtime_r(&now, &local_tm) == NULL) {
        perror("localtime_r");
        return 1;
    }
#else
    {
        struct tm *p = localtime(&now);
        if (p == NULL) {
            return 1;
        }
        local_tm = *p;
    }
#endif

    if (strftime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S %Z", &local_tm) == 0) {
        fprintf(stderr, "strftime failed\n");
        return 1;
    }
    printf("local: %s\n", buf);
    printf("asctime: %s", asctime(&local_tm));  /* includes newline */

    t0 = clock();
    for (i = 0; i < 10000000L; i++) {
        sink += 1.0;
    }
    t1 = clock();
    printf("busy loop: %.3f seconds of CPU (clock)\n",
           (double)(t1 - t0) / (double)CLOCKS_PER_SEC);
    printf("sink=%.0f\n", sink);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o timelab timelab.c
./timelab
```

Program 5 — `qsort` + `bsearch` + `stdlib` utilities (`sortlab.c`)

```
/* file: sortlab.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static int cmp_int_asc(const void *a, const void *b) {
```

```
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

static int cmp_str(const void *a, const void *b) {
    const char *const *sa = a;
    const char *const *sb = b;
    return strcmp(*sa, *sb);
}

int main(void) {
    int nums[] = {42, 7, 19, 3, 100, 19};
    size_t n = sizeof nums / sizeof nums[0];
    int key = 19;
    int *found;
    const char *words[] = {"pear", "apple", "orange", "banana"};
    size_t wn = sizeof words / sizeof words[0];
    size_t i;
    char *end;
    long port;

    qsort(nums, n, sizeof nums[0], cmp_int_asc);
    printf("sorted ints:");
    for (i = 0; i < n; i++) {
        printf(" %d", nums[i]);
    }
    printf("\n");

    found = bsearch(&key, nums, n, sizeof nums[0], cmp_int_asc);
    if (found != NULL) {
        printf("bsearch %d -> index %td\n", key, found - nums);
    }

    qsort(words, wn, sizeof words[0], cmp_str);
    printf("sorted words:");
    for (i = 0; i < wn; i++) {
        printf(" %s", words[i]);
    }
    printf("\n");

    port = strtol("8080", &end, 10);
    if (end != NULL && *end == '\0' && port > 0 && port <= 65535) {
        printf("parsed port: %ld\n", port);
    }

    printf("abs(-12)=%d, RAND_MAX=%d, rand sample=%d\n",
```

```
           abs(-12), RAND_MAX, rand() % 100);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o sortlab sortlab.c
./sortlab
```

Program 6 — Mini word frequency with `string.h` + heap (`wfreq.c`)

```
/* file: wfreq.c - counts words from stdin (letters/digits only) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_WORD 64
#define MAX_ENTRIES 256

struct Entry {
    char word[MAX_WORD];
    int count;
};

static int find_or_add(struct Entry *tab, int *n, const char *w) {
    int i;
    for (i = 0; i < *n; i++) {
        if (strcmp(tab[i].word, w) == 0) {
            tab[i].count++;
            return 0;
        }
    }
    if (*n >= MAX_ENTRIES) {
        return -1;
    }
    if (strlen(w) >= MAX_WORD) {
        return -1;
    }
    strcpy(tab[*n].word, w);  /* bounded by check above */
    tab[*n].count = 1;
    (*n)++;
    return 0;
}

static int cmp_entry_desc(const void *a, const void *b) {
```

```
    const struct Entry *ea = a;
    const struct Entry *eb = b;
    if (ea->count != eb->count) {
        return eb->count - ea->count;
    }
    return strcmp(ea->word, eb->word);
}

int main(void) {
    struct Entry table[MAX_ENTRIES];
    int n = 0;
    char word[MAX_WORD];
    int wi = 0;
    int ch;
    int i;

    while ((ch = getchar()) != EOF) {
        if (isalnum((unsigned char)ch)) {
            if (wi + 1 < MAX_WORD) {
                word[wi++] = (char)tolower((unsigned char)ch);
            }
        } else if (wi > 0) {
            word[wi] = '\0';
            if (find_or_add(table, &n, word) != 0) {
                fprintf(stderr, "table full or word too long\n");
                return EXIT_FAILURE;
            }
            wi = 0;
        }
    }
    if (wi > 0) {
        word[wi] = '\0';
        find_or_add(table, &n, word);
    }

    qsort(table, (size_t)n, sizeof table[0], cmp_entry_desc);
    for (i = 0; i < n; i++) {
        printf("%4d  %s\n", table[i].count, table[i].word);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o wfreq wfreq.c
echo 'To be or not to be' | ./wfreq
```

Best Practices for Using Standard Library Functions

1. Error Checking

Always check return values for error conditions:

```
FILE *file = fopen("data.txt", "r");
if (file == NULL) {
    perror("Error opening file");
    return 1;
}
```

2. Buffer Safety

Use safe versions of functions when available and ensure adequate buffer sizes:

```
char buffer[100];
// Safe approach
if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
    // Process input
}
```

Prefer `snprintf` over `sprintf`:

```
char path[64];
int n = snprintf(path, sizeof path, "/tmp/%s.log", "app");
if (n < 0 || (size_t)n >= sizeof path) {
    /* truncated or error */
}
```

3. Memory Management

Always free dynamically allocated memory:

```
int *ptr = malloc(10 * sizeof(int));
if (ptr != NULL) {
    // Use memory
    free(ptr);
    ptr = NULL;  // Prevent dangling pointer
}
```

4. Locale Awareness

Be aware of locale-specific functions:

```c
#include <locale.h>

setlocale(LC_ALL, "");  // Use system locale
```

5. Thread Safety

Some standard library functions are not thread-safe. Use reentrant versions when needed:

```c
// Thread-safe version (POSIX)
struct tm tm_result;
localtime_r(&rawtime, &tm_result);
```

6. Link math explicitly

On many Linux toolchains, math symbols live in `libm`:

```c
gcc -std=c17 -Wall -Wextra -o mathlab mathlab.c -lm
```

Exercises

```c
gcc -std=c17 -Wall -Wextra -o exN exN.c    # add -lm when using <math.h>
```

1. **Bounded concat** — Write `int str_cat(char *dest, size_t dest_sz, const char *src)` that appends without overflow and always NUL-terminates. Unit-test with full and almost-full buffers.
2. **`strtol` port parser** — Accept a string; accept only `1…65535`; reject junk, empty, negatives, and overflow. Return status codes, not just print.
3. **Case-insensitive compare** — Implement `int str_casecmp(const char *a, const char *b)` with `tolower((unsigned char)…)` and test `"Hello"` vs `"hello"`.
4. **Math: distance** — Given two `Vec2` points, print Euclidean distance with `hypot`. Compare naive `sqrt(dx*dx+dy*dy)` on large coordinates (optional exploration).
5. **`qsort` structs** — Sort an array of `{name, score}` by score descending, then name ascending.
6. **Time format** — Print "days since epoch" and a `strftime` format `YYYY-MM-DDThh:mm:ss`.
7. **`wfreq` upgrade** — Read from a filename in `argv[1]` instead of stdin; print only the top 10 words.
8. **ctype filter** — Read stdin, write only alphanumeric characters and spaces to stdout (like a crude sanitizer).

Summary

The C standard library provides a rich set of functions that form the foundation of most C programs. Mastering these functions is essential for effective C programming. Key points to remember:

1. **String Functions**: Essential for text processing and manipulation — prefer bounded copies and `snprintf`
2. **Mathematical Functions**: Provide a wide range of mathematical operations — link with `-lm`
3. **Character Functions**: Enable character classification and conversion — cast to `unsigned char`
4. **Time Functions**: Handle time-related operations and formatting — prefer reentrant APIs when available
5. **I/O Functions**: Enable file and console input/output operations — always check returns
6. **Memory Management**: Provide dynamic memory allocation capabilities — pair every `malloc` with `free`
7. **Utility Functions**: `qsort`, `bsearch`, `strtol`, `abs`, `rand` cover many everyday needs

Understanding and properly using these standard library functions will make your C programs more robust, portable, and efficient. Always consult the documentation for specific behavior and error handling requirements of each function.

Module 4: Functions and Modular Programming

Overview

This module explores the fundamental concepts of functions in C and how to organize code into modular, reusable components. You'll learn about function design, parameter passing, return values, and advanced function concepts like recursion and function pointers. The module also covers modular programming techniques, including header files, compilation units, and library creation.

Learning Objectives

By the end of this module, you will be able to: - Design and implement functions with proper parameters and return values - Understand parameter passing mechanisms (by value) - Create and use recursive functions effectively - Work with variable argument functions and inline functions - Utilize function pointers for callbacks and dynamic dispatch - Organize code into modular components using header and source files - Create and use both static and shared libraries - Understand linkage concepts and scope rules - Apply modular programming principles for maintainable code

Chapters

1. **Function Fundamentals** - Function definition, declaration, parameters, and return values
2. **Advanced Function Concepts** - Recursion, variable arguments, inline functions, and function pointers
3. **Modular Programming** - Header files, compilation units, and linkage concepts
4. **Standard Library Functions** - Commonly used functions from C standard library

Key Concepts Covered

- Function declaration and definition syntax
- Parameter passing by value
- Return values and the return statement
- Function prototypes and forward declarations
- Scope rules and name hiding
- Recursive function design and implementation
- Variable argument functions using stdarg.h
- Inline functions for performance optimization
- Function pointers and callback mechanisms

- Header file organization and include guards
- Compilation units and linking process
- Static vs. external linkage
- Creating and using libraries
- Standard library functions overview

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Understanding of basic programming concepts
- Familiarity with variable declaration and initialization

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Understanding of basic programming concepts
- Familiarity with debugging tools
- Make or CMake for building multi-file projects

Estimated Time to Complete

- Reading: 8-10 hours
- Exercises: 12-15 hours
- Projects: 6-8 hours

Assessment

- Complete all chapter exercises
- Successfully implement various function types
- Demonstrate understanding of modular programming concepts
- Pass function and modular programming quiz
- Submit a comprehensive program using advanced function features

Next Module

After completing this module, proceed to **Module 5: Arrays and Strings** to learn about data collections and text processing.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 4)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Online tutorials on function design patterns
- Debugging tools documentation (GDB, Valgrind)
- Best practices guides for modular programming

Arrays and Strings

Arrays

Introduction

Arrays store multiple values of the same type in a contiguous block of memory. They are the foundation of strings, buffers, tables, and many algorithms. This chapter covers declaration, indexing, traversal, passing arrays to functions, multidimensional layout, worked algorithm programs, common bugs, and a mini-lab.

Compile examples with:

```
gcc -std=c17 -Wall -Wextra program.c -o program
```

Array Declaration and Initialization

Basic Declaration

```
int numbers[10];          /* 10 integers (uninitialized if local) */
float temperatures[7];
char letters[26];
```

Initialization

```
/* Full initializer list */
int numbers[5] = {1, 2, 3, 4, 5};

/* Partial - remaining elements are zero */
int values[10] = {1, 2, 3};

/* All zeros */
int zeros[5] = {0};

/* Designated initializers (C99+) */
int array[10] = {[0] = 1, [5] = 10, [9] = 5};

/* Size inferred from initializer */
int primes[] = {2, 3, 5, 7, 11, 13};
```

Constant Arrays

```
const int DAYS_IN_MONTH[] = {
    31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
};
```

Access and Indexing

Indexing is zero-based. Valid indices for `int a[n]` are `0` through `n - 1`.

```
#include <stdio.h>

int main(void) {
    int numbers[5] = {10, 20, 30, 40, 50};

    printf("First: %d\n", numbers[0];   /* 10 */
    printf("Third: %d\n", numbers[2];   /* 30 */
    printf("Last:  %d\n", numbers[4];   /* 50 */

    numbers[1] = 25;
    numbers[3] = numbers[0] + numbers[2];  /* 40 */
    return 0;
}
```

No Automatic Bounds Checking

C does **not** check indices. Out-of-range access is undefined behavior:

```
int array[5] = {1, 2, 3, 4, 5};
array[0] = 10;   /* OK */
array[4] = 50;   /* OK */
/* array[-1] = 5; */   /* UB - do not */
/* array[10] = 100; */ /* UB - do not */
```

Traversal

```
#include <stdio.h>

int main(void) {
    int numbers[5] = {1, 2, 3, 4, 5};
    int size = 5;
```

```
    for (int i = 0; i < size; i++) {
        printf("a[%d] = %d\n", i, numbers[i]);
    }

    for (int i = size - 1; i >= 0; i--) {
        printf("rev a[%d] = %d\n", i, numbers[i]);
    }
    return 0;
}
```

`sizeof` and Array Length

```
#include <stdio.h>

#define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]))

int main(void) {
    int numbers[10];
    printf("len = %zu\n", ARRAY_LEN(numbers));  /* 10 */
    return 0;
}
```

Critical: `ARRAY_LEN` only works on a real array object, not on a pointer. After an array decays to a pointer (for example as a function parameter), `sizeof` yields the pointer size, not the array size. Always pass an explicit length.

Passing Arrays to Functions

When you pass an array, the callee receives a pointer to the first element. **Always pass the size** (or a sentinel convention you document).

Equivalent Parameter Declarations

These three mean the same thing for the callee — a pointer:

```
void f(int arr[], int n);
void f(int arr[10], int n);   /* 10 is ignored by the compiler for type */
void f(int *arr, int n);
```

Full Program: Size Parameter Pattern

```c
#include <stdio.h>

void print_ints(const int *arr, size_t n) {
    for (size_t i = 0; i < n; i++) {
        printf("%d%s", arr[i], (i + 1 < n) ? " " : "\n");
    }
}

int sum_ints(const int *arr, size_t n) {
    int s = 0;
    for (size_t i = 0; i < n; i++) {
        s += arr[i];
    }
    return s;
}

/* WRONG idea of "getting length inside the function" */
void wrong_len_demo(int arr[]) {
    /* sizeof(arr) is sizeof(int *), not the caller's array */
    printf("wrong \"length\" calculation: %zu\n",
           sizeof(arr) / sizeof(arr[0]));
}

int main(void) {
    int data[] = {3, 1, 4, 1, 5};
    size_t n = sizeof data / sizeof data[0];

    print_ints(data, n);
    printf("sum = %d\n", sum_ints(data, n));
    wrong_len_demo(data);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra pass_array.c -o pass_array
./pass_array
```

Optional: Static Array Size Hints (C99+)

```c
/* Documents that arr should point to at least n elements;
   some compilers can warn on misuse with the right flags. */
void process(size_t n, int arr[static n]);
```

Multidimensional Arrays and Row-Major Layout

2D Declaration

```
int matrix[3][4];   /* 3 rows, 4 columns */
int m[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Row-Major Storage

C stores multidimensional arrays in **row-major** order: all elements of row 0, then row 1, and so on. For `int a[R][C]`, element `a[i][j]` lives at offset:

```
index = i * C + j
```

in a flat view of `R * C` contiguous `int`s.

```
#include <stdio.h>

int main(void) {
    int a[2][3] = {
        {10, 20, 30},
        {40, 50, 60}
    };

    int *flat = &a[0][0];
    printf("a[1][2] via [i][j]  = %d\n", a[1][2]);      /* 60 */
    printf("a[1][2] via flat    = %d\n", flat[1 * 3 + 2]); /* 60 */

    printf("memory order: ");
    for (int k = 0; k < 6; k++) {
        printf("%d ", flat[k]);  /* 10 20 30 40 50 60 */
    }
    printf("\n");
    return 0;
}
```

Passing 2D Arrays

The callee needs the column count (or full type) so index arithmetic works:

```c
#include <stdio.h>

#define COLS 3

void print_matrix(const int m[][COLS], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%4d", m[i][j]);
        }
        printf("\n");
    }
}

/* Flat form: treat as one vector with explicit rows/cols */
void print_flat(const int *m, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%4d", m[i * cols + j]);
        }
        printf("\n");
    }
}

int main(void) {
    int m[2][COLS] = {{1, 2, 3}, {4, 5, 6}};
    print_matrix(m, 2);
    print_flat(&m[0][0], 2, COLS);
    return 0;
}
```

Worked Algorithms (Full Programs)

1. Reverse an Array In Place

```c
#include <stdio.h>

static void reverse(int *a, size_t n) {
    for (size_t i = 0; i < n / 2; i++) {
        int tmp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = tmp;
    }
}
```

```c
static void print_arr(const int *a, size_t n) {
    for (size_t i = 0; i < n; i++) {
        printf("%d%s", a[i], (i + 1 < n) ? " " : "\n");
    }
}

int main(void) {
    int a[] = {1, 2, 3, 4, 5};
    size_t n = sizeof a / sizeof a[0];

    printf("before: ");
    print_arr(a, n);
    reverse(a, n);
    printf("after:  ");
    print_arr(a, n);
    return 0;
}
```

2. Find Minimum and Maximum

```c
#include <stdio.h>
#include <stdbool.h>

/* Returns false if n == 0; otherwise writes *min_out / *max_out. */
static bool minmax(const int *a, size_t n, int *min_out, int *max_out) {
    if (a == NULL || min_out == NULL || max_out == NULL || n == 0) {
        return false;
    }
    int lo = a[0];
    int hi = a[0];
    for (size_t i = 1; i < n; i++) {
        if (a[i] < lo) lo = a[i];
        if (a[i] > hi) hi = a[i];
    }
    *min_out = lo;
    *max_out = hi;
    return true;
}

int main(void) {
    int a[] = {7, -2, 15, 0, 9, 15, -8};
    size_t n = sizeof a / sizeof a[0];
    int lo, hi;

    if (!minmax(a, n, &lo, &hi)) {
```

```
        fprintf(stderr, "empty array\n");
        return 1;
    }
    printf("min = %d, max = %d\n", lo, hi);
    return 0;
}
```

3. Frequency Count

Count how often each value appears in a small range (here: digits 0–9).

```
#include <stdio.h>
#include <string.h>

enum { RANGE = 10 };

static void count_digits(const int *a, size_t n, int freq[RANGE]) {
    memset(freq, 0, RANGE * sizeof freq[0]);
    for (size_t i = 0; i < n; i++) {
        int v = a[i];
        if (v >= 0 && v < RANGE) {
            freq[v]++;
        }
    }
}

int main(void) {
    int data[] = {1, 3, 1, 7, 3, 3, 9, 0, 1};
    size_t n = sizeof data / sizeof data[0];
    int freq[RANGE];

    count_digits(data, n, freq);

    printf("value : count\n");
    for (int v = 0; v < RANGE; v++) {
        if (freq[v] > 0) {
            printf("  %d   : %d\n", v, freq[v]);
        }
    }
    return 0;
}
```

4. Small Matrix Multiply (2×3 · 3×2 → 2×2)

```c
#include <stdio.h>

enum { A_ROWS = 2, A_COLS = 3, B_COLS = 2 };

/*
 * C = A * B
 * A is (A_ROWS x A_COLS), B is (A_COLS x B_COLS), C is (A_ROWS x B_COLS)
 * Row-major: C[i][j] = sum_k A[i][k] * B[k][j]
 */
static void matmul(const int A[A_ROWS][A_COLS],
                   const int B[A_COLS][B_COLS],
                   int C[A_ROWS][B_COLS]) {
    for (int i = 0; i < A_ROWS; i++) {
        for (int j = 0; j < B_COLS; j++) {
            int sum = 0;
            for (int k = 0; k < A_COLS; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}

static void print_2x2(const int M[A_ROWS][B_COLS]) {
    for (int i = 0; i < A_ROWS; i++) {
        for (int j = 0; j < B_COLS; j++) {
            printf("%4d", M[i][j]);
        }
        printf("\n");
    }
}

int main(void) {
    int A[A_ROWS][A_COLS] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int B[A_COLS][B_COLS] = {
        {7, 8},
        {9, 10},
        {11, 12}
    };
    int C[A_ROWS][B_COLS];

    matmul(A, B, C);
```

```
    printf("A * B =\n");
    print_2x2(C);
    /* Expected:
       58  64
       139 154
    */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra matmul.c -o matmul
./matmul
```

5. Histogram of Grades

Bucket scores into letter-grade ranges and print a text histogram.

```
#include <stdio.h>
#include <string.h>

enum { N_BUCKETS = 5 };

/* 0: F <60, 1: D 60-69, 2: C 70-79, 3: B 80-89, 4: A 90-100 */
static int grade_bucket(int score) {
    if (score < 60) return 0;
    if (score < 70) return 1;
    if (score < 80) return 2;
    if (score < 90) return 3;
    return 4;
}

static void histogram(const int *scores, size_t n, int buckets[N_BUCKETS]) {
    memset(buckets, 0, N_BUCKETS * sizeof buckets[0]);
    for (size_t i = 0; i < n; i++) {
        int s = scores[i];
        if (s < 0 || s > 100) {
            continue;  /* skip invalid */
        }
        buckets[grade_bucket(s)]++;
    }
}

static void print_bar(const char *label, int count) {
    printf("%s (%2d): ", label, count);
    for (int i = 0; i < count; i++) {
        putchar('#');
```

```
    }
    putchar('\n');
}

int main(void) {
    int scores[] = {
        95, 82, 77, 61, 54, 88, 91, 73, 66, 100,
        59, 84, 70, 45, 92
    };
    size_t n = sizeof scores / sizeof scores[0];
    int buckets[N_BUCKETS];
    const char *labels[N_BUCKETS] = {"F", "D", "C", "B", "A"};

    histogram(scores, n, buckets);

    printf("Grade histogram (%zu scores)\n", n);
    for (int i = 0; i < N_BUCKETS; i++) {
        print_bar(labels[i], buckets[i]);
    }
    return 0;
}
```

6. Linear Search and Selection Sort (Together)

```
#include <stdio.h>

static int linear_search(const int *a, size_t n, int target) {
    for (size_t i = 0; i < n; i++) {
        if (a[i] == target) {
            return (int)i;
        }
    }
    return -1;
}

static void selection_sort(int *a, size_t n) {
    for (size_t i = 0; i + 1 < n; i++) {
        size_t min_i = i;
        for (size_t j = i + 1; j < n; j++) {
            if (a[j] < a[min_i]) {
                min_i = j;
            }
        }
        if (min_i != i) {
            int t = a[i];
```

```
            a[i] = a[min_i];
            a[min_i] = t;
        }
    }
}

static void print_arr(const int *a, size_t n) {
    for (size_t i = 0; i < n; i++) {
        printf("%d%s", a[i], (i + 1 < n) ? " " : "\n");
    }
}

int main(void) {
    int a[] = {64, 25, 12, 22, 11};
    size_t n = sizeof a / sizeof a[0];

    printf("index of 22: %d\n", linear_search(a, n, 22));
    selection_sort(a, n);
    printf("sorted: ");
    print_arr(a, n);
    return 0;
}
```

Common Bugs: Wrong vs Right

Off-by-One in Loops

```
#include <stdio.h>

int main(void) {
    int a[5] = {1, 2, 3, 4, 5};

    /* WRONG: i <= 5 reads a[5] - out of bounds */
    /* for (int i = 0; i <= 5; i++) printf("%d\n", a[i]); */

    /* RIGHT: i < n */
    for (int i = 0; i < 5; i++) {
        printf("%d\n", a[i]);
    }
    return 0;
}
```

Forgetting That Arrays Decay

```c
#include <stdio.h>

/* WRONG: cannot recover length from decayed pointer */
static void bad_average(int arr[]) {
    size_t n = sizeof(arr) / sizeof(arr[0]);  /* pointer size! */
    int sum = 0;
    for (size_t i = 0; i < n; i++) {
        sum += arr[i];  /* may read garbage / crash */
    }
    printf("bad avg path used n=%zu\n", n);
    (void)sum;
}

/* RIGHT */
static double good_average(const int *arr, size_t n) {
    if (n == 0) {
        return 0.0;
    }
    long sum = 0;
    for (size_t i = 0; i < n; i++) {
        sum += arr[i];
    }
    return (double)sum / (double)n;
}

int main(void) {
    int a[] = {10, 20, 30, 40};
    size_t n = sizeof a / sizeof a[0];
    printf("avg = %.2f\n", good_average(a, n));
    bad_average(a);  /* educational only - do not copy this pattern */
    return 0;
}
```

Using Uninitialized Local Arrays

```c
/* WRONG */
int buf[10];
printf("%d\n", buf[0]);  /* indeterminate value */

/* RIGHT */
int buf[10] = {0};
/* or fill before read */
```

Copying with =

```
int a[3] = {1, 2, 3};
int b[3];
/* b = a; */  /* ILLEGAL - arrays are not assignable */

/* RIGHT */
for (int i = 0; i < 3; i++) {
    b[i] = a[i];
}
/* or memcpy(b, a, sizeof a); with <string.h> */
```

Returning a Pointer to a Local Array

```
/* WRONG */
int *bad(void) {
    int local[4] = {1, 2, 3, 4};
    return local;  /* dangling - storage ends when function returns */
}

/* RIGHT: caller owns the buffer */
void good(int *out, size_t n) {
    for (size_t i = 0; i < n; i++) {
        out[i] = (int)(i + 1);
    }
}
```

Practical Examples

Temperature Analysis

```
#include <stdio.h>

#define DAYS 7

int main(void) {
    float temperatures[DAYS] = {18.5f, 20.0f, 19.2f, 21.5f, 17.8f, 16.0f, 22.1f};

    float sum = 0.0f;
    float max = temperatures[0];
    float min = temperatures[0];

    for (int i = 0; i < DAYS; i++) {
```

```
        sum += temperatures[i];
        if (temperatures[i] > max) max = temperatures[i];
        if (temperatures[i] < min) min = temperatures[i];
    }

    printf("Average: %.2f\n", sum / DAYS);
    printf("Maximum: %.2f\n", max);
    printf("Minimum: %.2f\n", min);
    return 0;
}
```

Student Scores (Parallel Arrays)

```
#include <stdio.h>

#define STUDENTS 4

int main(void) {
    const char *names[STUDENTS] = {"Ada", "Linus", "Grace", "Ken"};
    float grades[STUDENTS] = {94.0f, 88.5f, 97.0f, 91.0f};

    int top = 0;
    float sum = 0.0f;
    for (int i = 0; i < STUDENTS; i++) {
        sum += grades[i];
        if (grades[i] > grades[top]) {
            top = i;
        }
    }

    printf("Top: %s (%.1f)\n", names[top], grades[top]);
    printf("Average: %.2f\n", sum / STUDENTS);
    return 0;
}
```

Mini-Lab: Array Toolkit

Implement a single file `array_lab.c` that:

1. Reads up to 32 integers (first number is `n`, then `n` values), **or** uses a built-in sample if you prefer non-interactive code.
2. Prints the array.
3. Prints min, max, and sum.
4. Prints a frequency table for values in `0..9` (ignore others).

5. Reverses a copy of the array and prints it (leave the original intact).

Skeleton:

```c
#include <stdio.h>
#include <string.h>

enum { MAX_N = 32 };

static void print_arr(const int *a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d%s", a[i], (i + 1 < n) ? " " : "\n");
    }
}

/* Exercise: implement minmax, sum, freq, reverse_into */

int main(void) {
    int sample[] = {1, 4, 1, 5, 9, 2, 6, 5};
    int n = (int)(sizeof sample / sizeof sample[0]);
    int copy[MAX_N];

    printf("data: ");
    print_arr(sample, n);

    /* Fill in the rest for the lab. */
    (void)copy;
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra array_lab.c -o array_lab
./array_lab
```

Exercises

1. **Rotate left by `k`.** Given an array and `k`, rotate elements left by `k` positions in place (or via a temp buffer). Test with `k = 0`, `k = n`, and `k > n` (normalize with `k % n`).
2. **Second largest.** Return the second-largest distinct value, or document a clear error if it does not exist.
3. **Merge sorted arrays.** Given two sorted `int` arrays, produce a third sorted array of length `n + m`.
4. **Saddle point.** In a small matrix, find an element that is the minimum of its row and maximum of its column (if any).

5. **Sparse frequency.** Count frequencies without assuming values are in `0..9` — use a parallel "value / count" table or sort-then-scan.
6. **Bounds-safe wrapper.** Write `int at(const int *a, size_t n, size_t i, int *out)` that returns `0` on success and `-1` if `i >= n`, never reading out of range.
7. **Row and column sums.** For a fixed `R×C` matrix, print each row sum and each column sum.
8. **Mini-lab completion.** Finish `array_lab.c` from above and add selection sort on a copy.

Summary

1. Arrays are contiguous same-type storage with zero-based indices.
2. C does not bounds-check — you own every index.
3. Arrays decay to pointers at function boundaries; **pass the length**.
4. Multidimensional arrays are row-major; `a[i][j]` sits at `i * cols + j` in flat form.
5. Classic operations — reverse, min/max, frequency, matrix multiply, histograms — are loops plus careful indexing.
6. Common bugs: off-by-one, `sizeof` on parameters, uninitialized data, illegal array assignment, dangling returns.

Solid array discipline is the base for strings, pointers, and almost every algorithm you will write in C.

Strings

Introduction

In C, a string is a contiguous sequence of characters ending with a null terminator `'\0'`. There is no built-in string type: you work with `char` arrays and `char *`, plus the helpers in `<string.h>` and careful buffer management.

This chapter covers representation, standard functions, **safe** copy/format patterns, DIY `strlen` / `strcpy` / `strcmp`, CSV parsing with `strtok_r` and manual scanners, buffer overflow awareness with safe alternatives, and exercises.

Compile with:

```
gcc -std=c17 -Wall -Wextra program.c -o program
```

String Representation

Character Arrays and Literals

```
char str1[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
char str2[6] = "Hello";   /* same content; size includes '\0' */
char str3[]  = "Hello";   /* size 6 inferred */

char *lit = "Hello";      /* pointer to string literal - treat as read-only */
char buf[] = "Hello";     /* mutable copy in your array */
```

Memory Layout

```
char s[] = "Hi";

 index:  0    1    2

        'H'  'i'  '\0'

 length strlen(s) == 2
 storage size sizeof s == 3
```

Literals Must Not Be Modified

```
char *p = "Hello";
/* p[0] = 'h'; */   /* undefined behavior - may crash */

char a[] = "Hello";
a[0] = 'h';         /* OK - a is a mutable array */
```

Prefer `const char *p = "Hello";` so the compiler helps you avoid writes.

Declaration Patterns

```
char name[64] = "Ada";     /* modifiable, fixed capacity */
char empty[32] = "";       /* just '\0' */
char raw[32];              /* uninitialized - do not read as a string yet */
char *msg = "ready";       /* points at literal */

#include <string.h>
memset(raw, 0, sizeof raw);  /* explicit clear */
```

Standard Library Overview

```
#include <string.h>

size_t strlen(const char *s);
char  *strcpy(char *dst, const char *src);           /* unsafe if dst too small */
char  *strncpy(char *dst, const char *src, size_t n);/* subtle; see below */
int    strcmp(const char *a, const char *b);
int    strncmp(const char *a, const char *b, size_t n);
char  *strcat(char *dst, const char *src);           /* unsafe if dst too small */
char  *strncat(char *dst, const char *src, size_t n);
char  *strchr(const char *s, int c);
char  *strstr(const char *haystack, const char *needle);
```

Also useful: `snprintf` from `<stdio.h>` for bounded formatting into a buffer.

Safe String Handling Patterns

Prefer `snprintf` over bare `strcpy` / `strcat`

`snprintf` always null-terminates when the buffer size is greater than zero, and it tells you how many characters would have been written.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[16];
    const char *user = "Ada Lovelace";

    int n = snprintf(buf, sizeof buf, "hi %s", user);
    if (n < 0) {
        /* encoding error (rare) */
        return 1;
    }
    if ((size_t)n >= sizeof buf) {
        /* truncated - buf is still a valid C string */
        printf("truncated: \"%s\" (needed %d bytes incl. nul logic)\n", buf, n + 1);
    } else {
        printf("full: \"%s\"\n", buf);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra snip.c -o snip
./snip
```

Building Paths or Messages Safely

```c
#include <stdio.h>

static int join3(char *out, size_t out_sz,
                 const char *a, const char *b, const char *c) {
    int n = snprintf(out, out_sz, "%s/%s/%s", a, b, c);
    if (n < 0) {
        return -1;
    }
    if ((size_t)n >= out_sz) {
        return -2;  /* truncated */
    }
    return 0;
}

int main(void) {
    char path[32];
    if (join3(path, sizeof path, "home", "ada", "notes.txt") == 0) {
        printf("%s\n", path);
    }
```

```
    return 0;
}
```

`strncpy` vs `snprintf` — Know the Trap

`strncpy` is **not** a "safe strcpy." It pads with zeros up to `n`, and if `src` length is `n`, it **does not** write a `'\0'`.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char dest[8];
    const char *src = "OVERFLOW!";  /* longer than dest */

    /* TRAP: dest may be not null-terminated */
    strncpy(dest, src, sizeof dest);
    /* dest has 8 chars, no '\0' - using it as a string is UB */

    /* Repair pattern if you must use strncpy: */
    dest[sizeof dest - 1] = '\0';

    /* BETTER: */
    char dest2[8];
    snprintf(dest2, sizeof dest2, "%s", src);  /* always terminated if size > 0 */

    printf("dest2 = \"%s\"\n", dest2);
    return 0;
}
```

Bounded Copy Helper (Recommended Pattern)

```
#include <stdio.h>
#include <string.h>

/* Returns 0 on full copy, -1 on truncation or bad args. Always terminates if dst_sz > 0. */
static int str_copy(char *dst, size_t dst_sz, const char *src) {
    if (dst == NULL || src == NULL || dst_sz == 0) {
        return -1;
    }
    int n = snprintf(dst, dst_sz, "%s", src);
    if (n < 0 || (size_t)n >= dst_sz) {
        return -1;
    }
```

```
    return 0;
}

int main(void) {
    char b[8];
    if (str_copy(b, sizeof b, "hello") == 0) {
        printf("ok: %s\n", b);
    }
    if (str_copy(b, sizeof b, "toolong!") != 0) {
        printf("truncated-or-fail: \"%s\"\n", b);
    }
    return 0;
}
```

Safe Concatenation

```
#include <stdio.h>
#include <string.h>

static int str_append(char *dst, size_t dst_sz, const char *src) {
    if (dst == NULL || src == NULL || dst_sz == 0) {
        return -1;
    }
    size_t used = strlen(dst);
    if (used >= dst_sz) {
        return -1;  /* dst already not a valid sized string */
    }
    int n = snprintf(dst + used, dst_sz - used, "%s", src);
    if (n < 0 || (size_t)n >= dst_sz - used) {
        return -1;
    }
    return 0;
}

int main(void) {
    char msg[32] = "Hello";
    str_append(msg, sizeof msg, ", ");
    str_append(msg, sizeof msg, "world");
    printf("%s\n", msg);
    return 0;
}
```

Input: Prefer `fgets` + Strip Newline

```c
#include <stdio.h>
#include <string.h>

static void strip_newline(char *s) {
    size_t n = strlen(s);
    if (n > 0 && s[n - 1] == '\n') {
        s[n - 1] = '\0';
    }
}

int main(void) {
    char line[64];

    /* BAD: scanf("%s") - no bounds unless you set a width */
    /* BAD: gets - removed from C; never use */

    printf("name: ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 1;
    }
    strip_newline(line);
    printf("hello, %s\n", line);
    return 0;
}
```

If you use `scanf` for a single token, always set a maximum field width: `scanf("%63s", line)` for a 64-byte buffer.

Implement `strlen` / `strcpy` / `strcmp` Yourself

Writing these clarifies how C strings work. Production code should still prefer the standard library and bounded helpers.

Full Learning Program

```c
#include <stdio.h>
#include <assert.h>

static size_t my_strlen(const char *s) {
    const char *p = s;
    while (*p != '\0') {
        p++;
```

```
    }
    return (size_t)(p - s);
}

/* Classic unbounded copy - destination must be large enough (like strcpy). */
static char *my_strcpy(char *dst, const char *src) {
    char *out = dst;
    while ((*dst++ = *src++) != '\0') {
        /* copy including '\0' */
    }
    return out;
}

/* Like strcmp: <0, 0, >0 */
static int my_strcmp(const char *a, const char *b) {
    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}

/* Bounded copy: always writes '\0' if dst_sz > 0. Returns total length of src
   (as snprintf does conceptually) so caller can detect truncation. */
static size_t my_strlcpy(char *dst, const char *src, size_t dst_sz) {
    size_t src_len = my_strlen(src);
    if (dst_sz == 0) {
        return src_len;
    }
    size_t copy = (src_len < dst_sz - 1) ? src_len : dst_sz - 1;
    for (size_t i = 0; i < copy; i++) {
        dst[i] = src[i];
    }
    dst[copy] = '\0';
    return src_len;
}

int main(void) {
    const char *s = "Hello";
    assert(my_strlen(s) == 5);
    assert(my_strlen("") == 0);

    char buf[16];
    my_strcpy(buf, s);
    assert(my_strcmp(buf, "Hello") == 0);
    assert(my_strcmp("abc", "abd") < 0);
```

```
    assert(my_strcmp("abd", "abc") > 0);

    char small[4];
    size_t need = my_strlcpy(small, "Hello", sizeof small);
    printf("small=\"%s\" need=%zu truncated=%s\n",
           small, need, (need >= sizeof small) ? "yes" : "no");

    printf("all my_string tests passed\n");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra my_string.c -o my_string
./my_string
```

Notes:

- Cast to `unsigned char` in compare so high-bit characters sort consistently.
- Unbounded `my_strcpy` is for learning; use `my_strlcpy` / `snprintf` in real code.

Tokenizing and CSV Parsing

`strtok` vs `strtok_r`

`strtok` keeps hidden static state — **not reentrant**, awkward in libraries and threads. Prefer `strtok_r` (POSIX) with an explicit context pointer, or parse manually for full control.

Full Program: CSV Line with `strtok_r`

```
#include <stdio.h>
#include <string.h>

enum { MAX_FIELDS = 16, FIELD_LEN = 64 };

/*
 * Split a mutable CSV line on commas into fields[][].
 * Note: strtok_r mutates the line (writes '\0' over delimiters).
 * Returns field count, or -1 on overflow of MAX_FIELDS.
 */
static int split_csv_strtok_r(char *line, char fields[][FIELD_LEN], int max_fields) {
    int count = 0;
    char *save = NULL;
    char *tok = strtok_r(line, ",", &save);

    while (tok != NULL) {
```

```
        if (count >= max_fields) {
            return -1;
        }
        /* Bounded copy into fixed field buffer */
        snprintf(fields[count], FIELD_LEN, "%s", tok);
        count++;
        tok = strtok_r(NULL, ",", &save);
    }
    return count;
}

int main(void) {
    char line[] = "ada,lovelace,36,math";  /* mutable */
    char fields[MAX_FIELDS][FIELD_LEN];

    int n = split_csv_strtok_r(line, fields, MAX_FIELDS);
    if (n < 0) {
        fprintf(stderr, "too many fields\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("field[%d] = \"%s\"\n", i, fields[i]);
    }
    /* Warning: original line is now shredded into tokens */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra csv_strtok.c -o csv_strtok
./csv_strtok
```

On platforms without `strtok_r`, use the manual parser below (portable C17).

Full Program: Manual CSV Parse (No `strtok`)

Does not need POSIX; leaves the original string intact if you copy fields out.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

enum { MAX_FIELDS = 16, FIELD_LEN = 64 };

static void trim_inplace(char *s) {
    /* leading */
    char *start = s;
```

```
    while (*start != '\0' && isspace((unsigned char)*start)) {
        start++;
    }
    if (start != s) {
        memmove(s, start, strlen(start) + 1);
    }
    /* trailing */
    size_t n = strlen(s);
    while (n > 0 && isspace((unsigned char)s[n - 1])) {
        s[--n] = '\0';
    }
}

/*
 * Parse comma-separated fields from src into fields.
 * Returns count or -1 if a field would not fit FIELD_LEN or too many fields.
 */
static int split_csv_manual(const char *src,
                            char fields[][FIELD_LEN],
                            int max_fields) {
    int count = 0;
    const char *p = src;

    while (*p != '\0') {
        if (count >= max_fields) {
            return -1;
        }
        /* skip leading spaces in field */
        while (*p != '\0' && *p != ',' && isspace((unsigned char)*p)) {
            p++;
        }
        const char *start = p;
        while (*p != '\0' && *p != ',') {
            p++;
        }
        size_t len = (size_t)(p - start);
        /* trim trailing spaces from the slice */
        while (len > 0 && isspace((unsigned char)start[len - 1])) {
            len--;
        }
        if (len + 1 > FIELD_LEN) {
            return -1;
        }
        memcpy(fields[count], start, len);
        fields[count][len] = '\0';
        count++;
```

```
        if (*p == ',') {
            p++;  /* consume comma */
        }
    }
    return count;
}

int main(void) {
    const char *line = " ada , lovelace , 36 , math ";
    char fields[MAX_FIELDS][FIELD_LEN];

    int n = split_csv_manual(line, fields, MAX_FIELDS);
    if (n < 0) {
        fprintf(stderr, "parse error\n");
        return 1;
    }
    printf("original still: \"%s\"\n", line);
    for (int i = 0; i < n; i++) {
        printf("field[%d] = \"%s\"\n", i, fields[i]);
    }
    (void)trim_inplace; /* available for whole-string trimming if needed */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra csv_manual.c -o csv_manual
./csv_manual
```

Simple Key=Value Line

```
#include <stdio.h>
#include <string.h>

/* Parse "key=value" into key/value buffers. Returns 0 on success. */
static int parse_kv(const char *line,
                    char *key, size_t key_sz,
                    char *val, size_t val_sz) {
    const char *eq = strchr(line, '=');
    if (eq == NULL) {
        return -1;
    }
    size_t klen = (size_t)(eq - line);
    if (klen + 1 > key_sz) {
        return -1;
    }
    memcpy(key, line, klen);
```

```
    key[klen] = '\0';

    if (snprintf(val, val_sz, "%s", eq + 1) < 0) {
        return -1;
    }
    if (strlen(eq + 1) >= val_sz) {
        return -1;  /* truncated */
    }
    return 0;
}

int main(void) {
    char k[32], v[64];
    if (parse_kv("port=8080", k, sizeof k, v, sizeof v) == 0) {
        printf("key=\"%s\" val=\"%s\"\n", k, v);
    }
    return 0;
}
```

Buffer Overflows: Awareness and Safe Alternatives

A **buffer overflow** happens when code writes past the end of an array. With strings, the usual causes are unbounded copies, missing null terminators, and `scanf("%s")` without a width.

This section is **defensive**. Do not use these ideas to attack systems. The goal is to recognize fragile patterns and replace them with bounded APIs.

Fragile Pattern (Do Not Ship)

```
/* ILLUSTRATION ONLY - unbounded write into a fixed buffer */
void fragile_copy(char *dst, const char *src) {
    while (*src) {
        *dst++ = *src++;  /* no length check */
    }
    *dst = '\0';
}
```

If `src` is longer than the space behind `dst`, memory after the buffer is overwritten. That can crash the program or corrupt other data. Treat it as a bug to eliminate.

Safe Alternatives Checklist

Fragile	Prefer
`gets`	`fgets(buf, sizeof buf, stdin)`
`scanf("%s", buf)`	`scanf("%63s", buf)` or `fgets`
`strcpy(dst, src)`	`snprintf(dst, dst_sz, "%s", src)`
`strcat(dst, src)`	`str_append` helper with remaining size
`strncpy` without forced `'\0'`	`snprintf` or copy + `dst[sz-1]='\0'`
`sprintf(dst, …)`	`snprintf(dst, dst_sz, …)`
DIY loops without `n`	index loops with `i + 1 < sz`

Full Program: Safe Line Reader and Copy

```c
#include <stdio.h>
#include <string.h>

enum { LINE_MAX = 128 };

static int read_line(char *buf, size_t buf_sz) {
    if (buf == NULL || buf_sz == 0) {
        return -1;
    }
    if (fgets(buf, (int)buf_sz, stdin) == NULL) {
        return -1;
    }
    size_t n = strlen(buf);
    if (n > 0 && buf[n - 1] == '\n') {
        buf[n - 1] = '\0';
    } else {
        /* Line may have been longer than buffer - drain rest of line */
        int c;
        int overflow = (n == buf_sz - 1);
        if (overflow) {
            while ((c = getchar()) != '\n' && c != EOF) {
                /* discard */
            }
            return -2;  /* truncated */
        }
    }
    return 0;
}

int main(void) {
    char name[LINE_MAX];
    char greeting[LINE_MAX];

    printf("Enter your name: ");
```

```
    int rc = read_line(name, sizeof name);
    if (rc == -1) {
        fprintf(stderr, "no input\n");
        return 1;
    }
    if (rc == -2) {
        fprintf(stderr, "input too long (truncated)\n");
    }

    if (snprintf(greeting, sizeof greeting, "Hello, %s!", name) < 0) {
        return 1;
    }
    printf("%s\n", greeting);
    return 0;
}
```

Compiler Help

```
gcc -std=c17 -Wall -Wextra -Wformat -Wformat-security program.c -o program
```

Some platforms also offer fortified library functions when optimization is on (`_FORTIFY_SOURCE`). Still write correct sizes yourself — fortification is a backstop, not a design.

More String Techniques

Remove Newline, Reverse, Case

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

static void strip_newline(char *s) {
    size_t n = strlen(s);
    if (n > 0 && s[n - 1] == '\n') {
        s[n - 1] = '\0';
    }
}

static void reverse_inplace(char *s) {
    size_t n = strlen(s);
    for (size_t i = 0; i < n / 2; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
```

```
        s[n - 1 - i] = t;
    }
}

static void to_lower_inplace(char *s) {
    for (; *s; s++) {
        *s = (char)tolower((unsigned char)*s);
    }
}

int main(void) {
    char s[] = "AbCdeF";
    to_lower_inplace(s);
    reverse_inplace(s);
    printf("%s\n", s);  /* fedcba */
    (void)strip_newline;
    return 0;
}
```

Palindrome Check

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

static int is_palindrome(const char *str) {
    size_t i = 0;
    size_t j = strlen(str);
    if (j == 0) {
        return 1;
    }
    j--;
    while (i < j) {
        while (i < j && !isalnum((unsigned char)str[i])) i++;
        while (i < j && !isalnum((unsigned char)str[j])) j--;
        if (tolower((unsigned char)str[i]) != tolower((unsigned char)str[j])) {
            return 0;
        }
        i++;
        j--;
    }
    return 1;
}

int main(void) {
```

```
    printf("%d\n", is_palindrome("A man, a plan, a canal: Panama")); /* 1 */
    printf("%d\n", is_palindrome("hello")); /* 0 */
    return 0;
}
```

Word Count

```
#include <stdio.h>
#include <ctype.h>

static int count_words(const char *str) {
    int count = 0;
    int in_word = 0;
    for (size_t i = 0; str[i] != '\0'; i++) {
        if (isspace((unsigned char)str[i])) {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

int main(void) {
    printf("%d\n", count_words("  hello   c strings "));  /* 3 */
    return 0;
}
```

Character Class Statistics

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

static void analyze(const char *str) {
    size_t letters = 0, digits = 0, spaces = 0, others = 0;
    for (size_t i = 0; str[i] != '\0'; i++) {
        unsigned char c = (unsigned char)str[i];
        if (isalpha(c)) letters++;
        else if (isdigit(c)) digits++;
        else if (isspace(c)) spaces++;
        else others++;
    }
```

```
    printf("letters=%zu digits=%zu spaces=%zu others=%zu len=%zu\n",
           letters, digits, spaces, others, strlen(str));
}

int main(void) {
    analyze("Hello, C17! 42");
    return 0;
}
```

Pitfalls Recap

1. **Forgetting `'\0'`** after manual copies.
2. **`sizeof` on `char *`** — that is pointer size, not string length.
3. **Modifying string literals.**
4. **`strncpy` without terminator.**
5. **Using `strtok` in concurrent or nested tokenization** — prefer `strtok_r` or manual parse.
6. **Assuming `scanf("%s")` is safe** without a width.

Exercises

1. **Bounded library.** Implement `str_copy`, `str_append`, and `str_eq` (boolean compare) with full size parameters and tests.
2. **DIY suite.** Write `my_strlen`, `my_strcmp`, `my_strncmp`, and `my_strlcpy`. Compare results against the C library for a table of sample strings.
3. **CSV grades.** Parse lines like `name,score` from a hardcoded multi-line string; compute average score. Use the manual splitter (no `strtok`).
4. **`strtok_r` word counter.** Tokenize on whitespace with `strtok_r` and count tokens. Then reimplement without mutating a second copy.
5. **Safe login prompt.** Read username and password with `fgets` into fixed buffers; reject truncated lines; never use `gets` or unbounded `scanf`.
6. **URL path join.** `join_url(char *out, size_t n, const char *base, const char *path)` using `snprintf`, handling trailing/leading slashes cleanly.
7. **Find and replace first occurrence.** Replace the first occurrence of a substring with another (same length for simplicity, or build into a second buffer with size checks).
8. **Overflow hunt.** Take a small program that uses `strcpy`/`sprintf` and rewrite every call site to a bounded form. List each change in comments.

Summary

1. C strings are **`char` sequences with `'\0'`** — capacity and length are not the same.
2. Prefer **`snprintf`**, **`fgets`**, and explicit sizes over `strcpy` / `gets` / bare `scanf("%s")`.
3. **`strncpy` is easy to misuse**; force termination or switch to `snprintf`.
4. Implementing **`strlen` / `strcpy` / `strcmp`** teaches the model; ship bounded wrappers.
5. **`strtok_r` or manual parsing** beats classic `strtok` for reentrancy and clarity.
6. Treat buffer overflows as **bugs to prevent** with correct sizes and safer APIs — never as something to exploit.

With disciplined buffer handling, C string code can be both efficient and robust.

Bounded String Copies: strncpy, strlcpy, memccpy and strscpy Compared

Bounded String Copies: strncpy, strlcpy, memccpy and strscpy Compared

After this page you will know exactly which bytes `strncpy`, `strlcpy`, `memccpy`, `snprintf` and a kernel-style `strscpy` write, how each one tells you about truncation, and which one to use when.

The problem: truncation, termination, and detection

Copying a C string into a fixed-size buffer sounds simple, but string operations in C present three distinct requirements that standard library functions fulfill differently:

1. **Guaranteed null termination:** Ensuring that the destination buffer always ends with a `'\0'` byte when `size > 0`, preventing subsequent read operations like `strlen` or `printf` from scanning past the allocated memory.
2. **Controlled bounds and safety:** Never writing past the allocated byte capacity of the destination buffer, and avoiding unnecessary reads past unmapped memory boundaries when processing untrusted input.
3. **Explicit truncation detection:** Providing a simple, unambiguous way for the caller to determine whether the source string fit entirely inside the destination buffer or was truncated.

`strcpy` has no bound at all. The bounded function that has been in ISO C since C89, `strncpy`, was designed for fixed-width, NUL-padded record fields, not for C strings. So it leaves the result unterminated when the source doesn't fit, and it writes padding NULs you didn't ask for when the source is short.

The alternatives are `strlcpy` (from OpenBSD, in glibc since 2.38 and in POSIX.1-2024), `memccpy` (POSIX for a long time, ISO C since C23), `snprintf`, and the Linux kernel's `strscpy`. They differ in whether they terminate, how far they read, how they report truncation and where they are available.

The lab

The program below fills an 8-byte buffer with `#` before each call, so bytes a function did not touch stay visible, and prints NUL bytes as `\0`. `strscpy` is a kernel API, not part of any libc, so `my_strscpy` is a small userspace equivalent written for this lab.

```
#define _DEFAULT_SOURCE /* glibc 2.38+ declares strlcpy under this */
#include <errno.h>
#include <stdio.h>
```

```c
#include <string.h>
#include <sys/types.h>

/* Kernel-style strscpy: copy at most size-1 bytes, always terminate
 * (when size > 0), return the copied length or -E2BIG on truncation. */
static ssize_t my_strscpy(char *dst, const char *src, size_t size) {
    if (size == 0)
        return -E2BIG;
    size_t i = 0;
    for (; i < size - 1 && src[i] != '\0'; i++)
        dst[i] = src[i];
    dst[i] = '\0';
    return src[i] == '\0' ? (ssize_t)i : -E2BIG;
}

static void show(const char *label, const char *buf, size_t n) {
    printf("%-9s [", label);
    for (size_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)buf[i];
        if (c == '\0')
            printf("\\0");
        else if (c == '#')
            printf("#");
        else
            putchar(c);
    }
    printf("]\n");
}

int main(void) {
    const char *src = "hostname-too-long";
    char buf[8];

    /* strncpy: pads with NULs when short, does NOT terminate when long. */
    memset(buf, '#', sizeof buf);
    strncpy(buf, "ok", sizeof buf);
    show("strncpy", buf, sizeof buf);
    memset(buf, '#', sizeof buf);
    strncpy(buf, src, sizeof buf);
    show("strncpy", buf, sizeof buf);
    printf("          terminated? %s\n", memchr(buf, '\0', sizeof buf) ? "yes" : "NO");

    /* strlcpy: always terminates, returns strlen(src) to detect truncation. */
    memset(buf, '#', sizeof buf);
    size_t need = strlcpy(buf, src, sizeof buf);
    show("strlcpy", buf, sizeof buf);
    printf("          returned %zu, truncated? %s\n", need, need >= sizeof buf ? "yes" : "no
```

```
    /* memccpy (C23): stops after copying the terminator, returns NULL if absent. */
    memset(buf, '#', sizeof buf);
    char *end = memccpy(buf, "ok", '\0', sizeof buf);
    show("memccpy", buf, sizeof buf);
    printf("          returned %s\n", end ? "pointer past the NUL" : "NULL");
    memset(buf, '#', sizeof buf);
    end = memccpy(buf, src, '\0', sizeof buf);
    if (!end)
        buf[sizeof buf - 1] = '\0'; /* caller must terminate on truncation */
    show("memccpy", buf, sizeof buf);
    printf("          returned %s\n", end ? "pointer past the NUL" : "NULL (truncated)");

    /* snprintf: always terminates, returns the length it wanted to write. */
    memset(buf, '#', sizeof buf);
    int want = snprintf(buf, sizeof buf, "%s", src);
    show("snprintf", buf, sizeof buf);
    printf("          returned %d, truncated? %s\n", want, (size_t)want >= sizeof buf ? "yes

    /* strscpy-style: length or -E2BIG. */
    memset(buf, '#', sizeof buf);
    ssize_t r = my_strscpy(buf, src, sizeof buf);
    show("strscpy", buf, sizeof buf);
    printf("          returned %zd (%s)\n", r, r == -E2BIG ? "-E2BIG" : "length");
    memset(buf, '#', sizeof buf);
    r = my_strscpy(buf, "ok", sizeof buf);
    show("strscpy", buf, sizeof buf);
    printf("          returned %zd\n", r);
    return 0;
}
```

It was compiled with gcc 14.2.0 and glibc 2.41. `_DEFAULT_SOURCE` at the top is needed because glibc only declares `strlcpy` under it (or `_GNU_SOURCE`) when you ask for strict `-std=c23`:

```
gcc -std=c23 -Wall -Wextra -O2 -o copies copies.c
```

GCC warned about two of the calls (more on that below):

```
copies.c: In function ‘main’:
copies.c:42:5: warning: ‘strncpy’ output truncated copying 8 bytes from a string of length 1
   42 |     strncpy(buf, src, sizeof buf);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~
copies.c:66:43: warning: ‘%s’ directive output truncated writing 17 bytes into a region of s
   34 |     const char *src = "hostname-too-long";
      |                       ~~~~~~~~~~~~~~~~~~~
......
   66 |     int want = snprintf(buf, sizeof buf, "%s", src);
```

```
      |                                                 ^~
copies.c:66:16: note: ‘snprintf’ output 18 bytes into a destination of size 8
   66 |     int want = snprintf(buf, sizeof buf, "%s", src);
      |                ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

Running `./copies` printed:

```
strncpy   [ok\0\0\0\0\0\0]
strncpy   [hostname]
          terminated? NO
strlcpy   [hostnam\0]
          returned 17, truncated? yes
memccpy   [ok\0#####]
          returned pointer past the NUL
memccpy   [hostnam\0]
          returned NULL (truncated)
snprintf  [hostnam\0]
          returned 17, truncated? yes
strscpy   [hostnam\0]
          returned -7 (-E2BIG)
strscpy   [ok\0#####]
          returned 2
```

The build is otherwise clean, and it also runs clean under `-fsanitize=address,undefined`.

Reading the results byte by byte

Understanding how each function behaves requires looking closely at the 8-byte destination array `buf` after each copy operation.

`strncpy`

When copying `"ok"` (2 characters plus NUL) into the 8-byte buffer, `strncpy` writes `'o'`, `'k'`, and then fills the remaining 6 bytes with `'\0'`. This padding ensures fixed-width record fields are completely zeroed out, preventing leftover garbage memory from leaking into disk structures or network packets.

However, when copying `"hostname-too-long"` (17 characters plus NUL) into the 8-byte buffer, `strncpy` writes exactly 8 bytes (`"hostname"`) and stops immediately without appending a `'\0'`. The memory contains no null byte anywhere inside the 8-byte boundary. Passing this non-terminated buffer to `printf("%s")` or `strlen` results in an out-of-bounds read until an arbitrary zero byte is encountered in surrounding memory. Furthermore, `strncpy` returns `dst` (a pointer to `buf`), providing no signal to the caller that the string was cut off.

strlcpy

When copying "hostname-too-long" into buf, strlcpy copies 7 bytes ("hostnam"), writes '\0' into the 8th position, and returns 17 (the length of the source string strlen(src)).

Because the returned length (17) is greater than or equal to the buffer capacity (8), the caller easily detects truncation by checking return_value >= sizeof(buf). However, to compute that return value, strlcpy must scan through all 17 characters of src to find its terminating null byte, even though it only writes 8 bytes into buf. If src is an untrusted or non-null-terminated buffer, strlcpy will read memory past the intended bounds.

memccpy

The memccpy function searches for a stop character (here '\0') while copying up to n bytes. When copying "ok", it copies 'o', 'k', '\0', stops immediately without padding the remaining 5 bytes, and returns a pointer to the byte directly after '\0' in buf.

When copying "hostname-too-long" into the 8-byte buffer, memccpy copies 8 bytes ("hostname") without finding '\0'. It stops and returns NULL. The destination buffer is left unterminated, but because NULL clearly signals truncation, the caller can explicitly set buf[sizeof(buf) - 1] = '\0' to guarantee safety. Crucially, memccpy never reads past the specified limit of 8 bytes in src.

snprintf

Calling snprintf(buf, sizeof(buf), "%s", src) copies 7 bytes, appends '\0' at index 7, and returns 17 (the total number of characters that would have been written had space been infinite).

While snprintf guarantees null termination, formatting engines introduce extra CPU overhead compared to direct string byte copies. Like strlcpy, snprintf reads the entire source string to calculate its full un-truncated return length.

strscpy

The kernel-style function my_strscpy copies at most size - 1 bytes and guarantees null termination whenever size > 0. When copying "hostname-too-long" into buf, it writes "hostnam\0" (7 characters plus NUL) and returns -E2BIG, which prints as -7 because E2BIG is 7 on Linux.

When copying "ok", my_strscpy writes 'o', 'k', '\0', leaves untouched bytes (#) intact, and returns 2 (the number of non-null characters copied). It reads at most size bytes of the source and stops at its terminator, so an unterminated source can't make it run off the end.

Function comparison table

The table below summarizes the key operational differences between the five copy mechanisms.

Function	Guaranteed `'\0'`?	Zero-Pads Buffer?	Reads Past `size` Bytes?	How Truncation Is Detected	Standard / Standardized In
`strncpy(dst, src, n)`	No (if `strlen(src) >= n`)	Yes (fills rest with `\0`)	No	Cannot be detected from return value alone (returns `dst`)	C89 / POSIX.1-2001
`strlcpy(dst, src, size)`	Yes (when `size > 0`)	No	Yes (scans full `strlen(src)`)	`return_value >= size`	POSIX.1-2024, BSDs, glibc 2.38+; not ISO C
`memccpy(dst, src, c, n)`	No (caller must set `dst[n-1] = '\0'` on `NULL`)	No	No	`return_value == NULL`	ISO C23; POSIX (XSI) before that
`snprintf(dst, size, "%s", src)`	Yes (when `size > 0`)	No	Yes (scans full `strlen(src)`)	`return_value >= size`	C99 / POSIX.1-2001
`strscpy(dst, src, size)`	Yes (when `size > 0`)	No	No	`return_value == -E2BIG`	Linux kernel only; write your own helper in userspace

Choosing the right copy function

Select string copy implementations based on your buffer boundary requirements and input trust level:

- **Fixed-width, NUL-padded record fields:** Use `strncpy` intentionally. This is appropriate for legacy binary record structures (such as database headers or fixed tar archive headers) where trailing bytes must be zero-cleared and strings do not rely on standard null termination.
- **General null-terminated C string copying (trusted input):** Use `strlcpy` (POSIX.1-2024, glibc 2.38+, the BSDs) or, where that isn't available, `snprintf`. Both functions guarantee null termination and allow simple truncation checks via `return_value >= size`.
- **Untrusted or potentially unterminated input:** Use `memccpy` or a `strscpy`-style helper. Because these functions stop reading as soon as either the destination capacity or the source null terminator is reached, they prevent out-of-bounds read vulnerabilities when handling raw network buffers or unbounded user inputs.
- **Whatever you pick:** check for truncation and treat it as an error. A truncated path, hostname or key is a different value, not a shorter version of the same one. The Linux kernel has deprecated `strncpy` and `strlcpy` in favour of `strscpy` for these reasons (see its deprecated-interfaces document).

Compiler warnings help detect misuse

Modern compilers help detect string buffer misuse statically. GCC has two warnings for this, both turned on by `-Wall`:

1. **`-Wstringop-truncation`:** Warns when `strncpy` is called with a destination bound equal to or smaller than the source string length, indicating that the destination will lack a terminating null byte.
2. **`-Wformat-truncation`:** Warns when calls to `snprintf` can be proven at compile time to result in truncated output.

When compiling `copies.c`, GCC emits diagnostic warnings for the `strncpy` call copying `src` into `buf`, as well as the `snprintf` call with a fixed 8-byte buffer. Using `-Werror` during build integration converts these warnings into build failures, forcing developers to fix improper string size assumptions.

Common mistakes

1. Passing total buffer size to `strncat` instead of remaining space

Unlike `strncpy` or `strlcpy`, the third parameter of `strncat(dst, src, count)` is the *maximum number of bytes to append*, not the total capacity of `dst`.

```
/* BUG: strncat appends up to sizeof(buf) bytes AFTER dst's current content */
char buf[16] = "hello";
strncat(buf, extra, sizeof(buf)); /* WRONG: result can need 5 + 16 + 1 = 22 bytes */

/* CORRECT: pass remaining available space minus space for '\0' */
size_t len = strlen(buf);
if (len < sizeof(buf)) {
    strncat(buf, extra, sizeof(buf) - len - 1);
}
```

2. Calling `sizeof` on a pointer parameter

Using `sizeof` on a function parameter declared as `char dst[]` or `char *dst` evaluates to the size of the pointer (typically 8 bytes on 64-bit architectures), not the size of the underlying array buffer.

```
void copy_path(char dst[256], const char *src) {
    /* BUG: sizeof(dst) evaluates to sizeof(char *), which is 8, not 256 */
    strlcpy(dst, src, sizeof(dst));
}

/* CORRECT: pass capacity explicitly */
void copy_path_correct(char *dst, size_t cap, const char *src) {
```

```
    strlcpy(dst, src, cap);
}
```

3. Ignoring the return value of `strlcpy` or `snprintf`

Calling `strlcpy` without checking its return value guarantees that the string is terminated, but ignores data loss.

```
/* BUG: Truncation occurs silently if src > sizeof(buf) */
strlcpy(buf, src, sizeof(buf));

/* CORRECT: Check return value for truncation */
size_t ret = strlcpy(buf, src, sizeof(buf));
if (ret >= sizeof(buf)) {
    /* Handle truncation error */
}
```

4. Off-by-one errors with `size` versus `size - 1`

Functions like `strlcpy`, `snprintf`, and `strscpy` take the full capacity of the buffer (`sizeof(buf)`), internally reserving the last byte for `'\0'`. Subtracting 1 manually when calling these functions reduces available capacity unnecessarily.

```
/* WRONG: subtracting 1 reduces 8-byte buffer capacity to 7 bytes */
strlcpy(buf, src, sizeof(buf) - 1);

/* CORRECT: pass full buffer size */
strlcpy(buf, src, sizeof(buf));
```

The `- 1` belongs only where you terminate by hand, as in the `memccpy` case: on `NULL`, write `buf[sizeof buf - 1] = '\0'`.

Key takeaways

- `strncpy` was designed for fixed-width NUL-padded fields, not C strings. It does not guarantee null termination when the source length equals or exceeds the destination size, and it zero-fills remaining buffer space.
- `strlcpy` and `snprintf` guarantee null termination when `size > 0` and simplify truncation detection (`ret >= size`), but both scan the entire source string, which can cause reading past expected bounds on untrusted input.
- `memccpy` and a `strscpy`-style helper bound both reads and writes, which makes them the right choice for untrusted or possibly unterminated input. `memccpy` leaves termination to you.
- Compiler flags `-Wall -Wextra` flag unsafe `strncpy` usage (`-Wstringop-truncation`) and format truncation (`-Wformat-truncation`).

Sources

- cppreference: strncpy
- cppreference: memccpy
- man7: strlcpy(3)
- man7: string_copying(7)
- Linux Kernel Documentation: Deprecated Interfaces
- GNU C Library release 2.38 notes
- POSIX.1-2024 strlcpy documentation

Advanced Arrays

Introduction

Advanced array concepts in C extend beyond basic one-dimensional arrays to include multi-dimensional arrays, variable-length arrays, and specialized array techniques. These concepts are essential for handling complex data structures and implementing efficient algorithms. Understanding advanced arrays enables programmers to work with matrices, tables, and other multi-dimensional data representations.

Multi-dimensional Arrays

Two-dimensional Arrays

Two-dimensional arrays are essentially arrays of arrays, often used to represent matrices or tables:

```
// Declaration
int matrix[3][4];  // 3 rows, 4 columns

// Initialization
int matrix1[2][3] = {{1, 2, 3}, {4, 5, 6}};
int matrix2[2][3] = {1, 2, 3, 4, 5, 6};  // Equivalent to above

// Partial initialization
int matrix3[3][3] = {{1, 2}, {4, 5}};  // Remaining elements set to 0

// Designated initializers (C99)
int matrix4[3][3] = {[0][0] = 1, [1][1] = 5, [2][2] = 9};
```

Accessing Multi-dimensional Arrays

```
int matrix[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};

// Access individual elements
int element = matrix[1][2];  // Element at row 1, column 2 (value: 7)

// Modify elements
matrix[0][0] = 100;
```

```c
// Traverse the array
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
```

Multi-dimensional Array Functions

When passing multi-dimensional arrays to functions, all dimensions except the first must be specified:

```c
// Function that takes a 3x4 matrix
void print_matrix(int matrix[][4], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Function with pointer notation (equivalent)
void print_matrix_ptr(int (*matrix)[4], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Usage
int matrix[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
print_matrix(matrix, 3);
```

Three-dimensional and Higher-dimensional Arrays

Three-dimensional Arrays

```
// Declaration
int cube[2][3][4];  // 2 layers, 3 rows, 4 columns

// Initialization
int cube1[2][2][2] = {
    {{1, 2}, {3, 4}},
    {{5, 6}, {7, 8}}
};

// Accessing elements
int element = cube1[1][0][1];  // Value: 6

// Traversal
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        for (int k = 0; k < 2; k++) {
            printf("cube[%d][%d][%d] = %d\n", i, j, k, cube1[i][j][k]);
        }
    }
}
```

Higher-dimensional Arrays

While C supports arrays of any dimension, higher-dimensional arrays become increasingly complex:

```
// Four-dimensional array
int hypercube[2][2][2][2];

// Accessing elements
hypercube[0][1][0][1] = 42;

// Generally, it's better to use structures or dynamic allocation for complex multi-dimensio
```

Variable-length Arrays (VLAs) - C99

Basic VLA Declaration

Variable-length arrays allow the size to be determined at runtime:

```c
#include <stdio.h>

int main() {
    int size;
    printf("Enter array size: ");
    scanf("%d", &size);

    // Variable-length array
    int array[size];  // Size determined at runtime

    // Initialize and use the array
    for (int i = 0; i < size; i++) {
        array[i] = i * i;
        printf("array[%d] = %d\n", i, array[i]);
    }

    return 0;
}
```

Multi-dimensional VLAs

```c
#include <stdio.h>

int main() {
    int rows, cols;
    printf("Enter rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Multi-dimensional VLA
    int matrix[rows][cols];

    // Initialize the matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = i * cols + j;
        }
    }

    // Print the matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
```

```
    return 0;
}
```

VLA Function Parameters

```
// Function that accepts VLA parameters
void process_matrix(int rows, int cols, int matrix[rows][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] *= 2;  // Double each element
        }
    }
}

// Usage
int main() {
    int rows = 3, cols = 4;
    int matrix[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};

    process_matrix(rows, cols, matrix);

    // Print modified matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Array of Arrays vs. Multi-dimensional Arrays

Array of Arrays

An array of arrays treats each row as a separate array:

```
// Array of pointers to arrays (jagged array)
int *matrix[3];
matrix[0] = malloc(2 * sizeof(int));  // First row: 2 elements
matrix[1] = malloc(4 * sizeof(int));  // Second row: 4 elements
matrix[2] = malloc(3 * sizeof(int));  // Third row: 3 elements
```

```
// Initialize elements
matrix[0][0] = 1; matrix[0][1] = 2;
matrix[1][0] = 3; matrix[1][1] = 4; matrix[1][2] = 5; matrix[1][3] = 6;
matrix[2][0] = 7; matrix[2][1] = 8; matrix[2][2] = 9;

// Free memory
free(matrix[0]);
free(matrix[1]);
free(matrix[2]);
```

Multi-dimensional Arrays

Multi-dimensional arrays have fixed dimensions and contiguous memory layout:

```
// True multi-dimensional array
int matrix[3][4];  // Fixed size: 3 rows, 4 columns

// All elements stored contiguously in memory
// Memory layout: [0][0] [0][1] [0][2] [0][3] [1][0] [1][1] ...
```

Jagged Arrays

Creating Jagged Arrays

Jagged arrays are arrays of arrays where each sub-array can have different lengths:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Create a jagged array
    int rows = 3;
    int *jagged[3];

    // Allocate memory for each row with different sizes
    jagged[0] = malloc(2 * sizeof(int));  // Row 0: 2 elements
    jagged[1] = malloc(4 * sizeof(int));  // Row 1: 4 elements
    jagged[2] = malloc(3 * sizeof(int));  // Row 2: 3 elements

    // Initialize elements
    jagged[0][0] = 1; jagged[0][1] = 2;
    jagged[1][0] = 3; jagged[1][1] = 4; jagged[1][2] = 5; jagged[1][3] = 6;
    jagged[2][0] = 7; jagged[2][1] = 8; jagged[2][2] = 9;
```

```
    // Print the jagged array
    for (int i = 0; i < rows; i++) {
        int cols = (i == 0) ? 2 : (i == 1) ? 4 : 3;
        for (int j = 0; j < cols; j++) {
            printf("%d ", jagged[i][j]);
        }
        printf("\n");
    }

    // Free memory
    for (int i = 0; i < rows; i++) {
        free(jagged[i]);
    }

    return 0;
}
```

Jagged Array Functions

```
// Function to create a jagged array
int** create_jagged_array(int rows, int *col_sizes) {
    int **array = malloc(rows * sizeof(int*));

    for (int i = 0; i < rows; i++) {
        array[i] = malloc(col_sizes[i] * sizeof(int));
    }

    return array;
}

// Function to free a jagged array
void free_jagged_array(int **array, int rows) {
    for (int i = 0; i < rows; i++) {
        free(array[i]);
    }
    free(array);
}

// Function to print a jagged array
void print_jagged_array(int **array, int rows, int *col_sizes) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < col_sizes[i]; j++) {
            printf("%d ", array[i][j]);
        }
        printf("\n");
```

```
    }
}
```

Array Algorithms and Operations

Matrix Operations

```c
#include <stdio.h>

#define ROWS 3
#define COLS 3

// Matrix addition
void matrix_add(int a[ROWS][COLS], int b[ROWS][COLS], int result[ROWS][COLS]) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}

// Matrix multiplication
void matrix_multiply(int a[ROWS][COLS], int b[COLS][ROWS], int result[ROWS][ROWS]) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < ROWS; j++) {
            result[i][j] = 0;
            for (int k = 0; k < COLS; k++) {
                result[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

// Transpose matrix
void matrix_transpose(int matrix[ROWS][COLS], int transpose[COLS][ROWS]) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }
}
```

Dynamic Array Resizing

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    int size;
    int capacity;
} DynamicArray;

// Initialize dynamic array
DynamicArray* create_array(int initial_capacity) {
    DynamicArray *arr = malloc(sizeof(DynamicArray));
    arr->data = malloc(initial_capacity * sizeof(int));
    arr->size = 0;
    arr->capacity = initial_capacity;
    return arr;
}

// Add element to dynamic array
void array_push(DynamicArray *arr, int value) {
    if (arr->size >= arr->capacity) {
        // Double the capacity
        arr->capacity *= 2;
        arr->data = realloc(arr->data, arr->capacity * sizeof(int));
    }

    arr->data[arr->size] = value;
    arr->size++;
}

// Free dynamic array
void free_array(DynamicArray *arr) {
    free(arr->data);
    free(arr);
}
```

Array Performance Considerations

Memory Layout

Arrays have contiguous memory layout, which provides: - Cache locality for sequential access - Predictable memory usage - Efficient indexing operations

```c
// Row-major order traversal (efficient)
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        process(matrix[i][j]);
    }
}

// Column-major order traversal (less efficient for C arrays)
for (int j = 0; j < cols; j++) {
    for (int i = 0; i < rows; i++) {
        process(matrix[i][j]);
    }
}
```

Cache Performance

```c
#define SIZE 1000

// Good cache performance - sequential access
void good_access_pattern(int matrix[SIZE][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            matrix[i][j] = i + j;
        }
    }
}

// Poor cache performance - non-sequential access
void poor_access_pattern(int matrix[SIZE][SIZE]) {
    for (int j = 0; j < SIZE; j++) {
        for (int i = 0; i < SIZE; i++) {
            matrix[i][j] = i + j;
        }
    }
}
```

Advanced Array Techniques

Array Pointers

```
// Pointer to array
int (*ptr)[5];  // Pointer to array of 5 integers
int array[5] = {1, 2, 3, 4, 5};
ptr = &array;   // Point to the entire array

// Access elements
printf("%d\n", (*ptr)[2]);  // Access third element
```

Array of Pointers

```
// Array of pointers
int *ptr_array[5];  // Array of 5 pointers to integers
int values[5] = {10, 20, 30, 40, 50};

// Point each pointer to an element
for (int i = 0; i < 5; i++) {
    ptr_array[i] = &values[i];
}

// Access through pointers
for (int i = 0; i < 5; i++) {
    printf("%d ", *ptr_array[i]);
}
```

Compound Literals (C99)

```
// Compound literal for array
int *ptr = (int[]){1, 2, 3, 4, 5};

// Use in function calls
void process_array(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}

// Call with compound literal
process_array((int[]){10, 20, 30, 40, 50}, 5);
```

Practical Examples

Sudoku Validator

```
#include <stdio.h>
#include <stdbool.h>

#define SIZE 9

bool is_valid_sudoku(int board[SIZE][SIZE]) {
    // Check rows
    for (int i = 0; i < SIZE; i++) {
        bool seen[SIZE + 1] = {false};
        for (int j = 0; j < SIZE; j++) {
            if (board[i][j] != 0) {
                if (seen[board[i][j]]) return false;
                seen[board[i][j]] = true;
            }
        }
    }

    // Check columns
    for (int j = 0; j < SIZE; j++) {
        bool seen[SIZE + 1] = {false};
        for (int i = 0; i < SIZE; i++) {
            if (board[i][j] != 0) {
                if (seen[board[i][j]]) return false;
                seen[board[i][j]] = true;
            }
        }
    }

    // Check 3x3 boxes
    for (int box_row = 0; box_row < 3; box_row++) {
        for (int box_col = 0; box_col < 3; box_col++) {
            bool seen[SIZE + 1] = {false};
            for (int i = 0; i < 3; i++) {
                for (int j = 0; j < 3; j++) {
                    int row = box_row * 3 + i;
                    int col = box_col * 3 + j;
                    if (board[row][col] != 0) {
                        if (seen[board[row][col]]) return false;
                        seen[board[row][col]] = true;
                    }
                }
            }
```

```
        }
    }

    return true;
}

int main() {
    int sudoku[SIZE][SIZE] = {
        {5, 3, 0, 0, 7, 0, 0, 0, 0},
        {6, 0, 0, 1, 9, 5, 0, 0, 0},
        {0, 9, 8, 0, 0, 0, 0, 6, 0},
        {8, 0, 0, 0, 6, 0, 0, 0, 3},
        {4, 0, 0, 8, 0, 3, 0, 0, 1},
        {7, 0, 0, 0, 2, 0, 0, 0, 6},
        {0, 6, 0, 0, 0, 0, 2, 8, 0},
        {0, 0, 0, 4, 1, 9, 0, 0, 5},
        {0, 0, 0, 0, 8, 0, 0, 7, 9}
    };

    if (is_valid_sudoku(sudoku)) {
        printf("Valid Sudoku\n");
    } else {
        printf("Invalid Sudoku\n");
    }

    return 0;
}
```

Image Processing Example

```
#include <stdio.h>
#include <stdlib.h>

#define WIDTH 100
#define HEIGHT 100

// Simple image structure
typedef struct {
    unsigned char pixels[HEIGHT][WIDTH][3];  // RGB pixels
    int width, height;
} Image;

// Convert to grayscale
void grayscale(Image *img) {
    for (int i = 0; i < img->height; i++) {
```

```
        for (int j = 0; j < img->width; j++) {
            // Calculate grayscale value
            unsigned char gray = (img->pixels[i][j][0] * 0.3 +
                                  img->pixels[i][j][1] * 0.59 +
                                  img->pixels[i][j][2] * 0.11);

            // Set all channels to grayscale value
            img->pixels[i][j][0] = gray;
            img->pixels[i][j][1] = gray;
            img->pixels[i][j][2] = gray;
        }
    }
}

// Apply simple blur filter
void blur(Image *img) {
    // Create a copy for processing
    unsigned char temp[HEIGHT][WIDTH][3];

    for (int i = 1; i < img->height - 1; i++) {
        for (int j = 1; j < img->width - 1; j++) {
            for (int c = 0; c < 3; c++) {
                // Average of 3x3 neighborhood
                int sum = 0;
                for (int di = -1; di <= 1; di++) {
                    for (int dj = -1; dj <= 1; dj++) {
                        sum += img->pixels[i + di][j + dj][c];
                    }
                }
                temp[i][j][c] = sum / 9;
            }
        }
    }

    // Copy back to original image
    for (int i = 1; i < img->height - 1; i++) {
        for (int j = 1; j < img->width - 1; j++) {
            for (int c = 0; c < 3; c++) {
                img->pixels[i][j][c] = temp[i][j][c];
            }
        }
    }
}
```

Summary

Advanced array concepts in C provide powerful tools for handling complex data structures and implementing efficient algorithms. Key points to remember:

1. **Multi-dimensional Arrays**: Arrays of arrays with fixed dimensions and contiguous memory layout
2. **Variable-length Arrays**: C99 feature allowing runtime-determined array sizes
3. **Jagged Arrays**: Arrays of arrays with different sub-array sizes
4. **Memory Layout**: Row-major order storage affects performance characteristics
5. **Array Pointers**: Distinction between pointers to arrays and arrays of pointers
6. **Performance Considerations**: Cache locality and access patterns significantly impact performance
7. **Advanced Techniques**: Compound literals, designated initializers, and dynamic resizing

Understanding these advanced array concepts is essential for efficient C programming, particularly when working with matrices, tables, and complex data structures.

Length-Prefixed Strings: an sds-Style String Library in Under 60 Lines

Length-Prefixed Strings: an sds-Style String Library in Under 60 Lines

Thesis

A C string is a pointer to bytes that ends at the first `'\0'`. That one decision has three consequences. Finding the length is O(n). Appending in a loop is $O(n^2)$, because each `strcat` rescans the whole string. And the string can't contain a NUL byte, so it isn't binary safe.

Pascal took the other route and stored the length in front of the data, originally in a single byte, which capped strings at 255 characters. The modern C version stores a full header in front of the bytes **and** keeps the trailing NUL. Then it hands the caller a pointer to the bytes, not to the header. That is the design of antirez's SDS, used by Redis. The result is still a `char *` that works with `printf` and `strlen`, but its length is O(1), appends are amortised O(1), and it can hold any bytes.

This chapter builds a minimal version of that design, measures it, and shows the one rule it makes you follow.

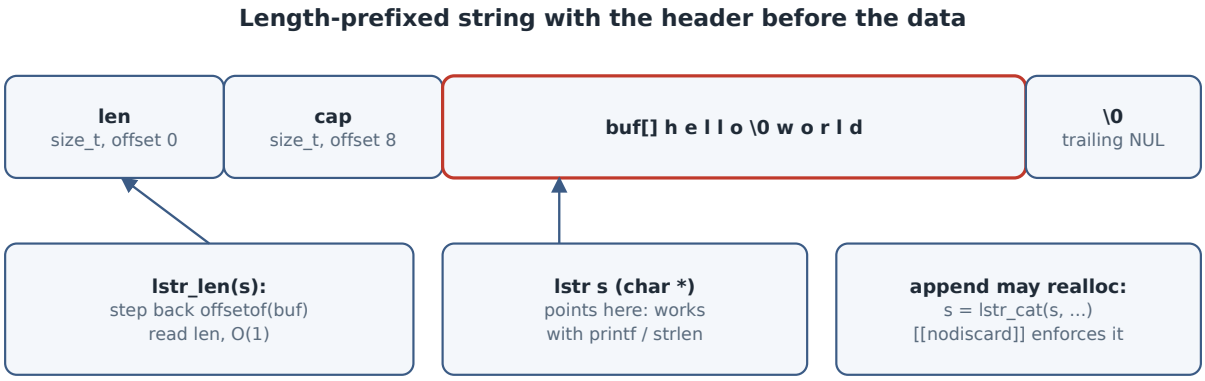


Figure 1: Length-prefixed string with the header before the data

The whole library

```
/* lstr.h - minimal sds-style length-prefixed string (single header) */
#ifndef LSTR_H
#define LSTR_H
#include <stddef.h>
```

```c
#include <stdlib.h>
#include <string.h>

#if defined(__STDC_VERSION__) && __STDC_VERSION__ > 201710L   /* GCC 14 -std=c23 reports 202
#define LSTR_MUST_USE [[nodiscard]]
#else
#define LSTR_MUST_USE __attribute__((warn_unused_result))   /* GCC/Clang */
#endif

typedef char *lstr;                 /* points at buf[], so it is also a C string */

struct lstr_hdr {
    size_t len;                     /* bytes in use, excluding the trailing NUL */
    size_t cap;                     /* bytes available in buf, excluding the NUL */
    char buf[];                     /* flexible array member */
};

#define LSTR_HDR(s) ((struct lstr_hdr *)((s) - offsetof(struct lstr_hdr, buf)))

static inline size_t lstr_len(const lstr s) { return LSTR_HDR(s)->len; }
static inline size_t lstr_cap(const lstr s) { return LSTR_HDR(s)->cap; }

static inline lstr lstr_newlen(const void *p, size_t n) {
    struct lstr_hdr *h = malloc(sizeof *h + n + 1);
    if (!h) return NULL;
    h->len = h->cap = n;
    if (n) memcpy(h->buf, p, n);
    h->buf[n] = '\0';
    return h->buf;
}
static inline lstr lstr_new(const char *c) { return lstr_newlen(c, strlen(c)); }
static inline void lstr_free(lstr s) { if (s) free(LSTR_HDR(s)); }

/* May move the string: always use the returned pointer (s = lstr_catlen(s, ...)). */
LSTR_MUST_USE static inline lstr lstr_catlen(lstr s, const void *p, size_t n) {
    struct lstr_hdr *h = LSTR_HDR(s);
    if (h->cap - h->len < n) {
        size_t need = h->len + n;
        size_t cap = need < 1024 * 1024 ? need * 2 : need + 1024 * 1024;
        h = realloc(h, sizeof *h + cap + 1);
        if (!h) return NULL;
        h->cap = cap;
    }
    memcpy(h->buf + h->len, p, n);
    h->len += n;
    h->buf[h->len] = '\0';
    return h->buf;
```

```
}
LSTR_MUST_USE static inline lstr lstr_cat(lstr s, const char *c) { return lstr_catlen(s, c,
#endif
```

Points to notice:

- **`buf[]` is a flexible array member.** The header and the bytes come from a single `malloc`, sized as `sizeof *h + n + 1`.
- **`LSTR_HDR` steps back from the user pointer** by `offsetof(struct lstr_hdr, buf)` to reach the header. That is why `lstr_len` is a single load.
- **Growth doubles up to 1 MiB, then adds 1 MiB at a time.** SDS uses the same policy (`SDS_MAX_PREALLOC`, `1024*1024` in `sds.h`). It keeps appends amortised O(1) without doubling huge buffers.
- **`lstr_catlen` can move the string**, so it returns the new pointer and is marked *must use*. The last section explains why.
- **Failure:** if `realloc` fails, the function returns `NULL` and the old block is still allocated. A caller who writes `s = lstr_cat(s, ...)` leaks it. SDS behaves the same way. Code that has to survive out-of-memory should keep the old pointer and check for `NULL` before overwriting it.

It is still a C string, and also binary safe

```c
#include <stdio.h>
#include "lstr.h"

int main(void) {
    lstr s = lstr_new("hello");
    printf("as a C string: %s (strlen %zu, lstr_len %zu)\n", s, strlen(s), lstr_len(s));

    s = lstr_catlen(s, "\0world", 6);           /* binary-safe: embedded NUL */
    printf("after binary append: strlen %zu, lstr_len %zu\n", strlen(s), lstr_len(s));
    fwrite(s, 1, lstr_len(s), stdout);
    putchar('\n');

    printf("header %zu bytes, len at offset %zu, buf at offset %zu\n",
           sizeof(struct lstr_hdr), offsetof(struct lstr_hdr, len),
           offsetof(struct lstr_hdr, buf));

    size_t last = lstr_cap(s), grows = 0;
    for (int i = 0; i < 100000; i++) {
        s = lstr_cat(s, "x");
        if (lstr_cap(s) != last) { grows++; last = lstr_cap(s); }
    }
    printf("100000 appends: len %zu, cap %zu, reallocations %zu\n", lstr_len(s), lstr_cap(s)
    lstr_free(s);
```

```
}
```

```
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <time.h>
#include "lstr.h"

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv

int main(void) {
    enum { N = 200000 };
    char *plain = calloc(N + 1, 1);
    double t = now();
    for (int i = 0; i < N; i++) strcat(plain, "x");   /* rescans the whole string each time
    double t_strcat = now() - t;

    lstr s = lstr_new("");
    t = now();
    for (int i = 0; i < N; i++) s = lstr_cat(s, "x");  /* O(1) to find the end */
    double t_lstr = now() - t;

    printf("%d one-byte appends: strcat %.1f ms, lstr_cat %.2f ms (same result: %s)\n",
           N, t_strcat * 1e3, t_lstr * 1e3, strcmp(plain, s) == 0 ? "yes" : "no");
    free(plain);
    lstr_free(s);
}
```

```
#include <stdio.h>
#include "lstr.h"

int main(void) {
    lstr s = lstr_new("short");
    lstr_cat(s, " and now much longer than the capacity");   /* BUG: result ignored */
    printf("%s\n", s);                                       /* s may point at freed memory
    lstr_free(s);
}
```

```
#!/bin/sh
set -x
cd "$(dirname "$0")"
gcc -std=c17 -O2 -Wall -Wextra -Werror demo.c -o demo && ./demo | cat -v
gcc -std=c17 -O2 -Wall -Wextra bench.c -o bench && ./bench
gcc -std=c17 -g -fsanitize=address,undefined -Wall -Wextra demo.c -o demo-asan && ./demo-asa
gcc -std=c23 -dM -E -x c /dev/null -o macros.txt
grep STDC_VERSION macros.txt
```

```
gcc -std=c23 -g -fsanitize=address -Wall -Wextra stale.c -o stale && ./stale 2> asan.txt
grep -E "ERROR|READ of|freed by|previously allocated|#[0-9] .*stale.c" asan.txt
```

All output is from GCC 14.2 and glibc 2.41 on x86-64 Linux. The ASan process ID is masked.

```
+ dirname ./run.sh
+ cd .
+ gcc -std=c17 -O2 -Wall -Wextra -Werror demo.c -o demo
+ ./demo
+ cat -v
as a C string: hello (strlen 5, lstr_len 5)
after binary append: strlen 5, lstr_len 11
hello^@world
header 16 bytes, len at offset 0, buf at offset 16
100000 appends: len 100011, cap 196606, reallocations 13
+ gcc -std=c17 -O2 -Wall -Wextra bench.c -o bench
+ ./bench
200000 one-byte appends: strcat 165.2 ms, lstr_cat 0.38 ms (same result: yes)
+ gcc -std=c17 -g -fsanitize=address,undefined -Wall -Wextra demo.c -o demo-asan
+ ./demo-asan
+ echo ASan/UBSan: clean
ASan/UBSan: clean
+ gcc -std=c23 -dM -E -x c /dev/null -o macros.txt
+ grep STDC_VERSION macros.txt
#define __STDC_VERSION__ 202000L
+ gcc -std=c23 -g -fsanitize=address -Wall -Wextra stale.c -o stale
stale.c: In function ‘main’:
stale.c:6:5: warning: ignoring return value of ‘lstr_cat’, declared with attribute ‘nodiscar
    6 |     lstr_cat(s, " and now much longer than the capacity");   /* BUG: result ignored
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
In file included from stale.c:2:
lstr.h:53:34: note: declared here
   53 | LSTR_MUST_USE static inline lstr lstr_cat(lstr s, const char *c) { return lstr_catle
      |                                  ^~~~~~~~
+ ./stale
+ grep -E ERROR|READ of|freed by|previously allocated|#[0-9] .*stale.c asan.txt
==PID==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000050 at pc 0x7f7171
READ of size 6 at 0x503000000050 thread T0
    #1 0x557c9b814594 in main /tmp/lab/stale.c:7
freed by thread T0 here:
    #3 0x557c9b814588 in main /tmp/lab/stale.c:6
previously allocated by thread T0 here:
    #3 0x557c9b81456e in main /tmp/lab/stale.c:5
```

What the run shows:

- **`%s` and `strlen` just work**, because `s` points at `buf` and the bytes end in a NUL.

- **Binary safety:** after appending `"\0world"`, `strlen` still says 5, while `lstr_len` correctly says 11. `fwrite(s, 1, lstr_len(s), ...)` writes all 11 bytes, and `cat -v` shows the embedded NUL as `^@`.
- **The header costs 16 bytes** on a 64-bit target: two `size_t`, with `buf` starting at offset 16. SDS goes further and picks among 1-, 3-, 5-, 9- or 17-byte packed headers (`sdshdr5` to `sdshdr64`, each with a `flags` byte) based on the length. That saves memory on millions of short keys, at the cost of a type dispatch on every access.
- **100,000 appends cause only 13 reallocations**, thanks to geometric growth.
- **O(n²) vs O(n):** 200,000 one-byte `strcat` calls took 165.2 ms in the run above, while `lstr_cat` took 0.38 ms and produced the same string. Two earlier runs gave 147.6/0.34 ms and 140.2/0.42 ms.

The one rule: always keep the returned pointer

Appending may `realloc` the block to a new address. That makes the API look like SDS's `s = sdscat(s, "...")`, and forgetting the assignment leaves `s` pointing at freed memory. `stale.c` makes that mistake. AddressSanitizer reports a `heap-use-after-free`: the block was allocated at line 5, freed by the `realloc` inside `lstr_cat` at line 6, and read by `printf` at line 7.

The compiler can catch this before it runs. Mark the append functions `[[nodiscard]]` (C23), or `__attribute__((warn_unused_result))` on older standards, and the ignored result becomes a `-Wunused-result` warning. Add `-Werror` to make it an error.

There is a portability trap here. **GCC 14 in `-std=c23` mode defines `__STDC_VERSION__` as `202000L`, not C23's final `202311L`**, as the run shows. A `#if __STDC_VERSION__ >= 202311L` check therefore silently falls back to the GCC attribute on GCC 14. That's why `lstr.h` tests for anything newer than C17 (`> 201710L`) instead.

When to use which

	NUL-terminated `char *`	length-prefixed (`lstr`/SDS)
length	O(n) `strlen`	O(1) header load
append in a loop	O(n²) with `strcat`	amortised O(1)
embedded `\0`	impossible	fine
pass to libc / OS APIs	native	native (`s` is a `char *`)
string literals, stack buffers	yes	no, heap only, freed with `lstr_free`
pointer stable across appends	n/a	**no**, always reassign

Use length-prefixed strings for owned, growing, possibly binary data: buffers, protocol messages, keys. Keep plain `char *` for literals and borrowed views. Never pass a plain `char *` to `lstr_len` or `lstr_free`: they would read a header that isn't there.

Takeaways

- Putting the header *before* the bytes and returning a pointer to the bytes gives you O(1) length and binary safety while staying compatible with `printf` and `strlen`.
- Geometric growth turns repeated appends from $O(n^2)$ into amortised O(1): 13 reallocations for 100,000 appends.
- Any function that may `realloc` must return the new pointer, and the API should make ignoring it a compile-time warning with `[[nodiscard]]`.
- When feature-testing C23, remember that GCC 14 reports `__STDC_VERSION__ == 202000L`.

Sources

- antirez/sds: README (design, `s = sdscat(s, ...)` usage, binary safety) and `sds.h` (`sdshdr5`…`sdshdr64` packed headers)
- C23 working draft N3220 (the `nodiscard` attribute, flexible array members, `__STDC_VERSION__` = `202311L`): https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf
- GCC manual: `warn_unused_result` function attribute
- AddressSanitizer

Module 5: Arrays and Strings

Overview

This module explores arrays and strings in C, fundamental data structures for storing and manipulating collections of data. You'll learn about array declaration, initialization, and access, as well as multi-dimensional arrays and advanced array concepts. The module also covers string handling in C, including string literals, character arrays, and the standard library functions for string manipulation.

Learning Objectives

By the end of this module, you will be able to: - Declare, initialize, and access arrays of various types - Work with multi-dimensional arrays and jagged arrays - Understand array bounds and buffer overflow prevention - Manipulate strings using character arrays and standard library functions - Implement string searching, comparison, and manipulation algorithms - Apply arrays and strings to solve practical programming problems - Understand the relationship between arrays and pointers - Use variable-length arrays (C99 feature) appropriately

Chapters

1. **Arrays** - Declaration, initialization, access, and basic operations
2. **Strings** - Character arrays, string literals, and string manipulation
3. **Advanced Arrays** - Multi-dimensional arrays, jagged arrays, and advanced concepts

Key Concepts Covered

- Array declaration and initialization syntax
- Array indexing and bounds checking
- Multi-dimensional arrays and initialization
- Array traversal and common operations
- String representation in C
- String literals vs. character arrays
- Standard library string functions
- String searching and manipulation
- Array algorithms (searching, sorting)
- Relationship between arrays and pointers

- Variable-length arrays (C99)
- Array of arrays vs. multi-dimensional arrays
- Jagged arrays using pointers
- Buffer overflow prevention techniques

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Completion of Module 4: Functions and Modular Programming
- Understanding of basic programming concepts
- Familiarity with variable declaration and initialization

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Understanding of basic programming concepts
- Familiarity with debugging tools

Estimated Time to Complete

- Reading: 8-10 hours
- Exercises: 12-15 hours
- Projects: 6-8 hours

Assessment

- Complete all chapter exercises
- Successfully implement array and string manipulation algorithms
- Demonstrate understanding of array bounds and string handling
- Pass array and string programming quiz
- Submit a comprehensive program using arrays and strings

Next Module

After completing this module, proceed to **Module 6: Pointers and Memory Management** to learn about memory addressing and dynamic allocation.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 5)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Online tutorials on array and string algorithms
- Debugging tools documentation (GDB, Valgrind)
- Best practices guides for secure string handling

Pointers

Pointer Fundamentals

Introduction

A pointer is a variable that stores the **address** of another object, not the object's value itself. Pointers unlock efficient parameter passing, dynamic data structures, and low-level system programming. This chapter builds intuition with step-by-step memory pictures, complete programs, `const` combinations, `void *`, array/pointer relationships, and out-parameters.

Compile with:

```
gcc -std=c17 -Wall -Wextra program.c -o program
```

What Is a Pointer?

Every object in your program lives at a memory address. The address-of operator `&` yields that address; the dereference operator `*` follows a pointer to the object.

```c
#include <stdio.h>

int main(void) {
    int x = 42;
    printf("Value of x:   %d\n", x);
    printf("Address of x: %p\n", (void *)&x);
    return 0;
}
```

Always cast to `void *` when printing with `%p`.

Visual Step-by-Step: `&` and `*`

Think of memory as numbered boxes. Variable names label boxes; pointers hold box numbers.

Step 1 — Create an `int`

```c
int x = 42;
```

```
  name: x

     42       address e.g. 0x1000
```

Step 2 — Point at it

```
int *p = &x;   /* p holds the address of x */
```

```
  name: p                        name: x

    0x1000                 42      0x1000
```

Step 3 — Read through the pointer

```
int y = *p;    /* y becomes 42 */
```

`*p` means "the `int` sitting at the address stored in `p`."

Step 4 — Write through the pointer

```
*p = 99;       /* x is now 99 */
```

```
  p    x: 99
```

Full Demo Program

```
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p = &x;

    printf("1) x=%d  &x=%p  p=%p  *p=%d\n",
           x, (void *)&x, (void *)p, *p);

    *p = 99;
    printf("2) after *p=99: x=%d  *p=%d\n", x, *p);

    x = 7;
```

```
    printf("3) after x=7:   x=%d  *p=%d\n", x, *p);

    int z = 100;
    p = &z;
    printf("4) p now points at z: *p=%d  x still %d\n", *p, x);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra ptr_steps.c -o ptr_steps
./ptr_steps
```

Declaration and Initialization

```
int *ptr;           /* pointer to int (uninitialized - dangerous) */
char *cptr;
float *fptr;

int x = 42;
int *p1 = &x;       /* initialized */
int *p2 = NULL;     /* safe empty pointer */
int *p3 = 0;        /* also null */
```

Null Pointers

```
#include <stdio.h>

int main(void) {
    int *p = NULL;

    if (p != NULL) {
        printf("%d\n", *p);
    } else {
        printf("Pointer is null - do not dereference\n");
    }
    return 0;
}
```

Never dereference `NULL`. On most systems that crashes; the language only says the behavior is undefined.

Swap via Pointers (Full Program)

Pass-by-value cannot swap the caller's variables. Pass pointers (addresses) so the function can write back.

Why Values Alone Fail

```c
#include <stdio.h>

/* This does NOT swap the caller's x and y */
static void swap_by_value(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main(void) {
    int x = 10, y = 20;
    swap_by_value(x, y);
    printf("still x=%d y=%d\n", x, y);  /* 10 20 */
    return 0;
}
```

Correct Swap with Pointers

```c
#include <stdio.h>

static void swap(int *a, int *b) {
    if (a == NULL || b == NULL) {
        return;
    }
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void) {
    int x = 10, y = 20;
    printf("before: x=%d y=%d\n", x, y);

    swap(&x, &y);

    printf("after:  x=%d y=%d\n", x, y);  /* 20 10 */
    return 0;
```

```
}
```

Picture of the call:

```
main:  x=10   y=20


swap:    a      b     (a and b hold addresses)
         *a   *b      (values exchanged)
```

```
gcc -std=c17 -Wall -Wextra swap.c -o swap
./swap
```

`const` and Pointers: Three Combinations

`const` can protect the **data**, the **pointer**, or **both**.

Declaration	Can change `*p`?	Can reassign `p`?
`const int *p`	no	yes
`int *const p`	yes	no
`const int *const p`	no	no
`int *p`	yes	yes

Full Program

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    int y = 20;

    /* 1) Pointer to const int - data is read-only through p */
    const int *p1 = &x;
    printf("p1 -> %d\n", *p1);
    /* *p1 = 99; */          /* ERROR: discards const */
    p1 = &y;                 /* OK: pointer may move */
    printf("p1 now -> %d\n", *p1);

    /* 2) Const pointer to int - pointer frozen, data mutable */
    int *const p2 = &x;
    *p2 = 11;                /* OK */
    /* p2 = &y; */           /* ERROR: p2 is const */
    printf("x is now %d\n", x);
```

```
    /* 3) Const pointer to const int - both frozen */
    const int *const p3 = &y;
    printf("p3 -> %d\n", *p3);
    /* *p3 = 0; */          /* ERROR */
    /* p3 = &x; */          /* ERROR */

    /* Reading is always fine for all three */
    printf("read-only views: %d %d %d\n", *p1, *p2, *p3);
    return 0;
}
```

Why `const int *` Appears in APIs

```
/* Caller knows print_ints will not modify the buffer */
void print_ints(const int *a, size_t n);
```

That is a promise to the reader and a constraint the compiler helps enforce (unless you cast const away — don't).

Note on `const int *` vs `int const *`

These are the **same**: pointer to const int. Read declarations right-to-left: `const int *p` → "`p` is a pointer to const int."

`void *` with Careful Casting

`void *` is a generic object pointer. You can convert to and from other object pointer types, but you **must not** dereference a `void *` without casting to a concrete type first.

Full Program: Generic Swap of Bytes

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Swap n bytes between the objects pointed to by a and b. */
static int memswap(void *a, void *b, size_t n) {
    if (a == NULL || b == NULL) {
        return -1;
    }
    if (n == 0) {
        return 0;
```

```
    }
    unsigned char *p = a;   /* implicit void* → unsigned char* */
    unsigned char *q = b;
    for (size_t i = 0; i < n; i++) {
        unsigned char t = p[i];
        p[i] = q[i];
        q[i] = t;
    }
    return 0;
}

static void print_ints(const char *label, const int *a, size_t n) {
    printf("%s:", label);
    for (size_t i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");
}

int main(void) {
    int u = 5, v = 9;
    memswap(&u, &v, sizeof u);
    printf("u=%d v=%d\n", u, v);

    int a[] = {1, 2, 3};
    int b[] = {7, 8, 9};
    memswap(a, b, sizeof a);
    print_ints("a", a, 3);
    print_ints("b", b, 3);

    /* Careful cast example: recover a typed pointer from void* */
    void *raw = a;
    int *typed = (int *)raw;   /* explicit cast for clarity */
    printf("typed[0]=%d\n", typed[0]);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra void_swap.c -o void_swap
./void_swap
```

Rules of Thumb for `void *`

1. Use for generic buffers, allocators (`malloc` returns `void *`), and callbacks.
2. Cast to the **actual** type before arithmetic or dereference.
3. Keep size information alongside the pointer when the type is erased.
4. Do not cast `void *` to function pointers (different category in C).

Pointer Arithmetic (Brief)

Arithmetic scales by the size of the pointed-to type:

```c
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;  /* same as &arr[0] */

    printf("*p       = %d\n", *p);       /* 10 */
    printf("*(p+1)   = %d\n", *(p + 1)); /* 20 */
    printf("p[2]     = %d\n", p[2]);     /* 30 - syntactic sugar */

    p++;
    printf("after p++: %d\n", *p);       /* 20 */
    return 0;
}
```

`p[i]` is defined as `*(p + i)`.

Pointers vs Arrays: Equivalence and Differences

Where They Look the Same

In most expression contexts, an array name **decays** to a pointer to its first element:

```c
int a[4] = {1, 2, 3, 4};
int *p = a;       /* decay */
int *q = &a[0];   /* same address */

/* These all refer to the third element (value 3) */
a[2];
*(a + 2);
p[2];
*(p + 2);
```

Where They Differ

	Array `int a[4]`	Pointer `int *p`
Storage	Holds 4 ints	Holds one address
`sizeof`	`4 * sizeof(int)`	`sizeof(int *)`
Assignability	`a = …` illegal	`p = …` OK
Address	`&a` is `int (*)[4]`	`&p` is `int **`

	Array `int a[4]`	Pointer `int *p`
Lifetime	As declared (e.g. auto)	Points wherever you set

Full Comparison Program

```c
#include <stdio.h>

static void show_param(int arr[]) {
    /* arr is really int * here */
    printf("  sizeof(param) = %zu (pointer, not array)\n", sizeof arr);
}

int main(void) {
    int a[4] = {1, 2, 3, 4};
    int *p = a;

    printf("sizeof(a) = %zu\n", sizeof a);   /* e.g. 16 */
    printf("sizeof(p) = %zu\n", sizeof p);   /* e.g. 8 */

    printf("a[2]=%d p[2]=%d\n", a[2], p[2]);

    p = &a[1];   /* OK */
    /* a = p; */ /* ERROR: array is not assignable */

    show_param(a);

    /* &a vs &a[0] */
    printf("&a    = %p (pointer to whole array)\n", (void *)&a);
    printf("&a[0] = %p (pointer to first int)\n", (void *)&a[0]);
    printf("same address, different types/sizeof arithmetic\n");
    return 0;
}
```

Out-Parameters: Return Status Separately from Data

A common C pattern: return an `int` status (`0` success, non-zero error) and write results through pointer out-parameters.

Full Program

```
#include <stdio.h>
#include <stdbool.h>

enum {
    OK = 0,
    ERR_NULL = -1,
    ERR_RANGE = -2,
    ERR_EMPTY = -3
};

/* Divide n / d into *quot_out. */
static int safe_div(int n, int d, int *quot_out) {
    if (quot_out == NULL) {
        return ERR_NULL;
    }
    if (d == 0) {
        return ERR_RANGE;
    }
    *quot_out = n / d;
    return OK;
}

/* Find max of a[0..n). */
static int array_max(const int *a, size_t n, int *max_out) {
    if (a == NULL || max_out == NULL) {
        return ERR_NULL;
    }
    if (n == 0) {
        return ERR_EMPTY;
    }
    int m = a[0];
    for (size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    *max_out = m;
    return OK;
}

/* Parse a simple non-negative integer from a C string. */
static int parse_u32(const char *s, unsigned *out) {
    if (s == NULL || out == NULL) {
        return ERR_NULL;
    }
```

```
    if (*s == '\0') {
        return ERR_EMPTY;
    }
    unsigned value = 0;
    for (const char *p = s; *p != '\0'; p++) {
        if (*p < '0' || *p > '9') {
            return ERR_RANGE;
        }
        value = value * 10u + (unsigned)(*p - '0');
    }
    *out = value;
    return OK;
}

int main(void) {
    int q;
    if (safe_div(42, 5, &q) == OK) {
        printf("42/5 = %d\n", q);
    }
    if (safe_div(42, 0, &q) != OK) {
        printf("division by zero handled\n");
    }

    int data[] = {3, 9, 2, 9, 5};
    int mx;
    if (array_max(data, 5, &mx) == OK) {
        printf("max = %d\n", mx);
    }

    unsigned u;
    if (parse_u32("12345", &u) == OK) {
        printf("parsed %u\n", u);
    }
    if (parse_u32("12x", &u) != OK) {
        printf("reject bad digits\n");
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra outparams.c -o outparams
./outparams
```

Design Tips

1. Document who allocates and who frees any pointer you return.
2. Prefer `const T *` for inputs you only read.

3. Check for `NULL` out-parameters when the API is public.
4. Do not leave `*out` half-written on failure unless you document it; either write only on success or define failure values.

Pointers and Functions (More)

Returning a Pointer into a Caller-Owned Array

```c
#include <stdio.h>

/* Returns pointer to first element equal to target, or NULL. */
static int *find_int(int *a, size_t n, int target) {
    for (size_t i = 0; i < n; i++) {
        if (a[i] == target) {
            return &a[i];
        }
    }
    return NULL;
}

int main(void) {
    int nums[] = {10, 50, 30, 40, 20};
    int *hit = find_int(nums, 5, 40);
    if (hit != NULL) {
        printf("found %d at offset %td\n", *hit, hit - nums);
        *hit = 41;  /* modify through returned pointer */
    }
    printf("nums[3]=%d\n", nums[3]);
    return 0;
}
```

Safe because the array lives in `main` longer than the pointer is used.

Unsafe: Returning Address of Local

```c
/* NEVER do this */
static int *bad(void) {
    int local = 42;
    return &local;  /* dangling */
}
```

Pointer Comparison and Size

```c
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p1 = &arr[1];
    int *p2 = &arr[4];

    if (p1 < p2) {
        printf("p1 before p2 in the same array\n");
    }
    printf("element distance: %td\n", p2 - p1);  /* 3 */

    printf("sizeof(int*)  = %zu\n", sizeof(int *));
    printf("sizeof(char*) = %zu\n", sizeof(char *));
    /* Same size on typical platforms; type still matters for arithmetic */
    return 0;
}
```

Relational comparisons are only defined for pointers into the same array object (or one past the end).

Best Practices

1. **Initialize** every pointer (`NULL` or a valid address).
2. **Check** for `NULL` before dereference when the value might be empty.
3. Prefer **`const`** on pointer parameters that only read data.
4. Do not return addresses of **automatic** locals.
5. Keep **size** next to raw buffers and **`void *`**.
6. Use **out-parameters + status** instead of overloading return values for both data and errors when both matter.
7. Compile with **`-Wall -Wextra`**; treat warnings as bugs.

Practical Examples

Pointer-Based String Reverse

```c
#include <stdio.h>
#include <string.h>

static void reverse_string(char *str) {
    if (str == NULL || *str == '\0') {
```

```
        return;
    }
    char *start = str;
    char *end = str + strlen(str) - 1;
    while (start < end) {
        char temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

int main(void) {
    char str[] = "Hello, World!";  /* must be mutable array */
    printf("Original: %s\n", str);
    reverse_string(str);
    printf("Reversed: %s\n", str);
    return 0;
}
```

Sum with Pointer Walking

```
#include <stdio.h>

static int sum_array(const int *arr, size_t n) {
    int sum = 0;
    const int *end = arr + n;
    for (const int *p = arr; p < end; p++) {
        sum += *p;
    }
    return sum;
}

int main(void) {
    int numbers[] = {1, 2, 3, 4, 5};
    size_t n = sizeof numbers / sizeof numbers[0];
    printf("Sum: %d\n", sum_array(numbers, n));
    return 0;
}
```

Exercises

1. **Trace on paper.** For `int x=3; int *p=&x; *p=5; int y=*p;`, draw boxes after each statement.
2. **Swap three ways.** Implement swap of two `int`s with pointers; of two `double`s; and a generic `memswap` with `void *` and `sizeof`.
3. **`const` experiments.** Write four small snippets that should fail to compile for illegal uses of `const int *`, `int *const`, and `const int *const`. Confirm with `gcc -std=c17 -Wall -Wextra`.
4. **Out-parameter min/max.** `int minmax(const int *a, size_t n, int *min_out, int *max_out)` returning status codes; write a driver that tests empty and null cases.
5. **Find and replace.** Return a pointer to the first space in a mutable string; replace that space with `'_'` through the pointer.
6. **Array vs pointer quiz.** Print `sizeof` for an array and for a pointer parameter that "receives" it. Explain the numbers in a comment.
7. **`void *` printer.** Write `void print_bytes(const void *p, size_t n)` that prints `n` bytes in hex by casting to `const unsigned char *`.
8. **Pointer distance.** Given two pointers into the same array, print how many elements lie between them. What happens if they are not from the same array? (Do not rely on that case.)

Summary

1. Pointers store **addresses**; `*` accesses the object; `&` produces an address.
2. **Swap and out-parameters** are the classic reasons to pass pointers to functions.
3. `const` can lock data, the pointer variable, or both — use it in APIs.
4. `void *` erases type; cast carefully and track sizes.
5. Arrays **decay** to pointers but are not the same as pointers (`sizeof`, assignment).
6. Always initialize, avoid dangling pointers, and return status separately when needed.

Mastering these fundamentals makes every later C topic — arrays, strings, dynamic memory, data structures — much easier to reason about.

Pointers and Arrays

Introduction

One of the most important relationships in C programming is between pointers and arrays. In fact, arrays and pointers are closely related concepts, with arrays often being treated as pointers in many contexts. Understanding this relationship is crucial for effective C programming, as it enables efficient array manipulation, dynamic memory allocation, and implementation of complex data structures.

Array-Pointer Equivalence

Arrays as Pointers

In most contexts, an array name is treated as a pointer to its first element:

```
int arr[5] = {10, 20, 30, 40, 50};

// These are equivalent:
int *ptr1 = arr;        // Array name decays to pointer
int *ptr2 = &arr[0];    // Explicit address of first element

printf("arr[0] = %d, *ptr1 = %d, *ptr2 = %d\n", arr[0], *ptr1, *ptr2);
```

Important Distinction

While arrays and pointers are often interchangeable, they are not identical:

```
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr;

// Arrays have fixed size
sizeof(arr);  // Returns 20 bytes (5 * sizeof(int))

// Pointers have size of address
sizeof(ptr);  // Returns 8 bytes on 64-bit system (size of pointer)

// Arrays cannot be reassigned
```

```c
// arr = ptr;  // Error: Cannot assign to array

// Pointers can be reassigned
ptr = &arr[2];  // OK: Pointer can be changed
```

Pointer Subscripting

Using Pointers as Arrays

Pointers can be used with array subscript notation:

```c
int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr;

// These are equivalent:
printf("arr[2] = %d\n", arr[2]);
printf("ptr[2] = %d\n", ptr[2]);
printf("*(arr + 2) = %d\n", *(arr + 2));
printf("*(ptr + 2) = %d\n", *(ptr + 2));
```

Array Access Methods

There are multiple equivalent ways to access array elements:

```c
int arr[5] = {10, 20, 30, 40, 50};

// Method 1: Array subscript notation
int value1 = arr[2];

// Method 2: Pointer arithmetic with dereference
int value2 = *(arr + 2);

// Method 3: Using pointer variable
int *ptr = arr;
int value3 = *(ptr + 2);
int value4 = ptr[2];

// All methods produce the same result
printf("%d %d %d %d\n", value1, value2, value3, value4);  // 30 30 30 30
```

Pointer Arithmetic with Arrays

Incrementing Pointers

Pointers can be incremented to traverse arrays:

```
int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr;

// Traverse array using pointer arithmetic
for (int i = 0; i < 5; i++) {
    printf("Element %d: %d\n", i, *ptr);
    ptr++;  // Move to next element
}

// Reset pointer to beginning
ptr = arr;

// Alternative traversal
for (int i = 0; i < 5; i++) {
    printf("Element %d: %d\n", i, *(ptr + i));
}
```

Pointer Difference

The difference between two pointers gives the number of elements between them:

```
int arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int *ptr1 = &arr[2];
int *ptr2 = &arr[8];

// Number of elements between ptr1 and ptr2
ptrdiff_t distance = ptr2 - ptr1;  // 6
printf("Distance: %td\n", distance);

// Check if pointer is within array bounds
if (ptr1 >= arr && ptr1 < arr + 10) {
    printf("ptr1 is within array bounds\n");
}
```

Multi-dimensional Arrays and Pointers

Two-dimensional Arrays

Two-dimensional arrays can be accessed using various pointer techniques:

```c
int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Method 1: Standard array notation
printf("matrix[1][2] = %d\n", matrix[1][2]);  // 7

// Method 2: Pointer to array
int (*ptr_to_row)[4] = matrix;  // Pointer to array of 4 integers
printf("ptr_to_row[1][2] = %d\n", ptr_to_row[1][2]);  // 7

// Method 3: Pointer to first element
int *ptr_to_element = &matrix[0][0];
printf("*(ptr_to_element + 1*4 + 2) = %d\n", *(ptr_to_element + 1*4 + 2));  // 7
```

Row-wise Traversal

```c
int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Traverse row by row
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}

// Using pointer arithmetic
int *ptr = &matrix[0][0];
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", *ptr);
        ptr++;
    }
    printf("\n");
}
```

Arrays as Function Parameters

Array Decay to Pointers

When arrays are passed to functions, they decay to pointers:

```
// These function declarations are equivalent:
void process_array1(int arr[]);
void process_array2(int *arr);
void process_array3(int arr[10]);  // Size is ignored

// Implementation
void process_array(int arr[], int size) {
    // sizeof(arr) returns pointer size, not array size!
    for (int i = 0; i < size; i++) {
        printf("Element %d: %d\n", i, arr[i]);
        // Equivalent to: printf("Element %d: %d\n", i, *(arr + i));
    }
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    process_array(numbers, 5);  // Pass array and size
    return 0;
}
```

Multi-dimensional Array Parameters

For multi-dimensional arrays, all dimensions except the first must be specified:

```
// Function accepting 2D array - second dimension must be specified
void process_matrix(int matrix[][4], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Alternative using pointer notation
void process_matrix_ptr(int (*matrix)[4], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", matrix[i][j]);
        }
```

```
        printf("\n");
    }
}

int main() {
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    process_matrix(matrix, 3);
    return 0;
}
```

Dynamic Arrays with Pointers

Allocating Dynamic Arrays

Pointers enable dynamic array allocation:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size;
    printf("Enter array size: ");
    scanf("%d", &size);

    // Allocate dynamic array
    int *arr = malloc(size * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Initialize array
    for (int i = 0; i < size; i++) {
        arr[i] = i * i;
    }

    // Use array
    for (int i = 0; i < size; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }
```

```
    // Free memory
    free(arr);
    arr = NULL;  // Prevent dangling pointer

    return 0;
}
```

Resizing Dynamic Arrays

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = malloc(5 * sizeof(int));
    int size = 5;

    // Initialize array
    for (int i = 0; i < size; i++) {
        arr[i] = i + 1;
    }

    printf("Original array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Resize array
    size = 10;
    arr = realloc(arr, size * sizeof(int));
    if (arr == NULL) {
        printf("Memory reallocation failed\n");
        return 1;
    }

    // Initialize new elements
    for (int i = 5; i < size; i++) {
        arr[i] = (i + 1) * 2;
    }

    printf("Resized array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
    // Free memory
    free(arr);
    return 0;
}
```

Pointer Arrays vs. Arrays of Pointers

Array of Pointers

An array of pointers stores multiple pointer values:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    // Array of string pointers
    char *strings[4] = {
        "Hello",
        "World",
        "C Programming",
        "Pointers"
    };

    // Print all strings
    for (int i = 0; i < 4; i++) {
        printf("String %d: %s\n", i, strings[i]);
    }

    // Dynamic string array
    char **dynamic_strings = malloc(3 * sizeof(char*));
    dynamic_strings[0] = malloc(6 * sizeof(char));
    strcpy(dynamic_strings[0], "First");

    dynamic_strings[1] = malloc(7 * sizeof(char));
    strcpy(dynamic_strings[1], "Second");

    dynamic_strings[2] = malloc(6 * sizeof(char));
    strcpy(dynamic_strings[2], "Third");

    // Print dynamic strings
    for (int i = 0; i < 3; i++) {
        printf("Dynamic string %d: %s\n", i, dynamic_strings[i]);
    }
```

```
    // Free dynamic memory
    for (int i = 0; i < 3; i++) {
        free(dynamic_strings[i]);
    }
    free(dynamic_strings);

    return 0;
}
```

Pointer to Array

A pointer to an array points to the entire array:

```
int arr[5] = {1, 2, 3, 4, 5};

// Pointer to array of 5 integers
int (*ptr_to_array)[5] = &arr;

// Access elements
printf("First element: %d\n", (*ptr_to_array)[0]);
printf("Third element: %d\n", (*ptr_to_array)[2]);

// Increment pointer (moves by entire array size)
ptr_to_array++;  // Points to next array of 5 integers
```

Advanced Array-Pointer Techniques

Pointer Arithmetic for Multi-dimensional Arrays

```
int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Treat 2D array as 1D array
int *ptr = &matrix[0][0];
for (int i = 0; i < 12; i++) {
    printf("%d ", ptr[i]);
    if ((i + 1) % 4 == 0) printf("\n");
}
```

Function Pointers for Array Operations

```c
#include <stdio.h>

// Function pointer type for array operations
typedef int (*array_operation)(int);

// Array operation functions
int square(int x) { return x * x; }
int cube(int x) { return x * x * x; }
int double_value(int x) { return x * 2; }

// Apply operation to array
void apply_operation(int *arr, int size, array_operation op) {
    for (int i = 0; i < size; i++) {
        arr[i] = op(arr[i]);
    }
}

int main() {
    int numbers[5] = {1, 2, 3, 4, 5};
    int size = 5;

    printf("Original: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    // Apply square operation
    apply_operation(numbers, size, square);

    printf("Squared: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}
```

Practical Examples

Matrix Transpose Using Pointers

```c
#include <stdio.h>

void transpose_matrix(int *matrix, int *transpose, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            // matrix[i][j] becomes transpose[j][i]
            *(transpose + j * rows + i) = *(matrix + i * cols + j);
        }
    }
}

int main() {
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    int transpose[4][3];

    transpose_matrix(&matrix[0][0], &transpose[0][0], 3, 4);

    printf("Original matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    printf("\nTransposed matrix:\n");
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

String Array Sorting

```c
#include <stdio.h>
#include <string.h>

void sort_strings(char *strings[], int count) {
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (strcmp(strings[j], strings[j + 1]) > 0) {
                // Swap pointers
                char *temp = strings[j];
                strings[j] = strings[j + 1];
                strings[j + 1] = temp;
            }
        }
    }
}

int main() {
    char *fruits[] = {"apple", "orange", "banana", "pear", "grape"};
    int count = 5;

    printf("Original order:\n");
    for (int i = 0; i < count; i++) {
        printf("%s ", fruits[i]);
    }
    printf("\n");

    sort_strings(fruits, count);

    printf("Sorted order:\n");
    for (int i = 0; i < count; i++) {
        printf("%s ", fruits[i]);
    }
    printf("\n");

    return 0;
}
```

Dynamic 2D Array

```c
#include <stdio.h>
#include <stdlib.h>

int** create_2d_array(int rows, int cols) {
```

```
    // Allocate array of row pointers
    int **matrix = malloc(rows * sizeof(int*));
    if (matrix == NULL) return NULL;

    // Allocate each row
    for (int i = 0; i < rows; i++) {
        matrix[i] = malloc(cols * sizeof(int));
        if (matrix[i] == NULL) {
            // Clean up on failure
            for (int j = 0; j < i; j++) {
                free(matrix[j]);
            }
            free(matrix);
            return NULL;
        }
    }

    return matrix;
}

void free_2d_array(int **matrix, int rows) {
    for (int i = 0; i < rows; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int main() {
    int rows = 3, cols = 4;
    int **matrix = create_2d_array(rows, cols);

    if (matrix == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Initialize matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = i * cols + j;
        }
    }

    // Print matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
```

```
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Free memory
    free_2d_array(matrix, rows);

    return 0;
}
```

Summary

The relationship between pointers and arrays is fundamental to C programming. Key points to remember:

1. **Array-Pointer Equivalence**: Arrays decay to pointers in most contexts
2. **Pointer Subscripting**: Pointers can use array notation and vice versa
3. **Pointer Arithmetic**: Operations are scaled by data type size
4. **Function Parameters**: Arrays pass as pointers to functions
5. **Dynamic Arrays**: Pointers enable runtime array allocation
6. **Multi-dimensional Arrays**: Various pointer techniques for 2D arrays
7. **Memory Management**: Proper allocation and deallocation of dynamic arrays

Understanding this relationship enables efficient array manipulation, dynamic memory allocation, and implementation of complex data structures in C.

Dynamic Memory Management

Introduction

Dynamic memory management is a crucial aspect of C programming that allows programs to allocate and deallocate memory at runtime. Unlike static memory allocation where the size and lifetime of variables are determined at compile time, dynamic memory allocation provides flexibility for handling data structures whose size is unknown until the program runs. This capability is essential for implementing complex data structures like linked lists, trees, and dynamic arrays.

Memory Layout in C Programs

Program Memory Sections

A C program's memory is typically organized into several sections:

1. **Text Segment**: Contains executable code
2. **Data Segment**: Contains global and static variables
3. **Stack**: Contains local variables and function call information
4. **Heap**: Contains dynamically allocated memory

Stack vs. Heap

```
// Stack allocation - automatic memory management
void stack_example() {
    int local_array[100];  // Allocated on stack
    // Automatically deallocated when function returns
}

// Heap allocation - manual memory management
void heap_example() {
    int *dynamic_array = malloc(100 * sizeof(int));  // Allocated on heap
    // Must be manually deallocated with free()
    free(dynamic_array);
}
```

Dynamic Memory Allocation Functions

malloc()

The `malloc()` function allocates a block of memory of specified size:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory for 10 integers
    int *arr = malloc(10 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Use the allocated memory
    for (int i = 0; i < 10; i++) {
        arr[i] = i * i;
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    // Free the allocated memory
    free(arr);
    arr = NULL;  // Prevent dangling pointer

    return 0;
}
```

calloc()

The `calloc()` function allocates memory and initializes it to zero:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate and zero-initialize memory for 10 integers
    int *arr = calloc(10, sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
```

```
    // Memory is already zero-initialized
    for (int i = 0; i < 10; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);  // All zeros
    }

    // Free the allocated memory
    free(arr);
    arr = NULL;

    return 0;
}
```

realloc()

The `realloc()` function changes the size of previously allocated memory:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Initial allocation
    int *arr = malloc(5 * sizeof(int));
    if (arr == NULL) {
        printf("Initial allocation failed\n");
        return 1;
    }

    // Initialize array
    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
    }

    printf("Original array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Resize array to hold 10 elements
    arr = realloc(arr, 10 * sizeof(int));
    if (arr == NULL) {
        printf("Reallocation failed\n");
        return 1;
    }
```

```
    // Initialize new elements
    for (int i = 5; i < 10; i++) {
        arr[i] = (i + 1) * 2;
    }

    printf("Resized array: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Free memory
    free(arr);
    return 0;
}
```

free()

The `free()` function deallocates previously allocated memory:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory
    int *ptr = malloc(100 * sizeof(int));
    if (ptr == NULL) {
        printf("Allocation failed\n");
        return 1;
    }

    // Use memory...
    for (int i = 0; i < 100; i++) {
        ptr[i] = i;
    }

    // Deallocate memory
    free(ptr);
    ptr = NULL;  // Prevent dangling pointer

    return 0;
}
```

Memory Allocation Strategies

Single Element Allocation

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    char name[50];
} Person;

int main() {
    // Allocate single structure
    Person *person = malloc(sizeof(Person));
    if (person == NULL) {
        printf("Allocation failed\n");
        return 1;
    }

    // Initialize structure
    person->id = 1;
    strcpy(person->name, "John Doe");

    printf("ID: %d, Name: %s\n", person->id, person->name);

    // Free memory
    free(person);
    person = NULL;

    return 0;
}
```

Array Allocation

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size;
    printf("Enter array size: ");
    scanf("%d", &size);

    // Allocate dynamic array
    int *arr = malloc(size * sizeof(int));
```

```
    if (arr == NULL) {
        printf("Allocation failed\n");
        return 1;
    }

    // Initialize array
    for (int i = 0; i < size; i++) {
        arr[i] = i * i;
    }

    // Use array
    printf("Array elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Free memory
    free(arr);
    arr = NULL;

    return 0;
}
```

2D Array Allocation

```
#include <stdio.h>
#include <stdlib.h>

int** create_2d_array(int rows, int cols) {
    // Allocate array of row pointers
    int **matrix = malloc(rows * sizeof(int*));
    if (matrix == NULL) return NULL;

    // Allocate each row
    for (int i = 0; i < rows; i++) {
        matrix[i] = malloc(cols * sizeof(int));
        if (matrix[i] == NULL) {
            // Clean up on failure
            for (int j = 0; j < i; j++) {
                free(matrix[j]);
            }
            free(matrix);
            return NULL;
        }
```

```
    }

    return matrix;
}

void free_2d_array(int **matrix, int rows) {
    for (int i = 0; i < rows; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int main() {
    int rows = 3, cols = 4;
    int **matrix = create_2d_array(rows, cols);

    if (matrix == NULL) {
        printf("Matrix allocation failed\n");
        return 1;
    }

    // Initialize matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = i * cols + j;
        }
    }

    // Print matrix
    printf("Matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Free memory
    free_2d_array(matrix, rows);

    return 0;
}
```

Memory Management Best Practices

Error Checking

Always check if memory allocation was successful:

```
int *ptr = malloc(100 * sizeof(int));
if (ptr == NULL) {
    // Handle allocation failure
    fprintf(stderr, "Memory allocation failed\n");
    exit(EXIT_FAILURE);
}
```

Preventing Dangling Pointers

Set pointers to NULL after freeing:

```
free(ptr);
ptr = NULL;  // Prevent dangling pointer
```

Avoiding Memory Leaks

Ensure every malloc/calloc/realloc has a corresponding free:

```
// Good practice
int *arr = malloc(10 * sizeof(int));
if (arr != NULL) {
    // Use array...
    free(arr);
    arr = NULL;
}
```

Memory Reallocation Safety

Handle realloc failure properly:

```
int *temp = realloc(arr, new_size * sizeof(int));
if (temp == NULL) {
    // realloc failed, original array still valid
    // Handle error without losing original data
    fprintf(stderr, "Reallocation failed\n");
    // arr is still valid
} else {
    // realloc succeeded
```

```
    arr = temp;
}
```

Common Memory Management Patterns

Dynamic String Handling

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* duplicate_string(const char *source) {
    if (source == NULL) return NULL;

    size_t len = strlen(source);
    char *copy = malloc((len + 1) * sizeof(char));
    if (copy == NULL) return NULL;

    strcpy(copy, source);
    return copy;
}

char* append_strings(const char *str1, const char *str2) {
    if (str1 == NULL || str2 == NULL) return NULL;

    size_t len1 = strlen(str1);
    size_t len2 = strlen(str2);
    char *result = malloc((len1 + len2 + 1) * sizeof(char));
    if (result == NULL) return NULL;

    strcpy(result, str1);
    strcat(result, str2);

    return result;
}

int main() {
    char *original = "Hello, ";
    char *append = "World!";

    char *copy = duplicate_string(original);
    char *combined = append_strings(original, append);

    if (copy != NULL) {
        printf("Copy: %s\n", copy);
```

```
        free(copy);
    }

    if (combined != NULL) {
        printf("Combined: %s\n", combined);
        free(combined);
    }

    return 0;
}
```

Dynamic Array with Resizing

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    size_t size;
    size_t capacity;
} DynamicArray;

DynamicArray* create_array(size_t initial_capacity) {
    DynamicArray *arr = malloc(sizeof(DynamicArray));
    if (arr == NULL) return NULL;

    arr->data = malloc(initial_capacity * sizeof(int));
    if (arr->data == NULL) {
        free(arr);
        return NULL;
    }

    arr->size = 0;
    arr->capacity = initial_capacity;
    return arr;
}

void push_back(DynamicArray *arr, int value) {
    if (arr->size >= arr->capacity) {
        // Double the capacity
        size_t new_capacity = arr->capacity * 2;
        int *temp = realloc(arr->data, new_capacity * sizeof(int));
        if (temp == NULL) return;  // Handle allocation failure

        arr->data = temp;
```

```
        arr->capacity = new_capacity;
    }

    arr->data[arr->size] = value;
    arr->size++;
}

void free_array(DynamicArray *arr) {
    if (arr != NULL) {
        free(arr->data);
        free(arr);
    }
}

int main() {
    DynamicArray *arr = create_array(2);
    if (arr == NULL) {
        printf("Array creation failed\n");
        return 1;
    }

    // Add elements (will trigger reallocation)
    for (int i = 0; i < 10; i++) {
        push_back(arr, i * i);
    }

    // Print elements
    printf("Array elements: ");
    for (size_t i = 0; i < arr->size; i++) {
        printf("%d ", arr->data[i]);
    }
    printf("\n");

    // Clean up
    free_array(arr);

    return 0;
}
```

Memory Debugging Tools

Valgrind (Linux/Unix)

Valgrind is a powerful tool for detecting memory errors:

```
# Compile with debugging information
gcc -g -Wall program.c -o program

# Run with Valgrind
valgrind --leak-check=full ./program
```

AddressSanitizer (GCC/Clang)

AddressSanitizer detects memory errors at runtime:

```
# Compile with AddressSanitizer
gcc -fsanitize=address -g program.c -o program

# Run program (errors will be reported automatically)
./program
```

Static Analysis Tools

Tools like cppcheck can detect potential memory issues:

```
# Static analysis
cppcheck --enable=all program.c
```

Common Memory Errors and Solutions

Memory Leaks

```
// Problem: Memory leak
void memory_leak() {
    int *ptr = malloc(100 * sizeof(int));
    // Missing free(ptr) - memory leak!
}

// Solution: Always free allocated memory
void no_memory_leak() {
    int *ptr = malloc(100 * sizeof(int));
    if (ptr != NULL) {
        // Use memory...
        free(ptr);
        ptr = NULL;
    }
}
```

Dangling Pointers

```
// Problem: Dangling pointer
void dangling_pointer() {
    int *ptr = malloc(sizeof(int));
    *ptr = 42;
    free(ptr);
    printf("%d\n", *ptr);  // Undefined behavior!
}

// Solution: Set pointer to NULL after freeing
void no_dangling_pointer() {
    int *ptr = malloc(sizeof(int));
    *ptr = 42;
    free(ptr);
    ptr = NULL;
    // if (ptr != NULL) printf("%d\n", *ptr);  // Safe check
}
```

Buffer Overflows

```
// Problem: Buffer overflow
void buffer_overflow() {
    int *arr = malloc(5 * sizeof(int));
    for (int i = 0; i < 10; i++) {
        arr[i] = i;  // Writing beyond allocated memory
    }
    free(arr);
}

// Solution: Always check bounds
void no_buffer_overflow() {
    int *arr = malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++) {  // Correct bound
        arr[i] = i;
    }
    free(arr);
}
```

Advanced Memory Management (C11 and Later)

aligned_alloc()

The `aligned_alloc()` function allocates aligned memory (C11):

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate 100 bytes aligned to 16-byte boundary
    void *ptr = aligned_alloc(16, 100);
    if (ptr == NULL) {
        printf("Aligned allocation failed\n");
        return 1;
    }

    printf("Allocated address: %p\n", ptr);
    printf("Alignment check: %s\n",
           ((uintptr_t)ptr % 16 == 0) ? "OK" : "FAIL");

    free(ptr);
    return 0;
}
```

Practical Examples

Memory Pool Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    void *memory;
    size_t block_size;
    size_t total_blocks;
    size_t free_blocks;
    char *free_list;
} MemoryPool;

MemoryPool* create_pool(size_t block_size, size_t num_blocks) {
    MemoryPool *pool = malloc(sizeof(MemoryPool));
    if (pool == NULL) return NULL;

    pool->memory = malloc(block_size * num_blocks);
    if (pool->memory == NULL) {
        free(pool);
        return NULL;
    }
```

```
    pool->block_size = block_size;
    pool->total_blocks = num_blocks;
    pool->free_blocks = num_blocks;

    // Initialize free list
    pool->free_list = malloc(num_blocks);
    if (pool->free_list == NULL) {
        free(pool->memory);
        free(pool);
        return NULL;
    }

    for (size_t i = 0; i < num_blocks; i++) {
        pool->free_list[i] = 1;  // 1 = free, 0 = allocated
    }

    return pool;
}

void* pool_alloc(MemoryPool *pool) {
    if (pool->free_blocks == 0) return NULL;

    for (size_t i = 0; i < pool->total_blocks; i++) {
        if (pool->free_list[i]) {
            pool->free_list[i] = 0;
            pool->free_blocks--;
            return (char*)pool->memory + (i * pool->block_size);
        }
    }

    return NULL;
}

void pool_free(MemoryPool *pool, void *ptr) {
    if (ptr == NULL || ptr < pool->memory ||
        ptr >= (char*)pool->memory + (pool->total_blocks * pool->block_size)) {
        return;
    }

    size_t index = ((char*)ptr - (char*)pool->memory) / pool->block_size;
    if (index < pool->total_blocks && !pool->free_list[index]) {
        pool->free_list[index] = 1;
        pool->free_blocks++;
    }
}

void destroy_pool(MemoryPool *pool) {
```

```
    if (pool != NULL) {
        free(pool->memory);
        free(pool->free_list);
        free(pool);
    }
}

int main() {
    // Create pool for 10 integers
    MemoryPool *pool = create_pool(sizeof(int), 10);
    if (pool == NULL) {
        printf("Pool creation failed\n");
        return 1;
    }

    // Allocate from pool
    int *numbers[5];
    for (int i = 0; i < 5; i++) {
        numbers[i] = (int*)pool_alloc(pool);
        if (numbers[i] != NULL) {
            *numbers[i] = i * 10;
            printf("Allocated %d at %p\n", *numbers[i], (void*)numbers[i]);
        }
    }

    // Free some allocations
    pool_free(pool, numbers[2]);
    pool_free(pool, numbers[4]);

    // Allocate again (should reuse freed blocks)
    int *new_num = (int*)pool_alloc(pool);
    if (new_num != NULL) {
        *new_num = 999;
        printf("Reallocated %d at %p\n", *new_num, (void*)new_num);
    }

    // Clean up
    destroy_pool(pool);

    return 0;
}
```

Smart Pointer-like Implementation

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    void *ptr;
    size_t ref_count;
    void (*destructor)(void*);
} SmartPointer;

SmartPointer* create_smart_pointer(size_t size, void (*destructor)(void*)) {
    SmartPointer *sp = malloc(sizeof(SmartPointer));
    if (sp == NULL) return NULL;

    sp->ptr = malloc(size);
    if (sp->ptr == NULL) {
        free(sp);
        return NULL;
    }

    sp->ref_count = 1;
    sp->destructor = destructor;

    return sp;
}

SmartPointer* smart_pointer_retain(SmartPointer *sp) {
    if (sp != NULL) {
        sp->ref_count++;
    }
    return sp;
}

void smart_pointer_release(SmartPointer *sp) {
    if (sp != NULL) {
        sp->ref_count--;
        if (sp->ref_count == 0) {
            if (sp->destructor != NULL) {
                sp->destructor(sp->ptr);
            } else {
                free(sp->ptr);
            }
            free(sp);
        }
    }
}
```

```c
int main() {
    // Create smart pointer for integer
    SmartPointer *sp = create_smart_pointer(sizeof(int), NULL);
    if (sp == NULL) {
        printf("Smart pointer creation failed\n");
        return 1;
    }

    // Use the smart pointer
    *(int*)sp->ptr = 42;
    printf("Value: %d\n", *(int*)sp->ptr);

    // Create another reference
    SmartPointer *sp2 = smart_pointer_retain(sp);
    printf("Reference count: %zu\n", sp->ref_count);

    // Release first reference
    smart_pointer_release(sp);
    printf("Reference count after release: %zu\n", sp2->ref_count);

    // Release second reference (triggers deallocation)
    smart_pointer_release(sp2);

    return 0;
}
```

Summary

Dynamic memory management is a powerful feature of C that provides flexibility for handling data structures whose size is unknown at compile time. Key points to remember:

1. **Memory Allocation Functions**: malloc, calloc, realloc, and free for memory management
2. **Memory Layout**: Understanding stack vs. heap allocation
3. **Error Handling**: Always check for allocation failures
4. **Memory Safety**: Prevent memory leaks, dangling pointers, and buffer overflows
5. **Best Practices**: Set pointers to NULL after freeing, check bounds, and use debugging tools
6. **Advanced Features**: aligned_alloc for aligned memory (C11+)
7. **Debugging Tools**: Valgrind, AddressSanitizer, and static analysis tools

Proper dynamic memory management is essential for writing robust, efficient C programs and avoiding common memory-related bugs.

Advanced Pointer Concepts

Introduction

Advanced pointer concepts in C extend beyond basic pointer usage to include sophisticated techniques like pointers to pointers, function pointers, void pointers, and const correctness. These concepts enable powerful programming paradigms such as generic programming, callback mechanisms, and complex data structures. Understanding these advanced concepts is essential for writing flexible, reusable, and efficient C code.

Pointers to Pointers

Basic Concept

A pointer to a pointer is a variable that stores the address of another pointer:

```c
#include <stdio.h>

int main() {
    int x = 42;
    int *ptr = &x;        // Pointer to integer
    int **ptr_to_ptr = &ptr;  // Pointer to pointer to integer

    printf("Value of x: %d\n", x);
    printf("Value via ptr: %d\n", *ptr);
    printf("Value via ptr_to_ptr: %d\n", **ptr_to_ptr);

    // Modifying through pointer to pointer
    **ptr_to_ptr = 100;
    printf("After modification, x = %d\n", x);

    return 0;
}
```

Practical Applications

Pointers to pointers are commonly used for:

1. **Modifying pointers in functions**

2. **Dynamic memory allocation for pointers**
3. **Implementing complex data structures**

```c
#include <stdio.h>
#include <stdlib.h>

// Function to allocate memory and update pointer
void allocate_memory(int **ptr, size_t size) {
    *ptr = malloc(size * sizeof(int));
    if (*ptr != NULL) {
        for (size_t i = 0; i < size; i++) {
            (*ptr)[i] = (int)i;
        }
    }
}

// Function to swap two pointers
void swap_pointers(int **ptr1, int **ptr2) {
    int *temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
}

int main() {
    int *arr1 = NULL;
    int *arr2 = NULL;

    // Allocate memory through pointer to pointer
    allocate_memory(&arr1, 5);
    allocate_memory(&arr2, 3);

    if (arr1 != NULL && arr2 != NULL) {
        printf("Array 1: ");
        for (int i = 0; i < 5; i++) {
            printf("%d ", arr1[i]);
        }
        printf("\n");

        printf("Array 2: ");
        for (int i = 0; i < 3; i++) {
            printf("%d ", arr2[i]);
        }
        printf("\n");

        // Swap pointers
        swap_pointers(&arr1, &arr2);
```

```
        printf("After swap:\n");
        printf("Array 1: ");
        for (int i = 0; i < 3; i++) {
            printf("%d ", arr1[i]);
        }
        printf("\n");

        // Free memory
        free(arr1);
        free(arr2);
    }

    return 0;
}
```

Function Pointers

Basic Declaration and Usage

Function pointers store the address of functions and can be used to call functions indirectly:

```
#include <stdio.h>

// Function prototypes
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }

int main() {
    // Function pointer declaration
    int (*operation)(int, int);

    // Assign function addresses
    operation = add;
    printf("5 + 3 = %d\n", operation(5, 3));

    operation = subtract;
    printf("5 - 3 = %d\n", operation(5, 3));

    operation = multiply;
    printf("5 * 3 = %d\n", operation(5, 3));

    return 0;
}
```

Function Pointer Arrays

Function pointers can be stored in arrays for dispatch tables:

```
#include <stdio.h>

// Calculator operations
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

int main() {
    // Array of function pointers
    int (*operations[])(int, int) = {add, subtract, multiply, divide};
    const char *op_names[] = {"+", "-", "*", "/"};

    int a = 10, b = 5;

    // Use function pointer array
    for (int i = 0; i < 4; i++) {
        int result = operations[i](a, b);
        printf("%d %s %d = %d\n", a, op_names[i], b, result);
    }

    return 0;
}
```

Callback Functions

Function pointers enable callback mechanisms:

```
#include <stdio.h>

// Callback function type
typedef void (*callback_func)(int value);

// Function that takes a callback
void process_array(int *arr, size_t size, callback_func callback) {
    for (size_t i = 0; i < size; i++) {
        callback(arr[i]);
    }
}

// Callback functions
void print_value(int value) {
```

```
    printf("Value: %d\n", value);
}

void square_value(int value) {
    printf("Square: %d\n", value * value);
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    size_t size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Printing values:\n");
    process_array(numbers, size, print_value);

    printf("\nPrinting squares:\n");
    process_array(numbers, size, square_value);

    return 0;
}
```

Function Pointers as Parameters

Function pointers are commonly used as parameters for generic algorithms:

```
#include <stdio.h>

// Comparison function type
typedef int (*compare_func)(const void *a, const void *b);

// Generic sorting function
void bubble_sort(void *arr, size_t count, size_t size, compare_func compare) {
    char *array = (char*)arr;

    for (size_t i = 0; i < count - 1; i++) {
        for (size_t j = 0; j < count - i - 1; j++) {
            char *elem1 = array + j * size;
            char *elem2 = array + (j + 1) * size;

            if (compare(elem1, elem2) > 0) {
                // Swap elements
                for (size_t k = 0; k < size; k++) {
                    char temp = elem1[k];
                    elem1[k] = elem2[k];
                    elem2[k] = temp;
                }
            }
```

```
            }
        }
    }

    // Comparison functions
    int compare_ints(const void *a, const void *b) {
        int int_a = *(const int*)a;
        int int_b = *(const int*)b;
        return (int_a > int_b) - (int_a < int_b);
    }

    int compare_strings(const void *a, const void *b) {
        const char *str_a = *(const char**)a;
        const char *str_b = *(const char**)b;
        return strcmp(str_a, str_b);
    }

    int main() {
        // Sort integers
        int numbers[] = {64, 34, 25, 12, 22, 11, 90};
        size_t num_count = sizeof(numbers) / sizeof(numbers[0]);

        printf("Original numbers: ");
        for (size_t i = 0; i < num_count; i++) {
            printf("%d ", numbers[i]);
        }
        printf("\n");

        bubble_sort(numbers, num_count, sizeof(int), compare_ints);

        printf("Sorted numbers: ");
        for (size_t i = 0; i < num_count; i++) {
            printf("%d ", numbers[i]);
        }
        printf("\n");

        return 0;
    }
```

Void Pointers

Basic Concept

Void pointers can point to any data type but cannot be dereferenced directly:

```c
#include <stdio.h>

int main() {
    int x = 42;
    float y = 3.14f;
    char z = 'A';

    // Void pointer can point to any type
    void *ptr;

    ptr = &x;
    printf("Integer: %d\n", *(int*)ptr);

    ptr = &y;
    printf("Float: %.2f\n", *(float*)ptr);

    ptr = &z;
    printf("Character: %c\n", *(char*)ptr);

    return 0;
}
```

Generic Functions with Void Pointers

Void pointers enable generic programming:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Generic swap function
void generic_swap(void *a, void *b, size_t size) {
    char *ptr_a = (char*)a;
    char *ptr_b = (char*)b;

    for (size_t i = 0; i < size; i++) {
        char temp = ptr_a[i];
        ptr_a[i] = ptr_b[i];
        ptr_b[i] = temp;
    }
}

// Generic copy function
void generic_copy(void *dest, const void *src, size_t size) {
    char *d = (char*)dest;
    const char *s = (const char*)src;
```

```
    for (size_t i = 0; i < size; i++) {
        d[i] = s[i];
    }
}

int main() {
    // Swap integers
    int a = 10, b = 20;
    printf("Before swap: a = %d, b = %d\n", a, b);
    generic_swap(&a, &b, sizeof(int));
    printf("After swap: a = %d, b = %d\n", a, b);

    // Swap doubles
    double x = 3.14, y = 2.71;
    printf("Before swap: x = %.2f, y = %.2f\n", x, y);
    generic_swap(&x, &y, sizeof(double));
    printf("After swap: x = %.2f, y = %.2f\n", x, y);

    // Copy structure
    struct {
        int id;
        char name[20];
    } person1 = {1, "John"}, person2;

    generic_copy(&person2, &person1, sizeof(person1));
    printf("Copied person: ID = %d, Name = %s\n", person2.id, person2.name);

    return 0;
}
```

Void Pointer Arrays

Arrays of void pointers can store pointers to different types:

```
#include <stdio.h>

int main() {
    int x = 42;
    float y = 3.14f;
    char z = 'A';

    // Array of void pointers
    void *array[3] = {&x, &y, &z};

    // Access elements with proper casting
```

```
    printf("Integer: %d\n", *(int*)array[0]);
    printf("Float: %.2f\n", *(float*)array[1]);
    printf("Character: %c\n", *(char*)array[2]);

    return 0;
}
```

Const Correctness with Pointers

Pointer to Constant

A pointer to constant prevents modification of the pointed-to value:

```
#include <stdio.h>

int main() {
    int x = 42;
    int y = 100;

    // Pointer to constant - cannot modify value
    const int *ptr_to_const = &x;

    // This is OK - can change where pointer points
    ptr_to_const = &y;

    // This is ERROR - cannot modify value through pointer
    // *ptr_to_const = 50;  // Compilation error

    printf("Value: %d\n", *ptr_to_const);

    return 0;
}
```

Constant Pointer

A constant pointer cannot be changed to point to a different location:

```
#include <stdio.h>

int main() {
    int x = 42;
    int y = 100;

    // Constant pointer - cannot change pointer value
```

```
    int *const const_ptr = &x;

    // This is OK - can modify value through pointer
    *const_ptr = 50;

    // This is ERROR - cannot change where pointer points
    // const_ptr = &y;  // Compilation error

    printf("Value: %d\n", *const_ptr);

    return 0;
}
```

Constant Pointer to Constant

Both the pointer and the pointed-to value are constant:

```
#include <stdio.h>

int main() {
    int x = 42;
    int y = 100;

    // Constant pointer to constant - nothing can be changed
    const int *const const_ptr_to_const = &x;

    // Both of these are ERROR
    // *const_ptr_to_const = 50;  // Cannot modify value
    // const_ptr_to_const = &y;   // Cannot change pointer

    printf("Value: %d\n", *const_ptr_to_const);

    return 0;
}
```

Const Correctness in Function Parameters

Using const in function parameters improves code safety and clarity:

```
#include <stdio.h>
#include <string.h>

// Function that doesn't modify the string
size_t safe_string_length(const char *str) {
    // str is a pointer to constant - cannot modify the string
```

```
    return strlen(str);
}

// Function that modifies the string
void unsafe_string_uppercase(char *str) {
    // str is a pointer to non-constant - can modify the string
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] >= 'a' && str[i] <= 'z') {
            str[i] = str[i] - 'a' + 'A';
        }
    }
}

// Function that takes a constant pointer to array
void print_array(const int *arr, size_t size) {
    // arr is a pointer to constant - cannot modify array elements
    for (size_t i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    const char *text = "Hello, World!";
    char mutable_text[] = "hello, world!";

    printf("Length: %zu\n", safe_string_length(text));

    printf("Before: %s\n", mutable_text);
    unsafe_string_uppercase(mutable_text);
    printf("After: %s\n", mutable_text);

    int numbers[] = {1, 2, 3, 4, 5};
    print_array(numbers, 5);

    return 0;
}
```

Advanced Pointer Techniques

Pointer Arithmetic with Structs

Pointer arithmetic can be used with structures:

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    char name[20];
    float score;
} Student;

int main() {
    // Array of structures
    Student class[3] = {
        {1, "Alice", 95.5},
        {2, "Bob", 87.0},
        {3, "Charlie", 92.5}
    };

    // Pointer to first element
    Student *ptr = class;

    // Access elements using pointer arithmetic
    for (int i = 0; i < 3; i++) {
        printf("Student %d: ID=%d, Name=%s, Score=%.1f\n",
               i + 1, ptr->id, ptr->name, ptr->score);
        ptr++;  // Move to next structure
    }

    return 0;
}
```

Flexible Array Members (C99)

Structures can have flexible array members:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int count;
    int data[];  // Flexible array member (C99)
} DynamicArray;

DynamicArray* create_dynamic_array(int count) {
    // Allocate memory for structure + array
    DynamicArray *arr = malloc(sizeof(DynamicArray) + count * sizeof(int));
```

```
    if (arr != NULL) {
        arr->count = count;
        // arr->data can now be used as array of 'count' integers
    }
    return arr;
}

int main() {
    DynamicArray *arr = create_dynamic_array(5);
    if (arr != NULL) {
        // Initialize array elements
        for (int i = 0; i < arr->count; i++) {
            arr->data[i] = i * i;
        }

        // Print array elements
        printf("Array elements: ");
        for (int i = 0; i < arr->count; i++) {
            printf("%d ", arr->data[i]);
        }
        printf("\n");

        // Free memory
        free(arr);
    }

    return 0;
}
```

Restricted Pointers (C99)

The `restrict` keyword tells the compiler that a pointer is the only way to access the object:

```
#include <stdio.h>

// Function with restricted pointers
void add_arrays(int *restrict result,
                const int *restrict a,
                const int *restrict b,
                size_t size) {
    // Compiler knows a, b, and result don't overlap
    for (size_t i = 0; i < size; i++) {
        result[i] = a[i] + b[i];
    }
}
```

```c
int main() {
    int a[] = {1, 2, 3, 4, 5};
    int b[] = {10, 20, 30, 40, 50};
    int result[5];

    add_arrays(result, a, b, 5);

    printf("Result: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    return 0;
}
```

Practical Examples

Generic Linked List Implementation

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    void *data;
    struct Node *next;
} Node;

typedef struct {
    Node *head;
    size_t data_size;
} LinkedList;

LinkedList* create_list(size_t data_size) {
    LinkedList *list = malloc(sizeof(LinkedList));
    if (list != NULL) {
        list->head = NULL;
        list->data_size = data_size;
    }
    return list;
}

void list_append(LinkedList *list, const void *data) {
    Node *new_node = malloc(sizeof(Node));
```

```
    if (new_node == NULL) return;

    new_node->data = malloc(list->data_size);
    if (new_node->data == NULL) {
        free(new_node);
        return;
    }

    // Copy data
    char *src = (char*)data;
    char *dest = (char*)new_node->data;
    for (size_t i = 0; i < list->data_size; i++) {
        dest[i] = src[i];
    }

    new_node->next = NULL;

    // Add to end of list
    if (list->head == NULL) {
        list->head = new_node;
    } else {
        Node *current = list->head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = new_node;
    }
}

void list_print_int(LinkedList *list) {
    Node *current = list->head;
    while (current != NULL) {
        printf("%d ", *(int*)current->data);
        current = current->next;
    }
    printf("\n");
}

void list_free(LinkedList *list) {
    Node *current = list->head;
    while (current != NULL) {
        Node *next = current->next;
        free(current->data);
        free(current);
        current = next;
    }
    free(list);
```

```
}

int main() {
    LinkedList *int_list = create_list(sizeof(int));

    // Add integers to list
    for (int i = 1; i <= 5; i++) {
        list_append(int_list, &i);
    }

    printf("List contents: ");
    list_print_int(int_list);

    list_free(int_list);

    return 0;
}
```

Event System with Function Pointers

```
#include <stdio.h>
#include <stdlib.h>

// Event types
typedef enum {
    EVENT_CLICK,
    EVENT_KEYPRESS,
    EVENT_MOUSEMOVE,
    EVENT_COUNT
} EventType;

// Event data structures
typedef struct {
    int x, y;
} ClickEvent;

typedef struct {
    char key;
} KeyPressEvent;

typedef struct {
    int x, y;
} MouseMoveEvent;

// Event handler function type
```

```c
typedef void (*EventHandler)(void *event_data);

// Event system structure
typedef struct {
    EventHandler handlers[EVENT_COUNT];
} EventSystem;

// Event handler functions
void handle_click(void *data) {
    ClickEvent *event = (ClickEvent*)data;
    printf("Click at (%d, %d)\n", event->x, event->y);
}

void handle_keypress(void *data) {
    KeyPressEvent *event = (KeyPressEvent*)data;
    printf("Key pressed: %c\n", event->key);
}

void handle_mousemove(void *data) {
    MouseMoveEvent *event = (MouseMoveEvent*)data;
    printf("Mouse moved to (%d, %d)\n", event->x, event->y);
}

// Event system functions
EventSystem* create_event_system() {
    EventSystem *system = malloc(sizeof(EventSystem));
    if (system != NULL) {
        for (int i = 0; i < EVENT_COUNT; i++) {
            system->handlers[i] = NULL;
        }
    }
    return system;
}

void register_handler(EventSystem *system, EventType type, EventHandler handler) {
    if (type >= 0 && type < EVENT_COUNT) {
        system->handlers[type] = handler;
    }
}

void trigger_event(EventSystem *system, EventType type, void *event_data) {
    if (type >= 0 && type < EVENT_COUNT && system->handlers[type] != NULL) {
        system->handlers[type](event_data);
    }
}
```

```c
void destroy_event_system(EventSystem *system) {
    free(system);
}

int main() {
    EventSystem *events = create_event_system();

    // Register event handlers
    register_handler(events, EVENT_CLICK, handle_click);
    register_handler(events, EVENT_KEYPRESS, handle_keypress);
    register_handler(events, EVENT_MOUSEMOVE, handle_mousemove);

    // Trigger events
    ClickEvent click = {100, 200};
    trigger_event(events, EVENT_CLICK, &click);

    KeyPressEvent keypress = {'A'};
    trigger_event(events, EVENT_KEYPRESS, &keypress);

    MouseMoveEvent mousemove = {50, 75};
    trigger_event(events, EVENT_MOUSEMOVE, &mousemove);

    destroy_event_system(events);

    return 0;
}
```

Summary

Advanced pointer concepts in C provide powerful mechanisms for creating flexible and efficient programs. Key points to remember:

1. **Pointers to Pointers**: Enable modification of pointers in functions and complex data structures
2. **Function Pointers**: Enable callback mechanisms, dispatch tables, and generic algorithms
3. **Void Pointers**: Enable generic programming and storage of different data types
4. **Const Correctness**: Improves code safety and communicates intent to other developers
5. **Advanced Techniques**: Flexible array members, restricted pointers, and pointer arithmetic with structures

These advanced concepts are essential for writing professional C code, implementing complex data structures, and creating reusable libraries. Understanding them enables programmers to leverage the full power of C's pointer system while maintaining code safety and clarity.

Common Pointer Pitfalls

Introduction

Pointers are one of the most powerful features of the C programming language, but they are also one of the most error-prone. Incorrect pointer usage can lead to subtle bugs, program crashes, security vulnerabilities, and undefined behavior. Understanding common pointer pitfalls and how to avoid them is crucial for writing robust, reliable C programs. This chapter explores the most frequent pointer-related errors and provides strategies for preventing them.

Dangling Pointers

What Are Dangling Pointers?

A dangling pointer is a pointer that points to memory that has been deallocated or is no longer valid. Dereferencing a dangling pointer leads to undefined behavior.

Common Causes

1. Premature Deallocation

```
#include <stdio.h>
#include <stdlib.h>

int* create_integer(int value) {
    int *ptr = malloc(sizeof(int));
    if (ptr != NULL) {
        *ptr = value;
    }
    return ptr;
}

int main() {
    int *ptr = create_integer(42);

    // Problem: Using pointer after deallocation
    free(ptr);
    printf("Value: %d\n", *ptr);  // Dangling pointer - undefined behavior!
```

```
    return 0;
}
```

2. Returning Pointer to Local Variable

```
#include <stdio.h>

int* get_pointer() {
    int local_var = 42;
    return &local_var;  // Problem: Returning pointer to local variable
}

int main() {
    int *ptr = get_pointer();
    printf("Value: %d\n", *ptr);  // Dangling pointer - undefined behavior!

    return 0;
}
```

Prevention Strategies

1. Set Pointers to NULL After Freeing

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(sizeof(int));
    if (ptr != NULL) {
        *ptr = 42;
        printf("Value: %d\n", *ptr);

        free(ptr);
        ptr = NULL;  // Prevent dangling pointer

        // Safe check before use
        if (ptr != NULL) {
            printf("Value: %d\n", *ptr);
        } else {
            printf("Pointer is NULL\n");
        }
    }

    return 0;
```

```
}
```

2. Avoid Returning Pointers to Local Variables

```c
#include <stdio.h>
#include <stdlib.h>

// Solution 1: Use dynamic allocation
int* create_integer_safe(int value) {
    int *ptr = malloc(sizeof(int));
    if (ptr != NULL) {
        *ptr = value;
    }
    return ptr;
}

// Solution 2: Use output parameters
void get_value(int *output) {
    *output = 42;  // Modify through parameter
}

int main() {
    // Using dynamic allocation
    int *ptr = create_integer_safe(42);
    if (ptr != NULL) {
        printf("Value: %d\n", *ptr);
        free(ptr);
    }

    // Using output parameter
    int value;
    get_value(&value);
    printf("Value: %d\n", value);

    return 0;
}
```

Memory Leaks

What Are Memory Leaks?

A memory leak occurs when dynamically allocated memory is not properly deallocated, making it unavailable for reuse during program execution.

Common Causes

1. Forgetting to Call free()

```c
#include <stdio.h>
#include <stdlib.h>

void memory_leak_example() {
    int *ptr = malloc(100 * sizeof(int));
    if (ptr != NULL) {
        // Use the memory...
        for (int i = 0; i < 100; i++) {
            ptr[i] = i;
        }

        // Problem: Forgot to call free() - memory leak!
        // free(ptr);
    }
}

int main() {
    for (int i = 0; i < 1000; i++) {
        memory_leak_example();  // Leaks 400 bytes each iteration
    }

    return 0;
}
```

2. Losing Pointer References

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr1 = malloc(100 * sizeof(int));
    int *ptr2 = malloc(100 * sizeof(int));

    if (ptr1 != NULL && ptr2 != NULL) {
        // Use memory...

        // Problem: Losing reference to ptr1 - memory leak!
        ptr1 = ptr2;

        free(ptr2);  // Only frees one block
        // ptr1's memory is leaked
```

```
    }

    return 0;
}
```

Prevention Strategies

1. Use RAII-like Patterns

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    size_t size;
} ManagedArray;

ManagedArray* create_array(size_t size) {
    ManagedArray *arr = malloc(sizeof(ManagedArray));
    if (arr != NULL) {
        arr->data = malloc(size * sizeof(int));
        if (arr->data == NULL) {
            free(arr);
            return NULL;
        }
        arr->size = size;
    }
    return arr;
}

void destroy_array(ManagedArray *arr) {
    if (arr != NULL) {
        free(arr->data);
        free(arr);
    }
}

int main() {
    ManagedArray *arr = create_array(100);
    if (arr != NULL) {
        // Use array...

        // Always clean up
        destroy_array(arr);
    }
```

```
    return 0;
}
```

2. Match Every malloc with free

```
#include <stdio.h>
#include <stdlib.h>

void safe_memory_usage() {
    int *ptr = malloc(100 * sizeof(int));
    if (ptr == NULL) {
        return;  // Handle allocation failure
    }

    // Use the memory...
    for (int i = 0; i < 100; i++) {
        ptr[i] = i;
    }

    // Always free allocated memory
    free(ptr);
    ptr = NULL;  // Prevent dangling pointer
}
```

Buffer Overflows

What Are Buffer Overflows?

A buffer overflow occurs when more data is written to a buffer than it can hold, potentially corrupting adjacent memory.

Common Causes

1. Array Index Out of Bounds

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    // Problem: Writing beyond array bounds
    for (int i = 0; i < 10; i++) {  // i goes to 9, but array only has 5 elements
```

```
        arr[i] = i * i;  // Buffer overflow when i >= 5
    }

    return 0;
}
```

2. String Buffer Overflows

```
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[10];

    // Problem: String too long for buffer
    strcpy(buffer, "This string is too long for the buffer!");
    // Buffer overflow - undefined behavior!

    return 0;
}
```

Prevention Strategies

1. Always Check Array Bounds

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Safe: Check bounds before access
    for (int i = 0; i < size; i++) {
        arr[i] = i * i;
    }

    // Alternative: Use safer loop
    for (int i = 0; i < 5; i++) {
        arr[i] = i * i;
    }

    return 0;
}
```

2. Use Safe String Functions

```c
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[10];

    // Safe: Use strncpy with proper bounds checking
    const char *source = "This string is too long";
    strncpy(buffer, source, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';  // Ensure null termination

    printf("Buffer: %s\n", buffer);

    return 0;
}
```

Uninitialized Pointers

What Are Uninitialized Pointers?

An uninitialized pointer contains garbage values and points to random memory locations. Dereferencing such pointers leads to undefined behavior.

Common Causes

1. Using Uninitialized Local Pointers

```c
#include <stdio.h>

int main() {
    int *ptr;  // Uninitialized pointer

    // Problem: Using uninitialized pointer
    *ptr = 42;  // Undefined behavior!

    return 0;
}
```

2. Partial Initialization

```c
#include <stdio.h>

int main() {
    int *ptr1, *ptr2;

    ptr1 = malloc(sizeof(int));
    if (ptr1 != NULL) {
        *ptr1 = 42;
    }

    // Problem: ptr2 is uninitialized
    *ptr2 = 100;  // Undefined behavior!

    free(ptr1);

    return 0;
}
```

Prevention Strategies

1. Always Initialize Pointers

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Good: Initialize to NULL
    int *ptr = NULL;

    // Safe: Check before use
    if (ptr != NULL) {
        *ptr = 42;
    }

    // Or initialize with valid address
    int value = 42;
    int *ptr2 = &value;  // Safe initialization

    printf("Value: %d\n", *ptr2);

    return 0;
}
```

2. Use Separate Declarations

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Good: Separate declarations for clarity
    int *ptr1 = NULL;
    int *ptr2 = NULL;

    ptr1 = malloc(sizeof(int));
    if (ptr1 != NULL) {
        *ptr1 = 42;
    }

    ptr2 = malloc(sizeof(int));
    if (ptr2 != NULL) {
        *ptr2 = 100;
    }

    // Safe usage
    if (ptr1 != NULL) {
        printf("ptr1: %d\n", *ptr1);
    }

    if (ptr2 != NULL) {
        printf("ptr2: %d\n", *ptr2);
    }

    // Clean up
    free(ptr1);
    free(ptr2);

    return 0;
}
```

Pointer Arithmetic Errors

Common Mistakes

1. Incorrect Pointer Arithmetic

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *ptr = arr;

    // Problem: Incorrect arithmetic
    ptr = ptr + 10;  // Points beyond array bounds

    // Undefined behavior when dereferencing
    // printf("Value: %d\n", *ptr);

    return 0;
}
```

2. Pointer Arithmetic with Different Types

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    char *char_ptr = (char*)arr;

    // Problem: Different pointer arithmetic
    printf("int ptr + 1: %p\n", (void*)(arr + 1));
    printf("char ptr + 1: %p\n", (void*)(char_ptr + 1));
    // These point to different memory locations!

    return 0;
}
```

Prevention Strategies

1. Validate Pointer Arithmetic

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *ptr = arr;
    int size = sizeof(arr) / sizeof(arr[0]);

    // Safe: Validate bounds before arithmetic
    int offset = 2;
    if (offset >= 0 && offset < size) {
        int *new_ptr = ptr + offset;
        printf("Value at offset %d: %d\n", offset, *new_ptr);
    } else {
        printf("Offset out of bounds\n");
    }

    return 0;
}
```

NULL Pointer Dereferencing

What Is NULL Pointer Dereferencing?

Dereferencing a NULL pointer leads to program crashes or undefined behavior.

Common Causes

1. Forgetting to Check Return Values

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(100 * sizeof(int));

    // Problem: Not checking for NULL
    *ptr = 42;  // Crash if malloc failed!

    free(ptr);
```

```
    return 0;
}
```

Prevention Strategies

1. Always Check for NULL

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(100 * sizeof(int));

    // Safe: Check for NULL
    if (ptr != NULL) {
        *ptr = 42;
        printf("Value: %d\n", *ptr);
        free(ptr);
    } else {
        printf("Memory allocation failed\n");
        return 1;
    }

    return 0;
}
```

Type Safety Issues

Common Problems

1. Incorrect Casting

```
#include <stdio.h>

int main() {
    int x = 42;
    int *int_ptr = &x;

    // Problem: Incorrect casting
    char *char_ptr = (char*)int_ptr;
    printf("Char value: %c\n", *char_ptr);  // Interprets as character

    return 0;
```

```
}
```

2. Function Pointer Mismatches

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
void print_message(void) { printf("Hello\n"); }

int main() {
    // Problem: Wrong function pointer type
    int (*func_ptr)(int, int) = (int (*)(int, int))print_message;

    // Undefined behavior when called
    // int result = func_ptr(5, 3);

    return 0;
}
```

Prevention Strategies

1. Use Proper Typedefs

```
#include <stdio.h>

// Safe: Use typedef for function pointers
typedef int (*binary_op)(int, int);

int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }

int main() {
    binary_op operations[] = {add, subtract};

    int result1 = operations[0](10, 5);  // Safe: add(10, 5)
    int result2 = operations[1](10, 5);  // Safe: subtract(10, 5)

    printf("10 + 5 = %d\n", result1);
    printf("10 - 5 = %d\n", result2);

    return 0;
}
```

Debugging Pointer Issues

Using Debugging Tools

1. Valgrind (Linux/Unix)

```
# Compile with debugging information
gcc -g -Wall program.c -o program

# Run with Valgrind
valgrind --leak-check=full ./program
```

2. AddressSanitizer (GCC/Clang)

```
# Compile with AddressSanitizer
gcc -fsanitize=address -g -Wall program.c -o program

# Run program (errors will be reported automatically)
./program
```

Static Analysis Tools

```
# Static analysis with cppcheck
cppcheck --enable=all program.c

# Static analysis with clang-analyzer
clang --analyze program.c
```

Best Practices for Pointer Safety

1. Initialize Pointers

```
// Good
int *ptr = NULL;

// Good
int value = 42;
int *ptr = &value;

// Bad
int *ptr;  // Uninitialized
```

2. Check for NULL

```
int *ptr = malloc(sizeof(int));
if (ptr != NULL) {
    *ptr = 42;
    // Use ptr...
    free(ptr);
    ptr = NULL;
}
```

3. Validate Array Bounds

```
int arr[10];
int size = sizeof(arr) / sizeof(arr[0]);

for (int i = 0; i < size; i++) {
    arr[i] = i;
}
```

4. Use Safe String Functions

```
char buffer[100];
const char *source = "Some text";

// Safe copy
strncpy(buffer, source, sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';
```

5. Match Allocation with Deallocation

```
// For malloc/calloc/realloc
int *ptr = malloc(100 * sizeof(int));
// ... use ptr ...
free(ptr);
ptr = NULL;

// For arrays
int *arr = calloc(100, sizeof(int));
// ... use arr ...
free(arr);
arr = NULL;
```

Practical Example: Safe Dynamic Array

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int *data;
    size_t size;
    size_t capacity;
} SafeArray;

SafeArray* safe_array_create(size_t initial_capacity) {
    SafeArray *arr = malloc(sizeof(SafeArray));
    if (arr == NULL) {
        return NULL;
    }

    arr->data = malloc(initial_capacity * sizeof(int));
    if (arr->data == NULL) {
        free(arr);
        return NULL;
    }

    arr->size = 0;
    arr->capacity = initial_capacity;
    return arr;
}

int safe_array_push(SafeArray *arr, int value) {
    if (arr == NULL) {
        return -1;  // Error: NULL pointer
    }

    // Resize if needed
    if (arr->size >= arr->capacity) {
        size_t new_capacity = arr->capacity * 2;
        int *temp = realloc(arr->data, new_capacity * sizeof(int));
        if (temp == NULL) {
            return -1;  // Error: Reallocation failed
        }
        arr->data = temp;
        arr->capacity = new_capacity;
    }

    arr->data[arr->size] = value;
```

```
    arr->size++;
    return 0;  // Success
}

int safe_array_get(SafeArray *arr, size_t index) {
    if (arr == NULL || index >= arr->size) {
        return 0;  // Error: Invalid parameters
    }
    return arr->data[index];
}

void safe_array_destroy(SafeArray *arr) {
    if (arr != NULL) {
        free(arr->data);
        free(arr);
    }
}

int main() {
    SafeArray *arr = safe_array_create(2);
    if (arr == NULL) {
        printf("Array creation failed\n");
        return 1;
    }

    // Add elements
    for (int i = 0; i < 10; i++) {
        if (safe_array_push(arr, i * i) != 0) {
            printf("Array push failed\n");
            safe_array_destroy(arr);
            return 1;
        }
    }

    // Print elements
    printf("Array elements: ");
    for (size_t i = 0; i < arr->size; i++) {
        printf("%d ", safe_array_get(arr, i));
    }
    printf("\n");

    // Clean up
    safe_array_destroy(arr);

    return 0;
}
```

Summary

Pointer-related errors are among the most common and dangerous bugs in C programs. Key strategies for avoiding these pitfalls include:

1. **Dangling Pointers**: Always set pointers to NULL after freeing and avoid returning pointers to local variables
2. **Memory Leaks**: Match every malloc/calloc/realloc with a corresponding free
3. **Buffer Overflows**: Always validate array bounds and use safe string functions
4. **Uninitialized Pointers**: Initialize all pointers to NULL or valid addresses
5. **NULL Pointer Dereferencing**: Always check for NULL before dereferencing
6. **Pointer Arithmetic**: Validate bounds and understand type-specific arithmetic
7. **Type Safety**: Use proper casting and typedefs for function pointers

Using debugging tools like Valgrind and AddressSanitizer, along with static analysis tools, can help detect many of these issues during development. Following best practices and defensive programming techniques will lead to more robust and reliable C programs.

When the Optimizer Deletes Your NULL Check

When the Optimizer Deletes Your NULL Check

Thesis

Dereferencing a null pointer in C is not "a crash". It is **undefined behaviour** (C17 §6.5.3.2 ¶4). The standard places no requirements on a program that does it, so the compiler may assume it never happens. Once the compiler has seen `r->len`, it concludes that `r` is not NULL, because otherwise the program would already be undefined. A later `if (r == NULL)` is then always false, and dead-code elimination removes it together with the error path it guarded.

Nothing in this chain is a compiler bug. Each step is a legal inference, and GCC documents it as `-fdelete-null-pointer-checks`, which is on by default on most targets. The bug is in the source, but it stays invisible until an optimisation level turns it into a security hole. In 2009 the Linux tun driver read `tun->sk` one line before `if (!tun) return POLLERR;`. GCC deleted the check, and with page zero mapped the result was a local root exploit. The kernel has built with `-fno-delete-null-pointer-checks` ever since.

This chapter reproduces the three ways to write "I already dereferenced it" (an explicit load, a call to a `nonnull`-declared library function, and your own `nonnull` attribute) and shows which tools catch each one. It also shows what `-fno-delete-null-pointer-checks` can and cannot rescue.

Mental model

The rule the optimizer applies is simple: **a pointer that has been used as if valid is valid from then on**. The surprises come from what counts as "used":

What the code does first	Why GCC treats `p` as non-null afterwards	Does `-fno-delete-null-pointer-checks` restore the later check?
`x = p->field;` or `*p`	Null dereference is undefined behaviour	yes
`memcpy(d, p, n)` (any `n`, even 0)	glibc declares `memcpy` with `__nonnull ((1, 2))`	yes
`p` is a parameter of a function you marked `__attribute__((nonnull))`	The attribute is a promise to the compiler	**no**. GCC's manual: "can currently not be disabled other than by removing the `nonnull` attribute"

The lesson for code review is that the bug is never the `if`. The bug is the use that comes before it.

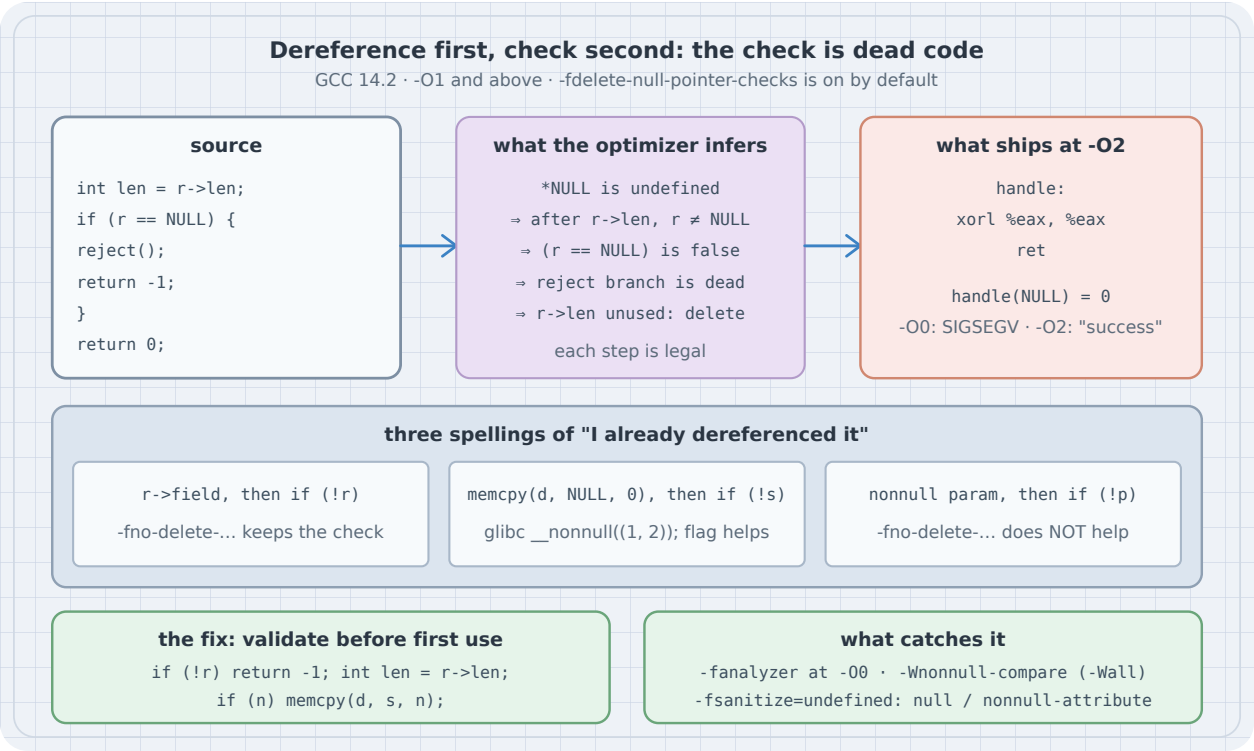


Figure 1: Dereference first, check second: the optimizer infers r is non-null, folds the comparison and deletes the reject path; three spellings of the earlier dereference; the fix and the detectors

Worked examples

Tested on Debian 13 x86_64, **GCC 14.2.0** (`gcc (Debian 14.2.0-19)`) and glibc 2.41, with `-std=c17 -Wall -Wextra`. The current upstream release is GCC 16.2 (2026-08-07). It documents `-fdelete-null-pointer-checks` and the `nonnull` attribute with the same wording quoted below, so the behaviour shown here is the documented behaviour, not a GCC 14 quirk. Clang was not available on the box, so nothing here was run with it.

Every case is driven by one script that builds a file at several levels and reports how each run ended:

```
#!/usr/bin/env bash
# run.sh - build one file at several -O levels and report how each run ended
src=$1; shift
for o in -O0 -O1 -O2 "$@"; do
  gcc -std=c17 -Wall -Wextra $o -o prog "$src" 2>/dev/null || exit 1
  out=$(./prog 2>&1); rc=$?
  [ $rc -gt 128 ] && end="killed by SIG$(kill -l $((rc - 128)))" || end="exit $rc"
  printf '%-34s %-36s %s\n' "$o" "${out//$'\n'/ | }" "$end"
done
```

Case 1: Dereference first, check second

```c
/* handle.c - dereference first, check second */
#include <stdio.h>

struct req { int len; char *buf; };

__attribute__((noinline)) int handle(struct req *r)
{
    int len = r->len;                   /* (1) UB if r == NULL */
    if (r == NULL) {                    /* (2) so r "cannot" be NULL here */
        fputs("handle: NULL request rejected\n", stderr);
        return -1;
    }
    (void)len;
    return 0;
}

int main(void)
{
    printf("handle(NULL) = %d\n", handle(NULL));
    return 0;
}
```

```
$ ./run.sh handle.c "-O2 -fno-delete-null-pointer-checks"
-O0                                                                 killed by SIGSEGV
-O1                                handle(NULL) = 0                 exit 0
-O2                                handle(NULL) = 0                 exit 0
-O2 -fno-delete-null-pointer-checks handle: NULL request rejected | handle(NULL) = -1 exit 0
```

The same source does three different things:

- At `-O0` the load from address 0 faults. The developer sees a crash and assumes that NULL "crashes".
- At `-O1` and `-O2` the function returns **0, success, for a NULL request**. The reject branch is gone, and so is the load, because `len` was never used. Nothing crashes, nothing is logged, and the caller carries on.
- With `-fno-delete-null-pointer-checks` the check survives. The dead load is still removed, so the function now does what its author meant.

The `-O2` code for `handle` is two instructions:

```
$ gcc -std=c17 -O2 -S -o - handle.c | sed -n '/^handle:/,/ret$/p' | grep -v cfi
handle:
.LFB3:
    xorl    %eax, %eax
    ret
```

The kernel's tun bug had the same shape, except that the loaded pointer *was* used later:

```
/* tun_poll.c - the shape of the 2009 Linux tun bug: dereference first, check second */
#include <stddef.h>

struct sock { int flags; };
struct tun_struct { struct sock *sk; int state; };

#define POLLERR 8

int tun_chr_poll(struct tun_struct *tun)
{
    struct sock *sk = tun->sk;      /* (1) dereference: UB if tun == NULL */
    if (!tun)                       /* (2) so the optimizer may assume tun != NULL */
        return POLLERR;
    return sk->flags & 1;
}
```

```
$ gcc -std=c17 -O0 -S -o - tun_poll.c | sed -n '/^tun_chr_poll:/,/ret$/p' | grep -v cfi
tun_chr_poll:
.LFB0:
    pushq   %rbp
    movq    %rsp, %rbp
    movq    %rdi, -24(%rbp)
    movq    -24(%rbp), %rax
    movq    (%rax), %rax
    movq    %rax, -8(%rbp)
    cmpq    $0, -24(%rbp)
    jne .L2
    movl    $8, %eax
    jmp .L3
.L2:
    movq    -8(%rbp), %rax
    movl    (%rax), %eax
    andl    $1, %eax
.L3:
    popq    %rbp
    ret
$ gcc -std=c17 -O2 -S -o - tun_poll.c | sed -n '/^tun_chr_poll:/,/ret$/p' | grep -v cfi
tun_chr_poll:
.LFB0:
    movq    (%rdi), %rax
    movl    (%rax), %eax
    andl    $1, %eax
    ret
$ gcc -std=c17 -O2 -fno-delete-null-pointer-checks -S -o - tun_poll.c | sed -n '/^tun_chr_po
tun_chr_poll:
```

```
.LFB0:
    movq    (%rdi), %rax
    testq   %rdi, %rdi
    je  .L3
    movl    (%rax), %eax
    andl    $1, %eax
    ret
```

At `-O0` there is a `cmpq $0` and a `POLLERR` path. At `-O2` both are gone. In user space the first `movq (%rdi)` faults, because Linux refuses to map low pages (`/proc/sys/vm/mmap_min_addr` is 4096 on this box). In the 2009 kernel an attacker could map page zero and put a fake `struct sock` there. The deleted check let execution continue into attacker-controlled data. The tun fix (commit `3c8a9c63d5fd`, July 2009) moved the dereference below the check. Ten days later, commit `a3ca86aea507` added `-fno-delete-null-pointer-checks` to the kernel's `KBUILD_CFLAGS`, and it is still in the top-level `Makefile` today. The flag keeps the `testq`. With it, the kernel trades a little optimisation for "a check I wrote stays where I wrote it".

Case 2: A zero-length `memcpy` counts as a dereference

This version never loads through the pointer. It only passes it to `memcpy` with a length of 0, which every C programmer "knows" does nothing:

```
/* copyz.c - a zero-length memcpy from NULL, then a NULL check on the source */
#include <stdio.h>
#include <string.h>

__attribute__((noinline)) int append(char *dst, const char *src, size_t n)
{
    memcpy(dst, src, n);            /* n == 0 and src == NULL: "harmless"? */
    if (src == NULL) {
        fputs("append: no source\n", stderr);
        return -1;
    }
    return (int)n;
}

int main(void)
{
    char buf[8];
    printf("append(buf, NULL, 0) = %d\n", append(buf, NULL, 0));
    return 0;
}
```

```
$ ./run.sh copyz.c "-O2 -fno-delete-null-pointer-checks"
-O0                                 append: no source | append(buf, NULL, 0) = -1 exit 0
-O1                                 append: no source | append(buf, NULL, 0) = -1 exit 0
```

```
-O2                                  append(buf, NULL, 0) = 0             exit 0
-O2 -fno-delete-null-pointer-checks append: no source | append(buf, NULL, 0) = -1 exit 0
```

This is worse than Case 1, because **no build crashes**. The `-O0` and `-O1` builds take the error path, so a debug-build test of "NULL source is rejected" passes. Only the `-O2` build returns success. Its code calls `memcpy` and returns `n`, and the test and the `fputs` are gone:

```
$ gcc -std=c17 -O2 -S -o - copyz.c | sed -n '/^append:/,/ret$/p' | grep -v cfi
append:
.LFB3:
    pushq   %rbx
    movq    %rdx, %rbx
    call    memcpy@PLT
    movl    %ebx, %eax
    popq    %rbx
    ret
```

The reason is in the header, not in the C language:

```
$ grep -n -A1 'extern void \*memcpy' /usr/include/string.h
43:extern void *memcpy (void *__restrict __dest, const void *__restrict __src,
44-          size_t __n) __THROW __nonnull ((1, 2));
```

`__nonnull ((1, 2))` expands to GCC's `nonnull` attribute. Up to C23, the standard says that when the length is 0 the pointer arguments "shall still have valid values" (C17 §7.24.1 ¶2). GCC treats the call as proof that `src` is not NULL, so the check after it is dead.

This corner of the language is changing. **N3322**, accepted for C2y, makes `memcpy(NULL, NULL, 0)`, `NULL + 0` and similar zero-length operations well defined. GCC 15 added a `nonnull_if_nonzero(ptr, len)` attribute so that libc headers can say exactly that. But headers decide what your compiler assumes, and glibc's development tree (checked 2026-10-19) still declares `memcpy` with `__nonnull ((1, 2))`. Until your libc changes and you compile in a mode that adopts N3322, write code that does not depend on it.

Case 3: Your own `nonnull` attribute, and the flag that does not help

Adding `nonnull` to a declaration looks like free documentation and free warnings. On a *definition*, it also tells GCC that it may delete the function's own defensive checks:

```c
/* parse_nonnull.c - a nonnull-declared function that still checks its argument */
#include <stdio.h>

__attribute__((nonnull(1), noinline))
int parse(const char *s)
{
    if (s == NULL)
        return -1;                      /* "defensive" check */
```

```
    return s[0];
}

int main(int argc, char **argv)
{
    const char *input = argc > 1 ? argv[1] : NULL;
    printf("parse -> %d\n", parse(input));
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -c parse_nonnull.c -o /dev/null
parse_nonnull.c: In function ‘parse’:
parse_nonnull.c:7:8: warning: ‘nonnull’ argument ‘s’ compared to NULL [-Wnonnull-compare]
    7 |     if (s == NULL)
      |        ^
$ ./run.sh parse_nonnull.c "-O2 -fno-delete-null-pointer-checks"
-O0                                parse -> -1                          exit 0
-O1                                                                     killed by SIGSEGV
-O2                                                                     killed by SIGSEGV
-O2 -fno-delete-null-pointer-checks                                     killed by SIGSEGV
$ gcc -std=c17 -O2 -S -o - parse_nonnull.c | sed -n '/^parse:/,/ret$/p' | grep -v cfi
parse:
.LFB3:
    movsbl  (%rdi), %eax
    ret
```

The NULL comes from a runtime value (`argc`), so GCC cannot see it at the call site and gives no `-Wnonnull` warning there. The only warning is `-Wnonnull-compare` on the definition, and it is in `-Wall`. This time **`-fno-delete-null-pointer-checks` changes nothing**. That is documented. For function definitions, the GCC manual says the optimisations based on `nonnull` parameters "can currently not be disabled other than by removing the `nonnull` attribute". If a function must survive NULL, don't mark it `nonnull`. If you mark it `nonnull`, treat `-Wnonnull-compare` as an error, because the check it points at is already gone.

Case 4: Finding them before the optimizer does

The GCC static analyzer has a dedicated warning, `-Wanalyzer-deref-before-check` (GCC 13 and later). Where you run it matters:

```
$ gcc -std=c17 -O2 -fanalyzer -c handle.c -o /dev/null && echo "(no warnings)"
(no warnings)
$ gcc -std=c17 -O1 -fanalyzer -c handle.c -o /dev/null && echo "(no warnings)"
(no warnings)
$ gcc -std=c17 -O0 -fanalyzer -fdiagnostics-plain-output -c handle.c -o /dev/null
handle.c: In function ‘handle’:
handle.c:8:9: warning: dereference of NULL ‘r’ [CWE-476] [-Wanalyzer-null-dereference]
```

```
handle.c:17:5: note: (1) entry to ‘main’
handle.c:19:5: note: (2) calling ‘handle’ from ‘main’
handle.c:6:31: note: (3) entry to ‘handle’
handle.c:8:9: note: (4) dereference of NULL ‘r’
handle.c:9:8: warning: check of ‘r’ for NULL after already dereferencing it [-Wanalyzer-dere
handle.c:8:9: note: (1) pointer ‘r’ is dereferenced here
handle.c:9:8: note: (2) pointer ‘r’ is checked for NULL here but it was already dereferenced
```

With GCC 14.2 the analyzer finds the bug at `-O0` and says nothing at `-O1` or `-O2`. By the time it runs in an optimised build, the check it would have flagged has already been folded away. Run `-fanalyzer` in a separate `-O0` job. It finds the `memcpy` shape too, and it even names the header attribute:

```
$ gcc -std=c17 -O0 -fanalyzer -fdiagnostics-plain-output -c copyz.c -o /dev/null
copyz.c: In function ‘append’:
copyz.c:7:5: warning: use of NULL ‘src’ where non-null expected [CWE-476] [-Wanalyzer-null-a
copyz.c:15:5: note: (1) entry to ‘main’
copyz.c:18:5: note: (2) calling ‘append’ from ‘main’
copyz.c:5:31: note: (3) entry to ‘append’
copyz.c:7:5: note: (4) argument 2 (‘src’) NULL where non-null expected
<built-in>: note: argument 2 of ‘__builtin_memcpy’ must be non-null
copyz.c:8:8: warning: check of ‘src’ for NULL after already dereferencing it [-Wanalyzer-der
copyz.c:7:5: note: (1) pointer ‘src’ is dereferenced here
copyz.c:8:8: note: (2) pointer ‘src’ is checked for NULL here but it was already dereference
```

UndefinedBehaviorSanitizer finds them at run time, as long as a test passes NULL:

```
$ gcc -std=c17 -O2 -g -fsanitize=undefined -o handle_ub handle.c && ./handle_ub
handle.c:8:9: runtime error: member access within null pointer of type 'struct req'
handle: NULL request rejected
handle(NULL) = -1
$ gcc -std=c17 -O2 -g -fsanitize=undefined -o copyz_ub copyz.c && ./copyz_ub
copyz.c:7:5: runtime error: null pointer passed as argument 2, which is declared to never be
append: no source
append(buf, NULL, 0) = -1
```

Look at the last two lines of each run. Under UBSan the `-O2` build takes the *reject* path, because the instrumentation keeps the check alive. That is the opposite of what the plain `-O2` build does. A sanitizer build is a detector, not a preview of what ships. Read the `runtime error:` line, and don’t trust the behaviour that follows it.

UBSan also flags code that today’s GCC does *not* exploit. Taking the address of a member through NULL, with no load, is still undefined:

```
/* addr.c - "I only took an address, I never loaded through it" */
#include <stdio.h>
```

```c
struct req { int len; char body[16]; };

__attribute__((noinline)) int body_first(struct req *r)
{
    char *body = r->body;           /* no load: just r + 4 */
    if (r == NULL) {
        fputs("body_first: NULL request\n", stderr);
        return -1;
    }
    return body[0];
}

int main(void)
{
    printf("body_first(NULL) = %d\n", body_first(NULL));
    return 0;
}
```

```
$ ./run.sh addr.c
-O0                                 body_first: NULL request | body_first(NULL) = -1 exit 0
-O1                                 body_first: NULL request | body_first(NULL) = -1 exit 0
-O2                                 body_first: NULL request | body_first(NULL) = -1 exit 0
$ gcc -std=c17 -O2 -g -fsanitize=undefined -o addr_ub addr.c && ./addr_ub
addr.c:8:11: runtime error: member access within null pointer of type 'struct req'
body_first: NULL request
body_first(NULL) = -1
```

GCC 14.2 keeps this check at every level, and UBSan still reports it. “My compiler doesn’t break it” describes one compiler version. It is not a guarantee in the language, and the next release is allowed to delete the check.

Case 5: The fix is ordering, not flags

```c
/* handle_fixed.c - check first, dereference second */
#include <stdio.h>

struct req { int len; char *buf; };

__attribute__((noinline)) int handle(struct req *r)
{
    if (r == NULL) {
        fputs("handle: NULL request rejected\n", stderr);
        return -1;
    }
    int len = r->len;
```

```
    (void)len;
    return 0;
}

int main(void)
{
    printf("handle(NULL) = %d\n", handle(NULL));
    return 0;
}
```

```
/* copyz_fixed.c - validate before the call, and skip empty copies */
#include <stdio.h>
#include <string.h>

__attribute__((noinline)) int append(char *dst, const char *src, size_t n)
{
    if (src == NULL) {
        fputs("append: no source\n", stderr);
        return -1;
    }
    if (n != 0)
        memcpy(dst, src, n);
    return (int)n;
}

int main(void)
{
    char buf[8];
    printf("append(buf, NULL, 0) = %d\n", append(buf, NULL, 0));
    return 0;
}
```

```
$ ./run.sh handle_fixed.c
-O0                             handle: NULL request rejected | handle(NULL) = -1 exit 0
-O1                             handle: NULL request rejected | handle(NULL) = -1 exit 0
-O2                             handle: NULL request rejected | handle(NULL) = -1 exit 0
$ ./run.sh copyz_fixed.c
-O0                             append: no source | append(buf, NULL, 0) = -1 exit 0
-O1                             append: no source | append(buf, NULL, 0) = -1 exit 0
-O2                             append: no source | append(buf, NULL, 0) = -1 exit 0
```

Once the check comes first, every level agrees. `if (n != 0)` around `memcpy` is not micro-optimisation. It keeps an "empty slice" `(NULL, 0)` from ever reaching a `nonnull` parameter. That is the usual way C code represents an empty buffer, and N3322 exists because of it.

The trap

The trap is testing NULL handling in a debug build and believing the result. In Case 1 the `-O0` build crashed, and a crash looks like "NULL is handled loudly". In Case 2 the `-O0` and `-O1` builds took the error path, so the unit test passed. Both shipped `-O2` builds return **success** for a NULL input, and they log nothing. A sanitizer build makes it worse: it takes the reject path and passes the test too. It also reports the UB on stderr, but only someone reading that log will notice.

The usual reactions miss the point:

- **"Add `-fno-delete-null-pointer-checks`."** It rescues Cases 1 and 2, and it is a reasonable belt-and-braces flag for kernels and firmware. It does nothing for Case 3, and the dereference before the check is still undefined. The next optimisation that reasons from UB (and there are many) is not covered by this flag.
- **"Add `nonnull` so the compiler warns callers."** It warns only when it can see a NULL constant, and it deletes your own checks. Case 3 shows both.
- **"`len` is never used, so the dereference doesn't matter."** That unused dereference is exactly what deleted the check in Case 1.

The boring rule

> Validate a pointer **before its first use**. A use includes `p->field`, `*p`, `&p->field`, and passing `p` to any function whose header marks it `nonnull`, including `memcpy`, `memset` and `strlen` with a length of 0. Guard zero-length library calls with `if (n != 0)`. Put `nonnull` only on functions that really never accept NULL, and build with `-Werror=nonnull-compare`. Run `-fanalyzer` in a separate `-O0` CI job and the test suite under `-fsanitize=undefined`, and include NULL inputs in the tests. Kernel-style code that may legitimately touch address 0 also gets `-fno-delete-null-pointer-checks`, but that flag is a seatbelt, not a fix.

Try this

1. Change Case 1 to use `len` after the check (`return len;`). Rerun `run.sh`. What does `-O2` do now, and why does the program crash again even though the check is still missing?
2. In Case 2, change `memcpy` to `memmove`, then to `memset(dst, 0, n)` with a NULL `dst`. Find each declaration in `/usr/include/string.h` and predict the `-O2` behaviour before you run it.
3. Add `-fisolate-erroneous-paths-attribute` to Case 3 and pass a literal `NULL` instead of `input`. What does the call site become? (Read the description of the option in the GCC manual first.)
4. Search a codebase you maintain for `if (!p)` or `if (p == NULL)` that appears *after* a use of `p` in the same function. Run `gcc -O0 -fanalyzer` over it and compare what each approach finds.
5. Write a two-line `container_of`-style macro that computes a struct address from a member pointer. Run it on NULL under `-fsanitize=undefined`. Is the arithmetic itself undefined, and what would N3322 change?

Sources

- ISO/IEC 9899:2018 (C17) §6.5.3.2 ¶4 (unary `*` on an invalid pointer, such as a null pointer, is undefined) and §7.24.1 ¶2 (when a length argument is zero, pointer arguments "shall still have valid values"); the C17 final draft N2176 is a convenient copy of this text.
- GCC manual, Optimize Options: `-fdelete-null-pointer-checks`, `-fisolate-erroneous-paths-derefer` `-fisolate-erroneous-paths-attribute`: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html
- GCC manual, Common Function Attributes: `nonnull` (including "can currently not be disabled other than by removing the `nonnull` attribute") and `nonnull_if_nonzero`: https://gcc.gnu.org/onlinedocs/gcc/Common-Attributes.html
- GCC manual, Static Analyzer Options: `-Wanalyzer-deref-before-check`, `-Wanalyzer-null-argument`: https://gcc.gnu.org/onlinedocs/gcc/Static-Analyzer-Options.html
- GCC manual, Instrumentation Options: `-fsanitize=null`, `-fsanitize=nonnull-attribute`: https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html
- WG14 N3322, N. Popov and A. Ballman, "Allow zero length operations on null pointers" (accepted for C2y): https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3322.pdf
- glibc `string/string.h` (development tree, `memcpy` declared `__nonnull ((1, 2))`): https://sourceware.org/git/?p=glibc.git;a=blob;f=string/string.h
- Linux commit 3c8a9c63d5fd, "tun/tap: Fix crashes if open() /dev/net/tun and then poll() it" (2009-07-05): https://github.com/torvalds/linux/commit/3c8a9c63d5fd738c261bd0ceece04d9c8357ca13
- Linux commit a3ca86aea507, "Add '-fno-delete-null-pointer-checks' to gcc CFLAGS" (2009-07-15), still present in the top-level `Makefile`: https://github.com/torvalds/linux/commit/a3ca86aea507904148870946d599e07a340b39bf
- GCC releases (GCC 16.2, 2026-08-07): https://gcc.gnu.org/

Module 6: Pointers and Memory Management

Overview

This module explores pointers and dynamic memory management in C, fundamental concepts for systems programming and efficient resource utilization. You'll learn about memory addresses, pointer arithmetic, dynamic memory allocation, and advanced pointer techniques. The module also covers common pointer pitfalls and best practices for safe memory management.

Learning Objectives

By the end of this module, you will be able to: - Understand memory addresses and pointer fundamentals - Perform pointer arithmetic and manipulation - Allocate and deallocate dynamic memory using malloc/free - Work with arrays and pointers interchangeably - Implement complex data structures using pointers - Apply function pointers for callbacks and dynamic dispatch - Understand void pointers and generic programming - Identify and prevent common pointer-related errors - Use const correctness with pointers - Apply advanced pointer concepts like pointers to pointers

Chapters

1. **Pointer Fundamentals** - Memory addresses, pointer declaration, initialization, and dereferencing
2. **Pointers and Arrays** - Array-pointer equivalence, pointer subscripting, and multi-dimensional arrays
3. **Dynamic Memory Management** - malloc, calloc, realloc, free, and memory debugging
4. **Advanced Pointer Concepts** - Pointers to pointers, function pointers, void pointers, and const correctness
5. **Common Pointer Pitfalls** - Dangling pointers, buffer overflows, segmentation faults, and memory leaks
6. **When the Optimizer Deletes Your NULL Check** - Dereference-before-check, `nonnull` and zero-length `memcpy`, `-fdelete-null-pointer-checks`, finding it with `-fanalyzer` and UBSan

Key Concepts Covered

- Understanding memory addresses and the & operator
- Pointer declaration, initialization, and dereferencing with *

- Pointer arithmetic and increment/decrement operations
- Array-pointer equivalence and pointer subscripting
- Dynamic memory allocation with malloc, calloc, realloc
- Memory deallocation with free and resource cleanup
- Function pointers and callback mechanisms
- Void pointers and generic programming techniques
- Const correctness with pointers
- Pointers to pointers and multi-level indirection
- Common pointer errors and debugging techniques
- Memory alignment and performance considerations

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Completion of Module 4: Functions and Modular Programming
- Completion of Module 5: Arrays and Strings
- Understanding of basic programming concepts
- Familiarity with variable declaration and initialization

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Memory debugging tools (Valgrind, AddressSanitizer)
- Understanding of basic programming concepts
- Familiarity with debugging tools

Estimated Time to Complete

- Reading: 10-12 hours
- Exercises: 15-20 hours
- Projects: 8-10 hours

Assessment

- Complete all chapter exercises
- Successfully implement dynamic data structures
- Demonstrate understanding of memory management concepts
- Pass pointer programming quiz
- Submit a comprehensive program using advanced pointer techniques

Next Module

After completing this module, proceed to **Module 7: Structures and User-Defined Types** to learn about creating custom data types.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 5)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Online tutorials on pointer arithmetic and memory management
- Debugging tools documentation (GDB, Valgrind, AddressSanitizer)
- Best practices guides for safe memory management

Structures

Structures

Introduction

Structures (structs) are user-defined data types in C that allow you to group related variables of different types under a single name. They are fundamental for creating complex data representations and implementing abstract data types. Structures enable programmers to model real-world entities and organize data in a logical, maintainable way.

Structure Declaration and Definition

Basic Structure Declaration

A structure is declared using the `struct` keyword followed by the structure name and a list of members enclosed in braces:

```
struct Person {
    char name[50];
    int age;
    float height;
    char gender;
};
```

Structure Variable Declaration

Structure variables can be declared in several ways:

```
// Method 1: Declare structure and variables separately
struct Person {
    char name[50];
    int age;
    float height;
    char gender;
};

struct Person person1, person2;

// Method 2: Declare structure and variables together
```

```c
struct {
    char name[50];
    int age;
    float height;
    char gender;
} person1, person2;

// Method 3: Use typedef for cleaner syntax
typedef struct {
    char name[50];
    int age;
    float height;
    char gender;
} Person;

Person person1, person2;
```

Structure Initialization

Designated Initializers (C99)

```c
typedef struct {
    char name[50];
    int age;
    float height;
    char gender;
} Person;

// Initialize all members
Person person1 = {"John Doe", 25, 5.9, 'M'};

// Designated initializers (C99)
Person person2 = {
    .name = "Jane Smith",
    .age = 30,
    .height = 5.5,
    .gender = 'F'
};

// Partial initialization
Person person3 = {
    .name = "Bob Johnson",
    .age = 35
    // height and gender will be zero-initialized
```

```
};
```

Nested Structure Initialization

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef struct {
    char name[50];
    Date birth_date;
    float salary;
} Employee;

// Nested structure initialization
Employee emp = {
    .name = "Alice Brown",
    .birth_date = {15, 6, 1990},
    .salary = 75000.0
};

// Designated initialization with nested structures
Employee emp2 = {
    .name = "Charlie Wilson",
    .birth_date.day = 20,
    .birth_date.month = 3,
    .birth_date.year = 1985,
    .salary = 80000.0
};
```

Accessing Structure Members

Dot Operator (.)

The dot operator is used to access members of a structure variable:

```
typedef struct {
    char name[50];
    int age;
    float height;
    char gender;
```

```
} Person;

int main() {
    Person person = {"John Doe", 25, 5.9, 'M'};

    // Access members using dot operator
    printf("Name: %s\n", person.name);
    printf("Age: %d\n", person.age);
    printf("Height: %.1f\n", person.height);
    printf("Gender: %c\n", person.gender);

    // Modify members
    person.age = 26;
    person.height = 6.0;
    strcpy(person.name, "John Smith");

    return 0;
}
```

Arrow Operator (->)

The arrow operator is used to access members through a pointer to a structure:

```
typedef struct {
    char name[50];
    int age;
    float height;
    char gender;
} Person;

int main() {
    Person person = {"John Doe", 25, 5.9, 'M'};
    Person *ptr = &person;

    // Access members using arrow operator
    printf("Name: %s\n", ptr->name);
    printf("Age: %d\n", ptr->age);
    printf("Height: %.1f\n", ptr->height);
    printf("Gender: %c\n", ptr->gender);

    // Modify members through pointer
    ptr->age = 26;
    ptr->height = 6.0;
    strcpy(ptr->name, "John Smith");

    return 0;
```

```
}
```

Arrays of Structures

Declaration and Initialization

```
typedef struct {
    char name[50];
    int id;
    float gpa;
} Student;

int main() {
    // Array of structures
    Student students[3] = {
        {"Alice", 101, 3.8},
        {"Bob", 102, 3.5},
        {"Charlie", 103, 3.9}
    };

    // Access array elements
    for (int i = 0; i < 3; i++) {
        printf("Student %d: %s, ID: %d, GPA: %.1f\n",
               i + 1, students[i].name, students[i].id, students[i].gpa);
    }

    return 0;
}
```

Dynamic Array of Structures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[50];
    int id;
    float gpa;
} Student;

int main() {
    int count = 3;
```

```
    // Dynamic array of structures
    Student *students = malloc(count * sizeof(Student));
    if (students == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Initialize array elements
    strcpy(students[0].name, "Alice");
    students[0].id = 101;
    students[0].gpa = 3.8;

    strcpy(students[1].name, "Bob");
    students[1].id = 102;
    students[1].gpa = 3.5;

    strcpy(students[2].name, "Charlie");
    students[2].id = 103;
    students[2].gpa = 3.9;

    // Access array elements
    for (int i = 0; i < count; i++) {
        printf("Student %d: %s, ID: %d, GPA: %.1f\n",
               i + 1, students[i].name, students[i].id, students[i].gpa);
    }

    // Free memory
    free(students);

    return 0;
}
```

Nested Structures

Basic Nested Structures

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef struct {
    char street[100];
```

```
    char city[50];
    char state[30];
    int zip_code;
} Address;

typedef struct {
    char name[50];
    Date birth_date;
    Address home_address;
    float salary;
} Employee;

int main() {
    Employee emp = {
        .name = "John Doe",
        .birth_date = {15, 6, 1990},
        .home_address = {
            .street = "123 Main St",
            .city = "Anytown",
            .state = "CA",
            .zip_code = 12345
        },
        .salary = 75000.0
    };

    // Access nested structure members
    printf("Name: %s\n", emp.name);
    printf("Birth Date: %d/%d/%d\n",
           emp.birth_date.month, emp.birth_date.day, emp.birth_date.year);
    printf("Address: %s, %s, %s %d\n",
           emp.home_address.street, emp.home_address.city,
           emp.home_address.state, emp.home_address.zip_code);
    printf("Salary: $%.2f\n", emp.salary);

    return 0;
}
```

Self-Referential Structures

Structures can contain pointers to their own type, enabling linked data structures:

```
typedef struct Node {
    int data;
    struct Node *next;  // Pointer to same structure type
} Node;
```

```c
int main() {
    // Create nodes
    Node node1 = {10, NULL};
    Node node2 = {20, NULL};
    Node node3 = {30, NULL};

    // Link nodes
    node1.next = &node2;
    node2.next = &node3;

    // Traverse linked list
    Node *current = &node1;
    while (current != NULL) {
        printf("Data: %d\n", current->data);
        current = current->next;
    }

    return 0;
}
```

Structure Assignment and Comparison

Structure Assignment

Structures can be assigned to each other if they are of the same type:

```c
typedef struct {
    char name[50];
    int age;
    float height;
} Person;

int main() {
    Person person1 = {"John Doe", 25, 5.9};
    Person person2;

    // Structure assignment
    person2 = person1;

    printf("Person 1: %s, %d, %.1f\n", person1.name, person1.age, person1.height);
    printf("Person 2: %s, %d, %.1f\n", person2.name, person2.age, person2.height);

    return 0;
}
```

Structure Comparison

Structures cannot be directly compared using comparison operators. You need to compare members individually:

```
typedef struct {
    char name[50];
    int age;
    float height;
} Person;

int compare_persons(const Person *p1, const Person *p2) {
    // Compare names
    int name_cmp = strcmp(p1->name, p2->name);
    if (name_cmp != 0) return name_cmp;

    // Compare ages
    if (p1->age != p2->age) return p1->age - p2->age;

    // Compare heights
    if (p1->height < p2->height) return -1;
    if (p1->height > p2->height) return 1;

    return 0;  // Structures are equal
}

int main() {
    Person person1 = {"John Doe", 25, 5.9};
    Person person2 = {"John Doe", 25, 5.9};

    if (compare_persons(&person1, &person2) == 0) {
        printf("Persons are equal\n");
    } else {
        printf("Persons are different\n");
    }

    return 0;
}
```

Passing Structures to Functions

Pass by Value

When passing structures by value, a copy of the entire structure is made:

```c
typedef struct {
    char name[50];
    int age;
    float height;
} Person;

void print_person(Person p) {
    printf("Name: %s\n", p.name);
    printf("Age: %d\n", p.age);
    printf("Height: %.1f\n", p.height);
}

void modify_person(Person p) {
    // Modifications affect only the copy
    p.age = 30;
    strcpy(p.name, "Modified Name");
}

int main() {
    Person person = {"John Doe", 25, 5.9};

    printf("Before modification:\n");
    print_person(person);

    modify_person(person);

    printf("After modification:\n");
    print_person(person);  // Original unchanged

    return 0;
}
```

Pass by Reference (Pointer)

Passing pointers to structures is more efficient and allows modification:

```c
typedef struct {
    char name[50];
    int age;
    float height;
} Person;

void print_person(const Person *p) {
    printf("Name: %s\n", p->name);
    printf("Age: %d\n", p->age);
    printf("Height: %.1f\n", p->height);
```

```
}

void modify_person(Person *p) {
    // Modifications affect the original structure
    p->age = 30;
    strcpy(p->name, "Modified Name");
}

int main() {
    Person person = {"John Doe", 25, 5.9};

    printf("Before modification:\n");
    print_person(&person);

    modify_person(&person);

    printf("After modification:\n");
    print_person(&person);  // Original changed

    return 0;
}
```

Structure Size and Memory Layout

Calculating Structure Size

```
#include <stdio.h>

typedef struct {
    char a;      // 1 byte
    int b;       // 4 bytes
    char c;      // 1 byte
} Example;

int main() {
    printf("Size of char: %zu bytes\n", sizeof(char));
    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of Example: %zu bytes\n", sizeof(Example));
    // Note: Size may be larger than sum due to padding

    return 0;
}
```

Memory Alignment

Compilers may add padding to ensure proper alignment:

```c
#include <stdio.h>

// Without pragma pack
typedef struct {
    char a;      // 1 byte
    int b;       // 4 bytes (may have 3 bytes padding before)
    char c;      // 1 byte (may have 3 bytes padding after)
} Unpacked;

// With pragma pack (1) - no padding
#pragma pack(1)
typedef struct {
    char a;      // 1 byte
    int b;       // 4 bytes (no padding)
    char c;      // 1 byte (no padding)
} Packed;
#pragma pack()

int main() {
    printf("Size of Unpacked: %zu bytes\n", sizeof(Unpacked));
    printf("Size of Packed: %zu bytes\n", sizeof(Packed));

    return 0;
}
```

Practical Examples

Student Management System

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    float grades[5];
    float average;
} Student;

void calculate_average(Student *student) {
```

```c
    float sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += student->grades[i];
    }
    student->average = sum / 5;
}

void print_student(const Student *student) {
    printf("ID: %d\n", student->id);
    printf("Name: %s\n", student->name);
    printf("Grades: ");
    for (int i = 0; i < 5; i++) {
        printf("%.1f ", student->grades[i]);
    }
    printf("\nAverage: %.2f\n", student->average);
    printf("------------------------\n");
}

int main() {
    Student students[3] = {
        {1, "Alice", {85.5, 90.0, 78.5, 92.0, 88.5}, 0},
        {2, "Bob", {76.0, 82.5, 79.0, 85.5, 81.0}, 0},
        {3, "Charlie", {95.0, 92.5, 98.0, 90.5, 94.0}, 0}
    };

    // Calculate averages
    for (int i = 0; i < 3; i++) {
        calculate_average(&students[i]);
    }

    // Print student information
    for (int i = 0; i < 3; i++) {
        print_student(&students[i]);
    }

    // Find student with highest average
    int top_student = 0;
    for (int i = 1; i < 3; i++) {
        if (students[i].average > students[top_student].average) {
            top_student = i;
        }
    }

    printf("Top student:\n");
    print_student(&students[top_student]);

    return 0;
```

```
}
```

Point Structure with Operations

```
#include <stdio.h>
#include <math.h>

typedef struct {
    double x;
    double y;
} Point;

// Function to create a point
Point create_point(double x, double y) {
    Point p = {x, y};
    return p;
}

// Function to calculate distance between two points
double distance(const Point *p1, const Point *p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx * dx + dy * dy);
}

// Function to move a point
void move_point(Point *p, double dx, double dy) {
    p->x += dx;
    p->y += dy;
}

// Function to print a point
void print_point(const Point *p) {
    printf("(%.2f, %.2f)", p->x, p->y);
}

int main() {
    Point p1 = create_point(0, 0);
    Point p2 = create_point(3, 4);

    printf("Point 1: ");
    print_point(&p1);
    printf("\n");

    printf("Point 2: ");
```

```
    print_point(&p2);
    printf("\n");

    printf("Distance: %.2f\n", distance(&p1, &p2));

    move_point(&p1, 1, 1);
    printf("After moving Point 1: ");
    print_point(&p1);
    printf("\n");

    printf("New distance: %.2f\n", distance(&p1, &p2));

    return 0;
}
```

Summary

Structures are fundamental to C programming, providing a way to group related data and create complex data types. Key points to remember:

1. **Declaration and Initialization**: Structures can be declared and initialized in various ways
2. **Member Access**: Use dot operator for structure variables, arrow operator for pointers
3. **Arrays of Structures**: Enable management of multiple related data items
4. **Nested Structures**: Allow creation of complex hierarchical data representations
5. **Function Parameters**: Pass by value creates copies, pass by pointer is more efficient
6. **Memory Layout**: Structures may have padding for alignment
7. **Self-Referential Structures**: Enable linked data structures like lists and trees

Understanding structures is essential for implementing complex data structures, managing related data efficiently, and creating well-organized C programs.

Unions and Enumerations

Introduction

Unions and enumerations are two important features in C that complement structures by providing additional ways to define and work with data types. Unions allow different data types to share the same memory location, making them useful for memory optimization and type-generic programming. Enumerations provide a way to define named integer constants, improving code readability and maintainability.

Unions

Union Declaration and Basics

A union is a special data type that allows storing different data types in the same memory location. Unlike structures where each member has its own memory, all members of a union share the same memory space.

```c
#include <stdio.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;

    printf("Memory size occupied by data: %zu\n", sizeof(data));

    data.i = 10;
    printf("data.i: %d\n", data.i);

    data.f = 220.5;
    printf("data.f: %f\n", data.f);

    // Note: Previous value of data.i is lost
    printf("data.i after setting data.f: %d\n", data.i);
```

```
    strcpy(data.str, "C Programming");
    printf("data.str: %s\n", data.str);

    return 0;
}
```

Union Memory Layout

All members of a union occupy the same memory location, so the size of a union is the size of its largest member:

```
#include <stdio.h>

union Example {
    char c;         // 1 byte
    int i;          // 4 bytes (typically)
    double d;       // 8 bytes (typically)
    char str[20];   // 20 bytes
};

int main() {
    union Example ex;

    printf("Size of char: %zu bytes\n", sizeof(char));
    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of double: %zu bytes\n", sizeof(double));
    printf("Size of char[20]: %zu bytes\n", sizeof(char[20]));
    printf("Size of union Example: %zu bytes\n", sizeof(ex));

    // All members share the same memory address
    printf("Address of ex.c: %p\n", (void*)&ex.c);
    printf("Address of ex.i: %p\n", (void*)&ex.i);
    printf("Address of ex.d: %p\n", (void*)&ex.d);
    printf("Address of ex.str: %p\n", (void*)ex.str);

    return 0;
}
```

Practical Union Usage

1. Memory Optimization

Unions are useful when you need to store different types of data but only one at a time:

```c
#include <stdio.h>
#include <string.h>

typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_STRING
} DataType;

typedef struct {
    DataType type;
    union {
        int i;
        float f;
        char str[50];
    } data;
} Variant;

void print_variant(const Variant *v) {
    switch (v->type) {
        case TYPE_INT:
            printf("Integer: %d\n", v->data.i);
            break;
        case TYPE_FLOAT:
            printf("Float: %.2f\n", v->data.f);
            break;
        case TYPE_STRING:
            printf("String: %s\n", v->data.str);
            break;
    }
}

int main() {
    Variant var1 = {TYPE_INT, .data.i = 42};
    Variant var2 = {TYPE_FLOAT, .data.f = 3.14f};
    Variant var3 = {TYPE_STRING, .data = {.str = "Hello, World!"}};

    print_variant(&var1);
    print_variant(&var2);
    print_variant(&var3);

    return 0;
}
```

2. Hardware Register Representation

Unions are commonly used in embedded programming to represent hardware registers:

```c
#include <stdio.h>
#include <stdint.h>

// 32-bit register with bit fields
typedef union {
    uint32_t value;
    struct {
        uint32_t enable : 1;
        uint32_t mode : 2;
        uint32_t reserved : 5;
        uint32_t frequency : 8;
        uint32_t address : 16;
    } bits;
} Register;

int main() {
    Register reg;

    // Set register value
    reg.value = 0x12345678;
    printf("Register value: 0x%08X\n", reg.value);
    printf("Enable: %d\n", reg.bits.enable);
    printf("Mode: %d\n", reg.bits.mode);
    printf("Frequency: %d\n", reg.bits.frequency);
    printf("Address: 0x%04X\n", reg.bits.address);

    // Modify individual bits
    reg.bits.enable = 1;
    reg.bits.mode = 2;
    printf("Modified register: 0x%08X\n", reg.value);

    return 0;
}
```

Anonymous Unions (C11)

C11 introduced anonymous unions, which allow direct access to union members without specifying the union name:

```c
#include <stdio.h>

typedef struct {
```

```
    int type;
    union {
        int integer_value;
        float float_value;
        char string_value[20];
    };  // Anonymous union
} DataRecord;

int main() {
    DataRecord record;

    record.type = 1;
    record.integer_value = 42;  // Direct access to union member
    printf("Integer value: %d\n", record.integer_value);

    record.type = 2;
    record.float_value = 3.14f;  // Direct access to union member
    printf("Float value: %.2f\n", record.float_value);

    return 0;
}
```

Enumerations

Basic Enumeration Declaration

Enumerations provide a way to define named integer constants, making code more readable and maintainable:

```
#include <stdio.h>

// Basic enumeration
enum Color {
    RED,
    GREEN,
    BLUE
};

// Enumeration with typedef
typedef enum {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
```

```
    SATURDAY,
    SUNDAY
} Day;

int main() {
    enum Color favorite_color = BLUE;
    Day today = WEDNESDAY;

    printf("Favorite color: %d\n", favorite_color);
    printf("Today is day: %d\n", today);

    // Using in switch statements
    switch (today) {
        case MONDAY:
            printf("It's Monday\n");
            break;
        case WEDNESDAY:
            printf("It's Wednesday\n");
            break;
        case FRIDAY:
            printf("It's Friday\n");
            break;
        default:
            printf("It's another day\n");
    }

    return 0;
}
```

Explicit Enumeration Values

Enumeration constants can be assigned specific values:

```
#include <stdio.h>

typedef enum {
    HTTP_OK = 200,
    HTTP_BAD_REQUEST = 400,
    HTTP_UNAUTHORIZED = 401,
    HTTP_FORBIDDEN = 403,
    HTTP_NOT_FOUND = 404,
    HTTP_INTERNAL_SERVER_ERROR = 500
} HttpStatusCode;

typedef enum {
    FLAG_NONE = 0,
```

```
    FLAG_READ = 1,
    FLAG_WRITE = 2,
    FLAG_EXECUTE = 4,
    FLAG_DELETE = 8
} FilePermission;

int main() {
    HttpStatusCode status = HTTP_NOT_FOUND;
    FilePermission permissions = FLAG_READ | FLAG_WRITE;

    printf("HTTP Status: %d\n", status);
    printf("Permissions: %d\n", permissions);

    // Check permissions using bitwise operations
    if (permissions & FLAG_READ) {
        printf("Read permission granted\n");
    }

    if (permissions & FLAG_EXECUTE) {
        printf("Execute permission granted\n");
    } else {
        printf("Execute permission denied\n");
    }

    return 0;
}
```

Enumeration with Negative Values

Enumerations can include negative values:

```
#include <stdio.h>

typedef enum {
    TEMPERATURE_FREEZING = 0,
    TEMPERATURE_COLD = 10,
    TEMPERATURE_WARM = 20,
    TEMPERATURE_HOT = 30,
    TEMPERATURE_BOILING = 100
} Temperature;

typedef enum {
    DIRECTION_NORTH = 0,
    DIRECTION_EAST = 90,
    DIRECTION_SOUTH = 180,
    DIRECTION_WEST = 270
```

```
} Direction;

// Enumeration with negative values
typedef enum {
    STATUS_ERROR = -1,
    STATUS_UNKNOWN = 0,
    STATUS_SUCCESS = 1
} Status;

int main() {
    Temperature temp = TEMPERATURE_HOT;
    Direction dir = DIRECTION_EAST;
    Status result = STATUS_SUCCESS;

    printf("Temperature: %d°C\n", temp);
    printf("Direction: %d°\n", dir);
    printf("Status: %d\n", result);

    return 0;
}
```

Enumeration Size and Type Specification (C23)

C23 introduces more control over enumeration types:

```
#include <stdio.h>

// Specify underlying type (C23 feature)
typedef enum : unsigned char {
    SMALL_RED,
    SMALL_GREEN,
    SMALL_BLUE
} SmallColor;

typedef enum : long long {
    LARGE_VALUE1 = 1000000000000LL,
    LARGE_VALUE2 = 2000000000000LL
} LargeEnum;

int main() {
    SmallColor color = SMALL_GREEN;
    LargeEnum large = LARGE_VALUE2;

    printf("Size of SmallColor: %zu bytes\n", sizeof(color));
    printf("Size of LargeEnum: %zu bytes\n", sizeof(large));
    printf("SmallColor value: %d\n", color);
```

```
    printf("LargeEnum value: %lld\n", large);

    return 0;
}
```

Combining Unions and Enumerations

Tagged Union Pattern

A common pattern combines unions with enumerations to create type-safe variant types:

```
#include <stdio.h>
#include <string.h>

typedef enum {
    VALUE_INTEGER,
    VALUE_FLOAT,
    VALUE_STRING,
    VALUE_BOOLEAN
} ValueType;

typedef struct {
    ValueType type;
    union {
        int integer_value;
        float float_value;
        char string_value[100];
        int boolean_value;  // 0 = false, 1 = true
    } data;
} Value;

// Constructor functions
Value create_integer(int value) {
    Value v = {VALUE_INTEGER, .data.integer_value = value};
    return v;
}

Value create_float(float value) {
    Value v = {VALUE_FLOAT, .data.float_value = value};
    return v;
}

Value create_string(const char *value) {
    Value v = {VALUE_STRING};
    strncpy(v.data.string_value, value, sizeof(v.data.string_value) - 1);
```

```
    v.data.string_value[sizeof(v.data.string_value) - 1] = '\0';
    return v;
}

Value create_boolean(int value) {
    Value v = {VALUE_BOOLEAN, .data.boolean_value = value ? 1 : 0};
    return v;
}

// Print function
void print_value(const Value *v) {
    switch (v->type) {
        case VALUE_INTEGER:
            printf("Integer: %d\n", v->data.integer_value);
            break;
        case VALUE_FLOAT:
            printf("Float: %.2f\n", v->data.float_value);
            break;
        case VALUE_STRING:
            printf("String: %s\n", v->data.string_value);
            break;
        case VALUE_BOOLEAN:
            printf("Boolean: %s\n", v->data.boolean_value ? "true" : "false");
            break;
    }
}

int main() {
    Value values[] = {
        create_integer(42),
        create_float(3.14f),
        create_string("Hello, World!"),
        create_boolean(1)
    };

    int count = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < count; i++) {
        print_value(&values[i]);
    }

    return 0;
}
```

Configuration Structure with Unions

Using unions and enumerations for flexible configuration:

```
#include <stdio.h>
#include <string.h>

typedef enum {
    CONFIG_TYPE_INT,
    CONFIG_TYPE_FLOAT,
    CONFIG_TYPE_STRING,
    CONFIG_TYPE_BOOL
} ConfigType;

typedef struct {
    char key[50];
    ConfigType type;
    union {
        int int_value;
        float float_value;
        char string_value[100];
        int bool_value;
    } value;
} ConfigItem;

typedef struct {
    ConfigItem *items;
    int count;
    int capacity;
} Config;

Config* create_config() {
    Config *config = malloc(sizeof(Config));
    if (config != NULL) {
        config->items = malloc(10 * sizeof(ConfigItem));
        if (config->items != NULL) {
            config->count = 0;
            config->capacity = 10;
        } else {
            free(config);
            config = NULL;
        }
    }
    return config;
}

void add_int_config(Config *config, const char *key, int value) {
```

```
    if (config->count >= config->capacity) {
        // Resize array
        ConfigItem *new_items = realloc(config->items,
                                        (config->capacity + 10) * sizeof(ConfigItem));
        if (new_items != NULL) {
            config->items = new_items;
            config->capacity += 10;
        } else {
            return;  // Allocation failed
        }
    }

    ConfigItem *item = &config->items[config->count];
    strncpy(item->key, key, sizeof(item->key) - 1);
    item->key[sizeof(item->key) - 1] = '\0';
    item->type = CONFIG_TYPE_INT;
    item->value.int_value = value;

    config->count++;
}

void print_config(const Config *config) {
    for (int i = 0; i < config->count; i++) {
        const ConfigItem *item = &config->items[i];
        printf("Key: %s, ", item->key);

        switch (item->type) {
            case CONFIG_TYPE_INT:
                printf("Value: %d (int)\n", item->value.int_value);
                break;
            case CONFIG_TYPE_FLOAT:
                printf("Value: %.2f (float)\n", item->value.float_value);
                break;
            case CONFIG_TYPE_STRING:
                printf("Value: %s (string)\n", item->value.string_value);
                break;
            case CONFIG_TYPE_BOOL:
                printf("Value: %s (bool)\n", item->value.bool_value ? "true" : "false");
                break;
        }
    }
}

int main() {
    Config *config = create_config();
    if (config == NULL) {
```

```
        printf("Failed to create config\n");
        return 1;
    }

    add_int_config(config, "window_width", 800);
    add_int_config(config, "window_height", 600);

    print_config(config);

    // Clean up
    free(config->items);
    free(config);

    return 0;
}
```

Practical Examples

Network Packet Header

Using unions and enumerations for network protocol implementation:

```
#include <stdio.h>
#include <stdint.h>

typedef enum {
    PACKET_TYPE_DATA = 0x01,
    PACKET_TYPE_ACK = 0x02,
    PACKET_TYPE_NACK = 0x03,
    PACKET_TYPE_CONTROL = 0x04
} PacketType;

typedef enum {
    CONTROL_START = 0x01,
    CONTROL_STOP = 0x02,
    CONTROL_RESET = 0x03
} ControlType;

typedef struct {
    uint8_t version : 4;
    uint8_t type : 4;
    uint16_t length;
    uint32_t sequence;
    union {
        struct {
```

```
            uint8_t data[1024];
        } data_packet;

        struct {
            uint32_t ack_sequence;
        } ack_packet;

        struct {
            uint8_t control_type;
            uint8_t reserved[3];
        } control_packet;
    } payload;
} NetworkPacket;

void print_packet_info(const NetworkPacket *packet) {
    printf("Packet Version: %d\n", packet->version);
    printf("Packet Type: 0x%02X\n", packet->type);
    printf("Packet Length: %d\n", packet->length);
    printf("Sequence Number: %u\n", packet->sequence);

    switch (packet->type) {
        case PACKET_TYPE_DATA:
            printf("Data Packet\n");
            break;
        case PACKET_TYPE_ACK:
            printf("ACK for sequence: %u\n", packet->payload.ack_packet.ack_sequence);
            break;
        case PACKET_TYPE_CONTROL:
            printf("Control Type: 0x%02X\n", packet->payload.control_packet.control_type);
            break;
        default:
            printf("Unknown Packet Type\n");
    }
}

int main() {
    NetworkPacket packet = {
        .version = 1,
        .type = PACKET_TYPE_CONTROL,
        .length = sizeof(packet.payload.control_packet),
        .sequence = 1000,
        .payload.control_packet = {
            .control_type = CONTROL_RESET,
            .reserved = {0}
        }
    };
```

```
    print_packet_info(&packet);

    return 0;
}
```

Mathematical Expression Evaluator

Using unions and enumerations for a simple expression evaluator:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef enum {
    TOKEN_NUMBER,
    TOKEN_OPERATOR,
    TOKEN_VARIABLE
} TokenType;

typedef enum {
    OP_ADD = '+',
    OP_SUBTRACT = '-',
    OP_MULTIPLY = '*',
    OP_DIVIDE = '/',
    OP_ASSIGN = '='
} Operator;

typedef struct {
    TokenType type;
    union {
        double number_value;
        Operator operator_value;
        char variable_name[20];
    } data;
} Token;

typedef struct {
    Token *tokens;
    int count;
    int capacity;
} Expression;

Expression* create_expression() {
    Expression *expr = malloc(sizeof(Expression));
    if (expr != NULL) {
        expr->tokens = malloc(10 * sizeof(Token));
```

```
        if (expr->tokens != NULL) {
            expr->count = 0;
            expr->capacity = 10;
        } else {
            free(expr);
            expr = NULL;
        }
    }
    return expr;
}

void add_number_token(Expression *expr, double value) {
    if (expr->count >= expr->capacity) {
        Token *new_tokens = realloc(expr->tokens,
                                    (expr->capacity + 10) * sizeof(Token));
        if (new_tokens != NULL) {
            expr->tokens = new_tokens;
            expr->capacity += 10;
        } else {
            return;
        }
    }

    Token *token = &expr->tokens[expr->count];
    token->type = TOKEN_NUMBER;
    token->data.number_value = value;
    expr->count++;
}

void add_operator_token(Expression *expr, Operator op) {
    if (expr->count >= expr->capacity) {
        Token *new_tokens = realloc(expr->tokens,
                                    (expr->capacity + 10) * sizeof(Token));
        if (new_tokens != NULL) {
            expr->tokens = new_tokens;
            expr->capacity += 10;
        } else {
            return;
        }
    }

    Token *token = &expr->tokens[expr->count];
    token->type = TOKEN_OPERATOR;
    token->data.operator_value = op;
    expr->count++;
}
```

```
void print_expression(const Expression *expr) {
    for (int i = 0; i < expr->count; i++) {
        const Token *token = &expr->tokens[i];

        switch (token->type) {
            case TOKEN_NUMBER:
                printf("%.2f ", token->data.number_value);
                break;
            case TOKEN_OPERATOR:
                printf("%c ", token->data.operator_value);
                break;
            case TOKEN_VARIABLE:
                printf("%s ", token->data.variable_name);
                break;
        }
    }
    printf("\n");
}

int main() {
    Expression *expr = create_expression();
    if (expr == NULL) {
        printf("Failed to create expression\n");
        return 1;
    }

    // Create expression: 3.14 + 2.5 * x
    add_number_token(expr, 3.14);
    add_operator_token(expr, OP_ADD);
    add_number_token(expr, 2.5);
    add_operator_token(expr, OP_MULTIPLY);
    // For simplicity, we'll represent variable as a number
    add_number_token(expr, 0);  // Placeholder for variable x

    printf("Expression: ");
    print_expression(expr);

    // Clean up
    free(expr->tokens);
    free(expr);

    return 0;
}
```

Summary

Unions and enumerations are powerful features that extend the capabilities of C programming:

1. **Unions**: Allow different data types to share the same memory location, useful for memory optimization and type-generic programming
2. **Memory Sharing**: All union members occupy the same memory space, with size equal to the largest member
3. **Practical Applications**: Hardware register representation, variant types, and memory-efficient data storage
4. **Anonymous Unions**: C11 feature enabling direct access to union members (C11)
5. **Enumerations**: Provide named integer constants for improved code readability and maintainability
6. **Explicit Values**: Enumerations can have specific values, including negative numbers
7. **Type Safety**: Combining unions with enumerations creates type-safe variant types
8. **Advanced Features**: C23 introduces more control over enumeration types

Understanding unions and enumerations enables programmers to write more efficient, readable, and maintainable C code, particularly in systems programming, embedded development, and protocol implementation.

Advanced Structure Concepts

Introduction

Advanced structure concepts in C extend beyond basic structure usage to include sophisticated features like bit fields, structure packing, flexible array members, and self-referential structures. These concepts enable programmers to optimize memory usage, implement complex data structures, and create efficient representations of real-world entities. Understanding these advanced features is essential for systems programming, embedded development, and performance-critical applications.

Bit Fields

Basic Bit Field Declaration

Bit fields allow you to specify the exact number of bits used for structure members, enabling efficient memory usage for small values:

```c
#include <stdio.h>

struct FilePermissions {
    unsigned int read : 1;     // 1 bit
    unsigned int write : 1;    // 1 bit
    unsigned int execute : 1;  // 1 bit
    unsigned int reserved : 5; // 5 bits
};

int main() {
    struct FilePermissions perm;

    // Set permissions
    perm.read = 1;
    perm.write = 1;
    perm.execute = 0;
    perm.reserved = 0;

    printf("Size of struct: %zu bytes\n", sizeof(perm));
    printf("Read: %d\n", perm.read);
    printf("Write: %d\n", perm.write);
    printf("Execute: %d\n", perm.execute);
```

```
    printf("Reserved: %d\n", perm.reserved);

    return 0;
}
```

Practical Bit Field Usage

1. Hardware Register Representation

Bit fields are commonly used to represent hardware registers with specific bit layouts:

```
#include <stdio.h>
#include <stdint.h>

struct GPIO_Register {
    uint32_t pin0  : 2;  // 2 bits for pin 0 configuration
    uint32_t pin1  : 2;  // 2 bits for pin 1 configuration
    uint32_t pin2  : 2;  // 2 bits for pin 2 configuration
    uint32_t pin3  : 2;  // 2 bits for pin 3 configuration
    uint32_t mode  : 4;  // 4 bits for mode selection
    uint32_t reserved : 20; // Remaining bits reserved
};

int main() {
    struct GPIO_Register gpio = {0};

    // Configure pins
    gpio.pin0 = 1;  // Output mode
    gpio.pin1 = 0;  // Input mode
    gpio.pin2 = 1;  // Output mode
    gpio.pin3 = 2;  // Alternate function
    gpio.mode = 5;  // Specific mode

    printf("GPIO Register Value: 0x%08X\n", *(uint32_t*)&gpio);
    printf("Pin 0 Configuration: %d\n", gpio.pin0);
    printf("Pin 1 Configuration: %d\n", gpio.pin1);
    printf("Mode: %d\n", gpio.mode);

    return 0;
}
```

2. Network Protocol Headers

Bit fields are useful for implementing network protocol headers with specific bit layouts:

```c
#include <stdio.h>
#include <stdint.h>

struct IP_Header {
    uint32_t version : 4;        // 4 bits
    uint32_t ihl : 4;           // 4 bits
    uint32_t tos : 8;           // 8 bits
    uint32_t total_length : 16; // 16 bits
    uint32_t identification : 16; // 16 bits
    uint32_t flags : 3;         // 3 bits
    uint32_t fragment_offset : 13; // 13 bits
    uint32_t ttl : 8;           // 8 bits
    uint32_t protocol : 8;      // 8 bits
    uint32_t checksum : 16;     // 16 bits
    uint32_t source_ip : 32;    // 32 bits
    uint32_t dest_ip : 32;      // 32 bits
};

int main() {
    struct IP_Header ip_header = {0};

    // Set header values
    ip_header.version = 4;           // IPv4
    ip_header.ihl = 5;              // 5 * 4 = 20 bytes
    ip_header.tos = 0;
    ip_header.total_length = 100;
    ip_header.identification = 12345;
    ip_header.flags = 2;            // Don't fragment
    ip_header.fragment_offset = 0;
    ip_header.ttl = 64;
    ip_header.protocol = 6;         // TCP
    ip_header.source_ip = 0xC0A80101; // 192.168.1.1
    ip_header.dest_ip = 0xC0A80102;   // 192.168.1.2

    printf("IP Header Size: %zu bytes\n", sizeof(ip_header));
    printf("Version: %d\n", ip_header.version);
    printf("Header Length: %d words\n", ip_header.ihl);
    printf("Total Length: %d bytes\n", ip_header.total_length);
    printf("Protocol: %d\n", ip_header.protocol);

    return 0;
}
```

Bit Field Limitations and Considerations

1. Portability Issues

Bit field layout is implementation-defined and may vary between compilers:

```
#include <stdio.h>

struct BitFieldExample {
    unsigned int a : 3;
    unsigned int b : 4;
    unsigned int c : 5;
};

int main() {
    struct BitFieldExample bf = {1, 2, 3};

    printf("Size: %zu bytes\n", sizeof(bf));
    printf("Values: a=%d, b=%d, c=%d\n", bf.a, bf.b, bf.c);

    // Bit layout is implementation-defined
    unsigned int *ptr = (unsigned int*)&bf;
    printf("Raw value: 0x%08X\n", *ptr);

    return 0;
}
```

2. Alignment and Padding

Bit fields may introduce padding for alignment:

```
#include <stdio.h>

struct PackedBits {
    unsigned int a : 1;
    unsigned int b : 1;
    unsigned int c : 1;
};

struct AlignedStruct {
    char x;
    struct PackedBits bits;
    char y;
};

int main() {
```

```
    printf("Size of PackedBits: %zu bytes\n", sizeof(struct PackedBits));
    printf("Size of AlignedStruct: %zu bytes\n", sizeof(struct AlignedStruct));

    return 0;
}
```

Structure Packing and Alignment

Default Structure Alignment

Compilers add padding to structures to ensure proper alignment:

```
#include <stdio.h>

struct Unpacked {
    char a;      // 1 byte
    int b;       // 4 bytes (may have 3 bytes padding before)
    char c;      // 1 byte (may have 3 bytes padding after)
};

#pragma pack(1)
struct Packed {
    char a;      // 1 byte
    int b;       // 4 bytes (no padding)
    char c;      // 1 byte (no padding)
};
#pragma pack()

int main() {
    printf("Size of Unpacked: %zu bytes\n", sizeof(struct Unpacked));
    printf("Size of Packed: %zu bytes\n", sizeof(struct Packed));

    // Show member offsets
    struct Unpacked u = {0};
    printf("Unpacked offsets - a: %td, b: %td, c: %td\n",
           (char*)&u.a - (char*)&u,
           (char*)&u.b - (char*)&u,
           (char*)&u.c - (char*)&u);

    struct Packed p = {0};
    printf("Packed offsets - a: %td, b: %td, c: %td\n",
           (char*)&p.a - (char*)&p,
           (char*)&p.b - (char*)&p,
           (char*)&p.c - (char*)&p);
```

```
    return 0;
}
```

Controlling Alignment with Pragma Pack

The `#pragma pack` directive controls structure packing:

```
#include <stdio.h>

// Default alignment
struct Default {
    char a;
    int b;
    char c;
};

// 1-byte alignment (no padding)
#pragma pack(1)
struct Packed1 {
    char a;
    int b;
    char c;
};
#pragma pack()

// 2-byte alignment
#pragma pack(2)
struct Packed2 {
    char a;
    int b;
    char c;
};
#pragma pack()

// 4-byte alignment
#pragma pack(4)
struct Packed4 {
    char a;
    int b;
    char c;
};
#pragma pack()

int main() {
    printf("Default alignment: %zu bytes\n", sizeof(struct Default));
    printf("1-byte packing: %zu bytes\n", sizeof(struct Packed1));
```

```
    printf("2-byte packing: %zu bytes\n", sizeof(struct Packed2));
    printf("4-byte packing: %zu bytes\n", sizeof(struct Packed4));

    return 0;
}
```

Using _Alignas (C11)

C11 introduces `_Alignas` for explicit alignment control:

```
#include <stdio.h>
#include <stdalign.h>

struct AlignedStruct {
    _Alignas(16) char data[16];  // Align to 16-byte boundary
    int value;
};

int main() {
    struct AlignedStruct s = {0};

    printf("Size: %zu bytes\n", sizeof(s));
    printf("Address of data: %p\n", (void*)s.data);
    printf("Alignment of data: %zu\n", (size_t)s.data % 16);

    return 0;
}
```

Flexible Array Members (C99)

Basic Flexible Array Member

Flexible array members allow structures to have variable-sized arrays at the end:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct DynamicArray {
    size_t count;
    int data[];  // Flexible array member (must be last)
};

struct DynamicArray* create_array(size_t count) {
```

```
    // Allocate memory for structure + array
    struct DynamicArray *arr = malloc(sizeof(struct DynamicArray) +
                                      count * sizeof(int));
    if (arr != NULL) {
        arr->count = count;
        // arr->data can now be used as array of 'count' integers
    }
    return arr;
}

int main() {
    struct DynamicArray *arr = create_array(5);
    if (arr != NULL) {
        // Initialize array elements
        for (size_t i = 0; i < arr->count; i++) {
            arr->data[i] = (int)(i * i);
        }

        // Print array elements
        printf("Array elements: ");
        for (size_t i = 0; i < arr->count; i++) {
            printf("%d ", arr->data[i]);
        }
        printf("\n");

        printf("Size of structure: %zu bytes\n", sizeof(struct DynamicArray));
        printf("Total allocated size: %zu bytes\n",
               sizeof(struct DynamicArray) + arr->count * sizeof(int));

        // Free memory
        free(arr);
    }

    return 0;
}
```

Flexible Array in String Structure

Using flexible array members for dynamic strings:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct DynamicString {
    size_t length;
```

```
    char data[];  // Flexible array member
};

struct DynamicString* create_string(const char *source) {
    size_t len = strlen(source);
    // Allocate memory for structure + string + null terminator
    struct DynamicString *str = malloc(sizeof(struct DynamicString) +
                                       len + 1);
    if (str != NULL) {
        str->length = len;
        strcpy(str->data, source);
    }
    return str;
}

struct DynamicString* append_strings(const struct DynamicString *s1,
                                     const char *s2) {
    size_t len2 = strlen(s2);
    size_t new_len = s1->length + len2;

    struct DynamicString *result = malloc(sizeof(struct DynamicString) +
                                          new_len + 1);
    if (result != NULL) {
        result->length = new_len;
        strcpy(result->data, s1->data);
        strcat(result->data, s2);
    }
    return result;
}

int main() {
    struct DynamicString *str1 = create_string("Hello, ");
    if (str1 != NULL) {
        printf("String 1: %s (length: %zu)\n", str1->data, str1->length);

        struct DynamicString *str2 = append_strings(str1, "World!");
        if (str2 != NULL) {
            printf("String 2: %s (length: %zu)\n", str2->data, str2->length);
            free(str2);
        }

        free(str1);
    }

    return 0;
}
```

Self-Referential Structures

Linked List Implementation

Self-referential structures are fundamental for implementing linked data structures:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

// Function to create a new node
struct Node* create_node(int data) {
    struct Node *node = malloc(sizeof(struct Node));
    if (node != NULL) {
        node->data = data;
        node->next = NULL;
    }
    return node;
}

// Function to insert at beginning
struct Node* insert_at_beginning(struct Node *head, int data) {
    struct Node *new_node = create_node(data);
    if (new_node != NULL) {
        new_node->next = head;
        head = new_node;
    }
    return head;
}

// Function to print the list
void print_list(struct Node *head) {
    struct Node *current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

// Function to free the list
void free_list(struct Node *head) {
    struct Node *current = head;
```

```
    while (current != NULL) {
        struct Node *next = current->next;
        free(current);
        current = next;
    }
}

int main() {
    struct Node *head = NULL;

    // Build the list: 3 -> 2 -> 1 -> NULL
    head = insert_at_beginning(head, 1);
    head = insert_at_beginning(head, 2);
    head = insert_at_beginning(head, 3);

    printf("Linked List: ");
    print_list(head);

    // Free the list
    free_list(head);

    return 0;
}
```

Binary Tree Implementation

Self-referential structures for binary trees:

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
};

// Function to create a new node
struct TreeNode* create_node(int data) {
    struct TreeNode *node = malloc(sizeof(struct TreeNode));
    if (node != NULL) {
        node->data = data;
        node->left = NULL;
        node->right = NULL;
    }
    return node;
```

```
}

// Function to insert a node in BST
struct TreeNode* insert(struct TreeNode *root, int data) {
    if (root == NULL) {
        return create_node(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

// In-order traversal
void inorder_traversal(struct TreeNode *root) {
    if (root != NULL) {
        inorder_traversal(root->left);
        printf("%d ", root->data);
        inorder_traversal(root->right);
    }
}

// Free the tree
void free_tree(struct TreeNode *root) {
    if (root != NULL) {
        free_tree(root->left);
        free_tree(root->right);
        free(root);
    }
}

int main() {
    struct TreeNode *root = NULL;

    // Insert nodes
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);
```

```
    printf("In-order traversal: ");
    inorder_traversal(root);
    printf("\n");

    // Free the tree
    free_tree(root);

    return 0;
}
```

Advanced Structure Techniques

Structure with Function Pointers (C Object-Oriented Approach)

Using function pointers to implement object-oriented concepts in C:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Forward declaration
struct Shape;

// Function pointer types
typedef double (*area_func)(const struct Shape *self);
typedef void (*print_func)(const struct Shape *self);

// Base structure
struct Shape {
    area_func area;
    print_func print;
    char name[20];
};

// Rectangle structure
struct Rectangle {
    struct Shape base;
    double width;
    double height;
};

// Circle structure
struct Circle {
    struct Shape base;
    double radius;
```

```
};

// Area calculation functions
double rectangle_area(const struct Shape *self) {
    const struct Rectangle *rect = (const struct Rectangle *)self;
    return rect->width * rect->height;
}

double circle_area(const struct Shape *self) {
    const struct Circle *circle = (const struct Circle *)self;
    return 3.14159 * circle->radius * circle->radius;
}

// Print functions
void rectangle_print(const struct Shape *self) {
    const struct Rectangle *rect = (const struct Rectangle *)self;
    printf("Rectangle: %s, Width: %.2f, Height: %.2f\n",
           self->name, rect->width, rect->height);
}

void circle_print(const struct Shape *self) {
    const struct Circle *circle = (const struct Circle *)self;
    printf("Circle: %s, Radius: %.2f\n",
           self->name, circle->radius);
}

// Constructor functions
struct Rectangle* create_rectangle(const char *name, double width, double height) {
    struct Rectangle *rect = malloc(sizeof(struct Rectangle));
    if (rect != NULL) {
        strncpy(rect->base.name, name, sizeof(rect->base.name) - 1);
        rect->base.name[sizeof(rect->base.name) - 1] = '\0';
        rect->base.area = rectangle_area;
        rect->base.print = rectangle_print;
        rect->width = width;
        rect->height = height;
    }
    return rect;
}

struct Circle* create_circle(const char *name, double radius) {
    struct Circle *circle = malloc(sizeof(struct Circle));
    if (circle != NULL) {
        strncpy(circle->base.name, name, sizeof(circle->base.name) - 1);
        circle->base.name[sizeof(circle->base.name) - 1] = '\0';
        circle->base.area = circle_area;
        circle->base.print = circle_print;
```

```
        circle->radius = radius;
    }
    return circle;
}

int main() {
    struct Rectangle *rect = create_rectangle("MyRectangle", 5.0, 3.0);
    struct Circle *circle = create_circle("MyCircle", 4.0);

    if (rect != NULL && circle != NULL) {
        // Polymorphic behavior
        struct Shape *shapes[] = {
            (struct Shape*)rect,
            (struct Shape*)circle
        };

        for (int i = 0; i < 2; i++) {
            shapes[i]->print(shapes[i]);
            printf("Area: %.2f\n", shapes[i]->area(shapes[i]));
        }

        free(rect);
        free(circle);
    }

    return 0;
}
```

Structure with Nested Anonymous Structures (C11)

Using nested anonymous structures for cleaner APIs:

```
#include <stdio.h>

struct Point3D {
    union {
        struct {
            double x, y, z;
        };
        double coords[3];
    };
};

struct Color {
    union {
        struct {
```

```
            unsigned char r, g, b, a;
        };
        unsigned char components[4];
    };
};

int main() {
    struct Point3D point = {1.0, 2.0, 3.0};
    struct Color color = {255, 128, 64, 255};

    // Access using named members
    printf("Point: (%.1f, %.1f, %.1f)\n", point.x, point.y, point.z);

    // Access using array
    printf("Coordinates: ");
    for (int i = 0; i < 3; i++) {
        printf("%.1f ", point.coords[i]);
    }
    printf("\n");

    // Access color components
    printf("Color: RGBA(%d, %d, %d, %d)\n",
           color.r, color.g, color.b, color.a);

    // Modify using array access
    for (int i = 0; i < 4; i++) {
        color.components[i] = 128;
    }

    printf("Modified color: RGBA(%d, %d, %d, %d)\n",
           color.r, color.g, color.b, color.a);

    return 0;
}
```

Practical Examples

Configuration Manager with Advanced Structures

Combining advanced structure concepts for a configuration manager:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
// Configuration entry types
typedef enum {
    CONFIG_INT,
    CONFIG_FLOAT,
    CONFIG_STRING,
    CONFIG_BOOL
} ConfigType;

// Configuration entry with bit fields
struct ConfigEntry {
    unsigned int type : 3;        // 3 bits for type
    unsigned int is_default : 1;  // 1 bit for default flag
    unsigned int is_locked : 1;   // 1 bit for lock flag
    unsigned int reserved : 27;   // 27 bits reserved

    char key[32];
    union {
        int int_value;
        float float_value;
        char *string_value;
        int bool_value;
    } value;
};

// Configuration manager
struct ConfigManager {
    struct ConfigEntry *entries;
    size_t count;
    size_t capacity;
};

struct ConfigManager* create_config_manager() {
    struct ConfigManager *cm = malloc(sizeof(struct ConfigManager));
    if (cm != NULL) {
        cm->entries = malloc(10 * sizeof(struct ConfigEntry));
        if (cm->entries != NULL) {
            cm->count = 0;
            cm->capacity = 10;
        } else {
            free(cm);
            cm = NULL;
        }
    }
    return cm;
}
```

```
int add_int_config(struct ConfigManager *cm, const char *key, int value) {
    if (cm->count >= cm->capacity) {
        struct ConfigEntry *new_entries = realloc(cm->entries,
            (cm->capacity + 10) * sizeof(struct ConfigEntry));
        if (new_entries == NULL) return -1;
        cm->entries = new_entries;
        cm->capacity += 10;
    }

    struct ConfigEntry *entry = &cm->entries[cm->count];
    entry->type = CONFIG_INT;
    entry->is_default = 0;
    entry->is_locked = 0;
    entry->reserved = 0;
    strncpy(entry->key, key, sizeof(entry->key) - 1);
    entry->key[sizeof(entry->key) - 1] = '\0';
    entry->value.int_value = value;

    cm->count++;
    return 0;
}

void print_config(const struct ConfigManager *cm) {
    for (size_t i = 0; i < cm->count; i++) {
        const struct ConfigEntry *entry = &cm->entries[i];
        printf("Key: %s, Type: %d, Default: %d, Locked: %d\n",
               entry->key, entry->type, entry->is_default, entry->is_locked);

        switch (entry->type) {
            case CONFIG_INT:
                printf("  Value: %d (int)\n", entry->value.int_value);
                break;
            // Add other cases as needed
        }
    }
}

void free_config_manager(struct ConfigManager *cm) {
    if (cm != NULL) {
        free(cm->entries);
        free(cm);
    }
}

int main() {
    struct ConfigManager *cm = create_config_manager();
    if (cm == NULL) {
```

```
        printf("Failed to create config manager\n");
        return 1;
    }

    add_int_config(cm, "window_width", 800);
    add_int_config(cm, "window_height", 600);
    add_int_config(cm, "max_fps", 60);

    print_config(cm);

    free_config_manager(cm);
    return 0;
}
```

Memory Pool with Flexible Array Members

Implementing a memory pool using flexible array members:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct MemoryBlock {
    size_t size;
    int is_free;
    char data[];  // Flexible array member
};

struct MemoryPool {
    size_t block_size;
    size_t total_blocks;
    struct MemoryBlock blocks[];  // Flexible array member
};

struct MemoryPool* create_memory_pool(size_t block_size, size_t num_blocks) {
    size_t pool_size = sizeof(struct MemoryPool) +
                       num_blocks * (sizeof(struct MemoryBlock) + block_size);

    struct MemoryPool *pool = malloc(pool_size);
    if (pool != NULL) {
        pool->block_size = block_size;
        pool->total_blocks = num_blocks;

        // Initialize blocks
        char *block_data = (char*)(pool->blocks);
        for (size_t i = 0; i < num_blocks; i++) {
```

```
            struct MemoryBlock *block = (struct MemoryBlock*)block_data;
            block->size = block_size;
            block->is_free = 1;
            block_data += sizeof(struct MemoryBlock) + block_size;
        }
    }
    return pool;
}

void* pool_alloc(struct MemoryPool *pool, size_t size) {
    if (size > pool->block_size) return NULL;

    char *block_data = (char*)(pool->blocks);
    for (size_t i = 0; i < pool->total_blocks; i++) {
        struct MemoryBlock *block = (struct MemoryBlock*)block_data;
        if (block->is_free && block->size >= size) {
            block->is_free = 0;
            return block->data;
        }
        block_data += sizeof(struct MemoryBlock) + pool->block_size;
    }
    return NULL;
}

void pool_free(struct MemoryPool *pool, void *ptr) {
    if (ptr == NULL) return;

    char *pool_start = (char*)pool;
    char *ptr_char = (char*)ptr;

    if (ptr_char >= pool_start &&
        ptr_char < pool_start + sizeof(struct MemoryPool) +
                   pool->total_blocks * (sizeof(struct MemoryBlock) + pool->block_size)) {

        char *block_data = (char*)(pool->blocks);
        for (size_t i = 0; i < pool->total_blocks; i++) {
            struct MemoryBlock *block = (struct MemoryBlock*)block_data;
            if (ptr_char == block->data) {
                block->is_free = 1;
                return;
            }
            block_data += sizeof(struct MemoryBlock) + pool->block_size;
        }
    }
}
```

```c
void free_memory_pool(struct MemoryPool *pool) {
    free(pool);
}

int main() {
    struct MemoryPool *pool = create_memory_pool(64, 10);
    if (pool == NULL) {
        printf("Failed to create memory pool\n");
        return 1;
    }

    // Allocate memory from pool
    int *numbers = (int*)pool_alloc(pool, 10 * sizeof(int));
    if (numbers != NULL) {
        for (int i = 0; i < 10; i++) {
            numbers[i] = i * i;
        }

        printf("Numbers: ");
        for (int i = 0; i < 10; i++) {
            printf("%d ", numbers[i]);
        }
        printf("\n");

        // Free memory
        pool_free(pool, numbers);
    }

    free_memory_pool(pool);
    return 0;
}
```

Summary

Advanced structure concepts in C provide powerful mechanisms for creating efficient and sophisticated data representations:

1. **Bit Fields**: Enable precise control over memory usage for small values and hardware register representation
2. **Structure Packing**: Control memory alignment and padding for optimization
3. **Flexible Array Members**: Allow structures to have variable-sized arrays at the end (C99)
4. **Self-Referential Structures**: Enable implementation of linked data structures like lists and trees
5. **Function Pointers in Structures**: Support object-oriented programming concepts in C
6. **Anonymous Structures**: Provide cleaner APIs (C11)

These advanced concepts are essential for systems programming, embedded development, and performance-critical applications where memory efficiency and precise control over data representation are crucial.

Abstract Data Types

Introduction

Abstract Data Types (ADTs) are a fundamental concept in software engineering that separate the interface of a data type from its implementation. In C, ADTs are implemented using structures, function pointers, and information hiding principles to create modular, maintainable, and reusable code. This approach enables programmers to encapsulate data and operations, providing a clean interface while hiding implementation details.

Information Hiding Principles

Separation of Interface and Implementation

Information hiding is achieved by separating the public interface (what users can access) from the private implementation (internal details):

```
// stack.h - Public interface
#ifndef STACK_H
#define STACK_H

#include <stddef.h>

// Opaque pointer - hides implementation details
typedef struct Stack Stack;

// Public functions
Stack* stack_create(size_t capacity);
void stack_destroy(Stack *stack);
int stack_push(Stack *stack, int value);
int stack_pop(Stack *stack);
int stack_peek(const Stack *stack);
int stack_is_empty(const Stack *stack);
int stack_is_full(const Stack *stack);
size_t stack_size(const Stack *stack);

#endif // STACK_H
```

```c
// stack.c - Private implementation
#include "stack.h"
#include <stdlib.h>
#include <stdbool.h>

// Actual structure definition (hidden from users)
struct Stack {
    int *data;
    size_t capacity;
    size_t top;
};

Stack* stack_create(size_t capacity) {
    Stack *stack = malloc(sizeof(Stack));
    if (stack != NULL) {
        stack->data = malloc(capacity * sizeof(int));
        if (stack->data != NULL) {
            stack->capacity = capacity;
            stack->top = 0;
        } else {
            free(stack);
            stack = NULL;
        }
    }
    return stack;
}

void stack_destroy(Stack *stack) {
    if (stack != NULL) {
        free(stack->data);
        free(stack);
    }
}

int stack_push(Stack *stack, int value) {
    if (stack == NULL || stack->top >= stack->capacity) {
        return -1; // Error
    }
    stack->data[stack->top] = value;
    stack->top++;
    return 0; // Success
}

int stack_pop(Stack *stack) {
    if (stack == NULL || stack->top == 0) {
        return -1; // Error
    }
```

```
    stack->top--;
    return stack->data[stack->top];
}

int stack_peek(const Stack *stack) {
    if (stack == NULL || stack->top == 0) {
        return -1; // Error
    }
    return stack->data[stack->top - 1];
}

int stack_is_empty(const Stack *stack) {
    return (stack == NULL || stack->top == 0);
}

int stack_is_full(const Stack *stack) {
    return (stack != NULL && stack->top >= stack->capacity);
}

size_t stack_size(const Stack *stack) {
    return (stack != NULL) ? stack->top : 0;
}
```

```
// main.c - Using the ADT
#include <stdio.h>
#include "stack.h"

int main() {
    Stack *stack = stack_create(10);
    if (stack == NULL) {
        printf("Failed to create stack\n");
        return 1;
    }

    // Use the stack
    stack_push(stack, 10);
    stack_push(stack, 20);
    stack_push(stack, 30);

    printf("Stack size: %zu\n", stack_size(stack));
    printf("Top element: %d\n", stack_peek(stack));

    while (!stack_is_empty(stack)) {
        printf("Popped: %d\n", stack_pop(stack));
    }
```

```
    stack_destroy(stack);
    return 0;
}
```

Opaque Pointers

Creating Opaque Pointers

Opaque pointers hide the implementation details of a data structure by only declaring the structure type in the header file:

```
// vector.h - Opaque pointer example
#ifndef VECTOR_H
#define VECTOR_H

#include <stddef.h>

// Forward declaration - no structure definition
typedef struct Vector Vector;

// Public interface
Vector* vector_create(size_t initial_capacity);
void vector_destroy(Vector *vector);
int vector_push_back(Vector *vector, int value);
int vector_pop_back(Vector *vector);
int vector_get(const Vector *vector, size_t index);
int vector_set(Vector *vector, size_t index, int value);
size_t vector_size(const Vector *vector);
size_t vector_capacity(const Vector *vector);

#endif // VECTOR_H
```

```
// vector.c - Implementation with opaque pointer
#include "vector.h"
#include <stdlib.h>

// Actual structure definition (private)
struct Vector {
    int *data;
    size_t size;
    size_t capacity;
};

Vector* vector_create(size_t initial_capacity) {
```

```
    Vector *vector = malloc(sizeof(Vector));
    if (vector != NULL) {
        vector->data = malloc(initial_capacity * sizeof(int));
        if (vector->data != NULL) {
            vector->size = 0;
            vector->capacity = initial_capacity;
        } else {
            free(vector);
            vector = NULL;
        }
    }
    return vector;
}

void vector_destroy(Vector *vector) {
    if (vector != NULL) {
        free(vector->data);
        free(vector);
    }
}

int vector_push_back(Vector *vector, int value) {
    if (vector == NULL) return -1;

    // Resize if needed
    if (vector->size >= vector->capacity) {
        size_t new_capacity = vector->capacity * 2;
        int *new_data = realloc(vector->data, new_capacity * sizeof(int));
        if (new_data == NULL) return -1;

        vector->data = new_data;
        vector->capacity = new_capacity;
    }

    vector->data[vector->size] = value;
    vector->size++;
    return 0;
}

int vector_pop_back(Vector *vector) {
    if (vector == NULL || vector->size == 0) return -1;
    vector->size--;
    return vector->data[vector->size];
}

int vector_get(const Vector *vector, size_t index) {
    if (vector == NULL || index >= vector->size) return -1;
```

```
    return vector->data[index];
}

int vector_set(Vector *vector, size_t index, int value) {
    if (vector == NULL || index >= vector->size) return -1;
    vector->data[index] = value;
    return 0;
}

size_t vector_size(const Vector *vector) {
    return (vector != NULL) ? vector->size : 0;
}

size_t vector_capacity(const Vector *vector) {
    return (vector != NULL) ? vector->capacity : 0;
}
```

Constructor and Destructor Patterns

Resource Acquisition Is Initialization (RAII) in C

Implementing constructor and destructor patterns for proper resource management:

```
// file_manager.h
#ifndef FILE_MANAGER_H
#define FILE_MANAGER_H

#include <stdio.h>

typedef struct FileManager FileManager;

FileManager* file_manager_create(const char *filename, const char *mode);
void file_manager_destroy(FileManager *fm);
int file_manager_write(FileManager *fm, const char *data);
int file_manager_read(FileManager *fm, char *buffer, size_t size);
long file_manager_get_size(FileManager *fm);

#endif // FILE_MANAGER_H
```

```
// file_manager.c
#include "file_manager.h"
#include <stdlib.h>
#include <string.h>
```

```
struct FileManager {
    FILE *file;
    char *filename;
    char *mode;
};

FileManager* file_manager_create(const char *filename, const char *mode) {
    FileManager *fm = malloc(sizeof(FileManager));
    if (fm == NULL) return NULL;

    fm->filename = malloc(strlen(filename) + 1);
    if (fm->filename == NULL) {
        free(fm);
        return NULL;
    }

    fm->mode = malloc(strlen(mode) + 1);
    if (fm->mode == NULL) {
        free(fm->filename);
        free(fm);
        return NULL;
    }

    strcpy(fm->filename, filename);
    strcpy(fm->mode, mode);

    fm->file = fopen(filename, mode);
    if (fm->file == NULL) {
        free(fm->mode);
        free(fm->filename);
        free(fm);
        return NULL;
    }

    return fm;
}

void file_manager_destroy(FileManager *fm) {
    if (fm != NULL) {
        if (fm->file != NULL) {
            fclose(fm->file);
        }
        free(fm->mode);
        free(fm->filename);
        free(fm);
    }
}
```

```c
int file_manager_write(FileManager *fm, const char *data) {
    if (fm == NULL || fm->file == NULL || data == NULL) return -1;
    return fputs(data, fm->file) >= 0 ? 0 : -1;
}

int file_manager_read(FileManager *fm, char *buffer, size_t size) {
    if (fm == NULL || fm->file == NULL || buffer == NULL) return -1;
    return fgets(buffer, (int)size, fm->file) != NULL ? 0 : -1;
}

long file_manager_get_size(FileManager *fm) {
    if (fm == NULL || fm->file == NULL) return -1;

    long current_pos = ftell(fm->file);
    fseek(fm->file, 0, SEEK_END);
    long size = ftell(fm->file);
    fseek(fm->file, current_pos, SEEK_SET);

    return size;
}
```

Generic Abstract Data Types

Using Void Pointers for Generic ADTs

Creating generic ADTs that can work with any data type:

```c
// generic_list.h
#ifndef GENERIC_LIST_H
#define GENERIC_LIST_H

#include <stddef.h>

typedef struct GenericList GenericList;

// Function pointer types for generic operations
typedef void (*destructor_func)(void *data);
typedef int (*compare_func)(const void *a, const void *b);
typedef void (*print_func)(const void *data);

GenericList* generic_list_create(destructor_func destructor);
void generic_list_destroy(GenericList *list);
int generic_list_append(GenericList *list, void *data);
int generic_list_prepend(GenericList *list, void *data);
```

```
void* generic_list_get(const GenericList *list, size_t index);
int generic_list_remove(GenericList *list, size_t index);
size_t generic_list_size(const GenericList *list);
void generic_list_print(const GenericList *list, print_func printer);

#endif // GENERIC_LIST_H
```

```
// generic_list.c
#include "generic_list.h"
#include <stdlib.h>

struct Node {
    void *data;
    struct Node *next;
};

struct GenericList {
    struct Node *head;
    size_t size;
    destructor_func destructor;
};

GenericList* generic_list_create(destructor_func destructor) {
    GenericList *list = malloc(sizeof(GenericList));
    if (list != NULL) {
        list->head = NULL;
        list->size = 0;
        list->destructor = destructor;
    }
    return list;
}

void generic_list_destroy(GenericList *list) {
    if (list != NULL) {
        struct Node *current = list->head;
        while (current != NULL) {
            struct Node *next = current->next;
            if (list->destructor != NULL) {
                list->destructor(current->data);
            }
            free(current);
            current = next;
        }
        free(list);
    }
}
```

```
int generic_list_append(GenericList *list, void *data) {
    if (list == NULL) return -1;

    struct Node *new_node = malloc(sizeof(struct Node));
    if (new_node == NULL) return -1;

    new_node->data = data;
    new_node->next = NULL;

    if (list->head == NULL) {
        list->head = new_node;
    } else {
        struct Node *current = list->head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = new_node;
    }

    list->size++;
    return 0;
}

void* generic_list_get(const GenericList *list, size_t index) {
    if (list == NULL || index >= list->size) return NULL;

    struct Node *current = list->head;
    for (size_t i = 0; i < index && current != NULL; i++) {
        current = current->next;
    }

    return (current != NULL) ? current->data : NULL;
}

int generic_list_remove(GenericList *list, size_t index) {
    if (list == NULL || index >= list->size) return -1;

    struct Node *to_remove;
    if (index == 0) {
        to_remove = list->head;
        list->head = to_remove->next;
    } else {
        struct Node *current = list->head;
        for (size_t i = 0; i < index - 1 && current != NULL; i++) {
            current = current->next;
        }
```

```
        if (current == NULL || current->next == NULL) return -1;

        to_remove = current->next;
        current->next = to_remove->next;
    }

    if (list->destructor != NULL) {
        list->destructor(to_remove->data);
    }
    free(to_remove);
    list->size--;

    return 0;
}

size_t generic_list_size(const GenericList *list) {
    return (list != NULL) ? list->size : 0;
}

void generic_list_print(const GenericList *list, print_func printer) {
    if (list == NULL || printer == NULL) return;

    struct Node *current = list->head;
    while (current != NULL) {
        printer(current->data);
        current = current->next;
    }
}
```

```
// Example usage with different data types
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "generic_list.h"

// Integer destructor
void int_destructor(void *data) {
    free(data);
}

// Integer printer
void int_printer(const void *data) {
    printf("%d ", *(int*)data);
}

// String destructor
```

```c
void string_destructor(void *data) {
    free(data);
}

// String printer
void string_printer(const void *data) {
    printf("%s ", (char*)data);
}

int main() {
    // Integer list
    GenericList *int_list = generic_list_create(int_destructor);
    if (int_list != NULL) {
        for (int i = 1; i <= 5; i++) {
            int *value = malloc(sizeof(int));
            *value = i * 10;
            generic_list_append(int_list, value);
        }

        printf("Integer list: ");
        generic_list_print(int_list, int_printer);
        printf("\n");

        generic_list_destroy(int_list);
    }

    // String list
    GenericList *string_list = generic_list_create(string_destructor);
    if (string_list != NULL) {
        char *str1 = malloc(20);
        char *str2 = malloc(20);
        char *str3 = malloc(20);

        strcpy(str1, "Hello");
        strcpy(str2, "World");
        strcpy(str3, "C");

        generic_list_append(string_list, str1);
        generic_list_append(string_list, str2);
        generic_list_append(string_list, str3);

        printf("String list: ");
        generic_list_print(string_list, string_printer);
        printf("\n");

        generic_list_destroy(string_list);
    }
```

```
    return 0;
}
```

Interface Design in C

Designing Clean APIs

Creating well-designed interfaces that are easy to use and maintain:

```
// hash_table.h - Well-designed hash table interface
#ifndef HASH_TABLE_H
#define HASH_TABLE_H

#include <stddef.h>
#include <stdbool.h>

typedef struct HashTable HashTable;

// Function pointer types for custom operations
typedef size_t (*hash_func)(const void *key);
typedef int (*compare_func)(const void *key1, const void *key2);
typedef void (*destructor_func)(void *data);

// Error codes
typedef enum {
    HASH_SUCCESS = 0,
    HASH_ERROR_NULL_POINTER = -1,
    HASH_ERROR_MEMORY = -2,
    HASH_ERROR_KEY_NOT_FOUND = -3,
    HASH_ERROR_DUPLICATE_KEY = -4
} HashError;

// Public functions
HashTable* hash_table_create(size_t initial_capacity,
                             hash_func hash_function,
                             compare_func compare_function,
                             destructor_func key_destructor,
                             destructor_func value_destructor);

void hash_table_destroy(HashTable *table);
HashError hash_table_insert(HashTable *table, void *key, void *value);
HashError hash_table_remove(HashTable *table, const void *key);
void* hash_table_lookup(const HashTable *table, const void *key);
bool hash_table_contains(const HashTable *table, const void *key);
```

```
size_t hash_table_size(const HashTable *table);
void hash_table_clear(HashTable *table);

// Utility functions
size_t hash_string(const void *key);  // Default string hash function
int compare_strings(const void *key1, const void *key2);  // Default string compare

#endif // HASH_TABLE_H
```

```
// hash_table.c - Implementation
#include "hash_table.h"
#include <stdlib.h>
#include <string.h>

// Hash table entry
struct Entry {
    void *key;
    void *value;
    struct Entry *next;
};

// Hash table structure
struct HashTable {
    struct Entry **buckets;
    size_t bucket_count;
    size_t size;
    hash_func hash_function;
    compare_func compare_function;
    destructor_func key_destructor;
    destructor_func value_destructor;
};

// Default hash function for strings
size_t hash_string(const void *key) {
    const char *str = (const char*)key;
    size_t hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;  // hash * 33 + c
    }

    return hash;
}

// Default compare function for strings
```

```c
int compare_strings(const void *key1, const void *key2) {
    return strcmp((const char*)key1, (const char*)key2);
}

HashTable* hash_table_create(size_t initial_capacity,
                             hash_func hash_function,
                             compare_func compare_function,
                             destructor_func key_destructor,
                             destructor_func value_destructor) {
    HashTable *table = malloc(sizeof(HashTable));
    if (table == NULL) return NULL;

    table->buckets = calloc(initial_capacity, sizeof(struct Entry*));
    if (table->buckets == NULL) {
        free(table);
        return NULL;
    }

    table->bucket_count = initial_capacity;
    table->size = 0;
    table->hash_function = hash_function ? hash_function : hash_string;
    table->compare_function = compare_function ? compare_function : compare_strings;
    table->key_destructor = key_destructor;
    table->value_destructor = value_destructor;

    return table;
}

void hash_table_destroy(HashTable *table) {
    if (table != NULL) {
        hash_table_clear(table);
        free(table->buckets);
        free(table);
    }
}

HashError hash_table_insert(HashTable *table, void *key, void *value) {
    if (table == NULL || key == NULL) return HASH_ERROR_NULL_POINTER;

    size_t index = table->hash_function(key) % table->bucket_count;

    // Check for duplicate key
    struct Entry *current = table->buckets[index];
    while (current != NULL) {
        if (table->compare_function(current->key, key) == 0) {
            return HASH_ERROR_DUPLICATE_KEY;
        }
```

```
        current = current->next;
    }

    // Create new entry
    struct Entry *new_entry = malloc(sizeof(struct Entry));
    if (new_entry == NULL) return HASH_ERROR_MEMORY;

    new_entry->key = key;
    new_entry->value = value;
    new_entry->next = table->buckets[index];
    table->buckets[index] = new_entry;
    table->size++;

    return HASH_SUCCESS;
}

void* hash_table_lookup(const HashTable *table, const void *key) {
    if (table == NULL || key == NULL) return NULL;

    size_t index = table->hash_function(key) % table->bucket_count;

    struct Entry *current = table->buckets[index];
    while (current != NULL) {
        if (table->compare_function(current->key, key) == 0) {
            return current->value;
        }
        current = current->next;
    }

    return NULL;
}

HashError hash_table_remove(HashTable *table, const void *key) {
    if (table == NULL || key == NULL) return HASH_ERROR_NULL_POINTER;

    size_t index = table->hash_function(key) % table->bucket_count;

    struct Entry *current = table->buckets[index];
    struct Entry *previous = NULL;

    while (current != NULL) {
        if (table->compare_function(current->key, key) == 0) {
            if (previous == NULL) {
                table->buckets[index] = current->next;
            } else {
                previous->next = current->next;
            }
```

```
            if (table->key_destructor != NULL) {
                table->key_destructor(current->key);
            }
            if (table->value_destructor != NULL) {
                table->value_destructor(current->value);
            }

            free(current);
            table->size--;
            return HASH_SUCCESS;
        }

        previous = current;
        current = current->next;
    }

    return HASH_ERROR_KEY_NOT_FOUND;
}

bool hash_table_contains(const HashTable *table, const void *key) {
    return hash_table_lookup(table, key) != NULL;
}

size_t hash_table_size(const HashTable *table) {
    return (table != NULL) ? table->size : 0;
}

void hash_table_clear(HashTable *table) {
    if (table == NULL) return;

    for (size_t i = 0; i < table->bucket_count; i++) {
        struct Entry *current = table->buckets[i];
        while (current != NULL) {
            struct Entry *next = current->next;

            if (table->key_destructor != NULL) {
                table->key_destructor(current->key);
            }
            if (table->value_destructor != NULL) {
                table->value_destructor(current->value);
            }

            free(current);
            current = next;
        }
        table->buckets[i] = NULL;
```

```
    }

    table->size = 0;
}
```

Practical Examples

Complete ADT Implementation: Dynamic Array

A complete implementation of a dynamic array ADT with comprehensive error handling:

```
// dynamic_array.h
#ifndef DYNAMIC_ARRAY_H
#define DYNAMIC_ARRAY_H

#include <stddef.h>

typedef struct DynamicArray DynamicArray;

// Error codes
typedef enum {
    DYNARRAY_SUCCESS = 0,
    DYNARRAY_ERROR_NULL_POINTER = -1,
    DYNARRAY_ERROR_OUT_OF_BOUNDS = -2,
    DYNARRAY_ERROR_MEMORY = -3,
    DYNARRAY_ERROR_INVALID_SIZE = -4
} DynamicArrayError;

// Function pointer types
typedef void (*destructor_func)(void *data);
typedef int (*compare_func)(const void *a, const void *b);

// Public interface
DynamicArray* dynamic_array_create(size_t element_size, destructor_func destructor);
void dynamic_array_destroy(DynamicArray *array);
DynamicArrayError dynamic_array_push_back(DynamicArray *array, const void *element);
DynamicArrayError dynamic_array_pop_back(DynamicArray *array, void *element);
DynamicArrayError dynamic_array_get(const DynamicArray *array, size_t index, void *element);
DynamicArrayError dynamic_array_set(DynamicArray *array, size_t index, const void *element);
DynamicArrayError dynamic_array_insert(DynamicArray *array, size_t index, const void *elemen
DynamicArrayError dynamic_array_remove(DynamicArray *array, size_t index);
size_t dynamic_array_size(const DynamicArray *array);
size_t dynamic_array_capacity(const DynamicArray *array);
DynamicArrayError dynamic_array_resize(DynamicArray *array, size_t new_size);
DynamicArrayError dynamic_array_clear(DynamicArray *array);
```

```
DynamicArrayError dynamic_array_sort(DynamicArray *array, compare_func compare);

#endif // DYNAMIC_ARRAY_H
```

```
// dynamic_array.c
#include "dynamic_array.h"
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 8
#define GROWTH_FACTOR 2

struct DynamicArray {
    void *data;
    size_t size;
    size_t capacity;
    size_t element_size;
    destructor_func destructor;
};

DynamicArray* dynamic_array_create(size_t element_size, destructor_func destructor) {
    if (element_size == 0) return NULL;

    DynamicArray *array = malloc(sizeof(DynamicArray));
    if (array == NULL) return NULL;

    array->data = malloc(INITIAL_CAPACITY * element_size);
    if (array->data == NULL) {
        free(array);
        return NULL;
    }

    array->size = 0;
    array->capacity = INITIAL_CAPACITY;
    array->element_size = element_size;
    array->destructor = destructor;

    return array;
}

void dynamic_array_destroy(DynamicArray *array) {
    if (array != NULL) {
        dynamic_array_clear(array);
        free(array->data);
        free(array);
    }
```

```
}

DynamicArrayError dynamic_array_push_back(DynamicArray *array, const void *element) {
    if (array == NULL || element == NULL) return DYNARRAY_ERROR_NULL_POINTER;

    if (array->size >= array->capacity) {
        size_t new_capacity = array->capacity * GROWTH_FACTOR;
        void *new_data = realloc(array->data, new_capacity * array->element_size);
        if (new_data == NULL) return DYNARRAY_ERROR_MEMORY;

        array->data = new_data;
        array->capacity = new_capacity;
    }

    void *dest = (char*)array->data + array->size * array->element_size;
    memcpy(dest, element, array->element_size);
    array->size++;

    return DYNARRAY_SUCCESS;
}

DynamicArrayError dynamic_array_pop_back(DynamicArray *array, void *element) {
    if (array == NULL) return DYNARRAY_ERROR_NULL_POINTER;
    if (array->size == 0) return DYNARRAY_ERROR_OUT_OF_BOUNDS;

    if (element != NULL) {
        void *src = (char*)array->data + (array->size - 1) * array->element_size;
        memcpy(element, src, array->element_size);
    }

    array->size--;
    return DYNARRAY_SUCCESS;
}

DynamicArrayError dynamic_array_get(const DynamicArray *array, size_t index, void *element)
    if (array == NULL || element == NULL) return DYNARRAY_ERROR_NULL_POINTER;
    if (index >= array->size) return DYNARRAY_ERROR_OUT_OF_BOUNDS;

    void *src = (char*)array->data + index * array->element_size;
    memcpy(element, src, array->element_size);

    return DYNARRAY_SUCCESS;
}

DynamicArrayError dynamic_array_set(DynamicArray *array, size_t index, const void *element)
    if (array == NULL || element == NULL) return DYNARRAY_ERROR_NULL_POINTER;
```

```
    if (index >= array->size) return DYNARRAY_ERROR_OUT_OF_BOUNDS;

    void *dest = (char*)array->data + index * array->element_size;
    memcpy(dest, element, array->element_size);

    return DYNARRAY_SUCCESS;
}

size_t dynamic_array_size(const DynamicArray *array) {
    return (array != NULL) ? array->size : 0;
}

size_t dynamic_array_capacity(const DynamicArray *array) {
    return (array != NULL) ? array->capacity : 0;
}

DynamicArrayError dynamic_array_clear(DynamicArray *array) {
    if (array == NULL) return DYNARRAY_ERROR_NULL_POINTER;

    if (array->destructor != NULL) {
        for (size_t i = 0; i < array->size; i++) {
            void *element = (char*)array->data + i * array->element_size;
            array->destructor(element);
        }
    }

    array->size = 0;
    return DYNARRAY_SUCCESS;
}

DynamicArrayError dynamic_array_sort(DynamicArray *array, compare_func compare) {
    if (array == NULL || compare == NULL) return DYNARRAY_ERROR_NULL_POINTER;

    // Simple bubble sort implementation
    for (size_t i = 0; i < array->size - 1; i++) {
        for (size_t j = 0; j < array->size - i - 1; j++) {
            void *elem1 = (char*)array->data + j * array->element_size;
            void *elem2 = (char*)array->data + (j + 1) * array->element_size;

            if (compare(elem1, elem2) > 0) {
                // Swap elements
                char *temp = malloc(array->element_size);
                if (temp == NULL) return DYNARRAY_ERROR_MEMORY;

                memcpy(temp, elem1, array->element_size);
                memcpy(elem1, elem2, array->element_size);
```

```
                memcpy(elem2, temp, array->element_size);

                free(temp);
            }
        }
    }

    return DYNARRAY_SUCCESS;
}
```

```
// Example usage
#include <stdio.h>
#include <stdlib.h>
#include "dynamic_array.h"

// Integer destructor
void int_destructor(void *data) {
    // No special cleanup needed for integers
}

// Integer compare function
int compare_ints(const void *a, const void *b) {
    int int_a = *(const int*)a;
    int int_b = *(const int*)b;
    return (int_a > int_b) - (int_a < int_b);
}

int main() {
    DynamicArray *array = dynamic_array_create(sizeof(int), int_destructor);
    if (array == NULL) {
        printf("Failed to create dynamic array\n");
        return 1;
    }

    // Add elements
    for (int i = 10; i >= 1; i--) {
        DynamicArrayError err = dynamic_array_push_back(array, &i);
        if (err != DYNARRAY_SUCCESS) {
            printf("Error adding element: %d\n", err);
            break;
        }
    }

    printf("Array size: %zu, capacity: %zu\n",
           dynamic_array_size(array), dynamic_array_capacity(array));
```

```
    // Print elements
    printf("Before sorting: ");
    for (size_t i = 0; i < dynamic_array_size(array); i++) {
        int value;
        if (dynamic_array_get(array, i, &value) == DYNARRAY_SUCCESS) {
            printf("%d ", value);
        }
    }
    printf("\n");

    // Sort array
    DynamicArrayError err = dynamic_array_sort(array, compare_ints);
    if (err == DYNARRAY_SUCCESS) {
        printf("After sorting: ");
        for (size_t i = 0; i < dynamic_array_size(array); i++) {
            int value;
            if (dynamic_array_get(array, i, &value) == DYNARRAY_SUCCESS) {
                printf("%d ", value);
            }
        }
        printf("\n");
    }

    dynamic_array_destroy(array);
    return 0;
}
```

Summary

Abstract Data Types in C provide a powerful mechanism for creating modular, maintainable, and reusable code:

1. **Information Hiding**: Separate interface from implementation using opaque pointers
2. **Opaque Pointers**: Hide structure details by only declaring types in headers
3. **Constructor/Destructor Patterns**: Ensure proper resource management
4. **Generic ADTs**: Use void pointers and function pointers for type flexibility
5. **Clean Interface Design**: Provide clear, consistent APIs with proper error handling
6. **Memory Management**: Implement proper allocation and deallocation strategies

These concepts enable C programmers to write code that is as maintainable and reusable as code written in object-oriented languages, while maintaining C's performance characteristics and low-level control.

Bit-Field Traps: Signedness, Promotion, Layout and Shared Words

Bit-Field Traps: Signedness, Promotion, Layout and Shared Words

Thesis

A bit-field looks like an ordinary struct member that happens to be narrow. It isn't. The C standard gives it its own rules, and leaves most of them to the implementation:

- Whether a **plain `int`** bit-field is signed or unsigned is implementation-defined (C17 §6.7.2 ¶5). C23 keeps that rule. A signed 1-bit field can only hold **0 and −1**, so `flag = 1` stores −1 and `flag == 1` is always false.
- In expressions a bit-field is promoted like a small integer. Whether `unsigned x : N` behaves as `int` or as `unsigned int` depends on **N** (C17 §6.3.1.1 ¶2).
- The order of bit-fields inside their storage unit, whether a field may straddle units, and the alignment of the unit are all implementation-defined or unspecified (C17 §6.7.2.1 ¶11). A bit-field struct is not a description of bytes on a wire.
- Adjacent bit-fields share one **memory location** (C17 §3.14). Two threads that write two *different* fields are in a data race.

The earlier structures chapter shows how to declare bit-fields and pack flags. This chapter runs each trap on GCC, shows the warning that would have caught it (when there is one), and ends with versions of the same code that every compiler reads the same way.

Mental model

Question	What C says	What GCC on x86-64 does
Is `int f : 1` signed?	implementation-defined	signed (`-funsigned-bitfields` changes it)
Type of `unsigned f : N` in `f - 1`	`int` if every value fits in `int`, otherwise `unsigned int`	`int` for N 31, `unsigned int` for N = 32
Which bits does the first field use?	implementation-defined	the least significant bits of the unit (SysV ABI)
May a field straddle a unit?	implementation-defined	no for `unsigned`: `a : 20, b : 20` uses 8 bytes
Base types other than `_Bool`, `int`, `signed`/`unsigned int`	implementation-defined	accepted (`uint8_t f : 4` makes a 1-byte struct)

Question	What C says	What GCC on x86-64 does
Can two threads write two adjacent fields?	no: one memory location, so a data race	each write is a read-modify-write of shared bytes

Worked examples

Tested on Debian 13 x86_64 with **GCC 14.2.0** (`gcc (Debian 14.2.0-19)`) and glibc 2.41. The current upstream release is GCC 16.2. Every rule shown here is either fixed by the standard or by the x86-64 SysV ABI, and documented for GCC under `-fsigned-bitfields`, so the newer compiler gives the same answers. Clang was not available on the box.

Case 1: A 1-bit `int` flag that can't equal 1

```c
/* one_bit.c - a 1-bit "flag" declared with plain int */
#include <stdio.h>

struct opts {
    int verbose : 1;        /* plain int: signedness is implementation-defined */
    int debug   : 1;
};

int main(void)
{
    struct opts o = { 0 };
    o.verbose = 1;
    printf("o.verbose = %d\n", o.verbose);
    if (o.verbose == 1)
        puts("verbose mode on");
    else
        puts("verbose mode OFF (o.verbose != 1)");
    return 0;
}
```

```
$ gcc -std=c17 -Wall -O2 -o one_bit one_bit.c && ./one_bit
o.verbose = -1
verbose mode OFF (o.verbose != 1)
$ gcc -std=c17 -Wall -O2 -funsigned-bitfields -o one_bit_u one_bit.c && ./one_bit_u
o.verbose = 1
verbose mode on
```

The same source prints opposite answers depending on one implementation-defined choice. GCC's default, documented under `-fsigned-bitfields`, is that a plain `int` bit-field is signed. In two's

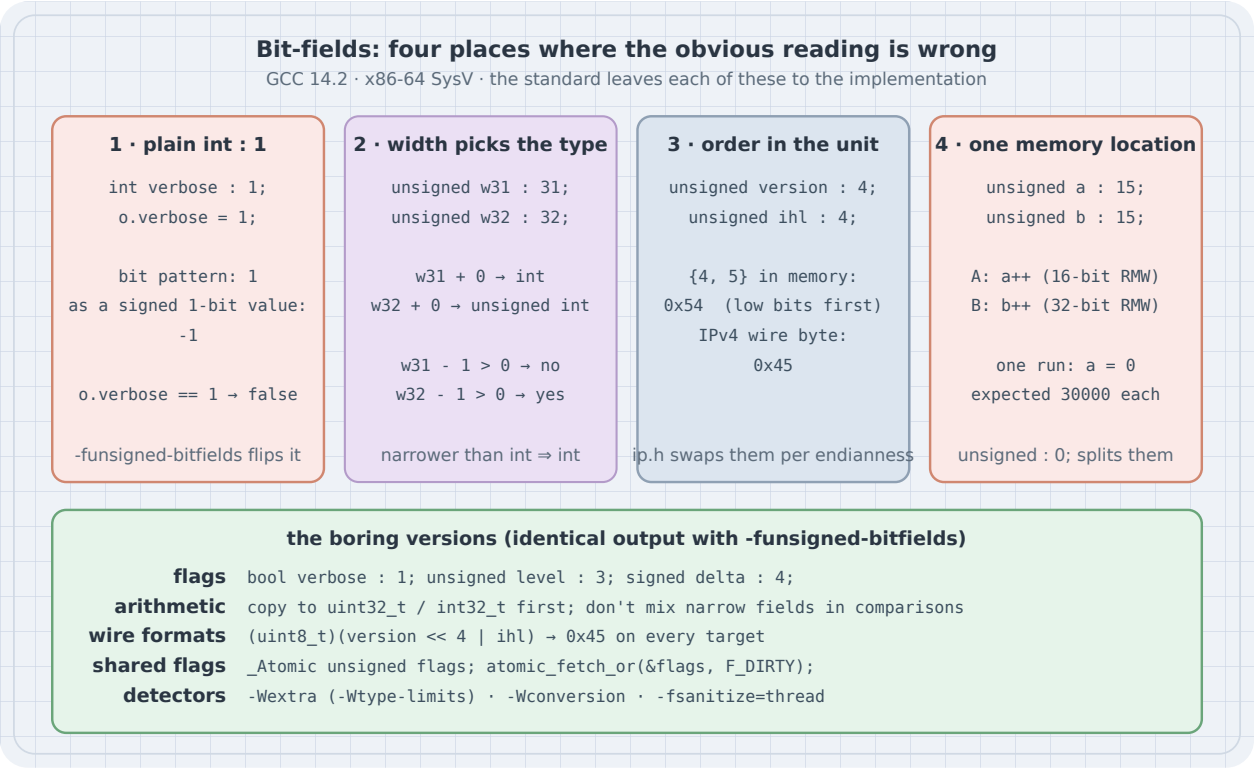


Figure 1: Four bit-field traps: plain int : 1 holds -1; narrower-than-int fields promote to int; allocation order gives 0x54 not 0x45; adjacent fields share a word and lose updates under threads; boring alternatives below

complement a 1-bit signed value has exactly two patterns: `0` is 0 and `1` is −1. `-Wall` alone printed nothing.

Flags are rarely set from constants, so here is the same field set from a runtime value:

```
/* one_bit_rt.c - the same flag, set from a runtime value and tested with != 0 vs == 1 */
#include <stdio.h>
#include <string.h>

struct opts {
    int verbose : 1;
};

static int want_verbose(int argc, char **argv)
{
    return argc > 1 && strcmp(argv[1], "-v") == 0;      /* 1 or 0 */
}

int main(int argc, char **argv)
{
    struct opts o = { 0 };
    o.verbose = want_verbose(argc, argv);
    int copy = o.verbose;                                   /* widen to int */
    printf("o.verbose = %d, copy = %d\n", o.verbose, copy);
    printf("o.verbose == 1 ? %s\n", o.verbose == 1 ? "yes" : "no");
    printf("o.verbose != 0 ? %s\n", o.verbose != 0 ? "yes" : "no");
    printf("o.verbose * 10 = %d\n", o.verbose * 10);
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o one_bit_rt one_bit_rt.c
one_bit_rt.c: In function ‘main’:
one_bit_rt.c:20:47: warning: comparison is always false due to limited range of data type [-
   20 |     printf("o.verbose == 1 ? %s\n", o.verbose == 1 ? "yes" : "no");
      |                                              ^~
$ ./one_bit_rt -v
o.verbose = -1, copy = -1
o.verbose == 1 ? no
o.verbose != 0 ? yes
o.verbose * 10 = -10
$ gcc -std=c17 -Wall -Wextra -Wconversion -O2 -c one_bit_rt.c -o /dev/null
one_bit_rt.c: In function ‘main’:
one_bit_rt.c:17:17: warning: conversion from ‘int’ to ‘signed char:1’ may change value [-Wco
   17 |     o.verbose = want_verbose(argc, argv);
      |                 ^~~~~~~~~~~~
one_bit_rt.c:20:47: warning: comparison is always false due to limited range of data type [-
   20 |     printf("o.verbose == 1 ? %s\n", o.verbose == 1 ? "yes" : "no");
```

```
      |                                                     ^~
```

Testing with `!= 0` (or plain `if (o.verbose)`) happens to work, so code that is only ever tested that way looks fine. Anything that uses the value as a *number* gets −1: copying it into an `int`, multiplying it, adding it to a counter, or comparing it with `1` or `true`. `-Wtype-limits` (in `-Wextra`) catches the comparison, and `-Wconversion` catches the store. GCC's own name for the type, `signed char:1`, tells you what it decided.

Case 2: The width of the field picks its type

```c
/* promote.c - a bit-field's type in expressions depends on its width */
#include <stdio.h>

#define TYPE(x) _Generic((x), int: "int", unsigned: "unsigned int", \
                         long: "long", unsigned long: "unsigned long", default: "other")

struct counters {
    unsigned w16 : 16;
    unsigned w31 : 31;
    unsigned w32 : 32;
};

int main(void)
{
    struct counters c = { 0, 0, 0 };
    printf("type of c.w16 + 0: %s\n", TYPE(c.w16 + 0));
    printf("type of c.w31 + 0: %s\n", TYPE(c.w31 + 0));
    printf("type of c.w32 + 0: %s\n", TYPE(c.w32 + 0));
    printf("c.w31 - 1 > 0 ? %s\n", c.w31 - 1 > 0 ? "yes" : "no");
    printf("c.w32 - 1 > 0 ? %s\n", c.w32 - 1 > 0 ? "yes" : "no");
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o promote promote.c && ./promote
type of c.w16 + 0: int
type of c.w31 + 0: int
type of c.w32 + 0: unsigned int
c.w31 - 1 > 0 ? no
c.w32 - 1 > 0 ? yes
```

The integer promotions (C17 §6.3.1.1 ¶2) apply to "a bit-field of type `_Bool`, `int`, `signed int`, or `unsigned int`": "If an `int` can represent all values of the original type (as restricted by the width, for a bit-field), the value is converted to an `int`." Every value of a 31-bit unsigned field fits in `int`, so `c.w31 - 1` is the `int` −1. A 32-bit field doesn't fit, so `c.w32 - 1` wraps to `UINT_MAX`. Two members that both say `unsigned` behave differently in the same expression. Changing a field's width in a later refactor can silently flip every comparison that uses it. The portable habit is to

copy the field into a variable of the type you mean (`uint32_t v = c.w31;`) before doing arithmetic on it.

Case 3: A bit-field struct is not a byte layout

```c
/* layout.c - what the bytes of a bit-field struct actually look like here */
#include <stdint.h>
#include <stdio.h>
#include <string.h>

struct hdr_u   { unsigned version : 4, ihl : 4; };          /* standard base type     */
struct hdr_u8  { uint8_t  version : 4, ihl : 4; };          /* GCC extension          */
struct straddle{ unsigned a : 20, b : 20; };                /* 40 bits: crosses 32    */
struct split   { unsigned a : 4; unsigned : 0; unsigned b : 4; };

static void dump(const char *name, const void *p, size_t n)
{
    const unsigned char *b = p;
    printf("%-16s sizeof %zu  bytes:", name, n);
    for (size_t i = 0; i < n; i++)
        printf(" %02x", b[i]);
    putchar('\n');
}

int main(void)
{
    struct hdr_u h;  struct hdr_u8 h8;  struct straddle s;  struct split sp;
    memset(&h, 0, sizeof h);   memset(&h8, 0, sizeof h8);    /* padding bits are */
    memset(&s, 0, sizeof s);   memset(&sp, 0, sizeof sp);    /* otherwise unspecified */
    h.version = 4;   h.ihl = 5;
    h8.version = 4;  h8.ihl = 5;
    s.a = 0xFFFFF;   s.b = 0x12345;
    sp.a = 0xA;      sp.b = 0xB;
    dump("hdr_u {4,5}", &h, sizeof h);
    dump("hdr_u8 {4,5}", &h8, sizeof h8);
    dump("straddle", &s, sizeof s);
    dump("split", &sp, sizeof sp);
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o layout layout.c && ./layout
hdr_u {4,5}      sizeof 4  bytes: 54 00 00 00
hdr_u8 {4,5}     sizeof 1  bytes: 54
straddle         sizeof 8  bytes: ff ff 0f 00 45 23 01 00
split            sizeof 8  bytes: 0a 00 00 00 0b 00 00 00
```

Four implementation-defined choices show up in four lines:

- **Order.** The first declared field (`version`) took the *low* nibble, giving `0x54`. The first byte of an IPv4 header puts version in the *high* nibble: `0x45`. The x86-64 SysV ABI allocates bit-fields from the least significant bit. Big-endian ABIs allocate from the most significant bit, where the same struct would produce `0x45`.
- **Unit size.** With `unsigned` as the base type, the struct is 4 bytes, the size of the declared type. With `uint8_t`, which GCC accepts as an extension, it is 1 byte. Both are legal, and neither is guaranteed by C.
- **Straddling.** `a : 20, b : 20` would fit in 5 bytes. GCC starts `b` in a new 32-bit unit rather than let it cross a 4-byte boundary, so `sizeof` is 8.
- **Zero width.** `unsigned : 0` forces the next field into a new unit, as the standard specifies.

glibc's own header shows the professional response. It refuses to rely on any one order:

```
$ grep -n -A10 '^struct iphdr' /usr/include/netinet/ip.h
44:struct iphdr
45-  {
46-#if __BYTE_ORDER == __LITTLE_ENDIAN
47-    unsigned int ihl:4;
48-    unsigned int version:4;
49-#elif __BYTE_ORDER == __BIG_ENDIAN
50-    unsigned int version:4;
51-    unsigned int ihl:4;
52-#else
53-# error  "Please fix <bits/endian.h>"
54-#endif
```

That works only because the header knows both of the orders that its supported ABIs use. For your own wire formats and file formats, write the bytes yourself with shifts and masks (Case 5).

Case 4: Two threads, two fields, one memory location

C11 defines a memory location as "either an object of scalar type, or a maximal sequence of adjacent bit-fields all having nonzero width" (C17 §3.14). Two threads may write two different memory locations without synchronisation. They may not write the same one. Adjacent bit-fields **are** the same one:

```c
/* race.c - two threads, two different bit-fields, one memory location */
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>

enum { N = 30000 };                 /* fits in 15 bits */

struct shared {
    unsigned a : 15;                /* written only by thread A */
```

```
#ifdef SPLIT
    unsigned   : 0;                /* zero-width bit-field: a new memory location */
#endif
    unsigned b : 15;               /* written only by thread B */
} s;

static pthread_barrier_t go;

/* An empty asm with a "memory" clobber keeps GCC from folding the loop into a
 * single "+= N": every iteration must really load, modify and store. */
#define KEEP_LOOP() __asm__ __volatile__("" ::: "memory")

static void *bump_a(void *arg) { (void)arg; pthread_barrier_wait(&go); for (int i = 0; i < N
static void *bump_b(void *arg) { (void)arg; pthread_barrier_wait(&go); for (int i = 0; i < N

int main(void)
{
    pthread_t ta, tb;
    pthread_barrier_init(&go, NULL, 2);
    pthread_create(&ta, NULL, bump_a, NULL);
    pthread_create(&tb, NULL, bump_b, NULL);
    pthread_join(ta, NULL);
    pthread_join(tb, NULL);
    printf("sizeof %zu  a = %5u  b = %5u  (expected %d each)%s\n", sizeof s,
           (unsigned)s.a, (unsigned)s.b, N, s.a == N && s.b == N ? "" : "  <- lost updates")
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -pthread -o race race.c && for i in 1 2 3 4 5; do ./race; d
sizeof 4  a =     0  b = 30000  (expected 30000 each)  <- lost updates
sizeof 4  a =     0  b = 30000  (expected 30000 each)  <- lost updates
sizeof 4  a =   159  b = 30000  (expected 30000 each)  <- lost updates
sizeof 4  a = 20065  b = 29995  (expected 30000 each)  <- lost updates
sizeof 4  a = 20372  b = 30059  (expected 30000 each)  <- lost updates
$ gcc -std=c17 -Wall -Wextra -O2 -pthread -DSPLIT -o race_split race.c && for i in 1 2 3 4 5
sizeof 8  a = 30000  b = 30000  (expected 30000 each)
sizeof 8  a = 30000  b = 30000  (expected 30000 each)
sizeof 8  a = 30000  b = 30000  (expected 30000 each)
sizeof 8  a = 30000  b = 30000  (expected 30000 each)
sizeof 8  a = 30000  b = 30000  (expected 30000 each)
```

Each thread touches only its own field, yet thread A's updates are mostly or entirely lost. The stores show why:

```
$ gcc -std=c17 -O2 -S -o - race.c | grep -E '^bump_[ab]:|movw|movl  %edx, s'
bump_b:
```

```
    movl    %edx, s(%rip)
bump_a:
    movw    %ax, (%rsi)
```

`s.b++` is compiled as "load 32 bits, change bits 15–29, store 32 bits". Every one of thread B's stores writes back whatever value of `a` it loaded, which is usually stale. `s.a++` is a 16-bit store, and bits 0–15 include **bit 15, the lowest bit of `b`**. So thread A can corrupt `b` as well, which is what the last two runs show (`b = 29995` and `b = 30059`, from a counter that only ever counts to 30000). The exact numbers change on every run, and a run can even come out right. ThreadSanitizer reports the race either way:

```
$ gcc -std=c17 -O1 -g -pthread -fsanitize=thread -o race_tsan race.c
$ ./race_tsan 2>&1 | grep -E 'WARNING|Write of size|Previous write|SUMMARY|^sizeof'
WARNING: ThreadSanitizer: data race (pid=249490)
  Previous write of size 4 at 0x555d0e6150a0 by thread T2:
SUMMARY: ThreadSanitizer: data race race.c:22 in bump_a
WARNING: ThreadSanitizer: data race (pid=249490)
  Write of size 4 at 0x555d0e6150a0 by thread T1:
  Previous write of size 4 at 0x555d0e6150a0 by thread T2:
SUMMARY: ThreadSanitizer: data race race.c:22 in bump_a
sizeof 4  a = 30000  b = 30000  (expected 30000 each)
```

In that TSan run the counts happened to come out right, but TSan still reported the race. The `-DSPLIT` build is clean, because a zero-width bit-field ends the memory location (C17 §3.14 note 2). There GCC uses two independent 16-bit stores:

```
$ gcc -std=c17 -O2 -DSPLIT -S -o - race.c | grep -E '^bump_[ab]:|movw|movl  %edx, s'
bump_b:
    movw    %ax, (%rsi)
bump_a:
    movw    %ax, (%rsi)
```

A separate non-bit-field member or a nested struct also splits the location. The `KEEP_LOOP()` barrier exists only to stop GCC from turning each loop into a single `+= 30000`, which would make the race window too small to see. It doesn't create the race. The race is in the struct definition.

Case 5: The boring versions

```
/* fixed.c - the boring versions: explicit signedness, shifts for wire formats, atomics for
#include <stdatomic.h>
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

struct opts {
    bool     verbose : 1;    /* _Bool bit-field: holds 0 or 1, converts like bool */
```

```
    unsigned level   : 3;    /* explicitly unsigned: 0..7 on every compiler */
    signed   delta   : 4;    /* explicitly signed: -8..7 on every compiler */
};

/* Wire formats: compute the byte, never memcpy a bit-field struct. */
static inline uint8_t ipv4_vihl(unsigned version, unsigned ihl)
{
    return (uint8_t)((version & 0xFu) << 4 | (ihl & 0xFu));
}

/* Flags shared between threads: one atomic word, masks instead of bit-fields. */
enum { F_DIRTY = 1u << 0, F_CLOSED = 1u << 1 };
static _Atomic unsigned flags;

int main(void)
{
    struct opts o = { 0 };
    o.verbose = 1;
    o.level = 5;
    o.delta = -3;
    printf("verbose = %d, level = %u, delta = %d\n", o.verbose, (unsigned)o.level, o.delta);
    printf("o.verbose == 1 ? %s\n", o.verbose == 1 ? "yes" : "no");
    printf("ipv4_vihl(4, 5) = 0x%02x\n", ipv4_vihl(4, 5));
    atomic_fetch_or(&flags, F_DIRTY);
    atomic_fetch_or(&flags, F_CLOSED);
    printf("flags = 0x%x\n", atomic_load(&flags));
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -Wconversion -O2 -o fixed fixed.c && ./fixed
verbose = 1, level = 5, delta = -3
o.verbose == 1 ? yes
ipv4_vihl(4, 5) = 0x45
flags = 0x3
$ gcc -std=c17 -Wall -Wextra -Wconversion -O2 -funsigned-bitfields -o fixed_u fixed.c && ./f
verbose = 1, level = 5, delta = -3
o.verbose == 1 ? yes
ipv4_vihl(4, 5) = 0x45
flags = 0x3
```

The build is clean even with `-Wconversion`, and flipping the implementation-defined default changes nothing:

- `bool verbose : 1` converts like `bool`: any nonzero value stores 1, and it reads back as 1.
- `unsigned` and `signed` are spelled out, so the range doesn't depend on the compiler.
- The IPv4 byte is computed, so it is `0x45` on every target, whatever the endianness or ABI.
- Flags shared between threads live in one `_Atomic unsigned` and change through

`atomic_fetch_or`. Two threads setting different flags can't lose each other's bits.

The trap

The trap is reading a bit-field declaration as if it were a portable specification. `int verbose : 1` reads as "one bit that means on", and on GCC it means "0 or −1". `unsigned seq : 31` reads as "an unsigned counter", and in arithmetic it is an `int`. `unsigned version : 4, ihl : 4` reads as "a header byte", and the bytes depend on the ABI. `a : 15; b : 15` reads as two independent fields, and to the memory model they are one location. None of this crashes. Each one passes tests on the platform where the code was written. The flag works with `if (flag)`, the counter works until it hits 0, the header round-trips through the same struct, and the race needs two threads at the same moment.

Packing with `#pragma pack` or `__attribute__((packed))` does not help. It changes padding, not signedness, promotion, order or memory locations.

The boring rule

> Write `bool`, `unsigned` or `signed` for every bit-field, never plain `int`. Copy a field into a fixed-width variable before you do arithmetic or comparisons with it. Never `memcpy`, cast or `fwrite` a bit-field struct to get bytes for a wire format or a file format. Build the bytes with shifts and masks. Never let two threads write fields of the same bit-field run. Put shared flags in one atomic word, or separate the fields with `unsigned : 0` or a non-bit-field member. Build with `-Wextra -Wconversion`, and run threaded tests under `-fsanitize=thread`.

Try this

1. Change `int verbose : 1` to `int verbose : 2` in `one_bit.c`. Does `== 1` work now? What values can the field hold, and which ones would still surprise you?
2. In `promote.c`, add `unsigned long long w40 : 40;` (a GCC extension). Predict the type of `c.w40 + 0` before you print it, and explain the answer with §6.3.1.1.
3. Rewrite `layout.c`'s `hdr_u` as `struct { unsigned ihl : 4, version : 4; }` and confirm you get `0x45`. Then explain why this "fix" is worse than `ipv4_vihl()`.
4. In `race.c`, make the fields `unsigned a : 16; unsigned b : 16;` and look at the stores with `-S`. Does the race still happen at `-O2`? Does TSan still report it, and is the program still racy according to the standard?
5. Find a bit-field struct in a codebase you use. Is it ever written to a file or socket, or written from two threads? Which of this chapter's rules does it break?

Sources

- ISO/IEC 9899:2018 (C17): §3.14 *memory location* (and note 2 on zero-width bit-fields); §6.3.1.1 ¶2 (integer promotions of bit-fields); §6.7.2 ¶5 (plain `int` bit-field signedness is implementation-defined); §6.7.2.1 ¶¶4–12 (bit-field types, allocation, straddling, order, zero width). The C17 final draft N2176 is a convenient copy of this text.
- cppreference, *Bit-fields* (C, including the C23 additions such as `_BitInt` bit-fields and the list of implementation-defined properties): https://en.cppreference.com/w/c/language/bit_field
- WG14 N2958, J. Myers, "Bit-fields, expressions and types" (C23-era analysis of bit-field types in expressions): https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2958.htm
- GCC manual, C Dialect Options: `-fsigned-bitfields`, `-funsigned-bitfields`: https://gcc.gnu.org/onlinedocs/gcc/C-Dialect-Options.html
- GCC manual, Warning Options: `-Wtype-limits`, `-Wconversion`, `-Woverflow`: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
- GCC manual, Structures, Unions, Enumerations and Bit-Fields implementation (implementation-defined behaviour for C): https://gcc.gnu.org/onlinedocs/gcc/Structures-unions-enumerations-and-bit-fields-implementation.html
- System V Application Binary Interface, AMD64 Architecture Processor Supplement, §3.1.2 "Bit-Fields": https://gitlab.com/x86-psABIs/x86-64-ABI
- SEI CERT C, INT12-C, "Do not make assumptions about the type of a plain int bit-field when used in an expression": https://wiki.sei.cmu.edu/confluence/display/c/INT12-C.+Do+not+make+assumptions+about+the+type+of+a+plain+int+bit-field+when+used+in+an+expression
- glibc `<netinet/ip.h>` (`struct iphdr`, endian-dependent field order)
- GCC releases (GCC 16.2, 2026-08-07): https://gcc.gnu.org/

Opaque Handles and Library APIs Other Languages Can Bind

Opaque Handles and Library APIs Other Languages Can Bind

After reading this chapter, you will be able to design, compile, and distribute C shared libraries that expose a clean, stable Application Binary Interface (ABI) easily consumed by any language with a Foreign Function Interface (FFI). You will know how to hide internal struct layouts behind opaque handles, return structured error codes instead of relying on global state, support caller-allocated memory through two-pass size queries, evolve configuration structs across releases without breaking binary compatibility, enforce strict symbol visibility at the dynamic linker level, and bind the resulting shared object from high-level runtimes using standard tools like Python's `ctypes`.

The ABI Contract: Why Naive C APIs Break Other Languages

C is the universal lingua franca of systems programming. Runtimes as varied as Python, Rust, Go, Zig, Java, and C# can invoke C functions directly through their respective FFI layers. However, an API that feels convenient when written for another C translation unit in the same repository often becomes an unmaintainable hazard across an ABI boundary.

When a header exposes concrete struct definitions, callers compile hard-coded byte offsets and struct sizes into their binaries. If a library author adds a diagnostic counter or reorders members in a minor patch, every external caller that allocated the struct on the stack or accessed a field will read corrupted memory or smash the stack. Similarly, returning raw pointers to library-allocated memory forces callers to guess which deallocator to call—a recipe for heap corruption when the host runtime and the C library link against different allocators or different versions of the C runtime library.

A production-grade C library API must follow strict constraints:

1. **Information hiding**: Data structures are declared as incomplete types in public headers and defined only inside private translation units.
2. **Explicit lifecycle**: Every resource allocated by the library must have a corresponding release function provided by the same library.
3. **Structured return values**: Functions communicate failure via predictable status integer enums and return output data through caller-provided pointer arguments.
4. **Caller-allocated memory**: Functions producing variable-length data allow the caller to query the required buffer size first, keeping memory ownership entirely in the caller's hands.
5. **No thread-local or process-wide side effects**: Configuration and state belong to explicit context handles, and callbacks accept `void *user` state pointers to prevent reliance on global variables.
6. **Controlled symbol export**: Internal helpers and private functions must never leak into the dynamic symbol table (`.dynsym`).

Anti-Patterns and Reliable Solutions

The following table contrasts common internal C idioms against their ABI-safe counterparts.

Fragile Idiom / Anti-Pattern	Operational Risk Across FFI	Robust Design Solution
Exposing concrete `struct` layout in the public header	Altering or adding fields changes struct size and member offsets, silently corrupting external callers.	Declare an opaque handle (`typedef struct tally tally;`) in the header; define the struct solely in the `.c` file.
Returning pointers to internal library buffers	Callers can mutate internal state or read stale data after subsequent calls; ownership of freeing memory is ambiguous.	Caller-owned buffers: caller supplies the buffer and capacity. Support a two-pass size query with a `NULL` buffer.
Relying on `errno` or static error strings	Not thread-safe without runtime thread-local storage support; high-level language FFI bridges often struggle to reliably capture global variables.	Every function returns an explicit integer status code (`TALLY_OK`, `TALLY_EINVAL`, etc.); outputs are returned through pointer parameters.
Aborting (`assert()`, `exit()`) on invalid arguments	A single bad pointer kills the host runtime (e.g., terminating an entire Python or Java process server).	Defensive argument validation: check all pointers and ranges, returning `TALLY_EINVAL` on error without crashing.
Functions taking naked callback function pointers	Callers cannot pass instance state or closures without resorting to global variables, breaking reentrancy and concurrency.	Accompany every callback function pointer with a `void *user` context pointer passed back on every invocation.
Exporting all functions by default	Leaks implementation details, increases dynamic symbol table bloat, slows load times, and invites naming collisions.	Build with `-fvisibility=hidden` and tag only public entry points with `__attribute__((visibility("default")))`.
Unversioned configuration structs	Adding fields to configuration structs breaks ABI compatibility with binaries compiled against earlier headers.	Place a `size_t size` member at the start of options structs so the library can inspect how much the caller knows.

Designing the Tally Library

To demonstrate these principles in practice, we will build a small, robust tally library named `libtally`. The library counts distinct words, supports case-sensitive and case-insensitive matching, provides a two-pass query for the most frequent word, and allows full iteration across collected counts via a closure-style callback.

Opaque Handles and Lifecycle Management

The public header exposes the handle as an incomplete type:

```
typedef struct tally tally;
```

Because the compiler does not know `sizeof(struct tally)` when compiling client code, clients cannot allocate `tally` on the stack (`tally t;` fails to compile) or directly access its fields (`t->count` fails to compile). The library controls allocation through `tally_new` and destruction through `tally_free`.

Following standard POSIX and ISO C conventions, `tally_free` safely treats a `NULL` pointer as a no-op, mirroring the behavior of `free(NULL)`:

```
void tally_free(tally *t)
{
    if (t == NULL)
        return;
    /* Clean up resources */
}
```

Return Codes and Out-Parameters

Functions return an integer status enum indicating success or a specific failure mode:

```
enum {
    TALLY_OK     =  0,
    TALLY_EINVAL = -1,
    TALLY_ENOMEM = -2,
    TALLY_ERANGE = -3
};
```

Actual computation results are delivered via pointer out-parameters. For example, retrieving a count takes the handle, the search key, and an out-parameter for the count:

```
int tally_get(const tally *t, const char *word, size_t *out_count);
```

If the caller passes `NULL` for `out_count`, the function immediately yields `TALLY_EINVAL` rather than attempting a memory dereference.

Caller-Owned Buffers and Two-Pass Size Queries

When an API returns a string or variable-length byte array, who allocates the buffer? If the library allocates the string with `malloc`, the caller must know which deallocator to invoke. In Python or Go, passing that pointer directly into standard language constructs can cause memory leaks or undefined behavior if the host runtime frees it using an incompatible allocator.

The standard solution is the two-pass buffer query, the same idea behind C's `snprintf` (called with a null buffer, it returns the length it would need) and many Windows APIs:

1. **Pass 1 (Query)**: The caller invokes the function with a `NULL` buffer pointer and a buffer length of zero. The function calculates the necessary capacity (including the null terminator), writes the size into an out-parameter `needed_len`, and returns `TALLY_OK`.
2. **Pass 2 (Fill)**: The caller allocates the requested number of bytes using its own preferred memory management mechanism and calls the function again, passing the pointer and the allocated capacity. If the caller provides insufficient capacity, the function returns `TALLY_ERANGE`.

```
int tally_top_word(const tally *t, char *buf, size_t buf_len, size_t *needed_len, size_t *ou
```

Versioned Options Structs

When a library introduces new configuration toggles across minor versions, naive struct signatures force an ABI break. By placing `size_t size` as the very first member of the options struct, the library can safely determine which version of the struct the caller was compiled against:

```
typedef struct tally_options {
    size_t size;        /* Must be sizeof(tally_options) */
    int case_sensitive;
} tally_options;
```

The caller initializes the struct with its current compile-time size:

```
tally_options opts = {
    .size = sizeof(opts),
    .case_sensitive = 1
};
```

The library validates `opts->size`. If the caller passes a size smaller than the base requirements, the library returns `TALLY_EINVAL`. If a future version adds new fields, older binaries passing a smaller `size` will cause the new library to apply sensible default values for the missing fields, preserving forward and backward compatibility.

Symbol Visibility and Dynamic Linker Sonames

By default, GCC exports every non-static function into the shared library's dynamic symbol table (`.dynsym`). In a shared library, this exposes private helper routines, pollutes the public namespace and costs performance. An exported function can be interposed by another library at load time, so calls to it go through the PLT and are harder for the compiler to inline.

To lock down exports, we compile with `-fvisibility=hidden` and annotate public APIs with a visibility macro:

```
#if defined(__GNUC__) && __GNUC__ >= 4
  #define TALLY_API __attribute__((visibility("default")))
#else
  #define TALLY_API
#endif
```

During linkage, setting `-Wl,-soname,libtally.so.1` embeds the library's logical ABI version directly into the ELF header (`DT_SONAME`). When an executable links against `libtally.so`, the dynamic linker (`ld.so`) records a dependency on `libtally.so.1` rather than the unversioned development symlink.

Library Implementation

Below are the complete source files for `libtally`. The implementation is written in ISO C17 and stays under 200 lines while remaining strictly defensive against malformed input.

```
#ifndef TALLY_H
#define TALLY_H

#include <stddef.h>

#if defined(_WIN32) || defined(__CYGWIN__)
  #ifdef TALLY_BUILDING_DLL
    #define TALLY_API __declspec(dllexport)
  #else
    #define TALLY_API __declspec(dllimport)
  #endif
#elif defined(__GNUC__) && __GNUC__ >= 4
  #define TALLY_API __attribute__((visibility("default")))
#else
  #define TALLY_API
#endif

enum {
    TALLY_OK     =  0,
    TALLY_EINVAL = -1,
    TALLY_ENOMEM = -2,
    TALLY_ERANGE = -3
};

typedef struct tally tally;

typedef struct tally_options {
    size_t size;
    int case_sensitive;
```

```
} tally_options;

typedef int (*tally_iter_fn)(const char *word, size_t count, void *user);

TALLY_API int tally_new(const tally_options *opts, tally **out);
TALLY_API void tally_free(tally *t);
TALLY_API int tally_add(tally *t, const char *word);
TALLY_API int tally_get(const tally *t, const char *word, size_t *out_count);
TALLY_API int tally_top_word(const tally *t, char *buf, size_t buf_len, size_t *needed_len,
TALLY_API int tally_iterate(const tally *t, tally_iter_fn callback, void *user);

#endif /* TALLY_H */
```

```
#include "tally.h"
#include <ctype.h>
#include <stdlib.h>
#include <string.h>

struct tally_entry {
    char *word;
    size_t count;
};

struct tally {
    struct tally_entry *entries;
    size_t count;
    size_t capacity;
    int case_sensitive;
};

static int words_equal(const char *a, const char *b, int case_sensitive)
{
    if (case_sensitive)
        return strcmp(a, b) == 0;
    while (*a && *b) {
        if (tolower((unsigned char)*a) != tolower((unsigned char)*b))
            return 0;
        a++;
        b++;
    }
    return *a == *b;
}

int tally_new(const tally_options *opts, tally **out)
{
    if (out == NULL)
```

```
        return TALLY_EINVAL;
    *out = NULL;

    int case_sensitive = 0;
    if (opts != NULL) {
        if (opts->size < sizeof(size_t))
            return TALLY_EINVAL;
        if (opts->size >= sizeof(tally_options))
            case_sensitive = opts->case_sensitive;
    }

    tally *t = calloc(1, sizeof(*t));
    if (t == NULL)
        return TALLY_ENOMEM;

    t->case_sensitive = case_sensitive;
    *out = t;
    return TALLY_OK;
}

void tally_free(tally *t)
{
    if (t == NULL)
        return;
    for (size_t i = 0; i < t->count; i++)
        free(t->entries[i].word);
    free(t->entries);
    free(t);
}

int tally_add(tally *t, const char *word)
{
    if (t == NULL || word == NULL)
        return TALLY_EINVAL;

    for (size_t i = 0; i < t->count; i++) {
        if (words_equal(t->entries[i].word, word, t->case_sensitive)) {
            t->entries[i].count++;
            return TALLY_OK;
        }
    }

    if (t->count == t->capacity) {
        size_t new_cap = (t->capacity == 0) ? 8 : t->capacity * 2;
        struct tally_entry *new_entries = realloc(t->entries, new_cap * sizeof(*new_entries)
        if (new_entries == NULL)
            return TALLY_ENOMEM;
```

```
        t->entries = new_entries;
        t->capacity = new_cap;
    }

    size_t len = strlen(word);
    char *copy = malloc(len + 1);
    if (copy == NULL)
        return TALLY_ENOMEM;
    memcpy(copy, word, len + 1);

    t->entries[t->count].word = copy;
    t->entries[t->count].count = 1;
    t->count++;
    return TALLY_OK;
}

int tally_get(const tally *t, const char *word, size_t *out_count)
{
    if (t == NULL || word == NULL || out_count == NULL)
        return TALLY_EINVAL;

    for (size_t i = 0; i < t->count; i++) {
        if (words_equal(t->entries[i].word, word, t->case_sensitive)) {
            *out_count = t->entries[i].count;
            return TALLY_OK;
        }
    }

    *out_count = 0;
    return TALLY_OK;
}

int tally_top_word(const tally *t, char *buf, size_t buf_len, size_t *needed_len, size_t *ou
{
    if (t == NULL || needed_len == NULL)
        return TALLY_EINVAL;

    if (t->count == 0) {
        *needed_len = 1;
        if (out_count != NULL)
            *out_count = 0;
        if (buf != NULL && buf_len > 0)
            buf[0] = '\0';
        return TALLY_OK;
    }

    size_t best_idx = 0;
```

```
    for (size_t i = 1; i < t->count; i++) {
        if (t->entries[i].count > t->entries[best_idx].count)
            best_idx = i;
    }

    const char *best_word = t->entries[best_idx].word;
    size_t req = strlen(best_word) + 1;
    *needed_len = req;
    if (out_count != NULL)
        *out_count = t->entries[best_idx].count;

    if (buf == NULL)
        return TALLY_OK;

    if (buf_len < req)
        return TALLY_ERANGE;

    memcpy(buf, best_word, req);
    return TALLY_OK;
}

int tally_iterate(const tally *t, tally_iter_fn callback, void *user)
{
    if (t == NULL || callback == NULL)
        return TALLY_EINVAL;

    for (size_t i = 0; i < t->count; i++) {
        int rc = callback(t->entries[i].word, t->entries[i].count, user);
        if (rc != 0)
            break;
    }
    return TALLY_OK;
}
```

Consuming the Library in C

The following demonstration client exercises the entire public API: initializing options, handling bad parameters defensively, querying counts, performing two-pass buffer size queries, and iterating with closures.

```
#include "tally.h"
#include <stdio.h>
#include <stdlib.h>

static int print_entry(const char *word, size_t count, void *user)
```

```
{
    size_t *total_words = (size_t *)user;
    *total_words += count;
    printf("  %s: %zu\n", word, count);
    return 0;
}

int main(void)
{
    tally_options opts = {
        .size = sizeof(opts),
        .case_sensitive = 0
    };

    tally *t = NULL;
    int rc = tally_new(&opts, &t);
    if (rc != TALLY_OK) {
        fprintf(stderr, "Failed to initialize tally: error %d\n", rc);
        return 1;
    }

    /* Verify defensive handling of NULL input */
    int null_check_1 = tally_add(NULL, "orphan");
    int null_check_2 = tally_add(t, NULL);
    printf("NULL check return values: %d, %d\n", null_check_1, null_check_2);

    /* Populate words (case-insensitive test) */
    tally_add(t, "pipeline");
    tally_add(t, "Pipeline");
    tally_add(t, "socket");
    tally_add(t, "socket");
    tally_add(t, "socket");
    tally_add(t, "buffer");

    /* Direct lookups */
    size_t count = 0;
    tally_get(t, "pipeline", &count);
    printf("Count for 'pipeline': %zu\n", count);
    tally_get(t, "socket", &count);
    printf("Count for 'socket': %zu\n", count);

    /* Two-pass buffer query for the most frequent word */
    size_t needed = 0;
    size_t top_count = 0;
    rc = tally_top_word(t, NULL, 0, &needed, &top_count);
    printf("Pass 1 (size query): rc=%d, needed_bytes=%zu\n", rc, needed);
```

```
    /* Verify bounds protection */
    char tiny_buf[2];
    rc = tally_top_word(t, tiny_buf, sizeof(tiny_buf), &needed, &top_count);
    printf("Small buffer check: rc=%d (expected ERANGE=%d)\n", rc, TALLY_ERANGE);

    /* Pass 2: Allocate caller-owned buffer and retrieve */
    char *word_buf = malloc(needed);
    if (word_buf != NULL) {
        rc = tally_top_word(t, word_buf, needed, &needed, &top_count);
        printf("Pass 2 (retrieval): '%s' with count %zu (rc=%d)\n", word_buf, top_count, rc)
        free(word_buf);
    }

    /* Iteration with accumulator passed via void *user */
    size_t aggregate = 0;
    puts("Tallied entries:");
    tally_iterate(t, print_entry, &aggregate);
    printf("Total tokens processed: %zu\n", aggregate);

    /* Cleanup */
    tally_free(t);
    tally_free(NULL); /* Safe no-op */

    return 0;
}
```

To build the shared object, set up the soname symlink, and compile the client executable against the library:

```
gcc -std=c17 -Wall -Wextra -O2 -fPIC -fvisibility=hidden -shared -Wl,-soname,libtally.so.1 -
```

The following command executed the compilation pipeline and ran the demo binary:

```
NULL check return values: -1, -1
Count for 'pipeline': 2
Count for 'socket': 3
Pass 1 (size query): rc=0, needed_bytes=7
Small buffer check: rc=-3 (expected ERANGE=-3)
Pass 2 (retrieval): 'socket' with count 3 (rc=0)
Tallied entries:
  pipeline: 2
  socket: 3
  buffer: 1
Total tokens processed: 6
```

Notice that both `NULL` invocations return `-1` (`TALLY_EINVAL`) without triggering a segmentation fault. The small buffer check returns `-3` (`TALLY_ERANGE`), protecting the caller against memory

corruption. When supplied with the queried buffer size, the library copies the string into memory allocated and managed strictly by the caller.

Next, verify that `-fvisibility=hidden` successfully suppressed internal functions from the exported dynamic symbol table:

```
nm -D --defined-only libtally.so.1
```

The following command inspected the exported symbols of the compiled shared library:

```
00000000000012e0 T tally_add
0000000000001280 T tally_free
0000000000001450 T tally_get
0000000000001630 T tally_iterate
0000000000001210 T tally_new
0000000000001520 T tally_top_word
```

The output shows only the six functions marked `TALLY_API`. The helper `words_equal` is missing for a simpler reason: it is `static`, and static functions never reach the dynamic symbol table. `-fvisibility=hidden` matters for the next helper you write. That's a non-`static` function in one `.c` file that another `.c` file of the library calls. Without the flag, it would be exported and someone would eventually bind to it.

The soname shows up on the consumer's side too. Running `readelf -d demo | grep NEEDED` on the client built above prints:

```
 0x0000000000000001 (NEEDED)             Shared library: [libtally.so.1]
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
```

The client recorded `libtally.so.1`, not the `libtally.so` development symlink it was linked through.

Binding from Python via ctypes

Because the API relies entirely on primitive scalar types, opaque pointers, explicit memory lengths, and standard calling conventions, binding `libtally` from Python requires no third-party generators or native C extensions. Standard library `ctypes` handles the interface directly.

```
import ctypes
import os
import sys

# Locate and load the shared library
so_path = os.path.abspath("libtally.so.1")
try:
    lib = ctypes.CDLL(so_path)
except OSError as err:
```

```
    print(f"Failed to load shared library: {err}", file=sys.stderr)
    sys.exit(1)

# Status constants
TALLY_OK = 0
TALLY_EINVAL = -1
TALLY_ENOMEM = -2
TALLY_ERANGE = -3

# Options struct matching tally_options layout
class TallyOptions(ctypes.Structure):
    _fields_ = [
        ("size", ctypes.c_size_t),
        ("case_sensitive", ctypes.c_int),
    ]

# Callback prototype: int (*)(const char *word, size_t count, void *user)
ITER_CALLBACK = ctypes.CFUNCTYPE(
    ctypes.c_int,
    ctypes.c_char_p,
    ctypes.c_size_t,
    ctypes.c_void_p
)

# Function prototype signatures
lib.tally_new.argtypes = [ctypes.POINTER(TallyOptions), ctypes.POINTER(ctypes.c_void_p)]
lib.tally_new.restype = ctypes.c_int

lib.tally_free.argtypes = [ctypes.c_void_p]
lib.tally_free.restype = None

lib.tally_add.argtypes = [ctypes.c_void_p, ctypes.c_char_p]
lib.tally_add.restype = ctypes.c_int

lib.tally_get.argtypes = [ctypes.c_void_p, ctypes.c_char_p, ctypes.POINTER(ctypes.c_size_t)]
lib.tally_get.restype = ctypes.c_int

lib.tally_top_word.argtypes = [
    ctypes.c_void_p,
    ctypes.c_char_p,
    ctypes.c_size_t,
    ctypes.POINTER(ctypes.c_size_t),
    ctypes.POINTER(ctypes.c_size_t)
]
lib.tally_top_word.restype = ctypes.c_int
```

```
lib.tally_iterate.argtypes = [ctypes.c_void_p, ITER_CALLBACK, ctypes.c_void_p]
lib.tally_iterate.restype = ctypes.c_int

def main():
    # Initialize versioned options struct
    opts = TallyOptions(size=ctypes.sizeof(TallyOptions), case_sensitive=0)
    handle = ctypes.c_void_p()

    rc = lib.tally_new(ctypes.byref(opts), ctypes.byref(handle))
    if rc != TALLY_OK:
        raise RuntimeError(f"tally_new failed with code {rc}")

    # Add sample data
    tokens = [b"stream", b"Stream", b"STREAM", b"buffer", b"kernel", b"kernel"]
    for token in tokens:
        rc = lib.tally_add(handle, token)
        if rc != TALLY_OK:
            raise RuntimeError(f"tally_add failed with code {rc}")

    # Single item lookup
    count_val = ctypes.c_size_t()
    lib.tally_get(handle, b"stream", ctypes.byref(count_val))
    print(f"Lookup 'stream': {count_val.value}")

    # Two-pass size query for top item
    needed = ctypes.c_size_t()
    top_count = ctypes.c_size_t()
    rc = lib.tally_top_word(handle, None, 0, ctypes.byref(needed), ctypes.byref(top_count))
    if rc != TALLY_OK:
        raise RuntimeError(f"Size query failed with code {rc}")

    # Allocate caller buffer in Python ctypes memory
    buf = ctypes.create_string_buffer(needed.value)
    rc = lib.tally_top_word(handle, buf, needed.value, ctypes.byref(needed), ctypes.byref(to
    if rc != TALLY_OK:
        raise RuntimeError(f"Buffer retrieval failed with code {rc}")

    print(f"Top word: {buf.value.decode('utf-8')} (occurrences: {top_count.value})")

    # Callback iteration collected into Python list
    results = []
    def python_collector(word_ptr, count, user_ptr):
        results.append((word_ptr.decode("utf-8"), count))
        return 0

    c_callback = ITER_CALLBACK(python_collector)
```

```
    rc = lib.tally_iterate(handle, c_callback, None)
    if rc != TALLY_OK:
        raise RuntimeError(f"Iteration failed with code {rc}")

    print("Iterated items:", results)

    # Release handle
    lib.tally_free(handle)

if __name__ == "__main__":
    main()
```

Run the binding script:

```
python3 bind.py
```

The following command executed the Python ctypes verification script:

```
Lookup 'stream': 3
Top word: stream (occurrences: 3)
Iterated items: [('stream', 3), ('buffer', 1), ('kernel', 2)]
```

Because `opts.case_sensitive` was set to `0`, all casing variations of `stream` collapse into a single bucket, making it the most frequent word. The Python script never interacted with internal C pointers or guessed struct offsets.

Pitfalls

1. Ownership of Returned Strings Across Language runtimes

Never return a heap-allocated `char *` from a C library that expects the caller to free it using standard `free()`. On platforms like Windows, a shared library may link against a different C Runtime (CRT) dynamic link library (DLL) than the host executable or runtime. Calling `free()` in one CRT on a pointer allocated by another results in immediate heap corruption. In garbage-collected environments like Python, Java, or Go, managing native heap lifecycles outside of the language runtime creates leaks.

Always prefer caller-allocated buffers with two-pass size queries. If the library must allocate memory on behalf of the caller, export an explicit deallocator (such as `tally_string_free(char *str)`) that calls `free()` inside the same DSO.

2. Callbacks Reentering the Library

When an API executes a user-provided callback during traversal (`tally_iterate`), the caller's code runs synchronously on the same thread. If the callback attempts to mutate the instance by

calling `tally_add` or destroys the instance with `tally_free`, the loop inside the library can read reallocated or freed arrays, which is a use-after-free. If a later version adds a lock, it can deadlock.

Document reentrancy invariants clearly: mark traversal functions as non-reentrant or enforce internal state guards (such as an iteration counter) that return an error if a reentrant mutation is attempted during an active callback.

3. Struct Size Versioning Rules

The versioned options pattern (`size_t size` as the first field) requires strict adherence to two rules:

1. **Strict member ordering**: New members must only ever be appended to the end of the struct. Never insert a field between existing fields or reorder fields, as this changes field offsets and corrupts reads by binaries compiled against older headers.
2. **Padding and alignment awareness**: Compilers pad struct members to match architecture alignment requirements. Always check that the provided `opts->size` reaches at least `offsetof(tally_options, field) + sizeof(field)` before reading that field from the caller.

Key Takeaways

- **Opaque handles eliminate ABI coupling**: Defining structs exclusively inside the C implementation hides memory layouts and allows internal data structures to change without breaking external callers.
- **Explicit lifecycles guarantee memory safety**: Pair every constructor (`tally_new`) with a destructor (`tally_free`) that tolerates `NULL` pointers.
- **Status return codes provide unambiguous control flow**: Return error enums from functions and pass outputs through pointers to ensure compatibility with foreign language error handling.
- **Two-pass size queries respect caller ownership**: Allowing callers to query buffer sizes with `NULL` pointers keeps heap allocation decisions firmly within the calling language.
- **Versioned structs support backward compatibility**: Placing a `size_t size` header on configuration structs allows APIs to grow without breaking existing compiled binaries.
- **Stateless callbacks enable clean foreign closures**: Accompanying callback function pointers with `void *user` parameters allows foreign runtimes to associate state without relying on global variables.
- **Hidden symbol visibility produces tidy DSOs**: Compiling with `-fvisibility=hidden` and annotating public interfaces with `__attribute__((visibility("default")))` restricts dynamic exports exclusively to intentional entry points.

Sources

- GCC Wiki: Visibility
- How To Write Shared Libraries (Ulrich Drepper)

- Python ctypes Documentation
- GCC manual: Code Gen Options (`-fvisibility`)

Module 7: Structures and User-Defined Types

Overview

This module explores structures and user-defined types in C, essential for creating complex data representations and abstract data types. You'll learn about structure declaration, initialization, and usage, as well as unions, enumerations, and advanced structure concepts. The module also covers information hiding principles and object-oriented programming techniques in C.

Learning Objectives

By the end of this module, you will be able to: - Declare, define, and use structures effectively - Initialize structures using various methods - Work with nested structures and arrays of structures - Understand structure packing and alignment - Implement unions and enumerations appropriately - Apply bit fields for memory-efficient data representation - Create abstract data types using information hiding principles - Design opaque pointers for encapsulation - Implement constructor/destructor patterns in C - Apply best practices for structure design and usage

Chapters

1. **Structures** - Structure declaration, definition, initialization, and usage
2. **Unions and Enumerations** - Union declaration and usage, enumeration types and values
3. **Advanced Structure Concepts** - Bit fields, structure packing, flexible array members, self-referential structures
4. **Abstract Data Types** - Information hiding principles, opaque pointers, constructor/destructor patterns, interface design
5. **Bit-Field Traps** - Plain `int` signedness, width-dependent promotion, allocation order vs wire formats, adjacent bit-fields as one memory location under threads

Key Concepts Covered

- Structure declaration and member access
- Structure initialization techniques
- Nested structures and arrays of structures
- Pointers to structures and the arrow operator
- Structure packing and memory alignment
- Union declaration and size optimization

- Enumeration types and explicit value assignment
- Bit fields for memory-efficient storage
- Flexible array members (C99)
- Self-referential structures for linked data
- Information hiding and encapsulation
- Opaque pointers for abstraction
- Constructor and destructor patterns
- Interface design in C

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Completion of Module 4: Functions and Modular Programming
- Completion of Module 5: Arrays and Strings
- Completion of Module 6: Pointers and Memory Management
- Understanding of basic programming concepts
- Familiarity with variable declaration and initialization

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Memory debugging tools (Valgrind, AddressSanitizer)
- Understanding of basic programming concepts
- Familiarity with debugging tools

Estimated Time to Complete

- Reading: 10-12 hours
- Exercises: 15-20 hours
- Projects: 8-10 hours

Assessment

- Complete all chapter exercises
- Successfully implement complex data structures using structures
- Demonstrate understanding of unions, enumerations, and bit fields
- Pass structure programming quiz
- Submit a comprehensive program using abstract data types

Next Module

After completing this module, proceed to **Module 8: File I/O and System Programming** to learn about file operations and system-level programming.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 6)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Online tutorials on data structure implementation
- Debugging tools documentation (GDB, Valgrind, AddressSanitizer)
- Best practices guides for structure design

File I/O

File Operations

File operations let programs persist data, process large datasets, and talk to the rest of the system. In C, the high-level interface lives in `<stdio.h>`: open a stream (`FILE *`), read or write, then close it. This chapter focuses on **Linux/POSIX** practice with complete, buildable programs.

Compile everything with:

```
gcc -std=c17 -Wall -Wextra -o program program.c
```

File Pointers and the `FILE` Structure

All stdio file work goes through a `FILE *`:

```
#include <stdio.h>

FILE *fp;  /* stream handle */
```

The exact layout of `FILE` is implementation-defined. Conceptually it holds buffering state, current position, error/EOF flags, and the underlying file descriptor.

Standard streams (already open):

Stream	Meaning	Typical fd
`stdin`	Standard input	0
`stdout`	Standard output	1
`stderr`	Standard error	2

Opening Files: `fopen`

```
FILE *fopen(const char *filename, const char *mode);
```

Returns a stream on success, or `NULL` on failure (check `errno`).

Modes

Mode	Access	Position	Create?	Truncate?
`"r"`	read	start	no	no
`"w"`	write	start	yes	yes
`"a"`	write	end	yes	no
`"r+"`	read/write	start	no	no
`"w+"`	read/write	start	yes	yes
`"a+"`	read/write	end (writes)	yes	no

Append `b` for binary (`"rb"`, `"wb"`, …). On Linux, `"b"` has little effect (no CRLF translation), but **always use it for non-text data** for portability and clarity.

Minimal open / close

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fp = fopen("data.txt", "r");
    if (fp == NULL) {
        perror("fopen data.txt");
        return EXIT_FAILURE;
    }

    /* ... use fp ... */

    if (fclose(fp) != 0) {
        perror("fclose");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

Text vs Binary Mode (Caveats)

Text mode (`"r"`, `"w"`, …)

- Oriented toward characters and lines
- On some systems (notably Windows), `\n` may be translated to `\r\n` on write and the reverse on read
- On Linux, translation is typically a no-op, but programs that care about **exact bytes** should still use binary mode

Binary mode (`"rb"`, `"wb"`, …)

- Bytes in, bytes out — no newline translation
- Correct choice for images, compressed data, structs dumped as bytes, executables
- **Caveat:** writing a raw `struct` with `fwrite` is **not** a portable file format (padding, endianness, alignment). Prefer explicit serialization for interchange

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *tf = fopen("note.txt", "w");
    FILE *bf = fopen("nums.bin", "wb");
    int nums[] = {1, 2, 3, 4, 5};

    if (tf == NULL || bf == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }

    fprintf(tf, "Hello, World!\n");
    if (fwrite(nums, sizeof nums[0], 5, bf) != 5) {
        fprintf(stderr, "short write or error on nums.bin\n");
    }

    fclose(tf);
    fclose(bf);
    return 0;
}
```

Portability note: Never assume `sizeof(int)` or endianness when exchanging binary files across machines. Use fixed-width types (`uint32_t`) and explicit byte order if the file leaves your process.

Closing and Buffering

```c
int fclose(FILE *stream);   /* 0 success, EOF failure */
int fflush(FILE *stream);   /* flush buffers to OS */
```

- **Always** `fclose` streams you opened (and check the result after writes — buffered data may fail only at flush/close).
- `fflush(stdout)` after prompts when you need the user to see text before a blocking read.
- `setvbuf` can change buffering mode (`_IOFBF`, `_IOLBF`, `_IONBF`).

```c
#include <stdio.h>

int main(void) {
    printf("Enter name: ");
    fflush(stdout);  /* ensure prompt appears */

    char name[64];
    if (fgets(name, sizeof name, stdin) == NULL) {
        return 1;
    }
    printf("Hi, %s", name);
    return 0;
}
```

Error Handling: `fopen`, `ferror`, `errno`

Check every open

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("missing.txt", "r");
    if (fp == NULL) {
        /* perror uses errno */
        perror("fopen missing.txt");

        /* or format yourself */
        fprintf(stderr, "fopen failed: %s (errno=%d)\n",
                strerror(errno), errno);
        return EXIT_FAILURE;
    }
    fclose(fp);
    return 0;
}
```

Common failure reasons: file not found (`ENOENT`), permission denied (`EACCES`), is a directory, too many open files, disk full on write.

Distinguish EOF from error

`fgetc` returns `EOF` both at end-of-file **and** on error. Use `feof` / `ferror`:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fp = fopen("example.txt", "r");
    int ch;

    if (fp == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }

    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }

    if (ferror(fp)) {
        fprintf(stderr, "read error on example.txt\n");
        clearerr(fp);
        fclose(fp);
        return EXIT_FAILURE;
    }
    if (feof(fp)) {
        /* normal end */
    }

    fclose(fp);
    return 0;
}
```

Function	Role
`feof(fp)`	Non-zero if EOF indicator is set
`ferror(fp)`	Non-zero if error indicator is set
`clearerr(fp)`	Clears both indicators
`perror(msg)`	Prints `msg: <strerror(errno)>\n` to stderr
`strerror(errno)`	Thread-aware alternatives exist (`strerror_r`) on POSIX

Pattern: cleanup on partial failure

When multiple resources open, close what you already opened before returning:

```c
FILE *in = fopen(src, "rb");
if (!in) {
```

```
    perror(src);
    return 1;
}
FILE *out = fopen(dst, "wb");
if (!out) {
    perror(dst);
    fclose(in);
    return 1;
}
/* ... */
fclose(out);
fclose(in);
```

Full Program 1: Copy a File

Binary-safe copy with block reads (faster than byte-at-a-time for large files):

```
/* file: fcopy.c */
#include <stdio.h>
#include <stdlib.h>

#define BUFSZ 8192

int main(int argc, char *argv[]) {
    FILE *src = NULL;
    FILE *dst = NULL;
    unsigned char buf[BUFSZ];
    size_t n;
    int status = EXIT_SUCCESS;

    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source> <destination>\n", argv[0]);
        return EXIT_FAILURE;
    }

    src = fopen(argv[1], "rb");
    if (src == NULL) {
        perror(argv[1]);
        return EXIT_FAILURE;
    }

    dst = fopen(argv[2], "wb");
    if (dst == NULL) {
```

```
        perror(argv[2]);
        fclose(src);
        return EXIT_FAILURE;
    }

    while ((n = fread(buf, 1, sizeof buf, src)) > 0) {
        if (fwrite(buf, 1, n, dst) != n) {
            fprintf(stderr, "write error: %s\n", argv[2]);
            status = EXIT_FAILURE;
            break;
        }
    }

    if (ferror(src)) {
        fprintf(stderr, "read error: %s\n", argv[1]);
        status = EXIT_FAILURE;
    }

    if (fclose(dst) != 0) {
        perror("fclose destination");
        status = EXIT_FAILURE;
    }
    if (fclose(src) != 0) {
        perror("fclose source");
        status = EXIT_FAILURE;
    }

    if (status == EXIT_SUCCESS) {
        fprintf(stderr, "copied %s -> %s\n", argv[1], argv[2]);
    }
    return status;
}
```

```
gcc -std=c17 -Wall -Wextra -o fcopy fcopy.c
./fcopy photo.jpg photo.bak
cmp photo.jpg photo.bak && echo identical
```

Full Program 2: Simplified `wc` (Lines, Words, Bytes)

Counts lines, words (whitespace-separated tokens), and bytes — like a small `wc`:

```c
/* file: mywc.c */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <stdbool.h>

struct counts {
    unsigned long lines;
    unsigned long words;
    unsigned long bytes;
};

static int count_stream(FILE *fp, struct counts *c) {
    int ch;
    bool in_word = false;

    c->lines = c->words = c->bytes = 0;

    while ((ch = fgetc(fp)) != EOF) {
        c->bytes++;

        if (ch == '\n') {
            c->lines++;
        }

        if (isspace((unsigned char)ch)) {
            in_word = false;
        } else if (!in_word) {
            in_word = true;
            c->words++;
        }
    }

    if (ferror(fp)) {
        return -1;
    }
    return 0;
}

static void print_counts(const struct counts *c, const char *name) {
    if (name != NULL) {
        printf("%8lu %8lu %8lu %s\n", c->lines, c->words, c->bytes, name);
    } else {
        printf("%8lu %8lu %8lu\n", c->lines, c->words, c->bytes);
    }
}
```

```
int main(int argc, char *argv[]) {
    struct counts total = {0, 0, 0};
    int i;
    int status = EXIT_SUCCESS;

    if (argc == 1) {
        struct counts c;
        if (count_stream(stdin, &c) != 0) {
            fprintf(stderr, "read error on stdin\n");
            return EXIT_FAILURE;
        }
        print_counts(&c, NULL);
        return EXIT_SUCCESS;
    }

    for (i = 1; i < argc; i++) {
        FILE *fp = fopen(argv[i], "rb");
        struct counts c;

        if (fp == NULL) {
            perror(argv[i]);
            status = EXIT_FAILURE;
            continue;
        }

        if (count_stream(fp, &c) != 0) {
            fprintf(stderr, "read error: %s\n", argv[i]);
            status = EXIT_FAILURE;
            fclose(fp);
            continue;
        }
        fclose(fp);

        print_counts(&c, argv[i]);
        total.lines += c.lines;
        total.words += c.words;
        total.bytes += c.bytes;
    }

    if (argc > 2) {
        print_counts(&total, "total");
    }

    return status;
}
```

```
gcc -std=c17 -Wall -Wextra -o mywc mywc.c
echo -e "hello world\nfoo" > sample.txt
./mywc sample.txt
./mywc sample.txt sample.txt
printf 'a b c' | ./mywc
```

Notes

- Binary mode avoids surprises on platforms with newline translation when counting bytes.
- `isspace((unsigned char)ch)` avoids undefined behavior for negative `char` values.
- Words follow a simple "transition into non-space" definition, similar in spirit to classic `wc`.

Full Program 3: Append Log Lines with Timestamps

Append-only log with ISO-like local timestamps:

```c
/* file: logappend.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <errno.h>

static int timestamp_now(char *buf, size_t buflen) {
    time_t t = time(NULL);
    struct tm tm_now;

    if (t == (time_t)-1) {
        return -1;
    }
    if (localtime_r(&t, &tm_now) == NULL) {
        return -1;
    }
    /* YYYY-MM-DD HH:MM:SS */
    if (strftime(buf, buflen, "%Y-%m-%d %H:%M:%S", &tm_now) == 0) {
        return -1;
    }
    return 0;
}

int main(int argc, char *argv[]) {
    const char *path;
```

```
FILE *fp;
char ts[32];
char line[1024];

if (argc != 2) {
    fprintf(stderr, "Usage: %s <logfile>\n", argv[0]);
    fprintf(stderr, "Reads lines from stdin and appends them with timestamps.\n");
    return EXIT_FAILURE;
}
path = argv[1];

fp = fopen(path, "a");
if (fp == NULL) {
    fprintf(stderr, "fopen %s: %s\n", path, strerror(errno));
    return EXIT_FAILURE;
}

while (fgets(line, sizeof line, stdin) != NULL) {
    size_t len = strlen(line);

    /* strip trailing newline for clean log formatting */
    if (len > 0 && line[len - 1] == '\n') {
        line[len - 1] = '\0';
    }

    if (timestamp_now(ts, sizeof ts) != 0) {
        fprintf(stderr, "timestamp failed\n");
        fclose(fp);
        return EXIT_FAILURE;
    }

    if (fprintf(fp, "[%s] %s\n", ts, line) < 0) {
        fprintf(stderr, "write failed: %s\n", strerror(errno));
        fclose(fp);
        return EXIT_FAILURE;
    }
    if (fflush(fp) != 0) {
        perror("fflush");
        fclose(fp);
        return EXIT_FAILURE;
    }
}

if (ferror(stdin)) {
    fprintf(stderr, "error reading stdin\n");
    fclose(fp);
    return EXIT_FAILURE;
```

```
    }

    if (fclose(fp) != 0) {
        perror("fclose");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o logappend logappend.c
echo "server started" | ./logappend app.log
echo "accepted client" | ./logappend app.log
cat app.log
# [2026-04-01 14:22:10] server started
# [2026-04-01 14:22:11] accepted client
```

`fopen(..., "a")` guarantees writes append even if another process also appends (with the usual caveats about line atomicity for large writes).

File Size and Status Helpers

```
/* file: fsize.c */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *fp;
    long size;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <file>\n", argv[0]);
        return EXIT_FAILURE;
    }

    fp = fopen(argv[1], "rb");
    if (fp == NULL) {
        perror(argv[1]);
        return EXIT_FAILURE;
    }

    if (fseek(fp, 0, SEEK_END) != 0) {
        perror("fseek");
```

```
        fclose(fp);
        return EXIT_FAILURE;
    }
    size = ftell(fp);
    if (size < 0) {
        perror("ftell");
        fclose(fp);
        return EXIT_FAILURE;
    }

    printf("%s: %ld bytes\n", argv[1], size);
    fclose(fp);
    return 0;
}
```

For huge files, prefer `fseeko` / `ftello` and `off_t` (define `_FILE_OFFSET_BITS=64` on some 32-bit systems). POSIX `stat()` is often clearer for metadata.

Best Practices

1. **Check** `fopen`, `fread`/`fwrite` counts, `ferror`, and `fclose` after writes.
2. **Close** everything you open; use reverse-order cleanup on error paths.
3. Use **`"rb"`/`"wb"`** for non-text or exact byte copies.
4. Prefer **block I/O** (`fread`/`fwrite` with an 4–64 KiB buffer) over `fgetc` for bulk data.
5. Use **`perror` / `strerror(errno)`** so users know *why* open failed.
6. **`fflush`** when the next operation needs data on disk or the user needs a prompt.
7. Do not ignore **short writes** — disks fill up; pipes break.
8. Treat raw `struct` dumps as non-portable binary unless you control every reader.

Exercises

Exercise 1 — Safe truncate-free write

Write `write_all.txt` using mode `"w"`. Then change the program to open with `"r+"` and overwrite only the first line without truncating the rest of the file. Explain when `"w"` would destroy data.

Exercise 2 — Byte-at-a-time vs block copy

Implement file copy with `fgetc`/`fputc` and again with an 8 KiB buffer. Time both on a ~50 MB file (`time ./fcopy ...`). Record the difference.

Exercise 3 — Extend `mywc`

Add a `-l` flag that prints only the line count (like `wc -l`). Keep the multi-file total behavior.

Exercise 4 — Rotating log

Modify `logappend` so that if the log exceeds 10000 bytes (`ftell` after open with `"a+"` and seek end), it renames `app.log` to `app.log.1` (use `rename(2)`) and opens a fresh log.

Exercise 5 — Error path drill

Point `fcopy` at a source you can read and a destination on a full filesystem or a path without write permission. Confirm you get a useful message and a non-zero exit code, and that the source stream is closed.

Exercise 6 — Binary round-trip

Write an array of ten `uint32_t` values to `data.bin` with `fwrite`, then read them back and print in hex. Re-open the same file in text mode and discuss whether anything appears different on Linux (and what would change on Windows).

Topic	Key API / idea

Summary

Topic	Key API / idea
Open / close	`fopen`, `fclose`
Modes	`"r"`/`"w"`/`"a"` and `+` / `b` variants
Text vs binary	Use `b` for exact bytes; watch portability of structs
Errors	`NULL` from `fopen`, `errno`, `ferror` vs `feof`
Bulk I/O	`fread` / `fwrite` with a fixed buffer
Practice programs	`fcopy`, `mywc`, `logappend`

Mastering careful open/check/copy/close patterns is the foundation for every later file-processing and system program in C.

File I/O Functions

Introduction

C provides a rich set of functions for performing input and output operations on files. These functions can be categorized into character I/O, line I/O, formatted I/O, and binary I/O functions. Each category serves different purposes and offers varying levels of control and convenience for file processing tasks.

Understanding these functions is crucial for effectively reading from and writing to files, whether you're processing text data, binary data, or structured information.

Compile everything with:

```
gcc -std=c17 -Wall -Wextra -o program program.c
```

Prefer complete programs: create input files, run tools, inspect outputs.

Character I/O Functions

Character I/O functions allow reading and writing files one character at a time. These functions are useful for simple text processing and when fine-grained control over file operations is needed.

fgetc() and getc()

Both functions read a single character from a file:

```
int fgetc(FILE *stream);
int getc(FILE *stream);
```

Key Points: - Returns the character read as an `unsigned char` cast to `int` - Returns `EOF` on end of file or error - `fgetc()` is a function, `getc()` may be a macro

fputc() and putc()

Both functions write a single character to a file:

```
int fputc(int char, FILE *stream);
int putc(int char, FILE *stream);
```

Key Points: - Writes the character (cast to `unsigned char`) to the file - Returns the written character on success - Returns `EOF` on error - `fputc()` is a function, `putc()` may be a macro

ungetc()

Pushes a character back onto the input stream:

```
int ungetc(int char, FILE *stream);
```

Key Points: - Allows "unread" operations for parsing - Only one character can be pushed back - Returns the character on success, `EOF` on failure

Examples

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

int main() {
    FILE *input, *output;
    int ch;

    input = fopen("input.txt", "r");
    output = fopen("output.txt", "w");

    if (input == NULL || output == NULL) {
        printf("Error opening files\n");
        exit(EXIT_FAILURE);
    }

    // Convert lowercase to uppercase
    while ((ch = fgetc(input)) != EOF) {
        if (islower(ch)) {
            ch = toupper(ch);
        }
        fputc(ch, output);
    }
```

```
    // Demonstrate ungetc
    ch = fgetc(input);
    if (ch != EOF) {
        ungetc(ch, input);  // Push character back
        ch = fgetc(input);  // Read it again
        printf("Character read twice: %c\n", ch);
    }

    fclose(input);
    fclose(output);

    return 0;
}
```

Line I/O Functions

Line I/O functions are designed for reading and writing entire lines of text, making them ideal for processing text files where data is organized in lines.

fgets()

Reads a line from a file:

```
char *fgets(char *str, int count, FILE *stream);
```

Key Points: - Reads at most `count-1` characters - Stops at newline or end of file - Always null-terminates the string - Stores newline in the buffer (if space available) - Returns `str` on success, `NULL` on error or EOF

fputs()

Writes a string to a file:

```
int fputs(const char *str, FILE *stream);
```

Key Points: - Writes string without null terminator - Does not append newline automatically - Returns non-negative value on success - Returns `EOF` on error

Examples

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    FILE *fp;
    char line[256];

    fp = fopen("lines.txt", "w");
    if (fp == NULL) {
        printf("Error opening file for writing\n");
        exit(EXIT_FAILURE);
    }

    // Write lines to file
    fputs("First line\n", fp);
    fputs("Second line\n", fp);
    fputs("Third line\n", fp);

    fclose(fp);

    // Read lines from file
    fp = fopen("lines.txt", "r");
    if (fp == NULL) {
        printf("Error opening file for reading\n");
        exit(EXIT_FAILURE);
    }

    while (fgets(line, sizeof(line), fp) != NULL) {
        // Remove newline if present
        line[strcspn(line, "\n")] = '\0';
        printf("Read line: '%s'\n", line);
    }

    fclose(fp);
    return 0;
}
```

Formatted I/O Functions

Formatted I/O functions provide high-level input and output operations similar to `printf()` and `scanf()`, but operate on files instead of standard streams.

fprintf()

Writes formatted output to a file:

```
int fprintf(FILE *stream, const char *format, ...);
```

Key Points: - Works like `printf()` but writes to a file - Returns number of characters written - Returns negative value on error

fscanf()

Reads formatted input from a file:

```
int fscanf(FILE *stream, const char *format, ...);
```

Key Points: - Works like `scanf()` but reads from a file - Returns number of items successfully read - Returns `EOF` on error or end of file

Examples

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    char name[50];
    double salary;
} Employee;

int main() {
    FILE *fp;
    Employee emp;

    // Write structured data to file
    fp = fopen("employees.txt", "w");
    if (fp == NULL) {
        printf("Error opening file for writing\n");
        exit(EXIT_FAILURE);
    }

    fprintf(fp, "%d %s %.2f\n", 1, "John Doe", 50000.0);
    fprintf(fp, "%d %s %.2f\n", 2, "Jane Smith", 60000.0);
    fprintf(fp, "%d %s %.2f\n", 3, "Bob Johnson", 55000.0);

    fclose(fp);
```

```
    // Read structured data from file
    fp = fopen("employees.txt", "r");
    if (fp == NULL) {
        printf("Error opening file for reading\n");
        exit(EXIT_FAILURE);
    }

    printf("Employee Records:\n");
    while (fscanf(fp, "%d %s %lf", &emp.id, emp.name, &emp.salary) == 3) {
        printf("ID: %d, Name: %s, Salary: %.2f\n",
               emp.id, emp.name, emp.salary);
    }

    fclose(fp);
    return 0;
}
```

Binary I/O Functions

Binary I/O functions allow reading and writing raw data in binary format, preserving the exact bit patterns of the data. These functions are essential for working with binary files like images, executables, or structured data.

fwrite()

Writes binary data to a file:

```
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream);
```

Key Points: - Writes `count` items of `size` bytes each - Returns number of items successfully written - Returns value less than `count` on error

fread()

Reads binary data from a file:

```
size_t fread(void *ptr, size_t size, size_t count, FILE *stream);
```

Key Points: - Reads `count` items of `size` bytes each - Returns number of items successfully read - Returns value less than `count` on error or EOF

Examples

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    char name[20];
    double value;
} Record;

int main() {
    FILE *fp;
    Record records[] = {
        {1, "Record1", 100.5},
        {2, "Record2", 200.75},
        {3, "Record3", 300.25}
    };
    Record read_records[3];
    size_t written, read;

    // Write binary data
    fp = fopen("records.bin", "wb");
    if (fp == NULL) {
        printf("Error opening file for writing\n");
        exit(EXIT_FAILURE);
    }

    written = fwrite(records, sizeof(Record), 3, fp);
    if (written != 3) {
        printf("Error writing records\n");
    }

    fclose(fp);

    // Read binary data
    fp = fopen("records.bin", "rb");
    if (fp == NULL) {
        printf("Error opening file for reading\n");
        exit(EXIT_FAILURE);
    }

    read = fread(read_records, sizeof(Record), 3, fp);
    if (read != 3) {
        printf("Error reading records\n");
    }
```

```
    fclose(fp);

    // Display read data
    printf("Read Records:\n");
    for (int i = 0; i < 3; i++) {
        printf("ID: %d, Name: %s, Value: %.2f\n",
               read_records[i].id, read_records[i].name, read_records[i].value);
    }

    return 0;
}
```

Practical Examples

CSV File Processor

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    int age;
    double salary;
} Employee;

int main() {
    FILE *input, *output;
    char line[256];
    Employee emp;
    double total_salary = 0;
    int count = 0;

    input = fopen("employees.csv", "r");
    output = fopen("report.txt", "w");

    if (input == NULL || output == NULL) {
        printf("Error opening files\n");
        exit(EXIT_FAILURE);
    }

    // Skip header line
    fgets(line, sizeof(line), input);
```

```
    fprintf(output, "Employee Report\n");
    fprintf(output, "===============\n");

    // Process each line
    while (fgets(line, sizeof(line), input) != NULL) {
        // Remove newline
        line[strcspn(line, "\n")] = '\0';

        // Parse CSV line
        if (sscanf(line, "%d,%[^,],%d,%lf",
                   &emp.id, emp.name, &emp.age, &emp.salary) == 4) {
            fprintf(output, "ID: %d, Name: %s, Age: %d, Salary: %.2f\n",
                    emp.id, emp.name, emp.age, emp.salary);
            total_salary += emp.salary;
            count++;
        }
    }

    if (count > 0) {
        fprintf(output, "\nAverage Salary: %.2f\n", total_salary / count);
    }

    fclose(input);
    fclose(output);

    printf("Report generated successfully\n");
    return 0;
}
```

Binary File Analyzer

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *fp;
    unsigned char buffer[16];
    long offset = 0;
    size_t bytes_read;

    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
```

```
    fp = fopen(argv[1], "rb");
    if (fp == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    printf("Offset  Hex Dump                  ASCII\n");
    printf("------  --------                  -----\n");

    while ((bytes_read = fread(buffer, 1, sizeof(buffer), fp)) > 0) {
        // Print offset
        printf("%06lx  ", offset);

        // Print hex values
        for (size_t i = 0; i < sizeof(buffer); i++) {
            if (i < bytes_read) {
                printf("%02x ", buffer[i]);
            } else {
                printf("   ");
            }

            // Add extra space after 8 bytes
            if (i == 7) printf(" ");
        }

        printf(" ");

        // Print ASCII representation
        for (size_t i = 0; i < bytes_read; i++) {
            if (buffer[i] >= 32 && buffer[i] <= 126) {
                printf("%c", buffer[i]);
            } else {
                printf(".");
            }
        }

        printf("\n");
        offset += bytes_read;
    }

    fclose(fp);
    return 0;
}
```

Error Handling and Best Practices

Comprehensive Error Checking

```c
#include <stdio.h>
#include <stdlib.h>

int safe_file_copy(const char *source, const char *destination) {
    FILE *src, *dst;
    int ch;

    // Open source file
    src = fopen(source, "rb");
    if (src == NULL) {
        perror("Error opening source file");
        return -1;
    }

    // Open destination file
    dst = fopen(destination, "wb");
    if (dst == NULL) {
        perror("Error opening destination file");
        fclose(src);
        return -1;
    }

    // Copy data
    while ((ch = fgetc(src)) != EOF) {
        if (fputc(ch, dst) == EOF) {
            perror("Error writing to destination file");
            fclose(src);
            fclose(dst);
            return -1;
        }
    }

    // Check for read errors
    if (ferror(src)) {
        perror("Error reading source file");
        fclose(src);
        fclose(dst);
        return -1;
    }

    // Close files
    if (fclose(src) != 0) {
```

```
        perror("Error closing source file");
        fclose(dst);
        return -1;
    }

    if (fclose(dst) != 0) {
        perror("Error closing destination file");
        return -1;
    }

    return 0;  // Success
}

int main() {
    if (safe_file_copy("source.txt", "destination.txt") == 0) {
        printf("File copied successfully\n");
    } else {
        printf("File copy failed\n");
    }

    return 0;
}
```

Full Programs: Character, Line, Formatted, Binary

Program 1 — ROT13 filter (`rot13.c`)

Character I/O over stdin/stdout (and optional files via shell redirection).

```
/* file: rot13.c */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

static int rot13_char(int ch) {
    if (ch >= 'a' && ch <= 'z') {
        return 'a' + (ch - 'a' + 13) % 26;
    }
    if (ch >= 'A' && ch <= 'Z') {
        return 'A' + (ch - 'A' + 13) % 26;
    }
    return ch;
}

int main(void) {
```

```
    int ch;
    while ((ch = fgetc(stdin)) != EOF) {
        if (fputc(rot13_char(ch), stdout) == EOF) {
            perror("fputc");
            return EXIT_FAILURE;
        }
    }
    if (ferror(stdin)) {
        perror("fgetc");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o rot13 rot13.c
echo 'Hello, World!' | ./rot13
echo 'Hello, World!' | ./rot13 | ./rot13   # identity
./rot13 < input.txt > output.txt
```

Program 2 — Line numberer with long-line handling (`nlnum.c`)

```
/* file: nlnum.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    FILE *in = stdin;
    char buf[64];
    unsigned long lineno = 0;
    int in_line = 0;  /* true if we already printed a number for current logical line */

    if (argc > 2) {
        fprintf(stderr, "Usage: %s [file]\n", argv[0]);
        return EXIT_FAILURE;
    }
    if (argc == 2) {
        in = fopen(argv[1], "r");
        if (in == NULL) {
            perror(argv[1]);
            return EXIT_FAILURE;
        }
    }

    while (fgets(buf, sizeof buf, in) != NULL) {
```

```
            if (!in_line) {
                lineno++;
                printf("%6lu\t", lineno);
                in_line = 1;
            }
            fputs(buf, stdout);
            if (strchr(buf, '\n') != NULL) {
                in_line = 0;  /* finished a full line */
            }
        }

        if (ferror(in)) {
            perror("read");
            if (in != stdin) {
                fclose(in);
            }
            return EXIT_FAILURE;
        }
        if (in != stdin) {
            fclose(in);
        }
        return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o nlnum nlnum.c
printf 'short\n' > sample.txt
# a line longer than the buffer still gets one line number:
python3 -c 'print("x"*100)' >> sample.txt
./nlnum sample.txt
```

Program 3 — CSV-ish records with `fprintf` / `fscanf` (`scores_io.c`)

```
/* file: scores_io.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

struct Score {
    char name[32];
    int points;
};

static int write_scores(const char *path, const struct Score *rows, size_t n) {
    FILE *fp = fopen(path, "w");
```

```
    size_t i;

    if (fp == NULL) {
        fprintf(stderr, "fopen %s: %s\n", path, strerror(errno));
        return -1;
    }
    for (i = 0; i < n; i++) {
        if (fprintf(fp, "%s %d\n", rows[i].name, rows[i].points) < 0) {
            perror("fprintf");
            fclose(fp);
            return -1;
        }
    }
    if (fclose(fp) != 0) {
        perror("fclose");
        return -1;
    }
    return 0;
}

static int read_scores(const char *path) {
    FILE *fp = fopen(path, "r");
    char name[32];
    int points;
    int nread;

    if (fp == NULL) {
        fprintf(stderr, "fopen %s: %s\n", path, strerror(errno));
        return -1;
    }

    printf("name\tpoints\n");
    printf("----\t------\n");
    while ((nread = fscanf(fp, "%31s %d", name, &points)) == 2) {
        printf("%s\t%d\n", name, points);
    }
    if (ferror(fp)) {
        perror("fscanf");
        fclose(fp);
        return -1;
    }
    /* nread==EOF or failed match ends loop - OK for this simple format */
    fclose(fp);
    return 0;
}

int main(void) {
```

```
    struct Score rows[] = {
        {"alice", 120},
        {"bob", 95},
        {"carol", 140}
    };

    if (write_scores("scores.txt", rows, 3) != 0) {
        return EXIT_FAILURE;
    }
    if (read_scores("scores.txt") != 0) {
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o scores_io scores_io.c
./scores_io
cat scores.txt
```

Caveat: `fscanf("%31s")` stops at whitespace — names with spaces need a different format (or `fgets` + parse).

Program 4 — Binary record dump / load (`binrec.c`)

```
/* file: binrec.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <errno.h>

struct Rec {
    uint32_t id;
    int32_t  value;
    char     tag[8];  /* fixed field; not a portable interchange format */
};

static int save_recs(const char *path, const struct Rec *recs, size_t n) {
    FILE *fp = fopen(path, "wb");
    size_t wrote;

    if (fp == NULL) {
        fprintf(stderr, "fopen %s: %s\n", path, strerror(errno));
        return -1;
    }
```

```
    wrote = fwrite(recs, sizeof recs[0], n, fp);
    if (wrote != n) {
        fprintf(stderr, "short write (%zu of %zu)\n", wrote, n);
        fclose(fp);
        return -1;
    }
    if (fclose(fp) != 0) {
        perror("fclose");
        return -1;
    }
    return 0;
}

static int load_and_print(const char *path) {
    FILE *fp = fopen(path, "rb");
    struct Rec r;
    size_t n;

    if (fp == NULL) {
        fprintf(stderr, "fopen %s: %s\n", path, strerror(errno));
        return -1;
    }
    while ((n = fread(&r, sizeof r, 1, fp)) == 1) {
        /* ensure tag is printable for display */
        char tag[9];
        memcpy(tag, r.tag, 8);
        tag[8] = '\0';
        printf("id=%u value=%d tag=\"%s\"\n", r.id, r.value, tag);
    }
    if (ferror(fp)) {
        perror("fread");
        fclose(fp);
        return -1;
    }
    fclose(fp);
    return 0;
}

int main(void) {
    struct Rec data[2];

    memset(data, 0, sizeof data);
    data[0].id = 1;
    data[0].value = 42;
    memcpy(data[0].tag, "alpha", 6);
    data[1].id = 2;
    data[1].value = -7;
```

```
    memcpy(data[1].tag, "beta", 5);

    if (save_recs("recs.bin", data, 2) != 0) {
        return EXIT_FAILURE;
    }
    if (load_and_print("recs.bin") != 0) {
        return EXIT_FAILURE;
    }

    /* hex peek of the file */
    {
        FILE *fp = fopen("recs.bin", "rb");
        unsigned char buf[16];
        size_t n;
        size_t i;
        if (fp) {
            n = fread(buf, 1, sizeof buf, fp);
            printf("first %zu bytes:", n);
            for (i = 0; i < n; i++) {
                printf(" %02x", buf[i]);
            }
            printf("\n");
            fclose(fp);
        }
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o binrec binrec.c
./binrec
xxd recs.bin | head
```

Portability note: raw `struct` dumps depend on padding and endianness. Fine for same-machine cache files; use explicit serialization for network or long-term formats.

Program 5 — `ungetc` simple number peek (`peek_num.c`)

```
/* file: peek_num.c */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

/* Read an integer from stdin; first non-digit is pushed back. */
static int read_int(int *out) {
    int ch;
```

```
    int sign = 1;
    long val = 0;
    int digits = 0;

    ch = fgetc(stdin);
    while (ch != EOF && isspace((unsigned char)ch)) {
        ch = fgetc(stdin);
    }
    if (ch == '+' || ch == '-') {
        sign = (ch == '-') ? -1 : 1;
        ch = fgetc(stdin);
    }
    while (ch != EOF && isdigit((unsigned char)ch)) {
        digits++;
        val = val * 10 + (ch - '0');
        ch = fgetc(stdin);
    }
    if (ch != EOF) {
        if (ungetc(ch, stdin) == EOF) {
            return -1;
        }
    }
    if (digits == 0) {
        return -1;
    }
    *out = (int)(sign * val);
    return 0;
}

int main(void) {
    int a, b;
    int ch;

    printf("Type: <int><op><int> e.g. 12+34\n");
    if (read_int(&a) != 0) {
        fprintf(stderr, "expected integer\n");
        return EXIT_FAILURE;
    }
    ch = fgetc(stdin);
    if (ch != '+' && ch != '-' && ch != '*' ) {
        fprintf(stderr, "expected operator, got %d\n", ch);
        return EXIT_FAILURE;
    }
    if (read_int(&b) != 0) {
        fprintf(stderr, "expected second integer\n");
        return EXIT_FAILURE;
    }
```

```
    switch (ch) {
    case '+': printf("%d\n", a + b); break;
    case '-': printf("%d\n", a - b); break;
    case '*': printf("%d\n", a * b); break;
    default:  return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o peek_num peek_num.c
printf '12+34\n' | ./peek_num
```

Program 6 — Block copy with progress (`bcopy.c`)

```
/* file: bcopy.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

#define BUF_SZ 4096

int main(int argc, char *argv[]) {
    FILE *in, *out;
    unsigned char buf[BUF_SZ];
    size_t n;
    unsigned long long total = 0;

    if (argc != 3) {
        fprintf(stderr, "Usage: %s <src> <dst>\n", argv[0]);
        return EXIT_FAILURE;
    }

    in = fopen(argv[1], "rb");
    if (in == NULL) {
        fprintf(stderr, "open %s: %s\n", argv[1], strerror(errno));
        return EXIT_FAILURE;
    }
    out = fopen(argv[2], "wb");
    if (out == NULL) {
        fprintf(stderr, "open %s: %s\n", argv[2], strerror(errno));
        fclose(in);
        return EXIT_FAILURE;
    }
```

```
    while ((n = fread(buf, 1, sizeof buf, in)) > 0) {
        size_t off = 0;
        while (off < n) {
            size_t w = fwrite(buf + off, 1, n - off, out);
            if (w == 0) {
                perror("fwrite");
                fclose(in);
                fclose(out);
                return EXIT_FAILURE;
            }
            off += w;
        }
        total += n;
    }
    if (ferror(in)) {
        perror("fread");
        fclose(in);
        fclose(out);
        return EXIT_FAILURE;
    }

    if (fclose(in) != 0 || fclose(out) != 0) {
        perror("fclose");
        return EXIT_FAILURE;
    }
    fprintf(stderr, "copied %llu bytes\n", total);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o bcopy bcopy.c
./bcopy scores.txt scores.bak
```

Error Handling Patterns (Quick Reference)

Function	Success	Failure / EOF
`fgetc` / `getc`	character as `int`	`EOF` — then check `ferror` vs `feof`
`fputc` / `putc`	character written	`EOF`
`fgets`	`str` pointer	`NULL`
`fputs`	non-negative	`EOF`
`fprintf`	char count 0	negative
`fscanf`	count of assigned fields	`EOF` or short count

Function	Success	Failure / EOF
`fread` / `fwrite`	items transferred	short count — check `ferror`

```c
int ch = fgetc(fp);
if (ch == EOF) {
    if (ferror(fp)) {
        perror("read");
    } else {
        /* clean EOF */
    }
}
```

Summary

C's file I/O functions provide comprehensive capabilities for working with files:

1. **Character I/O**: `fgetc()`, `fputc()`, `ungetc()` for single character operations
2. **Line I/O**: `fgets()`, `fputs()` for line-based text processing
3. **Formatted I/O**: `fprintf()`, `fscanf()` for structured data input/output
4. **Binary I/O**: `fwrite()`, `fread()` for raw binary data operations

Each category of functions serves specific purposes and offers different levels of control and convenience. Proper error handling and understanding of text vs. binary modes are essential for robust file processing applications.

Exercises

```c
gcc -std=c17 -Wall -Wextra -o exN exN.c
```

1. **Case flip** — Write a filter that toggles case of letters (`Hello` → `hELLO`) using `fgetc`/`fputc` and `ctype.h`.
2. **`nlnum` width** — Add `-w N` to control line-number field width (default 6).
3. **TSV writer** — Store three columns (string, int, double) with `fprintf` as tab-separated values; read them back with `fscanf` or `fgets`+parse.
4. **Binary double array** — Write 100 `double` values with `fwrite`, read them back, print the sum. Confirm file size is `100 * sizeof(double)`.

5. **Short write drill** — Modify `bcopy` to write in 1-byte `fwrite` loops (slow) and compare; still handle short writes correctly.
6. **`ungetc` comment skipper** — Skip C-style `//` comments from stdin: when you see `/` then `/`, discard until newline; use `ungetc` when the second char is not `/`.
7. **EOF vs error** — Open a directory as a file (e.g. `fopen(".", "r")` on Linux may fail) or force an error path; print whether you hit `ferror` or clean EOF on a normal file.

Advanced File Operations

Introduction

Advanced file operations in C go beyond basic reading and writing to include sophisticated techniques such as file positioning, random access, temporary file management, and concurrent access control. These operations are essential for building efficient file processing applications, database systems, and other programs that require fine-grained control over file access.

Understanding these advanced concepts enables programmers to create more robust and performant file-handling code, particularly when dealing with large files or complex data structures.

File Positioning

File positioning functions allow you to move the file pointer to specific locations within a file, enabling random access to data. This is particularly useful for databases, configuration files, and other structured data formats.

fseek()

Moves the file pointer to a specified position:

```
int fseek(FILE *stream, long offset, int origin);
```

Parameters: - `stream`: File pointer - `offset`: Number of bytes to move (can be negative) - `origin`: Reference point for the offset

Origin Values: - `SEEK_SET`: Beginning of file - `SEEK_CUR`: Current position - `SEEK_END`: End of file

Return Value: - `0` on success - Non-zero on error

ftell()

Returns the current position of the file pointer:

```
long ftell(FILE *stream);
```

Return Value: - Current file position (in bytes from beginning) - `-1L` on error

rewind()

Moves the file pointer to the beginning of the file:

```
void rewind(FILE *stream);
```

Examples

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char buffer[100];

    // Create a test file
    fp = fopen("test.txt", "w");
    if (fp == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    fprintf(fp, "Line 1: This is the first line.\n");
    fprintf(fp, "Line 2: This is the second line.\n");
    fprintf(fp, "Line 3: This is the third line.\n");
    fclose(fp);

    // Open for reading and demonstrate positioning
    fp = fopen("test.txt", "r");
    if (fp == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    // Read first line
    fgets(buffer, sizeof(buffer), fp);
    printf("First line: %s", buffer);

    // Move to beginning of file
    rewind(fp);
    fgets(buffer, sizeof(buffer), fp);
    printf("First line again: %s", buffer);

    // Move to position 20 from beginning
    fseek(fp, 20, SEEK_SET);
```

```
    fgets(buffer, sizeof(buffer), fp);
    printf("From position 20: %s", buffer);

    // Get current position
    long pos = ftell(fp);
    printf("Current position: %ld\n", pos);

    // Move 10 bytes forward from current position
    fseek(fp, 10, SEEK_CUR);
    pos = ftell(fp);
    printf("Position after moving 10 bytes forward: %ld\n", pos);

    fclose(fp);
    return 0;
}
```

Random Access Files

Random access files allow direct access to any part of the file without reading through preceding data. This is particularly useful for databases and other applications where quick access to specific records is required.

Structured Data Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define RECORD_SIZE 64

typedef struct {
    int id;
    char name[20];
    double value;
} Record;

// Write records to file
void write_records(const char *filename) {
    FILE *fp;
    Record records[] = {
        {1, "Record1", 100.5},
        {2, "Record2", 200.75},
        {3, "Record3", 300.25},
        {4, "Record4", 400.0},
```

```
        {5, "Record5", 500.75}
    };

    fp = fopen(filename, "wb");
    if (fp == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < 5; i++) {
        fwrite(&records[i], sizeof(Record), 1, fp);
    }

    fclose(fp);
}

// Read specific record by ID
int read_record(const char *filename, int record_id, Record *record) {
    FILE *fp;
    long position = (record_id - 1) * sizeof(Record);

    fp = fopen(filename, "rb");
    if (fp == NULL) {
        perror("Error opening file");
        return -1;
    }

    // Position to the specific record
    if (fseek(fp, position, SEEK_SET) != 0) {
        perror("Error seeking to record");
        fclose(fp);
        return -1;
    }

    // Read the record
    if (fread(record, sizeof(Record), 1, fp) != 1) {
        if (feof(fp)) {
            printf("Record not found\n");
        } else {
            perror("Error reading record");
        }
        fclose(fp);
        return -1;
    }

    fclose(fp);
    return 0;
```

```
}

// Update specific record by ID
int update_record(const char *filename, int record_id, const Record *new_record) {
    FILE *fp;
    long position = (record_id - 1) * sizeof(Record);

    fp = fopen(filename, "r+b");
    if (fp == NULL) {
        perror("Error opening file");
        return -1;
    }

    // Position to the specific record
    if (fseek(fp, position, SEEK_SET) != 0) {
        perror("Error seeking to record");
        fclose(fp);
        return -1;
    }

    // Write the updated record
    if (fwrite(new_record, sizeof(Record), 1, fp) != 1) {
        perror("Error writing record");
        fclose(fp);
        return -1;
    }

    fclose(fp);
    return 0;
}

int main() {
    Record record;

    // Create test file with records
    write_records("records.dat");

    // Read specific record
    if (read_record("records.dat", 3, &record) == 0) {
        printf("Record 3: ID=%d, Name=%s, Value=%.2f\n",
               record.id, record.name, record.value);
    }

    // Update record
    Record updated = {3, "Updated3", 999.99};
    if (update_record("records.dat", 3, &updated) == 0) {
```

```
        printf("Record 3 updated successfully\n");
    }

    // Read updated record
    if (read_record("records.dat", 3, &record) == 0) {
        printf("Updated Record 3: ID=%d, Name=%s, Value=%.2f\n",
               record.id, record.name, record.value);
    }

    return 0;
}
```

Temporary Files

Temporary files are useful for storing data temporarily during program execution. C provides functions to create temporary files that are automatically managed by the system.

tmpfile()

Creates a temporary binary file that is automatically deleted when closed:

```
FILE *tmpfile(void);
```

Return Value: - File pointer on success - `NULL` on error

tmpnam()

Generates a unique temporary filename:

```
char *tmpnam(char *str);
```

Parameters: - `str`: Buffer to store filename (if `NULL`, uses internal static buffer)

Return Value: - Pointer to temporary filename - `NULL` on error

mkstemp() (POSIX)

Creates and opens a unique temporary file (more secure):

```
int mkstemp(char *template);
```

Examples

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    FILE *tmp_fp;
    char tmp_name[L_tmpnam];
    int fd;

    // Method 1: Using tmpfile()
    tmp_fp = tmpfile();
    if (tmp_fp == NULL) {
        perror("Error creating temporary file");
        exit(EXIT_FAILURE);
    }

    fprintf(tmp_fp, "This is temporary data\n");
    fprintf(tmp_fp, "It will be deleted automatically\n");

    // Rewind and read back
    rewind(tmp_fp);
    int ch;
    while ((ch = fgetc(tmp_fp)) != EOF) {
        putchar(ch);
    }

    // File is automatically deleted when closed
    fclose(tmp_fp);

    // Method 2: Using tmpnam()
    if (tmpnam(tmp_name) == NULL) {
        printf("Error generating temporary filename\n");
        exit(EXIT_FAILURE);
    }

    printf("Temporary filename: %s\n", tmp_name);

    tmp_fp = fopen(tmp_name, "w+");
    if (tmp_fp == NULL) {
        perror("Error opening temporary file");
        exit(EXIT_FAILURE);
    }

    fprintf(tmp_fp, "Data in named temporary file\n");
```

```
    fclose(tmp_fp);

    // Note: tmpnam() doesn't delete the file automatically
    // You need to remove it manually
    if (remove(tmp_name) != 0) {
        perror("Error deleting temporary file");
    } else {
        printf("Temporary file deleted successfully\n");
    }

    return 0;
}
```

File Locking and Concurrent Access

When multiple processes or threads access the same file simultaneously, file locking mechanisms are necessary to prevent data corruption and ensure consistency.

flockfile() and funlockfile()

Provide explicit locking for FILE objects:

```
void flockfile(FILE *file);
void funlockfile(FILE *file);
```

ftrylockfile()

Attempts to lock a file without blocking:

```
int ftrylockfile(FILE *file);
```

Example with File Locking

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

void* writer_thread(void* arg) {
    FILE *fp = (FILE*)arg;
    int thread_id = *(int*)pthread_getspecific(pthread_self());
```

```
    for (int i = 0; i < 5; i++) {
        // Lock the file
        flockfile(fp);

        fprintf(fp, "Writer %d: Line %d\n", thread_id, i);
        fflush(fp);

        // Unlock the file
        funlockfile(fp);

        sleep(1);  // Simulate work
    }

    return NULL;
}

int main() {
    FILE *fp;
    pthread_t threads[3];
    int thread_ids[3] = {1, 2, 3};

    fp = fopen("shared.log", "w");
    if (fp == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    // Create writer threads
    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, writer_thread, fp);
    }

    // Wait for threads to complete
    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    fclose(fp);
    printf("All threads completed\n");

    return 0;
}
```

File Status and Information

C provides functions to query file status and obtain information about files without opening them.

fileno()

Returns the file descriptor associated with a FILE pointer:

```
int fileno(FILE *stream);
```

fstat() (POSIX)

Gets file status information:

```
#include <sys/stat.h>
int fstat(int fd, struct stat *buf);
```

Example with File Information

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

void print_file_info(const char *filename) {
    struct stat file_info;

    if (stat(filename, &file_info) != 0) {
        perror("Error getting file information");
        return;
    }

    printf("File: %s\n", filename);
    printf("Size: %ld bytes\n", file_info.st_size);
    printf("Permissions: %o\n", file_info.st_mode & 0777);
    printf("Last modified: %s", ctime(&file_info.st_mtime));
    printf("Number of links: %ld\n", file_info.st_nlink);

    // Check file type
    if (S_ISREG(file_info.st_mode)) {
        printf("Type: Regular file\n");
    } else if (S_ISDIR(file_info.st_mode)) {
        printf("Type: Directory\n");
    } else if (S_ISLNK(file_info.st_mode)) {
        printf("Type: Symbolic link\n");
    }
}
```

```c
int main() {
    FILE *fp;

    // Create a test file
    fp = fopen("info_test.txt", "w");
    if (fp == NULL) {
        perror("Error creating test file");
        exit(EXIT_FAILURE);
    }

    fprintf(fp, "This is a test file for information retrieval.\n");
    fprintf(fp, "It contains multiple lines of text.\n");
    fclose(fp);

    // Print file information
    print_file_info("info_test.txt");

    return 0;
}
```

Buffering Strategies

Understanding and controlling buffering can significantly impact file I/O performance.

setvbuf()

Controls buffering behavior:

```c
int setvbuf(FILE *stream, char *buffer, int mode, size_t size);
```

Mode Values: - `_IOFBF`: Full buffering - `_IOLBF`: Line buffering - `_IONBF`: No buffering

Example with Custom Buffering

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char buffer[1024];

    fp = fopen("buffer_test.txt", "w");
    if (fp == NULL) {
```

```
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    // Set custom buffer
    if (setvbuf(fp, buffer, _IOFBF, sizeof(buffer)) != 0) {
        printf("Error setting buffer\n");
        fclose(fp);
        exit(EXIT_FAILURE);
    }

    // Write data (will be buffered)
    for (int i = 0; i < 100; i++) {
        fprintf(fp, "Line %d: This is test data.\n", i);
    }

    // Force buffer flush
    fflush(fp);

    fclose(fp);
    printf("Data written with custom buffering\n");

    return 0;
}
```

Practical Examples

File Merger Utility

```
#include <stdio.h>
#include <stdlib.h>

int merge_files(const char *output, const char **inputs, int count) {
    FILE *out_fp, *in_fp;

    // Open output file
    out_fp = fopen(output, "wb");
    if (out_fp == NULL) {
        perror("Error opening output file");
        return -1;
    }

    // Process each input file
    for (int i = 0; i < count; i++) {
```

```
        in_fp = fopen(inputs[i], "rb");
        if (in_fp == NULL) {
            perror("Error opening input file");
            fclose(out_fp);
            return -1;
        }

        // Copy data
        int ch;
        while ((ch = fgetc(in_fp)) != EOF) {
            fputc(ch, out_fp);
        }

        fclose(in_fp);
    }

    fclose(out_fp);
    return 0;
}

int main() {
    const char *input_files[] = {"file1.txt", "file2.txt", "file3.txt"};

    // Create test files
    FILE *fp;
    fp = fopen("file1.txt", "w");
    fprintf(fp, "Content of file 1\n");
    fclose(fp);

    fp = fopen("file2.txt", "w");
    fprintf(fp, "Content of file 2\n");
    fclose(fp);

    fp = fopen("file3.txt", "w");
    fprintf(fp, "Content of file 3\n");
    fclose(fp);

    // Merge files
    if (merge_files("merged.txt", input_files, 3) == 0) {
        printf("Files merged successfully\n");
    } else {
        printf("Error merging files\n");
    }

    return 0;
}
```

File Splitter Utility

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int split_file(const char *input, long chunk_size) {
    FILE *in_fp, *out_fp;
    char out_name[256];
    int ch, chunk_num = 0;
    long bytes_written = 0;

    in_fp = fopen(input, "rb");
    if (in_fp == NULL) {
        perror("Error opening input file");
        return -1;
    }

    while ((ch = fgetc(in_fp)) != EOF) {
        // Open new output file if needed
        if (bytes_written == 0) {
            snprintf(out_name, sizeof(out_name), "%s.part%d", input, chunk_num);
            out_fp = fopen(out_name, "wb");
            if (out_fp == NULL) {
                perror("Error opening output file");
                fclose(in_fp);
                return -1;
            }
            chunk_num++;
        }

        // Write byte to current output file
        fputc(ch, out_fp);
        bytes_written++;

        // Close current output file if chunk is complete
        if (bytes_written >= chunk_size) {
            fclose(out_fp);
            bytes_written = 0;
        }
    }

    // Close last output file if open
    if (bytes_written > 0) {
        fclose(out_fp);
    }
```

```c
    fclose(in_fp);
    printf("File split into %d parts\n", chunk_num);
    return 0;
}

int main() {
    // Create test file
    FILE *fp = fopen("large_file.txt", "w");
    for (int i = 0; i < 1000; i++) {
        fprintf(fp, "This is line %d of the large test file.\n", i);
    }
    fclose(fp);

    // Split file into 1KB chunks
    if (split_file("large_file.txt", 1024) == 0) {
        printf("File split successfully\n");
    } else {
        printf("Error splitting file\n");
    }

    return 0;
}
```

Summary

Advanced file operations in C provide powerful capabilities for sophisticated file handling:

1. **File Positioning**: `fseek()`, `ftell()`, `rewind()` for random access
2. **Random Access Files**: Direct access to specific file locations
3. **Temporary Files**: `tmpfile()`, `tmpnam()` for temporary data storage
4. **File Locking**: Synchronization for concurrent file access
5. **File Information**: Status and metadata retrieval
6. **Buffering Control**: Performance optimization through custom buffering

These advanced techniques are essential for building robust file processing applications, database systems, and other programs requiring fine-grained control over file operations.

System-Level Programming

Introduction

System-level programming in C involves interacting directly with the operating system to perform tasks such as processing command-line arguments, accessing environment variables, managing process execution, and handling system signals. These capabilities allow C programs to integrate seamlessly with the operating system environment and provide more sophisticated functionality than basic file I/O operations.

Understanding system-level programming concepts is essential for developing robust applications that can adapt to different environments, handle errors gracefully, and interact with system resources effectively.

Environment: Linux/POSIX (macOS mostly fine). Compile with:

```
gcc -std=c17 -Wall -Wextra -o program program.c
```

Some examples use POSIX APIs (`sigaction`, `fork`, `unistd.h`).

Command-Line Arguments

Command-line arguments allow programs to receive input directly from the command line when they are executed. This is a fundamental way for programs to be configurable and flexible.

argc and argv

The `main()` function can accept two parameters for processing command-line arguments:

```
int main(int argc, char *argv[]);
```

Parameters: - `argc`: Argument count (number of command-line arguments) - `argv`: Argument vector (array of string pointers to the arguments)

Examples

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    printf("Program name: %s\n", argv[0]);
    printf("Number of arguments: %d\n", argc - 1);

    // Print all arguments
    for (int i = 1; i < argc; i++) {
        printf("Argument %d: %s\n", i, argv[i]);
    }

    return 0;
}
```

Practical Command-Line Processor

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void print_usage(const char *program_name) {
    printf("Usage: %s [options] <input_file> <output_file>\n", program_name);
    printf("Options:\n");
    printf("  -v, --verbose    Verbose output\n");
    printf("  -h, --help       Show this help message\n");
    printf("  -c, --copy       Copy mode (default)\n");
    printf("  -m, --move       Move mode\n");
}

int main(int argc, char *argv[]) {
    int verbose = 0;
    int move_mode = 0;
    const char *input_file = NULL;
    const char *output_file = NULL;

    // Process command-line arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-v") == 0 || strcmp(argv[i], "--verbose") == 0) {
            verbose = 1;
        } else if (strcmp(argv[i], "-h") == 0 || strcmp(argv[i], "--help") == 0) {
            print_usage(argv[0]);
```

```
            exit(EXIT_SUCCESS);
        } else if (strcmp(argv[i], "-m") == 0 || strcmp(argv[i], "--move") == 0) {
            move_mode = 1;
        } else if (strcmp(argv[i], "-c") == 0 || strcmp(argv[i], "--copy") == 0) {
            move_mode = 0;
        } else if (input_file == NULL) {
            input_file = argv[i];
        } else if (output_file == NULL) {
            output_file = argv[i];
        } else {
            fprintf(stderr, "Error: Too many arguments\n");
            print_usage(argv[0]);
            exit(EXIT_FAILURE);
        }
    }

    // Validate required arguments
    if (input_file == NULL || output_file == NULL) {
        fprintf(stderr, "Error: Input and output files required\n");
        print_usage(argv[0]);
        exit(EXIT_FAILURE);
    }

    if (verbose) {
        printf("Mode: %s\n", move_mode ? "Move" : "Copy");
        printf("Input file: %s\n", input_file);
        printf("Output file: %s\n", output_file);
    }

    // Perform the operation (simplified)
    if (move_mode) {
        if (verbose) printf("Moving file...\n");
        // In a real implementation, you would use rename()
    } else {
        if (verbose) printf("Copying file...\n");
        // In a real implementation, you would copy the file contents
    }

    printf("Operation completed successfully\n");
    return 0;
}
```

Environment Variables

Environment variables provide a way for programs to access system configuration information and user preferences. They are key-value pairs that are inherited from the parent process and can be

accessed by C programs.

getenv()

Retrieves the value of an environment variable:

```
char *getenv(const char *name);
```

Return Value: - Pointer to the value string on success - `NULL` if the variable is not found

Examples

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *home_dir, *path, *user, *shell;

    // Get common environment variables
    home_dir = getenv("HOME");
    path = getenv("PATH");
    user = getenv("USER");
    shell = getenv("SHELL");

    printf("Home directory: %s\n", home_dir ? home_dir : "Not set");
    printf("User: %s\n", user ? user : "Not set");
    printf("Shell: %s\n", shell ? shell : "Not set");
    printf("Path: %s\n", path ? path : "Not set");

    // Check for custom environment variable
    char *debug = getenv("DEBUG");
    if (debug != NULL && strcmp(debug, "1") == 0) {
        printf("Debug mode enabled\n");
    }

    return 0;
}
```

Setting Environment Variables

```
#include <stdio.h>
#include <stdlib.h>

int main() {
```

```
    // Set an environment variable
    if (setenv("MY_VARIABLE", "Hello, World!", 1) != 0) {
        perror("Error setting environment variable");
        exit(EXIT_FAILURE);
    }

    // Retrieve and display the variable
    char *value = getenv("MY_VARIABLE");
    printf("MY_VARIABLE = %s\n", value ? value : "Not set");

    // Unset the environment variable
    unsetenv("MY_VARIABLE");

    // Try to retrieve it again
    value = getenv("MY_VARIABLE");
    printf("MY_VARIABLE after unset = %s\n", value ? value : "Not set");

    return 0;
}
```

Process Management

Process management functions allow programs to control their execution flow, terminate gracefully, and manage exit status codes.

exit()

Terminates the program with a specified exit status:

```
void exit(int status);
```

atexit()

Registers functions to be called when the program exits:

```
int atexit(void (*func)(void));
```

Examples

```c
#include <stdio.h>
#include <stdlib.h>

void cleanup_handler1(void) {
    printf("Cleanup handler 1 called\n");
}

void cleanup_handler2(void) {
    printf("Cleanup handler 2 called\n");
}

void cleanup_handler3(void) {
    printf("Cleanup handler 3 called\n");
}

int main() {
    // Register cleanup handlers
    if (atexit(cleanup_handler1) != 0) {
        fprintf(stderr, "Failed to register cleanup handler 1\n");
        exit(EXIT_FAILURE);
    }

    if (atexit(cleanup_handler2) != 0) {
        fprintf(stderr, "Failed to register cleanup handler 2\n");
        exit(EXIT_FAILURE);
    }

    if (atexit(cleanup_handler3) != 0) {
        fprintf(stderr, "Failed to register cleanup handler 3\n");
        exit(EXIT_FAILURE);
    }

    printf("Main function executing\n");

    // Exit with success status
    exit(EXIT_SUCCESS);

    // This code will never be reached
    printf("This won't be printed\n");
}
```

Process Exit Status

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <number>\n", argv[0]);
        exit(EXIT_FAILURE);  // Exit with failure status
    }

    int number = atoi(argv[1]);

    if (number < 0) {
        fprintf(stderr, "Error: Negative numbers not allowed\n");
        exit(EXIT_FAILURE);  // Exit with failure status
    }

    if (number == 0) {
        printf("Zero is neither positive nor negative\n");
        exit(EXIT_SUCCESS);  // Exit with success status
    }

    printf("Number %d is positive\n", number);

    // Normal exit (equivalent to exit(EXIT_SUCCESS))
    return 0;
}
```

Signal Handling

Signals are software interrupts that allow the operating system to notify a process of various events. Signal handling enables programs to respond gracefully to events like user interrupts, termination requests, and other system events.

signal()

Sets up a signal handler:

```c
#include <signal.h>
void (*signal(int sig, void (*handler)(int)))(int);
```

Common Signals

Signal	Description	Default Action
`SIGINT`	Interrupt (Ctrl+C)	Terminate process
`SIGTERM`	Termination request	Terminate process
`SIGKILL`	Kill signal	Terminate process (cannot be caught)
`SIGSEGV`	Segmentation fault	Terminate process
`SIGUSR1`	User-defined signal 1	Terminate process
`SIGUSR2`	User-defined signal 2	Terminate process

Examples

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t signal_received = 0;

void signal_handler(int sig) {
    printf("\nReceived signal %d\n", sig);
    signal_received = 1;
}

int main() {
    // Set up signal handlers
    if (signal(SIGINT, signal_handler) == SIG_ERR) {
        perror("Error setting SIGINT handler");
        exit(EXIT_FAILURE);
    }

    if (signal(SIGTERM, signal_handler) == SIG_ERR) {
        perror("Error setting SIGTERM handler");
        exit(EXIT_FAILURE);
    }

    printf("Program running. Press Ctrl+C to interrupt.\n");

    // Main loop
    while (!signal_received) {
        printf("Working...\n");
        sleep(2);
    }

    printf("Cleaning up and exiting gracefully...\n");
```

```
    // Perform cleanup operations here
    // Close files, free memory, etc.

    exit(EXIT_SUCCESS);
}
```

Advanced Signal Handling

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

volatile sig_atomic_t sigint_count = 0;
volatile sig_atomic_t sigterm_received = 0;

void sigint_handler(int sig) {
    sigint_count++;
    printf("\nSIGINT received (%d times)\n", sigint_count);

    if (sigint_count >= 3) {
        printf("Three interrupts received. Exiting...\n");
        exit(EXIT_SUCCESS);
    }
}

void sigterm_handler(int sig) {
    printf("\nSIGTERM received. Cleaning up...\n");
    sigterm_received = 1;
}

void sigsegv_handler(int sig) {
    printf("\nSegmentation fault detected! Saving emergency data...\n");
    // Save critical data before terminating
    exit(EXIT_FAILURE);
}

int main() {
    // Set up signal handlers
    struct sigaction sa_int, sa_term, sa_segv;

    // SIGINT handler
    memset(&sa_int, 0, sizeof(sa_int));
    sa_int.sa_handler = sigint_handler;
```

```
    sigemptyset(&sa_int.sa_mask);
    sa_int.sa_flags = 0;
    sigaction(SIGINT, &sa_int, NULL);

    // SIGTERM handler
    memset(&sa_term, 0, sizeof(sa_term));
    sa_term.sa_handler = sigterm_handler;
    sigemptyset(&sa_term.sa_mask);
    sa_term.sa_flags = 0;
    sigaction(SIGTERM, &sa_term, NULL);

    // SIGSEGV handler
    memset(&sa_segv, 0, sizeof(sa_segv));
    sa_segv.sa_handler = sigsegv_handler;
    sigemptyset(&sa_segv.sa_mask);
    sa_segv.sa_flags = SA_RESETHAND;  // Only handle once
    sigaction(SIGSEGV, &sa_segv, NULL);

    printf("Advanced signal handling demo\n");
    printf("Press Ctrl+C up to 3 times, or send SIGTERM to terminate\n");

    // Main loop
    while (!sigterm_received) {
        printf("Working... (PID: %d)\n", getpid());
        sleep(3);
    }

    printf("Program terminating normally\n");
    return 0;
}
```

System Calls and Process Creation

System calls provide direct access to operating system services. While C's standard library provides higher-level interfaces, system calls offer more control and are sometimes necessary for specific tasks.

system()

Executes a command through the shell:

```
int system(const char *command);
```

Examples

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int result;

    // Execute system commands
    printf("Listing current directory:\n");
    result = system("ls -la");
    if (result == -1) {
        perror("Error executing command");
    }

    printf("\nCurrent date and time:\n");
    result = system("date");
    if (result == -1) {
        perror("Error executing command");
    }

    // Conditional execution based on system command result
    printf("\nChecking if 'git' is available:\n");
    result = system("which git > /dev/null 2>&1");
    if (result == 0) {
        printf("Git is available\n");
        system("git --version");
    } else {
        printf("Git is not available\n");
    }

    return 0;
}
```

Practical Examples

Configuration File Processor

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char database_host[100];
```

```
    int database_port;
    char log_level[20];
    int max_connections;
} Config;

int load_config(const char *filename, Config *config) {
    FILE *fp;
    char line[256];
    char key[100], value[100];

    // Set default values
    strcpy(config->database_host, "localhost");
    config->database_port = 5432;
    strcpy(config->log_level, "INFO");
    config->max_connections = 100;

    fp = fopen(filename, "r");
    if (fp == NULL) {
        // Try to load from environment variables
        char *env_host = getenv("DB_HOST");
        char *env_port = getenv("DB_PORT");
        char *env_log = getenv("LOG_LEVEL");
        char *env_max = getenv("MAX_CONN");

        if (env_host) strcpy(config->database_host, env_host);
        if (env_port) config->database_port = atoi(env_port);
        if (env_log) strcpy(config->log_level, env_log);
        if (env_max) config->max_connections = atoi(env_max);

        return 0;  // Success with environment variables
    }

    // Parse configuration file
    while (fgets(line, sizeof(line), fp) != NULL) {
        // Skip comments and empty lines
        if (line[0] == '#' || line[0] == '\n') continue;

        // Parse key=value pairs
        if (sscanf(line, "%[^=]=%s", key, value) == 2) {
            if (strcmp(key, "database_host") == 0) {
                strcpy(config->database_host, value);
            } else if (strcmp(key, "database_port") == 0) {
                config->database_port = atoi(value);
            } else if (strcmp(key, "log_level") == 0) {
                strcpy(config->log_level, value);
            } else if (strcmp(key, "max_connections") == 0) {
                config->max_connections = atoi(value);
```

```
            }
        }
    }

    fclose(fp);
    return 0;
}

void print_config(const Config *config) {
    printf("Configuration:\n");
    printf("  Database Host: %s\n", config->database_host);
    printf("  Database Port: %d\n", config->database_port);
    printf("  Log Level: %s\n", config->log_level);
    printf("  Max Connections: %d\n", config->max_connections);
}

int main(int argc, char *argv[]) {
    Config config;
    const char *config_file = "app.conf";

    // Check for command-line configuration file
    if (argc > 1) {
        config_file = argv[1];
    }

    // Load configuration
    if (load_config(config_file, &config) != 0) {
        fprintf(stderr, "Error loading configuration\n");
        exit(EXIT_FAILURE);
    }

    // Print configuration
    print_config(&config);

    // Use configuration in application
    printf("\nApplication starting with above configuration...\n");

    return 0;
}
```

Process Monitor

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

volatile sig_atomic_t terminate = 0;

void signal_handler(int sig) {
    printf("\nTermination signal received\n");
    terminate = 1;
}

int main(int argc, char *argv[]) {
    pid_t pid;
    int status;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <command>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    // Set up signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("Process monitor started\n");
    printf("Monitoring command: %s\n", argv[1]);

    while (!terminate) {
        // Fork a child process
        pid = fork();

        if (pid == -1) {
            perror("Error forking process");
            exit(EXIT_FAILURE);
        }

        if (pid == 0) {
            // Child process - execute the command
            execl("/bin/sh", "sh", "-c", argv[1], (char *)NULL);
            perror("Error executing command");
            exit(EXIT_FAILURE);
        } else {
```

```
            // Parent process - wait for child
            printf("Started process with PID %d\n", pid);

            // Wait for child to complete
            waitpid(pid, &status, 0);

            if (WIFEXITED(status)) {
                printf("Process exited with status %d\n", WEXITSTATUS(status));
            } else if (WIFSIGNALED(status)) {
                printf("Process terminated by signal %d\n", WTERMSIG(status));
            }

            // Wait before restarting (unless terminating)
            if (!terminate) {
                printf("Restarting in 5 seconds...\n");
                sleep(5);
            }
        }
    }

    printf("Process monitor terminated\n");
    return 0;
}
```

Full Programs: CLI, Env, Signals, Processes

Program 1 — Robust argument parser (`opts.c`)

```
/* file: opts.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Options {
    const char *input;
    const char *output;
    int verbose;
    int count;
};

static void usage(const char *prog) {
    fprintf(stderr,
            "Usage: %s [-v] [-n N] -i input -o output\n"
            "  -v        verbose\n"
```

```
            "  -n N      count (default 1)\n"
            "  -i path   input file\n"
            "  -o path   output file\n",
            prog);
}

int main(int argc, char *argv[]) {
    struct Options opt = {NULL, NULL, 0, 1};
    int i;

    for (i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-v") == 0) {
            opt.verbose = 1;
        } else if (strcmp(argv[i], "-n") == 0) {
            if (i + 1 >= argc) {
                usage(argv[0]);
                return EXIT_FAILURE;
            }
            opt.count = atoi(argv[++i]);
            if (opt.count <= 0) {
                fprintf(stderr, "-n must be positive\n");
                return EXIT_FAILURE;
            }
        } else if (strcmp(argv[i], "-i") == 0) {
            if (i + 1 >= argc) {
                usage(argv[0]);
                return EXIT_FAILURE;
            }
            opt.input = argv[++i];
        } else if (strcmp(argv[i], "-o") == 0) {
            if (i + 1 >= argc) {
                usage(argv[0]);
                return EXIT_FAILURE;
            }
            opt.output = argv[++i];
        } else if (strcmp(argv[i], "-h") == 0 || strcmp(argv[i], "--help") == 0) {
            usage(argv[0]);
            return EXIT_SUCCESS;
        } else {
            fprintf(stderr, "unknown option: %s\n", argv[i]);
            usage(argv[0]);
            return EXIT_FAILURE;
        }
    }

    if (opt.input == NULL || opt.output == NULL) {
```

```
        usage(argv[0]);
        return EXIT_FAILURE;
    }

    if (opt.verbose) {
        fprintf(stderr, "in=%s out=%s count=%d\n",
                opt.input, opt.output, opt.count);
    }
    printf("would process %s -> %s (%d times)\n",
           opt.input, opt.output, opt.count);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o opts opts.c
./opts -v -n 3 -i a.txt -o b.txt
./opts -h
./opts -i a.txt          # should fail with usage
```

Program 2 — Environment-driven config (`envcfg.c`)

```
/* file: envcfg.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Config {
    const char *home;
    const char *log_level;
    int max_items;
};

static int parse_positive_int(const char *s, int *out) {
    char *end;
    long v;

    if (s == NULL || out == NULL) {
        return -1;
    }
    v = strtol(s, &end, 10);
    if (end == s || *end != '\0' || v <= 0 || v > 1000000L) {
        return -1;
    }
    *out = (int)v;
    return 0;
}
```

```
int main(void) {
    struct Config cfg;
    const char *max_s;

    cfg.home = getenv("HOME");
    if (cfg.home == NULL) {
        cfg.home = ".";
    }

    cfg.log_level = getenv("APP_LOG_LEVEL");
    if (cfg.log_level == NULL) {
        cfg.log_level = "INFO";
    }

    max_s = getenv("APP_MAX_ITEMS");
    if (max_s == NULL) {
        cfg.max_items = 100;
    } else if (parse_positive_int(max_s, &cfg.max_items) != 0) {
        fprintf(stderr, "invalid APP_MAX_ITEMS=%s\n", max_s);
        return EXIT_FAILURE;
    }

    printf("home=%s\n", cfg.home);
    printf("log_level=%s\n", cfg.log_level);
    printf("max_items=%d\n", cfg.max_items);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o envcfg envcfg.c
./envcfg
APP_LOG_LEVEL=DEBUG APP_MAX_ITEMS=50 ./envcfg
APP_MAX_ITEMS=nope ./envcfg   # should fail
```

Program 3 — `atexit` cleanup and `exit` status (`cleanup_demo.c`)

```
/* file: cleanup_demo.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static FILE *g_log;

static void close_log(void) {
    if (g_log != NULL) {
```

```
        fprintf(g_log, "shutdown\n");
        fclose(g_log);
        g_log = NULL;
        fprintf(stderr, "atexit: log closed\n");
    }
}

int main(int argc, char *argv[]) {
    int fail = (argc > 1 && strcmp(argv[1], "fail") == 0);

    g_log = fopen("app_runtime.log", "w");
    if (g_log == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }
    if (atexit(close_log) != 0) {
        fprintf(stderr, "atexit registration failed\n");
        fclose(g_log);
        return EXIT_FAILURE;
    }

    fprintf(g_log, "started\n");
    printf("working...\n");

    if (fail) {
        fprintf(stderr, "failing with exit code 2\n");
        exit(2);  /* still runs atexit handlers */
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o cleanup_demo cleanup_demo.c
./cleanup_demo
cat app_runtime.log
./cleanup_demo fail; echo exit=$?
```

Note: `#include <string.h>` is required for `strcmp` — add it if your compiler rejects the program under pedantic modes.

Program 4 — Graceful SIGINT shutdown (`sigint_loop.c`)

```
/* file: sigint_loop.c */
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
```

```c
#include <unistd.h>
#include <string.h>

static volatile sig_atomic_t g_stop = 0;

static void on_sigint(int signo) {
    (void)signo;
    g_stop = 1;
}

int main(void) {
    struct sigaction sa;
    unsigned long ticks = 0;

    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;  /* no SA_RESTART: sleep may return early */

    if (sigaction(SIGINT, &sa, NULL) != 0) {
        perror("sigaction");
        return EXIT_FAILURE;
    }

    printf("pid=%d - press Ctrl+C to stop\n", (int)getpid());
    while (!g_stop) {
        printf("tick %lu\n", ticks++);
        sleep(1);
    }
    printf("clean shutdown after %lu ticks\n", ticks);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o sigint_loop sigint_loop.c
./sigint_loop
# Ctrl+C → clean message
```

Program 5 — `fork` + `exec` + `wait` mini runner (`run_cmd.c`)

```c
/* file: run_cmd.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c
#include <errno.h>

int main(int argc, char *argv[]) {
    pid_t pid;
    int status;

    if (argc < 2) {
        fprintf(stderr, "Usage: %s <command> [args...]\n", argv[0]);
        return EXIT_FAILURE;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return EXIT_FAILURE;
    }

    if (pid == 0) {
        /* child */
        execvp(argv[1], &argv[1]);
        fprintf(stderr, "execvp %s: %s\n", argv[1], strerror(errno));
        _exit(127);
    }

    /* parent */
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return EXIT_FAILURE;
    }

    if (WIFEXITED(status)) {
        printf("child exited %d\n", WEXITSTATUS(status));
        return WEXITSTATUS(status);
    }
    if (WIFSIGNALED(status)) {
        printf("child killed by signal %d\n", WTERMSIG(status));
        return 128 + WTERMSIG(status);
    }
    return EXIT_FAILURE;
}
```

```
gcc -std=c17 -Wall -Wextra -o run_cmd run_cmd.c
./run_cmd echo hello from child
./run_cmd false; echo status=$?
./run_cmd /no/such/cmd; echo status=$?
```

Program 6 — Echo lines with PID and env prefix (`line_echo.c`)

```c
/* file: line_echo.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    char line[256];
    const char *prefix = getenv("LINE_PREFIX");
    pid_t pid = getpid();

    if (prefix == NULL) {
        prefix = "";
    }

    while (fgets(line, sizeof line, stdin) != NULL) {
        /* strip newline for formatting */
        line[strcspn(line, "\n")] = '\0';
        printf("%s[pid=%d] %s\n", prefix, (int)pid, line);
        fflush(stdout);
    }
    if (ferror(stdin)) {
        perror("stdin");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -o line_echo line_echo.c
printf 'a\nb\n' | LINE_PREFIX='>>' ./line_echo
```

Exercises

```
gcc -std=c17 -Wall -Wextra -o exN exN.c
```

1. **`opts` long form** — Accept `--input=path` and `--verbose` in addition to short flags.
2. **Required env** — Fail startup if `APP_TOKEN` is unset or empty; print a clear error to stderr.
3. **Double atexit** — Register two `atexit` handlers; show they run in reverse order of registration.

4. **SIGTERM too** — Extend `sigint_loop` to stop on `SIGTERM` as well as `SIGINT` (one handler, two `sigaction` calls).
5. **Pipeline** — Use `run_cmd` to run `ls` and then `true`/`false`; document exit codes in a comment.
6. **Avoid `system()`** — Replace a `system("ls -l")` sketch with `fork`/`execvp`/`waitpid` for fixed argv `{"ls","-l",NULL}`.
7. **Argv0 banner** — Print only the basename of `argv[0]` (after last `/`) as the program name in usage errors.
8. **Timeout sketch** — After `fork`, parent uses `alarm(2)` or a timed wait strategy (research `sigtimedwait` or poll loop) to kill a hanging child — advanced stretch.

Summary

System-level programming in C provides powerful capabilities for interacting with the operating system:

1. **Command-Line Arguments**: Processing `argc` and `argv` for flexible program configuration
2. **Environment Variables**: Accessing system configuration through `getenv()`, `setenv()`, and `unsetenv()`
3. **Process Management**: Controlling program termination with `exit()` and cleanup with `atexit()`
4. **Signal Handling**: Responding to system events and interrupts gracefully (`sig_atomic_t`, `sigaction`)
5. **Process control**: `fork`, `exec*`, `waitpid` for running and supervising programs
6. **System Calls**: Executing shell commands carefully (`system` is convenient but shells inject risk)

These system-level programming concepts are essential for developing robust, production-ready C applications that can adapt to different environments and handle various runtime conditions effectively.

Module 8: File I/O and System Programming

Overview

This module covers file input/output operations and system-level programming in C. You'll learn how to work with files, handle different file modes, perform various I/O operations, and understand system programming concepts. The module also explores advanced file operations, command-line arguments, environment variables, and basic system calls.

Learning Objectives

By the end of this module, you will be able to: - Open, read, write, and close files using standard library functions - Understand different file modes and their appropriate usage - Perform character, line, formatted, and binary I/O operations - Manipulate file positions using seeking functions - Handle file errors and implement proper error checking - Work with command-line arguments and environment variables - Understand process creation and basic system calls - Implement file locking and concurrent access mechanisms - Apply best practices for file handling and system programming

Chapters

1. **File Operations** - File pointers, opening and closing files, text vs. binary modes, error handling
2. **File I/O Functions** - Character I/O, line I/O, formatted I/O, binary I/O functions
3. **Advanced File Operations** - File positioning, random access files, temporary files, file locking
4. **System-Level Programming** - Command-line arguments, environment variables, exit status, signal handling basics

Key Concepts Covered

- File pointers and the `FILE` structure
- File opening modes (`r`, `w`, `a`, `r+`, `w+`, `a+`, binary variants)
- Text vs. binary file operations
- Character I/O functions (`fgetc()`, `fputc()`, `ungetc()`)
- Line I/O functions (`fgets()`, `fputs()`)
- Formatted I/O functions (`fprintf()`, `fscanf()`)

- Binary I/O functions (`fread()`, `fwrite()`)
- File positioning functions (`fseek()`, `ftell()`, `rewind()`)
- Random access file operations
- Temporary file creation and management
- File locking mechanisms
- Command-line argument processing (`argc`, `argv`)
- Environment variable access (`getenv()`)
- Process exit handling (`exit()`, `atexit()`)
- Signal handling basics
- Error handling in file operations
- Buffering strategies and flushing

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Completion of Module 4: Functions and Modular Programming
- Completion of Module 5: Arrays and Strings
- Completion of Module 6: Pointers and Memory Management
- Completion of Module 7: Structures and User-Defined Types
- Understanding of basic programming concepts
- Familiarity with variable declaration and initialization

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- File system exploration tools
- Understanding of basic operating system concepts
- Familiarity with command-line interfaces

Estimated Time to Complete

- Reading: 8-10 hours
- Exercises: 12-15 hours
- Projects: 6-8 hours

Assessment

- Complete all chapter exercises
- Successfully implement file processing programs

- Demonstrate understanding of different file I/O techniques
- Pass file I/O and system programming quiz
- Submit a comprehensive file manipulation utility

Next Module

After completing this module, proceed to **Module 9: Advanced C Features and Modern Standards** to learn about modern C language features and standards.

Additional Resources

- "The C Programming Language" by Kernighan and Ritchie (Chapter 7)
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Operating system documentation
- File system documentation
- System programming guides
- POSIX standard documentation

Modern C

C99 Features

Introduction

The C99 standard (ISO/IEC 9899:1999) introduced numerous significant enhancements to the C programming language, making it more powerful, expressive, and suitable for modern programming tasks. These features addressed many limitations of the previous C89/C90 standard and brought C closer to contemporary programming language capabilities.

C99 features include new data types, improved control structures, enhanced initialization syntax, and better support for numerical computing. Understanding these features is essential for writing modern, efficient C code.

Variable-Length Arrays (VLAs)

One of the most significant additions in C99 was Variable-Length Arrays (VLAs), which allow the size of an array to be determined at runtime rather than compile time.

Basic VLA Declaration

```c
#include <stdio.h>

int main() {
    int size;

    printf("Enter array size: ");
    scanf("%d", &size);

    // Variable-length array - size determined at runtime
    int array[size];

    // Initialize and use the array
    for (int i = 0; i < size; i++) {
        array[i] = i * i;
        printf("array[%d] = %d\n", i, array[i]);
    }

    return 0;
}
```

Multi-dimensional VLAs

```c
#include <stdio.h>

int main() {
    int rows, cols;

    printf("Enter rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Multi-dimensional variable-length array
    int matrix[rows][cols];

    // Initialize the matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = i * cols + j;
        }
    }

    // Print the matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%4d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

VLA Function Parameters

```c
#include <stdio.h>

// Function that accepts VLA parameters
void process_matrix(int rows, int cols, int matrix[rows][cols]) {
    printf("Processing %dx%d matrix:\n", rows, cols);

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] *= 2;  // Double each element
        }
    }
}
```

```c
// Function to print a matrix
void print_matrix(int rows, int cols, int matrix[rows][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%4d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int rows = 3, cols = 4;

    // Declare and initialize a VLA
    int matrix[rows][cols] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    printf("Original matrix:\n");
    print_matrix(rows, cols, matrix);

    // Process the matrix
    process_matrix(rows, cols, matrix);

    printf("\nModified matrix:\n");
    print_matrix(rows, cols, matrix);

    return 0;
}
```

Designated Initializers

C99 introduced designated initializers, which allow you to initialize specific elements of arrays and structures by specifying their indices or member names.

Array Designated Initializers

```c
#include <stdio.h>

int main() {
    // Traditional initialization
    int array1[10] = {0, 0, 5, 0, 0, 0, 0, 8, 0, 0};
```

```
    // Designated initializers - only initialize specific elements
    int array2[10] = {[2] = 5, [7] = 8};

    // Mixed initialization
    int array3[10] = {1, 2, [5] = 10, 11, 12};

    printf("Array 1: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", array1[i]);
    }
    printf("\n");

    printf("Array 2: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", array2[i]);
    }
    printf("\n");

    printf("Array 3: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", array3[i]);
    }
    printf("\n");

    return 0;
}
```

Structure Designated Initializers

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    double salary;
    int age;
} Employee;

int main() {
    // Traditional initialization (order matters)
    Employee emp1 = {1, "John Doe", 50000.0, 30};

    // Designated initializers (order doesn't matter)
    Employee emp2 = {
```

```
        .id = 2,
        .name = "Jane Smith",
        .salary = 60000.0,
        .age = 28
    };

    // Partial initialization with designated initializers
    Employee emp3 = {
        .id = 3,
        .name = "Bob Johnson"
        // salary and age will be zero-initialized
    };

    printf("Employee 1: ID=%d, Name=%s, Salary=%.2f, Age=%d\n",
           emp1.id, emp1.name, emp1.salary, emp1.age);

    printf("Employee 2: ID=%d, Name=%s, Salary=%.2f, Age=%d\n",
           emp2.id, emp2.name, emp2.salary, emp2.age);

    printf("Employee 3: ID=%d, Name=%s, Salary=%.2f, Age=%d\n",
           emp3.id, emp3.name, emp3.salary, emp3.age);

    return 0;
}
```

Compound Literals

Compound literals allow you to create unnamed objects of any data type, including arrays and structures.

Array Compound Literals

```
#include <stdio.h>
#include <string.h>

void print_array(int size, int arr[size]) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    // Traditional approach
```

```c
    int temp_array[] = {1, 2, 3, 4, 5};
    print_array(5, temp_array);

    // Using compound literal
    print_array(5, (int[]){1, 2, 3, 4, 5});

    // Compound literal with VLA
    int size = 3;
    print_array(size, (int[size]){10, 20, 30});

    return 0;
}
```

Structure Compound Literals

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    int x, y;
} Point;

typedef struct {
    Point center;
    double radius;
} Circle;

void print_point(Point p) {
    printf("Point(%d, %d)\n", p.x, p.y);
}

void print_circle(Circle c) {
    printf("Circle(center=(%d, %d), radius=%.2f)\n",
           c.center.x, c.center.y, c.radius);
}

int main() {
    // Traditional approach
    Point p1 = {10, 20};
    print_point(p1);

    // Using compound literal
    print_point((Point){30, 40});

    // Structure with nested structure
```

```
    Circle c1 = {{0, 0}, 5.0};
    print_circle(c1);

    // Using compound literals for nested structures
    print_circle((Circle){{10, 10}, 7.5});

    return 0;
}
```

New Data Types

C99 introduced several new data types to improve numerical computing capabilities.

long long Type

```
#include <stdio.h>
#include <limits.h>

int main() {
    long long big_number = 123456789012345LL;
    unsigned long long huge_number = 123456789012345678 9ULL;

    printf("long long value: %lld\n", big_number);
    printf("unsigned long long value: %llu\n", huge_number);

    printf("LLONG_MIN: %lld\n", LLONG_MIN);
    printf("LLONG_MAX: %lld\n", LLONG_MAX);
    printf("ULLONG_MAX: %llu\n", ULLONG_MAX);

    return 0;
}
```

_Bool Type

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    // Using _Bool (C99)
    _Bool flag1 = 1;
    _Bool flag2 = 0;
```

```
    // Using bool with stdbool.h (recommended)
    bool is_valid = true;
    bool is_error = false;

    printf("_Bool values: %d, %d\n", flag1, flag2);
    printf("bool values: %d, %d\n", is_valid, is_error);

    // Boolean expressions
    int x = 10, y = 20;
    bool result = (x < y);
    printf("x < y: %s\n", result ? "true" : "false");

    return 0;
}
```

Inline Functions

The `inline` keyword suggests to the compiler that a function should be inlined to reduce function call overhead.

```
#include <stdio.h>

// Inline function suggestion
inline int square(int x) {
    return x * x;
}

// Static inline function (more portable)
static inline int cube(int x) {
    return x * x * x;
}

int main() {
    int num = 5;

    printf("Square of %d: %d\n", num, square(num));
    printf("Cube of %d: %d\n", num, cube(num));

    // Inline functions work in loops without function call overhead
    for (int i = 1; i <= 5; i++) {
        printf("Square of %d: %d\n", i, square(i));
    }

    return 0;
}
```

restrict Keyword

The `restrict` keyword is a hint to the compiler that a pointer is the only way to access a particular memory region, allowing for better optimization.

```c
#include <stdio.h>

// Without restrict - compiler must be conservative
void add_arrays_normal(int n, int *a, int *b, int *result) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i];
    }
}

// With restrict - compiler can optimize more aggressively
void add_arrays_restrict(int n, int *restrict a, int *restrict b, int *restrict result) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i];
    }
}

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int b[5] = {10, 20, 30, 40, 50};
    int result1[5], result2[5];

    add_arrays_normal(5, a, b, result1);
    add_arrays_restrict(5, a, b, result2);

    printf("Normal addition: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", result1[i]);
    }
    printf("\n");

    printf("Restrict addition: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", result2[i]);
    }
    printf("\n");

    return 0;
}
```

Flexible Array Members

C99 introduced flexible array members, allowing structures to have arrays of variable size at the end.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int count;
    double data[];  // Flexible array member (must be last)
} DynamicArray;

DynamicArray* create_dynamic_array(int count) {
    // Allocate memory for structure + array
    DynamicArray *arr = malloc(sizeof(DynamicArray) + count * sizeof(double));
    if (arr != NULL) {
        arr->count = count;
    }
    return arr;
}

void free_dynamic_array(DynamicArray *arr) {
    free(arr);
}

int main() {
    int size = 5;
    DynamicArray *arr = create_dynamic_array(size);

    if (arr != NULL) {
        // Initialize the flexible array
        for (int i = 0; i < arr->count; i++) {
            arr->data[i] = i * 1.5;
        }

        // Print the array
        printf("Dynamic array contents: ");
        for (int i = 0; i < arr->count; i++) {
            printf("%.1f ", arr->data[i]);
        }
        printf("\n");

        free_dynamic_array(arr);
    }
```

```
    return 0;
}
```

Practical Examples

Matrix Library Using C99 Features

```
#include <stdio.h>
#include <stdlib.h>

// Matrix structure using flexible array member
typedef struct {
    int rows, cols;
    double data[];  // Flexible array member
} Matrix;

// Create matrix
Matrix* create_matrix(int rows, int cols) {
    Matrix *m = malloc(sizeof(Matrix) + rows * cols * sizeof(double));
    if (m != NULL) {
        m->rows = rows;
        m->cols = cols;
    }
    return m;
}

// Free matrix
void free_matrix(Matrix *m) {
    free(m);
}

// Initialize matrix with designated initializers
void init_matrix(Matrix *m, double values[m->rows][m->cols]) {
    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
            m->data[i * m->cols + j] = values[i][j];
        }
    }
}

// Print matrix
void print_matrix(const Matrix *m) {
    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
```

```
            printf("%8.2f ", m->data[i * m->cols + j]);
        }
        printf("\n");
    }
}

// Matrix multiplication using VLAs
Matrix* multiply_matrices(const Matrix *a, const Matrix *b) {
    if (a->cols != b->rows) {
        printf("Error: Incompatible matrix dimensions\n");
        return NULL;
    }

    Matrix *result = create_matrix(a->rows, b->cols);
    if (result == NULL) {
        printf("Error: Memory allocation failed\n");
        return NULL;
    }

    // Use VLA for computation
    int rows = a->rows, cols = b->cols, inner = a->cols;
    double a_data[rows][inner], b_data[inner][cols], result_data[rows][cols];

    // Copy data to VLAs for easier access
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < inner; j++) {
            a_data[i][j] = a->data[i * a->cols + j];
        }
    }

    for (int i = 0; i < inner; i++) {
        for (int j = 0; j < cols; j++) {
            b_data[i][j] = b->data[i * b->cols + j];
        }
    }

    // Perform multiplication
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result_data[i][j] = 0;
            for (int k = 0; k < inner; k++) {
                result_data[i][j] += a_data[i][k] * b_data[k][j];
            }
        }
    }
```

```
    // Copy result back to matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result->data[i * cols + j] = result_data[i][j];
        }
    }

    return result;
}

int main() {
    // Create matrices using compound literals
    Matrix *m1 = create_matrix(2, 3);
    Matrix *m2 = create_matrix(3, 2);

    if (m1 == NULL || m2 == NULL) {
        printf("Error: Memory allocation failed\n");
        return 1;
    }

    // Initialize matrices
    double values1[2][3] = {{1, 2, 3}, {4, 5, 6}};
    double values2[3][2] = {{7, 8}, {9, 10}, {11, 12}};

    init_matrix(m1, values1);
    init_matrix(m2, values2);

    printf("Matrix 1:\n");
    print_matrix(m1);

    printf("\nMatrix 2:\n");
    print_matrix(m2);

    // Multiply matrices
    Matrix *result = multiply_matrices(m1, m2);
    if (result != NULL) {
        printf("\nResult of multiplication:\n");
        print_matrix(result);
        free_matrix(result);
    }

    free_matrix(m1);
    free_matrix(m2);

    return 0;
}
```

Summary

C99 introduced several powerful features that significantly enhanced the C programming language:

1. **Variable-Length Arrays (VLAs)**: Runtime-determined array sizes
2. **Designated Initializers**: Initialize specific array elements or structure members
3. **Compound Literals**: Create unnamed objects of any type
4. **New Data Types**: `long long`, `_Bool` for better numerical computing
5. **Inline Functions**: Suggest function inlining for performance
6. **restrict Keyword**: Hint for compiler optimization
7. **Flexible Array Members**: Variable-size arrays at structure end

These features make C99 code more expressive, efficient, and easier to write and maintain. Understanding and utilizing these features is essential for modern C programming.

C11 Features

Introduction

The C11 standard (ISO/IEC 9899:2011) introduced several important enhancements to the C programming language, focusing on improved type safety, concurrency support, and alignment with modern programming practices. These features addressed many of the limitations and safety concerns that had been identified in earlier versions of the language.

C11 features include static assertions for compile-time checking, anonymous structures and unions for more flexible data organization, thread-local storage for multi-threaded applications, and bounds-checking functions for improved security.

Static Assertions

Static assertions allow you to perform compile-time checks that can help catch errors early in the development process. This is particularly useful for validating assumptions about data types, structure sizes, or other compile-time constants.

_Static_assert

```
#include <stdio.h>
#include <stdint.h>

// Static assertion to ensure int is 32 bits
_Static_assert(sizeof(int) == 4, "int must be 32 bits");

// Static assertion to ensure pointer alignment
_Static_assert(sizeof(void*) >= sizeof(int), "Pointer must be at least as large as int");

typedef struct {
    uint8_t id;
    uint32_t timestamp;
    char data[256];
} Packet;

// Static assertion to ensure structure packing
_Static_assert(sizeof(Packet) == 261, "Packet structure size must be exactly 261 bytes");
```

```c
int main() {
    printf("All static assertions passed!\n");
    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of Packet: %zu bytes\n", sizeof(Packet));

    return 0;
}
```

Practical Static Assertion Examples

```c
#include <stdio.h>
#include <stdint.h>
#include <limits.h>

// Ensure char is 8 bits
_Static_assert(CHAR_BIT == 8, "char must be 8 bits");

// Ensure two's complement representation
_Static_assert(-1 == ~0, "System must use two's complement representation");

// Ensure floating-point format
_Static_assert(sizeof(float) == 4, "float must be 32 bits");
_Static_assert(sizeof(double) == 8, "double must be 64 bits");

// Custom assertion macro for better error messages
#define STATIC_ASSERT(condition, message) _Static_assert(condition, message)

// Assert that a structure has no padding
typedef struct {
    uint32_t a;
    uint16_t b;
    uint8_t c;
    uint8_t d;
} CompactStruct;

STATIC_ASSERT(sizeof(CompactStruct) == 8, "CompactStruct should be exactly 8 bytes");

int main() {
    printf("System characteristics validated at compile time\n");
    printf("Size of CompactStruct: %zu bytes\n", sizeof(CompactStruct));

    return 0;
}
```

Anonymous Structures and Unions

C11 introduced anonymous structures and unions, which allow you to embed structures and unions directly within other structures without requiring a member name.

Anonymous Structures

```c
#include <stdio.h>

typedef struct {
    int x, y;
} Point;

typedef struct {
    struct {  // Anonymous structure
        int width, height;
    };
    Point position;
} Rectangle;

int main() {
    Rectangle rect = {{100, 50}, {10, 20}};

    // Access anonymous structure members directly
    printf("Width: %d, Height: %d\n", rect.width, rect.height);
    printf("Position: (%d, %d)\n", rect.position.x, rect.position.y);

    // Modify anonymous structure members
    rect.width = 200;
    rect.height = 100;

    printf("Modified Width: %d, Height: %d\n", rect.width, rect.height);

    return 0;
}
```

Anonymous Unions

```c
#include <stdio.h>

typedef struct {
    enum { INTEGER, FLOAT, STRING } type;
    union {  // Anonymous union
        int i;
```

```
        float f;
        char *s;
    };
} Value;

void print_value(const Value *v) {
    switch (v->type) {
        case INTEGER:
            printf("Integer: %d\n", v->i);
            break;
        case FLOAT:
            printf("Float: %.2f\n", v->f);
            break;
        case STRING:
            printf("String: %s\n", v->s);
            break;
    }
}

int main() {
    Value values[3];

    // Initialize different types of values
    values[0].type = INTEGER;
    values[0].i = 42;

    values[1].type = FLOAT;
    values[1].f = 3.14f;

    values[2].type = STRING;
    values[2].s = "Hello, World!";

    // Print all values
    for (int i = 0; i < 3; i++) {
        print_value(&values[i]);
    }

    return 0;
}
```

Thread-Local Storage

C11 introduced the `_Thread_local` storage class specifier, which allows variables to have separate instances for each thread in a multi-threaded program.

Basic Thread-Local Storage

```c
#include <stdio.h>
#include <threads.h>
#include <stdlib.h>

// Thread-local variable
_Thread_local int thread_counter = 0;

// Function to be executed by threads
int thread_function(void *arg) {
    int thread_id = *(int*)arg;

    // Each thread has its own copy of thread_counter
    for (int i = 0; i < 5; i++) {
        thread_counter++;
        printf("Thread %d: counter = %d\n", thread_id, thread_counter);
        thrd_sleep(&(struct timespec){.tv_sec=1}, NULL);
    }

    return 0;
}

int main() {
    thrd_t threads[3];
    int thread_ids[3] = {1, 2, 3};

    // Create threads
    for (int i = 0; i < 3; i++) {
        if (thrd_create(&threads[i], thread_function, &thread_ids[i]) != thrd_success) {
            printf("Error creating thread %d\n", i);
            return 1;
        }
    }

    // Wait for threads to complete
    for (int i = 0; i < 3; i++) {
        thrd_join(threads[i], NULL);
    }

    printf("All threads completed\n");
    return 0;
}
```

Thread-Local Storage with Initialization

```c
#include <stdio.h>
#include <threads.h>
#include <stdlib.h>
#include <time.h>

// Thread-local variable with initializer
_Thread_local unsigned int thread_seed = 0;

// Function to initialize thread-local seed
void init_thread_seed(void) {
    if (thread_seed == 0) {
        thread_seed = (unsigned int)time(NULL) ^ (unsigned int)(uintptr_t)&thread_seed;
    }
}

int random_thread_function(void *arg) {
    int thread_id = *(int*)arg;

    // Initialize thread-local seed
    init_thread_seed();

    printf("Thread %d seed: %u\n", thread_id, thread_seed);

    // Generate some random numbers
    for (int i = 0; i < 5; i++) {
        int random_num = rand_r(&thread_seed) % 100;
        printf("Thread %d, Random %d: %d\n", thread_id, i, random_num);
        thrd_sleep(&(struct timespec){.tv_sec=0, .tv_nsec=100000000}, NULL);
    }

    return 0;
}

int main() {
    thrd_t threads[3];
    int thread_ids[3] = {1, 2, 3};

    // Create threads
    for (int i = 0; i < 3; i++) {
        if (thrd_create(&threads[i], random_thread_function, &thread_ids[i]) != thrd_success
            printf("Error creating thread %d\n", i);
            return 1;
        }
    }
```

```
    // Wait for threads to complete
    for (int i = 0; i < 3; i++) {
        thrd_join(threads[i], NULL);
    }

    printf("All threads completed\n");
    return 0;
}
```

Bounds-Checking Functions (Annex K)

C11 introduced bounds-checking functions as an optional extension (Annex K) to improve memory safety and prevent buffer overflows. These functions include additional parameters to specify buffer sizes.

Safe String Functions

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#ifdef __STDC_LIB_EXT1__
// Check if bounds-checking functions are available
void safe_string_operations(void) {
    char buffer[20];
    char source[] = "This is a long string that might overflow";
    errno_t result;

    // Safe string copy
    result = strcpy_s(buffer, sizeof(buffer), source);
    if (result != 0) {
        printf("strcpy_s failed: buffer too small\n");
    } else {
        printf("strcpy_s succeeded: %s\n", buffer);
    }

    // Safe string concatenation
    char buffer2[50] = "Hello, ";
    char append[] = "World!";

    result = strcat_s(buffer2, sizeof(buffer2), append);
    if (result != 0) {
        printf("strcat_s failed\n");
```

```
    } else {
        printf("strcat_s succeeded: %s\n", buffer2);
    }

    // Safe formatted output
    char formatted[30];
    result = sprintf_s(formatted, sizeof(formatted), "Number: %d, String: %s", 42, "test");
    if (result < 0) {
        printf("sprintf_s failed\n");
    } else {
        printf("sprintf_s succeeded: %s\n", formatted);
    }
}
#else
void safe_string_operations(void) {
    printf("Bounds-checking functions not available\n");
}
#endif

int main() {
    safe_string_operations();
    return 0;
}
```

Safe Memory Functions

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#ifdef __STDC_LIB_EXT1__
void safe_memory_operations(void) {
    char source[] = "Hello, World!";
    char destination[20];
    errno_t result;

    // Safe memory copy
    result = memcpy_s(destination, sizeof(destination), source, strlen(source) + 1);
    if (result != 0) {
        printf("memcpy_s failed\n");
    } else {
        printf("memcpy_s succeeded: %s\n", destination);
    }
```

```
    // Safe memory set
    char buffer[10];
    result = memset_s(buffer, sizeof(buffer), 'A', sizeof(buffer) - 1);
    if (result != 0) {
        printf("memset_s failed\n");
    } else {
        buffer[sizeof(buffer) - 1] = '\0';
        printf("memset_s succeeded: %s\n", buffer);
    }
}
#else
void safe_memory_operations(void) {
    printf("Bounds-checking functions not available\n");
}
#endif

int main() {
    safe_memory_operations();
    return 0;
}
```

Generic Selection

C11 introduced the `_Generic` keyword, which allows for type-generic programming by selecting different implementations based on the type of an expression.

Basic Generic Selection

```
#include <stdio.h>

// Generic print function using _Generic
#define print(x) _Generic((x), \
    int: print_int, \
    float: print_float, \
    double: print_double, \
    char*: print_string, \
    default: print_unknown \
)(x)

void print_int(int i) {
    printf("Integer: %d\n", i);
}

void print_float(float f) {
```

```
    printf("Float: %.2f\n", f);
}

void print_double(double d) {
    printf("Double: %.2f\n", d);
}

void print_string(char *s) {
    printf("String: %s\n", s);
}

void print_unknown(void *p) {
    printf("Unknown type: %p\n", p);
}

int main() {
    int i = 42;
    float f = 3.14f;
    double d = 2.71828;
    char *s = "Hello, World!";

    print(i);
    print(f);
    print(d);
    print(s);

    return 0;
}
```

Advanced Generic Selection

```
#include <stdio.h>
#include <math.h>

// Generic absolute value function
#define abs_val(x) _Generic((x), \
    int: abs, \
    long: labs, \
    long long: llabs, \
    float: fabsf, \
    double: fabs, \
    long double: fabsl \
)(x)

// Generic square function
```

```
#define square(x) _Generic((x), \
    int: square_int, \
    float: square_float, \
    double: square_double \
)(x)

int square_int(int x) {
    return x * x;
}

float square_float(float x) {
    return x * x;
}

double square_double(double x) {
    return x * x;
}

int main() {
    int i = -5;
    long l = -1000000L;
    float f = -3.14f;
    double d = -2.71828;

    printf("Absolute values:\n");
    printf("abs(%d) = %d\n", i, abs_val(i));
    printf("abs(%ld) = %ld\n", l, abs_val(l));
    printf("abs(%.2f) = %.2f\n", f, abs_val(f));
    printf("abs(%.2f) = %.2f\n", d, abs_val(d));

    printf("\nSquares:\n");
    printf("square(%d) = %d\n", i, square(i));
    printf("square(%.2f) = %.2f\n", f, square(f));
    printf("square(%.2f) = %.2f\n", d, square(d));

    return 0;
}
```

Alignment Support

C11 introduced alignment support through the `_Alignas` specifier and `_Alignof` operator, allowing for explicit control over memory alignment.

Alignment Specifier

```c
#include <stdio.h>
#include <stdalign.h>

// Align structure to 16-byte boundary
struct aligned_struct {
    _Alignas(16) char data[16];
    int value;
};

// Using alignas macro (more portable)
struct portable_aligned {
    alignas(32) double matrix[4][4];
    int id;
};

int main() {
    struct aligned_struct s1;
    struct portable_aligned s2;

    printf("Size of aligned_struct: %zu bytes\n", sizeof(s1));
    printf("Alignment of aligned_struct: %zu\n", _Alignof(struct aligned_struct));
    printf("Address of s1.data: %p (aligned to 16: %s)\n",
           (void*)s1.data, ((uintptr_t)s1.data % 16 == 0) ? "Yes" : "No");

    printf("\nSize of portable_aligned: %zu bytes\n", sizeof(s2));
    printf("Alignment of portable_aligned: %zu\n", alignof(struct portable_aligned));
    printf("Address of s2.matrix: %p (aligned to 32: %s)\n",
           (void*)s2.matrix, ((uintptr_t)s2.matrix % 32 == 0) ? "Yes" : "No");

    return 0;
}
```

Practical Examples

Thread-Safe Logger with Static Assertions

```c
#include <stdio.h>
#include <threads.h>
#include <time.h>
#include <stdarg.h>

// Static assertions for system requirements
```

```
_Static_assert(sizeof(void*) == 8, "64-bit system required");
_Static_assert(CHAR_BIT == 8, "8-bit char required");

// Thread-local logger instance
_Thread_local char log_buffer[1024];
_Thread_local int log_level = 0;

typedef enum {
    LOG_DEBUG,
    LOG_INFO,
    LOG_WARNING,
    LOG_ERROR
} LogLevel;

void log_message(LogLevel level, const char *format, ...) {
    const char *level_strings[] = {"DEBUG", "INFO", "WARNING", "ERROR"};

    if (level < log_level) return;

    // Get current time
    time_t now = time(NULL);
    struct tm *tm_info = localtime(&now);

    // Format timestamp
    strftime(log_buffer, sizeof(log_buffer), "%Y-%m-%d %H:%M:%S", tm_info);

    // Append log level and message
    int offset = strlen(log_buffer);
    snprintf(log_buffer + offset, sizeof(log_buffer) - offset,
             " [%s] ", level_strings[level]);

    // Format the actual message
    va_list args;
    va_start(args, format);
    offset = strlen(log_buffer);
    vsnprintf(log_buffer + offset, sizeof(log_buffer) - offset, format, args);
    va_end(args);

    printf("%s\n", log_buffer);
}

int logger_thread(void *arg) {
    int thread_id = *(int*)arg;

    // Set different log levels for different threads
    log_level = thread_id % 4;
```

```
    for (int i = 0; i < 3; i++) {
        log_message(LOG_DEBUG, "Thread %d debug message %d", thread_id, i);
        log_message(LOG_INFO, "Thread %d info message %d", thread_id, i);
        log_message(LOG_WARNING, "Thread %d warning message %d", thread_id, i);
        log_message(LOG_ERROR, "Thread %d error message %d", thread_id, i);
        thrd_sleep(&(struct timespec){.tv_sec=1}, NULL);
    }

    return 0;
}

int main() {
    thrd_t threads[3];
    int thread_ids[3] = {1, 2, 3};

    printf("Starting multi-threaded logger demo\n");

    // Create threads
    for (int i = 0; i < 3; i++) {
        if (thrd_create(&threads[i], logger_thread, &thread_ids[i]) != thrd_success) {
            printf("Error creating thread %d\n", i);
            return 1;
        }
    }

    // Wait for threads to complete
    for (int i = 0; i < 3; i++) {
        thrd_join(threads[i], NULL);
    }

    printf("Logger demo completed\n");
    return 0;
}
```

Summary

C11 introduced several important features that enhanced the safety, expressiveness, and concurrency support of the C programming language:

1. **Static Assertions**: Compile-time checks with `_Static_assert`
2. **Anonymous Structures/Unions**: Embed structures directly without member names
3. **Thread-Local Storage**: Per-thread variable instances with `_Thread_local`
4. **Bounds-Checking Functions**: Safer string and memory operations (Annex K)
5. **Generic Selection**: Type-generic programming with `_Generic`
6. **Alignment Support**: Explicit memory alignment control with `_Alignas` and `_Alignof`

These features make C11 a more robust and modern language while maintaining backward compatibility with earlier standards. Understanding and utilizing these features is essential for writing safe, efficient, and maintainable C code in contemporary development environments.

C17 and C23 Features

Introduction

The C17 standard (ISO/IEC 9899:2018) was primarily a technical corrigendum that fixed defects in the C11 standard without introducing significant new features. However, the upcoming C23 standard (ISO/IEC 9899:2024) represents a major evolution of the C language, introducing numerous modern features that make C more expressive, safer, and aligned with contemporary programming practices.

C23 features include typeof operators, binary literals, enhanced preprocessor directives, attribute syntax, and many other improvements that address long-standing limitations in the language. Understanding these features is crucial for writing modern C code.

C17: Technical Corrigendum

C17 was focused on fixing defects and inconsistencies in C11 rather than adding new features. Some notable changes include:

Defect Fixes

- Clarification of atomic operations semantics
- Fixes to alignment requirements
- Corrections to floating-point behavior
- Improvements to multi-threading support

Example of C17 Improvements

```
#include <stdio.h>
#include <stdalign.h>
#include <stdatomic.h>

// C17 clarified alignment requirements
_Alignas(16) struct aligned_data {
    int values[4];
};

// C17 improved atomic operations
```

```c
_Atomic int atomic_counter = 0;

int main() {
    printf("Alignment of aligned_data: %zu\n", alignof(struct aligned_data));

    // Atomic operations with clearer semantics
    atomic_fetch_add(&atomic_counter, 1);
    printf("Atomic counter: %d\n", atomic_load(&atomic_counter));

    return 0;
}
```

C23: typeof Operators

One of the most significant additions in C23 is the `typeof` operator, which allows you to declare variables with the same type as an expression. This feature brings C closer to C++'s `decltype` and makes generic programming much easier.

Basic typeof Usage

```c
#include <stdio.h>

#define max(a, b) ({ \
    typeof(a) _a = (a); \
    typeof(b) _b = (b); \
    _a > _b ? _a : _b; \
})

int main() {
    int x = 10, y = 20;
    double a = 3.14, b = 2.71;

    // typeof allows the macro to work with different types
    int max_int = max(x, y);
    double max_double = max(a, b);

    printf("Max of %d and %d: %d\n", x, y, max_int);
    printf("Max of %.2f and %.2f: %.2f\n", a, b, max_double);

    return 0;
}
```

typeof with Complex Expressions

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Declare variable with same type as array element
    typeof(array[0]) element = 42;

    // Declare variable with same type as array (pointer decay)
    typeof(array) *ptr = &array;

    // Declare variable with same type as complex expression
    typeof((int)3.14 + (long)42) result = 0;

    printf("Element: %d\n", element);
    printf("First array element via pointer: %d\n", (*ptr)[0]);
    printf("Result type size: %zu bytes\n", sizeof(result));

    return 0;
}
```

typeof_unqual for Unqualified Types

```c
#include <stdio.h>

int main() {
    const volatile int value = 42;

    // typeof preserves qualifiers
    typeof(value) qualified_var = 100;  // Still const volatile

    // typeof_unqual removes all qualifiers
    typeof_unqual(value) unqualified_var = 200;  // Just int

    printf("Qualified var type size: %zu\n", sizeof(qualified_var));
    printf("Unqualified var type size: %zu\n", sizeof(unqualified_var));

    return 0;
}
```

Binary Literals

C23 introduces binary literals, allowing you to write numbers in base-2 format using the `0b` or `0B` prefix.

Basic Binary Literals

```c
#include <stdio.h>

int main() {
    // Binary literals
    int binary1 = 0b1010;        // 10 in decimal
    int binary2 = 0B11110000;    // 240 in decimal
    int binary3 = 0b1111'1111;   // 255 in decimal (with digit separator)

    printf("0b1010 = %d\n", binary1);
    printf("0B11110000 = %d\n", binary2);
    printf("0b1111'1111 = %d\n", binary3);

    // Useful for bit manipulation
    unsigned char flags = 0b1010'0101;
    printf("Flags: 0x%02X (binary: ", flags);
    for (int i = 7; i >= 0; i--) {
        printf("%d", (flags >> i) & 1);
    }
    printf(")\n");

    return 0;
}
```

Bit Manipulation with Binary Literals

```c
#include <stdio.h>

// Bit manipulation macros using binary literals
#define BIT(n) (1U << (n))
#define SET_BIT(value, n) ((value) |= BIT(n))
#define CLEAR_BIT(value, n) ((value) &= ~BIT(n))
#define TOGGLE_BIT(value, n) ((value) ^= BIT(n))
#define CHECK_BIT(value, n) (((value) & BIT(n)) != 0)

int main() {
    unsigned char status = 0b0000'0000;
```

```
    printf("Initial status: 0b");
    for (int i = 7; i >= 0; i--) {
        printf("%d", (status >> i) & 1);
    }
    printf("\n");

    // Set some bits
    SET_BIT(status, 0);  // Set bit 0
    SET_BIT(status, 3);  // Set bit 3
    SET_BIT(status, 7);  // Set bit 7

    printf("After setting bits 0, 3, 7: 0b");
    for (int i = 7; i >= 0; i--) {
        printf("%d", (status >> i) & 1);
    }
    printf("\n");

    // Toggle a bit
    TOGGLE_BIT(status, 3);  // Toggle bit 3

    printf("After toggling bit 3: 0b");
    for (int i = 7; i >= 0; i--) {
        printf("%d", (status >> i) & 1);
    }
    printf("\n");

    // Check specific bits
    printf("Bit 0 is %s\n", CHECK_BIT(status, 0) ? "set" : "clear");
    printf("Bit 3 is %s\n", CHECK_BIT(status, 3) ? "set" : "clear");
    printf("Bit 5 is %s\n", CHECK_BIT(status, 5) ? "set" : "clear");

    return 0;
}
```

Enhanced Preprocessor Directives

C23 introduces several new preprocessor directives that make conditional compilation more powerful and flexible.

#elifdef and #elifndef

```c
#include <stdio.h>

#define DEBUG_MODE
// #define PRODUCTION_MODE

int main() {
#if defined(DEBUG_MODE)
    printf("Debug mode enabled\n");
#elifdef PRODUCTION_MODE
    printf("Production mode enabled\n");
#elifndef TESTING_MODE
    printf("Testing mode not defined, using default mode\n");
#else
    printf("Unknown mode\n");
#endif

    return 0;
}
```

#warning Directive

```c
#include <stdio.h>

// Compiler version check
#if __STDC_VERSION__ < 202311L
    #warning "This code is designed for C23 or later"
#endif

// Platform-specific warnings
#ifdef __APPLE__
    #warning "This code may need adjustments for macOS"
#endif

int main() {
    printf("Program running\n");

#ifdef __GNUC__
    #warning "Consider using Clang for better C23 support"
#endif

    return 0;
}
```

#embed Directive (Conceptual)

Note: The `#embed` directive is still being standardized and may not be available in all compilers yet.

```
// Example of proposed #embed directive
// #embed "shader.glsl" limit(1024) prefix(0xFF, 0xD8) suffix(0x00)

// This would embed the contents of shader.glsl as a byte array
// with a JPEG header prefix and null terminator suffix
```

Attribute Syntax

C23 introduces a standardized attribute syntax using double square brackets `[[]]`, making it easier to specify compiler-specific features in a portable way.

Standard Attributes

```c
#include <stdio.h>

// [[nodiscard]] - warn if return value is ignored
[[nodiscard]] int calculate_value(int x) {
    return x * x;
}

// [[maybe_unused]] - suppress unused variable warnings
void example_function([[maybe_unused]] int debug_param) {
    // debug_param might only be used in debug builds
#ifndef NDEBUG
    printf("Debug param: %d\n", debug_param);
#endif
}

// [[deprecated]] - mark functions as deprecated
[[deprecated("Use calculate_value instead")]]
int old_function(int x) {
    return x + x;
}

// [[fallthrough]] - indicate intentional fallthrough in switch
int process_state(int state) {
    switch (state) {
        case 1:
        case 2:
```

```
            printf("Low state\n");
            [[fallthrough]];
        case 3:
        case 4:
            printf("Medium state\n");
            [[fallthrough]];
        case 5:
            printf("High state\n");
            break;
        default:
            printf("Unknown state\n");
    }
    return 0;
}

int main() {
    // [[nodiscard]] example - compiler should warn if ignored
    calculate_value(5);  // Warning: ignoring return value

    example_function(42);

    // [[deprecated]] example
    old_function(10);  // Warning: deprecated function

    process_state(3);

    return 0;
}
```

Extended Integer Types

C23 introduces `_BitInt(N)` types for extended integer precision, allowing for integers with more than 64 bits.

_BitInt Example (Conceptual)

```
// Note: _BitInt is still being implemented in compilers
/*
#include <stdio.h>

int main() {
    // 128-bit integer
    _BitInt(128) big_number = 0;
```

```
    // 256-bit integer for cryptographic operations
    _BitInt(256) crypto_key = 0;

    printf("Size of 128-bit integer: %zu bytes\n", sizeof(big_number));
    printf("Size of 256-bit integer: %zu bytes\n", sizeof(crypto_key));

    return 0;
}
*/
```

Enhanced _Static_assert

C23 improves the `_Static_assert` macro by making the message parameter optional.

Enhanced Static Assertions

```c
#include <stdio.h>
#include <stdint.h>

// Traditional static assertion with message
_Static_assert(sizeof(int) >= 4, "int must be at least 32 bits");

// C23 allows static assertion without message
_Static_assert(sizeof(long long) > sizeof(long));

// Complex static assertions
typedef struct {
    uint32_t header;
    char data[256];
    uint32_t checksum;
} Packet;

_Static_assert(sizeof(Packet) == 264, "Packet structure size must be exactly 264 bytes");
_Static_assert(_Alignof(Packet) >= 4, "Packet must be aligned to at least 4 bytes");

int main() {
    printf("All static assertions passed\n");
    printf("Size of Packet: %zu bytes\n", sizeof(Packet));
    printf("Alignment of Packet: %zu bytes\n", _Alignof(Packet));

    return 0;
}
```

New Standard Library Functions

C23 adds several new standard library functions, including improved string handling and memory allocation functions.

strdup and strndup

```c
#define _GNU_SOURCE  // For strdup in some implementations
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Simple implementation of strdup if not available
char *my_strdup(const char *s) {
    size_t len = strlen(s) + 1;
    char *copy = malloc(len);
    if (copy) {
        memcpy(copy, s, len);
    }
    return copy;
}

int main() {
    const char *original = "Hello, World!";

    // Using strdup (if available in C23 implementation)
#ifdef _GNU_SOURCE
    char *copy1 = strdup(original);
    if (copy1) {
        printf("strdup copy: %s\n", copy1);
        free(copy1);
    }
#endif

    // Using custom implementation
    char *copy2 = my_strdup(original);
    if (copy2) {
        printf("my_strdup copy: %s\n", copy2);
        free(copy2);
    }

    // strndup example (copy at most n characters)
    char *copy3 = strndup(original, 5);
    if (copy3) {
        printf("strndup copy (5 chars): %s\n", copy3);
        free(copy3);
```

```
    }

    return 0;
}
```

Enhanced aligned_alloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate 100 bytes aligned to 16-byte boundary
    void *ptr = aligned_alloc(16, 100);
    if (ptr) {
        printf("Allocated address: %p\n", ptr);
        printf("Alignment check: %s\n",
               ((uintptr_t)ptr % 16 == 0) ? "OK" : "FAIL");

        // Use the memory...

        free(ptr);
    } else {
        printf("Allocation failed\n");
    }

    return 0;
}
```

Practical Examples

Modern Configuration System

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Using binary literals for configuration flags
typedef enum {
    CFG_DEBUG      = 0b0000'0001,
    CFG_LOGGING    = 0b0000'0010,
    CFG_NETWORK    = 0b0000'0100,
    CFG_DATABASE   = 0b0000'1000,
```

```
    CFG_SECURITY   = 0b0001'0000,
    CFG_COMPRESSION= 0b0010'0000
} ConfigFlags;

// Using typeof for generic configuration handling
#define CONFIG_GET(config, key) \
    typeof(((config)->key)) get_config_value(const Config *c, const char *k) { \
        if (strcmp(k, #key) == 0) return c->key; \
        /* Handle other keys */ \
        return (typeof(((config)->key)))0; \
    }

typedef struct {
    int max_connections;
    double timeout;
    bool enable_cache;
    ConfigFlags flags;
} Config;

// Using attributes for better code quality
[[nodiscard]] bool validate_config(const Config *config) {
    [[maybe_unused]] int debug_value = 42;

    if (config->max_connections <= 0) {
        return false;
    }

    if (config->timeout < 0.0) {
        return false;
    }

    return true;
}

[[deprecated("Use validate_config instead")]]
bool old_validate_config(Config *config) {
    return validate_config(config);
}

void print_config(const Config *config) {
    printf("Configuration:\n");
    printf("  Max Connections: %d\n", config->max_connections);
    printf("  Timeout: %.2f\n", config->timeout);
    printf("  Enable Cache: %s\n", config->enable_cache ? "Yes" : "No");
    printf("  Flags: 0b");
    for (int i = 7; i >= 0; i--) {
        printf("%d", (config->flags >> i) & 1);
```

```
    }
    printf("\n");
}

int main() {
    // Initialize configuration with binary literals and designated initializers
    Config config = {
        .max_connections = 100,
        .timeout = 30.0,
        .enable_cache = true,
        .flags = CFG_DEBUG | CFG_LOGGING | CFG_NETWORK
    };

    // Static assertion to ensure configuration structure size
    _Static_assert(sizeof(Config) > 0, "Config structure must not be empty");

    if (validate_config(&config)) {
        print_config(&config);
    } else {
        printf("Invalid configuration\n");
    }

    // Deprecated function usage (should generate warning)
    old_validate_config(&config);

    return 0;
}
```

Summary

C17 and C23 represent important milestones in the evolution of the C programming language:

1. **C17**: Technical corrigendum fixing defects in C11
2. **C23**: Major feature release with modern language enhancements

Key C23 features include:

1. **typeof Operators**: Type introspection and generic programming
2. **Binary Literals**: More intuitive bit manipulation
3. **Enhanced Preprocessor**: More powerful conditional compilation
4. **Attribute Syntax**: Standardized compiler hints
5. **Extended Integer Types**: Support for arbitrary precision integers
6. **Enhanced Static Assertions**: More flexible compile-time checking
7. **New Standard Library Functions**: Improved string and memory operations

These features make modern C more expressive, safer, and better aligned with contemporary programming practices while maintaining the language's core principles of efficiency and low-level control.

What C23 Removed: Trigraphs, K&R Definitions and () Meaning "Anything"

What C23 Removed: Trigraphs, K&R Definitions and () Meaning "Anything"

Thesis

Most of what is written about C23 lists the additions: `nullptr`, `constexpr`, `#embed`, `typeof`. Old code is more likely to be affected by the **removals**. C23 deletes features that had been in the language since 1989 or earlier:

- trigraphs
- function definitions with identifier lists (K&R style)
- function declarators without prototypes, so `int f();` now means `int f(void);`
- `bool`, `true` and `false` as macros: they are now keywords

C23 also makes two's complement mandatory and turns `realloc(p, 0)` into undefined behaviour. GCC 15 changed its default C dialect from `gnu17` to `gnu23` (GCC 15 porting guide), so these changes now reach code that never asked for C23. This chapter compiles each one as C17 and as C23 so you can see exactly what changes.

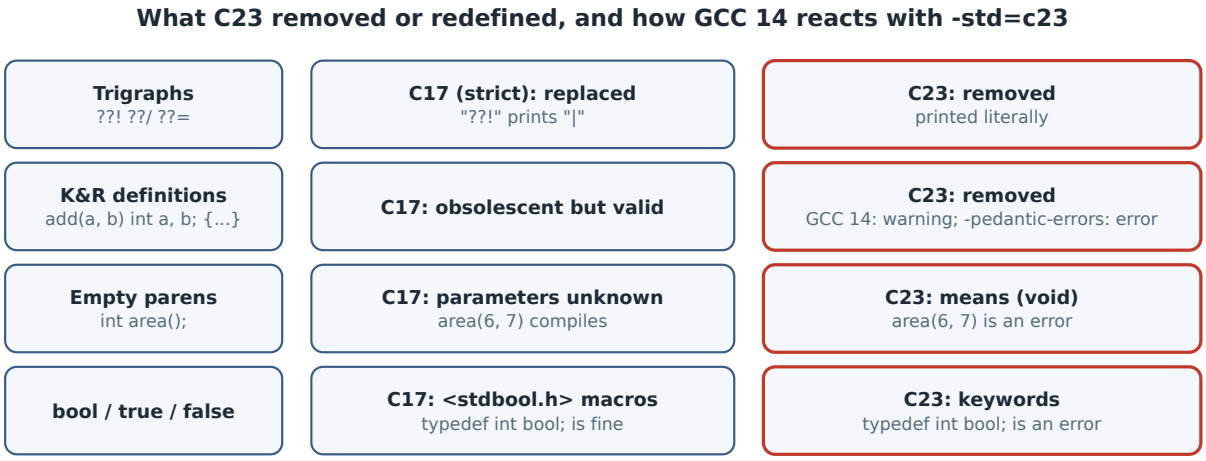


Figure 1: What C23 removed or redefined, and how GCC 14 reacts with -std=c23

The box has GCC 14.2.0, which supports `-std=c23` but still defaults to `gnu17`. Every result below uses an explicit `-std=` flag, so it matches what GCC 15's new default does for the C23 changes. GCC 15 was not run here.

The test files

```
/* trigraph.c */
#include <stdio.h>
int main(void) {
    printf("Really??! (and a path: C:??/temp)\n");
    return 0;
}
```

```
/* knr.c */
#include <stdio.h>
int add(a, b)
    int a, b;
{
    return a + b;
}
int main(void) { printf("%d\n", add(2, 3)); return 0; }
```

```
/* emptyparens.c */
#include <stdio.h>
int area();                         /* "unknown parameters" before C23 */
int main(void) {
    printf("%d\n", area(6, 7));
    return 0;
}
int area(int w, int h) { return w * h; }
```

```
/* booltypedef.c */
#include <stdio.h>
typedef int bool;
#define true 1
#define false 0
int main(void) { bool ok = true; printf("ok=%d\n", ok); return 0; }
```

(The filename comment above each listing isn't part of the compiled file. Line numbers in the diagnostics count from the `#include`.)

```
#!/bin/sh
# compile each file as strict C17 and as C23 with GCC 14
for f in trigraph knr emptyparens booltypedef; do
  for std in c17 c23; do
    echo "\$ gcc -std=$std -pedantic -Wall $f.c -o $f && ./$f"
    gcc -std=$std -pedantic -Wall $f.c -o $f 2>&1 && ./$f
    echo "[exit $?]"
  done
```

```
done
```

```
$ gcc -std=c17 -pedantic -Wall trigraph.c -o trigraph && ./trigraph
trigraph.c: In function ‘main’:
trigraph.c:3:19: warning: trigraph ??! converted to | [-Wtrigraphs]
    3 |     printf("Really??! (and a path: C:??/temp)\n");
trigraph.c:3:36: warning: trigraph ??/ converted to \ [-Wtrigraphs]
Really| (and a path: C: emp)
[exit 0]
$ gcc -std=c23 -pedantic -Wall trigraph.c -o trigraph && ./trigraph
trigraph.c: In function ‘main’:
trigraph.c:3:19: warning: trigraph ??! ignored, use -trigraphs to enable [-Wtrigraphs]
    3 |     printf("Really??! (and a path: C:??/temp)\n");
trigraph.c:3:38: warning: trigraph ??/ ignored, use -trigraphs to enable [-Wtrigraphs]
Really??! (and a path: C:??/temp)
[exit 0]
$ gcc -std=c17 -pedantic -Wall knr.c -o knr && ./knr
5
[exit 0]
$ gcc -std=c23 -pedantic -Wall knr.c -o knr && ./knr
knr.c: In function ‘add’:
knr.c:2:5: warning: old-style function definition [-Wold-style-definition]
    2 | int add(a, b)
      |     ^~~
5
[exit 0]
$ gcc -std=c17 -pedantic -Wall emptyparens.c -o emptyparens && ./emptyparens
42
[exit 0]
$ gcc -std=c23 -pedantic -Wall emptyparens.c -o emptyparens && ./emptyparens
emptyparens.c: In function ‘main’:
emptyparens.c:4:20: error: too many arguments to function ‘area’
    4 |     printf("%d\n", area(6, 7));
      |                    ^~~~
emptyparens.c:2:5: note: declared here
    2 | int area();                     /* "unknown parameters" before C23 */
      |     ^~~~
emptyparens.c: At top level:
emptyparens.c:7:5: error: conflicting types for ‘area’; have ‘int(int,  int)’
    7 | int area(int w, int h) { return w * h; }
      |     ^~~~
emptyparens.c:2:5: note: previous declaration of ‘area’ with type ‘int(void)’
    2 | int area();                     /* "unknown parameters" before C23 */
      |     ^~~~
[exit 1]
$ gcc -std=c17 -pedantic -Wall booltypedef.c -o booltypedef && ./booltypedef
```

```
ok=1
[exit 0]
$ gcc -std=c23 -pedantic -Wall booltypedef.c -o booltypedef && ./booltypedef
booltypedef.c:2:13: error: two or more data types in declaration specifiers
    2 | typedef int bool;
      |             ^~~~
booltypedef.c:2:1: warning: useless type name in empty declaration
    2 | typedef int bool;
      | ^~~~~~~
[exit 1]
```

1. Trigraphs are gone (N2940)

Trigraphs such as `??=`, `??/` and `??!` were added in C89 for keyboards without `#`, `\` and `|`. They are replaced in translation phase 1, *inside string literals too*. In strict C17, `"Really??!"` prints `Really|`, and `C:??/temp` turns into `C:` followed by a **tab** and `emp`, because `??/` became `\` and then `\t` became an escape. In C23 both strings print as written.

GNU dialects have ignored trigraphs all along unless you pass `-trigraphs`:

```
#!/bin/sh
# GNU dialects ignore trigraphs; K&R definitions under -pedantic-errors; the C23 fix
echo '$ gcc -std=gnu17 trigraph.c -o trigraph && ./trigraph'
gcc -std=gnu17 trigraph.c -o trigraph 2>&1 && ./trigraph
echo '$ gcc -std=c23 -pedantic-errors knr.c -o knr'
gcc -std=c23 -pedantic-errors knr.c -o knr 2>&1; echo "[exit $?]"
echo '$ gcc -std=c17 -Wstrict-prototypes -c emptyparens.c'
gcc -std=c17 -Wstrict-prototypes -c emptyparens.c 2>&1; echo "[exit $?]"
```

```
$ gcc -std=gnu17 trigraph.c -o trigraph && ./trigraph
trigraph.c: In function ‘main’:
trigraph.c:3:19: warning: trigraph ??! ignored, use -trigraphs to enable [-Wtrigraphs]
    3 |     printf("Really??! (and a path: C:??/temp)\n");
trigraph.c:3:38: warning: trigraph ??/ ignored, use -trigraphs to enable [-Wtrigraphs]
Really??! (and a path: C:??/temp)
$ gcc -std=c23 -pedantic-errors knr.c -o knr
knr.c: In function ‘add’:
knr.c:2:5: error: old-style function definition [-Wold-style-definition]
    2 | int add(a, b)
      |     ^~~
[exit 1]
$ gcc -std=c17 -Wstrict-prototypes -c emptyparens.c
emptyparens.c:2:1: warning: function declaration isn’t a prototype [-Wstrict-prototypes]
    2 | int area();                         /* "unknown parameters" before C23 */
      | ^~~
```

```
[exit 0]
```

Who is affected: code built with strict `-std=c89/c99/c11/c17` that contains `??` followed by one of `=/'()!<>-` in a string or comment. In practice that is usually accidental, like `"what??!"` in a log message, and C23 fixes it rather than breaking it. Code that really relies on trigraphs (EBCDIC-era sources) must keep an older `-std` or pass `-trigraphs`.

2. K&R function definitions are removed (N2432)

`int add(a, b) int a, b; { ... }` is no longer C. **GCC 14 doesn't reject it in C23 mode by default.** It accepts it as an extension and prints `-Wold-style-definition`. It becomes an error only with `-pedantic-errors` (second block above). Other compilers may already reject it, and GCC's leniency is not something to depend on.

The fix is mechanical: move the types into the parameter list, `int add(int a, int b)`. With `-Wold-style-definition` turned on, GCC finds every remaining case.

3. `()` now means `(void)` (N2841)

This change has the biggest effect. Before C23, `int area();` declared a function with **unknown** parameters. Calls weren't type-checked and every argument went through default promotions. In C23 it declares a function that takes **no** arguments, so `area(6, 7)` fails with "too many arguments", and the later definition `int area(int w, int h)` fails with "conflicting types … previous declaration … with type `int(void)`".

The GCC 15 porting guide gives two common real-world cases: a `void (*handler)();` holding the return value of `signal()`, and hand-written `void *malloc();` declarations that should be an `#include`. You can find these in C17 code before switching with `-Wstrict-prototypes` (last command above), which flags every non-prototype declaration.

4. `bool`, `true`, `false` are keywords (N2934)

C23 makes `bool`, `true`, `false`, `alignas`, `alignof`, `static_assert` and `thread_local` real keywords, and adds the new keyword `nullptr`. Projects that predate `<stdbool.h>` usually have their own `typedef int bool;`. GCC 14 rejects that with "two or more data types in declaration specifiers", and the porting guide shows GCC 15 adding the note " 'bool' is a keyword with '-std=c23' onwards". Delete the typedef and the macros. If the code has to build as both C17 and C23, guard them with `#if __STDC_VERSION__ < 202311L`.

Also changed: representation and `realloc(p, 0)`

- **Two's complement only** (N2412). Ones' complement and sign-magnitude are no longer allowed for signed integers. No real compiler is affected. The point is that code may now *assume* two's complement representation. Signed overflow is **still undefined behaviour**.
- **`realloc(p, 0)` is undefined behaviour** (N2464). Before C23 it was implementation-defined, and implementations disagreed about whether it frees `p` and what it returns. Write `free(p); p = NULL;` explicitly.

A migration checklist

1. Build with your current `-std` plus `-Wstrict-prototypes -Wold-style-definition -Wtrigraphs`, and fix every warning. All of these fixes are valid C17 as well.
2. Search for `typedef .* bool`, `#define true` and `#define false`, and guard them with `__STDC_VERSION__`.
3. Search for `realloc(` calls whose size can be zero.
4. Build with `-std=c23 -pedantic-errors` in CI. If you aren't ready, **pin** `-std=gnu17` in your build files so a GCC 15 upgrade doesn't change the language under you.

Takeaways

- C23 removes trigraphs and K&R definitions, makes `()` mean `(void)`, and turns `bool`/`true`/`false` into keywords.
- The `()` change and the `bool` keyword break real builds. Trigraph removal usually fixes accidental bugs.
- GCC 14 still accepts K&R definitions in `-std=c23` with only a warning, so use `-pedantic-errors` to enforce the standard.
- GCC 15 defaults to `gnu23`. Either port your code or pin `-std=` explicitly.

Sources

- WG14 N2940 *Removing trigraphs??!*, N2432 *Remove support for function definitions with identifier lists*, N2841 *No function declarators without prototypes*, N2934 *Revise spelling of keywords*, N2412 *Two's complement sign representation*, N2464 *Zero-size Reallocations are Undefined Behavior*
- C23 working draft N3220
- GCC, Porting to GCC 15 ("C23 by default") and GCC 15 changes

Modern C Development Practices

Introduction

Modern C development has evolved significantly from traditional approaches, incorporating contemporary software engineering practices, DevOps methodologies, and cloud-native technologies. Today's C developers work with sophisticated toolchains, containerized environments, continuous integration systems, and modern deployment strategies.

This chapter explores the current landscape of C development, including DevOps integration, containerized development environments, WebAssembly compilation, cloud-native applications, and modern testing practices. Understanding these practices is essential for professional C development in 2026 and beyond.

DevOps and C: CI/CD Pipelines

Continuous Integration and Continuous Deployment (CI/CD) pipelines are crucial for modern software development, ensuring code quality, automated testing, and reliable deployments.

GitHub Actions for C Projects

```
# .github/workflows/build.yml
name: C CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Install dependencies
      run: |
```

```
        sudo apt-get update
        sudo apt-get install -y build-essential cmake valgrind

    - name: Configure CMake
      run: cmake -B ${{github.workspace}}/build -DCMAKE_BUILD_TYPE=${{env.BUILD_TYPE}}

    - name: Build
      run: cmake --build ${{github.workspace}}/build --config ${{env.BUILD_TYPE}}

    - name: Test
      working-directory: ${{github.workspace}}/build
      run: ctest -C ${{env.BUILD_TYPE}}

    - name: Static Analysis
      run: |
        sudo apt-get install -y cppcheck
        cppcheck --enable=all --inconclusive src/

    - name: Memory Check
      run: |
        cd ${{github.workspace}}/build
        valgrind --leak-check=full --error-exitcode=1 ./my_program
```

GitLab CI for C Projects

```
# .gitlab-ci.yml
stages:
  - build
  - test
  - analyze
  - deploy

variables:
  BUILD_TYPE: Release

before_script:
  - apt-get update && apt-get install -y build-essential cmake

build_job:
  stage: build
  script:
    - cmake -B build -DCMAKE_BUILD_TYPE=$BUILD_TYPE
    - cmake --build build --config $BUILD_TYPE
  artifacts:
    paths:
```

```
      - build/

test_job:
  stage: test
  script:
    - cd build
    - ctest -C $BUILD_TYPE
  dependencies:
    - build_job

analyze_job:
  stage: analyze
  script:
    - apt-get install -y cppcheck
    - cppcheck --enable=all --inconclusive src/
  allow_failure: true

deploy_job:
  stage: deploy
  script:
    - echo "Deploying application..."
    - # Deployment commands here
  only:
    - main
```

Containerized Development

Containerization provides consistent development environments, simplifies dependency management, and enables reproducible builds.

Docker for C Development

```
# Dockerfile for C development environment
FROM ubuntu:22.04

# Install build tools and dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    gcc \
    g++ \
    gdb \
    valgrind \
    clang \
```

```
    clang-tools \
    cppcheck \
    git \
    vim \
    && rm -rf /var/lib/apt/lists/*

# Install additional tools
RUN apt-get update && apt-get install -y \
    lcov \
    doxygen \
    graphviz \
    && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR /app

# Copy project files
COPY . .

# Default command
CMD ["/bin/bash"]
```

Multi-stage Docker Build

```
# Multi-stage Dockerfile for C application
# Build stage
FROM ubuntu:22.04 AS builder

# Install build dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR /app

# Copy source code
COPY src/ src/
COPY include/ include/
COPY CMakeLists.txt .

# Build the application
RUN cmake -B build -DCMAKE_BUILD_TYPE=Release
RUN cmake --build build --config Release
```

```
# Runtime stage
FROM ubuntu:22.04

# Install runtime dependencies only
RUN apt-get update && apt-get install -y \
    libssl-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy built executable from builder stage
COPY --from=builder /app/build/myapp /usr/local/bin/myapp

# Create non-root user
RUN useradd -m -s /bin/bash appuser
USER appuser

# Expose port if needed
EXPOSE 8080

# Run the application
CMD ["myapp"]
```

Docker Compose for Development

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build: .
    volumes:
      - .:/app
      - /app/build
    working_dir: /app
    command: bash -c "cmake -B build && cmake --build build && ./build/myapp"
    environment:
      - DEBUG=1
    ports:
      - "8080:8080"
    depends_on:
      - database
      - cache

  database:
    image: postgres:15
    environment:
```

```
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  cache:
    image: redis:7
    ports:
      - "6379:6379"

  test:
    build: .
    volumes:
      - .:/app
    working_dir: /app
    command: bash -c "cmake -B build && cmake --build build && ctest -V"
    depends_on:
      - database
      - cache

volumes:
  db_data:
```

Cross-compilation

Modern C development often requires building for multiple target platforms and architectures.

CMake Cross-compilation

```
# CMakeLists.txt with cross-compilation support
cmake_minimum_required(VERSION 3.20)
project(MyApp C)

# Set C standard
set(CMAKE_C_STANDARD 23)
set(CMAKE_C_STANDARD_REQUIRED ON)

# Compiler-specific flags
if(CMAKE_C_COMPILER_ID STREQUAL "GNU")
    add_compile_options(-Wall -Wextra -Wpedantic)
elseif(CMAKE_C_COMPILER_ID STREQUAL "Clang")
```

```
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# Conditional compilation for different platforms
if(WIN32)
    add_compile_definitions(PLATFORM_WINDOWS)
elseif(APPLE)
    add_compile_definitions(PLATFORM_MACOS)
elseif(UNIX)
    add_compile_definitions(PLATFORM_LINUX)
endif()

# Source files
set(SOURCES
    src/main.c
    src/utils.c
    src/network.c
)

# Create executable
add_executable(myapp ${SOURCES})

# Link libraries based on platform
if(WIN32)
    target_link_libraries(myapp ws2_32)
elseif(UNIX)
    target_link_libraries(myapp pthread m)
endif()

# Installation rules
install(TARGETS myapp DESTINATION bin)
```

Cross-compilation Toolchain File

```
# arm-linux-gnueabihf.cmake
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR arm)

set(CMAKE_C_COMPILER arm-linux-gnueabihf-gcc)
set(CMAKE_CXX_COMPILER arm-linux-gnueabihf-g++)

set(CMAKE_FIND_ROOT_PATH /usr/arm-linux-gnueabihf)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
```

```
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
```

WebAssembly (WASM)

Compiling C to WebAssembly enables running C applications in web browsers and other WASM environments.

Emscripten Setup

```
# Install Emscripten
git clone https://github.com/emscripten-core/emsdk.git
cd emsdk
./emsdk install latest
./emsdk activate latest
source ./emsdk_env.sh
```

Simple C to WASM Example

```
// math_utils.c
#include <emscripten.h>

EMSCRIPTEN_KEEPALIVE
int add(int a, int b) {
    return a + b;
}

EMSCRIPTEN_KEEPALIVE
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

EMSCRIPTEN_KEEPALIVE
double calculate_circle_area(double radius) {
    return 3.14159 * radius * radius;
}
```

```
# Compile to WebAssembly
emcc math_utils.c -o math_utils.js \
    -s EXPORTED_FUNCTIONS='["_add", "_fibonacci", "_calculate_circle_area"]' \
    -s EXPORTED_RUNTIME_METHODS='["ccall", "cwrap"]' \
    -s MODULARIZE=1 \
```

```
    -s EXPORT_NAME="MathUtils"
```

HTML Integration

```
<!DOCTYPE html>
<html>
<head>
    <title>C WebAssembly Demo</title>
</head>
<body>
    <h1>C WebAssembly Math Demo</h1>
    <div id="output"></div>

    <script src="math_utils.js"></script>
    <script>
        MathUtils().then(function(Module) {
            // Wrap C functions
            const add = Module.cwrap('add', 'number', ['number', 'number']);
            const fibonacci = Module.cwrap('fibonacci', 'number', ['number']);
            const calculate_circle_area = Module.cwrap('calculate_circle_area', 'number', ['

            // Use the functions
            document.getElementById('output').innerHTML = `
                <p>5 + 3 = ${add(5, 3)}</p>
                <p>Fibonacci(10) = ${fibonacci(10)}</p>
                <p>Circle area (radius=5) = ${calculate_circle_area(5)}</p>
            `;
        });
    </script>
</body>
</html>
```

Cloud-native C Applications

Cloud-native development involves building applications that are designed to run in cloud environments, leveraging microservices, containers, and orchestration platforms.

Simple HTTP Server in C

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8080
#define BUFFER_SIZE 1024

void handle_request(int client_fd) {
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    // Read HTTP request
    bytes_read = read(client_fd, buffer, BUFFER_SIZE - 1);
    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';

        // Simple HTTP response
        const char *response =
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html\r\n"
            "Connection: close\r\n"
            "\r\n"
            "<html><body><h1>Hello from C!</h1>"
            "<p>Current time: %s</p>"
            "</body></html>";

        // Get current time
        time_t now = time(NULL);
        char *time_str = ctime(&now);
        time_str[strlen(time_str) - 1] = '\0'; // Remove newline

        // Send response
        char response_buffer[BUFFER_SIZE * 2];
        snprintf(response_buffer, sizeof(response_buffer), response, time_str);
        write(client_fd, response_buffer, strlen(response_buffer));
    }

    close(client_fd);
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
```

```
int addrlen = sizeof(address);

// Create socket
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Set socket options
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
               &opt, sizeof(opt))) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}

// Configure address
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind socket
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen for connections
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d\n", PORT);

// Main server loop
while (1) {
    // Accept connection
    if ((client_fd = accept(server_fd, (struct sockaddr *)&address,
                            (socklen_t*)&addrlen)) < 0) {
        perror("accept");
        continue;
    }

    // Handle request
    handle_request(client_fd);
}
```

```
    return 0;
}
```

Dockerfile for Cloud Deployment

```
# Dockerfile for cloud-native C application
FROM alpine:latest

# Install build tools
RUN apk add --no-cache build-base

# Set working directory
WORKDIR /app

# Copy source code
COPY server.c .

# Build the application
RUN gcc -o server server.c

# Create non-root user
RUN adduser -D -s /bin/sh appuser

# Switch to non-root user
USER appuser

# Expose port
EXPOSE 8080

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:8080/ || exit 1

# Run the application
CMD ["./server"]
```

Supply Chain Security

Modern C development requires attention to supply chain security, dependency management, and vulnerability scanning.

Conan Package Manager

```
# conanfile.txt
[requires]
zlib/1.2.13
openssl/3.1.0
libcurl/8.0.1

[generators]
CMakeDeps
CMakeToolchain

[options]
openssl:shared=False
zlib:shared=False
```

```
# CMakeLists.txt with Conan integration
cmake_minimum_required(VERSION 3.20)
project(MyApp C)

# Include Conan-generated files
include(${CMAKE_BINARY_DIR}/conan_toolchain.cmake)
include(${CMAKE_BINARY_DIR}/conan_deps.cmake)

# Find packages
find_package(ZLIB REQUIRED)
find_package(OpenSSL REQUIRED)
find_package(CURL REQUIRED)

# Create executable
add_executable(myapp src/main.c)

# Link libraries
target_link_libraries(myapp
    ZLIB::ZLIB
    OpenSSL::SSL
    CURL::libcurl
)
```

Vulnerability Scanning

```
# Using OSV-Scanner for vulnerability detection
osv-scanner --lockfile=conan.lock

# Using Snyk for security scanning
```

```
snyk test --file=conanfile.txt

# Using GitHub Dependabot (configured in .github/dependabot.yml)
```

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "gitsubmodule"
    directory: "/"
    schedule:
      interval: "weekly"

  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"
```

Modern Testing Practices

Contemporary C development incorporates advanced testing methodologies including fuzzing, property-based testing, and mutation testing.

Unity Testing Framework

```
// test_math.c
#include "unity.h"
#include "math_utils.h"

void setUp(void) {
    // Set up before each test
}

void tearDown(void) {
    // Clean up after each test
}

void test_addition_positive_numbers(void) {
    TEST_ASSERT_EQUAL_INT(5, add(2, 3));
    TEST_ASSERT_EQUAL_INT(0, add(-5, 5));
}

void test_addition_negative_numbers(void) {
    TEST_ASSERT_EQUAL_INT(-5, add(-2, -3));
```

```
    TEST_ASSERT_EQUAL_INT(-1, add(2, -3));
}

void test_fibonacci_base_cases(void) {
    TEST_ASSERT_EQUAL_INT(0, fibonacci(0));
    TEST_ASSERT_EQUAL_INT(1, fibonacci(1));
}

void test_fibonacci_recursive_cases(void) {
    TEST_ASSERT_EQUAL_INT(1, fibonacci(2));
    TEST_ASSERT_EQUAL_INT(2, fibonacci(3));
    TEST_ASSERT_EQUAL_INT(3, fibonacci(4));
    TEST_ASSERT_EQUAL_INT(5, fibonacci(5));
}

int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_addition_positive_numbers);
    RUN_TEST(test_addition_negative_numbers);
    RUN_TEST(test_fibonacci_base_cases);
    RUN_TEST(test_fibonacci_recursive_cases);
    return UNITY_END();
}
```

Property-Based Testing with CMocka

```
// property_test.c
#include <stdarg.h>
#include <stddef.h>
#include <setjmp.h>
#include <cmocka.h>
#include <stdlib.h>

// Property: Addition is commutative
static void test_addition_commutative(void **state) {
    (void) state; // unused

    for (int i = 0; i < 100; i++) {
        int a = rand() % 1000 - 500;  // -500 to 499
        int b = rand() % 1000 - 500;  // -500 to 499

        assert_int_equal(add(a, b), add(b, a));
    }
}
```

```
// Property: Adding zero is identity
static void test_addition_identity(void **state) {
    (void) state; // unused

    for (int i = 0; i < 100; i++) {
        int a = rand() % 1000 - 500;  // -500 to 499

        assert_int_equal(a, add(a, 0));
        assert_int_equal(a, add(0, a));
    }
}

int main(void) {
    const struct CMUnitTest tests[] = {
        cmocka_unit_test(test_addition_commutative),
        cmocka_unit_test(test_addition_identity),
    };

    return cmocka_run_group_tests(tests, NULL, NULL);
}
```

Fuzzing with AFL

```
// fuzz_target.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Function to fuzz test
int parse_config(const char *input, size_t len) {
    if (len < 4) return -1;

    // Simple parsing logic
    if (input[0] == 'C' && input[1] == 'F' && input[2] == 'G') {
        int value = 0;
        for (size_t i = 3; i < len && i < 10; i++) {
            if (input[i] >= '0' && input[i] <= '9') {
                value = value * 10 + (input[i] - '0');
            } else {
                break;
            }
        }
        return value;
    }
```

```
    return -1;
}

#ifdef __AFL_HAVE_MANUAL_CONTROL
int main() {
    __AFL_INIT();

    unsigned char *input = __AFL_FUZZ_TESTCASE_BUF;
    while (__AFL_LOOP(10000)) {
        size_t len = __AFL_FUZZ_TESTCASE_LEN;
        parse_config((char*)input, len);
    }

    return 0;
}
#else
int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <input_file>\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "rb");
    if (!fp) {
        perror("fopen");
        return 1;
    }

    fseek(fp, 0, SEEK_END);
    long len = ftell(fp);
    fseek(fp, 0, SEEK_SET);

    char *input = malloc(len);
    if (!input) {
        fclose(fp);
        return 1;
    }

    fread(input, 1, len, fp);
    fclose(fp);

    parse_config(input, len);
    free(input);

    return 0;
}
#endif
```

```
# Build for fuzzing
AFL_USE_ASAN=1 afl-gcc -o fuzz_target fuzz_target.c

# Create input directory
mkdir -p input
echo "CFG123" > input/sample.cfg

# Run fuzzer
afl-fuzz -i input -o output -- ./fuzz_target @@
```

Documentation as Code

Modern C projects integrate documentation generation into the development workflow.

Doxygen Configuration

```
# Doxyfile
PROJECT_NAME = "My C Application"
PROJECT_VERSION = "1.0.0"
PROJECT_BRIEF = "A modern C application with comprehensive documentation"

INPUT = src include
RECURSIVE = YES

GENERATE_HTML = YES
GENERATE_LATEX = NO
GENERATE_MAN = NO

EXTRACT_ALL = YES
EXTRACT_PRIVATE = NO
EXTRACT_STATIC = YES

WARN_IF_UNDOCUMENTED = YES
WARN_IF_DOC_ERROR = YES

FILE_PATTERNS = *.c *.h
RECURSIVE = YES

HAVE_DOT = YES
CALL_GRAPH = YES
CALLER_GRAPH = YES

GENERATE_TREEVIEW = YES
DISABLE_INDEX = NO
```

```
HTML_OUTPUT = docs/html
HTML_FILE_EXTENSION = .html
```

Example Documentation

```
/**
 * @file math_utils.h
 * @brief Mathematical utility functions
 * @author John Doe
 * @date 2024
 */

#ifndef MATH_UTILS_H
#define MATH_UTILS_H

/**
 * @brief Adds two integers
 * @param a First integer
 * @param b Second integer
 * @return Sum of a and b
 * @note This function handles both positive and negative integers
 * @warning Integer overflow is not checked
 *
 * Example usage:
 * @code
 * int result = add(5, 3);  // result = 8
 * @endcode
 */
int add(int a, int b);

/**
 * @brief Calculates the nth Fibonacci number
 * @param n The position in the Fibonacci sequence (0-indexed)
 * @return The nth Fibonacci number
 * @pre n >= 0
 * @post Return value >= 0
 *
 * @details This function uses a recursive approach for simplicity.
 * For large values of n, consider using an iterative implementation.
 *
 * Time complexity: O(2^n)
 * Space complexity: O(n)
 */
int fibonacci(int n);
```

```
#endif // MATH_UTILS_H
```

Practical Examples

Modern C Project Structure

```
my-c-project/
   src/
       main.c
       utils.c
       utils.h
   include/
       config.h
   tests/
       test_utils.c
       CMakeLists.txt
   docs/
       Doxyfile
       README.md
   cmake/
       FindMyLib.cmake
   .github/
       workflows/
           ci.yml
   docker/
       Dockerfile
       docker-compose.yml
   CMakeLists.txt
   conanfile.txt
   .gitignore
   README.md
```

Complete CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(MyCProject C)

# Set C standard to C23
set(CMAKE_C_STANDARD 23)
set(CMAKE_C_STANDARD_REQUIRED ON)

# Compiler options
if(CMAKE_C_COMPILER_ID STREQUAL "GNU" OR CMAKE_C_COMPILER_ID STREQUAL "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic -Werror)
```

```
    # Enable sanitizers in debug mode
    if(CMAKE_BUILD_TYPE STREQUAL "Debug")
        add_compile_options(-fsanitize=address -fsanitize=undefined)
        add_link_options(-fsanitize=address -fsanitize=undefined)
    endif()
endif()

# Include directories
include_directories(include)

# Source files
set(SOURCES
    src/main.c
    src/utils.c
)

# Create executable
add_executable(${PROJECT_NAME} ${SOURCES})

# Link libraries
target_link_libraries(${PROJECT_NAME} m)

# Enable testing
enable_testing()

# Add test subdirectory
add_subdirectory(tests)

# Installation
install(TARGETS ${PROJECT_NAME} DESTINATION bin)
install(DIRECTORY include/ DESTINATION include)

# Packaging
set(CPACK_PACKAGE_NAME "my-c-project")
set(CPACK_PACKAGE_VERSION "1.0.0")
set(CPACK_PACKAGE_DESCRIPTION "A modern C project")
set(CPACK_GENERATOR "TGZ;DEB;RPM")
include(CPack)
```

Summary

Modern C development practices encompass a wide range of contemporary software engineering methodologies:

1. **DevOps and CI/CD**: Automated building, testing, and deployment pipelines

2. **Containerized Development**: Consistent environments with Docker and containerization
3. **Cross-compilation**: Building for multiple target platforms and architectures
4. **WebAssembly**: Compiling C to run in web browsers and WASM environments
5. **Cloud-native Applications**: Designing applications for cloud deployment and orchestration
6. **Supply Chain Security**: Managing dependencies and scanning for vulnerabilities
7. **Modern Testing**: Fuzzing, property-based testing, and mutation testing
8. **Documentation as Code**: Integrated documentation generation and maintenance

These practices enable C developers to build robust, secure, and maintainable applications while leveraging modern development tools and methodologies. As C continues to evolve with new standards like C23, these practices become increasingly important for professional C development.

Advanced Preprocessor

Introduction

The C preprocessor is a powerful text processing tool that operates before the actual compilation begins. While often overlooked, mastering advanced preprocessor techniques can lead to more maintainable, efficient, and feature-rich C code. Modern C standards have enhanced preprocessor capabilities, making it even more versatile for metaprogramming and code generation.

This chapter explores advanced preprocessor features including macro definitions, conditional compilation, pragma directives, and token manipulation techniques that enable sophisticated code generation and platform-specific optimizations.

Macro Definitions and Function-like Macros

Macros are the most fundamental preprocessor feature, allowing for text substitution and code generation.

Advanced Macro Techniques

```
#include <stdio.h>
#include <stdlib.h>

// Stringification operator (#)
#define STRINGIFY(x) #x
#define PRINT_VAR(var) printf(#var " = %d\n", var)

// Token pasting operator (##)
#define CONCAT(a, b) a##b
#define DECLARE_VAR(type, name) type CONCAT(my_, name)

// Variadic macros (C99)
#define DEBUG_PRINT(fmt, ...) \
    fprintf(stderr, "[DEBUG] %s:%d: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)

#define LOG(level, fmt, ...) \
    printf("[" #level "] " fmt "\n", ##__VA_ARGS__)

// Macro overloading based on number of arguments
```

```c
#define GET_MACRO(_1, _2, _3, NAME, ...) NAME
#define ADD2(a, b) ((a) + (b))
#define ADD3(a, b, c) ((a) + (b) + (c))
#define ADD(...) GET_MACRO(__VA_ARGS__, ADD3, ADD2)(__VA_ARGS__)

int main() {
    int x = 42;

    // Stringification
    printf("Value of x: %s\n", STRINGIFY(x));
    PRINT_VAR(x);

    // Token pasting
    DECLARE_VAR(int, counter) = 100;
    printf("my_counter = %d\n", my_counter);

    // Variadic macros
    DEBUG_PRINT("This is a debug message");
    DEBUG_PRINT("Value of x is %d", x);
    LOG(INFO, "Application started");
    LOG(ERROR, "Failed to open file: %s", "config.txt");

    // Macro overloading
    printf("ADD(1, 2) = %d\n", ADD(1, 2));
    printf("ADD(1, 2, 3) = %d\n", ADD(1, 2, 3));

    return 0;
}
```

Generic Programming with Macros

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Generic swap macro
#define SWAP(a, b) do { \
    typeof(a) temp = (a); \
    (a) = (b); \
    (b) = temp; \
} while(0)

// Generic maximum macro
#define MAX(a, b) ({ \
    typeof(a) _a = (a); \
```

```
    typeof(b) _b = (b); \
    _a > _b ? _a : _b; \
})

// Generic container_of macro (similar to Linux kernel)
#define container_of(ptr, type, member) ({ \
    const typeof(((type *)0)->member) * __mptr = (ptr); \
    (type *)((char *)__mptr - offsetof(type, member)); \
})

#define offsetof(type, member) ((size_t) &((type *)0)->member)

typedef struct {
    int id;
    char name[50];
    double value;
} Item;

int main() {
    // Generic swap
    int x = 10, y = 20;
    printf("Before swap: x=%d, y=%d\n", x, y);
    SWAP(x, y);
    printf("After swap: x=%d, y=%d\n", x, y);

    double a = 3.14, b = 2.71;
    printf("Before swap: a=%.2f, b=%.2f\n", a, b);
    SWAP(a, b);
    printf("After swap: a=%.2f, b=%.2f\n", a, b);

    // Generic maximum
    printf("MAX(10, 20) = %d\n", MAX(10, 20));
    printf("MAX(3.14, 2.71) = %.2f\n", MAX(3.14, 2.71));

    // Container of example
    Item item = {1, "Test Item", 99.99};
    char *name_ptr = item.name;

    // Get the container from a member pointer
    Item *container = container_of(name_ptr, Item, name);
    printf("Container ID: %d, Name: %s, Value: %.2f\n",
           container->id, container->name, container->value);

    return 0;
}
```

Conditional Compilation

Conditional compilation allows you to include or exclude code based on preprocessor conditions, enabling platform-specific code and feature toggles.

Platform-Specific Compilation

```
#include <stdio.h>

// Platform detection
#if defined(_WIN32) || defined(_WIN64)
    #define PLATFORM_WINDOWS
    #include <windows.h>
#elif defined(__linux__)
    #define PLATFORM_LINUX
    #include <unistd.h>
#elif defined(__APPLE__)
    #define PLATFORM_MACOS
    #include <unistd.h>
#endif

// Compiler detection
#if defined(__GNUC__)
    #define COMPILER_GCC
    #define NORETURN __attribute__((noreturn))
    #define UNUSED __attribute__((unused))
#elif defined(_MSC_VER)
    #define COMPILER_MSVC
    #define NORETURN __declspec(noreturn)
    #define UNUSED
#endif

// Feature detection
#if __STDC_VERSION__ >= 202311L
    #define C23_AVAILABLE
    #define HAS_TYPEOF typeof
#elif __STDC_VERSION__ >= 201112L
    #define C11_AVAILABLE
    #define HAS_TYPEOF __typeof__
#else
    #define C99_AVAILABLE
#endif

// Debug vs Release builds
#ifdef DEBUG
    #define DBG_PRINT(fmt, ...) fprintf(stderr, "[DEBUG] " fmt "\n", ##__VA_ARGS__)
```

```
    #define ASSERT(condition) do { \
        if (!(condition)) { \
            fprintf(stderr, "Assertion failed: %s at %s:%d\n", \
                    #condition, __FILE__, __LINE__); \
            abort(); \
        } \
    } while(0)
#else
    #define DBG_PRINT(fmt, ...) do {} while(0)
    #define ASSERT(condition) do {} while(0)
#endif

// Feature toggles
#define FEATURE_NETWORKING 1
#define FEATURE_DATABASE 0

#if FEATURE_NETWORKING
    #include <sys/socket.h>
    void init_networking(void) {
        printf("Networking initialized\n");
    }
#else
    void init_networking(void) {
        printf("Networking disabled\n");
    }
#endif

#if FEATURE_DATABASE
    #include <sqlite3.h>
    void init_database(void) {
        printf("Database initialized\n");
    }
#else
    void init_database(void) {
        printf("Database disabled\n");
    }
#endif

int main() {
    printf("Platform: ");
#if defined(PLATFORM_WINDOWS)
    printf("Windows\n");
#elif defined(PLATFORM_LINUX)
    printf("Linux\n");
#elif defined(PLATFORM_MACOS)
    printf("macOS\n");
#else
```

```
    printf("Unknown\n");
#endif

    printf("Compiler: ");
#ifdef COMPILER_GCC
    printf("GCC\n");
#elif defined(COMPILER_MSVC)
    printf("MSVC\n");
#else
    printf("Unknown\n");
#endif

    printf("C Standard: ");
#ifdef C23_AVAILABLE
    printf("C23\n");
#elif defined(C11_AVAILABLE)
    printf("C11\n");
#else
    printf("C99 or earlier\n");
#endif

    // Debug printing
    DBG_PRINT("Debug message");
    DBG_PRINT("Value of x: %d", 42);

    // Assertions
    int x = 10;
    ASSERT(x > 0);

    // Feature initialization
    init_networking();
    init_database();

    return 0;
}
```

Version-Specific Code

```
#include <stdio.h>

// Version checking macros
#define VERSION_CHECK(major, minor, patch) \
    ((major) * 10000 + (minor) * 100 + (patch))

#define LIBRARY_VERSION VERSION_CHECK(2, 3, 1)
```

```c
// API version compatibility
#if LIBRARY_VERSION >= VERSION_CHECK(2, 0, 0)
    #define NEW_API_AVAILABLE
    void new_api_function(void) {
        printf("Using new API\n");
    }
#endif

#if LIBRARY_VERSION >= VERSION_CHECK(2, 3, 0)
    #define ENHANCED_API_AVAILABLE
    void enhanced_api_function(void) {
        printf("Using enhanced API\n");
    }
#endif

// Backward compatibility
#if LIBRARY_VERSION < VERSION_CHECK(2, 0, 0)
    void new_api_function(void) {
        printf("Using legacy implementation\n");
    }
#endif

int main() {
    printf("Library version: %d.%d.%d\n",
           LIBRARY_VERSION / 10000,
           (LIBRARY_VERSION % 10000) / 100,
           LIBRARY_VERSION % 100);

#ifdef NEW_API_AVAILABLE
    new_api_function();
#endif

#ifdef ENHANCED_API_AVAILABLE
    enhanced_api_function();
#endif

    return 0;
}
```

Pragma Directives

Pragma directives provide compiler-specific instructions and hints that can optimize code generation and control compilation behavior.

Common Pragma Directives

```c
#include <stdio.h>
#include <stdlib.h>

// Region folding (supported by many IDEs)
#pragma region Utility Functions

static inline int max(int a, int b) {
    return a > b ? a : b;
}

static inline int min(int a, int b) {
    return a < b ? a : b;
}

#pragma endregion

// Optimization hints
#pragma GCC push_options
#pragma GCC optimize("O3")

// Performance-critical function
double compute_pi(int iterations) {
    double pi = 0.0;
    double denominator = 1.0;

    for (int i = 0; i < iterations; i++) {
        if (i % 2 == 0) {
            pi += 4.0 / denominator;
        } else {
            pi -= 4.0 / denominator;
        }
        denominator += 2.0;
    }

    return pi;
}

#pragma GCC pop_options

// Warning control
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-variable"

void example_with_warnings(void) {
```

```
    int unused_var = 42;  // This warning is suppressed
    printf("Function with suppressed warnings\n");
}

#pragma GCC diagnostic pop

// Structure packing
#pragma pack(push, 1)
typedef struct {
    char a;      // 1 byte
    int b;       // 4 bytes (normally padded to offset 4)
    short c;     // 2 bytes (normally padded to offset 8)
} PackedStruct;  // Total size: 7 bytes instead of 12
#pragma pack(pop)

// Once-only inclusion (alternative to include guards)
#pragma once

// Message directives (compiler-specific)
#pragma message("Compiling with advanced preprocessor features")

int main() {
    printf("Max of 10 and 20: %d\n", max(10, 20));
    printf("Min of 10 and 20: %d\n", min(10, 20));

    double pi = compute_pi(1000000);
    printf("Computed PI: %.10f\n", pi);

    example_with_warnings();

    printf("Size of PackedStruct: %zu bytes\n", sizeof(PackedStruct));

    return 0;
}
```

Compiler-Specific Pragmas

```
#include <stdio.h>

// GCC-specific pragmas
#ifdef __GNUC__
    #pragma GCC diagnostic push
    #pragma GCC diagnostic ignored "-Wdeprecated-declarations"

    // Function inlining hints
```

```
    #pragma GCC push_options
    #pragma GCC optimize("inline-functions-called-once")

    static inline void optimized_function(void) {
        printf("This function may be inlined\n");
    }

    #pragma GCC pop_options
    #pragma GCC diagnostic pop
#endif

// MSVC-specific pragmas
#ifdef _MSC_VER
    #pragma warning(push)
    #pragma warning(disable: 4996)  // Disable deprecated function warnings

    // Function inlining
    #pragma inline_depth(255)
    #pragma inline_recursion(on)

    static __inline void msvc_optimized_function(void) {
        printf("MSVC optimized function\n");
    }

    #pragma warning(pop)
#endif

// Clang-specific pragmas
#ifdef __clang__
    #pragma clang diagnostic push
    #pragma clang diagnostic ignored "-Wdeprecated-declarations"

    // Loop optimization hints
    #pragma clang loop vectorize(enable)

    void clang_optimized_loop(int *array, int size) {
        for (int i = 0; i < size; i++) {
            array[i] *= 2;
        }
    }

    #pragma clang diagnostic pop
#endif

int main() {
#ifdef __GNUC__
    optimized_function();
```

```
#endif

#ifdef _MSC_VER
    msvc_optimized_function();
#endif

#ifdef __clang__
    int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    clang_optimized_loop(array, 10);
    printf("Array[0] after optimization: %d\n", array[0]);
#endif

    return 0;
}
```

Predefined Macros

C provides numerous predefined macros that offer information about the compilation environment.

System and Compiler Information

```
#include <stdio.h>
#include <time.h>

void print_compilation_info(void) {
    printf("=== Compilation Information ===\n");

    // Standard version
    printf("Standard version: %ld\n", __STDC_VERSION__);
#ifdef __STDC_VERSION__
    #if __STDC_VERSION__ >= 202311L
        printf("C Standard: C23\n");
    #elif __STDC_VERSION__ >= 201112L
        printf("C Standard: C11\n");
    #elif __STDC_VERSION__ >= 199901L
        printf("C Standard: C99\n");
    #else
        printf("C Standard: C89/C90\n");
    #endif
#endif

    // Compiler information
#ifdef __GNUC__
```

```
    printf("Compiler: GCC %d.%d.%d\n", __GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__);
#endif

#ifdef __clang__
    printf("Compiler: Clang %d.%d.%d\n", __clang_major__, __clang_minor__, __clang_patchleve
#endif

#ifdef _MSC_VER
    printf("Compiler: MSVC %d\n", _MSC_VER);
#endif

    // Platform information
#ifdef __x86_64__
    printf("Architecture: x86_64\n");
#elif defined(__i386__)
    printf("Architecture: x86\n");
#elif defined(__aarch64__)
    printf("Architecture: ARM64\n");
#elif defined(__arm__)
    printf("Architecture: ARM\n");
#endif

#ifdef __LP64__
    printf("Data model: LP64\n");
#endif

    // Date and time
    printf("Compilation date: %s\n", __DATE__);
    printf("Compilation time: %s\n", __TIME__);

    // File and line information
    printf("Current file: %s\n", __FILE__);
    printf("Current line: %d\n", __LINE__);
    printf("Function name: %s\n", __func__);

    printf("\n");
}

// Using predefined macros for debugging
#ifdef DEBUG
    #define TRACE() printf("TRACE: %s() in %s:%d\n", __func__, __FILE__, __LINE__)
#else
    #define TRACE() do {} while(0)
#endif

void function_a(void) {
    TRACE();
```

```
    printf("Inside function_a\n");
}

void function_b(void) {
    TRACE();
    printf("Inside function_b\n");
    function_a();
}

int main() {
    print_compilation_info();

    function_b();

    // Counter macro (implementation-defined)
    printf("This is compilation #%d\n", __COUNTER__);
    printf("This is compilation #%d\n", __COUNTER__);

    return 0;
}
```

Token Manipulation and Advanced Techniques

Advanced preprocessor techniques involve complex token manipulation and metaprogramming.

Deferred Macro Expansion

```
#include <stdio.h>

// Deferred expansion technique
#define EMPTY()
#define DEFER(id) id EMPTY()
#define OBSTRUCT(...) __VA_ARGS__ DEFER(EMPTY)()
#define EXPAND(...) __VA_ARGS__

#define EVAL(...) EVAL1(EVAL1(EVAL1(__VA_ARGS__)))
#define EVAL1(...) EVAL2(EVAL2(EVAL2(__VA_ARGS__)))
#define EVAL2(...) EVAL3(EVAL3(EVAL3(__VA_ARGS__)))
#define EVAL3(...) EVAL4(EVAL4(EVAL4(__VA_ARGS__)))
#define EVAL4(...) __VA_ARGS__

// Recursive macro example
#define WHEN(c) IF_##c
#define IF_1(t, f) t
```

```c
#define IF_0(t, f) f

#define COMMA() ,
#define SEMICOLON() ;

#define REPEAT_0(f, ...)
#define REPEAT_1(f, x, ...) f(x) REPEAT_0(f, __VA_ARGS__)
#define REPEAT_2(f, x, ...) f(x) REPEAT_1(f, __VA_ARGS__)
#define REPEAT_3(f, x, ...) f(x) REPEAT_2(f, __VA_ARGS__)
#define REPEAT_4(f, x, ...) f(x) REPEAT_3(f, __VA_ARGS__)

#define REPEAT_N(n, f, ...) EVAL(REPEAT_##n(f, __VA_ARGS__))

#define PRINT_ARG(x) printf("%s = %d\n", #x, x);

int main() {
    int a = 1, b = 2, c = 3, d = 4;

    printf("Repeating PRINT_ARG macro:\n");
    REPEAT_N(4, PRINT_ARG, a, b, c, d)

    return 0;
}
```

X-Macros Pattern

```c
#include <stdio.h>
#include <string.h>

// X-Macros for enumerations and string tables
#define COLOR_TABLE \
    X(RED, "red", 0xFF0000) \
    X(GREEN, "green", 0x00FF00) \
    X(BLUE, "blue", 0x0000FF) \
    X(YELLOW, "yellow", 0xFFFF00) \
    X(PURPLE, "purple", 0xFF00FF) \
    X(CYAN, "cyan", 0x00FFFF) \
    X(WHITE, "white", 0xFFFFFF) \
    X(BLACK, "black", 0x000000)

// Generate enumeration
#define X(name, string, value) COLOR_##name,
typedef enum {
    COLOR_TABLE
    COLOR_COUNT
```

```
} Color;
#undef X

// Generate string array
#define X(name, string, value) string,
static const char *color_strings[] = {
    COLOR_TABLE
};
#undef X

// Generate value array
#define X(name, string, value) value,
static const int color_values[] = {
    COLOR_TABLE
};
#undef X

// Generate function to get string
const char *color_to_string(Color color) {
    if (color >= 0 && color < COLOR_COUNT) {
        return color_strings[color];
    }
    return "unknown";
}

// Generate function to get value
int color_to_value(Color color) {
    if (color >= 0 && color < COLOR_COUNT) {
        return color_values[color];
    }
    return 0;
}

// Generate lookup function
Color string_to_color(const char *str) {
    for (int i = 0; i < COLOR_COUNT; i++) {
        if (strcmp(str, color_strings[i]) == 0) {
            return (Color)i;
        }
    }
    return COLOR_COUNT;  // Invalid color
}

int main() {
    printf("Color enumeration:\n");
    for (int i = 0; i < COLOR_COUNT; i++) {
```

```
        printf("  %d: %s (0x%06X)\n", i, color_to_string((Color)i), color_to_value((Color)i)
    }

    printf("\nString to color lookup:\n");
    const char *test_colors[] = {"red", "blue", "purple", "invalid"};
    for (int i = 0; i < 4; i++) {
        Color c = string_to_color(test_colors[i]);
        if (c < COLOR_COUNT) {
            printf("  %s -> %s (0x%06X)\n", test_colors[i], color_to_string(c), color_to_val
        } else {
            printf("  %s -> invalid color\n", test_colors[i]);
        }
    }

    return 0;
}
```

C23 Enhanced Preprocessor Features

C23 introduces several new preprocessor features that enhance metaprogramming capabilities.

#elifdef and #elifndef

```
#include <stdio.h>

// Define some macros for testing
#define DEBUG_MODE
// #define RELEASE_MODE
// #define TESTING_MODE

int main() {
    printf("Preprocessor conditional compilation:\n");

#if defined(DEBUG_MODE)
    printf("  Debug mode is enabled\n");
#elifdef RELEASE_MODE
    printf("  Release mode is enabled\n");
#elifndef TESTING_MODE
    printf("  Testing mode is not defined\n");
#else
    printf("  Unknown mode\n");
#endif

    return 0;
```

```
}
```

#warning Directive

```
#include <stdio.h>

// Compiler version check
#if __STDC_VERSION__ < 202311L
    #warning "This code is designed for C23 or later"
#endif

// Platform-specific warnings
#ifdef __APPLE__
    #warning "This code may need adjustments for macOS"
#endif

// Feature availability warnings
#ifndef __STDC_LIB_EXT1__
    #warning "Bounds-checking functions not available"
#endif

int main() {
    printf("Program running with warnings (if any)\n");

#ifdef __GNUC__
    #warning "Consider using Clang for better C23 support"
#endif

    return 0;
}
```

Digit Separators in Preprocessor

```
#include <stdio.h>

// Using digit separators in preprocessor expressions
#define ONE_MILLION 1'000'000
#define MAX_BUFFER_SIZE 10'000
#define BINARY_PATTERN 0b1010'0101'1100'1111

int main() {
    printf("One million: %d\n", ONE_MILLION);
    printf("Max buffer size: %d\n", MAX_BUFFER_SIZE);
    printf("Binary pattern: 0x%X\n", BINARY_PATTERN);
```

```
    // Using in preprocessor conditionals
#if ONE_MILLION > 500'000
    printf("One million is greater than 500 thousand\n");
#endif

    return 0;
}
```

Practical Examples

Configuration System Using Preprocessor

```
#include <stdio.h>

// Configuration system using preprocessor
// Define default values
#ifndef CONFIG_MAX_CONNECTIONS
    #define CONFIG_MAX_CONNECTIONS 100
#endif

#ifndef CONFIG_TIMEOUT
    #define CONFIG_TIMEOUT 30
#endif

#ifndef CONFIG_LOG_LEVEL
    #define CONFIG_LOG_LEVEL 2
#endif

// Feature flags
#define FEATURE_NETWORKING 1
#define FEATURE_DATABASE 1
#define FEATURE_SECURITY 0

// Platform-specific configurations
#ifdef _WIN32
    #define PATH_SEPARATOR '\\'
    #define DEFAULT_CONFIG_FILE "config.ini"
#else
    #define PATH_SEPARATOR '/'
    #define DEFAULT_CONFIG_FILE "/etc/myapp/config.ini"
#endif

// Debug configuration
#ifdef DEBUG
```

```
    #define LOG_LEVEL CONFIG_LOG_LEVEL
    #define ENABLE_ASSERTIONS 1
    #define VERBOSE_LOGGING 1
#else
    #define LOG_LEVEL (CONFIG_LOG_LEVEL - 1)
    #define ENABLE_ASSERTIONS 0
    #define VERBOSE_LOGGING 0
#endif

// Conditional compilation based on features
#if FEATURE_NETWORKING
    #include <sys/socket.h>
    #define NETWORK_BUFFER_SIZE 8192

    void init_networking(void) {
        printf("Networking initialized with buffer size: %d\n", NETWORK_BUFFER_SIZE);
    }
#else
    void init_networking(void) {
        printf("Networking disabled\n");
    }
#endif

#if FEATURE_DATABASE
    #include <sqlite3.h>
    #define DB_CONNECTION_POOL_SIZE 10

    void init_database(void) {
        printf("Database initialized with connection pool size: %d\n", DB_CONNECTION_POOL_SI
    }
#else
    void init_database(void) {
        printf("Database disabled\n");
    }
#endif

#if FEATURE_SECURITY
    #include <openssl/ssl.h>
    #define SSL_ENABLED 1

    void init_security(void) {
        printf("Security initialized with SSL enabled\n");
    }
#else
    #define SSL_ENABLED 0
    void init_security(void) {
        printf("Security disabled\n");
```

```
    }
#endif

void print_configuration(void) {
    printf("=== Application Configuration ===\n");
    printf("Max connections: %d\n", CONFIG_MAX_CONNECTIONS);
    printf("Timeout: %d seconds\n", CONFIG_TIMEOUT);
    printf("Log level: %d\n", LOG_LEVEL);
    printf("Path separator: '%c'\n", PATH_SEPARATOR);
    printf("Default config file: %s\n", DEFAULT_CONFIG_FILE);
    printf("Assertions: %s\n", ENABLE_ASSERTIONS ? "enabled" : "disabled");
    printf("Verbose logging: %s\n", VERBOSE_LOGGING ? "enabled" : "disabled");
    printf("\n");

    printf("=== Feature Status ===\n");
    printf("Networking: %s\n", FEATURE_NETWORKING ? "enabled" : "disabled");
    printf("Database: %s\n", FEATURE_DATABASE ? "enabled" : "disabled");
    printf("Security: %s\n", FEATURE_SECURITY ? "enabled" : "disabled");
    printf("SSL: %s\n", SSL_ENABLED ? "enabled" : "disabled");
    printf("\n");
}

int main() {
    print_configuration();

    init_networking();
    init_database();
    init_security();

    return 0;
}
```

Advanced Error Handling with Preprocessor

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

// Advanced error handling macros
#define STRINGIFY(x) #x
#define TOSTRING(x) STRINGIFY(x)

// Error checking with automatic cleanup
#define CHECK_ERRNO(call) do { \
```

```
    errno = 0; \
    call; \
    if (errno != 0) { \
        fprintf(stderr, "Error in %s at %s:%d: %s\n", \
                #call, __FILE__, __LINE__, strerror(errno)); \
        exit(EXIT_FAILURE); \
    } \
} while(0)

// Resource management with automatic cleanup
#define CLEANUP_VAR(type, name, cleanup_func) \
    type name = NULL; \
    defer_cleanup(cleanup_func, &name)

// Defer mechanism (simplified)
typedef struct defer_node {
    void (*func)(void*);
    void *arg;
    struct defer_node *next;
} defer_node_t;

static defer_node_t *defer_stack = NULL;

void defer_cleanup(void (*func)(void*), void *arg) {
    defer_node_t *node = malloc(sizeof(defer_node_t));
    node->func = func;
    node->arg = arg;
    node->next = defer_stack;
    defer_stack = node;
}

void execute_deferred(void) {
    while (defer_stack) {
        defer_node_t *node = defer_stack;
        defer_stack = node->next;
        node->func(node->arg);
        free(node);
    }
}

#define defer(func, arg) defer_cleanup(func, arg)

// Cleanup functions
void free_ptr(void *ptr) {
    free(*(void**)ptr);
}
```

```c
void fclose_ptr(void *ptr) {
    if (*(FILE**)ptr) {
        fclose(*(FILE**)ptr);
    }
}

int main() {
    printf("Advanced error handling with preprocessor\n");

    // File operations with automatic error checking
    CLEANUP_VAR(FILE*, fp, fclose_ptr);
    fp = fopen("test.txt", "w");
    if (!fp) {
        perror("fopen");
        exit(EXIT_FAILURE);
    }

    CHECK_ERRNO(fprintf(fp, "Hello, World!\n"));
    CHECK_ERRNO(fflush(fp));

    // Memory allocation with automatic cleanup
    CLEANUP_VAR(char*, buffer, free_ptr);
    buffer = malloc(100);
    if (!buffer) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(EXIT_FAILURE);
    }

    strcpy(buffer, "This is a test buffer");
    printf("Buffer content: %s\n", buffer);

    // Automatic cleanup when scope ends
    execute_deferred();

    return 0;
}
```

Summary

Advanced preprocessor techniques provide powerful metaprogramming capabilities in C:

1. **Macro Definitions**: Stringification, token pasting, variadic macros, and generic programming
2. **Conditional Compilation**: Platform-specific code, feature toggles, and version compatibility
3. **Pragma Directives**: Compiler hints, optimization control, and warning management

4. **Predefined Macros**: System information, compilation context, and debugging support
5. **Token Manipulation**: Deferred expansion, X-Macros, and complex metaprogramming
6. **C23 Enhancements**: Improved conditional directives and digit separators

Mastering these advanced preprocessor techniques enables C developers to write more maintainable, efficient, and feature-rich code while leveraging the full power of the C preprocessor for metaprogramming and code generation.

C23 `#embed`, `nullptr`, and `constexpr`

Overview

C23 (ISO/IEC 9899:2024) finally gives C three everyday tools other languages took for granted: **binary assets as first-class data** (`#embed`), a distinct **null pointer constant** (`nullptr`), and **`constexpr` objects** for values fixed at translation time. The overview chapter in this part mentions them briefly; this page is the operator-depth pass — what to type, what the compiler guarantees, and how to keep builds portable while toolchains catch up.

Baseline for the labs: **GCC 14+** or **Clang 19+** with `-std=c23` (or `-std=gnu23`). Check your compiler before relying on `#embed` in CI.

```
cc -std=c23 -Wall -Wextra -O2 embed_demo.c -o embed_demo
```

`#embed` — ship bytes without `xxd` hacks

`#embed` is a preprocessor directive that injects the contents of a file as a comma-separated list of integer constant expressions (by default, `unsigned char` values). Typical use: icons, small lookup tables, default config blobs, shader snippets — anything you used to generate with `xxd -i` or `ld -r -b binary`.

Basic embed into an array

```
/* embed_demo.c - C23 */
#include <stdio.h>
#include <stddef.h>

static const unsigned char logo[] = {
#embed "logo.png"
};

int main(void) {
    printf("logo.png: %zu bytes\n", sizeof logo);
    if (sizeof logo >= 8) {
        printf("header:");
        for (size_t i = 0; i < 8; i++) {
            printf(" %02x", logo[i]);
```

```
        }
        puts("");
    }
    return 0;
}
```

Place a small `logo.png` next to the source (or pass an absolute path). The array length is exactly the file size. Rebuilding picks up file changes — no separate codegen step.

Resource limits and empty files

```
/* Truncate or reject oversized assets at compile time */
static const unsigned char tip[] = {
#embed "tip.txt" limit(256)
};

/* Optional: custom empty-file expansion */
static const int maybe_empty[] = {
#embed "optional.bin" if_empty(0)
};
```

Clause	Meaning
`limit(N)`	Embed at most `N` elements; longer files are truncated for the directive (prefer failing the build in policy if truncation is unsafe — see Notes)
`prefix(…)` / `suffix(…)`	Extra constant expressions before/after the file bytes
`if_empty(…)`	Replacement list when the file has zero size
`offset(N)` (where supported)	Skip the first `N` bytes — confirm against your compiler's C23 embed support matrix

`#embed` paths are resolved like include paths (implementation-defined search rules; quote form usually searches relative to the current file). Keep assets inside the repo; do not `#embed` secrets from a home directory in shared builds.

When not to use `#embed`

- Multi-megabyte assets: prefer runtime files or `mmap` unless you intentionally want them in the binary.
- Generated code that must stay human-editable as C — keep a `.c` file.
- Toolchains stuck on C17: fall back to `xxd -i` or a small Python codegen in the Makefile until CI compilers move.

`nullptr` — a real null pointer constant

Historically `NULL` was `0` or `(void*)0`, which made overload-like `_Generic` selections and some warnings awkward. C23 adds `nullptr` of type `nullptr_t` (from `<stddef.h>`), which **only** converts to pointer types.

```
#include <stddef.h>
#include <stdio.h>

void greet(const char *name) {
    if (name == nullptr) {
        puts("hello, stranger");
        return;
    }
    printf("hello, %s\n", name);
}

int main(void) {
    greet(nullptr);
    greet("desk");
    /* nullptr is not an integer - good */
    /* int x = nullptr; */  /* constraint violation */
    return 0;
}
```

Prefer `nullptr` in new C23 code. `NULL` remains valid for compatibility. In headers shared with C++, `nullptr` matches C++ spelling and intent.

`constexpr` objects

C23 `constexpr` on object definitions requires an initializer that is a constant expression, and the object may be used in contexts that need true constants (where an ordinary `const` integer sometimes was not enough for every implementation).

```
#include <stdio.h>

constexpr int max_widgets = 64;
constexpr unsigned mask = (1u << 5) - 1u;

static int table[max_widgets]; /* size is a real constant */

int main(void) {
    printf("max=%d mask=%u\n", max_widgets, mask);
    return 0;
}
```

Use `constexpr` for protocol limits, shift masks, and array sizes you previously stuffed into `#define` macros. Macros remain appropriate for token pasting and include guards; prefer `constexpr` for typed values.

Combined sketch

```c
/* banner.c - C23 */
#include <stddef.h>
#include <stdio.h>

constexpr size_t banner_cap = 128;

static const unsigned char banner[] = {
#embed "banner.txt" limit(banner_cap)
};

void print_banner(const unsigned char *p, size_t n) {
    if (p == nullptr || n == 0) {
        return;
    }
    fwrite(p, 1, n, stdout);
}

int main(void) {
    print_banner(banner, sizeof banner);
    return 0;
}
```

Portability checklist

Feature	Gate
Language mode	`-std=c23` / `__STDC_VERSION__ >= 202311L`
`#embed`	Compiler + version matrix; provide `xxd` fallback in Makefile for older CI
`nullptr`	C23; polyfill as `#define nullptr ((void*)0)` only in transitional headers with care
`constexpr`	C23; older code keeps `#define` or `enum` constants

```c
#if __STDC_VERSION__ >= 202311L
#  define DESK_NULL nullptr
#else
#  define DESK_NULL NULL
#endif
```

Safety

- Do not `#embed` credentials, private keys, or production `.env` files into binaries you distribute.
- Treat `#embed` data as **untrusted input** if the file can be replaced in the build environment — validate magic headers at runtime when it matters.
- `nullptr` checks do not replace bounds checks on the objects you open after a non-null test.
- Huge embeds inflate binaries and caches; budget size like any other asset pipeline.

Try this

1. Create `tip.txt`, embed it with and without `limit(16)`, and print `sizeof` both ways.
2. Compile a one-line `int x = nullptr;` under `-std=c23` and read the diagnostic.
3. Replace a `#define MAX 32` array size with `constexpr int max = 32;` and confirm the project still builds at `-std=c23`.
4. Add a Makefile rule: if the compiler rejects `#embed`, generate `logo.h` with `xxd -i` as a fallback and document the flag that flips modes.

Sources

- ISO/IEC 9899:2024 (C23) — `#embed`, `nullptr` / `nullptr_t`, `constexpr` objects
- cppreference — `#embed`
- cppreference — `nullptr`
- GCC / Clang C23 status notes for your distro toolchain (`cc --version`)

Checked Integer Arithmetic with C23 `<stdckdint.h>`

Thesis

Integer overflow is the quiet bug behind undersized buffers, negative lengths, and "impossible" branches that the optimizer deletes. For decades the portable C answer was hand-written pre-checks (`if (a > INT_MAX - b)`), which are easy to get wrong for mixed signedness and nearly impossible to get right for 64-bit multiplication. C23 (ISO/IEC 9899:2024, §7.20) standardises the fix: three type-generic macros in `<stdckdint.h>` —

```
bool ckd_add(type1 *result, type2 a, type3 b);
bool ckd_sub(type1 *result, type2 a, type3 b);
bool ckd_mul(type1 *result, type2 a, type3 b);
```

Each computes the **mathematically exact** result, stores it (wrapped) in `*result`, and returns **`true` if it did not fit**. GCC 14 and Clang 18 ship the header and compile each call to an add/multiply plus a branch on the CPU's overflow or carry flag. There is no longer a reason to write overflow checks by hand in new code.

The boring operator habit: **every size, offset, or count that comes from outside the process passes through a `ckd_*` call (or an allocator that checks for you) before it reaches `malloc`, `memcpy`, or an array index.**

Mental model

```
        a (any integer type)      b (any integer type)
                  \                  /

         exact arithmetic in "infinite precision" - never UB


           convert to the type of *result   (the ONLY type that matters)


      fits exactly                      does not fit
  *result = exact value             *result = value wrapped to width
   return false                      return true   ← treat as error
```

Three consequences follow:

1. **The result type defines "overflow".** `ckd_add(&r8, 100, 28)` overflows for `int8_t r8` but not for `int r8`. To ask "does this fit in 64 bits?", make `*result` 64 bits wide.
2. **Operand types and signedness do not matter.** `ckd_sub(&size, 3u, 5u)` correctly reports that `-2` does not fit in `size_t`; `ckd_mul(&u, -1, -1)` correctly reports that `1` fits in `unsigned`. No casts, no usual-arithmetic-conversion surprises.
3. **The stored value on overflow is a wrapped leftover.** It is well-defined (no UB), but it is not an answer. Return an error; do not keep computing with it.

Approach	Signed safe?	Mixed signedness?	64×64 multiply?	Standard?
`a + b < a` after the fact	**No** — UB, optimizer may delete it	No	No	—
Pre-check `a > MAX - b`	Yes, if written perfectly	Painful	Painful (division)	Yes
Widen to `int64_t`, compare	Yes	Mostly	**No** (no portable 128-bit)	Yes
`__builtin_*_overflow`	Yes	Yes	Yes	GCC/Clang extension
C23 `ckd_*`	Yes	Yes	Yes	**ISO C23**

Worked examples

All listings were built and run on x86_64 Linux with GCC 14.2 (Debian 14.2.0-19), Clang 19.1.7, and glibc 2.41. Each listing produced identical output under both compilers unless noted.

Case 1: the check the optimizer is allowed to delete

The classic after-the-fact test looks reasonable and is wrong for signed types: if `a + 100` overflows, behaviour is undefined, so the compiler may assume it never does — and `a + 100 < a` is then always false.

```c
/* naive.c - the overflow check that the optimizer is allowed to delete. */
#include <limits.h>
#include <stdio.h>

/* Intent: "would a + 100 overflow?"  Signed overflow is undefined, so the
   compiler may assume a + 100 never wraps and fold this to 0. */
__attribute__((noinline))
int would_overflow(int a) {
    return a + 100 < a;
}

int main(void) {
```

```
    printf("would_overflow(INT_MAX) = %d\n", would_overflow(INT_MAX));
    return 0;
}
```

```
for o in -O0 -O2; do gcc   -std=c23 $o naive.c -o naive && echo "gcc $o: $(./naive)"; done
for o in -O0 -O2; do clang -std=c23 $o naive.c -o naive && echo "clang $o: $(./naive)"; done
```

```
gcc -O0: would_overflow(INT_MAX) = 0
gcc -O2: would_overflow(INT_MAX) = 0
clang -O0: would_overflow(INT_MAX) = 1
clang -O2: would_overflow(INT_MAX) = 0
```

Same source, three different "answers" to a yes/no question. GCC folds the comparison away even at `-O0`; the generated function is literally "return 0":

```
gcc -std=c23 -O2 -S -o - naive.c | sed -n '/would_overflow:/,/ret/p'
```

```
would_overflow:
.LFB3:
    .cfi_startproc
    xorl    %eax, %eax
    ret
```

UBSan sees the overflow at run time, which is how you *find* such sites — but finding is not fixing:

```
gcc -std=c23 -O2 -fsanitize=undefined naive.c -o naive_ub && ./naive_ub
```

```
naive.c:9:14: runtime error: signed integer overflow: 2147483647 + 100 cannot be represented
would_overflow(INT_MAX) = 1
```

The fix is one line: `int r; return ckd_add(&r, a, 100);`.

Case 2: what the macros actually report

This listing pins down the semantics in the mental model: result type decides, signedness of operands is irrelevant, and the stored value on overflow is the wrapped leftover.

```
/* basics.c - what ckd_add / ckd_sub / ckd_mul actually report. */
#include <limits.h>
#include <stdckdint.h>
#include <stdint.h>
#include <stdio.h>

int main(void) {
```

```
    int i;
    bool ov = ckd_add(&i, INT_MAX, 1);
    printf("INT_MAX + 1      -> int      ov=%d stored=%d\n", ov, i);

    /* Operands are evaluated as if in infinite precision; only the
       *result* type decides whether it "fits". */
    size_t n;
    ov = ckd_sub(&n, (size_t)3, (size_t)5);
    printf("3 - 5            -> size_t   ov=%d stored=%zu\n", ov, n);

    int8_t small;
    ov = ckd_add(&small, 100, 27);
    printf("100 + 27         -> int8_t   ov=%d stored=%d\n", ov, small);
    ov = ckd_add(&small, 100, 28);
    printf("100 + 28         -> int8_t   ov=%d stored=%d\n", ov, small);

    /* Mixed signedness is fine: -1 * -1 == 1 fits in unsigned. */
    unsigned u;
    ov = ckd_mul(&u, -1, -1);
    printf("-1 * -1          -> unsigned ov=%d stored=%u\n", ov, u);
    ov = ckd_mul(&u, -1, 1);
    printf("-1 * 1           -> unsigned ov=%d stored=%u\n", ov, u);

    /* Widening the result type is how you ask "does it fit in 64 bits?" */
    long long wide;
    ov = ckd_mul(&wide, INT_MAX, INT_MAX);
    printf("INT_MAX*INT_MAX  -> llong    ov=%d stored=%lld\n", ov, wide);
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -O2 basics.c -o basics && ./basics
```

```
INT_MAX + 1      -> int      ov=1 stored=-2147483648
3 - 5            -> size_t   ov=1 stored=18446744073709551614
100 + 27         -> int8_t   ov=0 stored=127
100 + 28         -> int8_t   ov=1 stored=-128
-1 * -1          -> unsigned ov=0 stored=1
-1 * 1           -> unsigned ov=1 stored=4294967295
INT_MAX*INT_MAX  -> llong    ov=0 stored=4611686014132420609
```

`bool` is a keyword in C23, so no `<stdbool.h>` is needed. Clang 19 (`clang -std=c23 …`) prints byte-identical output.

Case 3: sizing an allocation from an untrusted count

This is where overflow becomes a memory-safety bug. A count read from a file header, a network packet, or `argv` is multiplied by an element size and added to a header size. With a large enough count the product wraps to something tiny, `malloc` succeeds, and the subsequent loop writes far past the end.

```
/* alloc.c - sizing a buffer from an untrusted element count. */
#define _DEFAULT_SOURCE   /* -std=c23 is strict ISO; this exposes reallocarray() */
#include <errno.h>
#include <stdckdint.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct record { uint32_t id; uint32_t flags; };   /* 8 bytes */
struct table  { size_t count; size_t cap; struct record rows[]; };

/* Publishing each pointer through a volatile stops the optimizer from
   deleting an unused malloc/free pair and assuming it "succeeded". */
static void *volatile sink;

/* WRONG: the multiplication silently wraps for huge counts. */
static size_t naive_size(size_t count) {
    return sizeof(struct table) + count * sizeof(struct record);
}

/* RIGHT: every step that can overflow is checked; false means "fits". */
static bool checked_size(size_t count, size_t *out) {
    size_t body;
    if (ckd_mul(&body, count, sizeof(struct record))) return false;
    if (ckd_add(out, body, sizeof(struct table)))     return false;
    return true;
}

static struct table *table_new(size_t count) {
    size_t bytes;
    if (!checked_size(count, &bytes)) { errno = EOVERFLOW; return NULL; }
    struct table *t = malloc(bytes);
    if (t) { t->count = 0; t->cap = count; }
    return t;
}

int main(int argc, char **argv) {
    if (argc != 2) { fprintf(stderr, "usage: %s COUNT\n", argv[0]); return 2; }
    size_t count = strtoull(argv[1], NULL, 0);
```

```
    printf("count        = %zu\n", count);
    printf("naive bytes  = %zu\n", naive_size(count));

    size_t bytes;
    if (checked_size(count, &bytes))
        printf("checked      = %zu\n", bytes);
    else
        printf("checked      = overflow, refusing\n");

    struct table *t = sink = table_new(count);
    printf("table_new    = %s (%s)\n", t ? "ok" : "NULL", t ? "-" : strerror(errno));
    free(sink);

    /* The library already checks n * size for you in these two calls. */
    void *p = sink = calloc(count, sizeof(struct record));
    printf("calloc       = %s\n", p ? "ok" : "NULL");
    free(sink);
    p = sink = reallocarray(NULL, count, sizeof(struct record));
    printf("reallocarray = %s\n", p ? "ok" : "NULL");
    free(sink);
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -O2 alloc.c -o alloc
./alloc 1000
./alloc 0x2000000000000001
```

```
count        = 1000
naive bytes  = 8016
checked      = 8016
table_new    = ok (-)
calloc       = ok
reallocarray = ok

count        = 2305843009213693953
naive bytes  = 24
checked      = overflow, refusing
table_new    = NULL (Value too large for defined data type)
calloc       = NULL
reallocarray = NULL
```

The hostile count is `2^61 + 1`. Times 8 bytes it is `2^64 + 8`, which wraps to `8`; plus the 16-byte header gives a **24-byte** buffer for "two quintillion" records. The checked version refuses before calling `malloc`.

Two library calls already do the `n × size` check internally and are the first thing to reach for:

- `calloc(n, size)` — ISO C; zero-fills, returns `NULL` on overflow.
- `reallocarray(ptr, n, size)` — originally OpenBSD, in glibc since 2.26, standardised by **POSIX.1-2024**; behaves like `realloc(ptr, n * size)` but fails with `ENOMEM` when the multiplication overflows. Under strict `-std=c23`, glibc 2.41 declares it only when you request extensions with `_DEFAULT_SOURCE` (as the listing does); `_POSIX_C_SOURCE 202405L` alone is not enough there.

Why the `volatile sink`? Without it, Clang 19 at `-O2` deleted the unused `calloc`/`free` pair and printed `calloc = ok` for the hostile count — the optimizer may assume an allocation whose result is never used succeeded. A probe that is optimised away proves nothing; publish the pointer somewhere the compiler cannot see through. (The listing also avoids printing `errno` after `calloc`: ISO C does not require allocators to set it, and LLVM does not model it as changed by the call.)

Neither covers the `+ sizeof(struct table)` header term, which is why `checked_size` uses `ckd_mul` *and* `ckd_add`.

Case 4: parsing a bounded number digit by digit

Parsers are the other hot spot. Accumulating `v = v * 10 + digit` into the destination type and checking at each step rejects oversized input exactly at the digit that breaks the bound — no `strtoul` range dance, no `errno`, no locale.

```
/* parse.c - parse a decimal port/length field without trusting its size. */
#include <stdckdint.h>
#include <stdint.h>
#include <stdio.h>

/* Returns true on success. Rejects empty input, non-digits, and any value
   that does not fit in uint16_t - checked at every digit, not at the end. */
static bool parse_u16(const char *s, uint16_t *out) {
    uint16_t v = 0;
    if (*s == '\0') return false;
    for (; *s; s++) {
        if (*s < '0' || *s > '9') return false;
        if (ckd_mul(&v, v, 10))      return false;
        if (ckd_add(&v, v, *s - '0')) return false;
    }
    *out = v;
    return true;
}

int main(void) {
    const char *inputs[] = { "8443", "65535", "65536", "99999999999999999999", "12a", "" };
    for (size_t i = 0; i < sizeof inputs / sizeof inputs[0]; i++) {
        uint16_t port;
        if (parse_u16(inputs[i], &port))
            printf("%-22s -> %u\n", inputs[i], port);
```

```
        else
            printf("%-22s -> rejected\n", inputs[i]);
    }
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -Wconversion -O2 parse.c -o parse && ./parse
```

```
8443                  -> 8443
65535                 -> 65535
65536                 -> rejected
99999999999999999999  -> rejected
12a                   -> rejected
                      -> rejected
```

Note that `-Wconversion` is silent: `ckd_add(&v, v, *s - '0')` adds an `int` to a `uint16_t` and stores into `uint16_t` without any implicit narrowing — the narrowing is exactly what the macro checks.

Case 5: one header for C23 and older toolchains

`<stdckdint.h>` is a header shipped by the **compiler** (GCC 14, Clang 18), not by libc. Both GCC 14 and Clang 19 accept it even in `-std=c17` or older modes. For older compilers, map the macros onto the GCC/Clang builtins they are defined in terms of — and mind the **argument order**: the builtins put the result pointer *last*.

```
/* ckd_compat.h - use C23 <stdckdint.h> when present, else the GCC/Clang
   builtins it is defined in terms of. Note the argument order swap. */
#ifndef CKD_COMPAT_H
#define CKD_COMPAT_H

#if defined(__has_include) && !defined(CKD_FORCE_BUILTIN)
#  if __has_include(<stdckdint.h>)
#    include <stdckdint.h>
#    define CKD_IMPL "stdckdint.h"
#  endif
#endif

#ifndef CKD_IMPL
#  if defined(__GNUC__) || defined(__clang__)
     /* builtins: (a, b, result)   ckd_*: (result, a, b) */
#    define ckd_add(r, a, b) __builtin_add_overflow((a), (b), (r))
#    define ckd_sub(r, a, b) __builtin_sub_overflow((a), (b), (r))
#    define ckd_mul(r, a, b) __builtin_mul_overflow((a), (b), (r))
#    define CKD_IMPL "__builtin_*_overflow"
```

```
#  else
#    error "no checked-arithmetic support: need C23 <stdckdint.h> or GCC/Clang builtins"
#  endif
#endif

#endif /* CKD_COMPAT_H */
```

```
/* compat.c - same call sites, either implementation. */
#include <limits.h>
#include <stdio.h>
#include "ckd_compat.h"

int main(void) {
    long r;
    int ov = ckd_mul(&r, LONG_MAX / 2, 3);
    printf("impl=%s  ov=%d\n", CKD_IMPL, ov);
    return 0;
}
```

```
gcc   -std=c17 -Wall -Wextra                       compat.c -o compat && ./compat
gcc   -std=c17 -Wall -Wextra -DCKD_FORCE_BUILTIN compat.c -o compat && ./compat
clang -std=c11 -Wall -Wextra -DCKD_FORCE_BUILTIN compat.c -o compat && ./compat
```

```
impl=stdckdint.h  ov=1
impl=__builtin_*_overflow  ov=1
impl=__builtin_*_overflow  ov=1
```

Call sites use the C23 spelling everywhere; deleting the shim later is a one-line change.

The trap

Habits from the builtins, and trusting the leftover value. The compilers reject the worst mistakes, but not uniformly:

```
/* misuse.c - mistakes the compiler rejects. */
#include <stdckdint.h>

int main(void) {
    int  a = 1, b = 2, r;
    bool flag;
    char c;
    (void)ckd_add(&flag, a, b);   /* bool result: not allowed      */
    (void)ckd_add(&c, a, b);      /* plain char result: not allowed */
    (void)ckd_add(a, b, &r);      /* builtin argument order by habit */
```

```
    return r;
}
```

```
clang -std=c23 -c misuse.c -o /dev/null
```

```
misuse.c:8:11: error: result argument to checked integer operation must be a pointer to a no
    8 |     (void)ckd_add(&flag, a, b);   /* bool result: not allowed      */
      |           ^~~~~~~~~~~~~~~~~~~
/usr/lib/llvm-19/lib/clang/19/include/stdckdint.h:37:59: note: expanded from macro 'ckd_add'
   37 | #define ckd_add(R, A, B) __builtin_add_overflow((A), (B), (R))
      |                                                           ^~
misuse.c:9:11: error: result argument to checked integer operation must be a pointer to a no
    9 |     (void)ckd_add(&c, a, b);      /* plain char result: not allowed */
      |           ^~~~~~~~~~~~~~~~
/usr/lib/llvm-19/lib/clang/19/include/stdckdint.h:37:59: note: expanded from macro 'ckd_add'
   37 | #define ckd_add(R, A, B) __builtin_add_overflow((A), (B), (R))
      |                                                           ^~
misuse.c:10:11: error: operand argument to checked integer operation must be an integer type
   10 |     (void)ckd_add(a, b, &r);      /* builtin argument order by habit */
      |           ^~~~~~~~~~~~~~~~
/usr/lib/llvm-19/lib/clang/19/include/stdckdint.h:37:54: note: expanded from macro 'ckd_add'
   37 | #define ckd_add(R, A, B) __builtin_add_overflow((A), (B), (R))
      |                                                      ^~
3 errors generated.
```

```
gcc -std=c23 -c misuse.c -o /dev/null
```

```
misuse.c: In function ‘main’:
misuse.c:8:11: error: argument 3 in call to function ‘__builtin_add_overflow’ has pointer to
    8 |     (void)ckd_add(&flag, a, b);   /* bool result: not allowed      */
      |           ^~~~~~~
misuse.c:10:11: error: argument 2 in call to function ‘__builtin_add_overflow’ does not have
   10 |     (void)ckd_add(a, b, &r);      /* builtin argument order by habit */
      |           ^~~~~~~
```

GCC 14 silently accepts the plain-`char` result (the standard only *recommends* a diagnostic) — so code that builds with GCC can fail on Clang. Beyond what the compilers catch:

- **Ignoring the return value.** `ckd_add(&n, n, len);` with no `if` is just addition with extra steps. Wrap it: `if (ckd_add(&n, n, len)) return -EOVERFLOW;`.
- **Using `*result` after overflow.** It is the truncated bit pattern (24 bytes in Case 3). Never pass it on.
- **Checking in the wrong type.** `int total; ckd_add(&total, a_size_t, b_size_t)` checks fit in `int`, which may be what you want for a protocol field, but is wrong if the next line passes `total` to `malloc`. Make `*result` the type of the *consumer*.
- **Checking one step of a multi-step expression.** `header + n * size` is two operations;

both need checking (Case 3). So does every iteration of an accumulator (Case 4).

- **`_BitInt` and enums.** C23 excludes bit-precise integers, `bool`, plain `char`, and enumerations from `ckd_*` operands and results; convert to an ordinary integer type first.
- **`-fwrapv` as a fix.** It makes signed overflow wrap instead of being UB, so Case 1's comparison survives — but the program still computes a wrong, wrapped size. Wrapping is not detection.

The boring rule

> Any arithmetic on a size, count, offset, length, or index that is influenced by input goes through `ckd_add` / `ckd_sub` / `ckd_mul` — with `*result` typed as whatever consumes the value — and an overflow return becomes an error path, never a value. Prefer `calloc` / `reallocarray` when the expression is exactly `n × size`. Run tests under `-fsanitize=undefined` to find the sites you missed.

Checklist for review:

1. No after-the-fact overflow tests (`a + b < a`) on signed types anywhere.
2. Every `malloc`/`realloc` argument that is not a constant is either a `ckd_*` result or replaced by `calloc`/`reallocarray`.
3. Every `ckd_*` call is the condition of an `if` (or its result is otherwise tested).
4. Parsers accumulate into the destination type with `ckd_mul` + `ckd_add` per digit.
5. Builds that must support pre-C23 compilers use one shim header (Case 5), not ad-hoc builtins sprinkled through the code.

Try this

1. Suppose `table_new` in Case 3 used `naive_size` and allocated with `calloc(1, bytes)`. Would the overflow be caught? Explain why `calloc` only protects the multiplication it performs itself.
2. Extend `parse_u16` from Case 4 into a generic `parse_u64` and test it against `18446744073709551615` and `18446744073709551616`.
3. Write `bool checked_area(size_t w, size_t h, size_t bpp, size_t *out)` for an image decoder (`w × h × bpp` plus a 54-byte header) and feed it `w = h = 0x100000001`.
4. Compile Case 1 with `-fwrapv` under GCC and Clang at `-O2`. Does `would_overflow` now return 1? Is the program *correct* now, or merely deterministic?
5. Disassemble `checked_size` (`gcc -O2 -S alloc.c`) and find the `jc`/`jo`/`seto` (or `mul` + flag test) that implements the check. Count the instructions it costs.

Sources

- ISO/IEC 9899:2024 (C23) §7.20 *Checked integer arithmetic* — summarised at cppreference: `ckd_add` and `ckd_mul`
- GCC manual — Integer Overflow Builtins
- POSIX.1-2024 `reallocarray()` (Austin Group defect 1218); glibc 2.26 NEWS for its introduction

- Clang review D157331 — Implement C23 `<stdckdint.h>` (Clang 18)

C23 <stdbit.h>: Bit and Byte Utilities

Thesis

Every serious C code base has a `bits.h` full of `__builtin_clz`, `__builtin_popcount`, hand-rolled "round up to a power of two" loops, and `#ifdef __BYTE_ORDER__` blocks. C23 (ISO/IEC 9899:2024, §7.18) replaces that folklore with a standard header: **`<stdbit.h>`** — fourteen families of bit queries (`stdc_leading_zeros`, `stdc_count_ones`, `stdc_bit_width`, `stdc_bit_ceil`, …) plus three endianness macros.

The improvements over the builtins are not cosmetic:

- **Every function is defined for zero.** `__builtin_clz(0)` is undefined behaviour; `stdc_leading_zeros(0u)` is 32.
- **Type-generic and width-correct.** `stdc_leading_zeros` on a `uint8_t` counts within 8 bits, on a `uint64_t` within 64 — no `clz` vs `clzl` vs `clzll` mismatch.
- **Overflow is specified.** `stdc_bit_ceil` returns 0 when the next power of two does not fit, instead of wrapping silently.

glibc ships the header since **2.39** (January 2024), with out-of-line functions for each width (`_uc`, `_us`, `_ui`, `_ul`, `_ull`) and type-generic macros; GCC 14 adds `__builtin_stdc_*` builtins that glibc's macros use when available. The boring habit: **new bit-twiddling code calls `<stdbit.h>` with unsigned operands, and the old `bits.h` shrinks to a fallback shim.**

Mental model

```
bit:   31 30 29 ...  9  8  7  6  5  4         3  2  1  0
x:      0  0  0 ...  0  0  1  1  1  1         0  0  0  0
       leading zeros: 24    ones: 4    trailing zeros: 4

x = 0x000000F0 (unsigned int, 32 bits)
"first" functions count positions 1-based:  first_leading_one  = 25 (from the MSB)
                                            first_trailing_one =  5 (from the LSB)
                                            0 means "no such bit"
bit_width = 8   (bits needed: 32 - leading_zeros)
bit_floor = 0x80, bit_ceil = 0x100   (0 if not representable)
```

Family	Question it answers	`x == 0`	Return type
`leading_zeros` / `leading_ones`	run length from the MSB	width	`unsigned int`
`trailing_zeros` / `trailing_ones`	run length from the LSB	width / 0	`unsigned int`
`first_leading_{zero,one}`	1-based position from the MSB	1 / **0 = none**	`unsigned int`
`first_trailing_{zero,one}`	1-based position from the LSB	1 / **0 = none**	`unsigned int`
`count_zeros` / `count_ones`	population count	width / 0	`unsigned int`
`has_single_bit`	is it a power of two?	`false`	`bool`
`bit_width`	bits needed to represent `x` (log2 x + 1)	0	`unsigned int`
`bit_floor`	largest power of two `x`	0	same as argument
`bit_ceil`	smallest power of two `x`	1	same as argument; **0 if it does not fit**

Endianness, at compile time: `__STDC_ENDIAN_NATIVE__` equals `__STDC_ENDIAN_LITTLE__` or `__STDC_ENDIAN_BIG__` (or something else on exotic hardware). C23 gives you detection only — there is no standard byte-swap function in `<stdbit.h>`.

Worked examples

All listings were built and run on x86_64 Linux with GCC 14.2 (Debian 14.2.0-19), Clang 19.1.7, and glibc 2.41. Output was identical under both compilers except where *The trap* shows otherwise.

Case 1: a tour on one value

```c
/* tour.c - every <stdbit.h> family on one 32-bit value. */
#include <stdbit.h>
#include <stdio.h>

int main(void) {
    unsigned x = 0x00F0u;   /* ...0000 1111 0000 - bits 4..7 set */

    printf("x = 0x%08X (width %zu bits)\n", x, sizeof x * 8);
    printf("leading_zeros        %2u\n", stdc_leading_zeros(x));
    printf("leading_ones         %2u\n", stdc_leading_ones(x));
    printf("trailing_zeros       %2u\n", stdc_trailing_zeros(x));
    printf("trailing_ones        %2u\n", stdc_trailing_ones(x));
    printf("first_leading_one    %2u   (1-based from MSB)\n", stdc_first_leading_one(x));
    printf("first_trailing_one   %2u   (1-based from LSB)\n", stdc_first_trailing_one(x));
    printf("first_trailing_zero  %2u\n", stdc_first_trailing_zero(x));
```

```
    printf("count_ones           %2u\n", stdc_count_ones(x));
    printf("count_zeros          %2u\n", stdc_count_zeros(x));
    printf("has_single_bit       %2d\n", stdc_has_single_bit(x));
    printf("bit_width            %2u\n", stdc_bit_width(x));
    printf("bit_floor        0x%04X\n", stdc_bit_floor(x));
    printf("bit_ceil         0x%04X\n", stdc_bit_ceil(x));

    puts("-- edge cases --");
    printf("leading_zeros(0u)    %2u   (defined: the type's width)\n", stdc_leading_zeros(0u
    printf("first_trailing_one(0u) %u   (0 means \"no such bit\")\n", stdc_first_trailing_on
    printf("bit_width(0u)        %2u\n", stdc_bit_width(0u));
    printf("bit_ceil(0u)         %2u\n", stdc_bit_ceil(0u));
    printf("bit_ceil(0x80000001u) %u   (not representable -> 0)\n", stdc_bit_ceil(0x80000001
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -O2 tour.c -o tour && ./tour
```

```
x = 0x000000F0 (width 32 bits)
leading_zeros        24
leading_ones          0
trailing_zeros        4
trailing_ones         0
first_leading_one    25   (1-based from MSB)
first_trailing_one    5   (1-based from LSB)
first_trailing_zero   1
count_ones            4
count_zeros          28
has_single_bit        0
bit_width             8
bit_floor        0x0080
bit_ceil         0x0100
-- edge cases --
leading_zeros(0u)    32   (defined: the type's width)
first_trailing_one(0u) 0   (0 means "no such bit")
bit_width(0u)         0
bit_ceil(0u)          1
bit_ceil(0x80000001u) 0   (not representable -> 0)
```

Read the edge cases twice. `bit_ceil(0)` is 1 (the smallest power of two), `first_*` functions return **0 for "not found"**, so every position you get back must be decremented before use as a shift count or index.

Case 2: sizing a ring buffer with `stdc_bit_ceil`

Power-of-two capacities let a ring buffer replace `% capacity` with `& mask`. The old idiom — a loop or the "smear right" trick — silently wraps to 0 on huge inputs; `stdc_bit_ceil` *defines* that case as 0, which the caller can test.

```
/* ring.c - power-of-two ring buffer sized with stdc_bit_ceil. */
#include <stdbit.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

struct ring {
    size_t   mask;          /* capacity - 1; capacity is a power of two */
    size_t   head, tail;    /* free-running counters; index = counter & mask */
    uint32_t *slots;
};

static bool ring_init(struct ring *r, size_t want) {
    if (want == 0) return false;
    size_t cap = stdc_bit_ceil(want);      /* 0 if it cannot be represented */
    if (cap == 0) return false;
    r->slots = calloc(cap, sizeof *r->slots);
    if (!r->slots) return false;
    r->mask = cap - 1;
    r->head = r->tail = 0;
    return true;
}

static bool ring_push(struct ring *r, uint32_t v) {
    if (r->head - r->tail > r->mask) return false;   /* full */
    r->slots[r->head++ & r->mask] = v;
    return true;
}

static bool ring_pop(struct ring *r, uint32_t *v) {
    if (r->head == r->tail) return false;            /* empty */
    *v = r->slots[r->tail++ & r->mask];
    return true;
}

int main(void) {
    const size_t asks[] = { 1, 5, 64, 1000, SIZE_MAX / 2 + 2 };
    for (size_t i = 0; i < sizeof asks / sizeof asks[0]; i++) {
        size_t c = stdc_bit_ceil(asks[i]);
        printf("want %-20zu -> capacity %zu%s\n", asks[i], c,
               c ? "" : "  (not representable: refuse)");
```

```
    }

    struct ring r;
    if (!ring_init(&r, 5)) return 1;
    unsigned pushed = 0;
    for (uint32_t v = 100; ring_push(&r, v); v++) pushed++;
    printf("capacity %zu, pushed %u before full\n", r.mask + 1, pushed);

    uint32_t v;
    printf("popped:");
    while (ring_pop(&r, &v)) printf(" %u", v);
    printf("\n");
    free(r.slots);
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -O2 ring.c -o ring && ./ring
```

```
want 1                    -> capacity 1
want 5                    -> capacity 8
want 64                   -> capacity 64
want 1000                 -> capacity 1024
want 9223372036854775809  -> capacity 0  (not representable: refuse)
capacity 8, pushed 8 before full
popped: 100 101 102 103 104 105 106 107
```

`size_t` is `unsigned long` on LP64 Linux, so the type-generic macro selects the 64-bit implementation and returns a `size_t`-width result. The "full" test `head - tail > mask` relies on unsigned wrap of free-running counters, which is well-defined.

Case 3: a slot/ID allocator on a bitmap

File-descriptor tables, connection IDs, and inode bitmaps all ask "lowest free slot?". With one 64-bit word per 64 slots, `stdc_first_trailing_zero` answers in a single instruction per word, and `stdc_count_ones` gives occupancy.

```
/* slots.c - a 256-slot ID allocator on a bitmap of 64-bit words. */
#include <stdbit.h>
#include <stdint.h>
#include <stdio.h>

#define WORDS 4                       /* 4 × 64 = 256 slots */
static uint64_t used[WORDS];          /* bit set = slot in use */

/* Lowest free slot, or -1. stdc_first_trailing_zero is 1-based, 0 = none. */
```

```c
static int slot_alloc(void) {
    for (int w = 0; w < WORDS; w++) {
        unsigned pos = stdc_first_trailing_zero(used[w]);
        if (pos == 0) continue;                /* word full */
        unsigned bit = pos - 1;                /* convert to 0-based */
        used[w] |= UINT64_C(1) << bit;
        return w * 64 + (int)bit;
    }
    return -1;
}

static void slot_free(int id) {
    used[id / 64] &= ~(UINT64_C(1) << (id % 64));
}

static unsigned slots_in_use(void) {
    unsigned n = 0;
    for (int w = 0; w < WORDS; w++) n += stdc_count_ones(used[w]);
    return n;
}

int main(void) {
    int a = slot_alloc(), b = slot_alloc(), c = slot_alloc();
    printf("allocated %d %d %d, in use %u\n", a, b, c, slots_in_use());

    slot_free(b);
    printf("freed %d, next alloc reuses %d\n", b, slot_alloc());

    while (slot_alloc() >= 0) { }
    printf("after filling: in use %u, alloc -> %d\n", slots_in_use(), slot_alloc());

    slot_free(200);
    printf("freed 200, alloc -> %d\n", slot_alloc());
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -Wconversion -O2 slots.c -o slots && ./slots
```

```
allocated 0 1 2, in use 3
freed 1, next alloc reuses 1
after filling: in use 256, alloc -> -1
freed 200, alloc -> 200
```

The single line `unsigned bit = pos - 1;` is the whole cost of the 1-based convention — and forgetting it is an off-by-one that allocates slot 1 first and never hands out slot 0.

Case 4: a power-of-two latency histogram

Operators bucket latencies by order of magnitude (HDR-style). `stdc_bit_width(ns)` *is* the bucket index: bucket `b` holds values in `[2^(b-1), 2^b - 1]`, and 0 gets its own bucket — no `log2()`, no floating point, no special case for zero.

```
/* hist.c - power-of-two latency histogram: bucket = stdc_bit_width(ns). */
#include <stdbit.h>
#include <stdint.h>
#include <stdio.h>

#define BUCKETS 65   /* bit_width of a uint64_t is 0..64 */

int main(void) {
    /* Pretend these came from clock_gettime() deltas, in nanoseconds. */
    const uint64_t samples_ns[] = {
        0, 1, 3, 800, 950, 1024, 1500, 2047, 2048,
        120000, 130000, 250000, 4000000, 4100000, 900000000,
    };
    unsigned hist[BUCKETS] = {0};

    for (size_t i = 0; i < sizeof samples_ns / sizeof samples_ns[0]; i++)
        hist[stdc_bit_width(samples_ns[i])]++;     /* O(1), branch-free */

    printf("%-28s%s\n", "bucket (ns)", "count");
    for (unsigned b = 0; b < BUCKETS; b++) {
        if (!hist[b]) continue;
        uint64_t lo = b ? UINT64_C(1) << (b - 1) : 0;
        uint64_t hi = b ? (b == 64 ? UINT64_MAX : (UINT64_C(1) << b) - 1) : 0;
        printf("[%11llu, %11llu]  %u\n",
               (unsigned long long)lo, (unsigned long long)hi, hist[b]);
    }
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -Wconversion -O2 hist.c -o hist && ./hist
```

```
bucket (ns)                  count
[          0,           0]  1
[          1,           1]  1
[          2,           3]  1
[        512,        1023]  2
[       1024,        2047]  3
[       2048,        4095]  1
[      65536,      131071]  2
[     131072,      262143]  1
```

```
[    2097152,     4194303]  2
[  536870912,  1073741823]  1
```

Case 5: endianness — detect with `<stdbit.h>`, decode with shifts

The endianness macros are integer constant expressions, so they work in `#if`. But the robust way to *decode* wire formats is still to assemble bytes with shifts: correct on every host, no unaligned pointer casts, and the optimiser recognises the pattern.

```c
/* wire.c - endianness detection with <stdbit.h>, decoding with shifts. */
#include <stdbit.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

#if __STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_LITTLE__
#  define HOST_ORDER "little-endian"
#elif __STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_BIG__
#  define HOST_ORDER "big-endian"
#else
#  define HOST_ORDER "mixed/other"
#endif

/* Decode a big-endian (network order) u32 from bytes. Correct on every host;
   no casts of unaligned pointers, no #ifdef per architecture. */
uint32_t load_be32(const unsigned char *p) {
    return (uint32_t)p[0] << 24 | (uint32_t)p[1] << 16 |
           (uint32_t)p[2] << 8  | (uint32_t)p[3];
}

int main(void) {
    const unsigned char hdr[] = { 0x00, 0x00, 0x20, 0xFB, 0xDE, 0xAD };  /* len=8443 */
    uint32_t raw;
    memcpy(&raw, hdr, sizeof raw);

    printf("host byte order : %s\n", HOST_ORDER);
    printf("memcpy as-is    : %u (0x%08X)\n", raw, raw);
    printf("load_be32       : %u (0x%08X)\n", load_be32(hdr), load_be32(hdr));
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra -O2 wire.c -o wire && ./wire
```

```
host byte order : little-endian
memcpy as-is    : 4213178368 (0xFB200000)
load_be32       : 8443 (0x000020FB)
```

The portable shift expression compiles to a single load plus byte swap on x86_64 with both compilers:

```
gcc -std=c23 -O2 -S -o - wire.c | sed -n '/^load_be32:/,/ret/p' | grep -v '^\s*\.'
```

```
load_be32:
    movl    (%rdi), %eax
    bswap   %eax
    ret
```

Use `__STDC_ENDIAN_NATIVE__` for what it is good at: a `static_assert` that a memory-mapped on-disk format really is native-endian, or picking a fast path. Do not build decoders that are only correct on one byte order.

Case 6: one shim for older libcs

glibc before 2.39 (still common in long-term-support distributions) has no `<stdbit.h>`. Wrap only the calls your project uses and test both paths. The fallback below is deliberately **64-bit only** — exactly the width restriction the real type-generic macros remove — and it guards the zero case the builtins leave undefined.

```
/* stdbit_compat.h - the three <stdbit.h> calls this project uses, on any
   GCC/Clang + any libc. Prefer the real header; fall back carefully. */
#ifndef STDBIT_COMPAT_H
#define STDBIT_COMPAT_H

#if defined(__has_include) && !defined(STDBIT_FORCE_FALLBACK)
#  if __has_include(<stdbit.h>)
#    include <stdbit.h>
#    define STDBIT_IMPL "libc <stdbit.h>"
#  endif
#endif

#ifndef STDBIT_IMPL
#  include <stdint.h>
#  define STDBIT_IMPL "fallback"
/* __builtin_clzll / __builtin_ctzll are UNDEFINED for 0 - guard it. */
static inline unsigned compat_bit_width_u64(uint64_t x) {
    return x ? 64u - (unsigned)__builtin_clzll(x) : 0u;
}
static inline unsigned compat_count_ones_u64(uint64_t x) {
    return (unsigned)__builtin_popcountll(x);
```

```
}
static inline unsigned compat_first_trailing_zero_u64(uint64_t x) {
    return ~x ? (unsigned)__builtin_ctzll(~x) + 1u : 0u;   /* 1-based, 0 = none */
}
#  define stdc_bit_width(x)          compat_bit_width_u64(x)
#  define stdc_count_ones(x)         compat_count_ones_u64(x)
#  define stdc_first_trailing_zero(x) compat_first_trailing_zero_u64(x)
#endif

#endif /* STDBIT_COMPAT_H */
```

```
/* compat.c - same results from the real header and the fallback. */
#include <stdint.h>
#include <stdio.h>
#include "stdbit_compat.h"

int main(void) {
    const uint64_t v[] = { 0, 1, 0xF0, UINT64_MAX };
    printf("impl: %s\n", STDBIT_IMPL);
    for (int i = 0; i < 4; i++)
        printf("x=%-20llu width=%2u ones=%2u first_trailing_zero=%2u\n",
               (unsigned long long)v[i], stdc_bit_width(v[i]),
               stdc_count_ones(v[i]), stdc_first_trailing_zero(v[i]));
    return 0;
}
```

```
gcc -std=c23 -Wall -Wextra compat.c -o c1 && ./c1
gcc -std=c17 -Wall -Wextra -DSTDBIT_FORCE_FALLBACK compat.c -o c2 && ./c2
```

```
impl: libc <stdbit.h>
x=0                   width= 0 ones= 0 first_trailing_zero= 1
x=1                   width= 1 ones= 1 first_trailing_zero= 2
x=240                 width= 8 ones= 4 first_trailing_zero= 1
x=18446744073709551615 width=64 ones=64 first_trailing_zero= 0
impl: fallback
x=0                   width= 0 ones= 0 first_trailing_zero= 1
x=1                   width= 1 ones= 1 first_trailing_zero= 2
x=240                 width= 8 ones= 4 first_trailing_zero= 1
x=18446744073709551615 width=64 ones=64 first_trailing_zero= 0
```

The trap

Integer promotion and signed arguments. The type-generic macros pick a width from the *type of the expression you pass*. In C, `uint8_t >> 1` is an `int`, and `int` is signed. The standard

says these macros take unsigned integer types only, but whether you get a diagnostic depends on the toolchain:

```c
/* trap.c - integer promotion and signedness meet type-generic bit macros. */
#include <stdbit.h>
#include <stdint.h>
#include <stdio.h>

int main(void) {
    uint8_t b = 0x10;                       /* 0001 0000 */
    int flags = -1;                         /* all 32 bits set */

    printf("leading_zeros(b)                 = %u\n", stdc_leading_zeros(b));
    printf("leading_zeros(b >> 1)            = %u\n", stdc_leading_zeros(b >> 1));
    printf("leading_zeros((uint8_t)(b >> 1)) = %u\n", stdc_leading_zeros((uint8_t)(b >> 1)));
    printf("count_ones(flags)                = %u\n", stdc_count_ones(flags));
    printf("count_ones((unsigned)flags)      = %u\n", stdc_count_ones((unsigned)flags));
    return 0;
}
```

GCC 14 maps the macros onto its `__builtin_stdc_*` builtins and rejects both mistakes:

```
gcc -std=c23 -Wall -Wextra trap.c -o trap
```

```
In file included from trap.c:2:
trap.c: In function ‘main’:
trap.c:11:55: error: argument 1 in call to function ‘__builtin_stdc_leading_zeros’ has signe
   11 |     printf("leading_zeros(b >> 1)            = %u\n", stdc_leading_zeros(b >> 1));
      |                                                       ^~~~~~~~~~~~~~~~~~
trap.c:13:55: error: argument 1 in call to function ‘__builtin_stdc_count_ones’ has signed t
   13 |     printf("count_ones(flags)                = %u\n", stdc_count_ones(flags));
      |                                                       ^~~~~~~~~~~~~~~
```

Clang 19 has no such builtins, so glibc's header falls back to a `sizeof`-based macro that converts the argument to `unsigned long long`. It compiles **without a warning** and prints:

```
clang -std=c23 -Wall -Wextra trap.c -o trap && ./trap
```

```
leading_zeros(b)                 = 3
leading_zeros(b >> 1)            = 28
leading_zeros((uint8_t)(b >> 1)) = 4
count_ones(flags)                = 64
count_ones((unsigned)flags)      = 32
```

`b >> 1` was counted as a 32-bit value (28 leading zeros instead of 4), and `-1` was sign-extended to 64 bits before counting (64 ones from a 32-bit `int`). Same source, two toolchains, one compile error and one silent wrong answer.

Related mistakes:

- **Forgetting the 1-based `first_*` convention** (Case 3) — subtract 1, and treat 0 as "none".
- **Using `stdc_bit_ceil` without checking for 0** (Case 2) — 0 means "does not fit", and `0 - 1` as a mask is all ones.
- **Swapping in `__builtin_clz` "because it's the same"** — it is not defined for 0 and is fixed at `unsigned int` width.
- **Treating endianness macros as a decoder** (Case 5) — detection is not conversion.

The boring rule

> Bit queries go through `<stdbit.h>` with an operand whose type is explicitly unsigned and of the width you mean — cast after any shift or arithmetic (`(uint8_t)(b >> 1)`, `(uint32_t)flags`). Treat `first_*` results as 1-based with 0 for "none", treat `bit_ceil` returning 0 as an error, and decode byte orders with shifts. Build with GCC 14 in at least one CI job so its builtins reject signed arguments the other toolchains accept.

Review checklist:

1. No new `__builtin_clz/ctz/popcount` calls outside a single compat header.
2. Every `stdc_*` argument is unsigned by type, not by hope; no shifted narrow types without a cast.
3. Every `stdc_first_*` result is tested for 0 and decremented before use.
4. Every `stdc_bit_ceil` result is tested for 0.
5. Wire-format decoders use byte shifts; `__STDC_ENDIAN_NATIVE__` appears only in `#if`/`static_assert`.

Try this

1. Add `stdc_leading_ones` to Case 1 for `x = 0xFF000000u` and predict the output before running it.
2. Extend Case 3 with `slot_alloc_highest()` using `stdc_first_leading_zero` on each word, scanning from the last word down. Remember the position counts from the MSB.
3. In Case 4, add a sample of `UINT64_MAX` and confirm it lands in bucket 64 with the upper bound printed correctly.
4. Write `load_le64` and `store_be16` in Case 5's style and check the assembly at `-O2` for single `mov`/`bswap`/`rol` instructions.
5. Compile `trap.c` with both compilers on your machine. If you only have Clang, add `-Wsign-conversion` and see whether it catches either line — then decide which CI job your team needs.

Sources

- ISO/IEC 9899:2024 (C23) §7.18 *Bit and byte utilities* — overview at cppreference: Bit manipulation (since C23)

- The GNU C Library manual (2.39) — Bit Manipulation
- glibc 2.39 release announcement — `<stdbit.h>` added

defer: Scope-Based Cleanup (TS 25755)

`defer`: Scope-Based Cleanup with TS 25755

Thesis

Every C programmer has written the cleanup ladder: acquire a file, a buffer, a lock, and on each failure `goto` the label that releases exactly what has been acquired so far. It works, and the Linux kernel is built on it. It is also where leaks and double-frees live, because the release code sits far from the acquisition and every new resource means editing two places in the right order.

WG14 answered with a **Technical Specification**, ISO/IEC TS 25755, *"defer, a mechanism for general purpose, lexical scope-based undo"* (project editor JeanHeyd Meneide). It adds one statement:

```
defer statement-or-block;      /* the keyword is _Defer; <stddefer.h> spells it defer */
```

The deferred block runs when control leaves the **enclosing block** for any reason the language can see: falling off the end, `return`, `break`, `continue`, or `goto` out of it. Multiple deferred blocks run in reverse order. Release now sits one line under acquisition, and the compiler, not you, maintains the ladder.

Where things stand in late 2026:

Implementation	Status
Clang 22.1+	Implemented (WG14 draft N3734) behind **`-fdefer-ts`**, C mode only. Clang 23.1.3, the current release, behaves the same and still marks the feature "subject to change".
GCC	Not in any release. The GCC 16 release notes (16.1/16.2) don't list it, and Debian's GCC 14.2 has no `<stddefer.h>`. Patches have been posted to gcc-patches.
The TS itself	WG14 finished the text in 2026 and sent it to ISO for publication. ISO's catalogue page still lists ISO/IEC TS 25755 as under development, so treat details as provisional.

A TS is not part of C23. It is an optional extension that compilers may implement and that may later be folded into C2y. This chapter uses Clang 23 for `defer` and shows the portable fallback that works on today's GCC.

Mental model

```
{                                   ← block E opens
    A = acquire_a();
    defer release(A);               ← registered (not run)
    B = acquire_b();
    defer release(B);               ← registered (not run)
    ... return / break / goto out / end of block ...
}                                   ← leaving E: run release(B), then release(A)
```

Rule	Consequence
Deferred blocks belong to the **enclosing block**, not the function	A `defer` inside a loop body runs at the end of *every iteration*. Go programmers, take note: Go's `defer` is function-scoped
They run **LIFO**	Release order is the reverse of acquisition order, which is almost always what you want
They read variables **when they run**	A deferred `printf("%d", n)` prints the value `n` has at scope exit, not at the `defer` line
`return expr;` evaluates `expr` **first**, then runs deferred blocks	A deferred block cannot change the function's return value
You may not jump **into** a defer's scope or **out of** a deferred block	`goto` past a `defer`, and `return`/`break`/`goto` inside a deferred block, are compile errors
`exit`, `abort`, `longjmp`, `_Exit`, signals do **not** run deferred blocks	`defer` is lexical. It is not a destructor and not a `finally` for the whole process

Worked examples

All listings were built and run on x86_64 Linux with **Clang 23.1.3** (LLVM release tarball) and Debian's **GCC 14.2.0**, glibc 2.41.

Case 0: is it there?

```c
#include <stdio.h>
#include <stddefer.h>
int main(void) {
#ifdef __STDC_DEFER_TS25755__
    printf("__STDC_DEFER_TS25755__ = %d\n", __STDC_DEFER_TS25755__);
#else
    printf("no defer TS\n");
#endif
    {
        defer printf("deferred\n");
        printf("body\n");
    }
    return 0;
}
```

```
clang -std=c23 -fdefer-ts -Wall -Wextra probe.c -o probe && ./probe
clang -std=c23 probe.c -o probe
gcc -std=c23 -c probe.c
```

```
__STDC_DEFER_TS25755__ = 1
body
deferred
probe.c:10:9: error: use of undeclared identifier 'defer'
   10 |         defer printf("deferred\n");
      |         ^~~~~
1 error generated.
probe.c:2:10: fatal error: stddefer.h: No such file or directory
    2 | #include <stddefer.h>
      |          ^~~~~~~~~~
compilation terminated.
```

Without `-fdefer-ts`, Clang's `<stddefer.h>` defines nothing, because the header only maps `defer` to `_Defer` when `__STDC_DEFER_TS25755__` is set. GCC 14 has no header at all. The macro's value is `1` here: Clang provides only the `_Defer` keyword, and the lowercase spelling comes from the header.

Case 1: when deferred blocks run

```c
/* order.c - when deferred blocks run: LIFO, per block, per loop iteration */
#include <stddefer.h>
#include <stdio.h>

static int step(const char *what, int v) {
```

```
    printf("  %s\n", what);
    return v;
}

static int answer(void) {
    int x = 1;
    defer { printf("  defer sees x=%d, sets it to 99\n", x); x = 99; }
    return step("return expression evaluated", x);
}

int main(void) {
    puts("1. LIFO within one block");
    {
        defer puts("  first registered, runs last");
        defer puts("  second registered, runs first");
        puts("  end of block body");
    }

    puts("2. block scope, not function scope");
    for (int i = 0; i < 3; i++) {
        defer printf("  cleanup for i=%d\n", i);
        if (i == 1)
            continue;               /* still runs the defer */
        printf("  body i=%d\n", i);
    }

    puts("3. the value is captured at exit, not at the defer statement");
    {
        int n = 1;
        defer printf("  deferred print sees n=%d\n", n);
        n = 42;
    }

    puts("4. return value is computed before deferred blocks run");
    printf("  answer() returned %d\n", answer());
    return 0;
}
```

```
clang -std=c23 -fdefer-ts -Wall -Wextra order.c -o order && ./order
```

```
1. LIFO within one block
  end of block body
  second registered, runs first
  first registered, runs last
2. block scope, not function scope
  body i=0
```

```
  cleanup for i=0
  cleanup for i=1
  body i=2
  cleanup for i=2
3. the value is captured at exit, not at the defer statement
  deferred print sees n=42
4. return value is computed before deferred blocks run
  return expression evaluated
  defer sees x=1, sets it to 99
  answer() returned 1
```

Section 2 shows the property that makes `defer` useful in loops: the `continue` at `i == 1` skipped the body's `printf`, but not the cleanup. Section 4 is the one that bites. Keep it in mind for Case 2.

Case 2: replacing a cleanup ladder

The task: read up to `max` bytes from one file, upper-case them, and write them to another. There are three resources and five failure points. Here is the classic version:

```c
/* upcase_goto.c - classic C cleanup: one label per acquired resource */
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>

int upcase_file(const char *in_path, const char *out_path, size_t max) {
    int rc = -1;
    FILE *in = fopen(in_path, "rb");
    if (!in) { perror(in_path); goto out; }

    char *buf = malloc(max);
    if (!buf) { perror("malloc"); goto close_in; }

    FILE *out = fopen(out_path, "wb");
    if (!out) { perror(out_path); goto free_buf; }

    size_t n = fread(buf, 1, max, in);
    if (ferror(in)) { perror("fread"); goto close_out; }
    for (size_t i = 0; i < n; i++)
        buf[i] = (char)toupper((unsigned char)buf[i]);
    if (fwrite(buf, 1, n, out) != n) { perror("fwrite"); goto close_out; }
    rc = (int)n;

close_out:
    if (fclose(out) != 0 && rc >= 0) { perror("fclose"); rc = -1; }
free_buf:
```

```
    free(buf);
close_in:
    fclose(in);
out:
    return rc;
}

int main(int argc, char **argv) {
    if (argc != 3) { fprintf(stderr, "usage: %s IN OUT\n", argv[0]); return 2; }
    int n = upcase_file(argv[1], argv[2], 4096);
    printf("result: %d\n", n);
    return n < 0;
}
```

And a direct translation to `defer`:

```
/* upcase_defer.c - the same function with TS 25755 defer */
#include <ctype.h>
#include <stddefer.h>
#include <stdio.h>
#include <stdlib.h>

int upcase_file(const char *in_path, const char *out_path, size_t max) {
    FILE *in = fopen(in_path, "rb");
    if (!in) { perror(in_path); return -1; }
    defer fclose(in);

    char *buf = malloc(max);
    if (!buf) { perror("malloc"); return -1; }
    defer free(buf);

    FILE *out = fopen(out_path, "wb");
    if (!out) { perror(out_path); return -1; }
    int rc = -1;
    defer {
        if (fclose(out) != 0 && rc >= 0) { perror("fclose"); rc = -1; }
    }

    size_t n = fread(buf, 1, max, in);
    if (ferror(in)) { perror("fread"); return -1; }
    for (size_t i = 0; i < n; i++)
        buf[i] = (char)toupper((unsigned char)buf[i]);
    if (fwrite(buf, 1, n, out) != n) { perror("fwrite"); return -1; }
    rc = (int)n;
    return rc;
}
```

```
int main(int argc, char **argv) {
    if (argc != 3) { fprintf(stderr, "usage: %s IN OUT\n", argv[0]); return 2; }
    int n = upcase_file(argv[1], argv[2], 4096);
    printf("result: %d\n", n);
    return n < 0;
}
```

Build both with AddressSanitizer (which includes LeakSanitizer on Linux), then run each against a good input, a missing input, an unwritable directory, and `/dev/full`. `/dev/full` accepts `fwrite` into stdio's buffer but fails when the buffer is flushed at `fclose`.

```
printf 'hello, defer\n' > in.txt
for v in goto defer; do
  clang -std=c23 -fdefer-ts -g -O1 -fsanitize=address -Wall -Wextra upcase_$v.c -o up_$v
  echo "== $v"
  ./up_$v in.txt out.txt; cat out.txt
  ./up_$v missing.txt out.txt; echo "exit=$?"
  ./up_$v in.txt /nonexistent/dir/out.txt; echo "exit=$?"
  ./up_$v in.txt /dev/full; echo "exit=$?"
done
```

```
== goto
result: 13
HELLO, DEFER
missing.txt: No such file or directory
result: -1
exit=1
/nonexistent/dir/out.txt: No such file or directory
result: -1
exit=1
fclose: No space left on device
result: -1
exit=1
== defer
result: 13
HELLO, DEFER
missing.txt: No such file or directory
result: -1
exit=1
/nonexistent/dir/out.txt: No such file or directory
result: -1
exit=1
fclose: No space left on device
result: 13
exit=0
```

No sanitizer reports, so both versions release everything on every path. The `defer` version is shorter

and harder to get wrong when a fourth resource arrives. **But look at the last run.** The `defer` version printed the `fclose` error and then reported success. `return rc;` evaluated `rc` (13) before the deferred block ran, so setting `rc = -1` inside the deferred block changed a local that nobody would read again. The data was lost and the caller was told everything was fine.

Case 3: release with `defer`, commit explicitly

The fix is a rule, not a trick. Use `defer` for releases whose failure you can't act on: `free`, closing a file you only read, unlocking. An operation whose failure *is* the result, such as closing or `fsync`ing a file you wrote, or `COMMIT`, goes on the success path explicitly. The deferred block only covers the error paths.

```c
/* upcase_defer_fixed.c - defer for release, an explicit close where the error matters */
#include <ctype.h>
#include <stdbool.h>
#include <stddefer.h>
#include <stdio.h>
#include <stdlib.h>

int upcase_file(const char *in_path, const char *out_path, size_t max) {
    FILE *in = fopen(in_path, "rb");
    if (!in) { perror(in_path); return -1; }
    defer fclose(in);                       /* read side: close errors don't matter */

    char *buf = malloc(max);
    if (!buf) { perror("malloc"); return -1; }
    defer free(buf);

    FILE *out = fopen(out_path, "wb");
    if (!out) { perror(out_path); return -1; }
    bool out_closed = false;
    defer { if (!out_closed) fclose(out); } /* error paths only */

    size_t n = fread(buf, 1, max, in);
    if (ferror(in)) { perror("fread"); return -1; }
    for (size_t i = 0; i < n; i++)
        buf[i] = (char)toupper((unsigned char)buf[i]);
    if (fwrite(buf, 1, n, out) != n) { perror("fwrite"); return -1; }

    out_closed = true;                      /* success path: the close is the commit */
    if (fclose(out) != 0) { perror("fclose"); return -1; }
    return (int)n;
}

int main(int argc, char **argv) {
    if (argc != 3) { fprintf(stderr, "usage: %s IN OUT\n", argv[0]); return 2; }
```

```
    int n = upcase_file(argv[1], argv[2], 4096);
    printf("result: %d\n", n);
    return n < 0;
}
```

```
clang -std=c23 -fdefer-ts -g -O1 -fsanitize=address -Wall -Wextra upcase_defer_fixed.c -o up
./up_fixed in.txt out.txt; echo "exit=$?"
./up_fixed missing.txt out.txt; echo "exit=$?"
./up_fixed in.txt /dev/full; echo "exit=$?"
```

```
result: 13
exit=0
missing.txt: No such file or directory
result: -1
exit=1
fclose: No space left on device
result: -1
exit=1
```

Case 4: the same shape on today's GCC

Until GCC ships the TS, the portable way to get scope-bound cleanup is the `cleanup` variable attribute, which GCC and Clang have supported for many years. It attaches a function to a *variable*, not to a block of code, so you write one small helper per resource type. The "disarm, then close explicitly" pattern from Case 3 carries over unchanged.

```
/* upcase_cleanup.c - pre-TS idiom: GCC/Clang __attribute__((cleanup)) */
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>

static void close_file(FILE **f) { if (*f) fclose(*f); }
static void free_mem(char **p)    { free(*p); }

#define AUTO_FILE __attribute__((cleanup(close_file))) FILE *
#define AUTO_MEM  __attribute__((cleanup(free_mem))) char *

int upcase_file(const char *in_path, const char *out_path, size_t max) {
    AUTO_FILE in = fopen(in_path, "rb");
    if (!in) { perror(in_path); return -1; }

    AUTO_MEM buf = malloc(max);
    if (!buf) { perror("malloc"); return -1; }

    AUTO_FILE out = fopen(out_path, "wb");
```

```
    if (!out) { perror(out_path); return -1; }

    size_t n = fread(buf, 1, max, in);
    if (ferror(in)) { perror("fread"); return -1; }
    for (size_t i = 0; i < n; i++)
        buf[i] = (char)toupper((unsigned char)buf[i]);
    if (fwrite(buf, 1, n, out) != n) { perror("fwrite"); return -1; }

    FILE *o = out;
    out = NULL;                         /* disarm: we close it ourselves */
    if (fclose(o) != 0) { perror("fclose"); return -1; }
    return (int)n;
}

int main(int argc, char **argv) {
    if (argc != 3) { fprintf(stderr, "usage: %s IN OUT\n", argv[0]); return 2; }
    int n = upcase_file(argv[1], argv[2], 4096);
    printf("result: %d\n", n);
    return n < 0;
}
```

```
gcc -std=c23 -g -O1 -fsanitize=address -Wall -Wextra upcase_cleanup.c -o up_cleanup_gcc
./up_cleanup_gcc in.txt out.txt; echo "exit=$?"
./up_cleanup_gcc missing.txt out.txt; echo "exit=$?"
./up_cleanup_gcc in.txt /dev/full; echo "exit=$?"
clang -std=c23 -Wall -Wextra upcase_cleanup.c -o up_cleanup_clang && ./up_cleanup_clang in.t
```

```
result: 13
exit=0
missing.txt: No such file or directory
result: -1
exit=1
fclose: No space left on device
result: -1
exit=1
result: 13
```

goto ladder	`__attribute__((cleanup))`	TS 25755 `defer`
Standard C	Yes	No (GNU extension)	Optional TS, Clang only today
Unit of cleanup	Label	Variable	Any statement or block

`goto` ladder	`__attribute__((cleanup))`	TS 25755 `defer`
Arbitrary cleanup code (`unlock`, `printf`, `rollback`)	Yes	Only via a helper per type	Yes, inline
Runs on `continue`/`break` out of a loop body	Only if you wrote it	Yes	Yes
Can affect the return value	Yes	No	No

The trap

Jumps the compiler rejects

```
/* illegal.c - jumps the defer TS forbids */
#include <stddefer.h>
#include <stdio.h>
#include <stdlib.h>

int jump_past(int fail) {
    if (fail)
        goto done;                  /* jumps over the defer into its scope */
    char *p = malloc(16);
    defer free(p);
done:
    return 0;
}

int leave_from_defer(void) {
    defer {
        return 1;                   /* cannot leave a deferred block early */
```

```
    }
    return 0;
}
```

```
clang -std=c23 -fdefer-ts -c illegal.c
```

```
illegal.c:8:9: error: cannot jump from this goto statement to its label
    8 |         goto done;                /* jumps over the defer into its scope */
      |         ^
illegal.c:10:5: note: jump bypasses defer statement
   10 |     defer free(p);
      |     ^
/opt/llvm-23/lib/clang/23/include/stddefer.h:16:15: note: expanded from macro 'defer'
   16 | #define defer _Defer
      |               ^
illegal.c:17:9: error: cannot return from a defer statement
   17 |         return 1;                /* cannot leave a deferred block early */
      |         ^
2 errors generated.
```

Both are good errors. A `goto` that skips a `defer` would leave the compiler unsure whether to run it. A deferred block that returns would abandon the remaining cleanup. The fix for the first is usually to open a new block `{ … }` for the resource, so the label sits outside the defer's scope.

Leaving without leaving the block

```
/* exit_skips.c - exit() and longjmp leave without running deferred blocks */
#include <stddefer.h>
#include <stdio.h>
#include <stdlib.h>

static void work(int code) {
    defer puts("deferred cleanup ran");
    if (code)
        exit(code);
    puts("returning normally");
}

int main(int argc, char **argv) {
    (void)argv;
    work(argc > 1);
    return 0;
}
```

```
clang -std=c23 -fdefer-ts -Wall exit_skips.c -o exit_skips
./exit_skips; echo "exit=$?"
./exit_skips now; echo "exit=$?"
```

```
returning normally
deferred cleanup ran
exit=0
exit=1
```

`exit()` never returns to `work`, so the deferred `puts` never runs. The same is true of `abort`, `_Exit`, `longjmp` over the frame, and a fatal signal. Memory and file descriptors are reclaimed by the kernel anyway. A temporary file that should have been `unlink`ed, or a lock file, is not. Process-wide cleanup still belongs in `atexit` handlers or the caller.

A homegrown `defer` macro meets `<stddefer.h>`

Plenty of codebases already have a function-like macro called `defer`, usually a runtime cleanup stack like the one in the advanced-preprocessor chapter. `<stddefer.h>` is a system header. Its `#define defer _Defer` replaces yours without a redefinition warning, and every old call site still parses:

```
// clash.c
#include <stdio.h>
#include <stdlib.h>

static void defer_cleanup(void (*func)(void *), void *arg) {
    puts("registered with the old runtime stack");
    (void)func; (void)arg;
}
#define defer(func, arg) defer_cleanup(func, arg)   /* legacy macro */

#include <stddefer.h>                                /* added later */

static void free_ptr(void *p) { free(*(void **)p); puts("freed"); }

int main(void) {
    void *buf = malloc(64);
    defer(free_ptr, &buf);
    puts("body");
    return 0;
}
```

```
$ clang -std=c23 -fdefer-ts -Wall -Wextra clash.c -o clash
clash.c:17:11: warning: left operand of comma operator has no effect [-Wunused-value]
   17 |     defer(free_ptr, &buf);
```

```
      |           ^~~~~~~~
clash.c:17:21: warning: expression result unused [-Wunused-value]
   17 |     defer(free_ptr, &buf);
      |                     ^~~~
clash.c:5:13: warning: unused function 'defer_cleanup' [-Wunused-function]
    5 | static void defer_cleanup(void (*func)(void *), void *arg) {
      |             ^~~~~~~~~~~~~
3 warnings generated.
$ ./clash
body
$ clang -std=c23 -fdefer-ts -g -fsanitize=address clash.c -o clash_asan && ./clash_asan
body

=================================================================
==460565==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 64 byte(s) in 1 object(s) allocated from:
    #0 0x563937b29b94 in malloc /home/runner/work/llvm-project/llvm-project/compiler-rt/lib/
    #1 0x563937b6b7d8 in main /workspace/scratch/cdefer/clash.c:16:17
...
```

`defer(free_ptr, &buf);` is now `_Defer (free_ptr, &buf);`. That defers a comma expression that does nothing. The old runtime never sees the registration and the buffer leaks. The only clues are `-Wunused-value` and `-Wunused-function`. Before you include `<stddefer.h>`, rename the legacy macro (`git grep -n 'define defer'`), or use only the `_Defer` keyword and leave the header out.

Mutating the result from a deferred block

Case 2's `/dev/full` run is the trap. A deferred block that assigns to the variable being returned looks like Go's named-result trick, but C has no named results. `return rc;` copies the value before the deferred blocks run. If a cleanup step can fail in a way the caller must hear about, it isn't cleanup. Do it explicitly on the success path (Case 3).

The boring rule

> Put `defer release(x);` on the line right after `x` is acquired and checked. Use `defer` only for releases whose failure you can ignore: `free`, read-side `fclose`, unlocking, restoring a saved value. Writes are committed explicitly on the success path, with the deferred block covering only the error paths through a "done" flag. Remember that deferred blocks are per block (and per loop iteration) and are skipped by `exit` and `longjmp`. Gate the code on `__STDC_DEFER_TS25755__`, build it with `-fdefer-ts`, and keep a `cleanup`-attribute fallback while GCC catches up.

Try this

1. Add a fourth resource to `upcase_goto.c` (say, a second output buffer) and count how many lines change. Do the same in `upcase_defer_fixed.c`.
2. Write a `with_lock(pthread_mutex_t *m)` pattern: `pthread_mutex_lock(m); defer pthread_mutex_unlock(m);` inside a loop that `continue`s on odd iterations, and prove with a counter that the mutex is unlocked every time.
3. Wrap Case 3 in a header that uses `defer` when `__STDC_DEFER_TS25755__` is defined and the `cleanup` attribute otherwise. Build it with both compilers.
4. Move the `defer free(buf);` *above* the `if (!buf)` check. Is `free(NULL)` on the failure path a bug? What about `defer fclose(in);` above `if (!in)`?
5. Replace `exit(code)` in `exit_skips.c` with a `longjmp` to a `setjmp` in `main` and confirm the deferred block is still skipped. Then read the TS's wording on `longjmp` and explain why the compiler can't help here.

Sources

- WG14, ISO/IEC TS 25755 working draft N3928, *defer, a mechanism for general purpose, lexical scope-based undo* (ed. JeanHeyd Meneide): https://thephd.dev/_vendor/future_cxx/technical%20specification/C%20-%20defer/C%20-%20defer%20Technical%20Specification.pdf
- WG14 N3734 (the draft Clang implements): https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3734.pdf
- ISO catalogue entry, ISO/IEC TS 25755: https://www.iso.org/standard/91402.html
- Clang 22 release notes ("Implemented the `defer` draft Technical Specification … `-fdefer-ts`"): https://releases.llvm.org/22.1.0/tools/clang/docs/ReleaseNotes.html
- LLVM commit 71bfdd1, "Clang Add support for the C `_Defer` TS (#162848)": https://github.com/llvm/llvm-project/commit/71bfdd130403
- GCC 16 release series changes (no `defer` entry): https://gcc.gnu.org/gcc-16/changes.html
- GCC manual, `cleanup` variable attribute: https://gcc.gnu.org/onlinedocs/gcc/Common-Variable-Attributes.html

GCC -ftrivial-auto-var-init: Zero and Pattern Init

`-ftrivial-auto-var-init`: Making Uninitialized Stack Boring

Thesis

Uninitialized automatic (stack) variables are a classic C footgun: reads are undefined behaviour, and in practice they leak whatever previous frame left behind — sometimes secrets, often "works on my machine" nondeterminism. Fixing every declaration by hand does not scale across a large codebase. GCC and Clang offer a compile-time mitigator:

```
-ftrivial-auto-var-init=zero     # or =pattern
```

The compiler inserts initialization for automatic variables that lack an explicit initializer. Security-sensitive and predictability-sensitive builds can flip the default without rewriting every function. This is not a substitute for correct initialization in new code, and it has sharp edges around **padding**, **partial writes**, **unions**, and **control-flow corners** (notably some `switch` locals). This chapter measures the flag on the box's GCC and documents what upstream says beyond that version.

Where things stand (2026-10)

Toolchain	Status
GCC 14.2.0 (Debian on this lab box)	`-ftrivial-auto-var-init=uninitialized|pattern` present. Default remains `uninitialized`. Docs: initializes autos **and** struct/union **padding** to zero when the flag is on. No `-fzero-init-padding-bits=` on 14.2.
GCC 15+	Still has `-ftrivial-auto-var-init`. Separately, GCC 15 tightened when **explicit** initializers zero union padding; use `-fzero-init-padding-bits=unions` (or `=all`) / `{}` (C23) when you need the old "zero all the bytes" behaviour for initialized objects. That flag is **not** the same as trivial-auto-var-init.

Toolchain	Status
GCC 16	Confirm in the release notes of the GCC you ship; this chapter's runnable cases use **14.2.0**. Treat 16 as "re-read Optimize Options before relying on new padding knobs."
Clang	Has supported the same idea longer; historically `=zero` needed an extra "I know this may go away" gate on some versions. Not installed on this lab box — verify your Clang's `clang --help` before copying flags.

GCC's own docs are clear on a subtlety: even with `=zero` or `=pattern`, the compiler **still treats** the variable as uninitialized for `-Wuninitialized` / analyzer purposes and optimizes *as if* it were uninitialized. The inserted stores are a safety net for the abstract machine's UB-shaped holes, not a promise that tools will stop nagging.

Mental model

```
auto object, no initializer

          default (=uninitialized):  contents = leftover stack garbage
          =zero:                     all bytes 0x00 (fields + padding)
          =pattern:                  fields get 0xFE.. pattern;
                                      padding still zeroed (GCC docs)
```

Choice	Typical use
`uninitialized`	Language default; fastest; UB on read before write
`zero`	Production hardening: predictable, matches "calloc-shaped" intuition
`pattern`	Bug finding: 0xFE repeats stand out in crash dumps; do not rely on the value as API

Worked examples

All listings built and run on x86_64 Linux with **GCC 14.2.0** (Debian 14.2.0-19). Clang was not available on the lab host.

Case 0: does the flag exist?

```
gcc -Q --help=optimizers 2>/dev/null | grep trivial-auto-var-init
```

```
  -ftrivial-auto-var-init=[uninitialized|pattern|zero]  uninitialized
```

Case 1: garbage vs zero vs pattern

```
/* tav_demo.c - stack pollution, then an uninitialized struct */
#include <stdio.h>
#include <stdint.h>

struct Padded {
    char a;   /* offset 0 */
    /* 3 bytes padding */
    int  b;   /* offset 4 */
    char c;   /* offset 8 */
    /* 3 bytes end padding → sizeof 12 on this ABI */
};

__attribute__((noinline)) void pollute_stack(void) {
    volatile unsigned char junk[512];
    volatile unsigned char sink = 0;
    for (int i = 0; i < 512; i++) {
        junk[i] = (unsigned char)(0xA5 ^ (i * 17));
        sink ^= junk[i];
    }
    (void)sink;
}

__attribute__((noinline, optimize("O0"))) void show_uninit(const char *label) {
    struct Padded s; /* no initializer on purpose */
    unsigned char *p = (unsigned char *)&s;
    int nz = 0;
    printf("%s sizeof=%zu bytes:", label, sizeof s);
    for (size_t i = 0; i < sizeof s; i++) {
        printf(" %02x", p[i]);
        if (p[i]) nz++;
    }
    printf("  (nonzero=%d)\n", nz);
}

int main(void) {
    pollute_stack();
```

```
    show_uninit("uninit");
    return 0;
}
```

```
gcc -O0 tav_demo.c -o tav_def && ./tav_def
gcc -O0 -ftrivial-auto-var-init=zero tav_demo.c -o tav_z && ./tav_z
gcc -O0 -ftrivial-auto-var-init=pattern tav_demo.c -o tav_p && ./tav_p
```

Representative output from this lab (the default line's exact bytes vary; nonzero count is the point):

```
# default
uninit sizeof=12 bytes: 09 18 6b 7a 55 a4 b7 86 91 e0 f3 c2  (nonzero=12)
# =zero
uninit sizeof=12 bytes: 00 00 00 00 00 00 00 00 00 00 00 00  (nonzero=0)
# =pattern  (0xFE in fields; padding bytes are 00)
uninit sizeof=12 bytes: fe 00 00 00 fe fe fe fe fe 00 00 00  (nonzero=6)
```

Read the pattern line carefully: `a=fe`, padding `00 00 00`, `b=fe fe fe fe`, `c=fe`, end padding `00 00 00`. GCC documents that with this option it **also zeros padding** of automatic structs/unions. Pattern init is for *named members*; padding stays zero.

Case 2: partial writes still surprise people

```
/* same struct; write a and b only */
__attribute__((noinline, optimize("O0"))) void show_partial(const char *label) {
    struct Padded s;
    s.a = 'X';           /* 0x58 */
    s.b = 0x11223344;
    /* c never written */
    unsigned char *p = (unsigned char *)&s;
    printf("%s after a+b writes:", label);
    for (size_t i = 0; i < sizeof s; i++) printf(" %02x", p[i]);
    putchar('\n');
}
```

```
# default (c + padding still garbage from the stack)
partial after a+b writes: 58 e0 f3 c2 44 33 22 11 19 68 7b 4a
# =zero (unwritten c and padding stay 00)
partial after a+b writes: 58 00 00 00 44 33 22 11 00 00 00 00
# =pattern (unwritten c stays 0xFE; padding 00)
partial after a+b writes: 58 00 00 00 44 33 22 11 fe 00 00 00
```

Without the flag, writing two fields does **not** scrub the rest of the object. `memcmp`, `memcpy` of the whole struct, hashing, or putting the bytes on the wire will ship padding and untouched members.

With `=zero`, the unwritten pieces stay zero after the compiler's inserted init — until you overwrite them.

Case 3: unions

```
union U {
    uint32_t w;
    unsigned char bytes[4];
};

__attribute__((noinline, optimize("O0"))) void show_union(const char *label) {
    union U u; /* no initializer */
    printf("%s union bytes:", label);
    for (int i = 0; i < 4; i++) printf(" %02x", u.bytes[i]);
    putchar('\n');
}
```

```
# default (example)
union union bytes: 00 02 00 00
# =zero
union union bytes: 00 00 00 00
# =pattern
union union bytes: fe fe fe fe
```

For *uninitialized* autos, trivial-auto-var-init covers the storage. For **explicitly initialized** unions/structs, C's rules about padding are a different story: GCC 15 changed some cases where union padding used to be zeroed and introduced `-fzero-init-padding-bits=`. On GCC 14.2 that flag does not exist (`unrecognized command-line option`). If you need every byte zero under an initializer, `memset(&obj, 0, sizeof obj)` or C23 `{}` remain the portable hammers.

Case 4: warnings do not go away

```
gcc -O0 -Wall -ftrivial-auto-var-init=zero tav_demo.c -o /dev/null
```

You will still see `-Wmaybe-uninitialized` (and friends) on reads of `s` / `u`. That is intentional per GCC docs: the program is patched for safety, but the language-level "this was never initialized" analysis remains. Do not use the flag as an excuse to ignore warnings on hot paths you own.

The trap

- **Believing `{0}` always zeros padding for automatic structs.** Static-duration objects get zeroed storage; automatic padding has historically been the weak spot. Trivial-auto-var-init helps the *uninitialized* case; explicit initializers need their own discipline (`memset`, C23 `{}`, or

GCC 15+ padding flags).

- **Shipping `=pattern` to production.** 0xFE is for crashing loudly in test builds. Production hardening usually wants `=zero`.
- **Assuming cost is free.** The compiler inserts stores (often `memset`-shaped). Measure on your hot functions; prefer enabling it globally in security builds and opting out only where profiles demand.
- **Locals declared between `switch (x)` and the first `case`.** GCC documents that the current implementation cannot initialize those; `-Wtrivial-auto-var-init` reports them. Move declarations into the case scopes or before the switch.
- **Thinking the flag initializes heap memory.** It does not. `malloc` is still raw; use `calloc` / `memset`.

The boring rule

New code: initialize on declaration (`= {0}`, C23 `{}`, or real values). Large legacy trees: compile with `-ftrivial-auto-var-init=zero` in hardened profiles, keep `-Wall -Wextra`, and `memset` any object you put on the wire or compare by raw bytes. Re-check GCC release notes when you jump majors — padding knobs have been moving.

Try this

1. Run Case 1 on your laptop GCC/Clang; confirm `=pattern` padding zeros.
2. Add `-Wtrivial-auto-var-init` to a `switch`-heavy file and fix any reported locals.
3. Benchmark one allocator-heavy translation unit with and without `=zero` at `-O2`.

Sources

- GCC Optimize Options — `-ftrivial-auto-var-init`: https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Optimize-Options.html#index-ftrivial-auto-var-init
- GCC 15 changes (padding / `-fzero-init-padding-bits`): https://gcc.gnu.org/gcc-15/changes.html
- Memfault — C structure padding initialization (background): https://interrupt.memfault.com/blog/c-struct-padding-initialization

GCC 16: What Changed for C

GCC 16: What Changed for C

Thesis

GCC **16.1** shipped on **30 April 2026**. The headline everyone saw on social media was the C++ default flipping from GNU C++17 to GNU C++20. That matters for C++ shops. For **C** shops the story is quieter but not empty: new GNU extensions (`_Maxof` / `_Minof`, richer `counted_by`), more C2y diagnostics treated as constraint violations, SARIF becoming the only machine-readable diagnostic sink (`json` removed), and stricter levels for `-Wunused-but-set-*` when you enable `-Wall` / `-Wextra`.

This chapter is the C migration checklist. It cites the GCC 16 changes page and the companion porting notes. **GCC 16 is not installed on this lab box** (Debian ships **14.2.0**). Runnable probes below use 14.2; GCC 16 behaviour is documented from upstream, not invented.

Where things stand (2026-10)

Toolchain	Status for this chapter
GCC 14.2.0 (Debian on this lab)	Default C dialect is **GNU C17**. `-ftrivial-auto-var-init` and C23 flags work; no `_Maxof` / `_Minof`.
GCC 15	Default C dialect moved to **GNU C23** (`-std=gnu23`). Still the baseline many distros will hold through late 2026.
GCC 16.1 / 16.2	C++ default → `gnu++20`. C keeps the C23-era default; adds the C section features listed below. Do **not** assume "builds on 15 builds on 16" without pinning `-std=` and re-reading warnings.

Mental model

```
Your Makefile / meson / cmake

           never pins -std= …  →  you silently inherit the compiler default
                                  (C17 on GCC 14, C23/gnu23 on 15+, C++20 on GCC 16++)
           pins -std=gnu23 / c23 / gnu17 …
                                  →  behaviour-stable across toolchain bumps

Machine-readable diagnostics
```

```
GCC 15:  -fdiagnostics-format=json   (removed in 16)
GCC 16+:  SARIF only (-fdiagnostics-format=sarif / add-output=sarif)
```

Rule	Consequence
Pin the language dialect in the build	A default change never surprises CI
Treat warning-level bumps as build breaks	`-Werror` plus stricter unused-but-set levels will fail overnight
Prefer SARIF for bots	Text scrapers and the old `json` sink die with GCC 16
Read the **C** section of the changes page	Do not migrate a C codebase off C++-only advice

What actually changed for C in GCC 16

From the official **C** section of the GCC 16 changes page (mirrored; `gcc.gnu.org` sometimes returns 403 to automated fetchers):

GNU extensions

1. **`counted_by` on pointer members.** The attribute that tells the analyzer / bounds sanitizers how long a flexible array is can now annotate **pointer** members as well as flexible-array members. Useful for "pointer + length" structs that never used a trailing array.
2. **`_Maxof` and `_Minof`.** Operators that yield the maximum / minimum representable value of an integer type — the compile-time cousin of `<limits.h>` macros, without naming the type twice.
3. **Compound literals with empty initializers and a variable size.** A VLA-shaped compound literal `(T[n]){}` is now accepted where the older front end rejected it.
4. **Nested functions that do not touch the enclosing frame** are guaranteed **not** to need a trampoline, even at `-O0`. That removes a classic executable-stack surprise for pure helpers nested for scope only.

C2y alignment

GCC 16 also lands more of the C2y draft:

- Static assertions usable **inside expressions**.
- Unspecified and variably-modified arrays usable as type names in `_Generic` associations.
- Situations that were silent undefined behaviour in older modes and are now **constraint violations** (hard diagnostics).

None of these require you to pass `-std=c2y` tomorrow morning; they matter when you *do* raise the dialect and when you read new errors on old code.

Diagnostics and tooling (C and C++ share these)

- The diagnostic format named `json` for `-fdiagnostics-format=` is **removed**. Machine consumers must move to **SARIF**.
- SARIF sinks respect the dump / output directory, nest logical locations, and can carry control-flow kinds (`throw`, `catch`, `setjmp`, …).
- Optional HTML diagnostic output via `-fdiagnostics-add-output=experimental-html`.
- `-fanalyzer` gained more C++ awareness; for pure C it remains useful. New `-fanalyzer-assume-nothrow` quiets exception-shaped assumptions when you compile C with `-fexceptions` only for C++ interop.

Warnings: unused-but-set levels

The porting guide (community write-ups quoting it, including the ELCSoftware migration note of 3 May 2026) documents that **`-Wunused-but-set-variable`** and **`-Wunused-but-set-parameter`** now have **levels**, and the defaults attached to `-Wall` / `-Wextra` are **stricter**. Patterns that only touch a variable through `++` / `--` / compound assignment can newly warn. Mitigation if you cannot fix the code yet:

```
-Wunused-but-set-variable=1 -Wunused-but-set-parameter=1
```

Prefer deleting the dead store, using the value, or `(void)x`.

Worked examples

All runnable cases: **GCC 14.2.0** on x86_64 Linux (Debian). Cases marked "docs only" describe GCC 16 APIs that 14.2 does not implement.

Case 0: what default dialect do we actually get?

```
# probe_default_std.sh - ask cc1 what language it thinks it is compiling
echo 'int main(void){return 0;}' > /tmp/probe.c
gcc -v -c /tmp/probe.c -o /tmp/probe.o 2>&1 | grep 'GNU C'
```

```
GNU C17 (Debian 14.2.0-19) version 14.2.0 (x86_64-linux-gnu)
```

On this box the default is still **C17**. On a GCC 15/16 install you should see **GNU C23** (or whichever default that release documents). Pin anyway:

```
CFLAGS ?= -std=gnu23 -Wall -Wextra
```

Case 1: pin the standard so GCC 16 cannot surprise you

```c
/* pin_std.c - force the dialect; works on 14, 15, and 16 */
#if !defined(__STDC_VERSION__)
#  error "expected a C compiler"
#endif

#include <stdio.h>

int main(void) {
    printf("STDC_VERSION=%ld\n", (long)__STDC_VERSION__);
#if defined(__GNUC__)
    printf("GCC %d.%d\n", __GNUC__, __GNUC_MINOR__);
#endif
    return 0;
}
```

```
gcc -std=gnu17 pin_std.c -o pin17 && ./pin17
gcc -std=gnu23 pin_std.c -o pin23 && ./pin23
```

Representative output on GCC 14.2:

```
STDC_VERSION=201710
GCC 14.2
STDC_VERSION=202000
GCC 14.2
```

(`202000L` is GCC's current stand-in for C23; the exact macro value can move as WG14 finalises.)

Case 2: `_Maxof` / `_Minof` (GCC 16 — docs only on this lab)

```c
/* maxof_demo.c - requires GCC 16+; will not compile on 14.2 */
#include <stdio.h>

int main(void) {
    printf("max int = %d\n", (int)_Maxof(int));
    printf("min int = %d\n", (int)_Minof(int));
    printf("max unsigned = %u\n", (unsigned)_Maxof(unsigned));
    return 0;
}
```

```
# On this lab (GCC 14.2):
gcc -std=gnu23 maxof_demo.c -o maxof_demo
# error: '_Maxof' undeclared (expected - feature is GCC 16)
```

```
# On GCC 16 (from docs; not run here):
# gcc-16 -std=gnu23 maxof_demo.c -o maxof_demo && ./maxof_demo
```

Honest status: **not run** — GCC 16 is not installed, and this scout run must not `apt install` toolchains. When you have `gcc-16`, the listing above is the smoke test.

Case 3: `counted_by` on a pointer member (shape for GCC 16)

```c
/* counted_by_ptr.c - attribute shape; analyzer benefit is GCC 16+ */
#include <stddef.h>
#include <stdlib.h>

struct Buf {
    size_t n;
    char *data __attribute__((counted_by(n)));
};

struct Buf *make(size_t n) {
    struct Buf *b = malloc(sizeof *b);
    if (!b) return NULL;
    b->n = n;
    b->data = calloc(n, 1);
    if (!b->data) { free(b); return NULL; }
    return b;
}
```

On older GCC the attribute on a **pointer** member was ignored or rejected; GCC 16 documents pointer-member support. Keep the length field authoritative and update it before the pointer when you grow the buffer — the attribute does not magically track realloc.

Case 4: SARIF instead of `json`

```
# Before GCC 16 (still works on 14.2):
gcc -fdiagnostics-format=json -c pin_std.c -o /tmp/x.o 2>&1 | head -c 120; echo

# GCC 16 migration (docs; flag names from the changes page):
# gcc-16 -fdiagnostics-add-output=sarif -c pin_std.c -o build/pin_std.o
# → writes build/pin_std.c.sarif next to the object (dump-dir aware)
```

If an internal bot still passes `-fdiagnostics-format=json`, it will fail hard on GCC 16. Switch the consumer to SARIF before the toolchain bump, not after.

Case 5: unused-but-set under `-Wall`

```c
/* unused_but_set.c */
int f(int x) {
    int y = x;
    y++;          /* value of y never read */
    return x;
}
```

```
gcc -Wall -Wextra -c unused_but_set.c -o /tmp/ubs.o 2>&1 || true
```

On **GCC 14.2** this particular pattern may still be silent depending on exact warning configuration (observed: no diagnostic on this lab). On **GCC 16**, porting notes warn that the default level attached to `-Wall`/`-Wextra` is stricter and can flag increments that never feed a read. Fix the code; only then dial the level back.

The trap

- **Reading a C++ migration thread and applying it to C.** `concept` / `requires` as identifiers, `operator!=` synthesis, `char8_t` — those are C++20 problems. Your `.c` files do not care. Your shared headers compiled as C++ might.
- **Assuming the lab GCC is the release you are writing policy for.** Document the version in the PR / runbook: "verified on 14.2; required on 16.1+".
- **Leaving diagnostic scrapers on `json`.** That format is gone in 16; the failure mode is empty annotations, not a soft warning.
- **Installing GCC 16 with `apt`/`sudo` in a scout sandbox.** Policy for these books: cite upstream and skip the install. Reproduce on a real workstation or CI image that already has the toolchain.

The boring rule

Pin `-std=` (usually `-std=gnu23` for new C), pin warning policy, migrate bots to SARIF **before** the GCC 16 bump, and treat every new `_Maxof` / `counted_by` use as an explicit dependency on GCC 16.

Try this

1. Print your CI image's `gcc -v` line and record the default `GNU Cxx` dialect.
2. Add `-std=gnu23 -Wall -Wextra` to a leaf library and fix what breaks on your current compiler first.
3. Point one diagnostic consumer at SARIF on GCC 15 (where both sinks may still exist) so the GCC 16 cut-over is a no-op.

4. When you do get `gcc-16`, compile Case 2 and Case 3 and paste the real output into this chapter's next revision.

Sources

- GCC 16 Release Series — Changes (also mirrored at `gcc.sourceware.org`)
- Porting to GCC 16 (linked from the changes page; same 403 caveats on some fetchers)
- Community migration notes summarising the porting guide (e.g. ELCSoftware, 3 May 2026) for warning-level details
- Inspiration: linuxiac on X — GCC 16.1 / C++20 default (C++ headline; this chapter covers the C side)

The XOR-Swap Trap and a Type-Safe C23 SWAP Macro

The XOR-Swap Trap and a Type-Safe C23 `SWAP` Macro

Thesis

`a ^= b; b ^= a; a ^= b;` swaps two integers without a temporary. It turns up in interview answers, old embedded code and "clever C" threads. On a modern compiler and CPU it has three problems and no benefit:

1. **Aliasing.** If `a` and `b` name the same object, the first XOR sets it to zero, and from then on it stays zero.
2. **Undefined behaviour in the one-liner.** `a ^= b ^= a ^= b` modifies `a` twice without a sequence point between the two modifications.
3. **It isn't faster.** Through pointers it compiles to *more* instructions in a serial dependency chain. With values in registers, the compiler emits the same code for both versions.

A temporary is correct, and C23's standard `typeof` lets you wrap one in a macro that evaluates each operand once and refuses mismatched types. All numbers and diagnostics below come from gcc 14.2.0 (Debian) on x86-64.

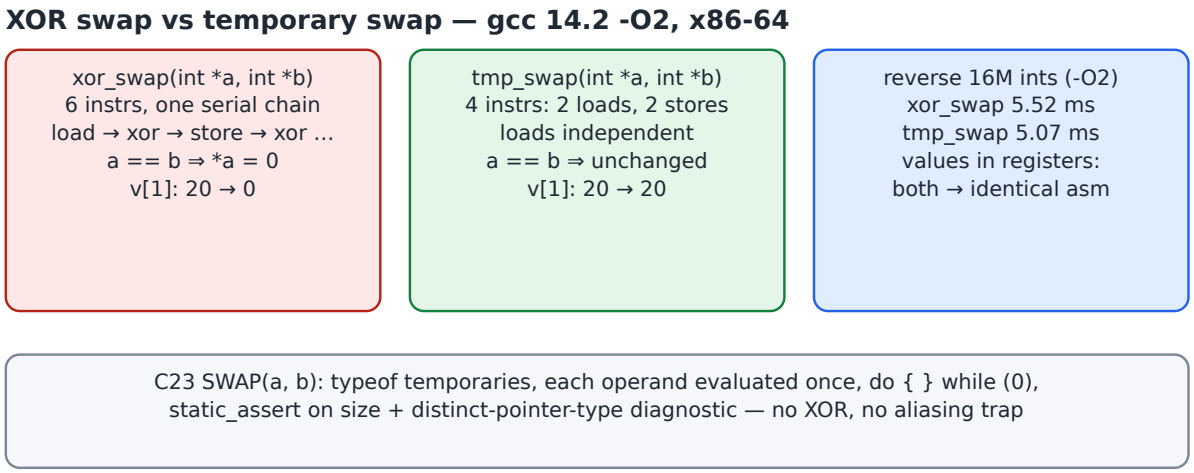


Figure 1: XOR swap vs temporary swap: instruction count, aliasing result, benchmark, and the C23 SWAP macro

Trap 1: swapping an element with itself

A self-swap happens more often than it sounds. A Fisher–Yates shuffle swaps `v[i]` with `v[j]` where `j` may equal `i`, and a partition step may swap an element with itself.

```
#include <stdio.h>
static void xor_swap(int *a, int *b) { *a ^= *b; *b ^= *a; *a ^= *b; }
static void tmp_swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
int main(void) {
    int v[4] = {10, 20, 30, 40};
    int i = 1, j = 1;                       /* e.g. a shuffle that picked the same index */
    tmp_swap(&v[i], &v[j]);  printf("tmp_swap(&v[1], &v[1]) -> v[1] = %d\n", v[1]);
    xor_swap(&v[i], &v[j]);  printf("xor_swap(&v[1], &v[1]) -> v[1] = %d\n", v[1]);
    int x = 3, y = 5;
    xor_swap(&x, &y);        printf("xor_swap(3, 5)         -> %d %d\n", x, y);
    return 0;
}
```

`gcc -std=c23 -O2 -Wall -Wextra alias.c && ./a.out`:

```
tmp_swap(&v[1], &v[1]) -> v[1] = 20
xor_swap(&v[1], &v[1]) -> v[1] = 0
xor_swap(3, 5)         -> 5 3
```

The compiler gives no warning, and the bug only shows up when two indices happen to collide. With `*a` and `*b` the same object, `*a ^= *b` computes `x ^ x`, which is `0`, and the next two XORs keep it at zero. The usual "fix" is `if (a != b)`, which adds a branch to an idiom whose only selling point was saving a register.

Trap 2: the one-liner is undefined

```
#include <stdio.h>
int main(void) { int a = 3, b = 5; a ^= b ^= a ^= b; printf("a=%d b=%d\n", a, b); }
```

```
oneliner.c: In function ‘main’:
oneliner.c:2:38: warning: operation on ‘a’ may be undefined [-Wsequence-point]
    2 | int main(void) { int a = 3, b = 5; a ^= b ^= a ^= b; printf("a=%d b=%d\n", a, b); }
      |                                      ^~
a=5 b=3
```

It printed the "right" answer on this build, which is what makes it dangerous. The C standard makes it undefined to modify the same scalar twice without the modifications being sequenced (evaluation order; C23 N3220 §6.5). GCC's `-Wsequence-point`, which `-Wall` enables, flags it. Treat that warning as an error.

Trap 3: it is not faster

```
void xor_swap(int *a, int *b) { *a ^= *b; *b ^= *a; *a ^= *b; }
void tmp_swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
int  xor_regs(int a, int b) { a ^= b; b ^= a; a ^= b; return a - b; }
int  tmp_regs(int a, int b) { int t = a; a = b; b = t; return a - b; }
```

`gcc -std=c23 -O2 -S -masm=intel` (directives trimmed):

```
xor_swap:
    mov eax, DWORD PTR [rdi]
    xor eax, DWORD PTR [rsi]
    mov DWORD PTR [rdi], eax
    xor eax, DWORD PTR [rsi]
    mov DWORD PTR [rsi], eax
    xor DWORD PTR [rdi], eax
    ret
tmp_swap:
    mov eax, DWORD PTR [rdi]
    mov edx, DWORD PTR [rsi]
    mov DWORD PTR [rdi], edx
    mov DWORD PTR [rsi], eax
    ret
xor_regs:
    mov eax, esi
    sub eax, edi
    ret
tmp_regs:
    mov eax, esi
    sub eax, edi
    ret
```

Three observations:

- **Through pointers**, `xor_swap` is 6 instructions. Each one depends on the previous result, and the final `xor DWORD PTR [rdi], eax` is a read-modify-write to memory. The compiler *must* keep that shape, because it can't rule out `a == b`, and the XOR version gives a different result (zero) in that case. `tmp_swap` is 4 instructions, with two loads that can issue in parallel followed by two stores.
- **In registers**, both versions compile to the same two instructions. GCC sees through the XOR sequence and just renames registers. A temporary in source code doesn't cost a register in the output.
- The "saves a register" argument comes from machines with very few registers and no register renaming. x86-64 has 16 general-purpose registers plus renaming in hardware.

A micro-benchmark that reverses a 16M-element `int` array in place with each swap (`bench.c`, best of 20 passes, `clock_gettime(CLOCK_MONOTONIC)`, 8-vCPU Intel Xeon VM):

```
-O2: reverse 16M ints, best of 20: xor_swap 5.52 ms, tmp_swap 5.07 ms (v[0]=0)
-O3: reverse 16M ints, best of 20: xor_swap 5.93 ms, tmp_swap 5.26 ms (v[0]=0)
```

Three more runs of the `-O2` binary gave 8.43/7.58, 7.93/7.27 and 7.42/6.81 ms. Absolute times move around on a shared VM, but `tmp_swap` won every run by about 8–12%. This loop is mostly memory-bound, so the gap is modest. The point is that the XOR version never wins.

Doing it right: a C23 `SWAP` macro

C23 standardises `typeof`, which GCC and Clang had long supported as an extension (cppreference). That makes a type-generic swap possible without passing the type as an argument:

```c
#include <stdio.h>
#include <assert.h>
/* C23: typeof is standard. Same-type check via a pointer assignment, evaluated once. */
#define SWAP(a, b) do {                                          \
        typeof(a) *swap_pa_ = &(a);                               \
        typeof(b) *swap_pb_ = &(b);                               \
        static_assert(sizeof *swap_pa_ == sizeof *swap_pb_,       \
                      "SWAP: operands differ in size");           \
        (void)(swap_pa_ == swap_pb_); /* distinct types -> diagnostic */ \
        typeof(a) swap_t_ = *swap_pa_;                            \
        *swap_pa_ = *swap_pb_;                                    \
        *swap_pb_ = swap_t_;                                      \
    } while (0)
```

Design choices:

- **Each operand is evaluated once.** The macro takes `&(a)` and `&(b)` a single time and works through the pointers, so `SWAP(v[i++], v[2])` increments `i` exactly once. A naive `t = a; a = b; b = t;` macro would evaluate `v[i++]` twice.
- **Self-swap is harmless** because it uses a temporary, not XOR.
- **Type checks.** The `static_assert` catches operands of different sizes at compile time. Comparing the two pointers catches different types of the *same* size (`int` vs `float`), using GCC's `-Wcompare-distinct-pointer-types` (warning options). Build with `-Werror` to make that a hard error.
- `do { … } while (0)` makes the macro a single statement, so `if (x) SWAP(p, q); else …` parses as intended. A GCC statement expression isn't needed, because a swap produces no value.
- The operands must be lvalues. Bit-fields and `register` variables can't have their address taken, so swap those with a plain temporary.

The test driver:

```c
int main(void) {
    int v[3] = {1, 2, 3}; int i = 0;
```

```
    SWAP(v[i++], v[2]);           /* i++ evaluated exactly once */
    printf("v = {%d, %d, %d}, i = %d\n", v[0], v[1], v[2], i);
    SWAP(v[1], v[1]);             /* self-swap is harmless */
    printf("self-swap v[1] = %d\n", v[1]);
    double d = 1.5, e = 2.5; SWAP(d, e); printf("d=%.1f e=%.1f\n", d, e);
#ifdef BAD
    int n = 1; long long w = 2; SWAP(n, w);
    float f = 1; SWAP(n, f);
#endif
    return 0;
}
```

```
v = {3, 2, 1}, i = 1
self-swap v[1] = 2
d=2.5 e=1.5
$ gcc -std=c23 -DBAD -Wall macro.c
macro.c:7:9: error: static assertion failed: "SWAP: operands differ in size"
macro.c:9:25: warning: comparison of distinct pointer types lacks a cast [-Wcompare-distinct
macro.c:9:25: warning: comparison of distinct pointer types lacks a cast [-Wcompare-distinct
```

The size assertion fires for `int` vs `long long`, and the distinct-pointer warning fires for `int` vs `float`. GCC also reports that warning on the `long long` line, which is why it appears twice.

Takeaways

- Never use XOR swap through pointers: it zeroes the value when the pointers alias, and it compiles to a longer dependency chain.
- `a ^= b ^= a ^= b` is undefined behaviour, so keep `-Wall` (which includes `-Wsequence-point`) on and treat it as fatal.
- Write the temporary. The optimiser removes it whenever it can.
- For a reusable swap, use C23 `typeof` with take-the-address-once semantics, plus a size `static_assert` and a pointer-type check.

C23 typeof and typeof_unqual: type of an expression

C23 typeof and typeof_unqual: type of an expression

After reading this chapter, you will be able to inspect and derive types from expressions at compile time using C23 `typeof` and `typeof_unqual`, build type-safe macros without repeating type names, strip qualifiers like `const` and `volatile` when needed, and migrate existing GCC `__typeof__` constructs to standardized C23 syntax.

Overview

Until C23, C developers writing generic macros or type-dependent abstractions relied either on explicit type parameters, compiler extensions like GCC's `__typeof__`, or macro hacks that often broke under qualification or array decay. C23 (ISO/IEC 9899:2024) standardizes expression-based type deduction with two key operators: `typeof` and `typeof_unqual`.

Both operators evaluate their operand solely as a static type expression at compile time. Like `sizeof`, neither operator evaluates runtime side effects inside its operand (unless the operand is a variable-length array type).

To compile the examples in this chapter, use GCC 14+ or Clang 19+ with the C23 standard flag:

```
gcc -std=c23 -Wall -Wextra -O2 typeof-demo.c -o typeof-demo
```

Basic declaration from expressions

The `typeof` operator takes either an expression or a parenthesized type name and yields its type. This allows you to declare variables, pointers, and array indices matched directly to existing objects without hardcoding type names.

```
#include <stdio.h>

int main(void) {
  int xs[] = {1, 2, 3};
  typeof(xs[0]) sum = 0;
  for (typeof(sizeof xs) i = 0; i < sizeof xs / sizeof xs[0]; i++)
    sum += xs[i];
  typeof(sum) *p = &sum;
  printf("sum=%d sizeof=%zu\n", *p, sizeof(typeof(*p)));
  return 0;
```

```
}
```

The command used to build and execute this demonstration is:

```
gcc -std=c23 -Wall -Wextra -O2 typeof-demo.c -o typeof-demo && ./typeof-demo
```

```
<<OUTPUT:run1>>
```

In `typeof-demo.c`: - `typeof(xs[0])` resolves to `int`, matching the element type of `xs`. - `typeof(sizeof xs)` resolves to `size_t`, ensuring the loop index counter matches the integer type returned by `sizeof`. - `typeof(sum) *p` declares a pointer to the type of `sum` (`int *`). - `sizeof(typeof(*p))` evaluates the byte size of the type pointed to by `p`.

`typeof` vs `typeof_unqual`

The difference between `typeof` and `typeof_unqual` lies in how type qualifiers (`const`, `volatile`, `restrict`, and `_Atomic`) are handled.

- `typeof(expr)` preserves all type qualifiers of the expression's type.
- `typeof_unqual(expr)` strips top-level type qualifiers, yielding an unqualified version of the type.

```
const volatile int status_flag = 42;

/* target1 has type: const volatile int */
typeof(status_flag) target1 = status_flag;

/* target2 has type: int */
typeof_unqual(status_flag) target2 = status_flag;
```

Stripping qualifiers is essential when declaring writable temporary storage inside macros or local working buffers derived from `const`-qualified input parameters.

Syntax	Preserves `const` / `volatile` / `_Atomic`	Typical Use Case
`typeof(expr)`	Yes	Direct variable declaration, pointer derivation matching target semantics
`typeof_unqual(expr)`	No (strips top-level qualifiers)	Temporary variables in macros, local mutable accumulators

Structuring type-safe macros with `typeof_unqual`

A classic pitfall in C macros is evaluating an argument with side effects multiple times. The standard idiom uses local temporaries, but without `typeof_unqual`, passing a `const` value produces a `const`

temporary that cannot be reassigned or modified.

```
#include <stdio.h>

#define MAX(a, b) ({                          \
  typeof_unqual(a) _a = (a);                  \
  typeof_unqual(b) _b = (b);                  \
  _a > _b ? _a : _b;                          \
})

int main(void) {
  const int x = 10;
  const int y = 20;
  int result = MAX(x, y);
  printf("max result: %d\n", result);
  return 0;
}
```

Because `_a` and `_b` are declared with `typeof_unqual`, they remain mutable inside the block expression even when initialized from `const` inputs `x` and `y`.

Difference vs GCC historical `__typeof__`

Before C23, GCC and Clang provided `__typeof__` and `typeof` as GNU language extensions. Standard C23 incorporates `typeof` and `typeof_unqual` directly into the core language syntax.

When updating existing codebases:

1. **Keyword standardization**: Standard C23 introduces `typeof` and `typeof_unqual` as keywords. Older headers using `__typeof__` remain supported for backward compatibility under `-std=gnu23` or older standard modes.
2. **Qualifier handling**: GCC's extension `__typeof__` retained qualifiers, but had no standard companion for stripping them. `typeof_unqual` replaces custom macro workarounds designed to strip `const`.
3. **Strict ISO conformance**: When compiling with strict `-std=c23` (rather than `-std=gnu23`), use `typeof` and `typeof_unqual` rather than GNU extension keywords.

```
/* Legacy GNU C extension */
__typeof__(x) temp1 = x;

/* Standard C23 */
typeof(x) temp2 = x;
typeof_unqual(x) temp3 = x;
```

Common mistakes

- **Modifying variables inside `typeof` operands**: `typeof` does not execute the operand expression at runtime. Writing `typeof(x++) y = 0;` does not increment `x`.
- **Confusing pointer target qualification with top-level qualification**: `typeof_unqual(const int *)` strips qualifiers from the pointer itself, yielding `const int *` (a mutable pointer to a constant integer). To remove the `const` from the pointed-to type, unqualify the dereferenced object: `typeof_unqual(*p) *`.
- **Assuming `typeof` decays array types**: Unlike function argument passing, `typeof(array)` retains the full array type (e.g., `int[3]`), not a pointer type (`int *`).

Summary

- C23 `typeof` inspects an expression or type name at compile time to produce its exact type.
- C23 `typeof_unqual` derives the underlying type while discarding top-level `const`, `volatile`, `restrict`, and `_Atomic` qualifiers.
- Expressions inside `typeof` and `typeof_unqual` are unevaluated constant type queries at translation time.
- Standard C23 `typeof` supersedes vendor extensions like GCC `__typeof__`, providing a portable syntax across modern C toolchains.

Verified output

```
sum=6 sizeof=4
```

Sources

- cppreference — typeof and typeof_unqual
- ISO/IEC JTC1/SC22/WG14 N3096 (C23 Working Draft)

Module 9: Advanced C Features and Modern Standards

Overview

This module explores advanced features introduced in modern C standards (C99, C11, C17, and C23) as well as contemporary development practices. You'll learn about variable-length arrays, designated initializers, static assertions, thread-local storage, and many other features that make modern C more powerful and expressive. The module also covers modern development practices including DevOps integration, containerized development, and cloud-native C programming.

Learning Objectives

By the end of this module, you will be able to: - Understand and apply C99 features including variable-length arrays and designated initializers - Utilize C11 features such as static assertions and thread-local storage - Implement C17/C23 features including typeof operators and binary literals - Apply modern C development practices and tools - Understand contemporary C ecosystem including WebAssembly and cloud-native development - Work with advanced preprocessor features - Implement generic programming techniques using modern C features - Apply best practices for modern C development

Chapters

1. **C99 Features** - Variable-length arrays, designated initializers, compound literals, restrict keyword, inline functions
2. **C11 Features** - Static assertions, anonymous structures/unions, thread-local storage, bounds-checking functions
3. **C17 and C23 Features** - typeof operators, binary literals, enhanced preprocessor directives, attributes
4. **Modern C Development Practices** - DevOps integration, containerized development, WebAssembly, cloud-native C
5. **Advanced Preprocessor** - Macro definitions, conditional compilation, pragma directives, token manipulation
6. **C23 `#embed`, `nullptr`, and `constexpr`** - Binary assets, null pointer constant, typed translation-time values
7. **Checked Integer Arithmetic** - C23 `<stdckdint.h>` (`ckd_add`/`ckd_sub`/`ckd_mul`), overflow-safe allocation sizing, `reallocarray`, portable fallback

8. **C23 `<stdbit.h>`** - Bit and byte utilities, defined zero handling, power-of-two ring buffers, endianness
9. **`defer` (TS 25755)** - Scope-based cleanup, Clang `-fdefer-ts`, the return-value trap, `cleanup`-attribute fallback

Key Concepts Covered

- Variable-length arrays (VLAs)
- Designated initializers
- Compound literals
- restrict keyword
- inline functions
- Static assertions (_Static_assert)
- Anonymous structures and unions
- Thread-local storage (_Thread_local)
- Bounds-checking functions (Annex K)
- typeof and typeof_unqual operators
- Binary integer constants (0b1010)
- Enhanced preprocessor directives (#elifdef, #elifndef, #warning, #embed)
- Attribute syntax ([[]])
- Generic programming techniques
- DevOps and CI/CD for C projects
- Containerized C development with Docker
- WebAssembly compilation with Emscripten
- Cloud-native C applications
- Modern testing practices (fuzzing, property-based testing)
- Supply chain security and dependency management

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Completion of Module 4: Functions and Modular Programming
- Completion of Module 5: Arrays and Strings
- Completion of Module 6: Pointers and Memory Management
- Completion of Module 7: Structures and User-Defined Types
- Completion of Module 8: File I/O and System Programming
- Understanding of basic programming concepts
- Familiarity with C language fundamentals

Tools and Technologies

- Modern C compiler (GCC 14+, Clang 18+, MSVC 2022)

- Build systems (CMake, Meson)
- Package managers (Conan, vcpkg)
- Containerization tools (Docker)
- WebAssembly toolchains (Emscripten)
- CI/CD platforms (GitHub Actions, GitLab CI)
- Testing frameworks (Unity, CMocka)
- Static analysis tools (clang-analyzer, cppcheck)
- Fuzzing tools (AFL, libFuzzer)

Estimated Time to Complete

- Reading: 12-15 hours
- Exercises: 15-20 hours
- Projects: 10-12 hours

Assessment

- Complete all chapter exercises
- Successfully implement programs using modern C features
- Demonstrate understanding of contemporary C development practices
- Pass modern C features and practices quiz
- Submit a comprehensive project using modern C standards

Next Module

After completing this module, proceed to **Module 10: Data Structures and Algorithms** to learn about implementing classic data structures and algorithms in C.

Additional Resources

- "Modern C" by Jens Gustedt
- C standard documentation (ISO/IEC 9899:2024 for C23)
- Compiler documentation (GCC, Clang, MSVC)
- WebAssembly documentation
- Docker documentation
- CI/CD platform documentation
- Modern C development blogs and resources

Data Structures

Linear Data Structures

Introduction

Linear data structures are fundamental building blocks in computer science that organize data elements in a sequential manner. These structures include dynamic arrays, linked lists, stacks, queues, and their variations. Understanding linear data structures is crucial for efficient programming, as they form the foundation for more complex data structures and algorithms.

In this chapter, we'll explore the implementation and usage of various linear data structures in C, focusing on dynamic memory management, performance characteristics, and practical applications.

Dynamic Arrays (Vectors)

Dynamic arrays, also known as vectors, are resizable arrays that can grow or shrink during program execution. They provide the benefits of arrays (random access, cache locality) with the flexibility of dynamic sizing.

Basic Dynamic Array Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int *data;
    size_t size;
    size_t capacity;
} DynamicArray;

// Initialize dynamic array
DynamicArray* create_array(size_t initial_capacity) {
    DynamicArray *arr = malloc(sizeof(DynamicArray));
    if (arr == NULL) return NULL;

    arr->data = malloc(initial_capacity * sizeof(int));
    if (arr->data == NULL) {
        free(arr);
        return NULL;
```

```
    }

    arr->size = 0;
    arr->capacity = initial_capacity;
    return arr;
}

// Free dynamic array
void free_array(DynamicArray *arr) {
    if (arr != NULL) {
        free(arr->data);
        free(arr);
    }
}

// Add element to end of array
void push_back(DynamicArray *arr, int value) {
    if (arr->size >= arr->capacity) {
        // Double the capacity
        size_t new_capacity = arr->capacity * 2;
        int *new_data = realloc(arr->data, new_capacity * sizeof(int));
        if (new_data == NULL) return;  // Handle allocation failure

        arr->data = new_data;
        arr->capacity = new_capacity;
    }

    arr->data[arr->size] = value;
    arr->size++;
}

// Remove element from end of array
int pop_back(DynamicArray *arr) {
    if (arr->size == 0) return 0;  // Handle empty array

    arr->size--;
    return arr->data[arr->size];
}

// Access element at index
int at(DynamicArray *arr, size_t index) {
    if (index >= arr->size) return 0;  // Handle out of bounds
    return arr->data[index];
}

// Set element at index
```

```c
void set(DynamicArray *arr, size_t index, int value) {
    if (index >= arr->size) return;  // Handle out of bounds
    arr->data[index] = value;
}

// Insert element at index
void insert(DynamicArray *arr, size_t index, int value) {
    if (index > arr->size) return;  // Handle invalid index

    if (arr->size >= arr->capacity) {
        size_t new_capacity = arr->capacity * 2;
        int *new_data = realloc(arr->data, new_capacity * sizeof(int));
        if (new_data == NULL) return;

        arr->data = new_data;
        arr->capacity = new_capacity;
    }

    // Shift elements to the right
    for (size_t i = arr->size; i > index; i--) {
        arr->data[i] = arr->data[i - 1];
    }

    arr->data[index] = value;
    arr->size++;
}

// Remove element at index
void erase(DynamicArray *arr, size_t index) {
    if (index >= arr->size) return;  // Handle invalid index

    // Shift elements to the left
    for (size_t i = index; i < arr->size - 1; i++) {
        arr->data[i] = arr->data[i + 1];
    }

    arr->size--;
}

// Print array contents
void print_array(DynamicArray *arr) {
    printf("Array (size=%zu, capacity=%zu): ", arr->size, arr->capacity);
    for (size_t i = 0; i < arr->size; i++) {
        printf("%d ", arr->data[i]);
    }
    printf("\n");
}
```

```c
int main() {
    DynamicArray *arr = create_array(2);
    if (arr == NULL) {
        printf("Failed to create array\n");
        return 1;
    }

    // Test basic operations
    push_back(arr, 10);
    push_back(arr, 20);
    print_array(arr);

    push_back(arr, 30);  // This should trigger a resize
    print_array(arr);

    insert(arr, 1, 15);
    print_array(arr);

    set(arr, 0, 5);
    print_array(arr);

    printf("Element at index 2: %d\n", at(arr, 2));

    erase(arr, 1);
    print_array(arr);

    int popped = pop_back(arr);
    printf("Popped element: %d\n", popped);
    print_array(arr);

    free_array(arr);
    return 0;
}
```

Generic Dynamic Array

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    void **data;
    size_t size;
    size_t capacity;
    size_t element_size;
```

```c
} GenericArray;

// Create generic array
GenericArray* create_generic_array(size_t initial_capacity, size_t element_size) {
    GenericArray *arr = malloc(sizeof(GenericArray));
    if (arr == NULL) return NULL;

    arr->data = malloc(initial_capacity * sizeof(void*));
    if (arr->data == NULL) {
        free(arr);
        return NULL;
    }

    arr->size = 0;
    arr->capacity = initial_capacity;
    arr->element_size = element_size;
    return arr;
}

// Free generic array
void free_generic_array(GenericArray *arr) {
    if (arr != NULL) {
        for (size_t i = 0; i < arr->size; i++) {
            free(arr->data[i]);
        }
        free(arr->data);
        free(arr);
    }
}

// Add element to generic array
int push_generic(GenericArray *arr, const void *element) {
    if (arr->size >= arr->capacity) {
        size_t new_capacity = arr->capacity * 2;
        void **new_data = realloc(arr->data, new_capacity * sizeof(void*));
        if (new_data == NULL) return -1;

        arr->data = new_data;
        arr->capacity = new_capacity;
    }

    arr->data[arr->size] = malloc(arr->element_size);
    if (arr->data[arr->size] == NULL) return -1;

    memcpy(arr->data[arr->size], element, arr->element_size);
    arr->size++;
    return 0;
```

```
}

// Print generic array with custom print function
void print_generic_array(GenericArray *arr, void (*print_func)(const void*)) {
    printf("Generic Array (size=%zu, capacity=%zu): ", arr->size, arr->capacity);
    for (size_t i = 0; i < arr->size; i++) {
        print_func(arr->data[i]);
        printf(" ");
    }
    printf("\n");
}

// Example with strings
void print_string(const void *str) {
    printf("%s", (const char*)str);
}

int main() {
    // Create array of strings
    GenericArray *str_array = create_generic_array(2, sizeof(char*));
    if (str_array == NULL) {
        printf("Failed to create array\n");
        return 1;
    }

    const char *strings[] = {"Hello", "World", "Generic", "Array"};

    for (int i = 0; i < 4; i++) {
        push_generic(str_array, &strings[i]);
    }

    print_generic_array(str_array, print_string);

    free_generic_array(str_array);
    return 0;
}
```

Linked Lists

Linked lists are linear data structures where elements are stored in nodes, and each node contains a reference to the next node in the sequence. They provide efficient insertion and deletion operations but have slower access times compared to arrays.

Singly Linked List

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

typedef struct {
    Node *head;
    size_t size;
} LinkedList;

// Create new node
Node* create_node(int data) {
    Node *node = malloc(sizeof(Node));
    if (node == NULL) return NULL;

    node->data = data;
    node->next = NULL;
    return node;
}

// Initialize linked list
LinkedList* create_list(void) {
    LinkedList *list = malloc(sizeof(LinkedList));
    if (list == NULL) return NULL;

    list->head = NULL;
    list->size = 0;
    return list;
}

// Free linked list
void free_list(LinkedList *list) {
    if (list == NULL) return;

    Node *current = list->head;
    while (current != NULL) {
        Node *next = current->next;
        free(current);
        current = next;
    }

    free(list);
```

```
}

// Insert at beginning
void push_front(LinkedList *list, int data) {
    Node *new_node = create_node(data);
    if (new_node == NULL) return;

    new_node->next = list->head;
    list->head = new_node;
    list->size++;
}

// Insert at end
void push_back(LinkedList *list, int data) {
    Node *new_node = create_node(data);
    if (new_node == NULL) return;

    if (list->head == NULL) {
        list->head = new_node;
    } else {
        Node *current = list->head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = new_node;
    }
    list->size++;
}

// Remove from beginning
int pop_front(LinkedList *list) {
    if (list->head == NULL) return 0;

    Node *temp = list->head;
    int data = temp->data;
    list->head = list->head->next;
    free(temp);
    list->size--;

    return data;
}

// Find element
Node* find(LinkedList *list, int data) {
    Node *current = list->head;
    while (current != NULL) {
        if (current->data == data) {
```

```
            return current;
        }
        current = current->next;
    }
    return NULL;
}

// Insert after node
void insert_after(Node *node, int data) {
    if (node == NULL) return;

    Node *new_node = create_node(data);
    if (new_node == NULL) return;

    new_node->next = node->next;
    node->next = new_node;
}

// Delete node with specific data
void delete_node(LinkedList *list, int data) {
    if (list->head == NULL) return;

    // If head node holds the data
    if (list->head->data == data) {
        Node *temp = list->head;
        list->head = list->head->next;
        free(temp);
        list->size--;
        return;
    }

    // Search for the node to delete
    Node *current = list->head;
    while (current->next != NULL && current->next->data != data) {
        current = current->next;
    }

    // If node was found
    if (current->next != NULL) {
        Node *temp = current->next;
        current->next = current->next->next;
        free(temp);
        list->size--;
    }
}

// Print list
```

```
void print_list(LinkedList *list) {
    printf("List (size=%zu): ", list->size);
    Node *current = list->head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

int main() {
    LinkedList *list = create_list();
    if (list == NULL) {
        printf("Failed to create list\n");
        return 1;
    }

    // Test operations
    push_back(list, 10);
    push_back(list, 20);
    push_front(list, 5);
    print_list(list);

    insert_after(list->head, 7);
    print_list(list);

    Node *found = find(list, 20);
    if (found != NULL) {
        printf("Found node with data: %d\n", found->data);
    }

    delete_node(list, 7);
    print_list(list);

    int popped = pop_front(list);
    printf("Popped element: %d\n", popped);
    print_list(list);

    free_list(list);
    return 0;
}
```

Doubly Linked List

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct DNode {
    int data;
    struct DNode *next;
    struct DNode *prev;
} DNode;

typedef struct {
    DNode *head;
    DNode *tail;
    size_t size;
} DoublyLinkedList;

// Create new node
DNode* create_dnode(int data) {
    DNode *node = malloc(sizeof(DNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->next = NULL;
    node->prev = NULL;
    return node;
}

// Initialize doubly linked list
DoublyLinkedList* create_dlist(void) {
    DoublyLinkedList *list = malloc(sizeof(DoublyLinkedList));
    if (list == NULL) return NULL;

    list->head = NULL;
    list->tail = NULL;
    list->size = 0;
    return list;
}

// Free doubly linked list
void free_dlist(DoublyLinkedList *list) {
    if (list == NULL) return;

    DNode *current = list->head;
    while (current != NULL) {
        DNode *next = current->next;
        free(current);
```

```
        current = next;
    }

    free(list);
}

// Insert at beginning
void push_front_dlist(DoublyLinkedList *list, int data) {
    DNode *new_node = create_dnode(data);
    if (new_node == NULL) return;

    if (list->head == NULL) {
        list->head = list->tail = new_node;
    } else {
        new_node->next = list->head;
        list->head->prev = new_node;
        list->head = new_node;
    }
    list->size++;
}

// Insert at end
void push_back_dlist(DoublyLinkedList *list, int data) {
    DNode *new_node = create_dnode(data);
    if (new_node == NULL) return;

    if (list->tail == NULL) {
        list->head = list->tail = new_node;
    } else {
        new_node->prev = list->tail;
        list->tail->next = new_node;
        list->tail = new_node;
    }
    list->size++;
}

// Remove from beginning
int pop_front_dlist(DoublyLinkedList *list) {
    if (list->head == NULL) return 0;

    DNode *temp = list->head;
    int data = temp->data;

    if (list->head == list->tail) {
        list->head = list->tail = NULL;
    } else {
        list->head = list->head->next;
```

```
        list->head->prev = NULL;
    }

    free(temp);
    list->size--;
    return data;
}

// Remove from end
int pop_back_dlist(DoublyLinkedList *list) {
    if (list->tail == NULL) return 0;

    DNode *temp = list->tail;
    int data = temp->data;

    if (list->head == list->tail) {
        list->head = list->tail = NULL;
    } else {
        list->tail = list->tail->prev;
        list->tail->next = NULL;
    }

    free(temp);
    list->size--;
    return data;
}

// Print list forward
void print_dlist_forward(DoublyLinkedList *list) {
    printf("Forward: ");
    DNode *current = list->head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

// Print list backward
void print_dlist_backward(DoublyLinkedList *list) {
    printf("Backward: ");
    DNode *current = list->tail;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->prev;
    }
    printf("\n");
```

```
}

int main() {
    DoublyLinkedList *list = create_dlist();
    if (list == NULL) {
        printf("Failed to create list\n");
        return 1;
    }

    // Test operations
    push_back_dlist(list, 10);
    push_back_dlist(list, 20);
    push_front_dlist(list, 5);
    push_back_dlist(list, 30);

    print_dlist_forward(list);
    print_dlist_backward(list);

    int front = pop_front_dlist(list);
    printf("Popped from front: %d\n", front);

    int back = pop_back_dlist(list);
    printf("Popped from back: %d\n", back);

    print_dlist_forward(list);

    free_dlist(list);
    return 0;
}
```

Stacks

A stack is a Last-In-First-Out (LIFO) data structure where elements are added and removed from the same end, called the top.

Array-based Stack

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define STACK_SIZE 100

typedef struct {
```

```
    int data[STACK_SIZE];
    int top;
} ArrayStack;

// Initialize stack
void init_array_stack(ArrayStack *stack) {
    stack->top = -1;
}

// Check if stack is empty
bool is_empty_array(ArrayStack *stack) {
    return stack->top == -1;
}

// Check if stack is full
bool is_full_array(ArrayStack *stack) {
    return stack->top == STACK_SIZE - 1;
}

// Push element onto stack
bool push_array(ArrayStack *stack, int value) {
    if (is_full_array(stack)) {
        return false;  // Stack overflow
    }

    stack->data[++stack->top] = value;
    return true;
}

// Pop element from stack
int pop_array(ArrayStack *stack) {
    if (is_empty_array(stack)) {
        return 0;  // Stack underflow
    }

    return stack->data[stack->top--];
}

// Peek at top element
int peek_array(ArrayStack *stack) {
    if (is_empty_array(stack)) {
        return 0;  // Stack is empty
    }

    return stack->data[stack->top];
}
```

```c
// Print stack contents
void print_array_stack(ArrayStack *stack) {
    if (is_empty_array(stack)) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack (top to bottom): ");
    for (int i = stack->top; i >= 0; i--) {
        printf("%d ", stack->data[i]);
    }
    printf("\n");
}

int main() {
    ArrayStack stack;
    init_array_stack(&stack);

    // Test stack operations
    push_array(&stack, 10);
    push_array(&stack, 20);
    push_array(&stack, 30);

    print_array_stack(&stack);

    printf("Top element: %d\n", peek_array(&stack));

    int popped = pop_array(&stack);
    printf("Popped element: %d\n", popped);

    print_array_stack(&stack);

    return 0;
}
```

Linked List-based Stack

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct StackNode {
    int data;
    struct StackNode *next;
} StackNode;
```

```c
typedef struct {
    StackNode *top;
    size_t size;
} LinkedStack;

// Create new stack node
StackNode* create_stack_node(int data) {
    StackNode *node = malloc(sizeof(StackNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->next = NULL;
    return node;
}

// Initialize stack
LinkedStack* create_linked_stack(void) {
    LinkedStack *stack = malloc(sizeof(LinkedStack));
    if (stack == NULL) return NULL;

    stack->top = NULL;
    stack->size = 0;
    return stack;
}

// Free stack
void free_linked_stack(LinkedStack *stack) {
    if (stack == NULL) return;

    StackNode *current = stack->top;
    while (current != NULL) {
        StackNode *next = current->next;
        free(current);
        current = next;
    }

    free(stack);
}

// Check if stack is empty
bool is_empty_linked(LinkedStack *stack) {
    return stack->top == NULL;
}

// Push element onto stack
bool push_linked(LinkedStack *stack, int value) {
```

```
    StackNode *new_node = create_stack_node(value);
    if (new_node == NULL) return false;

    new_node->next = stack->top;
    stack->top = new_node;
    stack->size++;

    return true;
}

// Pop element from stack
int pop_linked(LinkedStack *stack) {
    if (is_empty_linked(stack)) {
        return 0;  // Stack underflow
    }

    StackNode *temp = stack->top;
    int data = temp->data;
    stack->top = stack->top->next;
    free(temp);
    stack->size--;

    return data;
}

// Peek at top element
int peek_linked(LinkedStack *stack) {
    if (is_empty_linked(stack)) {
        return 0;  // Stack is empty
    }

    return stack->top->data;
}

// Print stack contents
void print_linked_stack(LinkedStack *stack) {
    if (is_empty_linked(stack)) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack (top to bottom): ");
    StackNode *current = stack->top;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
```

```
    printf("\n");
}

int main() {
    LinkedStack *stack = create_linked_stack();
    if (stack == NULL) {
        printf("Failed to create stack\n");
        return 1;
    }

    // Test stack operations
    push_linked(stack, 10);
    push_linked(stack, 20);
    push_linked(stack, 30);

    print_linked_stack(stack);

    printf("Top element: %d\n", peek_linked(stack));

    int popped = pop_linked(stack);
    printf("Popped element: %d\n", popped);

    print_linked_stack(stack);

    free_linked_stack(stack);
    return 0;
}
```

Queues

A queue is a First-In-First-Out (FIFO) data structure where elements are added at the rear and removed from the front.

Circular Buffer Queue

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define QUEUE_SIZE 100

typedef struct {
    int data[QUEUE_SIZE];
    int front;
```

```
    int rear;
    int size;
} CircularQueue;

// Initialize queue
void init_circular_queue(CircularQueue *queue) {
    queue->front = 0;
    queue->rear = -1;
    queue->size = 0;
}

// Check if queue is empty
bool is_empty_queue(CircularQueue *queue) {
    return queue->size == 0;
}

// Check if queue is full
bool is_full_queue(CircularQueue *queue) {
    return queue->size == QUEUE_SIZE;
}

// Enqueue (add) element to queue
bool enqueue(CircularQueue *queue, int value) {
    if (is_full_queue(queue)) {
        return false;  // Queue overflow
    }

    queue->rear = (queue->rear + 1) % QUEUE_SIZE;
    queue->data[queue->rear] = value;
    queue->size++;

    return true;
}

// Dequeue (remove) element from queue
int dequeue(CircularQueue *queue) {
    if (is_empty_queue(queue)) {
        return 0;  // Queue underflow
    }

    int data = queue->data[queue->front];
    queue->front = (queue->front + 1) % QUEUE_SIZE;
    queue->size--;

    return data;
}
```

```c
// Peek at front element
int front_queue(CircularQueue *queue) {
    if (is_empty_queue(queue)) {
        return 0;  // Queue is empty
    }

    return queue->data[queue->front];
}

// Print queue contents
void print_queue(CircularQueue *queue) {
    if (is_empty_queue(queue)) {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue (front to rear): ");
    int index = queue->front;
    for (int i = 0; i < queue->size; i++) {
        printf("%d ", queue->data[index]);
        index = (index + 1) % QUEUE_SIZE;
    }
    printf("\n");
}

int main() {
    CircularQueue queue;
    init_circular_queue(&queue);

    // Test queue operations
    enqueue(&queue, 10);
    enqueue(&queue, 20);
    enqueue(&queue, 30);

    print_queue(&queue);

    printf("Front element: %d\n", front_queue(&queue));

    int dequeued = dequeue(&queue);
    printf("Dequeued element: %d\n", dequeued);

    print_queue(&queue);

    return 0;
}
```

Linked List-based Queue

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct QueueNode {
    int data;
    struct QueueNode *next;
} QueueNode;

typedef struct {
    QueueNode *front;
    QueueNode *rear;
    size_t size;
} LinkedQueue;

// Create new queue node
QueueNode* create_queue_node(int data) {
    QueueNode *node = malloc(sizeof(QueueNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->next = NULL;
    return node;
}

// Initialize queue
LinkedQueue* create_linked_queue(void) {
    LinkedQueue *queue = malloc(sizeof(LinkedQueue));
    if (queue == NULL) return NULL;

    queue->front = NULL;
    queue->rear = NULL;
    queue->size = 0;
    return queue;
}

// Free queue
void free_linked_queue(LinkedQueue *queue) {
    if (queue == NULL) return;

    QueueNode *current = queue->front;
    while (current != NULL) {
        QueueNode *next = current->next;
        free(current);
        current = next;
```

```
    }

    free(queue);
}

// Check if queue is empty
bool is_empty_linked_queue(LinkedQueue *queue) {
    return queue->front == NULL;
}

// Enqueue (add) element to queue
bool enqueue_linked(LinkedQueue *queue, int value) {
    QueueNode *new_node = create_queue_node(value);
    if (new_node == NULL) return false;

    if (is_empty_linked_queue(queue)) {
        queue->front = queue->rear = new_node;
    } else {
        queue->rear->next = new_node;
        queue->rear = new_node;
    }

    queue->size++;
    return true;
}

// Dequeue (remove) element from queue
int dequeue_linked(LinkedQueue *queue) {
    if (is_empty_linked_queue(queue)) {
        return 0;  // Queue underflow
    }

    QueueNode *temp = queue->front;
    int data = temp->data;

    queue->front = queue->front->next;
    if (queue->front == NULL) {
        queue->rear = NULL;  // Queue is now empty
    }

    free(temp);
    queue->size--;

    return data;
}

// Peek at front element
```

```c
int front_linked_queue(LinkedQueue *queue) {
    if (is_empty_linked_queue(queue)) {
        return 0;  // Queue is empty
    }

    return queue->front->data;
}

// Print queue contents
void print_linked_queue(LinkedQueue *queue) {
    if (is_empty_linked_queue(queue)) {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue (front to rear): ");
    QueueNode *current = queue->front;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

int main() {
    LinkedQueue *queue = create_linked_queue();
    if (queue == NULL) {
        printf("Failed to create queue\n");
        return 1;
    }

    // Test queue operations
    enqueue_linked(queue, 10);
    enqueue_linked(queue, 20);
    enqueue_linked(queue, 30);

    print_linked_queue(queue);

    printf("Front element: %d\n", front_linked_queue(queue));

    int dequeued = dequeue_linked(queue);
    printf("Dequeued element: %d\n", dequeued);

    print_linked_queue(queue);

    free_linked_queue(queue);
    return 0;
```

```
}
```

Practical Examples

Expression Evaluation Using Stacks

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
#include <ctype.h>

#define STACK_SIZE 100

typedef struct {
    int data[STACK_SIZE];
    int top;
} Stack;

void init_stack(Stack *stack) {
    stack->top = -1;
}

bool is_empty(Stack *stack) {
    return stack->top == -1;
}

bool is_full(Stack *stack) {
    return stack->top == STACK_SIZE - 1;
}

bool push(Stack *stack, int value) {
    if (is_full(stack)) return false;
    stack->data[++stack->top] = value;
    return true;
}

int pop(Stack *stack) {
    if (is_empty(stack)) return 0;
    return stack->data[stack->top--];
}

int peek(Stack *stack) {
    if (is_empty(stack)) return 0;
```

```
    return stack->data[stack->top];
}

// Check if character is operator
bool is_operator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/');
}

// Get precedence of operator
int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        default:
            return 0;
    }
}

// Convert infix to postfix
void infix_to_postfix(const char *infix, char *postfix) {
    Stack stack;
    init_stack(&stack);

    int i, j = 0;
    for (i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];

        if (isdigit(c)) {
            postfix[j++] = c;
        } else if (c == '(') {
            push(&stack, c);
        } else if (c == ')') {
            while (!is_empty(&stack) && peek(&stack) != '(') {
                postfix[j++] = pop(&stack);
            }
            pop(&stack);  // Pop '('
        } else if (is_operator(c)) {
            while (!is_empty(&stack) && precedence(peek(&stack)) >= precedence(c)) {
                postfix[j++] = pop(&stack);
            }
            push(&stack, c);
        }
    }
```

```
    while (!is_empty(&stack)) {
        postfix[j++] = pop(&stack);
    }

    postfix[j] = '\0';
}

// Evaluate postfix expression
int evaluate_postfix(const char *postfix) {
    Stack stack;
    init_stack(&stack);

    for (int i = 0; postfix[i] != '\0'; i++) {
        char c = postfix[i];

        if (isdigit(c)) {
            push(&stack, c - '0');
        } else if (is_operator(c)) {
            int b = pop(&stack);
            int a = pop(&stack);

            switch (c) {
                case '+':
                    push(&stack, a + b);
                    break;
                case '-':
                    push(&stack, a - b);
                    break;
                case '*':
                    push(&stack, a * b);
                    break;
                case '/':
                    push(&stack, a / b);
                    break;
            }
        }
    }

    return pop(&stack);
}

int main() {
    char infix[] = "3+4*2/(1-5)";
    char postfix[100];

    printf("Infix expression: %s\n", infix);
```

```
    infix_to_postfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);

    int result = evaluate_postfix(postfix);
    printf("Result: %d\n", result);

    return 0;
}
```

Summary

Linear data structures are fundamental to computer science and programming:

1. **Dynamic Arrays**: Provide random access with dynamic resizing capabilities
2. **Linked Lists**: Offer efficient insertion/deletion with sequential access
3. **Stacks**: Implement LIFO behavior for expression evaluation and backtracking
4. **Queues**: Implement FIFO behavior for scheduling and breadth-first processing

Each structure has its own trade-offs in terms of memory usage, access patterns, and performance characteristics. Understanding these trade-offs is crucial for selecting the appropriate data structure for specific applications.

Key considerations when choosing linear data structures: - **Access Pattern**: Random vs. sequential access - **Insertion/Deletion Frequency**: At ends vs. middle - **Memory Constraints**: Fixed vs. dynamic allocation - **Performance Requirements**: Time complexity for operations - **Implementation Complexity**: Simple vs. complex structures

Mastering linear data structures provides a solid foundation for understanding more complex data structures and algorithms.

Tree Structures

Introduction

Tree structures are hierarchical data structures that organize data in a parent-child relationship, where each node can have zero or more child nodes. Trees are fundamental in computer science and are used in various applications such as file systems, databases, expression parsing, and decision-making algorithms.

In this chapter, we'll explore different types of tree structures, from basic binary trees to more complex balanced trees like AVL and red-black trees. We'll implement these structures in C and examine their properties, operations, and performance characteristics.

Binary Trees

A binary tree is a tree data structure where each node has at most two children, referred to as the left child and the right child.

Basic Binary Tree Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

// Create new tree node
TreeNode* create_node(int data) {
    TreeNode *node = malloc(sizeof(TreeNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
```

```
}

// Free entire tree
void free_tree(TreeNode *root) {
    if (root == NULL) return;

    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

// Insert node in binary tree (level-order insertion)
TreeNode* insert_level_order(TreeNode *root, int data) {
    TreeNode *new_node = create_node(data);
    if (new_node == NULL) return root;

    if (root == NULL) {
        return new_node;
    }

    // Use queue for level-order insertion
    TreeNode *queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front < rear) {
        TreeNode *current = queue[front++];

        if (current->left == NULL) {
            current->left = new_node;
            return root;
        } else {
            queue[rear++] = current->left;
        }

        if (current->right == NULL) {
            current->right = new_node;
            return root;
        } else {
            queue[rear++] = current->right;
        }
    }

    return root;
}

// Tree traversals
```

```c
void inorder_traversal(TreeNode *root) {
    if (root == NULL) return;

    inorder_traversal(root->left);
    printf("%d ", root->data);
    inorder_traversal(root->right);
}

void preorder_traversal(TreeNode *root) {
    if (root == NULL) return;

    printf("%d ", root->data);
    preorder_traversal(root->left);
    preorder_traversal(root->right);
}

void postorder_traversal(TreeNode *root) {
    if (root == NULL) return;

    postorder_traversal(root->left);
    postorder_traversal(root->right);
    printf("%d ", root->data);
}

// Level-order traversal (breadth-first)
void level_order_traversal(TreeNode *root) {
    if (root == NULL) return;

    TreeNode *queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front < rear) {
        TreeNode *current = queue[front++];
        printf("%d ", current->data);

        if (current->left != NULL) {
            queue[rear++] = current->left;
        }

        if (current->right != NULL) {
            queue[rear++] = current->right;
        }
    }
}
```

```c
// Tree height
int tree_height(TreeNode *root) {
    if (root == NULL) return 0;

    int left_height = tree_height(root->left);
    int right_height = tree_height(root->right);

    return 1 + (left_height > right_height ? left_height : right_height);
}

// Node count
int node_count(TreeNode *root) {
    if (root == NULL) return 0;

    return 1 + node_count(root->left) + node_count(root->right);
}

// Leaf count
int leaf_count(TreeNode *root) {
    if (root == NULL) return 0;

    if (root->left == NULL && root->right == NULL) {
        return 1;
    }

    return leaf_count(root->left) + leaf_count(root->right);
}

int main() {
    TreeNode *root = NULL;

    // Build tree: 1, 2, 3, 4, 5, 6, 7
    for (int i = 1; i <= 7; i++) {
        root = insert_level_order(root, i);
    }

    printf("Inorder traversal: ");
    inorder_traversal(root);
    printf("\n");

    printf("Preorder traversal: ");
    preorder_traversal(root);
    printf("\n");

    printf("Postorder traversal: ");
    postorder_traversal(root);
    printf("\n");
```

```
    printf("Level-order traversal: ");
    level_order_traversal(root);
    printf("\n");

    printf("Tree height: %d\n", tree_height(root));
    printf("Node count: %d\n", node_count(root));
    printf("Leaf count: %d\n", leaf_count(root));

    free_tree(root);
    return 0;
}
```

Binary Search Trees (BST)

A binary search tree is a binary tree where each node follows the property that all values in the left subtree are less than the node's value, and all values in the right subtree are greater than the node's value.

BST Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct BSTNode {
    int data;
    struct BSTNode *left;
    struct BSTNode *right;
} BSTNode;

// Create new BST node
BSTNode* create_bst_node(int data) {
    BSTNode *node = malloc(sizeof(BSTNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// Free BST
void free_bst(BSTNode *root) {
```

```
    if (root == NULL) return;

    free_bst(root->left);
    free_bst(root->right);
    free(root);
}

// Insert node in BST
BSTNode* insert_bst(BSTNode *root, int data) {
    if (root == NULL) {
        return create_bst_node(data);
    }

    if (data < root->data) {
        root->left = insert_bst(root->left, data);
    } else if (data > root->data) {
        root->right = insert_bst(root->right, data);
    }
    // If data == root->data, we don't insert duplicates

    return root;
}

// Search in BST
BSTNode* search_bst(BSTNode *root, int data) {
    if (root == NULL || root->data == data) {
        return root;
    }

    if (data < root->data) {
        return search_bst(root->left, data);
    } else {
        return search_bst(root->right, data);
    }
}

// Find minimum value node
BSTNode* find_min(BSTNode *root) {
    if (root == NULL) return NULL;

    while (root->left != NULL) {
        root = root->left;
    }

    return root;
}
```

```
// Find maximum value node
BSTNode* find_max(BSTNode *root) {
    if (root == NULL) return NULL;

    while (root->right != NULL) {
        root = root->right;
    }

    return root;
}

// Delete node from BST
BSTNode* delete_bst(BSTNode *root, int data) {
    if (root == NULL) return root;

    if (data < root->data) {
        root->left = delete_bst(root->left, data);
    } else if (data > root->data) {
        root->right = delete_bst(root->right, data);
    } else {
        // Node to be deleted found

        // Case 1: Node has no children
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }

        // Case 2: Node has one child
        if (root->left == NULL) {
            BSTNode *temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL) {
            BSTNode *temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: Node has two children
        BSTNode *temp = find_min(root->right);
        root->data = temp->data;
        root->right = delete_bst(root->right, temp->data);
    }
```

```
    return root;
}

// Inorder traversal (gives sorted order for BST)
void inorder_bst(BSTNode *root) {
    if (root == NULL) return;

    inorder_bst(root->left);
    printf("%d ", root->data);
    inorder_bst(root->right);
}

// Check if tree is valid BST
bool is_valid_bst(BSTNode *root, long min_val, long max_val) {
    if (root == NULL) return true;

    if (root->data <= min_val || root->data >= max_val) {
        return false;
    }

    return is_valid_bst(root->left, min_val, root->data) &&
           is_valid_bst(root->right, root->data, max_val);
}

bool validate_bst(BSTNode *root) {
    return is_valid_bst(root, LONG_MIN, LONG_MAX);
}

int main() {
    BSTNode *root = NULL;

    // Insert nodes
    int values[] = {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45};
    int n = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < n; i++) {
        root = insert_bst(root, values[i]);
    }

    printf("BST inorder traversal (sorted): ");
    inorder_bst(root);
    printf("\n");

    printf("Is valid BST: %s\n", validate_bst(root) ? "Yes" : "No");

    // Search operations
    int search_values[] = {25, 35, 90};
```

```
    for (int i = 0; i < 3; i++) {
        BSTNode *found = search_bst(root, search_values[i]);
        printf("Search %d: %s\n", search_values[i],
               found ? "Found" : "Not found");
    }

    // Find min and max
    BSTNode *min_node = find_min(root);
    BSTNode *max_node = find_max(root);
    printf("Minimum value: %d\n", min_node ? min_node->data : -1);
    printf("Maximum value: %d\n", max_node ? max_node->data : -1);

    // Delete operations
    printf("Deleting 20...\n");
    root = delete_bst(root, 20);
    printf("BST after deletion: ");
    inorder_bst(root);
    printf("\n");

    printf("Deleting 30...\n");
    root = delete_bst(root, 30);
    printf("BST after deletion: ");
    inorder_bst(root);
    printf("\n");

    printf("Deleting 50...\n");
    root = delete_bst(root, 50);
    printf("BST after deletion: ");
    inorder_bst(root);
    printf("\n");

    free_bst(root);
    return 0;
}
```

AVL Trees

AVL trees are self-balancing binary search trees where the heights of the two child subtrees of any node differ by at most one. This ensures O(log n) time complexity for search, insert, and delete operations.

AVL Tree Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>

typedef struct AVLNode {
    int data;
    struct AVLNode *left;
    struct AVLNode *right;
    int height;
} AVLNode;

// Get height of node
int get_height(AVLNode *node) {
    return node ? node->height : 0;
}

// Get balance factor
int get_balance(AVLNode *node) {
    return node ? get_height(node->left) - get_height(node->right) : 0;
}

// Create new AVL node
AVLNode* create_avl_node(int data) {
    AVLNode *node = malloc(sizeof(AVLNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;  // New node is initially at leaf level
    return node;
}

// Free AVL tree
void free_avl(AVLNode *root) {
    if (root == NULL) return;

    free_avl(root->left);
    free_avl(root->right);
    free(root);
}

// Update height of node
```

```
void update_height(AVLNode *node) {
    if (node != NULL) {
        node->height = 1 + fmax(get_height(node->left), get_height(node->right));
    }
}

// Right rotate
AVLNode* right_rotate(AVLNode *y) {
    AVLNode *x = y->left;
    AVLNode *T2 = x->right;

    // Perform rotation
    x->right = y;
    y->left = T2;

    // Update heights
    update_height(y);
    update_height(x);

    // Return new root
    return x;
}

// Left rotate
AVLNode* left_rotate(AVLNode *x) {
    AVLNode *y = x->right;
    AVLNode *T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

    // Update heights
    update_height(x);
    update_height(y);

    // Return new root
    return y;
}

// Insert node in AVL tree
AVLNode* insert_avl(AVLNode *root, int data) {
    // 1. Perform normal BST insertion
    if (root == NULL) {
        return create_avl_node(data);
    }
```

```
    if (data < root->data) {
        root->left = insert_avl(root->left, data);
    } else if (data > root->data) {
        root->right = insert_avl(root->right, data);
    } else {
        // Equal keys not allowed
        return root;
    }

    // 2. Update height of this ancestor node
    update_height(root);

    // 3. Get balance factor
    int balance = get_balance(root);

    // 4. If unbalanced, perform rotations

    // Left Left Case
    if (balance > 1 && data < root->left->data) {
        return right_rotate(root);
    }

    // Right Right Case
    if (balance < -1 && data > root->right->data) {
        return left_rotate(root);
    }

    // Left Right Case
    if (balance > 1 && data > root->left->data) {
        root->left = left_rotate(root->left);
        return right_rotate(root);
    }

    // Right Left Case
    if (balance < -1 && data < root->right->data) {
        root->right = right_rotate(root->right);
        return left_rotate(root);
    }

    // Return unchanged node pointer
    return root;
}

// Find node with minimum value
AVLNode* find_min_avl(AVLNode *root) {
    if (root == NULL) return NULL;
```

```
    while (root->left != NULL) {
        root = root->left;
    }

    return root;
}

// Delete node from AVL tree
AVLNode* delete_avl(AVLNode *root, int data) {
    // 1. Perform standard BST deletion
    if (root == NULL) return root;

    if (data < root->data) {
        root->left = delete_avl(root->left, data);
    } else if (data > root->data) {
        root->right = delete_avl(root->right, data);
    } else {
        // Node to be deleted found
        if (root->left == NULL || root->right == NULL) {
            AVLNode *temp = root->left ? root->left : root->right;

            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else {
                *root = *temp;
            }

            free(temp);
        } else {
            AVLNode *temp = find_min_avl(root->right);
            root->data = temp->data;
            root->right = delete_avl(root->right, temp->data);
        }
    }

    if (root == NULL) return root;

    // 2. Update height of current node
    update_height(root);

    // 3. Get balance factor
    int balance = get_balance(root);

    // 4. If unbalanced, perform rotations

    // Left Left Case
```

```
    if (balance > 1 && get_balance(root->left) >= 0) {
        return right_rotate(root);
    }

    // Left Right Case
    if (balance > 1 && get_balance(root->left) < 0) {
        root->left = left_rotate(root->left);
        return right_rotate(root);
    }

    // Right Right Case
    if (balance < -1 && get_balance(root->right) <= 0) {
        return left_rotate(root);
    }

    // Right Left Case
    if (balance < -1 && get_balance(root->right) > 0) {
        root->right = right_rotate(root->right);
        return left_rotate(root);
    }

    return root;
}

// Search in AVL tree
AVLNode* search_avl(AVLNode *root, int data) {
    if (root == NULL || root->data == data) {
        return root;
    }

    if (data < root->data) {
        return search_avl(root->left, data);
    } else {
        return search_avl(root->right, data);
    }
}

// Inorder traversal
void inorder_avl(AVLNode *root) {
    if (root == NULL) return;

    inorder_avl(root->left);
    printf("%d ", root->data);
    inorder_avl(root->right);
}

// Check if tree is balanced
```

```
bool is_balanced(AVLNode *root) {
    if (root == NULL) return true;

    int balance = get_balance(root);
    if (abs(balance) > 1) return false;

    return is_balanced(root->left) && is_balanced(root->right);
}

int main() {
    AVLNode *root = NULL;

    // Insert nodes
    int values[] = {10, 20, 30, 40, 50, 25};
    int n = sizeof(values) / sizeof(values[0]);

    printf("Inserting values: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", values[i]);
        root = insert_avl(root, values[i]);
    }
    printf("\n");

    printf("AVL tree inorder traversal: ");
    inorder_avl(root);
    printf("\n");

    printf("Is tree balanced: %s\n", is_balanced(root) ? "Yes" : "No");
    printf("Root height: %d\n", get_height(root));

    // Search operations
    int search_values[] = {25, 35, 60};
    for (int i = 0; i < 3; i++) {
        AVLNode *found = search_avl(root, search_values[i]);
        printf("Search %d: %s\n", search_values[i],
               found ? "Found" : "Not found");
    }

    // Delete operations
    printf("Deleting 30...\n");
    root = delete_avl(root, 30);
    printf("AVL tree after deletion: ");
    inorder_avl(root);
    printf("\n");

    printf("Is tree balanced: %s\n", is_balanced(root) ? "Yes" : "No");
```

```
    printf("Root height: %d\n", get_height(root));

    free_avl(root);
    return 0;
}
```

Red-Black Trees

Red-black trees are another type of self-balancing binary search tree with additional properties that ensure the tree remains approximately balanced during insertions and deletions.

Red-Black Tree Properties

1. Every node is either red or black
2. The root is black
3. All leaves (NIL nodes) are black
4. If a node is red, both its children are black
5. Every path from a node to its descendant leaves contains the same number of black nodes

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef enum { RED, BLACK } Color;

typedef struct RBNode {
    int data;
    Color color;
    struct RBNode *left;
    struct RBNode *right;
    struct RBNode *parent;
} RBNode;

// Global NIL node (sentinel)
RBNode *NIL;

// Initialize NIL node
void init_nil(void) {
    NIL = malloc(sizeof(RBNode));
    NIL->color = BLACK;
    NIL->left = NIL->right = NIL->parent = NULL;
}

// Create new red-black node
```

```c
RBNode* create_rb_node(int data) {
    RBNode *node = malloc(sizeof(RBNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->color = RED;  // New nodes are red by default
    node->left = node->right = node->parent = NIL;
    return node;
}

// Free red-black tree
void free_rb_tree(RBNode *root) {
    if (root == NULL || root == NIL) return;

    free_rb_tree(root->left);
    free_rb_tree(root->right);
    free(root);
}

// Left rotate
void left_rotate(RBNode **root, RBNode *x) {
    RBNode *y = x->right;

    // Turn y's left subtree into x's right subtree
    x->right = y->left;
    if (y->left != NIL) {
        y->left->parent = x;
    }

    // Link x's parent to y
    y->parent = x->parent;
    if (x->parent == NIL) {
        *root = y;
    } else if (x == x->parent->left) {
        x->parent->left = y;
    } else {
        x->parent->right = y;
    }

    // Put x on y's left
    y->left = x;
    x->parent = y;
}

// Right rotate
void right_rotate(RBNode **root, RBNode *y) {
```

```
    RBNode *x = y->left;

    // Turn x's right subtree into y's left subtree
    y->left = x->right;
    if (x->right != NIL) {
        x->right->parent = y;
    }

    // Link y's parent to x
    x->parent = y->parent;
    if (y->parent == NIL) {
        *root = x;
    } else if (y == y->parent->right) {
        y->parent->right = x;
    } else {
        y->parent->left = x;
    }

    // Put y on x's right
    x->right = y;
    y->parent = x;
}

// Fixup after insertion
void insert_fixup(RBNode **root, RBNode *z) {
    while (z->parent->color == RED) {
        if (z->parent == z->parent->parent->left) {
            RBNode *y = z->parent->parent->right;

            if (y->color == RED) {
                // Case 1: Uncle is red
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    // Case 2: Uncle is black and z is right child
                    z = z->parent;
                    left_rotate(root, z);
                }
                // Case 3: Uncle is black and z is left child
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                right_rotate(root, z->parent->parent);
            }
        } else {
```

```
            RBNode *y = z->parent->parent->left;

            if (y->color == RED) {
                // Case 1: Uncle is red
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->left) {
                    // Case 2: Uncle is black and z is left child
                    z = z->parent;
                    right_rotate(root, z);
                }
                // Case 3: Uncle is black and z is right child
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                left_rotate(root, z->parent->parent);
            }
        }
    }
    (*root)->color = BLACK;
}

// Insert node in red-black tree
RBNode* insert_rb(RBNode **root, int data) {
    RBNode *z = create_rb_node(data);
    if (z == NULL) return NULL;

    RBNode *y = NIL;
    RBNode *x = *root;

    // Find insertion point
    while (x != NIL) {
        y = x;
        if (z->data < x->data) {
            x = x->left;
        } else {
            x = x->right;
        }
    }

    z->parent = y;
    if (y == NIL) {
        *root = z;
    } else if (z->data < y->data) {
        y->left = z;
```

```
    } else {
        y->right = z;
    }

    // Fix red-black properties
    insert_fixup(root, z);

    return z;
}

// Inorder traversal
void inorder_rb(RBNode *root) {
    if (root == NULL || root == NIL) return;

    inorder_rb(root->left);
    printf("%d(%s) ", root->data, root->color == RED ? "R" : "B");
    inorder_rb(root->right);
}

int main() {
    init_nil();

    RBNode *root = NIL;

    // Insert nodes
    int values[] = {7, 3, 18, 10, 22, 8, 11, 26};
    int n = sizeof(values) / sizeof(values[0]);

    printf("Inserting values into red-black tree: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", values[i]);
        insert_rb(&root, values[i]);
    }
    printf("\n");

    printf("Red-black tree inorder traversal: ");
    inorder_rb(root);
    printf("\n");

    free_rb_tree(root);
    free(NIL);
    return 0;
}
```

B-Trees

B-trees are balanced tree data structures that maintain sorted data and allow for efficient insertion, deletion, and search operations. They are commonly used in databases and file systems.

B-Tree Implementation (Simplified)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define T 3  // Minimum degree

typedef struct BTreeNode {
    int n;                     // Number of keys
    int keys[2 * T - 1];       // Array of keys
    struct BTreeNode *children[2 * T];  // Array of child pointers
    bool leaf;                 // True if node is leaf
} BTreeNode;

// Create new B-tree node
BTreeNode* create_btree_node(bool leaf) {
    BTreeNode *node = malloc(sizeof(BTreeNode));
    if (node == NULL) return NULL;

    node->n = 0;
    node->leaf = leaf;

    for (int i = 0; i < 2 * T; i++) {
        node->children[i] = NULL;
    }

    return node;
}

// Search for key in B-tree
BTreeNode* search_btree(BTreeNode *root, int key) {
    int i = 0;
    while (i < root->n && key > root->keys[i]) {
        i++;
    }

    if (i < root->n && key == root->keys[i]) {
        return root;
    }
```

```
    if (root->leaf) {
        return NULL;
    }

    return search_btree(root->children[i], key);
}

// Split child node
void split_child(BTreeNode *parent, int i, BTreeNode *child) {
    BTreeNode *z = create_btree_node(child->leaf);
    z->n = T - 1;

    // Copy the last T-1 keys of child to z
    for (int j = 0; j < T - 1; j++) {
        z->keys[j] = child->keys[j + T];
    }

    // Copy the last T children of child to z
    if (!child->leaf) {
        for (int j = 0; j < T; j++) {
            z->children[j] = child->children[j + T];
        }
    }

    child->n = T - 1;

    // Shift parent's children to make space for z
    for (int j = parent->n; j >= i + 1; j--) {
        parent->children[j + 1] = parent->children[j];
    }

    parent->children[i + 1] = z;

    // Shift parent's keys to make space
    for (int j = parent->n - 1; j >= i; j--) {
        parent->keys[j + 1] = parent->keys[j];
    }

    parent->keys[i] = child->keys[T - 1];
    parent->n++;
}

// Insert non-full node
void insert_non_full(BTreeNode *node, int key) {
    int i = node->n - 1;

    if (node->leaf) {
```

```
        // Find location to insert key and shift keys
        while (i >= 0 && node->keys[i] > key) {
            node->keys[i + 1] = node->keys[i];
            i--;
        }

        node->keys[i + 1] = key;
        node->n++;
    } else {
        // Find child to recurse on
        while (i >= 0 && node->keys[i] > key) {
            i--;
        }
        i++;

        // Split child if it's full
        if (node->children[i]->n == 2 * T - 1) {
            split_child(node, i, node->children[i]);

            if (node->keys[i] < key) {
                i++;
            }
        }

        insert_non_full(node->children[i], key);
    }
}

// Insert key in B-tree
BTreeNode* insert_btree(BTreeNode *root, int key) {
    if (root == NULL) {
        root = create_btree_node(true);
        root->keys[0] = key;
        root->n = 1;
        return root;
    }

    // If root is full, split it
    if (root->n == 2 * T - 1) {
        BTreeNode *new_root = create_btree_node(false);
        new_root->children[0] = root;
        split_child(new_root, 0, root);

        int i = 0;
        if (new_root->keys[0] < key) {
            i++;
        }
```

```
        insert_non_full(new_root->children[i], key);
        return new_root;
    } else {
        insert_non_full(root, key);
        return root;
    }
}

// Print B-tree
void print_btree(BTreeNode *root, int level) {
    if (root == NULL) return;

    for (int i = 0; i < level; i++) {
        printf("  ");
    }

    printf("Level %d: [", level);
    for (int i = 0; i < root->n; i++) {
        printf("%d ", root->keys[i]);
    }
    printf("]\n");

    if (!root->leaf) {
        for (int i = 0; i <= root->n; i++) {
            print_btree(root->children[i], level + 1);
        }
    }
}

int main() {
    BTreeNode *root = NULL;

    // Insert nodes
    int values[] = {10, 20, 5, 6, 12, 30, 7, 17};
    int n = sizeof(values) / sizeof(values[0]);

    printf("Inserting values into B-tree: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", values[i]);
        root = insert_btree(root, values[i]);
    }
    printf("\n\n");

    printf("B-tree structure:\n");
    print_btree(root, 0);
```

```
    // Search operations
    int search_values[] = {6, 15, 17};
    for (int i = 0; i < 3; i++) {
        BTreeNode *found = search_btree(root, search_values[i]);
        printf("Search %d: %s\n", search_values[i],
               found ? "Found" : "Not found");
    }

    return 0;
}
```

Practical Examples

File System Directory Tree

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

typedef struct DirectoryNode {
    char name[100];
    bool is_directory;
    long size;  // For files
    struct DirectoryNode *first_child;
    struct DirectoryNode *next_sibling;
} DirectoryNode;

// Create new directory node
DirectoryNode* create_directory_node(const char *name, bool is_dir, long size) {
    DirectoryNode *node = malloc(sizeof(DirectoryNode));
    if (node == NULL) return NULL;

    strncpy(node->name, name, sizeof(node->name) - 1);
    node->name[sizeof(node->name) - 1] = '\0';
    node->is_directory = is_dir;
    node->size = size;
    node->first_child = NULL;
    node->next_sibling = NULL;

    return node;
}

// Add child to directory
```

```c
void add_child(DirectoryNode *parent, DirectoryNode *child) {
    if (parent->first_child == NULL) {
        parent->first_child = child;
    } else {
        DirectoryNode *current = parent->first_child;
        while (current->next_sibling != NULL) {
            current = current->next_sibling;
        }
        current->next_sibling = child;
    }
}

// Print directory tree
void print_directory_tree(DirectoryNode *root, int level) {
    if (root == NULL) return;

    for (int i = 0; i < level; i++) {
        printf("  ");
    }

    if (root->is_directory) {
        printf("[%s]\n", root->name);
    } else {
        printf("%s (%ld bytes)\n", root->name, root->size);
    }

    // Print children
    DirectoryNode *child = root->first_child;
    while (child != NULL) {
        print_directory_tree(child, level + 1);
        child = child->next_sibling;
    }
}

// Calculate directory size
long calculate_directory_size(DirectoryNode *root) {
    if (root == NULL) return 0;

    if (!root->is_directory) {
        return root->size;
    }

    long total_size = 0;
    DirectoryNode *child = root->first_child;
    while (child != NULL) {
        total_size += calculate_directory_size(child);
```

```
        child = child->next_sibling;
    }

    return total_size;
}

// Find file by name
DirectoryNode* find_file(DirectoryNode *root, const char *name) {
    if (root == NULL) return NULL;

    if (strcmp(root->name, name) == 0) {
        return root;
    }

    // Search in children
    DirectoryNode *child = root->first_child;
    while (child != NULL) {
        DirectoryNode *found = find_file(child, name);
        if (found != NULL) {
            return found;
        }
        child = child->next_sibling;
    }

    return NULL;
}

int main() {
    // Create file system structure
    DirectoryNode *root = create_directory_node("root", true, 0);

    // Create directories
    DirectoryNode *home = create_directory_node("home", true, 0);
    DirectoryNode *usr = create_directory_node("usr", true, 0);
    DirectoryNode *john = create_directory_node("john", true, 0);
    DirectoryNode *documents = create_directory_node("documents", true, 0);
    DirectoryNode *pictures = create_directory_node("pictures", true, 0);

    // Create files
    DirectoryNode *file1 = create_directory_node("readme.txt", false, 1024);
    DirectoryNode *file2 = create_directory_node("report.pdf", false, 204800);
    DirectoryNode *file3 = create_directory_node("photo.jpg", false, 512000);
    DirectoryNode *file4 = create_directory_node("vacation.png", false, 1024000);

    // Build tree structure
    add_child(root, home);
    add_child(root, usr);
```

```
    add_child(home, john);
    add_child(john, documents);
    add_child(john, pictures);
    add_child(documents, file1);
    add_child(documents, file2);
    add_child(pictures, file3);
    add_child(pictures, file4);

    // Print directory tree
    printf("File System Directory Tree:\n");
    print_directory_tree(root, 0);

    // Calculate sizes
    printf("\nDirectory Sizes:\n");
    printf("Root directory size: %ld bytes\n", calculate_directory_size(root));
    printf("John's directory size: %ld bytes\n", calculate_directory_size(john));
    printf("Documents directory size: %ld bytes\n", calculate_directory_size(documents));

    // Find files
    printf("\nFile Search:\n");
    DirectoryNode *found = find_file(root, "photo.jpg");
    if (found) {
        printf("Found file: %s (%ld bytes)\n", found->name, found->size);
    } else {
        printf("File not found\n");
    }

    return 0;
}
```

Summary

Tree structures are essential data structures in computer science with various applications:

1. **Binary Trees**: Basic hierarchical structure with at most two children per node
2. **Binary Search Trees**: Ordered binary trees with efficient search, insert, and delete operations
3. **AVL Trees**: Self-balancing BSTs that maintain O(log n) operations
4. **Red-Black Trees**: Another self-balancing BST with relaxed balancing requirements
5. **B-Trees**: Multi-way trees optimized for disk storage and databases

Key properties and considerations:

- **Time Complexity**:
  - BST (unbalanced): O(n) worst case, O(log n) average
  - Balanced trees (AVL, Red-Black): O(log n) guaranteed

  - B-Trees: O(log n) with better cache performance

- **Space Complexity**: O(n) for n nodes
- **Use Cases**:
  - BST: Expression trees, decision trees
  - AVL/Red-Black: In-memory databases, symbol tables
  - B-Trees: File systems, database indexes

Understanding tree structures and their variations is crucial for efficient data organization and retrieval in complex applications.

Hash Tables

Introduction

Hash tables (also known as hash maps) are data structures that store key-value pairs and provide efficient insertion, deletion, and lookup operations with an average time complexity of O(1). They achieve this efficiency by using a hash function to compute an index into an array of buckets or slots, from which the desired value can be found.

Hash tables are fundamental in computer science and are used extensively in databases, caches, sets, and many other applications where fast data retrieval is essential.

Hash Function Design

A good hash function is crucial for the performance of a hash table. It should distribute keys uniformly across the hash table to minimize collisions and ensure efficient operations.

Basic Hash Functions

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Simple hash function for integers
unsigned int hash_int(int key, int table_size) {
    return key % table_size;
}

// Simple hash function for strings (djb2 algorithm)
unsigned int hash_string(const char *str, int table_size) {
    unsigned long hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;  // hash * 33 + c
    }

    return hash % table_size;
```

```c
}

// Simple hash function for strings (sdbm algorithm)
unsigned int hash_string_sdbm(const char *str, int table_size) {
    unsigned long hash = 0;
    int c;

    while ((c = *str++)) {
        hash = c + (hash << 6) + (hash << 16) - hash;
    }

    return hash % table_size;
}

// Universal hash function
unsigned int hash_universal(unsigned int key, int table_size,
                            unsigned int a, unsigned int b, unsigned int p) {
    // Universal hashing: h(key) = ((a * key + b) mod p) mod table_size
    // where p is a prime number >= table_size
    return ((a * key + b) % p) % table_size;
}

int main() {
    int table_size = 100;

    // Test integer hash function
    printf("Hash values for integers:\n");
    for (int i = 1; i <= 10; i++) {
        printf("Key %d -> Hash %u\n", i, hash_int(i, table_size));
    }

    printf("\nHash values for strings (djb2):\n");
    const char *strings[] = {"hello", "world", "hash", "table", "example"};
    for (int i = 0; i < 5; i++) {
        printf("Key '%s' -> Hash %u\n", strings[i], hash_string(strings[i], table_size));
    }

    printf("\nHash values for strings (sdbm):\n");
    for (int i = 0; i < 5; i++) {
        printf("Key '%s' -> Hash %u\n", strings[i], hash_string_sdbm(strings[i], table_size)
    }

    return 0;
}
```

Collision Resolution Strategies

When two keys hash to the same index, a collision occurs. There are several strategies to handle collisions:

Chaining

In chaining, each bucket contains a linked list of elements that hash to the same index.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define TABLE_SIZE 10

typedef struct HashNode {
    char key[100];
    int value;
    struct HashNode *next;
} HashNode;

typedef struct {
    HashNode *buckets[TABLE_SIZE];
    int size;
} HashTable;

// Create new hash node
HashNode* create_hash_node(const char *key, int value) {
    HashNode *node = malloc(sizeof(HashNode));
    if (node == NULL) return NULL;

    strncpy(node->key, key, sizeof(node->key) - 1);
    node->key[sizeof(node->key) - 1] = '\0';
    node->value = value;
    node->next = NULL;

    return node;
}

// Initialize hash table
HashTable* create_hash_table(void) {
    HashTable *table = malloc(sizeof(HashTable));
    if (table == NULL) return NULL;

    for (int i = 0; i < TABLE_SIZE; i++) {
```

```
        table->buckets[i] = NULL;
    }

    table->size = 0;
    return table;
}

// Free hash table
void free_hash_table(HashTable *table) {
    if (table == NULL) return;

    for (int i = 0; i < TABLE_SIZE; i++) {
        HashNode *current = table->buckets[i];
        while (current != NULL) {
            HashNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(table);
}

// Hash function for strings
unsigned int hash(const char *key) {
    unsigned long hash = 5381;
    int c;

    while ((c = *key++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % TABLE_SIZE;
}

// Insert key-value pair
bool hash_table_insert(HashTable *table, const char *key, int value) {
    if (table == NULL || key == NULL) return false;

    unsigned int index = hash(key);

    // Check if key already exists
    HashNode *current = table->buckets[index];
    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            current->value = value;  // Update existing value
            return true;
```

```
        }
        current = current->next;
    }

    // Create new node and insert at beginning of chain
    HashNode *new_node = create_hash_node(key, value);
    if (new_node == NULL) return false;

    new_node->next = table->buckets[index];
    table->buckets[index] = new_node;
    table->size++;

    return true;
}

// Search for key
bool hash_table_search(HashTable *table, const char *key, int *value) {
    if (table == NULL || key == NULL) return false;

    unsigned int index = hash(key);

    HashNode *current = table->buckets[index];
    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            if (value != NULL) {
                *value = current->value;
            }
            return true;
        }
        current = current->next;
    }

    return false;
}

// Delete key
bool hash_table_delete(HashTable *table, const char *key) {
    if (table == NULL || key == NULL) return false;

    unsigned int index = hash(key);

    HashNode *current = table->buckets[index];
    HashNode *prev = NULL;

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            if (prev == NULL) {
```

```
                // First node in chain
                table->buckets[index] = current->next;
            } else {
                prev->next = current->next;
            }

            free(current);
            table->size--;
            return true;
        }

        prev = current;
        current = current->next;
    }

    return false;
}

// Print hash table
void print_hash_table(HashTable *table) {
    if (table == NULL) return;

    printf("Hash Table (size: %d):\n", table->size);
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf("Bucket %d: ", i);

        HashNode *current = table->buckets[i];
        if (current == NULL) {
            printf("empty\n");
        } else {
            while (current != NULL) {
                printf("(%s: %d) -> ", current->key, current->value);
                current = current->next;
            }
            printf("NULL\n");
        }
    }
}

int main() {
    HashTable *table = create_hash_table();
    if (table == NULL) {
        printf("Failed to create hash table\n");
        return 1;
    }
```

```
    // Insert key-value pairs
    hash_table_insert(table, "apple", 5);
    hash_table_insert(table, "banana", 3);
    hash_table_insert(table, "orange", 8);
    hash_table_insert(table, "grape", 12);
    hash_table_insert(table, "kiwi", 7);

    print_hash_table(table);

    // Search for keys
    int value;
    if (hash_table_search(table, "apple", &value)) {
        printf("Found 'apple': %d\n", value);
    }

    if (hash_table_search(table, "mango", &value)) {
        printf("Found 'mango': %d\n", value);
    } else {
        printf("'mango' not found\n");
    }

    // Update existing key
    hash_table_insert(table, "apple", 10);
    printf("After updating 'apple':\n");
    if (hash_table_search(table, "apple", &value)) {
        printf("Found 'apple': %d\n", value);
    }

    // Delete key
    if (hash_table_delete(table, "banana")) {
        printf("Deleted 'banana'\n");
    }

    print_hash_table(table);

    free_hash_table(table);
    return 0;
}
```

Open Addressing

In open addressing, all elements are stored in the hash table itself. When a collision occurs, alternative slots are probed until an empty slot is found.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define TABLE_SIZE 13
#define EMPTY_CELL -1
#define DELETED_CELL -2

typedef struct {
    char key[100];
    int value;
    int status;  // EMPTY_CELL, DELETED_CELL, or occupied
} HashEntry;

typedef struct {
    HashEntry entries[TABLE_SIZE];
    int size;
} OpenHashTable;

// Initialize open hash table
OpenHashTable* create_open_hash_table(void) {
    OpenHashTable *table = malloc(sizeof(OpenHashTable));
    if (table == NULL) return NULL;

    for (int i = 0; i < TABLE_SIZE; i++) {
        table->entries[i].status = EMPTY_CELL;
    }

    table->size = 0;
    return table;
}

// Free open hash table
void free_open_hash_table(OpenHashTable *table) {
    free(table);
}

// Hash function
unsigned int hash_open(const char *key) {
    unsigned long hash = 5381;
    int c;

    while ((c = *key++)) {
        hash = ((hash << 5) + hash) + c;
    }
```

```
    return hash % TABLE_SIZE;
}

// Linear probing hash function
unsigned int hash_linear_probe(const char *key, int attempt) {
    return (hash_open(key) + attempt) % TABLE_SIZE;
}

// Quadratic probing hash function
unsigned int hash_quadratic_probe(const char *key, int attempt) {
    return (hash_open(key) + attempt * attempt) % TABLE_SIZE;
}

// Double hashing
unsigned int hash_double(const char *key, int attempt) {
    unsigned int hash1 = hash_open(key);
    unsigned int hash2 = 7 - (hash_open(key) % 7);  // Second hash function
    return (hash1 + attempt * hash2) % TABLE_SIZE;
}

// Insert with linear probing
bool open_hash_table_insert(OpenHashTable *table, const char *key, int value) {
    if (table == NULL || key == NULL) return false;

    int attempt = 0;
    unsigned int index;

    do {
        index = hash_linear_probe(key, attempt);

        if (table->entries[index].status == EMPTY_CELL ||
            table->entries[index].status == DELETED_CELL) {
            // Found empty or deleted slot
            strncpy(table->entries[index].key, key, sizeof(table->entries[index].key) - 1);
            table->entries[index].key[sizeof(table->entries[index].key) - 1] = '\0';
            table->entries[index].value = value;
            table->entries[index].status = 0;  // Occupied
            table->size++;
            return true;
        } else if (strcmp(table->entries[index].key, key) == 0) {
            // Key already exists, update value
            table->entries[index].value = value;
            return true;
        }

        attempt++;
    } while (attempt < TABLE_SIZE);
```

```
    // Hash table is full
    return false;
}

// Search with linear probing
bool open_hash_table_search(OpenHashTable *table, const char *key, int *value) {
    if (table == NULL || key == NULL) return false;

    int attempt = 0;
    unsigned int index;

    do {
        index = hash_linear_probe(key, attempt);

        if (table->entries[index].status == EMPTY_CELL) {
            // Key not found
            return false;
        } else if (table->entries[index].status != DELETED_CELL &&
                   strcmp(table->entries[index].key, key) == 0) {
            // Key found
            if (value != NULL) {
                *value = table->entries[index].value;
            }
            return true;
        }

        attempt++;
    } while (attempt < TABLE_SIZE);

    // Key not found
    return false;
}

// Delete with linear probing
bool open_hash_table_delete(OpenHashTable *table, const char *key) {
    if (table == NULL || key == NULL) return false;

    int attempt = 0;
    unsigned int index;

    do {
        index = hash_linear_probe(key, attempt);

        if (table->entries[index].status == EMPTY_CELL) {
            // Key not found
            return false;
```

```
        } else if (table->entries[index].status != DELETED_CELL &&
                   strcmp(table->entries[index].key, key) == 0) {
            // Key found, mark as deleted
            table->entries[index].status = DELETED_CELL;
            table->size--;
            return true;
        }

        attempt++;
    } while (attempt < TABLE_SIZE);

    // Key not found
    return false;
}

// Print open hash table
void print_open_hash_table(OpenHashTable *table) {
    if (table == NULL) return;

    printf("Open Hash Table (size: %d):\n", table->size);
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf("Index %d: ", i);
        if (table->entries[i].status == EMPTY_CELL) {
            printf("EMPTY\n");
        } else if (table->entries[i].status == DELETED_CELL) {
            printf("DELETED\n");
        } else {
            printf("(%s: %d)\n", table->entries[i].key, table->entries[i].value);
        }
    }
}

int main() {
    OpenHashTable *table = create_open_hash_table();
    if (table == NULL) {
        printf("Failed to create hash table\n");
        return 1;
    }

    // Insert key-value pairs
    open_hash_table_insert(table, "apple", 5);
    open_hash_table_insert(table, "banana", 3);
    open_hash_table_insert(table, "orange", 8);
    open_hash_table_insert(table, "grape", 12);
    open_hash_table_insert(table, "kiwi", 7);

    print_open_hash_table(table);
```

```
    // Search for keys
    int value;
    if (open_hash_table_search(table, "apple", &value)) {
        printf("Found 'apple': %d\n", value);
    }

    if (open_hash_table_search(table, "mango", &value)) {
        printf("Found 'mango': %d\n", value);
    } else {
        printf("'mango' not found\n");
    }

    // Update existing key
    open_hash_table_insert(table, "apple", 10);
    printf("After updating 'apple':\n");
    if (open_hash_table_search(table, "apple", &value)) {
        printf("Found 'apple': %d\n", value);
    }

    // Delete key
    if (open_hash_table_delete(table, "banana")) {
        printf("Deleted 'banana'\n");
    }

    print_open_hash_table(table);

    free_open_hash_table(table);
    return 0;
}
```

Dynamic Hash Tables

Dynamic hash tables can resize themselves when the load factor becomes too high, ensuring efficient performance.

Resizable Hash Table with Chaining

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define INITIAL_SIZE 8
```

```c
#define LOAD_FACTOR_THRESHOLD 0.75

typedef struct DynamicHashNode {
    char key[100];
    int value;
    struct DynamicHashNode *next;
} DynamicHashNode;

typedef struct {
    DynamicHashNode **buckets;
    int size;
    int capacity;
} DynamicHashTable;

// Create new hash node
DynamicHashNode* create_dynamic_hash_node(const char *key, int value) {
    DynamicHashNode *node = malloc(sizeof(DynamicHashNode));
    if (node == NULL) return NULL;

    strncpy(node->key, key, sizeof(node->key) - 1);
    node->key[sizeof(node->key) - 1] = '\0';
    node->value = value;
    node->next = NULL;

    return node;
}

// Initialize dynamic hash table
DynamicHashTable* create_dynamic_hash_table(void) {
    DynamicHashTable *table = malloc(sizeof(DynamicHashTable));
    if (table == NULL) return NULL;

    table->buckets = calloc(INITIAL_SIZE, sizeof(DynamicHashNode*));
    if (table->buckets == NULL) {
        free(table);
        return NULL;
    }

    table->size = 0;
    table->capacity = INITIAL_SIZE;
    return table;
}

// Free dynamic hash table
void free_dynamic_hash_table(DynamicHashTable *table) {
    if (table == NULL) return;
```

```c
    for (int i = 0; i < table->capacity; i++) {
        DynamicHashNode *current = table->buckets[i];
        while (current != NULL) {
            DynamicHashNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(table->buckets);
    free(table);
}

// Hash function
unsigned int dynamic_hash(const char *key, int capacity) {
    unsigned long hash = 5381;
    int c;

    while ((c = *key++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % capacity;
}

// Resize hash table
bool resize_hash_table(DynamicHashTable *table) {
    int old_capacity = table->capacity;
    int new_capacity = old_capacity * 2;

    DynamicHashNode **old_buckets = table->buckets;
    DynamicHashNode **new_buckets = calloc(new_capacity, sizeof(DynamicHashNode*));
    if (new_buckets == NULL) return false;

    table->buckets = new_buckets;
    table->capacity = new_capacity;

    // Rehash all elements
    for (int i = 0; i < old_capacity; i++) {
        DynamicHashNode *current = old_buckets[i];
        while (current != NULL) {
            DynamicHashNode *next = current->next;

            unsigned int new_index = dynamic_hash(current->key, new_capacity);
            current->next = table->buckets[new_index];
            table->buckets[new_index] = current;
```

```
            current = next;
        }
    }

    free(old_buckets);
    return true;
}

// Insert key-value pair
bool dynamic_hash_table_insert(DynamicHashTable *table, const char *key, int value) {
    if (table == NULL || key == NULL) return false;

    // Check if resize is needed
    double load_factor = (double)table->size / table->capacity;
    if (load_factor > LOAD_FACTOR_THRESHOLD) {
        if (!resize_hash_table(table)) {
            return false;
        }
    }

    unsigned int index = dynamic_hash(key, table->capacity);

    // Check if key already exists
    DynamicHashNode *current = table->buckets[index];
    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            current->value = value;  // Update existing value
            return true;
        }
        current = current->next;
    }

    // Create new node and insert at beginning of chain
    DynamicHashNode *new_node = create_dynamic_hash_node(key, value);
    if (new_node == NULL) return false;

    new_node->next = table->buckets[index];
    table->buckets[index] = new_node;
    table->size++;

    return true;
}

// Search for key
bool dynamic_hash_table_search(DynamicHashTable *table, const char *key, int *value) {
    if (table == NULL || key == NULL) return false;
```

```
    unsigned int index = dynamic_hash(key, table->capacity);

    DynamicHashNode *current = table->buckets[index];
    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            if (value != NULL) {
                *value = current->value;
            }
            return true;
        }
        current = current->next;
    }

    return false;
}

// Delete key
bool dynamic_hash_table_delete(DynamicHashTable *table, const char *key) {
    if (table == NULL || key == NULL) return false;

    unsigned int index = dynamic_hash(key, table->capacity);

    DynamicHashNode *current = table->buckets[index];
    DynamicHashNode *prev = NULL;

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            if (prev == NULL) {
                // First node in chain
                table->buckets[index] = current->next;
            } else {
                prev->next = current->next;
            }

            free(current);
            table->size--;
            return true;
        }

        prev = current;
        current = current->next;
    }

    return false;
}
```

```c
// Print dynamic hash table
void print_dynamic_hash_table(DynamicHashTable *table) {
    if (table == NULL) return;

    printf("Dynamic Hash Table (size: %d, capacity: %d, load factor: %.2f):\n",
           table->size, table->capacity, (double)table->size / table->capacity);

    for (int i = 0; i < table->capacity; i++) {
        printf("Bucket %d: ", i);

        DynamicHashNode *current = table->buckets[i];
        if (current == NULL) {
            printf("empty\n");
        } else {
            while (current != NULL) {
                printf("(%s: %d) -> ", current->key, current->value);
                current = current->next;
            }
            printf("NULL\n");
        }
    }
}

int main() {
    DynamicHashTable *table = create_dynamic_hash_table();
    if (table == NULL) {
        printf("Failed to create hash table\n");
        return 1;
    }

    // Insert many key-value pairs to trigger resizing
    char key[20];
    for (int i = 0; i < 20; i++) {
        sprintf(key, "key%d", i);
        dynamic_hash_table_insert(table, key, i * 10);

        // Print status periodically
        if (i % 5 == 4) {
            printf("After inserting %d elements:\n", i + 1);
            print_dynamic_hash_table(table);
            printf("\n");
        }
    }

    // Search for some keys
    int value;
```

```
    if (dynamic_hash_table_search(table, "key5", &value)) {
        printf("Found 'key5': %d\n", value);
    }

    if (dynamic_hash_table_search(table, "key15", &value)) {
        printf("Found 'key15': %d\n", value);
    }

    // Delete some keys
    if (dynamic_hash_table_delete(table, "key10")) {
        printf("Deleted 'key10'\n");
    }

    printf("Final hash table:\n");
    print_dynamic_hash_table(table);

    free_dynamic_hash_table(table);
    return 0;
}
```

Hash Table Performance Analysis

Load Factor and Performance

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define TABLE_SIZE 1000

typedef struct PerfNode {
    int key;
    int value;
    struct PerfNode *next;
} PerfNode;

typedef struct {
    PerfNode *buckets[TABLE_SIZE];
    int size;
} PerfHashTable;

// Create performance hash table
PerfHashTable* create_perf_hash_table(void) {
    PerfHashTable *table = malloc(sizeof(PerfHashTable));
```

```
    if (table == NULL) return NULL;

    for (int i = 0; i < TABLE_SIZE; i++) {
        table->buckets[i] = NULL;
    }

    table->size = 0;
    return table;
}

// Free performance hash table
void free_perf_hash_table(PerfHashTable *table) {
    if (table == NULL) return;

    for (int i = 0; i < TABLE_SIZE; i++) {
        PerfNode *current = table->buckets[i];
        while (current != NULL) {
            PerfNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(table);
}

// Hash function
unsigned int perf_hash(int key) {
    return key % TABLE_SIZE;
}

// Insert with performance measurement
bool perf_hash_table_insert(PerfHashTable *table, int key, int value) {
    if (table == NULL) return false;

    unsigned int index = perf_hash(key);

    PerfNode *new_node = malloc(sizeof(PerfNode));
    if (new_node == NULL) return false;

    new_node->key = key;
    new_node->value = value;
    new_node->next = table->buckets[index];
    table->buckets[index] = new_node;
    table->size++;

    return true;
```

```
}

// Calculate average chain length
double calculate_average_chain_length(PerfHashTable *table) {
    if (table == NULL) return 0.0;

    int total_length = 0;
    int non_empty_buckets = 0;

    for (int i = 0; i < TABLE_SIZE; i++) {
        if (table->buckets[i] != NULL) {
            non_empty_buckets++;
            int length = 0;
            PerfNode *current = table->buckets[i];
            while (current != NULL) {
                length++;
                current = current->next;
            }
            total_length += length;
        }
    }

    return non_empty_buckets > 0 ? (double)total_length / non_empty_buckets : 0.0;
}

// Calculate load factor
double calculate_load_factor(PerfHashTable *table) {
    if (table == NULL) return 0.0;
    return (double)table->size / TABLE_SIZE;
}

// Find maximum chain length
int find_max_chain_length(PerfHashTable *table) {
    if (table == NULL) return 0;

    int max_length = 0;

    for (int i = 0; i < TABLE_SIZE; i++) {
        int length = 0;
        PerfNode *current = table->buckets[i];
        while (current != NULL) {
            length++;
            current = current->next;
        }
        if (length > max_length) {
            max_length = length;
        }
```

```
    }

    return max_length;
}

int main() {
    PerfHashTable *table = create_perf_hash_table();
    if (table == NULL) {
        printf("Failed to create hash table\n");
        return 1;
    }

    // Insert elements with different patterns
    printf("Performance Analysis of Hash Table:\n");
    printf("===================================\n\n");

    // Pattern 1: Sequential keys
    printf("Inserting 500 sequential keys (0-499):\n");
    clock_t start = clock();
    for (int i = 0; i < 500; i++) {
        perf_hash_table_insert(table, i, i * 2);
    }
    clock_t end = clock();

    printf("Time taken: %f seconds\n", ((double)(end - start)) / CLOCKS_PER_SEC);
    printf("Load factor: %.2f\n", calculate_load_factor(table));
    printf("Average chain length: %.2f\n", calculate_average_chain_length(table));
    printf("Maximum chain length: %d\n\n", find_max_chain_length(table));

    // Pattern 2: Random keys
    printf("Inserting 500 random keys:\n");
    srand(42);  // Fixed seed for reproducibility
    start = clock();
    for (int i = 0; i < 500; i++) {
        int random_key = rand() % 10000;
        perf_hash_table_insert(table, random_key, random_key * 2);
    }
    end = clock();

    printf("Time taken: %f seconds\n", ((double)(end - start)) / CLOCKS_PER_SEC);
    printf("Load factor: %.2f\n", calculate_load_factor(table));
    printf("Average chain length: %.2f\n", calculate_average_chain_length(table));
    printf("Maximum chain length: %d\n\n", find_max_chain_length(table));

    free_perf_hash_table(table);
    return 0;
}
```

Practical Examples

Symbol Table Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define SYMBOL_TABLE_SIZE 101

typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_STRING,
    TYPE_FUNCTION
} SymbolType;

typedef struct Symbol {
    char name[100];
    SymbolType type;
    union {
        int int_val;
        float float_val;
        char *string_val;
        void (*func_ptr)(void);
    } value;
    struct Symbol *next;
} Symbol;

typedef struct {
    Symbol *buckets[SYMBOL_TABLE_SIZE];
} SymbolTable;

// Create symbol table
SymbolTable* create_symbol_table(void) {
    SymbolTable *table = malloc(sizeof(SymbolTable));
    if (table == NULL) return NULL;

    for (int i = 0; i < SYMBOL_TABLE_SIZE; i++) {
        table->buckets[i] = NULL;
    }

    return table;
}
```

```c
// Free symbol table
void free_symbol_table(SymbolTable *table) {
    if (table == NULL) return;

    for (int i = 0; i < SYMBOL_TABLE_SIZE; i++) {
        Symbol *current = table->buckets[i];
        while (current != NULL) {
            Symbol *next = current->next;
            if (current->type == TYPE_STRING) {
                free(current->value.string_val);
            }
            free(current);
            current = next;
        }
    }

    free(table);
}

// Hash function for symbol names
unsigned int symbol_hash(const char *name) {
    unsigned long hash = 5381;
    int c;

    while ((c = *name++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % SYMBOL_TABLE_SIZE;
}

// Add integer symbol
bool add_integer_symbol(SymbolTable *table, const char *name, int value) {
    if (table == NULL || name == NULL) return false;

    unsigned int index = symbol_hash(name);

    Symbol *new_symbol = malloc(sizeof(Symbol));
    if (new_symbol == NULL) return false;

    strncpy(new_symbol->name, name, sizeof(new_symbol->name) - 1);
    new_symbol->name[sizeof(new_symbol->name) - 1] = '\0';
    new_symbol->type = TYPE_INT;
    new_symbol->value.int_val = value;
    new_symbol->next = table->buckets[index];
    table->buckets[index] = new_symbol;
```

```
    return true;
}

// Add float symbol
bool add_float_symbol(SymbolTable *table, const char *name, float value) {
    if (table == NULL || name == NULL) return false;

    unsigned int index = symbol_hash(name);

    Symbol *new_symbol = malloc(sizeof(Symbol));
    if (new_symbol == NULL) return false;

    strncpy(new_symbol->name, name, sizeof(new_symbol->name) - 1);
    new_symbol->name[sizeof(new_symbol->name) - 1] = '\0';
    new_symbol->type = TYPE_FLOAT;
    new_symbol->value.float_val = value;
    new_symbol->next = table->buckets[index];
    table->buckets[index] = new_symbol;

    return true;
}

// Add string symbol
bool add_string_symbol(SymbolTable *table, const char *name, const char *value) {
    if (table == NULL || name == NULL || value == NULL) return false;

    unsigned int index = symbol_hash(name);

    Symbol *new_symbol = malloc(sizeof(Symbol));
    if (new_symbol == NULL) return false;

    strncpy(new_symbol->name, name, sizeof(new_symbol->name) - 1);
    new_symbol->name[sizeof(new_symbol->name) - 1] = '\0';
    new_symbol->type = TYPE_STRING;
    new_symbol->value.string_val = malloc(strlen(value) + 1);
    if (new_symbol->value.string_val == NULL) {
        free(new_symbol);
        return false;
    }
    strcpy(new_symbol->value.string_val, value);
    new_symbol->next = table->buckets[index];
    table->buckets[index] = new_symbol;

    return true;
}

// Lookup symbol
```

```
Symbol* lookup_symbol(SymbolTable *table, const char *name) {
    if (table == NULL || name == NULL) return NULL;

    unsigned int index = symbol_hash(name);

    Symbol *current = table->buckets[index];
    while (current != NULL) {
        if (strcmp(current->name, name) == 0) {
            return current;
        }
        current = current->next;
    }

    return NULL;
}

// Print symbol table
void print_symbol_table(SymbolTable *table) {
    if (table == NULL) return;

    printf("Symbol Table:\n");
    printf("==============\n");

    for (int i = 0; i < SYMBOL_TABLE_SIZE; i++) {
        Symbol *current = table->buckets[i];
        while (current != NULL) {
            printf("Name: %-15s Type: ", current->name);

            switch (current->type) {
                case TYPE_INT:
                    printf("int     Value: %d\n", current->value.int_val);
                    break;
                case TYPE_FLOAT:
                    printf("float   Value: %.2f\n", current->value.float_val);
                    break;
                case TYPE_STRING:
                    printf("string  Value: %s\n", current->value.string_val);
                    break;
                case TYPE_FUNCTION:
                    printf("function\n");
                    break;
            }

            current = current->next;
        }
    }
}
```

```c
int main() {
    SymbolTable *table = create_symbol_table();
    if (table == NULL) {
        printf("Failed to create symbol table\n");
        return 1;
    }

    // Add various symbols
    add_integer_symbol(table, "count", 42);
    add_integer_symbol(table, "max_size", 1000);
    add_float_symbol(table, "pi", 3.14159f);
    add_float_symbol(table, "e", 2.71828f);
    add_string_symbol(table, "name", "John Doe");
    add_string_symbol(table, "version", "1.0.0");

    printf("Symbol table contents:\n");
    print_symbol_table(table);

    // Lookup symbols
    printf("\nSymbol lookups:\n");
    printf("===============\n");

    Symbol *symbol = lookup_symbol(table, "count");
    if (symbol != NULL) {
        printf("Found 'count': %d\n", symbol->value.int_val);
    }

    symbol = lookup_symbol(table, "pi");
    if (symbol != NULL) {
        printf("Found 'pi': %.5f\n", symbol->value.float_val);
    }

    symbol = lookup_symbol(table, "name");
    if (symbol != NULL) {
        printf("Found 'name': %s\n", symbol->value.string_val);
    }

    symbol = lookup_symbol(table, "nonexistent");
    if (symbol != NULL) {
        printf("Found 'nonexistent'\n");
    } else {
        printf("'nonexistent' not found\n");
    }

    free_symbol_table(table);
    return 0;
```

```
}
```

Cache Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <time.h>

#define CACHE_SIZE 16
#define MAX_KEY_LENGTH 100
#define MAX_VALUE_LENGTH 1000

typedef struct CacheNode {
    char key[MAX_KEY_LENGTH];
    char value[MAX_VALUE_LENGTH];
    time_t timestamp;
    int access_count;
    struct CacheNode *next;
} CacheNode;

typedef struct {
    CacheNode *buckets[CACHE_SIZE];
    int size;
    int max_size;
} LRUCache;

// Create LRU cache
LRUCache* create_lru_cache(int max_size) {
    LRUCache *cache = malloc(sizeof(LRUCache));
    if (cache == NULL) return NULL;

    for (int i = 0; i < CACHE_SIZE; i++) {
        cache->buckets[i] = NULL;
    }

    cache->size = 0;
    cache->max_size = max_size;
    return cache;
}

// Free LRU cache
void free_lru_cache(LRUCache *cache) {
    if (cache == NULL) return;
```

```
    for (int i = 0; i < CACHE_SIZE; i++) {
        CacheNode *current = cache->buckets[i];
        while (current != NULL) {
            CacheNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(cache);
}

// Hash function
unsigned int cache_hash(const char *key) {
    unsigned long hash = 5381;
    int c;

    while ((c = *key++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % CACHE_SIZE;
}

// Remove least recently used item
void remove_lru_item(LRUCache *cache) {
    if (cache == NULL || cache->size == 0) return;

    time_t oldest_time = time(NULL);
    CacheNode *oldest_node = NULL;
    int oldest_bucket = -1;
    CacheNode *oldest_prev = NULL;

    // Find the oldest accessed item
    for (int i = 0; i < CACHE_SIZE; i++) {
        CacheNode *current = cache->buckets[i];
        CacheNode *prev = NULL;

        while (current != NULL) {
            if (current->timestamp < oldest_time) {
                oldest_time = current->timestamp;
                oldest_node = current;
                oldest_bucket = i;
                oldest_prev = prev;
            }
            prev = current;
```

```
            current = current->next;
        }
    }

    // Remove the oldest item
    if (oldest_node != NULL) {
        if (oldest_prev == NULL) {
            cache->buckets[oldest_bucket] = oldest_node->next;
        } else {
            oldest_prev->next = oldest_node->next;
        }
        free(oldest_node);
        cache->size--;
    }
}

// Put item in cache
bool cache_put(LRUCache *cache, const char *key, const char *value) {
    if (cache == NULL || key == NULL || value == NULL) return false;

    // Check if key already exists
    unsigned int index = cache_hash(key);
    CacheNode *current = cache->buckets[index];
    CacheNode *prev = NULL;

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            // Update existing item
            strncpy(current->value, value, MAX_VALUE_LENGTH - 1);
            current->value[MAX_VALUE_LENGTH - 1] = '\0';
            current->timestamp = time(NULL);
            current->access_count++;
            return true;
        }
        prev = current;
        current = current->next;
    }

    // Check if cache is full
    if (cache->size >= cache->max_size) {
        remove_lru_item(cache);
    }

    // Create new cache node
    CacheNode *new_node = malloc(sizeof(CacheNode));
    if (new_node == NULL) return false;
```

```
    strncpy(new_node->key, key, MAX_KEY_LENGTH - 1);
    new_node->key[MAX_KEY_LENGTH - 1] = '\0';
    strncpy(new_node->value, value, MAX_VALUE_LENGTH - 1);
    new_node->value[MAX_VALUE_LENGTH - 1] = '\0';
    new_node->timestamp = time(NULL);
    new_node->access_count = 1;
    new_node->next = cache->buckets[index];
    cache->buckets[index] = new_node;
    cache->size++;

    return true;
}

// Get item from cache
bool cache_get(LRUCache *cache, const char *key, char *value, size_t value_size) {
    if (cache == NULL || key == NULL || value == NULL) return false;

    unsigned int index = cache_hash(key);
    CacheNode *current = cache->buckets[index];

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            // Found item, update timestamp and access count
            strncpy(value, current->value, value_size - 1);
            value[value_size - 1] = '\0';
            current->timestamp = time(NULL);
            current->access_count++;
            return true;
        }
        current = current->next;
    }

    return false;
}

// Print cache statistics
void print_cache_stats(LRUCache *cache) {
    if (cache == NULL) return;

    printf("Cache Statistics:\n");
    printf("  Size: %d/%d\n", cache->size, cache->max_size);
    printf("  Buckets:\n");

    for (int i = 0; i < CACHE_SIZE; i++) {
        CacheNode *current = cache->buckets[i];
        if (current != NULL) {
            printf("    Bucket %d:\n", i);
```

```
            while (current != NULL) {
                printf("      Key: %-15s Value: %-20s Accesses: %d\n",
                       current->key, current->value, current->access_count);
                current = current->next;
            }
        }
    }
}

int main() {
    LRUCache *cache = create_lru_cache(5);
    if (cache == NULL) {
        printf("Failed to create cache\n");
        return 1;
    }

    // Add items to cache
    cache_put(cache, "user1", "John Doe");
    cache_put(cache, "user2", "Jane Smith");
    cache_put(cache, "config", "debug=true");
    cache_put(cache, "version", "1.2.3");
    cache_put(cache, "theme", "dark");

    printf("Initial cache state:\n");
    print_cache_stats(cache);

    // Access some items
    char value[100];
    if (cache_get(cache, "user1", value, sizeof(value))) {
        printf("\nRetrieved 'user1': %s\n", value);
    }

    if (cache_get(cache, "config", value, sizeof(value))) {
        printf("Retrieved 'config': %s\n", value);
    }

    // Add more items to trigger LRU eviction
    cache_put(cache, "new_item1", "value1");
    cache_put(cache, "new_item2", "value2");

    printf("\nCache after adding new items:\n");
    print_cache_stats(cache);

    // Try to access evicted item
    if (cache_get(cache, "user2", value, sizeof(value))) {
        printf("\nRetrieved 'user2': %s\n", value);
    } else {
```

```
        printf("\n'user2' was evicted from cache\n");
    }

    free_lru_cache(cache);
    return 0;
}
```

Summary

Hash tables are powerful data structures that provide efficient key-value storage and retrieval:

1. **Hash Functions**: Critical for distributing keys uniformly across the table
2. **Collision Resolution**:
    - Chaining: Store collided elements in linked lists
    - Open Addressing: Probe for alternative slots in the table
3. **Dynamic Resizing**: Maintain performance by resizing when load factor is too high
4. **Performance Characteristics**:
    - Average case: O(1) for insert, delete, and search
    - Worst case: O(n) when all keys collide
    - Space complexity: O(n)

Key considerations when implementing hash tables: - **Hash Function Quality**: Should minimize collisions and distribute keys uniformly - **Load Factor Management**: Resize when load factor exceeds threshold - **Collision Resolution Strategy**: Choose based on use case and performance requirements - **Memory Management**: Properly handle dynamic allocation and deallocation

Hash tables are widely used in: - Database indexing - Caching systems - Symbol tables in compilers - Sets and maps in programming languages - Network routers and switches - Cryptographic applications

Understanding hash table implementation and optimization techniques is essential for building high-performance applications that require fast data access.

Roaring Bitmaps from Scratch: Array, Bitmap and Run Containers

Roaring Bitmaps from Scratch: Array, Bitmap and Run Containers

After this page you will understand how a Roaring bitmap is laid out, you will have built a small two-level version in C that switches a container from array to bitmap on its own, and you will be able to work out when an array, bitmap or run container is the smallest choice.

Why Compressed Bitmaps?

Bitmaps are fundamental data structures in database engines, search engines, and data analytics systems. In an inverted index, a bitmap can represent the set of document IDs containing a specific search term. In analytical engines, bitmaps speed up filters, set operations (intersection, union, difference) and counting.

A plain uncompressed bitmap maps each integer identifier directly to a bit position in an array of words. For a 32-bit unsigned integer space (0 to $2^{32}-1$), a traditional dense bitmap requires exactly 2^{32} bits = 512 MiB of contiguous memory. While this provides $O(1)$ set membership checks and fast bitwise CPU vectorization, it introduces massive memory overhead when representing sparse datasets. For instance, storing just 1,000 sparse 32-bit integers in a plain bitmap still consumes 512 MiB, whereas a simple array of 32-bit integers (`uint32_t`) takes only 4,000 bytes.

Conversely, sorted arrays of 32-bit integers are memory-efficient for sparse data (consuming 4 bytes per entry), but performing set membership or intersections requires binary search or multi-way merge operations, scaling with $O(\log N)$ or $O(N)$ time complexity instead of fast word-aligned bitwise arithmetic.

To solve this dilemma, compressed bitmap formats balance memory efficiency with high-performance operations. Early compressed bitmap designs such as WAH (Word-Aligned Hybrid) and EWAH (Enhanced Word-Aligned Hybrid) rely on Run-Length Encoding (RLE). While RLE compresses long sequences of identical bits effectively, it limits random access speed because indexing into an arbitrary position requires decoding previous compressed runs sequentially.

Roaring bitmaps, introduced by Chambi, Lemire, Kaser and Godin (2016) and described in more detail by Lemire et al. (2018), take a different approach. They split the 32-bit space into two levels and pick a container format for each chunk to suit how dense it is. Lucene, Druid, ClickHouse and Pilosa use them, and CRoaring is the reference C library.

The Two-Level Layout

Roaring splits every 32-bit integer x into two 16-bit components:

1. **High 16 bits (`x >> 16`)**: Serves as a chunk key (ranging from 0 to 65535).
2. **Low 16 bits (`x & 0xFFFF`)**: Stored inside a container corresponding to that chunk key.

Because the upper 16 bits partition the value space into $2^{16} = 65,536$ logical chunks, each individual container never holds more than $65,536$ distinct 16-bit integers (0 to $65,535$).

Empty chunks take no memory. The top level is a sorted array of the keys that are in use, each paired with its container.

Layer	Component	Range / Description	Memory Footprint
Top Level	Key Array / Directory	Sorted list of active high-16-bit chunk keys	Dynamic (2 bytes per key + pointer)
Container Level	Low-16 Storage	Holds low-16-bit values for a specific chunk key	Up to 8 KiB payload per chunk

So memory is only spent on parts of the 32-bit range that actually hold values.

Containers and the 4096 Rule

Inside each chunk, Roaring selects one of three primary container types depending on the density and pattern of the low-16-bit values:

1. **Array Container**: Stores the low 16 bits of each value as a sorted array of `uint16_t`. It is used when the container holds at most 4,096 values.
2. **Bitmap Container**: Stores presence or absence of values using a fixed 2^{16}-bit array ($65,536$ bits $= 8,192$ bytes $= 8$ KiB), represented as 1,024 64-bit words (`uint64_t`). It is used when a container holds more than 4,096 values.
3. **Run Container**: Stores continuous sequences (runs) of consecutive values as pairs of 16-bit integers: `(start_value, run_length)`. Each run takes $2 + 2 = 4$ bytes, plus 2 bytes for the run count. (The format stores length minus one, so a full chunk is the single run `(0, 65535)`.)

The Break-Even Calculation

The boundary between array containers and bitmap containers is strictly derived from memory efficiency:

- Each value in an **Array Container** consumes 2 bytes (16 bits).
- A **Bitmap Container** occupies a fixed payload of $8,192$ bytes ($65,536$ bits), regardless of how many values are present.

Setting the memory consumption of both structures equal determines the threshold:

$$\text{Cardinality} \times 2 \text{ bytes} = 8,192 \text{ bytes}$$

$$\text{Cardinality} = \frac{8,192}{2} = 4,096$$

When a container holds fewer than 4,096 values, an array container consumes less memory than an 8 KiB bitmap container (e.g., 1,000 values consume $2,000$ bytes). At 4,096 values, an array container consumes exactly $4,096 \times 2 = 8,192$ bytes, matching the fixed bitmap size. When cardinality exceeds 4,096, an array container would consume more than 8,192 bytes (e.g., 5,000 values would consume $10,000$ bytes), making the bitmap container strictly more compact.

Run Container Trade-off

Run containers are for consecutive values. Take the 10,000 integers $100,000$ to $109,999$. They all have key 1, and their low parts are 34,464 to 44,463. As an array that would be 20,000 bytes, more than the 4,096-value limit allows, so they go in an 8,192-byte bitmap. As one run, `(34464, 9999)`, they take 2 bytes (count) + 4 bytes (run) = 6 bytes.

Implementation Walk-through

The program below has array and bitmap containers. It turns an array container into a bitmap when the array is full at 4,096 values, finds containers by binary search on the keys, and reports what each container would cost as a run container. It does **not** store run containers or implement set operations; those are exercises at the end.

```
/* A minimal Roaring-style bitmap: 32-bit values split into a 16-bit key
 * (high bits, selects a container) and a 16-bit low part stored in either
 * an array container (sorted uint16_t, up to 4096 values) or a bitmap
 * container (65536 bits = 8 KiB). Run containers are measured, not stored. */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ARRAY_MAX 4096

typedef struct {
    uint16_t key;
    int is_bitmap;
    uint32_t card;      /* number of values in the container */
    uint16_t *array;    /* used when !is_bitmap, capacity ARRAY_MAX */
    uint64_t *bits;     /* used when is_bitmap, 1024 words */
} container;
```

```
typedef struct {
    container *c;
    size_t n, cap;
} roaring;

static container *find_or_add(roaring *r, uint16_t key) {
    size_t lo = 0, hi = r->n;
    while (lo < hi) { /* binary search on keys */
        size_t mid = (lo + hi) / 2;
        if (r->c[mid].key < key) lo = mid + 1; else hi = mid;
    }
    if (lo < r->n && r->c[lo].key == key)
        return &r->c[lo];
    if (r->n == r->cap) {
        r->cap = r->cap ? r->cap * 2 : 4;
        r->c = realloc(r->c, r->cap * sizeof *r->c);
        if (!r->c) { perror("realloc"); exit(1); }
    }
    memmove(&r->c[lo + 1], &r->c[lo], (r->n - lo) * sizeof *r->c);
    r->n++;
    container *c = &r->c[lo];
    memset(c, 0, sizeof *c);
    c->key = key;
    c->array = malloc(ARRAY_MAX * sizeof *c->array);
    if (!c->array) { perror("malloc"); exit(1); }
    return c;
}

static void to_bitmap(container *c) {
    c->bits = calloc(1024, sizeof *c->bits);
    if (!c->bits) { perror("calloc"); exit(1); }
    for (uint32_t i = 0; i < c->card; i++)
        c->bits[c->array[i] >> 6] |= UINT64_C(1) << (c->array[i] & 63);
    free(c->array);
    c->array = NULL;
    c->is_bitmap = 1;
}

static void roaring_add(roaring *r, uint32_t x) {
    container *c = find_or_add(r, (uint16_t)(x >> 16));
    uint16_t low = (uint16_t)x;
    if (c->is_bitmap) {
        uint64_t m = UINT64_C(1) << (low & 63);
        if (!(c->bits[low >> 6] & m)) { c->bits[low >> 6] |= m; c->card++; }
        return;
    }
```

```
    uint32_t lo = 0, hi = c->card; /* binary search in the sorted array */
    while (lo < hi) {
        uint32_t mid = (lo + hi) / 2;
        if (c->array[mid] < low) lo = mid + 1; else hi = mid;
    }
    if (lo < c->card && c->array[lo] == low)
        return;
    if (c->card == ARRAY_MAX) { /* array full: switch representation */
        to_bitmap(c);
        roaring_add(r, x);
        return;
    }
    memmove(&c->array[lo + 1], &c->array[lo], (c->card - lo) * sizeof *c->array);
    c->array[lo] = low;
    c->card++;
}

static int roaring_contains(const roaring *r, uint32_t x) {
    uint16_t key = (uint16_t)(x >> 16), low = (uint16_t)x;
    for (size_t i = 0; i < r->n; i++) {
        const container *c = &r->c[i];
        if (c->key != key) continue;
        if (c->is_bitmap) return (c->bits[low >> 6] >> (low & 63)) & 1;
        for (uint32_t j = 0; j < c->card; j++) if (c->array[j] == low) return 1;
        return 0;
    }
    return 0;
}

static int test_low(const container *c, uint32_t v) {
    if (c->is_bitmap) return (c->bits[v >> 6] >> (v & 63)) & 1;
    for (uint32_t j = 0; j < c->card; j++) if (c->array[j] == v) return 1;
    return 0;
}

/* Count runs of consecutive values: a run container needs 4 bytes per run. */
static uint32_t count_runs(const container *c) {
    uint32_t runs = 0;
    int prev = 0;
    for (uint32_t v = 0; v < 65536; v++) {
        int cur = test_low(c, v);
        if (cur && !prev) runs++;
        prev = cur;
    }
    return runs;
}
```

```
static void report(const char *name, const roaring *r) {
    uint64_t total = 0, bytes = 0, best = 0;
    for (size_t i = 0; i < r->n; i++) {
        const container *c = &r->c[i];
        uint32_t runs = count_runs(c);
        uint64_t arr = 2ull * c->card, bm = 8192, run = 2 + 4ull * runs;
        uint64_t now = c->is_bitmap ? bm : arr;
        uint64_t min = arr < bm ? arr : bm;
        if (run < min) min = run;
        total += c->card; bytes += now; best += min;
    }
    printf("%-14s values=%-8llu containers=%-3zu bytes=%-8llu with-runs=%-8llu plain-u32=%ll
           name, (unsigned long long)total, r->n, (unsigned long long)bytes,
           (unsigned long long)best, (unsigned long long)total * 4);
    for (size_t i = 0; i < r->n && i < 3; i++)
        printf("    key=%-5u %-6s card=%u\n", r->c[i].key,
               r->c[i].is_bitmap ? "bitmap" : "array", r->c[i].card);
}

static void roaring_free(roaring *r) {
    for (size_t i = 0; i < r->n; i++) { free(r->c[i].array); free(r->c[i].bits); }
    free(r->c);
}

int main(void) {
    roaring sparse = {0}, dense = {0}, ranges = {0};
    srand(42);
    for (int i = 0; i < 1000; i++) roaring_add(&sparse, (uint32_t)rand() * 7u);
    for (uint32_t i = 0; i < 200000; i++) if (rand() % 2) roaring_add(&dense, i);
    for (uint32_t i = 1000000; i < 1300000; i++) roaring_add(&ranges, i);

    report("sparse", &sparse);
    report("dense-random", &dense);
    report("one-range", &ranges);

    printf("contains(1000123)=%d contains(999999)=%d\n",
           roaring_contains(&ranges, 1000123), roaring_contains(&ranges, 999999));
    roaring_free(&sparse); roaring_free(&dense); roaring_free(&ranges);
    return 0;
}
```

Function Breakdowns

- `find_or_add`: Performs a binary search over the array of container keys (`r->c[mid].key`). If the high 16 bits match an existing key, it returns that container. If missing, it uses `memmove` to make room and inserts a new container in key order. To keep the code short, every new

array container is allocated at full capacity, `ARRAY_MAX` values (8 KiB), up front. A real library grows arrays as needed.

- `to_bitmap`: Upgrades an array container to a bitmap container. It allocates 1,024 64-bit words (8,192 bytes) zero-initialized via `calloc`, iterates over the existing sorted `array` entries, sets bit `(val & 63)` in word `(val >> 6)`, frees the array buffer, and sets `is_bitmap = 1`.
- `roaring_add`: Extracts high (`x >> 16`) and low (`x & 0xFFFF`) parts. If the container is already a bitmap, it sets the corresponding bit. If it is an array container, it uses binary search to find the insertion index for `low`. If the value exists, it returns early; if the array contains `ARRAY_MAX` elements, it calls `to_bitmap(c)` and then `roaring_add` again, which now takes the bitmap path.
- `roaring_contains`: kept deliberately simple, it scans the keys and the array linearly. Exercise 3 asks you to make it binary-search both.
- `count_runs`: Scans low-16 bit values from 0 to 65535 to count transitions from 0 to 1 (`cur && !prev`), calculating how many run segments `(start, length)` would be needed to store the container's contents.
- `report`: for each set, prints the total payload in its current containers (`bytes`), the payload if every container used its cheapest format, run containers included (`with-runs`), and a plain `uint32_t` array (`plain-u32`, 4 bytes per value). The counts are payload only: no key array, no container structs, no allocator overhead, and not the full-size array allocations mentioned above.

Compile it. With gcc 14.2 this gives no warnings, and an `-fsanitize=address,undefined` build prints the same output with no reports:

```
gcc -std=c17 -Wall -Wextra -O2 -o roaring roaring.c
```

Run it:

```
./roaring
```

```
sparse         values=1000     containers=995 bytes=2000     with-runs=2000     plain-u32=40
    key=30    array  card=1
    key=91    array  card=1
    key=109   array  card=1
dense-random   values=99704    containers=4   bytes=28006    with-runs=27970    plain-u32=39
    key=0     bitmap card=32552
    key=1     bitmap card=32790
    key=2     bitmap card=32647
one-range      values=300000   containers=5   bytes=40960    with-runs=30       plain-u32=12
    key=15    bitmap card=48576
    key=16    bitmap card=65536
    key=17    bitmap card=65536
contains(1000123)=1 contains(999999)=0
```

Reading the Results

Each set prints a summary line and its first three containers.

1. **Sparse Dataset (`sparse`)**:
    - Holds 1,000 random values spread across the full 32-bit range.
    - The values are scattered over the whole range, so almost every value gets its own container: 995 containers for 1,000 values.
    - Each is an array container with one value, 2 bytes of payload, so the payload total is 2,000 bytes against 4,000 for a plain `uint32_t` array.
    - *That comparison flatters Roaring.* The payload count leaves out the 995 key entries and container structs, which are far bigger than 2 bytes each (and this toy allocates 8 KiB per array on top). For data this sparse, per-container overhead dominates, and a plain sorted array is the smaller format.
2. **Dense Random Dataset (`dense-random`)**:
    - Holds 99,704 values, about half of `0` to `199,999`, chosen at random.
    - Keys 0, 1 and 2 cover full 65,536-value chunks with about 32,600 values each, far above 4,096, so they are bitmaps at 8,192 bytes each.
    - Key 3 only covers 196,608 to 199,999 and holds about 1,700 values, so it stays an array (about 3,400 bytes). That gives 28,006 bytes in total.
    - A plain array would need 398,816 bytes. Random data has few long runs, so run containers barely help here (27,970).
3. **Sequential Range Dataset (`one-range`)**:
    - Stores 300,000 consecutive integers spanning `1,000,000` to `1,299,999`.
    - The range touches keys 15 to 19: five containers, each with far more than 4,096 values, so five bitmaps, 40,960 bytes.
    - Within each container the values are one unbroken run (a full chunk is `(0, 65535)`; the two ends are partial runs), so as run containers they would need $2 + 4 = 6$ bytes each, 30 bytes in all.
    - The last line checks membership: 1,000,123 is in the range and 999,999 is not.

What Real Roaring Adds

CRoaring and the other production libraries add a lot on top of this.

Run Optimization Pass

Adding values one at a time doesn't turn containers into run containers. In CRoaring you ask for it explicitly (some operations, such as adding a whole range, can produce run containers directly):

```
roaring_bitmap_run_optimize(bitmap);
```

It checks each array and bitmap container and converts it to a run container where that is smaller.

Specialized Container-Pair Operations

When executing set intersections (`AND`), unions (`OR`), differences (`ANDNOT`), or symmetric differences (`XOR`), CRoaring dispatches optimized operation algorithms based on the type pair of the two target containers:

- **Array vs Array**: a merge of two sorted arrays, switching to galloping search when one array is much smaller than the other. CRoaring also has SIMD versions on CPUs that support them.
- **Bitmap vs Bitmap**: works 64 bits at a time (or wider with SIMD) and counts the result with popcount instructions, so the new cardinality comes almost for free.
- **Array vs Bitmap**: Iterates through the array elements, checking bit positions in the bitmap container directly.

Serialization Specification

Roaring has a shared serialization format, `RoaringFormatSpec`. A serialized bitmap starts with a cookie that also says whether there are run containers. Next comes each container's key and cardinality, then (in some cases) offsets, then the container data. Everything is little-endian, so the C, Java, Go and other implementations can read each other's output.

64-bit Variants

For 64-bit values you need another level over the high bits. CRoaring's C++ `Roaring64Map` maps the high 32 bits to an ordinary 32-bit Roaring bitmap, and its C `roaring64` API uses an adaptive radix tree instead.

Production Usage with CRoaring

For real use, take CRoaring (dual Apache-2.0 / MIT licence). It ships a single-file amalgamation, `roaring.c` plus `roaring.h`, that you can drop into a project.

> **Note**
>
> **Not run here.** CRoaring is not installed on the test machine. The snippet follows the API in CRoaring's README.

```c
#include <stdio.h>
#include "roaring.h" /* compile together with the amalgamated roaring.c */

int main(void) {
    /* Create a new bitmap */
    roaring_bitmap_t *r = roaring_bitmap_create();

    /* Add values */
```

```
    roaring_bitmap_add(r, 100);
    roaring_bitmap_add_range_closed(r, 100000, 200000);

    /* Optimize container representations */
    roaring_bitmap_run_optimize(r);

    /* Membership check & cardinality */
    if (roaring_bitmap_contains(r, 100050)) {
        printf("Cardinality: %llu\n",
               (unsigned long long)roaring_bitmap_get_cardinality(r));
    }

    /* Cleanup */
    roaring_bitmap_free(r);
    return 0;
}
```

Common Mistakes

1. Assuming Bitmaps Are Always Smaller Than Arrays

A bitmap container is 8,192 bytes however few bits are set. Placing 10 scattered elements in a bitmap container consumes 8,192 bytes, whereas an array container consumes 20 bytes. Switching to bitmap containers before reaching the 4,096 break-even threshold increases memory usage.

2. Ignoring Per-Container Overhead on Sparse Data

Summing payload sizes, as `report` does, understates the cost of sparse data. Each container also needs a key entry, its own struct (`key`, `card`, pointers) and allocator overhead. With one value per container, as in the `sparse` set, that overhead is many times the 2-byte payload.

3. Inserting Unsorted Data into Array Containers

Array containers must stay sorted: binary search and merge-based set operations depend on it. Appending a value at the end instead of inserting it in place breaks lookups and lets duplicates in. The same goes for the key array.

4. Integer Overflow during High/Low Bit Splitting

When extracting 16-bit keys and low values from a 32-bit integer, bit shifts and type casting must be handled carefully:

```
uint16_t high = (uint16_t)(x >> 16);
uint16_t low  = (uint16_t)(x & 0xFFFF);
```

Do the split on an unsigned 32-bit type. Right-shifting a negative `int` gives an implementation-defined result. Storing the low part in something wider than 16 bits without masking makes `low >> 6` index past the 1,024-word bitmap.

Exercises

1. **Implement run containers**: add a third container kind holding an array of `struct { uint16_t start; uint16_t length; }`, and a `run_optimize` pass that converts each container to whichever of the three formats is smallest. Check your sizes against the `with-runs` column.
2. **Implement Container Set Intersection (`AND`)**: Write a function `roaring_and(const roaring *r1, const roaring *r2)` that creates a new Roaring bitmap containing only elements present in both inputs. Implement array-array merging and bitmap-bitmap bitwise AND logic.
3. **Faster lookups**: make `roaring_contains` binary-search the keys and the array instead of scanning them.
4. **Right-size arrays**: allocate array containers small and grow them by doubling, then measure the real heap use of the `sparse` set (for example with `mallinfo2`) and compare it with the payload figure.

Summary

A Roaring bitmap splits each 32-bit value into a 16-bit key, which picks a container, and a 16-bit low part, which is stored in it. Arrays win up to 4,096 values (2 bytes each against a fixed 8,192-byte bitmap), bitmaps win above that, and run containers win for consecutive values. Every container stays small and fast to search, but for very sparse data the per-container overhead can cost more than a plain array.

- **Array Containers** store up to 4,096 sorted `uint16_t` values (2 bytes per entry).
- **Bitmap Containers** store dense allocations across 2^{16} bits using 1,024 64-bit words ($8,192$ bytes).
- **Run Containers** compress consecutive sequence ranges using `(start, length)` pairs (4 bytes per run).

Sources

- Roaring Bitmap Official Site
- Roaring Bitmaps: Implementation of an Optimized Software Library (Lemire et al., 2018)
- Better bitmap performance with Roaring bitmaps (Chambi et al., 2016)
- CRoaring GitHub Repository
- Roaring Format Specification

Graph Algorithms

Introduction

Graphs are fundamental data structures used to model relationships between objects. A graph consists of vertices (nodes) connected by edges, and can represent various real-world scenarios such as social networks, transportation networks, computer networks, and dependency relationships.

In this chapter, we'll explore graph representations, traversal algorithms, shortest path algorithms, and minimum spanning tree algorithms. We'll implement these algorithms in C and examine their applications and performance characteristics.

Graph Representations

Graphs can be represented in several ways, each with its own advantages and disadvantages.

Adjacency Matrix

An adjacency matrix is a 2D array where the cell at position (i, j) indicates whether there is an edge from vertex i to vertex j.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>

#define MAX_VERTICES 100

typedef struct {
    int vertices;
    int matrix[MAX_VERTICES][MAX_VERTICES];
} AdjMatrixGraph;

// Create adjacency matrix graph
AdjMatrixGraph* create_adj_matrix_graph(int vertices) {
    AdjMatrixGraph *graph = malloc(sizeof(AdjMatrixGraph));
    if (graph == NULL) return NULL;

    graph->vertices = vertices;
```

```
    // Initialize matrix with 0s (no edges)
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            graph->matrix[i][j] = 0;
        }
    }

    return graph;
}

// Add edge to adjacency matrix graph
void add_edge_matrix(AdjMatrixGraph *graph, int src, int dest, int weight) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }

    graph->matrix[src][dest] = weight;
    // For undirected graph, also add reverse edge
    graph->matrix[dest][src] = weight;
}

// Remove edge from adjacency matrix graph
void remove_edge_matrix(AdjMatrixGraph *graph, int src, int dest) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }

    graph->matrix[src][dest] = 0;
    graph->matrix[dest][src] = 0;
}

// Check if edge exists
bool has_edge_matrix(AdjMatrixGraph *graph, int src, int dest) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return false;
    }

    return graph->matrix[src][dest] != 0;
}

// Print adjacency matrix
void print_adj_matrix(AdjMatrixGraph *graph) {
    if (graph == NULL) return;
```

```
    printf("Adjacency Matrix:\n");
    printf("    ");
    for (int i = 0; i < graph->vertices; i++) {
        printf("%3d", i);
    }
    printf("\n");

    for (int i = 0; i < graph->vertices; i++) {
        printf("%3d ", i);
        for (int j = 0; j < graph->vertices; j++) {
            printf("%3d", graph->matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    AdjMatrixGraph *graph = create_adj_matrix_graph(5);
    if (graph == NULL) {
        printf("Failed to create graph\n");
        return 1;
    }

    // Add edges
    add_edge_matrix(graph, 0, 1, 1);
    add_edge_matrix(graph, 0, 4, 1);
    add_edge_matrix(graph, 1, 2, 1);
    add_edge_matrix(graph, 1, 3, 1);
    add_edge_matrix(graph, 1, 4, 1);
    add_edge_matrix(graph, 2, 3, 1);
    add_edge_matrix(graph, 3, 4, 1);

    print_adj_matrix(graph);

    printf("\nEdge checks:\n");
    printf("Edge (0,1): %s\n", has_edge_matrix(graph, 0, 1) ? "Yes" : "No");
    printf("Edge (0,2): %s\n", has_edge_matrix(graph, 0, 2) ? "Yes" : "No");

    free(graph);
    return 0;
}
```

Adjacency List

An adjacency list represents a graph as an array of lists, where each list contains the neighbors of a vertex.

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_VERTICES 100

typedef struct AdjListNode {
    int vertex;
    int weight;
    struct AdjListNode *next;
} AdjListNode;

typedef struct {
    int vertices;
    AdjListNode *adj_list[MAX_VERTICES];
} AdjListGraph;

// Create new adjacency list node
AdjListNode* create_adj_list_node(int vertex, int weight) {
    AdjListNode *node = malloc(sizeof(AdjListNode));
    if (node == NULL) return NULL;

    node->vertex = vertex;
    node->weight = weight;
    node->next = NULL;
    return node;
}

// Create adjacency list graph
AdjListGraph* create_adj_list_graph(int vertices) {
    AdjListGraph *graph = malloc(sizeof(AdjListGraph));
    if (graph == NULL) return NULL;

    graph->vertices = vertices;

    // Initialize adjacency lists
    for (int i = 0; i < vertices; i++) {
        graph->adj_list[i] = NULL;
    }

    return graph;
}

// Free adjacency list graph
void free_adj_list_graph(AdjListGraph *graph) {
    if (graph == NULL) return;
```

```
    for (int i = 0; i < graph->vertices; i++) {
        AdjListNode *current = graph->adj_list[i];
        while (current != NULL) {
            AdjListNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(graph);
}

// Add edge to adjacency list graph
void add_edge_list(AdjListGraph *graph, int src, int dest, int weight) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }

    // Add edge from src to dest
    AdjListNode *new_node = create_adj_list_node(dest, weight);
    new_node->next = graph->adj_list[src];
    graph->adj_list[src] = new_node;

    // For undirected graph, also add reverse edge
    new_node = create_adj_list_node(src, weight);
    new_node->next = graph->adj_list[dest];
    graph->adj_list[dest] = new_node;
}

// Remove edge from adjacency list graph
void remove_edge_list(AdjListGraph *graph, int src, int dest) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }

    // Remove edge from src to dest
    AdjListNode *current = graph->adj_list[src];
    AdjListNode *prev = NULL;

    while (current != NULL) {
        if (current->vertex == dest) {
            if (prev == NULL) {
                graph->adj_list[src] = current->next;
            } else {
                prev->next = current->next;
```

```
            }
            free(current);
            break;
        }
        prev = current;
        current = current->next;
    }

    // Remove edge from dest to src
    current = graph->adj_list[dest];
    prev = NULL;

    while (current != NULL) {
        if (current->vertex == src) {
            if (prev == NULL) {
                graph->adj_list[dest] = current->next;
            } else {
                prev->next = current->next;
            }
            free(current);
            break;
        }
        prev = current;
        current = current->next;
    }
}

// Print adjacency list
void print_adj_list(AdjListGraph *graph) {
    if (graph == NULL) return;

    printf("Adjacency List:\n");
    for (int i = 0; i < graph->vertices; i++) {
        printf("%d: ", i);
        AdjListNode *current = graph->adj_list[i];
        while (current != NULL) {
            printf("(%d, %d) -> ", current->vertex, current->weight);
            current = current->next;
        }
        printf("NULL\n");
    }
}

int main() {
    AdjListGraph *graph = create_adj_list_graph(5);
    if (graph == NULL) {
```

```
        printf("Failed to create graph\n");
        return 1;
    }

    // Add edges
    add_edge_list(graph, 0, 1, 1);
    add_edge_list(graph, 0, 4, 1);
    add_edge_list(graph, 1, 2, 1);
    add_edge_list(graph, 1, 3, 1);
    add_edge_list(graph, 1, 4, 1);
    add_edge_list(graph, 2, 3, 1);
    add_edge_list(graph, 3, 4, 1);

    print_adj_list(graph);

    free_adj_list_graph(graph);
    return 0;
}
```

Graph Traversal Algorithms

Breadth-First Search (BFS)

BFS explores vertices level by level, visiting all neighbors of a vertex before moving to the next level.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_VERTICES 100

typedef struct Queue {
    int items[MAX_VERTICES];
    int front;
    int rear;
} Queue;

// Initialize queue
Queue* create_queue(void) {
    Queue *queue = malloc(sizeof(Queue));
    if (queue == NULL) return NULL;

    queue->front = 0;
    queue->rear = -1;
```

```
    return queue;
}

// Check if queue is empty
bool is_queue_empty(Queue *queue) {
    return queue->rear < queue->front;
}

// Enqueue
void enqueue(Queue *queue, int item) {
    if (queue->rear < MAX_VERTICES - 1) {
        queue->items[++queue->rear] = item;
    }
}

// Dequeue
int dequeue(Queue *queue) {
    if (is_queue_empty(queue)) {
        return -1;
    }
    return queue->items[queue->front++];
}

typedef struct BFSGraph {
    int vertices;
    struct AdjListNode **adj_list;
} BFSGraph;

typedef struct AdjListNode {
    int vertex;
    struct AdjListNode *next;
} AdjListNode;

// Create new adjacency list node
AdjListNode* create_bfs_node(int vertex) {
    AdjListNode *node = malloc(sizeof(AdjListNode));
    if (node == NULL) return NULL;

    node->vertex = vertex;
    node->next = NULL;
    return node;
}

// Create BFS graph
BFSGraph* create_bfs_graph(int vertices) {
    BFSGraph *graph = malloc(sizeof(BFSGraph));
    if (graph == NULL) return NULL;
```

```
    graph->vertices = vertices;
    graph->adj_list = malloc(vertices * sizeof(AdjListNode*));
    if (graph->adj_list == NULL) {
        free(graph);
        return NULL;
    }

    for (int i = 0; i < vertices; i++) {
        graph->adj_list[i] = NULL;
    }

    return graph;
}

// Free BFS graph
void free_bfs_graph(BFSGraph *graph) {
    if (graph == NULL) return;

    for (int i = 0; i < graph->vertices; i++) {
        AdjListNode *current = graph->adj_list[i];
        while (current != NULL) {
            AdjListNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(graph->adj_list);
    free(graph);
}

// Add edge to BFS graph
void add_bfs_edge(BFSGraph *graph, int src, int dest) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }

    // Add edge from src to dest
    AdjListNode *new_node = create_bfs_node(dest);
    new_node->next = graph->adj_list[src];
    graph->adj_list[src] = new_node;

    // For undirected graph, also add reverse edge
    new_node = create_bfs_node(src);
    new_node->next = graph->adj_list[dest];
```

```
    graph->adj_list[dest] = new_node;
}

// BFS traversal
void bfs(BFSGraph *graph, int start_vertex) {
    if (graph == NULL || start_vertex < 0 || start_vertex >= graph->vertices) {
        return;
    }

    bool *visited = calloc(graph->vertices, sizeof(bool));
    if (visited == NULL) return;

    Queue *queue = create_queue();
    if (queue == NULL) {
        free(visited);
        return;
    }

    visited[start_vertex] = true;
    enqueue(queue, start_vertex);

    printf("BFS Traversal starting from vertex %d: ", start_vertex);

    while (!is_queue_empty(queue)) {
        int current_vertex = dequeue(queue);
        printf("%d ", current_vertex);

        AdjListNode *current = graph->adj_list[current_vertex];
        while (current != NULL) {
            int adj_vertex = current->vertex;
            if (!visited[adj_vertex]) {
                visited[adj_vertex] = true;
                enqueue(queue, adj_vertex);
            }
            current = current->next;
        }
    }

    printf("\n");

    free(visited);
    free(queue);
}

int main() {
    BFSGraph *graph = create_bfs_graph(6);
```

```
    if (graph == NULL) {
        printf("Failed to create graph\n");
        return 1;
    }

    // Add edges to create a sample graph
    add_bfs_edge(graph, 0, 1);
    add_bfs_edge(graph, 0, 2);
    add_bfs_edge(graph, 1, 3);
    add_bfs_edge(graph, 1, 4);
    add_bfs_edge(graph, 2, 4);
    add_bfs_edge(graph, 3, 5);
    add_bfs_edge(graph, 4, 5);

    bfs(graph, 0);
    bfs(graph, 2);

    free_bfs_graph(graph);
    return 0;
}
```

Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_VERTICES 100

typedef struct DFSGraph {
    int vertices;
    struct AdjListNode **adj_list;
} DFSGraph;

typedef struct AdjListNode {
    int vertex;
    struct AdjListNode *next;
} AdjListNode;

// Create new adjacency list node
AdjListNode* create_dfs_node(int vertex) {
    AdjListNode *node = malloc(sizeof(AdjListNode));
    if (node == NULL) return NULL;
```

```
    node->vertex = vertex;
    node->next = NULL;
    return node;
}

// Create DFS graph
DFSGraph* create_dfs_graph(int vertices) {
    DFSGraph *graph = malloc(sizeof(DFSGraph));
    if (graph == NULL) return NULL;

    graph->vertices = vertices;
    graph->adj_list = malloc(vertices * sizeof(AdjListNode*));
    if (graph->adj_list == NULL) {
        free(graph);
        return NULL;
    }

    for (int i = 0; i < vertices; i++) {
        graph->adj_list[i] = NULL;
    }

    return graph;
}

// Free DFS graph
void free_dfs_graph(DFSGraph *graph) {
    if (graph == NULL) return;

    for (int i = 0; i < graph->vertices; i++) {
        AdjListNode *current = graph->adj_list[i];
        while (current != NULL) {
            AdjListNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(graph->adj_list);
    free(graph);
}

// Add edge to DFS graph
void add_dfs_edge(DFSGraph *graph, int src, int dest) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }
```

```
    // Add edge from src to dest
    AdjListNode *new_node = create_dfs_node(dest);
    new_node->next = graph->adj_list[src];
    graph->adj_list[src] = new_node;

    // For undirected graph, also add reverse edge
    new_node = create_dfs_node(src);
    new_node->next = graph->adj_list[dest];
    graph->adj_list[dest] = new_node;
}

// Recursive DFS helper function
void dfs_recursive(DFSGraph *graph, int vertex, bool *visited) {
    visited[vertex] = true;
    printf("%d ", vertex);

    AdjListNode *current = graph->adj_list[vertex];
    while (current != NULL) {
        int adj_vertex = current->vertex;
        if (!visited[adj_vertex]) {
            dfs_recursive(graph, adj_vertex, visited);
        }
        current = current->next;
    }
}

// DFS traversal
void dfs(DFSGraph *graph, int start_vertex) {
    if (graph == NULL || start_vertex < 0 || start_vertex >= graph->vertices) {
        return;
    }

    bool *visited = calloc(graph->vertices, sizeof(bool));
    if (visited == NULL) return;

    printf("DFS Traversal starting from vertex %d: ", start_vertex);
    dfs_recursive(graph, start_vertex, visited);
    printf("\n");

    free(visited);
}

// Iterative DFS using stack
void dfs_iterative(DFSGraph *graph, int start_vertex) {
    if (graph == NULL || start_vertex < 0 || start_vertex >= graph->vertices) {
        return;
```

```
    }

    bool *visited = calloc(graph->vertices, sizeof(bool));
    if (visited == NULL) return;

    int stack[MAX_VERTICES];
    int top = -1;

    stack[++top] = start_vertex;

    printf("DFS Iterative Traversal starting from vertex %d: ", start_vertex);

    while (top >= 0) {
        int current_vertex = stack[top--];

        if (!visited[current_vertex]) {
            visited[current_vertex] = true;
            printf("%d ", current_vertex);

            // Push all adjacent vertices to stack
            AdjListNode *current = graph->adj_list[current_vertex];
            while (current != NULL) {
                int adj_vertex = current->vertex;
                if (!visited[adj_vertex]) {
                    stack[++top] = adj_vertex;
                }
                current = current->next;
            }
        }
    }

    printf("\n");

    free(visited);
}

int main() {
    DFSGraph *graph = create_dfs_graph(6);
    if (graph == NULL) {
        printf("Failed to create graph\n");
        return 1;
    }

    // Add edges to create a sample graph
    add_dfs_edge(graph, 0, 1);
    add_dfs_edge(graph, 0, 2);
```

```
    add_dfs_edge(graph, 1, 3);
    add_dfs_edge(graph, 1, 4);
    add_dfs_edge(graph, 2, 4);
    add_dfs_edge(graph, 3, 5);
    add_dfs_edge(graph, 4, 5);

    dfs(graph, 0);
    dfs_iterative(graph, 0);

    free_dfs_graph(graph);
    return 0;
}
```

Shortest Path Algorithms

Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>

#define MAX_VERTICES 100

typedef struct DijkstraGraph {
    int vertices;
    struct DijkstraNode **adj_list;
} DijkstraGraph;

typedef struct DijkstraNode {
    int vertex;
    int weight;
    struct DijkstraNode *next;
} DijkstraNode;

// Create new adjacency list node
DijkstraNode* create_dijkstra_node(int vertex, int weight) {
    DijkstraNode *node = malloc(sizeof(DijkstraNode));
    if (node == NULL) return NULL;

    node->vertex = vertex;
    node->weight = weight;
```

```
    node->next = NULL;
    return node;
}

// Create Dijkstra graph
DijkstraGraph* create_dijkstra_graph(int vertices) {
    DijkstraGraph *graph = malloc(sizeof(DijkstraGraph));
    if (graph == NULL) return NULL;

    graph->vertices = vertices;
    graph->adj_list = malloc(vertices * sizeof(DijkstraNode*));
    if (graph->adj_list == NULL) {
        free(graph);
        return NULL;
    }

    for (int i = 0; i < vertices; i++) {
        graph->adj_list[i] = NULL;
    }

    return graph;
}

// Free Dijkstra graph
void free_dijkstra_graph(DijkstraGraph *graph) {
    if (graph == NULL) return;

    for (int i = 0; i < graph->vertices; i++) {
        DijkstraNode *current = graph->adj_list[i];
        while (current != NULL) {
            DijkstraNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(graph->adj_list);
    free(graph);
}

// Add edge to Dijkstra graph
void add_dijkstra_edge(DijkstraGraph *graph, int src, int dest, int weight) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
        return;
    }
```

```
    // Add edge from src to dest
    DijkstraNode *new_node = create_dijkstra_node(dest, weight);
    new_node->next = graph->adj_list[src];
    graph->adj_list[src] = new_node;

    // For undirected graph, also add reverse edge
    new_node = create_dijkstra_node(src, weight);
    new_node->next = graph->adj_list[dest];
    graph->adj_list[dest] = new_node;
}

// Min heap implementation for Dijkstra's algorithm
typedef struct {
    int *vertices;
    int *distances;
    int size;
    int capacity;
    int *pos;  // Position of vertices in heap
} MinHeap;

// Create min heap
MinHeap* create_min_heap(int capacity) {
    MinHeap *heap = malloc(sizeof(MinHeap));
    if (heap == NULL) return NULL;

    heap->capacity = capacity;
    heap->size = 0;
    heap->vertices = malloc(capacity * sizeof(int));
    heap->distances = malloc(capacity * sizeof(int));
    heap->pos = malloc(capacity * sizeof(int));

    if (heap->vertices == NULL || heap->distances == NULL || heap->pos == NULL) {
        free(heap->vertices);
        free(heap->distances);
        free(heap->pos);
        free(heap);
        return NULL;
    }

    return heap;
}

// Free min heap
void free_min_heap(MinHeap *heap) {
    if (heap == NULL) return;

    free(heap->vertices);
```

```
    free(heap->distances);
    free(heap->pos);
    free(heap);
}

// Check if heap is empty
bool is_heap_empty(MinHeap *heap) {
    return heap->size == 0;
}

// Min heapify operation
void min_heapify(MinHeap *heap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < heap->size && heap->distances[left] < heap->distances[smallest]) {
        smallest = left;
    }

    if (right < heap->size && heap->distances[right] < heap->distances[smallest]) {
        smallest = right;
    }

    if (smallest != idx) {
        // Swap positions
        int temp_pos = heap->pos[heap->vertices[smallest]];
        heap->pos[heap->vertices[smallest]] = heap->pos[heap->vertices[idx]];
        heap->pos[heap->vertices[idx]] = temp_pos;

        // Swap vertices
        int temp_vertex = heap->vertices[smallest];
        heap->vertices[smallest] = heap->vertices[idx];
        heap->vertices[idx] = temp_vertex;

        // Swap distances
        int temp_dist = heap->distances[smallest];
        heap->distances[smallest] = heap->distances[idx];
        heap->distances[idx] = temp_dist;

        min_heapify(heap, smallest);
    }
}

// Extract minimum distance vertex
int extract_min(MinHeap *heap) {
```

```
    if (is_heap_empty(heap)) return -1;

    int root_vertex = heap->vertices[0];

    // Replace root with last element
    int last_vertex = heap->vertices[heap->size - 1];
    heap->vertices[0] = last_vertex;

    // Update position of last vertex
    heap->pos[root_vertex] = heap->size - 1;
    heap->pos[last_vertex] = 0;

    // Reduce heap size and heapify root
    heap->size--;
    min_heapify(heap, 0);

    return root_vertex;
}

// Decrease key operation
void decrease_key(MinHeap *heap, int vertex, int new_dist) {
    int idx = heap->pos[vertex];
    heap->distances[idx] = new_dist;

    // Travel up while heap property is violated
    while (idx && heap->distances[idx] < heap->distances[(idx - 1) / 2]) {
        // Swap positions
        int temp_pos = heap->pos[heap->vertices[(idx - 1) / 2]];
        heap->pos[heap->vertices[(idx - 1) / 2]] = heap->pos[heap->vertices[idx]];
        heap->pos[heap->vertices[idx]] = temp_pos;

        // Swap vertices
        int temp_vertex = heap->vertices[(idx - 1) / 2];
        heap->vertices[(idx - 1) / 2] = heap->vertices[idx];
        heap->vertices[idx] = temp_vertex;

        // Swap distances
        int temp_dist = heap->distances[(idx - 1) / 2];
        heap->distances[(idx - 1) / 2] = heap->distances[idx];
        heap->distances[idx] = temp_dist;

        idx = (idx - 1) / 2;
    }
}

// Check if vertex is in min heap
bool is_in_min_heap(MinHeap *heap, int vertex) {
```

```
    return (heap->pos[vertex] < heap->size);
}

// Dijkstra's algorithm
void dijkstra(DijkstraGraph *graph, int src) {
    if (graph == NULL || src < 0 || src >= graph->vertices) {
        return;
    }

    int *distances = malloc(graph->vertices * sizeof(int));
    int *parent = malloc(graph->vertices * sizeof(int));
    bool *visited = calloc(graph->vertices, sizeof(bool));

    if (distances == NULL || parent == NULL || visited == NULL) {
        free(distances);
        free(parent);
        free(visited);
        return;
    }

    // Initialize distances and parent array
    for (int i = 0; i < graph->vertices; i++) {
        distances[i] = INT_MAX;
        parent[i] = -1;
    }

    distances[src] = 0;

    // Create min heap
    MinHeap *min_heap = create_min_heap(graph->vertices);
    if (min_heap == NULL) {
        free(distances);
        free(parent);
        free(visited);
        return;
    }

    // Initialize min heap with all vertices
    for (int i = 0; i < graph->vertices; i++) {
        min_heap->vertices[i] = i;
        min_heap->distances[i] = distances[i];
        min_heap->pos[i] = i;
    }

    min_heap->size = graph->vertices;
```

```
    // Main loop of Dijkstra's algorithm
    while (!is_heap_empty(min_heap)) {
        // Extract minimum distance vertex
        int u = extract_min(min_heap);
        visited[u] = true;

        // Update distance values of adjacent vertices
        DijkstraNode *current = graph->adj_list[u];
        while (current != NULL) {
            int v = current->vertex;

            // If shortest distance to v is not finalized yet,
            // and distance to v through u is less than its current distance
            if (!visited[v] && distances[u] != INT_MAX &&
                distances[u] + current->weight < distances[v]) {
                distances[v] = distances[u] + current->weight;
                parent[v] = u;

                // Update distance value in min heap
                decrease_key(min_heap, v, distances[v]);
            }

            current = current->next;
        }
    }

    // Print shortest distances
    printf("Shortest distances from vertex %d:\n", src);
    for (int i = 0; i < graph->vertices; i++) {
        if (distances[i] == INT_MAX) {
            printf("Vertex %d: INF\n", i);
        } else {
            printf("Vertex %d: %d\n", i, distances[i]);
        }
    }

    free(distances);
    free(parent);
    free(visited);
    free_min_heap(min_heap);
}

int main() {
    DijkstraGraph *graph = create_dijkstra_graph(6);
    if (graph == NULL) {
        printf("Failed to create graph\n");
        return 1;
```

```
    }

    // Add weighted edges
    add_dijkstra_edge(graph, 0, 1, 4);
    add_dijkstra_edge(graph, 0, 2, 3);
    add_dijkstra_edge(graph, 1, 2, 1);
    add_dijkstra_edge(graph, 1, 3, 2);
    add_dijkstra_edge(graph, 2, 3, 4);
    add_dijkstra_edge(graph, 3, 4, 2);
    add_dijkstra_edge(graph, 4, 5, 6);

    dijkstra(graph, 0);

    free_dijkstra_graph(graph);
    return 0;
}
```

Minimum Spanning Tree Algorithms

Prim's Algorithm

Prim's algorithm finds a minimum spanning tree for a weighted undirected graph.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>

#define MAX_VERTICES 100

typedef struct PrimGraph {
    int vertices;
    struct PrimNode **adj_list;
} PrimGraph;

typedef struct PrimNode {
    int vertex;
    int weight;
    struct PrimNode *next;
} PrimNode;

// Create new adjacency list node
PrimNode* create_prim_node(int vertex, int weight) {
    PrimNode *node = malloc(sizeof(PrimNode));
    if (node == NULL) return NULL;
```

```
    node->vertex = vertex;
    node->weight = weight;
    node->next = NULL;
    return node;
}

// Create Prim graph
PrimGraph* create_prim_graph(int vertices) {
    PrimGraph *graph = malloc(sizeof(PrimGraph));
    if (graph == NULL) return NULL;

    graph->vertices = vertices;
    graph->adj_list = malloc(vertices * sizeof(PrimNode*));
    if (graph->adj_list == NULL) {
        free(graph);
        return NULL;
    }

    for (int i = 0; i < vertices; i++) {
        graph->adj_list[i] = NULL;
    }

    return graph;
}

// Free Prim graph
void free_prim_graph(PrimGraph *graph) {
    if (graph == NULL) return;

    for (int i = 0; i < graph->vertices; i++) {
        PrimNode *current = graph->adj_list[i];
        while (current != NULL) {
            PrimNode *next = current->next;
            free(current);
            current = next;
        }
    }

    free(graph->adj_list);
    free(graph);
}

// Add edge to Prim graph
void add_prim_edge(PrimGraph *graph, int src, int dest, int weight) {
    if (graph == NULL || src < 0 || src >= graph->vertices ||
        dest < 0 || dest >= graph->vertices) {
```

```
        return;
    }

    // Add edge from src to dest
    PrimNode *new_node = create_prim_node(dest, weight);
    new_node->next = graph->adj_list[src];
    graph->adj_list[src] = new_node;

    // For undirected graph, also add reverse edge
    new_node = create_prim_node(src, weight);
    new_node->next = graph->adj_list[dest];
    graph->adj_list[dest] = new_node;
}

// Min heap implementation for Prim's algorithm
typedef struct {
    int *vertices;
    int *keys;
    int *parent;
    int size;
    int capacity;
    int *pos;  // Position of vertices in heap
} PrimHeap;

// Create min heap for Prim's algorithm
PrimHeap* create_prim_heap(int capacity) {
    PrimHeap *heap = malloc(sizeof(PrimHeap));
    if (heap == NULL) return NULL;

    heap->capacity = capacity;
    heap->size = 0;
    heap->vertices = malloc(capacity * sizeof(int));
    heap->keys = malloc(capacity * sizeof(int));
    heap->parent = malloc(capacity * sizeof(int));
    heap->pos = malloc(capacity * sizeof(int));

    if (heap->vertices == NULL || heap->keys == NULL ||
        heap->parent == NULL || heap->pos == NULL) {
        free(heap->vertices);
        free(heap->keys);
        free(heap->parent);
        free(heap->pos);
        free(heap);
        return NULL;
    }

    return heap;
```

```
}

// Free Prim heap
void free_prim_heap(PrimHeap *heap) {
    if (heap == NULL) return;

    free(heap->vertices);
    free(heap->keys);
    free(heap->parent);
    free(heap->pos);
    free(heap);
}

// Check if heap is empty
bool is_prim_heap_empty(PrimHeap *heap) {
    return heap->size == 0;
}

// Min heapify operation
void prim_heapify(PrimHeap *heap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < heap->size && heap->keys[left] < heap->keys[smallest]) {
        smallest = left;
    }

    if (right < heap->size && heap->keys[right] < heap->keys[smallest]) {
        smallest = right;
    }

    if (smallest != idx) {
        // Swap positions
        int temp_pos = heap->pos[heap->vertices[smallest]];
        heap->pos[heap->vertices[smallest]] = heap->pos[heap->vertices[idx]];
        heap->pos[heap->vertices[idx]] = temp_pos;

        // Swap vertices
        int temp_vertex = heap->vertices[smallest];
        heap->vertices[smallest] = heap->vertices[idx];
        heap->vertices[idx] = temp_vertex;

        // Swap keys
        int temp_key = heap->keys[smallest];
        heap->keys[smallest] = heap->keys[idx];
```

```
        heap->keys[idx] = temp_key;

        prim_heapify(heap, smallest);
    }
}

// Extract minimum key vertex
int extract_min_prim(PrimHeap *heap) {
    if (is_prim_heap_empty(heap)) return -1;

    int root_vertex = heap->vertices[0];

    // Replace root with last element
    int last_vertex = heap->vertices[heap->size - 1];
    heap->vertices[0] = last_vertex;

    // Update position of last vertex
    heap->pos[root_vertex] = heap->size - 1;
    heap->pos[last_vertex] = 0;

    // Reduce heap size and heapify root
    heap->size--;
    prim_heapify(heap, 0);

    return root_vertex;
}

// Decrease key operation
void decrease_key_prim(PrimHeap *heap, int vertex, int new_key) {
    int idx = heap->pos[vertex];
    heap->keys[idx] = new_key;

    // Travel up while heap property is violated
    while (idx && heap->keys[idx] < heap->keys[(idx - 1) / 2]) {
        // Swap positions
        int temp_pos = heap->pos[heap->vertices[(idx - 1) / 2]];
        heap->pos[heap->vertices[(idx - 1) / 2]] = heap->pos[heap->vertices[idx]];
        heap->pos[heap->vertices[idx]] = temp_pos;

        // Swap vertices
        int temp_vertex = heap->vertices[(idx - 1) / 2];
        heap->vertices[(idx - 1) / 2] = heap->vertices[idx];
        heap->vertices[idx] = temp_vertex;

        // Swap keys
        int temp_key = heap->keys[(idx - 1) / 2];
```

```
        heap->keys[(idx - 1) / 2] = heap->keys[idx];
        heap->keys[idx] = temp_key;

        idx = (idx - 1) / 2;
    }
}

// Prim's algorithm for Minimum Spanning Tree
void prim_mst(PrimGraph *graph) {
    if (graph == NULL) return;

    int *parent = malloc(graph->vertices * sizeof(int));
    int *key = malloc(graph->vertices * sizeof(int));
    bool *in_mst = calloc(graph->vertices, sizeof(bool));

    if (parent == NULL || key == NULL || in_mst == NULL) {
        free(parent);
        free(key);
        free(in_mst);
        return;
    }

    // Create min heap
    PrimHeap *min_heap = create_prim_heap(graph->vertices);
    if (min_heap == NULL) {
        free(parent);
        free(key);
        free(in_mst);
        return;
    }

    // Initialize min heap with all vertices
    for (int i = 0; i < graph->vertices; i++) {
        parent[i] = -1;
        key[i] = INT_MAX;
        min_heap->vertices[i] = i;
        min_heap->keys[i] = key[i];
        min_heap->parent[i] = parent[i];
        min_heap->pos[i] = i;
    }

    // Make key value of 0th vertex as 0
    min_heap->keys[0] = 0;
    key[0] = 0;
    min_heap->parent[0] = -1;
```

```
    min_heap->size = graph->vertices;

    // Main loop of Prim's algorithm
    while (!is_prim_heap_empty(min_heap)) {
        // Extract minimum key vertex
        int u = extract_min_prim(min_heap);
        in_mst[u] = true;

        // Update key values and parent indices of adjacent vertices
        PrimNode *current = graph->adj_list[u];
        while (current != NULL) {
            int v = current->vertex;

            // If v is not in MST and weight of (u,v) is smaller than current key of v
            if (!in_mst[v] && current->weight < key[v]) {
                key[v] = current->weight;
                parent[v] = u;
                decrease_key_prim(min_heap, v, key[v]);
            }

            current = current->next;
        }
    }

    // Print MST edges
    printf("Minimum Spanning Tree Edges:\n");
    int total_weight = 0;
    for (int i = 1; i < graph->vertices; i++) {
        printf("%d - %d: %d\n", parent[i], i, key[i]);
        total_weight += key[i];
    }
    printf("Total weight: %d\n", total_weight);

    free(parent);
    free(key);
    free(in_mst);
    free_prim_heap(min_heap);
}

int main() {
    PrimGraph *graph = create_prim_graph(5);
    if (graph == NULL) {
        printf("Failed to create graph\n");
        return 1;
    }

    // Add weighted edges to create a sample graph
```

```
    add_prim_edge(graph, 0, 1, 2);
    add_prim_edge(graph, 0, 3, 6);
    add_prim_edge(graph, 1, 2, 3);
    add_prim_edge(graph, 1, 3, 8);
    add_prim_edge(graph, 1, 4, 5);
    add_prim_edge(graph, 2, 4, 7);
    add_prim_edge(graph, 3, 4, 9);

    prim_mst(graph);

    free_prim_graph(graph);
    return 0;
}
```

Practical Examples

Social Network Analysis

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define MAX_USERS 100
#define MAX_NAME_LENGTH 50

typedef struct User {
    int id;
    char name[MAX_NAME_LENGTH];
    struct User **friends;
    int friend_count;
    int max_friends;
} User;

typedef struct SocialNetwork {
    User *users[MAX_USERS];
    int user_count;
} SocialNetwork;

// Create new user
User* create_user(int id, const char *name) {
    User *user = malloc(sizeof(User));
    if (user == NULL) return NULL;
```

```
    user->id = id;
    strncpy(user->name, name, MAX_NAME_LENGTH - 1);
    user->name[MAX_NAME_LENGTH - 1] = '\0';
    user->friend_count = 0;
    user->max_friends = 10;
    user->friends = malloc(user->max_friends * sizeof(User*));
    if (user->friends == NULL) {
        free(user);
        return NULL;
    }

    return user;
}

// Free user
void free_user(User *user) {
    if (user == NULL) return;
    free(user->friends);
    free(user);
}

// Create social network
SocialNetwork* create_social_network(void) {
    SocialNetwork *network = malloc(sizeof(SocialNetwork));
    if (network == NULL) return NULL;

    for (int i = 0; i < MAX_USERS; i++) {
        network->users[i] = NULL;
    }

    network->user_count = 0;
    return network;
}

// Free social network
void free_social_network(SocialNetwork *network) {
    if (network == NULL) return;

    for (int i = 0; i < MAX_USERS; i++) {
        if (network->users[i] != NULL) {
            free_user(network->users[i]);
        }
    }

    free(network);
}
```

```
// Add user to network
bool add_user(SocialNetwork *network, int id, const char *name) {
    if (network == NULL || id < 0 || id >= MAX_USERS || name == NULL) {
        return false;
    }

    if (network->users[id] != NULL) {
        return false;  // User already exists
    }

    User *user = create_user(id, name);
    if (user == NULL) {
        return false;
    }

    network->users[id] = user;
    network->user_count++;
    return true;
}

// Add friendship between users
bool add_friendship(SocialNetwork *network, int user1_id, int user2_id) {
    if (network == NULL || user1_id < 0 || user1_id >= MAX_USERS ||
        user2_id < 0 || user2_id >= MAX_USERS) {
        return false;
    }

    User *user1 = network->users[user1_id];
    User *user2 = network->users[user2_id];

    if (user1 == NULL || user2 == NULL) {
        return false;
    }

    // Add user2 to user1's friend list
    if (user1->friend_count >= user1->max_friends) {
        user1->max_friends *= 2;
        User **temp = realloc(user1->friends, user1->max_friends * sizeof(User*));
        if (temp == NULL) {
            return false;
        }
        user1->friends = temp;
    }
    user1->friends[user1->friend_count++] = user2;

    // Add user1 to user2's friend list
    if (user2->friend_count >= user2->max_friends) {
```

```
        user2->max_friends *= 2;
        User **temp = realloc(user2->friends, user2->max_friends * sizeof(User*));
        if (temp == NULL) {
            return false;
        }
        user2->friends = temp;
    }
    user2->friends[user2->friend_count++] = user1;

    return true;
}

// Find mutual friends
void find_mutual_friends(SocialNetwork *network, int user1_id, int user2_id) {
    if (network == NULL || user1_id < 0 || user1_id >= MAX_USERS ||
        user2_id < 0 || user2_id >= MAX_USERS) {
        return;
    }

    User *user1 = network->users[user1_id];
    User *user2 = network->users[user2_id];

    if (user1 == NULL || user2 == NULL) {
        return;
    }

    printf("Mutual friends between %s and %s:\n", user1->name, user2->name);

    bool found = false;
    for (int i = 0; i < user1->friend_count; i++) {
        User *friend1 = user1->friends[i];
        for (int j = 0; j < user2->friend_count; j++) {
            if (user2->friends[j] == friend1) {
                printf("  %s\n", friend1->name);
                found = true;
                break;
            }
        }
    }

    if (!found) {
        printf("  No mutual friends found\n");
    }
}

// Calculate degree centrality (number of connections)
void calculate_degree_centrality(SocialNetwork *network) {
```

```
    if (network == NULL) return;

    printf("Degree Centrality (Number of Friends):\n");
    printf("======================================\n");

    for (int i = 0; i < MAX_USERS; i++) {
        if (network->users[i] != NULL) {
            printf("%s: %d friends\n",
                   network->users[i]->name,
                   network->users[i]->friend_count);
        }
    }
}

// Print social network
void print_social_network(SocialNetwork *network) {
    if (network == NULL) return;

    printf("Social Network:\n");
    printf("===============\n");

    for (int i = 0; i < MAX_USERS; i++) {
        if (network->users[i] != NULL) {
            User *user = network->users[i];
            printf("%s (ID: %d): ", user->name, user->id);

            if (user->friend_count == 0) {
                printf("No friends\n");
            } else {
                printf("Friends: ");
                for (int j = 0; j < user->friend_count; j++) {
                    printf("%s", user->friends[j]->name);
                    if (j < user->friend_count - 1) {
                        printf(", ");
                    }
                }
                printf("\n");
            }
        }
    }
}

int main() {
    SocialNetwork *network = create_social_network();
    if (network == NULL) {
        printf("Failed to create social network\n");
        return 1;
```

```
    }

    // Add users
    add_user(network, 0, "Alice");
    add_user(network, 1, "Bob");
    add_user(network, 2, "Charlie");
    add_user(network, 3, "Diana");
    add_user(network, 4, "Eve");

    // Add friendships
    add_friendship(network, 0, 1);  // Alice - Bob
    add_friendship(network, 0, 2);  // Alice - Charlie
    add_friendship(network, 1, 2);  // Bob - Charlie
    add_friendship(network, 1, 3);  // Bob - Diana
    add_friendship(network, 2, 4);  // Charlie - Eve
    add_friendship(network, 3, 4);  // Diana - Eve

    print_social_network(network);

    printf("\n");
    find_mutual_friends(network, 0, 3);  // Alice and Diana
    find_mutual_friends(network, 1, 4);  // Bob and Eve

    printf("\n");
    calculate_degree_centrality(network);

    free_social_network(network);
    return 0;
}
```

Summary

Graph algorithms are essential for solving complex problems in computer science and real-world applications:

1. **Graph Representations**:
   - Adjacency Matrix: Good for dense graphs, O(1) edge lookup
   - Adjacency List: Good for sparse graphs, space-efficient
2. **Graph Traversal**:
   - BFS: Level-by-level exploration, useful for shortest path in unweighted graphs
   - DFS: Deep exploration, useful for connectivity and cycle detection
3. **Shortest Path Algorithms**:
   - Dijkstra's Algorithm: Finds shortest paths in weighted graphs with non-negative edges

- Time complexity: O((V + E) log V) with binary heap

4. **Minimum Spanning Tree**:
    - Prim's Algorithm: Greedy approach to find MST
    - Time complexity: O((V + E) log V) with binary heap

Key considerations when working with graph algorithms: - **Graph Density**: Choose appropriate representation (matrix vs. list) - **Edge Weights**: Consider whether negative weights are allowed - **Directed vs. Undirected**: Algorithm implementation varies - **Performance Requirements**: Balance time and space complexity

Graph algorithms have numerous applications: - Social network analysis - Network routing protocols - Transportation and logistics - Computer network design - Circuit design - Game development (pathfinding) - Recommendation systems - Dependency resolution

Understanding graph algorithms and their implementations is crucial for solving complex computational problems efficiently.

Sorting and Searching

Introduction

Sorting and searching are fundamental operations in computer science that are used extensively in software development. Sorting arranges data in a specific order, making it easier to search, analyze, and process. Searching finds specific elements within a dataset, which is essential for data retrieval and manipulation.

In this chapter, we'll explore various sorting algorithms including comparison-based and non-comparison-based approaches, as well as different searching techniques. We'll implement these algorithms in C and analyze their time and space complexity.

Comparison-Based Sorting Algorithms

Bubble Sort

Bubble sort repeatedly steps through the list, compares adjacent elements, and swaps them if they're in the wrong order.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>

// Bubble sort implementation
void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        // Last i elements are already in place
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
```

```
        // If no swapping occurred, array is sorted
        if (!swapped) {
            break;
        }
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    bubble_sort(arr, n);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Selection Sort

Selection sort finds the minimum element from the unsorted part and places it at the beginning.

```
#include <stdio.h>

// Selection sort implementation
void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Find the minimum element in the remaining unsorted array
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
```

```
        }

        // Swap the found minimum element with the first element
        if (min_idx != i) {
            int temp = arr[min_idx];
            arr[min_idx] = arr[i];
            arr[i] = temp;
        }
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    selection_sort(arr, n);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Insertion Sort

Insertion sort builds the final sorted array one item at a time by inserting each element into its correct position.

```
#include <stdio.h>

// Insertion sort implementation
void insertion_sort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
```

```
        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    insertion_sort(arr, n);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Merge Sort

Merge sort is a divide-and-conquer algorithm that divides the array into halves, sorts them, and then merges them.

```
#include <stdio.h>
#include <stdlib.h>

// Merge two sorted subarrays
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
```

```
// Create temporary arrays
int *left_arr = malloc(n1 * sizeof(int));
int *right_arr = malloc(n2 * sizeof(int));

if (left_arr == NULL || right_arr == NULL) {
    free(left_arr);
    free(right_arr);
    return;
}

// Copy data to temporary arrays
for (int i = 0; i < n1; i++) {
    left_arr[i] = arr[left + i];
}
for (int i = 0; i < n2; i++) {
    right_arr[i] = arr[mid + 1 + i];
}

// Merge the temporary arrays back into arr[left..right]
int i = 0, j = 0, k = left;

while (i < n1 && j < n2) {
    if (left_arr[i] <= right_arr[j]) {
        arr[k] = left_arr[i];
        i++;
    } else {
        arr[k] = right_arr[j];
        j++;
    }
    k++;
}

// Copy remaining elements of left_arr[], if any
while (i < n1) {
    arr[k] = left_arr[i];
    i++;
    k++;
}

// Copy remaining elements of right_arr[], if any
while (j < n2) {
    arr[k] = right_arr[j];
    j++;
    k++;
}
```

```c
    free(left_arr);
    free(right_arr);
}

// Merge sort implementation
void merge_sort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    merge_sort(arr, 0, n - 1);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Quick Sort

Quick sort is a divide-and-conquer algorithm that picks an element as a pivot and partitions the array around it.

```c
#include <stdio.h>

// Partition function for quick sort
int partition(int arr[], int low, int high) {
    int pivot = arr[high];  // Choose the rightmost element as pivot
    int i = low - 1;        // Index of smaller element

    for (int j = low; j < high; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++;  // Increment index of smaller element
            // Swap arr[i] and arr[j]
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    // Swap arr[i+1] and arr[high] (pivot)
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Quick sort implementation
void quick_sort(int arr[], int low, int high) {
    if (low < high) {
        // Partition the array and get pivot index
        int pi = partition(arr, low, high);

        // Recursively sort elements before and after partition
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
```

```c
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    quick_sort(arr, 0, n - 1);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Heap Sort

Heap sort uses a binary heap data structure to sort elements efficiently.

```c
#include <stdio.h>

// Heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;    // Initialize largest as root
    int left = 2 * i + 1;   // Left child
    int right = 2 * i + 2;  // Right child

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    // If right child is larger than largest so far
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // If largest is not root
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}
```

```c
// Heap sort implementation
void heap_sort(int arr[], int n) {
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    // Extract elements from heap one by one
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // Call heapify on the reduced heap
        heapify(arr, i, 0);
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    heap_sort(arr, n);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Non-Comparison Sorting Algorithms

Counting Sort

Counting sort counts the occurrences of each element and uses this information to place elements in the correct position.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Counting sort implementation
void counting_sort(int arr[], int n, int max_val) {
    // Create count array to store count of each element
    int *count = calloc(max_val + 1, sizeof(int));
    int *output = malloc(n * sizeof(int));

    if (count == NULL || output == NULL) {
        free(count);
        free(output);
        return;
    }

    // Store count of each element
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    // Change count[i] so that count[i] now contains actual
    // position of this element in output array
    for (int i = 1; i <= max_val; i++) {
        count[i] += count[i - 1];
    }

    // Build the output array
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    // Copy the output array to arr[], so that arr[] now
    // contains sorted elements
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }

    free(count);
```

```
    free(output);
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {4, 2, 2, 8, 3, 3, 1};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    // Find maximum value in array
    int max_val = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }

    counting_sort(arr, n, max_val);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Radix Sort

Radix sort sorts numbers by processing individual digits, starting from the least significant digit to the most significant.

```
#include <stdio.h>
#include <stdlib.h>

// Get maximum value in array
int get_max(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
```

```
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

// Counting sort for a particular digit (exp)
void counting_sort_for_radix(int arr[], int n, int exp) {
    int *output = malloc(n * sizeof(int));
    int count[10] = {0};

    if (output == NULL) return;

    // Store count of occurrences in count[]
    for (int i = 0; i < n; i++) {
        count[(arr[i] / exp) % 10]++;
    }

    // Change count[i] so that count[i] now contains actual
    // position of this digit in output[]
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }

    // Build the output array
    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy the output array to arr[], so that arr[] now
    // contains sorted numbers according to current digit
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }

    free(output);
}

// Radix sort implementation
void radix_sort(int arr[], int n) {
    // Find the maximum number to know number of digits
    int max = get_max(arr, n);

    // Do counting sort for every digit
    for (int exp = 1; max / exp > 0; exp *= 10) {
```

```
        counting_sort_for_radix(arr, n, exp);
    }
}

// Print array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {170, 45, 75, 90, 2, 802, 24, 66};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    radix_sort(arr, n);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Bucket Sort

Bucket sort distributes elements into buckets and then sorts each bucket individually.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NUM_BUCKETS 10

typedef struct Node {
    float data;
    struct Node *next;
} Node;

// Create new node
Node* create_node(float data) {
    Node *node = malloc(sizeof(Node));
    if (node == NULL) return NULL;
```

```
    node->data = data;
    node->next = NULL;
    return node;
}

// Insert node in sorted order
void insert_sorted(Node **head, float data) {
    Node *new_node = create_node(data);
    if (new_node == NULL) return;

    // If list is empty or new node should be first
    if (*head == NULL || (*head)->data >= data) {
        new_node->next = *head;
        *head = new_node;
        return;
    }

    // Find the node before the point of insertion
    Node *current = *head;
    while (current->next != NULL && current->next->data < data) {
        current = current->next;
    }

    new_node->next = current->next;
    current->next = new_node;
}

// Bucket sort implementation
void bucket_sort(float arr[], int n) {
    // Create buckets
    Node *buckets[NUM_BUCKETS];
    for (int i = 0; i < NUM_BUCKETS; i++) {
        buckets[i] = NULL;
    }

    // Put array elements in different buckets
    for (int i = 0; i < n; i++) {
        int bucket_index = (int)(arr[i] * NUM_BUCKETS);
        // Handle edge case where arr[i] is 1.0
        if (bucket_index >= NUM_BUCKETS) {
            bucket_index = NUM_BUCKETS - 1;
        }
        insert_sorted(&buckets[bucket_index], arr[i]);
    }

    // Concatenate all buckets into arr[]
```

```
    int index = 0;
    for (int i = 0; i < NUM_BUCKETS; i++) {
        Node *current = buckets[i];
        while (current != NULL) {
            arr[index++] = current->data;
            Node *temp = current;
            current = current->next;
            free(temp);
        }
    }
}

// Print array
void print_array(float arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%.2f ", arr[i]);
    }
    printf("\n");
}

int main() {
    float arr[] = {0.897, 0.565, 0.656, 0.1234, 0.665, 0.3434};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array(arr, n);

    bucket_sort(arr, n);

    printf("Sorted array: ");
    print_array(arr, n);

    return 0;
}
```

Searching Algorithms

Linear Search

Linear search sequentially checks each element until the target is found or the end of the array is reached.

```
#include <stdio.h>

// Linear search implementation
```

```c
int linear_search(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;  // Return index of found element
        }
    }
    return -1;  // Element not found
}

int main() {
    int arr[] = {10, 20, 80, 30, 60, 50, 110, 100, 130, 170};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 110;

    int result = linear_search(arr, n, target);

    if (result != -1) {
        printf("Element %d found at index %d\n", target, result);
    } else {
        printf("Element %d not found in array\n", target);
    }

    return 0;
}
```

Binary Search

Binary search works on sorted arrays by repeatedly dividing the search interval in half.

```c
#include <stdio.h>

// Iterative binary search
int binary_search_iterative(int arr[], int n, int target) {
    int left = 0;
    int right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;  // Element found
        } else if (arr[mid] < target) {
            left = mid + 1;  // Search right half
        } else {
            right = mid - 1;  // Search left half
        }
```

```c
    }

    return -1;  // Element not found
}

// Recursive binary search
int binary_search_recursive(int arr[], int left, int right, int target) {
    if (left > right) {
        return -1;  // Element not found
    }

    int mid = left + (right - left) / 2;

    if (arr[mid] == target) {
        return mid;  // Element found
    } else if (arr[mid] < target) {
        return binary_search_recursive(arr, mid + 1, right, target);
    } else {
        return binary_search_recursive(arr, left, mid - 1, target);
    }
}

int main() {
    int arr[] = {2, 3, 4, 10, 40, 50, 80};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 10;

    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Iterative binary search
    int result1 = binary_search_iterative(arr, n, target);
    if (result1 != -1) {
        printf("Element %d found at index %d (iterative)\n", target, result1);
    } else {
        printf("Element %d not found (iterative)\n", target);
    }

    // Recursive binary search
    int result2 = binary_search_recursive(arr, 0, n - 1, target);
    if (result2 != -1) {
        printf("Element %d found at index %d (recursive)\n", target, result2);
    } else {
```

```
        printf("Element %d not found (recursive)\n", target);
    }

    return 0;
}
```

Interpolation Search

Interpolation search is an improvement over binary search for uniformly distributed sorted arrays.

```
#include <stdio.h>

// Interpolation search implementation
int interpolation_search(int arr[], int n, int target) {
    int low = 0;
    int high = n - 1;

    while (low <= high && target >= arr[low] && target <= arr[high]) {
        // If array has only one element
        if (low == high) {
            if (arr[low] == target) return low;
            return -1;
        }

        // Probing the position with keeping uniform distribution in mind
        int pos = low + (((double)(high - low) / (arr[high] - arr[low])) * (target - arr[low

        // Condition of target found
        if (arr[pos] == target) {
            return pos;
        }

        // If target is larger, target is in upper part
        if (arr[pos] < target) {
            low = pos + 1;
        }
        // If target is smaller, target is in lower part
        else {
            high = pos - 1;
        }
    }

    return -1;  // Element not found
}

int main() {
```

```
    int arr[] = {10, 12, 13, 16, 18, 19, 20, 21, 22, 23, 24, 33, 35, 42, 47};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 18;

    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int result = interpolation_search(arr, n, target);

    if (result != -1) {
        printf("Element %d found at index %d\n", target, result);
    } else {
        printf("Element %d not found\n", target);
    }

    return 0;
}
```

Advanced Searching Techniques

Exponential Search

Exponential search finds the range where the element might be present and then performs binary search in that range.

```
#include <stdio.h>

// Binary search helper function
int binary_search(int arr[], int left, int right, int target) {
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

```c
// Exponential search implementation
int exponential_search(int arr[], int n, int target) {
    // If first element is target
    if (arr[0] == target) {
        return 0;
    }

    // Find range for binary search by repeated doubling
    int i = 1;
    while (i < n && arr[i] <= target) {
        i *= 2;
    }

    // Call binary search for the found range
    return binary_search(arr, i / 2, (i < n) ? i : n - 1, target);
}

int main() {
    int arr[] = {2, 3, 4, 10, 40, 50, 80, 100, 120, 150, 200};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 100;

    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int result = exponential_search(arr, n, target);

    if (result != -1) {
        printf("Element %d found at index %d\n", target, result);
    } else {
        printf("Element %d not found\n", target);
    }

    return 0;
}
```

Jump Search

Jump search creates blocks and jumps through them to find the target block, then performs linear search within that block.

```c
#include <stdio.h>
#include <math.h>

// Jump search implementation
int jump_search(int arr[], int n, int target) {
    int step = sqrt(n);
    int prev = 0;

    // Finding the block where element is present (if it is present)
    while (arr[(int)fmin(step, n) - 1] < target) {
        prev = step;
        step += sqrt(n);
        if (prev >= n) {
            return -1;
        }
    }

    // Doing a linear search for target in block beginning with prev
    while (arr[prev] < target) {
        prev++;

        // If we reached next block or end of array, element is not present
        if (prev == fmin(step, n)) {
            return -1;
        }
    }

    // If element is found
    if (arr[prev] == target) {
        return prev;
    }

    return -1;
}

int main() {
    int arr[] = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 55;

    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int result = jump_search(arr, n, target);
```

```
    if (result != -1) {
        printf("Element %d found at index %d\n", target, result);
    } else {
        printf("Element %d not found\n", target);
    }

    return 0;
}
```

Selection Algorithms

Quickselect (Finding kth Smallest Element)

Quickselect finds the kth smallest element in an unordered list using a partitioning approach similar to quicksort.

```
#include <stdio.h>
#include <stdlib.h>

// Partition function (same as in quicksort)
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Quickselect algorithm to find kth smallest element
int quickselect(int arr[], int low, int high, int k) {
    if (low <= high) {
        // Partition the array and get pivot index
```

```c
        int pi = partition(arr, low, high);

        // If pivot is the kth element
        if (pi == k) {
            return arr[pi];
        }
        // If kth element is in left subarray
        else if (pi > k) {
            return quickselect(arr, low, pi - 1, k);
        }
        // If kth element is in right subarray
        else {
            return quickselect(arr, pi + 1, high, k);
        }
    }

    return -1;  // Error case
}

// Find kth smallest element
int find_kth_smallest(int arr[], int n, int k) {
    if (k < 0 || k >= n) {
        return -1;  // Invalid k
    }

    // Create a copy of array to avoid modifying original
    int *temp = malloc(n * sizeof(int));
    if (temp == NULL) return -1;

    for (int i = 0; i < n; i++) {
        temp[i] = arr[i];
    }

    int result = quickselect(temp, 0, n - 1, k);
    free(temp);

    return result;
}

int main() {
    int arr[] = {12, 3, 5, 7, 4, 19, 26};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 3;  // Find 4th smallest element (0-indexed)

    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
```

```
    }
    printf("\n");

    int kth_smallest = find_kth_smallest(arr, n, k);

    if (kth_smallest != -1) {
        printf("The %dth smallest element is: %d\n", k + 1, kth_smallest);
    } else {
        printf("Invalid k value\n");
    }

    return 0;
}
```

Performance Comparison and Analysis

Sorting Algorithm Performance Test

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

#define ARRAY_SIZE 10000

// Function prototypes
void bubble_sort(int arr[], int n);
void selection_sort(int arr[], int n);
void insertion_sort(int arr[], int n);
void merge_sort(int arr[], int left, int right);
void quick_sort(int arr[], int low, int high);
void heap_sort(int arr[], int n);

// Generate random array
void generate_random_array(int arr[], int n) {
    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }
}

// Copy array
void copy_array(int src[], int dest[], int n) {
    for (int i = 0; i < n; i++) {
```

```c
        dest[i] = src[i];
    }
}

// Test sorting algorithm performance
double test_sorting_algorithm(void (*sort_func)(int[], int), int arr[], int n, const char *n
    int *test_arr = malloc(n * sizeof(int));
    if (test_arr == NULL) return -1;

    copy_array(arr, test_arr, n);

    clock_t start = clock();
    sort_func(test_arr, n);
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("%-20s: %.6f seconds\n", name, time_taken);

    free(test_arr);
    return time_taken;
}

// Test sorting algorithm with range parameters
double test_sorting_algorithm_range(void (*sort_func)(int[], int, int), int arr[], int n, co
    int *test_arr = malloc(n * sizeof(int));
    if (test_arr == NULL) return -1;

    copy_array(arr, test_arr, n);

    clock_t start = clock();
    sort_func(test_arr, 0, n - 1);
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("%-20s: %.6f seconds\n", name, time_taken);

    free(test_arr);
    return time_taken;
}

// Bubble sort implementation
void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
```

```
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

// Selection sort implementation
void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        if (min_idx != i) {
            int temp = arr[min_idx];
            arr[min_idx] = arr[i];
            arr[i] = temp;
        }
    }
}

// Insertion sort implementation
void insertion_sort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Merge function for merge sort
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
```

```
    int *left_arr = malloc(n1 * sizeof(int));
    int *right_arr = malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) {
        left_arr[i] = arr[left + i];
    }
    for (int i = 0; i < n2; i++) {
        right_arr[i] = arr[mid + 1 + i];
    }

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (left_arr[i] <= right_arr[j]) {
            arr[k] = left_arr[i];
            i++;
        } else {
            arr[k] = right_arr[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = left_arr[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = right_arr[j];
        j++;
        k++;
    }

    free(left_arr);
    free(right_arr);
}

// Merge sort implementation
void merge_sort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```c
// Partition function for quick sort
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Quick sort implementation
void quick_sort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

// Heapify function for heap sort
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    if (largest != i) {
        int temp = arr[i];
```

```
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

// Heap sort implementation
void heap_sort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main() {
    int *original_arr = malloc(ARRAY_SIZE * sizeof(int));
    if (original_arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Generating random array of size %d...\n", ARRAY_SIZE);
    generate_random_array(original_arr, ARRAY_SIZE);

    printf("\nSorting Algorithm Performance Comparison:\n");
    printf("==========================================\n");

    // Test various sorting algorithms
    test_sorting_algorithm(bubble_sort, original_arr, ARRAY_SIZE, "Bubble Sort");
    test_sorting_algorithm(selection_sort, original_arr, ARRAY_SIZE, "Selection Sort");
    test_sorting_algorithm(insertion_sort, original_arr, ARRAY_SIZE, "Insertion Sort");
    test_sorting_algorithm_range(merge_sort, original_arr, ARRAY_SIZE, "Merge Sort");
    test_sorting_algorithm_range(quick_sort, original_arr, ARRAY_SIZE, "Quick Sort");
    test_sorting_algorithm(heap_sort, original_arr, ARRAY_SIZE, "Heap Sort");

    free(original_arr);
    return 0;
}
```

Practical Examples

Phone Book Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define MAX_CONTACTS 1000
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 20

typedef struct {
    char name[MAX_NAME_LENGTH];
    char phone[MAX_PHONE_LENGTH];
} Contact;

typedef struct {
    Contact contacts[MAX_CONTACTS];
    int count;
} PhoneBook;

// Create phone book
PhoneBook* create_phone_book(void) {
    PhoneBook *book = malloc(sizeof(PhoneBook));
    if (book == NULL) return NULL;

    book->count = 0;
    return book;
}

// Free phone book
void free_phone_book(PhoneBook *book) {
    free(book);
}

// Add contact to phone book
bool add_contact(PhoneBook *book, const char *name, const char *phone) {
    if (book == NULL || name == NULL || phone == NULL || book->count >= MAX_CONTACTS) {
        return false;
    }

    strncpy(book->contacts[book->count].name, name, MAX_NAME_LENGTH - 1);
    book->contacts[book->count].name[MAX_NAME_LENGTH - 1] = '\0';
```

```
    strncpy(book->contacts[book->count].phone, phone, MAX_PHONE_LENGTH - 1);
    book->contacts[book->count].phone[MAX_PHONE_LENGTH - 1] = '\0';

    book->count++;
    return true;
}

// Linear search for contact by name
int search_contact_linear(PhoneBook *book, const char *name) {
    if (book == NULL || name == NULL) return -1;

    for (int i = 0; i < book->count; i++) {
        if (strcmp(book->contacts[i].name, name) == 0) {
            return i;
        }
    }

    return -1;
}

// Comparison function for sorting contacts by name
int compare_contacts(const void *a, const void *b) {
    const Contact *contact_a = (const Contact *)a;
    const Contact *contact_b = (const Contact *)b;
    return strcmp(contact_a->name, contact_b->name);
}

// Sort phone book by name
void sort_phone_book(PhoneBook *book) {
    if (book == NULL) return;

    qsort(book->contacts, book->count, sizeof(Contact), compare_contacts);
}

// Binary search for contact by name (requires sorted phone book)
int search_contact_binary(PhoneBook *book, const char *name) {
    if (book == NULL || name == NULL) return -1;

    int left = 0;
    int right = book->count - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int cmp = strcmp(book->contacts[mid].name, name);

        if (cmp == 0) {
            return mid;
```

```
        } else if (cmp < 0) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}

// Print all contacts
void print_phone_book(PhoneBook *book) {
    if (book == NULL) return;

    printf("Phone Book (%d contacts):\n", book->count);
    printf("========================\n");

    for (int i = 0; i < book->count; i++) {
        printf("%-20s: %s\n", book->contacts[i].name, book->contacts[i].phone);
    }
    printf("\n");
}

// Print specific contact
void print_contact(PhoneBook *book, int index) {
    if (book == NULL || index < 0 || index >= book->count) return;

    printf("Name: %s\n", book->contacts[index].name);
    printf("Phone: %s\n", book->contacts[index].phone);
}

int main() {
    PhoneBook *book = create_phone_book();
    if (book == NULL) {
        printf("Failed to create phone book\n");
        return 1;
    }

    // Add contacts
    add_contact(book, "Alice Johnson", "555-1234");
    add_contact(book, "Bob Smith", "555-5678");
    add_contact(book, "Charlie Brown", "555-9012");
    add_contact(book, "Diana Prince", "555-3456");
    add_contact(book, "Eve Wilson", "555-7890");
    add_contact(book, "Frank Miller", "555-2345");
    add_contact(book, "Grace Lee", "555-6789");
```

```
    printf("Unsorted phone book:\n");
    print_phone_book(book);

    // Linear search
    printf("Linear search for 'Charlie Brown':\n");
    int index = search_contact_linear(book, "Charlie Brown");
    if (index != -1) {
        print_contact(book, index);
    } else {
        printf("Contact not found\n");
    }

    // Sort phone book
    printf("\nSorting phone book...\n");
    sort_phone_book(book);

    printf("\nSorted phone book:\n");
    print_phone_book(book);

    // Binary search
    printf("Binary search for 'Eve Wilson':\n");
    index = search_contact_binary(book, "Eve Wilson");
    if (index != -1) {
        print_contact(book, index);
    } else {
        printf("Contact not found\n");
    }

    free_phone_book(book);
    return 0;
}
```

Summary

Sorting and searching algorithms are fundamental to computer science and software development:

1. **Comparison-Based Sorting**:
    - Bubble Sort: $O(n^2)$ time, simple but inefficient
    - Selection Sort: $O(n^2)$ time, minimal memory writes
    - Insertion Sort: $O(n^2)$ time, efficient for small datasets
    - Merge Sort: $O(n \log n)$ time, stable and consistent
    - Quick Sort: $O(n \log n)$ average time, in-place sorting
    - Heap Sort: $O(n \log n)$ time, in-place and consistent
2. **Non-Comparison Sorting**:
    - Counting Sort: $O(n + k)$ time, for small range integers

- Radix Sort: O(d × (n + k)) time, for fixed-width integers
- Bucket Sort: O(n + k) average time, for uniformly distributed data

3. **Searching Algorithms**:

   - Linear Search: O(n) time, works on unsorted data
   - Binary Search: O(log n) time, requires sorted data
   - Interpolation Search: O(log log n) average time, for uniformly distributed data
   - Exponential Search: O(log n) time, for unbounded searches
   - Jump Search: O(√n) time, alternative to binary search

4. **Selection Algorithms**:

   - Quickselect: O(n) average time, finds kth smallest element

Key considerations when choosing algorithms: - **Data Size**: Small datasets may benefit from simple algorithms - **Data Distribution**: Uniform distribution enables faster algorithms - **Memory Constraints**: In-place algorithms save memory - **Stability Requirements**: Stable sorts preserve relative order - **Performance Requirements**: Balance time and space complexity

Understanding these algorithms and their trade-offs is essential for: - Optimizing application performance - Choosing appropriate data structures - Implementing efficient algorithms - Solving complex computational problems - Passing technical interviews

Mastering sorting and searching techniques provides a strong foundation for advanced algorithmic thinking and problem-solving skills.

Module 10: Data Structures and Algorithms

Overview

This module explores fundamental data structures and algorithms essential for efficient C programming. You'll learn about linear data structures like dynamic arrays, linked lists, stacks, and queues, as well as more complex structures such as trees, hash tables, and graphs. The module also covers essential algorithms for sorting, searching, and graph traversal, providing a solid foundation for solving complex computational problems.

Learning Objectives

By the end of this module, you will be able to: - Implement and use dynamic arrays (vectors) effectively - Design and implement various types of linked lists - Understand and apply stack and queue data structures - Implement tree structures including binary trees and balanced trees - Design and use hash tables for efficient data retrieval - Apply graph algorithms for network and pathfinding problems - Implement classic sorting and searching algorithms - Analyze time and space complexity of algorithms - Choose appropriate data structures for specific problems - Apply best practices for data structure implementation in C

Chapters

1. **Linear Data Structures** - Dynamic arrays, linked lists, stacks, queues, deques, priority queues
2. **Tree Structures** - Binary trees, binary search trees, AVL trees, red-black trees, B-trees
3. **Hash Tables** - Hash function design, collision resolution, dynamic resizing, performance analysis
4. **Graph Algorithms** - Graph representations, traversal algorithms, shortest path algorithms, minimum spanning trees
5. **Sorting and Searching** - Comparison-based sorting, non-comparison sorting, binary search variations, selection algorithms

Key Concepts Covered

- Dynamic array implementation and resizing strategies
- Singly, doubly, and circular linked lists
- Stack implementation (array-based and linked-based)

- Queue implementation (circular buffer and linked-based)
- Deque and priority queue implementations
- Binary tree traversal (inorder, preorder, postorder)
- Binary search tree operations (insert, delete, search)
- AVL tree balancing and rotations
- Red-black tree properties and operations
- B-trees and B+ trees for database applications
- Hash function design and evaluation
- Collision resolution strategies (chaining, open addressing)
- Dynamic hash table resizing
- Graph representations (adjacency matrix, adjacency list)
- Breadth-first search (BFS) and depth-first search (DFS)
- Dijkstra's shortest path algorithm
- A* pathfinding algorithm
- Minimum spanning tree algorithms (Prim's, Kruskal's)
- Comparison-based sorting (quicksort, mergesort, heapsort)
- Non-comparison sorting (radix sort, counting sort, bucket sort)
- Binary search and its variations
- Selection algorithms (quickselect)
- Time and space complexity analysis
- Amortized analysis techniques
- Memory management for data structures
- Error handling in data structure operations

Prerequisites

- Completion of Module 1: Foundations and Environment Setup
- Completion of Module 2: Data Types and Variables
- Completion of Module 3: Control Flow
- Completion of Module 4: Functions and Modular Programming
- Completion of Module 5: Arrays and Strings
- Completion of Module 6: Pointers and Memory Management
- Completion of Module 7: Structures and User-Defined Types
- Completion of Module 8: File I/O and System Programming
- Completion of Module 9: Advanced C Features and Modern Standards
- Understanding of basic programming concepts
- Familiarity with C language fundamentals
- Knowledge of pointers and dynamic memory management

Tools and Technologies

- C compiler (GCC, Clang, or MSVC)
- Text editor or IDE
- Memory debugging tools (Valgrind, AddressSanitizer)
- Performance profiling tools (gprof, perf)

- Graph visualization tools (optional)
- Understanding of basic algorithms and data structures

Estimated Time to Complete

- Reading: 15-20 hours
- Exercises: 20-25 hours
- Projects: 15-20 hours

Assessment

- Complete all chapter exercises
- Successfully implement various data structures from scratch
- Demonstrate understanding of algorithm complexity analysis
- Pass data structures and algorithms quiz
- Submit comprehensive implementations of classic data structures

Next Module

After completing this module, proceed to **Module 11: Network Programming** to learn about socket programming and network communication.

Additional Resources

- "Introduction to Algorithms" by Cormen, Leiserson, Rivest, and Stein
- "Data Structures and Algorithms in C" by Adam Drozdek
- "The Algorithm Design Manual" by Steven Skiena
- Online algorithm visualization tools
- Competitive programming platforms (LeetCode, HackerRank)
- Open source data structure libraries

Network

Network Fundamentals

Network programming lets C programs exchange data across hosts. This chapter covers the models and addressing you need, then focuses on **real POSIX C**: endianness, `htons`/`htonl`, `inet_pton`/`inet_ntop`, filling `sockaddr_in`, and a complete **UDP echo client and server**.

Environment: Linux, `gcc -std=c17 -Wall -Wextra`. Link nothing extra for these examples (just libc).

```
gcc -std=c17 -Wall -Wextra -o prog prog.c
```

Computer Network Basics

A network is a set of interconnected devices that share a common communication path. Scope ranges from a USB gadget link to the global Internet.

Topologies (conceptual)

1. **Bus** — shared medium
2. **Star** — central switch/hub
3. **Ring** — circular path
4. **Mesh** — multiple redundant paths
5. **Tree** — hierarchical aggregation

Network types by scope

Type	Scope	Example
PAN	Person / desk	Bluetooth
LAN	Building / campus	Office Ethernet
MAN	City	Metro fiber
WAN	Large region	ISP backbone
Internet	Global	Public IP routing

OSI and TCP/IP Models

OSI (7 layers)

1. Physical — bits on wire/radio
2. Data Link — frames, MAC
3. Network — routing (IP)
4. Transport — end-to-end (TCP/UDP)
5. Session — dialogs
6. Presentation — encoding/encryption
7. Application — HTTP, DNS, …

TCP/IP (practical 4 layers)

TCP/IP layer	Rough OSI	Examples
Link	1–2	Ethernet, Wi-Fi
Internet	3	IPv4, IPv6, ICMP
Transport	4	TCP, UDP
Application	5–7	HTTP, DNS, SSH

In socket programming you mostly touch **transport + addressing (IP + port)**.

IP Addressing and Ports

IPv4

32-bit addresses written dotted-decimal: `192.168.1.10`.

Special addresses:

- `127.0.0.1` — loopback (this host)
- `0.0.0.0` — "any" address when binding a server
- `255.255.255.255` — limited broadcast (local link)

Historical classful ranges (A/B/C) matter less than **CIDR** (`192.168.0.0/24`), but you still see class language in older docs.

IPv6 (awareness)

128-bit addresses: `2001:db8::1`. Use `AF_INET6`, `struct sockaddr_in6`, and `inet_pton(AF_INET6, ...)`. This chapter's full programs use IPv4 for clarity; the conversion APIs are dual-stack ready.

Ports

16-bit service numbers (0–65535):

Range	Name	Examples
0–1023	Well-known	22 SSH, 80 HTTP, 443 HTTPS
1024–49151	Registered	Many app servers
49152–65535	Ephemeral	Client source ports

Binding ports below 1024 usually requires root/capabilities on Linux.

TCP vs UDP (what sockets expose)

	TCP (`SOCK_STREAM`)	UDP (`SOCK_DGRAM`)
Connection	Handshake (3-way)	None
Reliability	Retransmit, ordered	Best-effort
Message bounds	Byte stream	Datagram boundaries kept
Use cases	HTTP, SSH, databases	DNS, games, discovery, echo labs

This chapter ends with **UDP echo** because it is the smallest complete network program (no listen/accept).

Sockets and Address Structures

A **socket** is an endpoint (a file descriptor on Unix). You create one with `socket()`, then `bind`/`connect`/`sendto`/`recvfrom` as needed.

Headers

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netdb.h>
```

```c
#include <errno.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
```

Structures

```c
/* IPv4 socket address */
struct sockaddr_in {
    sa_family_t    sin_family; /* AF_INET */
    in_port_t      sin_port;   /* port in network byte order */
    struct in_addr sin_addr;   /* IPv4 address, network byte order */
    /* padding (sin_zero) often present */
};

struct in_addr {
    uint32_t s_addr;           /* address in network byte order */
};

/* Generic shape used by bind/connect (cast to/from) */
struct sockaddr {
    sa_family_t sa_family;
    char        sa_data[14];
};
```

Always set unused fields to zero (`memset` the whole `sockaddr_in`) before filling fields.

Endianness: Why Byte Order Matters

Multi-byte integers can be stored **little-endian** (least significant byte first — common on x86/x86-64) or **big-endian** (most significant byte first). **Network byte order is big-endian.**

If you put a host `uint16_t` port into a packet without conversion, peers on different architectures (or even the same machine's stack) will disagree about the value.

Program: inspect endianness and multi-byte layout

```c
/* file: endian_demo.c */
#include <stdio.h>
#include <stdint.h>
#include <arpa/inet.h>
```

```
static void print_bytes(const char *label, const void *p, size_t n) {
    const unsigned char *b = p;
    size_t i;
    printf("%s:", label);
    for (i = 0; i < n; i++) {
        printf(" %02x", b[i]);
    }
    printf("\n");
}

int main(void) {
    uint32_t host_u32 = 0x12345678u;
    uint16_t host_u16 = 0xABCDu;
    uint32_t net_u32;
    uint16_t net_u16;

    {
        uint16_t probe = 0x0102;
        const unsigned char *p = (const unsigned char *)&probe;
        if (p[0] == 0x01 && p[1] == 0x02) {
            printf("This host appears big-endian\n");
        } else if (p[0] == 0x02 && p[1] == 0x01) {
            printf("This host appears little-endian\n");
        } else {
            printf("Unexpected byte order probe\n");
        }
    }

    print_bytes("host uint32 0x12345678", &host_u32, sizeof host_u32);
    print_bytes("host uint16 0xABCD", &host_u16, sizeof host_u16);

    net_u32 = htonl(host_u32);
    net_u16 = htons(host_u16);

    print_bytes("htonl(0x12345678)", &net_u32, sizeof net_u32);
    print_bytes("htons(0xABCD)", &net_u16, sizeof net_u16);

    printf("ntohl(htonl(x)) == x ? %s\n",
           ntohl(net_u32) == host_u32 ? "yes" : "no");
    printf("ntohs(htons(x)) == x ? %s\n",
           ntohs(net_u16) == host_u16 ? "yes" : "no");

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o endian_demo endian_demo.c
./endian_demo
```

On a little-endian host you typically see:

```
This host appears little-endian
host uint32 0x12345678: 78 56 34 12
htonl(0x12345678): 12 34 56 78
```

Network order always shows the "natural" human hex order in memory for that value's big-endian encoding.

---

htons / htonl / ntohs / ntohl

```
#include <arpa/inet.h>

uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

Function	Direction	Width
`htons`	host → network	16-bit (ports)
`htonl`	host → network	32-bit (IPv4 `s_addr` pieces, some protocol fields)
`ntohs`	network → host	16-bit
`ntohl`	network → host	32-bit

Demo program

```
/* file: hton_demo.c */
#include <stdio.h>
#include <stdint.h>
#include <arpa/inet.h>

int main(void) {
    uint16_t port_host = 8080;
    uint16_t port_net = htons(port_host);
    uint32_t word_host = 0x12345678u;
```

```
    printf("port host:  %u (0x%04x)\n", port_host, port_host);
    printf("port net:   %u (0x%04x)  /* value after htons */\n",
           port_net, port_net);
    printf("port back:  %u\n", ntohs(port_net));

    printf("word host:  0x%08x\n", word_host);
    printf("word net:   0x%08x\n", htonl(word_host));
    printf("word back:  0x%08x\n", ntohl(htonl(word_host)));

    /* Rule: store htons(port) into sin_port; print with ntohs(sin.sin_port) */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o hton_demo hton_demo.c
./hton_demo
```

Rule of thumb: every multi-byte field that crosses the wire (or sits in `sockaddr_in`) is converted **to network order on write** and **to host order on read**.

`inet_pton` and `inet_ntop`

Prefer these over older `inet_addr` / `inet_ntoa` (which are IPv4-only, weaker error reporting, and `inet_ntoa` is not thread-safe).

```
#include <arpa/inet.h>

int inet_pton(int af, const char *src, void *dst);
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
```

- `inet_pton`: text → binary. Returns `1` success, `0` invalid text, `-1` bad `af` / error.
- `inet_ntop`: binary → text. Returns `dst` on success, `NULL` on error.
- Buffer size: `INET_ADDRSTRLEN` (IPv4) or `INET6_ADDRSTRLEN` (IPv6).

Full conversion + `sockaddr_in` setup demo

```
/* file: addr_demo.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
```

```c
#include <netinet/in.h>

int main(void) {
    const char *ip_text = "192.168.1.100";
    const uint16_t port = 9000;
    struct sockaddr_in addr;
    char ip_out[INET_ADDRSTRLEN];

    memset(&addr, 0, sizeof addr);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);

    if (inet_pton(AF_INET, ip_text, &addr.sin_addr) != 1) {
        fprintf(stderr, "inet_pton failed for %s\n", ip_text);
        return EXIT_FAILURE;
    }

    if (inet_ntop(AF_INET, &addr.sin_addr, ip_out, sizeof ip_out) == NULL) {
        perror("inet_ntop");
        return EXIT_FAILURE;
    }

    printf("text in:  %s\n", ip_text);
    printf("text out: %s\n", ip_out);
    printf("port:     %u (wire field raw after htons: 0x%04x)\n",
           ntohs(addr.sin_port), addr.sin_port);
    printf("s_addr:   0x%08x (network order bits)\n",
           addr.sin_addr.s_addr);

    /* Bind-all pattern for servers */
    memset(&addr, 0, sizeof addr);
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY); /* 0.0.0.0 */
    addr.sin_port = htons(9000);
    printf("server bind: 0.0.0.0:%u\n", ntohs(addr.sin_port));

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o addr_demo addr_demo.c
./addr_demo
```

Filling `sockaddr_in` checklist

```
struct sockaddr_in sa;
memset(&sa, 0, sizeof sa);
sa.sin_family = AF_INET;
sa.sin_port   = htons(port);
inet_pton(AF_INET, "127.0.0.1", &sa.sin_addr);
/* or: sa.sin_addr.s_addr = htonl(INADDR_ANY); */
```

When calling `bind` / `sendto` / `recvfrom`, cast:

```
bind(fd, (struct sockaddr *)&sa, sizeof sa);
```

Minimal UDP Echo: Server and Client

UDP echo: client sends a datagram; server replies with the same payload to the source address.

Server (`udp_echo_server.c`)

```
/* file: udp_echo_server.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>

#define BUF_SIZE 2048

int main(int argc, char *argv[]) {
    int fd;
    uint16_t port;
    struct sockaddr_in srv;
    struct sockaddr_in cli;
    socklen_t cli_len;
    char buf[BUF_SIZE];
    char addrbuf[INET_ADDRSTRLEN];
    ssize_t n;

    if (argc != 2) {
```

```
    fprintf(stderr, "Usage: %s <port>\n", argv[0]);
    return EXIT_FAILURE;
}
port = (uint16_t)atoi(argv[1]);
if (port == 0) {
    fprintf(stderr, "invalid port\n");
    return EXIT_FAILURE;
}

fd = socket(AF_INET, SOCK_DGRAM, 0);
if (fd < 0) {
    perror("socket");
    return EXIT_FAILURE;
}

memset(&srv, 0, sizeof srv);
srv.sin_family = AF_INET;
srv.sin_addr.s_addr = htonl(INADDR_ANY);
srv.sin_port = htons(port);

if (bind(fd, (struct sockaddr *)&srv, sizeof srv) < 0) {
    perror("bind");
    close(fd);
    return EXIT_FAILURE;
}

printf("UDP echo server listening on port %u\n", port);

for (;;) {
    cli_len = sizeof cli;
    n = recvfrom(fd, buf, sizeof buf, 0,
                 (struct sockaddr *)&cli, &cli_len);
    if (n < 0) {
        perror("recvfrom");
        continue;
    }

    if (inet_ntop(AF_INET, &cli.sin_addr, addrbuf, sizeof addrbuf) == NULL) {
        strncpy(addrbuf, "?", sizeof addrbuf);
        addrbuf[sizeof addrbuf - 1] = '\0';
    }
    printf("from %s:%u (%zd bytes)\n",
           addrbuf, ntohs(cli.sin_port), n);

    if (sendto(fd, buf, (size_t)n, 0,
               (struct sockaddr *)&cli, cli_len) < 0) {
        perror("sendto");
```

```
        }
    }

    /* unreachable in this simple loop */
    /* close(fd); */
    /* return 0; */
}
```

Client (`udp_echo_client.c`)

```
/* file: udp_echo_client.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>

#define BUF_SIZE 2048

int main(int argc, char *argv[]) {
    int fd;
    struct sockaddr_in srv;
    char buf[BUF_SIZE];
    ssize_t n;
    size_t len;

    if (argc != 4) {
        fprintf(stderr, "Usage: %s <server_ip> <port> <message>\n", argv[0]);
        return EXIT_FAILURE;
    }

    fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (fd < 0) {
        perror("socket");
        return EXIT_FAILURE;
    }

    memset(&srv, 0, sizeof srv);
    srv.sin_family = AF_INET;
    srv.sin_port = htons((uint16_t)atoi(argv[2]));
    if (inet_pton(AF_INET, argv[1], &srv.sin_addr) != 1) {
        fprintf(stderr, "invalid server IP: %s\n", argv[1]);
        close(fd);
```

```
        return EXIT_FAILURE;
    }

    len = strlen(argv[3]);
    if (len == 0 || len >= BUF_SIZE) {
        fprintf(stderr, "message empty or too long\n");
        close(fd);
        return EXIT_FAILURE;
    }

    if (sendto(fd, argv[3], len, 0,
               (struct sockaddr *)&srv, sizeof srv) < 0) {
        perror("sendto");
        close(fd);
        return EXIT_FAILURE;
    }

    n = recvfrom(fd, buf, sizeof buf - 1, 0, NULL, NULL);
    if (n < 0) {
        perror("recvfrom");
        close(fd);
        return EXIT_FAILURE;
    }
    buf[n] = '\0';
    printf("echo: %s\n", buf);

    close(fd);
    return 0;
}
```

Build and run (two terminals)

```
gcc -std=c17 -Wall -Wextra -o udp_echo_server udp_echo_server.c
gcc -std=c17 -Wall -Wextra -o udp_echo_client udp_echo_client.c

# terminal 1
./udp_echo_server 9000

# terminal 2
./udp_echo_client 127.0.0.1 9000 "hello UDP"
# echo: hello UDP
```

What just happened

1. Server `socket` + `bind` to `0.0.0.0:9000`

2. Client `sendto` a datagram to `127.0.0.1:9000`
3. Kernel delivers payload; `recvfrom` also fills **client address**
4. Server `sendto` same bytes back to that address
5. Client `recvfrom` prints the reply

No `listen`, no `accept` — that is TCP territory (next chapter).

Optional: see the packets

```
# while the exchange runs
sudo tcpdump -i lo -n udp port 9000
```

Network Errors (practical)

Always check return values:

```
if (fd < 0) {
    fprintf(stderr, "socket: %s\n", strerror(errno));
    exit(EXIT_FAILURE);
}
if (bind(fd, (struct sockaddr *)&srv, sizeof srv) < 0) {
    fprintf(stderr, "bind: %s\n", strerror(errno));
    close(fd);
    exit(EXIT_FAILURE);
}
```

Symptom	Typical cause
`EADDRINUSE`	Port already bound
`EACCES`	Port < 1024 without privilege
`ECONNREFUSED`	More common with TCP; UDP may get ICMP unreachable later
`EINVAL`	Wrong address length / family
`ENETUNREACH` / `EHOSTUNREACH`	Routing problem

UDP clients often hang in `recvfrom` if the server is down (no automatic error). Use `setsockopt` timeouts (`SO_RCVTIMEO`) for production clients.

```c
struct timeval tv = {.tv_sec = 3, .tv_usec = 0};
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
```

Hostname Lookup (brief, modern)

Prefer `getaddrinfo` over deprecated `gethostbyname`:

```c
/* sketch - full dual-stack client belongs with TCP chapter */
struct addrinfo hints, *res;
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_DGRAM;
if (getaddrinfo("localhost", "9000", &hints, &res) != 0) {
    /* handle error */
}
/* use res->ai_addr with sendto; freeaddrinfo(res) later */
```

Protocol Quick Reference

- **TCP** — reliable stream; next chapter builds connect/accept.
- **UDP** — datagrams; this chapter's echo lab.
- **ICMP** — diagnostics (`ping`); raw sockets need privileges.

Exercises

Exercise 1 — Endianness report

Run `endian_demo` on your machine. Paste the byte listings for `0x12345678` before and after `htonl`. Explain which listing matches network order.

Exercise 2 — Port round-trip

Write a program that takes a port number as `argv[1]`, stores `htons` into a `sockaddr_in`, then prints `ntohs(sin_port)`. Confirm the printed port equals the argument for values `80`, `8080`, and `65535`.

Exercise 3 — `inet_pton` validation

Feed `inet_pton` the strings `127.0.0.1`, `256.0.0.1`, `::1` (with `AF_INET`), and `not-an-ip`. Print the return code for each. Then convert `::1` with `AF_INET6` successfully.

Exercise 4 — Bind failure

Start `udp_echo_server 9000` twice. Capture the error from the second bind. Kill the first and confirm the second starts.

Exercise 5 — Echo timeout

Add `SO_RCVTIMEO` of 2 seconds to the client. Run the client without a server and show that it fails cleanly instead of hanging forever.

Exercise 6 — Multi-message server log

Modify the server to prefix each log line with a packet counter. Send five client messages and verify counters 1…5.

Exercise 7 — Binary payload

Send 8 raw bytes (not a C string) from a small client using an `unsigned char` array. Have the server reply with the same bytes. Confirm length stays 8 (no reliance on `\0`).

Summary

Concept	C takeaway
TCP/IP model	Sockets sit at transport + app
Endianness	Network = big-endian; use `hton*` / `ntoh*`
Text binary IP	`inet_pton` / `inet_ntop`
Address struct	Zero, set `AF_INET`, `htons` port, `inet_pton` or `INADDR_ANY`
UDP echo	`socket` → `bind`/`sendto`/`recvfrom`
Errors	Check every call; `strerror(errno)`

You now have the addressing and byte-order tools every C network program needs, plus a complete UDP exchange you can extend into real services. The next chapter builds **TCP** socket programming on the same foundations.

Socket Programming

Introduction

Socket programming is the foundation of network communication in C. Sockets provide an interface for applications to communicate over a network using various protocols such as TCP and UDP. This chapter covers the essential concepts and functions needed to create network applications using sockets.

Socket Basics

A socket is an endpoint for communication between two machines over a network. In Unix-like systems, sockets are treated as file descriptors, allowing the use of standard I/O functions for network communication.

Socket Creation

The first step in socket programming is creating a socket using the `socket()` function:

```
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

Parameters: - `domain`: Communication domain (e.g., AF_INET for IPv4, AF_INET6 for IPv6) - `type`: Communication type (e.g., SOCK_STREAM for TCP, SOCK_DGRAM for UDP) - `protocol`: Specific protocol (usually 0 for default)

Return Value: - Socket file descriptor on success - -1 on error

Example Socket Creation

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
```

```
    int server_fd;

    // Create TCP socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    printf("Socket created successfully with file descriptor: %d\n", server_fd);

    // Close socket
    close(server_fd);
    return 0;
}
```

Socket Address Structures

Different address structures are used for different address families:

IPv4 Address Structure

```
struct sockaddr_in {
    sa_family_t    sin_family; // Address family (AF_INET)
    in_port_t      sin_port;   // Port number (network byte order)
    struct in_addr sin_addr;   // IP address (network byte order)
    char           sin_zero[8]; // Padding to match sockaddr size
};
```

Generic Socket Address Structure

```
struct sockaddr {
    sa_family_t sa_family;     // Address family
    char        sa_data[14];   // Address data
};
```

Server-Side Socket Programming

Creating a server involves several steps: socket creation, binding, listening, and accepting connections.

Binding a Socket

The `bind()` function associates a socket with a specific address and port:

```
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

Listening for Connections

The `listen()` function makes a socket listen for incoming connections:

```
int listen(int sockfd, int backlog);
```

Accepting Connections

The `accept()` function accepts an incoming connection:

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

Complete TCP Server Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    char *hello = "Hello from server";

    // Create socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
```

```
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Forcefully attaching socket to the port
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
               &opt, sizeof(opt))) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}

// Configure address structure
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind the socket to the network address and port
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen for incoming connections
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d\n", PORT);

// Accept incoming connection
if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
                         (socklen_t*)&addrlen)) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}

// Read message from client
int valread = read(new_socket, buffer, BUFFER_SIZE);
printf("Client: %s\n", buffer);

// Send message to client
send(new_socket, hello, strlen(hello), 0);
printf("Hello message sent\n");

// Close sockets
close(new_socket);
```

```
    close(server_fd);

    return 0;
}
```

Client-Side Socket Programming

Creating a client involves socket creation and connecting to a server.

Connecting to a Server

The `connect()` function establishes a connection to a server:

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

Complete TCP Client Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char *hello = "Hello from client";
    char buffer[BUFFER_SIZE] = {0};

    // Create socket file descriptor
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    // Configure server address
```

```
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary form
    if(inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }

    // Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // Send message to server
    send(sock, hello, strlen(hello), 0);
    printf("Hello message sent\n");

    // Read message from server
    int valread = read(sock, buffer, BUFFER_SIZE);
    printf("Server: %s\n", buffer);

    // Close socket
    close(sock);

    return 0;
}
```

UDP Socket Programming

UDP (User Datagram Protocol) provides connectionless communication. Unlike TCP, there's no need for connection establishment.

UDP Server Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

```c
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in servaddr, cliaddr;

    // Create socket file descriptor
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0 ) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Clear address structures
    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    // Configure server address
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    // Bind the socket with the server address
    if (bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    int len, n;
    n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE, MSG_WAITALL,
                 (struct sockaddr *) &cliaddr, &len);
    buffer[n] = '\0';
    printf("Client : %s\n", buffer);

    char *hello = "Hello from server";
    sendto(sockfd, (const char *)hello, strlen(hello), MSG_CONFIRM,
           (const struct sockaddr *) &cliaddr, len);
    printf("Hello message sent.\n");

    close(sockfd);
    return 0;
}
```

UDP Client Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in servaddr;

    // Create socket file descriptor
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Clear server address structure
    memset(&servaddr, 0, sizeof(servaddr));

    // Configure server address
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = INADDR_ANY;

    int n;
    char *hello = "Hello from client";
    sendto(sockfd, (const char *)hello, strlen(hello), MSG_CONFIRM,
           (const struct sockaddr *) &servaddr, sizeof(servaddr));
    printf("Hello message sent.\n");

    socklen_t len;
    n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE, MSG_WAITALL,
                 (struct sockaddr *) &servaddr, &len);
    buffer[n] = '\0';
    printf("Server : %s\n", buffer);

    close(sockfd);
    return 0;
}
```

Advanced Socket Options

Socket Options

The `setsockopt()` and `getsockopt()` functions allow configuration of socket behavior:

```
#include <sys/socket.h>

int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);

int getsockopt(int sockfd, int level, int optname,
               void *optval, socklen_t *optlen);
```

Common Socket Options

1. **SO_REUSEADDR** - Allow reuse of local addresses
2. **SO_KEEPALIVE** - Enable keep-alive messages
3. **SO_LINGER** - Control behavior on close
4. **TCP_NODELAY** - Disable Nagle's algorithm

Example: Setting Socket Options

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>

int main() {
    int sockfd;
    int opt = 1;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Enable socket reuse
    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) == -1) {
        perror("setsockopt SO_REUSEADDR failed");
        exit(EXIT_FAILURE);
    }
```

```
    // Enable keep-alive
    if (setsockopt(sockfd, SOL_SOCKET, SO_KEEPALIVE, &opt, sizeof(opt)) == -1) {
        perror("setsockopt SO_KEEPALIVE failed");
        exit(EXIT_FAILURE);
    }

    printf("Socket options set successfully\n");

    close(sockfd);
    return 0;
}
```

Error Handling in Socket Programming

Proper error handling is crucial in socket programming due to the unreliable nature of networks.

Common Socket Errors

1. **EACCES** - Permission denied
2. **EADDRINUSE** - Address already in use
3. **ECONNREFUSED** - Connection refused
4. **ETIMEDOUT** - Connection timed out
5. **EHOSTUNREACH** - No route to host

Robust Error Handling Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>

int create_tcp_socket() {
    int sockfd;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        switch (errno) {
            case EACCES:
                fprintf(stderr, "Permission denied to create socket\n");
```

```
                break;
            case EMFILE:
                fprintf(stderr, "Process file descriptor limit reached\n");
                break;
            case ENFILE:
                fprintf(stderr, "System file descriptor limit reached\n");
                break;
            case ENOBUFS:
                fprintf(stderr, "Insufficient buffer space\n");
                break;
            default:
                perror("socket creation failed");
        }
        return -1;
    }

    return sockfd;
}

int main() {
    int sockfd = create_tcp_socket();
    if (sockfd == -1) {
        exit(EXIT_FAILURE);
    }

    printf("Socket created successfully: %d\n", sockfd);
    close(sockfd);
    return 0;
}
```

Practical Examples

Simple Echo Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8080
#define BUFFER_SIZE 1024
```

```
void handle_client(int client_fd) {
    char buffer[BUFFER_SIZE];
    int bytes_read;

    while ((bytes_read = read(client_fd, buffer, BUFFER_SIZE - 1)) > 0) {
        buffer[bytes_read] = '\0';
        printf("Received: %s", buffer);

        // Echo back to client
        write(client_fd, buffer, bytes_read);

        // Exit if client sends "quit"
        if (strncmp(buffer, "quit", 4) == 0) {
            break;
        }
    }

    close(client_fd);
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // Configure address
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
```

```
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Echo server listening on port %d\n", PORT);

    // Accept and handle clients
    while (1) {
        if ((client_fd = accept(server_fd, (struct sockaddr *)&address,
                                (socklen_t*)&addrlen)) < 0) {
            perror("accept");
            continue;
        }

        printf("Client connected\n");
        handle_client(client_fd);
        printf("Client disconnected\n");
    }

    close(server_fd);
    return 0;
}
```

Simple Echo Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
```

```
char buffer[BUFFER_SIZE] = {0};

// Create socket
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    printf("Socket creation error\n");
    return -1;
}

// Configure server address
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);

// Convert IPv4 address
if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
    printf("Invalid address\n");
    return -1;
}

// Connect to server
if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection failed\n");
    return -1;
}

printf("Connected to echo server. Type 'quit' to exit.\n");

// Main loop
while (1) {
    printf("Enter message: ");
    fgets(buffer, BUFFER_SIZE, stdin);

    // Remove newline
    buffer[strcspn(buffer, "\n")] = 0;

    // Send message
    send(sock, buffer, strlen(buffer), 0);

    // Exit if user types "quit"
    if (strcmp(buffer, "quit") == 0) {
        break;
    }

    // Receive echo
    int valread = read(sock, buffer, BUFFER_SIZE);
    buffer[valread] = '\0';
    printf("Echo: %s\n", buffer);
}
```

```
    close(sock);
    return 0;
}
```

Summary

Socket programming in C involves several key concepts:

1. **Socket Creation** - Using `socket()` to create endpoints for communication
2. **Server Programming** - Binding, listening, and accepting connections
3. **Client Programming** - Connecting to servers
4. **TCP vs UDP** - Connection-oriented vs connectionless communication
5. **Socket Options** - Configuring socket behavior with `setsockopt()`
6. **Error Handling** - Properly handling network errors and exceptions

These fundamentals form the basis for building more complex network applications. The next chapter will cover advanced networking concepts such as non-blocking I/O, select/poll, and multiplexing.

Advanced Networking

Introduction

Advanced networking concepts build upon the fundamentals of socket programming to create more sophisticated and efficient network applications. This chapter covers non-blocking I/O, multiplexing with select/poll, concurrent server architectures, and other advanced techniques that are essential for building scalable network applications.

Non-blocking I/O

Non-blocking I/O allows a program to continue execution while waiting for network operations to complete, rather than blocking until the operation finishes.

Setting Non-blocking Mode

There are two main ways to set a socket to non-blocking mode:

1. **Using fcntl()**:

```
#include <fcntl.h>

int flags = fcntl(sockfd, F_GETFL, 0);
fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);
```

2. **Using ioctl()**:

```
#include <sys/ioctl.h>

int nonblocking = 1;
ioctl(sockfd, FIONBIO, &nonblocking);
```

Non-blocking Socket Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```c
#include <fcntl.h>
#include <errno.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
    int sockfd;
    struct sockaddr_in server_addr;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Set socket to non-blocking mode
    int flags = fcntl(sockfd, F_GETFL, 0);
    if (fcntl(sockfd, F_SETFL, flags | O_NONBLOCK) < 0) {
        perror("fcntl failed");
        exit(EXIT_FAILURE);
    }

    // Configure server address
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(80);
    server_addr.sin_addr.s_addr = inet_addr("93.184.216.34"); // example.com

    // Attempt to connect (will likely return immediately with EINPROGRESS)
    int result = connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));

    if (result < 0) {
        if (errno == EINPROGRESS) {
            printf("Connection in progress...\n");
            // Use select() or poll() to wait for connection completion
        } else {
            perror("connect failed");
            close(sockfd);
            exit(EXIT_FAILURE);
        }
    }

    close(sockfd);
    return 0;
}
```

I/O Multiplexing

I/O multiplexing allows a single process to monitor multiple file descriptors, waiting until one or more of them become "ready" for I/O operations.

select()

The `select()` function monitors multiple file descriptors to see if any of them are ready for I/O:

```
#include <sys/select.h>

int select(int nfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

Example: Using select() for Multiple Clients

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX_CLIENTS 30
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket, client_socket[MAX_CLIENTS], max_clients = MAX_CLIENTS;
    int activity, i, valread, sd;
    int max_sd;
    struct sockaddr_in address;

    char buffer[BUFFER_SIZE];

    fd_set readfds;

    // Initialize all client_socket[] to 0 (not valid)
    for (i = 0; i < max_clients; i++) {
        client_socket[i] = 0;
    }

    // Create socket
```

```
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Set master socket to allow multiple connections
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, (char *)&opt, sizeof(opt)) < 0) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}

// Configure address
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind socket
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

printf("Listener on port %d\n", PORT);

// Listen for connections
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

// Accept connections
int addrlen = sizeof(address);
puts("Waiting for connections...");

while (1) {
    // Clear the socket set
    FD_ZERO(&readfds);

    // Add master socket to set
    FD_SET(server_fd, &readfds);
    max_sd = server_fd;

    // Add child sockets to set
    for (i = 0; i < max_clients; i++) {
        // Socket descriptor
        sd = client_socket[i];
```

```
        // If valid socket descriptor then add to read list
        if (sd > 0)
            FD_SET(sd, &readfds);

        // Highest file descriptor number, needed for select()
        if (sd > max_sd)
            max_sd = sd;
    }

    // Wait for an activity on one of the sockets
    activity = select(max_sd + 1, &readfds, NULL, NULL, NULL);

    if ((activity < 0) && (errno != EINTR)) {
        printf("select error");
    }

    // If something happened on the master socket, then it's an incoming connection
    if (FD_ISSET(server_fd, &readfds)) {
        if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&ad
            perror("accept");
            exit(EXIT_FAILURE);
        }

        // Inform user of socket number
        printf("New connection, socket fd is %d, ip is : %s, port : %d\n",
               new_socket, inet_ntoa(address.sin_addr), ntohs(address.sin_port));

        // Add new socket to array of sockets
        for (i = 0; i < max_clients; i++) {
            // If position is empty
            if (client_socket[i] == 0) {
                client_socket[i] = new_socket;
                printf("Adding to list of sockets as %d\n", i);
                break;
            }
        }
    }

    // Else it's some IO operation on some other socket
    for (i = 0; i < max_clients; i++) {
        sd = client_socket[i];

        if (FD_ISSET(sd, &readfds)) {
            // Check if it was for closing, and also read the incoming message
            if ((valread = read(sd, buffer, BUFFER_SIZE)) == 0) {
                // Somebody disconnected, get his details and print
```

```
                    getpeername(sd, (struct sockaddr*)&address, (socklen_t*)&addrlen);
                    printf("Host disconnected, ip %s, port %d\n",
                           inet_ntoa(address.sin_addr), ntohs(address.sin_port));

                    // Close the socket and mark as 0 in list for reuse
                    close(sd);
                    client_socket[i] = 0;
                } else {
                    // Set the string terminating NULL byte on the end of the data read
                    buffer[valread] = '\0';
                    send(sd, buffer, strlen(buffer), 0);
                }
            }
        }
    }

    return 0;
}
```

poll()

The `poll()` function provides similar functionality to `select()` but with some advantages:

```
#include <poll.h>

int poll(struct pollfd *fds, nfds_t nfds, int timeout);
```

Example: Using poll()

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <poll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX_CLIENTS 30
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
```

```
struct sockaddr_in address;
int addrlen = sizeof(address);

struct pollfd fds[MAX_CLIENTS + 1];
int nfds = 1; // Only server socket initially

// Initialize pollfd structure
for (int i = 0; i < MAX_CLIENTS + 1; i++) {
    fds[i].fd = -1;
    fds[i].events = POLLIN;
}

// Create server socket
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Set socket options
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, (char *)&opt, sizeof(opt)) < 0) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}

// Configure address
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind and listen
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

// Add server socket to poll
fds[0].fd = server_fd;
fds[0].events = POLLIN;

printf("Server listening on port %d\n", PORT);
```

```
    while (1) {
        int poll_count = poll(fds, nfds, -1);

        if (poll_count == -1) {
            perror("poll");
            exit(EXIT_FAILURE);
        }

        // Check server socket for new connections
        if (fds[0].revents & POLLIN) {
            if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&ad
                perror("accept");
                exit(EXIT_FAILURE);
            }

            printf("New connection, socket fd is %d\n", new_socket);

            // Add new socket to poll array
            int i;
            for (i = 1; i < MAX_CLIENTS + 1; i++) {
                if (fds[i].fd == -1) {
                    fds[i].fd = new_socket;
                    nfds = (i + 1 > nfds) ? i + 1 : nfds;
                    break;
                }
            }

            if (i == MAX_CLIENTS + 1) {
                printf("Too many clients\n");
                close(new_socket);
            }
        }

        // Check client sockets for data
        for (int i = 1; i < nfds; i++) {
            if (fds[i].fd != -1 && (fds[i].revents & POLLIN)) {
                char buffer[BUFFER_SIZE];
                int valread = read(fds[i].fd, buffer, BUFFER_SIZE);

                if (valread == 0) {
                    // Client disconnected
                    printf("Client disconnected, socket fd %d\n", fds[i].fd);
                    close(fds[i].fd);
                    fds[i].fd = -1;
                } else {
                    // Echo data back
                    buffer[valread] = '\0';
```

```
                    printf("Received: %s", buffer);
                    send(fds[i].fd, buffer, valread, 0);
                }
            }
        }
    }

    return 0;
}
```

Concurrent Server Architectures

Threading Approach

Using threads to handle multiple clients concurrently:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

void *handle_client(void *arg) {
    int client_fd = *(int*)arg;
    char buffer[BUFFER_SIZE];
    int valread;

    // Detach thread to automatically clean up when finished
    pthread_detach(pthread_self());

    // Free the argument (allocated in main)
    free(arg);

    // Handle client communication
    while ((valread = read(client_fd, buffer, BUFFER_SIZE)) > 0) {
        buffer[valread] = '\0';
        printf("Received from client: %s", buffer);

        // Echo back to client
```

```
        send(client_fd, buffer, valread, 0);

        // Exit if client sends "quit"
        if (strncmp(buffer, "quit", 4) == 0) {
            break;
        }
    }

    printf("Client disconnected\n");
    close(client_fd);
    return NULL;
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);

    // Create server socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // Configure address
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 10) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
```

```
    }

    printf("Server listening on port %d\n", PORT);

    while (1) {
        // Accept connection
        if ((client_fd = accept(server_fd, (struct sockaddr *)&address,
                                (socklen_t*)&addrlen)) < 0) {
            perror("accept");
            continue;
        }

        printf("New client connected\n");

        // Create thread to handle client
        pthread_t thread_id;
        int *new_sock = malloc(sizeof(int));
        *new_sock = client_fd;

        if (pthread_create(&thread_id, NULL, handle_client, (void*)new_sock) < 0) {
            perror("could not create thread");
            free(new_sock);
            close(client_fd);
        }
    }

    close(server_fd);
    return 0;
}
```

Forking Approach

Using processes to handle multiple clients:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/wait.h>

#define PORT 8080
#define BUFFER_SIZE 1024
```

```
void handle_client(int client_fd) {
    char buffer[BUFFER_SIZE];
    int valread;

    while ((valread = read(client_fd, buffer, BUFFER_SIZE)) > 0) {
        buffer[valread] = '\0';
        printf("Received from client: %s", buffer);

        // Echo back to client
        send(client_fd, buffer, valread, 0);

        // Exit if client sends "quit"
        if (strncmp(buffer, "quit", 4) == 0) {
            break;
        }
    }

    printf("Client disconnected\n");
    close(client_fd);
    exit(0);
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);

    // Create server socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // Configure address
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
```

```
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 10) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Server listening on port %d\n", PORT);

    while (1) {
        // Accept connection
        if ((client_fd = accept(server_fd, (struct sockaddr *)&address,
                                (socklen_t*)&addrlen)) < 0) {
            perror("accept");
            continue;
        }

        printf("New client connected\n");

        // Fork child process to handle client
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
            close(client_fd);
        } else if (pid == 0) {
            // Child process
            close(server_fd); // Close listening socket in child
            handle_client(client_fd);
        } else {
            // Parent process
            close(client_fd); // Close client socket in parent
            // Clean up zombie processes
            while (waitpid(-1, NULL, WNOHANG) > 0);
        }
    }

    close(server_fd);
    return 0;
}
```

Advanced Socket Options

Socket Buffer Sizes

Adjusting socket buffer sizes can improve performance:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>

int main() {
    int sockfd;
    int buffer_size;
    socklen_t len = sizeof(buffer_size);

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Get current receive buffer size
    if (getsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &buffer_size, &len) < 0) {
        perror("getsockopt failed");
        exit(EXIT_FAILURE);
    }
    printf("Default receive buffer size: %d bytes\n", buffer_size);

    // Get current send buffer size
    if (getsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &buffer_size, &len) < 0) {
        perror("getsockopt failed");
        exit(EXIT_FAILURE);
    }
    printf("Default send buffer size: %d bytes\n", buffer_size);

    // Set new buffer sizes
    buffer_size = 64 * 1024; // 64KB
    if (setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &buffer_size, sizeof(buffer_size)) < 0) {
        perror("setsockopt SO_RCVBUF failed");
        exit(EXIT_FAILURE);
    }

    if (setsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &buffer_size, sizeof(buffer_size)) < 0) {
        perror("setsockopt SO_SNDBUF failed");
        exit(EXIT_FAILURE);
    }
```

```
    printf("New buffer sizes set to %d bytes\n", buffer_size);

    close(sockfd);
    return 0;
}
```

Timeout Options

Setting timeouts for socket operations:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <time.h>

int main() {
    int sockfd;
    struct timeval timeout;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Set receive timeout
    timeout.tv_sec = 5;  // 5 seconds
    timeout.tv_usec = 0;

    if (setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0) {
        perror("setsockopt SO_RCVTIMEO failed");
        exit(EXIT_FAILURE);
    }

    // Set send timeout
    timeout.tv_sec = 3;  // 3 seconds

    if (setsockopt(sockfd, SOL_SOCKET, SO_SNDTIMEO, &timeout, sizeof(timeout)) < 0) {
        perror("setsockopt SO_SNDTIMEO failed");
        exit(EXIT_FAILURE);
    }

    printf("Socket timeouts set successfully\n");
```

```
    close(sockfd);
    return 0;
}
```

Signal-Driven I/O

Signal-driven I/O allows a process to be notified when a socket is ready for I/O operations:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>

int sockfd;

void signal_handler(int sig) {
    if (sig == SIGIO) {
        printf("Socket is ready for I/O\n");
        // Handle I/O operations here
    }
}

int main() {
    struct sockaddr_in address;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Set socket to non-blocking
    int flags = fcntl(sockfd, F_GETFL, 0);
    fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);

    // Set up signal handler
    signal(SIGIO, signal_handler);
```

```
    // Enable asynchronous I/O on socket
    if (fcntl(sockfd, F_SETOWN, getpid()) < 0) {
        perror("fcntl F_SETOWN failed");
        exit(EXIT_FAILURE);
    }

    if (fcntl(sockfd, F_SETFL, flags | O_ASYNC) < 0) {
        perror("fcntl F_SETFL O_ASYNC failed");
        exit(EXIT_FAILURE);
    }

    // Configure address
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_port = htons(80);
    address.sin_addr.s_addr = inet_addr("93.184.216.34"); // example.com

    // Attempt to connect
    connect(sockfd, (struct sockaddr*)&address, sizeof(address));

    // Main loop
    while (1) {
        pause(); // Wait for signals
    }

    close(sockfd);
    return 0;
}
```

Practical Examples

High-Performance Echo Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>

#define PORT 8080
```

```c
#define MAX_CLIENTS 100
#define BUFFER_SIZE 4096

typedef struct {
    int fd;
    char buffer[BUFFER_SIZE];
    int buffer_pos;
    int buffer_len;
} client_t;

int main() {
    int server_fd;
    struct sockaddr_in address;
    client_t clients[MAX_CLIENTS];
    int max_clients = MAX_CLIENTS;
    fd_set read_fds, write_fds;
    int max_fd;

    // Initialize client structures
    for (int i = 0; i < max_clients; i++) {
        clients[i].fd = -1;
        clients[i].buffer_pos = 0;
        clients[i].buffer_len = 0;
    }

    // Create server socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    int opt = 1;
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, (char *)&opt, sizeof(opt)) < 0) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // Configure address
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind and listen
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
```

```
    }

    if (listen(server_fd, 10) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("High-performance echo server listening on port %d\n", PORT);

    while (1) {
        // Clear file descriptor sets
        FD_ZERO(&read_fds);
        FD_ZERO(&write_fds);

        // Add server socket to read set
        FD_SET(server_fd, &read_fds);
        max_fd = server_fd;

        // Add client sockets to appropriate sets
        for (int i = 0; i < max_clients; i++) {
            if (clients[i].fd > 0) {
                FD_SET(clients[i].fd, &read_fds);

                // If we have data to send, add to write set
                if (clients[i].buffer_len > 0) {
                    FD_SET(clients[i].fd, &write_fds);
                }

                if (clients[i].fd > max_fd) {
                    max_fd = clients[i].fd;
                }
            }
        }

        // Wait for activity
        int activity = select(max_fd + 1, &read_fds, &write_fds, NULL, NULL);

        if (activity < 0 && errno != EINTR) {
            perror("select error");
            exit(EXIT_FAILURE);
        }

        // Check for new connections
        if (FD_ISSET(server_fd, &read_fds)) {
            int new_socket;
            int addrlen = sizeof(address);
```

```
        if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&ad
            perror("accept");
            continue;
        }

        // Add new socket to clients array
        int i;
        for (i = 0; i < max_clients; i++) {
            if (clients[i].fd == -1) {
                clients[i].fd = new_socket;
                clients[i].buffer_pos = 0;
                clients[i].buffer_len = 0;
                printf("New connection: socket %d\n", new_socket);
                break;
            }
        }

        if (i == max_clients) {
            printf("Too many clients, closing new connection\n");
            close(new_socket);
        }
    }

    // Check for data from clients
    for (int i = 0; i < max_clients; i++) {
        int sd = clients[i].fd;

        if (sd < 0) continue;

        // Check for data to read
        if (FD_ISSET(sd, &read_fds)) {
            int valread = read(sd, clients[i].buffer + clients[i].buffer_len,
                               BUFFER_SIZE - clients[i].buffer_len);

            if (valread <= 0) {
                // Client disconnected
                getpeername(sd, (struct sockaddr*)&address, (socklen_t*)&addrlen);
                printf("Client disconnected: %s:%d\n",
                       inet_ntoa(address.sin_addr), ntohs(address.sin_port));
                close(sd);
                clients[i].fd = -1;
                clients[i].buffer_pos = 0;
                clients[i].buffer_len = 0;
            } else {
                // Add data to buffer
                clients[i].buffer_len += valread;
```

```
                    printf("Received %d bytes from client\n", valread);
                }
            }

            // Check for ability to write
            if (sd > 0 && FD_ISSET(sd, &write_fds) && clients[i].buffer_len > 0) {
                int to_send = clients[i].buffer_len - clients[i].buffer_pos;
                int sent = write(sd, clients[i].buffer + clients[i].buffer_pos, to_send);

                if (sent < 0) {
                    if (errno != EAGAIN && errno != EWOULDBLOCK) {
                        printf("Write error, closing connection\n");
                        close(sd);
                        clients[i].fd = -1;
                        clients[i].buffer_pos = 0;
                        clients[i].buffer_len = 0;
                    }
                } else {
                    clients[i].buffer_pos += sent;

                    // If we've sent all data, reset buffer
                    if (clients[i].buffer_pos >= clients[i].buffer_len) {
                        clients[i].buffer_pos = 0;
                        clients[i].buffer_len = 0;
                    }
                }
            }
        }
    }

    return 0;
}
```

Summary

Advanced networking concepts in C include:

1. **Non-blocking I/O** - Preventing programs from blocking on network operations
2. **I/O Multiplexing** - Using `select()` and `poll()` to handle multiple connections
3. **Concurrent Servers** - Implementing threading and forking approaches for handling multiple clients
4. **Advanced Socket Options** - Configuring buffer sizes, timeouts, and other socket parameters
5. **Signal-Driven I/O** - Using signals to notify when sockets are ready for I/O

These techniques are essential for building high-performance, scalable network applications. The next chapter will cover specific network protocols and their implementation in C.

Network Protocols

Introduction

Network protocols define the rules and conventions for communication between network devices. Understanding and implementing common network protocols is essential for developing robust network applications. This chapter covers the implementation of widely-used protocols such as HTTP, FTP, DNS, and others in C.

HTTP Protocol Implementation

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the World Wide Web. Understanding HTTP is crucial for developing web applications and services.

HTTP Request Structure

An HTTP request consists of: 1. **Request Line** - Method, URI, and HTTP version 2. **Headers** - Key-value pairs providing metadata 3. **Body** - Optional data (for POST, PUT requests)

Example HTTP GET request:

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Connection: close
```

HTTP Response Structure

An HTTP response consists of: 1. **Status Line** - HTTP version, status code, and reason phrase 2. **Headers** - Key-value pairs providing metadata 3. **Body** - Response content

Example HTTP response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234
Connection: close

<HTML>...</HTML>
```

Simple HTTP Client

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define BUFFER_SIZE 4096

int create_socket(const char *hostname, int port) {
    int sockfd;
    struct hostent *server;
    struct sockaddr_in serv_addr;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("ERROR opening socket");
        return -1;
    }

    // Get server IP address
    server = gethostbyname(hostname);
    if (server == NULL) {
        fprintf(stderr, "ERROR, no such host\n");
        return -1;
    }

    // Configure server address
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    memcpy(server->h_addr, &serv_addr.sin_addr.s_addr, server->h_length);
    serv_addr.sin_port = htons(port);

    // Connect to server
    if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("ERROR connecting");
        return -1;
    }

    return sockfd;
}
```

```c
int http_get(const char *hostname, const char *path) {
    int sockfd;
    char request[BUFFER_SIZE];
    char response[BUFFER_SIZE];
    int bytes_sent, bytes_received;

    // Create socket and connect
    sockfd = create_socket(hostname, 80);
    if (sockfd < 0) {
        return -1;
    }

    // Create HTTP GET request
    snprintf(request, BUFFER_SIZE,
             "GET %s HTTP/1.1\r\n"
             "Host: %s\r\n"
             "Connection: close\r\n"
             "\r\n",
             path, hostname);

    // Send request
    bytes_sent = send(sockfd, request, strlen(request), 0);
    if (bytes_sent < 0) {
        perror("ERROR sending request");
        close(sockfd);
        return -1;
    }

    printf("HTTP Request sent:\n%s", request);

    // Receive response
    while ((bytes_received = recv(sockfd, response, BUFFER_SIZE - 1, 0)) > 0) {
        response[bytes_received] = '\0';
        printf("%s", response);
    }

    if (bytes_received < 0) {
        perror("ERROR receiving response");
        close(sockfd);
        return -1;
    }

    close(sockfd);
    return 0;
}

int main() {
```

```
    const char *hostname = "httpbin.org";
    const char *path = "/get";

    printf("Sending HTTP GET request to %s%s\n", hostname, path);
    http_get(hostname, path);

    return 0;
}
```

Simple HTTP Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8080
#define BUFFER_SIZE 4096

void send_http_response(int client_fd, const char *status, const char *content_type, const c
    char response[BUFFER_SIZE];
    time_t now;
    char time_str[100];

    // Get current time
    time(&now);
    strftime(time_str, sizeof(time_str), "%a, %d %b %Y %H:%M:%S GMT", gmtime(&now));

    // Create HTTP response
    int content_length = strlen(body);
    snprintf(response, BUFFER_SIZE,
             "HTTP/1.1 %s\r\n"
             "Date: %s\r\n"
             "Server: SimpleHTTPServer/1.0\r\n"
             "Content-Type: %s\r\n"
             "Content-Length: %d\r\n"
             "Connection: close\r\n"
             "\r\n"
             "%s",
             status, time_str, content_type, content_length, body);
```

```
    // Send response
    send(client_fd, response, strlen(response), 0);
}

void handle_request(int client_fd, const char *request) {
    printf("Received request:\n%s\n", request);

    // Parse request line (first line)
    char method[16], path[256], version[16];
    if (sscanf(request, "%15s %255s %15s", method, path, version) != 3) {
        send_http_response(client_fd, "400 Bad Request", "text/plain", "Bad Request");
        return;
    }

    // Handle different HTTP methods and paths
    if (strcmp(method, "GET") == 0) {
        if (strcmp(path, "/") == 0 || strcmp(path, "/index.html") == 0) {
            const char *html =
                "<!DOCTYPE html>\n"
                "<html>\n"
                "<head><title>Simple HTTP Server</title></head>\n"
                "<body><h1>Welcome to Simple HTTP Server</h1>\n"
                "<p>This is a simple HTTP server implemented in C.</p>\n"
                "</body>\n"
                "</html>";
            send_http_response(client_fd, "200 OK", "text/html", html);
        } else if (strcmp(path, "/api/time") == 0) {
            time_t now = time(NULL);
            char time_str[100];
            snprintf(time_str, sizeof(time_str), "{\"time\": \"%s\"}", ctime(&now));
            send_http_response(client_fd, "200 OK", "application/json", time_str);
        } else {
            const char *not_found =
                "<!DOCTYPE html>\n"
                "<html>\n"
                "<head><title>404 Not Found</title></head>\n"
                "<body><h1>404 Not Found</h1>\n"
                "<p>The requested resource was not found.</p>\n"
                "</body>\n"
                "</html>";
            send_http_response(client_fd, "404 Not Found", "text/html", not_found);
        }
    } else {
        send_http_response(client_fd, "405 Method Not Allowed", "text/plain", "Method Not Al
    }
}
```

```c
int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // Configure address
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 10) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("HTTP Server listening on port %d\n", PORT);

    while (1) {
        // Accept connection
        if ((client_fd = accept(server_fd, (struct sockaddr *)&address,
                                (socklen_t*)&addrlen)) < 0) {
            perror("accept");
            continue;
        }
```

```
        // Read request
        int bytes_read = read(client_fd, buffer, BUFFER_SIZE - 1);
        if (bytes_read > 0) {
            buffer[bytes_read] = '\0';
            handle_request(client_fd, buffer);
        }

        // Close client connection
        close(client_fd);
    }

    close(server_fd);
    return 0;
}
```

FTP Protocol Implementation

FTP (File Transfer Protocol) is a standard network protocol used for transferring files between a client and server on a computer network.

FTP Client Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define BUFFER_SIZE 1024
#define FTP_PORT 21

int ftp_connect(const char *hostname, int port) {
    int sockfd;
    struct hostent *server;
    struct sockaddr_in serv_addr;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("ERROR opening socket");
        return -1;
```

```
    }

    // Get server IP address
    server = gethostbyname(hostname);
    if (server == NULL) {
        fprintf(stderr, "ERROR, no such host\n");
        return -1;
    }

    // Configure server address
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    memcpy(server->h_addr, &serv_addr.sin_addr.s_addr, server->h_length);
    serv_addr.sin_port = htons(port);

    // Connect to server
    if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("ERROR connecting");
        return -1;
    }

    return sockfd;
}

int ftp_login(int sockfd, const char *username, const char *password) {
    char buffer[BUFFER_SIZE];
    int bytes_received;

    // Read welcome message
    bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("%s", buffer);
    }

    // Send USER command
    snprintf(buffer, BUFFER_SIZE, "USER %s\r\n", username);
    send(sockfd, buffer, strlen(buffer), 0);

    // Read response
    bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("%s", buffer);
    }

    // Send PASS command
```

```
    snprintf(buffer, BUFFER_SIZE, "PASS %s\r\n", password);
    send(sockfd, buffer, strlen(buffer), 0);

    // Read response
    bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("%s", buffer);

        // Check if login was successful
        if (strncmp(buffer, "230", 3) == 0) {
            return 0; // Success
        }
    }

    return -1; // Failure
}

int ftp_list(int sockfd) {
    char buffer[BUFFER_SIZE];

    // Send LIST command
    snprintf(buffer, BUFFER_SIZE, "LIST\r\n");
    send(sockfd, buffer, strlen(buffer), 0);

    // Read response
    int bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("%s", buffer);
        return 0;
    }

    return -1;
}

int main() {
    int sockfd;
    const char *hostname = "ftp.example.com"; // Replace with actual FTP server
    const char *username = "anonymous";
    const char *password = "user@example.com";

    printf("Connecting to FTP server: %s\n", hostname);

    // Connect to FTP server
    sockfd = ftp_connect(hostname, FTP_PORT);
    if (sockfd < 0) {
```

```
        exit(EXIT_FAILURE);
    }

    // Login to FTP server
    if (ftp_login(sockfd, username, password) < 0) {
        printf("FTP login failed\n");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("FTP login successful\n");

    // List directory contents
    printf("Directory listing:\n");
    ftp_list(sockfd);

    // Close connection
    close(sockfd);
    return 0;
}
```

DNS Protocol Implementation

DNS (Domain Name System) translates domain names to IP addresses. Understanding DNS is important for network programming.

Simple DNS Resolver

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

// DNS header structure
struct dns_header {
    unsigned short id;       // identification number
    unsigned short flags;    // flags
    unsigned short qdcount;  // number of question entries
    unsigned short ancount;  // number of answer entries
    unsigned short nscount;  // number of authority entries
```

```
    unsigned short arcount;  // number of resource entries
};

// DNS question structure
struct dns_question {
    unsigned short qtype;
    unsigned short qclass;
};

// Function to convert domain name to DNS format
void dns_format(unsigned char* dns, unsigned char* host) {
    int lock = 0, i;
    strcat((char*)host, ".");

    for(i = 0; i < strlen((char*)host); i++) {
        if(host[i] == '.') {
            *dns++ = i - lock;
            for(; lock < i; lock++) {
                *dns++ = host[lock];
            }
            lock++;
        }
    }
    *dns++ = '\0';
}

int dns_query(const char *hostname) {
    int sockfd;
    struct sockaddr_in dest;
    unsigned char buf[65536];
    struct dns_header *dns = NULL;
    struct dns_question *qinfo = NULL;

    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sockfd < 0) {
        perror("socket creation failed");
        return -1;
    }

    // Configure destination address (Google DNS)
    dest.sin_family = AF_INET;
    dest.sin_port = htons(53);
    dest.sin_addr.s_addr = inet_addr("8.8.8.8");

    // Set up DNS header
    dns = (struct dns_header *)&buf;
```

```
dns->id = (unsigned short)htons(getpid());
dns->flags = htons(0x0100); // Standard query
dns->qdcount = htons(1);    // One question
dns->ancount = 0;
dns->nscount = 0;
dns->arcount = 0;

// Set up question section
unsigned char *qname = (unsigned char*)&buf[sizeof(struct dns_header)];
dns_format(qname, (unsigned char*)hostname);

qinfo = (struct dns_question*)&buf[sizeof(struct dns_header) + strlen((char*)qname) + 1]
qinfo->qtype = htons(1);  // Type A (IPv4 address)
qinfo->qclass = htons(1); // Class IN (Internet)

// Send DNS query
int packet_size = sizeof(struct dns_header) + strlen((char*)qname) + 1 + sizeof(struct d
if (sendto(sockfd, (char*)buf, packet_size, 0, (struct sockaddr*)&dest, sizeof(dest)) <
    perror("sendto failed");
    close(sockfd);
    return -1;
}

// Receive DNS response
socklen_t dest_len = sizeof(dest);
int received_bytes = recvfrom(sockfd, (char*)buf, 65536, 0, (struct sockaddr*)&dest, &de
if (received_bytes < 0) {
    perror("recvfrom failed");
    close(sockfd);
    return -1;
}

// Parse DNS response
dns = (struct dns_header*)buf;
printf("DNS Response:\n");
printf("  ID: %d\n", ntohs(dns->id));
printf("  Questions: %d\n", ntohs(dns->qdcount));
printf("  Answers: %d\n", ntohs(dns->ancount));

// Extract IP addresses from answer section
unsigned char *reader = &buf[sizeof(struct dns_header) + strlen((char*)qname) + 1 + size

for(int i = 0; i < ntohs(dns->ancount); i++) {
    // Skip name (for simplicity, assuming it's a pointer)
    reader += 2; // Skip name pointer
```

```
        unsigned short type = ntohs(*(unsigned short*)reader);
        reader += 2;

        unsigned short class = ntohs(*(unsigned short*)reader);
        reader += 2;

        unsigned int ttl = ntohl(*(unsigned int*)reader);
        reader += 4;

        unsigned short data_len = ntohs(*(unsigned short*)reader);
        reader += 2;

        if (type == 1 && class == 1) { // Type A, Class IN
            unsigned char ip[4];
            for(int j = 0; j < 4; j++) {
                ip[j] = *reader++;
            }
            printf("  IP Address: %d.%d.%d.%d\n", ip[0], ip[1], ip[2], ip[3]);
        } else {
            reader += data_len;
        }
    }

    close(sockfd);
    return 0;
}

int main() {
    const char *hostname = "google.com";

    printf("Resolving hostname: %s\n", hostname);
    dns_query(hostname);

    return 0;
}
```

SMTP Protocol Implementation

SMTP (Simple Mail Transfer Protocol) is used for sending emails. Here's a simple SMTP client implementation:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```c
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define BUFFER_SIZE 1024
#define SMTP_PORT 25

int smtp_connect(const char *smtp_server) {
    int sockfd;
    struct hostent *server;
    struct sockaddr_in serv_addr;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("ERROR opening socket");
        return -1;
    }

    // Get server IP address
    server = gethostbyname(smtp_server);
    if (server == NULL) {
        fprintf(stderr, "ERROR, no such host\n");
        return -1;
    }

    // Configure server address
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    memcpy(server->h_addr, &serv_addr.sin_addr.s_addr, server->h_length);
    serv_addr.sin_port = htons(SMTP_PORT);

    // Connect to server
    if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("ERROR connecting");
        return -1;
    }

    return sockfd;
}

int smtp_send_command(int sockfd, const char *command) {
    char buffer[BUFFER_SIZE];
    int bytes_sent, bytes_received;

    // Send command
```

```
    bytes_sent = send(sockfd, command, strlen(command), 0);
    if (bytes_sent < 0) {
        perror("ERROR sending command");
        return -1;
    }

    // Receive response
    bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Server: %s", buffer);
        return 0;
    }

    return -1;
}

int smtp_send_email(const char *smtp_server, const char *from, const char *to,
                    const char *subject, const char *body) {
    int sockfd;
    char command[BUFFER_SIZE];

    // Connect to SMTP server
    sockfd = smtp_connect(smtp_server);
    if (sockfd < 0) {
        return -1;
    }

    // Read server greeting
    char buffer[BUFFER_SIZE];
    int bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Server: %s", buffer);
    }

    // Send HELO command
    snprintf(command, BUFFER_SIZE, "HELO localhost\r\n");
    smtp_send_command(sockfd, command);

    // Send MAIL FROM command
    snprintf(command, BUFFER_SIZE, "MAIL FROM:<%s>\r\n", from);
    smtp_send_command(sockfd, command);

    // Send RCPT TO command
    snprintf(command, BUFFER_SIZE, "RCPT TO:<%s>\r\n", to);
    smtp_send_command(sockfd, command);
```

```
    // Send DATA command
    snprintf(command, BUFFER_SIZE, "DATA\r\n");
    smtp_send_command(sockfd, command);

    // Send email content
    snprintf(command, BUFFER_SIZE,
             "From: %s\r\n"
             "To: %s\r\n"
             "Subject: %s\r\n"
             "\r\n"
             "%s\r\n"
             ".\r\n",
             from, to, subject, body);
    send(sockfd, command, strlen(command), 0);

    // Read response
    bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Server: %s", buffer);
    }

    // Send QUIT command
    snprintf(command, BUFFER_SIZE, "QUIT\r\n");
    smtp_send_command(sockfd, command);

    close(sockfd);
    return 0;
}

int main() {
    const char *smtp_server = "smtp.example.com"; // Replace with actual SMTP server
    const char *from = "sender@example.com";
    const char *to = "recipient@example.com";
    const char *subject = "Test Email";
    const char *body = "This is a test email sent from a C program.";

    printf("Sending email via SMTP...\n");
    smtp_send_email(smtp_server, from, to, subject, body);

    return 0;
}
```

Practical Examples

Protocol Analyzer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <netinet/ip_icmp.h>

void analyze_ip_packet(const unsigned char *buffer, int size) {
    struct iphdr *iph = (struct iphdr*)buffer;

    printf("IP Header\n");
    printf("   |-Version           : %d\n", (unsigned int)iph->version);
    printf("   |-Internet Header Length : %d DWORDS or %d Bytes\n",
           (unsigned int)iph->ihl, ((unsigned int)(iph->ihl))*4);
    printf("   |-Type Of Service   : %d\n", (unsigned int)iph->tos);
    printf("   |-Total Length      : %d Bytes\n", ntohs(iph->tot_len));
    printf("   |-Identification    : %d\n", ntohs(iph->id));
    printf("   |-Time To Live      : %d\n", (unsigned int)iph->ttl);
    printf("   |-Protocol          : %d\n", (unsigned int)iph->protocol);
    printf("   |-Header Checksum   : %d\n", ntohs(iph->check));
    printf("   |-Source IP         : %s\n", inet_ntoa(*(struct in_addr*)&iph->saddr));
    printf("   |-Destination IP    : %s\n", inet_ntoa(*(struct in_addr*)&iph->daddr));

    // Analyze protocol-specific headers
    switch (iph->protocol) {
        case 6: // TCP
            printf("TCP Protocol\n");
            break;
        case 17: // UDP
            printf("UDP Protocol\n");
            break;
        case 1: // ICMP
            printf("ICMP Protocol\n");
            break;
        default:
            printf("Other Protocol\n");
            break;
```

```
    }
}

int main() {
    int sockfd;
    unsigned char buffer[65536];
    struct sockaddr_in dest;
    socklen_t dest_len = sizeof(dest);

    // Create raw socket (requires root privileges)
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (sockfd < 0) {
        perror("socket creation failed (try running as root)");
        exit(EXIT_FAILURE);
    }

    printf("Protocol Analyzer started...\n");

    // Capture packets
    while (1) {
        int packet_size = recvfrom(sockfd, buffer, 65536, 0, (struct sockaddr*)&dest, &dest_
        if (packet_size < 0) {
            perror("recvfrom failed");
            continue;
        }

        printf("\n----------------------------------------\n");
        analyze_ip_packet(buffer, packet_size);
        printf("----------------------------------------\n");
    }

    close(sockfd);
    return 0;
}
```

Summary

Network protocol implementation in C involves understanding and working with various standard protocols:

1. **HTTP** - The foundation of web communication with request/response patterns
2. **FTP** - File transfer protocol for uploading and downloading files
3. **DNS** - Domain name resolution system
4. **SMTP** - Email transmission protocol
5. **Protocol Analysis** - Examining network packets at the protocol level

Key concepts for protocol implementation: - Understanding protocol message formats and structures - Properly formatting requests and parsing responses - Handling different protocol states and error conditions - Managing connections and data flow - Implementing security considerations

These implementations form the basis for more complex network applications and services. The next chapter will cover network security considerations and best practices.

Network Security

Introduction

Network security is a critical aspect of modern network programming. As networks become more complex and interconnected, ensuring the confidentiality, integrity, and availability of data becomes increasingly important. This chapter covers essential network security concepts, encryption techniques, secure communication protocols, and best practices for developing secure network applications in C.

Security Fundamentals

CIA Triad

The foundation of information security is based on three core principles:

1. **Confidentiality** - Ensuring that information is accessible only to those authorized to have access
2. **Integrity** - Safeguarding the accuracy and completeness of information and processing methods
3. **Availability** - Ensuring that authorized users have access to information and associated assets when required

Common Security Threats

1. **Eavesdropping** - Unauthorized interception of network communications
2. **Man-in-the-Middle (MITM)** - Intercepting and potentially altering communication between two parties
3. **Denial of Service (DoS)** - Overwhelming a system to make it unavailable
4. **Injection Attacks** - Inserting malicious code into applications
5. **Buffer Overflows** - Exploiting memory management vulnerabilities

Cryptography Basics

Cryptography is the practice of securing communication from adversaries. Understanding basic cryptographic concepts is essential for implementing secure network applications.

Symmetric Encryption

Symmetric encryption uses the same key for both encryption and decryption.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Simple XOR cipher (for demonstration only - not secure)
void xor_encrypt_decrypt(char *data, size_t data_len, const char *key, size_t key_len) {
    for (size_t i = 0; i < data_len; i++) {
        data[i] ^= key[i % key_len];
    }
}

int main() {
    char message[] = "This is a secret message";
    char key[] = "mykey";
    size_t message_len = strlen(message);

    printf("Original message: %s\n", message);

    // Encrypt
    xor_encrypt_decrypt(message, message_len, key, strlen(key));
    printf("Encrypted message: ");
    for (size_t i = 0; i < message_len; i++) {
        printf("%02x ", (unsigned char)message[i]);
    }
    printf("\n");

    // Decrypt (XOR is symmetric)
    xor_encrypt_decrypt(message, message_len, key, strlen(key));
    printf("Decrypted message: %s\n", message);

    return 0;
}
```

Hash Functions

Hash functions create fixed-size digests of variable-length input data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Simple hash function (for demonstration only - not cryptographically secure)
```

```c
unsigned int simple_hash(const char *str) {
    unsigned int hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c; // hash * 33 + c
    }

    return hash;
}

int main() {
    const char *message1 = "Hello, World!";
    const char *message2 = "Hello, World!";
    const char *message3 = "Hello, world!";

    printf("Hash of '%s': 0x%08x\n", message1, simple_hash(message1));
    printf("Hash of '%s': 0x%08x\n", message2, simple_hash(message2));
    printf("Hash of '%s': 0x%08x\n", message3, simple_hash(message3));

    return 0;
}
```

Secure Socket Layer (SSL/TLS)

SSL/TLS provides secure communication over computer networks. OpenSSL is a widely-used library for implementing SSL/TLS in C applications.

Installing OpenSSL

```
# Ubuntu/Debian
sudo apt-get install libssl-dev

# macOS (with Homebrew)
brew install openssl

# CentOS/RHEL
sudo yum install openssl-devel
```

Simple HTTPS Client with OpenSSL

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <openssl/ssl.h>
#include <openssl/err.h>

#define BUFFER_SIZE 4096

int create_socket(const char *hostname, int port) {
    int sockfd;
    struct hostent *server;
    struct sockaddr_in serv_addr;

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("ERROR opening socket");
        return -1;
    }

    // Get server IP address
    server = gethostbyname(hostname);
    if (server == NULL) {
        fprintf(stderr, "ERROR, no such host\n");
        return -1;
    }

    // Configure server address
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    memcpy(server->h_addr, &serv_addr.sin_addr.s_addr, server->h_length);
    serv_addr.sin_port = htons(port);

    // Connect to server
    if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("ERROR connecting");
        return -1;
    }

    return sockfd;
```

```
}

SSL_CTX* init_ssl_ctx() {
    const SSL_METHOD *method;
    SSL_CTX *ctx;

    // Initialize OpenSSL
    SSL_library_init();
    SSL_load_error_strings();
    OpenSSL_add_all_algorithms();

    // Create SSL method
    method = TLS_client_method();

    // Create SSL context
    ctx = SSL_CTX_new(method);
    if (!ctx) {
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    return ctx;
}

int https_get(const char *hostname, const char *path) {
    int sockfd;
    SSL_CTX *ctx;
    SSL *ssl;
    char request[BUFFER_SIZE];
    char response[BUFFER_SIZE];
    int bytes_sent, bytes_received;

    // Initialize SSL context
    ctx = init_ssl_ctx();

    // Create socket and connect
    sockfd = create_socket(hostname, 443);
    if (sockfd < 0) {
        SSL_CTX_free(ctx);
        return -1;
    }

    // Create SSL structure
    ssl = SSL_new(ctx);
    SSL_set_fd(ssl, sockfd);

    // Perform SSL handshake
```

```
    if (SSL_connect(ssl) <= 0) {
        ERR_print_errors_fp(stderr);
        SSL_free(ssl);
        close(sockfd);
        SSL_CTX_free(ctx);
        return -1;
    }

    printf("SSL connection established\n");
    printf("SSL version: %s\n", SSL_get_version(ssl));
    printf("Cipher: %s\n", SSL_get_cipher(ssl));

    // Create HTTPS GET request
    snprintf(request, BUFFER_SIZE,
             "GET %s HTTP/1.1\r\n"
             "Host: %s\r\n"
             "Connection: close\r\n"
             "\r\n",
             path, hostname);

    // Send request over SSL
    bytes_sent = SSL_write(ssl, request, strlen(request));
    if (bytes_sent <= 0) {
        ERR_print_errors_fp(stderr);
        SSL_free(ssl);
        close(sockfd);
        SSL_CTX_free(ctx);
        return -1;
    }

    printf("HTTPS Request sent:\n%s", request);

    // Receive response over SSL
    while ((bytes_received = SSL_read(ssl, response, BUFFER_SIZE - 1)) > 0) {
        response[bytes_received] = '\0';
        printf("%s", response);
    }

    if (bytes_received < 0) {
        ERR_print_errors_fp(stderr);
        SSL_free(ssl);
        close(sockfd);
        SSL_CTX_free(ctx);
        return -1;
    }

    // Cleanup
```

```
    SSL_free(ssl);
    close(sockfd);
    SSL_CTX_free(ctx);

    return 0;
}

int main() {
    const char *hostname = "httpbin.org";
    const char *path = "/get";

    printf("Sending HTTPS GET request to %s%s\n", hostname, path);
    https_get(hostname, path);

    return 0;
}
```

Authentication and Authorization

Authentication verifies the identity of users, while authorization determines what resources they can access.

Simple Authentication System

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <crypt.h>

#define MAX_USERS 100
#define USERNAME_MAX 32
#define PASSWORD_MAX 128

typedef struct {
    char username[USERNAME_MAX];
    char password_hash[PASSWORD_MAX];
} User;

User users[MAX_USERS];
int user_count = 0;

// Add a new user
int add_user(const char *username, const char *password) {
```

```
    if (user_count >= MAX_USERS) {
        return -1;
    }

    // Hash the password (using crypt - not recommended for production)
    char *salt = "$6$salt$"; // SHA-512 with salt "salt"
    char *hash = crypt(password, salt);

    if (hash == NULL) {
        return -1;
    }

    strncpy(users[user_count].username, username, USERNAME_MAX - 1);
    strncpy(users[user_count].password_hash, hash, PASSWORD_MAX - 1);
    user_count++;

    return 0;
}

// Authenticate a user
int authenticate(const char *username, const char *password) {
    for (int i = 0; i < user_count; i++) {
        if (strcmp(users[i].username, username) == 0) {
            char *salt = users[i].password_hash;
            char *hash = crypt(password, salt);

            if (hash != NULL && strcmp(hash, users[i].password_hash) == 0) {
                return 1; // Authentication successful
            }
            return 0; // Wrong password
        }
    }
    return -1; // User not found
}

int main() {
    // Add some users
    add_user("admin", "admin123");
    add_user("user1", "password1");
    add_user("user2", "password2");

    // Test authentication
    const char *test_users[] = {"admin", "user1", "unknown"};
    const char *test_passwords[] = {"admin123", "wrongpass", "password1"};

    for (int i = 0; i < 3; i++) {
```

```
        printf("Authenticating user '%s' with password '%s': ",
               test_users[i], test_passwords[i]);

        int result = authenticate(test_users[i], test_passwords[i]);
        switch (result) {
            case 1:
                printf("SUCCESS\n");
                break;
            case 0:
                printf("FAILED (wrong password)\n");
                break;
            case -1:
                printf("FAILED (user not found)\n");
                break;
        }
    }

    return 0;
}
```

Secure Coding Practices

Input Validation

Proper input validation is crucial for preventing injection attacks and buffer overflows.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

// Validate email address format
int validate_email(const char *email) {
    if (email == NULL) return 0;

    int len = strlen(email);
    if (len == 0 || len > 254) return 0;

    // Check for @ symbol
    char *at = strchr(email, '@');
    if (at == NULL || at == email || at == email + len - 1) return 0;

    // Check for domain part
    char *domain = at + 1;
    if (strlen(domain) == 0) return 0;
```

```
    // Check for valid characters
    for (int i = 0; i < len; i++) {
        if (!isalnum(email[i]) && email[i] != '@' && email[i] != '.' &&
            email[i] != '-' && email[i] != '_') {
            return 0;
        }
    }

    return 1;
}

// Safe string copy with bounds checking
char* safe_strncpy(char *dest, const char *src, size_t dest_size) {
    if (dest == NULL || src == NULL || dest_size == 0) {
        return NULL;
    }

    size_t src_len = strlen(src);
    size_t copy_len = (src_len < dest_size - 1) ? src_len : dest_size - 1;

    strncpy(dest, src, copy_len);
    dest[copy_len] = '\0';

    return dest;
}

int main() {
    // Test email validation
    const char *emails[] = {
        "user@example.com",
        "invalid.email",
        "@example.com",
        "user@",
        "user@.com",
        "valid.email@domain.co.uk"
    };

    for (int i = 0; i < 6; i++) {
        printf("Email '%s' is %s\n",
               emails[i],
               validate_email(emails[i]) ? "VALID" : "INVALID");
    }

    // Test safe string copy
    char dest[10];
    const char *src = "This is a very long string";
```

```
    printf("\nOriginal source: %s\n", src);
    if (safe_strncpy(dest, src, sizeof(dest)) != NULL) {
        printf("Safe copy result: %s\n", dest);
    } else {
        printf("Safe copy failed\n");
    }

    return 0;
}
```

Buffer Overflow Prevention

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Vulnerable function (DO NOT USE)
void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input); // Buffer overflow risk
    printf("Buffer content: %s\n", buffer);
}

// Secure function
void secure_function(const char *input) {
    char buffer[10];
    // Use strncpy with proper bounds checking
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0'; // Ensure null termination
    printf("Buffer content: %s\n", buffer);
}

// Even more secure using snprintf
void very_secure_function(const char *input) {
    char buffer[10];
    // Use snprintf for guaranteed bounds checking
    snprintf(buffer, sizeof(buffer), "%s", input);
    printf("Buffer content: %s\n", buffer);
}

int main() {
    const char *safe_input = "Hello";
    const char *unsafe_input = "This is a very long string that will cause buffer overflow";

    printf("Testing secure function with safe input:\n");
```

```
    secure_function(safe_input);

    printf("\nTesting secure function with unsafe input:\n");
    secure_function(unsafe_input);

    printf("\nTesting very secure function with unsafe input:\n");
    very_secure_function(unsafe_input);

    return 0;
}
```

Firewall and Access Control

Simple Packet Filter

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_RULES 100

typedef enum {
    ACTION_ALLOW,
    ACTION_DENY
} action_t;

typedef enum {
    PROTO_TCP,
    PROTO_UDP,
    PROTO_ANY
} protocol_t;

typedef struct {
    action_t action;
    protocol_t protocol;
    unsigned int src_ip;
    unsigned int dst_ip;
    unsigned short src_port;
    unsigned short dst_port;
} rule_t;

rule_t rules[MAX_RULES];
int rule_count = 0;

// Add a firewall rule
```

```
int add_rule(action_t action, protocol_t protocol,
             const char *src_ip, const char *dst_ip,
             unsigned short src_port, unsigned short dst_port) {
    if (rule_count >= MAX_RULES) {
        return -1;
    }

    rules[rule_count].action = action;
    rules[rule_count].protocol = protocol;
    // In a real implementation, convert IP strings to network byte order integers
    rules[rule_count].src_ip = 0; // Simplified
    rules[rule_count].dst_ip = 0; // Simplified
    rules[rule_count].src_port = src_port;
    rules[rule_count].dst_port = dst_port;

    rule_count++;
    return 0;
}

// Check if a packet should be allowed
int check_packet(protocol_t protocol, unsigned short src_port, unsigned short dst_port) {
    // Check rules in order (first match wins)
    for (int i = 0; i < rule_count; i++) {
        // Simplified matching - in reality, would check IP addresses too
        if ((rules[i].protocol == PROTO_ANY || rules[i].protocol == protocol) &&
            (rules[i].src_port == 0 || rules[i].src_port == src_port) &&
            (rules[i].dst_port == 0 || rules[i].dst_port == dst_port)) {

            return (rules[i].action == ACTION_ALLOW) ? 1 : 0;
        }
    }

    // Default deny
    return 0;
}

int main() {
    // Add some firewall rules
    add_rule(ACTION_DENY, PROTO_TCP, NULL, NULL, 0, 23);  // Deny Telnet
    add_rule(ACTION_ALLOW, PROTO_TCP, NULL, NULL, 0, 80); // Allow HTTP
    add_rule(ACTION_ALLOW, PROTO_TCP, NULL, NULL, 0, 443); // Allow HTTPS
    add_rule(ACTION_DENY, PROTO_ANY, NULL, NULL, 0, 0);   // Deny everything else

    // Test packets
    struct {
        protocol_t protocol;
```

```
        unsigned short src_port;
        unsigned short dst_port;
        const char *description;
    } test_packets[] = {
        {PROTO_TCP, 12345, 80, "HTTP request"},
        {PROTO_TCP, 12345, 443, "HTTPS request"},
        {PROTO_TCP, 12345, 23, "Telnet connection"},
        {PROTO_UDP, 12345, 53, "DNS query"}
    };

    for (int i = 0; i < 4; i++) {
        int allowed = check_packet(test_packets[i].protocol,
                                   test_packets[i].src_port,
                                   test_packets[i].dst_port);

        printf("Packet: %s -> %s\n",
               test_packets[i].description,
               allowed ? "ALLOWED" : "DENIED");
    }

    return 0;
}
```

Secure Communication Patterns

Mutual Authentication

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Simple mutual authentication protocol
typedef struct {
    char client_id[32];
    char server_id[32];
    unsigned char nonce[16]; // Random number
    unsigned char session_key[32];
} auth_context_t;

// Generate random nonce (simplified)
void generate_nonce(unsigned char *nonce, size_t len) {
    // In a real implementation, use a cryptographically secure random number generator
    for (size_t i = 0; i < len; i++) {
        nonce[i] = rand() % 256;
    }
```

```
}

// Simple HMAC-like function (for demonstration only)
void simple_hmac(const char *key, const char *message, unsigned char *digest) {
    // In a real implementation, use a proper HMAC function
    unsigned int hash = 5381;
    const char *ptr = key;

    while (*ptr) {
        hash = ((hash << 5) + hash) + *ptr++;
    }

    ptr = message;
    while (*ptr) {
        hash = ((hash << 5) + hash) + *ptr++;
    }

    // Convert to byte array
    for (int i = 0; i < 4; i++) {
        digest[i] = (hash >> (i * 8)) & 0xFF;
    }
}

int mutual_authenticate_client(const char *client_id, const char *server_id,
                               const char *client_secret, const char *server_secret) {
    auth_context_t ctx;
    unsigned char client_hmac[16];
    unsigned char server_hmac[16];

    // Initialize context
    strncpy(ctx.client_id, client_id, sizeof(ctx.client_id) - 1);
    strncpy(ctx.server_id, server_id, sizeof(ctx.server_id) - 1);
    generate_nonce(ctx.nonce, sizeof(ctx.nonce));

    printf("Client: Starting mutual authentication with server %s\n", server_id);

    // Client sends client_id and nonce to server
    printf("Client: Sending client_id='%s' and nonce\n", client_id);

    // Server responds with its own nonce and HMAC
    generate_nonce(ctx.nonce, sizeof(ctx.nonce)); // Server's nonce
    simple_hmac(server_secret, ctx.client_id, server_hmac);

    printf("Server: Sending nonce and HMAC\n");

    // Client verifies server's HMAC and sends its own HMAC
    simple_hmac(client_secret, ctx.server_id, client_hmac);
```

```
    // Server verifies client's HMAC
    printf("Server: Verifying client HMAC\n");
    printf("Client: Verifying server HMAC\n");

    // If both verifications pass, establish session key
    generate_nonce(ctx.session_key, sizeof(ctx.session_key));
    printf("Mutual authentication successful. Session key established.\n");

    return 0;
}

int main() {
    srand(time(NULL));

    const char *client_id = "client123";
    const char *server_id = "server456";
    const char *client_secret = "client_secret_key";
    const char *server_secret = "server_secret_key";

    printf("Starting mutual authentication...\n");
    mutual_authenticate_client(client_id, server_id, client_secret, server_secret);

    return 0;
}
```

Practical Examples

Secure File Transfer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <openssl/ssl.h>
#include <openssl/err.h>

#define BUFFER_SIZE 4096
#define PORT 8443

// Simple file encryption function
int encrypt_file(const char *input_file, const char *output_file, const char *key) {
```

```
    FILE *in_fp, *out_fp;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;
    size_t key_len = strlen(key);
    size_t key_index = 0;

    in_fp = fopen(input_file, "rb");
    if (!in_fp) {
        perror("Error opening input file");
        return -1;
    }

    out_fp = fopen(output_file, "wb");
    if (!out_fp) {
        perror("Error opening output file");
        fclose(in_fp);
        return -1;
    }

    // XOR encryption (for demonstration only)
    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, in_fp)) > 0) {
        for (size_t i = 0; i < bytes_read; i++) {
            buffer[i] ^= key[key_index++ % key_len];
        }
        fwrite(buffer, 1, bytes_read, out_fp);
    }

    fclose(in_fp);
    fclose(out_fp);

    return 0;
}

// Secure file transfer server
int secure_file_server() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);

    // Create server socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
```

```
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // Configure address
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Secure file server listening on port %d\n", PORT);

    while (1) {
        // Accept connection
        if ((client_fd = accept(server_fd, (struct sockaddr *)&address,
                                (socklen_t*)&addrlen)) < 0) {
            perror("accept");
            continue;
        }

        printf("Client connected\n");

        // In a real implementation, would establish SSL connection here
        // and transfer encrypted files

        close(client_fd);
    }

    close(server_fd);
    return 0;
}

int main() {
```

```
    printf("Network Security Examples\n");
    printf("========================\n\n");

    // Example: File encryption
    const char *test_file = "test.txt";
    const char *encrypted_file = "test.encrypted";
    const char *key = "my_secret_key";

    // Create a test file
    FILE *fp = fopen(test_file, "w");
    if (fp) {
        fprintf(fp, "This is a secret file that needs to be encrypted.\n");
        fprintf(fp, "It contains sensitive information.\n");
        fclose(fp);

        printf("Created test file: %s\n", test_file);

        // Encrypt the file
        if (encrypt_file(test_file, encrypted_file, key) == 0) {
            printf("File encrypted successfully: %s\n", encrypted_file);
        } else {
            printf("File encryption failed\n");
        }
    }

    return 0;
}
```

Summary

Network security in C programming involves several critical aspects:

1. **Cryptography Fundamentals** - Understanding encryption, hashing, and secure key management
2. **SSL/TLS Implementation** - Using OpenSSL for secure communication
3. **Authentication and Authorization** - Implementing secure user verification systems
4. **Secure Coding Practices** - Preventing common vulnerabilities like buffer overflows
5. **Firewall and Access Control** - Implementing network-level security controls
6. **Secure Communication Patterns** - Establishing trusted communication channels

Key principles for secure network programming: - Always validate and sanitize input - Use established cryptographic libraries rather than implementing your own - Implement proper error handling without revealing sensitive information - Follow the principle of least privilege - Keep software and libraries up to date - Use secure communication protocols (HTTPS, SSL/TLS) - Implement proper logging and monitoring

These security practices are essential for developing robust and trustworthy network applications that can withstand modern security threats.

Module 11: Network Programming

Overview

Network programming is a critical skill for developing modern applications that communicate over networks. This module covers the fundamentals of network programming in C, from basic concepts to advanced techniques. You'll learn how to create network applications that can communicate over local networks and the internet.

Learning Objectives

By the end of this module, you should be able to: - Understand fundamental networking concepts and protocols - Implement client-server applications using sockets - Work with different network protocols (TCP, UDP) - Handle network errors and implement robust communication - Implement advanced networking features like non-blocking I/O - Understand network security considerations

Chapters

1. **Network Fundamentals** - Basic networking concepts, OSI model, IP addressing, and network protocols
2. **Socket Programming** - Creating sockets, binding, connecting, and basic client-server communication
3. **Advanced Networking** - Non-blocking I/O, select/poll, multiplexing, and advanced socket options
4. **Network Protocols** - HTTP, FTP, DNS, and other common protocols implementation
5. **Network Security** - Secure communication, encryption, and network security best practices

Prerequisites

Before starting this module, you should have a solid understanding of: - C programming fundamentals - Pointers and memory management - File I/O operations - Basic understanding of computer networks

Key Concepts

- OSI and TCP/IP models
- IP addressing and subnetting
- Socket programming fundamentals
- Client-server architecture
- TCP and UDP protocols
- Network error handling
- Concurrent network programming
- Network security principles

Embedded

Embedded Basics

Introduction

Embedded systems are specialized computing systems designed to perform dedicated functions within larger mechanical or electrical systems. Unlike general-purpose computers, they are often resource-constrained and optimized for specific tasks. This chapter builds C-focused foundations you can practice on a Linux host: bit helpers, packed structs, `volatile`, register overlays over a buffer, a button-debounce state machine, and high-level cross-compile notes for `arm-none-eabi-gcc`.

Many examples are **host simulations** (plain `gcc` on Linux). They teach patterns you will use on bare metal without requiring a board.

Host compile convention:

```
gcc -std=c17 -Wall -Wextra -o program program.c
./program
```

What Are Embedded Systems?

An embedded system is a computer with a dedicated function inside a larger product, often with real-time constraints.

Characteristics

1. **Dedicated function** — not a general-purpose desktop OS workload
2. **Resource constraints** — limited RAM, flash, CPU, energy
3. **Real-time operation** — deadlines matter
4. **Reliability** — long uptime expectations
5. **Cost sensitivity** — BOM-driven design
6. **Power efficiency** — batteries and thermal limits

Types (rough scale)

Scale	Typical hardware	Examples
Small	8/16-bit MCU, KB of RAM	sensors, toys, simple appliances
Medium	32-bit MCU, tens–hundreds of KB	motor control, IoT nodes
Sophisticated	MPU + OS (Linux/RTOS), MB+	routers, automotive domains, phones

Microcontroller vs Microprocessor

Microcontroller (MCU): CPU + memory + peripherals on one chip; minimal external parts.

Microprocessor (MPU): CPU core; RAM, flash, and many peripherals are external; higher performance, more complex board design.

Typical Components

- Processor (CPU)
- Memory (Flash/ROM, RAM, sometimes EEPROM)
- GPIO and serial (UART, SPI, I²C)
- ADC/DAC, timers/counters
- Interrupt controller, clock tree, power management

Development Environments and Cross-Compilation

Embedded builds usually **cross-compile**: the compiler runs on a host (e.g. x86_64 Linux) and emits code for a different target (e.g. ARM Cortex-M).

High-Level Toolchain Notes (`arm-none-eabi-gcc`)

Piece	Role
`arm-none-eabi-gcc`	C compiler for bare-metal ARM (no Linux userspace ABI)
`arm-none-eabi-as` / `ld`	Assembler / linker (often driven by gcc)
`arm-none-eabi-objcopy`	ELF → binary/ihex for flashing
`arm-none-eabi-objdump` / `size`	Inspect sections and code size
`arm-none-eabi-gdb`	Debug via probe (OpenOCD, J-Link, etc.)

Piece	Role
Linker script (`.ld`)	Places `.text`, `.data`, `.bss`, stack at real addresses
Startup (`.s` / `.c`)	Reset handler, copy `.data`, zero `.bss`, call `main`

Typical flags (illustrative Cortex-M3):

```
# Install on Debian/Ubuntu-style hosts (example package name):
# sudo apt install gcc-arm-none-eabi binutils-arm-none-eabi

arm-none-eabi-gcc -std=c17 -Wall -Wextra \
  -mcpu=cortex-m3 -mthumb \
  -ffunction-sections -fdata-sections \
  -T linker_script.ld \
  -nostartfiles \
  -o firmware.elf startup.c main.c

arm-none-eabi-objcopy -O binary firmware.elf firmware.bin
arm-none-eabi-size firmware.elf
```

What those flags mean (high level):

- `-mcpu=cortex-m3 -mthumb` — generate Thumb code for that core
- `-ffunction-sections -fdata-sections` — enable linker garbage collection of unused sections (with `-Wl,--gc-sections`)
- `-T linker_script.ld` — memory map for flash/RAM
- `-nostartfiles` — you supply startup (or use a vendor CRT carefully)

You do **not** need this toolchain for the host simulation programs below; use native `gcc` on Linux for those.

Other Tools You Will Meet

- IDEs: vendor tools, VS Code + Cortex-Debug, CLion, etc.
- Debug probes: SWD/JTAG
- Simulators: QEMU (machine-dependent), Renode

Memory Organization

Region	Typical use
Flash / ROM	Code (`.text`), constants (`.rodata`), initializers
RAM	`.data` (copied at boot), `.bss` (zeroed), heap, stack
EEPROM / NVM	Configuration that survives power loss
MMIO registers	Peripheral control/status (volatile)

```
/* Conceptual Cortex-M style map (numbers vary by chip) */
/* 0x00000000  code flash
 * 0x20000000  SRAM
 * 0x40000000  peripherals
 * 0xE0000000  system (NVIC, SysTick, ...)
 */
```

Startup Sketch (bare metal idea)

```
/* Conceptual only - addresses come from the linker script */
extern unsigned int _etext, _data, _edata, _bss, _ebss;

void Reset_Handler(void) {
    unsigned int *src = &_etext;
    unsigned int *dst = &_data;
    while (dst < &_edata) {
        *dst++ = *src++;
    }
    dst = &_bss;
    while (dst < &_ebss) {
        *dst++ = 0;
    }
    main();
    for (;;) {
        /* halt if main returns */
    }
}
```

Section Placement Reminders

```
int initialized_var = 42;     /* .data */
int uninitialized_var;        /* .bss */
const int constant_var = 100; /* .rodata */
```

```
void example_function(void) {
    static int static_var = 10; /* .data */
    static int static_uninit;   /* .bss */
}
```

Bit Manipulation Helpers

Bit ops are daily drivers for GPIO, flags, and control registers. Prefer unsigned types and `1U` shifts.

```
/* bit_helpers.c - host-runnable */
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

static inline uint32_t bit_mask(unsigned bit) {
    return 1u << bit;
}

static inline void set_bit(uint32_t *reg, unsigned bit) {
    *reg |= bit_mask(bit);
}

static inline void clear_bit(uint32_t *reg, unsigned bit) {
    *reg &= ~bit_mask(bit);
}

static inline void toggle_bit(uint32_t *reg, unsigned bit) {
    *reg ^= bit_mask(bit);
}

static inline bool test_bit(uint32_t reg, unsigned bit) {
    return (reg & bit_mask(bit)) != 0u;
}

/* Multi-bit field helpers (field in [lsb, lsb+width)) */
static inline void write_field(uint32_t *reg, unsigned lsb, unsigned width,
                               uint32_t value) {
    uint32_t mask = ((1u << width) - 1u) << lsb;
    *reg = (*reg & ~mask) | ((value << lsb) & mask);
}

static inline uint32_t read_field(uint32_t reg, unsigned lsb, unsigned width) {
```

```
    return (reg >> lsb) & ((1u << width) - 1u);
}

int main(void) {
    uint32_t gpio = 0;

    set_bit(&gpio, 3);
    set_bit(&gpio, 1);
    printf("after set 3 and 1: 0x%08X\n", gpio);

    clear_bit(&gpio, 1);
    printf("after clear 1:     0x%08X\n", gpio);

    toggle_bit(&gpio, 3);
    toggle_bit(&gpio, 3);
    printf("after toggle 3 x2: 0x%08X (bit3=%d)\n",
           gpio, test_bit(gpio, 3));

    /* e.g. mode field bits [5:4] = 0b10 */
    write_field(&gpio, 4, 2, 0x2);
    printf("field[5:4]=%u reg=0x%08X\n",
           read_field(gpio, 4, 2), gpio);

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o bit_helpers bit_helpers.c
./bit_helpers
```

Macro form (common in headers; use carefully with side-effect arguments):

```
#define SET_BIT(reg, bit)    ((reg) |=  (1U << (bit)))
#define CLEAR_BIT(reg, bit)  ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit) ((reg) ^=  (1U << (bit)))
#define CHECK_BIT(reg, bit)  (((reg) >> (bit)) & 1U)
```

Shift safety: shifting by width of the type is undefined. Keep `bit < 32` for `uint32_t`.

`volatile` — Why Embedded Needs It

The compiler may cache memory values in registers or remove "useless" reads/writes. Memory-mapped registers and variables shared with ISRs **change outside the abstract machine**. Mark them `volatile` so each access in the source becomes a real access.

```c
/* volatile_demo.c */
#include <stdio.h>
#include <stdint.h>

/* Simulated peripheral register (on bare metal this would be MMIO) */
static volatile uint32_t FAKE_UART_STATUS;
static volatile uint32_t FAKE_UART_DATA;

#define STATUS_RX_FULL (1u << 0)

static void hw_sim_inject(char c) {
    FAKE_UART_DATA = (uint32_t)(unsigned char)c;
    FAKE_UART_STATUS |= STATUS_RX_FULL;
}

static char uart_getc_blocking(void) {
    /* Without volatile on STATUS, an optimizer could read once and loop forever */
    while ((FAKE_UART_STATUS & STATUS_RX_FULL) == 0u) {
        /* spin - in real chips an IRQ or WFI would be better */
    }
    char c = (char)FAKE_UART_DATA;
    FAKE_UART_STATUS &= ~STATUS_RX_FULL;
    return c;
}

int main(void) {
    hw_sim_inject('Z');
    char c = uart_getc_blocking();
    printf("received: %c\n", c);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o volatile_demo volatile_demo.c
./volatile_demo
```

Rules of thumb:

- MMIO registers: `volatile` (often via pointer typedef).
- Flags set in an ISR, read in main: `volatile`.
- `volatile` is **not** a substitute for mutexes on multi-core or for atomic RMW correctness across threads — it only limits *compiler* optimizations on that object.

Packed Structs

Compilers insert padding for alignment. Hardware register maps and wire formats often need **exact** layouts. Use packing attributes carefully (unaligned access cost/faults on some CPUs).

```
/* packed_structs.c */
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>

struct Loose {
    uint8_t  a;
    uint32_t b;
    uint8_t  c;
};

struct Packed {
    uint8_t  a;
    uint32_t b;
    uint8_t  c;
} __attribute__((packed));

/* Bit-fields: handy for docs; packing/order is implementation-defined - verify */
struct StatusBits {
    uint8_t ready : 1;
    uint8_t error : 1;
    uint8_t mode  : 2;
    uint8_t       : 4; /* padding bits */
};

int main(void) {
    printf("sizeof(Loose)  = %zu\n", sizeof(struct Loose));
    printf("sizeof(Packed) = %zu\n", sizeof(struct Packed));
    printf("offset b Loose  = %zu\n", offsetof(struct Loose, b));
    printf("offset b Packed = %zu\n", offsetof(struct Packed, b));

    struct Packed p = { .a = 1, .b = 0x11223344u, .c = 0xFE };
    const unsigned char *bytes = (const unsigned char *)&p;
    printf("Packed bytes:");
    for (size_t i = 0; i < sizeof p; i++) {
        printf(" %02X", bytes[i]);
    }
    printf("\n");

    struct StatusBits st = { .ready = 1, .error = 0, .mode = 3 };
    printf("status ready=%u error=%u mode=%u\n", st.ready, st.error, st.mode);
    return 0;
```

```
}
```

```
gcc -std=c17 -Wall -Wextra -o packed_structs packed_structs.c
./packed_structs
```

On a typical 64-bit Linux host, `Loose` is larger than 6 bytes due to padding; `Packed` is 6 bytes. Always check with `sizeof` / `offsetof` for your ABI.

Memory-Mapped Register Pattern as Struct Overlay

On hardware, a peripheral's registers sit at a fixed address. In C you often overlay a `struct` on that address. On the host, overlay the same struct on a **byte buffer** to practice without silicon.

```
/* mmio_overlay.c - simulate a tiny GPIO block in a buffer */
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdbool.h>

typedef struct {
    volatile uint32_t DIR;    /* 0x00: 1 = output */
    volatile uint32_t DATA;   /* 0x04: pin levels */
    volatile uint32_t SET;    /* 0x08: write 1 to set bit in DATA */
    volatile uint32_t CLEAR;  /* 0x0C: write 1 to clear bit in DATA */
} GpioBlock;

/* Host "device memory" */
static uint8_t device_mem[sizeof(GpioBlock)];

static GpioBlock *gpio_at_buffer(void) {
    return (GpioBlock *)(void *)device_mem;
}

/* Emulate SET/CLEAR side effects a real peripheral would do in hardware */
static void gpio_bus_write(GpioBlock *g, volatile uint32_t *reg, uint32_t value) {
    if (reg == &g->SET) {
        g->DATA |= value;
    } else if (reg == &g->CLEAR) {
        g->DATA &= ~value;
    } else {
        *reg = value;
    }
}
```

```
static void gpio_init_outputs(GpioBlock *g, uint32_t mask) {
    gpio_bus_write(g, &g->DIR, g->DIR | mask);
}

static void gpio_set_pins(GpioBlock *g, uint32_t mask) {
    gpio_bus_write(g, &g->SET, mask);
}

static void gpio_clear_pins(GpioBlock *g, uint32_t mask) {
    gpio_bus_write(g, &g->CLEAR, mask);
}

int main(void) {
    memset(device_mem, 0, sizeof device_mem);
    GpioBlock *gpio = gpio_at_buffer();

    const uint32_t LED = (1u << 0);
    gpio_init_outputs(gpio, LED);

    gpio_set_pins(gpio, LED);
    printf("DIR=0x%X DATA=0x%X (LED on)\n", gpio->DIR, gpio->DATA);

    gpio_clear_pins(gpio, LED);
    printf("DIR=0x%X DATA=0x%X (LED off)\n", gpio->DIR, gpio->DATA);

    /* On bare metal you would instead write:
     *   #define GPIOA ((GpioBlock *)0x40020000u)
     *   GPIOA->DIR |= LED;
     * with the real memory map - never do that to a random address on Linux.
     */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o mmio_overlay mmio_overlay.c
./mmio_overlay
```

Bare-metal sketch (do not run as a normal Linux userspace poke):

```
#define GPIOA_BASE 0x40020000u
#define GPIOA ((volatile GpioBlock *)GPIOA_BASE)
```

Software Model: Button Debounce State Machine

Mechanical buttons bounce. A small state machine filters chatter before you act on a press/release.

```
/* button_debounce_fsm.c */
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>

typedef enum {
    BTN_IDLE,
    BTN_DEBOUNCE_PRESS,
    BTN_PRESSED,
    BTN_DEBOUNCE_RELEASE
} BtnState;

typedef struct {
    BtnState state;
    uint32_t ticks;       /* time spent in current state */
    uint32_t threshold;   /* ticks required to accept transition */
    bool     stable_down; /* debounced level: true = pressed */
} BtnFsm;

static void btn_init(BtnFsm *b, uint32_t threshold) {
    memset(b, 0, sizeof *b);
    b->state = BTN_IDLE;
    b->threshold = threshold;
}

/* raw_down: instantaneous sample (active high here) */
static void btn_update(BtnFsm *b, bool raw_down) {
    switch (b->state) {
        case BTN_IDLE:
            if (raw_down) {
                b->state = BTN_DEBOUNCE_PRESS;
                b->ticks = 0;
            }
            break;

        case BTN_DEBOUNCE_PRESS:
            if (!raw_down) {
                b->state = BTN_IDLE;
                b->ticks = 0;
            } else if (++b->ticks >= b->threshold) {
                b->state = BTN_PRESSED;
```

```
                b->stable_down = true;
                printf("event: PRESS accepted\n");
            }
            break;

        case BTN_PRESSED:
            if (!raw_down) {
                b->state = BTN_DEBOUNCE_RELEASE;
                b->ticks = 0;
            }
            break;

        case BTN_DEBOUNCE_RELEASE:
            if (raw_down) {
                b->state = BTN_PRESSED;
                b->ticks = 0;
            } else if (++b->ticks >= b->threshold) {
                b->state = BTN_IDLE;
                b->stable_down = false;
                printf("event: RELEASE accepted\n");
            }
            break;
    }
}

/* Simulated noisy waveform: 0=up, 1=down. Called once per "ms". */
int main(void) {
    BtnFsm btn;
    btn_init(&btn, 3);  /* need 3 stable samples */

    /* bounce on press, solid hold, bounce on release */
    const char *wave =
        "000"          /* idle */
        "10110111"     /* noisy press */
        "1111111"      /* held */
        "01001000"     /* noisy release */
        "00000";

    printf("raw sequence (1=down): %s\n", wave);
    for (size_t i = 0; wave[i] != '\0'; i++) {
        bool raw = (wave[i] == '1');
        btn_update(&btn, raw);
        printf("t=%02zu raw=%d state=%d stable=%d\n",
               i, raw, (int)btn.state, btn.stable_down);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o button_debounce_fsm button_debounce_fsm.c
./button_debounce_fsm
```

Tune `threshold` to your sample rate (e.g. 20 ms at 1 kHz sampling → threshold 20).

Classic Patterns (Still Useful)

LED Blink Superloop (conceptual MMIO)

```c
#include <stdint.h>

#define GPIO_PORT_ADDR 0x40020000u
#define GPIO_DIR  (*(volatile uint32_t *)(GPIO_PORT_ADDR + 0x00))
#define GPIO_DATA (*(volatile uint32_t *)(GPIO_PORT_ADDR + 0x04))
#define LED_PIN 0

/* On host, prefer the buffer overlay example - these addresses are not yours. */

void delay(volatile uint32_t count) {
    while (count--) {
        __asm__ volatile("nop");
    }
}

void led_init(void) {
    GPIO_DIR |= (1u << LED_PIN);
}

int main(void) {
    led_init();
    for (;;) {
        GPIO_DATA |= (1u << LED_PIN);
        delay(1000000);
        GPIO_DATA &= ~(1u << LED_PIN);
        delay(1000000);
    }
}
```

Interrupt Sketch

```
void UART_IRQHandler(void) {
    uint32_t status = UART_STATUS_REG;
    if (status & UART_RX_INTERRUPT) {
        char data = (char)UART_DATA_REG;
        process_received_data(data);
        UART_STATUS_REG = UART_RX_INTERRUPT; /* clear */
    }
}
```

Keep ISRs short: set flags / push to a ring buffer; do heavy work in the main loop.

Timing

Busy-wait delays waste power and drift with optimization and clock changes. Prefer hardware timers or a SysTick counter for production firmware. Software `nop` loops are only for crude bring-up.

Practical Host Lab: Mini "Firmware" Module

Combine bits + overlay + FSM in one translation unit you can test on Linux.

```
/* mini_fw_sim.c */
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>

typedef struct {
    volatile uint32_t DIR;
    volatile uint32_t DATA;
} SimpleGpio;

static uint8_t ram[sizeof(SimpleGpio)];
static SimpleGpio *GPIO;

static inline void set_bit_u32(volatile uint32_t *r, unsigned b) {
    *r |= (1u << b);
}
static inline void clear_bit_u32(volatile uint32_t *r, unsigned b) {
    *r &= ~(1u << b);
}
```

```
#define LED 0u
#define BTN 1u

typedef enum { S_OFF, S_ON } LedMode;

int main(void) {
    memset(ram, 0, sizeof ram);
    GPIO = (SimpleGpio *)(void *)ram;

    set_bit_u32(&GPIO->DIR, LED);    /* LED out */
    clear_bit_u32(&GPIO->DIR, BTN);  /* BTN in  */

    LedMode mode = S_OFF;
    /* Pretend samples of button (active high) every step */
    const int samples[] = {0, 0, 1, 1, 1, 1, 0, 0};
    int stable = 0;
    int last = 0;

    for (size_t i = 0; i < sizeof samples / sizeof samples[0]; i++) {
        int raw = samples[i];
        if (raw == last) {
            stable++;
        } else {
            stable = 0;
            last = raw;
        }
        if (stable == 2 && raw == 1) {
            mode = (mode == S_OFF) ? S_ON : S_OFF;
            printf("toggle -> %s\n", mode == S_ON ? "ON" : "OFF");
        }
        if (mode == S_ON) {
            set_bit_u32(&GPIO->DATA, LED);
        } else {
            clear_bit_u32(&GPIO->DATA, LED);
        }
        printf("i=%zu raw=%d DATA=0x%X\n", i, raw, GPIO->DATA);
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o mini_fw_sim mini_fw_sim.c
./mini_fw_sim
```

Exercises

Exercise 1 — Bit library

Implement `set`, `clear`, `toggle`, `test`, and `write_field` / `read_field` in a small header. Write tests that print expected masks for bits 0, 7, 31 on `uint32_t`. Reject `bit >= 32` safely.

```
gcc -std=c17 -Wall -Wextra -o bit_tests bit_tests.c
```

Exercise 2 — Packed vs unpacked

Define a packet:

```
u8 type; u16 length; u8 flags; u32 crc;
```

Print `sizeof` and offsets with and without `__attribute__((packed))`. Serialize to a `uint8_t` buffer with explicit byte writes (portable) and compare to a packed memcpy approach.

Exercise 3 — Overlay UART

Model a UART with `STATUS` (RX_FULL, TX_EMPTY) and `DATA` registers in a buffer. Write `uart_putc` / `uart_getc` that spin on flags. Inject bytes from `main` by writing the buffer as the "hardware" would.

Exercise 4 — Debounce FSM extension

Add a long-press state: if pressed longer than `L` ticks, emit `LONG_PRESS` once. Draw the state diagram in comments.

Exercise 5 — Cross-compile dry run

If `arm-none-eabi-gcc` is installed, compile a tiny `main` that only returns 0:

```
arm-none-eabi-gcc -std=c17 -Wall -Wextra -mcpu=cortex-m3 -mthumb \
  -c -o main.o main.c
arm-none-eabi-size main.o
```

Note: without startup and a linker script you will not produce a full flashable image; the goal is to confirm the cross compiler runs and inspect section sizes.

Exercise 6 — Volatile experiment

Write a loop waiting on a non-volatile global that a second function would change (on one core with `-O2`, the wait may spin forever if the compiler caches the load). Then mark the flag `volatile` and show the wait completes when you flip the flag between calls (e.g. cooperative single-thread simulation).

Summary

Embedded C fundamentals covered here:

1. **Architecture** — MCU vs MPU, memory map, startup and sections
2. **Cross-compilation** — role of `arm-none-eabi-gcc`, flags, linker script, objcopy
3. **Bit helpers** — set/clear/toggle/test and field pack/unpack
4. `volatile` — MMIO and ISR-shared flags; not a lock
5. **Packed structs** — layout control and portability caveats
6. **Struct overlay** — register blocks on real addresses or host buffers
7. **Debounce FSM** — software model for noisy digital inputs
8. **Host labs** — practice patterns safely with Linux `gcc`

Next chapters go deeper into low-level techniques, embedded-specific C patterns, real-time structure, and hardware interfaces.

Low-Level Programming

Introduction

Low-level programming in embedded systems involves direct interaction with hardware components through memory-mapped I/O, bit manipulation, and precise control of system resources. This chapter explores advanced techniques for accessing hardware peripherals, managing memory at the register level, and optimizing code for resource-constrained environments.

Memory-Mapped I/O

Memory-mapped I/O allows software to interact with hardware peripherals by reading from and writing to specific memory addresses that correspond to hardware registers.

Direct Register Access

```
#include <stdint.h>

// Example register definitions for a microcontroller
#define PERIPH_BASE       0x40000000
#define GPIOA_BASE        (PERIPH_BASE + 0x00000)
#define USART1_BASE       (PERIPH_BASE + 0x10000)

// GPIOA registers
#define GPIOA_MODER       (*(volatile uint32_t*)(GPIOA_BASE + 0x00))
#define GPIOA_OTYPER      (*(volatile uint32_t*)(GPIOA_BASE + 0x04))
#define GPIOA_OSPEEDR     (*(volatile uint32_t*)(GPIOA_BASE + 0x08))
#define GPIOA_PUPDR       (*(volatile uint32_t*)(GPIOA_BASE + 0x0C))
#define GPIOA_IDR         (*(volatile uint32_t*)(GPIOA_BASE + 0x10))
#define GPIOA_ODR         (*(volatile uint32_t*)(GPIOA_BASE + 0x14))
#define GPIOA_BSRR        (*(volatile uint32_t*)(GPIOA_BASE + 0x18))
#define GPIOA_LCKR        (*(volatile uint32_t*)(GPIOA_BASE + 0x1C))

// USART1 registers
#define USART1_SR         (*(volatile uint32_t*)(USART1_BASE + 0x00))
#define USART1_DR         (*(volatile uint32_t*)(USART1_BASE + 0x04))
#define USART1_BRR        (*(volatile uint32_t*)(USART1_BASE + 0x08))
#define USART1_CR1        (*(volatile uint32_t*)(USART1_BASE + 0x0C))
```

```
#define USART1_CR2        (*(volatile uint32_t*)(USART1_BASE + 0x10))
#define USART1_CR3        (*(volatile uint32_t*)(USART1_BASE + 0x14))

// Configure GPIO pin as output
void gpio_config_output(uint8_t pin) {
    // Set pin mode to output (01)
    GPIOA_MODER &= ~(3UL << (pin * 2));    // Clear mode bits
    GPIOA_MODER |= (1UL << (pin * 2));     // Set output mode

    // Set output type to push-pull (0)
    GPIOA_OTYPER &= ~(1UL << pin);

    // Set output speed to high (11)
    GPIOA_OSPEEDR |= (3UL << (pin * 2));

    // Set pull-up/pull-down to no pull (00)
    GPIOA_PUPDR &= ~(3UL << (pin * 2));
}

// Set GPIO pin high
void gpio_set(uint8_t pin) {
    GPIOA_BSRR = (1UL << pin);
}

// Clear GPIO pin (set low)
void gpio_clear(uint8_t pin) {
    GPIOA_BSRR = (1UL << (pin + 16));
}

// Toggle GPIO pin
void gpio_toggle(uint8_t pin) {
    GPIOA_ODR ^= (1UL << pin);
}

// Read GPIO pin state
uint8_t gpio_read(uint8_t pin) {
    return (GPIOA_IDR & (1UL << pin)) ? 1 : 0;
}
```

Bit Manipulation Techniques

Bit manipulation is fundamental in embedded programming for efficient hardware control.

Advanced Bit Operations

```
#include <stdint.h>

// Bit manipulation macros
#define BIT(n)                    (1UL << (n))
#define BIT_MASK(len)             (BIT(len) - 1)
#define BIT_FIELD(val, pos, len)  (((val) & BIT_MASK(len)) << (pos))

#define SET_BIT(reg, bit)         ((reg) |= BIT(bit))
#define CLEAR_BIT(reg, bit)       ((reg) &= ~BIT(bit))
#define TOGGLE_BIT(reg, bit)      ((reg) ^= BIT(bit))
#define CHECK_BIT(reg, bit)       (((reg) & BIT(bit)) != 0)

#define SET_BITS(reg, pos, len, val) \
    ((reg) = ((reg) & ~(BIT_MASK(len) << (pos))) | BIT_FIELD(val, pos, len))

#define GET_BITS(reg, pos, len) \
    (((reg) >> (pos)) & BIT_MASK(len))

// Example usage
void example_bit_operations(void) {
    volatile uint32_t register_value = 0;

    // Set bit 5
    SET_BIT(register_value, 5);

    // Clear bits 2-3
    CLEAR_BITS(register_value, 2, 2);

    // Set bits 8-11 to value 0xA
    SET_BITS(register_value, 8, 4, 0xA);

    // Get value of bits 8-11
    uint32_t extracted_value = GET_BITS(register_value, 8, 4);

    // Check if bit 5 is set
    if (CHECK_BIT(register_value, 5)) {
        // Bit 5 is set
    }
}
```

Bit Fields

Bit fields provide a convenient way to access specific bits within a structure:

```c
#include <stdint.h>

// GPIO register structure using bit fields
typedef struct {
    volatile uint32_t MODER   : 32;  // Port mode register
    volatile uint32_t OTYPER  : 16;  // Port output type register
    volatile uint32_t OSPEEDR : 32;  // Port output speed register
    volatile uint32_t PUPDR   : 32;  // Port pull-up/pull-down register
    volatile uint32_t IDR     : 16;  // Port input data register
    volatile uint32_t ODR     : 16;  // Port output data register
    volatile uint32_t BSRR    : 32;  // Port bit set/reset register
    volatile uint32_t LCKR    : 32;  // Port configuration lock register
    // ... other registers
} GPIO_TypeDef;

// Access GPIO registers using bit fields
#define GPIOA_BASE_ADDR  0x40020000
#define GPIOA            ((GPIO_TypeDef *)GPIOA_BASE_ADDR)

void gpio_bitfield_example(void) {
    // Configure pin 0 as output using bit fields
    GPIOA->MODER &= ~(3UL << 0);    // Clear mode bits for pin 0
    GPIOA->MODER |= (1UL << 0);     // Set output mode for pin 0

    // Set pin 0 high
    GPIOA->BSRR = (1UL << 0);
}
```

Inline Assembly

Inline assembly allows embedding assembly code directly within C functions for maximum performance and hardware control.

Basic Inline Assembly

```c
#include <stdint.h>

// Simple inline assembly example
void nop_instruction(void) {
    __asm__ volatile ("nop");
}

// Inline assembly with input/output operands
uint32_t rotate_left(uint32_t value, uint32_t shift) {
```

```
    uint32_t result;

    __asm__ volatile (
        "ror %0, %1, %2"
        : "=r" (result)           // Output operand
        : "r" (value), "r" (shift) // Input operands
    );

    return result;
}

// Memory barrier using inline assembly
void memory_barrier(void) {
    __asm__ volatile ("" ::: "memory");
}
```

Advanced Inline Assembly

```
// Atomic increment using inline assembly
int32_t atomic_increment(volatile int32_t *value) {
    int32_t result;

    __asm__ volatile (
        "1: ldrex  %0, [%1]     \n"  // Load exclusive
        "   add    %0, %0, #1   \n"  // Increment
        "   strex  r1, %0, [%1] \n"  // Store exclusive
        "   cmp    r1, #0       \n"  // Check if successful
        "   bne    1b           \n"  // Retry if not successful
        : "=&r" (result)
        : "r" (value)
        : "r1", "cc", "memory"
    );

    return result;
}

// Disable interrupts
void disable_interrupts(void) {
    __asm__ volatile (
        "cpsid i"
        ::: "memory"
    );
}

// Enable interrupts
```

```c
void enable_interrupts(void) {
    __asm__ volatile (
        "cpsie i"
        ::: "memory"
    );
}
```

Memory Management

Efficient memory management is critical in resource-constrained embedded systems.

Custom Memory Allocator

```c
#include <stdint.h>
#include <stddef.h>

#define HEAP_SIZE 4096
static uint8_t heap[HEAP_SIZE];
static size_t heap_top = 0;

// Simple memory allocator
void* simple_malloc(size_t size) {
    // Align to 4-byte boundary
    size = (size + 3) & ~3;

    if (heap_top + size > HEAP_SIZE) {
        return NULL;  // Out of memory
    }

    void *ptr = &heap[heap_top];
    heap_top += size;

    return ptr;
}

// Simple memory deallocator (does nothing in this simple implementation)
void simple_free(void *ptr) {
    // In a real implementation, you would track allocated blocks
    // This simple version just ignores deallocation
    (void)ptr;
}

// Reset heap (for demonstration)
void heap_reset(void) {
```

```
    heap_top = 0;
}
```

Memory Sections and Linker Scripts

Understanding memory sections and linker scripts is crucial for embedded development:

```
// Example of placing variables in specific memory sections
__attribute__((section(".ramfunc")))
void fast_function(void) {
    // Function placed in RAM for faster execution
}

__attribute__((aligned(16)))
uint32_t aligned_buffer[32];  // 16-byte aligned buffer

// Place variable in specific memory region
__attribute__((section(".ccmram")))
uint32_t ccm_data[100];  // Placed in Core Coupled Memory

// Zero-initialize variable in .bss section
static uint32_t bss_variable;

// Initialize variable in .data section
static uint32_t data_variable = 0x12345678;

// Read-only variable in .rodata section
static const uint32_t rodata_variable = 0xABCDEF00;
```

Hardware Abstraction Layer (HAL)

Creating a hardware abstraction layer improves code portability and maintainability.

GPIO HAL Implementation

```
#include <stdint.h>

// GPIO port enumeration
typedef enum {
    GPIO_PORT_A,
    GPIO_PORT_B,
    GPIO_PORT_C,
    // ... other ports
```

```
} gpio_port_t;

// GPIO pin enumeration
typedef enum {
    GPIO_PIN_0 = 0,
    GPIO_PIN_1,
    GPIO_PIN_2,
    GPIO_PIN_3,
    GPIO_PIN_4,
    GPIO_PIN_5,
    GPIO_PIN_6,
    GPIO_PIN_7,
    GPIO_PIN_8,
    GPIO_PIN_9,
    GPIO_PIN_10,
    GPIO_PIN_11,
    GPIO_PIN_12,
    GPIO_PIN_13,
    GPIO_PIN_14,
    GPIO_PIN_15
} gpio_pin_t;

// GPIO mode enumeration
typedef enum {
    GPIO_MODE_INPUT = 0,
    GPIO_MODE_OUTPUT,
    GPIO_MODE_AF,
    GPIO_MODE_ANALOG
} gpio_mode_t;

// GPIO output type enumeration
typedef enum {
    GPIO_OTYPE_PP = 0,  // Push-pull
    GPIO_OTYPE_OD       // Open-drain
} gpio_otype_t;

// GPIO speed enumeration
typedef enum {
    GPIO_SPEED_LOW = 0,
    GPIO_SPEED_MEDIUM,
    GPIO_SPEED_HIGH,
    GPIO_SPEED_VERY_HIGH
} gpio_speed_t;

// GPIO pull-up/pull-down enumeration
typedef enum {
    GPIO_PUPD_NONE = 0,
```

```
    GPIO_PUPD_UP,
    GPIO_PUPD_DOWN
} gpio_pupd_t;

// GPIO configuration structure
typedef struct {
    gpio_mode_t mode;
    gpio_otype_t otype;
    gpio_speed_t speed;
    gpio_pupd_t pupd;
} gpio_config_t;

// Base addresses for GPIO ports
#define GPIOA_BASE  0x40020000
#define GPIOB_BASE  0x40020400
#define GPIOC_BASE  0x40020800

// GPIO register structure
typedef struct {
    volatile uint32_t MODER;    // GPIO port mode register
    volatile uint32_t OTYPER;   // GPIO port output type register
    volatile uint32_t OSPEEDR;  // GPIO port output speed register
    volatile uint32_t PUPDR;    // GPIO port pull-up/pull-down register
    volatile uint32_t IDR;      // GPIO port input data register
    volatile uint32_t ODR;      // GPIO port output data register
    volatile uint32_t BSRR;     // GPIO port bit set/reset register
    volatile uint32_t LCKR;     // GPIO port configuration lock register
    volatile uint32_t AFR[2];   // GPIO alternate function registers
} gpio_reg_t;

// Get GPIO register pointer
static gpio_reg_t* get_gpio_reg(gpio_port_t port) {
    switch (port) {
        case GPIO_PORT_A: return (gpio_reg_t*)GPIOA_BASE;
        case GPIO_PORT_B: return (gpio_reg_t*)GPIOB_BASE;
        case GPIO_PORT_C: return (gpio_reg_t*)GPIOC_BASE;
        default: return NULL;
    }
}

// Configure GPIO pin
int gpio_init(gpio_port_t port, gpio_pin_t pin, const gpio_config_t *config) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio == NULL || config == NULL) {
        return -1;
    }
```

```
    uint32_t pin_pos = (uint32_t)pin * 2;

    // Configure mode
    gpio->MODER &= ~(3UL << pin_pos);
    gpio->MODER |= ((uint32_t)config->mode << pin_pos);

    // Configure output type (only for output modes)
    if (config->mode == GPIO_MODE_OUTPUT || config->mode == GPIO_MODE_AF) {
        gpio->OTYPER &= ~(1UL << pin);
        gpio->OTYPER |= ((uint32_t)config->otype << pin);
    }

    // Configure speed
    gpio->OSPEEDR &= ~(3UL << pin_pos);
    gpio->OSPEEDR |= ((uint32_t)config->speed << pin_pos);

    // Configure pull-up/pull-down
    gpio->PUPDR &= ~(3UL << pin_pos);
    gpio->PUPDR |= ((uint32_t)config->pupd << pin_pos);

    return 0;
}

// Set GPIO pin high
void gpio_set(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        gpio->BSRR = (1UL << pin);
    }
}

// Clear GPIO pin (set low)
void gpio_clear(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        gpio->BSRR = (1UL << (pin + 16));
    }
}

// Toggle GPIO pin
void gpio_toggle(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        gpio->ODR ^= (1UL << pin);
    }
}
```

```c
// Read GPIO pin state
int gpio_read(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        return (gpio->IDR & (1UL << pin)) ? 1 : 0;
    }
    return -1;
}
```

Interrupt Management

Proper interrupt management is essential for responsive embedded systems.

Nested Vectored Interrupt Controller (NVIC)

```c
// NVIC registers
#define NVIC_BASE  0xE000E100
#define NVIC_ISER  ((volatile uint32_t*)(NVIC_BASE + 0x000))  // Interrupt set-enable
#define NVIC_ICER  ((volatile uint32_t*)(NVIC_BASE + 0x080))  // Interrupt clear-enable
#define NVIC_ISPR  ((volatile uint32_t*)(NVIC_BASE + 0x100))  // Interrupt set-pending
#define NVIC_ICPR  ((volatile uint32_t*)(NVIC_BASE + 0x180))  // Interrupt clear-pending
#define NVIC_IPR   ((volatile uint8_t*)(NVIC_BASE + 0x300))   // Interrupt priority

// Enable interrupt
void nvic_enable_irq(uint8_t irq_number) {
    NVIC_ISER[irq_number >> 5] = (1UL << (irq_number & 0x1F));
}

// Disable interrupt
void nvic_disable_irq(uint8_t irq_number) {
    NVIC_ICER[irq_number >> 5] = (1UL << (irq_number & 0x1F));
}

// Set interrupt priority
void nvic_set_priority(uint8_t irq_number, uint8_t priority) {
    NVIC_IPR[irq_number] = (priority << 4) & 0xFF;
}

// Global interrupt enable/disable
void enable_irq(void) {
    __asm__ volatile ("cpsie i" ::: "memory");
}

void disable_irq(void) {
```

```
    __asm__ volatile ("cpsid i" ::: "memory");
}
```

Practical Examples

UART Driver Implementation

```
#include <stdint.h>

// UART registers
typedef struct {
    volatile uint32_t SR;     // Status register
    volatile uint32_t DR;     // Data register
    volatile uint32_t BRR;    // Baud rate register
    volatile uint32_t CR1;    // Control register 1
    volatile uint32_t CR2;    // Control register 2
    volatile uint32_t CR3;    // Control register 3
    volatile uint32_t GTPR;   // Guard time and prescaler register
} uart_reg_t;

#define USART1_BASE  0x40011000
#define USART1       ((uart_reg_t*)USART1_BASE)

// UART status flags
#define UART_FLAG_TXE   (1UL << 7)  // Transmit data register empty
#define UART_FLAG_RXNE  (1UL << 5)  // Read data register not empty
#define UART_FLAG_TC    (1UL << 6)  // Transmission complete

// UART control bits
#define UART_CR1_TE     (1UL << 3)  // Transmitter enable
#define UART_CR1_RE     (1UL << 2)  // Receiver enable
#define UART_CR1_UE     (1UL << 0)  // UART enable

// Initialize UART
int uart_init(uint32_t baudrate, uint32_t peripheral_clock) {
    // Enable UART clock (implementation-specific)
    // RCC->APB2ENR |= RCC_APB2ENR_USART1EN;

    // Configure baud rate
    uint32_t baud_div = (peripheral_clock + (baudrate / 2)) / baudrate;
    USART1->BRR = baud_div;

    // Configure UART (8N1, no flow control)
    USART1->CR1 = UART_CR1_TE | UART_CR1_RE | UART_CR1_UE;
```

```
    return 0;
}

// Send character
void uart_putc(char c) {
    // Wait for transmit data register to be empty
    while (!(USART1->SR & UART_FLAG_TXE));

    // Send character
    USART1->DR = c;
}

// Send string
void uart_puts(const char *str) {
    while (*str) {
        uart_putc(*str++);
    }
}

// Receive character
char uart_getc(void) {
    // Wait for receive data register to be not empty
    while (!(USART1->SR & UART_FLAG_RXNE));

    // Read character
    return (char)(USART1->DR & 0xFF);
}

// Check if data is available
int uart_available(void) {
    return (USART1->SR & UART_FLAG_RXNE) ? 1 : 0;
}

// Example usage
int main(void) {
    // Initialize UART for 115200 baud
    uart_init(115200, 16000000);  // 16MHz peripheral clock

    uart_puts("Hello, Embedded World!\r\n");

    while (1) {
        if (uart_available()) {
            char c = uart_getc();
            uart_putc(c);  // Echo character
        }
    }
```

```
    return 0;
}
```

Summary

Low-level programming in embedded systems involves:

1. **Memory-Mapped I/O** - Direct hardware register access
2. **Bit Manipulation** - Efficient control of individual bits and bit fields
3. **Inline Assembly** - Maximum performance and hardware control
4. **Memory Management** - Custom allocators and memory section control
5. **Hardware Abstraction** - Portable and maintainable code
6. **Interrupt Management** - Responsive system behavior
7. **Peripheral Drivers** - Interface with hardware components

These techniques enable developers to create efficient, reliable embedded applications that make optimal use of limited system resources while maintaining precise control over hardware behavior.

Embedded C Techniques

Introduction

Embedded C programming requires specialized techniques and language extensions to efficiently utilize limited resources while maintaining code reliability and performance. This chapter explores advanced C programming techniques specifically tailored for embedded systems, including compiler extensions, optimization strategies, and specialized language features.

Compiler Extensions and Attributes

Modern embedded compilers provide extensions that enhance C's capabilities for embedded development.

GCC Attributes

GCC provides numerous attributes to control code generation and optimization:

```
#include <stdint.h>

// Align variable to specific boundary
__attribute__((aligned(16)))
uint32_t aligned_array[32];

// Place function in specific section
__attribute__((section(".ramfunc")))
void fast_function(void) {
    // Code placed in RAM for faster execution
}

// Never return function (noreturn)
__attribute__((noreturn))
void system_reset(void) {
    // Reset system
    // This function never returns
    while (1);
}

// Weak symbol (can be overridden)
```

```c
__attribute__((weak))
void default_handler(void) {
    // Default interrupt handler
    while (1);
}

// Pack structure to minimize padding
__attribute__((packed))
struct packed_data {
    uint8_t byte1;
    uint16_t word1;
    uint8_t byte2;
};

// Always inline function
__attribute__((always_inline))
static inline uint32_t fast_multiply(uint32_t a, uint32_t b) {
    return a * b;
}

// Never inline function
__attribute__((noinline))
void critical_function(void) {
    // Function that should never be inlined
}

// Specify function call convention
__attribute__((long_call))
void far_function(void) {
    // Function that may be far away in memory
}
```

IAR and Keil Extensions

Different compilers provide their own extensions:

```c
// IAR specific pragmas
#pragma location = "MY_SECTION"
__no_init uint32_t persistent_data;  // No initialization

// Keil specific attributes
__align(16)
uint32_t keil_aligned_array[32];

// ARM-specific intrinsics
#include <arm_acle.h>
```

```
void example_intrinsics(void) {
    // Count leading zeros
    uint32_t value = 0x00100000;
    uint32_t clz = __clz(value);  // Count leading zeros

    // Reverse bits
    uint32_t reversed = __rbit(value);

    // Byte-swap
    uint32_t swapped = __rev(value);
}
```

Optimization Techniques

Embedded systems require careful optimization to balance performance, size, and power consumption.

Compiler Optimization Levels

```
// Optimization levels (GCC examples)
// -O0: No optimization (default for debugging)
// -O1: Basic optimization
// -O2: Full optimization
// -O3: Aggressive optimization
// -Os: Optimize for size
// -Ofast: Fastest possible (may violate standards)

// Function-specific optimization
__attribute__((optimize("O3")))
void optimized_function(void) {
    // This function will be optimized with -O3 regardless of global settings
}

__attribute__((optimize("Os")))
void size_optimized_function(void) {
    // This function will be optimized for size
}
```

Manual Optimization Techniques

```c
#include <stdint.h>

// Loop unrolling
void manual_unroll_example(uint32_t *array, uint32_t size) {
    uint32_t i;

    // Unrolled loop (process 4 elements per iteration)
    for (i = 0; i < size - 3; i += 4) {
        array[i] += 1;
        array[i+1] += 1;
        array[i+2] += 1;
        array[i+3] += 1;
    }

    // Handle remaining elements
    for (; i < size; i++) {
        array[i] += 1;
    }
}

// Strength reduction
uint32_t strength_reduction_example(uint32_t x, uint32_t iterations) {
    // Instead of multiplication in loop
    uint32_t result = 0;
    uint32_t multiplier = x;

    for (uint32_t i = 0; i < iterations; i++) {
        result += multiplier;  // Addition instead of multiplication
        multiplier += x;       // Increment instead of multiplication
    }

    return result;
}

// Lookup tables for expensive calculations
static const uint8_t sine_table[256] = {
    // Pre-calculated sine values
    128, 131, 134, 137, 140, 143, 146, 149, 152, 155, 158, 161, 164, 167, 170, 173,
    176, 179, 182, 185, 188, 191, 194, 196, 199, 202, 205, 207, 210, 213, 215, 218,
    // ... more values
};

uint8_t fast_sine(uint8_t angle) {
    return sine_table[angle];
}
```

Fixed-Point Arithmetic

Floating-point operations are expensive on many embedded processors. Fixed-point arithmetic provides an efficient alternative.

```
#include <stdint.h>

// Fixed-point types (Q-format)
typedef int32_t q16_16;  // 16 integer bits, 16 fractional bits
typedef int16_t q8_8;    // 8 integer bits, 8 fractional bits

// Convert between integer and fixed-point
#define INT_TO_Q16_16(x) ((q16_16)((x) << 16))
#define Q16_16_TO_INT(x) ((int32_t)((x) >> 16))

// Fixed-point multiplication (Q16.16)
q16_16 q16_16_mul(q16_16 a, q16_16 b) {
    // Multiply and shift to maintain Q16.16 format
    return (q16_16)(((int64_t)a * b) >> 16);
}

// Fixed-point division (Q16.16)
q16_16 q16_16_div(q16_16 a, q16_16 b) {
    // Divide and shift to maintain Q16.16 format
    return (q16_16)(((int64_t)a << 16) / b);
}

// Fixed-point addition and subtraction (same format)
q16_16 q16_16_add(q16_16 a, q16_16 b) {
    return a + b;
}

q16_16 q16_16_sub(q16_16 a, q16_16 b) {
    return a - b;
}

// Example usage
void fixed_point_example(void) {
    q16_16 a = INT_TO_Q16_16(3);     // 3.0 in Q16.16
    q16_16 b = INT_TO_Q16_16(2) + (1 << 15);  // 2.5 in Q16.16

    q16_16 result = q16_16_mul(a, b);  // 3.0 * 2.5 = 7.5
    int32_t integer_result = Q16_16_TO_INT(result);  // Convert to integer
}
```

Volatile Keyword and Memory-Mapped I/O

The volatile keyword is crucial for embedded programming to prevent compiler optimizations that could break hardware interaction.

```
#include <stdint.h>

// Memory-mapped hardware registers
#define GPIOA_BASE  0x40020000
#define GPIOA_ODR   (*(volatile uint32_t*)(GPIOA_BASE + 0x14))
#define GPIOA_IDR   (*(volatile uint32_t*)(GPIOA_BASE + 0x10))

// Without volatile, compiler might optimize away repeated reads
void non_volatile_example(void) {
    uint32_t reg_value = GPIOA_IDR;  // Compiler might cache this value
    // ... some code ...
    reg_value = GPIOA_IDR;  // Compiler might use cached value instead of re-reading
}

// With volatile, compiler will always read from hardware
void volatile_example(void) {
    volatile uint32_t reg_value = GPIOA_IDR;  // Always read from hardware
    // ... some code ...
    reg_value = GPIOA_IDR;  // Always read from hardware again
}

// Volatile pointers
volatile uint32_t * const gpio_odr = (volatile uint32_t*)(GPIOA_BASE + 0x14);

void pointer_example(void) {
    *gpio_odr = 0x00000001;  // Set pin 0
    *gpio_odr = 0x00000002;  // Set pin 1
}
```

Interrupt Service Routines

Properly written ISRs are critical for real-time embedded systems.

```
#include <stdint.h>

// Global variables used in ISR should be volatile
volatile uint32_t isr_counter = 0;
volatile uint8_t data_ready = 0;
volatile uint8_t received_data = 0;
```

```
// Interrupt service routine
void USART1_IRQHandler(void) __attribute__((interrupt));
void USART1_IRQHandler(void) {
    // Read status register
    uint32_t status = USART1->SR;

    // Check for receive interrupt
    if (status & USART_SR_RXNE) {
        // Read received data
        received_data = USART1->DR;
        data_ready = 1;
        isr_counter++;
    }

    // Check for transmit interrupt
    if (status & USART_SR_TXE) {
        // Disable transmit interrupt
        USART1->CR1 &= ~USART_CR1_TXEIE;
    }
}

// Critical section management
uint32_t enter_critical_section(void) {
    uint32_t primask;
    __asm__ volatile ("mrs %0, primask" : "=r" (primask));
    __asm__ volatile ("cpsid i");
    return primask;
}

void exit_critical_section(uint32_t primask) {
    __asm__ volatile ("msr primask, %0" : : "r" (primask));
}

// Safe access to shared variables
uint32_t get_isr_counter(void) {
    uint32_t counter;
    uint32_t primask = enter_critical_section();
    counter = isr_counter;
    exit_critical_section(primask);
    return counter;
}
```

Memory-Constrained Programming

Techniques for working with limited memory resources.

Stack Management

```
#include <stdint.h>

// Monitor stack usage
extern uint32_t _estack;      // End of stack (top)
extern uint32_t _stack_start; // Start of stack

// Stack pattern for usage monitoring
#define STACK_PATTERN  0xDEADBEEF

void init_stack_pattern(void) {
    uint32_t *stack_ptr = &_stack_start;
    while (stack_ptr < &_estack) {
        *stack_ptr++ = STACK_PATTERN;
    }
}

uint32_t get_stack_usage(void) {
    uint32_t *stack_ptr = &_stack_start;
    while (stack_ptr < &_estack && *stack_ptr == STACK_PATTERN) {
        stack_ptr++;
    }
    return (uint32_t)&_estack - (uint32_t)stack_ptr;
}

// Avoid large local variables
void bad_function(void) {
    uint8_t large_buffer[2048];  // Large stack allocation
    // ... use buffer ...
}

void good_function(void) {
    static uint8_t large_buffer[2048];  // Static allocation
    // ... use buffer ...
}

// Or use dynamic allocation with careful management
uint8_t *get_buffer(void) {
    static uint8_t buffer[2048];
    static uint8_t in_use = 0;

    if (!in_use) {
        in_use = 1;
        return buffer;
    }
    return NULL;  // Buffer in use
```

```
}

void release_buffer(void) {
    // In a real implementation, you'd track which buffer is released
}
```

Code Size Optimization

```
#include <stdint.h>

// Use smaller data types when possible
void size_optimized_loop(uint8_t count) {
    for (uint8_t i = 0; i < count; i++) {
        // Process data
    }
}

// Instead of:
void size_unoptimized_loop(uint32_t count) {
    for (uint32_t i = 0; i < count; i++) {
        // Process data
    }
}

// Use function pointers for common operations
typedef void (*handler_func_t)(void);

void handler_a(void) { /* ... */ }
void handler_b(void) { /* ... */ }
void handler_c(void) { /* ... */ }

static const handler_func_t handlers[] = {
    handler_a,
    handler_b,
    handler_c
};

void call_handler(uint8_t index) {
    if (index < sizeof(handlers) / sizeof(handlers[0])) {
        handlers[index]();
    }
}

// Use lookup tables instead of complex calculations
static const uint8_t crc8_table[256] = {
```

```
    0x00, 0x07, 0x0E, 0x09, 0x1C, 0x1B, 0x12, 0x15,
    0x38, 0x3F, 0x36, 0x31, 0x24, 0x23, 0x2A, 0x2D,
    // ... table values
};

uint8_t crc8(uint8_t *data, uint32_t length) {
    uint8_t crc = 0;
    for (uint32_t i = 0; i < length; i++) {
        crc = crc8_table[crc ^ data[i]];
    }
    return crc;
}
```

Real-Time Considerations

Techniques for ensuring deterministic behavior in real-time systems.

Timing Analysis

```
#include <stdint.h>

// Measure execution time
uint32_t get_cycle_count(void) {
    uint32_t cycle_count;
    __asm__ volatile ("rdcycle %0" : "=r" (cycle_count));
    return cycle_count;
}

void timing_analysis_example(void) {
    uint32_t start_time = get_cycle_count();

    // Code to measure
    complex_calculation();

    uint32_t end_time = get_cycle_count();
    uint32_t execution_time = end_time - start_time;

    // Store or report execution time
}

// Avoid operations with variable execution time
void deterministic_delay(uint32_t cycles) {
    // Use timer instead of loop with unknown iterations
    uint32_t start = get_cycle_count();
```

```
    while ((get_cycle_count() - start) < cycles) {
        // Wait
    }
}

// Use constant-time operations when possible
uint32_t constant_time_compare(const uint8_t *a, const uint8_t *b, uint32_t len) {
    uint32_t result = 0;
    for (uint32_t i = 0; i < len; i++) {
        result |= a[i] ^ b[i];  // Always compare all bytes
    }
    return result;  // 0 if equal, non-zero if different
}
```

Hardware-Specific Features

Leveraging processor-specific features for maximum efficiency.

ARM Cortex-M Specific Features

```
#include <stdint.h>

// Use Cortex-M specific instructions
void cortex_m_optimizations(void) {
    // Count leading zeros
    uint32_t value = 0x00100000;
    uint32_t clz;
    __asm__ volatile ("clz %0, %1" : "=r" (clz) : "r" (value));

    // Reverse bits
    uint32_t reversed;
    __asm__ volatile ("rbit %0, %1" : "=r" (reversed) : "r" (value));

    // Byte-reverse word
    uint32_t rev;
    __asm__ volatile ("rev %0, %1" : "=r" (rev) : "r" (value));
}

// Use bit-band region for atomic bit operations
#define BITBAND_SRAM_REF   0x20000000
#define BITBAND_SRAM_BASE  0x22000000
#define BITBAND_SRAM(a,b)  ((BITBAND_SRAM_BASE + (a-BITBAND_SRAM_REF)*32 + (b*4)))

// Atomic bit manipulation using bit-band
```

```c
void bitband_example(volatile uint32_t *reg, uint8_t bit) {
    // Create bit-band alias
    volatile uint32_t *bit_alias = (volatile uint32_t*)BITBAND_SRAM((uint32_t)reg, bit);

    *bit_alias = 1;  // Atomically set bit
    *bit_alias = 0;  // Atomically clear bit
}
```

Practical Examples

Efficient Sensor Data Processing

```c
#include <stdint.h>

// Circular buffer for sensor data
#define BUFFER_SIZE 64

typedef struct {
    uint16_t buffer[BUFFER_SIZE];
    uint8_t head;
    uint8_t tail;
    uint8_t count;
} circular_buffer_t;

static circular_buffer_t sensor_buffer;

// Initialize circular buffer
void buffer_init(circular_buffer_t *buf) {
    buf->head = 0;
    buf->tail = 0;
    buf->count = 0;
}

// Add data to buffer
int buffer_put(circular_buffer_t *buf, uint16_t data) {
    if (buf->count >= BUFFER_SIZE) {
        return -1;  // Buffer full
    }

    buf->buffer[buf->head] = data;
    buf->head = (buf->head + 1) % BUFFER_SIZE;
    buf->count++;

    return 0;
}
```

```
// Get data from buffer
int buffer_get(circular_buffer_t *buf, uint16_t *data) {
    if (buf->count == 0) {
        return -1;  // Buffer empty
    }

    *data = buf->buffer[buf->tail];
    buf->tail = (buf->tail + 1) % BUFFER_SIZE;
    buf->count--;

    return 0;
}

// Moving average filter
#define AVERAGE_WINDOW 8

uint16_t moving_average(uint16_t new_value) {
    static uint16_t window[AVERAGE_WINDOW];
    static uint8_t index = 0;
    static uint32_t sum = 0;
    static uint8_t count = 0;

    // Remove oldest value from sum
    sum -= window[index];

    // Add new value to sum and window
    window[index] = new_value;
    sum += new_value;

    // Update index
    index = (index + 1) % AVERAGE_WINDOW;

    // Update count
    if (count < AVERAGE_WINDOW) {
        count++;
    }

    // Return average
    return (uint16_t)(sum / count);
}
```

Power Management

```
#include <stdint.h>

// Power management states
typedef enum {
    POWER_STATE_ACTIVE,
    POWER_STATE_SLEEP,
    POWER_STATE_DEEP_SLEEP
} power_state_t;

// Enter sleep mode
void enter_sleep_mode(void) {
    // Disable unnecessary peripherals
    // RCC->APB1ENR &= ~RCC_APB1ENR_TIM2EN;  // Disable timer

    // Configure wake-up sources
    // EXTI->IMR |= EXTI_IMR_MR0;  // Enable EXTI line 0

    // Enter sleep mode
    __asm__ volatile ("wfi");  // Wait for interrupt
}

// Enter deep sleep mode
void enter_deep_sleep_mode(void) {
    // Save critical data to retention memory

    // Configure deep sleep
    // SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;

    // Enter deep sleep
    __asm__ volatile ("wfi");
}

// Dynamic frequency scaling
void set_performance_level(uint8_t level) {
    switch (level) {
        case 0:  // Low power
            // SystemClock_Config_LowPower();
            break;
        case 1:  // Medium power
            // SystemClock_Config_Medium();
            break;
        case 2:  // High performance
            // SystemClock_Config_High();
            break;
    }
```

```
}
```

Summary

Embedded C techniques involve specialized programming approaches to maximize efficiency in resource-constrained environments:

1. **Compiler Extensions** - Using attributes and pragmas for optimal code generation
2. **Optimization Techniques** - Balancing performance, size, and power consumption
3. **Fixed-Point Arithmetic** - Efficient alternative to floating-point operations
4. **Volatile Keyword** - Proper handling of hardware registers and shared variables
5. **Interrupt Management** - Writing efficient and safe ISRs
6. **Memory-Constrained Programming** - Techniques for limited RAM and Flash
7. **Real-Time Considerations** - Ensuring deterministic behavior
8. **Hardware-Specific Features** - Leveraging processor capabilities

These techniques enable developers to create efficient, reliable embedded applications that make optimal use of limited system resources while maintaining real-time performance requirements.

Real-time Programming

Introduction

Real-time programming in embedded systems requires deterministic behavior, precise timing control, and efficient resource management. This chapter explores the principles and techniques for developing real-time embedded applications, including scheduling strategies, timing analysis, and meeting strict temporal constraints.

Real-time System Characteristics

Real-time systems must respond to events within specified time constraints. Understanding these constraints is fundamental to successful real-time programming.

Types of Real-time Systems

1. **Hard Real-time Systems** - Missed deadlines can cause catastrophic failures
   - Examples: Aircraft control systems, medical devices, automotive safety systems
2. **Soft Real-time Systems** - Missed deadlines degrade performance but don't cause failures
   - Examples: Multimedia streaming, user interfaces, gaming
3. **Firm Real-time Systems** - Some deadlines can be missed with limited consequences
   - Examples: Industrial process control, data logging

Real-time Requirements

```
#include <stdint.h>
#include <stdbool.h>

// Real-time task structure
typedef struct {
    void (*function)(void);     // Task function
    uint32_t period;            // Task period in milliseconds
    uint32_t deadline;          // Task deadline in milliseconds
    uint32_t execution_time;    // Worst-case execution time
    uint32_t next_release;      // Next release time
    uint8_t priority;           // Task priority
```

```
    bool enabled;               // Task enabled flag
} rt_task_t;

// Real-time system parameters
typedef struct {
    uint32_t system_time;       // Current system time
    uint32_t max_response_time; // Maximum observed response time
    uint32_t missed_deadlines;  // Count of missed deadlines
} rt_system_t;

static rt_system_t rt_system;
```

Task Scheduling

Effective task scheduling is crucial for meeting real-time constraints while maximizing system utilization.

Rate Monotonic Scheduling (RMS)

Rate Monotonic Scheduling assigns priorities based on task periods - shorter periods get higher priorities.

```
#include <stdint.h>
#include <stdbool.h>

#define MAX_TASKS 16

static rt_task_t task_list[MAX_TASKS];
static uint8_t task_count = 0;

// Add task to system
int rt_add_task(void (*function)(void), uint32_t period, uint32_t deadline,
                uint32_t execution_time, uint8_t priority) {
    if (task_count >= MAX_TASKS) {
        return -1;  // Task list full
    }

    task_list[task_count].function = function;
    task_list[task_count].period = period;
    task_list[task_count].deadline = deadline;
    task_list[task_count].execution_time = execution_time;
    task_list[task_count].next_release = period;
    task_list[task_count].priority = priority;
    task_list[task_count].enabled = true;
```

```
    task_count++;
    return 0;
}

// Rate Monotonic priority assignment
void rt_assign_rm_priorities(void) {
    // Sort tasks by period (shortest first)
    for (int i = 0; i < task_count - 1; i++) {
        for (int j = 0; j < task_count - i - 1; j++) {
            if (task_list[j].period > task_list[j + 1].period) {
                // Swap tasks
                rt_task_t temp = task_list[j];
                task_list[j] = task_list[j + 1];
                task_list[j + 1] = temp;
            }
        }
    }

    // Assign priorities (higher number = higher priority)
    for (int i = 0; i < task_count; i++) {
        task_list[i].priority = task_count - i;
    }
}

// Check Rate Monotonic schedulability
bool rt_check_rm_schedulability(void) {
    // Calculate total utilization
    double total_utilization = 0.0;

    for (int i = 0; i < task_count; i++) {
        double utilization = (double)task_list[i].execution_time / task_list[i].period;
        total_utilization += utilization;
    }

    // Rate Monotonic bound: n * (2^(1/n) - 1)
    double rm_bound = task_count * (pow(2.0, 1.0/task_count) - 1.0);

    return (total_utilization <= rm_bound);
}
```

Earliest Deadline First (EDF) Scheduling

EDF scheduling dynamically assigns priorities based on task deadlines.

```
// Find highest priority task (earliest deadline)
int rt_find_highest_priority_task(void) {
    int highest_priority_task = -1;
    uint32_t earliest_deadline = UINT32_MAX;

    for (int i = 0; i < task_count; i++) {
        if (task_list[i].enabled &&
            task_list[i].next_release <= rt_system.system_time &&
            task_list[i].deadline < earliest_deadline) {
            earliest_deadline = task_list[i].deadline;
            highest_priority_task = i;
        }
    }

    return highest_priority_task;
}

// Update task deadlines
void rt_update_deadlines(void) {
    for (int i = 0; i < task_count; i++) {
        if (task_list[i].enabled) {
            // If task is ready to run
            if (rt_system.system_time >= task_list[i].next_release) {
                // Set absolute deadline
                task_list[i].deadline = task_list[i].next_release +
                                        task_list[i].period;
            }
        }
    }
}
```

Timing and Synchronization

Precise timing control is essential for real-time systems.

Timer Management

```
#include <stdint.h>

// Timer structure
typedef struct {
    uint32_t reload_value;      // Timer reload value
    uint32_t current_value;     // Current timer value
    bool running;               // Timer running flag
```

```
    bool expired;               // Timer expired flag
    void (*callback)(void);     // Expiration callback
} timer_t;

#define MAX_TIMERS 8
static timer_t timers[MAX_TIMERS];

// Initialize timer system
void timer_init(void) {
    for (int i = 0; i < MAX_TIMERS; i++) {
        timers[i].running = false;
        timers[i].expired = false;
        timers[i].callback = NULL;
    }

    // Configure hardware timer
    // TIM2->PSC = SystemCoreClock / 1000000 - 1;  // 1MHz timer
    // TIM2->ARR = 999;  // 1ms interrupt
    // TIM2->DIER |= TIM_DIER_UIE;  // Enable update interrupt
    // TIM2->CR1 |= TIM_CR1_CEN;    // Start timer
}

// Create timer
int timer_create(uint32_t milliseconds, void (*callback)(void)) {
    for (int i = 0; i < MAX_TIMERS; i++) {
        if (!timers[i].running) {
            timers[i].reload_value = milliseconds;
            timers[i].current_value = milliseconds;
            timers[i].callback = callback;
            timers[i].running = true;
            timers[i].expired = false;
            return i;
        }
    }
    return -1;  // No free timers
}

// Timer tick handler (called by timer interrupt)
void timer_tick_handler(void) {
    rt_system.system_time++;

    // Update all running timers
    for (int i = 0; i < MAX_TIMERS; i++) {
        if (timers[i].running) {
            if (timers[i].current_value > 0) {
                timers[i].current_value--;
            }
```

```
            if (timers[i].current_value == 0) {
                timers[i].expired = true;
                timers[i].running = false;

                if (timers[i].callback) {
                    timers[i].callback();
                }
            }
        }
    }

    // Update task deadlines
    rt_update_deadlines();
}

// Get system time
uint32_t rt_get_system_time(void) {
    return rt_system.system_time;
}

// Delay function (blocking)
void rt_delay_ms(uint32_t milliseconds) {
    uint32_t start_time = rt_system.system_time;
    while ((rt_system.system_time - start_time) < milliseconds) {
        // Wait
    }
}
```

Semaphore Implementation

Semaphores provide synchronization between tasks and ISRs.

```
#include <stdint.h>
#include <stdbool.h>

#define MAX_SEMAPHORES 16

typedef struct {
    volatile int32_t count;     // Semaphore count
    uint8_t max_count;          // Maximum count
    bool initialized;           // Initialization flag
} semaphore_t;

static semaphore_t semaphores[MAX_SEMAPHORES];
```

```c
// Initialize semaphore
int sem_init(uint8_t sem_id, uint8_t initial_count, uint8_t max_count) {
    if (sem_id >= MAX_SEMAPHORES) {
        return -1;
    }

    semaphores[sem_id].count = initial_count;
    semaphores[sem_id].max_count = max_count;
    semaphores[sem_id].initialized = true;

    return 0;
}

// Wait on semaphore (blocking)
int sem_wait(uint8_t sem_id) {
    if (sem_id >= MAX_SEMAPHORES || !semaphores[sem_id].initialized) {
        return -1;
    }

    // Disable interrupts to ensure atomic operation
    uint32_t primask = __get_PRIMASK();
    __disable_irq();

    while (semaphores[sem_id].count <= 0) {
        // Re-enable interrupts and wait
        __set_PRIMASK(primask);

        // In a real RTOS, this would yield to other tasks
        // For simple implementation, just wait
        __asm__ volatile ("nop");

        // Disable interrupts again
        __disable_irq();
    }

    semaphores[sem_id].count--;

    // Restore interrupt state
    __set_PRIMASK(primask);

    return 0;
}

// Signal semaphore (non-blocking)
int sem_signal(uint8_t sem_id) {
    if (sem_id >= MAX_SEMAPHORES || !semaphores[sem_id].initialized) {
        return -1;
```

```
    }

    // Disable interrupts to ensure atomic operation
    uint32_t primask = __get_PRIMASK();
    __disable_irq();

    if (semaphores[sem_id].count < semaphores[sem_id].max_count) {
        semaphores[sem_id].count++;
    }

    // Restore interrupt state
    __set_PRIMASK(primask);

    return 0;
}

// Try wait on semaphore (non-blocking)
int sem_try_wait(uint8_t sem_id) {
    if (sem_id >= MAX_SEMAPHORES || !semaphores[sem_id].initialized) {
        return -1;
    }

    // Disable interrupts to ensure atomic operation
    uint32_t primask = __get_PRIMASK();
    __disable_irq();

    int result = -1;
    if (semaphores[sem_id].count > 0) {
        semaphores[sem_id].count--;
        result = 0;
    }

    // Restore interrupt state
    __set_PRIMASK(primask);

    return result;
}
```

Interrupt Handling in Real-time Systems

Proper interrupt management is crucial for maintaining real-time performance.

Interrupt Latency Management

```
#include <stdint.h>

// Measure interrupt latency
static volatile uint32_t interrupt_timestamp = 0;
static volatile uint32_t interrupt_latency = 0;

// Interrupt service routine with latency measurement
void EXTI0_IRQHandler(void) {
    // Record interrupt entry time
    interrupt_timestamp = rt_system.system_time;

    // Clear interrupt flag
    // EXTI->PR = EXTI_PR_PR0;

    // Process interrupt
    process_external_interrupt();

    // Calculate latency
    interrupt_latency = rt_system.system_time - interrupt_timestamp;
}

// Critical section management
typedef struct {
    uint32_t primask;
    bool interrupts_disabled;
} critical_section_t;

// Enter critical section
critical_section_t enter_critical_section(void) {
    critical_section_t cs;
    cs.primask = __get_PRIMASK();
    __disable_irq();
    cs.interrupts_disabled = true;
    return cs;
}

// Exit critical section
void exit_critical_section(critical_section_t cs) {
    if (cs.interrupts_disabled) {
        __set_PRIMASK(cs.primask);
    }
}

// Deferred interrupt processing
#define INTERRUPT_QUEUE_SIZE 16
```

```
typedef enum {
    INTERRUPT_TYPE_GPIO,
    INTERRUPT_TYPE_UART,
    INTERRUPT_TYPE_TIMER
} interrupt_type_t;

typedef struct {
    interrupt_type_t type;
    uint32_t data;
    uint32_t timestamp;
} interrupt_event_t;

static interrupt_event_t interrupt_queue[INTERRUPT_QUEUE_SIZE];
static volatile uint8_t queue_head = 0;
static volatile uint8_t queue_tail = 0;
static volatile uint8_t queue_count = 0;

// Add interrupt event to queue
void queue_interrupt_event(interrupt_type_t type, uint32_t data) {
    critical_section_t cs = enter_critical_section();

    if (queue_count < INTERRUPT_QUEUE_SIZE) {
        interrupt_queue[queue_head].type = type;
        interrupt_queue[queue_head].data = data;
        interrupt_queue[queue_head].timestamp = rt_system.system_time;

        queue_head = (queue_head + 1) % INTERRUPT_QUEUE_SIZE;
        queue_count++;
    }

    exit_critical_section(cs);
}

// Process queued interrupts
void process_interrupt_queue(void) {
    while (queue_count > 0) {
        critical_section_t cs = enter_critical_section();

        interrupt_event_t event = interrupt_queue[queue_tail];
        queue_tail = (queue_tail + 1) % INTERRUPT_QUEUE_SIZE;
        queue_count--;

        exit_critical_section(cs);

        // Process the event
        switch (event.type) {
```

```
            case INTERRUPT_TYPE_GPIO:
                process_gpio_interrupt(event.data);
                break;
            case INTERRUPT_TYPE_UART:
                process_uart_interrupt(event.data);
                break;
            case INTERRUPT_TYPE_TIMER:
                process_timer_interrupt(event.data);
                break;
        }
    }
}
```

Resource Management

Efficient resource management is essential for predictable real-time behavior.

Memory Pool Management

```
#include <stdint.h>
#include <stdbool.h>
#include <string.h>

#define MEMORY_POOL_SIZE 1024
#define BLOCK_SIZE 32
#define NUM_BLOCKS (MEMORY_POOL_SIZE / BLOCK_SIZE)

typedef struct {
    uint8_t pool[MEMORY_POOL_SIZE];
    bool block_used[NUM_BLOCKS];
    uint8_t free_blocks;
} memory_pool_t;

static memory_pool_t memory_pool;

// Initialize memory pool
void mem_pool_init(void) {
    memset(memory_pool.block_used, 0, sizeof(memory_pool.block_used));
    memory_pool.free_blocks = NUM_BLOCKS;
}

// Allocate memory block
void* mem_pool_alloc(void) {
    critical_section_t cs = enter_critical_section();
```

```
    if (memory_pool.free_blocks == 0) {
        exit_critical_section(cs);
        return NULL;  // No free blocks
    }

    // Find free block
    for (int i = 0; i < NUM_BLOCKS; i++) {
        if (!memory_pool.block_used[i]) {
            memory_pool.block_used[i] = true;
            memory_pool.free_blocks--;
            exit_critical_section(cs);
            return &memory_pool.pool[i * BLOCK_SIZE];
        }
    }

    exit_critical_section(cs);
    return NULL;  // Should not reach here
}

// Free memory block
void mem_pool_free(void* ptr) {
    if (ptr == NULL) return;

    // Calculate block index
    uintptr_t offset = (uintptr_t)ptr - (uintptr_t)memory_pool.pool;
    uint32_t block_index = offset / BLOCK_SIZE;

    if (block_index < NUM_BLOCKS) {
        critical_section_t cs = enter_critical_section();
        if (memory_pool.block_used[block_index]) {
            memory_pool.block_used[block_index] = false;
            memory_pool.free_blocks++;
        }
        exit_critical_section(cs);
    }
}
```

Priority Inheritance

Priority inheritance prevents priority inversion in resource sharing.

```
#include <stdint.h>
#include <stdbool.h>

#define MAX_PRIORITY 255
```

```
typedef struct {
    uint8_t original_priority;
    uint8_t current_priority;
    bool owned;
    uint8_t owner_task_id;
} resource_t;

#define MAX_RESOURCES 16
static resource_t resources[MAX_RESOURCES];

// Initialize resource
void resource_init(uint8_t resource_id) {
    if (resource_id < MAX_RESOURCES) {
        resources[resource_id].original_priority = 0;
        resources[resource_id].current_priority = 0;
        resources[resource_id].owned = false;
        resources[resource_id].owner_task_id = 0;
    }
}

// Acquire resource with priority inheritance
int resource_acquire(uint8_t resource_id, uint8_t task_id, uint8_t task_priority) {
    if (resource_id >= MAX_RESOURCES) {
        return -1;
    }

    critical_section_t cs = enter_critical_section();

    if (!resources[resource_id].owned) {
        // Resource is free, acquire it
        resources[resource_id].owned = true;
        resources[resource_id].owner_task_id = task_id;
        resources[resource_id].original_priority = task_priority;
        resources[resource_id].current_priority = task_priority;
    } else {
        // Resource is owned by another task
        uint8_t owner_id = resources[resource_id].owner_task_id;

        // Check if priority inversion would occur
        if (task_priority > resources[resource_id].current_priority) {
            // Apply priority inheritance
            resources[resource_id].current_priority = task_priority;
            // In a real system, you would boost the owner task's priority
            task_list[owner_id].priority = task_priority;
        }
```

```
        exit_critical_section(cs);
        return -2;  // Resource busy
    }

    exit_critical_section(cs);
    return 0;  // Success
}

// Release resource
int resource_release(uint8_t resource_id, uint8_t task_id) {
    if (resource_id >= MAX_RESOURCES) {
        return -1;
    }

    critical_section_t cs = enter_critical_section();

    if (resources[resource_id].owned &&
        resources[resource_id].owner_task_id == task_id) {
        // Release resource
        resources[resource_id].owned = false;
        resources[resource_id].owner_task_id = 0;
        // Restore original priority
        // In a real system, you would restore the owner task's original priority
    } else {
        exit_critical_section(cs);
        return -2;  // Not owner or not owned
    }

    exit_critical_section(cs);
    return 0;  // Success
}
```

Timing Analysis and Deadline Monitoring

Monitoring and analyzing timing behavior ensures real-time constraints are met.

Deadline Monitoring

```
#include <stdint.h>
#include <stdbool.h>

// Task execution tracking
typedef struct {
    uint32_t start_time;
```

```
    uint32_t end_time;
    uint32_t execution_time;
    uint32_t deadline;
    bool deadline_missed;
} task_execution_t;

#define EXECUTION_HISTORY_SIZE 32
static task_execution_t execution_history[MAX_TASKS][EXECUTION_HISTORY_SIZE];
static uint8_t history_index[MAX_TASKS] = {0};

// Start task execution tracking
void task_start_execution(uint8_t task_id) {
    if (task_id < MAX_TASKS) {
        execution_history[task_id][history_index[task_id]].start_time =
            rt_system.system_time;
        execution_history[task_id][history_index[task_id]].deadline =
            task_list[task_id].deadline;
    }
}

// End task execution tracking
void task_end_execution(uint8_t task_id) {
    if (task_id < MAX_TASKS) {
        uint8_t idx = history_index[task_id];
        execution_history[task_id][idx].end_time = rt_system.system_time;
        execution_history[task_id][idx].execution_time =
            execution_history[task_id][idx].end_time -
            execution_history[task_id][idx].start_time;

        // Check for deadline miss
        if (rt_system.system_time > execution_history[task_id][idx].deadline) {
            execution_history[task_id][idx].deadline_missed = true;
            rt_system.missed_deadlines++;
        } else {
            execution_history[task_id][idx].deadline_missed = false;
        }

        // Update maximum response time
        if (execution_history[task_id][idx].execution_time >
            rt_system.max_response_time) {
            rt_system.max_response_time =
                execution_history[task_id][idx].execution_time;
        }

        // Advance history index
        history_index[task_id] = (history_index[task_id] + 1) % EXECUTION_HISTORY_SIZE;
    }
```

```
}

// Get task statistics
typedef struct {
    uint32_t avg_execution_time;
    uint32_t max_execution_time;
    uint32_t min_execution_time;
    uint32_t deadline_misses;
    double deadline_miss_rate;
} task_stats_t;

task_stats_t get_task_stats(uint8_t task_id) {
    task_stats_t stats = {0};

    if (task_id >= MAX_TASKS) {
        return stats;
    }

    uint32_t total_execution_time = 0;
    uint32_t valid_samples = 0;
    stats.max_execution_time = 0;
    stats.min_execution_time = UINT32_MAX;

    for (int i = 0; i < EXECUTION_HISTORY_SIZE; i++) {
        if (execution_history[task_id][i].end_time > 0) {
            uint32_t exec_time = execution_history[task_id][i].execution_time;
            total_execution_time += exec_time;
            valid_samples++;

            if (exec_time > stats.max_execution_time) {
                stats.max_execution_time = exec_time;
            }
            if (exec_time < stats.min_execution_time) {
                stats.min_execution_time = exec_time;
            }

            if (execution_history[task_id][i].deadline_missed) {
                stats.deadline_misses++;
            }
        }
    }

    if (valid_samples > 0) {
        stats.avg_execution_time = total_execution_time / valid_samples;
        stats.deadline_miss_rate = (double)stats.deadline_misses / valid_samples;
    }
```

```
    return stats;
}
```

Practical Examples

Real-time Control System

```
#include <stdint.h>
#include <math.h>

// PID Controller
typedef struct {
    float kp;           // Proportional gain
    float ki;           // Integral gain
    float kd;           // Derivative gain
    float setpoint;     // Desired value
    float integral;     // Integral term
    float previous_error; // Previous error for derivative
    float output_min;   // Minimum output
    float output_max;   // Maximum output
} pid_controller_t;

// Initialize PID controller
void pid_init(pid_controller_t *pid, float kp, float ki, float kd,
              float output_min, float output_max) {
    pid->kp = kp;
    pid->ki = ki;
    pid->kd = kd;
    pid->setpoint = 0;
    pid->integral = 0;
    pid->previous_error = 0;
    pid->output_min = output_min;
    pid->output_max = output_max;
}

// Compute PID output
float pid_compute(pid_controller_t *pid, float feedback, float dt) {
    // Calculate error
    float error = pid->setpoint - feedback;

    // Proportional term
    float proportional = pid->kp * error;

    // Integral term
    pid->integral += error * dt;
```

```
    float integral = pid->ki * pid->integral;

    // Derivative term
    float derivative = pid->kd * (error - pid->previous_error) / dt;

    // Calculate output
    float output = proportional + integral + derivative;

    // Clamp output to limits
    if (output > pid->output_max) {
        output = pid->output_max;
        // Prevent integral windup
        pid->integral -= error * dt;
    } else if (output < pid->output_min) {
        output = pid->output_min;
        // Prevent integral windup
        pid->integral -= error * dt;
    }

    // Save error for next derivative calculation
    pid->previous_error = error;

    return output;
}

// Set PID setpoint
void pid_set_setpoint(pid_controller_t *pid, float setpoint) {
    pid->setpoint = setpoint;
}

// Real-time motor control task
static pid_controller_t motor_pid;
static float motor_position = 0.0;
static float motor_speed = 0.0;

void motor_control_task(void) {
    static uint32_t last_time = 0;
    uint32_t current_time = rt_system.system_time;
    float dt = (current_time - last_time) / 1000.0;  // Convert to seconds
    last_time = current_time;

    // Read motor position (simulated)
    // In real implementation: motor_position = read_encoder();

    // Compute PID output
    float control_output = pid_compute(&motor_pid, motor_position, dt);
```

```
    // Apply control output to motor
    // In real implementation: set_motor_pwm(control_output);

    // Update motor model (simulated)
    motor_speed = control_output;
    motor_position += motor_speed * dt;
}

// Initialize motor control system
void motor_control_init(void) {
    // Initialize PID controller
    pid_init(&motor_pid, 2.0, 1.0, 0.1, -100.0, 100.0);
    pid_set_setpoint(&motor_pid, 100.0);  // Set target position

    // Add motor control task (10ms period)
    rt_add_task(motor_control_task, 10, 10, 2, 0);
}
```

Real-time Data Acquisition System

```
#include <stdint.h>

// Data acquisition system
#define ADC_CHANNELS 8
#define SAMPLE_BUFFER_SIZE 256

typedef struct {
    uint16_t samples[ADC_CHANNELS][SAMPLE_BUFFER_SIZE];
    uint16_t write_index[ADC_CHANNELS];
    uint16_t read_index[ADC_CHANNELS];
    uint32_t sample_count[ADC_CHANNELS];
    uint32_t overrun_count[ADC_CHANNELS];
} data_acquisition_t;

static data_acquisition_t data_acq;

// Initialize data acquisition system
void data_acq_init(void) {
    for (int i = 0; i < ADC_CHANNELS; i++) {
        data_acq.write_index[i] = 0;
        data_acq.read_index[i] = 0;
        data_acq.sample_count[i] = 0;
        data_acq.overrun_count[i] = 0;
    }
}
```

```
// ADC interrupt handler
void ADC_IRQHandler(void) {
    static uint8_t current_channel = 0;

    // Read ADC value (simulated)
    // uint16_t adc_value = ADC1->DR;
    uint16_t adc_value = 0;  // Simulated value

    // Store sample
    uint16_t write_idx = data_acq.write_index[current_channel];
    data_acq.samples[current_channel][write_idx] = adc_value;

    // Update indices
    data_acq.write_index[current_channel] = (write_idx + 1) % SAMPLE_BUFFER_SIZE;
    data_acq.sample_count[current_channel]++;

    // Check for buffer overrun
    if (data_acq.write_index[current_channel] == data_acq.read_index[current_channel]) {
        data_acq.overrun_count[current_channel]++;
        // Handle overrun (e.g., increment read index to maintain FIFO behavior)
        data_acq.read_index[current_channel] =
            (data_acq.read_index[current_channel] + 1) % SAMPLE_BUFFER_SIZE;
    }

    // Move to next channel
    current_channel = (current_channel + 1) % ADC_CHANNELS;

    // Configure ADC for next channel
    // ADC1->SQR3 = current_channel;  // Set channel in sequence register
}

// Read samples from buffer
int data_acq_read_samples(uint8_t channel, uint16_t *buffer, uint16_t max_samples) {
    if (channel >= ADC_CHANNELS) {
        return -1;
    }

    critical_section_t cs = enter_critical_section();

    uint16_t available_samples = 0;
    uint16_t read_idx = data_acq.read_index[channel];
    uint16_t write_idx = data_acq.write_index[channel];

    // Calculate available samples
    if (write_idx >= read_idx) {
        available_samples = write_idx - read_idx;
```

```
    } else {
        available_samples = SAMPLE_BUFFER_SIZE - read_idx + write_idx;
    }

    // Limit to requested samples
    if (available_samples > max_samples) {
        available_samples = max_samples;
    }

    // Copy samples
    for (uint16_t i = 0; i < available_samples; i++) {
        buffer[i] = data_acq.samples[channel][read_idx];
        read_idx = (read_idx + 1) % SAMPLE_BUFFER_SIZE;
    }

    // Update read index
    data_acq.read_index[channel] = read_idx;

    exit_critical_section(cs);

    return available_samples;
}

// Data processing task
void data_processing_task(void) {
    static uint16_t sample_buffer[64];

    // Process samples from channel 0
    int samples_read = data_acq_read_samples(0, sample_buffer, 64);

    if (samples_read > 0) {
        // Perform data processing (e.g., filtering, FFT, etc.)
        // process_samples(sample_buffer, samples_read);
    }
}

// Initialize data acquisition system
void data_acquisition_system_init(void) {
    data_acq_init();

    // Add data processing task (50ms period)
    rt_add_task(data_processing_task, 50, 50, 10, 0);

    // Enable ADC interrupt
    // NVIC_EnableIRQ(ADC_IRQn);
}
```

Summary

Real-time programming in embedded systems requires careful consideration of:

1. **Scheduling Algorithms** - Rate Monotonic and Earliest Deadline First scheduling
2. **Timing Control** - Precise timer management and delay functions
3. **Synchronization** - Semaphores, mutexes, and critical sections
4. **Interrupt Handling** - Low-latency ISRs and deferred processing
5. **Resource Management** - Memory pools and priority inheritance
6. **Timing Analysis** - Deadline monitoring and performance measurement
7. **Control Systems** - PID controllers and real-time feedback loops

Key principles for successful real-time programming: - Minimize interrupt latency and execution time - Use deterministic algorithms and data structures - Carefully manage shared resources to avoid priority inversion - Monitor and analyze timing behavior to ensure constraints are met - Implement proper error handling for missed deadlines - Balance performance requirements with system complexity

These techniques enable developers to create reliable, predictable embedded systems that meet strict real-time requirements while efficiently utilizing limited system resources.

Hardware Interfaces

Introduction

Embedded systems interact with the physical world through various hardware interfaces. Understanding how to properly interface with different types of hardware components is essential for developing functional embedded applications. This chapter covers the most common hardware interfaces used in embedded systems, including GPIO, SPI, I2C, UART, and other specialized interfaces.

GPIO (General Purpose Input/Output)

GPIO pins are the most fundamental interface in embedded systems, providing direct digital input and output capabilities.

GPIO Configuration and Control

```
#include <stdint.h>
#include <stdbool.h>

// GPIO pin modes
typedef enum {
    GPIO_MODE_INPUT = 0,
    GPIO_MODE_OUTPUT,
    GPIO_MODE_AF,        // Alternate function
    GPIO_MODE_ANALOG
} gpio_mode_t;

// GPIO output types
typedef enum {
    GPIO_OTYPE_PP = 0,   // Push-pull
    GPIO_OTYPE_OD        // Open-drain
} gpio_otype_t;

// GPIO speed settings
typedef enum {
    GPIO_SPEED_LOW = 0,
    GPIO_SPEED_MEDIUM,
```

```
    GPIO_SPEED_HIGH,
    GPIO_SPEED_VERY_HIGH
} gpio_speed_t;

// GPIO pull-up/pull-down settings
typedef enum {
    GPIO_PUPD_NONE = 0,
    GPIO_PUPD_UP,
    GPIO_PUPD_DOWN
} gpio_pupd_t;

// GPIO port definitions
typedef enum {
    GPIO_PORT_A = 0,
    GPIO_PORT_B,
    GPIO_PORT_C,
    GPIO_PORT_D,
    GPIO_PORT_E,
    GPIO_PORT_F,
    GPIO_PORT_G,
    GPIO_PORT_H
} gpio_port_t;

// GPIO pin definitions
typedef enum {
    GPIO_PIN_0 = 0,
    GPIO_PIN_1,
    GPIO_PIN_2,
    GPIO_PIN_3,
    GPIO_PIN_4,
    GPIO_PIN_5,
    GPIO_PIN_6,
    GPIO_PIN_7,
    GPIO_PIN_8,
    GPIO_PIN_9,
    GPIO_PIN_10,
    GPIO_PIN_11,
    GPIO_PIN_12,
    GPIO_PIN_13,
    GPIO_PIN_14,
    GPIO_PIN_15
} gpio_pin_t;

// GPIO register structure
typedef struct {
    volatile uint32_t MODER;    // Mode register
    volatile uint32_t OTYPER;   // Output type register
```

```
    volatile uint32_t OSPEEDR;  // Output speed register
    volatile uint32_t PUPDR;    // Pull-up/pull-down register
    volatile uint32_t IDR;      // Input data register
    volatile uint32_t ODR;      // Output data register
    volatile uint32_t BSRR;     // Bit set/reset register
    volatile uint32_t LCKR;     // Configuration lock register
    volatile uint32_t AFR[2];   // Alternate function registers
} gpio_reg_t;

// Base addresses for GPIO ports
#define GPIOA_BASE  0x40020000
#define GPIOB_BASE  0x40020400
#define GPIOC_BASE  0x40020800
#define GPIOD_BASE  0x40020C00
#define GPIOE_BASE  0x40021000
#define GPIOF_BASE  0x40021400
#define GPIOG_BASE  0x40021800
#define GPIOH_BASE  0x40021C00

// Get GPIO register pointer
static gpio_reg_t* get_gpio_reg(gpio_port_t port) {
    switch (port) {
        case GPIO_PORT_A: return (gpio_reg_t*)GPIOA_BASE;
        case GPIO_PORT_B: return (gpio_reg_t*)GPIOB_BASE;
        case GPIO_PORT_C: return (gpio_reg_t*)GPIOC_BASE;
        case GPIO_PORT_D: return (gpio_reg_t*)GPIOD_BASE;
        case GPIO_PORT_E: return (gpio_reg_t*)GPIOE_BASE;
        case GPIO_PORT_F: return (gpio_reg_t*)GPIOF_BASE;
        case GPIO_PORT_G: return (gpio_reg_t*)GPIOG_BASE;
        case GPIO_PORT_H: return (gpio_reg_t*)GPIOH_BASE;
        default: return NULL;
    }
}

// Configure GPIO pin
int gpio_init(gpio_port_t port, gpio_pin_t pin, gpio_mode_t mode,
              gpio_otype_t otype, gpio_speed_t speed, gpio_pupd_t pupd) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio == NULL) {
        return -1;
    }

    uint32_t pin_pos = (uint32_t)pin * 2;

    // Configure mode
    gpio->MODER &= ~(3UL << pin_pos);
```

```
    gpio->MODER |= ((uint32_t)mode << pin_pos);

    // Configure output type (only for output modes)
    if (mode == GPIO_MODE_OUTPUT || mode == GPIO_MODE_AF) {
        gpio->OTYPER &= ~(1UL << pin);
        gpio->OTYPER |= ((uint32_t)otype << pin);
    }

    // Configure speed
    gpio->OSPEEDR &= ~(3UL << pin_pos);
    gpio->OSPEEDR |= ((uint32_t)speed << pin_pos);

    // Configure pull-up/pull-down
    gpio->PUPDR &= ~(3UL << pin_pos);
    gpio->PUPDR |= ((uint32_t)pupd << pin_pos);

    return 0;
}

// Set GPIO pin high
void gpio_set(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        gpio->BSRR = (1UL << pin);
    }
}

// Clear GPIO pin (set low)
void gpio_clear(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        gpio->BSRR = (1UL << (pin + 16));
    }
}

// Toggle GPIO pin
void gpio_toggle(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
    if (gpio != NULL) {
        gpio->ODR ^= (1UL << pin);
    }
}

// Read GPIO pin state
int gpio_read(gpio_port_t port, gpio_pin_t pin) {
    gpio_reg_t *gpio = get_gpio_reg(port);
```

```
    if (gpio != NULL) {
        return (gpio->IDR & (1UL << pin)) ? 1 : 0;
    }
    return -1;
}

// Configure GPIO pin as input with pull-up
int gpio_config_input_pullup(gpio_port_t port, gpio_pin_t pin) {
    return gpio_init(port, pin, GPIO_MODE_INPUT, GPIO_OTYPE_PP,
                     GPIO_SPEED_LOW, GPIO_PUPD_UP);
}

// Configure GPIO pin as output push-pull
int gpio_config_output(gpio_port_t port, gpio_pin_t pin) {
    return gpio_init(port, pin, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP,
                     GPIO_SPEED_HIGH, GPIO_PUPD_NONE);
}
```

SPI (Serial Peripheral Interface)

SPI is a synchronous serial communication interface used for short-distance communication between microcontrollers and peripheral devices.

SPI Driver Implementation

```
#include <stdint.h>
#include <stdbool.h>

// SPI register structure
typedef struct {
    volatile uint32_t CR1;      // Control register 1
    volatile uint32_t CR2;      // Control register 2
    volatile uint32_t SR;       // Status register
    volatile uint32_t DR;       // Data register
    volatile uint32_t CRCPR;    // CRC polynomial register
    volatile uint32_t RXCRCR;   // RX CRC register
    volatile uint32_t TXCRCR;   // TX CRC register
    volatile uint32_t I2SCFGR;  // I2S configuration register
    volatile uint32_t I2SPR;    // I2S prescaler register
} spi_reg_t;

// SPI base addresses
#define SPI1_BASE  0x40013000
#define SPI2_BASE  0x40003800
```

```
#define SPI3_BASE  0x40003C00

#define SPI1  ((spi_reg_t*)SPI1_BASE)
#define SPI2  ((spi_reg_t*)SPI2_BASE)
#define SPI3  ((spi_reg_t*)SPI3_BASE)

// SPI control register bits
#define SPI_CR1_CPHA    (1UL << 0)   // Clock phase
#define SPI_CR1_CPOL    (1UL << 1)   // Clock polarity
#define SPI_CR1_MSTR    (1UL << 2)   // Master selection
#define SPI_CR1_BR      (7UL << 3)   // Baud rate control
#define SPI_CR1_SPE     (1UL << 6)   // SPI enable
#define SPI_CR1_LSBFIRST (1UL << 7)  // Frame format
#define SPI_CR1_SSI     (1UL << 8)   // Internal slave select
#define SPI_CR1_SSM     (1UL << 9)   // Software slave management
#define SPI_CR1_RXONLY  (1UL << 10)  // Receive only
#define SPI_CR1_DFF     (1UL << 11)  // Data frame format
#define SPI_CR1_CRCNEXT (1UL << 12)  // CRC transfer next
#define SPI_CR1_CRCEN   (1UL << 13)  // Hardware CRC calculation
#define SPI_CR1_BIDIOE  (1UL << 14)  // Output enable in bidirectional mode
#define SPI_CR1_BIDIMODE (1UL << 15) // Bidirectional data mode enable

// SPI status register bits
#define SPI_SR_RXNE     (1UL << 0)   // Receive buffer not empty
#define SPI_SR_TXE      (1UL << 1)   // Transmit buffer empty
#define SPI_SR_CHSIDE   (1UL << 2)   // Channel side
#define SPI_SR_UDR      (1UL << 3)   // Underrun flag
#define SPI_SR_CRCERR   (1UL << 4)   // CRC error flag
#define SPI_SR_MODF     (1UL << 5)   // Mode fault
#define SPI_SR_OVR      (1UL << 6)   // Overrun flag
#define SPI_SR_BSY      (1UL << 7)   // Busy flag

// SPI configuration structure
typedef struct {
    uint32_t baud_rate;         // Baud rate prescaler
    bool master;                // Master mode
    bool cpol;                  // Clock polarity
    bool cpha;                  // Clock phase
    bool lsb_first;             // LSB first
    bool bidirectional;         // Bidirectional mode
    uint8_t data_size;          // Data size (8 or 16 bits)
} spi_config_t;

// Initialize SPI peripheral
int spi_init(spi_reg_t *spi, const spi_config_t *config) {
    if (spi == NULL || config == NULL) {
        return -1;
```

```
    }

    // Disable SPI
    spi->CR1 &= ~SPI_CR1_SPE;

    // Configure SPI
    uint32_t cr1 = 0;

    // Set baud rate
    cr1 |= (config->baud_rate << 3) & SPI_CR1_BR;

    // Set master/slave mode
    if (config->master) {
        cr1 |= SPI_CR1_MSTR;
        cr1 |= SPI_CR1_SSI;  // Set internal slave select
    }

    // Set clock polarity and phase
    if (config->cpol) cr1 |= SPI_CR1_CPOL;
    if (config->cpha) cr1 |= SPI_CR1_CPHA;

    // Set data frame format
    if (config->data_size == 16) {
        cr1 |= SPI_CR1_DFF;
    }

    // Set frame format
    if (config->lsb_first) {
        cr1 |= SPI_CR1_LSBFIRST;
    }

    // Enable software slave management
    cr1 |= SPI_CR1_SSM;

    spi->CR1 = cr1;

    // Configure bidirectional mode
    if (config->bidirectional) {
        spi->CR1 |= SPI_CR1_BIDIMODE;
        spi->CR1 |= SPI_CR1_BIDIOE;  // Enable output
    }

    // Enable SPI
    spi->CR1 |= SPI_CR1_SPE;

    return 0;
}
```

```
// SPI transmit function
int spi_transmit(spi_reg_t *spi, const uint8_t *data, uint32_t length) {
    if (spi == NULL || data == NULL || length == 0) {
        return -1;
    }

    for (uint32_t i = 0; i < length; i++) {
        // Wait for transmit buffer to be empty
        while (!(spi->SR & SPI_SR_TXE));

        // Send data
        spi->DR = data[i];

        // Wait for transmission to complete
        while (spi->SR & SPI_SR_BSY);
    }

    return 0;
}

// SPI receive function
int spi_receive(spi_reg_t *spi, uint8_t *data, uint32_t length) {
    if (spi == NULL || data == NULL || length == 0) {
        return -1;
    }

    for (uint32_t i = 0; i < length; i++) {
        // Wait for receive buffer to be empty
        while (!(spi->SR & SPI_SR_TXE));

        // Send dummy data to initiate reception
        spi->DR = 0xFF;

        // Wait for data to be received
        while (!(spi->SR & SPI_SR_RXNE));

        // Read received data
        data[i] = spi->DR;
    }

    return 0;
}

// SPI transmit and receive function (full duplex)
int spi_transceive(spi_reg_t *spi, const uint8_t *tx_data,
                   uint8_t *rx_data, uint32_t length) {
```

```
    if (spi == NULL || tx_data == NULL || rx_data == NULL || length == 0) {
        return -1;
    }

    for (uint32_t i = 0; i < length; i++) {
        // Wait for transmit buffer to be empty
        while (!(spi->SR & SPI_SR_TXE));

        // Send data
        spi->DR = tx_data[i];

        // Wait for data to be received
        while (!(spi->SR & SPI_SR_RXNE));

        // Read received data
        rx_data[i] = spi->DR;
    }

    return 0;
}

// SPI chip select control
void spi_select_device(gpio_port_t cs_port, gpio_pin_t cs_pin) {
    gpio_clear(cs_port, cs_pin);  // Active low CS
}

void spi_deselect_device(gpio_port_t cs_port, gpio_pin_t cs_pin) {
    gpio_set(cs_port, cs_pin);    // Inactive high CS
}
```

I2C (Inter-Integrated Circuit)

I2C is a multi-master, multi-slave, packet-switched, single-ended, serial communication bus used for short-distance communication.

I2C Driver Implementation

```
#include <stdint.h>
#include <stdbool.h>

// I2C register structure
typedef struct {
    volatile uint32_t CR1;      // Control register 1
    volatile uint32_t CR2;      // Control register 2
```

```
    volatile uint32_t OAR1;     // Own address register 1
    volatile uint32_t OAR2;     // Own address register 2
    volatile uint32_t DR;       // Data register
    volatile uint32_t SR1;      // Status register 1
    volatile uint32_t SR2;      // Status register 2
    volatile uint32_t CCR;      // Clock control register
    volatile uint32_t TRISE;    // TRISE register
    volatile uint32_t FLTR;     // FLTR register
} i2c_reg_t;

// I2C base addresses
#define I2C1_BASE  0x40005400
#define I2C2_BASE  0x40005800
#define I2C3_BASE  0x40005C00

#define I2C1  ((i2c_reg_t*)I2C1_BASE)
#define I2C2  ((i2c_reg_t*)I2C2_BASE)
#define I2C3  ((i2c_reg_t*)I2C3_BASE)

// I2C control register 1 bits
#define I2C_CR1_PE      (1UL << 0)   // Peripheral enable
#define I2C_CR1_SMBUS   (1UL << 1)   // SMBus mode
#define I2C_CR1_SMBTYPE (1UL << 3)   // SMBus type
#define I2C_CR1_ENARP   (1UL << 4)   // ARP enable
#define I2C_CR1_ENPEC   (1UL << 5)   // PEC enable
#define I2C_CR1_ENGC    (1UL << 6)   // General call enable
#define I2C_CR1_NOSTRETCH (1UL << 7) // Clock stretching disable
#define I2C_CR1_START   (1UL << 8)   // Start generation
#define I2C_CR1_STOP    (1UL << 9)   // Stop generation
#define I2C_CR1_ACK     (1UL << 10)  // Acknowledge enable
#define I2C_CR1_POS     (1UL << 11)  // Acknowledge/PEC position
#define I2C_CR1_PEC     (1UL << 12)  // Packet error checking
#define I2C_CR1_ALERT   (1UL << 13)  // SMBus alert
#define I2C_CR1_SWRST   (1UL << 15)  // Software reset

// I2C status register 1 bits
#define I2C_SR1_SB      (1UL << 0)   // Start bit
#define I2C_SR1_ADDR    (1UL << 1)   // Address sent/matched
#define I2C_SR1_BTF     (1UL << 2)   // Byte transfer finished
#define I2C_SR1_ADD10   (1UL << 3)   // 10-bit header sent
#define I2C_SR1_STOPF   (1UL << 4)   // Stop detection
#define I2C_SR1_RXNE    (1UL << 6)   // Data register not empty
#define I2C_SR1_TXE     (1UL << 7)   // Data register empty
#define I2C_SR1_BERR    (1UL << 8)   // Bus error
#define I2C_SR1_ARLO    (1UL << 9)   // Arbitration lost
#define I2C_SR1_AF      (1UL << 10)  // Acknowledge failure
#define I2C_SR1_OVR     (1UL << 11)  // Overrun/underrun
```

```
#define I2C_SR1_PECERR  (1UL << 12)  // PEC error
#define I2C_SR1_TIMEOUT (1UL << 14)  // Timeout or Tlow error
#define I2C_SR1_SMBALERT (1UL << 15) // SMBus alert

// I2C configuration structure
typedef struct {
    uint32_t clock_speed;       // I2C clock speed in Hz
    bool master;                // Master mode
    uint16_t own_address;       // Own address (slave mode)
    bool ack_enable;            // Acknowledge enable
} i2c_config_t;

// Initialize I2C peripheral
int i2c_init(i2c_reg_t *i2c, const i2c_config_t *config) {
    if (i2c == NULL || config == NULL) {
        return -1;
    }

    // Disable I2C
    i2c->CR1 &= ~I2C_CR1_PE;

    // Configure I2C clock
    uint32_t freq_mhz = 0;  // Peripheral clock frequency in MHz
    // freq_mhz = get_pclk1_frequency() / 1000000;

    // Configure CR2 register (frequency)
    i2c->CR2 = freq_mhz & 0x3F;

    // Configure CCR register (clock control)
    uint32_t ccr_value = 0;
    if (config->clock_speed <= 100000) {
        // Standard mode (up to 100kHz)
        ccr_value = (freq_mhz * 1000000) / (config->clock_speed * 2);
        if (ccr_value < 4) ccr_value = 4;
    } else {
        // Fast mode (up to 400kHz)
        ccr_value = (freq_mhz * 1000000) / (config->clock_speed * 3);
        if (ccr_value < 1) ccr_value = 1;
        ccr_value |= (1UL << 15);  // Fast mode
    }
    i2c->CCR = ccr_value;

    // Configure TRISE register (rise time)
    if (config->clock_speed <= 100000) {
        // Standard mode
        i2c->TRISE = freq_mhz + 1;
    } else {
```

```
        // Fast mode
        i2c->TRISE = (freq_mhz * 300) / 1000 + 1;
    }

    // Configure own address (slave mode)
    if (!config->master) {
        i2c->OAR1 = (config->own_address << 1) | (1UL << 14);
    }

    // Enable acknowledge
    if (config->ack_enable) {
        i2c->CR1 |= I2C_CR1_ACK;
    }

    // Enable I2C
    i2c->CR1 |= I2C_CR1_PE;

    return 0;
}

// Generate start condition
static int i2c_start(i2c_reg_t *i2c) {
    // Generate start condition
    i2c->CR1 |= I2C_CR1_START;

    // Wait for start condition to be generated
    uint32_t timeout = 100000;
    while (!(i2c->SR1 & I2C_SR1_SB) && timeout--) {
        if (i2c->SR1 & (I2C_SR1_ARLO | I2C_SR1_BERR | I2C_SR1_AF)) {
            return -1;  // Error occurred
        }
    }

    if (timeout == 0) {
        return -2;  // Timeout
    }

    return 0;
}

// Generate stop condition
static void i2c_stop(i2c_reg_t *i2c) {
    i2c->CR1 |= I2C_CR1_STOP;
}

// Send address
```

```
static int i2c_send_address(i2c_reg_t *i2c, uint8_t address, bool read) {
    // Send address
    i2c->DR = (address << 1) | (read ? 1 : 0);

    // Wait for address to be sent
    uint32_t timeout = 100000;
    while (!(i2c->SR1 & I2C_SR1_ADDR) && timeout--) {
        if (i2c->SR1 & (I2C_SR1_ARLO | I2C_SR1_BERR | I2C_SR1_AF)) {
            return -1;  // Error occurred
        }
    }

    if (timeout == 0) {
        return -2;  // Timeout
    }

    // Clear ADDR flag by reading SR1 and SR2
    volatile uint32_t temp = i2c->SR1;
    temp = i2c->SR2;
    (void)temp;  // Prevent compiler warning

    return 0;
}

// Send data
static int i2c_send_data(i2c_reg_t *i2c, const uint8_t *data, uint32_t length) {
    for (uint32_t i = 0; i < length; i++) {
        // Wait for transmit buffer to be empty
        uint32_t timeout = 100000;
        while (!(i2c->SR1 & I2C_SR1_TXE) && timeout--) {
            if (i2c->SR1 & (I2C_SR1_ARLO | I2C_SR1_BERR | I2C_SR1_AF | I2C_SR1_OVR)) {
                return -1;  // Error occurred
            }
        }

        if (timeout == 0) {
            return -2;  // Timeout
        }

        // Send data
        i2c->DR = data[i];
    }

    // Wait for byte transfer to finish
    uint32_t timeout = 100000;
    while (!(i2c->SR1 & I2C_SR1_BTF) && timeout--) {
        if (i2c->SR1 & (I2C_SR1_ARLO | I2C_SR1_BERR | I2C_SR1_AF | I2C_SR1_OVR)) {
```

```
            return -1;  // Error occurred
        }
    }

    if (timeout == 0) {
        return -2;  // Timeout
    }

    return 0;
}

// Receive data
static int i2c_receive_data(i2c_reg_t *i2c, uint8_t *data, uint32_t length) {
    // Enable acknowledge for all bytes except the last one
    if (length > 1) {
        i2c->CR1 |= I2C_CR1_ACK;
    } else {
        i2c->CR1 &= ~I2C_CR1_ACK;
    }

    for (uint32_t i = 0; i < length; i++) {
        // For the last byte, generate stop condition before reading
        if (i == length - 1) {
            i2c->CR1 &= ~I2C_CR1_ACK;  // Disable acknowledge
            i2c_stop(i2c);             // Generate stop condition
        }

        // Wait for data to be received
        uint32_t timeout = 100000;
        while (!(i2c->SR1 & I2C_SR1_RXNE) && timeout--) {
            if (i2c->SR1 & (I2C_SR1_ARLO | I2C_SR1_BERR | I2C_SR1_AF | I2C_SR1_OVR)) {
                return -1;  // Error occurred
            }
        }

        if (timeout == 0) {
            return -2;  // Timeout
        }

        // Read data
        data[i] = i2c->DR;
    }

    return 0;
}

// I2C write function
```

```
int i2c_write(i2c_reg_t *i2c, uint8_t device_address,
              const uint8_t *data, uint32_t length) {
    if (i2c == NULL || data == NULL || length == 0) {
        return -1;
    }

    // Generate start condition
    if (i2c_start(i2c) != 0) {
        return -2;
    }

    // Send device address with write bit
    if (i2c_send_address(i2c, device_address, false) != 0) {
        i2c_stop(i2c);
        return -3;
    }

    // Send data
    if (i2c_send_data(i2c, data, length) != 0) {
        i2c_stop(i2c);
        return -4;
    }

    // Generate stop condition
    i2c_stop(i2c);

    return 0;
}

// I2C read function
int i2c_read(i2c_reg_t *i2c, uint8_t device_address,
             uint8_t *data, uint32_t length) {
    if (i2c == NULL || data == NULL || length == 0) {
        return -1;
    }

    // Generate start condition
    if (i2c_start(i2c) != 0) {
        return -2;
    }

    // Send device address with read bit
    if (i2c_send_address(i2c, device_address, true) != 0) {
        i2c_stop(i2c);
        return -3;
    }
```

```
    // Receive data
    if (i2c_receive_data(i2c, data, length) != 0) {
        return -4;
    }

    return 0;
}

// I2C write-read function (repeated start)
int i2c_write_read(i2c_reg_t *i2c, uint8_t device_address,
                   const uint8_t *write_data, uint32_t write_length,
                   uint8_t *read_data, uint32_t read_length) {
    if (i2c == NULL || write_data == NULL || read_data == NULL ||
        write_length == 0 || read_length == 0) {
        return -1;
    }

    // Generate start condition
    if (i2c_start(i2c) != 0) {
        return -2;
    }

    // Send device address with write bit
    if (i2c_send_address(i2c, device_address, false) != 0) {
        i2c_stop(i2c);
        return -3;
    }

    // Send write data
    if (i2c_send_data(i2c, write_data, write_length) != 0) {
        i2c_stop(i2c);
        return -4;
    }

    // Generate repeated start condition
    if (i2c_start(i2c) != 0) {
        i2c_stop(i2c);
        return -5;
    }

    // Send device address with read bit
    if (i2c_send_address(i2c, device_address, true) != 0) {
        i2c_stop(i2c);
        return -6;
    }

    // Receive read data
```

```
    if (i2c_receive_data(i2c, read_data, read_length) != 0) {
        return -7;
    }

    return 0;
}
```

UART (Universal Asynchronous Receiver-Transmitter)

UART is used for asynchronous serial communication between devices.

UART Driver Implementation

```
#include <stdint.h>
#include <stdbool.h>

// UART register structure
typedef struct {
    volatile uint32_t SR;       // Status register
    volatile uint32_t DR;       // Data register
    volatile uint32_t BRR;      // Baud rate register
    volatile uint32_t CR1;      // Control register 1
    volatile uint32_t CR2;      // Control register 2
    volatile uint32_t CR3;      // Control register 3
    volatile uint32_t GTPR;     // Guard time and prescaler register
} uart_reg_t;

// UART base addresses
#define USART1_BASE  0x40011000
#define USART2_BASE  0x40004400
#define USART3_BASE  0x40004800
#define UART4_BASE   0x40004C00
#define UART5_BASE   0x40005000

#define USART1  ((uart_reg_t*)USART1_BASE)
#define USART2  ((uart_reg_t*)USART2_BASE)
#define USART3  ((uart_reg_t*)USART3_BASE)
#define UART4   ((uart_reg_t*)UART4_BASE)
#define UART5   ((uart_reg_t*)UART5_BASE)

// UART status register bits
#define UART_SR_PE      (1UL << 0)   // Parity error
#define UART_SR_FE      (1UL << 1)   // Framing error
#define UART_SR_NE      (1UL << 2)   // Noise error
```

```
#define UART_SR_ORE     (1UL << 3)   // Overrun error
#define UART_SR_IDLE    (1UL << 4)   // IDLE line detected
#define UART_SR_RXNE    (1UL << 5)   // Read data register not empty
#define UART_SR_TC      (1UL << 6)   // Transmission complete
#define UART_SR_TXE     (1UL << 7)   // Transmit data register empty
#define UART_SR_LBD     (1UL << 8)   // LIN break detection flag
#define UART_SR_CTS     (1UL << 9)   // CTS flag

// UART control register 1 bits
#define UART_CR1_SBK    (1UL << 0)   // Send break
#define UART_CR1_RWU    (1UL << 1)   // Receiver wakeup
#define UART_CR1_RE     (1UL << 2)   // Receiver enable
#define UART_CR1_TE     (1UL << 3)   // Transmitter enable
#define UART_CR1_IDLEIE (1UL << 4)   // IDLE interrupt enable
#define UART_CR1_RXNEIE (1UL << 5)   // RXNE interrupt enable
#define UART_CR1_TCIE   (1UL << 6)   // Transmission complete interrupt enable
#define UART_CR1_TXEIE  (1UL << 7)   // TXE interrupt enable
#define UART_CR1_PEIE   (1UL << 8)   // PE interrupt enable
#define UART_CR1_PS     (1UL << 9)   // Parity selection
#define UART_CR1_PCE    (1UL << 10)  // Parity control enable
#define UART_CR1_WAKE   (1UL << 11)  // Wakeup method
#define UART_CR1_M      (1UL << 12)  // Word length
#define UART_CR1_UE     (1UL << 13)  // USART enable
#define UART_CR1_OVER8  (1UL << 15)  // Oversampling mode

// UART configuration structure
typedef struct {
    uint32_t baud_rate;         // Baud rate
    uint8_t data_bits;          // Data bits (8 or 9)
    uint8_t stop_bits;          // Stop bits (1 or 2)
    bool parity_enable;         // Parity enable
    bool parity_even;           // Even parity (true) or odd parity (false)
    uint32_t peripheral_clock;   // Peripheral clock frequency
} uart_config_t;

// Initialize UART peripheral
int uart_init(uart_reg_t *uart, const uart_config_t *config) {
    if (uart == NULL || config == NULL) {
        return -1;
    }

    // Disable UART
    uart->CR1 &= ~UART_CR1_UE;

    // Configure baud rate
    uint32_t baud_div = 0;
```

```
    if (config->baud_rate != 0) {
        baud_div = (config->peripheral_clock + (config->baud_rate / 2)) / config->baud_rate;
    }
    uart->BRR = baud_div;

    // Configure data bits
    if (config->data_bits == 9) {
        uart->CR1 |= UART_CR1_M;
    } else {
        uart->CR1 &= ~UART_CR1_M;
    }

    // Configure parity
    if (config->parity_enable) {
        uart->CR1 |= UART_CR1_PCE;
        if (config->parity_even) {
            uart->CR1 &= ~UART_CR1_PS;  // Even parity
        } else {
            uart->CR1 |= UART_CR1_PS;   // Odd parity
        }
    } else {
        uart->CR1 &= ~UART_CR1_PCE;
    }

    // Configure stop bits
    uart->CR2 &= ~(3UL << 12);  // Clear stop bits
    if (config->stop_bits == 2) {
        uart->CR2 |= (2UL << 12);   // 2 stop bits
    } else {
        uart->CR2 |= (0UL << 12);   // 1 stop bit
    }

    // Enable transmitter and receiver
    uart->CR1 |= UART_CR1_TE | UART_CR1_RE;

    // Enable UART
    uart->CR1 |= UART_CR1_UE;

    return 0;
}

// Send character
int uart_putc(uart_reg_t *uart, char c) {
    if (uart == NULL) {
        return -1;
    }
```

```
    // Wait for transmit data register to be empty
    uint32_t timeout = 100000;
    while (!(uart->SR & UART_SR_TXE) && timeout--) {
        // Wait
    }

    if (timeout == 0) {
        return -2;  // Timeout
    }

    // Send character
    uart->DR = c;

    return 0;
}

// Send string
int uart_puts(uart_reg_t *uart, const char *str) {
    if (uart == NULL || str == NULL) {
        return -1;
    }

    while (*str) {
        if (uart_putc(uart, *str++) != 0) {
            return -2;
        }
    }

    return 0;
}

// Receive character
int uart_getc(uart_reg_t *uart, char *c) {
    if (uart == NULL || c == NULL) {
        return -1;
    }

    // Check if data is available
    if (!(uart->SR & UART_SR_RXNE)) {
        return -2;  // No data available
    }

    // Read character
    *c = (char)(uart->DR & 0xFF);

    // Check for errors
    if (uart->SR & (UART_SR_PE | UART_SR_FE | UART_SR_NE | UART_SR_ORE)) {
```

```
        return -3;  // Error occurred
    }

    return 0;
}

// Check if data is available
bool uart_available(uart_reg_t *uart) {
    if (uart == NULL) {
        return false;
    }
    return (uart->SR & UART_SR_RXNE) ? true : false;
}

// Flush UART receive buffer
void uart_flush(uart_reg_t *uart) {
    if (uart == NULL) {
        return;
    }

    // Read all available data
    while (uart->SR & UART_SR_RXNE) {
        volatile uint32_t temp = uart->DR;
        (void)temp;  // Prevent compiler warning
    }
}
```

Practical Examples

Sensor Interface Example

```
#include <stdint.h>
#include <stdbool.h>

// Temperature sensor (I2C) driver
#define TEMP_SENSOR_ADDR  0x48

typedef struct {
    i2c_reg_t *i2c;
    uint8_t address;
} temp_sensor_t;

// Initialize temperature sensor
int temp_sensor_init(temp_sensor_t *sensor, i2c_reg_t *i2c) {
    if (sensor == NULL || i2c == NULL) {
```

```
        return -1;
    }

    sensor->i2c = i2c;
    sensor->address = TEMP_SENSOR_ADDR;

    // Configure sensor (example configuration)
    uint8_t config_data[] = {0x01, 0x60};  // Configuration register and value
    return i2c_write(sensor->i2c, sensor->address, config_data, 2);
}

// Read temperature
float temp_sensor_read(temp_sensor_t *sensor) {
    if (sensor == NULL) {
        return -1.0;
    }

    // Read temperature register
    uint8_t reg_addr = 0x00;  // Temperature register
    uint8_t data[2];

    if (i2c_write_read(sensor->i2c, sensor->address,
                       &reg_addr, 1, data, 2) != 0) {
        return -1.0;
    }

    // Convert to temperature value
    int16_t raw_temp = (data[0] << 8) | data[1];
    float temperature = raw_temp / 256.0;

    return temperature;
}

// EEPROM (I2C) driver
#define EEPROM_ADDR  0x50

typedef struct {
    i2c_reg_t *i2c;
    uint8_t address;
} eeprom_t;

// Initialize EEPROM
int eeprom_init(eeprom_t *eeprom, i2c_reg_t *i2c) {
    if (eeprom == NULL || i2c == NULL) {
        return -1;
    }
```

```
    eeprom->i2c = i2c;
    eeprom->address = EEPROM_ADDR;

    return 0;
}

// Write data to EEPROM
int eeprom_write(eeprom_t *eeprom, uint16_t address,
                 const uint8_t *data, uint32_t length) {
    if (eeprom == NULL || data == NULL || length == 0) {
        return -1;
    }

    // Write data in chunks (EEPROM page size is typically 16-64 bytes)
    uint32_t written = 0;
    while (written < length) {
        uint32_t chunk_size = (length - written > 16) ? 16 : (length - written);

        // Prepare data with address
        uint8_t write_data[18];  // 2 bytes address + 16 bytes data
        write_data[0] = (address + written) >> 8;     // High byte of address
        write_data[1] = (address + written) & 0xFF;   // Low byte of address
        for (uint32_t i = 0; i < chunk_size; i++) {
            write_data[2 + i] = data[written + i];
        }

        // Write chunk
        if (i2c_write(eeprom->i2c, eeprom->address,
                      write_data, 2 + chunk_size) != 0) {
            return -2;
        }

        written += chunk_size;

        // Wait for write to complete (EEPROM typical write time is 5ms)
        rt_delay_ms(5);
    }

    return 0;
}

// Read data from EEPROM
int eeprom_read(eeprom_t *eeprom, uint16_t address,
                uint8_t *data, uint32_t length) {
    if (eeprom == NULL || data == NULL || length == 0) {
        return -1;
    }
```

```
    // Send address
    uint8_t addr_data[2] = {(address >> 8) & 0xFF, address & 0xFF};
    if (i2c_write(eeprom->i2c, eeprom->address, addr_data, 2) != 0) {
        return -2;
    }

    // Read data
    return i2c_read(eeprom->i2c, eeprom->address, data, length);
}

// SPI Flash memory driver
typedef struct {
    spi_reg_t *spi;
    gpio_port_t cs_port;
    gpio_pin_t cs_pin;
} flash_t;

// Initialize flash memory
int flash_init(flash_t *flash, spi_reg_t *spi,
               gpio_port_t cs_port, gpio_pin_t cs_pin) {
    if (flash == NULL || spi == NULL) {
        return -1;
    }

    flash->spi = spi;
    flash->cs_port = cs_port;
    flash->cs_pin = cs_pin;

    // Configure CS pin as output
    gpio_config_output(cs_port, cs_pin);
    gpio_set(cs_port, cs_pin);  // Deselect flash

    return 0;
}

// Send command to flash
static int flash_send_command(flash_t *flash, const uint8_t *cmd, uint32_t cmd_len,
                              const uint8_t *data_out, uint8_t *data_in, uint32_t data_len)
    if (flash == NULL || cmd == NULL || cmd_len == 0) {
        return -1;
    }

    // Select flash
    spi_select_device(flash->cs_port, flash->cs_pin);

    // Send command
```

```
    if (spi_transmit(flash->spi, cmd, cmd_len) != 0) {
        spi_deselect_device(flash->cs_port, flash->cs_pin);
        return -2;
    }

    // Send/receive data
    if (data_out != NULL && data_in != NULL) {
        if (spi_transceive(flash->spi, data_out, data_in, data_len) != 0) {
            spi_deselect_device(flash->cs_port, flash->cs_pin);
            return -3;
        }
    } else if (data_out != NULL) {
        if (spi_transmit(flash->spi, data_out, data_len) != 0) {
            spi_deselect_device(flash->cs_port, flash->cs_pin);
            return -4;
        }
    } else if (data_in != NULL) {
        if (spi_receive(flash->spi, data_in, data_len) != 0) {
            spi_deselect_device(flash->cs_port, flash->cs_pin);
            return -5;
        }
    }

    // Deselect flash
    spi_deselect_device(flash->cs_port, flash->cs_pin);

    return 0;
}

// Read flash ID
int flash_read_id(flash_t *flash, uint8_t *id, uint32_t id_len) {
    if (flash == NULL || id == NULL || id_len == 0) {
        return -1;
    }

    uint8_t cmd = 0x9F;  // Read ID command
    return flash_send_command(flash, &cmd, 1, NULL, id, id_len);
}

// Read data from flash
int flash_read(flash_t *flash, uint32_t address, uint8_t *data, uint32_t length) {
    if (flash == NULL || data == NULL || length == 0) {
        return -1;
    }

    // Prepare read command with address
    uint8_t cmd[4] = {0x03,                    // Read command
```

```
                        (address >> 16) & 0xFF,  // Address high byte
                        (address >> 8) & 0xFF,   // Address middle byte
                        address & 0xFF};         // Address low byte

    return flash_send_command(flash, cmd, 4, NULL, data, length);
}

// Write data to flash (page program)
int flash_write(flash_t *flash, uint32_t address, const uint8_t *data, uint32_t length) {
    if (flash == NULL || data == NULL || length == 0) {
        return -1;
    }

    // Prepare write command with address
    uint8_t cmd[4] = {0x02,                  // Page program command
                        (address >> 16) & 0xFF,  // Address high byte
                        (address >> 8) & 0xFF,   // Address middle byte
                        address & 0xFF};         // Address low byte

    return flash_send_command(flash, cmd, 4, data, NULL, length);
}
```

Summary

Hardware interfaces are fundamental to embedded systems, enabling communication with various peripheral devices and sensors:

1. **GPIO** - Basic digital input/output for controlling LEDs, reading switches, etc.
2. **SPI** - High-speed synchronous communication for flash memory, displays, and sensors
3. **I2C** - Multi-master communication for EEPROMs, temperature sensors, and other devices
4. **UART** - Asynchronous serial communication for debug output and device communication

Key considerations for hardware interface implementation: - Proper initialization and configuration of peripheral registers - Error handling and timeout mechanisms - Efficient data transfer methods - Proper use of chip select signals for SPI devices - Addressing schemes for I2C devices - Baud rate configuration for UART communication - Interrupt-driven vs. polling-based approaches - Resource sharing and synchronization in multi-threaded environments

These interfaces form the foundation for connecting embedded systems to the physical world, enabling the development of sophisticated applications that interact with sensors, actuators, displays, storage devices, and communication modules.

Module 12: Embedded Systems Programming

Overview

Embedded systems programming is a specialized area of C development focused on creating software for dedicated computer systems that perform specific functions within larger mechanical or electrical systems. This module covers the unique challenges and techniques involved in programming for resource-constrained environments, real-time requirements, and direct hardware interaction.

Learning Objectives

By the end of this module, you should be able to: - Understand the fundamentals of embedded systems and their constraints - Program at the hardware level with direct memory-mapped I/O - Implement real-time programming techniques and scheduling - Work with embedded C extensions and specialized compilers - Interface with various hardware components and sensors - Debug and optimize code for embedded environments - Understand power management and resource optimization

Chapters

1. **Embedded Basics** - Introduction to embedded systems, architectures, and development environments
2. **Low-Level Programming** - Direct hardware access, memory-mapped I/O, and bit manipulation
3. **Embedded C Techniques** - Specialized C features for embedded systems, compiler extensions
4. **Real-time Programming** - Real-time constraints, scheduling, and timing considerations
5. **Hardware Interfaces** - Working with GPIO, SPI, I2C, UART, and other hardware interfaces

Prerequisites

Before starting this module, you should have a solid understanding of: - C programming fundamentals - Pointers and memory management - Basic electronics concepts - Computer architecture fundamentals

Key Concepts

- Resource-constrained programming
- Real-time constraints and deterministic behavior
- Direct hardware manipulation
- Memory-mapped I/O
- Bit manipulation and bit fields
- Interrupt handling
- Power management
- Cross-compilation for embedded targets
- Hardware-software interface design

Performance

Performance Analysis

Introduction

Performance analysis is the process of measuring, evaluating, and understanding the runtime behavior of programs to identify bottlenecks and optimization opportunities. In C programming, where performance is often critical, proper performance analysis is essential for creating efficient applications.

This chapter covers various techniques and tools for analyzing program performance, including profiling, benchmarking, and measurement methodologies.

Performance Metrics

Understanding the right metrics is crucial for effective performance analysis.

Time Complexity Metrics

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Simple timing function
double get_time(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

// Example function to measure
void example_function(int n) {
    volatile int sum = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            sum += i * j;
        }
    }
}
```

```c
// Measure execution time
void measure_execution_time(void) {
    double start_time, end_time;

    start_time = get_time();
    example_function(1000);
    end_time = get_time();

    printf("Execution time: %.6f seconds\n", end_time - start_time);
}
```

Memory Usage Metrics

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Memory tracking structure
typedef struct {
    size_t allocated_bytes;
    size_t freed_bytes;
    size_t current_usage;
    size_t peak_usage;
    size_t allocation_count;
    size_t deallocation_count;
} memory_stats_t;

static memory_stats_t mem_stats = {0};

// Custom malloc wrapper for tracking
void* tracked_malloc(size_t size) {
    void* ptr = malloc(size);
    if (ptr != NULL) {
        mem_stats.allocated_bytes += size;
        mem_stats.current_usage += size;
        mem_stats.allocation_count++;

        if (mem_stats.current_usage > mem_stats.peak_usage) {
            mem_stats.peak_usage = mem_stats.current_usage;
        }
    }
    return ptr;
}

// Custom free wrapper for tracking
```

```c
void tracked_free(void* ptr, size_t size) {
    if (ptr != NULL) {
        free(ptr);
        mem_stats.freed_bytes += size;
        mem_stats.current_usage -= size;
        mem_stats.deallocation_count++;
    }
}

// Print memory statistics
void print_memory_stats(void) {
    printf("Memory Statistics:\n");
    printf("  Allocated: %zu bytes\n", mem_stats.allocated_bytes);
    printf("  Freed: %zu bytes\n", mem_stats.freed_bytes);
    printf("  Current Usage: %zu bytes\n", mem_stats.current_usage);
    printf("  Peak Usage: %zu bytes\n", mem_stats.peak_usage);
    printf("  Allocations: %zu\n", mem_stats.allocation_count);
    printf("  Deallocations: %zu\n", mem_stats.deallocation_count);
}

// Reset memory statistics
void reset_memory_stats(void) {
    memset(&mem_stats, 0, sizeof(mem_stats));
}
```

CPU Usage Metrics

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/resource.h>

// Get CPU time usage
double get_cpu_time(void) {
    struct rusage usage;
    if (getrusage(RUSAGE_SELF, &usage) == 0) {
        return usage.ru_utime.tv_sec + usage.ru_utime.tv_usec / 1e6 +
               usage.ru_stime.tv_sec + usage.ru_stime.tv_usec / 1e6;
    }
    return -1.0;
}

// Example CPU-intensive function
void cpu_intensive_task(void) {
    volatile double result = 0.0;
    for (int i = 0; i < 1000000; i++) {
```

```
        result += i * 3.14159;
    }
}

// Measure CPU time
void measure_cpu_time(void) {
    double start_cpu, end_cpu;

    start_cpu = get_cpu_time();
    cpu_intensive_task();
    end_cpu = get_cpu_time();

    if (start_cpu >= 0 && end_cpu >= 0) {
        printf("CPU time used: %.6f seconds\n", end_cpu - start_cpu);
    }
}
```

Profiling Tools

Profiling tools provide detailed insights into program execution and resource usage.

GNU gprof

GNU gprof is a profiling tool that comes with GCC:

```
# Compile with profiling enabled
gcc -pg -o program program.c

# Run the program
./program

# Generate profiling report
gprof program gmon.out > analysis.txt
```

Example program for gprof profiling:

```
#include <stdio.h>
#include <stdlib.h>

// Function to profile
void function_a(void) {
    volatile int sum = 0;
    for (int i = 0; i < 1000000; i++) {
        sum += i;
    }
```

```
}

void function_b(void) {
    volatile int product = 1;
    for (int i = 1; i < 1000; i++) {
        product *= i;
    }
}

void function_c(void) {
    volatile double result = 0.0;
    for (int i = 0; i < 500000; i++) {
        result += i * 1.5;
    }
}

int main(void) {
    printf("Starting performance analysis...\n");

    // Call functions multiple times
    for (int i = 0; i < 100; i++) {
        function_a();
        if (i % 2 == 0) {
            function_b();
        }
        if (i % 5 == 0) {
            function_c();
        }
    }

    printf("Performance analysis complete.\n");
    return 0;
}
```

Valgrind Callgrind

Valgrind with Callgrind provides detailed call graph profiling:

```
# Compile normally
gcc -g -O2 -o program program.c

# Run with callgrind
valgrind --tool=callgrind ./program

# View results with kcachegrind
kcachegrind callgrind.out.*
```

perf (Linux Performance Events)

perf is a powerful performance analysis tool on Linux:

```
# Record performance data
perf record -g ./program

# View report
perf report

# Generate flame graph
perf script | flamegraph.pl > perf.svg
```

Custom Profiling Framework

Creating a custom profiling framework for specific needs:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_PROFILERS 100

// Profiler structure
typedef struct {
    char name[64];
    struct timespec start_time;
    long long total_time_ns;
    int call_count;
    int is_active;
} profiler_t;

static profiler_t profilers[MAX_PROFILERS];
static int profiler_count = 0;

// Find or create profiler
static profiler_t* get_profiler(const char* name) {
    // Try to find existing profiler
    for (int i = 0; i < profiler_count; i++) {
        if (strcmp(profilers[i].name, name) == 0) {
            return &profilers[i];
        }
    }
```

```
    // Create new profiler if space available
    if (profiler_count < MAX_PROFILERS) {
        profiler_t* p = &profilers[profiler_count];
        strncpy(p->name, name, sizeof(p->name) - 1);
        p->name[sizeof(p->name) - 1] = '\0';
        p->total_time_ns = 0;
        p->call_count = 0;
        p->is_active = 0;
        profiler_count++;
        return p;
    }

    return NULL;
}

// Start profiling a section
void profile_start(const char* name) {
    profiler_t* p = get_profiler(name);
    if (p != NULL && !p->is_active) {
        clock_gettime(CLOCK_MONOTONIC, &p->start_time);
        p->is_active = 1;
    }
}

// Stop profiling a section
void profile_stop(const char* name) {
    profiler_t* p = get_profiler(name);
    if (p != NULL && p->is_active) {
        struct timespec end_time;
        clock_gettime(CLOCK_MONOTONIC, &end_time);

        long long elapsed_ns = (end_time.tv_sec - p->start_time.tv_sec) * 1000000000LL +
                               (end_time.tv_nsec - p->start_time.tv_nsec);

        p->total_time_ns += elapsed_ns;
        p->call_count++;
        p->is_active = 0;
    }
}

// Print profiling results
void profile_print_results(void) {
    printf("\n=== Profiling Results ===\n");
    printf("%-30s %12s %10s %15s\n", "Function", "Calls", "Time(ms)", "Avg Time( s)");
    printf("%-30s %12s %10s %15s\n", "--------", "-----", "-------", "------------");

    for (int i = 0; i < profiler_count; i++) {
```

```
        profiler_t* p = &profilers[i];
        double total_time_ms = p->total_time_ns / 1e6;
        double avg_time_us = (p->call_count > 0) ?
                             (p->total_time_ns / 1e3) / p->call_count : 0;

        printf("%-30s %12d %10.3f %15.3f\n",
               p->name, p->call_count, total_time_ms, avg_time_us);
    }
}

// Reset profiling data
void profile_reset(void) {
    profiler_count = 0;
    memset(profilers, 0, sizeof(profilers));
}

// Example usage
void example_function_1(void) {
    profile_start("example_function_1");

    // Simulate work
    volatile int sum = 0;
    for (int i = 0; i < 100000; i++) {
        sum += i;
    }

    profile_stop("example_function_1");
}

void example_function_2(void) {
    profile_start("example_function_2");

    // Simulate work
    volatile double product = 1.0;
    for (int i = 1; i < 1000; i++) {
        product *= i * 0.1;
    }

    profile_stop("example_function_2");
}

int main(void) {
    printf("Starting custom profiling example...\n");

    // Profile different functions
    for (int i = 0; i < 1000; i++) {
        example_function_1();
```

```
        if (i % 2 == 0) {
            example_function_2();
        }
    }

    // Print results
    profile_print_results();

    return 0;
}
```

Benchmarking Framework

Creating a systematic benchmarking framework:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

// Benchmark result structure
typedef struct {
    double min_time;
    double max_time;
    double avg_time;
    double median_time;
    double stddev;
    int iterations;
} benchmark_result_t;

// Benchmark function type
typedef void (*benchmark_func_t)(void);

// Run benchmark
benchmark_result_t run_benchmark(benchmark_func_t func, int iterations) {
    benchmark_result_t result = {0};
    double* times = malloc(iterations * sizeof(double));

    if (times == NULL) {
        result.iterations = -1;
        return result;
    }

    struct timespec start, end;
```

```
    // Warm up
    for (int i = 0; i < 10; i++) {
        func();
    }

    // Run benchmark iterations
    for (int i = 0; i < iterations; i++) {
        clock_gettime(CLOCK_MONOTONIC, &start);
        func();
        clock_gettime(CLOCK_MONOTONIC, &end);

        times[i] = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
    }

    // Calculate statistics
    result.iterations = iterations;
    result.min_time = times[0];
    result.max_time = times[0];
    double sum = 0.0;

    for (int i = 0; i < iterations; i++) {
        if (times[i] < result.min_time) result.min_time = times[i];
        if (times[i] > result.max_time) result.max_time = times[i];
        sum += times[i];
    }

    result.avg_time = sum / iterations;

    // Calculate median
    // Sort times array (simplified bubble sort for small arrays)
    for (int i = 0; i < iterations - 1; i++) {
        for (int j = 0; j < iterations - i - 1; j++) {
            if (times[j] > times[j + 1]) {
                double temp = times[j];
                times[j] = times[j + 1];
                times[j + 1] = temp;
            }
        }
    }

    if (iterations % 2 == 0) {
        result.median_time = (times[iterations/2 - 1] + times[iterations/2]) / 2.0;
    } else {
        result.median_time = times[iterations/2];
    }

    // Calculate standard deviation
```

```
    double sum_sq_diff = 0.0;
    for (int i = 0; i < iterations; i++) {
        double diff = times[i] - result.avg_time;
        sum_sq_diff += diff * diff;
    }
    result.stddev = sqrt(sum_sq_diff / iterations);

    free(times);
    return result;
}

// Print benchmark results
void print_benchmark_result(const char* name, const benchmark_result_t* result) {
    if (result->iterations < 0) {
        printf("Benchmark '%s' failed\n", name);
        return;
    }

    printf("Benchmark: %s\n", name);
    printf("  Iterations: %d\n", result->iterations);
    printf("  Min Time: %.6f ms\n", result->min_time * 1000);
    printf("  Max Time: %.6f ms\n", result->max_time * 1000);
    printf("  Avg Time: %.6f ms\n", result->avg_time * 1000);
    printf("  Median Time: %.6f ms\n", result->median_time * 1000);
    printf("  Std Dev: %.6f ms\n", result->stddev * 1000);
    printf("\n");
}

// Example functions to benchmark
void benchmark_function_1(void) {
    volatile int sum = 0;
    for (int i = 0; i < 10000; i++) {
        sum += i;
    }
}

void benchmark_function_2(void) {
    volatile double product = 1.0;
    for (int i = 1; i < 1000; i++) {
        product *= (i * 0.01);
    }
}

int main(void) {
    printf("Starting benchmark tests...\n\n");

    // Run benchmarks
```

```
    benchmark_result_t result1 = run_benchmark(benchmark_function_1, 1000);
    print_benchmark_result("Sum Calculation", &result1);

    benchmark_result_t result2 = run_benchmark(benchmark_function_2, 1000);
    print_benchmark_result("Product Calculation", &result2);

    return 0;
}
```

Cache Performance Analysis

Understanding cache behavior is crucial for performance optimization:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define CACHE_LINE_SIZE 64
#define ARRAY_SIZE 1000000

// Test cache-friendly access pattern
void cache_friendly_access(int* array, int size) {
    for (int i = 0; i < size; i++) {
        array[i] += 1;
    }
}

// Test cache-unfriendly access pattern
void cache_unfriendly_access(int* array, int size) {
    for (int stride = 1; stride < size; stride *= 2) {
        for (int i = 0; i < size; i += stride) {
            array[i] += 1;
        }
    }
}

// Measure cache performance
void measure_cache_performance(void) {
    int* array = malloc(ARRAY_SIZE * sizeof(int));
    if (array == NULL) {
        printf("Failed to allocate memory\n");
        return;
    }

    struct timespec start, end;
```

```
    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i;
    }

    // Measure cache-friendly access
    clock_gettime(CLOCK_MONOTONIC, &start);
    cache_friendly_access(array, ARRAY_SIZE);
    clock_gettime(CLOCK_MONOTONIC, &end);

    double friendly_time = (end.tv_sec - start.tv_sec) +
                           (end.tv_nsec - start.tv_nsec) / 1e9;

    // Reset array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i;
    }

    // Measure cache-unfriendly access
    clock_gettime(CLOCK_MONOTONIC, &start);
    cache_unfriendly_access(array, ARRAY_SIZE);
    clock_gettime(CLOCK_MONOTONIC, &end);

    double unfriendly_time = (end.tv_sec - start.tv_sec) +
                             (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Cache Performance Analysis:\n");
    printf("  Cache-friendly access time: %.6f seconds\n", friendly_time);
    printf("  Cache-unfriendly access time: %.6f seconds\n", unfriendly_time);
    printf("  Performance ratio: %.2f\n", unfriendly_time / friendly_time);

    free(array);
}
```

Memory Access Pattern Analysis

Analyzing memory access patterns for optimization:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MATRIX_SIZE 1000
```

```c
// Row-major access (cache-friendly)
void row_major_access(int matrix[MATRIX_SIZE][MATRIX_SIZE]) {
    for (int i = 0; i < MATRIX_SIZE; i++) {
        for (int j = 0; j < MATRIX_SIZE; j++) {
            matrix[i][j] += 1;
        }
    }
}

// Column-major access (cache-unfriendly)
void column_major_access(int matrix[MATRIX_SIZE][MATRIX_SIZE]) {
    for (int j = 0; j < MATRIX_SIZE; j++) {
        for (int i = 0; i < MATRIX_SIZE; i++) {
            matrix[i][j] += 1;
        }
    }
}

// Measure memory access patterns
void measure_memory_access_patterns(void) {
    int(*matrix)[MATRIX_SIZE] = malloc(MATRIX_SIZE * sizeof(*matrix));
    if (matrix == NULL) {
        printf("Failed to allocate memory\n");
        return;
    }

    struct timespec start, end;

    // Initialize matrix
    for (int i = 0; i < MATRIX_SIZE; i++) {
        for (int j = 0; j < MATRIX_SIZE; j++) {
            matrix[i][j] = i * MATRIX_SIZE + j;
        }
    }

    // Measure row-major access
    clock_gettime(CLOCK_MONOTONIC, &start);
    row_major_access(matrix);
    clock_gettime(CLOCK_MONOTONIC, &end);

    double row_time = (end.tv_sec - start.tv_sec) +
                      (end.tv_nsec - start.tv_nsec) / 1e9;

    // Reset matrix
    for (int i = 0; i < MATRIX_SIZE; i++) {
        for (int j = 0; j < MATRIX_SIZE; j++) {
```

```
            matrix[i][j] = i * MATRIX_SIZE + j;
        }
    }

    // Measure column-major access
    clock_gettime(CLOCK_MONOTONIC, &start);
    column_major_access(matrix);
    clock_gettime(CLOCK_MONOTONIC, &end);

    double col_time = (end.tv_sec - start.tv_sec) +
                      (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Memory Access Pattern Analysis:\n");
    printf("  Row-major access time: %.6f seconds\n", row_time);
    printf("  Column-major access time: %.6f seconds\n", col_time);
    printf("  Performance ratio: %.2f\n", col_time / row_time);

    free(matrix);
}
```

Practical Examples

Comprehensive Performance Analysis Tool

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

// Performance analysis tool
typedef struct {
    char name[64];
    double execution_time;
    size_t memory_usage;
    int cpu_cycles;
} performance_data_t;

// Simple hash table implementation for performance data
#define HASH_TABLE_SIZE 101

typedef struct perf_node {
    performance_data_t data;
    struct perf_node* next;
} perf_node_t;
```

```c
static perf_node_t* hash_table[HASH_TABLE_SIZE];

// Hash function
static unsigned int hash(const char* str) {
    unsigned int hash = 5381;
    int c;
    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;
    }
    return hash % HASH_TABLE_SIZE;
}

// Add performance data
void add_performance_data(const char* name, double execution_time,
                          size_t memory_usage, int cpu_cycles) {
    unsigned int index = hash(name);

    perf_node_t* node = malloc(sizeof(perf_node_t));
    if (node == NULL) return;

    strncpy(node->data.name, name, sizeof(node->data.name) - 1);
    node->data.name[sizeof(node->data.name) - 1] = '\0';
    node->data.execution_time = execution_time;
    node->data.memory_usage = memory_usage;
    node->data.cpu_cycles = cpu_cycles;
    node->next = hash_table[index];
    hash_table[index] = node;
}

// Find performance data
performance_data_t* find_performance_data(const char* name) {
    unsigned int index = hash(name);
    perf_node_t* node = hash_table[index];

    while (node != NULL) {
        if (strcmp(node->data.name, name) == 0) {
            return &node->data;
        }
        node = node->next;
    }

    return NULL;
}

// Print all performance data
void print_all_performance_data(void) {
```

```
    printf("\n=== Performance Analysis Report ===\n");
    printf("%-30s %12s %15s %12s\n", "Function", "Time (ms)", "Memory (bytes)", "CPU Cycles"
    printf("%-30s %12s %15s %12s\n", "--------", "--------", "-------------", "----------");

    for (int i = 0; i < HASH_TABLE_SIZE; i++) {
        perf_node_t* node = hash_table[i];
        while (node != NULL) {
            printf("%-30s %12.3f %15zu %12d\n",
                   node->data.name,
                   node->data.execution_time * 1000,
                   node->data.memory_usage,
                   node->data.cpu_cycles);
            node = node->next;
        }
    }
}

// Clear performance data
void clear_performance_data(void) {
    for (int i = 0; i < HASH_TABLE_SIZE; i++) {
        perf_node_t* node = hash_table[i];
        while (node != NULL) {
            perf_node_t* next = node->next;
            free(node);
            node = next;
        }
        hash_table[i] = NULL;
    }
}

// Example performance test functions
void test_function_1(void) {
    volatile int sum = 0;
    for (int i = 0; i < 100000; i++) {
        sum += i;
    }
}

void test_function_2(void) {
    volatile double product = 1.0;
    for (int i = 1; i < 1000; i++) {
        product *= i * 0.1;
    }
}

void test_function_3(void) {
    char* buffer = malloc(1024);
```

```
    if (buffer != NULL) {
        memset(buffer, 0, 1024);
        free(buffer);
    }
}

int main(void) {
    struct timespec start, end;

    printf("Running comprehensive performance analysis...\n");

    // Test function 1
    clock_gettime(CLOCK_MONOTONIC, &start);
    test_function_1();
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time1 = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
    add_performance_data("test_function_1", time1, 0, 0);

    // Test function 2
    clock_gettime(CLOCK_MONOTONIC, &start);
    test_function_2();
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time2 = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
    add_performance_data("test_function_2", time2, 0, 0);

    // Test function 3
    clock_gettime(CLOCK_MONOTONIC, &start);
    test_function_3();
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time3 = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
    add_performance_data("test_function_3", time3, 1024, 0);

    // Print results
    print_all_performance_data();

    // Clean up
    clear_performance_data();

    return 0;
}
```

Summary

Performance analysis is a critical skill for C programmers, involving:

1. **Metrics Collection** - Time, memory, and CPU usage measurements

2. **Profiling Tools** - gprof, Valgrind, perf, and custom profilers
3. **Benchmarking** - Systematic performance testing with statistical analysis
4. **Cache Analysis** - Understanding memory access patterns and cache behavior
5. **Memory Analysis** - Tracking allocations and identifying memory bottlenecks

Key principles for effective performance analysis: - Use appropriate tools for different types of analysis - Measure performance in realistic conditions - Collect sufficient data for statistical significance - Understand the trade-offs between accuracy and overhead - Focus on the most critical performance bottlenecks - Document performance characteristics for future reference - Continuously monitor performance during development

These techniques enable developers to create high-performance C applications that efficiently utilize system resources while meeting performance requirements.

Optimization Techniques

Introduction

Performance optimization in C programming involves applying various techniques to improve execution speed, reduce memory usage, and enhance overall efficiency. This chapter explores fundamental and advanced optimization strategies, including algorithmic improvements, loop optimizations, data structure selection, and compiler-assisted optimizations.

Algorithmic Optimization

The most significant performance improvements often come from choosing better algorithms and data structures.

Time Complexity Analysis

Understanding algorithmic complexity is fundamental to optimization:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// O(n^2) bubble sort
void bubble_sort(int* arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// O(n log n) quick sort
void quick_sort(int* arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
```

```
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

int partition(int* arr, int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}

// O(n log n) merge sort
void merge_sort(int* arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        merge_sort(arr, l, m);
        merge_sort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void merge(int* arr, int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int* L = malloc(n1 * sizeof(int));
    int* R = malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
```

```
    j = 0;
    k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}

// Performance comparison
void compare_sorting_algorithms(void) {
    const int size = 10000;
    int* arr1 = malloc(size * sizeof(int));
    int* arr2 = malloc(size * sizeof(int));
    int* arr3 = malloc(size * sizeof(int));

    // Initialize arrays with random data
    for (int i = 0; i < size; i++) {
        arr1[i] = arr2[i] = arr3[i] = rand() % 10000;
    }

    struct timespec start, end;

    // Test bubble sort
    clock_gettime(CLOCK_MONOTONIC, &start);
    bubble_sort(arr1, size);
```

```
    clock_gettime(CLOCK_MONOTONIC, &end);
    double bubble_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test quick sort
    clock_gettime(CLOCK_MONOTONIC, &start);
    quick_sort(arr2, 0, size - 1);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double quick_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test merge sort
    clock_gettime(CLOCK_MONOTONIC, &start);
    merge_sort(arr3, 0, size - 1);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double merge_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Sorting Algorithm Performance (n=%d):\n", size);
    printf("  Bubble Sort: %.6f seconds\n", bubble_time);
    printf("  Quick Sort:  %.6f seconds\n", quick_time);
    printf("  Merge Sort:  %.6f seconds\n", merge_time);

    free(arr1);
    free(arr2);
    free(arr3);
}
```

Search Algorithm Optimization

Choosing the right search algorithm can dramatically improve performance:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// O(n) linear search
int linear_search(int* arr, int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

// O(log n) binary search (requires sorted array)
int binary_search(int* arr, int n, int target) {
    int left = 0;
```

```
    int right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        }

        if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}

// Hash table implementation for O(1) average case search
#define HASH_TABLE_SIZE 1009

typedef struct hash_node {
    int key;
    int value;
    struct hash_node* next;
} hash_node_t;

static hash_node_t* hash_table[HASH_TABLE_SIZE];

unsigned int hash_function(int key) {
    return (unsigned int)(key % HASH_TABLE_SIZE);
}

void hash_table_insert(int key, int value) {
    unsigned int index = hash_function(key);

    hash_node_t* node = malloc(sizeof(hash_node_t));
    node->key = key;
    node->value = value;
    node->next = hash_table[index];
    hash_table[index] = node;
}

int hash_table_search(int key) {
    unsigned int index = hash_function(key);
    hash_node_t* node = hash_table[index];
```

```
    while (node != NULL) {
        if (node->key == key) {
            return node->value;
        }
        node = node->next;
    }

    return -1;
}

void hash_table_clear(void) {
    for (int i = 0; i < HASH_TABLE_SIZE; i++) {
        hash_node_t* node = hash_table[i];
        while (node != NULL) {
            hash_node_t* next = node->next;
            free(node);
            node = next;
        }
        hash_table[i] = NULL;
    }
}

// Performance comparison
void compare_search_algorithms(void) {
    const int size = 100000;
    int* sorted_array = malloc(size * sizeof(int));
    int* unsorted_array = malloc(size * sizeof(int));

    // Initialize arrays
    for (int i = 0; i < size; i++) {
        sorted_array[i] = i * 2;  // Even numbers
        unsorted_array[i] = rand() % (size * 2);
        hash_table_insert(unsorted_array[i], i);
    }

    int target = sorted_array[size / 2];  // Target in middle

    struct timespec start, end;

    // Test linear search
    clock_gettime(CLOCK_MONOTONIC, &start);
    int linear_result = linear_search(unsorted_array, size, target);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double linear_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test binary search
```

```
    clock_gettime(CLOCK_MONOTONIC, &start);
    int binary_result = binary_search(sorted_array, size, target);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double binary_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test hash table search
    clock_gettime(CLOCK_MONOTONIC, &start);
    int hash_result = hash_table_search(target);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double hash_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Search Algorithm Performance (n=%d):\n", size);
    printf("  Linear Search: %.9f seconds (result: %d)\n", linear_time, linear_result);
    printf("  Binary Search: %.9f seconds (result: %d)\n", binary_time, binary_result);
    printf("  Hash Search:   %.9f seconds (result: %d)\n", hash_time, hash_result);

    free(sorted_array);
    free(unsorted_array);
    hash_table_clear();
}
```

Loop Optimization

Loop optimization techniques can significantly improve performance by reducing overhead and improving cache locality.

Loop Unrolling

Loop unrolling reduces loop overhead by processing multiple iterations in each loop cycle:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Standard loop
void standard_loop(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = arr[i] * 2 + 1;
    }
}

// Unrolled loop (factor of 4)
void unrolled_loop(int* arr, int n) {
    int i;
```

```
    // Process 4 elements per iteration
    for (i = 0; i < n - 3; i += 4) {
        arr[i] = arr[i] * 2 + 1;
        arr[i + 1] = arr[i + 1] * 2 + 1;
        arr[i + 2] = arr[i + 2] * 2 + 1;
        arr[i + 3] = arr[i + 3] * 2 + 1;
    }

    // Handle remaining elements
    for (; i < n; i++) {
        arr[i] = arr[i] * 2 + 1;
    }
}

// Duff's device (advanced loop unrolling)
void duff_device(int* to, int* from, int count) {
    int n = (count + 7) / 8;
    switch (count % 8) {
        case 0: do { *to++ = *from++;
        case 7:      *to++ = *from++;
        case 6:      *to++ = *from++;
        case 5:      *to++ = *from++;
        case 4:      *to++ = *from++;
        case 3:      *to++ = *from++;
        case 2:      *to++ = *from++;
        case 1:      *to++ = *from++;
                } while (--n > 0);
    }
}

// Performance comparison
void compare_loop_optimizations(void) {
    const int size = 1000000;
    int* arr1 = malloc(size * sizeof(int));
    int* arr2 = malloc(size * sizeof(int));

    // Initialize arrays
    for (int i = 0; i < size; i++) {
        arr1[i] = arr2[i] = i;
    }

    struct timespec start, end;

    // Test standard loop
    clock_gettime(CLOCK_MONOTONIC, &start);
    standard_loop(arr1, size);
```

```
    clock_gettime(CLOCK_MONOTONIC, &end);
    double standard_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9

    // Test unrolled loop
    clock_gettime(CLOCK_MONOTONIC, &start);
    unrolled_loop(arr2, size);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double unrolled_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9

    printf("Loop Optimization Performance (n=%d):\n", size);
    printf("  Standard Loop: %.6f seconds\n", standard_time);
    printf("  Unrolled Loop: %.6f seconds\n", unrolled_time);
    printf("  Speedup: %.2fx\n", standard_time / unrolled_time);

    free(arr1);
    free(arr2);
}
```

Loop Fusion and Fission

Combining or splitting loops can improve cache performance:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000000

// Loop fusion example
void fused_loops(float* a, float* b, float* c, float* d, int n) {
    for (int i = 0; i < n; i++) {
        a[i] = b[i] + c[i];  // First operation
        d[i] = a[i] * 2.0f;  // Second operation (depends on first)
    }
}

// Loop fission example
void fissioned_loops(float* a, float* b, float* c, float* d, int n) {
    // First loop
    for (int i = 0; i < n; i++) {
        a[i] = b[i] + c[i];
    }

    // Second loop
    for (int i = 0; i < n; i++) {
        d[i] = a[i] * 2.0f;
```

```
    }
}

// Cache-friendly loop fusion
void cache_friendly_fusion(float* a, float* b, float* c, float* d, int n) {
    const int block_size = 1024;  // Cache line friendly

    for (int block = 0; block < n; block += block_size) {
        int end = (block + block_size < n) ? block + block_size : n;

        // Process block
        for (int i = block; i < end; i++) {
            a[i] = b[i] + c[i];
            d[i] = a[i] * 2.0f;
        }
    }
}

// Performance comparison
void compare_loop_transformations(void) {
    float* a1 = malloc(SIZE * sizeof(float));
    float* b1 = malloc(SIZE * sizeof(float));
    float* c1 = malloc(SIZE * sizeof(float));
    float* d1 = malloc(SIZE * sizeof(float));

    float* a2 = malloc(SIZE * sizeof(float));
    float* b2 = malloc(SIZE * sizeof(float));
    float* c2 = malloc(SIZE * sizeof(float));
    float* d2 = malloc(SIZE * sizeof(float));

    float* a3 = malloc(SIZE * sizeof(float));
    float* b3 = malloc(SIZE * sizeof(float));
    float* c3 = malloc(SIZE * sizeof(float));
    float* d3 = malloc(SIZE * sizeof(float));

    // Initialize arrays
    for (int i = 0; i < SIZE; i++) {
        b1[i] = b2[i] = b3[i] = (float)i;
        c1[i] = c2[i] = c3[i] = (float)(i * 2);
    }

    struct timespec start, end;

    // Test fused loops
    clock_gettime(CLOCK_MONOTONIC, &start);
    fused_loops(a1, b1, c1, d1, SIZE);
```

```
    clock_gettime(CLOCK_MONOTONIC, &end);
    double fused_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test fissioned loops
    clock_gettime(CLOCK_MONOTONIC, &start);
    fissioned_loops(a2, b2, c2, d2, SIZE);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double fissioned_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e

    // Test cache-friendly fusion
    clock_gettime(CLOCK_MONOTONIC, &start);
    cache_friendly_fusion(a3, b3, c3, d3, SIZE);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double cache_friendly_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec)

    printf("Loop Transformation Performance (n=%d):\n", SIZE);
    printf("  Fused Loops:        %.6f seconds\n", fused_time);
    printf("  Fissioned Loops:    %.6f seconds\n", fissioned_time);
    printf("  Cache-Friendly:     %.6f seconds\n", cache_friendly_time);

    free(a1); free(b1); free(c1); free(d1);
    free(a2); free(b2); free(c2); free(d2);
    free(a3); free(b3); free(c3); free(d3);
}
```

Data Structure Optimization

Choosing the right data structures can have a dramatic impact on performance.

Array vs Linked List

Different data structures have different performance characteristics:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Array-based implementation
typedef struct {
    int* data;
    int size;
    int capacity;
} array_list_t;
```

```
array_list_t* array_list_create(int initial_capacity) {
    array_list_t* list = malloc(sizeof(array_list_t));
    list->data = malloc(initial_capacity * sizeof(int));
    list->size = 0;
    list->capacity = initial_capacity;
    return list;
}

void array_list_append(array_list_t* list, int value) {
    if (list->size >= list->capacity) {
        list->capacity *= 2;
        list->data = realloc(list->data, list->capacity * sizeof(int));
    }
    list->data[list->size++] = value;
}

int array_list_get(array_list_t* list, int index) {
    if (index >= 0 && index < list->size) {
        return list->data[index];
    }
    return -1;
}

void array_list_free(array_list_t* list) {
    free(list->data);
    free(list);
}

// Linked list implementation
typedef struct node {
    int data;
    struct node* next;
} node_t;

typedef struct {
    node_t* head;
    int size;
} linked_list_t;

linked_list_t* linked_list_create(void) {
    linked_list_t* list = malloc(sizeof(linked_list_t));
    list->head = NULL;
    list->size = 0;
    return list;
}

void linked_list_append(linked_list_t* list, int value) {
```

```
    node_t* new_node = malloc(sizeof(node_t));
    new_node->data = value;
    new_node->next = NULL;

    if (list->head == NULL) {
        list->head = new_node;
    } else {
        node_t* current = list->head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = new_node;
    }
    list->size++;
}

int linked_list_get(linked_list_t* list, int index) {
    node_t* current = list->head;
    for (int i = 0; i < index && current != NULL; i++) {
        current = current->next;
    }
    return (current != NULL) ? current->data : -1;
}

void linked_list_free(linked_list_t* list) {
    node_t* current = list->head;
    while (current != NULL) {
        node_t* next = current->next;
        free(current);
        current = next;
    }
    free(list);
}

// Performance comparison
void compare_data_structures(void) {
    const int size = 100000;

    struct timespec start, end;

    // Test array list
    clock_gettime(CLOCK_MONOTONIC, &start);
    array_list_t* arr_list = array_list_create(1000);
    for (int i = 0; i < size; i++) {
        array_list_append(arr_list, i);
    }
    for (int i = 0; i < size; i += 100) {
```

```
        array_list_get(arr_list, i);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    double array_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test linked list
    clock_gettime(CLOCK_MONOTONIC, &start);
    linked_list_t* link_list = linked_list_create();
    for (int i = 0; i < size; i++) {
        linked_list_append(link_list, i);
    }
    for (int i = 0; i < size; i += 100) {
        linked_list_get(link_list, i);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    double linked_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Data Structure Performance (n=%d):\n", size);
    printf("  Array List:  %.6f seconds\n", array_time);
    printf("  Linked List: %.6f seconds\n", linked_time);
    printf("  Ratio: %.2fx\n", linked_time / array_time);

    array_list_free(arr_list);
    linked_list_free(link_list);
}
```

Compiler Optimization Techniques

Understanding and leveraging compiler optimizations is crucial for performance.

Compiler Flags and Pragmas

```
#include <stdio.h>
#include <stdlib.h>

// Function inlining hints
static inline int fast_add(int a, int b) {
    return a + b;
}

// Restrict keyword for pointer aliasing
void vector_add(float* restrict a, float* restrict b, float* restrict c, int n) {
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
```

```
    }
}

// Loop unrolling pragma
void optimized_loop(float* arr, int n) {
    #pragma GCC unroll 4
    for (int i = 0; i < n; i++) {
        arr[i] = arr[i] * 2.0f + 1.0f;
    }
}

// Vectorization hints
void vectorized_operation(float* arr, int n) {
    #pragma GCC ivdep
    for (int i = 0; i < n; i++) {
        arr[i] = arr[i] * arr[i] + 2.0f * arr[i] + 1.0f;
    }
}

// Branch prediction hints
void branch_prediction_example(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        if (__builtin_expect(arr[i] > 0, 1)) {  // Likely true
            arr[i] *= 2;
        } else {
            arr[i] = 0;
        }
    }
}

// Memory alignment
struct aligned_struct {
    int a;
    double b;
    char c;
} __attribute__((aligned(16)));

// Example usage
void compiler_optimization_examples(void) {
    const int size = 1000000;
    float* a = malloc(size * sizeof(float));
    float* b = malloc(size * sizeof(float));
    float* c = malloc(size * sizeof(float));

    // Initialize arrays
    for (int i = 0; i < size; i++) {
        a[i] = b[i] = (float)i;
```

```
    }

    // Test optimized operations
    vector_add(a, b, c, size);
    optimized_loop(c, size);
    vectorized_operation(c, size);

    printf("Compiler optimization examples completed\n");

    free(a);
    free(b);
    free(c);
}
```

Mathematical Optimization

Mathematical optimizations can significantly improve performance in numerical computations.

Fast Mathematical Operations

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

// Fast integer square root (Newton's method)
int fast_sqrt(int x) {
    if (x == 0) return 0;

    int guess = x;
    int better_guess;

    do {
        better_guess = (guess + x / guess) / 2;
        if (better_guess >= guess) break;
        guess = better_guess;
    } while (1);

    return guess;
}

// Fast integer power
int fast_power(int base, int exp) {
    int result = 1;
```

```
    while (exp > 0) {
        if (exp & 1) {
            result *= base;
        }
        base *= base;
        exp >>= 1;
    }
    return result;
}

// Bit manipulation for multiplication by powers of 2
int multiply_by_power_of_2(int value, int power) {
    return value << power;  // Equivalent to value * (2^power)
}

// Bit manipulation for division by powers of 2
int divide_by_power_of_2(int value, int power) {
    return value >> power;  // Equivalent to value / (2^power) for positive numbers
}

// Fast absolute value
int fast_abs(int x) {
    int mask = x >> (sizeof(int) * 8 - 1);
    return (x + mask) ^ mask;
}

// Performance comparison
void compare_mathematical_optimizations(void) {
    const int iterations = 1000000;

    struct timespec start, end;

    // Test standard sqrt
    clock_gettime(CLOCK_MONOTONIC, &start);
    volatile double sum1 = 0;
    for (int i = 1; i <= iterations; i++) {
        sum1 += sqrt(i);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    double sqrt_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test fast sqrt
    clock_gettime(CLOCK_MONOTONIC, &start);
    volatile int sum2 = 0;
    for (int i = 1; i <= iterations; i++) {
        sum2 += fast_sqrt(i);
    }
```

```
    clock_gettime(CLOCK_MONOTONIC, &end);
    double fast_sqrt_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e

    // Test standard power
    clock_gettime(CLOCK_MONOTONIC, &start);
    volatile int sum3 = 0;
    for (int i = 1; i <= 1000; i++) {
        sum3 += (int)pow(2, i % 10);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    double power_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Test fast power
    clock_gettime(CLOCK_MONOTONIC, &start);
    volatile int sum4 = 0;
    for (int i = 1; i <= 1000; i++) {
        sum4 += fast_power(2, i % 10);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    double fast_power_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1

    printf("Mathematical Optimization Performance:\n");
    printf("  Standard sqrt: %.6f seconds\n", sqrt_time);
    printf("  Fast sqrt:     %.6f seconds\n", fast_sqrt_time);
    printf("  Standard power: %.6f seconds\n", power_time);
    printf("  Fast power:     %.6f seconds\n", fast_power_time);
}
```

Practical Examples

Image Processing Optimization

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define IMAGE_WIDTH 1920
#define IMAGE_HEIGHT 1080

// Simple image structure
typedef struct {
    unsigned char* data;
    int width;
    int height;
```

```
} image_t;

// Create image
image_t* create_image(int width, int height) {
    image_t* img = malloc(sizeof(image_t));
    img->data = malloc(width * height * sizeof(unsigned char));
    img->width = width;
    img->height = height;
    return img;
}

// Free image
void free_image(image_t* img) {
    free(img->data);
    free(img);
}

// Naive blur implementation
void naive_blur(image_t* src, image_t* dst) {
    for (int y = 1; y < src->height - 1; y++) {
        for (int x = 1; x < src->width - 1; x++) {
            int sum = 0;
            for (int ky = -1; ky <= 1; ky++) {
                for (int kx = -1; kx <= 1; kx++) {
                    sum += src->data[(y + ky) * src->width + (x + kx)];
                }
            }
            dst->data[y * src->width + x] = sum / 9;
        }
    }
}

// Optimized blur with loop unrolling
void optimized_blur(image_t* src, image_t* dst) {
    for (int y = 1; y < src->height - 1; y++) {
        for (int x = 1; x < src->width - 1; x++) {
            // Unroll the inner loops
            int sum = src->data[(y-1) * src->width + (x-1)] +
                      src->data[(y-1) * src->width + x] +
                      src->data[(y-1) * src->width + (x+1)] +
                      src->data[y * src->width + (x-1)] +
                      src->data[y * src->width + x] +
                      src->data[y * src->width + (x+1)] +
                      src->data[(y+1) * src->width + (x-1)] +
                      src->data[(y+1) * src->width + x] +
                      src->data[(y+1) * src->width + (x+1)];
```

```
            dst->data[y * src->width + x] = sum / 9;
        }
    }
}

// SIMD-style optimization using multiple operations per loop
void simd_style_blur(image_t* src, image_t* dst) {
    for (int y = 1; y < src->height - 1; y++) {
        // Process 4 pixels at a time
        for (int x = 1; x < src->width - 4; x += 4) {
            // Process 4 pixels in parallel (simplified)
            for (int i = 0; i < 4; i++) {
                int sum = src->data[(y-1) * src->width + (x+i-1)] +
                          src->data[(y-1) * src->width + (x+i)] +
                          src->data[(y-1) * src->width + (x+i+1)] +
                          src->data[y * src->width + (x+i-1)] +
                          src->data[y * src->width + (x+i)] +
                          src->data[y * src->width + (x+i+1)] +
                          src->data[(y+1) * src->width + (x+i-1)] +
                          src->data[(y+1) * src->width + (x+i)] +
                          src->data[(y+1) * src->width + (x+i+1)];
                dst->data[y * src->width + (x+i)] = sum / 9;
            }
        }
    }
}

// Performance comparison
void compare_image_processing(void) {
    image_t* src = create_image(IMAGE_WIDTH, IMAGE_HEIGHT);
    image_t* dst1 = create_image(IMAGE_WIDTH, IMAGE_HEIGHT);
    image_t* dst2 = create_image(IMAGE_WIDTH, IMAGE_HEIGHT);
    image_t* dst3 = create_image(IMAGE_WIDTH, IMAGE_HEIGHT);

    // Initialize source image with random data
    for (int i = 0; i < IMAGE_WIDTH * IMAGE_HEIGHT; i++) {
        src->data[i] = rand() % 256;
    }

    struct timespec start, end;

    // Test naive blur
    clock_gettime(CLOCK_MONOTONIC, &start);
    naive_blur(src, dst1);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double naive_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
```

```
    // Test optimized blur
    clock_gettime(CLOCK_MONOTONIC, &start);
    optimized_blur(src, dst2);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double optimized_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e

    // Test SIMD-style blur
    clock_gettime(CLOCK_MONOTONIC, &start);
    simd_style_blur(src, dst3);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double simd_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Image Processing Performance (%dx%d):\n", IMAGE_WIDTH, IMAGE_HEIGHT);
    printf("  Naive Blur:     %.6f seconds\n", naive_time);
    printf("  Optimized Blur: %.6f seconds\n", optimized_time);
    printf("  SIMD-style:     %.6f seconds\n", simd_time);
    printf("  Speedup:        %.2fx\n", naive_time / simd_time);

    free_image(src);
    free_image(dst1);
    free_image(dst2);
    free_image(dst3);
}
```

Summary

Optimization techniques in C programming encompass a wide range of approaches:

1. **Algorithmic Optimization** - Choosing better algorithms and data structures
2. **Loop Optimization** - Unrolling, fusion, and restructuring loops
3. **Data Structure Optimization** - Selecting appropriate data structures for the task
4. **Compiler Optimization** - Leveraging compiler features and flags
5. **Mathematical Optimization** - Fast mathematical operations and bit manipulation
6. **Memory Access Optimization** - Improving cache locality and reducing memory overhead

Key principles for effective optimization: - Profile first to identify actual bottlenecks - Focus on algorithmic improvements before micro-optimizations - Understand the trade-offs between performance and maintainability - Use appropriate compiler optimization flags - Consider the target hardware architecture - Test optimizations thoroughly to ensure correctness - Document optimization decisions for future maintenance

These techniques enable developers to create high-performance C applications that efficiently utilize system resources while maintaining code quality and readability.

Memory Optimization

Introduction

Memory optimization is a critical aspect of performance tuning in C programs, especially in resource-constrained environments and high-performance applications. Efficient memory management can significantly impact program performance through improved cache utilization, reduced allocation overhead, and better data locality. This chapter explores techniques for optimizing memory usage, from allocation strategies to cache-friendly data structures.

Memory Hierarchy and Performance

Understanding the memory hierarchy is fundamental to effective memory optimization. Modern computer systems have multiple levels of memory with vastly different access times.

Memory Latency Comparison

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

// Example illustrating memory access time differences
void memory_latency_example(void) {
    printf("Typical Access Times:\n");
    printf("- CPU Register: ~0.3 ns\n");
    printf("- L1 Cache:     ~1 ns\n");
    printf("- L2 Cache:     ~4 ns\n");
    printf("- L3 Cache:     ~15 ns\n");
    printf("- Main Memory:  ~100 ns\n");
    printf("- SSD:          ~10  s\n");
    printf("- HDD:          ~10 ms\n");
}
```

Cache Behavior Impact

Cache misses can cause significant performance degradation. Understanding how data is accessed affects cache performance:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MATRIX_SIZE 1024
#define ITERATIONS 100

// Row-major access (cache-friendly)
double row_major_sum(double matrix[MATRIX_SIZE][MATRIX_SIZE]) {
    double sum = 0.0;
    for (int i = 0; i < MATRIX_SIZE; i++) {
        for (int j = 0; j < MATRIX_SIZE; j++) {
            sum += matrix[i][j];
        }
    }
    return sum;
}

// Column-major access (cache-unfriendly)
double column_major_sum(double matrix[MATRIX_SIZE][MATRIX_SIZE]) {
    double sum = 0.0;
    for (int j = 0; j < MATRIX_SIZE; j++) {
        for (int i = 0; i < MATRIX_SIZE; i++) {
            sum += matrix[i][j];
        }
    }
    return sum;
}

void cache_performance_comparison(void) {
    double (*matrix)[MATRIX_SIZE] = malloc(MATRIX_SIZE * sizeof(*matrix));

    // Initialize matrix
    for (int i = 0; i < MATRIX_SIZE; i++) {
        for (int j = 0; j < MATRIX_SIZE; j++) {
            matrix[i][j] = (double)(i * MATRIX_SIZE + j);
        }
    }

    clock_t start, end;

    // Row-major timing
    start = clock();
    for (int i = 0; i < ITERATIONS; i++) {
        volatile double sum = row_major_sum(matrix);
    }
    end = clock();
```

```
    double row_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Column-major timing
    start = clock();
    for (int i = 0; i < ITERATIONS; i++) {
        volatile double sum = column_major_sum(matrix);
    }
    end = clock();
    double col_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Row-major time: %.4f seconds\n", row_time);
    printf("Column-major time: %.4f seconds\n", col_time);
    printf("Performance difference: %.2fx\n", col_time / row_time);

    free(matrix);
}
```

Memory Allocation Optimization

Standard memory allocation functions like `malloc()` (see system programming) and `free()` (see linear structures) can be significant performance bottlenecks in performance-critical applications.

Custom Memory Allocators

For specific use cases, custom allocators can provide significant performance improvements:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>

// Simple pool allocator
typedef struct {
    void *memory_pool;
    size_t pool_size;
    size_t block_size;
    size_t num_blocks;
    uint8_t *free_blocks;
    size_t next_free;
} pool_allocator_t;

// Initialize pool allocator
pool_allocator_t* pool_init(size_t block_size, size_t num_blocks) {
    pool_allocator_t *pool = malloc(sizeof(pool_allocator_t));
```

```
    if (!pool) return NULL;

    pool->pool_size = block_size * num_blocks;
    pool->memory_pool = malloc(pool->pool_size);
    if (!pool->memory_pool) {
        free(pool);
        return NULL;
    }

    pool->block_size = block_size;
    pool->num_blocks = num_blocks;
    pool->free_blocks = calloc(num_blocks, sizeof(uint8_t));
    pool->next_free = 0;

    return pool;
}

// Allocate block from pool
void* pool_alloc(pool_allocator_t *pool) {
    if (!pool || pool->next_free >= pool->num_blocks) {
        return NULL;  // Pool exhausted
    }

    // Find free block
    for (size_t i = 0; i < pool->num_blocks; i++) {
        size_t index = (pool->next_free + i) % pool->num_blocks;
        if (!pool->free_blocks[index]) {
            pool->free_blocks[index] = 1;
            pool->next_free = (index + 1) % pool->num_blocks;
            return (char*)pool->memory_pool + (index * pool->block_size);
        }
    }

    return NULL;  // Should not happen
}

// Free block back to pool
void pool_free(pool_allocator_t *pool, void *ptr) {
    if (!pool || !ptr) return;

    // Calculate block index
    ptrdiff_t offset = (char*)ptr - (char*)pool->memory_pool;
    if (offset < 0 || offset >= (ptrdiff_t)pool->pool_size) return;

    size_t index = offset / pool->block_size;
    if (index < pool->num_blocks) {
        pool->free_blocks[index] = 0;
```

```
        pool->next_free = index;  // Prefer this block for next allocation
    }
}

// Destroy pool allocator
void pool_destroy(pool_allocator_t *pool) {
    if (pool) {
        free(pool->memory_pool);
        free(pool->free_blocks);
        free(pool);
    }
}

// Example usage
void pool_allocator_example(void) {
    // Create pool for 100 blocks of 64 bytes each
    pool_allocator_t *pool = pool_init(64, 100);
    if (!pool) {
        printf("Failed to create pool\n");
        return;
    }

    // Allocate some blocks
    void *ptr1 = pool_alloc(pool);
    void *ptr2 = pool_alloc(pool);
    void *ptr3 = pool_alloc(pool);

    printf("Allocated 3 blocks\n");

    // Free some blocks
    pool_free(pool, ptr1);
    pool_free(pool, ptr3);

    printf("Freed 2 blocks\n");

    // Allocate more blocks (should reuse freed ones)
    void *ptr4 = pool_alloc(pool);
    void *ptr5 = pool_alloc(pool);

    printf("Allocated 2 more blocks\n");

    // Cleanup
    pool_destroy(pool);
}
```

Stack Allocation vs Heap Allocation

Stack allocation is significantly faster than heap allocation but has size and lifetime limitations:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ARRAY_SIZE 10000
#define ITERATIONS 100000

// Stack allocation
void stack_allocation_test(void) {
    // Fast allocation on stack
    int array[ARRAY_SIZE];

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i;
    }
}

// Heap allocation
void heap_allocation_test(void) {
    // Slower allocation on heap
    int *array = malloc(ARRAY_SIZE * sizeof(int));
    if (array) {
        // Initialize array
        for (int i = 0; i < ARRAY_SIZE; i++) {
            array[i] = i;
        }
        free(array);
    }
}

void allocation_performance_comparison(void) {
    clock_t start, end;

    // Stack allocation timing
    start = clock();
    for (int i = 0; i < ITERATIONS; i++) {
        stack_allocation_test();
    }
    end = clock();
    double stack_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Heap allocation timing
```

```
    start = clock();
    for (int i = 0; i < ITERATIONS; i++) {
        heap_allocation_test();
    }
    end = clock();
    double heap_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Stack allocation time: %.4f seconds\n", stack_time);
    printf("Heap allocation time: %.4f seconds\n", heap_time);
    printf("Stack is %.2fx faster\n", heap_time / stack_time);
}
```

Data Structure Optimization

Choosing the right data structures and organizing them for cache efficiency can dramatically improve performance.

Structure of Arrays vs Array of Structures

The way data is organized in memory can significantly impact cache performance:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define NUM_POINTS 1000000

// Array of Structures (AoS) - cache-unfriendly for component-wise operations
typedef struct {
    float x, y, z;
} point_aos_t;

// Structure of Arrays (SoA) - cache-friendly for component-wise operations
typedef struct {
    float *x, *y, *z;
} point_soa_t;

// Initialize AoS
point_aos_t* init_aos(int count) {
    point_aos_t *points = malloc(count * sizeof(point_aos_t));
    for (int i = 0; i < count; i++) {
        points[i].x = (float)i;
        points[i].y = (float)(i * 2);
        points[i].z = (float)(i * 3);
```

```
    }
    return points;
}

// Initialize SoA
point_soa_t* init_soa(int count) {
    point_soa_t *points = malloc(sizeof(point_soa_t));
    points->x = malloc(count * sizeof(float));
    points->y = malloc(count * sizeof(float));
    points->z = malloc(count * sizeof(float));

    for (int i = 0; i < count; i++) {
        points->x[i] = (float)i;
        points->y[i] = (float)(i * 2);
        points->z[i] = (float)(i * 3);
    }
    return points;
}

// Process all x coordinates (AoS version)
float process_x_aos(point_aos_t *points, int count) {
    float sum = 0.0f;
    for (int i = 0; i < count; i++) {
        sum += points[i].x * 2.0f;
    }
    return sum;
}

// Process all x coordinates (SoA version)
float process_x_soa(point_soa_t *points, int count) {
    float sum = 0.0f;
    for (int i = 0; i < count; i++) {
        sum += points->x[i] * 2.0f;
    }
    return sum;
}

void aos_vs_soa_comparison(void) {
    point_aos_t *aos_points = init_aos(NUM_POINTS);
    point_soa_t *soa_points = init_soa(NUM_POINTS);

    clock_t start, end;

    // AoS timing
    start = clock();
    volatile float aos_result = process_x_aos(aos_points, NUM_POINTS);
    end = clock();
```

```
    double aos_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // SoA timing
    start = clock();
    volatile float soa_result = process_x_soa(soa_points, NUM_POINTS);
    end = clock();
    double soa_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("AoS processing time: %.4f seconds\n", aos_time);
    printf("SoA processing time: %.4f seconds\n", soa_time);
    printf("SoA is %.2fx faster\n", aos_time / soa_time);

    // Cleanup
    free(aos_points);
    free(soa_points->x);
    free(soa_points->y);
    free(soa_points->z);
    free(soa_points);
}
```

Memory Alignment

Proper memory alignment can improve performance by ensuring efficient memory access:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdalign.h>
#include <time.h>

#define ARRAY_SIZE 1000000

// Unaligned structure
typedef struct {
    char a;      // 1 byte
    int b;       // 4 bytes (3 bytes padding before)
    short c;     // 2 bytes
    // 1 byte padding at end
} unaligned_struct_t;

// Aligned structure
typedef struct {
    alignas(16) int b;    // 4 bytes, aligned to 16-byte boundary
    char a;               // 1 byte
    short c;              // 2 bytes
    // 9 bytes padding at end for 16-byte alignment
} aligned_struct_t;
```

```
void alignment_performance_test(void) {
    // Allocate arrays
    unaligned_struct_t *unaligned_array = malloc(ARRAY_SIZE * sizeof(unaligned_struct_t));
    aligned_struct_t *aligned_array = malloc(ARRAY_SIZE * sizeof(aligned_struct_t));

    clock_t start, end;

    // Test unaligned access
    start = clock();
    for (int i = 0; i < ARRAY_SIZE; i++) {
        unaligned_array[i].b = i * 2;
    }
    end = clock();
    double unaligned_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Test aligned access
    start = clock();
    for (int i = 0; i < ARRAY_SIZE; i++) {
        aligned_array[i].b = i * 2;
    }
    end = clock();
    double aligned_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Unaligned access time: %.4f seconds\n", unaligned_time);
    printf("Aligned access time: %.4f seconds\n", aligned_time);
    printf("Aligned is %.2fx faster\n", unaligned_time / aligned_time);

    // Print structure sizes
    printf("Unaligned struct size: %zu bytes\n", sizeof(unaligned_struct_t));
    printf("Aligned struct size: %zu bytes\n", sizeof(aligned_struct_t));

    free(unaligned_array);
    free(aligned_array);
}
```

Memory Layout Optimization

Understanding and optimizing memory layout can lead to significant performance improvements.

Data Locality

Keeping related data close together in memory improves cache performance:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MATRIX_ROWS 1000
#define MATRIX_COLS 1000

// Poor data locality - accessing data with large strides
void poor_locality_example(int **matrix) {
    long sum = 0;
    // Access data column by column (poor cache usage)
    for (int col = 0; col < MATRIX_COLS; col++) {
        for (int row = 0; row < MATRIX_ROWS; row++) {
            sum += matrix[row][col];
        }
    }
    // Prevent optimization
    volatile long dummy = sum;
}

// Good data locality - accessing data sequentially
void good_locality_example(int **matrix) {
    long sum = 0;
    // Access data row by row (good cache usage)
    for (int row = 0; row < MATRIX_ROWS; row++) {
        for (int col = 0; col < MATRIX_COLS; col++) {
            sum += matrix[row][col];
        }
    }
    // Prevent optimization
    volatile long dummy = sum;
}

void data_locality_comparison(void) {
    // Allocate matrix
    int **matrix = malloc(MATRIX_ROWS * sizeof(int*));
    for (int i = 0; i < MATRIX_ROWS; i++) {
        matrix[i] = malloc(MATRIX_COLS * sizeof(int));
        for (int j = 0; j < MATRIX_COLS; j++) {
            matrix[i][j] = i * MATRIX_COLS + j;
        }
    }

    clock_t start, end;

    // Poor locality timing
    start = clock();
```

```
    poor_locality_example(matrix);
    end = clock();
    double poor_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Good locality timing
    start = clock();
    good_locality_example(matrix);
    end = clock();
    double good_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Poor locality time: %.4f seconds\n", poor_time);
    printf("Good locality time: %.4f seconds\n", good_time);
    printf("Good locality is %.2fx faster\n", poor_time / good_time);

    // Cleanup
    for (int i = 0; i < MATRIX_ROWS; i++) {
        free(matrix[i]);
    }
    free(matrix);
}
```

Memory Pooling

Memory pooling can reduce allocation overhead and improve data locality:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define POOL_SIZE 1024 * 1024  // 1MB pool
#define BLOCK_SIZE 256

typedef struct memory_block {
    struct memory_block *next;
    size_t size;
    int in_use;
    char data[1];  // Flexible array member
} memory_block_t;

typedef struct {
    char *pool_start;
    char *pool_end;
    memory_block_t *free_list;
    size_t total_size;
} memory_pool_t;
```

```
// Initialize memory pool
memory_pool_t* pool_create(size_t size) {
    memory_pool_t *pool = malloc(sizeof(memory_pool_t));
    if (!pool) return NULL;

    pool->pool_start = malloc(size);
    if (!pool->pool_start) {
        free(pool);
        return NULL;
    }

    pool->pool_end = pool->pool_start + size;
    pool->total_size = size;
    pool->free_list = (memory_block_t*)pool->pool_start;

    // Initialize first block to cover entire pool
    pool->free_list->next = NULL;
    pool->free_list->size = size - sizeof(memory_block_t);
    pool->free_list->in_use = 0;

    return pool;
}

// Allocate memory from pool
void* pool_malloc(memory_pool_t *pool, size_t size) {
    if (!pool || size == 0) return NULL;

    // Add space for block header
    size_t required_size = sizeof(memory_block_t) + size;

    // Find suitable block
    memory_block_t *prev = NULL;
    memory_block_t *current = pool->free_list;

    while (current) {
        if (!current->in_use && current->size >= size) {
            // Found suitable block
            current->in_use = 1;

            // Split block if large enough
            size_t remaining = current->size - size;
            if (remaining > sizeof(memory_block_t) + 32) {  // Minimum block size
                memory_block_t *new_block = (memory_block_t*)
                    ((char*)current + sizeof(memory_block_t) + size);
                new_block->size = remaining - sizeof(memory_block_t);
                new_block->in_use = 0;
                new_block->next = current->next;
```

```
                current->size = size;
                current->next = new_block;
            }

            return current->data;
        }

        prev = current;
        current = current->next;
    }

    return NULL;  // No suitable block found
}

// Free memory back to pool
void pool_free(memory_pool_t *pool, void *ptr) {
    if (!pool || !ptr) return;

    // Get block header
    memory_block_t *block = (memory_block_t*)((char*)ptr - sizeof(memory_block_t));
    block->in_use = 0;

    // Coalesce adjacent free blocks
    memory_block_t *current = pool->free_list;
    while (current) {
        if (!current->in_use) {
            // Check next block
            memory_block_t *next = current->next;
            if (next && !next->in_use) {
                // Merge with next block
                current->size += sizeof(memory_block_t) + next->size;
                current->next = next->next;
                continue;  // Check same block again
            }
        }
        current = current->next;
    }
}

// Destroy memory pool
void pool_destroy(memory_pool_t *pool) {
    if (pool) {
        free(pool->pool_start);
        free(pool);
    }
}
```

```
// Example usage
void memory_pool_example(void) {
    memory_pool_t *pool = pool_create(POOL_SIZE);
    if (!pool) {
        printf("Failed to create memory pool\n");
        return;
    }

    printf("Created memory pool of %zu bytes\n", POOL_SIZE);

    // Allocate some blocks
    void *ptr1 = pool_malloc(pool, 128);
    void *ptr2 = pool_malloc(pool, 256);
    void *ptr3 = pool_malloc(pool, 64);

    printf("Allocated 3 blocks\n");

    // Free some blocks
    pool_free(pool, ptr1);
    pool_free(pool, ptr3);

    printf("Freed 2 blocks\n");

    // Allocate more blocks (should reuse freed space)
    void *ptr4 = pool_malloc(pool, 96);
    void *ptr5 = pool_malloc(pool, 160);

    printf("Allocated 2 more blocks\n");

    // Cleanup
    pool_destroy(pool);
}
```

Memory Access Patterns

Understanding and optimizing memory access patterns can significantly improve performance.

Sequential vs Random Access

Sequential memory access is much faster than random access due to cache behavior:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```c
#define ARRAY_SIZE 1000000
#define ACCESS_COUNT 1000000

void sequential_access_test(int *array) {
    long sum = 0;
    for (int i = 0; i < ARRAY_SIZE; i++) {
        sum += array[i];
    }
    // Prevent optimization
    volatile long dummy = sum;
}

void random_access_test(int *array) {
    long sum = 0;
    // Access elements in random order
    for (int i = 0; i < ARRAY_SIZE; i++) {
        int index = rand() % ARRAY_SIZE;
        sum += array[index];
    }
    // Prevent optimization
    volatile long dummy = sum;
}

void access_pattern_comparison(void) {
    int *array = malloc(ARRAY_SIZE * sizeof(int));
    if (!array) {
        printf("Failed to allocate array\n");
        return;
    }

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i;
    }

    clock_t start, end;

    // Sequential access timing
    start = clock();
    sequential_access_test(array);
    end = clock();
    double sequential_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Random access timing
    srand(42);  // Fixed seed for reproducible results
    start = clock();
```

```
    random_access_test(array);
    end = clock();
    double random_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Sequential access time: %.4f seconds\n", sequential_time);
    printf("Random access time: %.4f seconds\n", random_time);
    printf("Sequential is %.2fx faster\n", random_time / sequential_time);

    free(array);
}
```

Practical Memory Optimization Techniques

Reducing Memory Fragmentation

Memory fragmentation can lead to inefficient memory usage and allocation failures:

```
#include <stdio.h>
#include <stdlib.h>

#define POOL_SIZE 10000

// Simple example showing fragmentation
void fragmentation_example(void) {
    void *ptrs[1000];

    printf("Demonstrating memory fragmentation:\n");

    // Allocate many small blocks
    for (int i = 0; i < 1000; i++) {
        ptrs[i] = malloc(10);
        if (!ptrs[i]) {
            printf("Allocation failed at %d\n", i);
            return;
        }
    }

    // Free every other block
    for (int i = 0; i < 1000; i += 2) {
        free(ptrs[i]);
        ptrs[i] = NULL;
    }

    // Try to allocate a larger block
    void *large_block = malloc(500);
```

```
    if (large_block) {
        printf("Large block allocation succeeded\n");
        free(large_block);
    } else {
        printf("Large block allocation failed due to fragmentation\n");
    }

    // Free remaining blocks
    for (int i = 1; i < 1000; i += 2) {
        if (ptrs[i]) {
            free(ptrs[i]);
        }
    }
}
```

Memory Prefetching

Modern processors support prefetching instructions to improve memory access performance:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ARRAY_SIZE 1000000

// Manual prefetching example
void prefetch_example(int *array) {
    long sum = 0;

    for (int i = 0; i < ARRAY_SIZE; i++) {
        // Prefetch next cache line (typically 64 bytes)
        if (i + 16 < ARRAY_SIZE) {
            __builtin_prefetch(&array[i + 16], 0, 3);
        }

        sum += array[i];
    }

    // Prevent optimization
    volatile long dummy = sum;
}

// Without prefetching
void no_prefetch_example(int *array) {
    long sum = 0;
```

```
    for (int i = 0; i < ARRAY_SIZE; i++) {
        sum += array[i];
    }

    // Prevent optimization
    volatile long dummy = sum;
}

void prefetch_comparison(void) {
    int *array = malloc(ARRAY_SIZE * sizeof(int));
    if (!array) {
        printf("Failed to allocate array\n");
        return;
    }

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i;
    }

    clock_t start, end;

    // Without prefetching
    start = clock();
    no_prefetch_example(array);
    end = clock();
    double no_prefetch_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // With prefetching
    start = clock();
    prefetch_example(array);
    end = clock();
    double prefetch_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Without prefetching: %.4f seconds\n", no_prefetch_time);
    printf("With prefetching: %.4f seconds\n", prefetch_time);
    printf("Prefetching improvement: %.2fx\n", no_prefetch_time / prefetch_time);

    free(array);
}
```

Memory Debugging and Analysis

Tools and techniques for analyzing memory usage and identifying optimization opportunities.

Simple Memory Tracker

A basic memory tracking system to monitor allocations:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Simple memory tracker
typedef struct mem_block {
    struct mem_block *next;
    void *ptr;
    size_t size;
    const char *file;
    int line;
} mem_block_t;

static mem_block_t *mem_list = NULL;
static size_t total_allocated = 0;
static size_t peak_allocated = 0;
static size_t allocation_count = 0;

// Track memory allocation
void* tracked_malloc(size_t size, const char *file, int line) {
    void *ptr = malloc(size);
    if (!ptr) return NULL;

    // Add to tracking list
    mem_block_t *block = malloc(sizeof(mem_block_t));
    if (!block) {
        free(ptr);
        return NULL;
    }

    block->ptr = ptr;
    block->size = size;
    block->file = file;
    block->line = line;
    block->next = mem_list;
    mem_list = block;

    total_allocated += size;
    allocation_count++;

    if (total_allocated > peak_allocated) {
        peak_allocated = total_allocated;
    }
```

```
    return ptr;
}

// Track memory deallocation
void tracked_free(void *ptr, const char *file, int line) {
    if (!ptr) return;

    // Find and remove from tracking list
    mem_block_t *prev = NULL;
    mem_block_t *current = mem_list;

    while (current) {
        if (current->ptr == ptr) {
            if (prev) {
                prev->next = current->next;
            } else {
                mem_list = current->next;
            }

            total_allocated -= current->size;
            free(current);
            free(ptr);
            return;
        }

        prev = current;
        current = current->next;
    }

    // Not found in tracking list - possible double free
    printf("Warning: Attempt to free untracked pointer at %s:%d\n", file, line);
    free(ptr);
}

// Print memory statistics
void print_memory_stats(void) {
    printf("\n=== Memory Statistics ===\n");
    printf("Total allocations: %zu\n", allocation_count);
    printf("Currently allocated: %zu bytes\n", total_allocated);
    printf("Peak allocation: %zu bytes\n", peak_allocated);

    if (mem_list) {
        printf("Leaked blocks:\n");
        mem_block_t *current = mem_list;
        while (current) {
            printf("  %p: %zu bytes at %s:%d\n",
```

```
                    current->ptr, current->size, current->file, current->line);
            current = current->next;
        }
    } else {
        printf("No memory leaks detected\n");
    }
}

// Cleanup tracking (call at program exit)
void cleanup_memory_tracking(void) {
    mem_block_t *current = mem_list;
    while (current) {
        mem_block_t *next = current->next;
        free(current->ptr);
        free(current);
        current = next;
    }
    mem_list = NULL;
}

// Macros for easier usage
#define MALLOC(size) tracked_malloc(size, __FILE__, __LINE__)
#define FREE(ptr) tracked_free(ptr, __FILE__, __LINE__)

// Example usage
void memory_tracking_example(void) {
    printf("Memory tracking example:\n");

    // Allocate some memory
    int *array1 = (int*)MALLOC(100 * sizeof(int));
    char *string = (char*)MALLOC(50);

    // Use the memory
    for (int i = 0; i < 100; i++) {
        array1[i] = i;
    }
    strcpy(string, "Hello, World!");

    printf("Allocated array and string\n");
    print_memory_stats();

    // Free one block
    FREE(array1);
    printf("Freed array\n");
    print_memory_stats();
```

```
    // Intentionally leak string to demonstrate leak detection
    printf("Intentionally leaking string...\n");
    print_memory_stats();

    // Cleanup
    cleanup_memory_tracking();
}
```

Summary

Memory optimization is a multifaceted discipline that requires understanding of:

1. **Memory Hierarchy** - How different levels of memory affect performance
2. **Allocation Strategies** - Custom allocators, stack vs heap allocation
3. **Data Structure Design** - Structure of Arrays vs Array of Structures, alignment
4. **Memory Layout** - Data locality, memory pooling
5. **Access Patterns** - Sequential vs random access, prefetching
6. **Fragmentation Management** - Reducing memory fragmentation
7. **Analysis Tools** - Memory tracking and debugging

Key principles for effective memory optimization:

- **Profile First** - Measure before and after optimization to verify improvements
- **Understand Your Data** - Know how your data is accessed and sized
- **Consider the Cache** - Design for cache-friendly access patterns
- **Minimize Allocations** - Reduce allocation frequency and overhead
- **Track Memory Usage** - Monitor for leaks and fragmentation
- **Use Appropriate Tools** - Leverage profilers and analysis tools

These techniques, when applied appropriately, can lead to significant performance improvements in C programs, especially in resource-constrained or performance-critical applications.

Advanced Optimization

Introduction

Advanced optimization techniques go beyond basic algorithmic improvements to leverage hardware-specific features, compiler optimizations, and parallel processing capabilities. This chapter explores sophisticated methods for maximizing performance in C programs, including vectorization, parallel processing, compiler-assisted optimizations, and specialized techniques for specific architectures.

Compiler Optimization

Modern compilers provide powerful optimization capabilities that can significantly improve performance with minimal programmer effort.

Optimization Levels

Different compiler optimization levels offer varying degrees of optimization:

```
// Example showing the impact of different optimization levels
// Compile with: gcc -O0, -O1, -O2, -O3, -Os

#include <stdio.h>
#include <time.h>

#define ITERATIONS 100000000

// Simple loop that can be optimized
int compute_sum(int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += i * 2 + 1;
    }
    return sum;
}

void optimization_levels_demo(void) {
    clock_t start = clock();
    volatile int result = compute_sum(ITERATIONS);
    clock_t end = clock();
```

```
    double time_spent = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Computation time: %.4f seconds\n", time_spent);
}
```

Compiler Hints and Pragmas

Compiler hints can guide optimization decisions:

```
#include <stdio.h>
#include <stdlib.h>

// Function likely to be called frequently
__attribute__((hot))
int hot_function(int x) {
    return x * x + 2 * x + 1;
}

// Function unlikely to be called
__attribute__((cold))
void error_handler(void) {
    printf("Error occurred!\n");
    exit(1);
}

// Loop unrolling hint
void loop_unrolling_example(void) {
    int array[1000];

    // Hint to unroll this loop
    #pragma GCC unroll 4
    for (int i = 0; i < 1000; i++) {
        array[i] = i * i;
    }
}

// Vectorization hints
void vectorization_example(float *a, float *b, float *c, int n) {
    // Tell compiler this loop can be vectorized
    #pragma GCC ivdep
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}
```

Profile-Guided Optimization (PGO)

PGO uses runtime profiling data to optimize code placement and branch prediction:

```
// Example of code that benefits from PGO
#include <stdio.h>

int process_data(int data) {
    // Branch that depends on input data
    if (data < 100) {
        // Common case
        return data * 2;
    } else if (data < 1000) {
        // Less common case
        return data + 100;
    } else {
        // Rare case
        return data / 2;
    }
}

void pgo_example(void) {
    // In a real PGO scenario, you would run the program with
    // representative data to generate profile data
    for (int i = 0; i < 10000; i++) {
        int result = process_data(i);
        // Use result to prevent optimization
        volatile int dummy = result;
    }
}
```

Vectorization

Vectorization allows processing multiple data elements simultaneously using SIMD (Single Instruction, Multiple Data) instructions.

Manual Vectorization with Intrinsics

Using compiler intrinsics for explicit vectorization:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#ifdef __SSE__
```

```
#include <xmmintrin.h>  // SSE
#endif

#ifdef __AVX__
#include <immintrin.h>  // AVX
#endif

// Scalar addition
void scalar_add(float *a, float *b, float *c, int n) {
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}

#ifdef __SSE__
// SSE vectorized addition
void sse_add(float *a, float *b, float *c, int n) {
    int i = 0;

    // Process 4 elements at a time
    for (; i <= n - 4; i += 4) {
        __m128 va = _mm_load_ps(&a[i]);
        __m128 vb = _mm_load_ps(&b[i]);
        __m128 vc = _mm_add_ps(va, vb);
        _mm_store_ps(&c[i], vc);
    }

    // Handle remaining elements
    for (; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}
#endif

#ifdef __AVX__
// AVX vectorized addition
void avx_add(float *a, float *b, float *c, int n) {
    int i = 0;

    // Process 8 elements at a time
    for (; i <= n - 8; i += 8) {
        __m256 va = _mm256_load_ps(&a[i]);
        __m256 vb = _mm256_load_ps(&b[i]);
        __m256 vc = _mm256_add_ps(va, vb);
        _mm256_store_ps(&c[i], vc);
    }
```

```
    // Handle remaining elements
    for (; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}
#endif

void vectorization_comparison(void) {
    const int size = 1000000;
    float *a = malloc(size * sizeof(float));
    float *b = malloc(size * sizeof(float));
    float *c = malloc(size * sizeof(float));

    // Initialize arrays
    for (int i = 0; i < size; i++) {
        a[i] = (float)i;
        b[i] = (float)(i * 2);
    }

    clock_t start, end;

    // Scalar version
    start = clock();
    scalar_add(a, b, c, size);
    end = clock();
    double scalar_time = ((double)(end - start)) / CLOCKS_PER_SEC;

#ifdef __SSE__
    // SSE version
    start = clock();
    sse_add(a, b, c, size);
    end = clock();
    double sse_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("SSE time: %.4f seconds (%.2fx speedup)\n",
           sse_time, scalar_time / sse_time);
#endif

#ifdef __AVX__
    // AVX version
    start = clock();
    avx_add(a, b, c, size);
    end = clock();
    double avx_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("AVX time: %.4f seconds (%.2fx speedup)\n",
           avx_time, scalar_time / avx_time);
#endif
```

```
    printf("Scalar time: %.4f seconds\n", scalar_time);

    free(a);
    free(b);
    free(c);
}
```

Auto-Vectorization

Modern compilers can automatically vectorize suitable loops:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ARRAY_SIZE 1000000

// Compiler can auto-vectorize this function
void auto_vectorizable_loop(float *a, float *b, float *c, int n) {
    // Conditions for auto-vectorization:
    // 1. No dependencies between iterations
    // 2. Simple arithmetic operations
    // 3. Contiguous memory access
    // 4. Known loop bounds

    for (int i = 0; i < n; i++) {
        c[i] = a[i] * b[i] + 1.0f;
    }
}

// This loop cannot be auto-vectorized due to dependency
void non_vectorizable_loop(int *a, int n) {
    // Each iteration depends on the previous one
    for (int i = 1; i < n; i++) {
        a[i] = a[i-1] + i;
    }
}

void auto_vectorization_example(void) {
    float *a = malloc(ARRAY_SIZE * sizeof(float));
    float *b = malloc(ARRAY_SIZE * sizeof(float));
    float *c = malloc(ARRAY_SIZE * sizeof(float));

    // Initialize arrays
    for (int i = 0; i < ARRAY_SIZE; i++) {
        a[i] = (float)i;
```

```
        b[i] = (float)(i * 2);
    }

    clock_t start = clock();
    auto_vectorizable_loop(a, b, c, ARRAY_SIZE);
    clock_t end = clock();

    double time_spent = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Auto-vectorized loop time: %.4f seconds\n", time_spent);

    // Verify result (check first few elements)
    printf("First 5 results: ");
    for (int i = 0; i < 5; i++) {
        printf("%.0f ", c[i]);
    }
    printf("\n");

    free(a);
    free(b);
    free(c);
}
```

Parallel Processing

Utilizing multiple CPU cores can dramatically improve performance for suitable workloads.

OpenMP for Shared-Memory Parallelism

OpenMP provides a simple way to parallelize loops and sections:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

#define ARRAY_SIZE 1000000

// Sequential matrix multiplication
void sequential_matrix_mult(int n, double *a, double *b, double *c) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++) {
                sum += a[i * n + k] * b[k * n + j];
```

```
            }
            c[i * n + j] = sum;
        }
    }
}

// Parallel matrix multiplication with OpenMP
void parallel_matrix_mult(int n, double *a, double *b, double *c) {
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++) {
                sum += a[i * n + k] * b[k * n + j];
            }
            c[i * n + j] = sum;
        }
    }
}

// Parallel reduction example
double parallel_sum(double *array, int n) {
    double sum = 0.0;

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < n; i++) {
        sum += array[i];
    }

    return sum;
}

void openmp_example(void) {
    const int size = 1000;
    double *a = malloc(size * size * sizeof(double));
    double *b = malloc(size * size * sizeof(double));
    double *c = malloc(size * size * sizeof(double));

    // Initialize matrices
    for (int i = 0; i < size * size; i++) {
        a[i] = (double)(i % 100);
        b[i] = (double)((i + 1) % 100);
    }

    // Measure sequential version
    clock_t start = clock();
    sequential_matrix_mult(size, a, b, c);
```

```
    clock_t end = clock();
    double sequential_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Measure parallel version
    start = clock();
    parallel_matrix_mult(size, a, b, c);
    end = clock();
    double parallel_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Sequential time: %.4f seconds\n", sequential_time);
    printf("Parallel time: %.4f seconds\n", parallel_time);
    printf("Speedup: %.2fx\n", sequential_time / parallel_time);

    // Show number of threads
    printf("Number of OpenMP threads: %d\n", omp_get_max_threads());

    free(a);
    free(b);
    free(c);
}
```

Thread-Level Parallelism with Pthreads

Manual thread management for fine-grained control:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define NUM_THREADS 4
#define ARRAY_SIZE 1000000

typedef struct {
    int *array;
    int start;
    int end;
    long long sum;
} thread_data_t;

// Thread function to compute partial sum
void* compute_partial_sum(void *arg) {
    thread_data_t *data = (thread_data_t*)arg;

    data->sum = 0;
    for (int i = data->start; i < data->end; i++) {
```

```
        data->sum += data->array[i];
    }

    return NULL;
}

long long parallel_sum_pthreads(int *array, int n) {
    pthread_t threads[NUM_THREADS];
    thread_data_t thread_data[NUM_THREADS];

    // Divide work among threads
    int chunk_size = n / NUM_THREADS;

    for (int i = 0; i < NUM_THREADS; i++) {
        thread_data[i].array = array;
        thread_data[i].start = i * chunk_size;
        thread_data[i].end = (i == NUM_THREADS - 1) ? n : (i + 1) * chunk_size;

        pthread_create(&threads[i], NULL, compute_partial_sum, &thread_data[i]);
    }

    // Wait for all threads and accumulate results
    long long total_sum = 0;
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
        total_sum += thread_data[i].sum;
    }

    return total_sum;
}

long long sequential_sum(int *array, int n) {
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += array[i];
    }
    return sum;
}

void pthreads_example(void) {
    int *array = malloc(ARRAY_SIZE * sizeof(int));

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i % 1000;
    }
```

```
    // Sequential sum
    clock_t start = clock();
    long long seq_sum = sequential_sum(array, ARRAY_SIZE);
    clock_t end = clock();
    double sequential_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Parallel sum
    start = clock();
    long long par_sum = parallel_sum_pthreads(array, ARRAY_SIZE);
    end = clock();
    double parallel_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Sequential sum: %lld (time: %.4f seconds)\n", seq_sum, sequential_time);
    printf("Parallel sum: %lld (time: %.4f seconds)\n", par_sum, parallel_time);
    printf("Speedup: %.2fx\n", sequential_time / parallel_time);

    free(array);
}
```

Architecture-Specific Optimizations

Different CPU architectures have unique features that can be leveraged for performance.

Cache Optimization Techniques

Optimizing for cache hierarchy and behavior:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MATRIX_SIZE 1024

// Cache-unfriendly matrix transpose
void transpose_unfriendly(int n, int *src, int *dst) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dst[j * n + i] = src[i * n + j];
        }
    }
}

// Cache-friendly matrix transpose with blocking
void transpose_blocked(int n, int *src, int *dst) {
```

```
    const int block_size = 32;  // Should match cache line size

    for (int ii = 0; ii < n; ii += block_size) {
        for (int jj = 0; jj < n; jj += block_size) {
            for (int i = ii; i < ii + block_size && i < n; i++) {
                for (int j = jj; j < jj + block_size && j < n; j++) {
                    dst[j * n + i] = src[i * n + j];
                }
            }
        }
    }
}

void cache_optimization_example(void) {
    int *matrix = malloc(MATRIX_SIZE * MATRIX_SIZE * sizeof(int));
    int *transposed = malloc(MATRIX_SIZE * MATRIX_SIZE * sizeof(int));

    // Initialize matrix
    for (int i = 0; i < MATRIX_SIZE * MATRIX_SIZE; i++) {
        matrix[i] = i;
    }

    clock_t start, end;

    // Unfriendly transpose
    start = clock();
    transpose_unfriendly(MATRIX_SIZE, matrix, transposed);
    end = clock();
    double unfriendly_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Blocked transpose
    start = clock();
    transpose_blocked(MATRIX_SIZE, matrix, transposed);
    end = clock();
    double blocked_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Unfriendly transpose: %.4f seconds\n", unfriendly_time);
    printf("Blocked transpose: %.4f seconds\n", blocked_time);
    printf("Improvement: %.2fx\n", unfriendly_time / blocked_time);

    free(matrix);
    free(transposed);
}
```

Branch Prediction Optimization

Improving branch prediction accuracy for better performance:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ARRAY_SIZE 1000000

// Function with unpredictable branches
void unpredictable_branches(int *array, int n) {
    for (int i = 0; i < n; i++) {
        // Random condition leads to poor branch prediction
        if (rand() % 2) {
            array[i] *= 2;
        } else {
            array[i] += 1;
        }
    }
}

// Function with predictable branches
void predictable_branches(int *array, int n) {
    // Process even indices differently from odd indices
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            array[i] *= 2;
        } else {
            array[i] += 1;
        }
    }
}

// Branchless version using conditional moves
void branchless_version(int *array, int n) {
    for (int i = 0; i < n; i++) {
        int is_even = (i % 2 == 0);
        array[i] = array[i] * (1 + is_even) + (1 - is_even);
    }
}

void branch_prediction_example(void) {
    int *array1 = malloc(ARRAY_SIZE * sizeof(int));
    int *array2 = malloc(ARRAY_SIZE * sizeof(int));
    int *array3 = malloc(ARRAY_SIZE * sizeof(int));
```

```
    // Initialize arrays
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array1[i] = array2[i] = array3[i] = i;
    }

    clock_t start, end;

    // Unpredictable branches
    srand(42);
    start = clock();
    unpredictable_branches(array1, ARRAY_SIZE);
    end = clock();
    double unpredictable_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Predictable branches
    start = clock();
    predictable_branches(array2, ARRAY_SIZE);
    end = clock();
    double predictable_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Branchless version
    start = clock();
    branchless_version(array3, ARRAY_SIZE);
    end = clock();
    double branchless_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Unpredictable branches: %.4f seconds\n", unpredictable_time);
    printf("Predictable branches: %.4f seconds\n", predictable_time);
    printf("Branchless version: %.4f seconds\n", branchless_time);
    printf("Predictable improvement: %.2fx\n", unpredictable_time / predictable_time);
    printf("Branchless improvement: %.2fx\n", unpredictable_time / branchless_time);

    free(array1);
    free(array2);
    free(array3);
}
```

Memory-Level Parallelism

Exploiting memory-level parallelism to hide latency:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```c
#define ARRAY_SIZE 1000000

// Function that can benefit from memory-level parallelism
void memory_parallelism_example(void) {
    double *a = malloc(ARRAY_SIZE * sizeof(double));
    double *b = malloc(ARRAY_SIZE * sizeof(double));
    double *c = malloc(ARRAY_SIZE * sizeof(double));
    double *d = malloc(ARRAY_SIZE * sizeof(double));

    // Initialize arrays
    for (int i = 0; i < ARRAY_SIZE; i++) {
        a[i] = (double)i;
        b[i] = (double)(i * 2);
        c[i] = (double)(i * 3);
        d[i] = 0.0;
    }

    clock_t start = clock();

    // Independent operations that can be executed in parallel
    // Modern processors can execute multiple loads/stores simultaneously
    for (int i = 0; i < ARRAY_SIZE; i++) {
        d[i] = a[i] + b[i] + c[i];
    }

    clock_t end = clock();
    double time_spent = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Memory parallelism example time: %.4f seconds\n", time_spent);
    printf("First few results: ");
    for (int i = 0; i < 5; i++) {
        printf("%.0f ", d[i]);
    }
    printf("\n");

    free(a);
    free(b);
    free(c);
    free(d);
}
```

Advanced Profiling and Analysis

Using sophisticated tools for performance analysis:

```c
#include <stdio.h>
#include <stdlib.h>

#ifdef __linux__
#include <sys/resource.h>
#endif

// Function to measure memory usage
void print_memory_usage(void) {
#ifdef __linux__
    struct rusage usage;
    if (getrusage(RUSAGE_SELF, &usage) == 0) {
        printf("Maximum resident set size: %ld KB\n", usage.ru_maxrss);
        printf("Page reclaims: %ld\n", usage.ru_minflt);
        printf("Page faults: %ld\n", usage.ru_majflt);
    }
#endif
}

// Function to demonstrate performance counters (conceptual)
void performance_counter_example(void) {
    printf("Performance counter example:\n");
    printf("- Instructions per cycle (IPC)\n");
    printf("- Cache miss rates\n");
    printf("- Branch misprediction rate\n");
    printf("- Memory bandwidth utilization\n");

    // In practice, you would use tools like:
    // - perf (Linux)
    // - Intel VTune
    // - Apple Instruments
    // - GNU gprof
}
```

Optimization Decision Making

Making informed decisions about which optimizations to apply:

```c
#include <stdio.h>
#include <time.h>

// Example showing when optimization matters
void optimization_decision_example(void) {
    printf("When to optimize:\n");
    printf("1. Profile first - identify actual bottlenecks\n");
```

```
    printf("2. Consider algorithmic improvements before micro-optimizations\n");
    printf("3. Optimize hot paths - functions called frequently\n");
    printf("4. Balance performance with maintainability\n");
    printf("5. Consider target platform and constraints\n");
    printf("6. Measure impact of each optimization\n");

    // Simple example of measuring optimization impact
    const int iterations = 100000000;

    clock_t start = clock();
    volatile long long sum = 0;
    for (int i = 0; i < iterations; i++) {
        sum += i;
    }
    clock_t end = clock();

    double time_spent = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSimple loop time: %.4f seconds\n", time_spent);
}
```

Practical Optimization Workflow

A systematic approach to performance optimization:

```
#include <stdio.h>

void optimization_workflow(void) {
    printf("Performance Optimization Workflow:\n\n");

    printf("1. Profiling and Measurement\n");
    printf("   - Use profiling tools to identify bottlenecks\n");
    printf("   - Measure baseline performance\n");
    printf("   - Set performance goals\n\n");

    printf("2. Algorithmic Analysis\n");
    printf("   - Review algorithmic complexity\n");
    printf("   - Consider alternative algorithms\n");
    printf("   - Optimize data structures\n\n");

    printf("3. Implementation Optimization\n");
    printf("   - Apply compiler optimizations\n");
    printf("   - Use appropriate data layouts\n");
    printf("   - Minimize memory allocations\n\n");

    printf("4. Hardware-Specific Tuning\n");
```

```
    printf("   - Leverage SIMD instructions\n");
    printf("   - Utilize multiple cores\n");
    printf("   - Optimize for cache hierarchy\n\n");

    printf("5. Validation and Testing\n");
    printf("   - Verify correctness after optimization\n");
    printf("   - Measure performance improvements\n");
    printf("   - Test on target hardware\n\n");

    printf("6. Documentation and Maintenance\n");
    printf("   - Document optimization decisions\n");
    printf("   - Monitor performance over time\n");
    printf("   - Maintain balance between performance and readability\n");
}
```

Summary

Advanced optimization techniques require a deep understanding of both the problem domain and the underlying hardware. Key takeaways include:

1. **Compiler Optimization** - Leverage compiler flags and hints for automatic improvements
2. **Vectorization** - Use SIMD instructions to process multiple data elements simultaneously
3. **Parallel Processing** - Distribute work across multiple CPU cores using OpenMP or pthreads
4. **Architecture-Specific Optimizations** - Tailor code for specific CPU features and cache behaviors
5. **Memory-Level Parallelism** - Exploit the ability to perform multiple memory operations simultaneously
6. **Branch Prediction** - Structure code to improve CPU branch prediction accuracy
7. **Systematic Approach** - Follow a methodical workflow for identifying and addressing performance bottlenecks

Important principles to remember:

- **Profile First** - Always measure before and after optimization
- **Algorithmic Improvements** - Often provide bigger gains than micro-optimizations
- **Platform Awareness** - Consider target hardware characteristics
- **Maintainability** - Balance performance with code readability and maintainability
- **Verification** - Ensure optimizations don't introduce bugs or change behavior

These advanced techniques, when applied appropriately, can yield substantial performance improvements in performance-critical C applications. However, they should be used judiciously and only after identifying actual bottlenecks through profiling.

ISPC 1.32: AVX10.2, APX, and Zen 6 Targets

ISPC 1.32: AVX10.2, APX, and Zen 6 Targets

Writing SIMD by hand with intrinsics is slow, brittle, and hard to port across SSE, AVX2, AVX-512, and whatever comes next. **Intel Implicit SPMD Program Compiler (ISPC)** is a C-like language with a small set of extensions so you write one *serial-looking* program and the compiler maps a gang of program instances onto the CPU's SIMD lanes. **ISPC 1.32.0** (13 October 2026) is the release that brings full **AVX10.2** and **APX** code generation for the Diamond Rapids and Nova Lake target families, plus an explicit `--cpu=znver6` for AMD Zen 6. This chapter teaches the SPMD mental model, how those new targets relate to packing width (`-x4` … `-x64`), and the OS-support traps around APX and AMX that the multi-target dispatcher will *not* catch for you.

Learning goals

By the end of this chapter you can:

- Explain SPMD in ISPC: one source program, many instances, `uniform` vs `varying`
- Name what AVX10.2 and APX add for SPMD C on the `avx10.2dmr` and `avx10.2nvl` families
- Choose `--target` / `--cpu=znver6` and explain lane packing (`-x16` vs `-x32`)
- Use `--opt=disable-apx` when the OS may not manage APX state
- Sketch a docs-faithful `export/foreach` kernel and how it links to C

Thesis

ISPC's job is a thin abstraction over SIMD: you keep a C-like syntax and a clear map from source to lanes, instead of writing `_mm512_*` by hand. Version 1.32 does not invent a new programming model — it unlocks **new code-generation backends** so the same SPMD kernels can compile for upcoming Intel AVX10.2/APX CPUs and for Zen 6's native `float16` path.

Mental model — SPMD, gangs, packing

In SPMD, you write what looks like a scalar loop body. At runtime (and in the generated code), a **gang** of program instances runs together, one per SIMD lane of the chosen target width.

Qualifier	Meaning
`uniform`	One value shared by the whole gang (scalars, loop bounds, pointers you share)
`varying`	One value per program instance (the data that rides in vector registers)

Qualifier	Meaning
`foreach`	Parallel iteration that assigns work across the gang

Packing is how many instances fit in one gang for a given target name:

Target family (ISPC 1.32)	Typical packing	Notes from release notes / user's guide
`avx10.2dmr-xN`	N {4,8,16,32,64}	Diamond Rapids; AVX10.2 + APX; AMX-capable variants exist
`avx10.2nvl-xN`	N {4,8,16,32,64}	Nova Lake; AVX10.2 + APX; **no** AMX (unlike dmr)
`--cpu=znver6`	maps to `avx512icl-x16`	AMD Zen 6; native `float16` arithmetic

Macro `ISPC_TARGET_AVX10_2NVL` is predefined on the Nova Lake family. Wider packing uses more lanes per gang; it is not automatically "faster" — it changes register pressure, masking, and how well your data layout fills the gang.

```
C host                          ISPC gang (e.g. avx10.2nvl-x16)
--------                        --------------------------------
float *a, *b, *c;               foreach (i = 0 ... n-1) {
call ispc_add(a,b,c,n);           c[i] = a[i] + b[i];   // varying loads/stores
                                }
                                // 16 instances of that assignment in SIMD
```

What AVX10.2 and APX enable

From the ISPC 1.32.0 release notes (LLVM 23.1.2 backend):

- **AVX10.2** — newer x86 vector ISA; ISPC emits it for the `avx10.2dmr` and `avx10.2nvl` targets so SPMD kernels use the hardware's current vector ops instead of falling back to older AVX-512 subsets.
- **APX (Advanced Performance Extensions)** — extended GPRs and related sub-features (`egpr`, `ndd`, `push2pop2`, `ppx`, `ccmp`, `cf`, `nf`, `zu`, `jmpabs`). On `avx10.2dmr` / `avx10.2nvl`, APX is **enabled by default**.
- `pdep32` / `pext32` / `pdep64` / `pext64` — new stdlib helpers (uniform and varying) for parallel bit deposit/extract; BMI2 where available, specialized for constant masks (Morton codes, packing).

The trap: CPUID ≠ OS support

The multi-target **runtime dispatcher** selects `avx10.2dmr`, `avx10.2nvl`, and AMX-capable variants from **CPUID only**. It does **not** check whether the OS has enabled APX or AMX XSAVE state.

Situation	What to do
Binary may run on an OS without APX	Compile with `--opt=disable-apx` (or a subset, e.g. `--opt=disable-apx=egpr`)
`--target=host` on Nova Lake / Diamond Rapids without OS APX	ISPC keeps the detected target but **disables APX** codegen
AMX tiles (`avx512spr`, `avx512gnr`, `avx10.2dmr`)	Your app must verify OS AMX permission before executing AMX; dispatcher does not

Dispatch stubs themselves are always compiled without APX so the trampoline stays portable.

Worked example — SPMD add (docs-faithful)

ISPC is not installed on this writer machine; the listing below follows the official ISPC user's guide / website SPMD model and 1.32 target naming. Treat output as illustrative until you compile locally.

```
// simple.ispc - gang of instances adds two float arrays
export void simple_add(uniform float a[], uniform float b[],
                       uniform float c[], uniform int count) {
    foreach (i = 0 ... count) {
        c[i] = a[i] + b[i];
    }
}
```

```
// main.c - call the export from ordinary C
#include <stdio.h>
#include <stdlib.h>

void simple_add(float a[], float b[], float c[], int count);

int main(void) {
    const int n = 1024;
    float *a = malloc(n * sizeof *a);
    float *b = malloc(n * sizeof *b);
    float *c = malloc(n * sizeof *c);
    for (int i = 0; i < n; i++) {
        a[i] = (float)i;
        b[i] = 1.0f;
    }
    simple_add(a, b, c, n);
    printf("c[0]=%.1f c[n-1]=%.1f\n", c[0], c[n - 1]);
    free(a); free(b); free(c);
    return 0;
}
```

```
# Diamond Rapids / Nova Lake style targets (ISPC 1.32+)
ispc --target=avx10.2nvl-x16 simple.ispc -o simple.o
# or multi-target dispatch:
# ispc --target=avx2-i32x8,avx10.2nvl-x16,avx10.2dmr-x16 simple.ispc -o simple.o

# Zen 6 CPU tuning (maps to avx512icl-x16 + native float16)
ispc --cpu=znver6 --target=avx512icl-x16 simple.ispc -o simple.o

# Portable APX-off binary for mixed OS fleets
ispc --target=avx10.2nvl-x16 --opt=disable-apx simple.ispc -o simple.o

cc -O2 main.c simple.o -o simple_add
./simple_add
```

Expected shape of output once built on a machine with ISPC 1.32:

```
c[0]=1.0 c[n-1]=1024.0
```

Case — picking packing width

Goal	Prefer
Match AVX-512 512-bit float lanes	often `-x16` for `float`
Experiment with double-pumped / wider gangs	`-x32` / `-x64` (more instances, more masking cost)
Ship one binary for many CPUs	multi-target list + runtime dispatcher; remember APX/AMX OS caveats

The trap

- Treating **dispatcher selection** as "safe to run APX/AMX" — it is not; add an OS check or `--opt=disable-apx`.
- Confusing `avx10.2nvl` (no AMX) with `avx10.2dmr` (AMX-capable family).
- Assuming **Zen 6** (`--cpu=znver6`) is an AVX10.2 target — it maps to `avx512icl-x16` with native `float16`, per the user's guide.
- Stripping symbols / skipping the C header ISPC generates for `export` and then wondering why the linker cannot find the kernel.

The boring rule

Write SPMD kernels with clear `uniform` / `varying` boundaries; pick `--target` for the ISA you need; for AVX10.2 fleets, decide **APX on vs `--opt=disable-apx`** as a deployment property, not an afterthought.

Try this

1. Install ISPC 1.32 from ispc.github.io (no system package install required for this chapter's verification path — use the official binary tarball).
2. Run `ispc --support-matrix` and locate `avx10.2nvl`, `avx10.2dmr`, and `znver6`.
3. Compile the add kernel once with `--opt=disable-apx` and once without; compare `objdump -d` for APX-related encodings if you have a disassembler.
4. Read the "Updating ISPC Programs For Changes In ISPC 1.32.0" section of the user's guide before upgrading multi-target builds.

Sources

- ISPC home: https://ispc.github.io/
- ISPC user's guide: https://ispc.github.io/ispc.html
- Release notes (v1.32.0): https://raw.githubusercontent.com/ispc/ispc/main/docs/ReleaseNotes.txt
- GitHub releases: https://github.com/ispc/ispc/releases

Arena Allocators: Bump Pointers, Alignment, and Lifetimes Instead of free()

Arena Allocators: Bump Pointers, Alignment, and Lifetimes Instead of `free()`

Thesis

Most `malloc`/`free` bugs come from tracking the lifetime of every object one at a time. Many programs don't have a million independent lifetimes, though. They have a few: *this request*, *this frame*, *this parse*, *until exit*. An **arena** (also called a region, bump or linear allocator) matches that shape. It takes one big block and allocates by moving a pointer forward. It frees everything at once by moving the pointer back.

The core is about ten lines. The details that make it correct are:

- **Alignment**: pad before each allocation.
- **Overflow-checked sizes**: never compute `count * size` blindly.
- **An explicit out-of-memory policy.**
- **Honesty about the cost**: a pointer that outlives a reset is a use-after-free that no allocator will catch for you, unless you teach AddressSanitizer about the arena.

Mental model

	`malloc`/`free`	Arena
Allocate	general-purpose search of size-class bins	pad + bounds check + pointer add
Free one object	`free(p)`	not supported (by design)
Free a whole structure	walk it and `free` every node	`beg = mark` or `free(base)`: O(1)
Per-object overhead	chunk header (8–16 B on glibc x86-64) plus rounding	only alignment padding
Lifetime bugs	per object: double free, leak, UAF	per *region*: pointers that escape the region
Tool support	ASan/Valgrind understand it out of the box	ASan only if you add manual poisoning

Worked examples

All listings were compiled with **GCC 14.2.0** (Debian), `-std=c17 -Wall -Wextra -O2` (warning-free) against glibc 2.41, and run on an 8-vCPU x86-64 Xeon VM. The current GCC release is **16.2**.

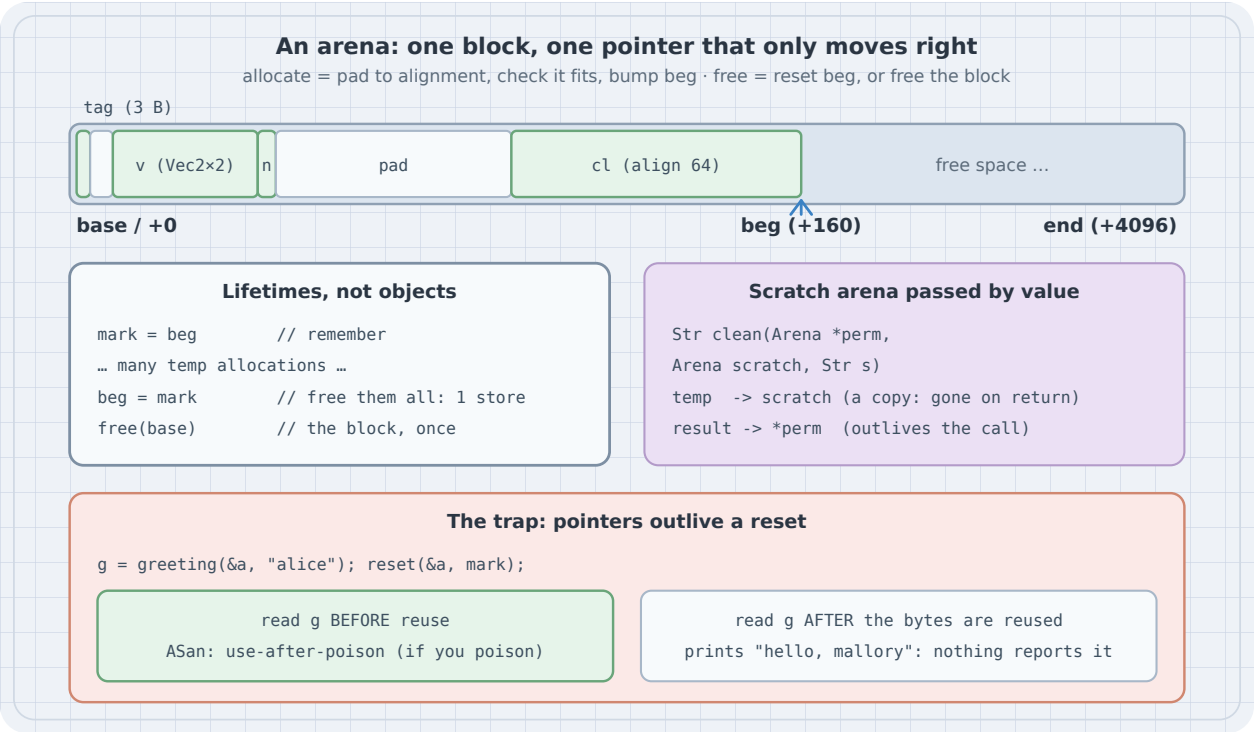


Figure 1: An arena: one block carved left to right by bumping beg, with alignment padding; mark/reset and scratch-by-value lifetimes; and the trap of pointers that outlive a reset

Nothing here depends on the compiler version beyond C11 `_Alignof`/`alignas` and the sanitizer interface. Every output block is pasted from a real run.

The whole allocator

```
// arena.h - a minimal bump ("linear") arena allocator
#ifndef ARENA_H
#define ARENA_H
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

#if defined(__has_feature)
#  if __has_feature(address_sanitizer)
#    define ARENA_ASAN 1
#  endif
#endif
#if defined(__SANITIZE_ADDRESS__)
#  define ARENA_ASAN 1
#endif
#ifdef ARENA_ASAN
```

```
#  include <sanitizer/asan_interface.h>
// Out-of-line on purpose: GCC 14's -Wmaybe-uninitialized misfires when fresh malloc memory
// is passed straight to the poison call's const volatile void * parameter.
__attribute__((noinline)) static void arena_poison_(void *p, size_t n) { ASAN_POISON_MEMORY_
#  define ARENA_POISON(p, n)   arena_poison_((p), (n))
#  define ARENA_UNPOISON(p, n) ASAN_UNPOISON_MEMORY_REGION((p), (n))
#else
#  define ARENA_POISON(p, n)   ((void)(p), (void)(n))
#  define ARENA_UNPOISON(p, n) ((void)(p), (void)(n))
#endif

typedef struct {
    char *beg;   // next free byte
    char *end;   // one past the last usable byte
} Arena;

// Carve an arena out of one malloc'd block. The caller frees *base once, at the end.
static inline Arena arena_new(size_t cap, void **base)
{
    char *mem = malloc(cap);
    *base = mem;
    Arena a = { mem, mem ? mem + cap : NULL };
    if (mem) ARENA_POISON(a.beg, (size_t)(a.end - a.beg));   // ASan: unallocated bytes are
    return a;
}

// Allocate count objects of size bytes each, aligned to align (a power of two), zeroed.
// Returns NULL when the arena is exhausted or count*size would overflow.
static inline void *arena_alloc(Arena *a, ptrdiff_t size, ptrdiff_t align, ptrdiff_t count)
{
    ptrdiff_t pad = -(uintptr_t)a->beg & (uintptr_t)(align - 1);
    ptrdiff_t avail = a->end - a->beg - pad;
    if (avail < 0 || count > avail / size) {
        return NULL;                        // checked: no count*size overflow
    }
    char *p = a->beg + pad;
    a->beg = p + count * size;
    ARENA_UNPOISON(p, (size_t)(count * size));
    return memset(p, 0, (size_t)(count * size));
}

#define arena_new_of(a, T, n) ((T *)arena_alloc((a), sizeof(T), _Alignof(T), (n)))

// Scratch lifetimes: remember where we were, and roll back to it later.
static inline char *arena_mark(const Arena *a) { return a->beg; }
static inline void arena_reset(Arena *a, char *mark)
{
```

```
    ARENA_POISON(mark, (size_t)(a->beg - mark));
    a->beg = mark;
}
#endif
```

Five decisions in this header deserve a sentence each:

- **Signed sizes** (`ptrdiff_t`). `avail` can go negative when the padding alone doesn't fit. The check `count > avail / size` then rejects any request whose `count * size` would exceed the space left, **without ever computing the overflowing product**.
- **Alignment padding** is `-(uintptr_t)beg & (align - 1)`. That is the distance to the next multiple of a power-of-two `align`. `arena_new_of` feeds it `_Alignof(T)`, so every type gets its own alignment, not a one-size-fits-all 16.
- **Zeroing** in `arena_alloc` makes "zero is a valid initial state" designs free to use and removes a whole class of uninitialised reads. The cost is touching every byte you allocate, so drop the `memset` if you measure it mattering.
- **Out-of-memory returns `NULL`.** Some arenas abort instead, or `longjmp` to a per-request handler. Pick one policy and use it everywhere.
- **ASan poisoning** is compiled in only under `-fsanitize=address`. Case 4 is the reason it exists.

Case 1: Sizes, padding and the alignment `malloc` doesn't promise

```
// basics.c - what a bump allocator actually does with sizes and alignment
#include <stdio.h>
#include <stdalign.h>
#include "arena.h"

typedef struct { double x, y; } Vec2;          // alignment 8
typedef struct { alignas(64) char line[64]; } CacheLine;   // alignment 64

int main(void)
{
    void *base;
    Arena a = arena_new(4096, &base);
    char *start = a.beg;

    char     *tag  = arena_new_of(&a, char, 3);
    Vec2     *v    = arena_new_of(&a, Vec2, 2);
    int      *n    = arena_new_of(&a, int, 1);
    CacheLine *cl  = arena_new_of(&a, CacheLine, 1);

    printf("block %% 64 = %zu (malloc only promises %zu)\n",
           (size_t)((uintptr_t)start % 64), alignof(max_align_t));
    printf("tag  at +%-4td size %zu\n", tag - start, 3 * sizeof *tag);
    printf("v    at +%-4td size %zu  (padded to align %zu)\n", (char *)v - start, 2 * sizeof
```

```
    printf("n    at +%-4td size %zu\n", (char *)n - start, sizeof *n);
    printf("cl   at +%-4td size %zu  (align %zu: addr %% 64 = %zu)\n", (char *)cl - start, s
           alignof(CacheLine), (size_t)((uintptr_t)cl % 64));
    printf("used %td of 4096 bytes, 4 allocations, 1 malloc\n", a.beg - start);

    // Overflow-checked: a huge count fails cleanly instead of wrapping.
    void *huge = arena_alloc(&a, sizeof(Vec2), alignof(Vec2), PTRDIFF_MAX / 4);
    printf("huge request -> %s\n", huge ? "non-NULL (BUG)" : "NULL");

    free(base);   // every allocation above is released here, at once
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o basics basics.c && ./basics
block % 64 = 32 (malloc only promises 16)
tag  at +0    size 3
v    at +8    size 32  (padded to align 8)
n    at +40   size 4
cl   at +96   size 64  (align 64: addr % 64 = 0)
used 160 of 4096 bytes, 4 allocations, 1 malloc
huge request -> NULL
```

Read the offsets. `tag` takes 3 bytes, then 5 bytes of padding, because `Vec2` needs 8-byte alignment. `CacheLine` asks for 64-byte alignment, which is *more* than `malloc` guarantees: C only promises alignment for types with a fundamental alignment, which is `alignof(max_align_t)`, 16 here. The block itself happened to start 32 bytes past a 64-byte boundary. So the arena padded from offset 44 all the way to 96, where the *address* is a multiple of 64. Alignment is about addresses, not offsets: the same code against a different block would pad differently. The last line shows the overflow check working. The request for about 2^61 `Vec2` objects fails cleanly instead of wrapping to a small size.

Case 2: Two lifetimes, and a scratch arena passed by value

Real programs have at least two lifetimes: results that must survive, and temporaries that only exist while a function runs. Give each one its own arena.

```
// scratch.c - two lifetimes: a permanent arena for results, a scratch arena passed BY VALUE
#include <ctype.h>
#include <stdio.h>
#include "arena.h"

typedef struct { char *data; ptrdiff_t len; } Str;

// Upper-cased, de-duplicated-spaces copy of s. The temporary work buffer comes from
// `scratch`, a COPY of the caller's arena: anything allocated in it is gone on return.
static Str clean(Arena *perm, Arena scratch, Str s)
```

```
{
    char *tmp = arena_new_of(&scratch, char, s.len);      // temporary: dies with this frame
    if (!tmp) return (Str){ NULL, 0 };
    ptrdiff_t n = 0;
    for (ptrdiff_t i = 0; i < s.len; i++) {
        if (s.data[i] == ' ' && (n == 0 || tmp[n - 1] == ' ')) continue;
        tmp[n++] = (char)toupper((unsigned char)s.data[i]);
    }
    while (n > 0 && tmp[n - 1] == ' ') n--;
    Str out = { arena_new_of(perm, char, n + 1), n };     // permanent: outlives the call
    if (!out.data) return (Str){ NULL, 0 };
    memcpy(out.data, tmp, (size_t)n);
    return out;
}

int main(void)
{
    void *pbase, *sbase;
    Arena perm = arena_new(1 << 16, &pbase);
    Arena scratch = arena_new(1 << 16, &sbase);
    char *p0 = perm.beg, *s0 = scratch.beg;

    const char *in[] = { "  hello   arena  ", "bump   pointers are   cheap", " free   it all
    Str out[3];
    for (int i = 0; i < 3; i++) {
        out[i] = clean(&perm, scratch, (Str){ (char *)in[i], (ptrdiff_t)strlen(in[i]) });
        printf("\"%s\"\n", out[i].data ? out[i].data : "(out of memory)");
    }
    printf("perm used %td bytes, scratch used %td bytes (each call reused the same scratch s
           perm.beg - p0, scratch.beg - s0);

    // The same thing with explicit mark/reset, for code that holds an Arena * everywhere.
    char *mark = arena_mark(&scratch);
    char *big = arena_new_of(&scratch, char, 4000);
    big[0] = 'x';
    printf("after a 4000-byte temp: scratch used %td\n", scratch.beg - s0);
    arena_reset(&scratch, mark);
    printf("after reset:            scratch used %td\n", scratch.beg - s0);

    free(sbase);
    free(pbase);
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o scratch scratch.c && ./scratch
"HELLO ARENA"
```

```
"BUMP POINTERS ARE CHEAP"
"FREE IT ALL AT ONCE"
perm used 56 bytes, scratch used 0 bytes (each call reused the same scratch space)
after a 4000-byte temp: scratch used 4000
after reset:            scratch used 0
```

`clean` takes `perm` by pointer, because its allocations must persist, and **`scratch` by value**. The callee bumps its *copy* of `beg`, so when it returns, the caller's `scratch` is exactly as it was. Every call reused the same scratch bytes, and `scratch used 0 bytes` at the end. No `free`, no reset call, no way to forget one. For code that already holds an `Arena *` everywhere, `arena_mark`/`arena_reset` gives the same effect explicitly.

Case 3: What it buys, measured

```c
// bench.c - 1M small nodes: malloc/free per node vs one arena, two workloads
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <sys/resource.h>
#include <time.h>
#include "arena.h"

typedef struct Node { struct Node *left, *right; uint64_t key; char name[16]; } Node;
enum { N = 1000000, RUNS = 5 };

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}
static long minflt(void)
{
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

static uint64_t rng;
static uint64_t next(void) { rng ^= rng << 13; rng ^= rng >> 7; rng ^= rng << 17; return rng

// Workload "list": allocation-dominated (push N nodes on a list).
// Workload "tree": search-dominated (insert N random keys into an unbalanced BST).
static Node *insert(Node *root, Node *n, int tree)
{
    if (!tree) { n->right = root; return n; }
    Node **slot = &root;
```

```
    while (*slot) slot = n->key < (*slot)->key ? &(*slot)->left : &(*slot)->right;
    *slot = n;
    return root;
}
static void free_all(Node *n)                       // per-node teardown that malloc forces
{
    while (n) { free_all(n->left); Node *r = n->right; free(n); n = r; }
}

static double median(double *v)
{
    for (int i = 1; i < RUNS; i++)
        for (int j = i; j > 0 && v[j] < v[j - 1]; j--) { double t = v[j]; v[j] = v[j - 1]; v
    return v[RUNS / 2];
}

static void run(int tree, Arena *a)
{
    double bm[RUNS], tm[RUNS], ba[RUNS], ta[RUNS];
    long fm = 0, fa = 0;
    for (int r = 0; r < RUNS; r++) {
        rng = 0x9E3779B97F4A7C15u;
        Node *root = NULL;
        long f0 = minflt();
        double t0 = now();
        for (int i = 0; i < N; i++) {
            Node *n = calloc(1, sizeof *n);
            if (!n) exit(1);
            n->key = next();
            root = insert(root, n, tree);
        }
        double t1 = now();
        free_all(root);
        double t2 = now();
        fm = minflt() - f0;
        bm[r] = t1 - t0; tm[r] = t2 - t1;

        rng = 0x9E3779B97F4A7C15u;
        root = NULL;
        char *mark = arena_mark(a);
        f0 = minflt();
        t0 = now();
        for (int i = 0; i < N; i++) {
            Node *n = arena_new_of(a, Node, 1);
            if (!n) exit(1);
            n->key = next();
            root = insert(root, n, tree);
```

```
        }
        t1 = now();
        arena_reset(a, mark);                          // one store releases every node
        t2 = now();
        fa = minflt() - f0;
        ba[r] = t1 - t0; ta[r] = t2 - t1;
    }
    printf("%s workload\n", tree ? "tree (search-dominated)" : "list (allocation-dominated)"
    printf("  malloc  build %7.1f ms  teardown %7.3f ms  minor faults %6ld\n", median(bm) *
    printf("  arena   build %7.1f ms  teardown %7.3f ms  minor faults %6ld\n", median(ba) *
}

int main(void)
{
    void *base;                                        // one long-lived arena, reset per run
    Arena a = arena_new((size_t)N * sizeof(Node), &base);
    if (!base) return 1;
    run(0, &a);
    run(1, &a);
    free(base);
    printf("(median of %d runs, %d nodes of %zu bytes; faults from the last run; the arena's
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o bench bench.c && ./bench
list (allocation-dominated) workload
  malloc  build     8.1 ms  teardown   6.835 ms  minor faults      0
  arena   build     6.5 ms  teardown   0.000 ms  minor faults      0
tree (search-dominated) workload
  malloc  build   578.1 ms  teardown  59.858 ms  minor faults      0
  arena   build   384.4 ms  teardown   0.000 ms  minor faults      0
(median of 5 runs, 1000000 nodes of 40 bytes; faults from the last run; the arena's first ru
```

Two results hold up. One doesn't:

- **Teardown is the big, robust win.** Freeing a million nodes one at a time costs roughly 7 ms for a list and roughly 55–60 ms for a tree. The tree walk misses cache on every node. The arena's teardown is a single store, which rounds to 0.000 ms.
- **Allocation itself is somewhat faster** in the allocation-dominated list workload (about 6.5 ms vs 8 ms). Nodes are also denser: 40 bytes each with no 16-byte chunk header in between.
- **Don't trust the tree *build* column.** It is dominated by cache misses during the search, not by the allocator. On this VM it moved by ±40% between variants of the same benchmark. In an earlier variant that `malloc`'d a fresh 40 MB arena block on every run, the arena tree build was about 10% **slower** than `malloc` (440–470 ms vs 400 ms). That run paid 9,766 minor page faults to first-touch the new block: exactly 40,000,000 / 4096, rounded up. A 40 MB request is far above glibc's `M_MMAP_THRESHOLD`, so unless it can be carved out of heap space `malloc` has just released, it arrives as fresh, untouched pages. The arena wins on build only

when the block is reused, and a long-lived, reset-per-request arena is how they're meant to be used anyway.

Case 4: The trap, a pointer that outlives its reset

```c
// dangling.c - the arena trap: a pointer that outlives its reset
#include <stdio.h>
#include "arena.h"

static const char *greeting(Arena *a, const char *name)
{
    char *s = arena_new_of(a, char, 64);
    if (!s) return "(oom)";
    snprintf(s, 64, "hello, %s", name);
    return s;
}

int main(int argc, char **argv)
{
    int early = argc > 1 && strcmp(argv[1], "early") == 0;  // read the stale pointer before
    void *base;
    Arena a = arena_new(4096, &base);

    char *mark = arena_mark(&a);
    const char *g = greeting(&a, "alice");      // g points into the arena...
    printf("before reset: %s\n", g);
    arena_reset(&a, mark);                      // ...and this just made it dangling
    if (early) printf("stale read:   %s\n", g);  // bytes are poisoned until reused

    const char *h = greeting(&a, "mallory");    // reuses the same bytes
    printf("after reset:  %s   (same address? %s)\n", g, g == h ? "yes" : "no");

    free(base);
    return 0;
}
```

Plain `-O2` build, read after reuse, then read before reuse:

```
$ ./dangling
before reset: hello, alice
after reset:  hello, mallory   (same address? yes)
$ ./dangling early
before reset: hello, alice
stale read:   hello, alice
after reset:  hello, mallory   (same address? yes)
```

Both runs are bugs: `g` refers to an allocation the program has released. Neither run complains. As far as C and the hardware can tell, the bytes are still part of a live `malloc` block, so there is no crash and nothing to catch. Now build with AddressSanitizer. The arena's `ARENA_POISON`/`ARENA_UNPOISON` hooks switch on, so released bytes are marked unaddressable:

```
$ gcc -std=c17 -Wall -Wextra -O1 -g -fsanitize=address -fno-omit-frame-pointer -o dangling-a
$ ./dangling-asan early
=================================================================
==309323==ERROR: AddressSanitizer: use-after-poison on address 0x521000000100 at pc 0x7f8175
READ of size 13 at 0x521000000100 thread T0
    #0 0x7f8175c9c79b in printf_common ../../../../src/libsanitizer/sanitizer_common/sanitiz
    #1 0x7f8175cc849a in vprintf ../../../../src/libsanitizer/sanitizer_common/sanitizer_com
    #2 0x7f8175cca08e in printf ../../../../src/libsanitizer/sanitizer_common/sanitizer_comm
    #3 0x5642669db52c in main /tmp/arena/dangling.c:23
    #4 0x7f8175a35ca7  (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId: 5e5b51f4c39af9ac5
    #5 0x7f8175a35d64 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x29d64) (BuildI
    #6 0x5642669db150 in _start (/tmp/arena/dangling-asan+0x1150) (BuildId: b7a93a7760343370

0x521000000100 is located 0 bytes inside of 4096-byte region [0x521000000100,0x521000001100)
allocated by thread T0 here:
    #0 0x7f8175cf4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x5642669db398 in arena_new /tmp/arena/arena.h:37
    #2 0x5642669db398 in main /tmp/arena/dangling.c:17
    #3 0x7f8175a35ca7  (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId: 5e5b51f4c39af9ac5

SUMMARY: AddressSanitizer: use-after-poison ../../../../src/libsanitizer/sanitizer_common/sa
[... shadow-byte map trimmed ...]
```

```
$ ./dangling-asan
before reset: hello, alice
after reset:  hello, mallory   (same address? yes)
```

That second run is the real lesson. **Poisoning catches a stale read only until the bytes are handed out again.** Once `greeting(&a, "mallory")` re-allocates the same address, `g` is a perfectly valid pointer as far as any tool can tell. It silently reads someone else's data. With `malloc`, ASan holds freed chunks in a quarantine and delays their reuse, so a stale read after `free` is usually still caught. An arena that knows nothing of that reuses the exact address on the very next allocation.

The trap

The trap is letting a pointer escape the lifetime of its arena. The bug isn't in the allocator. It's in an API that returns arena memory to a caller who doesn't know which arena, or which reset, owns it:

- caching a `const char *` from a per-request arena in a global or long-lived struct;

- keeping an element pointer while "growing" an arena-backed array by copying it to a new allocation (the old pointer still "works" until reuse);
- passing a scratch arena by *pointer* where the code assumed by value (or the reverse), so temporaries outlive, or are freed under, their users;
- resetting a shared arena in one thread while another still reads from it. The ASan interface is explicitly not thread-safe for concurrent poisoning of the same region either.

Two subtler costs are worth knowing:

- ASan's manual poisoning works at 8-byte granularity. `ASAN_UNPOISON_MEMORY_REGION` "may unpoison a super-region" of what you asked for, so tiny overruns into padding can go unreported.
- This arena places allocations back to back, with no redzones, so an overrun from one object into the next is *never* reported. The sanitizer docs suggest leaving poisoned redzones between chunks (see Try this #3).

The boring rule

> Use an arena when a group of objects shares one lifetime (a request, a frame, a parse, a phase) and free the group, never the member. Size requests with an overflow check, align with `_Alignof(T)`, and choose one out-of-memory policy. Pass scratch arenas **by value** so that temporaries clean themselves up, and keep long-lived results in a separate arena passed by pointer. Never let arena pointers outlive their arena's reset. Make that impossible to miss by keeping arenas out of globals and APIs short-lived. In debug builds, poison on reset under ASan, and keep a build mode where `arena_alloc` calls `malloc`, so the ordinary tools see every object.

Try this

1. Change `basics.c` to allocate `CacheLine` first. How much padding is left, and why does the answer depend on the address `malloc` returned?
2. Add `void *arena_realloc_last(Arena *a, void *p, ptrdiff_t old, ptrdiff_t new)` that grows in place when `p` was the most recent allocation and copies otherwise. Use it for a growable string builder and count how many copies it makes for 1,000 appends.
3. Give `arena_alloc` a debug-only 16-byte poisoned redzone after every allocation, as the ASan manual-poisoning guide suggests. Write a one-byte overrun from `tag` into `v`, and confirm that ASan reports it now and didn't before.
4. Add `-DARENA_MALLOC` support so that `arena_alloc` calls `calloc` and records each pointer in a list that `arena_reset` frees. Run `dangling.c` under ASan in that mode. What does ASan report for the "late" read now?
5. Rework `bench.c` so that each run uses a fresh arena block created with `arena_new`. Reproduce the page-fault cost from Case 3, then pre-touch the block with `memset` before timing. Which column moves?

Sources

- ISO C, §6.2.8 Alignment of objects and §7.22.3 Memory management functions ("suitably aligned so that it may be assigned to a pointer to any type of object with a fundamental alignment requirement"; the same section numbers and wording in C11 and C17). Quoted from the public C11 committee draft N1570: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf
- GNU C Library manual, "Aligned Memory Blocks" and "Memory Allocation Tunables" (`M_MMAP_THRESHOLD`, `M_TRIM_THRESHOLD`): https://sourceware.org/glibc/manual/latest/html_node/Aligned-Memory-Blocks.html, https://sourceware.org/glibc/manual/latest/html_node/Memory-Allocation-Tunables.html
- AddressSanitizer manual poisoning (the custom-arena workflow, 8-byte alignment, `allow_user_poisoning`): https://github.com/google/sanitizers/wiki/AddressSanitizerManualPoisoning
- Sanitizer interface header `asan_interface.h` (`__asan_poison_memory_region` / `__asan_unpoison_memory_` semantics and thread-safety note): https://github.com/llvm/llvm-project/blob/main/compiler-rt/include/sanitizer/asan_interface.h
- GCC manual, Instrumentation Options (`-fsanitize=address`): https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html
- GCC releases (GCC 16.2 current): https://gcc.gnu.org/releases.html
- Linux `getrusage(2)` (`ru_minflt`): https://man7.org/linux/man-pages/man2/getrusage.2.html

Double-Ended Stack Allocators: Two Lifetimes, One Buffer, No Fragmentation

Double-Ended Stack Allocators: Two Lifetimes, One Buffer, No Fragmentation

After this page you can split one fixed buffer between two independent allocation lifetimes by growing one bump stack up from the bottom and another down from the top. Either side can reset without fragmenting the other. You will use the same layout to pack variable-sized records with an index into a fixed-size packet, and make it thread-safe the right way: one atomic word for both heads, because "an atomic add on each head" overlaps allocations under contention, which the stress test below shows.

Thesis

The arena chapter's bump allocator frees everything at once. That breaks down when *two* subsystems with different lifetimes share one memory budget, such as streamed level data and a swappable character model in a game, or payloads and an index in a network packet. Interleave their allocations in one arena and freeing one subsystem leaves holes the other is still pinning. Give each its own end of the buffer instead. Each side is still a stack with O(1) allocation and O(1) reset, the free space is always one contiguous gap in the middle, and the split between the two sides adjusts itself at run time.

Mental model

The invariant is `0 <= lo <= hi <= cap`. Bytes `[0, lo)` belong to the bottom stack, `[hi, cap)` to the top stack, and `[lo, hi)` is free. An allocation fails exactly when it would make `lo > hi`. Note what you give up: each side frees only in LIFO order (marks and rewinds) or all at once.

Worked examples

The allocator

```
// dstack.h: a double-ended stack allocator over one caller-owned buffer.
// The low head grows up from 0, the high head grows down from cap;
// the buffer is full when they would cross.
#ifndef DSTACK_H
#define DSTACK_H
#include <stddef.h>
```

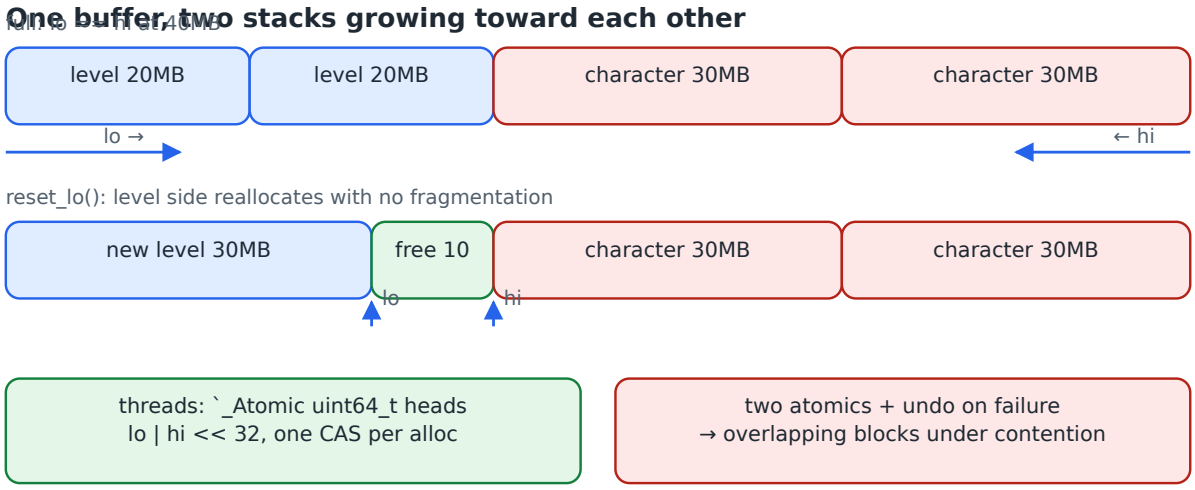


Figure 1: A 100 MB buffer with level data growing up from the bottom and character data growing down from the top; resetting the low side frees a contiguous 40 MB for a new 30 MB level allocation; for threads, both heads are packed into one atomic word updated by CAS

```
#include <stdint.h>

typedef struct {
    unsigned char *base;
    size_t cap;
    size_t lo;   // first free byte from the bottom
    size_t hi;   // one past the last free byte from the top
} DStack;

static inline void dstack_init(DStack *s, void *buf, size_t cap) {
    s->base = buf; s->cap = cap; s->lo = 0; s->hi = cap;
}

// align must be a power of two. Alignment is applied to the address,
// so it works whatever the alignment of buf.
static inline void *dstack_alloc_lo(DStack *s, size_t size, size_t align) {
    uintptr_t b = (uintptr_t)s->base;
    uintptr_t p = (b + s->lo + (align - 1)) & ~(uintptr_t)(align - 1);
    size_t off = (size_t)(p - b);
    if (off > s->hi || size > s->hi - off) return NULL;   // would cross
    s->lo = off + size;
    return s->base + off;
}

static inline void *dstack_alloc_hi(DStack *s, size_t size, size_t align) {
    uintptr_t b = (uintptr_t)s->base;
```

```
    if (size > s->hi - s->lo) return NULL;
    uintptr_t p = (b + s->hi - size) & ~(uintptr_t)(align - 1);  // round down
    if (p < b + s->lo) return NULL;                              // would cross
    s->hi = (size_t)(p - b);
    return s->base + s->hi;
}

// Marks let one side free everything allocated after the mark, in LIFO order.
static inline size_t dstack_mark_lo(const DStack *s) { return s->lo; }
static inline size_t dstack_mark_hi(const DStack *s) { return s->hi; }
static inline void dstack_rewind_lo(DStack *s, size_t m) { s->lo = m; }
static inline void dstack_rewind_hi(DStack *s, size_t m) { s->hi = m; }
static inline void dstack_reset_lo(DStack *s) { s->lo = 0; }
static inline void dstack_reset_hi(DStack *s) { s->hi = s->cap; }
static inline size_t dstack_free(const DStack *s) { return s->hi - s->lo; }
#endif
```

Two details matter. The top side rounds the *address* down to the alignment, so its padding sits above the block rather than below it. And both sides check for crossing *before* moving the head, using subtraction (`size > s->hi - off`) so a huge `size` can't overflow the comparison.

Case 1: two lifetimes without fragmentation

```
#include <stdio.h>
#include <string.h>
#include <stdalign.h>
#include "dstack.h"

#define MB (1024u * 1024u)
static alignas(64) unsigned char heap[100 * MB];

static void show(const char *what, const DStack *s) {
    printf("%-34s lo=%3zuMB hi=%3zuMB free=%3zuMB\n", what,
           s->lo / MB, s->hi / MB, dstack_free(s) / MB);
}

int main(void) {
    DStack s;
    dstack_init(&s, heap, sizeof heap);

    // Level data from the bottom, character data from the top, interleaved in time.
    dstack_alloc_lo(&s, 20 * MB, 16);  dstack_alloc_hi(&s, 30 * MB, 16);
    dstack_alloc_lo(&s, 20 * MB, 16);  dstack_alloc_hi(&s, 30 * MB, 16);
    show("level 20+20, character 30+30", &s);
    printf("one more 1MB level chunk: %s\n", dstack_alloc_lo(&s, MB, 16) ? "ok" : "NULL (hea
```

```
    dstack_reset_lo(&s);                        // player moved: drop all level data
    show("level reset", &s);
    void *lvl = dstack_alloc_lo(&s, 30 * MB, 16);
    printf("new 30MB level chunk: %s\n", lvl ? "ok, contiguous" : "NULL");
    show("after new level", &s);

    // Scoped scratch on the top side: mark, use, rewind.
    size_t m = dstack_mark_hi(&s);
    char *tmp = dstack_alloc_hi(&s, 4096, 64);
    printf("scratch at offset %td, 64-aligned: %s\n",
           (unsigned char *)tmp - heap, ((uintptr_t)tmp % 64 == 0) ? "yes" : "no");
    dstack_rewind_hi(&s, m);
    show("scratch rewound", &s);
    return 0;
}
```

Verified with **gcc (Debian 14.2.0-19) 14.2.0**, glibc 2.41, under ASan and UBSan:

```
gcc -std=c23 -O2 -Wall -Wextra -Werror -fsanitize=address,undefined -o demo demo.c && ./demo
```

```
level 20+20, character 30+30       lo= 40MB hi= 40MB free=  0MB
one more 1MB level chunk: NULL (heads would cross)
level reset                        lo=  0MB hi= 40MB free= 40MB
new 30MB level chunk: ok, contiguous
after new level                    lo= 30MB hi= 40MB free= 10MB
scratch at offset 41938944, 64-aligned: yes
scratch rewound                    lo= 30MB hi= 40MB free= 10MB
```

With one arena and the same interleaving (level, character, level, character), freeing the level data would leave two 20 MB holes, and a 30 MB request would fail with 40 MB free. Here the level side's free space is always one run. The scratch line shows the other use of a top side: a mark-and-rewind region for temporaries that doesn't disturb either long-lived stack.

Case 2: variable-sized records plus an index in a fixed packet

Payload bytes grow up from the bottom, and a `uint16_t` offset per record grows down from the top. Record i is found in O(1), and its length is the distance to the next record's offset (or to `lo` for the last one). Adding a record is all-or-nothing: if the index slot doesn't fit, the payload allocation is rewound too.

```
// Variable-sized records with random access: payloads grow up from the
// bottom, a table of uint16_t offsets grows down from the top.
#include <stdio.h>
#include <string.h>
#include <stdalign.h>
#include "dstack.h"
```

```
typedef struct { DStack s; uint16_t *index; size_t n; } Packet;

static int packet_add(Packet *p, const void *rec, size_t len) {
    size_t mlo = dstack_mark_lo(&p->s), mhi = dstack_mark_hi(&p->s);
    unsigned char *dst = dstack_alloc_lo(&p->s, len, 1);
    uint16_t *slot = dst ? dstack_alloc_hi(&p->s, sizeof *slot, alignof(uint16_t)) : NULL;
    if (!slot) {                          // all or nothing: undo the payload too
        dstack_rewind_lo(&p->s, mlo); dstack_rewind_hi(&p->s, mhi);
        return -1;
    }
    memcpy(dst, rec, len);
    *slot = (uint16_t)(dst - p->s.base);
    p->index = slot;                      // the newest entry is lowest in memory
    return (int)p->n++;
}

// Record i (0 = first added) and its length.
static const unsigned char *packet_get(const Packet *p, size_t i, size_t *len) {
    const uint16_t *top = (const uint16_t *)(p->s.base + p->s.cap);
    uint16_t off = top[-1 - (ptrdiff_t)i];
    uint16_t end = (i + 1 < p->n) ? top[-2 - (ptrdiff_t)i] : (uint16_t)p->s.lo;
    *len = (size_t)(end - off);
    return p->s.base + off;
}

int main(void) {
    static alignas(8) unsigned char mtu[64];
    Packet p = {0};
    dstack_init(&p.s, mtu, sizeof mtu);
    const char *cmds[] = {"MOVE 12 40", "FIRE", "CHAT hello there", "PING", "SPAWN orc 7 7 3
    for (size_t i = 0; i < sizeof cmds / sizeof *cmds; i++) {
        int r = packet_add(&p, cmds[i], strlen(cmds[i]));
        printf("add %-18s -> %s (free %zu)\n", cmds[i], r < 0 ? "full" : "ok", dstack_free(&
    }
    for (size_t i = 0; i < p.n; i++) {
        size_t len; const unsigned char *r = packet_get(&p, i, &len);
        printf("[%zu] off=%2td len=%2zu %.*s\n", i, r - mtu, len, (int)len, (const char *)r)
    }
    return 0;
}
```

```
gcc -std=c23 -O2 -Wall -Wextra -Werror -fsanitize=address,undefined -o packet packet.c && ./
```

```
add MOVE 12 40         -> ok (free 52)
add FIRE               -> ok (free 46)
```

```
add CHAT hello there   -> ok (free 28)
add PING               -> ok (free 22)
add SPAWN orc 7 7 3    -> ok (free 5)
add NOP                -> ok (free 0)
add BYE                -> full (free 0)
[0] off= 0 len=10 MOVE 12 40
[1] off=10 len= 4 FIRE
[2] off=14 len=16 CHAT hello there
[3] off=30 len= 4 PING
[4] off=34 len=15 SPAWN orc 7 7 3
[5] off=49 len= 3 NOP
```

The 64-byte packet holds six commands and their index with zero bytes to spare. No length prefixes are needed, because the index already implies them. The receiver needs the record count (or `lo`) in a header to find where the index starts.

Case 3: sharing it between threads

For a single stack, `atomic_fetch_add` on the head is a complete lock-free allocator. It's tempting to extend that to two heads: `fetch_add` on `lo`, check against `hi`, and `fetch_sub` to undo if the heads crossed. The undo is the bug. Undos from different threads land in a different order than their bumps, so a late undo can move `lo` back *below* a block another thread has just been given, and the next allocation overlaps it. The fix is to make "check for crossing and move a head" a single step. Pack both 32-bit heads into one 64-bit atomic and update it with compare-and-swap. A failed allocation then never changes the state.

```
#define _POSIX_C_SOURCE 200809L
// Two lock-free variants of a shared double-ended stack, stress-tested for
// overlapping allocations. Build: gcc -std=c23 -O2 -pthread mt.c
#include <stdatomic.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { CAP = 4096, THREADS = 8, PER_THREAD = 256, ROUNDS = 20000 };

// --- Variant A: one atomic per head, fetch_add then check, undo on failure.
static _Atomic size_t lo_a, hi_a;
static long alloc_naive(int top, size_t n) {
    if (!top) {
        size_t off = atomic_fetch_add(&lo_a, n);
        if (off + n > atomic_load(&hi_a)) { atomic_fetch_sub(&lo_a, n); return -1; }
        return (long)off;
    }
```

```
    size_t end = atomic_fetch_sub(&hi_a, n);
    if (end < n || end - n < atomic_load(&lo_a)) { atomic_fetch_add(&hi_a, n); return -1; }
    return (long)(end - n);
}

// --- Variant B: both heads packed in one 64-bit word, updated by CAS.
static _Atomic uint64_t heads;   // low 32 bits: lo, high 32 bits: hi
static long alloc_packed(int top, size_t n) {
    uint64_t cur = atomic_load_explicit(&heads, memory_order_relaxed), next;
    long off;
    do {
        uint32_t lo = (uint32_t)cur, hi = (uint32_t)(cur >> 32);
        if (n > hi - lo) return -1;                    // full: no state change
        if (!top) { off = lo; next = ((uint64_t)hi << 32) | (lo + n); }
        else      { off = hi - n; next = ((uint64_t)(hi - n) << 32) | lo; }
    } while (!atomic_compare_exchange_weak_explicit(&heads, &cur, next,
                 memory_order_acq_rel, memory_order_relaxed));
    return off;
}

typedef struct { long off[PER_THREAD]; unsigned len[PER_THREAD]; int n, fails; unsigned seed
static long (*alloc_fn)(int, size_t);
static Log logs[THREADS];
static pthread_barrier_t go;

static void *worker(void *arg) {
    Log *L = arg;
    L->n = L->fails = 0;
    pthread_barrier_wait(&go);
    for (int i = 0; i < PER_THREAD; i++) {
        unsigned len = 1 + rand_r(&L->seed) % 48;
        long off = alloc_fn(rand_r(&L->seed) & 1, len);
        if (off < 0) { L->fails++; continue; }
        L->off[L->n] = off; L->len[L->n++] = len;
    }
    return NULL;
}

static int run(const char *name, long (*fn)(int, size_t)) {
    static unsigned char owner[CAP];
    long overlaps = 0, bad_rounds = 0, wasted = 0;
    alloc_fn = fn;
    for (int r = 0; r < ROUNDS; r++) {
        atomic_store(&lo_a, 0); atomic_store(&hi_a, CAP);
        atomic_store(&heads, (uint64_t)CAP << 32);
        pthread_t t[THREADS];
```

```
        pthread_barrier_init(&go, NULL, THREADS);
        for (int i = 0; i < THREADS; i++) { logs[i].seed = r * 131u + i; pthread_create(&t[i
        for (int i = 0; i < THREADS; i++) pthread_join(t[i], NULL);
        pthread_barrier_destroy(&go);
        memset(owner, 0, sizeof owner);
        long ov = 0, used = 0;
        for (int i = 0; i < THREADS; i++)
            for (int k = 0; k < logs[i].n; k++)
                for (unsigned b = 0; b < logs[i].len[k]; b++) {
                    long p = logs[i].off[k] + b;
                    if (p < 0 || p >= CAP || owner[p]) ov++; else { owner[p] = 1; used++; }
                }
        overlaps += ov; bad_rounds += ov > 0;
        wasted += CAP - used;                 // bytes never handed out
    }
    printf("%-26s rounds=%d  rounds with overlap=%ld  overlapping bytes=%ld  avg unused byte
           name, ROUNDS, bad_rounds, overlaps, wasted / ROUNDS);
    return bad_rounds != 0;
}

int main(void) {
    int a = run("fetch_add + undo", alloc_naive);
    int b = run("packed heads + CAS", alloc_packed);
    return (a << 1) | b;
}
```

```
gcc -std=c23 -O2 -Wall -Wextra -Werror -pthread -o mt mt.c && time ./mt; echo "exit=$?"
```

```
fetch_add + undo           rounds=20000  rounds with overlap=3858  overlapping bytes=27122
packed heads + CAS         rounds=20000  rounds with overlap=0  overlapping bytes=0  avg unu
exit=2

real    0m10.153s
user    0m13.124s
sys 0m13.093s
```

That's 8 threads on 8 CPUs, a 4 KiB buffer, and 256 random 1–48 byte requests per thread per round, enough to fill the buffer every round. The two-atomic version handed out overlapping memory in about 19% of rounds. The packed version never did, and it filled the buffer completely. The CAS loop can retry under contention, so it is lock-free rather than wait-free. If one subsystem's allocation rate dominates, a per-thread or per-side front cache is the next step. The packed version limits `cap` to 4 GiB; use `unsigned __int128` with a 16-byte CAS, or a mutex, beyond that.

Things that bite

- **Lifetimes, not sizes, decide the split.** If two "sides" in fact free in interleaved order, a double-ended stack just moves the fragmentation into rewinds that can't happen. Each side must free LIFO or all at once.
- **Pointers into a reset side dangle.** As with any arena, ASan won't see it unless you poison the freed range yourself (see the arena chapter's `ARENA_POISON`).
- **Alignment on the top side rounds down.** Computing `hi - size` and then rounding *up* would push the block past `hi` into memory the top stack already owns.
- **No destructors.** Anything owning a file descriptor or a `malloc`'d buffer must be cleaned up before its side resets.

Try this

1. Add `dstack_high_water(const DStack *)`, which tracks the smallest gap ever seen, and use it to size the buffer from a real run.
2. Make `packet_add` store a 1-byte record type in the index entry next to the offset, and measure how many more `PING`s fit than with a type byte in each payload.
3. In `mt.c`, replace the undo in `alloc_naive` with "leak the bytes on failure" (no `fetch_sub`). Do overlaps disappear? What happens to `avg unused bytes/round`, and why does that still make it unusable near capacity?

Sources

- Alan Wolfe, "The Double Headed Stack Allocator" (2026-10-03): https://blog.demofox.org/2026/10/03/the-double-headed-stack-allocator/
- C atomics, `atomic_compare_exchange_weak_explicit`: https://en.cppreference.com/w/c/atomic/atomic_compare_exchange
- `atomic_fetch_add`: https://en.cppreference.com/w/c/atomic/atomic_fetch_add
- `alignas` and `alignof` in C: https://en.cppreference.com/w/c/language/_Alignas
- POSIX `pthread_barrier_wait`: https://pubs.opengroup.org/onlinepubs/9799919799/functions/pthread_barrier_wait.html
- D. E. Knuth, *The Art of Computer Programming*, Vol. 1, §2.2.2 (two stacks sharing one memory area, growing toward each other).

How Redis wraps malloc: zmalloc accounting, used_memory vs RSS, and fragmentation

A server that enforces a memory limit needs a number it can trust for "how much am I using?", and plain `malloc` doesn't provide one. Redis solves this with a thin wrapper layer, `zmalloc.c`: every allocation goes through `zmalloc`/`zfree`, which keep a `used_memory` counter. This chapter rebuilds that layer in about 50 lines, uses it to show the gap between `used_memory` and what the kernel reports (RSS), and explains why freeing memory often doesn't lower RSS.

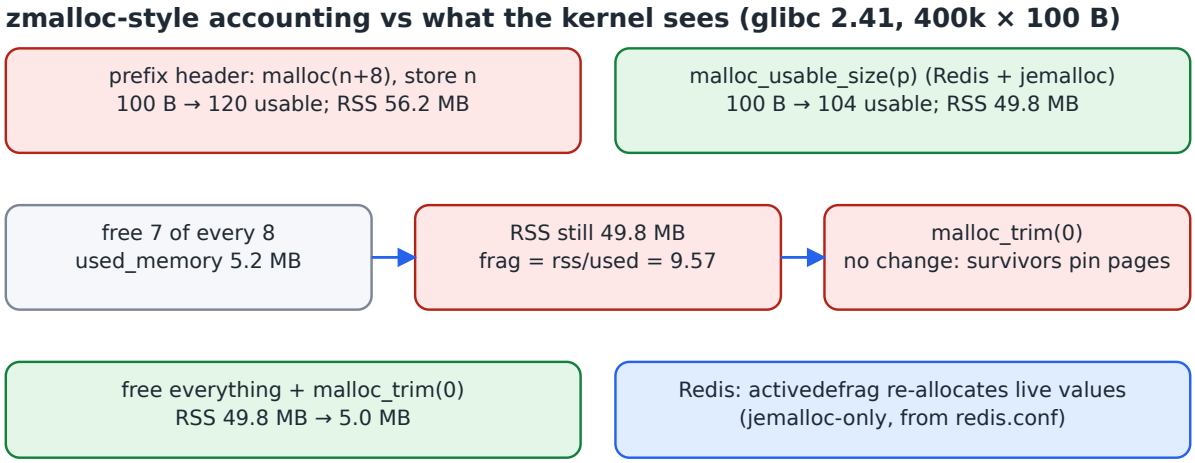


Figure 1: Two accounting strategies, then fragmentation after freeing 7 of every 8 objects

Two ways to know a block's size

`zfree(p)` has to subtract the size of `p` from the counter, but `free` doesn't tell you what that size was. Redis's source handles both cases:

- **The allocator can report it.** In `zmalloc.h`, jemalloc builds define `HAVE_MALLOC_SIZE` and `zmalloc_size(p)` as `je_malloc_usable_size(p)`; tcmalloc and macOS have their own equivalents. Then `PREFIX_SIZE` is 0.
- **It can't.** Then `zmalloc.c` sets `PREFIX_SIZE` to `sizeof(size_t)`, allocates `size + PREFIX_SIZE`, stores the size in the header, and returns the pointer just past it.

The real `update_zmalloc_stat_alloc` keeps the count in per-thread entries and only occasionally checks the global total and peak, to avoid contention on a single atomic. The version below uses one atomic counter, which is enough to show the idea:

```c
/* A zmalloc-style accounting wrapper: two strategies, one counter. */
#define _GNU_SOURCE
#include <malloc.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static _Atomic size_t used_memory;

#ifdef USE_PREFIX
/* No way to ask the allocator for a block's size: store it ourselves. */
#define PREFIX_SIZE sizeof(size_t)
void *zmalloc(size_t n) {
    size_t *p = malloc(n + PREFIX_SIZE);
    if (!p) abort();
    *p = n;
    atomic_fetch_add(&used_memory, n + PREFIX_SIZE);
    return (char *)p + PREFIX_SIZE;
}
void zfree(void *ptr) {
    if (!ptr) return;
    size_t *p = (size_t *)((char *)ptr - PREFIX_SIZE);
    atomic_fetch_sub(&used_memory, *p + PREFIX_SIZE);
    free(p);
}
#define STRATEGY "prefix header"
#else
/* Allocator knows the real block size (what Redis does with jemalloc). */
void *zmalloc(size_t n) {
    void *p = malloc(n);
    if (!p) abort();
    atomic_fetch_add(&used_memory, malloc_usable_size(p));
    return p;
}
void zfree(void *p) {
    if (!p) return;
    atomic_fetch_sub(&used_memory, malloc_usable_size(p));
    free(p);
}
#define STRATEGY "malloc_usable_size"
#endif

static size_t rss_bytes(void) {          /* like zmalloc_get_rss on Linux */
    long pages = 0, rss = 0;
    FILE *f = fopen("/proc/self/statm", "r");
```

```
    if (f) { if (fscanf(f, "%ld %ld", &pages, &rss) != 2) rss = 0; fclose(f); }
    return (size_t)rss * 4096;
}

static void report(const char *when, size_t requested) {
    size_t used = atomic_load(&used_memory), rss = rss_bytes();
    printf("%-26s requested=%9zu used_memory=%9zu rss=%9zu frag=%.2f\n",
           when, requested, used, rss, used ? (double)rss / used : 0.0);
}

enum { N = 400000, SZ = 100 };
static void *objs[N];

int main(void) {
    printf("strategy: %s\n", STRATEGY);
    void *one = zmalloc(SZ);
    printf("zmalloc(%d): usable=%zu\n", SZ, malloc_usable_size(
#ifdef USE_PREFIX
        (char *)one - PREFIX_SIZE
#else
        one
#endif
    ));
    zfree(one);
    report("start", 0);
    for (int i = 0; i < N; i++) { objs[i] = zmalloc(SZ); memset(objs[i], 1, SZ); }
    report("after 400k x 100B", (size_t)N * SZ);
    for (int i = 0; i < N; i++) if (i % 8) { zfree(objs[i]); objs[i] = NULL; }
    report("freed 7 of every 8", (size_t)N / 8 * SZ);
    malloc_trim(0);
    report("after malloc_trim(0)", (size_t)N / 8 * SZ);
    for (int i = 0; i < N; i++) zfree(objs[i]);
    report("freed all", 0);
    malloc_trim(0);
    report("freed all + malloc_trim(0)", 0);
    return 0;
}
```

Real output (gcc 14.2 -O2, glibc 2.41, x86_64)

```
$ ./zm_usable
strategy: malloc_usable_size
zmalloc(100): usable=104
start                      requested=        0 used_memory=        0 rss=  1675264 frag=0.00
```

```
after 400k x 100B          requested= 40000000 used_memory= 41600000 rss= 49778688 frag=1.20
freed 7 of every 8         requested=  5000000 used_memory=  5200000 rss= 49778688 frag=9.57
after malloc_trim(0)       requested=  5000000 used_memory=  5200000 rss= 49774592 frag=9.57
freed all                  requested=        0 used_memory=        0 rss= 49774592 frag=0.00
freed all + malloc_trim(0) requested=        0 used_memory=        0 rss=  4976640 frag=0.00
$ ./zm_prefix
strategy: prefix header
zmalloc(100): usable=120
start                      requested=        0 used_memory=        0 rss=  1572864 frag=0.00
after 400k x 100B          requested= 40000000 used_memory= 43200000 rss= 56160256 frag=1.30
freed 7 of every 8         requested=  5000000 used_memory=  5400000 rss= 56160256 frag=10.4
after malloc_trim(0)       requested=  5000000 used_memory=  5400000 rss= 56160256 frag=10.4
freed all                  requested=        0 used_memory=        0 rss= 56160256 frag=0.00
freed all + malloc_trim(0) requested=        0 used_memory=        0 rss=  4960256 frag=0.00
```

Both builds were also run under `-fsanitize=address` and came out clean.

What the numbers say

The allocator rounds up, and the two strategies see it differently. glibc gave 104 usable bytes for a 100-byte request (a 112-byte chunk). The prefix version asks for 108 bytes and gets 120 usable bytes (a 128-byte chunk), so the 8-byte header really costs 16 bytes per object. Its counter (`n + 8`) still claims 43.2 MB, while the chunks behind it total 51.2 MB. RSS bears this out: 56.2 MB for the prefix build against 49.8 MB for the other. `malloc_usable_size` counts what the allocator actually reserved, which is why Redis prefers it. Note that the man page says it is "intended to only be used for diagnostics and statistics". Accounting is a statistic. Never write into the extra bytes.

Freeing doesn't give memory back if the survivors are spread out. After freeing 7 of every 8 objects, `used_memory` fell to 5.2 MB but RSS stayed at 49.8 MB. With 112-byte chunks, each 4 KiB page holds about 36 objects, so about 4 survivors remain on *every* page. No page is empty, and nothing can be returned. `malloc_trim(0)` also changed nothing. It can only release whole free pages, and there were none. Only after the last objects were freed did the same call bring RSS down to 5.0 MB.

This is what Redis's ratio measures. `INFO memory` reports `mem_fragmentation_ratio`, the "Ratio between used_memory_rss and used_memory". The 9.57 above is that ratio for this pathological free pattern. In this toy program the ratio also includes the binary's own pages (about 1.6 MB RSS at start), which is why the docs point to `allocator_frag_ratio` as "the true (external) fragmentation metric".

What Redis does about it: active defrag

You can't move a C object that other code holds a pointer to, so a general-purpose allocator can't compact the heap. Redis can, because it owns every pointer to its values. `redis.conf` describes `activedefrag`: when fragmentation passes a threshold, Redis "will start to create new copies of the

values in contiguous memory regions by exploiting certain specific Jemalloc features … and at the same time, will release the old copies". It is "disabled by default, and only works if you compiled Redis to use the copy of Jemalloc we ship". I did not run Redis or jemalloc for this chapter; this section is written from those primary sources.

You can apply the same idea in your own C code without jemalloc. If a structure owns all references to its objects (a hash table of values, say), you can rebuild it after heavy churn: allocate fresh copies, swap them in, then free the old ones. This turns scattered survivors into dense pages and gives `malloc_trim` free pages to return.

Checklist

- ☐ Route every allocation through one wrapper so the counter means something.
- ☐ Prefer `malloc_usable_size` (or `je_malloc_usable_size`) over a size header: it is more accurate, and the header also costs allocator rounding.
- ☐ Track RSS alongside `used_memory`, and alert on the ratio, not just the counter.
- ☐ Remember that `free` lowers your counter but not necessarily RSS. `malloc_trim` only returns whole free pages.
- ☐ For long-lived, high-churn heaps that own all their pointers, plan compaction (rebuild and swap), as Redis's active defrag does.

restrict and Pointer Aliasing

restrict and Pointer Aliasing

Thesis

`restrict` (C99) is a promise you make to the compiler: for the lifetime of that pointer, no other pointer in the same scope will access the same object. That promise unlocks vectorisation and load/store reordering. Break the promise and you get silent wrong answers — not a warning.

Mental model

```
add(n, out, a, b) with restrict on all three
     → compiler may assume out[i] is never a[j] or b[j]
     → safe to keep a[i], b[i] in registers across the store to out[i]
```

Without `restrict`, a legal (if pathological) call is `add(n, a, a, b)` where stores to `out` change later loads from `a`.

Worked example

Verified with **GCC 14.2** `-O3 -std=c11`.

```c
/* restrict.c - fragment */
void add_restrict(size_t n, double *restrict out,
                  const double *restrict a, const double *restrict b) {
  for (size_t i = 0; i < n; i++) out[i] = a[i] + b[i];
}

void add_alias(size_t n, double *out, const double *a, const double *b) {
  for (size_t i = 0; i < n; i++) out[i] = a[i] + b[i];
}
```

A microbenchmark on this box (`n=1<<22`, 50 repeats, disjoint buffers) printed:

```
n=4194304 restrict_ms=16.66 alias_ms=6.76 out0=3.0
```

Treat microbenchmarks carefully: both versions already vectorise at `-O3` when buffers do not alias, so wall-clock can go either way depending on cache state. The value of `restrict` is the **permission** it gives the optimiser, visible in the generated assembly (`objdump -d`) more reliably than in a single timing run.

Trap

Lying with `restrict`. If `out` overlaps `a`, the compiler may emit code that is wrong for that call. Never slap `restrict` on API pointers "for speed" unless the contract forbids overlap (document it).

Boring rule

Use `restrict` on hot kernels where the contract is "distinct buffers", document that contract, and prefer sanitizers / tests with deliberate overlap to catch misuse. Do not use it as a general style pepper.

Try this

1. Compare `add_restrict` vs `add_alias` assembly at `-O3` with `objdump -d`.
2. Call `add_restrict(n, a+1, a, b)` (overlap) and observe wrong results under optimisation.
3. Read how `memcpy` vs `memmove` relate to the same aliasing idea.

Sources

- ISO/IEC 9899:1999 §6.7.3 (`restrict`)
- GCC manual: Restricted Pointers

Timing Code with RDTSC and RDTSCP: Fences, Overhead, and Converting Ticks

Timing Code with RDTSC and RDTSCP: Fences, Overhead, and Converting Ticks

Thesis

The time-stamp counter (TSC) is the cheapest clock on an x86-64 machine: one instruction, no system call, and a resolution well below a nanosecond. That makes it tempting for micro-timing, and it is also why so many TSC measurements are wrong. Used naively, `rdtsc` has three problems:

1. **It is not ordered.** The CPU executes out of order, so a bare `rdtsc` can read the counter before earlier work has finished, or after later work has started.
2. **It has a cost of its own.** A pair of reads costs tens of ticks, which is about the same size as the short regions people try to time with it.
3. **Ticks are not cycles.** On any CPU with an *invariant* TSC, the counter ticks at a fixed reference rate. It does not follow the core clock as it turbos up and down.

This chapter fixes each problem in C, using the GCC/Clang intrinsics `__rdtsc`, `__rdtscp` and `_mm_lfence`, and compares the result with `clock_gettime`. Everything was built with gcc 14.2.0 `-O2 -std=c23` and run on an 8-vCPU Intel Xeon VM whose kernel clocksource is `kvm-clock`.

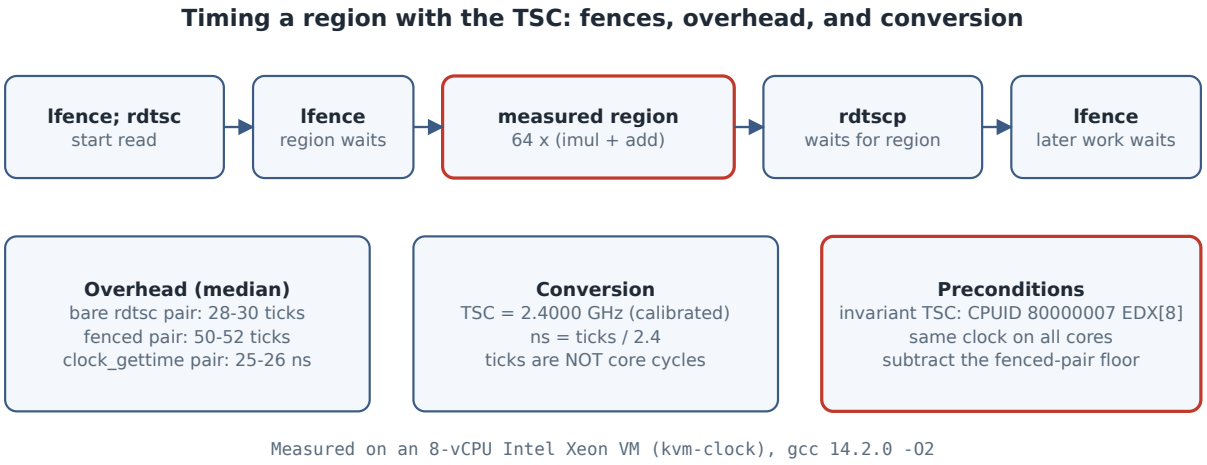


Figure 1: Timing a region with the TSC: lfence/rdtsc/lfence before, rdtscp/lfence after, with measured overheads and the conversion to ns

Step 0: is the TSC usable at all?

The Intel SDM (Vol. 3B, "Time-Stamp Counter") describes the **invariant TSC**: on processors that report it, the counter "will run at a constant rate in all ACPI P-, C-, and T-states". Software detects it with `CPUID.80000007H:EDX[8]`, and `RDTSCP` support with `CPUID.80000001H:EDX[27]`. GCC's `<cpuid.h>` makes that a few lines:

```
#include <cpuid.h>
#include <stdio.h>
int main(void) {
    unsigned a, b, c, d;
    if (!__get_cpuid(0x80000007, &a, &b, &c, &d)) { puts("leaf 0x80000007 unsupported"); ret
    printf("invariant TSC (CPUID 0x80000007 EDX[8]): %s\n", (d >> 8) & 1 ? "yes" : "no");
    if (__get_cpuid(0x80000001, &a, &b, &c, &d))
        printf("RDTSCP (CPUID 0x80000001 EDX[27]):       %s\n", (d >> 27) & 1 ? "yes" : "no"
    return 0;
}
```

```
invariant TSC (CPUID 0x80000007 EDX[8]): yes
RDTSCP (CPUID 0x80000001 EDX[27]):       yes
```

On Linux the same facts appear as the `constant_tsc` and `nonstop_tsc` flags in `/proc/cpuinfo`. This VM has both, plus `rdtscp` and `tsc_known_freq`. If the invariant bit is missing, stop here: the counter can change rate with frequency scaling, and you should use `clock_gettime`.

Step 1: order the reads

The SDM's instruction reference for `RDTSC` is explicit. It is not a serializing instruction and "does not necessarily wait until all previous instructions have been executed before reading the counter". It gives the fix as well: execute `LFENCE` *before* `RDTSC` so that earlier instructions finish first, and `LFENCE` *after* it so that later instructions cannot start before the read.

`RDTSCP` is half-ordered. It "waits until all previous instructions have executed and all previous loads are globally visible", but later instructions may still begin before it reads the counter. So the end of a region needs `rdtscp` followed by `lfence`.

```
#include <x86intrin.h>   /* __rdtsc, __rdtscp, _mm_lfence */

static inline uint64_t tsc_begin(void) {      /* fence, then read: earlier work must retire
    _mm_lfence();
    uint64_t t = __rdtsc();
    _mm_lfence();                              /* keep later loads from starting before the
    return t;
}
static inline uint64_t tsc_end(void) {        /* rdtscp waits for earlier instructions */
    unsigned aux;
```

```
    uint64_t t = __rdtscp(&aux);
    _mm_lfence();                              /* ...but does not stop later ones running ea
    return t;
}
```

This is the same shape as Intel's benchmarking white paper (Paoloni, *How to Benchmark Code Execution Times on Intel IA-32 and IA-64*), with `lfence` in place of the paper's `cpuid`. `cpuid` fully serializes, but it is far slower and traps to the hypervisor inside a VM. `__rdtscp` also stores `IA32_TSC_AUX` into `aux`. On Linux that holds the CPU number, so you can throw away samples where the thread migrated between the two reads.

Step 2: measure the measurement

Before timing anything, time *nothing*. Each loop below takes 1,000,001 samples of an empty region and reports min/median/p99. The harness (full listing in the appendix) also calibrates the TSC against `CLOCK_MONOTONIC_RAW` over 200 ms:

```
TSC frequency: 2.4000 GHz (ticks per ns)

bare rdtsc pair (ticks)      min    24.0  p50    28.0  p99    34.0 ticks
fenced pair (ticks)          min    44.0  p50    52.0  p99    58.0 ticks
clock_gettime pair (ns)      min    21.0  p50    26.0  p99    34.0 ns
64 dep. mul-add, fenced      min    80.0  p50    89.2  p99   105.8 ns
64 dep. mul-add, clock_gettime min    82.0  p50    93.0  p99   107.0 ns
```

How to read this:

- **The TSC runs at 2.4000 GHz here.** That is the nominal frequency of the virtual CPU, so `ns = ticks / 2.4`.
- **A bare pair costs 28–30 ticks** (about 12 ns), and **the fenced pair costs 50–52 ticks** (about 21 ns). That 50-tick floor is part of every fenced number, so subtract it, or keep the region large enough that it doesn't matter.
- **A `clock_gettime` pair costs 25–26 ns.** On Linux, `clock_gettime` runs in the vDSO and does not enter the kernel. With `kvm-clock` it reads the TSC and scales it using parameters the hypervisor shares with the guest. It costs about the same as the fenced TSC pair, and returns nanoseconds you don't need to calibrate.

The last two lines time a real region: 64 iterations of a dependent `x = x * A + C`. The compiler emits a tight `imulq`/`addq` loop (checked with `gcc -S`). Both clocks agree: about 89 ns median, which is roughly 214 ticks.

Step 3: ticks are not core cycles

On most x86-64 cores, `imul r64, r64` has 3-cycle latency and `add` has 1, so 64 iterations of the dependency chain take about **256 core cycles**. Subtract the fenced-pair floor from the median

and you get about 214 − 51 **163 TSC ticks**. The region took fewer ticks than cycles because this core was running faster than the 2.4 GHz reference clock. The ratio suggests about 1.57 × 2.4 3.8 GHz. That figure is an inference from latency tables, not a measurement; to count real core cycles, use the `cycles` event in `perf stat`.

That is the main thing to understand about an invariant TSC: it measures **wall time** at a fixed rate. That makes it a good clock, but it can't tell you how many cycles your code took.

When to use which

Need	Use
Wall time, portable, nanoseconds	`clock_gettime(CLOCK_MONOTONIC)` (vDSO, ~25 ns per pair here)
Smallest possible overhead in a hot loop you will aggregate	bare `__rdtsc()` and many samples. Accept reordering noise
One short region, ordered	`lfence; rdtsc; lfence` … `rdtscp; lfence`, minus the fenced-pair floor
Core cycles or instructions	`perf stat -e cycles,instructions`, not the TSC
No invariant TSC (old or odd hardware)	`clock_gettime` only

Pitfalls

- **Forgetting the floor.** A region measured as "52 ticks" with fenced reads is empty.
- **Reporting the mean.** The p99 values above are about 2× the median even on an idle VM. Interrupts and vCPU preemption produce a long tail. Report min and median, and keep the distribution.
- **Assuming the reference rate.** Calibrate, or read it from `tsc_known_freq`/`dmesg`. Don't hard-code a number from the CPU name.
- **Migrating threads.** Pin the thread (`taskset -c 2 ./tsc`) or check `aux` from `rdtscp`. Invariant TSCs are synchronized across cores on modern platforms, but pinning also removes migration noise.
- **Non-x86 targets.** `x86intrin.h` is x86-only. AArch64 has `CNTVCT_EL0`, a fixed-frequency virtual counter with similar semantics. Hide whichever one you use behind a function.

Takeaways

- Check `CPUID.80000007H:EDX[8]` (or `constant_tsc`/`nonstop_tsc`) before trusting the TSC.
- Fence both ends: `lfence; rdtsc; lfence` to start, `rdtscp; lfence` to finish.
- Measure an empty region first and subtract it.
- Divide by the calibrated TSC frequency to get nanoseconds, and remember that this is wall time, not core cycles.
- If you are not trying to shave the last 10 ns off the probe, `clock_gettime` is just as accurate.

Sources

- Intel® 64 and IA-32 Architectures Software Developer's Manual, Vol. 2B instruction reference for RDTSC and RDTSCP, and Vol. 3B §"Time-Stamp Counter" (invariant TSC)
- Gabriele Paoloni, How to Benchmark Code Execution Times on Intel IA-32 and IA-64 Instruction Set Architectures (Intel white paper)
- Linux man-pages, clock_gettime(2) and vdso(7)
- GCC, x86 built-in functions

Appendix: the full harness

```
/* tsc.c - timing with RDTSC/RDTSCP vs clock_gettime (x86-64, gcc/clang) */
#define _GNU_SOURCE
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <x86intrin.h>   /* __rdtsc, __rdtscp, _mm_lfence */

static inline uint64_t tsc_begin(void) {      /* fence, then read: earlier work must retire
    _mm_lfence();
    uint64_t t = __rdtsc();
    _mm_lfence();                              /* keep later loads from starting before the
    return t;
}
static inline uint64_t tsc_end(void) {        /* rdtscp waits for earlier instructions */
    unsigned aux;
    uint64_t t = __rdtscp(&aux);
    _mm_lfence();                              /* ...but does not stop later ones running ea
    return t;
}
static uint64_t now_ns(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ull + (uint64_t)ts.tv_nsec;
}
static int cmp_u64(const void *a, const void *b) {
    uint64_t x = *(const uint64_t *)a, y = *(const uint64_t *)b;
    return (x > y) - (x < y);
}
#define N 1000001
static uint64_t s[N];
static void report(const char *name, double scale, const char *unit) {
    qsort(s, N, sizeof s[0], cmp_u64);
    printf("%-28s min %7.1f  p50 %7.1f  p99 %7.1f %s\n", name,
```

```
           s[0] * scale, s[N / 2] * scale, s[N / 100 * 99] * scale, unit);
}
volatile uint64_t sink;

int main(void) {
    /* 1. calibrate: TSC ticks per ns against CLOCK_MONOTONIC_RAW over 200 ms */
    uint64_t n0 = now_ns(), t0 = tsc_begin();
    while (now_ns() - n0 < 200000000ull) { }
    uint64_t n1 = now_ns(), t1 = tsc_end();
    double ghz = (double)(t1 - t0) / (double)(n1 - n0);
    printf("TSC frequency: %.4f GHz (ticks per ns)\n\n", ghz);

    /* 2. cost of the measurement itself: empty region */
    for (int i = 0; i < N; i++) { uint64_t a = __rdtsc(); uint64_t b = __rdtsc(); s[i] = b -
    report("bare rdtsc pair (ticks)", 1, "ticks");
    for (int i = 0; i < N; i++) { uint64_t a = tsc_begin(); uint64_t b = tsc_end(); s[i] = b
    report("fenced pair (ticks)", 1, "ticks");
    for (int i = 0; i < N; i++) { uint64_t a = now_ns(); uint64_t b = now_ns(); s[i] = b - a
    report("clock_gettime pair (ns)", 1, "ns");

    /* 3. a real region: 64 dependent multiply-adds (~4 cycles each on most cores) */
    for (int i = 0; i < N; i++) {
        uint64_t x = (uint64_t)i | 1;
        uint64_t a = tsc_begin();
        for (int k = 0; k < 64; k++) x = x * 6364136223846793005ull + 1442695040888963407ull
        uint64_t b = tsc_end();
        sink = x; s[i] = b - a;
    }
    report("64 dep. mul-add, fenced", 1 / ghz, "ns");
    for (int i = 0; i < N; i++) {
        uint64_t x = (uint64_t)i | 1;
        uint64_t a = now_ns();
        for (int k = 0; k < 64; k++) x = x * 6364136223846793005ull + 1442695040888963407ull
        uint64_t b = now_ns();
        sink = x; s[i] = b - a;
    }
    report("64 dep. mul-add, clock_gettime", 1, "ns");
    return 0;
}
```

Build and run with `gcc -O2 -std=c23 -Wall -Wextra -o tsc tsc.c && ./tsc`.

Module 13: Performance Optimization

Overview

Performance optimization is a critical aspect of C programming, especially in resource-constrained environments and high-performance applications. This module covers techniques for analyzing, measuring, and improving the performance of C programs through profiling, algorithm optimization, memory management, and advanced optimization strategies.

Learning Objectives

By the end of this module, you should be able to: - Analyze program performance using profiling tools - Identify and eliminate performance bottlenecks - Optimize algorithms and data structures for better performance - Manage memory efficiently to reduce overhead - Apply compiler optimization techniques - Understand the trade-offs between performance and other factors - Implement caching strategies and algorithmic improvements - Measure and benchmark performance improvements

Chapters

1. **Performance Analysis** - Profiling tools, benchmarking techniques, and performance metrics
2. **Optimization Techniques** - Algorithm optimization, loop optimization, and code restructuring
3. **Memory Optimization** - Memory allocation strategies, cache optimization, and memory layout
4. **Advanced Optimization** - Compiler optimizations, vectorization, and parallel processing
5. **ISPC 1.32: AVX10.2, APX, and Zen 6 Targets** - SPMD vectorization with Intel's ISPC on current x86 targets
6. **Arena Allocators** - Bump-pointer allocation, alignment padding, scratch lifetimes, and the use-after-reset trap

Prerequisites

Before starting this module, you should have a solid understanding of: - C programming fundamentals - Data structures and algorithms - Memory management concepts - Basic understanding of computer architecture

Key Concepts

- Profiling and performance measurement
- Algorithmic complexity analysis
- Memory hierarchy and cache optimization
- Compiler optimization flags and techniques
- Loop optimization and vectorization
- Memory allocation strategies
- Performance benchmarking
- Trade-offs between performance and maintainability

Testing

Testing Methodologies

Testing ensures C code behaves correctly under normal use, edge cases, and failure conditions. This chapter focuses on **practical unit testing in C on Linux/POSIX**: small harnesses you can compile with `gcc`, table-driven tests, TDD, boundary analysis, coverage with `gcov`/`lcov`, and sanitizers.

Build flags used throughout this chapter:

```
gcc -std=c17 -Wall -Wextra -o program program.c
```

Add `-g` for debugging symbols, and sanitizer flags when hunting memory/UB bugs (covered below).

Why Testing Matters

- **Bug detection early**: cheaper than production incidents
- **Documentation**: tests show intended API contracts
- **Safe refactoring**: change internals without fear if tests pass
- **Regression control**: catch breakages when you touch old code

Types of Testing (C-focused)

Level	What you test	Typical tools in C
Unit	One function / small module	`assert`, custom macros, Unity, CUnit
Integration	Several `.c` files / libraries together	Same harness + real I/O stubs
System	Whole binary (CLI, server)	Shell scripts, golden files
Regression	"This used to work" cases	Any automated suite in CI

Unit testing characteristics

- **Small and focused** — one behavior per test when possible
- **Fast** — milliseconds, not minutes
- **Independent** — no order dependence

- **Repeatable** — same result every run
- **Automated** — no human clicks

Style 1: `assert.h` Unit Tests

`<assert.h>` is enough for tiny libraries. Failed asserts abort the process (good for local runs; less pretty for large suites).

```c
/* file: test_add.c */
#include <assert.h>
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

void test_add(void) {
    assert(add(2, 3) == 5);
    assert(add(-1, 1) == 0);
    assert(add(0, 0) == 0);
    assert(add(1000000, 2000000) == 3000000);
}

int main(void) {
    test_add();
    printf("all asserts passed\n");
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o test_add test_add.c
./test_add
```

Note: In release builds, `NDEBUG` disables asserts. Keep tests in a separate binary that never defines `NDEBUG`.

Style 2: Minimal Custom `TEST` Macro Harness

A few macros give you named tests, a pass/fail counter, and a non-zero exit code — without external libraries.

```
/* file: mini_test.h */
#ifndef MINI_TEST_H
#define MINI_TEST_H

#include <stdio.h>
#include <string.h>

static int g_tests_run;
static int g_tests_failed;

#define TEST_BEGIN() \
    do { g_tests_run = 0; g_tests_failed = 0; } while (0)

#define TEST_END()                                                          \
    do {                                                                    \
        printf("\n%d tests, %d failed\n", g_tests_run, g_tests_failed);     \
        return g_tests_failed ? 1 : 0;                                      \
    } while (0)

#define RUN_TEST(fn)                                                        \
    do {                                                                    \
        g_tests_run++;                                                      \
        printf("  RUN  %s ... ", #fn);                                      \
        fflush(stdout);                                                     \
        if ((fn)() == 0) {                                                  \
            printf("OK\n");                                                 \
        } else {                                                            \
            g_tests_failed++;                                               \
            printf("FAIL\n");                                               \
        }                                                                   \
    } while (0)

/* Return 0 on success, 1 on failure from these macros */
#define EXPECT_EQ_INT(actual, expected)                                     \
    do {                                                                    \
        int _a = (actual);                                                  \
        int _e = (expected);                                                \
        if (_a != _e) {                                                     \
            fprintf(stderr, "\n    %s:%d: expected %d, got %d\n",            \
                    __FILE__, __LINE__, _e, _a);                            \
            return 1;                                                       \
        }                                                                   \
    } while (0)

#define EXPECT_TRUE(cond)                                                   \
    do {                                                                    \
        if (!(cond)) {                                                      \
```

```
            fprintf(stderr, "\n    %s:%d: condition false: %s\n",          \
                    __FILE__, __LINE__, #cond);                            \
            return 1;                                                      \
        }                                                                  \
    } while (0)

#define EXPECT_STREQ(actual, expected)                                     \
    do {                                                                   \
        const char *_a = (actual);                                         \
        const char *_e = (expected);                                       \
        if (_a == NULL || _e == NULL || strcmp(_a, _e) != 0) {             \
            fprintf(stderr, "\n    %s:%d: expected \"%s\", got \"%s\"\n", \
                    __FILE__, __LINE__,                                    \
                    _e ? _e : "(null)", _a ? _a : "(null)");               \
            return 1;                                                      \
        }                                                                  \
    } while (0)

#endif /* MINI_TEST_H */
```

```
/* file: test_clamp.c - uses mini_test.h */
#include "mini_test.h"

int clamp(int x, int lo, int hi) {
    if (x < lo) {
        return lo;
    }
    if (x > hi) {
        return hi;
    }
    return x;
}

static int test_clamp_inside(void) {
    EXPECT_EQ_INT(clamp(5, 0, 10), 5);
    return 0;
}

static int test_clamp_below(void) {
    EXPECT_EQ_INT(clamp(-3, 0, 10), 0);
    return 0;
}

static int test_clamp_above(void) {
    EXPECT_EQ_INT(clamp(99, 0, 10), 10);
    return 0;
```

```
}

int main(void) {
    TEST_BEGIN();
    RUN_TEST(test_clamp_inside);
    RUN_TEST(test_clamp_below);
    RUN_TEST(test_clamp_above);
    TEST_END();
}
```

```
gcc -std=c17 -Wall -Wextra -o test_clamp test_clamp.c
./test_clamp
```

Style 3: Unity-Style Suite That Actually Builds

Unity is a popular embedded-friendly C unit framework. You can either vendor its sources or use a **Unity-compatible subset** that builds as a single file for learning.

Option A — Self-contained "unity-like" runner (no download)

```
/* file: unity_lite.h - small subset of Unity API names */
#ifndef UNITY_LITE_H
#define UNITY_LITE_H

#include <stdio.h>
#include <string.h>

static int Unity_tests;
static int Unity_failures;
static const char *Unity_current;

#define UNITY_BEGIN() \
    do { Unity_tests = 0; Unity_failures = 0; printf("Unity-lite\n"); } while (0)

#define UNITY_END()                                                    \
    do {                                                               \
        printf("------------\n%d Tests %d Failures\n",                 \
               Unity_tests, Unity_failures);                           \
        return Unity_failures ? 1 : 0;                                 \
    } while (0)
```

```
#define RUN_TEST(fn)                                                    \
    do {                                                                \
        Unity_tests++;                                                  \
        Unity_current = #fn;                                            \
        printf("TEST(%s)\n", Unity_current);                            \
        fn();                                                           \
    } while (0)

#define TEST_ASSERT_EQUAL_INT(expected, actual)                         \
    do {                                                                \
        int _e = (int)(expected);                                       \
        int _a = (int)(actual);                                         \
        if (_e != _a) {                                                 \
            Unity_failures++;                                           \
            printf("  FAIL: %s:%d expected %d was %d\n",                \
                   __FILE__, __LINE__, _e, _a);                         \
        }                                                               \
    } while (0)

#define TEST_ASSERT_TRUE(cond)                                          \
    do {                                                                \
        if (!(cond)) {                                                  \
            Unity_failures++;                                           \
            printf("  FAIL: %s:%d %s is false\n",                       \
                   __FILE__, __LINE__, #cond);                          \
        }                                                               \
    } while (0)

#define TEST_ASSERT_EQUAL_STRING(expected, actual)                      \
    do {                                                                \
        const char *_e = (expected);                                    \
        const char *_a = (actual);                                      \
        if (_e == NULL || _a == NULL || strcmp(_e, _a) != 0) {          \
            Unity_failures++;                                           \
            printf("  FAIL: %s:%d expected \"%s\" was \"%s\"\n",        \
                   __FILE__, __LINE__,                                  \
                   _e ? _e : "(null)", _a ? _a : "(null)");             \
        }                                                               \
    } while (0)

#endif
```

```
/* file: test_leap_unity.c */
#include "unity_lite.h"

/* Production code under test */
```

```c
int is_leap_year(int year) {
    if (year % 400 == 0) {
        return 1;
    }
    if (year % 100 == 0) {
        return 0;
    }
    if (year % 4 == 0) {
        return 1;
    }
    return 0;
}

void test_common_year(void) {
    TEST_ASSERT_EQUAL_INT(0, is_leap_year(2019));
}

void test_leap_divisible_by_4(void) {
    TEST_ASSERT_EQUAL_INT(1, is_leap_year(2020));
}

void test_century_not_leap(void) {
    TEST_ASSERT_EQUAL_INT(0, is_leap_year(1900));
}

void test_400_year_leap(void) {
    TEST_ASSERT_EQUAL_INT(1, is_leap_year(2000));
}

int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_common_year);
    RUN_TEST(test_leap_divisible_by_4);
    RUN_TEST(test_century_not_leap);
    RUN_TEST(test_400_year_leap);
    return UNITY_END();
}
```

```
gcc -std=c17 -Wall -Wextra -o test_leap test_leap_unity.c
./test_leap
```

Option B — Real Unity (vendored)

If you clone Unity into `third_party/Unity`:

```
# Example layout
# third_party/Unity/src/unity.c
# third_party/Unity/src/unity.h
# third_party/Unity/src/unity_internals.h

gcc -std=c17 -Wall -Wextra \
  -I third_party/Unity/src \
  -o test_leap \
  test_leap_unity_real.c third_party/Unity/src/unity.c
```

Real Unity uses the same names (`UNITY_BEGIN`, `RUN_TEST`, `TEST_ASSERT_EQUAL_INT`, `UNITY_END`). Your tests stay almost identical to Option A.

Test-Driven Development (TDD) Walkthrough: `is_leap_year()`

TDD cycle: **Red → Green → Refactor**.

We implement Gregorian leap-year rules:

- Divisible by 400 → leap
- Else divisible by 100 → not leap
- Else divisible by 4 → leap
- Else → not leap

Step 0 — Scaffold (no real logic yet)

```
/* leap.c + test_leap_tdd.c evolution */

int is_leap_year(int year) {
    (void)year;
    return 0; /* deliberately wrong / incomplete */
}
```

Step 1 — RED: first failing test

```
/* test_leap_tdd.c */
#include "unity_lite.h"

int is_leap_year(int year); /* or #include "leap.h" */
```

```c
void test_2020_is_leap(void) {
    TEST_ASSERT_EQUAL_INT(1, is_leap_year(2020));
}

int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_2020_is_leap);
    return UNITY_END();
}
```

```
gcc -std=c17 -Wall -Wextra -o test_leap_tdd test_leap_tdd.c leap.c
./test_leap_tdd
# FAIL: expected 1 was 0
```

Step 2 — GREEN: minimal code that passes

```c
/* leap.c - first green */
int is_leap_year(int year) {
    return year % 4 == 0;
}
```

```
./test_leap_tdd
# OK for 2020
```

Step 3 — RED: century rule

```c
void test_1900_not_leap(void) {
    TEST_ASSERT_EQUAL_INT(0, is_leap_year(1900));
}

/* main: also RUN_TEST(test_1900_not_leap); */
```

```
./test_leap_tdd
# FAIL for 1900 (1900 % 4 == 0 with naive rule)
```

Step 4 — GREEN: handle centuries

```c
int is_leap_year(int year) {
    if (year % 100 == 0) {
        return 0;
```

```
    }
    return year % 4 == 0;
}
```

Step 5 — RED: year 2000 must be leap

```
void test_2000_is_leap(void) {
    TEST_ASSERT_EQUAL_INT(1, is_leap_year(2000));
}
```

Step 6 — GREEN: full Gregorian rule

```
int is_leap_year(int year) {
    if (year % 400 == 0) {
        return 1;
    }
    if (year % 100 == 0) {
        return 0;
    }
    if (year % 4 == 0) {
        return 1;
    }
    return 0;
}
```

Step 7 — REFACTOR: keep tests green

```
int is_leap_year(int year) {
    return (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
}
```

Re-run the suite after every refactor. All four (or more) tests should still pass.

Alternate TDD target: `clamp()`

Same cycle with fewer domain rules:

Phase	Test idea	Implementation
Red	`clamp(5,0,10)==5`	`return 0;` fails
Green	return `x` if in range	`return x;` (too simple)
Red	`clamp(-1,0,10)==0`	fails
Green	lower bound	`if (x < lo) return lo; return x;`

Phase	Test idea	Implementation
Red	`clamp(99,0,10)==10`	fails
Green	upper bound	full clamp
Refactor	assert `lo <= hi` or document	keep tests green

Boundary Value Testing

Bugs cluster at edges. Design cases for **just inside**, **on**, and **just outside** valid ranges.

Integers

```c
#include "unity_lite.h"
#include <limits.h>

int clamp(int x, int lo, int hi) {
    if (x < lo) return lo;
    if (x > hi) return hi;
    return x;
}

void test_clamp_boundaries(void) {
    /* lo / hi themselves */
    TEST_ASSERT_EQUAL_INT(0, clamp(0, 0, 10));
    TEST_ASSERT_EQUAL_INT(10, clamp(10, 0, 10));

    /* just outside */
    TEST_ASSERT_EQUAL_INT(0, clamp(-1, 0, 10));
    TEST_ASSERT_EQUAL_INT(10, clamp(11, 0, 10));

    /* extreme ints */
    TEST_ASSERT_EQUAL_INT(INT_MIN, clamp(INT_MIN, INT_MIN, INT_MAX));
    TEST_ASSERT_EQUAL_INT(INT_MAX, clamp(INT_MAX, INT_MIN, INT_MAX));
    TEST_ASSERT_EQUAL_INT(0, clamp(INT_MIN, 0, 10));
    TEST_ASSERT_EQUAL_INT(10, clamp(INT_MAX, 0, 10));
}
```

Strings

```c
#include <string.h>
#include <stddef.h>

/* Returns length, or (size_t)-1 on NULL; never walks past maxn */
size_t safe_strnlen(const char *s, size_t maxn) {
    size_t i;
    if (s == NULL) {
        return (size_t)-1;
    }
    for (i = 0; i < maxn; i++) {
        if (s[i] == '\0') {
            return i;
        }
    }
    return maxn;
}

void test_safe_strnlen_boundaries(void) {
    TEST_ASSERT_EQUAL_INT((int)(size_t)-1, (int)safe_strnlen(NULL, 10));
    TEST_ASSERT_EQUAL_INT(0, (int)safe_strnlen("", 10));
    TEST_ASSERT_EQUAL_INT(0, (int)safe_strnlen("abc", 0));
    TEST_ASSERT_EQUAL_INT(3, (int)safe_strnlen("abc", 3));
    TEST_ASSERT_EQUAL_INT(3, (int)safe_strnlen("abc", 100));
    TEST_ASSERT_EQUAL_INT(3, (int)safe_strnlen("abcdef", 3)); /* capped */
}
```

Boundary checklist for strings

- `NULL`
- empty `""`
- length 1
- exact buffer size
- longer than limit
- non-ASCII / embedded `\0` if relevant

Table-Driven Tests in C

Encode many cases in an array of structs. One loop runs them all — easy to extend.

```
/* file: test_leap_table.c */
#include <stdio.h>

int is_leap_year(int year) {
    return (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
}

struct leap_case {
    int year;
    int expected; /* 1 leap, 0 not */
    const char *note;
};

static const struct leap_case cases[] = {
    {2019, 0, "common"},
    {2020, 1, "div by 4"},
    {1900, 0, "century"},
    {2000, 1, "div by 400"},
    {1600, 1, "div by 400"},
    {2100, 0, "century"},
    {1,    0, "year 1"},
    {4,    1, "year 4"},
};

int main(void) {
    int failed = 0;
    size_t i;
    size_t n = sizeof cases / sizeof cases[0];

    for (i = 0; i < n; i++) {
        int got = is_leap_year(cases[i].year);
        if (got != cases[i].expected) {
            fprintf(stderr, "FAIL year=%d (%s): expected %d got %d\n",
                    cases[i].year, cases[i].note,
                    cases[i].expected, got);
            failed++;
        } else {
            printf("OK   year=%d (%s)\n", cases[i].year, cases[i].note);
        }
    }

    printf("%zu cases, %d failed\n", n, failed);
    return failed ? 1 : 0;
```

```
}
```

```
gcc -std=c17 -Wall -Wextra -o test_leap_table test_leap_table.c
./test_leap_table
```

Table-driven style also works for `clamp`, parsers, and error-code matrices.

Code Coverage with `gcov` / `lcov`

Coverage shows which lines ran during tests — not that logic is correct, but that untested paths exist.

Instrument and run

```
# Compile with coverage (debug-friendly)
gcc -std=c17 -Wall -Wextra -g --coverage \
  -o test_leap test_leap_unity.c

./test_leap

# Per-file summary
gcov test_leap_unity.c
# creates test_leap_unity.c.gcov - open in an editor

# Optional pretty HTML report (install lcov)
lcov --capture --directory . --output-file coverage.info
genhtml coverage.info --output-directory coverage_html
# open coverage_html/index.html
```

Tips

- Clean between runs: `rm -f *.gcda *.gcno *.gcov coverage.info`
- Link with `--coverage` (or `-lgcov` on some toolchains) when splitting objects
- Aim for high branch coverage on pure logic; 100% line coverage bug-free

Example `.gcov` line prefixes:

- `#####:` — never executed
- `1:` — executed once

- `branch  0 taken 0%` — branch never taken (needs another case)

Sanitizers: Address and Undefined Behavior

GCC/Clang sanitizers catch classes of bugs tests alone may miss.

AddressSanitizer (ASan)

Detects heap/stack buffer overflows, use-after-free, some leaks.

```c
/* file: bug_overflow.c */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(4 * sizeof(int));
    if (!p) {
        return 1;
    }
    p[4] = 42; /* one past the end - undefined behavior */
    printf("%d\n", p[0]);
    free(p);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address -o bug_overflow bug_overflow.c
./bug_overflow
# ASan prints stack of the bad write and the allocation site
```

UndefinedBehaviorSanitizer (UBSan)

```c
/* file: bug_ub.c */
#include <limits.h>
#include <stdio.h>

int main(void) {
    int x = INT_MAX;
    x = x + 1; /* signed overflow - UB */
    printf("%d\n", x);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=undefined -o bug_ub bug_ub.c
./bug_ub
```

Combined (common for tests)

```
gcc -std=c17 -Wall -Wextra -g \
  -fsanitize=address,undefined \
  -o test_leap test_leap_unity.c
./test_leap
```

CI idea: run the unit suite under ASan+UBSan on every commit.

ThreadSanitizer (`-fsanitize=thread`) is for races; requires pthreads and is slower.

Integration and System Testing (brief)

- **Integration**: link `module_a.o` + `module_b.o` and exercise the boundary API.
- **System**: run the finished CLI with shell scripts:

```
#!/usr/bin/env bash
set -euo pipefail
./myprog --input sample.txt > out.txt
diff -u expected.txt out.txt
```

- **Regression**: keep golden files under `tests/fixtures/` and fail the pipeline on diff.

Black-Box vs White-Box (quick map)

Approach	You know	Techniques
Black box	Spec only	Equivalence classes, boundaries, decision tables
White box	Source	Statement/branch/path coverage, data-flow
Gray box	Partial	Spec + known module structure

In unit tests you usually do **white/gray box**; acceptance tests stay closer to black box.

FIRST and AAA

- **FIRST**: Fast, Independent, Repeatable, Self-validating, Timely
- **AAA**: Arrange (setup) → Act (call SUT) → Assert (check)

```c
void test_clamp_above(void) {
    /* Arrange */
    int x = 99, lo = 0, hi = 10;
    /* Act */
    int got = clamp(x, lo, hi);
    /* Assert */
    TEST_ASSERT_EQUAL_INT(10, got);
}
```

Best Practices

1. Keep production code free of test-only `#ifdef` noise when possible — separate `*_test.c` files.
2. Prefer pure functions for hard logic (easy to table-test).
3. One clear reason to fail per test (or clearly named multi-assert tests).
4. Run tests and sanitizers before every push.
5. Treat flaky tests as bugs (timing, order, shared files).
6. Do not commit coverage HTML; generate it in CI artifacts.

Exercises

Exercise 1 — Custom harness for `clamp`

Implement `int clamp(int x, int lo, int hi)` and a `mini_test.h` suite covering inside, below, above, and `INT_MIN`/`INT_MAX`. Build with:

```c
gcc -std=c17 -Wall -Wextra -o test_clamp test_clamp.c
```

Exercise 2 — TDD `int max3(int a, int b, int c)`

Write failing tests first, implement, then refactor. Include cases where the max is in the first, second, and third argument, and all equal.

Exercise 3 — Table-driven string trim

Write `void trim_inplace(char *s)` that removes leading/trailing ASCII spaces. Drive tests from a table of `{input, expected}` pairs (use a writable buffer per case). Include empty string, all spaces, no spaces, and mixed.

Exercise 4 — Coverage gap hunt

Take `is_leap_year` and deliberately omit the year-2000 test. Run `gcov` and show which branch is under-tested. Add the missing case and re-check.

Exercise 5 — Sanitizer lab

Write a small program with a deliberate heap overflow. Confirm it "works" without sanitizers (undefined — may print garbage or crash). Rebuild with `-fsanitize=address` and capture the report. Fix the bug and re-run clean.

Exercise 6 — Boundary design

For a function `int parse_port(const char *s, int *out)` that accepts `"1"`…`"65535"`, list at least ten boundary/equivalence cases (NULL, `""`, `"0"`, `"1"`, `"65535"`, `"65536"`, leading zeros, non-digits, etc.) and implement tests.

Summary

Technique	When to use
`assert`	Tiny checks, throwaway scripts
Custom `TEST` macros	Portable unit suites with no deps
Unity-style runner	Familiar API / embeddable projects
TDD	New pure logic, clear contracts
Table-driven tests	Many similar cases
Boundary analysis	Ranges, sizes, parsers
`gcov`/`lcov`	Find untested lines/branches
ASan/UBSan	Memory safety and undefined behavior

Testing in C is mostly **discipline and tooling**, not language magic. A small harness plus tables, boundaries, coverage, and sanitizers will catch most defects before they leave your machine.

Debugging Techniques

Debugging is the systematic process of finding and fixing defects. On Linux, the core toolkit is **GDB**, **printf/logging macros**, **Valgrind Memcheck**, **strace/ltrace**, **core dumps**, and compiler sanitizers (see the testing chapter for ASan/UBSan).

Always compile with symbols when debugging:

```
gcc -std=c17 -Wall -Wextra -g -O0 -o program program.c
```

`-O0` keeps the mapping between source lines and machine code straightforward. Optimized builds (`-O2`) can reorder code and eliminate variables, which confuses stepping.

The Debugging Process

1. **Reproduce** — reliable steps beat intermittent luck
2. **Isolate** — smallest input / fewest modules that fail
3. **Hypothesize** — what would make the observed symptoms true?
4. **Observe** — debugger, logs, sanitizers, traces
5. **Fix** — change the root cause, not the symptom
6. **Verify** — re-run failing case + nearby regression tests

---

GDB: Session-Style Walkthroughs

GDB (GNU Debugger) inspects a running or crashed process: breakpoints, stepping, printing, stack traces, and watchpoints.

Program under study

```
/* file: buggy_sum.c */
#include <stdio.h>

int sum_range(int n) {
    int total = 0;
    int i;
    /* Off-by-one: should be i <= n for inclusive sum 1..n */
    for (i = 1; i < n; i++) {
        total += i;
    }
    return total;
}

int main(void) {
    int n = 5;
    int s = sum_range(n);
    printf("sum 1..%d = %d (expected 15)\n", n, s);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -O0 -o buggy_sum buggy_sum.c
gdb ./buggy_sum
```

Transcript 1 — break, run, print, next, continue

```
$ gdb ./buggy_sum
(gdb) break main
Breakpoint 1 at 0x11a9: file buggy_sum.c, line 14.
(gdb) run
Starting program: .../buggy_sum

Breakpoint 1, main () at buggy_sum.c:14
14          int n = 5;
(gdb) next
15          int s = sum_range(n);
(gdb) print n
$1 = 5
(gdb) step
sum_range (n=5) at buggy_sum.c:5
5           int total = 0;
(gdb) next
7           for (i = 1; i < n; i++) {
```

```
(gdb) print i
$2 = 0
(gdb) next
8               total += i;
(gdb) print i
$3 = 1
(gdb) print total
$4 = 0
(gdb) next
7           for (i = 1; i < n; i++) {
(gdb) print total
$5 = 1
(gdb) break buggy_sum.c:9
Breakpoint 2 at 0x1178: file buggy_sum.c, line 9.
(gdb) continue
Continuing.

Breakpoint 2, sum_range (n=5) at buggy_sum.c:9
9           return total;
(gdb) print total
$6 = 10
(gdb) print n
$7 = 5
(gdb) quit
```

Observation: for `n = 5`, loop runs `i = 1..4` so total is 10, not 15. Fix: `i <= n`.

Transcript 2 — backtrace on a crash

```
/* file: crash_null.c */
#include <stdio.h>

void boom(int *p) {
    *p = 42; /* segfault if p is NULL */
}

void middle(void) {
    int *p = NULL;
    boom(p);
}

int main(void) {
    middle();
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -O0 -o crash_null crash_null.c
gdb ./crash_null
```

```
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x0000555555555155 in boom (p=0x0) at crash_null.c:5
5           *p = 42;
(gdb) backtrace
#0  boom (p=0x0) at crash_null.c:5
#1  0x0000555555555170 in middle () at crash_null.c:10
#2  0x0000555555555185 in main () at crash_null.c:14
(gdb) frame 1
#1  middle () at crash_null.c:10
10          boom(p);
(gdb) print p
$1 = (int *) 0x0
(gdb) info locals
p = 0x0
(gdb) list
```

Commands to memorize

Command	Purpose
`break main` / `b file.c:42`	Set breakpoint
`break foo if x > 5`	Conditional breakpoint
`run` / `r`	Start program (args: `run arg1 arg2`)
`next` / `n`	Step over
`step` / `s`	Step into
`continue` / `c`	Resume
`print expr` / `p`	Evaluate expression
`backtrace` / `bt`	Call stack
`frame N`	Select stack frame
`info locals`	Locals in current frame
`list`	Show source
`quit`	Exit

Transcript 3 — watchpoints

```
/* file: watch_demo.c */
#include <stdio.h>

int main(void) {
    int counter = 0;
    int i;
    for (i = 0; i < 5; i++) {
```

```
        counter += i;
    }
    printf("%d\n", counter);
    return 0;
}
```

```
(gdb) break main
(gdb) run
(gdb) next
(gdb) watch counter
Hardware watchpoint 2: counter
(gdb) continue
Hardware watchpoint 2: counter

Old value = 0
New value = 1
main () at watch_demo.c:8
(gdb) continue
...
(gdb) delete 2
```

Use `watch *pointer` for heap locations after they are allocated. Software watchpoints are slower; hardware watchpoints are limited in count but efficient.

Handy GDB extras

```
(gdb) break 10 if x > 5
(gdb) info breakpoints
(gdb) delete 1
(gdb) set var i = 99
(gdb) finish          # run until current function returns
(gdb) until 40        # continue until line 40 in this frame
(gdb) disassemble main
(gdb) info registers
```

TUI mode (source + assembly panes): start with `gdb -tui ./program` or press `C-x a` inside GDB.

`printf` Debugging and `DEBUG_LOG` Macros

Print debugging is fast for I/O-heavy paths and remote systems without an interactive debugger. Make it **toggleable** and **location-aware**.

```
/* file: debug_log.h */
#ifndef DEBUG_LOG_H
#define DEBUG_LOG_H

#include <stdio.h>

/* Compile with -DDEBUG=1 to enable, or #define DEBUG 1 before include */
#ifndef DEBUG
#define DEBUG 0
#endif

#if DEBUG
#define DEBUG_LOG(fmt, ...)                                             \
    do {                                                                \
        fprintf(stderr, "[DEBUG] %s:%d:%s(): " fmt "\n",                \
                __FILE__, __LINE__, __func__, ##__VA_ARGS__);           \
    } while (0)
#else
#define DEBUG_LOG(fmt, ...)                                             \
    do { /* no-op */ } while (0)
#endif

/* Always-on diagnostic (errors, rare paths) */
#define ERROR_LOG(fmt, ...)                                             \
    do {                                                                \
        fprintf(stderr, "[ERROR] %s:%d:%s(): " fmt "\n",                \
                __FILE__, __LINE__, __func__, ##__VA_ARGS__);           \
    } while (0)

#endif
```

```
/* file: sum_debug.c */
#include "debug_log.h"

int sum_range(int n) {
    int total = 0;
    int i;
    DEBUG_LOG("enter n=%d", n);
    for (i = 1; i <= n; i++) {
        total += i;
        DEBUG_LOG("i=%d total=%d", i, total);
    }
    DEBUG_LOG("leave total=%d", total);
    return total;
}
```

```c
int main(void) {
    int s = sum_range(5);
    printf("sum=%d\n", s);
    return 0;
}
```

```
# Logs on
gcc -std=c17 -Wall -Wextra -DDEBUG=1 -o sum_debug sum_debug.c
./sum_debug

# Logs compiled out
gcc -std=c17 -Wall -Wextra -o sum_debug sum_debug.c
./sum_debug
```

Patterns

- Log **entry/exit** of suspect functions with key arguments
- Log **loop indices** only for small `n` or every Nth iteration
- Prefer `stderr` so stdout can still be piped/compared
- Never leave secrets (passwords, tokens) in permanent logs
- For binary data, log lengths and a few hex bytes, not megabytes

```c
/* Hex dump snippet */
static void dump_bytes(const unsigned char *p, size_t n) {
    size_t i;
    for (i = 0; i < n; i++) {
        fprintf(stderr, "%02x%s", p[i], ((i + 1) % 16 == 0) ? "\n" : " ");
    }
    if (n % 16) {
        fputc('\n', stderr);
    }
}
```

Rubber duck and binary-search debugging

- **Rubber duck**: explain each line aloud; contradictions surface quickly.
- **Binary search**: comment out half the work (or `git bisect`) until the bad change is isolated.

```
git bisect start
git bisect bad                # current broken revision
git bisect good v1.0          # known good tag
```

```
# build + test each checkout, then:
git bisect good   # or: git bisect bad
git bisect reset
```

Valgrind Memcheck: Intentional Leak and Reading the Report

Valgrind Memcheck detects invalid reads/writes, use of uninitialized values, double-free, and leaks.

Example program with a real leak and a bad read

```
/* file: leaky.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *make_greeting(const char *name) {
    size_t n = strlen(name);
    char *buf = malloc(n + 8); /* enough for "Hello, " + name + NUL roughly */
    if (buf == NULL) {
        return NULL;
    }
    /* Intentionally wrong size path for demo: we allocated n+8, use carefully */
    sprintf(buf, "Hello, %s", name);
    return buf; /* caller must free - we will "forget" */
}

int main(void) {
    char *msg = make_greeting("Ada");
    if (msg == NULL) {
        return 1;
    }
    printf("%s\n", msg);

    /* Invalid read demo (optional): read one past strlen */
    /* printf("past end byte maybe junk: %d\n", msg[strlen(msg) + 1]); */

    /* BUG: forgot free(msg) - definite leak */
    /* free(msg); */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -O0 -o leaky leaky.c
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all \
  --track-origins=yes ./leaky
```

How to read typical output

```
==12345== HEAP SUMMARY:
==12345==     in use at exit: 11 bytes in 1 blocks
==12345==   total heap usage: 1 allocs, 0 frees, 11 bytes allocated
==12345==
==12345== 11 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12345==    at 0x...: malloc (vg_replace_malloc.c:...)
==12345==    by 0x...: make_greeting (leaky.c:8)
==12345==    by 0x...: main (leaky.c:18)
```

Phrase	Meaning
definitely lost	No pointer to the block remains — real leak
indirectly lost	Lost because a root pointer was lost (e.g. list head)
possibly lost	Interior pointer only (suspicious)
still reachable	Pointers exist at exit (globals, caches) — may be intentional
Invalid read/write	Access outside allocated region or after free
Conditional jump depends on uninitialised value	Branch on garbage
Invalid free / Mismatched free	Double free or `free` vs `delete` style issues

Fix for the demo: `free(msg);` before `return 0`.

Common Memcheck flags

```
valgrind --leak-check=full --show-leak-kinds=all \
  --track-origins=yes --errors-for-leak-kinds=all \
  ./program args...
```

Run under the same `-g` build you use for GDB. Suppressions (`.supp` files) hide known third-party noise — use sparingly.

`strace` and `ltrace` (brief)

`strace` — system calls

Shows what the kernel sees: open, read, write, connect, etc.

```
# Trace a simple program
strace -o strace.log ./buggy_sum
# Follow forks (servers, shells)
strace -f -o strace.log ./server
# Only file-related calls
strace -e trace=file ./myprog
# Network-related
strace -e trace=network ./client
```

Example snippet when a file is missing:

```
openat(AT_FDCWD, "config.txt", O_RDONLY) = -1 ENOENT (No such file or directory)
```

That single line often beats minutes of guessing about path logic.

`ltrace` — library calls

```
ltrace -o ltrace.log ./leaky
# Focus on malloc family
ltrace -e malloc+free+realloc ./leaky
```

Useful when you care about libc (`malloc`, `printf`, `connect`) rather than raw syscalls. Availability varies by distro/package.

Core Dumps: Capture and Analyze

When a process dies with a fatal signal (e.g. `SIGSEGV`), the kernel can write a **core** image of memory for post-mortem GDB.

Enable dumps

```
# Current shell only
ulimit -c unlimited

# Check limit
ulimit -c
```

```
# On many systems, core pattern is controlled by:
cat /proc/sys/kernel/core_pattern
# Example patterns: core, core.%p, or systemd-coredump pipes
```

If `core_pattern` pipes to `systemd-coredump`, use:

```
coredumpctl list
coredumpctl gdb ./crash_null
# or: coredumpctl dump -o core.crash
```

Produce a core

```
gcc -std=c17 -Wall -Wextra -g -O0 -o crash_null crash_null.c
ulimit -c unlimited
./crash_null
# may create file "core" or "core.<pid>" depending on system
ls -l core*
```

Analyze with GDB

```
gdb ./crash_null core
# or: gdb ./crash_null core.12345
```

```
(gdb) bt
#0  boom (p=0x0) at crash_null.c:5
#1  middle () at crash_null.c:10
#2  main () at crash_null.c:14
(gdb) print p
$1 = (int *) 0x0
(gdb) list
```

Same skills as live debugging: `bt`, `frame`, `print`, `info locals`.

Tips

- The binary path and **build ID** should match the one that crashed
- Keep `-g` builds (or separate debug packages) for production post-mortems
- Never ship secrets in cores; cores can contain full memory

Common Failure Modes

Segmentation faults

Typical causes:

- Null or wild pointer dereference
- Buffer overflow past array/heap end
- Use after free
- Stack overflow (deep recursion)

Workflow: GDB `bt` → inspect pointer → Valgrind or ASan if memory corruption is subtle.

Logic errors

Program runs, wrong answer. Prefer:

- Unit tests with known golden values
- GDB watchpoints on intermediate results
- Temporary `DEBUG_LOG` of invariants

Memory leaks and corruption

Valgrind / ASan first; then fix ownership (who allocates, who frees).

Race conditions (threads)

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=thread -o races races.c -pthread
./races
```

Combine with careful locking; races are hard to reproduce with `printf` alone.

Static Analysis (quick)

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -c file.c
cppcheck --enable=all --inconclusive --std=c17 file.c
```

Static tools do not replace runtime debugging; they reduce the number of bugs that reach GDB.

Debugging Best Practices

1. **Reproduce first** — write a failing test or a minimal input file.
2. **One change at a time** — random edits hide the real fix.
3. **Trust tools more than memory** — print the actual value of `errno`, sizes, indices.
4. **Keep a short log** — hypothesis, experiment, result.
5. **Prefer `-g -O0` while hunting**, then re-test optimized builds.
6. **Use version control** — `git bisect` for regressions.
7. **Take breaks** — fatigue creates new bugs.

Exercises

Exercise 1 — GDB fix of `sum_range`

Use the buggy `sum_range` that uses `i < n`. Under GDB, break in the loop, print `i` and `total` for `n = 5`, confirm the off-by-one, fix the loop, and re-run outside GDB so the program prints `15`.

Exercise 2 — `DEBUG_LOG` toggle

Add `debug_log.h` to a small multi-function program. Build once with `-DDEBUG=1` and once without. Confirm the non-debug binary produces no debug lines and is smaller / quieter.

Exercise 3 — Valgrind leak

Write a program that allocates a linked list of 10 nodes and "forgets" to free them. Run Memcheck with `--leak-check=full`. Then free the list correctly and show a clean report (`All heap blocks were freed -- no leaks are possible`).

Exercise 4 — Invalid write

Allocate `malloc(4)` and write 8 bytes. Compare:

1. Normal run (may appear to work — UB)
2. `valgrind ./prog`
3. `gcc -fsanitize=address`

Describe which tool gave the clearest diagnosis.

Exercise 5 — Core dump

Enable `ulimit -c unlimited`, run `crash_null`, open the core in GDB, and paste (or retype) the `bt` output. Identify the null pointer in which frame.

Exercise 6 — `strace` detective

Write a program that tries to open three paths in order until one succeeds. Hide the working path among missing ones. Use `strace -e openat` (or `trace=file`) to determine which path was used without reading the source.

Summary

Tool	Best for
GDB	Breakpoints, stack traces, watchpoints, live state
DEBUG_LOG	Fast insight, remote/headless runs
Valgrind Memcheck	Leaks, invalid access, uninit values
ASan/UBSan	Fast memory/UB checks (see testing chapter)
strace	"Why did open/connect fail?"
ltrace	Library call sequences
Core + GDB	Post-crash forensics

Debugging is a skill of **observation under control**. Master GDB and Memcheck early; use logs and traces when interactivity is limited; always finish with a regression test so the same bug stays fixed.

Error Handling

Error handling is a critical aspect of robust software development. In C programming, where memory management and low-level operations are common, proper error handling becomes even more important. This chapter explores various error handling strategies and techniques in C.

Introduction to Error Handling

Error handling refers to the process of anticipating, detecting, and resolving programming, application, or communication errors. In C, error handling is primarily manual since the language does not have built-in exception handling mechanisms like some other languages.

Why Error Handling is Important

- **Program Stability**: Prevents crashes and unexpected termination
- **Data Integrity**: Ensures data consistency and prevents corruption
- **User Experience**: Provides meaningful feedback to users
- **Debugging**: Helps identify and resolve issues quickly
- **Security**: Prevents vulnerabilities that could be exploited

Error Handling Strategies in C

Return Codes

Return codes are the most common error handling mechanism in C. Functions return specific values to indicate success or failure.

Basic Return Code Pattern

```c
#include <stdio.h>
#include <stdlib.h>

// Function that returns 0 on success, non-zero on error
int divide(int a, int b, int *result) {
    if (b == 0) {
        return -1; // Error: division by zero
    }
```

```
    *result = a / b;
    return 0; // Success
}

int main() {
    int result;
    int error = divide(10, 2, &result);

    if (error != 0) {
        printf("Error occurred during division\n");
        return 1;
    }

    printf("Result: %d\n", result);
    return 0;
}
```

Enumerated Return Codes

Using enums for return codes makes the code more readable and maintainable.

```
#include <stdio.h>

typedef enum {
    SUCCESS = 0,
    ERROR_INVALID_INPUT,
    ERROR_DIVISION_BY_ZERO,
    ERROR_MEMORY_ALLOCATION,
    ERROR_FILE_NOT_FOUND
} ErrorCode;

ErrorCode safe_divide(int a, int b, int *result) {
    if (a < 0 || b < 0) {
        return ERROR_INVALID_INPUT;
    }

    if (b == 0) {
        return ERROR_DIVISION_BY_ZERO;
    }

    *result = a / b;
    return SUCCESS;
}

int main() {
    int result;
```

```
    ErrorCode error = safe_divide(10, 2, &result);

    switch (error) {
        case SUCCESS:
            printf("Result: %d\n", result);
            break;
        case ERROR_INVALID_INPUT:
            printf("Error: Invalid input values\n");
            break;
        case ERROR_DIVISION_BY_ZERO:
            printf("Error: Division by zero\n");
            break;
        default:
            printf("Unknown error occurred\n");
            break;
    }

    return error;
}
```

Global Error Variables

C provides global variables like `errno` for system-level error reporting.

Using errno

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main() {
    FILE *file = fopen("nonexistent.txt", "r");

    if (file == NULL) {
        printf("Error opening file: %s\n", strerror(errno));
        return 1;
    }

    fclose(file);
    return 0;
}
```

Custom Error Variables

For application-specific errors, you can create your own global error variables.

```
#include <stdio.h>
#include <string.h>

// Global error variable
static int app_errno = 0;

// Error codes
#define APP_SUCCESS 0
#define APP_ERROR_INVALID_DATA 1
#define APP_ERROR_BUFFER_OVERFLOW 2
#define APP_ERROR_NETWORK_FAILURE 3

// Function to get error description
const char* app_strerror(int error_code) {
    switch (error_code) {
        case APP_SUCCESS:
            return "Success";
        case APP_ERROR_INVALID_DATA:
            return "Invalid data";
        case APP_ERROR_BUFFER_OVERFLOW:
            return "Buffer overflow";
        case APP_ERROR_NETWORK_FAILURE:
            return "Network failure";
        default:
            return "Unknown error";
    }
}

// Function that sets app_errno
int process_data(const char *data) {
    if (data == NULL) {
        app_errno = APP_ERROR_INVALID_DATA;
        return -1;
    }

    if (strlen(data) > 100) {
        app_errno = APP_ERROR_BUFFER_OVERFLOW;
        return -1;
    }

    // Process data...
    app_errno = APP_SUCCESS;
    return 0;
```

```
}

int main() {
    int result = process_data(NULL);

    if (result != 0) {
        printf("Error: %s\n", app_strerror(app_errno));
        return 1;
    }

    printf("Data processed successfully\n");
    return 0;
}
```

Advanced Error Handling Techniques

Setjmp and Longjmp

C provides `setjmp` and `longjmp` for non-local jumps, which can be used for exception-like behavior.

```
#include <stdio.h>
#include <stdlib.h>
#include <setjmp.h>
#include <string.h>

// Jump buffer for error handling
static jmp_buf error_buffer;

// Custom exception-like function
void throw_error(const char *message) {
    printf("Error: %s\n", message);
    longjmp(error_buffer, 1);
}

// Function that might fail
void risky_function(const char *data) {
    if (data == NULL) {
        throw_error("Null pointer passed to risky_function");
    }

    if (strlen(data) == 0) {
        throw_error("Empty string passed to risky_function");
    }
```

```
    printf("Processing data: %s\n", data);
}

int main() {
    // Set jump point
    if (setjmp(error_buffer) == 0) {
        // Normal execution path
        printf("Calling risky_function...\n");
        risky_function("Hello, World!");
        risky_function(NULL); // This will cause an error
    } else {
        // Error handling path
        printf("Error handled, continuing execution...\n");
    }

    printf("Program continues...\n");
    return 0;
}
```

Resource Acquisition Is Initialization (RAII)

While not native to C, RAII principles can be implemented using goto statements for cleanup.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *name;
    int *data;
    FILE *file;
} ResourceContainer;

int process_resources() {
    ResourceContainer container = {0};
    int result = -1; // Assume failure

    // Allocate resources
    container.name = malloc(100);
    if (container.name == NULL) {
        fprintf(stderr, "Failed to allocate name buffer\n");
        goto cleanup;
    }

    container.data = malloc(1000 * sizeof(int));
    if (container.data == NULL) {
```

```
        fprintf(stderr, "Failed to allocate data array\n");
        goto cleanup;
    }

    container.file = fopen("data.txt", "w");
    if (container.file == NULL) {
        fprintf(stderr, "Failed to open file\n");
        goto cleanup;
    }

    // Use resources
    strcpy(container.name, "Test Data");
    for (int i = 0; i < 1000; i++) {
        container.data[i] = i * i;
        fprintf(container.file, "%d\n", container.data[i]);
    }

    result = 0; // Success

cleanup:
    // Clean up resources in reverse order
    if (container.file != NULL) {
        fclose(container.file);
    }

    if (container.data != NULL) {
        free(container.data);
    }

    if (container.name != NULL) {
        free(container.name);
    }

    return result;
}

int main() {
    if (process_resources() != 0) {
        printf("Error occurred during resource processing\n");
        return 1;
    }

    printf("Resources processed successfully\n");
    return 0;
}
```

Error Handling Best Practices

Check All Return Values

Always check return values from functions, especially system calls and library functions.

```
#include <stdio.h>
#include <stdlib.h>

int safe_file_operations() {
    FILE *file = fopen("example.txt", "w");
    if (file == NULL) {
        perror("Failed to open file");
        return -1;
    }

    if (fprintf(file, "Hello, World!\n") < 0) {
        perror("Failed to write to file");
        fclose(file);
        return -1;
    }

    if (fclose(file) != 0) {
        perror("Failed to close file");
        return -1;
    }

    return 0;
}
```

Use Assertions for Debugging

Assertions help catch programming errors during development.

```
#include <stdio.h>
#include <assert.h>

int calculate_average(int *array, int size) {
    // Precondition checks
    assert(array != NULL);
    assert(size > 0);

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += array[i];
    }
```

```
    return sum / size;
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int average = calculate_average(numbers, 5);
    printf("Average: %d\n", average);

    // These would trigger assertions:
    // calculate_average(NULL, 5);  // Assertion failed
    // calculate_average(numbers, 0);  // Assertion failed

    return 0;
}
```

Provide Meaningful Error Messages

Error messages should be clear and informative to help with debugging.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef enum {
    ERR_SUCCESS = 0,
    ERR_NULL_POINTER,
    ERR_INVALID_SIZE,
    ERR_MEMORY_ALLOCATION,
    ERR_FILE_OPERATION
} ErrorType;

const char* get_error_message(ErrorType error, const char *context) {
    static char message[256];

    switch (error) {
        case ERR_SUCCESS:
            snprintf(message, sizeof(message), "Success in %s", context);
            break;
        case ERR_NULL_POINTER:
            snprintf(message, sizeof(message), "Null pointer error in %s", context);
            break;
        case ERR_INVALID_SIZE:
            snprintf(message, sizeof(message), "Invalid size parameter in %s", context);
            break;
        case ERR_MEMORY_ALLOCATION:
```

```
            snprintf(message, sizeof(message), "Memory allocation failed in %s", context);
            break;
        case ERR_FILE_OPERATION:
            snprintf(message, sizeof(message), "File operation failed in %s", context);
            break;
        default:
            snprintf(message, sizeof(message), "Unknown error in %s", context);
            break;
    }

    return message;
}

int safe_memory_copy(char *dest, const char *src, size_t size) {
    if (dest == NULL) {
        fprintf(stderr, "%s\n", get_error_message(ERR_NULL_POINTER, "safe_memory_copy"));
        return -1;
    }

    if (src == NULL) {
        fprintf(stderr, "%s\n", get_error_message(ERR_NULL_POINTER, "safe_memory_copy"));
        return -1;
    }

    if (size == 0) {
        fprintf(stderr, "%s\n", get_error_message(ERR_INVALID_SIZE, "safe_memory_copy"));
        return -1;
    }

    strncpy(dest, src, size - 1);
    dest[size - 1] = '\0';

    return 0;
}
```

Implement Error Logging

Logging errors helps with debugging and monitoring in production environments.

```
#include <stdio.h>
#include <time.h>
#include <stdarg.h>

typedef enum {
    LOG_DEBUG,
    LOG_INFO,
```

```
    LOG_WARNING,
    LOG_ERROR,
    LOG_CRITICAL
} LogLevel;

void log_message(LogLevel level, const char *format, ...) {
    const char *level_strings[] = {
        "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"
    };

    // Get current time
    time_t now = time(NULL);
    struct tm *tm_info = localtime(&now);
    char timestamp[20];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", tm_info);

    // Print log level and timestamp
    printf("[%s] [%s] ", timestamp, level_strings[level]);

    // Print formatted message
    va_list args;
    va_start(args, format);
    vprintf(format, args);
    va_end(args);

    printf("\n");
}

int divide_with_logging(int a, int b, int *result) {
    log_message(LOG_DEBUG, "divide_with_logging called with a=%d, b=%d", a, b);

    if (b == 0) {
        log_message(LOG_ERROR, "Division by zero attempted");
        return -1;
    }

    *result = a / b;
    log_message(LOG_INFO, "Division successful: %d / %d = %d", a, b, *result);
    return 0;
}
```

Error Handling in Different Contexts

File I/O Error Handling

File operations require careful error handling due to various failure points.

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    double value;
} Record;

int read_records(const char *filename, Record *records, int max_records, int *record_count)
    FILE *file = fopen(filename, "rb");
    if (file == NULL) {
        fprintf(stderr, "Error opening file %s: %s\n", filename, strerror(errno));
        return -1;
    }

    *record_count = 0;
    Record temp;

    while (*record_count < max_records) {
        size_t elements_read = fread(&temp, sizeof(Record), 1, file);
        if (elements_read == 1) {
            records[*record_count] = temp;
            (*record_count)++;
        } else {
            if (feof(file)) {
                // End of file reached normally
                break;
            } else if (ferror(file)) {
                // Error occurred
                fprintf(stderr, "Error reading from file: %s\n", strerror(errno));
                fclose(file);
                return -1;
            }
        }
    }

    if (fclose(file) != 0) {
        fprintf(stderr, "Error closing file: %s\n", strerror(errno));
        return -1;
    }

    return 0;
}
```

Memory Management Error Handling

Proper memory management is crucial in C to prevent leaks and corruption.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *data;
    size_t size;
    size_t capacity;
} DynamicBuffer;

DynamicBuffer* create_buffer(size_t initial_capacity) {
    DynamicBuffer *buffer = malloc(sizeof(DynamicBuffer));
    if (buffer == NULL) {
        fprintf(stderr, "Failed to allocate buffer structure\n");
        return NULL;
    }

    buffer->data = malloc(initial_capacity);
    if (buffer->data == NULL) {
        fprintf(stderr, "Failed to allocate buffer data\n");
        free(buffer);
        return NULL;
    }

    buffer->size = 0;
    buffer->capacity = initial_capacity;
    return buffer;
}

int append_to_buffer(DynamicBuffer *buffer, const char *data, size_t data_size) {
    if (buffer == NULL || data == NULL) {
        return -1;
    }

    // Check if we need to resize
    if (buffer->size + data_size > buffer->capacity) {
        size_t new_capacity = buffer->capacity * 2;
        while (new_capacity < buffer->size + data_size) {
            new_capacity *= 2;
        }

        char *new_data = realloc(buffer->data, new_capacity);
        if (new_data == NULL) {
```

```
            fprintf(stderr, "Failed to resize buffer\n");
            return -1;
        }

        buffer->data = new_data;
        buffer->capacity = new_capacity;
    }

    // Copy data
    memcpy(buffer->data + buffer->size, data, data_size);
    buffer->size += data_size;

    return 0;
}

void destroy_buffer(DynamicBuffer *buffer) {
    if (buffer != NULL) {
        free(buffer->data);
        free(buffer);
    }
}
```

Conclusion

Error handling is a fundamental aspect of writing robust C programs. By using appropriate error handling strategies, checking return values, providing meaningful error messages, and implementing proper cleanup procedures, developers can create more reliable and maintainable software. The key is to anticipate potential failure points and handle them gracefully, ensuring that programs can recover from errors or fail safely when recovery is not possible.

Code Quality

Code quality is a measure of how well software code meets specified requirements and satisfies user needs. High-quality code is not only functional but also maintainable, readable, and efficient. This chapter explores various aspects of code quality in C programming and best practices to achieve it — with **complete programs** you can compile under a strict warning set.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o prog prog.c
```

Treat `-Werror` as a quality gate once the code is past the first sketch.

Introduction to Code Quality

Code quality encompasses multiple dimensions that contribute to the overall excellence of software. It's not just about whether the code works, but also about how well it works, how easy it is to maintain, and how reliably it performs over time.

Dimensions of Code Quality

1. **Correctness**: The code produces the expected results
2. **Reliability**: The code performs consistently under various conditions
3. **Efficiency**: The code uses resources optimally
4. **Maintainability**: The code is easy to modify and extend
5. **Readability**: The code is easy to understand
6. **Testability**: The code can be easily tested
7. **Portability**: The code works across different platforms
8. **Security**: The code is resistant to vulnerabilities

Coding Standards and Style Guidelines

Consistent coding standards improve code readability and maintainability. They provide a common framework for developers to follow.

Naming Conventions

```
// Good naming conventions
int user_age;                    // Descriptive variable names
double calculate_circle_area(double radius);  // Descriptive function names
const int MAX_BUFFER_SIZE = 1024;  // Constants in uppercase

// Avoid cryptic names
int x;          // What does x represent?
int f(int a);   // What does f do?
```

Code Formatting

Consistent indentation and spacing improve code readability:

```
// Well-formatted code
int calculate_sum(int array[], int size) {
    int sum = 0;

    for (int i = 0; i < size; i++) {
        sum += array[i];
    }

    return sum;
}

// Poorly formatted code (avoid this)
int calculate_sum(int array[],int size){
int sum=0;
for(int i=0;i<size;i++){
sum+=array[i];}
return sum;}
```

Commenting Best Practices

Comments should explain the "why" rather than the "what":

```
// Good comments - explain the reasoning
void initialize_buffer(Buffer *buf) {
    // Initialize to zero to prevent undefined behavior
    buf->size = 0;
    buf->capacity = INITIAL_CAPACITY;

    // Allocate memory with extra space to reduce reallocations
```

```
    buf->data = malloc(buf->capacity * sizeof(int));
}

// Avoid redundant comments
int x = 5;  // Set x to 5 (obvious from code)
```

Code Review Practices

Code reviews are essential for maintaining code quality and sharing knowledge among team members.

Benefits of Code Reviews

- **Bug Detection**: Catch defects before they reach production
- **Knowledge Sharing**: Spread expertise across the team
- **Consistency**: Ensure adherence to coding standards
- **Mentoring**: Help junior developers improve
- **Design Feedback**: Get input on architectural decisions

Code Review Checklist

1. **Functionality**: Does the code meet requirements?
2. **Correctness**: Are there any logical errors?
3. **Readability**: Is the code easy to understand?
4. **Maintainability**: Is the code easy to modify?
5. **Performance**: Are there any efficiency concerns?
6. **Security**: Are there potential vulnerabilities?
7. **Standards**: Does the code follow established guidelines?

Example Code Review

```
// Before review
int process(int*a,int b){int c=0;for(int d=0;d<b;d++){c+=a[d];}return c;}

// After review
/**
 * Calculate the sum of integers in an array.
 *
 * @param array Array of integers to sum
 * @param size Number of elements in the array
 * @return Sum of all elements in the array
 */
int calculate_array_sum(int *array, int size) {
```

```
    // Validate input parameters
    if (array == NULL || size < 0) {
        return -1; // Error case
    }

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += array[i];
    }

    return sum;
}
```

Static Analysis Tools

Static analysis tools examine source code without executing it to find potential issues.

GCC with Warnings

GCC provides extensive warning options to catch common issues:

```
# Compile with comprehensive warnings
gcc -Wall -Wextra -Werror -pedantic -std=c99 program.c

# Explanation of flags:
# -Wall: Enable most warning messages
# -Wextra: Enable additional warnings
# -Werror: Treat warnings as errors
# -pedantic: Issue warnings for non-standard C
# -std=c99: Specify C standard
```

Cppcheck

Cppcheck is a static analysis tool specifically designed for C/C++:

```
# Basic usage
cppcheck program.c

# More thorough analysis
cppcheck --enable=all --inconclusive program.c

# Check specific directories
cppcheck --enable=all src/
```

Clang Static Analyzer

The Clang Static Analyzer detects bugs and potential issues:

```
# Analyze with Clang
clang --analyze program.c

# Generate HTML reports
scan-build make
```

Code Formatting and Style Tools

Automated tools help maintain consistent code style across projects.

Clang-Format

Clang-Format automatically formats C code according to specified rules:

```
# Format a single file
clang-format -i program.c

# Format multiple files
find . -name "*.c" -o -name "*.h" | xargs clang-format -i

# Use a specific style
clang-format -style=Google -i program.c
```

AStyle (Artistic Style)

AStyle is another popular code formatter:

```
# Format with Allman style
astyle --style=allman *.c *.h

# Format with K&R style
astyle --style=kr *.c *.h
```

Test Coverage Analysis

Test coverage measures how much of your code is exercised by tests.

GCC with Coverage

GCC can generate coverage information:

```
# Compile with coverage instrumentation
gcc -fprofile-arcs -ftest-coverage -o program program.c

# Run the program
./program

# Generate coverage report
gcov program.c

# Generate HTML report with lcov
lcov --capture --directory . --output-file coverage.info
genhtml coverage.info --output-directory coverage_report
```

Coverage Metrics

1. **Statement Coverage**: Percentage of statements executed
2. **Branch Coverage**: Percentage of branches taken
3. **Function Coverage**: Percentage of functions called
4. **Line Coverage**: Percentage of lines executed

Performance Analysis

Performance analysis helps identify bottlenecks and optimize code.

Profiling with gprof

gprof provides function-level profiling information:

```
# Compile with profiling enabled
gcc -pg -o program program.c

# Run the program
./program

# Generate profiling report
gprof program gmon.out > analysis.txt
```

Profiling with perf

perf is a powerful Linux profiling tool:

```
# Profile a program
perf record ./program

# Analyze results
perf report

# View call graph
perf record -g ./program
perf report -g
```

Security Best Practices

Security should be considered throughout the development process.

Input Validation

Always validate input to prevent buffer overflows and injection attacks:

```
#include <stdio.h>
#include <string.h>

// Unsafe function (vulnerable to buffer overflow)
void unsafe_copy(char *dest, char *src) {
    strcpy(dest, src);  // Dangerous!
}

// Safe function with bounds checking
int safe_copy(char *dest, size_t dest_size, const char *src) {
    if (dest == NULL || src == NULL || dest_size == 0) {
        return -1; // Error
    }

    size_t src_len = strlen(src);
    if (src_len >= dest_size) {
        return -1; // Buffer too small
    }

    strcpy(dest, src);
    return 0; // Success
}
```

Memory Safety

Proper memory management prevents common security vulnerabilities:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Secure memory handling
char* duplicate_string(const char *source) {
    if (source == NULL) {
        return NULL;
    }

    size_t len = strlen(source);
    char *copy = malloc(len + 1);

    if (copy == NULL) {
        return NULL; // Allocation failed
    }

    strcpy(copy, source);
    return copy;
}

void free_string(char *str) {
    if (str != NULL) {
        free(str);
        str = NULL; // Prevent dangling pointer (local copy only)
    }
}
```

Secure Coding Practices

1. **Use Safe Functions**: Prefer `strncpy` over `strcpy`, `snprintf` over `sprintf`
2. **Validate Pointers**: Always check for NULL pointers
3. **Bounds Checking**: Verify array indices and buffer sizes
4. **Integer Overflow**: Check for arithmetic overflow
5. **Format Strings**: Use constant format strings with printf/scanf

Documentation and Code Comments

Good documentation is essential for code quality and maintainability.

Self-Documenting Code

Write code that is easy to understand without excessive comments:

```
// Self-documenting approach
typedef enum {
    USER_STATUS_ACTIVE,
    USER_STATUS_INACTIVE,
    USER_STATUS_SUSPENDED
} UserStatus;

typedef struct {
    char username[50];
    UserStatus status;
    time_t last_login;
} User;

int is_user_active(const User *user) {
    return user != NULL && user->status == USER_STATUS_ACTIVE;
}

// Avoid unclear code
int check_user(int a, int b) {  // What do a and b represent?
    return a == 1 && b > 0;     // What does this check mean?
}
```

API Documentation

Document public APIs with clear descriptions and examples:

```
/**
 * @brief Calculate the factorial of a non-negative integer.
 *
 * This function calculates the factorial of a given non-negative integer
 * using an iterative approach. The factorial of n is the product of all
 * positive integers less than or equal to n.
 *
 * @param n The non-negative integer to calculate factorial for
 * @return The factorial of n, or -1 if n is negative
 *
 * @note This function can overflow for large values of n (n > 20)
 *
 * Example usage:
 * @code
 * int result = factorial(5);  // Returns 120
 * @endcode
```

```
 */
int factorial(int n) {
    if (n < 0) {
        return -1; // Error case
    }

    if (n == 0 || n == 1) {
        return 1;
    }

    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }

    return result;
}
```

Refactoring and Code Improvement

Regular refactoring improves code quality over time.

Code Smells to Avoid

1. **Long Functions**: Functions that are too long and complex
2. **Duplicated Code**: Same or similar code in multiple places
3. **Complex Conditionals**: Overly complex if/else statements
4. **Magic Numbers**: Unexplained numeric constants in code
5. **Inappropriate Intimacy**: Functions that access internal data directly

Refactoring Example

```
// Before refactoring - complex and hard to understand
double calculate_discount(double price, int customer_type, int quantity) {
    if (customer_type == 1) {
        if (quantity > 100) {
            return price * 0.2;
        } else if (quantity > 50) {
            return price * 0.15;
        } else {
            return price * 0.1;
        }
    } else if (customer_type == 2) {
```

```
        if (quantity > 100) {
            return price * 0.3;
        } else if (quantity > 50) {
            return price * 0.25;
        } else {
            return price * 0.2;
        }
    } else {
        if (quantity > 100) {
            return price * 0.1;
        } else if (quantity > 50) {
            return price * 0.05;
        } else {
            return 0;
        }
    }
}

// After refactoring - clean and maintainable
typedef enum {
    CUSTOMER_REGULAR,
    CUSTOMER_PREMIUM,
    CUSTOMER_VIP
} CustomerType;

typedef struct {
    double threshold_100;
    double threshold_50;
    double base_rate;
} DiscountRates;

static const DiscountRates DISCOUNT_RATES[] = {
    [CUSTOMER_REGULAR] = {0.2, 0.15, 0.1},
    [CUSTOMER_PREMIUM] = {0.3, 0.25, 0.2},
    [CUSTOMER_VIP] = {0.1, 0.05, 0.0}
};

double calculate_discount(double price, CustomerType customer_type, int quantity) {
    if (customer_type < 0 || customer_type >= sizeof(DISCOUNT_RATES)) {
        return 0; // Invalid customer type
    }

    const DiscountRates *rates = &DISCOUNT_RATES[customer_type];

    if (quantity > 100) {
        return price * rates->threshold_100;
```

```
    } else if (quantity > 50) {
        return price * rates->threshold_50;
    } else {
        return price * rates->base_rate;
    }
}
```

Full Programs: Quality Gates You Can Run Locally

Program 1 — Strict-warning "clean" module (`clamp_lib`)

A small pure function plus a self-test binary. Clean under `-Werror`.

```
/* file: clamp.c */
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

/* Returns x clamped to [lo, hi]. If lo > hi, returns lo (documented choice). */
int clamp(int x, int lo, int hi) {
    if (lo > hi) {
        return lo;
    }
    if (x < lo) {
        return lo;
    }
    if (x > hi) {
        return hi;
    }
    return x;
}

#ifdef CLAMP_MAIN
int main(void) {
    struct {
        int x, lo, hi, want;
    } cases[] = {
        {5, 0, 10, 5},
        {-1, 0, 10, 0},
        {99, 0, 10, 10},
        {0, 0, 0, 0},
        {INT_MAX, 0, 10, 10},
        {INT_MIN, -10, 10, -10},
    };
    size_t i;
```

```
    int failed = 0;

    for (i = 0; i < sizeof cases / sizeof cases[0]; i++) {
        int got = clamp(cases[i].x, cases[i].lo, cases[i].hi);
        if (got != cases[i].want) {
            fprintf(stderr, "case %zu: clamp(%d,%d,%d)=%d want %d\n",
                    i, cases[i].x, cases[i].lo, cases[i].hi, got, cases[i].want);
            failed = 1;
        }
    }
    if (failed) {
        return EXIT_FAILURE;
    }
    puts("clamp: all cases passed");
    return EXIT_SUCCESS;
}
#endif
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -DCLAMP_MAIN -o clamp_test clamp.c
./clamp_test
```

Program 2 — Warning archaeology (`smell.c` → `clean.c`)

Start with a deliberately smelly program, record warnings, then fix.

```
/* file: smell.c - DO NOT ship; for warning lab only */
#include <stdio.h>
#include <string.h>

int helper();  /* no prototype details */

int main() {
    int unused;
    char buf[8];
    char *p = 0;

    strcpy(buf, "toolongforbuffer");  /* overflow risk */
    if (p)
        printf("%s\n", p);
    printf("%d\n", helper(1, 2));
    return 0;
}

int helper(int a, int b) {
    return a + b;
```

```
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -c smell.c 2> smell_warnings.txt
cat smell_warnings.txt
```

Clean rewrite:

```
/* file: clean.c */
#include <stdio.h>
#include <string.h>

static int helper(int a, int b);

int main(void) {
    char buf[8];

    /* bounded copy + explicit truncation detection */
    if (snprintf(buf, sizeof buf, "%s", "toolongforbuffer") >= (int)sizeof buf) {
        fprintf(stderr, "warning: truncated\n");
    }
    printf("buf=\"%s\"\n", buf);
    printf("sum=%d\n", helper(1, 2));
    return 0;
}

static int helper(int a, int b) {
    return a + b;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o clean clean.c
./clean
```

Program 3 — Complexity reduction: nested ifs → early return

```
/* file: access.c */
#include <stdio.h>
#include <stdbool.h>

struct User {
    bool active;
    bool banned;
    int age;
    int role;  /* 0 guest, 1 user, 2 admin */
};
```

```
/* Opaque quality: one exit path per failure reason, easy to test. */
const char *can_edit(const struct User *u) {
    if (u == NULL) {
        return "null user";
    }
    if (!u->active) {
        return "inactive";
    }
    if (u->banned) {
        return "banned";
    }
    if (u->age < 13) {
        return "underage";
    }
    if (u->role < 1) {
        return "guest cannot edit";
    }
    return NULL;  /* allowed */
}

int main(void) {
    struct User samples[] = {
        {true, false, 20, 1},
        {false, false, 20, 1},
        {true, true, 20, 2},
        {true, false, 10, 1},
        {true, false, 25, 0},
    };
    size_t i;

    for (i = 0; i < sizeof samples / sizeof samples[0]; i++) {
        const char *err = can_edit(&samples[i]);
        if (err == NULL) {
            printf("sample %zu: OK\n", i);
        } else {
            printf("sample %zu: DENY (%s)\n", i, err);
        }
    }
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o access access.c
./access
```

Program 4 — Static assertions and size contracts

```
/* file: layout_check.c */
#include <stdio.h>
#include <stddef.h>
#include <stdint.h>
#include <assert.h>

struct PacketHdr {
    uint16_t type;
    uint16_t len;
    uint32_t id;
};

/* Catch ABI surprises at compile time */
_Static_assert(sizeof(struct PacketHdr) == 8, "PacketHdr must be 8 bytes");
_Static_assert(offsetof(struct PacketHdr, id) == 4, "id offset");

int main(void) {
    struct PacketHdr h = {1, 16, 42};
    printf("type=%u len=%u id=%u size=%zu\n",
           h.type, h.len, h.id, sizeof h);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o layout_check layout_check.c
./layout_check
```

Program 5 — Local quality script (`check.sh` + target)

```
/* file: avg.c */
#include <stdio.h>
#include <stdlib.h>

double average(const int *a, size_t n) {
    size_t i;
    long sum = 0;

    if (a == NULL || n == 0) {
        return 0.0;
    }
    for (i = 0; i < n; i++) {
        sum += a[i];
    }
```

```
    return (double)sum / (double)n;
}

int main(void) {
    int v[] = {1, 2, 3, 4};
    printf("%.2f\n", average(v, 4));
    return 0;
}
```

```
# file: check.sh
#!/usr/bin/env bash
set -euo pipefail
CFLAGS=(-std=c17 -Wall -Wextra -Wpedantic -Werror -g)
gcc "${CFLAGS[@]}" -o avg avg.c
./avg
# optional tools if installed:
if command -v cppcheck >/dev/null; then
  cppcheck --enable=warning,style --error-exitcode=1 avg.c
fi
if command -v valgrind >/dev/null; then
  valgrind --error-exitcode=1 --leak-check=full ./avg
fi
echo "quality gate: OK"
```

```
chmod +x check.sh
./check.sh
```

Program 6 — Review checklist applied to a PR-sized change

Before you merge any non-trivial C change, walk this list (paste into PR template if you want):

```
[ ] Builds with -std=c17 -Wall -Wextra (and -Werror in CI)
[ ] No new compiler warnings
[ ] Every public function documents ownership (who frees?)
[ ] Error paths free / close what success paths free / close
[ ] No unbounded strcpy/sprintf/gets
[ ] Integer sizes / casts reviewed for truncation
[ ] Tests or a tiny main demo for the changed logic
[ ] Sanitizer or Valgrind run if memory/pointers touched
```

Example “before” (fails several checks):

```
char *load(const char *path) {
    FILE *fp = fopen(path, "r");
    char *buf = malloc(1024);
```

```
    fread(buf, 1, 1024, fp);  /* no checks, no NUL, leak on error */
    return buf;
}
```

Example “after”:

```
/* file: load_text.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

/* Caller frees the returned buffer. Returns NULL on error. */
char *load_text(const char *path, size_t max_n) {
    FILE *fp;
    char *buf;
    size_t n;

    if (path == NULL || max_n == 0) {
        return NULL;
    }
    fp = fopen(path, "rb");
    if (fp == NULL) {
        return NULL;
    }
    buf = malloc(max_n + 1);
    if (buf == NULL) {
        fclose(fp);
        return NULL;
    }
    n = fread(buf, 1, max_n, fp);
    if (ferror(fp)) {
        free(buf);
        fclose(fp);
        return NULL;
    }
    buf[n] = '\0';
    fclose(fp);
    return buf;
}

int main(int argc, char *argv[]) {
    char *s;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <file>\n", argv[0]);
        return EXIT_FAILURE;
```

```
    }
    s = load_text(argv[1], 4096);
    if (s == NULL) {
        fprintf(stderr, "load failed: %s\n", strerror(errno));
        return EXIT_FAILURE;
    }
    fputs(s, stdout);
    free(s);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o load_text load_text.c
echo 'hello quality' > t.txt
./load_text t.txt
```

Continuous Integration and Quality Gates

Continuous Integration (CI) helps maintain code quality by automatically testing and validating changes.

CI Pipeline Components

1. **Automated Builds**: Compile code on every change
2. **Static Analysis**: Run code quality tools automatically
3. **Unit Tests**: Execute test suite on every change
4. **Code Coverage**: Measure test coverage
5. **Security Scans**: Check for vulnerabilities
6. **Deployment**: Automatically deploy passing builds

Example CI Configuration

```
# .github/workflows/ci.yml
name: C CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
```

```yaml
    - uses: actions/checkout@v4

    - name: Install dependencies
      run: |
        sudo apt-get update
        sudo apt-get install -y gcc valgrind cppcheck

    - name: Compile with warnings as errors
      run: gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o program program.c

    - name: Static analysis
      run: cppcheck --enable=warning,style --error-exitcode=1 .

    - name: Run tests
      run: ./run_tests.sh

    - name: Memory check
      run: valgrind --error-exitcode=1 --leak-check=full ./program
```

Local gate CI gate

If it does not pass `check.sh` on your laptop, it should not pass in CI. Keep the same flags in both places.

Exercises

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o exN exN.c
```

1. **Warning farm** — Write a 20-line file that triggers at least four distinct `-Wall -Wextra` diagnostics. Fix them one by one until `-Werror` is clean. Keep the smelly version in a comment.
2. **`clamp` table** — Add five more edge cases to `clamp_test` (including `lo > hi`) and make the policy explicit in a comment.
3. **Refactor nest** — Take a function with three nested `if`s from your own code (or invent one) and rewrite with early returns. Confirm behavior with a small table of cases.
4. **`_Static_assert`** — Define a struct with intentional padding; assert `sizeof` and an `offsetof`. Change a field type and watch the assert fire.
5. **`load_text` growth** — Extend `load_text` to grow with `realloc` until EOF (cap at e.g. 1 MiB). Prove no leaks under Valgrind or ASan.

6. **`check.sh` for multi-file** — Point the script at `main.c` + `util.c`, fail the script if any command fails, and add an optional sanitizer build.
7. **Review role-play** — Review `smell.c` as if it were a PR. Write five bullet comments a reviewer should leave.
8. **Complexity budget** — Count decision points (`if`/`case`/`&&`/`||`/`for`/`while`) in one function; if over 10, split it.

Conclusion

Code quality is not a one-time achievement but an ongoing commitment. By following established coding standards, conducting regular code reviews, using automated tools, and continuously refactoring, developers can maintain high-quality codebases that are robust, maintainable, and secure.

Practical baseline for this book:

Gate	Command / habit
Warnings	`-Wall -Wextra -Wpedantic` (+ `-Werror` in CI)
Memory	ASan/UBSan or Valgrind on pointer-heavy changes
Structure	Early returns, clear ownership, small functions
Proof	Table-driven tests or a tiny self-check `main`
Process	Checklist + same flags locally and in CI

The investment in code quality pays dividends throughout the software lifecycle, reducing bugs, improving maintainability, and enhancing the overall developer experience.

Compiler Sanitizers (ASan, UBSan, TSan, MSan)

Overview

Compiler sanitizers are **instrumented builds** that turn many classes of memory and concurrency bugs into **loud, early failures** instead of silent corruption. For operators and CI, the boring habit is: ship tests under **AddressSanitizer + UndefinedBehaviorSanitizer**, run a separate **ThreadSanitizer** job for threaded code, and treat **MemorySanitizer** as an advanced Clang-only gate when you can rebuild the whole dependency graph.

Tool	Flag	Catches (high level)	Typical slowdown
ASan (AddressSanitizer)	`-fsanitize=address`	Heap/stack/global OOB, use-after-free, double-free, some use-after-return/scope; leaks via integrated LSan on Linux	~2×
UBSan (UndefinedBehaviorSanitizer)	`-fsanitize=undefined`	Signed overflow, null deref, misaligned access, invalid shifts, …	small
TSan (ThreadSanitizer)	`-fsanitize=thread`	Data races	~5–15×
MSan (MemorySanitizer)	`-fsanitize=memory`	Use of uninitialized values (Clang)	~3×

Hard rule: ASan, TSan, and MSan are **mutually exclusive** in one binary. ASan + UBSan **can** be combined: `-fsanitize=address,undefined`.

Baseline compile line (GCC or Clang):

```
cc -std=c17 -Wall -Wextra -g -O1 -fno-omit-frame-pointer \
  -fsanitize=address,undefined \
  -o prog prog.c
```

Use the compiler driver (`cc` / `gcc` / `clang`) for the **link** step so the sanitizer runtime is linked. Linking with bare `ld` usually fails or produces a binary that cannot report errors.

Mental model

```
source    instrumented object code    link sanitizer runtime


           run tests / binary

         first real bug    stderr report + non-zero exit
```

Sanitizers are **bug detectors for test and debug builds**, not production hardening. Do not ship ASan-instrumented binaries to users expecting a security boundary — the runtime is not designed as a hardened sandbox.

AddressSanitizer (ASan)

Planted use-after-free

```c
/* file: asan_uaf.c */
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int *p = malloc(sizeof *p);
    if (p == NULL) {
        return 1;
    }
    *p = 42;
    free(p);
    /* BOOM: use-after-free */
    printf("%d\n", *p);
    return 0;
}
```

```
cc -std=c17 -Wall -Wextra -g -O1 -fno-omit-frame-pointer \
  -fsanitize=address -o asan_uaf asan_uaf.c
./asan_uaf
```

Expected shape of the report (exact addresses vary):

```
==PID==ERROR: AddressSanitizer: heap-use-after-free on address …
READ of size 4 at … thread T0
    #0 … in main asan_uaf.c:…
…
```

```
freed by thread T0 here:
    #0 … in free …
    #1 … in main asan_uaf.c:…
previously allocated by thread T0 here:
    #0 … in malloc …
    #1 … in main asan_uaf.c:…
==PID==ABORTING
```

Read the report top-down: **error type**, **access**, **stack of the bad use**, then **who freed**, then **who allocated**.

Heap buffer overflow

```c
/* file: asan_oob.c */
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int *a = malloc(4 * sizeof *a);
    if (a == NULL) {
        return 1;
    }
    for (int i = 0; i <= 4; i++) { /* last iteration is OOB */
        a[i] = i;
    }
    printf("%d\n", a[0]);
    free(a);
    return 0;
}
```

```
cc -std=c17 -Wall -Wextra -g -O1 -fsanitize=address -o asan_oob asan_oob.c
./asan_oob
# Expect: heap-buffer-overflow
```

Useful ASan runtime knobs

```
# List all options
ASAN_OPTIONS=help=1 ./asan_uaf

# Common CI defaults
ASAN_OPTIONS=detect_leaks=1:abort_on_error=1:halt_on_error=1 ./tests

# Symbolize with llvm-symbolizer when stacks are raw
ASAN_SYMBOLIZER_PATH="$(command -v llvm-symbolizer)" ./asan_uaf
```

On Linux, LeakSanitizer is integrated with ASan and often on by default. Suppress third-party leak noise with `LSAN_OPTIONS=suppressions=lsan.supp` rather than turning leaks off globally unless you have a documented reason.

UndefinedBehaviorSanitizer (UBSan)

UBSan catches **language undefined behavior** that may not be a classic memory corruption yet still means the program is not trustworthy.

```c
/* file: ubsan_overflow.c */
#include <limits.h>
#include <stdio.h>

int main(void) {
    int x = INT_MAX;
    /* signed overflow is UB */
    int y = x + 1;
    printf("%d\n", y);
    return 0;
}
```

```
cc -std=c17 -Wall -Wextra -g -O1 -fsanitize=undefined -o ubsan_overflow ubsan_overflow.c
./ubsan_overflow
# Expect a runtime diagnostic about signed integer overflow
```

Combine with ASan for day-to-day CI:

```
cc -std=c17 -Wall -Wextra -g -O1 -fno-omit-frame-pointer \
  -fsanitize=address,undefined -o tests tests.c
```

Pass UBSan-specific knobs via `UBSAN_OPTIONS` (not packed into `ASAN_OPTIONS` when both runtimes are present).

ThreadSanitizer (TSan)

TSan finds **data races**: unsynchronized concurrent access where at least one access is a write.

```c
/* file: tsan_race.c - needs -pthread */
#include <pthread.h>
#include <stdio.h>
```

```c
static int counter;

static void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < 100000; i++) {
        counter++; /* race */
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, worker, NULL);
    pthread_create(&b, NULL, worker, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("%d\n", counter);
    return 0;
}
```

```
cc -std=c17 -Wall -Wextra -g -O1 -fsanitize=thread -pthread -o tsan_race tsan_race.c
./tsan_race
# Expect: WARNING: ThreadSanitizer: data race
```

Requirements: compile **and** link with `-fsanitize=thread`. Prefer instrumenting **all** objects in the process; non-instrumented shared libraries can hide races or add noise. Runtime knobs live in `TSAN_OPTIONS` (for example `history_size=7` when stacks fail to restore).

Fix the race with a mutex, atomics, or by removing shared mutable state — then re-run until TSan is silent.

MemorySanitizer (MSan) — brief

MSan (Clang) detects **uses of uninitialized memory**. It is stricter than ASan about "read before write."

```
clang -std=c17 -Wall -Wextra -g -O1 -fno-omit-frame-pointer \
  -fsanitize=memory -fsanitize-memory-track-origins=2 \
  -o msan_demo msan_demo.c
```

Operator constraints:

- Instrument **every** dependency, including libc wrappers Clang ships for MSan — mixed instrumented/non-instrumented code produces false positives.

- Do not combine with ASan/TSan.
- Prefer a dedicated CI job on a known toolchain image, not a laptop one-off, until the rebuild story is solid.

If you cannot rebuild the world, stick to ASan+UBSan (+ Valgrind Memcheck for selected tests) and schedule MSan later.

Interpreting reports (operator checklist)

1. **Reproduce with symbols:** `-g`, preferably `-O1`, `-fno-omit-frame-pointer`.
2. **Read the first error only** — ASan aborts on the first finding by design; fix it, then re-run.
3. **Map frames to your code** before blaming the standard library; library frames often sit above the real bug.
4. **Confirm it is your build:** wrong binary, stale `.o`, or a non-instrumented `dlopen` plugin are common false friends.
5. **One sanitizer family per job** in CI matrices (see below).

CI gates (practical matrix)

Job	Flags	Notes
`asan-ubsan`	`-fsanitize=address,undefined`	Default PR gate for C tests
`tsan`	`-fsanitize=thread`	Only if the suite exercises threads
`msan`	`-fsanitize=memory`	Clang + fully instrumented deps
`release`	no sanitizers, `-O2`/`-O3`	Ship artifacts; do not mix

Example Makefile fragment:

```
CFLAGS_SAN = -std=c17 -Wall -Wextra -g -O1 -fno-omit-frame-pointer

test-asan: tests.c
    $(CC) $(CFLAGS_SAN) -fsanitize=address,undefined -o tests-asan tests.c
    ASAN_OPTIONS=detect_leaks=1:abort_on_error=1 ./tests-asan

test-tsan: tests_threaded.c
    $(CC) $(CFLAGS_SAN) -fsanitize=thread -pthread -o tests-tsan tests_threaded.c
    ./tests-tsan
```

Fail the pipeline on **any** sanitizer abort. Do not “warn-only” ASan in CI — that trains the team to ignore red stacks.

Common false friends

Symptom / belief	Reality
"ASan false positive"	ASan is not expected to false-positive on fully instrumented code; re-check lifetime and OOB logic
Combining ASan + TSan	Unsupported; use separate binaries/jobs
Sanitizer "fixes" the bug	It only **detects**; you still patch the code
Running ASan at `-O0` only	Works, but `-O1` is the documented sweet spot for speed + traces
Ignoring leaks because "process exits"	Still a bug signal in long-lived services and tests; triage with LSan suppressions carefully
Shipping ASan to production for "safety"	Wrong tool — use hardened allocators / privilege drop / fuzzing separately
TSan clean no concurrency bugs	Deadlocks, logic races, and missed wakes can remain
UBSan silent at `-O2` without the flag	Without `-fsanitize=undefined`, the compiler may exploit UB

Selective opt-out (rare): `__attribute__((no_sanitize("address")))` on a function, gated with `__has_feature(address_sanitizer)` / `__SANITIZE_ADDRESS__`. Prefer fixing the code; use ignorelists only for third-party pain you cannot rebuild yet.

Try this

1. Build `asan_uaf.c` and `asan_oob.c`; save both reports and highlight the allocation/free stacks.
2. Add `-fsanitize=undefined` to a project that uses intentional unsigned wrap — confirm you did **not** enable unsigned overflow in the default `undefined` group unless you opted in.
3. Run `tsan_race.c`, then fix it with `pthread_mutex_t` and show a clean TSan run.
4. Wire `test-asan` into CI so a planted UAF fails the job.

Sources

- Clang AddressSanitizer

- Clang UndefinedBehaviorSanitizer
- Clang ThreadSanitizer
- Clang MemorySanitizer
- AddressSanitizer wiki (Google)

Memory-Safe C in Practice: Fil-C, counted_by, and -fbounds-safety

Memory-Safe C in Practice: Fil-C, `counted_by`, and `-fbounds-safety`

Thesis

Sanitizers find memory bugs **in the runs you test**. This chapter is about the other question: what stops an out-of-bounds write or a use-after-free **in production**, on input you never tried? As of October 2026 there are three practical answers for C on Linux, and they work at different levels:

Option	What it is	Status (Oct 2026)	Unit of protection
Fil-C	A memory-safe *implementation* of C and C++. Every pointer carries an invisible capability and memory is garbage collected. You rebuild the program **and its libc** with it	0.686 (released 2026-10-04). Linux x86_64/ARM64 only, based on Clang 20.1.8	The **allocation**, rounded up to 16 bytes
`counted_by`	An attribute that ties a flexible array member to the struct field holding its length. The compiler then checks indexing (`-fsanitize=bounds`) and size queries	Upstream Clang (including 23.1.3) and GCC 15+. Not in GCC 14	The **array member**, by its declared count
`-fbounds-safety`	Apple's bounds-annotation dialect of C (`__counted_by` on pointers, wide local pointers, compile-time rejection of unchecked arithmetic)	Upstream Clang documents it as "a design document … not available for users yet". Clang 23.1.3 rejects the flag. It ships in Apple's Clang fork	The **pointer**, by its annotation

None of them is a free upgrade. This chapter runs the same few bugs through a plain build, `_FORTIFY_SOURCE=3`, AddressSanitizer, `counted_by`, and Fil-C, and shows what each one stops, what each one misses, and what it costs.

Mental model

```
                    detects in tests     stops in production     what you rebuild

ASan                 yes (byte-exact)     no (not hardening)      your code
_FORTIFY_SOURCE=3    some libc calls      same calls, abort       your code
counted_by + bounds  indexed FAM access   only with -trap         your code + annotations
Fil-C                yes (per object)     yes, panics             your code + libc + every d
```

Question	Fil-C	ASan	`counted_by`
Overflow into the *next* allocation	Panic	Report	Only for annotated arrays
Overflow into the *next field* of the same struct	**Not caught**	**Not caught**	Only for annotated arrays
Use after free	Panic, every time	Report, unless the memory has been reused	No
Off-by-a-few past `malloc(n)`	Not caught until the 16-byte rounding is exhausted	Report	Caught exactly, for annotated arrays
Integer → pointer round trip through memory	Panic (the integer carries no capability)	Allowed	Allowed
Typical cost here	~1.7× time, ~2× memory	~1.8× time, ~2× memory	A compare per checked access

The rows that matter most are the gaps. Fil-C protects *objects*, not fields. ASan is a debugging tool that an attacker can step around. `counted_by` only knows the arrays you annotate.

Worked examples

All transcripts below come from this machine:

```
$ gcc --version | head -1
gcc (Debian 14.2.0-19) 14.2.0
$ clang --version | head -1
clang version 23.1.3 (https://github.com/llvm/llvm-project 0d261d1ca552c95a8f007e061c787ac71
```

```
$ $FILC/build/bin/clang --version | head -1
clang version 20.1.8 (Fil-C 0.686 git@github.com:pizlonator/fil-c.git 163fae598eaf249b74065b
$ clang -fbounds-safety -c packet.c -o /dev/null
clang: error: unknown argument: '-fbounds-safety'
```

`clang` is the LLVM 23.1.3 release and `gcc` is Debian's 14.2. `$FILC` is the unpacked `filc-0.686-linux-x86_64` release tarball, after its `./setup.sh` has run. That tarball is the musl-based distribution, so `$FILC/build/bin/clang` links against Fil-C's own libc, not the system glibc.

Case 1: an overflow into the next allocation

```c
// heap.c - copy a name into an 8-byte heap buffer without checking its length
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv) {
    const char *who = argc > 1 ? argv[1] : "kim";
    char *tag  = malloc(8);
    char *next = malloc(8);
    strcpy(next, "intact");
    strcpy(tag, who);                       /* no length check */
    printf("tag=%s next=%s\n", tag, next);
    free(next);
    free(tag);
    return 0;
}
```

```
$ gcc -O2 -g heap.c -o heap_gcc
$ gcc -O2 -g -D_FORTIFY_SOURCE=3 heap.c -o heap_fortify
$ clang -O1 -g -fsanitize=address heap.c -o heap_asan
$ $FILC/build/bin/clang -O2 -g heap.c -o heap_fil
$ ./heap_gcc kim
tag=kim next=intact
$ ./heap_gcc kimberly-from-accounts-payable-admin; echo "exit=$?"
tag=kimberly-from-accounts-payable-admin next=dmin
double free or corruption (out)
Aborted
exit=134
$ ./heap_fortify kimberly-from-accounts-payable-admin; echo "exit=$?"
*** buffer overflow detected ***: terminated
Aborted
exit=134
$ ./heap_asan kimberly-from-accounts-payable-admin; echo "exit=$?"
```

```
=================================================================
==490270==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7bb331de0018 at pc 0x55
WRITE of size 37 at 0x7bb331de0018 thread T0
    #0 0x5558e5d16b14 in strcpy /home/runner/work/llvm-project/llvm-project/compiler-rt/lib/
    #1 0x5558e5d73822 in main /workspace/scratch/memsafe/heap.c:11:5
    #2 0x7f9332a98ca7 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:5
    #3 0x7f9332a98d64 in __libc_start_main csu/../csu/libc-start.c:360:3
    #4 0x5558e5c83350 in _start (/workspace/scratch/memsafe/heap_asan+0x2d350)
...
$ ./heap_fil kimberly-from-accounts-payable-admin; echo "exit=$?"
filc safety error: cannot write pointer with ptr >= upper.
    pointer: 0x7f8bb39082e0,0x7f8bb39082d0,0x7f8bb39082e0
    expected 1 writable bytes.
semantic origin:
    (libc.so) src/string/stpcpy.c:12:12: __stpcpy
check scheduled at:
    (libc.so) src/string/stpcpy.c:12:13: __stpcpy
    (libc.so) src/string/strcpy.c:5:2: strcpy
    (heap_fil) heap.c:11:5: main
    (libc.so) src/env/__libc_start_main.c:79:7: __libc_start_main
    (libpizlo.so) <runtime>: start_program
[490283] filc panic: thwarted a futile attempt to violate memory safety.
Trace/breakpoint trap
exit=133
```

The ASan report is cut after the first frames.

- **Plain build.** `next` now reads `dmin`. The copy ran over the 8-byte buffer, through glibc's chunk header, and into the neighbouring string. The program only died later, inside `free`, far from the bug. On a different input it would not have died at all.
- **`_FORTIFY_SOURCE=3`.** glibc's checked `strcpy` knew the destination was an 8-byte `malloc` and aborted before writing. That only works because the size was visible at the call.
- **ASan** gives the best *diagnosis*: the exact write size (37) and the allocation it overflowed. It's built for tests, though. Its redzones and quarantine are probabilistic barriers, not a security boundary, and it roughly doubles memory.
- **Fil-C** stops the write at the first byte past the object (`ptr >= upper`) and panics with a stack. The trace goes through Fil-C's own `stpcpy`. The libc is compiled with the same checks, so there is no unchecked `strcpy` to hide in.

Case 2: use after free

```c
// session.c - a dangling pointer meets a recycled allocation
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
struct session { char user[16]; int is_admin; };

int main(void) {
    struct session *s = malloc(sizeof *s);
    strcpy(s->user, "guest");
    s->is_admin = 0;
    free(s);                                    /* logout... */

    struct session *t = malloc(sizeof *t);      /* ...someone else logs in */
    strcpy(t->user, "root");
    t->is_admin = 1;

    printf("stale pointer sees: user=%s is_admin=%d\n", s->user, s->is_admin);
    free(t);
    return 0;
}
```

```
$ gcc -O0 -g session.c -o session_gcc 2>/dev/null
$ ./session_gcc
stale pointer sees: user=root is_admin=1
$ clang -O1 -g -fsanitize=address session.c -o session_asan
$ ./session_asan; echo "exit=$?"
=================================================================
==490351==ERROR: AddressSanitizer: heap-use-after-free on address 0x7b6d9ede0050 at pc 0x55b
READ of size 4 at 0x7b6d9ede0050 thread T0
    #0 0x55b9421cb85d in main /workspace/scratch/memsafe/session.c:18:69
    #1 0x7f3d9fc71ca7 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:5
    #2 0x7f3d9fc71d64 in __libc_start_main csu/../csu/libc-start.c:360:3
...
$ $FILC/build/bin/clang -O2 -g session.c -o session_fil
$ ./session_fil; echo "exit=$?"
filc safety error: cannot access pointer to free object.
    pointer: 0x7fbd53d10550,0x7fbd53d10550,0x7fbd53d10550,free
    expected valid capability.
semantic origin:
    (session_fil) session.c:18:69: main
check scheduled at:
    (session_fil) session.c:14:25: main
    (libc.so) src/env/__libc_start_main.c:79:7: __libc_start_main
    (libpizlo.so) <runtime>: start_program
[490394] filc panic: thwarted a futile attempt to violate memory safety.
Trace/breakpoint trap
exit=133
```

The plain build (`-O0`, so the transcript is stable) shows the exploit shape. glibc handed the freed chunk straight to the next `malloc` of the same size, so the stale pointer now sees `root` with `is_admin=1`. With `-O2 -Wall`, GCC warns `pointer 's' used after 'free'`, and the program

prints whatever the allocator left in the chunk, which is different on every run. ASan caught it because the freed chunk was still in quarantine. With enough allocation churn in between, ASan's quarantine recycles too.

Fil-C's report reads `pointer to free object`. Freeing an object sets its capability's upper bound to its lower bound, so every later access through any old pointer panics deterministically. The memory is reclaimed by the garbage collector once nothing points to it, which means a dangling pointer can never see another object's data.

Case 3: teach the compiler how long an array is

A flexible array member (`data[]`) is the classic place where the compiler loses track of a size. `counted_by` tells it which field holds the count:

```c
// packet.c - a flexible array member that knows its own length
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#if defined(__has_attribute)
#  if __has_attribute(counted_by)
#    define COUNTED_BY(n) __attribute__((counted_by(n)))
#  endif
#endif
#ifndef COUNTED_BY
#  define COUNTED_BY(n)
#endif

struct packet {
    size_t len;
    unsigned char data[] COUNTED_BY(len);
};

[[gnu::noinline]] static size_t room(struct packet *p) {
    return __builtin_dynamic_object_size(p->data, 1);
}

[[gnu::noinline]] static void fill(struct packet *p, size_t last) {
    for (size_t i = 0; i <= last; i++)
        p->data[i] = (unsigned char)i;
}

[[gnu::noinline]] static void copy_in(struct packet *p, const void *src, size_t n) {
    memcpy(p->data, src, n);
}

int main(int argc, char **argv) {
```

```
    const char *mode = argc > 1 ? argv[1] : "room";
    struct packet *p = calloc(1, sizeof *p + 16);
    if (!p) return 1;

    if (!strcmp(mode, "late")) {           /* write first, set the count after */
        fill(p, 15);
        p->len = 16;
        puts("late: returned");
        free(p);
        return 0;
    }

    p->len = 16;                           /* count before any use of data[] */
    if (!strcmp(mode, "room")) {
        size_t r = room(p);
        if (r == (size_t)-1) puts("room(p) = unknown");
        else                 printf("room(p) = %zu\n", r);
    } else if (!strcmp(mode, "fill")) {
        fill(p, p->len);                   /* off by one: last valid is len-1 */
        puts("fill: returned");
    } else if (!strcmp(mode, "copy")) {
        char src[32] = {0};
        copy_in(p, src, sizeof src);       /* 32 bytes into 16 */
        puts("copy: returned");
    }
    free(p);
    return 0;
}
```

```
$ gcc -std=c2x -O2 -g -fsanitize=bounds packet.c -o packet_gcc
$ clang -std=c23 -O2 -g -fsanitize=bounds packet.c -o packet_clang
$ ./packet_gcc room; ./packet_clang room
room(p) = unknown
room(p) = 16
$ ./packet_gcc fill
fill: returned
$ ./packet_clang fill
packet.c:26:12: runtime error: index 16 out of bounds for type 'unsigned char[] __counted_by
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior packet.c:26:12
fill: returned
$ clang -std=c23 -O2 -g -fsanitize=bounds -fsanitize-trap=bounds packet.c -o packet_trap
$ ./packet_trap fill; echo "exit=$?"
Illegal instruction
exit=132
```

- `room(p)` is `__builtin_dynamic_object_size(p->data, 1)` inside a non-inlined function. Clang reads `p->len` and answers `16`. GCC 14 doesn't know the attribute, so the `COUNTED_BY`

macro expands to nothing and the answer is "unknown".
- With `-fsanitize=bounds`, Clang checks every `p->data[i]` against `p->len` and reports the off-by-one at `index 16`. GCC 14 can't, because it has nothing to compare against.
- By default UBSan reports and **continues** (`fill: returned`). For hardening rather than diagnosis, add `-fsanitize-trap=bounds`. The check then compiles to a trap instruction with no runtime library, and the process dies with `SIGILL` (exit 132) at the bad write.

The `#if __has_attribute(counted_by)` guard is the portable pattern. The annotation documents the invariant on every compiler and enforces it on the ones that understand it.

Case 4: what Fil-C asks you to change

Fil-C is meant to be "fanatically compatible", and most code builds unchanged. The idiom it breaks on purpose is turning a pointer into an integer and back, because the capability can't survive the trip through an integer stored in memory:

```
// handles.c - hand out integer handles for objects, the way many C APIs do
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#if defined(__FILC__) && !defined(RAW_CAST)
#include <stdfil.h>
static zexact_ptrtable *table;
static uintptr_t to_handle(void *p)       { return zexact_ptrtable_encode(table, p); }
static void     *from_handle(uintptr_t h) { return zexact_ptrtable_decode(table, h); }
static void      handles_init(void)       { table = zexact_ptrtable_new(); }
#else
static uintptr_t to_handle(void *p)       { return (uintptr_t)p; }
static void     *from_handle(uintptr_t h) { return (void *)h; }
static void      handles_init(void)       { }
#endif

struct widget { int id; };

[[gnu::noinline]] static uintptr_t widget_open(int id) {
    struct widget *w = malloc(sizeof *w);
    w->id = id;
    return to_handle(w);
}

[[gnu::noinline]] static int widget_id(uintptr_t h) {
    struct widget *w = from_handle(h);
    return w->id;
}

int main(void) {
```

```
    handles_init();
    uintptr_t h = widget_open(7);
    printf("widget id %d\n", widget_id(h));
    return 0;
}
```

```
$ gcc -std=c2x -O2 -g -Wall handles.c -o handles_gcc && ./handles_gcc
widget id 7
$ $FILC/build/bin/clang -std=c2x -O2 -g -Wall -DRAW_CAST handles.c -o handles_raw
$ ./handles_raw; echo "exit=$?"
filc safety error: cannot read pointer with null object.
    pointer: 0x7f58043042d0,<null>
    expected 4 bytes.
semantic origin:
    (handles_raw) handles.c:28:15: widget_id
check scheduled at:
    (handles_raw) handles.c:28:15: widget_id
    (handles_raw) handles.c:34:30: main
    (libc.so) src/env/__libc_start_main.c:79:7: __libc_start_main
    (libpizlo.so) <runtime>: start_program
[490466] filc panic: thwarted a futile attempt to violate memory safety.
Trace/breakpoint trap
exit=133
$ $FILC/build/bin/clang -std=c2x -O2 -g -Wall handles.c -o handles_fil && ./handles_fil
widget id 7
```

`(void *)h` produces the right address with a **null** capability, so the first dereference panics. The fix is to keep an explicit table, and `<stdfil.h>` ships one. `zexact_ptrtable_encode` returns exactly the pointer's integer value, so handles look the same as before, and `decode` gives the capability back. A decoded handle to a freed object still has a null capability, so stale handles stay caught.

Small cases like this can mislead. When the integer never leaves a register, the optimizer folds `(int *)(uintptr_t)p` back into `p` and the raw cast happens to work at `-O2`. It's the trip through memory, a struct field or a hash table, that loses the capability. Test the real code path, not a toy.

Case 5: what it costs

```
// bench2.c - two small CPU-bound kernels: array sort and a pointer tree
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static unsigned long long rng = 88172645463325252ULL;
static int next_rand(void) {                          /* xorshift64: same stream on every li
    rng ^= rng << 13; rng ^= rng >> 7; rng ^= rng << 17;
```

```
    return (int)(rng >> 33);
}

static double ms_since(struct timespec a) {
    struct timespec b; clock_gettime(CLOCK_MONOTONIC, &b);
    return (b.tv_sec - a.tv_sec) * 1e3 + (b.tv_nsec - a.tv_nsec) / 1e6;
}

static void sort(int *v, long lo, long hi) {          /* plain quicksort */
    while (lo < hi) {
        int p = v[(lo + hi) / 2]; long i = lo, j = hi;
        while (i <= j) {
            while (v[i] < p) i++;
            while (v[j] > p) j--;
            if (i <= j) { int t = v[i]; v[i] = v[j]; v[j] = t; i++; j--; }
        }
        if (j - lo < hi - i) { sort(v, lo, j); lo = i; } else { sort(v, i, hi); hi = j; }
    }
}

struct tnode { struct tnode *l, *r; int key; };

static void free_tree(struct tnode *t) {
    if (!t) return;
    free_tree(t->l); free_tree(t->r); free(t);
}

static struct tnode *insert(struct tnode *t, int key) {
    struct tnode **slot = &t, *root = t;
    while (*slot) slot = key < (*slot)->key ? &(*slot)->l : &(*slot)->r;
    *slot = calloc(1, sizeof **slot);
    (*slot)->key = key;
    return root ? root : *slot;
}

int main(void) {
    enum { N = 1 << 20, TN = 1 << 18, LOOKUPS = 1 << 22 };
    int *v = malloc(N * sizeof *v);
    for (long i = 0; i < N; i++) v[i] = next_rand();
    struct timespec t0; clock_gettime(CLOCK_MONOTONIC, &t0);
    for (int r = 0; r < 5; r++) { for (long i = 0; i < N; i++) v[i] ^= next_rand(); sort(v,
    double sort_ms = ms_since(t0);

    struct tnode *root = NULL;
    for (int i = 0; i < TN; i++) root = insert(root, next_rand());
    clock_gettime(CLOCK_MONOTONIC, &t0);
```

```
    long hits = 0;
    for (int i = 0; i < LOOKUPS; i++) {
        int k = next_rand();
        for (struct tnode *n = root; n; n = k < n->key ? n->l : n->r)
            if (n->key == k) { hits++; break; }
    }
    double tree_ms = ms_since(t0);
    printf("sort: %.0f ms   tree: %.0f ms   (check %d %ld)\n", sort_ms, tree_ms, v[N / 2] &
    free_tree(root); free(v);
    return 0;
}
```

```
$ clang -O2 bench2.c -o bench_clang
$ clang -O2 -fsanitize=address bench2.c -o bench_asan
$ $FILC/build/bin/clang -O2 bench2.c -o bench_fil
$ /usr/bin/time -f "maxrss=%M KiB wall=%e s" ./bench_clang
sort: 474 ms   tree: 1764 ms   (check 1 499)
maxrss=13888 KiB wall=2.32 s
$ /usr/bin/time -f "maxrss=%M KiB wall=%e s" ./bench_asan
sort: 529 ms   tree: 3335 ms   (check 1 499)
maxrss=26840 KiB wall=4.09 s
$ /usr/bin/time -f "maxrss=%M KiB wall=%e s" ./bench_fil
sort: 642 ms   tree: 3213 ms   (check 1 499)
maxrss=28208 KiB wall=4.04 s
```

The benchmark uses its own xorshift generator, so all three builds sort and search the same data even though Fil-C's musl `rand()` differs from glibc's (`check 1 499` matches). Across three runs each on this 8-core box, the numbers stayed within about 10%:

- **Array sort:** Fil-C ~1.3× the plain build.
- **Pointer-chasing tree lookups:** ~1.8×.
- **Peak memory:** ~2× for both ASan and Fil-C.

Fil-C's own write-up quotes "about 4x in the bad cases". Treat these numbers as one data point, not a budget, and measure your hot path.

The trap

Memory safety is about *objects*. Your bug might be about *fields*. Overflow `name[8]` by four bytes so that it lands exactly on `is_admin`, inside the same 12-byte allocation:

```
// field.c - the same bug, but the victim lives inside the same object
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
struct account {
    char name[8];
    int  is_admin;
};

int main(int argc, char **argv) {
    const char *who = argc > 1 ? argv[1] : "kim";
    struct account *a = calloc(1, sizeof *a);
    strcpy(a->name, who);                    /* overflows into is_admin */
    printf("name=%s is_admin=%d\n", a->name, a->is_admin);
    free(a);
    return 0;
}
```

```
$ ./field_gcc 'kimberly!AA'
name=kimberly!AA is_admin=4276513
$ ./field_asan 'kimberly!AA'
name=kimberly!AA is_admin=4276513
$ ./field_fil 'kimberly!AA'
name=kimberly!AA is_admin=4276513
$ ./field_fortify 'kimberly!AA'; echo "exit=$?"
*** buffer overflow detected ***: terminated
Aborted
exit=134
```

ASan and Fil-C both let `is_admin` become `4276513`. The write never left the object, so neither has anything to object to. Only `_FORTIFY_SOURCE=3` caught it, because `strcpy` into `a->name` is checked with `__builtin_dynamic_object_size(a->name, 1)`, the size of the *member*. Fortify only covers the libc calls it wraps, though. A hand-written byte loop into `a->name` gets past fortify, ASan, and Fil-C alike. What does catch that loop is `-fsanitize=bounds`, because `name` has a declared size of 8 (`index 8 out of bounds for type 'char[8]'`).

Fil-C's bounds are also not byte-exact. Allocations are rounded up to the runtime's 16-byte minimum alignment (`stdfil.h` documents this as "may allocate slightly more than `count`"), so a 13-byte copy into `malloc(8)` passes:

```
$ ./heap_fil 'kimberly!AAA'
tag=kimberly!AAA next=intact
$ ./heap_asan 'kimberly!AAA'
=================================================================
==491062==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7c039f9e0018 at pc 0x55
WRITE of size 13 at 0x7c039f9e0018 thread T0
...
```

That's still *memory-safe*, since no other object can be reached. But it means Fil-C is not a replacement for ASan as a bug finder.

`counted_by` has its own traps. The count is a promise you have to keep:

```
$ ./packet_clang late
packet.c:26:12: runtime error: index 0 out of bounds for type 'unsigned char[] __counted_by(
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior packet.c:26:12
late: returned
$ ./packet_clang copy; echo "exit=$?"
copy: returned
exit=0
$ $FILC/build/bin/clang -std=c2x -O2 -g packet.c -o packet_fil
$ ./packet_fil fill
fill: returned
$ ./packet_fil copy
filc safety error: cannot write 32 bytes when upper - ptr = 24 (destination ptr = 0x7f55ef10
    (libpizlo.so) <runtime>: memmove
    (packet_fil) packet.c:30:5: copy_in
...
```

- **Set the count first.** In `late` mode the array is filled while `len` is still `0` (from `calloc`), so the first, perfectly valid write is reported as `index 0 out of bounds`. Treat `p->len = n` as part of the allocation.
- **Indexing is checked. Bulk copies are not.** `memcpy` of 32 bytes into the 16-byte `data[]` ran to completion in the Clang build (`copy: returned`). `-fsanitize=bounds` instruments `p->data[i]`, not `memcpy`. Fil-C caught the same copy (`cannot write 32 bytes when upper - ptr = 24`) but not the one-byte `fill` overrun, which landed in the 16-byte rounding slack.

The boring rule

Keep ASan + UBSan in CI. They find the bugs. Hardening is a separate decision:

- **Every compiler you ship with:** annotate every flexible array member with a `__has_attribute`-guarded `counted_by`, assign the count before the first element, and build release binaries with `-fsanitize=bounds -fsanitize-trap=bounds` plus `-D_FORTIFY_SOURCE=3`. The checks are cheap and need no runtime library.
- **A process that parses hostile input and you can rebuild the whole stack:** build it with Fil-C. Budget about 2× memory and measure CPU, replace integer pointer round trips with `zexact_ptrtable`, and treat a Fil-C panic as a crash that would otherwise have been an exploit.
- **Fields that guard security decisions** (`is_admin`, lengths, function pointers) should not sit right after a fixed-size buffer that string APIs write into. Neither Fil-C nor ASan sees overflows between fields.
- **`-fbounds-safety`:** wait for upstream Clang to enable it, or use Apple's toolchain, before you plan around it.

Try this

1. Replace `strcpy(a->name, who)` in `field.c` with a hand-written `for` loop that copies bytes until `'\0'`. Confirm that the `-D_FORTIFY_SOURCE=3` build no longer catches the overflow and the Clang `-fsanitize=bounds` build does. Then fix it with `snprintf(a->name, sizeof a->name, "%s", who)`.
2. Reorder `struct account` so that `is_admin` comes first. Which of the four builds now detects `'kimberly!AA'`, and why?
3. Clang 23 also accepts `counted_by` on **pointer** members. Write `struct buf { size_t cap; unsigned char *data COUNTED_BY(cap); };`, index one past `cap` under `-fsanitize=bounds`, and check which of your compilers report it.
4. In `bench2.c`, change `struct tnode` to hold a 64-byte key. Rerun all three builds and see how Fil-C's memory ratio moves when objects contain more data and fewer pointers.
5. Build `heap.c` with `$FILC/build/bin/clang` and run it under `strace -f`. Find where Fil-C's runtime sets up its heap and compare the syscalls to the glibc build.

Sources

- Fil-C 0.686 release (2026-10-04), README and binaries: https://github.com/pizlonator/fil-c/releases/tag/v0.686
- Fil-C, *InvisiCaps: The Fil-C Capability Model* (integer→pointer gives a null capability; freed objects; "about 4x in the bad cases"): https://fil-c.org/invisicaps
- Fil-C `stdfil.h` (`zgc_alloc` minalign, `zptrtable`, `zexact_ptrtable`), in the release tarball at `pizfix/stdfil-include/stdfil.h`
- Clang, *`-fbounds-safety`: Enforcing bounds safety for C* (status note: "design document … not available for users yet"): https://clang.llvm.org/docs/BoundsSafety.html
- Clang attribute reference, `counted_by`: https://clang.llvm.org/docs/AttributeReference.html#counted-by-counted-by-or-null-sized-by-sized-by-or-null
- GCC 15 release series changes (`counted_by` for flexible array members): https://gcc.gnu.org/gcc-15/changes.html
- GCC manual, common variable attributes (`counted_by`): https://gcc.gnu.org/onlinedocs/gcc/Common-Variable-Attributes.html
- Clang UndefinedBehaviorSanitizer (`-fsanitize=bounds`, `-fsanitize-trap`): https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
- glibc manual, Source Fortification (`_FORTIFY_SOURCE=3`): https://sourceware.org/glibc/manual/latest/html_node/Source-Fortification.html

GCC Diagnostics: JSON and SARIF for CI

GCC Diagnostics: JSON and SARIF for CI

Thesis

Human-readable compiler warnings are for humans. CI, IDEs, and security dashboards want **structured** diagnostics: machine-stable rule ids, file/line/column, and related notes. GCC can emit that today with `-fdiagnostics-format=`:

Value	Where it goes	Shape
`text`	stderr (default)	Classic caret diagnostics
`json` / `json-stderr`	stderr	JSON array of diagnostics
`json-file`	`<source>.gcc.json` next to the input	same JSON, in a file
`sarif` paths: `sarif-stderr` / `sarif-file`	stderr or `<source>.sarif`	SARIF 2.1.0 run

This chapter is verified on **GCC 14.2.0** (Debian 14.2.0-19) — the toolchain on this lab box. GCC 16 release notes advertise further diagnostics work (including HTML-oriented experiments upstream); we do **not** invent GCC 16 output here. Re-run the same commands on your builder's GCC before you pin CI parsers.

Mental model

```
gcc -Wall -c foo.c

            text          → humans / logs
            json*         → jq, custom bots, lighter parsers
            sarif*        → GitHub code scanning, IDEs, SARIF viewers
```

SARIF wraps a `runs[]` entry with tool metadata (`driver.name`, `version`, `rules[]`) and `results[]` that point at `physicalLocation.region`. JSON mode is a flatter list of `{kind, message, option, locations, children}` objects — fine for a five-line `jq` gate, less standard across vendors.

Worked examples

All listings: **GCC 14.2.0**, `x86_64-linux-gnu`, C17.

Case 0: which formats exist?

```
gcc -fdiagnostics-format=help
```

```
gcc: error: unrecognized argument in option ‘-fdiagnostics-format=help’
gcc: note: valid arguments to ‘-fdiagnostics-format=’ are: json json-file json-stderr sarif-
```

The “error” is GCC rejecting the literal `help` token; the **note** is the catalogue you want. Default remains `text`.

Case 1: text baseline

```
/* uninit.c */
#include <stdio.h>
int main(void) {
    int x;
    printf("%d\n", x);
    return 0;
}
```

```
gcc -Wall -Wextra -c uninit.c -o uninit.o
```

```
uninit.c: In function ‘main’:
uninit.c:5:5: warning: ‘x’ is used uninitialized [-Wuninitialized]
    5 |     printf("%d\n", x);
      |     ^~~~~~~~~~~~~~~~~
uninit.c:4:9: note: ‘x’ was declared here
    4 |     int x;
      |         ^
```

Case 2: SARIF file for dashboards

```
rm -f uninit.c.sarif uninit.o
gcc -Wall -Wextra -fdiagnostics-format=sarif-file -c uninit.c -o uninit.o
wc -c uninit.c.sarif
python3 - <<'PY'
import json
d = json.load(open("uninit.c.sarif"))
run = d["runs"][0]
print("tool:", run["tool"]["driver"]["fullName"])
r = run["results"][0]
print("ruleId:", r["ruleId"])
print("level:", r["level"])
```

```
print("message:", r["message"]["text"])
print("region:", r["locations"][0]["physicalLocation"]["region"])
PY
```

```
3205 uninit.c.sarif
tool: GNU C17 (Debian 14.2.0-19) version 14.2.0 (x86_64-linux-gnu)
ruleId: -Wuninitialized
level: warning
message: ‘x’ is used uninitialized
region: {'startLine': 5, 'startColumn': 5, 'endColumn': 22}
```

The document declares `$schema` for SARIF 2.1.0 and embeds the source snippet under `artifacts`. CI can fail the job when `results` is non-empty, or upload the `.sarif` to a code-scanning product that already speaks the format.

`sarif-stderr` writes the same document to stderr — useful when the build tree is read-only and you capture logs instead of files.

Case 3: JSON for a tiny gate

```
gcc -Wall -Wextra -fdiagnostics-format=json-file -c uninit.c -o /dev/null
python3 - <<'PY'
import json, sys
d = json.load(open("uninit.c.gcc.json"))
warns = [x for x in d if x.get("kind") == "warning"]
print("warnings:", len(warns), "option:", warns[0]["option"])
sys.exit(1 if warns else 0)
PY
```

```
warnings: 1 option: -Wuninitialized
```

(exit status 1) — that is the whole CI policy for a “no new warnings” gate on a single file. Prefer `json-file` / `sarif-file` when parallel `gcc` invocations would interleave stderr JSON.

Case 4: plain text without carets (log-friendly)

```
gcc -Wall -fdiagnostics-plain-output -c uninit.c -o /dev/null
```

```
uninit.c: In function ‘main’:
uninit.c:5:5: warning: ‘x’ is used uninitialized [-Wuninitialized]
uninit.c:4:9: note: ‘x’ was declared here
```

Same facts, no source underline. Handy when caret columns confuse log scrapers that are not ready for SARIF yet.

Case 5: analyzer events still flow through text (and into SARIF when enabled)

```c
/* bounds.c */
int sum3(int *a) {
    return a[0] + a[1] + a[2] + a[3]; /* one past */
}
int main(void) {
    int v[3] = {1, 2, 3};
    return sum3(v);
}
```

```
gcc -Wall -fanalyzer -c bounds.c -o bounds.o
```

```
bounds.c: In function ‘sum3’:
bounds.c:3:34: warning: stack-based buffer over-read [CWE-126] [-Wanalyzer-out-of-bounds]
    3 |     return a[0] + a[1] + a[2] + a[3]; /* one past */
      |                                  ~^~~
  ‘main’: events 1-3
    |
    |    5 | int main(void) {
    |      |     ^~~~
    |      |     |
    |      |     (1) entry to ‘main’
    |    6 |     int v[3] = {1,2,3};
    |      |         ~
    |      |         |
    |      |         (2) capacity: 12 bytes
    |    7 |     return sum3(v);
    |      |            ~~~~~~~
    |      |            |
    |      |            (3) calling ‘sum3’ from ‘main’
    |
    +--> ‘sum3’: events 4-5
           |
           |    2 | int sum3(int *a) {
           |      |     ^~~~
           |      |     |
           |      |     (4) entry to ‘sum3’
           |    3 |     return a[0] + a[1] + a[2] + a[3]; /* one past */
           |      |                                 ~~~~
           |      |                                  |
           |      |                                  (5) out-of-bounds read from byte 12 til
```

Pair `-fanalyzer` with `-fdiagnostics-format=sarif-file` when you want those path events in a tool that understands SARIF graphs. Keep analyzer runs in a separate CI job: they are slower than `-Wall` alone.

The trap

Parsing caret text with regex. Column-adjusted Unicode quotes (`‘x’`), multi-line notes, and analyzer path diagrams will break your scraper on the first GCC upgrade. Prefer JSON or SARIF and key on `option` / `ruleId`.

Interleaved JSON on stderr from `make -j`. Two compilers writing JSON arrays to one log produce invalid JSON. Use `*-file` outputs or per-object log files.

Assuming SARIF means “GCC 16 only”. File and stderr SARIF emitters are already on GCC 14.2. New GCC releases may add sinks (HTML experiments, richer analyzer graphs); treat release notes as a delta, not as permission to skip verifying your installed compiler.

The boring rule

> Ship CI warnings as **SARIF** (or GCC JSON) from the **same** GCC major you run in production builds. Fail on `results` / `kind=warning` you have not waived. Keep `text` for interactive builds; never scrape carets in automation.

Try this

1. Add a Make pattern rule: `%.o: %.c` compiles with `-fdiagnostics-format=sarif-file` and a script that `jq`s every `*.sarif` under `build/`.
2. Emit both `-Wuninitialized` and a deliberate `-Wunused-variable`; assert your gate counts two `ruleId`s.
3. Run Case 5 with `-fdiagnostics-format=sarif-file` and inspect whether analyzer path events appear as related locations or separate results on your GCC.
4. On a machine with GCC 15 or 16, re-run Case 0 and diff the valid-argument list against 14.2 — document only what that binary prints.

Sources

- GCC Optimize / Instrumentation options: `-fdiagnostics-format=` (Diagnostic Message Formatting Options)
- OASIS SARIF 2.1.0
- GCC 14 changes: https://gcc.gnu.org/gcc-14/changes.html (baseline for this lab)
- GCC 16 changes (read when you have that toolchain): https://gcc.gnu.org/gcc-16/changes.html — do not trust secondary blogs over the installed `gcc --version`

Testing the Error Paths: Deterministic Failure Injection

Testing the Error Paths: Deterministic Failure Injection

Thesis

The `goto`-cleanup ladder is C's answer to "undo what I did if a later step fails". It is also the code your tests run least. Happy-path tests never take it, sanitizers only watch the paths that actually run, and code review reads it once. The leaks, double frees, and leaked descriptors hiding in error paths surface in production, on the day `malloc` or `read` finally fails.

The fix is mechanical. Put a **named failure point** in front of every fallible call. In a test build, make any one of them fail on its n-th hit. Then **sweep** the whole space, every point at every hit, with AddressSanitizer and LeakSanitizer watching. In a release build the failure points compile to a constant `false`, and the object code is byte-for-byte identical to code that never had them.

This is the C version of Mitchell Hashimoto's Tripwire library for Zig's `errdefer`. It also matches the "fail the n-th allocation, then increase n" loop that SQLite has used for its out-of-memory and I/O-error testing for years.

Mental model

```
int fd = TW_FAIL(TW_OPEN, EMFILE) ? -1   : open(path, O_RDONLY | O_CLOEXEC);
buf    = TW_FAIL(TW_MALLOC, ENOMEM) ? NULL : malloc(cap);
         \______________________/   \___/   \___________/
          test build: true on the    what the   the real call
          armed point's nth hit,     real call
          and sets errno             returns on failure
          release build: false
```

Four rules make it work:

Rule	Why
A failure point returns **exactly what the real call returns on failure**, and sets `errno`	The code under test can't tell an injected failure from a real one
Points are **named**, not just counted	A report says `TW_REALLOC hit 2`, not "allocation #7", and adding one allocation elsewhere does not renumber everything
Each injection runs in a **fresh child process**	One double free can't abort the sweep, and leak and descriptor counts start clean

Rule	Why
The harness checks the point **actually fired**	A point your fixture never reaches is untested, and the sweep must say so instead of printing `ok`

Worked examples

All listings were run on Debian 13 x86_64 with **GCC 14.2.0** (`gcc (Debian 14.2.0-19)`), glibc 2.41, and Linux 6.12. The current upstream release is GCC 16.2 (2026-08-07). Nothing here depends on a newer compiler: `-Wuse-after-free` has existed since GCC 12, and ASan/LSan long before that. The files live in one directory:

```
mkdir -p tripwire/logs && cd tripwire
```

Case 1: A loader whose happy-path test passes under ASan

`config_load` reads a whole file and duplicates its first line as the name. It is written in the common ladder style, with labels that free "everything allocated so far". Three bugs are planted, all of them in error paths.

```
/* config.h - load a small config file: first line is the name. */
#ifndef CONFIG_H
#define CONFIG_H
#include <stddef.h>

struct config {
    char *name; /* first line, owned */
    char *body; /* whole file, owned, not NUL-terminated */
    size_t len;
};

int config_load(const char *path, struct config *out); /* 0 or -1 + errno */
void config_free(struct config *c);
#endif
```

```
/* config.c - a typical goto-ladder loader, with three planted bugs. */
#define _POSIX_C_SOURCE 200809L /* open flags, strndup, ssize_t */
#include "config.h"
#include "tripwire.h"

#include <errno.h>
#include <fcntl.h>
#include <stdlib.h>
```

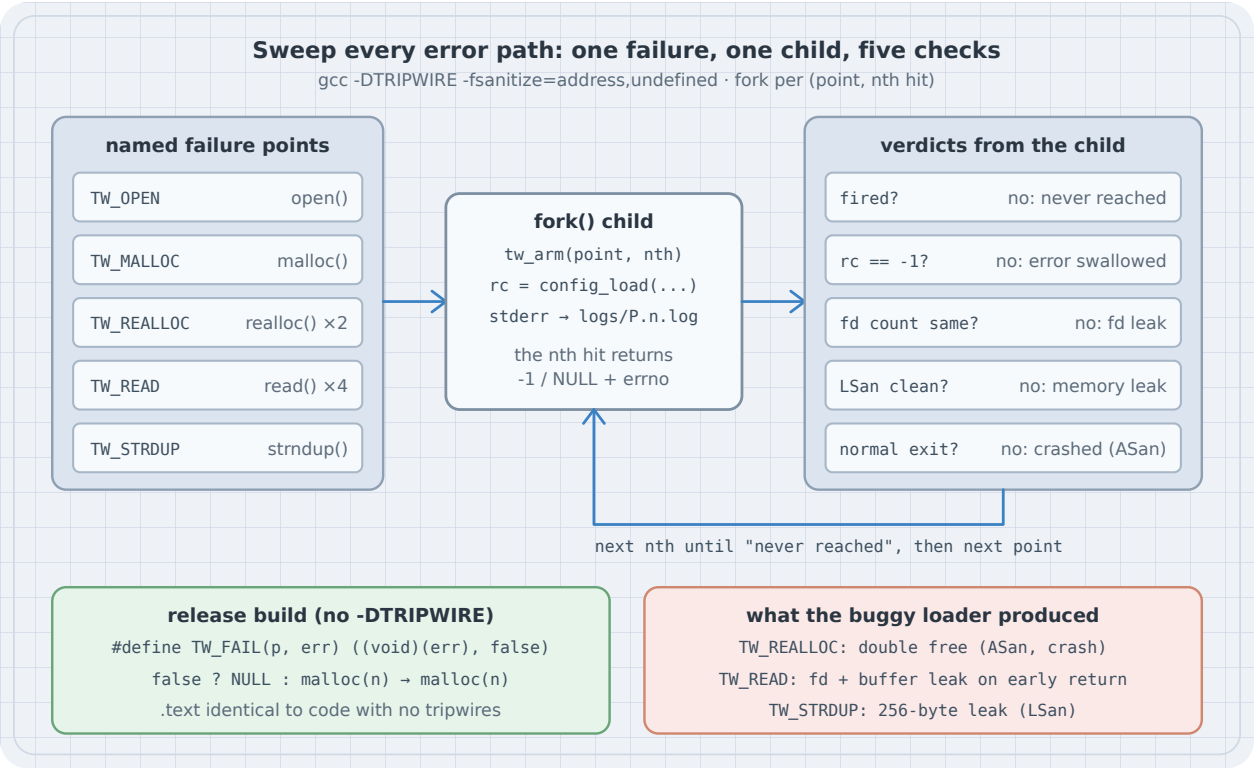


Figure 1: Sweep: named points, one forked child per (point, nth hit), five verdicts; release builds fold the macro away

```c
#include <string.h>
#include <unistd.h>

int config_load(const char *path, struct config *out)
{
    int fd = TW_FAIL(TW_OPEN, EMFILE) ? -1 : open(path, O_RDONLY | O_CLOEXEC);
    if (fd < 0)
        return -1;

    size_t cap = 64, len = 0;
    char *buf = TW_FAIL(TW_MALLOC, ENOMEM) ? NULL : malloc(cap);
    if (!buf)
        goto out_close;

    for (;;) {
        if (len == cap) {
            char *nbuf = TW_FAIL(TW_REALLOC, ENOMEM) ? NULL : realloc(buf, cap * 2);
            if (!nbuf) {
                free(buf);
                goto out_free; /* BUG 1: out_free frees buf again */
            }
```

```
            buf = nbuf;
            cap *= 2;
        }
        ssize_t n = TW_FAIL(TW_READ, EIO) ? -1 : read(fd, buf + len, cap - len);
        if (n < 0)
            return -1; /* BUG 2: leaks buf and fd */
        if (n == 0)
            break;
        len += (size_t)n;
    }

    char *nl = memchr(buf, '\n', len);
    size_t nlen = nl ? (size_t)(nl - buf) : len;
    char *name = TW_FAIL(TW_STRDUP, ENOMEM) ? NULL : strndup(buf, nlen);
    if (!name)
        goto out_close; /* BUG 3: skips free(buf) */

    out->name = name;
    out->body = buf;
    out->len = len;
    close(fd);
    return 0;

out_free:
    free(buf);
out_close:
    close(fd);
    return -1;
}

void config_free(struct config *c)
{
    free(c->name);
    free(c->body);
    *c = (struct config){0};
}
```

`tripwire.h` (shown in full in Case 2) is included, but in this build `TW_FAIL` is the constant `false`, so this is just the plain loader. The test most projects ship:

```
/* test_happy.c - the test most projects stop at. */
#include "config.h"

#include <stdio.h>

int main(void)
{
```

```
    struct config c = {0};
    if (config_load("fixture.conf", &c) != 0) {
        perror("config_load");
        return 1;
    }
    printf("name=%s len=%zu\n", c.name, c.len);
    config_free(&c);
    return 0;
}
```

`fixture.conf` is 183 bytes: a first line `web-frontend` followed by ten `keyNN = value-NN` lines.

```
$ gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined -o test_happy test_happy.c config.
config.c: In function ‘config_load’:
config.c:54:5: warning: pointer ‘buf’ used after ‘free’ [-Wuse-after-free]
   54 |     free(buf);
      |     ^~~~~~~~~
config.c:27:17: note: call to ‘free’ here
   27 |                 free(buf);
      |                 ^~~~~~~~~
$ ./test_happy
name=web-frontend len=183
exit=0
```

GCC's `-Wuse-after-free` (in `-Wall`) catches **bug 1**, because the `free` / `goto` / `free` sequence is visible within one function. Bugs 2 and 3 are invisible to it. Both are legal C that drops a resource on the floor. The test passes, and ASan and LSan stay silent because no error path ran.

Case 2: The tripwire header and its zero-cost release build

Points are declared once with an X-macro, which gives both the enum and printable names. The test-only implementation is a counter.

```
/* tripwire.h - named failure points that vanish from release builds. */
#ifndef TRIPWIRE_H
#define TRIPWIRE_H

#include <stdbool.h>

/* Every fallible call site gets a name. Add new points here. */
#define TW_POINTS(X) \
    X(TW_OPEN)       \
    X(TW_MALLOC)     \
    X(TW_REALLOC)    \
    X(TW_READ)       \
    X(TW_STRDUP)
```

```
#define TW_ENUM(name) name,
enum tw_point { TW_POINTS(TW_ENUM) TW_NPOINTS };
#undef TW_ENUM

#ifdef TRIPWIRE
/* Test build: a real function that may fail the call. */
bool tw_fail(enum tw_point p, int err);
void tw_arm(enum tw_point p, unsigned nth); /* fail the nth hit of p */
void tw_reset(void);
bool tw_fired(void);                        /* did the armed point trip? */
const char *tw_name(enum tw_point p);
#define TW_FAIL(p, err) tw_fail((p), (err))
#else
/* Release build: a constant the compiler folds away. */
#define TW_FAIL(p, err) ((void)(err), false)
#endif

#endif
```

```
/* tripwire.c - only compiled into test binaries (-DTRIPWIRE). */
#include "tripwire.h"

#include <errno.h>

static enum tw_point armed = TW_NPOINTS; /* nothing armed */
static unsigned target, hits;
static bool fired;

#define TW_NAME(name) #name,
static const char *const names[] = { TW_POINTS(TW_NAME) };

bool tw_fail(enum tw_point p, int err)
{
    if (p != armed || ++hits != target)
        return false;
    fired = true;
    errno = err; /* look exactly like the real failure */
    return true;
}

void tw_arm(enum tw_point p, unsigned nth) { armed = p; target = nth; hits = 0; fired = fals
void tw_reset(void) { tw_arm(TW_NPOINTS, 0); }
bool tw_fired(void) { return fired; }
const char *tw_name(enum tw_point p) { return names[p]; }
```

The claim that release builds pay nothing is cheap to check. Strip every `TW_FAIL(...) ? x :` prefix with `sed` to get the loader as if tripwires had never existed, then compare machine code. This

uses the fixed loader from Case 4, but the result is the same for any file.

```
#!/bin/sh
# zero_cost.sh - is the release build identical to code with no tripwires?
set -e
sed -E 's/TW_FAIL\(TW_[A-Z]+, E[A-Z]+\) \? [^:]+ : //' config_fixed.c \
    | grep -v '#include "tripwire.h"' > config_plain.c
gcc -std=c17 -O2 -c config_fixed.c -o release.o          # no -DTRIPWIRE
gcc -std=c17 -O2 -c config_plain.c -o plain.o
gcc -std=c17 -O2 -DTRIPWIRE -c config_fixed.c -o test.o
size release.o plain.o test.o
echo "tw_ symbols in release.o: $(nm release.o | grep -c tw_ || true)"
echo "tw_ symbols in test.o:    $(nm test.o | grep -c tw_ || true)"
objcopy -O binary --only-section=.text release.o release.text
objcopy -O binary --only-section=.text plain.o plain.text
cmp release.text plain.text && echo ".text of release.o and plain.o: identical"
```

```
$ ./zero_cost.sh
   text    data     bss     dec     hex filename
    486       0       0     486     1e6 release.o
    486       0       0     486     1e6 plain.o
    598       0       0     598     256 test.o
tw_ symbols in release.o: 0
tw_ symbols in test.o:    1
.text of release.o and plain.o: identical
```

The release object's `.text` is **identical** to the code with no tripwires, and it has no `tw_` symbols. In the test build the macro becomes a real call to `tw_fail`, which costs 112 bytes in this function and only exists in test binaries.

Case 3: The sweep finds all three bugs

The harness arms one `(point, nth)` pair per child and runs the loader. It then checks, in order, that the point fired, that the error was reported, that no descriptor leaked, and that LeakSanitizer is clean (`__lsan_do_recoverable_leak_check()`). A crash is caught from the exit status. Sanitizer output goes to a per-injection log, and the sweep prints the last `SUMMARY:` line.

```
/* test_sweep.c - fail every point at every hit, one child per failure. */
#define _POSIX_C_SOURCE 200809L
#include "config.h"
#include "tripwire.h"

#include <dirent.h>
#include <fcntl.h>
#include <sanitizer/lsan_interface.h>
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

enum { OK = 0, NOT_REACHED = 10, NOT_REPORTED = 11, FD_LEAK = 12, MEM_LEAK = 13 };

static int count_fds(void)
{
    int n = 0;
    DIR *d = opendir("/proc/self/fd");
    for (struct dirent *e; d && (e = readdir(d));)
        n += e->d_name[0] != '.';
    if (d)
        closedir(d);
    return n - 1; /* the DIR's own descriptor */
}

static int child(const char *fixture, enum tw_point p, unsigned nth)
{
    struct config c = {0};
    int fds = count_fds();
    tw_arm(p, nth);
    int rc = config_load(fixture, &c);
    if (!tw_fired())
        return NOT_REACHED; /* the nth hit never happened */
    if (rc != -1)
        return NOT_REPORTED; /* error swallowed */
    if (count_fds() != fds)
        return FD_LEAK;
    if (__lsan_do_recoverable_leak_check())
        return MEM_LEAK;
    return OK;
}

/* Last "SUMMARY:" line the sanitizer wrote, if any. */
static void summary(const char *log, char *out, size_t n)
{
    char line[512];
    FILE *f = fopen(log, "r");
    out[0] = '\0';
    while (f && fgets(line, sizeof line, f))
        if (strncmp(line, "SUMMARY:", 8) == 0)
            snprintf(out, n, "%.*s", (int)strcspn(line + 9, "\n"), line + 9);
    if (f)
        fclose(f);
}
```

```
int main(int argc, char **argv)
{
    const char *fixture = argc > 1 ? argv[1] : "fixture.conf";
    int failures = 0;

    for (int p = 0; p < TW_NPOINTS; p++) {
        for (unsigned nth = 1;; nth++) {
            char log[64];
            snprintf(log, sizeof log, "logs/%s.%u.log", tw_name(p), nth);
            fflush(stdout);
            pid_t pid = fork();
            if (pid == 0) {
                int fd = open(log, O_WRONLY | O_CREAT | O_TRUNC, 0644);
                dup2(fd, STDERR_FILENO);
                close(fd);
                _exit(child(fixture, p, nth));
            }
            int st;
            waitpid(pid, &st, 0);
            int code = WIFEXITED(st) ? WEXITSTATUS(st) : 128 + WTERMSIG(st);
            if (code == NOT_REACHED) {
                if (nth == 1) {
                    printf("%-10s never reached: this fixture cannot test it\n", tw_name(p))
                    failures++;
                }
                break;
            }
            char sum[256];
            summary(log, sum, sizeof sum);
            const char *verdict = code == OK             ? "ok"
                                : code == NOT_REPORTED ? "FAIL error swallowed"
                                : code == FD_LEAK      ? "FAIL fd leak"
                                : code == MEM_LEAK     ? "FAIL memory leak"
                                                       : "FAIL crashed";
            printf("%-10s hit %u: %s%s%s\n", tw_name(p), nth, verdict, *sum ? " | " : "", su
            failures += code != OK;
        }
    }
    printf("%d failing injection(s)\n", failures);
    return failures != 0;
}
```

```
gcc -std=c17 -Wall -Wextra -g -DTRIPWIRE -fsanitize=address,undefined \
    -o test_sweep test_sweep.c config.c tripwire.c   # same warning as Case 1
```

```
$ ./test_sweep
```

```
TW_OPEN    hit 1: ok
TW_MALLOC  hit 1: ok
TW_REALLOC hit 1: FAIL crashed | AddressSanitizer: double-free ../../../../src/libsanitizer/asa
TW_REALLOC hit 2: FAIL crashed | AddressSanitizer: double-free ../../../../src/libsanitizer/asa
TW_READ    hit 1: FAIL fd leak
TW_READ    hit 2: FAIL fd leak
TW_READ    hit 3: FAIL fd leak
TW_READ    hit 4: FAIL fd leak
TW_STRDUP  hit 1: FAIL memory leak | AddressSanitizer: 256 byte(s) leaked in 1 allocation(s).
7 failing injection(s)
exit=1

$ grep -m1 -A3 ERROR logs/TW_REALLOC.1.log
==116016==ERROR: AddressSanitizer: attempting double-free on 0x506000000020 in thread T0:
    #0 0x7f7f722f38f8 in free ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:52
    #1 0x56374c5752a8 in config_load /tmp/tripwire/config.c:54
    #2 0x56374c5746c5 in child /tmp/tripwire/test_sweep.c:33
```

Every planted bug is pinned to a named point:

- `TW_REALLOC` → **double free (bug 1).** The `realloc` failure path frees `buf`, then jumps to a label that frees it again. ASan aborts the child with the exact line, `config.c:54`.
- `TW_READ` → **fd leak (bug 2).** The early `return -1` skips both `free(buf)` and `close(fd)`. The descriptor check fires first, and LSan would have reported the buffer too. Note that **every** read iteration is a separate test. A bug that only appears on the fourth read would be caught here as well.
- `TW_STRDUP` → **memory leak (bug 3).** `goto out_close` skips `free(buf)`. By then `buf` has been `realloc`ed to 256 bytes, which is exactly what LSan reports.
- `TW_OPEN` and `TW_MALLOC` are `ok`. Those paths really were correct, and now there is evidence for it.

Case 4: One cleanup block, and a clean sweep

The bugs share a cause: each label encodes a guess about which resources are live at that point, and the guesses rot as the function changes. The robust shape gives every resource an "empty" value (`-1`, `NULL`) up front. There is **one** exit label that releases whatever is non-empty. Success hands ownership to the caller by setting the locals back to empty. `errno` is saved around the cleanup, because `close` may change it.

```c
/* config_fixed.c - one cleanup block, every resource starts "empty". */
#define _POSIX_C_SOURCE 200809L /* open flags, strndup, ssize_t */
#include "config.h"
#include "tripwire.h"

#include <errno.h>
#include <fcntl.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <unistd.h>

int config_load(const char *path, struct config *out)
{
    int rc = -1, saved;
    int fd = -1;
    char *buf = NULL, *name = NULL;
    size_t cap = 64, len = 0;

    fd = TW_FAIL(TW_OPEN, EMFILE) ? -1 : open(path, O_RDONLY | O_CLOEXEC);
    if (fd < 0)
        goto out;

    buf = TW_FAIL(TW_MALLOC, ENOMEM) ? NULL : malloc(cap);
    if (!buf)
        goto out;

    for (;;) {
        if (len == cap) {
            char *nbuf = TW_FAIL(TW_REALLOC, ENOMEM) ? NULL : realloc(buf, cap * 2);
            if (!nbuf)
                goto out; /* buf is still valid and still ours */
            buf = nbuf;
            cap *= 2;
        }
        ssize_t n = TW_FAIL(TW_READ, EIO) ? -1 : read(fd, buf + len, cap - len);
        if (n < 0)
            goto out;
        if (n == 0)
            break;
        len += (size_t)n;
    }

    char *nl = memchr(buf, '\n', len);
    name = TW_FAIL(TW_STRDUP, ENOMEM) ? NULL : strndup(buf, nl ? (size_t)(nl - buf) : len);
    if (!name)
        goto out;

    /* Success: hand ownership to the caller, then clean up as usual. */
    *out = (struct config){ .name = name, .body = buf, .len = len };
    name = buf = NULL;
    rc = 0;
out:
    saved = errno; /* close() may overwrite it */
    free(name);
    free(buf);
```

```
    if (fd >= 0)
        close(fd);
    errno = saved;
    return rc;
}

void config_free(struct config *c)
{
    free(c->name);
    free(c->body);
    *c = (struct config){0};
}
```

```
gcc -std=c17 -Wall -Wextra -g -DTRIPWIRE -fsanitize=address,undefined \
    -o test_sweep_fixed test_sweep.c config_fixed.c tripwire.c
```

```
$ ./test_sweep_fixed
TW_OPEN    hit 1: ok
TW_MALLOC  hit 1: ok
TW_REALLOC hit 1: ok
TW_REALLOC hit 2: ok
TW_READ    hit 1: ok
TW_READ    hit 2: ok
TW_READ    hit 3: ok
TW_READ    hit 4: ok
TW_STRDUP  hit 1: ok
0 failing injection(s)
exit=0
$ ./test_sweep_fixed tiny.conf
TW_OPEN    hit 1: ok
TW_MALLOC  hit 1: ok
TW_REALLOC never reached: this fixture cannot test it
TW_READ    hit 1: ok
TW_READ    hit 2: ok
TW_STRDUP  hit 1: ok
1 failing injection(s)
exit=1
```

The full fixture gives nine injections and nine passes, with no compiler warnings. Keep the second run in mind: it is the trap.

The trap

A failure point the fixture never reaches is a test that silently passes. The bug in Case 3's `realloc` path only exists for files larger than the initial 64-byte buffer. Run the *buggy* loader's

sweep against a 13-byte fixture:

```
printf 'tiny\nk = v\n' > tiny.conf
```

```
$ ./test_sweep tiny.conf
TW_OPEN    hit 1: ok
TW_MALLOC  hit 1: ok
TW_REALLOC never reached: this fixture cannot test it
TW_READ    hit 1: FAIL fd leak
TW_READ    hit 2: FAIL fd leak
TW_STRDUP  hit 1: FAIL memory leak | AddressSanitizer: 64 byte(s) leaked in 1 allocation(s).
4 failing injection(s)
exit=1
```

The double free is gone from the report. It hasn't been fixed. The code is just never executed. A harness that only checks "returned -1 and no leaks" would count that row as a pass. Checking `tw_fired()` turns "untested" into a visible failure: `never reached: this fixture cannot test it`. The fixed loader's sweep against `tiny.conf` fails for the same reason, which is correct. Coverage of error paths depends on the input as much as on the injection, so keep at least one fixture that drives every point. Treat "never reached" as a red build, not as a skip.

Other ways to fool yourself:

- **Injecting failures the real call can't produce.** `read` can return a short count or fail with `EINTR`. `malloc(0)` may legitimately return `NULL`. Injected behaviour must match the man page, otherwise you test code paths that only exist in your harness.
- **Testing in the same process.** Without `fork`, the first double free aborts the whole sweep, and one leak pollutes every later leak check.
- **Counting instead of naming.** "Fail allocation #7" breaks every time someone adds an allocation upstream. Named points keep the numbering stable.
- **Shipping the test build.** `-DTRIPWIRE` belongs in the test target only. Case 2's `nm` check is a cheap CI assertion that release objects carry no `tw_` symbols.

The boring rule

Every fallible call in code that owns resources gets a named `TW_FAIL` point. Every such function gets a sweep in CI that forks a child per `(point, nth hit)`, runs under ASan and LSan, checks the return value and the descriptor count, and fails the build on "never reached". Write cleanup as one exit block over resources that start "empty", and verify with `nm` that release objects contain no tripwire code.

Try this

1. Record the injected `errno` in `tripwire.c` and expose it (say, `tw_errno()`). Make the child assert that `config_load` returns with that exact `errno`. Then delete the `saved = errno` lines

from `config_fixed.c` and find a point where `close()` would clobber it. (Hint: it may not on Linux, so check what POSIX promises.)
2. Add a `TW_CLOSE` point and decide what the loader should do when `close` fails on the success path.
3. Add an `errorAfter`-style mode, "fail every hit from n on", and rerun the sweep. Which points behave differently from single failures?
4. Port the harness to a function you own that has a ladder of three or more labels. Count the bugs before you start, then count them after.
5. Turn Case 2's `zero_cost.sh` into a CI step that fails if `nm release.o | grep tw_` finds anything.

Sources

- Mitchell Hashimoto, "Don't Trip[wire] Yourself: Testing Error Recovery in Zig" (2026-01-21): https://mitchellh.com/writing/tripwire
- SQLite, "How SQLite Is Tested": Out-Of-Memory Testing and I/O Error Testing: https://www.sqlite.org/testing.html#oomtesting
- AddressSanitizer and LeakSanitizer (Google sanitizers wiki): https://github.com/google/sanitizers/wiki/AddressSanitizer, https://github.com/google/sanitizers/wiki/AddressSanitizerLeakSanitizer
- `__lsan_do_recoverable_leak_check` interface, compiler-rt `sanitizer/lsan_interface.h`: https://github.com/llvm/llvm-project/blob/main/compiler-rt/include/sanitizer/lsan_interface.h
- GCC manual, `-Wuse-after-free`: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html#index-Wuse-after-free
- SEI CERT C, MEM12-C "Consider using a goto chain when leaving a function on error when using and releasing resources": https://wiki.sei.cmu.edu/confluence/display/c/MEM12-C.+Consider+using+a+goto+chain+when+leaving+a+function+on+error+when+using+and+releasing+resources
- POSIX.1-2024 `read()`, `close()`, and `malloc()` (error returns and `errno`): https://pubs.opengroup.org/onlinepubs/9799919799/functions/read.html, https://pubs.opengroup.org/onlinepubs/9799919799/functions/close.html, https://pubs.opengroup.org/onlinepubs/9799919799/functions/malloc.html
- GCC releases (GCC 16.2, 2026-08-07): https://gcc.gnu.org/

setjmp/longjmp for Error Unwinding: The Rules That Bite

setjmp/longjmp for Error Unwinding: The Rules That Bite

Thesis

`setjmp`/`longjmp` lets a deeply nested parser report an error with one jump back to a single handler, without threading return codes through every level. That is how Lua raises errors (`luaD_throw` → `LUAI_THROW`, which is `longjmp` or `_longjmp` in ldo.c) and how many hand-written parsers work. The jump is cheap, but it is also blind: it discards every stack frame in between without running any of their cleanup. To use it safely you need four rules, all of which this chapter reproduces in real runs:

1. A local in the `setjmp` caller that changes after `setjmp` must be `volatile`, or its value after the jump is unspecified.
2. Anything the skipped frames own has to be reachable from somewhere the handler can free it.
3. The function that called `setjmp` must still be running when `longjmp` happens.
4. When jumping out of a signal handler, use `sigsetjmp(env, 1)`/`siglongjmp`. Otherwise the signal can stay blocked.

Mental model

`setjmp(env)` saves "where we are": the stack pointer, the program counter, and some registers. It returns 0 the first time. A later `longjmp(env, v)` restores that state, so `setjmp` appears to return a second time, now with `v`. If you pass 0, it returns 1 instead. Anything between the two points is simply abandoned. Nothing like C++ destructors or Go `defer` runs.

The standard also limits where `setjmp` may appear. It may appear only as the entire controlling expression of an `if`/`switch`/loop, as one operand of a comparison against an integer constant expression (or the operand of `!`) when that is the entire controlling expression, or as a whole expression statement. In any other context, including `int r = setjmp(env);`, the behaviour is undefined (cppreference). The return value is for control flow only.

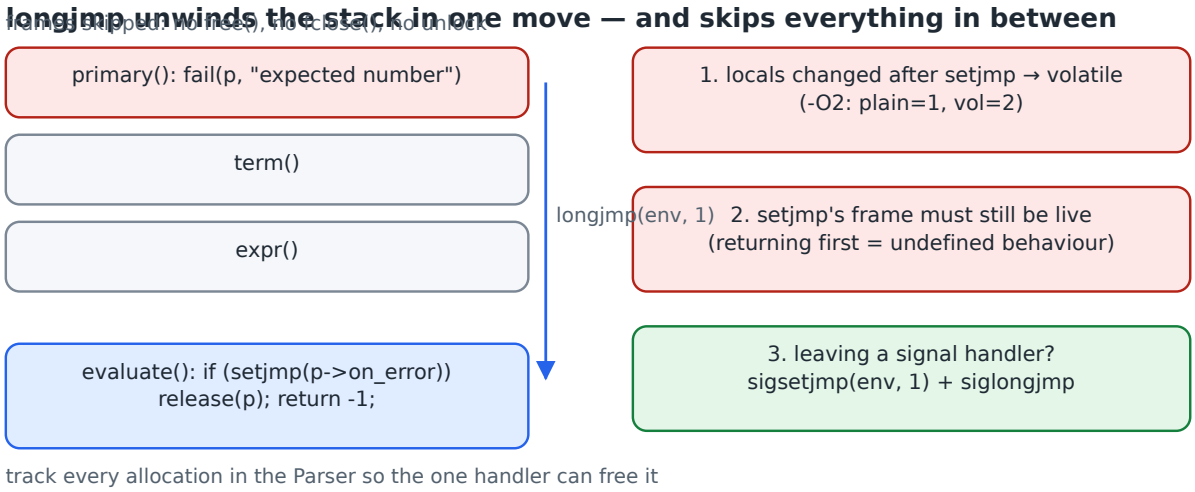


Figure 1: longjmp jumps from the innermost parser frame straight back to setjmp in evaluate, skipping cleanup in the frames between; three rules on the right

Worked example 1: the volatile rule, made visible

```c
#include <setjmp.h>
#include <stdio.h>

static jmp_buf env;
__attribute__((noinline)) static void fail(void) { longjmp(env, 1); }

int main(void) {
    int plain = 1;
    volatile int vol = 1;
    if (setjmp(env) == 0) {
        plain = 2;          /* modified after setjmp, not volatile */
        vol = 2;
        fail();
    }
    printf("plain=%d vol=%d\n", plain, vol);
    return 0;
}
```

```
gcc -std=c17 -O0 -Wall -Wextra -o clobber_O0 clobber.c && ./clobber_O0
gcc -std=c17 -O2 -Wall -Wextra -o clobber_O2 clobber.c && ./clobber_O2
```

```
-O0: plain=2 vol=2
-O2: plain=1 vol=2
```

At `-O0`, `plain` lives in memory, so the store of 2 survives. At `-O2` the optimizer is free to keep `plain` in a register or treat it as a known constant, because the standard makes its value unspecified after the jump. Here it read back the old value, 1. The Linux man page states the condition precisely. A local is unspecified after `longjmp` when it is local to the `setjmp` caller, it was changed between `setjmp` and `longjmp`, and it isn't `volatile`.

Don't rely on warnings to catch this. GCC documents `-Wclobbered` as warning "for variables that might be changed by longjmp or vfork", and `-Wextra` enables it. Yet this exact program built silently with `-O2 -Wall -Wextra -Wclobbered` on GCC 14.2. The warning is heuristic, so the rule needs to be followed in code.

Worked example 2: a parser that unwinds and doesn't leak

The parser below handles `+ - * /` and parentheses with plain recursive descent. Every error goes through `fail()`, which records a message and calls `longjmp` back to `evaluate()`. To show the cleanup problem, every number allocates a small heap block, standing in for AST nodes or token buffers.

```
/* Recursive-descent calculator: errors unwind with longjmp to one handler. */
#include <ctype.h>
#include <setjmp.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    const char *s;
    jmp_buf on_error;
    const char *msg;
    long pos;
    void *owned[64];      /* allocations to free if we unwind */
    int nowned;
} Parser;

static _Noreturn void fail(Parser *p, const char *msg) {
    p->msg = msg;
    longjmp(p->on_error, 1);
}

static void *track(Parser *p, size_t n) {
    void *q = malloc(n);
    if (!q || p->nowned == 64) fail(p, "out of memory");
    p->owned[p->nowned++] = q;
    return q;
}

static void skip(Parser *p) { while (isspace((unsigned char)*p->s)) p->s++; }
```

```
static long expr(Parser *p, const char *start);

static long primary(Parser *p, const char *start) {
    skip(p);
    if (*p->s == '(') {
        p->s++;
        long v = expr(p, start);
        skip(p);
        if (*p->s != ')') { p->pos = p->s - start; fail(p, "expected ')'"); }
        p->s++;
        return v;
    }
    if (!isdigit((unsigned char)*p->s)) { p->pos = p->s - start; fail(p, "expected number");
    long *scratch = track(p, sizeof *scratch);   /* pretend we need heap per node */
    *scratch = strtol(p->s, (char **)&p->s, 10);
    return *scratch;
}

static long term(Parser *p, const char *start) {
    long v = primary(p, start);
    for (;;) {
        skip(p);
        char op = *p->s;
        if (op != '*' && op != '/') return v;
        long op_pos = p->s - start;
        p->s++;
        long r = primary(p, start);
        if (op == '/' && r == 0) { p->pos = op_pos; fail(p, "division by zero"); }
        v = op == '*' ? v * r : v / r;
    }
}

static long expr(Parser *p, const char *start) {
    long v = term(p, start);
    for (;;) {
        skip(p);
        char op = *p->s;
        if (op != '+' && op != '-') return v;
        p->s++;
        long r = term(p, start);
        v = op == '+' ? v + r : v - r;
    }
}

static void release(Parser *p) {
    for (int i = 0; i < p->nowned; i++) free(p->owned[i]);
    p->nowned = 0;
```

```
}

/* Returns 0 on success, -1 on error (message in p->msg). */
static int evaluate(const char *src, long *out, Parser *p) {
    p->s = src; p->nowned = 0; p->msg = NULL; p->pos = 0;
    if (setjmp(p->on_error) != 0) {
#ifndef LEAK
        release(p);            /* unwinding skipped every free on the way up */
#endif
        return -1;
    }
    long v = expr(p, src);
    skip(p);
    if (*p->s) { p->pos = p->s - src; fail(p, "trailing input"); }
    release(p);
    *out = v;
    return 0;
}

int main(void) {
    const char *inputs[] = { "2 + 3 * (4 - 1)", "2 * (3 + ", "10 / (5 - 5)", "7 7", "((1+2)*
    Parser p;
    for (size_t i = 0; i < sizeof inputs / sizeof *inputs; i++) {
        long v;
        if (evaluate(inputs[i], &v, &p) == 0)
            printf("%-18s = %ld\n", inputs[i], v);
        else
            printf("%-18s ! %s at offset %ld\n", inputs[i], p.msg, p.pos);
    }
    return 0;
}
```

```
gcc -std=c17 -O2 -g -Wall -Wextra -fsanitize=address,undefined -o parse parse.c && ./parse
```

```
2 + 3 * (4 - 1)    = 11
2 * (3 +           ! expected number at offset 9
10 / (5 - 5)       ! division by zero at offset 3
7 7                ! trailing input at offset 2
((1+2)*(3+4))      = 21
```

Three design choices make this safe:

- The `jmp_buf` lives in the `Parser`, not in a global. Each parse has its own target, which is what the man page recommends (“each setjmp() call should employ a unique jmp_buf variable”), and nested or concurrent parsers can’t overwrite each other’s target.
- The state the handler reads (`msg`, `pos`, `owned`, `nowned`) lives in memory reached through a pointer, not in locals of `evaluate()` that change after `setjmp`. That keeps it out of reach of

rule 1.
- Allocations are recorded in `owned[]`, so the single handler can free what the abandoned frames can't.

Remove that one `release(p)` call from the error path (`-DLEAK`) and AddressSanitizer counts exactly what the skipped frames dropped:

```
Direct leak of 48 byte(s) in 6 object(s) allocated from:
SUMMARY: AddressSanitizer: 48 byte(s) leaked in 6 allocation(s).
```

That is 2 blocks from `2 * (3 +`, 3 from `10 / (5 - 5)` and 1 from `7 7`, six in total, each allocated by a frame that `longjmp` jumped over. The same applies to `FILE *` handles, held mutexes and partially built structures. If a skipped frame owns it, the handler has to be able to find it. An arena that the handler resets in one call is the cleanest version of this.

Worked example 3: leaving a signal handler

When a signal handler runs, the kernel blocks that signal until the handler returns. A jump out of the handler never returns, so whether the signal mask gets restored depends on which function you used:

```
#define _POSIX_C_SOURCE 200809L
#include <setjmp.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>

static sigjmp_buf senv;
static jmp_buf jenv;
static int use_sig;

static void on_usr1(int sig) {
    (void)sig;
    if (use_sig) siglongjmp(senv, 1); else longjmp(jenv, 1);
}

static int usr1_blocked(void) {
    sigset_t cur;
    sigprocmask(SIG_BLOCK, NULL, &cur);
    return sigismember(&cur, SIGUSR1);
}

int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_usr1;          /* SIGUSR1 is blocked while the handler runs */
```

```
    sigaction(SIGUSR1, &sa, NULL);

    use_sig = 0;
    if (setjmp(jenv) == 0) raise(SIGUSR1);
    printf("after longjmp:            SIGUSR1 blocked=%d\n", usr1_blocked());

    sigset_t none; sigemptyset(&none); sigprocmask(SIG_SETMASK, &none, NULL);

    use_sig = 1;
    if (sigsetjmp(senv, 1) == 0) raise(SIGUSR1);
    printf("after siglongjmp(savemask=1): SIGUSR1 blocked=%d\n", usr1_blocked());
    return 0;
}
```

```
after longjmp:            SIGUSR1 blocked=1
after siglongjmp(savemask=1): SIGUSR1 blocked=0
```

On glibc 2.41 a plain `longjmp` leaves `SIGUSR1` blocked, so a second `SIGUSR1` would never be delivered. `sigsetjmp(env, 1)` saves the mask and `siglongjmp` restores it. The man page records the history: before glibc 2.19 `setjmp` could follow BSD semantics (saving the mask) depending on feature-test macros. Since 2.19 `<setjmp.h>` exposes only the System V version, and programs that need the mask restored are told to use `sigsetjmp` with a nonzero `savesigs`. POSIX itself leaves it unspecified whether `setjmp` saves the mask, so portable code should use `sigsetjmp` whenever a signal handler is involved.

Even then, jumping out of a handler is only well-defined in limited cases. The man page warns that if the handler interrupted a non-async-signal-safe function such as `malloc` or `printf`, calling any non-async-signal-safe function after the jump is undefined behaviour. Here the handler interrupted `raise()`, which is async-signal-safe.

Things that bite

Trap	What happens	Fix
Non-volatile local changed after `setjmp`	Stale value after the jump (`plain=1` at `-O2`)	`volatile`, or keep the state in a struct reached through a pointer
Frames own resources	Leaks, unclosed files, mutexes left locked	Register ownership in the context, or use an arena the handler resets
`setjmp` caller already returned	Undefined behaviour: the saved stack frame no longer exists	Call `setjmp` in a function that stays live for the whole protected region
Jump target from another thread	Undefined behaviour (man page)	One `jmp_buf` per thread or context

Trap	What happens	Fix
Jumping out of a signal handler with `longjmp`	Signal stays blocked on glibc	`sigsetjmp(env, 1)` + `siglongjmp`
`int r = setjmp(env);`	Undefined behaviour: not a permitted context	`if (setjmp(env) != 0) { ... }` or `switch (setjmp(env))`
VLA in scope at `setjmp`, scope since left	Undefined behaviour even within the same function, and skipped VLAs may leak (cppreference)	Don't mix VLAs with non-local jumps

When to choose it

Use `setjmp`/`longjmp` when there is one well-defined entry point (`evaluate`, `lua_pcall`-style), errors are rare, and the code between entry and failure is your own and can register what it owns. Prefer return codes or a `goto cleanup` per function when frames own resources you can't easily centralise. The same applies when callbacks pass through code you don't control, since a jump through a library's frames skips its cleanup too.

Try this

1. Change `plain` in `clobber.c` to `volatile int` and confirm both builds print `plain=2`. Then try `-O1` and `-Os`.
2. Add a nesting-depth counter to `Parser` and `fail(p, "too deep")` past 100 levels. Feed it 10,000 `(` characters and check that ASan stays quiet.
3. In `mask.c`, replace the `sigsetjmp(senv, 1)` call with `sigsetjmp(senv, 0)` and predict the output before running it.

Sources

- C reference: `setjmp`, `longjmp`
- Linux man-pages: setjmp(3) (caveats, glibc 2.19 history, undefined behaviour list)
- POSIX.1-2024: sigsetjmp, longjmp
- Lua 5.4: ldo.c (`LUAI_THROW`)
- GCC: Warning options, `-Wclobbered`

Module 14: Testing

This module turns "it seems to work" into **evidence**. You will unit-test pure C with portable harnesses, drive design with TDD, hunt bugs with GDB and Valgrind, handle errors without exceptions, and raise code quality with warnings, sanitizers, and review habits.

Environment: Linux/POSIX, `gcc -std=c17 -Wall -Wextra`, plus optional `gcov/lcov`, Valgrind, and sanitizers.

Learning goals

By the end of this module you should be able to:

- Write **automated unit tests** with `assert`, a tiny custom macro harness, or a Unity-style runner
- Apply **TDD** (red → green → refactor) to small pure functions
- Design **boundary** and **table-driven** cases for integers and strings
- Measure **coverage** with `gcov/lcov` and interpret gaps
- Run tests under **AddressSanitizer** and **UndefinedBehaviorSanitizer**
- Drive **GDB** sessions (break, print, backtrace, watch) and read **core dumps**
- Detect leaks and invalid access with **Valgrind Memcheck**
- Use **strace/ltrace** and structured **DEBUG_LOG** macros
- Choose return codes, `errno`, and cleanup patterns for robust APIs
- Apply practical **quality** gates: `-Werror`, static analysis, review checklists, local `check.sh`

Chapters (map)

#	Chapter	What you will practice
1	Testing Methodologies	Unit harnesses, Unity-style suites, TDD (`is_leap_year` / `clamp`), boundaries, table-driven tests, `gcov`/`lcov`, ASan/UBSan
2	Debugging Techniques	GDB transcripts, `DEBUG_LOG`, Valgrind, strace/ltrace, core dumps, systematic isolation
3	Error Handling	Return codes, enums, `errno`, cleanup on failure, partial-success pitfalls
4	Code Quality	Warning archaeology, early-return design, `_Static_assert`, load/ownership patterns, CI-style local gates
5	Compiler Sanitizers	ASan, UBSan, TSan, brief MSan: flags, reports, CI matrix, false friends
6	Memory-Safe C in Practice	Fil-C vs ASan vs `_FORTIFY_SOURCE=3`, `counted_by` + `-fsanitize=bounds`, `-fbounds-safety` status, costs and blind spots
7	GCC Diagnostics: JSON and SARIF	`-fdiagnostics-format=` json/sarif file sinks, CI gates, analyzer pairing
8	Testing the Error Paths: Deterministic Failure Injection	Named `TW_FAIL` points, fork-per-injection sweeps under ASan/LSan, zero-cost release builds, one-exit cleanup

Suggested order

1. **Testing Methodologies** first — build a harness and a few pure functions you trust.

2. **Debugging Techniques** — deliberately break code and recover with GDB/Valgrind.

3. **Error Handling** — make APIs fail loudly and cleanly.

4. **Code Quality** — lock in habits that keep the above sustainable (`-Werror`, checklists, scripts).
5. **Compiler Sanitizers** — deepen ASan/UBSan and add TSan/MSan operator practice beyond the harness chapter.
6. **Memory-Safe C in Practice** — move from finding memory bugs in tests to stopping them in production.
7. **GCC Diagnostics (JSON/SARIF)** — turn `-Wall` into CI-consumable artifacts without scraping carets.
8. **Testing the Error Paths** — inject failures at every `malloc`/`open`/`read` so cleanup code is exercised, not just reviewed.

Tooling cheat sheet

```
# Unit test binary
gcc -std=c17 -Wall -Wextra -g -o test_foo test_foo.c
./test_foo

# Coverage
gcc -std=c17 -Wall -Wextra -g --coverage -o test_foo test_foo.c
./test_foo && gcov test_foo.c

# Sanitizers
gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined -o test_foo test_foo.c

# Strict quality compile
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -o prog prog.c

# Debug
gcc -std=c17 -Wall -Wextra -g -O0 -o prog prog.c
gdb ./prog
valgrind --leak-check=full ./prog
```

How this module connects

- **Earlier modules** (functions, pointers, memory, file I/O) supply the bugs and APIs you will test.
- **Later practice** (projects, networking) should inherit the same habit: small tests for pure logic, integration checks for I/O and sockets.
- Treat Module 14 as a **skill overlay**: re-test older chapters' utilities under a harness once you have one.

Module self-check

Each chapter ends with exercises. A solid bar for Module 14:

☐ A multi-test binary exits non-zero on failure

☐ At least one function was written strictly with TDD

☐ A table of 8 cases drives a pure function

☐ `gcov` (or HTML via `lcov`) reviewed once

- ☐ ASan or Valgrind used to find a planted memory bug
- ☐ A GDB `backtrace` obtained from a segfault or core file
- ☐ A "smelly" file cleaned until `-Werror` is silent
- ☐ One PR-style checklist applied to a small change

Work through the chapters in order, keep every example compiling with `-Wall -Wextra`, and prefer **complete programs** you can re-run when you change the code under test.

Estimated time

- Reading: 5–8 hours
- Labs (tests + GDB + sanitizers + quality gates): 8–12 hours

Next

Apply these habits to **Module 15 (Advanced)** projects and to any workbook project: ship tests and a quality script with the feature, not after.

Advanced

Module 15.1: Multithreading in C

Introduction

Multithreading is a powerful technique that allows programs to perform multiple tasks concurrently within a single process. In C programming, multithreading can significantly improve performance for CPU-intensive applications and enable responsive user interfaces. This module explores the fundamentals of multithreading using the POSIX Threads (pthreads) library, which is the standard threading API on Unix-like systems.

Learning Objectives

By the end of this module, you will be able to: - Understand the concepts of processes and threads - Create and manage threads using the pthread library - Implement thread synchronization mechanisms - Handle thread communication and data sharing - Debug and optimize multithreaded applications - Apply best practices for thread safety

1. Fundamentals of Multithreading

1.1 Processes vs Threads

A **process** is an independent program execution unit that has its own memory space, file descriptors, and system resources. A **thread** is a lightweight subprocess that exists within a process and shares the process's resources.

Key differences: - Processes have separate memory spaces; threads share memory - Process creation is expensive; thread creation is relatively cheap - Inter-process communication requires special mechanisms; threads can communicate directly through shared memory - Processes are isolated; threads can affect each other if not properly synchronized

1.2 Thread Lifecycle

A thread goes through several states during its lifetime: 1. **New**: Thread is being created 2. **Runnable**: Thread is ready to run 3. **Running**: Thread is executing 4. **Blocked**: Thread is waiting for a resource 5. **Terminated**: Thread has finished execution

1.3 Thread Attributes

Threads can have various attributes that control their behavior: - **Detached vs Joinable**: Detached threads clean up automatically; joinable threads require explicit joining - **Scheduling policy**: Determines how the thread is scheduled (SCHED_FIFO, SCHED_RR, SCHED_OTHER) - **Stack size**: Controls the amount of stack memory allocated to the thread

2. Creating and Managing Threads

2.1 Basic Thread Creation

The pthread library provides the `pthread_create()` function to create threads:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Thread function
void* thread_function(void* arg) {
    int thread_id = *(int*)arg;
    printf("Hello from thread %d\n", thread_id);

    // Simulate some work
    sleep(1);

    printf("Thread %d finishing\n", thread_id);
    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    int id1 = 1, id2 = 2;

    // Create threads
    if (pthread_create(&thread1, NULL, thread_function, &id1) != 0) {
        perror("pthread_create failed");
        exit(EXIT_FAILURE);
    }

    if (pthread_create(&thread2, NULL, thread_function, &id2) != 0) {
        perror("pthread_create failed");
        exit(EXIT_FAILURE);
    }

    // Wait for threads to complete
```

```
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("All threads completed\n");
    return 0;
}
```

2.2 Thread Arguments and Return Values

Threads can receive arguments and return values:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int start;
    int end;
    long long sum;
} thread_data_t;

void* calculate_sum(void* arg) {
    thread_data_t* data = (thread_data_t*)arg;

    data->sum = 0;
    for (int i = data->start; i <= data->end; i++) {
        data->sum += i;
    }

    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    thread_data_t data1 = {1, 50000000, 0};
    thread_data_t data2 = {50000001, 100000000, 0};

    // Create threads
    pthread_create(&thread1, NULL, calculate_sum, &data1);
    pthread_create(&thread2, NULL, calculate_sum, &data2);

    // Wait for completion
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    long long total_sum = data1.sum + data2.sum;
```

```
    printf("Sum of 1 to 100000000: %lld\n", total_sum);

    return 0;
}
```

3. Thread Synchronization

3.1 Race Conditions

A race condition occurs when multiple threads access shared data concurrently, and the outcome depends on the timing of their execution. Consider this problematic code:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int shared_counter = 0;

void* increment_counter(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        shared_counter++;  // Race condition here!
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Expected: 2000000, Actual: %d\n", shared_counter);
    return 0;
}
```

3.2 Mutexes

Mutexes (mutual exclusions) are the most common synchronization primitive:

```
#include <pthread.h>
#include <stdio.h>
```

```c
#include <stdlib.h>

int shared_counter = 0;
pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;

void* increment_counter(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&counter_mutex);
        shared_counter++;
        pthread_mutex_unlock(&counter_mutex);
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Expected: 2000000, Actual: %d\n", shared_counter);

    pthread_mutex_destroy(&counter_mutex);
    return 0;
}
```

3.3 Condition Variables

Condition variables allow threads to wait for specific conditions:

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int buffer = -1;
int count = 0;
pthread_mutex_t buffer_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t buffer_not_empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t buffer_not_full = PTHREAD_COND_INITIALIZER;

void* producer(void* arg) {
    for (int i = 0; i < 10; i++) {
```

```
        pthread_mutex_lock(&buffer_mutex);

        // Wait while buffer is full
        while (count == 1) {
            pthread_cond_wait(&buffer_not_full, &buffer_mutex);
        }

        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);

        // Signal consumer that buffer is not empty
        pthread_cond_signal(&buffer_not_empty);
        pthread_mutex_unlock(&buffer_mutex);

        sleep(1);  // Simulate work
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&buffer_mutex);

        // Wait while buffer is empty
        while (count == 0) {
            pthread_cond_wait(&buffer_not_empty, &buffer_mutex);
        }

        int item = buffer;
        count = 0;
        printf("Consumed: %d\n", item);

        // Signal producer that buffer is not full
        pthread_cond_signal(&buffer_not_full);
        pthread_mutex_unlock(&buffer_mutex);

        sleep(1);  // Simulate work
    }
    return NULL;
}

int main() {
    pthread_t producer_thread, consumer_thread;

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);
```

```
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    pthread_mutex_destroy(&buffer_mutex);
    pthread_cond_destroy(&buffer_not_empty);
    pthread_cond_destroy(&buffer_not_full);

    return 0;
}
```

4. Advanced Threading Concepts

4.1 Thread-Specific Data

Thread-specific data allows each thread to have its own copy of a variable:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

pthread_key_t thread_key;

void destructor(void* value) {
    printf("Destroying thread-specific data: %d\n", *(int*)value);
    free(value);
}

void* thread_function(void* arg) {
    int* thread_data = malloc(sizeof(int));
    *thread_data = *(int*)arg;

    pthread_setspecific(thread_key, thread_data);

    int* retrieved_data = (int*)pthread_getspecific(thread_key);
    printf("Thread %d has data: %d\n", *(int*)arg, *retrieved_data);

    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    int data1 = 100, data2 = 200;

    pthread_key_create(&thread_key, destructor);
```

```
    pthread_create(&thread1, NULL, thread_function, &data1);
    pthread_create(&thread2, NULL, thread_function, &data2);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_key_delete(thread_key);
    return 0;
}
```

4.2 Thread Cancellation

Threads can be cancelled by other threads:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void cleanup_handler(void* arg) {
    printf("Cleanup handler called with argument: %d\n", *(int*)arg);
    free(arg);
}

void* cancellable_thread(void* arg) {
    int* data = malloc(sizeof(int));
    *data = 42;

    // Install cleanup handler
    pthread_cleanup_push(cleanup_handler, data);

    // Enable cancellation
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);

    while (1) {
        printf("Working...\n");
        sleep(1);

        // Cancellation point
        pthread_testcancel();
    }

    // This won't be reached due to infinite loop
    pthread_cleanup_pop(0);
    return NULL;
```

```
}

int main() {
    pthread_t thread;

    pthread_create(&thread, NULL, cancellable_thread, NULL);

    sleep(3);  // Let thread work for a while

    printf("Cancelling thread...\n");
    pthread_cancel(thread);

    pthread_join(thread, NULL);
    printf("Thread cancelled and joined\n");

    return 0;
}
```

5. Performance Considerations

5.1 Thread Pool Implementation

A thread pool can improve performance by reusing threads:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NUM_THREADS 4
#define QUEUE_SIZE 10

typedef struct {
    void (*function)(void* arg);
    void* arg;
} task_t;

typedef struct {
    task_t task_queue[QUEUE_SIZE];
    int front, rear, count;
    pthread_mutex_t queue_mutex;
    pthread_cond_t queue_not_empty;
    pthread_cond_t queue_not_full;
    int shutdown;
    pthread_t threads[NUM_THREADS];
```

```
} threadpool_t;

threadpool_t pool = {0};

void* worker_thread(void* arg) {
    while (1) {
        pthread_mutex_lock(&pool.queue_mutex);

        // Wait for tasks or shutdown
        while (pool.count == 0 && !pool.shutdown) {
            pthread_cond_wait(&pool.queue_not_empty, &pool.queue_mutex);
        }

        if (pool.shutdown) {
            pthread_mutex_unlock(&pool.queue_mutex);
            break;
        }

        // Get task from queue
        task_t task = pool.task_queue[pool.front];
        pool.front = (pool.front + 1) % QUEUE_SIZE;
        pool.count--;

        pthread_mutex_unlock(&pool.queue_mutex);
        pthread_cond_signal(&pool.queue_not_full);

        // Execute task
        task.function(task.arg);
    }
    return NULL;
}

int threadpool_init() {
    pthread_mutex_init(&pool.queue_mutex, NULL);
    pthread_cond_init(&pool.queue_not_empty, NULL);
    pthread_cond_init(&pool.queue_not_full, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&pool.threads[i], NULL, worker_thread, NULL);
    }

    return 0;
}

int threadpool_add_task(void (*function)(void*), void* arg) {
    pthread_mutex_lock(&pool.queue_mutex);
```

```
    // Wait if queue is full
    while (pool.count == QUEUE_SIZE) {
        pthread_cond_wait(&pool.queue_not_full, &pool.queue_mutex);
    }

    // Add task to queue
    pool.task_queue[pool.rear].function = function;
    pool.task_queue[pool.rear].arg = arg;
    pool.rear = (pool.rear + 1) % QUEUE_SIZE;
    pool.count++;

    pthread_mutex_unlock(&pool.queue_mutex);
    pthread_cond_signal(&pool.queue_not_empty);

    return 0;
}

int threadpool_shutdown() {
    pthread_mutex_lock(&pool.queue_mutex);
    pool.shutdown = 1;
    pthread_mutex_unlock(&pool.queue_mutex);

    pthread_cond_broadcast(&pool.queue_not_empty);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(pool.threads[i], NULL);
    }

    pthread_mutex_destroy(&pool.queue_mutex);
    pthread_cond_destroy(&pool.queue_not_empty);
    pthread_cond_destroy(&pool.queue_not_full);

    return 0;
}

// Example task function
void example_task(void* arg) {
    int task_id = *(int*)arg;
    printf("Executing task %d\n", task_id);
    sleep(1);
    printf("Task %d completed\n", task_id);
}

int main() {
    threadpool_init();

    // Add tasks to thread pool
```

```
    for (int i = 0; i < 10; i++) {
        int* task_id = malloc(sizeof(int));
        *task_id = i;
        threadpool_add_task(example_task, task_id);
    }

    sleep(5);  // Let tasks execute

    threadpool_shutdown();
    printf("Thread pool shutdown complete\n");

    return 0;
}
```

6. Debugging Multithreaded Applications

6.1 Common Issues

1. **Deadlocks**: Threads waiting for each other indefinitely
2. **Race conditions**: Unpredictable behavior due to timing
3. **Priority inversion**: High-priority thread blocked by low-priority thread
4. **Thread starvation**: Thread never gets CPU time

6.2 Debugging Tools

- **Valgrind with Helgrind**: Detects race conditions and threading errors
- **GDB**: Can debug individual threads
- **ThreadSanitizer**: Compiler-based tool for detecting threading bugs

Example Valgrind command:

```
valgrind --tool=helgrind ./multithreaded_program
```

7. Best Practices

7.1 Design Principles

1. **Minimize shared data**: Reduce the need for synchronization
2. **Use immutable data**: Avoid race conditions by not modifying shared data
3. **Keep critical sections small**: Reduce lock contention
4. **Avoid nested locks**: Prevent deadlocks
5. **Use RAII for resource management**: Automatic cleanup

7.2 Performance Optimization

1. **Profile before optimizing**: Identify actual bottlenecks
2. **Consider Amdahl's Law**: Understand the limits of parallelization
3. **Use appropriate synchronization primitives**: Mutex for mutual exclusion, condition variables for signaling
4. **Avoid false sharing**: Ensure cache lines aren't shared unnecessarily
5. **Consider lock-free data structures**: For high-performance scenarios

Summary

Multithreading in C provides powerful capabilities for concurrent programming but requires careful design and implementation to avoid common pitfalls. Key concepts include:

- Understanding the difference between processes and threads
- Properly creating and managing threads with pthread_create and pthread_join
- Implementing synchronization with mutexes and condition variables
- Handling thread-specific data and cancellation
- Optimizing performance with thread pools and careful resource management

Mastering multithreading requires practice and understanding of both the technical details and the underlying concepts. Always prioritize correctness over performance and use appropriate debugging tools to identify issues in multithreaded code.

Module 15.2: System Programming in C

Introduction

System programming involves writing software that interacts directly with the operating system and hardware. In C, system programming encompasses a wide range of topics including process management, file system operations, inter-process communication, and low-level system interfaces. This module explores the essential concepts and techniques for writing system-level code in C.

Learning Objectives

By the end of this module, you will be able to: - Understand system calls and their role in system programming - Manage processes and implement process control - Perform advanced file system operations - Implement inter-process communication mechanisms - Work with signals and signal handling - Understand memory management at the system level - Apply best practices for secure and robust system programming

1. System Calls and APIs

1.1 What are System Calls?

System calls are the interface between user-space programs and the kernel. They provide controlled access to system resources and services. In C, system calls are typically accessed through wrapper functions in the standard library.

1.2 Common System Call Categories

1. **Process Control**: fork(), exec(), wait(), exit()
2. **File Management**: open(), read(), write(), close(), stat()
3. **Device Management**: ioctl()
4. **Information Maintenance**: getpid(), getuid(), gettimeofday()
5. **Communication**: pipe(), socket(), send(), receive()

1.3 Error Handling in System Calls

System calls return -1 on error and set the global `errno` variable:

```c
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

int main() {
    if (close(-1) == -1) {
        fprintf(stderr, "Error: %s\n", strerror(errno));
        // Error: Bad file descriptor
    }
    return 0;
}
```

2. Process Management

2.1 Process Creation with fork()

The `fork()` system call creates a new process by duplicating the calling process:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // Child process
        printf("Child process (PID: %d)\n", getpid());
        exit(EXIT_SUCCESS);
    } else {
        // Parent process
        printf("Parent process (PID: %d), Child PID: %d\n", getpid(), pid);
        wait(NULL);  // Wait for child to complete
        printf("Child process completed\n");
    }

    return 0;
}
```

2.2 Process Execution with exec()

The `exec()` family of functions replaces the current process image:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process - execute ls command
        char *args[] = {"ls", "-l", "/home", NULL};
        execvp("ls", args);

        // If execvp returns, it failed
        perror("execvp failed");
        exit(EXIT_FAILURE);
    } else if (pid > 0) {
        // Parent process
        wait(NULL);
        printf("Child process completed\n");
    } else {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

2.3 Process Groups and Sessions

Process groups and sessions help organize related processes:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Create new session
        if (setsid() == -1) {
```

```
            perror("setsid failed");
            exit(EXIT_FAILURE);
        }

        printf("New session leader (PID: %d)\n", getpid());
        printf("Session ID: %d\n", getsid(0));

        // Simulate daemon work
        sleep(10);
        exit(EXIT_SUCCESS);
    } else if (pid > 0) {
        printf("Parent process (PID: %d)\n", getpid());
        wait(NULL);
    }

    return 0;
}
```

3. Advanced File System Operations

3.1 File Metadata with stat()

The `stat()` family of functions retrieves file metadata:

```
#include <stdio.h>
#include <sys/stat.h>
#include <time.h>
#include <pwd.h>
#include <grp.h>

void print_file_info(const char *filename) {
    struct stat file_stat;

    if (stat(filename, &file_stat) == -1) {
        perror("stat failed");
        return;
    }

    printf("File: %s\n", filename);
    printf("Size: %ld bytes\n", file_stat.st_size);
    printf("Permissions: %o\n", file_stat.st_mode & 0777);
    printf("Inode: %ld\n", file_stat.st_ino);
    printf("Links: %d\n", file_stat.st_nlink);

    // Convert timestamps to readable format
```

```
    printf("Last access: %s", ctime(&file_stat.st_atime));
    printf("Last modification: %s", ctime(&file_stat.st_mtime));
    printf("Last status change: %s", ctime(&file_stat.st_ctime));

    // Get user and group names
    struct passwd *pwd = getpwuid(file_stat.st_uid);
    struct group *grp = getgrgid(file_stat.st_gid);

    if (pwd) printf("Owner: %s\n", pwd->pw_name);
    if (grp) printf("Group: %s\n", grp->gr_name);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    print_file_info(argv[1]);
    return 0;
}
```

3.2 Directory Operations

Working with directories using opendir(), readdir(), and closedir():

```
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
#include <string.h>

void list_directory(const char *path) {
    DIR *dir = opendir(path);
    if (!dir) {
        perror("opendir failed");
        return;
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        // Skip current and parent directory entries
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        printf("%s\n", entry->d_name);
```

```
        // Get file type information
        switch (entry->d_type) {
            case DT_REG:
                printf("  (regular file)\n");
                break;
            case DT_DIR:
                printf("  (directory)\n");
                break;
            case DT_LNK:
                printf("  (symbolic link)\n");
                break;
            case DT_CHR:
                printf("  (character device)\n");
                break;
            case DT_BLK:
                printf("  (block device)\n");
                break;
            case DT_FIFO:
                printf("  (named pipe)\n");
                break;
            case DT_SOCK:
                printf("  (socket)\n");
                break;
            default:
                printf("  (unknown)\n");
                break;
        }
    }

    closedir(dir);
}

int main(int argc, char *argv[]) {
    const char *path = (argc > 1) ? argv[1] : ".";
    list_directory(path);
    return 0;
}
```

3.3 Hard Links and Symbolic Links

Creating and working with links:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>
```

```c
int main() {
    const char *original = "original.txt";
    const char *hard_link = "hard_link.txt";
    const char *sym_link = "sym_link.txt";

    // Create original file
    FILE *file = fopen(original, "w");
    if (file) {
        fprintf(file, "This is the original file.\n");
        fclose(file);
    }

    // Create hard link
    if (link(original, hard_link) == -1) {
        perror("link failed");
    } else {
        printf("Hard link created: %s\n", hard_link);
    }

    // Create symbolic link
    if (symlink(original, sym_link) == -1) {
        perror("symlink failed");
    } else {
        printf("Symbolic link created: %s\n", sym_link);
    }

    // Show link information
    struct stat stat_buf;
    if (stat(original, &stat_buf) == 0) {
        printf("Original file link count: %d\n", stat_buf.st_nlink);
    }

    return 0;
}
```

4. Inter-Process Communication (IPC)

4.1 Pipes

Creating anonymous pipes for communication between related processes:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c
#include <string.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[100];

    // Create pipe
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(EXIT_FAILURE);
    }

    pid = fork();
    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        // Child process - write to pipe
        close(pipefd[0]);  // Close read end

        const char *message = "Hello from child process!";
        write(pipefd[1], message, strlen(message) + 1);

        close(pipefd[1]);  // Close write end
        exit(EXIT_SUCCESS);
    } else {
        // Parent process - read from pipe
        close(pipefd[1]);  // Close write end

        ssize_t bytes_read = read(pipefd[0], buffer, sizeof(buffer));
        if (bytes_read > 0) {
            printf("Parent received: %s\n", buffer);
        }

        close(pipefd[0]);  // Close read end
        wait(NULL);  // Wait for child to complete
    }

    return 0;
}
```

4.2 Named Pipes (FIFOs)

Creating named pipes for communication between unrelated processes:

```c
#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define FIFO_NAME "/tmp/my_fifo"

int main() {
    // Create named pipe
    if (mkfifo(FIFO_NAME, 0666) == -1) {
        perror("mkfifo failed");
        return 1;
    }

    pid_t pid = fork();
    if (pid == 0) {
        // Child process - write to FIFO
        int fd = open(FIFO_NAME, O_WRONLY);
        if (fd == -1) {
            perror("open failed");
            exit(EXIT_FAILURE);
        }

        const char *message = "Message through named pipe";
        write(fd, message, strlen(message) + 1);
        close(fd);
        exit(EXIT_SUCCESS);
    } else {
        // Parent process - read from FIFO
        int fd = open(FIFO_NAME, O_RDONLY);
        if (fd == -1) {
            perror("open failed");
            return 1;
        }

        char buffer[100];
        ssize_t bytes_read = read(fd, buffer, sizeof(buffer));
        if (bytes_read > 0) {
            printf("Received: %s\n", buffer);
        }

        close(fd);
```

```
        wait(NULL);
        unlink(FIFO_NAME);  // Remove FIFO
    }

    return 0;
}
```

4.3 Shared Memory

Using shared memory for efficient data sharing:

```
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    const int SHM_SIZE = 1024;
    const key_t shm_key = 0x1234;

    // Create shared memory segment
    int shmid = shmget(shm_key, SHM_SIZE, IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget failed");
        return 1;
    }

    pid_t pid = fork();
    if (pid == 0) {
        // Child process - write to shared memory
        char *shm_ptr = (char*)shmat(shmid, NULL, 0);
        if (shm_ptr == (char*)-1) {
            perror("shmat failed");
            exit(EXIT_FAILURE);
        }

        strcpy(shm_ptr, "Hello from shared memory!");
        shmdt(shm_ptr);
        exit(EXIT_SUCCESS);
    } else {
        // Parent process - read from shared memory
        sleep(1);  // Wait for child to write

        char *shm_ptr = (char*)shmat(shmid, NULL, 0);
```

```
        if (shm_ptr == (char*)-1) {
            perror("shmat failed");
            return 1;
        }

        printf("Received: %s\n", shm_ptr);

        shmdt(shm_ptr);
        shmctl(shmid, IPC_RMID, NULL);  // Remove shared memory
        wait(NULL);
    }

    return 0;
}
```

5. Signal Handling

5.1 Basic Signal Handling

Handling signals with signal() or sigaction():

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void signal_handler(int sig) {
    switch (sig) {
        case SIGINT:
            printf("\nReceived SIGINT (Ctrl+C)\n");
            break;
        case SIGTERM:
            printf("\nReceived SIGTERM\n");
            break;
        default:
            printf("\nReceived signal %d\n", sig);
            break;
    }
}

int main() {
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
```

```
    printf("Process PID: %d\n", getpid());
    printf("Send signals to test handling\n");
    printf("Press Ctrl+C to send SIGINT\n");

    // Infinite loop - wait for signals
    while (1) {
        printf("Working...\n");
        sleep(2);
    }

    return 0;
}
```

5.2 Advanced Signal Handling with sigaction()

More robust signal handling using sigaction():

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

volatile sig_atomic_t signal_received = 0;

void sigint_handler(int sig, siginfo_t *info, void *context) {
    printf("\nReceived SIGINT from PID %d\n", info->si_pid);
    signal_received = 1;
}

void sigusr1_handler(int sig, siginfo_t *info, void *context) {
    printf("\nReceived SIGUSR1\n");
    signal_received = 2;
}

int main() {
    struct sigaction sa_int, sa_usr1;

    // Setup SIGINT handler
    memset(&sa_int, 0, sizeof(sa_int));
    sa_int.sa_sigaction = sigint_handler;
    sa_int.sa_flags = SA_SIGINFO;
    sigaction(SIGINT, &sa_int, NULL);

    // Setup SIGUSR1 handler
    memset(&sa_usr1, 0, sizeof(sa_usr1));
```

```
    sa_usr1.sa_sigaction = sigusr1_handler;
    sa_usr1.sa_flags = SA_SIGINFO;
    sigaction(SIGUSR1, &sa_usr1, NULL);

    printf("Process PID: %d\n", getpid());
    printf("Send SIGINT (Ctrl+C) or SIGUSR1 to test\n");

    while (!signal_received) {
        printf("Working... (signal_received = %d)\n", signal_received);
        sleep(2);
    }

    printf("Exiting due to signal %d\n", signal_received);
    return 0;
}
```

6. Memory Management

6.1 Memory Mapping with mmap()

Using mmap() for file mapping and shared memory:

```
#include <stdio.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    const char *filename = "mapped_file.txt";
    const char *content = "This content is mapped to memory.\n";

    // Create and write to file
    int fd = open(filename, O_CREAT | O_RDWR, 0666);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }

    write(fd, content, strlen(content));

    // Map file to memory
    struct stat sb;
    fstat(fd, &sb);
```

```
    char *mapped = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (mapped == MAP_FAILED) {
        perror("mmap failed");
        close(fd);
        return 1;
    }

    printf("Original content: %s", mapped);

    // Modify mapped content
    strcpy(mapped, "Modified content through memory mapping!\n");

    // Unmap and cleanup
    munmap(mapped, sb.st_size);
    close(fd);

    // Verify changes
    fd = open(filename, O_RDONLY);
    char buffer[100];
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';
        printf("File content after modification: %s", buffer);
    }

    close(fd);
    unlink(filename);
    return 0;
}
```

6.2 Memory Protection

Setting memory protection with mprotect():

```
#include <stdio.h>
#include <sys/mman.h>
#include <stdlib.h>
#include <signal.h>
#include <setjmp.h>

static jmp_buf jmp_env;

void segv_handler(int sig) {
    printf("Segmentation fault caught!\n");
    longjmp(jmp_env, 1);
```

```
}

int main() {
    // Allocate memory with mmap
    size_t page_size = getpagesize();
    char *memory = mmap(NULL, page_size, PROT_READ | PROT_WRITE,
                        MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

    if (memory == MAP_FAILED) {
        perror("mmap failed");
        return 1;
    }

    // Set up signal handler
    signal(SIGSEGV, segv_handler);

    // Write to memory (should work)
    strcpy(memory, "Hello, protected memory!");
    printf("Before protection: %s\n", memory);

    // Set memory to read-only
    if (mprotect(memory, page_size, PROT_READ) == -1) {
        perror("mprotect failed");
        munmap(memory, page_size);
        return 1;
    }

    // Try to write to protected memory
    if (setjmp(jmp_env) == 0) {
        strcpy(memory, "This should fail!");
        printf("Write succeeded (unexpected)\n");
    } else {
        printf("Write to protected memory failed as expected\n");
    }

    printf("After protection: %s\n", memory);

    // Cleanup
    munmap(memory, page_size);
    return 0;
}
```

7. Security Considerations

7.1 Secure Programming Practices

1. **Input Validation**: Always validate and sanitize input
2. **Buffer Overflow Prevention**: Use safe string functions
3. **Privilege Management**: Drop privileges when not needed
4. **Secure File Operations**: Set appropriate permissions
5. **Environment Sanitization**: Clear sensitive environment variables

7.2 Example of Secure File Handling

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <errno.h>
#include <string.h>

int create_secure_file(const char *filename) {
    // Create file with restrictive permissions
    int fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0600);
    if (fd == -1) {
        return -1;
    }

    // Verify file was created with correct permissions
    struct stat st;
    if (fstat(fd, &st) == -1) {
        close(fd);
        unlink(filename);
        return -1;
    }

    if ((st.st_mode & 0777) != 0600) {
        close(fd);
        unlink(filename);
        errno = EACCES;
        return -1;
    }

    return fd;
}

int main() {
    const char *filename = "secure_file.txt";
```

```c
    int fd = create_secure_file(filename);

    if (fd == -1) {
        perror("Failed to create secure file");
        return 1;
    }

    const char *data = "Sensitive data";
    write(fd, data, strlen(data));
    close(fd);

    printf("Secure file created successfully\n");

    // Verify permissions
    struct stat st;
    if (stat(filename, &st) == 0) {
        printf("File permissions: %o\n", st.st_mode & 0777);
    }

    unlink(filename);
    return 0;
}
```

8. Best Practices

8.1 Error Handling

1. Check all system call return values
2. Use errno for detailed error information
3. Implement proper cleanup on error
4. Log errors appropriately

8.2 Resource Management

1. Always close file descriptors
2. Free allocated memory
3. Remove temporary files
4. Detach shared memory segments

8.3 Portability

1. Use feature test macros
2. Handle platform differences
3. Avoid non-standard extensions

4. **Test on multiple platforms**

Summary

System programming in C requires a deep understanding of operating system concepts and careful attention to detail. Key areas covered in this module include:

- Process management with fork(), exec(), and wait()
- Advanced file system operations with stat() and directory functions
- Inter-process communication through pipes, FIFOs, and shared memory
- Signal handling for process control
- Memory management with mmap() and mprotect()
- Security considerations for robust system code

Mastering system programming enables developers to create efficient, low-level applications that interact directly with the operating system. Always prioritize security, robustness, and proper error handling when writing system-level code.

Module 15.3: Domain-Specific Applications in C

Introduction

Domain-specific applications are software solutions designed to address particular problem domains with specialized requirements. In C programming, developing domain-specific applications requires leveraging the language's low-level capabilities while implementing domain-specific logic and algorithms. This module explores the development of applications in various domains including embedded systems, scientific computing, networking, and real-time systems.

Learning Objectives

By the end of this module, you will be able to: - Understand the characteristics of domain-specific applications - Design and implement applications for specific domains - Apply domain-specific algorithms and data structures - Optimize performance for domain requirements - Handle domain-specific constraints and requirements - Integrate with domain-specific hardware or protocols

1. Characteristics of Domain-Specific Applications

1.1 Domain Requirements

Domain-specific applications have unique requirements that distinguish them from general-purpose software:

1. **Performance Constraints**: Real-time systems require deterministic response times
2. **Resource Limitations**: Embedded systems often have limited memory and processing power
3. **Reliability Requirements**: Safety-critical systems demand high reliability
4. **Domain-Specific Logic**: Specialized algorithms and data structures
5. **Integration Needs**: Interface with specific hardware or protocols

1.2 Design Considerations

When designing domain-specific applications in C:

1. **Modularity**: Separate domain logic from infrastructure code
2. **Abstraction**: Hide domain complexity behind clean interfaces
3. **Optimization**: Focus on domain-critical performance paths
4. **Testing**: Implement domain-specific test scenarios
5. **Documentation**: Provide domain-specific context and examples

2. Embedded Systems Applications

2.1 Microcontroller Programming

Programming for microcontrollers requires careful resource management:

```
#include <stdint.h>
#include <stdbool.h>

// Hardware register definitions (example for ARM Cortex-M)
#define GPIO_BASE_ADDR    0x40020000
#define GPIO_MODER_REG    (*(volatile uint32_t*)(GPIO_BASE_ADDR + 0x00))
#define GPIO_ODR_REG      (*(volatile uint32_t*)(GPIO_BASE_ADDR + 0x14))

// GPIO pin configuration
typedef enum {
    GPIO_MODE_INPUT = 0,
    GPIO_MODE_OUTPUT = 1,
    GPIO_MODE_AF = 2,
    GPIO_MODE_ANALOG = 3
} gpio_mode_t;

typedef enum {
    GPIO_SPEED_LOW = 0,
    GPIO_SPEED_MEDIUM = 1,
    GPIO_SPEED_HIGH = 2,
    GPIO_SPEED_VERY_HIGH = 3
} gpio_speed_t;

typedef struct {
    uint8_t pin;
    gpio_mode_t mode;
    gpio_speed_t speed;
    bool pull_up;
    bool pull_down;
} gpio_config_t;

// GPIO driver functions
void gpio_init(const gpio_config_t *config) {
    // Configure pin mode
    uint32_t mode_mask = 0x3 << (config->pin * 2);
    GPIO_MODER_REG = (GPIO_MODER_REG & ~mode_mask) |
                     ((config->mode & 0x3) << (config->pin * 2));

    // Configure output speed (simplified)
    // Additional configuration for pull-up/pull-down would go here
}
```

```
void gpio_write(uint8_t pin, bool value) {
    if (value) {
        GPIO_ODR_REG |= (1 << pin);
    } else {
        GPIO_ODR_REG &= ~(1 << pin);
    }
}

bool gpio_read(uint8_t pin) {
    return (GPIO_ODR_REG & (1 << pin)) != 0;
}

// Example application: LED blinker
void led_blinker_task(void) {
    static uint32_t counter = 0;
    const uint32_t BLINK_PERIOD = 1000;  // ms

    gpio_config_t led_config = {
        .pin = 13,  // Assuming LED on pin 13
        .mode = GPIO_MODE_OUTPUT,
        .speed = GPIO_SPEED_LOW,
        .pull_up = false,
        .pull_down = false
    };

    gpio_init(&led_config);

    // Main loop (typically called by RTOS or timer interrupt)
    while (1) {
        if (counter % BLINK_PERIOD == 0) {
            static bool led_state = false;
            led_state = !led_state;
            gpio_write(13, led_state);
        }
        counter++;

        // In a real system, this would be a delay function
        // or the function would be called periodically
    }
}
```

2.2 Real-Time Operating System Integration

Integrating with RTOS for deterministic behavior:

```c
#include <stdint.h>
#include <stddef.h>

// Simplified RTOS API (similar to FreeRTOS)
typedef void* TaskHandle_t;
typedef void* QueueHandle_t;
typedef void* SemaphoreHandle_t;

// Task function prototype
typedef void (*TaskFunction_t)(void* pvParameters);

// RTOS function declarations
TaskHandle_t xTaskCreate(TaskFunction_t pvTaskCode,
                         const char * const pcName,
                         const uint16_t usStackDepth,
                         void * const pvParameters,
                         uint32_t uxPriority,
                         TaskHandle_t * const pxCreatedTask);

QueueHandle_t xQueueCreate(uint32_t uxQueueLength,
                           uint32_t uxItemSize);

uint32_t xQueueSend(QueueHandle_t xQueue,
                    const void * const pvItemToQueue,
                    uint32_t xTicksToWait);

uint32_t xQueueReceive(QueueHandle_t xQueue,
                       void * const pvBuffer,
                       uint32_t xTicksToWait);

SemaphoreHandle_t xSemaphoreCreateBinary(void);
void xSemaphoreTake(SemaphoreHandle_t xSemaphore, uint32_t xBlockTime);
void xSemaphoreGive(SemaphoreHandle_t xSemaphore);

// Domain-specific application: Sensor data processing
typedef struct {
    uint16_t temperature;
    uint16_t humidity;
    uint32_t timestamp;
} sensor_data_t;

// Task handles
TaskHandle_t sensor_task_handle;
TaskHandle_t processing_task_handle;
TaskHandle_t communication_task_handle;

// Communication queue
```

```
QueueHandle_t sensor_queue;

// Synchronization semaphore
SemaphoreHandle_t data_ready_semaphore;

// Sensor reading task
void sensor_task(void* pvParameters) {
    sensor_data_t sensor_data;

    while (1) {
        // Simulate sensor reading
        sensor_data.temperature = 250 + (rand() % 20);  // 25.0°C ± 1.0°C
        sensor_data.humidity = 500 + (rand() % 100);    // 50.0% ± 5.0%
        sensor_data.timestamp = 0;  // In real system, use actual timestamp

        // Send data to processing task
        if (xQueueSend(sensor_queue, &sensor_data, 100) != 0) {
            // Handle queue full error
        }

        // Signal that data is ready
        xSemaphoreGive(data_ready_semaphore);

        // Delay for 1 second (1000ms)
        // In real RTOS, use vTaskDelay() or similar
    }
}

// Data processing task
void processing_task(void* pvParameters) {
    sensor_data_t sensor_data;

    while (1) {
        // Wait for data to be ready
        xSemaphoreTake(data_ready_semaphore, 0xFFFFFFFF);  // Wait indefinitely

        // Receive sensor data
        if (xQueueReceive(sensor_queue, &sensor_data, 0) == 0) {
            // Process sensor data
            float temp_celsius = sensor_data.temperature / 10.0f;
            float humidity_percent = sensor_data.humidity / 10.0f;

            // Apply calibration or filtering if needed
            // ... processing logic ...

            // Forward processed data to communication task
            // ... communication logic ...
```

```
        }
    }
}

// Application initialization
void application_init(void) {
    // Create communication queue
    sensor_queue = xQueueCreate(10, sizeof(sensor_data_t));

    // Create synchronization semaphore
    data_ready_semaphore = xSemaphoreCreateBinary();

    // Create tasks
    sensor_task_handle = xTaskCreate(sensor_task,
                                     "SensorTask",
                                     256,    // Stack size
                                     NULL,   // Parameters
                                     1,      // Priority
                                     NULL);  // Task handle

    processing_task_handle = xTaskCreate(processing_task,
                                         "ProcessingTask",
                                         512,    // Stack size
                                         NULL,   // Parameters
                                         2,      // Priority
                                         NULL);  // Task handle

    // In a real system, you would also create communication task
    // and start the scheduler
}
```

3. Scientific Computing Applications

3.1 Numerical Libraries Integration

Integrating with numerical libraries for scientific computing:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

// Matrix operations for scientific computing
typedef struct {
    size_t rows;
```

```
    size_t cols;
    double *data;
} matrix_t;

// Create matrix
matrix_t* matrix_create(size_t rows, size_t cols) {
    matrix_t *mat = malloc(sizeof(matrix_t));
    if (!mat) return NULL;

    mat->rows = rows;
    mat->cols = cols;
    mat->data = calloc(rows * cols, sizeof(double));

    if (!mat->data) {
        free(mat);
        return NULL;
    }

    return mat;
}

// Free matrix
void matrix_destroy(matrix_t *mat) {
    if (mat) {
        free(mat->data);
        free(mat);
    }
}

// Get element at (row, col)
double matrix_get(const matrix_t *mat, size_t row, size_t col) {
    if (row >= mat->rows || col >= mat->cols) return 0.0;
    return mat->data[row * mat->cols + col];
}

// Set element at (row, col)
void matrix_set(matrix_t *mat, size_t row, size_t col, double value) {
    if (row < mat->rows && col < mat->cols) {
        mat->data[row * mat->cols + col] = value;
    }
}

// Matrix multiplication
matrix_t* matrix_multiply(const matrix_t *a, const matrix_t *b) {
    if (a->cols != b->rows) return NULL;

    matrix_t *result = matrix_create(a->rows, b->cols);
```

```
    if (!result) return NULL;

    for (size_t i = 0; i < a->rows; i++) {
        for (size_t j = 0; j < b->cols; j++) {
            double sum = 0.0;
            for (size_t k = 0; k < a->cols; k++) {
                sum += matrix_get(a, i, k) * matrix_get(b, k, j);
            }
            matrix_set(result, i, j, sum);
        }
    }

    return result;
}

// Solve linear system using Gaussian elimination
int matrix_solve_linear_system(matrix_t *a, matrix_t *b) {
    if (a->rows != a->cols || a->rows != b->rows || b->cols != 1) {
        return -1;  // Invalid dimensions
    }

    size_t n = a->rows;

    // Forward elimination
    for (size_t i = 0; i < n; i++) {
        // Find pivot
        size_t max_row = i;
        for (size_t k = i + 1; k < n; k++) {
            if (fabs(matrix_get(a, k, i)) > fabs(matrix_get(a, max_row, i))) {
                max_row = k;
            }
        }

        // Swap rows if needed
        if (max_row != i) {
            for (size_t j = 0; j < n; j++) {
                double temp = matrix_get(a, i, j);
                matrix_set(a, i, j, matrix_get(a, max_row, j));
                matrix_set(a, max_row, j, temp);
            }
            double temp = matrix_get(b, i, 0);
            matrix_set(b, i, 0, matrix_get(b, max_row, 0));
            matrix_set(b, max_row, 0, temp);
        }

        // Check for singular matrix
```

```
        if (fabs(matrix_get(a, i, i)) < 1e-10) {
            return -2;  // Singular matrix
        }

        // Eliminate column
        for (size_t k = i + 1; k < n; k++) {
            double factor = matrix_get(a, k, i) / matrix_get(a, i, i);
            for (size_t j = i; j < n; j++) {
                double new_value = matrix_get(a, k, j) - factor * matrix_get(a, i, j);
                matrix_set(a, k, j, new_value);
            }
            double new_b = matrix_get(b, k, 0) - factor * matrix_get(b, i, 0);
            matrix_set(b, k, 0, new_b);
        }
    }

    // Back substitution
    for (int i = n - 1; i >= 0; i--) {
        double sum = matrix_get(b, i, 0);
        for (size_t j = i + 1; j < n; j++) {
            sum -= matrix_get(a, i, j) * matrix_get(b, j, 0);
        }
        matrix_set(b, i, 0, sum / matrix_get(a, i, i));
    }

    return 0;  // Success
}

// Example: Solve a system of linear equations
void example_linear_system(void) {
    // Solve:
    // 2x + 3y + z = 1
    // x + 4y + 2z = 2
    // 3x + y + 5z = 3

    matrix_t *a = matrix_create(3, 3);
    matrix_t *b = matrix_create(3, 1);

    if (!a || !b) {
        printf("Memory allocation failed\n");
        return;
    }

    // Coefficient matrix A
    matrix_set(a, 0, 0, 2.0); matrix_set(a, 0, 1, 3.0); matrix_set(a, 0, 2, 1.0);
    matrix_set(a, 1, 0, 1.0); matrix_set(a, 1, 1, 4.0); matrix_set(a, 1, 2, 2.0);
    matrix_set(a, 2, 0, 3.0); matrix_set(a, 2, 1, 1.0); matrix_set(a, 2, 2, 5.0);
```

```
    // Constants vector B
    matrix_set(b, 0, 0, 1.0);
    matrix_set(b, 1, 0, 2.0);
    matrix_set(b, 2, 0, 3.0);

    printf("Solving linear system:\n");
    printf("2x + 3y + z = 1\n");
    printf("x + 4y + 2z = 2\n");
    printf("3x + y + 5z = 3\n\n");

    int result = matrix_solve_linear_system(a, b);

    if (result == 0) {
        printf("Solution:\n");
        printf("x = %.6f\n", matrix_get(b, 0, 0));
        printf("y = %.6f\n", matrix_get(b, 1, 0));
        printf("z = %.6f\n", matrix_get(b, 2, 0));
    } else {
        printf("Failed to solve system (error code: %d)\n", result);
    }

    matrix_destroy(a);
    matrix_destroy(b);
}
```

3.2 Fast Fourier Transform (FFT)

Implementing FFT for signal processing:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <complex.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

// Complex number type
typedef double complex complex_t;

// FFT implementation using Cooley-Tukey algorithm
void fft_recursive(complex_t *x, size_t n, size_t step) {
    if (step < n) {
        fft_recursive(x, n, step * 2);
```

```
        fft_recursive(x + step, n, step * 2);

        for (size_t i = 0; i < n; i += step * 2) {
            complex_t t = cexp(-I * M_PI * i / n) * x[i + step];
            x[i + step] = x[i] - t;
            x[i] += t;
        }
    }
}

void fft(complex_t *x, size_t n) {
    // Make sure n is a power of 2
    if (n & (n - 1)) {
        fprintf(stderr, "FFT size must be a power of 2\n");
        return;
    }

    // Bit-reversal permutation
    size_t j = 0;
    for (size_t i = 0; i < n; i++) {
        if (i < j) {
            complex_t temp = x[i];
            x[i] = x[j];
            x[j] = temp;
        }

        size_t k = n >> 1;
        while (k <= j) {
            j -= k;
            k >>= 1;
        }
        j += k;
    }

    // Recursive FFT
    fft_recursive(x, n, 1);
}

// Inverse FFT
void ifft(complex_t *x, size_t n) {
    // Conjugate the input
    for (size_t i = 0; i < n; i++) {
        x[i] = conj(x[i]);
    }

    // Apply FFT
    fft(x, n);
```

```
    // Conjugate and normalize
    for (size_t i = 0; i < n; i++) {
        x[i] = conj(x[i]) / n;
    }
}

// Generate a test signal
void generate_test_signal(complex_t *signal, size_t n, double frequency) {
    for (size_t i = 0; i < n; i++) {
        double t = (double)i / n;
        // Composite signal: sum of sine waves
        double value = sin(2 * M_PI * frequency * t) +
                       0.5 * sin(2 * M_PI * 3 * frequency * t) +
                       0.3 * sin(2 * M_PI * 5 * frequency * t);
        signal[i] = value + 0.0 * I;  // Real signal
    }
}

// Example: FFT analysis of a signal
void example_fft_analysis(void) {
    const size_t N = 64;  // Must be power of 2
    const double sample_rate = 1000.0;  // Hz
    const double signal_frequency = 50.0;  // Hz

    complex_t *signal = malloc(N * sizeof(complex_t));
    if (!signal) {
        fprintf(stderr, "Memory allocation failed\n");
        return;
    }

    // Generate test signal
    generate_test_signal(signal, N, signal_frequency);

    printf("Original signal samples:\n");
    for (size_t i = 0; i < 8; i++) {  // Show first 8 samples
        printf("Sample[%zu] = %.3f\n", i, creal(signal[i]));
    }

    // Apply FFT
    fft(signal, N);

    printf("\nFrequency domain representation:\n");
    printf("Frequency (Hz)\tMagnitude\tPhase (rad)\n");

    // Analyze frequency components
    for (size_t i = 0; i <= N/2; i++) {
```

```
        double frequency = i * sample_rate / N;
        double magnitude = cabs(signal[i]) / N;
        double phase = carg(signal[i]);

        // Only show significant components
        if (magnitude > 0.01) {
            printf("%.1f\t\t%.3f\t\t%.3f\n", frequency, magnitude, phase);
        }
    }

    // Apply inverse FFT to verify
    ifft(signal, N);

    printf("\nReconstructed signal (first 8 samples):\n");
    for (size_t i = 0; i < 8; i++) {
        printf("Sample[%zu] = %.3f\n", i, creal(signal[i]));
    }

    free(signal);
}
```

4. Networking Applications

4.1 Protocol Implementation

Implementing custom network protocols:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <arpa/inet.h>

// Custom protocol packet structure
typedef struct {
    uint16_t magic;        // Protocol identifier
    uint16_t version;      // Protocol version
    uint32_t sequence;     // Packet sequence number
    uint32_t timestamp;    // Timestamp (seconds since epoch)
    uint16_t type;         // Packet type
    uint16_t length;       // Payload length
    uint8_t payload[0];    // Variable length payload
} __attribute__((packed)) protocol_packet_t;

#define PROTOCOL_MAGIC 0xCAFE
```

```c
#define PROTOCOL_VERSION 1

// Packet types
#define PACKET_TYPE_DATA 1
#define PACKET_TYPE_ACK 2
#define PACKET_TYPE_NACK 3
#define PACKET_TYPE_HEARTBEAT 4

// Protocol error codes
typedef enum {
    PROTOCOL_SUCCESS = 0,
    PROTOCOL_ERROR_INVALID_MAGIC = -1,
    PROTOCOL_ERROR_INVALID_VERSION = -2,
    PROTOCOL_ERROR_BUFFER_TOO_SMALL = -3,
    PROTOCOL_ERROR_INVALID_LENGTH = -4
} protocol_error_t;

// Create a protocol packet
protocol_packet_t* protocol_create_packet(uint16_t type,
                                          const void *payload,
                                          uint16_t payload_length) {
    size_t packet_size = sizeof(protocol_packet_t) + payload_length;
    protocol_packet_t *packet = malloc(packet_size);

    if (!packet) {
        return NULL;
    }

    // Initialize packet header
    packet->magic = htons(PROTOCOL_MAGIC);
    packet->version = htons(PROTOCOL_VERSION);
    packet->sequence = 0;  // Would be set by sequence manager
    packet->timestamp = htonl((uint32_t)time(NULL));
    packet->type = htons(type);
    packet->length = htons(payload_length);

    // Copy payload
    if (payload && payload_length > 0) {
        memcpy(packet->payload, payload, payload_length);
    }

    return packet;
}

// Validate a received packet
protocol_error_t protocol_validate_packet(const protocol_packet_t *packet,
                                          size_t buffer_size) {
```

```
    // Check buffer size
    if (buffer_size < sizeof(protocol_packet_t)) {
        return PROTOCOL_ERROR_BUFFER_TOO_SMALL;
    }

    // Check magic number
    if (ntohs(packet->magic) != PROTOCOL_MAGIC) {
        return PROTOCOL_ERROR_INVALID_MAGIC;
    }

    // Check version
    if (ntohs(packet->version) != PROTOCOL_VERSION) {
        return PROTOCOL_ERROR_INVALID_VERSION;
    }

    // Check payload length
    uint16_t payload_length = ntohs(packet->length);
    size_t total_size = sizeof(protocol_packet_t) + payload_length;

    if (buffer_size < total_size) {
        return PROTOCOL_ERROR_BUFFER_TOO_SMALL;
    }

    return PROTOCOL_SUCCESS;
}

// Parse packet payload based on type
void protocol_parse_payload(const protocol_packet_t *packet) {
    uint16_t type = ntohs(packet->type);
    uint16_t length = ntohs(packet->length);

    printf("Packet Type: %u, Length: %u\n", type, length);

    switch (type) {
        case PACKET_TYPE_DATA:
            printf("Data packet received: ");
            if (length > 0) {
                // Print first 32 bytes of payload as hex
                for (int i = 0; i < length && i < 32; i++) {
                    printf("%02x ", packet->payload[i]);
                }
                if (length > 32) printf("...");
            }
            printf("\n");
            break;

        case PACKET_TYPE_ACK:
```

```
            printf("Acknowledgment packet\n");
            break;

        case PACKET_TYPE_NACK:
            printf("Negative acknowledgment packet\n");
            break;

        case PACKET_TYPE_HEARTBEAT:
            printf("Heartbeat packet\n");
            break;

        default:
            printf("Unknown packet type\n");
            break;
    }
}

// Example: Protocol usage
void example_protocol_usage(void) {
    printf("Creating protocol packets...\n");

    // Create data packet
    const char *message = "Hello, Protocol World!";
    protocol_packet_t *data_packet = protocol_create_packet(
        PACKET_TYPE_DATA, message, strlen(message));

    if (data_packet) {
        printf("Created data packet:\n");
        protocol_parse_payload(data_packet);

        // Validate the packet
        protocol_error_t result = protocol_validate_packet(
            data_packet, sizeof(protocol_packet_t) + strlen(message));

        if (result == PROTOCOL_SUCCESS) {
            printf("Packet validation successful\n");
        } else {
            printf("Packet validation failed: %d\n", result);
        }

        free(data_packet);
    }

    // Create heartbeat packet
    protocol_packet_t *heartbeat_packet = protocol_create_packet(
        PACKET_TYPE_HEARTBEAT, NULL, 0);
```

```
    if (heartbeat_packet) {
        printf("\nCreated heartbeat packet:\n");
        protocol_parse_payload(heartbeat_packet);
        free(heartbeat_packet);
    }
}
```

4.2 HTTP Client Implementation

Simple HTTP client for web services:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

// HTTP request structure
typedef struct {
    char method[16];
    char host[256];
    int port;
    char path[512];
    char *headers;
    char *body;
    size_t body_length;
} http_request_t;

// HTTP response structure
typedef struct {
    int status_code;
    char status_text[64];
    char *headers;
    char *body;
    size_t body_length;
} http_response_t;

// Create HTTP request
http_request_t* http_create_request(const char *method,
                                    const char *host,
                                    int port,
                                    const char *path) {
    http_request_t *req = malloc(sizeof(http_request_t));
```

```
    if (!req) return NULL;

    strncpy(req->method, method, sizeof(req->method) - 1);
    strncpy(req->host, host, sizeof(req->host) - 1);
    req->port = port;
    strncpy(req->path, path, sizeof(req->path) - 1);

    req->headers = NULL;
    req->body = NULL;
    req->body_length = 0;

    return req;
}

// Add header to request
void http_add_header(http_request_t *req, const char *header) {
    size_t current_len = req->headers ? strlen(req->headers) : 0;
    size_t header_len = strlen(header);
    size_t new_len = current_len + header_len + 3;  // +3 for \r\n\0

    char *new_headers = realloc(req->headers, new_len);
    if (!new_headers) return;

    req->headers = new_headers;

    if (current_len == 0) {
        strcpy(req->headers, header);
    } else {
        strcat(req->headers, "\r\n");
        strcat(req->headers, header);
    }
}

// Set request body
void http_set_body(http_request_t *req, const char *body, size_t length) {
    req->body = malloc(length);
    if (req->body) {
        memcpy(req->body, body, length);
        req->body_length = length;
    }
}

// Build HTTP request string
char* http_build_request_string(const http_request_t *req) {
    size_t req_size = strlen(req->method) + strlen(req->path) + 16;  // Method + Path + HTTP
    req_size += strlen(req->host) + 16;  // Host header
```

```
    if (req->headers) {
        req_size += strlen(req->headers) + 2;  // Headers + \r\n
    }

    if (req->body) {
        req_size += req->body_length + 16;  // Content-Length header
    }

    req_size += 32;  // Extra space for safety

    char *request = malloc(req_size);
    if (!request) return NULL;

    // Build request line
    snprintf(request, req_size, "%s %s HTTP/1.1\r\n", req->method, req->path);

    // Add Host header
    char host_header[300];
    snprintf(host_header, sizeof(host_header), "Host: %s:%d", req->host, req->port);
    strcat(request, host_header);
    strcat(request, "\r\n");

    // Add custom headers
    if (req->headers) {
        strcat(request, req->headers);
        strcat(request, "\r\n");
    }

    // Add Content-Length if body exists
    if (req->body) {
        char content_length[32];
        snprintf(content_length, sizeof(content_length), "Content-Length: %zu\r\n", req->bod
        strcat(request, content_length);
    }

    // End headers
    strcat(request, "\r\n");

    // Add body
    if (req->body) {
        size_t current_len = strlen(request);
        memcpy(request + current_len, req->body, req->body_length);
        request[current_len + req->body_length] = '\0';
    }

    return request;
}
```

```c
// Simple HTTP GET request
int http_get(const char *url, char **response_body, size_t *response_length) {
    // Parse URL (simplified - assumes http://host:port/path format)
    if (strncmp(url, "http://", 7) != 0) {
        fprintf(stderr, "Only HTTP URLs supported\n");
        return -1;
    }

    const char *host_start = url + 7;
    const char *port_start = strchr(host_start, ':');
    const char *path_start = strchr(host_start, '/');

    if (!path_start) {
        fprintf(stderr, "Invalid URL format\n");
        return -1;
    }

    char host[256];
    int port = 80;

    if (port_start && port_start < path_start) {
        // Extract host
        size_t host_len = port_start - host_start;
        if (host_len >= sizeof(host)) {
            fprintf(stderr, "Host name too long\n");
            return -1;
        }
        strncpy(host, host_start, host_len);
        host[host_len] = '\0';

        // Extract port
        port = atoi(port_start + 1);
    } else {
        // Extract host only
        size_t host_len = path_start - host_start;
        if (host_len >= sizeof(host)) {
            fprintf(stderr, "Host name too long\n");
            return -1;
        }
        strncpy(host, host_start, host_len);
        host[host_len] = '\0';
    }

    const char *path = path_start;

    // Create socket
```

```c
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket creation failed");
        return -1;
    }

    // Resolve hostname
    struct hostent *server = gethostbyname(host);
    if (!server) {
        fprintf(stderr, "Failed to resolve hostname: %s\n", host);
        close(sock);
        return -1;
    }

    // Connect to server
    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    memcpy(&server_addr.sin_addr.s_addr, server->h_addr, server->h_length);
    server_addr.sin_port = htons(port);

    if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("connection failed");
        close(sock);
        return -1;
    }

    // Create HTTP request
    http_request_t *req = http_create_request("GET", host, port, path);
    if (!req) {
        close(sock);
        return -1;
    }

    http_add_header(req, "Connection: close");
    http_add_header(req, "User-Agent: C-HTTP-Client/1.0");

    char *request_string = http_build_request_string(req);
    if (!request_string) {
        free(req);
        close(sock);
        return -1;
    }

    // Send request
    if (send(sock, request_string, strlen(request_string), 0) < 0) {
        perror("send failed");
```

```
        free(request_string);
        free(req);
        close(sock);
        return -1;
    }

    // Receive response
    char buffer[4096];
    ssize_t bytes_received;
    char *full_response = NULL;
    size_t total_received = 0;

    while ((bytes_received = recv(sock, buffer, sizeof(buffer) - 1, 0)) > 0) {
        char *new_response = realloc(full_response, total_received + bytes_received + 1);
        if (!new_response) {
            free(full_response);
            free(request_string);
            free(req);
            close(sock);
            return -1;
        }

        full_response = new_response;
        memcpy(full_response + total_received, buffer, bytes_received);
        total_received += bytes_received;
        full_response[total_received] = '\0';
    }

    close(sock);
    free(request_string);
    free(req);

    if (bytes_received < 0) {
        perror("recv failed");
        free(full_response);
        return -1;
    }

    // Parse response (simplified)
    *response_body = full_response;
    *response_length = total_received;

    return 0;
}

// Example: HTTP client usage
void example_http_client(void) {
```

```
    printf("Making HTTP GET request...\n");

    char *response_body;
    size_t response_length;

    // Make a simple HTTP request
    int result = http_get("http://httpbin.org/get", &response_body, &response_length);

    if (result == 0) {
        printf("HTTP request successful\n");
        printf("Response length: %zu bytes\n", response_length);

        // Print first 500 characters of response
        size_t print_length = (response_length < 500) ? response_length : 500;
        printf("Response (first %zu chars):\n", print_length);
        fwrite(response_body, 1, print_length, stdout);
        printf("\n");

        free(response_body);
    } else {
        printf("HTTP request failed\n");
    }
}
```

5. Best Practices for Domain-Specific Applications

5.1 Domain Analysis

1. **Requirements Gathering**: Understand domain-specific constraints and requirements
2. **Performance Analysis**: Identify critical performance paths
3. **Resource Constraints**: Account for memory, CPU, and I/O limitations
4. **Safety Considerations**: Implement appropriate error handling and recovery

5.2 Design Patterns

1. **Modular Architecture**: Separate domain logic from infrastructure
2. **Abstraction Layers**: Hide domain complexity behind clean interfaces
3. **Configuration Management**: Support domain-specific configuration
4. **Extensibility**: Design for future domain requirements

5.3 Testing Strategies

1. **Domain-Specific Tests**: Implement tests that validate domain requirements
2. **Edge Case Testing**: Test boundary conditions specific to the domain

3. **Performance Testing**: Validate performance under domain-specific loads
4. **Integration Testing**: Test integration with domain-specific systems

Summary

Domain-specific applications in C require a deep understanding of both the programming language and the target domain. Key concepts covered in this module include:

- Characteristics and requirements of domain-specific applications
- Embedded systems programming with microcontroller integration
- Scientific computing with numerical libraries and algorithms
- Networking applications with custom protocols and HTTP clients
- Best practices for design, implementation, and testing

Developing domain-specific applications in C allows for highly optimized, efficient solutions that can take full advantage of system resources while meeting specialized requirements. Success in this area requires both strong C programming skills and domain expertise.

Module 15.4: Capstone Project

Introduction

The capstone project represents the culmination of your C programming journey, integrating all the concepts, techniques, and best practices you've learned throughout this course. This project challenges you to design, implement, and document a substantial application that demonstrates mastery of advanced C programming concepts while solving a real-world problem.

Learning Objectives

By completing this capstone project, you will demonstrate the ability to: - Design and implement a complex, multi-module C application - Apply advanced programming techniques including multithreading, memory management, and system programming - Integrate domain-specific knowledge with C programming skills - Follow professional software development practices including documentation, testing, and version control - Debug and optimize complex applications - Present technical work effectively

Project Requirements

1. Technical Requirements

Your capstone project must include:

1. **Multi-file Structure**: Organized into logical modules with proper header files
2. **Advanced C Features**: Use of at least three advanced C concepts (e.g., multithreading, memory mapping, signal handling, etc.)
3. **Error Handling**: Comprehensive error handling with appropriate recovery mechanisms
4. **Memory Management**: Proper dynamic memory allocation and deallocation
5. **Input/Output**: Robust file I/O and/or network communication
6. **Data Structures**: Implementation of appropriate data structures for the problem domain
7. **Testing**: Unit tests and integration tests for critical components
8. **Documentation**: Complete documentation including user guide and API reference

2. Project Scope

The project should be substantial enough to demonstrate advanced skills but manageable within the timeframe. Consider projects that:

- Solve a real-world problem or fill a gap in existing tools
- Demonstrate integration of multiple concepts from the course
- Include both algorithmic complexity and systems programming aspects
- Have clear requirements and measurable outcomes

3. Deliverables

1. **Source Code**: Complete, well-organized C source code
2. **Documentation**:
    - README with project overview, installation, and usage instructions
    - Technical documentation explaining design decisions
    - API documentation for any libraries or modules created
3. **Testing**: Test suite with clear test cases and expected outcomes
4. **Presentation**: A presentation (written or oral) explaining the project
5. **Reflection**: A written reflection on the development process and lessons learned

Project Ideas

1. System Monitoring Tool

Create a comprehensive system monitoring application that collects and displays system metrics:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <sys/sysinfo.h>
#include <sys/statvfs.h>

// Data structures for system metrics
typedef struct {
    double cpu_usage;
    unsigned long memory_total;
    unsigned long memory_free;
    unsigned long memory_available;
    double load_average[3];
    time_t timestamp;
} system_metrics_t;
```

```c
typedef struct {
    char filesystem[256];
    unsigned long total_space;
    unsigned long free_space;
    unsigned long available_space;
} disk_metrics_t;

// Thread-safe metrics storage
typedef struct {
    system_metrics_t *metrics;
    size_t count;
    size_t capacity;
    pthread_mutex_t mutex;
} metrics_storage_t;

// Global metrics storage
static metrics_storage_t g_metrics = {
    .metrics = NULL,
    .count = 0,
    .capacity = 0,
    .mutex = PTHREAD_MUTEX_INITIALIZER
};

// Function to collect CPU usage
double get_cpu_usage() {
    static unsigned long long prev_idle = 0;
    static unsigned long long prev_total = 0;

    FILE *fp = fopen("/proc/stat", "r");
    if (!fp) return -1.0;

    char buffer[1024];
    fgets(buffer, sizeof(buffer), fp);
    fclose(fp);

    unsigned long long user, nice, system, idle, iowait, irq, softirq, steal;
    sscanf(buffer, "cpu %llu %llu %llu %llu %llu %llu %llu %llu",
           &user, &nice, &system, &idle, &iowait, &irq, &softirq, &steal);

    unsigned long long total = user + nice + system + idle + iowait + irq + softirq + steal;
    unsigned long long total_idle = idle + iowait;

    double cpu_usage = 0.0;
    if (prev_total != 0) {
        unsigned long long total_diff = total - prev_total;
        unsigned long long idle_diff = total_idle - prev_idle;
```

```
        cpu_usage = 100.0 * (total_diff - idle_diff) / total_diff;
    }

    prev_idle = total_idle;
    prev_total = total;

    return cpu_usage;
}

// Function to collect memory information
void get_memory_info(unsigned long *total, unsigned long *free, unsigned long *available) {
    struct sysinfo si;
    if (sysinfo(&si) == 0) {
        *total = si.totalram * si.mem_unit;
        *free = si.freeram * si.mem_unit;
        *available = si.freeram * si.mem_unit + si.bufferram * si.mem_unit;
    }
}

// Function to collect disk space information
int get_disk_info(const char *path, disk_metrics_t *disk) {
    struct statvfs buf;
    if (statvfs(path, &buf) != 0) {
        return -1;
    }

    strncpy(disk->filesystem, path, sizeof(disk->filesystem) - 1);
    disk->total_space = buf.f_blocks * buf.f_frsize;
    disk->free_space = buf.f_bfree * buf.f_frsize;
    disk->available_space = buf.f_bavail * buf.f_frsize;

    return 0;
}

// Function to collect system load average
void get_load_average(double loadavg[3]) {
    FILE *fp = fopen("/proc/loadavg", "r");
    if (fp) {
        fscanf(fp, "%lf %lf %lf", &loadavg[0], &loadavg[1], &loadavg[2]);
        fclose(fp);
    }
}

// Metrics collection thread function
void* metrics_collection_thread(void* arg) {
    while (1) {
```

```
        system_metrics_t metrics = {0};
        metrics.timestamp = time(NULL);

        // Collect metrics
        metrics.cpu_usage = get_cpu_usage();
        get_memory_info(&metrics.memory_total, &metrics.memory_free, &metrics.memory_availab
        get_load_average(metrics.load_average);

        // Store metrics safely
        pthread_mutex_lock(&g_metrics.mutex);

        // Expand storage if needed
        if (g_metrics.count >= g_metrics.capacity) {
            size_t new_capacity = (g_metrics.capacity == 0) ? 10 : g_metrics.capacity * 2;
            system_metrics_t *new_metrics = realloc(g_metrics.metrics,
                                                    new_capacity * sizeof(system_metrics_t));
            if (new_metrics) {
                g_metrics.metrics = new_metrics;
                g_metrics.capacity = new_capacity;
            }
        }

        // Add new metrics
        if (g_metrics.count < g_metrics.capacity) {
            g_metrics.metrics[g_metrics.count] = metrics;
            g_metrics.count++;
        }

        pthread_mutex_unlock(&g_metrics.mutex);

        // Wait for 5 seconds before next collection
        sleep(5);
    }

    return NULL;
}

// Function to display current metrics
void display_current_metrics() {
    pthread_mutex_lock(&g_metrics.mutex);

    if (g_metrics.count > 0) {
        system_metrics_t *latest = &g_metrics.metrics[g_metrics.count - 1];

        printf("=== System Metrics ===\n");
        printf("Timestamp: %s", ctime(&latest->timestamp));
```

```
        printf("CPU Usage: %.2f%%\n", latest->cpu_usage);
        printf("Memory Total: %lu MB\n", latest->memory_total / (1024 * 1024));
        printf("Memory Free: %lu MB\n", latest->memory_free / (1024 * 1024));
        printf("Memory Available: %lu MB\n", latest->memory_available / (1024 * 1024));
        printf("Load Average: %.2f, %.2f, %.2f\n",
               latest->load_average[0], latest->load_average[1], latest->load_average[2]);

        // Display disk information
        disk_metrics_t disk;
        if (get_disk_info("/", &disk) == 0) {
            printf("Disk (/): %.2f GB total, %.2f GB free, %.2f GB available\n",
                   disk.total_space / (1024.0 * 1024 * 1024),
                   disk.free_space / (1024.0 * 1024 * 1024),
                   disk.available_space / (1024.0 * 1024 * 1024));
        }
    }

    pthread_mutex_unlock(&g_metrics.mutex);
}

// Main function for system monitor
int main_system_monitor() {
    printf("Starting System Monitor...\n");

    // Create metrics collection thread
    pthread_t collector_thread;
    if (pthread_create(&collector_thread, NULL, metrics_collection_thread, NULL) != 0) {
        perror("Failed to create metrics collection thread");
        return 1;
    }

    // Main loop - display metrics and handle user input
    char command[10];
    while (1) {
        printf("\nCommands: [d]isplay metrics, [q]uit: ");
        if (fgets(command, sizeof(command), stdin)) {
            if (command[0] == 'd' || command[0] == 'D') {
                display_current_metrics();
            } else if (command[0] == 'q' || command[0] == 'Q') {
                break;
            }
        }
    }

    // Clean up
    pthread_cancel(collector_thread);
```

```
    pthread_join(collector_thread, NULL);

    if (g_metrics.metrics) {
        free(g_metrics.metrics);
    }

    printf("System Monitor stopped.\n");
    return 0;
}
```

2. Custom Database Engine

Implement a simple embedded database with basic SQL-like functionality:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>

// Data types
typedef enum {
    TYPE_INTEGER,
    TYPE_STRING,
    TYPE_FLOAT
} column_type_t;

// Column definition
typedef struct {
    char name[64];
    column_type_t type;
    size_t offset;
} column_t;

// Table definition
typedef struct {
    char name[64];
    column_t *columns;
    size_t column_count;
    size_t row_size;
    void *data;
    size_t row_count;
    size_t capacity;
} table_t;

// Database structure
```

```
typedef struct {
    table_t *tables;
    size_t table_count;
    size_t capacity;
} database_t;

// Global database instance
static database_t g_db = {0};

// Create a new table
table_t* db_create_table(const char *name, size_t column_count) {
    // Expand tables array if needed
    if (g_db.table_count >= g_db.capacity) {
        size_t new_capacity = (g_db.capacity == 0) ? 4 : g_db.capacity * 2;
        table_t *new_tables = realloc(g_db.tables, new_capacity * sizeof(table_t));
        if (!new_tables) return NULL;

        g_db.tables = new_tables;
        g_db.capacity = new_capacity;
    }

    // Initialize new table
    table_t *table = &g_db.tables[g_db.table_count];
    strncpy(table->name, name, sizeof(table->name) - 1);
    table->columns = calloc(column_count, sizeof(column_t));
    table->column_count = column_count;
    table->row_size = 0;
    table->data = NULL;
    table->row_count = 0;
    table->capacity = 0;

    g_db.table_count++;
    return table;
}

// Add a column to a table
int table_add_column(table_t *table, const char *name, column_type_t type) {
    if (table->column_count >= 64) return -1;  // Max columns reached

    column_t *column = &table->columns[table->column_count - 1];
    strncpy(column->name, name, sizeof(column->name) - 1);

    // Calculate offset based on previous columns
    size_t offset = 0;
    for (size_t i = 0; i < table->column_count - 1; i++) {
        switch (table->columns[i].type) {
            case TYPE_INTEGER:
```

```
                offset = (offset + sizeof(int) - 1) & ~(sizeof(int) - 1);
                offset += sizeof(int);
                break;
            case TYPE_FLOAT:
                offset = (offset + sizeof(double) - 1) & ~(sizeof(double) - 1);
                offset += sizeof(double);
                break;
            case TYPE_STRING:
                offset = (offset + sizeof(char*) - 1) & ~(sizeof(char*) - 1);
                offset += sizeof(char*);
                break;
        }
    }

    column->type = type;
    column->offset = offset;

    // Update row size
    switch (type) {
        case TYPE_INTEGER:
            table->row_size = (offset + sizeof(int) - 1) & ~(sizeof(int) - 1);
            table->row_size += sizeof(int);
            break;
        case TYPE_FLOAT:
            table->row_size = (offset + sizeof(double) - 1) & ~(sizeof(double) - 1);
            table->row_size += sizeof(double);
            break;
        case TYPE_STRING:
            table->row_size = (offset + sizeof(char*) - 1) & ~(sizeof(char*) - 1);
            table->row_size += sizeof(char*);
            break;
    }

    return 0;
}

// Insert a row into a table
int table_insert_row(table_t *table, void *row_data) {
    // Expand data array if needed
    if (table->row_count >= table->capacity) {
        size_t new_capacity = (table->capacity == 0) ? 16 : table->capacity * 2;
        void *new_data = realloc(table->data, new_capacity * table->row_size);
        if (!new_data) return -1;

        table->data = new_data;
        table->capacity = new_capacity;
    }
```

```
    // Copy row data
    void *dest = (char*)table->data + table->row_count * table->row_size;
    memcpy(dest, row_data, table->row_size);
    table->row_count++;

    return 0;
}

// Simple query execution
void db_execute_query(const char *query) {
    printf("Executing query: %s\n", query);

    // Simple parser for demonstration
    if (strncmp(query, "CREATE TABLE", 12) == 0) {
        // Parse table creation
        char table_name[64];
        if (sscanf(query, "CREATE TABLE %63s", table_name) == 1) {
            table_t *table = db_create_table(table_name, 4);  // Max 4 columns for demo
            if (table) {
                printf("Table '%s' created successfully\n", table_name);
            } else {
                printf("Failed to create table '%s'\n", table_name);
            }
        }
    } else if (strncmp(query, "INSERT INTO", 11) == 0) {
        printf("Insert operation executed\n");
    } else if (strncmp(query, "SELECT", 6) == 0) {
        printf("Select query executed\n");
    } else {
        printf("Unsupported query\n");
    }
}

// Database initialization
void db_init() {
    g_db.tables = NULL;
    g_db.table_count = 0;
    g_db.capacity = 0;
    printf("Database initialized\n");
}

// Database cleanup
void db_cleanup() {
    for (size_t i = 0; i < g_db.table_count; i++) {
        table_t *table = &g_db.tables[i];
        free(table->columns);
```

```
        free(table->data);
    }
    free(g_db.tables);
    g_db.tables = NULL;
    g_db.table_count = 0;
    g_db.capacity = 0;
    printf("Database cleaned up\n");
}

// Main function for database engine
int main_database_engine() {
    db_init();

    printf("Custom Database Engine\n");
    printf("Enter SQL-like commands (type 'quit' to exit):\n");

    char query[256];
    while (1) {
        printf("> ");
        if (!fgets(query, sizeof(query), stdin)) break;

        // Remove newline
        query[strcspn(query, "\n")] = 0;

        if (strcmp(query, "quit") == 0) break;

        db_execute_query(query);
    }

    db_cleanup();
    return 0;
}
```

3. Network File Transfer System

Create a secure file transfer system with client-server architecture:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```c
#include <pthread.h>

#define PORT 8080
#define BUFFER_SIZE 4096
#define MAX_FILENAME 256

// Protocol commands
#define CMD_UPLOAD 1
#define CMD_DOWNLOAD 2
#define CMD_LIST 3
#define CMD_QUIT 4

// Protocol packet structure
typedef struct {
    uint32_t command;
    uint32_t data_length;
    char filename[MAX_FILENAME];
} __attribute__((packed)) protocol_header_t;

// Thread arguments for client handling
typedef struct {
    int client_socket;
    struct sockaddr_in client_addr;
} client_args_t;

// Function to send a file to client
int send_file(int client_socket, const char *filename) {
    int file_fd = open(filename, O_RDONLY);
    if (file_fd < 0) {
        perror("Failed to open file");
        return -1;
    }

    // Get file size
    struct stat st;
    if (fstat(file_fd, &st) < 0) {
        perror("Failed to get file stats");
        close(file_fd);
        return -1;
    }

    // Send file size to client
    uint32_t file_size = htonl(st.st_size);
    if (send(client_socket, &file_size, sizeof(file_size), 0) < 0) {
        perror("Failed to send file size");
        close(file_fd);
        return -1;
```

```
    }

    // Send file data
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
    while ((bytes_read = read(file_fd, buffer, BUFFER_SIZE)) > 0) {
        if (send(client_socket, buffer, bytes_read, 0) < 0) {
            perror("Failed to send file data");
            close(file_fd);
            return -1;
        }
    }

    close(file_fd);
    return 0;
}

// Function to receive a file from client
int receive_file(int client_socket, const char *filename) {
    // Receive file size
    uint32_t file_size;
    if (recv(client_socket, &file_size, sizeof(file_size), 0) < 0) {
        perror("Failed to receive file size");
        return -1;
    }

    file_size = ntohl(file_size);

    // Create file
    int file_fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (file_fd < 0) {
        perror("Failed to create file");
        return -1;
    }

    // Receive file data
    char buffer[BUFFER_SIZE];
    size_t total_received = 0;

    while (total_received < file_size) {
        size_t to_receive = (file_size - total_received < BUFFER_SIZE) ?
                            (file_size - total_received) : BUFFER_SIZE;

        ssize_t bytes_received = recv(client_socket, buffer, to_receive, 0);
        if (bytes_received < 0) {
            perror("Failed to receive file data");
            close(file_fd);
```

```
            return -1;
        }

        if (write(file_fd, buffer, bytes_received) < 0) {
            perror("Failed to write file data");
            close(file_fd);
            return -1;
        }

        total_received += bytes_received;
    }

    close(file_fd);
    return 0;
}

// Function to list files in directory
void list_files(int client_socket) {
    FILE *fp = popen("ls -1", "r");
    if (!fp) {
        perror("Failed to execute ls command");
        return;
    }

    char buffer[BUFFER_SIZE];
    size_t total_size = 0;

    // First pass: calculate total size
    while (fgets(buffer, sizeof(buffer), fp)) {
        total_size += strlen(buffer);
    }

    // Send total size
    uint32_t size = htonl(total_size);
    send(client_socket, &size, sizeof(size), 0);

    // Second pass: send file list
    rewind(fp);
    while (fgets(buffer, sizeof(buffer), fp)) {
        send(client_socket, buffer, strlen(buffer), 0);
    }

    pclose(fp);
}

// Client handling thread
void* handle_client(void* arg) {
```

```
client_args_t *client_args = (client_args_t*)arg;
int client_socket = client_args->client_socket;

printf("Client connected: %s:%d\n",
       inet_ntoa(client_args->client_addr.sin_addr),
       ntohs(client_args->client_addr.sin_port));

free(client_args);

// Handle client requests
while (1) {
    protocol_header_t header;
    ssize_t bytes_received = recv(client_socket, &header, sizeof(header), 0);

    if (bytes_received <= 0) {
        break;  // Client disconnected or error
    }

    header.command = ntohl(header.command);
    header.data_length = ntohl(header.data_length);

    switch (header.command) {
        case CMD_UPLOAD:
            printf("Upload request for file: %s\n", header.filename);
            if (receive_file(client_socket, header.filename) == 0) {
                printf("File uploaded successfully: %s\n", header.filename);
            } else {
                printf("Failed to upload file: %s\n", header.filename);
            }
            break;

        case CMD_DOWNLOAD:
            printf("Download request for file: %s\n", header.filename);
            if (send_file(client_socket, header.filename) == 0) {
                printf("File sent successfully: %s\n", header.filename);
            } else {
                printf("Failed to send file: %s\n", header.filename);
            }
            break;

        case CMD_LIST:
            printf("File list request\n");
            list_files(client_socket);
            break;

        case CMD_QUIT:
            printf("Client requested to quit\n");
```

```
                goto cleanup;

            default:
                printf("Unknown command: %u\n", header.command);
                break;
        }
    }

cleanup:
    close(client_socket);
    printf("Client disconnected\n");
    return NULL;
}

// Server main function
int main_file_transfer_server() {
    int server_fd;
    struct sockaddr_in address;
    int opt = 1;

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("listen failed");
        exit(EXIT_FAILURE);
```

```
    }

    printf("File transfer server listening on port %d\n", PORT);

    // Accept connections
    while (1) {
        int client_socket;
        struct sockaddr_in client_addr;
        socklen_t client_addr_len = sizeof(client_addr);

        if ((client_socket = accept(server_fd, (struct sockaddr *)&client_addr,
                                    &client_addr_len)) < 0) {
            perror("accept failed");
            continue;
        }

        // Create thread to handle client
        client_args_t *client_args = malloc(sizeof(client_args_t));
        client_args->client_socket = client_socket;
        client_args->client_addr = client_addr;

        pthread_t client_thread;
        if (pthread_create(&client_thread, NULL, handle_client, client_args) != 0) {
            perror("Failed to create client thread");
            free(client_args);
            close(client_socket);
        } else {
            pthread_detach(client_thread);  // Automatically clean up thread
        }
    }

    close(server_fd);
    return 0;
}

// Client main function
int main_file_transfer_client(const char *server_ip) {
    int client_fd;
    struct sockaddr_in serv_addr;

    // Create socket
    if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return -1;
    }
```

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);

// Convert IPv4 address from text to binary
if (inet_pton(AF_INET, server_ip, &serv_addr.sin_addr) <= 0) {
    printf("Invalid address/ Address not supported\n");
    return -1;
}

// Connect to server
if (connect(client_fd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    perror("Connection Failed");
    return -1;
}

printf("Connected to server %s:%d\n", server_ip, PORT);

// Simple client interface
char command[10];
while (1) {
    printf("\nCommands: [u]pload, [d]ownload, [l]ist, [q]uit: ");
    if (fgets(command, sizeof(command), stdin)) {
        if (command[0] == 'u' || command[0] == 'U') {
            char filename[MAX_FILENAME];
            printf("Enter filename to upload: ");
            if (fgets(filename, sizeof(filename), stdin)) {
                filename[strcspn(filename, "\n")] = 0;  // Remove newline

                protocol_header_t header = {0};
                header.command = htonl(CMD_UPLOAD);
                strncpy(header.filename, filename, sizeof(header.filename) - 1);

                send(client_fd, &header, sizeof(header), 0);
                if (receive_file(client_fd, filename) == 0) {
                    printf("File uploaded successfully\n");
                } else {
                    printf("Failed to upload file\n");
                }
            }
        } else if (command[0] == 'd' || command[0] == 'D') {
            char filename[MAX_FILENAME];
            printf("Enter filename to download: ");
            if (fgets(filename, sizeof(filename), stdin)) {
                filename[strcspn(filename, "\n")] = 0;  // Remove newline

                protocol_header_t header = {0};
                header.command = htonl(CMD_DOWNLOAD);
```

```
                    strncpy(header.filename, filename, sizeof(header.filename) - 1);

                    send(client_fd, &header, sizeof(header), 0);
                    if (send_file(client_fd, filename) == 0) {
                        printf("File downloaded successfully\n");
                    } else {
                        printf("Failed to download file\n");
                    }
                }
            } else if (command[0] == 'l' || command[0] == 'L') {
                protocol_header_t header = {0};
                header.command = htonl(CMD_LIST);
                send(client_fd, &header, sizeof(header), 0);

                // Receive file list
                uint32_t list_size;
                recv(client_fd, &list_size, sizeof(list_size), 0);
                list_size = ntohl(list_size);

                char *list_buffer = malloc(list_size + 1);
                if (list_buffer) {
                    recv(client_fd, list_buffer, list_size, 0);
                    list_buffer[list_size] = '\0';
                    printf("Files on server:\n%s", list_buffer);
                    free(list_buffer);
                }
            } else if (command[0] == 'q' || command[0] == 'Q') {
                protocol_header_t header = {0};
                header.command = htonl(CMD_QUIT);
                send(client_fd, &header, sizeof(header), 0);
                break;
            }
        }
    }

    close(client_fd);
    return 0;
}
```

Project Development Process

1. Planning Phase

1. **Project Selection**: Choose a project that interests you and matches your skill level
2. **Requirements Analysis**: Define clear, measurable requirements
3. **Design Documentation**: Create architecture and design documents

4. **Technology Selection**: Choose appropriate libraries, tools, and frameworks
5. **Timeline Planning**: Create a realistic development timeline

2. Implementation Phase

1. **Environment Setup**: Configure development environment and tools
2. **Version Control**: Initialize Git repository and establish branching strategy
3. **Modular Development**: Implement features incrementally
4. **Testing**: Write and execute tests for each module
5. **Documentation**: Maintain documentation throughout development

3. Testing and Debugging Phase

1. **Unit Testing**: Test individual functions and modules
2. **Integration Testing**: Test module interactions
3. **System Testing**: Test complete system functionality
4. **Performance Testing**: Validate performance requirements
5. **Debugging**: Identify and fix issues

4. Deployment and Presentation Phase

1. **Final Testing**: Conduct comprehensive testing
2. **Documentation Finalization**: Complete all documentation
3. **Packaging**: Package application for distribution
4. **Presentation Preparation**: Prepare presentation materials
5. **Project Submission**: Submit all deliverables

Evaluation Criteria

Your capstone project will be evaluated based on:

Technical Excellence (40%)

- Code quality and organization
- Proper use of C programming concepts
- Error handling and robustness
- Performance optimization
- Security considerations

Design and Architecture (25%)

- System design quality
- Modularity and maintainability
- Appropriate use of data structures and algorithms
- Scalability considerations

Documentation (20%)

- Code comments and readability
- User documentation
- Technical documentation
- API documentation

Testing (10%)

- Test coverage
- Test quality
- Bug identification and resolution

Presentation (5%)

- Clarity of presentation
- Technical depth
- Professionalism

Best Practices

1. Code Quality

- Follow consistent coding standards
- Use meaningful variable and function names
- Write modular, reusable code
- Include comprehensive error handling
- Document complex algorithms and decisions

2. Project Management

- Use version control effectively
- Break project into manageable milestones
- Track progress and adjust plans as needed
- Seek feedback regularly
- Maintain regular backups

3. Collaboration

- If working in a team, establish clear roles
- Use collaborative tools effectively
- Communicate regularly with team members
- Review each other's code
- Resolve conflicts constructively

Resources and Support

Tools and Libraries

- **Development Environment**: GCC, Clang, or preferred C compiler
- **Debugging**: GDB, Valgrind
- **Version Control**: Git
- **Documentation**: Doxygen for API documentation
- **Testing**: CUnit, Unity, or custom testing framework

Learning Resources

- **Books**: "The C Programming Language" by Kernighan and Ritchie
- **Online**: C standard documentation, man pages
- **Communities**: Stack Overflow, C programming forums
- **Tutorials**: System programming tutorials, networking guides

Conclusion

The capstone project is your opportunity to demonstrate mastery of C programming and apply it to solve real-world problems. Choose a project that challenges you but is achievable within the timeframe. Focus on writing clean, well-documented code and following professional software development practices.

Remember that the goal is not just to complete a project, but to learn and grow as a developer. Take the time to understand the concepts you're implementing, and don't hesitate to explore beyond the basic requirements. Good luck with your capstone project!

Static Linking: Why glibc Binaries Are Big, and When musl Is the Answer

Static Linking: Why glibc Binaries Are Big, and When musl Is the Answer

Thesis

Link a one-line `puts("hello")` program statically against glibc and you get roughly **750 KB**. Link the same source against musl and you get about **25 KB**. Neither number is a bug. Both follow from how a static archive (`libc.a`) works: the linker copies in every object file that resolves a symbol that something already copied in needs, and it keeps going until nothing is missing. glibc's stdio is built for locales, wide characters and loadable character-set converters, so pulling in `puts` drags in the converter machinery and, behind that, a small copy of the dynamic loader. musl's stdio does not have that dependency chain.

The size is the visible part. The part that bites is behaviour: a "fully static" glibc binary still reads `/etc/nsswitch.conf` at run time, and for anything beyond the built-in `files` and `dns` sources it tries to load **shared** NSS modules from the host. The linker warns you about this. This chapter measures the size, traces where it comes from, and shows when static glibc, static musl or plain dynamic linking is the right choice.

Mental model

```
your main.o needs: puts


libc.a(ioputs.o)        needs __io_vtables
libc.a(vtables.o)       needs _IO_new_file_setbuf
libc.a(fileops.o)       needs __wcsmbs_named_conv   ← wide-char support in every FILE
libc.a(wcsmbsload.o)    needs __gconv_find_transform
libc.a(gconv_db.o)      needs __gconv_find_shlib     ← iconv converters live in .so files
libc.a(gconv_dl.o)      needs __libc_dlopen_mode
libc.a(dl-libc.o)       needs _dl_open               ← so a static binary carries a loader
...                     425 archive members in total (musl: 29)
```

Fact	Consequence
The linker pulls **whole object files** from `libc.a`, transitively	One call can bring in hundreds of objects. `-ffunction-sections -Wl,--gc-sections` on *your* code cannot undo libc's internal references

Fact	Consequence
glibc's `FILE` supports wide orientation and locale conversion	`puts` reaches iconv (`gconv`) and from there `dlopen`
glibc NSS (users, groups, hosts) is plugin based	Static binaries still read `nsswitch.conf` and may `dlopen` `libnss_*.so.2` of the host
musl implements NSS-like lookups directly (files and DNS only)	Small and self-contained, but it ignores `nsswitch.conf`, so LDAP/SSSD/systemd users are invisible to it
A dynamic binary shares one `libc.so.6`	~16 KB on disk, but needs a compatible glibc on the target

Worked examples

Tested on Debian 13 (x86_64) with GCC 14.2.0, glibc 2.41 and musl 1.2.5 (`musl-tools` provides the `musl-gcc` wrapper). The mechanisms described here have been stable for many glibc and musl releases; exact byte counts vary by version and distribution.

Case 1: Measure the three builds

```c
/* hello.c - the smallest useful program */
#include <stdio.h>

int main(void)
{
    puts("hello");
    return 0;
}
```

```
gcc      -O2         -o hello-dyn   hello.c
gcc      -O2 -static -o hello-glibc hello.c
musl-gcc -O2 -static -o hello-musl  hello.c
for f in hello-dyn hello-glibc hello-musl; do strip -o $f.s $f; done
ls -l hello-dyn hello-glibc hello-musl *.s | awk '{print $5, $9}'
size hello-dyn hello-glibc hello-musl
```

```
15952 hello-dyn
14472 hello-dyn.s
758448 hello-glibc
676288 hello-glibc.s
24640 hello-musl
17768 hello-musl.s
   text    data     bss     dec     hex filename
```

```
   1306     584       8    1898     76a hello-dyn
 640899   23688   22576  687163   a7c3b hello-glibc
   4956     336    1688    6980    1b44 hello-musl
```

Stripping removes symbol tables, not code: the glibc build is still 676 KB. The `text` column is the honest comparison. Static glibc carries **640 KB** of code for `puts`, static musl about **5 KB**.

```
ldd hello-dyn; ldd hello-glibc; ldd hello-musl
```

```
    linux-vdso.so.1 (0x00007f0fec57d000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f0fec373000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f0fec57f000)
    not a dynamic executable
    not a dynamic executable
```

Case 2: Ask the linker why

A link map records every archive member that was pulled in and the symbol that caused it:

```
gcc -O2 -static -o hello-glibc hello.c -Wl,-Map=glibc.map
awk '/^Archive member included/{f=1;next} /^Discarded|^Allocating|^Memory map/{f=0} f' glibc
grep -o 'libc\.a([^)]*)' members.txt | sort -u | wc -l
```

```
425
```

The same count for musl (`musl-gcc ... -Wl,-Map=musl.map`) is **29**. To follow the chain, look up each member and the line after it, which names the object and symbol that required it:

```
# why.sh - print which object (and symbol) pulled a libc.a member into the link
w() { grep -A1 "^/[^ ]*libc.a($1)" members.txt | tail -1 | sed 's#.*libc.a(#libc.a(#; s#^ *#
for m in ioputs.o vtables.o fileops.o wcsmbsload.o gconv_db.o gconv_dl.o dl-libc.o dl-open.o
  printf '%-14s <- %s\n' "$m" "$(w $m)"
done
```

```
ioputs.o       <- /tmp/cc4rLaBC.o (puts)
vtables.o      <- libc.a(ioputs.o) (__io_vtables)
fileops.o      <- libc.a(vtables.o) (_IO_new_file_setbuf)
wcsmbsload.o   <- libc.a(fileops.o) (__wcsmbs_named_conv)
gconv_db.o     <- libc.a(wcsmbsload.o) (__gconv_find_transform)
gconv_dl.o     <- libc.a(gconv_db.o) (__gconv_find_shlib)
dl-libc.o      <- libc.a(gconv_dl.o) (__libc_dlopen_mode)
dl-open.o      <- libc.a(dl-libc.o) (_dl_open)
dl-support.o   <- libc.a(libc-start.o) (_dl_aux_init)
```

(The first line names the temporary object GCC made from `hello.c`.) Two independent roots

explain most of the size:

1. **stdio → iconv → dlopen.** Any glibc `FILE` may be switched to wide orientation and convert through a character set whose converter lives in a `gconv` shared module, so `fileops.o` references the converter loader, which references `dlopen`.
2. **Startup → loader support.** `__libc_start_main` in a static binary initialises TLS, IRELATIVE relocations (for the CPU-specific `memcpy`/`strlen` variants), tunables and auxv handling through `dl-support.o` and friends.

The largest symbols confirm it:

```
nm --size-sort -S -t d hello-glibc | tail -8
```

```
0000000004537792 0000000000007845 T _dl_relocate_object_no_relro
0000000004379296 0000000000008401 t printf_positional
0000000004565200 0000000000008551 t __printf_fp_buffer_1.isra.0
0000000004387712 0000000000009636 T __printf_buffer
0000000004758976 0000000000013272 r translit_from_tbl
0000000004780416 0000000000013800 R __tens
0000000004876416 0000000000016384 B __pthread_keys
0000000004728768 0000000000023548 r translit_to_tbl
```

A program that only calls `puts` contains the dynamic relocator, the full `printf` engine with floating-point formatting, and transliteration tables.

Case 3: Users and hosts

```c
/* lookup.c - resolve a user and a host name */
#include <netdb.h>
#include <pwd.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    const char *host = argc > 1 ? argv[1] : "localhost";
    struct passwd *pw = getpwnam("root");
    printf("root's shell: %s\n", pw ? pw->pw_shell : "(not found)");

    struct addrinfo *res;
    int rc = getaddrinfo(host, NULL, NULL, &res);
    printf("getaddrinfo(%s): %s\n", host, rc ? gai_strerror(rc) : "ok");
    if (!rc) freeaddrinfo(res);
    return 0;
}
```

```
gcc -O2 -static -o lookup-glibc lookup.c
```

```
/usr/bin/ld: /tmp/ccONpojX.o: in function `main':
lookup.c:(.text.startup+0x4c): warning: Using 'getaddrinfo' in statically linked application
/usr/bin/ld: lookup.c:(.text.startup+0x1d): warning: Using 'getpwnam' in statically linked a
```

The build succeeds, which is why people miss the warning. Compare what each binary reads at run time:

```
musl-gcc -O2 -static -o lookup-musl lookup.c
strace -e trace=open,openat ./lookup-musl localhost
strace -e trace=open,openat ./lookup-glibc localhost 2>&1 | grep -v locale
```

```
open("/etc/passwd", O_RDONLY|O_LARGEFILE|O_CLOEXEC) = 5
root's shell: /bin/bash
open("/etc/hosts", O_RDONLY|O_LARGEFILE|O_CLOEXEC) = 5
getaddrinfo(localhost): ok
+++ exited with 0 +++
openat(AT_FDCWD, "/etc/nsswitch.conf", O_RDONLY|O_CLOEXEC) = 5
openat(AT_FDCWD, "/etc/passwd", O_RDONLY|O_CLOEXEC) = 5
openat(AT_FDCWD, "/etc/host.conf", O_RDONLY|O_CLOEXEC) = 5
openat(AT_FDCWD, "/etc/resolv.conf", O_RDONLY|O_CLOEXEC) = 5
openat(AT_FDCWD, "/etc/hosts", O_RDONLY|O_CLOEXEC) = 5
openat(AT_FDCWD, "/etc/gai.conf", O_RDONLY|O_CLOEXEC) = 5
root's shell: /bin/bash
getaddrinfo(localhost): ok
+++ exited with 0 +++
```

musl reads `/etc/passwd` and `/etc/hosts` directly and never looks at `nsswitch.conf`. (It uses `open(2)`, not `openat(2)`, which is why the trace includes both.) Static glibc follows `nsswitch.conf`. On this host it says `passwd: files` and `hosts: files dns`, and since glibc 2.34 those two sources are built into libc, so nothing is loaded. On a host configured with `passwd: files systemd sss` or `hosts: files mdns4_minimal dns`, the static binary tries to `dlopen libnss_systemd.so.2`, `libnss_sss.so.2` and so on from the host. Those modules must come from the **same glibc version** the binary was linked with.

The trap

"Static" glibc is not self-contained, and the failure is silent. Ship `lookup-glibc` into a minimal container or onto a host with a different glibc, and one of three things happens when a lookup needs a plugin:

- the module is missing: the lookup quietly returns "not found" (directory users do not exist, mDNS names do not resolve);
- the module is present but from another glibc: the lookup can fail or crash inside the module, because glibc's internal interfaces are not stable between versions;
- the module is present and matching: it works, on that host, by luck.

None of these produces an error at startup, so tests on the build machine pass. musl avoids the problem by not supporting plugins at all, which is its own trap: a musl binary on an LDAP or SSSD host will not see directory users, whatever `nsswitch.conf` says.

Pick on purpose:

Need	Choose
Runs on any Linux, only local files and DNS	Static **musl**
Must honour the host's NSS (LDAP, SSSD, systemd-homed, mDNS)	**Dynamic** glibc, built against the oldest glibc you support
Static glibc for other reasons (a rescue tool, an initramfs)	Avoid NSS calls entirely, or ship the exact matching `libnss_*` modules beside it

The boring rule

> Treat `-static` with glibc as a portability claim you must prove, not a flag. Read the linker's NSS warnings as errors (`-Wl,--fatal-warnings` makes them fail the build). Use static musl for small, self-contained tools that only need local files and DNS, dynamic glibc for anything that must see the host's users and names, and measure size with `size`, not `ls`.

Try this

1. Add `-Wl,--fatal-warnings` to the static glibc build of `lookup.c` and confirm that the NSS warnings now fail the link.
2. Replace `puts` with `write(1, "hello\n", 6)` and rebuild statically with glibc. How many archive members remain, and does `gconv_db.o` still appear in the map?
3. Build `lookup.c` statically against musl, run it in a `FROM scratch` container with only `/etc/passwd` and `/etc/hosts` copied in, and then do the same with the static glibc build.
4. Use `-Wl,--trace-symbol=__libc_dlopen_mode` to print every object that defines or references the loader entry point in each build.

Sources

- GNU C Library manual: Name Service Switch and NSS module internals
- glibc 2.34 release notes (NEWS): `nss_files` and `nss_dns` built into libc
- glibc FAQ / wiki: statically linked programs and NSS
- musl documentation: Functional differences from glibc (name resolution, no NSS)
- GNU ld manual: `-Map`, `--trace-symbol`, `--fatal-warnings`
- `nsswitch.conf(5)`, `getaddrinfo(3)`, `getpwnam(3)` man pages

Fil-C: Memory-Safe C with Invisible Capabilities

Fil-C: Memory-Safe C with Invisible Capabilities

Thesis

Fil-C is a memory-safe implementation of C and C++ that keeps the languages you already write: clang extensions, atomics, threads, `mmap`, signals, even a large slice of inline assembly. Every pointer carries a **capability** that says which object it may touch and how. Out-of-bounds access, use-after-free, and capability corruption become **Fil-C panics**, not silent exploitation. There is no `unsafe` keyword and only limited FFI to unchecked code.

The engineering bet is *fanatical compatibility* on stock hardware (Linux x86_64 and ARM64). Pointers stay **64-bit** in the C address space; the capability lives in an invisible sidecar (InvisiCaps). As of **v0.686** (4 October 2026) the compiler is based on clang **20.1.8**. This chapter explains the capability model, why overhead is often acceptable (including the Fil-xz story), and a concrete byval/tail-padding bug that Anthropic helped fix — without installing Fil-C on this machine.

Mental model

```
C program sees:     intval  (ordinary 64-bit pointer arithmetic / casts)
Fil-C also keeps:   lower   (trusted object base + header)
                    upper   (exclusive end; lower==upper   no data access)
                    aux     (flags + pointer to aux capability table)

flight pointer (in registers):  (lower, intval)
rest pointer   (in the heap):   intval in payload;
                                lower in aux allocation (or atomic box)
```

Idea	Consequence
Capability, not tag	Hitting "valid memory" is not enough; the *pointer* must be allowed to that object (unlike ASan / MTE / HWAsan, which attackers can often bypass)
InvisiCaps	`sizeof(T*)` stays 8 on 64-bit; metadata is not in the C-visible object layout
Concurrent GC (FUGC)	`free` can deterministically disable all pointers to an object; UAF panics instead of corrupting capabilities
No `unsafe`	Safety is the ABI; escapes are limited FFI, not a keyword
Flight vs rest	Locals carry lower+intval; heap stores intval in the payload and lower in an aux page sized like the payload

Compared with hardware capabilities (CHERI), InvisiCaps aim for similar enforcement while remaining binary-compatible with ordinary 64-bit pointers. Fil-C has iterated PLUT → SideCaps → MonoCaps → InvisiCaps; earlier models were slower, required knowing types at `malloc` time, or both.

Worked examples

Fil-C is **not installed** on this box (no system package installs for scout runs). The cases below use ordinary gcc to show *what* Fil-C checks, and cite upstream docs for *how* Fil-C checks it. Version facts are from the Fil-C site and the v0.686 GitHub release notes (verified October 2026).

Case 1: The bug class Fil-C turns into panics

```c
/* uaf_oob_demo.c - illustrates the class of errors Fil-C panics on.
 * Under Yolo-C (ordinary gcc/clang) these are undefined behaviour.
 * Under Fil-C, capability checks make them deterministic panics.
 * We do NOT run the buggy paths here; we only compile the safe skeleton. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    char *p = malloc(4);
    if (!p) return 1;
    memcpy(p, "abcd", 4);

    /* SAFE: in-bounds write */
    p[3] = 'Z';
    printf("ok: %c%c%c%c\n", p[0], p[1], p[2], p[3]);

    /* Fil-C would panic on either of these (do not enable):
     *   p[4] = 'X';          // OOB - intval past upper
     *   free(p); p[0] = 'Y'; // UAF - free collapses upper to lower + free bit
     */
    free(p);
    return 0;
}
```

```
gcc -O2 -Wall -o uaf_oob_demo uaf_oob_demo.c
./uaf_oob_demo
```

```
ok: abcZ
```

Under Fil-C, freeing sets the object's upper bound equal to lower and sets a free bit in the aux word, so a later load/store through any surviving pointer panics with a use-after-free diagnosis rather than reinterpreting recycled memory.

Case 2: InvisiCaps at rest (why unions and atomics still work)

When a pointer is stored into the heap:

1. The **intval** lands in the object's payload (so integer loads see a normal address).
2. The **lower** lands in an **aux allocation** the same size as the payload (created lazily when the first pointer is stored).
3. Atomic / `volatile` / `std::atomic` pointer slots use 16-byte **atomic boxes** in the aux table so races cannot forge a capability — you may load the "wrong" valid lower, but not a corrupted one.

Objects with no pointers (strings, framebuffers) keep a NULL aux word: the GC treats them as leaves and space overhead stays far below 2×.

```
Object A payload:  [ ... | intval → B | ... ]
Object A aux:      [ ... | lower_of_B | ... ]   ← invisible to C
Object B header:   lower points here; upper = end of B
```

Case 3: Fil-xz and the overhead story

Filip Pizło reports that **xz** has extremely low Fil-C overhead — low enough that Fil-xz is often competitive with, and sometimes faster than, a distro's older xz build, because xz's own algorithmic gains outweigh the safety tax. Upstream docs describe roughly ~**1.5**× in good cases and up to about ~**4**× in bad cases under InvisiCaps, with more optimisation headroom still open (calling convention, InvisiCap tuning, pointer chasing).

The practical takeaway for C engineers: measure *your* hot path. Compression, parsers, and other CPU-bound code with few pointer-heavy data structures are the optimistic region; pointer-chasing graphs are the pessimistic one.

Case 4: Anthropic byval / tail-padding fix

Commit `3485bcd` (9 October 2026) fixed an uninitialized-memory issue in Fil-C's by-value argument path:

> the compiler [was] using a smaller size when *checking* byval arguments, and a larger size when *copying* them. This is due to the behavior of **tail padding**.

LLVM may copy a struct using its size *including* trailing padding, while capability checks used the logical (possibly smaller) size. The gap could leave padding bytes unchecked or copy past the capability. The commit message credits Anthropic with the bug report, almost all of the new tests (`byval-copy-test*`, `byval_tailpad_*`, misaligned demote cases), and most of the fix in `FilPizlonator.cpp` / `filc_runtime.c`.

Why it matters: Fil-C's calling convention copies arguments into thread-local payload and capability buffers with 8-byte rounding and alignment padding. Byval/byref edges are exactly where "invisible" metadata and ABI padding meet — a classic place for safety compilers to get the size wrong.

Case 5: What v0.686 shipped

From the v0.686 release notes (4 October 2026), highlights include:

- Initial **C++ coroutine** support and limited `musttail` (may panic on stack overflow where Yolo-C would not)
- ARM64 GC / weak-load memory-ordering fixes
- fork(2) deadlocks with global init and exiting threads fixed
- Structs containing unions containing pointers as args/returns fixed
- `/opt/fil` package bumps (OpenSSL 3.6.5, xz 5.8.x line in the 0.685 rebase, etc.)

Download slices: self-contained **pizfix** tarballs (musl, no root) and **/opt/fil** (glibc, root) for x86_64 and ARM64.

The trap

Treating Fil-C like AddressSanitizer. ASan finds bugs in tests; Fil-C is a **capability ABI** meant to stop exploitation in production. If you plan an `unsafe` escape hatch "just for this module", you are fighting the design — Fil-C deliberately has none. The other trap is installing a full Fil-C userland via apt/sudo on a shared scout box: use the documented pizfix tarball on a disposable machine, or read the docs and measure later.

The boring rule

Write normal C. Prefer patterns Fil-C already runs (openssl, CPython, SQLite, OpenSSH in `/opt/fil`). When something panics, fix the memory error — do not reach for an unchecked FFI. For size/ABI surprises (byval, unions, tail padding), assume the capability size and the copy size must agree.

Try this

1. On a disposable Linux VM, unpack `filc-0.686-linux-x86_64.tar.xz` from the GitHub release (no root required for pizfix) and rebuild a small project you own with `filcc`.
2. Deliberately add an OOB write and confirm you get a Fil-C panic with a useful origin, then remove it.
3. Read InvisiCaps once end-to-end; sketch flight vs rest pointers from memory.
4. If you ship compression, compare wall time of distro `xz` vs a Fil-C build of current xz on *your* corpus before deciding overhead is unacceptable.

Sources

- Fil-C project: https://github.com/pizlonator/fil-c
- Fil-C site and InvisiCaps: https://fil-c.org/, https://fil-c.org/invisicaps
- Release v0.686 notes: https://github.com/pizlonator/fil-c/releases/tag/v0.686
- Byval / tail-padding fix (Anthropic): https://github.com/pizlonator/fil-c/commit/3485bcdbc52605955993c6ff596b4794f5756850
- Calling convention (argument buffers / padding): https://fil-c.org/calling_convention
- Inspiration (overhead / xz, release, Anthropic thanks): https://x.com/filpizlo/status/2109805487290003465, https://x.com/filpizlo/status/2106887794962600414, https://x.com/filpizlo/status/2108600110762647568

File-Descriptor Hygiene: CLOEXEC, close_range, and O_CLOFORK

File-Descriptor Hygiene: CLOEXEC, close_range, and O_CLOFORK

Thesis

`fork()` copies the calling process's file-descriptor table. `execve()` then keeps every descriptor whose close-on-exec flag is clear. The default for `open()`, `pipe()`, `socket()`, `accept()`, and `dup()` is **clear**. A worker, a helper, a pager, a `system()` grandchild — any of them can inherit a pipe write end, a listening socket, a secret, or a log file you meant to keep to yourself.

`O_CLOEXEC` / `FD_CLOEXEC` is the opt-out for **exec**. It does nothing at **fork**. A prefork server that never `exec`s still hands every unmarked descriptor to every child. POSIX.1-2024 added `O_CLOFORK` / `FD_CLOFORK` for that case. Linux does not implement them: not in this kernel (6.12), not in glibc 2.41, and not in the Linux man-pages 6.19 `fcntl(2)` VERSIONS note ("POSIX.1-2024 specifies `FD_CLOFORK` and `F_DUPFD_CLOFORK`, but Linux doesn't support them").

On Linux the working kit is: create every descriptor with an atomic `*CLOEXEC` constructor, and before `exec` sweep the rest with `close_range()`. Setting `FD_CLOEXEC` with `fcntl()` *after* `open()` is a race against any other thread that `fork`s.

Mental model

```
open / pipe / socket / accept / dup     →  FD_CLOEXEC clear  (inherited)
open(..., O_CLOEXEC)  and friends       →  FD_CLOEXEC set    (closed on exec only)
POSIX.1-2024 O_CLOFORK                  →  FD_CLOFORK set    (closed in the child at fork)
                                           Linux: not implemented
```

Flag / call	When it fires	On this Linux (6.12, glibc 2.41)
`O_CLOEXEC`, `FD_CLOEXEC`	`execve` / `posix_spawn`	Yes. Linux 2.6.23; POSIX.1-2008
`SOCK_CLOEXEC`, `pipe2`, `dup3`, `accept4`, `F_DUPFD_CLOEXEC`	At creation, sets `FD_CLOEXEC`	Yes. `SOCK_CLOEXEC` / `dup3` / `pipe2` are in POSIX.1-2024
`close_range(first, last, 0)`	Now: close every fd in the range	Yes. Linux 5.9, glibc 2.34
`CLOSE_RANGE_CLOEXEC`	Now: *mark* the range close-on-exec	Yes. Linux 5.11

Flag / call	When it fires	On this Linux (6.12, glibc 2.41)
`O_CLOFORK`, `FD_CLOFORK`, `F_DUPFD_CLOFORK`, `SOCK_CLOFORK`	`fork` in the child	**No.** Headers do not define them; `open(..., O_CLOFORK)` does not compile

A pipe is a single open file description with two descriptors. `read()` returns 0 only when **every** write end is closed. If the child still holds a write end, the reader waits forever. That is the leak that looks like a hang.

Worked examples

Tested on Debian 13 (x86_64), Linux **6.12.94+**, GCC **14.2.0**, glibc **2.41**. Compile with `-std=c17 -Wall -Wextra -O2`. Several programs start with `close_range(3, ~0U, 0)` so the listing is about the descriptors *we* open, not whatever the parent process already leaked. In this captured session fds 0–2 are `/dev/null` and two pipes; in a terminal they are usually the tty. The `man` pages are not installed on this box; quotes below are from man-pages 6.19 (checked on man7.org, 2026-10-17) and from POSIX.1-2024.

Build the fd-dump helper once. Later cases `exec` it:

```
/* listfd.c - dump /proc/self/fd plus FD_CLOEXEC for each open descriptor */
#define _GNU_SOURCE
#include <dirent.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

struct rec {
    int fd;
    int cloexec;
    char target[128];
};

static int cmp_fd(const void *a, const void *b)
{
    return ((const struct rec *)a)->fd - ((const struct rec *)b)->fd;
}

int main(void)
{
    DIR *d = opendir("/proc/self/fd");
    if (!d) {
```

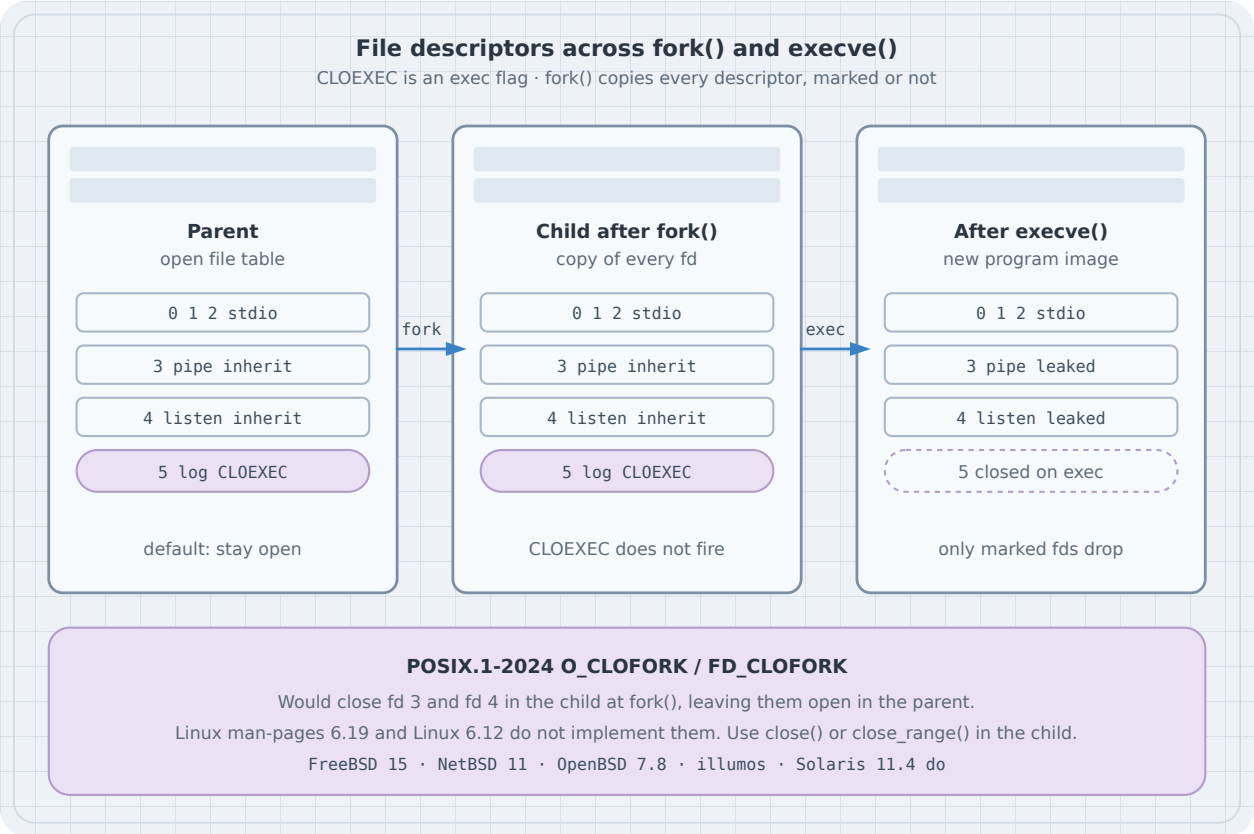


Figure 1: Parent fd table copied by fork; CLOEXEC drops a descriptor only at exec

```
        perror("opendir");
        return 1;
    }
    int dfd = dirfd(d);
    struct rec recs[64];
    int nrec = 0;
    struct dirent *ent;
    while ((ent = readdir(d)) != NULL && nrec < 64) {
        if (ent->d_name[0] == '.')
            continue;
        int fd = atoi(ent->d_name);
        char path[64];
        snprintf(path, sizeof path, "/proc/self/fd/%d", fd);
        ssize_t n = readlink(path, recs[nrec].target,
                             sizeof recs[nrec].target - 1);
        if (n < 0)
            continue;
        recs[nrec].target[n] = '\0';
        int flags = fcntl(fd, F_GETFD);
        recs[nrec].fd = fd;
        recs[nrec].cloexec = (flags >= 0 && (flags & FD_CLOEXEC));
        nrec++;
    }
    qsort(recs, (size_t)nrec, sizeof recs[0], cmp_fd);
    for (int i = 0; i < nrec; i++) {
        printf("fd %2d  %-8s  %s%s\n",
               recs[i].fd,
               recs[i].cloexec ? "CLOEXEC" : "inherit",
               recs[i].target,
               recs[i].fd == dfd ? "  (opendir)" : "");
    }
    closedir(d);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o listfd listfd.c
```

glibc's `opendir("/proc/self/fd")` itself opens a descriptor with `O_CLOEXEC`. That extra line is the directory handle `listfd` used to walk the table, not a leak from the parent.

Case 0: What this libc actually defines

```c
/* probe.c - what this libc and kernel actually define */
#define _GNU_SOURCE
#include <fcntl.h>
#include <linux/close_range.h>
#include <stdio.h>
#include <sys/socket.h>
#include <unistd.h>
#ifdef __GLIBC__
#include <gnu/libc-version.h>
#endif

int main(void)
{
#ifdef __GLIBC__
    printf("glibc %s\n", gnu_get_libc_version());
#endif
    printf("O_CLOEXEC           = 0%o (0x%x)\n", O_CLOEXEC, O_CLOEXEC);
    printf("FD_CLOEXEC          = %d\n", FD_CLOEXEC);
    printf("F_DUPFD_CLOEXEC     = %d\n", F_DUPFD_CLOEXEC);
    printf("SOCK_CLOEXEC        = 0%o (0x%x)\n", SOCK_CLOEXEC, SOCK_CLOEXEC);
    printf("CLOSE_RANGE_UNSHARE = %u\n", CLOSE_RANGE_UNSHARE);
    printf("CLOSE_RANGE_CLOEXEC = %u\n", CLOSE_RANGE_CLOEXEC);

#ifdef O_CLOFORK
    printf("O_CLOFORK           = 0x%x\n", O_CLOFORK);
#else
    printf("O_CLOFORK           not defined\n");
#endif
#ifdef FD_CLOFORK
    printf("FD_CLOFORK          = %d\n", FD_CLOFORK);
#else
    printf("FD_CLOFORK          not defined\n");
#endif
#ifdef F_DUPFD_CLOFORK
    printf("F_DUPFD_CLOFORK     = %d\n", F_DUPFD_CLOFORK);
#else
    printf("F_DUPFD_CLOFORK     not defined\n");
#endif
#ifdef SOCK_CLOFORK
    printf("SOCK_CLOFORK        = 0x%x\n", SOCK_CLOFORK);
#else
    printf("SOCK_CLOFORK        not defined\n");
#endif
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o probe probe.c
./probe
```

```
glibc 2.41
O_CLOEXEC           = 02000000 (0x80000)
FD_CLOEXEC          = 1
F_DUPFD_CLOEXEC     = 1030
SOCK_CLOEXEC        = 02000000 (0x80000)
CLOSE_RANGE_UNSHARE = 2
CLOSE_RANGE_CLOEXEC = 4
O_CLOFORK           not defined
FD_CLOFORK          not defined
F_DUPFD_CLOFORK     not defined
SOCK_CLOFORK        not defined
```

`O_CLOEXEC` and `SOCK_CLOEXEC` are the same bit (`02000000`). Passing `O_CLOFORK` without an `#ifdef` fails at compile time:

```
oclofork_use.c:2:54: error: ‘O_CLOFORK’ undeclared (first use in this function)
```

Case 1: A leaked pipe write end hides EOF

`read()` on a pipe returns 0 only when no process still holds a write end. `fork()` duplicates both ends. If the child is the reader and keeps the inherited write end, the parent's `close()` is not enough.

```c
/* leak_pipe.c - a leaked write end keeps the reader from seeing EOF */
#define _GNU_SOURCE
#include <errno.h>
#include <poll.h>
#include <stdio.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int leak = argc > 1 && strcmp(argv[1], "leak") == 0;
    int p[2];
    if (pipe(p) == -1) {
        perror("pipe");
        return 1;
    }

    fflush(stdout);
```

```
pid_t pid = fork();
if (pid == -1) {
    perror("fork");
    return 1;
}
if (pid == 0) {
    /* Child is the reader. Closing the inherited write end is what
       lets read() return 0 once the parent drops its write end. */
    if (!leak)
        close(p[1]);
    char buf[64];
    for (;;) {
        struct pollfd po = { .fd = p[0], .events = POLLIN };
        int pr = poll(&po, 1, 400);
        if (pr == 0) {
            printf("child: poll timeout - write end still open, no EOF\n");
            fflush(stdout);
            _exit(2);
        }
        if (pr < 0) {
            perror("poll");
            _exit(1);
        }
        ssize_t n = read(p[0], buf, sizeof buf);
        if (n == 0) {
            printf("child: read returned 0 (EOF)\n");
            fflush(stdout);
            _exit(0);
        }
        if (n < 0) {
            perror("read");
            _exit(1);
        }
        printf("child: read %zd byte(s)\n", n);
        fflush(stdout);
    }
}

close(p[0]);
const char msg[] = "hello\n";
if (write(p[1], msg, sizeof msg - 1) != (ssize_t)(sizeof msg - 1)) {
    perror("write");
    return 1;
}
close(p[1]); /* parent drops its write end */
int st;
if (waitpid(pid, &st, 0) == -1) {
```

```
        perror("waitpid");
        return 1;
    }
    printf("parent: child exit %d\n", WIFEXITED(st) ? WEXITSTATUS(st) : -1);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o leak_pipe leak_pipe.c
./leak_pipe
./leak_pipe leak
```

```
child: read 6 byte(s)
child: read returned 0 (EOF)
parent: child exit 0
```

```
child: read 6 byte(s)
child: poll timeout - write end still open, no EOF
parent: child exit 2
```

Six bytes is `"hello\n"`. The data arrives either way. EOF does not, unless the child closes its copy of the write end. `pipe2(p, O_CLOEXEC)` would have marked both ends close-on-exec, which helps a later `exec` and does **not** close the child's write end across a plain `fork`. For a fork-only worker, the child still has to `close(p[1])`.

Case 2: The child inherits the listening socket

A prefork server that `listen()`s and then `fork()`s workers wants the children to inherit that socket. Almost every other `fork` does not. `SOCK_CLOEXEC` does not change the fork case.

```
/* leak_listen.c - a child inherits the listening socket across fork() */
#define _GNU_SOURCE
#include <arpa/inet.h>
#include <fcntl.h>
#include <linux/close_range.h>
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <unistd.h>

static void dump_one(const char *who, int fd)
{
    char path[64], target[128];
    snprintf(path, sizeof path, "/proc/self/fd/%d", fd);
```

```
    ssize_t n = readlink(path, target, sizeof target - 1);
    if (n < 0) {
        printf("%s: fd %d gone\n", who, fd);
        return;
    }
    target[n] = '\0';
    int flags = fcntl(fd, F_GETFD);
    printf("%s pid %d  fd %d -> %s  FD_CLOEXEC=%s\n",
           who, getpid(), fd, target,
           (flags >= 0 && (flags & FD_CLOEXEC)) ? "yes" : "no");
}

int main(int argc, char **argv)
{
    int cloexec = 0, do_exec = 0;
    const char *helper = "./listfd";
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "cloexec") == 0)
            cloexec = 1;
        else if (strcmp(argv[i], "exec") == 0)
            do_exec = 1;
        else if (strncmp(argv[i], "helper=", 7) == 0)
            helper = argv[i] + 7;
    }

    /* Drop inherited junk so the listing is about this socket. */
    if (close_range(3, ~0U, 0) == -1) {
        perror("close_range");
        return 1;
    }

    int type = SOCK_STREAM | (cloexec ? SOCK_CLOEXEC : 0);
    int s = socket(AF_INET, type, 0);
    if (s == -1) {
        perror("socket");
        return 1;
    }
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK),
        .sin_port = 0,
    };
    if (bind(s, (struct sockaddr *)&addr, sizeof addr) == -1) {
        perror("bind");
        return 1;
    }
    if (listen(s, 1) == -1) {
```

```
        perror("listen");
        return 1;
    }
    socklen_t alen = sizeof addr;
    if (getsockname(s, (struct sockaddr *)&addr, &alen) == -1) {
        perror("getsockname");
        return 1;
    }
    printf("parent listening on 127.0.0.1:%u  fd=%d  SOCK_CLOEXEC=%s\n",
           ntohs(addr.sin_port), s, cloexec ? "yes" : "no");
    dump_one("parent", s);
    fflush(stdout);

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        if (do_exec) {
            execl(helper, "listfd", (char *)NULL);
            perror("execl");
            _exit(1);
        }
        dump_one("child ", s);
        fflush(stdout);
        _exit(0);
    }
    int st;
    waitpid(pid, &st, 0);
    close(s);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o leak_listen leak_listen.c
./leak_listen
./leak_listen cloexec
./leak_listen exec helper=./listfd
./leak_listen cloexec exec helper=./listfd
```

Plain `fork`, no `exec`. The child has the listening socket, `FD_CLOEXEC` clear:

```
parent listening on 127.0.0.1:53065  fd=3  SOCK_CLOEXEC=no
parent pid 921716  fd 3 -> socket:[10082516]  FD_CLOEXEC=no
child  pid 921717  fd 3 -> socket:[10082516]  FD_CLOEXEC=no
```

`SOCK_CLOEXEC` sets the flag and **still** copies the descriptor across `fork`. Both processes share the

same kernel socket (same inode number):

```
parent listening on 127.0.0.1:42277  fd=3  SOCK_CLOEXEC=yes
parent pid 921718  fd 3 -> socket:[10076076]  FD_CLOEXEC=yes
child  pid 921719  fd 3 -> socket:[10076076]  FD_CLOEXEC=yes
```

`fork` + `exec` without `SOCK_CLOEXEC` leaks the listener into the new image. `listfd` still sees `fd 3` as that socket:

```
parent listening on 127.0.0.1:58355  fd=3  SOCK_CLOEXEC=no
parent pid 921720  fd 3 -> socket:[10076080]  FD_CLOEXEC=no
fd  0  inherit   /dev/null
fd  1  inherit   pipe:[10081799]
fd  2  inherit   pipe:[10081799]
fd  3  inherit   socket:[10076080]
fd  4  CLOEXEC   /proc/921721/fd  (opendir)
```

`fork` + `exec` *with* `SOCK_CLOEXEC` drops it. `fd 3` in the new image is `listfd`'s own `opendir`, not the listener:

```
parent listening on 127.0.0.1:45693  fd=3  SOCK_CLOEXEC=yes
parent pid 921722  fd 3 -> socket:[10076084]  FD_CLOEXEC=yes
fd  0  inherit   /dev/null
fd  1  inherit   pipe:[10081799]
fd  2  inherit   pipe:[10081799]
fd  3  CLOEXEC   /proc/921723/fd  (opendir)
```

A helper that inherited a listener can `accept()` connections the parent thought it owned. That is a correctness bug and, on a privileged socket, a privilege bug.

Case 3: Audit with `/proc/self/fd`

`/proc/self/fd/N` is a symlink to the open file. `fcntl(fd, F_GETFD)` reports the close-on-exec bit (`FD_CLOEXEC` is the value 1). Open one file without the flag and one with it, then `exec listfd`.

```c
/* audit_exec.c - open two files, one with O_CLOEXEC, then exec listfd */
#define _GNU_SOURCE
#include <fcntl.h>
#include <linux/close_range.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    const char *helper = (argc > 1) ? argv[1] : "./listfd";
```

```
    /* Isolate the demo from whatever the parent leaked to us. */
    if (close_range(3, ~0U, 0) == -1) {
        perror("close_range");
        return 1;
    }

    int plain = open("/dev/null", O_RDONLY);
    int marked = open("/dev/null", O_RDONLY | O_CLOEXEC);
    if (plain < 0 || marked < 0) {
        perror("open");
        return 1;
    }
    printf("parent: plain fd=%d  cloexec fd=%d  then exec %s\n",
           plain, marked, helper);
    fflush(stdout);
    execl(helper, "listfd", (char *)NULL);
    perror("execl");
    return 1;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o audit_exec audit_exec.c
./audit_exec ./listfd
```

```
parent: plain fd=3  cloexec fd=4  then exec ./listfd
fd  0  inherit   /dev/null
fd  1  inherit   pipe:[10081799]
fd  2  inherit   pipe:[10081799]
fd  3  inherit   /dev/null
fd  4  CLOEXEC   /proc/921724/fd  (opendir)
```

The unmarked `/dev/null` survived as fd 3. The `O_CLOEXEC` `/dev/null` is gone. fd 4 is a **new** descriptor `listfd` opened on `/proc/self/fd`; it is not the file the parent marked. Read `/proc/self/fd` by target and by the `CLOEXEC` column, not by number reuse.

Case 4: Atomic constructors

`open(2)` is explicit that `fcntl(F_SETFD)` after `open` is not enough in a multithreaded program. Every fd-creating call has (or grew) a flag that sets `FD_CLOEXEC` in the same syscall that allocates the descriptor.

```
/* atomic_ctors.c - pipe2, dup3, SOCK_CLOEXEC, F_DUPFD_CLOEXEC, accept4 */
#define _GNU_SOURCE
#include <arpa/inet.h>
#include <fcntl.h>
#include <linux/close_range.h>
```

```c
#include <netinet/in.h>
#include <stdio.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <unistd.h>

static void show(const char *tag, int fd)
{
    int flags = fcntl(fd, F_GETFD);
    printf("%-22s fd=%-3d FD_CLOEXEC=%s\n",
           tag, fd,
           (flags >= 0 && (flags & FD_CLOEXEC)) ? "yes" : "no");
}

int main(void)
{
    if (close_range(3, ~0U, 0) == -1) {
        perror("close_range");
        return 1;
    }

    int p[2];
    if (pipe2(p, O_CLOEXEC) == -1) {
        perror("pipe2");
        return 1;
    }
    show("pipe2(O_CLOEXEC) r", p[0]);
    show("pipe2(O_CLOEXEC) w", p[1]);

    int d = dup3(p[0], 20, O_CLOEXEC);
    if (d == -1) {
        perror("dup3");
        return 1;
    }
    show("dup3(..., O_CLOEXEC)", d);

    int extra = fcntl(p[0], F_DUPFD_CLOEXEC, 30);
    if (extra == -1) {
        perror("F_DUPFD_CLOEXEC");
        return 1;
    }
    show("F_DUPFD_CLOEXEC", extra);

    int sock = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);
    if (sock == -1) {
        perror("socket");
        return 1;
```

```
    }
    show("SOCK_CLOEXEC", sock);

    int lst = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);
    if (lst == -1) {
        perror("socket listen");
        return 1;
    }
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK),
        .sin_port = 0,
    };
    if (bind(lst, (struct sockaddr *)&addr, sizeof addr) == -1 ||
        listen(lst, 1) == -1) {
        perror("bind/listen");
        return 1;
    }
    socklen_t alen = sizeof addr;
    if (getsockname(lst, (struct sockaddr *)&addr, &alen) == -1) {
        perror("getsockname");
        return 1;
    }

    fflush(stdout);
    pid_t pid = fork();
    if (pid == 0) {
        int c = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);
        if (c == -1 || connect(c, (struct sockaddr *)&addr, sizeof addr) == -1)
            _exit(1);
        _exit(0);
    }
    int acc = accept4(lst, NULL, NULL, SOCK_CLOEXEC);
    if (acc == -1) {
        perror("accept4");
        return 1;
    }
    show("accept4(SOCK_CLOEXEC)", acc);
    int st;
    waitpid(pid, &st, 0);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o atomic_ctors atomic_ctors.c
./atomic_ctors
```

```
pipe2(O_CLOEXEC) r     fd=3   FD_CLOEXEC=yes
pipe2(O_CLOEXEC) w     fd=4   FD_CLOEXEC=yes
dup3(..., O_CLOEXEC)   fd=20  FD_CLOEXEC=yes
F_DUPFD_CLOEXEC        fd=30  FD_CLOEXEC=yes
SOCK_CLOEXEC           fd=5   FD_CLOEXEC=yes
accept4(SOCK_CLOEXEC)  fd=7   FD_CLOEXEC=yes
```

Instead of	Use
`pipe` + two `fcntl`s	`pipe2(p, O_CLOEXEC)`
`dup` / `dup2`	`dup3(old, new, O_CLOEXEC)` (`dup3` fails if `old == new`)
`fcntl(fd, F_DUPFD, n)`	`fcntl(fd, F_DUPFD_CLOEXEC, n)`
`socket(af, type, proto)`	`socket(af, type | SOCK_CLOEXEC, proto)`
`accept`	`accept4(s, addr, len, SOCK_CLOEXEC)`
`fopen(path, "r")`	`fopen(path, "re")` (`e` is POSIX.1-2008: `O_CLOEXEC`)

`dup()` and `dup2()` **clear** `FD_CLOEXEC` on the new descriptor, even when the old one had it set. `F_DUPFD` does the same. That is why `dup3` and `F_DUPFD_CLOEXEC` exist.

Case 5: `close_range`, including `CLOSE_RANGE_CLOEXEC`

`close_range(first, last, flags)` is one syscall for "everything from here up". `last = ~0U` means "to the end of the table". `flags = 0` closes. `CLOSE_RANGE_CLOEXEC` (Linux 5.11) **marks** instead of closing, which is the right move when you still need the descriptors until `execve` — for example a seccomp profile you install after marking, which then forbids `close_range` itself.

glibc 2.34 declares the wrapper in `<unistd.h>` under `_GNU_SOURCE`. The flag constants live in `<linux/close_range.h>`:

```
#define CLOSE_RANGE_UNSHARE (1U << 1)
#define CLOSE_RANGE_CLOEXEC (1U << 2)
```

```
/* close_range_demo.c - CLOSE_RANGE_CLOEXEC then a real close */
#define _GNU_SOURCE
#include <fcntl.h>
#include <linux/close_range.h>
#include <stdio.h>
#include <unistd.h>

static void show(const char *phase, int fd)
```

```
{
    int flags = fcntl(fd, F_GETFD);
    if (flags == -1) {
        printf("%s: fd %d  gone\n", phase, fd);
        return;
    }
    printf("%s: fd %d  FD_CLOEXEC=%s\n",
           phase, fd, (flags & FD_CLOEXEC) ? "yes" : "no");
}

int main(void)
{
    if (close_range(3, ~0U, 0) == -1) {
        perror("close_range");
        return 1;
    }

    int a = open("/dev/null", O_RDONLY);
    int b = open("/dev/null", O_RDONLY);
    int c = open("/dev/zero", O_RDONLY);
    if (a < 0 || b < 0 || c < 0) {
        perror("open");
        return 1;
    }
    printf("opened fds %d %d %d\n", a, b, c);
    show("after open", a);
    show("after open", b);
    show("after open", c);

    if (close_range((unsigned)a, ~0U, CLOSE_RANGE_CLOEXEC) == -1) {
        perror("close_range CLOEXEC");
        return 1;
    }
    show("after CLOEXEC flag", a);
    show("after CLOEXEC flag", b);
    show("after CLOEXEC flag", c);

    if (close_range((unsigned)a, ~0U, 0) == -1) {
        perror("close_range close");
        return 1;
    }
    show("after close", a);
    show("after close", b);
    show("after close", c);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o close_range_demo close_range_demo.c
./close_range_demo
```

```
opened fds 3 4 5
after open: fd 3  FD_CLOEXEC=no
after open: fd 4  FD_CLOEXEC=no
after open: fd 5  FD_CLOEXEC=no
after CLOEXEC flag: fd 3  FD_CLOEXEC=yes
after CLOEXEC flag: fd 4  FD_CLOEXEC=yes
after CLOEXEC flag: fd 5  FD_CLOEXEC=yes
after close: fd 3  gone
after close: fd 4  gone
after close: fd 5  gone
```

The usual pre-exec idiom from `close_range(2)`:

```
close_range(3, ~0U, CLOSE_RANGE_UNSHARE | CLOSE_RANGE_CLOEXEC);
execve(...);
```

`CLOSE_RANGE_UNSHARE` snapshots the fd table off any thread that still shares it (`CLONE_FILES`), then the mark applies to the private copy. Errors closing individual descriptors are ignored; a bad `flags` or `first > last` returns `-1` / `EINVAL`.

Case 6: The threads + `fork` race, shown as a window

`open(2)`:

> the use of this flag is essential in some multithreaded programs, because using a separate `fcntl(2)` `F_SETFD` operation to set the `FD_CLOEXEC` flag does not suffice to avoid race conditions where one thread opens a file descriptor and attempts to set its close-on-exec flag using `fcntl(2)` at the same time as another thread does a `fork(2)` plus `execve(2)`.

The window is two syscalls wide:

```
thread A:  fd = open(path, O_RDONLY);     /* FD_CLOEXEC clear */
           /*  ← thread B: fork(); execve(helper)  */
           fcntl(fd, F_SETFD, FD_CLOEXEC); /* too late for that child */
```

A single-threaded program can put `fork` in that same gap. That is this listing: `open` without `O_CLOEXEC`, `fork` + `exec`, *then* `fcntl` in the parent.

```
/* fcntl_window.c - the open / fcntl / fork window, shown without threads */
#define _GNU_SOURCE
#include <fcntl.h>
#include <linux/close_range.h>
#include <stdio.h>
```

```c
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    const char *helper = (argc > 1) ? argv[1] : "./listfd";

    if (close_range(3, ~0U, 0) == -1) {
        perror("close_range");
        return 1;
    }

    int fd = open("/dev/null", O_RDONLY); /* no O_CLOEXEC */
    if (fd < 0) {
        perror("open");
        return 1;
    }
    printf("parent: opened /dev/null as fd %d, about to fork, then fcntl\n", fd);
    fflush(stdout);

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        /* Child execs before the parent can set FD_CLOEXEC. */
        execl(helper, "listfd", (char *)NULL);
        perror("execl");
        _exit(1);
    }

    if (fcntl(fd, F_SETFD, FD_CLOEXEC) == -1) {
        perror("fcntl");
        return 1;
    }
    int flags = fcntl(fd, F_GETFD);
    printf("parent: fcntl(F_SETFD, FD_CLOEXEC) done; now FD_CLOEXEC=%s\n",
           (flags & FD_CLOEXEC) ? "yes" : "no");
    int st;
    waitpid(pid, &st, 0);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -O2 -o fcntl_window fcntl_window.c
./fcntl_window ./listfd
```

```
parent: opened /dev/null as fd 3, about to fork, then fcntl
fd  0  inherit   /dev/null
fd  1  inherit   pipe:[10081799]
fd  2  inherit   pipe:[10081799]
fd  3  inherit   /dev/null
fd  4  CLOEXEC   /proc/921729/fd  (opendir)
parent: fcntl(F_SETFD, FD_CLOEXEC) done; now FD_CLOEXEC=yes
```

The child's image still has `/dev/null` as fd 3, `inherit`. The parent's later `fcntl` cannot reach it. With threads you do not get to place the `fork`; another thread can. `O_CLOEXEC` on the `open` (or `pipe2` / `accept4` / …) closes the window because the descriptor is born marked.

Case 7: POSIX.1-2024 `O_CLOFORK`, and who actually has it

POSIX.1-2024 `open()`:

> The `FD_CLOEXEC` file descriptor flag associated with the new file descriptor shall be cleared unless the `O_CLOEXEC` flag is set in `oflag`. The `FD_CLOFORK` file descriptor flag associated with the new file descriptor shall be cleared unless the `O_CLOFORK` flag is set in `oflag`.

POSIX.1-2024 `fork()`:

> The child process shall have its own copy of the parent's file descriptors, except for those whose `FD_CLOFORK` flag is set.

`fcntl(F_SETFD)` with `FD_CLOFORK` set means the descriptor "shall not be inherited by any child process". `F_DUPFD_CLOFORK` is the dup analogue of `F_DUPFD_CLOEXEC`. `SOCK_CLOFORK` is specified on `socket()` / `accept4()` the same way.

That is exactly the prefork-server hole `O_CLOEXEC` does not fill. Linux does not ship it.

System	`O_CLOFORK` / `FD_CLOFORK`
Linux (this box 6.12; man-pages 6.19; mainline `fcntl.h` `VALID_OPEN_FLAGS` has `O_CLOEXEC` and not `O_CLOFORK`)	No. A 2011 patch was never merged; a later attempt was NAKed.
glibc 2.41, musl (as of the gnulib 2025 survey)	Macros not defined
FreeBSD 15.0 (landed 2025-07-06, `O_CLOFORK` `0x08000000`)	Yes
NetBSD 11 (kernel, 2025-07-16)	Yes

System	`O_CLOFORK` / `FD_CLOFORK`
OpenBSD 7.8	Yes
DragonFly BSD (master, 2025)	Yes
illumos, **Solaris 11.4**	Yes (Solaris had it first)
macOS 15, AIX 7.3	No, as of the 2025 gnulib survey

Austin Group defect 1851 asked whether `FD_CLOFORK` should survive `exec`. POSIX.1-2024 as published says it remains set on descriptors that stay open. OpenBSD, FreeBSD, NetBSD, Solaris 11.4.78, and illumos **clear** it on `exec`. The interpretation (2025-08) restates the published text and refers the concern to the sponsor, with a proposed fix that the flag "shall be cleared". On a system that has `O_CLOFORK`, do not assume the bit is still set after `execve`.

Until Linux grows the flag, a child that must not see a descriptor has to `close` it (or `close_range` it) itself. `O_CLOEXEC` only helps if that child `exec`s.

The trap

Treating `O_CLOEXEC` as "do not leak to children". It is close-on-**exec**. A long-lived `fork()` worker — a prefork HTTP server, a process pool, a privilege-separated helper that keeps running the same binary — inherits the listening socket, the log, the metrics pipe, and anything else you did not close. The other trap is `open` then `fcntl(F_SETFD, FD_CLOEXEC)` in a program that has threads. The man page calls that race out by name. `dup()` / `dup2()` / `F_DUPFD` are a third: they clear the flag on the copy.

The boring rule

Create every descriptor with the atomic `*CLOEXEC` constructor (`O_CLOEXEC`, `SOCK_CLOEXEC`, `pipe2`, `dup3`, `accept4`, `F_DUPFD_CLOEXEC`, `fopen(..., "re")`). In a child that will `exec`, sweep the rest with `close_range(3, ~0U, CLOSE_RANGE_CLOEXEC)` (or close them) *before* `execve`. In a child that will not `exec`, close what it must not hold; Linux has no `O_CLOFORK`. Never rely on `fcntl` after `open` if another thread can `fork`.

Try this

1. Run `listfd` from a shell and from a `system("./listfd")` (or `popen`) inside a program that has a listening socket. Compare the tables.
2. Change Case 1 to `pipe2(p, O_CLOEXEC)` and `exec` a reader in the child. Confirm EOF appears. Then `fork` without `exec` and confirm you still have to `close` the child's write end.
3. In Case 2, connect to the leaked listener from a third process while the helper is alive; `accept` in the helper. Then mark `SOCK_CLOEXEC` and show the helper can no longer `accept`.

4. Replace the `fcntl` in Case 6 with `open(..., O_RDONLY | O_CLOEXEC)` and confirm fd 3 is gone from the child's `listfd` output.
5. On a FreeBSD 15 or OpenBSD 7.8 box, rebuild Case 2 with `SOCK_CLOFORK` (or `O_CLOFORK` on a file) and confirm the child does not have the descriptor after `fork` with no `exec`.

Sources

- POSIX.1-2024 `open()`: https://pubs.opengroup.org/onlinepubs/9799919799/functions/open.html
- POSIX.1-2024 `fork()` (child fd table and `FD_CLOFORK`): https://pubs.opengroup.org/onlinepubs/9799919799/functions/fork.html
- POSIX.1-2024 `fcntl()` (`F_DUPFD_CLOEXEC`, `F_DUPFD_CLOFORK`, `FD_CLOEXEC`, `FD_CLOFORK`): https://pubs.opengroup.org/onlinepubs/9799919799/functions/fcntl.html
- Linux `open(2)` (`O_CLOEXEC` and the threads/`fork` race): https://man7.org/linux/man-pages/man2/open.2.html
- Linux `fcntl(2)` (VERSIONS: `FD_CLOFORK` / `F_DUPFD_CLOFORK` not on Linux; man-pages 6.19): https://man7.org/linux/man-pages/man2/fcntl.2.html
- Linux `close_range(2)` (`CLOSE_RANGE_CLOEXEC` since 5.11, glibc 2.34): https://man7.org/linux/man-pages/man2/close_range.2.html
- Linux `pipe(2)` / `pipe2`, `dup(2)` / `dup3`, `accept(2)` / `accept4`, `socket(2)` (`SOCK_CLOEXEC`; `SOCK_CLOFORK` specified by POSIX.1-2024 and not implemented on Linux)
- UAPI `close_range` flags: `<linux/close_range.h>` on this box; kernel `include/uapi/linux/close_range.h`
- glibc `close_range` wrapper: `<unistd.h>` under `_GNU_SOURCE` (glibc 2.34); descriptor flags: https://www.gnu.org/software/libc/manual/html_node/Descriptor-Flags.html
- Austin Group defect 1851 (`FD_CLOFORK` across `exec`): https://www.austingroupbugs.net/view.php?id=1851
- Linux man-pages thread, 2025-04-21: POSIX.1-2024 added `*_CLOFORK`; Linux unsupported, earlier idea NAKed — https://www.spinics.net/lists/linux-man/msg29355.html
- FreeBSD 15 `O_CLOFORK` (2025-07-06): https://cgit.freebsd.org/src/commit/?id=02ffca404e65b2058720e1e9d5
- NetBSD kernel `O_CLOFORK` (2025-07-16): http://mail-index.netbsd.org/source-changes/2025/07/16/msg157460.html
- OpenBSD 7.8 `open(2)` (`O_CLOFORK`): https://man.openbsd.org/OpenBSD-7.8/open.2

fork(), vfork() or posix_spawn(): Starting Programs in Modern C

`fork()`, `vfork()` or `posix_spawn()`: Starting Programs in Modern C

Thesis

The textbook way to run another program is `fork()` followed by `exec()`. It is short and flexible, and it gets expensive as the parent's memory grows. `fork()` has to duplicate the parent's address space (page tables, with every page marked copy-on-write) only for `exec()` to throw that copy away immediately. The 2019 HotOS paper *A fork() in the road* argues that `fork` is a poor default for this job. This chapter measures the cost and shows the replacement: **`posix_spawn()`**. It is as fast as `vfork()`, it reports exec failures directly, and it configures the child declaratively instead of with arbitrary code between `fork` and `exec`.

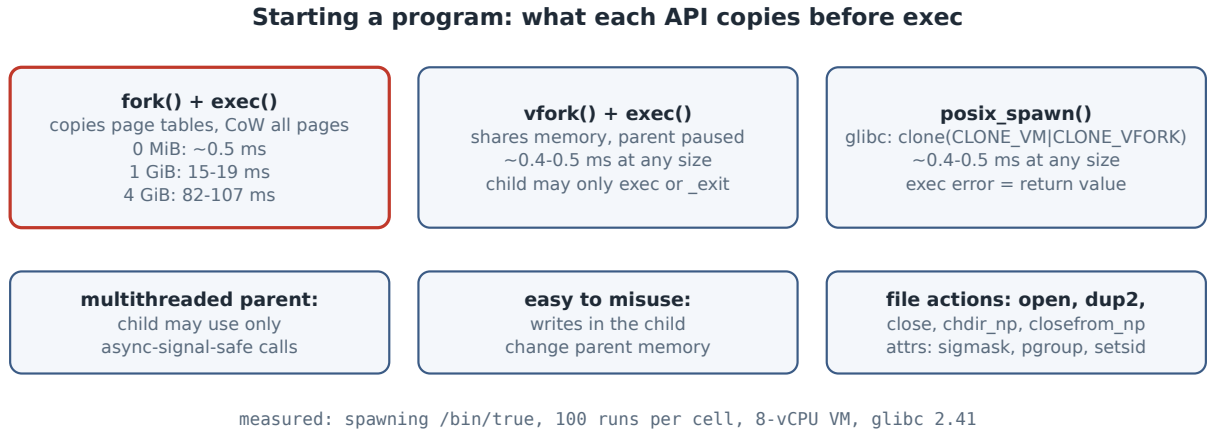


Figure 1: Starting a program: what each API copies before exec

Everything was run with GCC 14.2.0 and glibc 2.41 on Linux 6.12, on an 8-vCPU x86-64 VM with 16 GB of RAM.

The three APIs

	`fork()` + `exec()`	`vfork()` + `exec()`	`posix_spawn()`
Child's memory	copy-on-write duplicate	**shared** with the parent	shared (glibc 2.24)
Parent during the gap	keeps running	suspended until `exec`/`_exit`	suspended until `exec`

	`fork()` + `exec()`	`vfork()` + `exec()`	`posix_spawn()`
What the child may do before `exec`	anything (single-threaded parent); only async-signal-safe calls (multithreaded parent)	only `exec` or `_exit`	nothing: you describe it with file actions and attributes
Exec failure reported as	child exit status you invent	child exit status you invent	return value
Standard	POSIX	removed from POSIX.1-2008	POSIX

According to the `posix_spawn(3)` man page, "since glibc 2.24, the posix_spawn() function commences by calling clone(2) with CLONE_VM and CLONE_VFORK flags." In other words, glibc does the `vfork` trick for you, on a separate stack and with the dangerous parts handled.

Measurement: cost versus parent memory

The parent maps and touches a heap of 0, 1 or 4 GiB, then starts `/bin/true` 100 times with each method and waits for every child. The heap is `mmap`ed and the program calls `madvise` on it. The `4k` argument asks for no transparent huge pages, and the default asks for them. `AnonHuge` reports how much of the heap actually received huge pages.

```
// spawn.c: cost of starting /bin/true from a parent with a large resident heap
#define _GNU_SOURCE
#include <spawn.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

extern char **environ;
static char *argv_true[] = {"/bin/true", NULL};

static pid_t via_fork(void) {
    pid_t pid = fork();
    if (pid == 0) { execv(argv_true[0], argv_true); _exit(127); }
    return pid;
}
static pid_t via_vfork(void) {
    pid_t pid = vfork();
    if (pid == 0) { execv(argv_true[0], argv_true); _exit(127); }
```

```
    return pid;
}
static pid_t via_spawn(void) {
    pid_t pid;
    int e = posix_spawn(&pid, argv_true[0], NULL, NULL, argv_true, environ);
    return e ? -1 : pid;
}

static long anon_huge_kib(void) {
    FILE *f = fopen("/proc/self/smaps_rollup", "r");
    char line[256]; long kib = -1;
    while (f && fgets(line, sizeof line, f))
        if (sscanf(line, "AnonHugePages: %ld kB", &kib) == 1) break;
    if (f) fclose(f);
    return kib;
}

static double bench(pid_t (*start)(void), int n) {
    struct timespec a, b;
    clock_gettime(CLOCK_MONOTONIC, &a);
    for (int i = 0; i < n; i++) {
        int st; pid_t p = start();
        if (p < 0 || waitpid(p, &st, 0) < 0 || !WIFEXITED(st) || WEXITSTATUS(st)) {
            fprintf(stderr, "spawn failed\n"); exit(1);
        }
    }
    clock_gettime(CLOCK_MONOTONIC, &b);
    return ((b.tv_sec - a.tv_sec) * 1e9 + (b.tv_nsec - a.tv_nsec)) / n / 1e3;
}

int main(int argc, char **argv) {
    size_t mib = argc > 1 ? strtoul(argv[1], NULL, 10) : 0;
    int small_pages = argc > 2 && strcmp(argv[2], "4k") == 0;
    char *heap = NULL;
    if (mib) {
        heap = mmap(NULL, mib << 20, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1
        if (heap == MAP_FAILED) { perror("mmap"); return 1; }
        madvise(heap, mib << 20, small_pages ? MADV_NOHUGEPAGE : MADV_HUGEPAGE);
        memset(heap, 1, mib << 20);           /* make it resident */
    }
    int n = 100;
    printf("heap %5zu MiB %s (AnonHuge %7ld MiB):  fork+exec %8.1f us   vfork+exec %6.1f us
           mib, small_pages ? "4K " : "THP", anon_huge_kib() >> 10, bench(via_fork, n), benc
    if (heap) munmap(heap, mib << 20);
    return 0;
}
```

```
$ gcc -std=c23 -O2 -Wall -Wextra spawn.c -o spawn
$ for m in 0 1024 4096; do ./spawn $m; ./spawn $m 4k; done
heap     0 MiB THP (AnonHuge       0 MiB):  fork+exec    545.7 us   vfork+exec  397.4 us   p
heap     0 MiB 4K  (AnonHuge       0 MiB):  fork+exec    546.3 us   vfork+exec  414.8 us   p
heap  1024 MiB THP (AnonHuge     178 MiB):  fork+exec  15299.9 us   vfork+exec  441.4 us   p
heap  1024 MiB 4K  (AnonHuge       0 MiB):  fork+exec  19366.4 us   vfork+exec  475.4 us   p
heap  4096 MiB THP (AnonHuge     178 MiB):  fork+exec 106975.3 us   vfork+exec  499.0 us   p
heap  4096 MiB 4K  (AnonHuge       0 MiB):  fork+exec  82345.9 us   vfork+exec  437.6 us   p
```

- **`fork+exec` grows with resident memory:** about 0.55 ms with no heap, 15–19 ms with 1 GiB and 82–107 ms with 4 GiB. That is two orders of magnitude slower for the same `/bin/true`.
- **`vfork` and `posix_spawn` stay flat** at 0.38–0.52 ms at every heap size, because they never copy the address space.
- The THP rows show only 178 MiB of huge pages actually allocated, so this run tells you nothing about huge pages either way. The 4 KiB rows are the clean comparison.

The cost isn't only time. A `fork` from a 4 GiB process also briefly *commits* another 4 GiB of copy-on-write memory, which can fail outright under `vm.overcommit_memory=2` even though the child is about to `exec` (fork(2) lists `ENOMEM`).

Errors and child setup

```c
// errs.c: how each API reports "the program does not exist", plus file actions
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <spawn.h>
#include <stdio.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

extern char **environ;

int main(void) {
    char *bad[] = {"/no/such/program", NULL};

    /* fork + exec: the error happens in the child; the parent sees only a status */
    pid_t pid = fork();
    if (pid == 0) { execv(bad[0], bad); _exit(127); }
    int st; waitpid(pid, &st, 0);
    printf("fork+exec:   fork() succeeded, child exit status %d\n", WEXITSTATUS(st));

    /* posix_spawn: glibc reports the exec failure as the return value */
```

```
    int e = posix_spawn(&pid, bad[0], NULL, NULL, bad, environ);
    printf("posix_spawn: returned %d (%s)\n", e, strerror(e));

    /* file actions: run `pwd` in /tmp with stdout redirected to a file */
    posix_spawn_file_actions_t fa;
    posix_spawn_file_actions_init(&fa);
    posix_spawn_file_actions_addopen(&fa, STDOUT_FILENO, "/tmp/spawn-out.txt",
                                     O_WRONLY | O_CREAT | O_TRUNC, 0644);
    posix_spawn_file_actions_addchdir_np(&fa, "/tmp");
    char *pwd[] = {"pwd", NULL};
    e = posix_spawnp(&pid, "pwd", &fa, NULL, pwd, environ);
    posix_spawn_file_actions_destroy(&fa);
    waitpid(pid, &st, 0);
    printf("posix_spawnp(pwd): returned %d, child status %d\n", e, WEXITSTATUS(st));
    return 0;
}
```

```
$ gcc -std=c23 -O2 -Wall -Wextra errs.c -o errs && ./errs && cat /tmp/spawn-out.txt
fork+exec:   fork() succeeded, child exit status 127
posix_spawn: returned 2 (No such file or directory)
posix_spawnp(pwd): returned 0, child status 0
/tmp
```

With `fork`+`exec`, the parent learns only that the child exited with 127, the conventional "command not found" code that this program chose itself. To get the real `errno` you need a close-on-exec pipe so the child can send it back. `posix_spawn` returns `ENOENT` (2) directly.

The child is configured with **file actions**, which are applied in order in the child before `exec`. Here they open a file as stdout and `chdir` to `/tmp`. The available actions are `addopen`, `adddup2`, `addclose`, and the glibc extensions `addchdir_np` (glibc 2.29) and `addclosefrom_np` (glibc 2.34). **Attributes** (`posix_spawnattr_*`) set the signal mask, default signal dispositions, process group and session. These cover most of what people write between `fork` and `exec`.

When to use which

- **Starting a program: `posix_spawn`/`posix_spawnp`.** It is fast at any heap size, reports errors properly and is safe in multithreaded programs.
- **`fork` without `exec`:** prefork servers, snapshotting (Redis `BGSAVE`), sandboxes that fork a template. In these cases the copy is the point. Remember that in a **multithreaded** parent the child may only use async-signal-safe functions until it calls `exec`, because other threads' locks (including malloc's) may be held forever (POSIX `fork`).
- **`vfork`: almost never directly.** The child shares the parent's memory and stack. Any write other than calling `exec` or `_exit`, including returning from the function that called `vfork`, corrupts the parent (`vfork(2)`). `posix_spawn` gives you the same speed safely.
- **Needs `posix_spawn` can't express** (seccomp filters, namespaces, `setrlimit` in the child): use `clone3`/`pidfd`-based helpers or a small exec-wrapper program. A `fork` from a large

process is the expensive option here too.

Pitfalls

- **`system()` and `popen()` use a shell.** glibc runs them through `/bin/sh -c`. They cost an extra exec and bring shell quoting bugs. Use `posix_spawnp` with an argv array.
- **Descriptor leaks.** The spawned child inherits every descriptor that isn't close-on-exec. Create descriptors with `O_CLOEXEC`, or add `addclosefrom_np`, before spawning.
- **`posix_spawnp` searches `PATH`; `posix_spawn` doesn't.** Pass an absolute path to `posix_spawn`.
- **Exit status 127 is just a convention.** In a `fork`+`exec` error path it is ambiguous with a real program that exits 127. Prefer an errno pipe or `posix_spawn`.

Takeaways

- `fork`+`exec` cost grows with the parent's resident memory: 0.55 ms → about 19 ms (1 GiB) → about 82 ms (4 GiB) here. `posix_spawn` stayed at about 0.5 ms.
- glibc's `posix_spawn` is a safe `vfork`: `clone(CLONE_VM | CLONE_VFORK)` on its own stack.
- It returns exec errors directly and configures the child with file actions and attributes.
- Use `fork` when you want the copy, and `posix_spawn` when you want a new program.

Sources

- Linux man-pages, `posix_spawn(3)`, `fork(2)`, `vfork(2)`
- POSIX.1-2024, `fork` (async-signal-safe rule in multithreaded processes)
- Baumann, Appavoo, Krieger, Roscoe, *A fork() in the road*, HotOS 2019 (PDF)

Work Stealing in C: a Chase–Lev Deque with C11 Atomics

Work Stealing in C: a Chase–Lev Deque with C11 Atomics

Thesis

A work-stealing scheduler gives every worker thread its own double-ended queue of tasks. The owner pushes and pops at the **bottom**, like a stack: newest first and cache-hot. It takes no lock. An idle worker becomes a **thief** and steals from the **top** of someone else's deque: oldest first, which in divide-and-conquer code is the *biggest* chunk of work. Cilk made this scheduler famous in the 1990s. Blumofe and Leiserson proved it efficient: randomized stealing has provably good time and space bounds. Chase and Lev (SPAA 2005) gave the lock-free, growable deque most runtimes still use.

The data structure is about 100 lines of C11. Every line of it can be wrong in a way that passes most test runs. The owner's `take()` needs a **sequentially consistent fence** between "I lowered `bottom`" and "let me read `top`". Without it, an owner and a thief can both walk away with the same task, even on x86. This chapter builds the deque with the C11 orderings from Lê, Pop, Cohen and Zappa Nardelli (PPoPP 2013), proves it with a counting stress test, removes the fence to watch it break, and then uses it to run a tiny parallel scheduler.

Mental model

```
               top (thieves CAS here)                bottom (owner only)

index:   0  1  2 3      4      5      6      7      8  9
slots: [ ][ ][ ][A0  ][A1   ][A2   ][ ...  ][A7   ][ ][ ]      items live in [top, bottom)
                 oldest = biggest chunk      newest = smallest chunk

push(x)   owner:  buf[bottom] = x;  release fence;  bottom++
take()    owner:  bottom--;  SEQ_CST FENCE;  t = top
                  t <  bottom → item is mine, no CAS
                  t == bottom → last item: CAS top, t → t+1 (race the thieves)
                  t >  bottom → empty, restore bottom
steal()   thief:  t = top;  SEQ_CST FENCE;  b = bottom
                  t < b → read buf[t], CAS top, t → t+1 (fail = ABORT, someone else won)
```

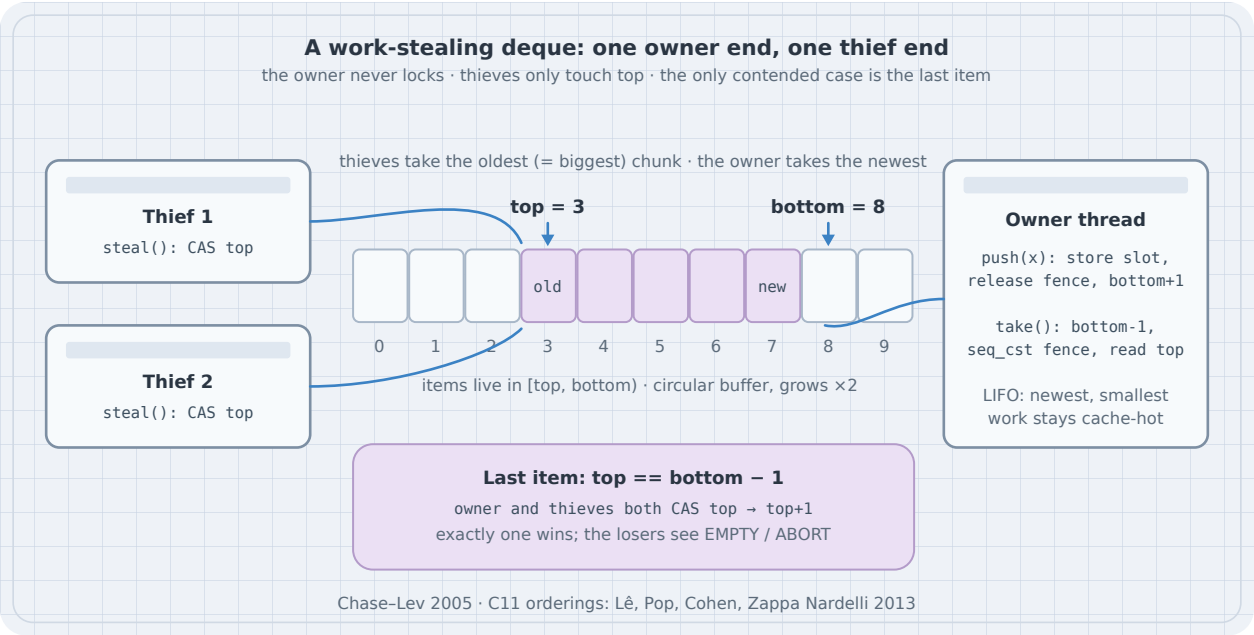


Figure 1: Chase–Lev deque: thieves steal at top with a CAS, the owner pushes and takes at bottom, and the last item is decided by a CAS on top

Operation	Who	Atomics it needs	Why
`push`	owner	release fence before `bottom++`	A thief that sees the new `bottom` must see the item
`take`	owner	**seq_cst fence** between the `bottom` store and the `top` load; seq_cst CAS on the last item	Either the owner sees a thief's `top`, or the thief sees the owner's `bottom`. Never neither
`steal`	any	acquire `top`, **seq_cst fence**, acquire `bottom`; seq_cst CAS on `top`	Pairs with `take`'s fence. The CAS makes "one winner per slot" true

Operation	Who	Atomics it needs	Why
grow	owner	release store of the new array pointer	A thief that loads the new pointer sees the copied items

`top` only ever increases, so a 64-bit CAS on it has no ABA problem. `bottom` is written only by the owner. The only contended moment is the **last item**, where the owner, too, has to win a CAS.

Worked examples

Tested on Debian 13 x86_64 (8 vCPUs, a shared VM), Linux 6.12, **GCC 14.2.0** (`gcc (Debian 14.2.0-19)`), glibc 2.41, `-std=c17`. Nothing here depends on a newer compiler. Current upstream GCC (16.1) reads the same `<stdatomic.h>` API, and C23 keeps it apart from removing `ATOMIC_VAR_INIT`. The deque lives in two files that every case links against.

```
/* chase_lev.h - Chase-Lev work-stealing deque with C11 atomics (interface)
 * Algorithm: Chase & Lev, SPAA 2005; C11 orderings after Lê, Pop, Cohen,
 * Zappa Nardelli, PPoPP 2013. One owner pushes/takes at the bottom;
 * any number of thieves steal from the top. */
#ifndef CHASE_LEV_H
#define CHASE_LEV_H
#include <stdatomic.h>
#include <stddef.h>
#include <stdint.h>

#define CL_EMPTY INTPTR_MIN        /* take/steal: nothing there        */
#define CL_ABORT (INTPTR_MIN + 1)  /* steal: lost a race, may retry    */

#ifdef CL_UNSIGNED                 /* deliberately wrong: see Case 4   */
typedef size_t cl_index;
#else
typedef int64_t cl_index;
#endif

typedef struct cl_array {
    cl_index size;                 /* power of two                     */
    struct cl_array *prev;         /* retired arrays, freed at destroy */
    _Atomic intptr_t buf[];
} cl_array;

typedef struct {
```

```
    _Alignas(64) _Atomic cl_index top;     /* thieves CAS this end    */
    _Alignas(64) _Atomic cl_index bottom;  /* only the owner writes   */
    _Alignas(64) _Atomic(cl_array *) array;
} cl_deque;

void     cl_init(cl_deque *q, cl_index size);    /* size: power of two */
void     cl_destroy(cl_deque *q);
void     cl_push(cl_deque *q, intptr_t x);       /* owner only */
intptr_t cl_take(cl_deque *q);                   /* owner only */
intptr_t cl_steal(cl_deque *q);                  /* any thread */
#endif
```

```
/* chase_lev.c - Chase-Lev deque implementation (one owner, many thieves) */
#include <stdlib.h>
#include "chase_lev.h"

static cl_array *cl_array_new(cl_index size)
{
    cl_array *a = malloc(sizeof *a + (size_t)size * sizeof a->buf[0]);
    if (!a)
        abort();
    a->size = size;
    a->prev = NULL;
    for (cl_index i = 0; i < size; i++)
        atomic_init(&a->buf[i], 0);
    return a;
}

void cl_init(cl_deque *q, cl_index size)
{
    atomic_init(&q->top, 0);
    atomic_init(&q->bottom, 0);
    atomic_init(&q->array, cl_array_new(size));
}

void cl_destroy(cl_deque *q)
{
    cl_array *a = atomic_load_explicit(&q->array, memory_order_relaxed);
    while (a) {
        cl_array *p = a->prev;
        free(a);
        a = p;
    }
}

/* Owner only. Thieves may still be reading the old array, so it is
```

```
 * retired, not freed. */
static cl_array *cl_grow(cl_deque *q, cl_array *a, cl_index t, cl_index b)
{
    cl_array *n = cl_array_new(a->size * 2);
    for (cl_index i = t; i < b; i++)
        atomic_store_explicit(&n->buf[i & (n->size - 1)],
            atomic_load_explicit(&a->buf[i & (a->size - 1)], memory_order_relaxed),
            memory_order_relaxed);
    n->prev = a;
    atomic_store_explicit(&q->array, n, memory_order_release);
    return n;
}

void cl_push(cl_deque *q, intptr_t x)               /* owner */
{
    cl_index b = atomic_load_explicit(&q->bottom, memory_order_relaxed);
    cl_index t = atomic_load_explicit(&q->top, memory_order_acquire);
    cl_array *a = atomic_load_explicit(&q->array, memory_order_relaxed);
    if (b - t > a->size - 1)
        a = cl_grow(q, a, t, b);
    atomic_store_explicit(&a->buf[b & (a->size - 1)], x, memory_order_relaxed);
    atomic_thread_fence(memory_order_release);  /* item before bottom */
    atomic_store_explicit(&q->bottom, b + 1, memory_order_relaxed);
}

intptr_t cl_take(cl_deque *q)                       /* owner */
{
    cl_index b = atomic_load_explicit(&q->bottom, memory_order_relaxed) - 1;
    cl_array *a = atomic_load_explicit(&q->array, memory_order_relaxed);
    atomic_store_explicit(&q->bottom, b, memory_order_relaxed);
#ifndef CL_NO_TAKE_FENCE                        /* deliberately wrong: Case 3 */
    atomic_thread_fence(memory_order_seq_cst);  /* bottom store before top load */
#endif
    cl_index t = atomic_load_explicit(&q->top, memory_order_relaxed);
    intptr_t x;
    if (t <= b) {                                   /* non-empty */
        x = atomic_load_explicit(&a->buf[b & (a->size - 1)], memory_order_relaxed);
        if (t == b) {                               /* last item: race the thieves */
            if (!atomic_compare_exchange_strong_explicit(&q->top, &t, t + 1,
                    memory_order_seq_cst, memory_order_relaxed))
                x = CL_EMPTY;                       /* a thief won */
            atomic_store_explicit(&q->bottom, b + 1, memory_order_relaxed);
        }
    } else {                                        /* was empty: undo */
        x = CL_EMPTY;
        atomic_store_explicit(&q->bottom, b + 1, memory_order_relaxed);
    }
```

```
    return x;
}

intptr_t cl_steal(cl_deque *q)                          /* any thread */
{
    cl_index t = atomic_load_explicit(&q->top, memory_order_acquire);
    atomic_thread_fence(memory_order_seq_cst);  /* top load before bottom load */
    cl_index b = atomic_load_explicit(&q->bottom, memory_order_acquire);
    if (!(t < b))
        return CL_EMPTY;
    cl_array *a = atomic_load_explicit(&q->array, memory_order_acquire);
    intptr_t x = atomic_load_explicit(&a->buf[t & (a->size - 1)], memory_order_relaxed);
    if (!atomic_compare_exchange_strong_explicit(&q->top, &t, t + 1,
            memory_order_seq_cst, memory_order_relaxed))
        return CL_ABORT;                        /* another thief or the owner won */
    return x;
}
```

Two `-D` switches exist only so later cases can break the code on purpose. `CL_NO_TAKE_FENCE` deletes the one fence in `take()`, and `CL_UNSIGNED` makes the indices `size_t`. Never ship either.

Case 1: One thread, both ends

```
/* basic.c - single-threaded behaviour: LIFO for the owner, FIFO for thieves */
#include <stdio.h>
#include "chase_lev.h"

int main(void)
{
    cl_deque q;
    cl_init(&q, 2);                   /* tiny on purpose: forces two grows */
    for (intptr_t i = 1; i <= 5; i++)
        cl_push(&q, i);
    printf("array size after 5 pushes: %lld\n",
           (long long)atomic_load(&q.array)->size);
    printf("steal -> %ld (oldest)\n", (long)cl_steal(&q));
    printf("take  -> %ld (newest)\n", (long)cl_take(&q));
    printf("take  -> %ld\n", (long)cl_take(&q));
    printf("steal -> %ld\n", (long)cl_steal(&q));
    printf("take  -> %ld\n", (long)cl_take(&q));
    intptr_t e = cl_take(&q);
    printf("take  -> %s\n", e == CL_EMPTY ? "EMPTY" : "?");
    cl_destroy(&q);
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o basic basic.c chase_lev.c && ./basic
array size after 5 pushes: 8
steal -> 1 (oldest)
take  -> 5 (newest)
take  -> 4
steal -> 2
take  -> 3
take  -> EMPTY
```

The capacity started at 2 and grew twice ($2 \to 4 \to 8$) during five pushes. Thieves see FIFO order (1, then 2). The owner sees LIFO order (5, 4, 3). The empty sentinel is `INTPTR_MIN`, so items are any `intptr_t` except the two sentinels. In a real scheduler an item is a task pointer.

Case 2: One owner, seven thieves, two million items

The test has to *count*. Every item is a number from 1 to N, and `seen[v]` counts how many times `v` came out. A correct deque gives `lost 0 duplicated 0` every time.

```c
/* stress.c - one owner, several thieves: every item must come out exactly once */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include "chase_lev.h"

enum { N = 2000000, THIEVES = 7 };

static cl_deque q;
static _Atomic unsigned char seen[N + 1];
static _Atomic long consumed;
static long stolen[THIEVES], aborts[THIEVES];

static void eat(intptr_t v)
{
    atomic_fetch_add_explicit(&seen[v], 1, memory_order_relaxed);
    atomic_fetch_add_explicit(&consumed, 1, memory_order_relaxed);
}

static void *thief(void *arg)
{
    long id = (long)arg;
    while (atomic_load_explicit(&consumed, memory_order_relaxed) < N) {
        intptr_t v = cl_steal(&q);
        if (v == CL_ABORT)
            aborts[id]++;
        else if (v != CL_EMPTY) {
            eat(v);
```

```
            stolen[id]++;
        }
    }
    return NULL;
}

int main(void)
{
    cl_init(&q, 64);
    pthread_t th[THIEVES];
    for (long i = 0; i < THIEVES; i++)
        pthread_create(&th[i], NULL, thief, (void *)i);

    long taken = 0;
    for (intptr_t v = 1; v <= N; v++) {
        cl_push(&q, v);
        if (v % 3 == 0) {                 /* owner works on its newest items */
            intptr_t x = cl_take(&q);
            if (x != CL_EMPTY) { eat(x); taken++; }
        }
    }
    for (intptr_t x; (x = cl_take(&q)) != CL_EMPTY; taken++)
        eat(x);                           /* drain what thieves left */
    for (int i = 0; i < THIEVES; i++)
        pthread_join(th[i], NULL);

    long lost = 0, dup = 0, st = 0, ab = 0;
    for (long v = 1; v <= N; v++) {
        unsigned c = atomic_load(&seen[v]);
        lost += (c == 0);
        dup += (c > 1);
    }
    for (int i = 0; i < THIEVES; i++) { st += stolen[i]; ab += aborts[i]; }
    printf("items %d  owner took %ld  thieves stole %ld  steal aborts %ld\n",
           N, taken, st, ab);
    printf("final array size %lld\n", (long long)atomic_load(&q.array)->size);
    printf("lost %ld  duplicated %ld  -> %s\n", lost, dup,
           lost == 0 && dup == 0 ? "OK" : "BROKEN");
    cl_destroy(&q);
    return lost || dup;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -pthread -o stress stress.c chase_lev.c
$ for i in 1 2 3; do ./stress; done
items 2000000  owner took 666667  thieves stole 1333333  steal aborts 678870
final array size 262144
```

```
lost 0  duplicated 0  -> OK
items 2000000  owner took 669554  thieves stole 1330446  steal aborts 788784
final array size 262144
lost 0  duplicated 0  -> OK
items 2000000  owner took 675706  thieves stole 1324294  steal aborts 786425
final array size 131072
lost 0  duplicated 0  -> OK
```

"Steal aborts" are CAS losses: another thief, or the owner taking the last item, got there first. They are normal and cheap. A thief just tries again. The final array size varies from run to run because it depends on how far ahead of the thieves the owner got.

Case 3: Delete one fence and the deque duplicates and loses work

```
$ gcc -std=c17 -Wall -Wextra -O2 -pthread -DCL_NO_TAKE_FENCE -o stress_nofence stress.c chas
$ for i in 1 2 3; do ./stress_nofence; echo "exit $?"; done
items 2000000  owner took 669678  thieves stole 1330322  steal aborts 628972
final array size 524288
lost 0  duplicated 0  -> OK
exit 0
items 2000000  owner took 668223  thieves stole 1331777  steal aborts 700731
final array size 32768
lost 0  duplicated 0  -> OK
exit 0
items 2000000  owner took 667793  thieves stole 1332207  steal aborts 431824
final array size 524288
lost 0  duplicated 0  -> OK
exit 0
```

Three green runs. This is exactly what makes concurrency bugs dangerous. Now run each build ten times and count the failures:

```
#!/usr/bin/env bash
# hunt.sh - run the stress test 10 times per build; count runs that lost or duplicated items
set -u
for flags in "-O2" "-O2 -DCL_NO_TAKE_FENCE" "-O2 -flto -DCL_NO_TAKE_FENCE" "-O0 -DCL_NO_TAKE
  gcc -std=c17 -Wall -Wextra -pthread $flags -o hunt_bin stress.c chase_lev.c || exit 1
  broken=0
  for i in $(seq 10); do ./hunt_bin > /dev/null || broken=$((broken + 1)); done
  printf '%-30s broken in %2d of 10 runs\n' "$flags" "$broken"
done
```

```
$ ./hunt.sh
-O2                            broken in  0 of 10 runs
-O2 -DCL_NO_TAKE_FENCE         broken in  3 of 10 runs
```

```
-O2 -flto -DCL_NO_TAKE_FENCE   broken in  2 of 10 runs
-O0 -DCL_NO_TAKE_FENCE         broken in  2 of 10 runs
```

The correct `-O2` build passed 10 of 10 runs. The build without the fence lost and duplicated items in some runs at every optimisation level, including `-flto`, where `take()` is inlined into the owner loop. The failure rate changes from batch to batch. While writing this chapter, the same no-fence source also went 20 of 20 clean in a header-only, `static inline` build. The bug is in the source. Whether you *see* it depends on inlining and timing.

Here is what happens without the fence, with two items at `top = t` and `t+1`:

1. The owner lowers `bottom` to `t+1`. On x86 that store sits in the core's **store buffer**. x86-TSO lets a later load pass an earlier store, so the owner's load of `top` can complete first and read `t`.
2. A thief steals slot `t` (`top` becomes `t+1`). Then a thief (the same one or another) reads `top = t+1` and still sees the *old* `bottom = t+2`, because the owner's store has not left its store buffer. It takes slot `t+1` with a successful CAS.
3. The owner compares its stale `t` with `b = t+1`, decides "more than one item, no CAS needed", and also takes slot `t+1`. That item is **duplicated**. `top` (`t+2`) is now past `bottom` (`t+1`), so the owner's next push lands in a slot no thief will ever read. That item is **lost**. This is why the counts above are always "lost N duplicated N".

The seq_cst fence in `take()` pairs with the one in `steal()`. Of the two threads, at least one must see the other's write. You can see what it costs on x86:

```
$ gcc -std=c17 -O2 -S -o - chase_lev.c | grep -E '^cl_[a-z]+:|lock|mfence'
cl_init:
cl_destroy:
cl_push:
cl_take:
    lock orq    $0, (%rsp)
    lock cmpxchgq   %r8, (%rdi)
cl_steal:
    lock orq    $0, (%rsp)
    lock cmpxchgq   %rcx, (%rdi)
$ gcc -std=c17 -O2 -DCL_NO_TAKE_FENCE -S -o - chase_lev.c | grep -E '^cl_[a-z]+:|lock|mfence
cl_init:
cl_destroy:
cl_push:
cl_take:
    lock cmpxchgq   %r8, (%rdi)
cl_steal:
    lock orq    $0, (%rsp)
    lock cmpxchgq   %rcx, (%rdi)
```

GCC 14 implements a seq_cst fence as `lock or $0,(%rsp)`, a locked no-op on the stack that drains the store buffer. `cl_push` has no locked instruction at all: a release fence on x86 only stops the *compiler* from reordering. Removing the fence from `take()` saves one locked instruction per pop and costs you correctness.

Case 4: Unsigned indices turn “empty” into “an item”

`take()` begins with `b = bottom - 1`. With `bottom == 0` and a signed index, `b` is -1 and `t <= b` is false: empty. With `size_t`, `b` wraps to 2^64 − 1 and `t <= b` is true:

```
/* unsigned_trap.c - take() on an empty deque, signed vs unsigned indices */
#include <stdio.h>
#include "chase_lev.h"

int main(void)
{
    cl_deque q;
    cl_init(&q, 4);
    intptr_t x = cl_take(&q);                  /* nothing was ever pushed */
    cl_index t = atomic_load(&q.top), b = atomic_load(&q.bottom);
    printf("index type: %s\n", (cl_index)-1 > 0 ? "unsigned (size_t)" : "signed (int64_t)");
    printf("take on empty -> %s\n", x == CL_EMPTY ? "EMPTY" : "an item (!)");
    printf("afterwards: top=%llu bottom=%llu, next push sees b - t = %llu\n",
           (unsigned long long)t, (unsigned long long)b,
           (unsigned long long)(b - t));
    cl_destroy(&q);
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -o trap_signed unsigned_trap.c chase_lev.c && ./trap_signed
index type: signed (int64_t)
take on empty -> EMPTY
afterwards: top=0 bottom=0, next push sees b - t = 0
$ gcc -std=c17 -Wall -Wextra -O2 -DCL_UNSIGNED -o trap_unsigned unsigned_trap.c chase_lev.c
index type: unsigned (size_t)
take on empty -> an item (!)
afterwards: top=0 bottom=18446744073709551615, next push sees b - t = 18446744073709551615
```

The unsigned build returned a garbage “item” from a deque that never held one. It also left `bottom` at 2^64 − 1, so the next `push` would compute `b - t` as 2^64 − 1 and try to grow the array forever. Use a signed 64-bit index (`int64_t` or `ptrdiff_t`). It cannot overflow in practice: at a billion pushes a second it lasts about 292 years.

Case 5: ThreadSanitizer does not see fences

```
$ gcc -std=c17 -Wall -Wextra -O1 -g -pthread -fsanitize=thread -o stress_tsan stress.c chase
$ ./stress_tsan
items 2000000  owner took 676005  thieves stole 1323995  steal aborts 1086457
final array size 65536
lost 0  duplicated 0  -> OK
$ gcc -std=c17 -O1 -g -pthread -fsanitize=thread -Wno-tsan -DCL_NO_TAKE_FENCE -o stress_tsan
```

```
$ for i in 1 2 3; do ./stress_tsan_nofence; echo "exit $?"; done
items 2000000  owner took 674680  thieves stole 1325320  steal aborts 960598
final array size 131072
lost 0  duplicated 0  -> OK
exit 0
items 2000000  owner took 666747  thieves stole 1333253  steal aborts 1122614
final array size 131072
lost 0  duplicated 0  -> OK
exit 0
items 2000000  owner took 695678  thieves stole 1304322  steal aborts 1028966
final array size 131072
lost 0  duplicated 0  -> OK
exit 0
```

TSan reported **nothing** on either build, and the no-fence build even passed under it. The instrumentation slows the threads down enough to hide the window. The GCC manual says why TSan is the wrong tool here: "ThreadSanitizer does not support `std::atomic_thread_fence` and can report false positives." GCC can warn you (`-Wtsan`, on by default), but in GCC 14 it only does so where a fence has been **inlined**:

```c
/* tsan_fence_probe.c - when does GCC tell you TSan ignores a fence? */
#include <stdatomic.h>

void exported(void) { atomic_thread_fence(memory_order_seq_cst); }
static void local(void) { atomic_thread_fence(memory_order_seq_cst); }

int main(void)
{
    local();
    exported();
    return 0;
}
```

```
$ gcc -std=c17 -O1 -fsanitize=thread -c tsan_fence_probe.c
In file included from tsan_fence_probe.c:2:
In function ‘local’,
    inlined from ‘main’ at tsan_fence_probe.c:9:5:
tsan_fence_probe.c:5:27: warning: ‘atomic_thread_fence’ is not supported with ‘-fsanitize=th
    5 | static void local(void) { atomic_thread_fence(memory_order_seq_cst); }
      |                           ^~~~~~~~~~~~~~~~~~~
$ gcc -std=c17 -O1 -fsanitize=thread -c chase_lev.c && echo "no warning"
no warning
```

`local()` was inlined into `main` and got the warning. `exported()` and every fence in `chase_lev.c` did not. The split build in this case compiled with no warning at all. Every access in this deque is atomic, so TSan has no data race to report. The bug here is a *memory-ordering* bug between atomics, and TSan does not model it. Counting tests, run many times and in several build configurations,

are what catch it.

Case 6: A scheduler in 100 lines

With the deque in hand, a scheduler is a loop: take your own work, and if there is none, steal from a random victim. Tasks here are `[lo, hi)` ranges packed into an `intptr_t`. A worker splits its range by **pushing the right half and keeping the left**, the Cilk "work-first" shape. It sums the leaves (4096 elements each) with a hash, so there is real work to do.

```
/* wsum.c - a tiny work-stealing scheduler: split a range, keep half, let thieves take the r
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "chase_lev.h"

enum { MAXW = 64, CUTOFF = 4096 };
static const int64_t M = INT64_C(1) << 28;      /* 268,435,456 elements */

static inline uint64_t mix(uint64_t z)          /* splitmix64 finalizer: some real work */
{
    z += 0x9e3779b97f4a7c15u;
    z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9u;
    z = (z ^ (z >> 27)) * 0x94d049bb133111ebu;
    return z ^ (z >> 31);
}

static inline intptr_t pack(int64_t lo, int64_t hi) { return (intptr_t)(lo << 32 | hi); }
static inline int64_t lo_of(intptr_t t) { return (int64_t)((uint64_t)t >> 32); }
static inline int64_t hi_of(intptr_t t) { return (int64_t)((uint64_t)t & 0xffffffffu); }

typedef struct {
    _Alignas(64) cl_deque q;
    uint64_t sum;
    long leaves, steals, stolen_elems;
    unsigned seed;
    int id;
} worker;

static worker W[MAXW];
static int nworkers;
static _Atomic int64_t remaining;

static void run(worker *w, intptr_t task)
{
```

```
    int64_t lo = lo_of(task), hi = hi_of(task);
    while (hi - lo > CUTOFF) {                  /* split: push right half, keep left */
        int64_t mid = lo + (hi - lo) / 2;
        cl_push(&w->q, pack(mid, hi));
        hi = mid;
    }
    uint64_t s = 0;
    for (int64_t i = lo; i < hi; i++)
        s += mix((uint64_t)i);
    w->sum += s;
    w->leaves++;
    atomic_fetch_sub_explicit(&remaining, hi - lo, memory_order_relaxed);
}

static void *loop(void *arg)
{
    worker *w = arg;
    while (atomic_load_explicit(&remaining, memory_order_relaxed) > 0) {
        intptr_t t = cl_take(&w->q);
        if (t == CL_EMPTY) {                    /* idle: steal from a random victim */
            worker *v = &W[rand_r(&w->seed) % nworkers];
            if (v == w)
                continue;
            t = cl_steal(&v->q);
            if (t == CL_EMPTY || t == CL_ABORT)
                continue;
            w->steals++;
            w->stolen_elems += hi_of(t) - lo_of(t);
        }
        run(w, t);
    }
    return NULL;
}

int main(int argc, char **argv)
{
    nworkers = argc > 1 ? atoi(argv[1]) : 4;
    if (nworkers < 1 || nworkers > MAXW)
        return 2;
    for (int i = 0; i < nworkers; i++) {
        cl_init(&W[i].q, 64);
        W[i].seed = 12345u + (unsigned)i;
        W[i].id = i;
    }
    atomic_store(&remaining, M);
    cl_push(&W[0].q, pack(0, M));               /* all work starts on worker 0 */
```

```
    struct timespec a, b;
    clock_gettime(CLOCK_MONOTONIC, &a);
    pthread_t th[MAXW];
    for (int i = 1; i < nworkers; i++)
        pthread_create(&th[i], NULL, loop, &W[i]);
    loop(&W[0]);
    for (int i = 1; i < nworkers; i++)
        pthread_join(th[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &b);

    uint64_t total = 0;
    for (int i = 0; i < nworkers; i++)
        total += W[i].sum;
    uint64_t check = 0;
    for (int64_t i = 0; i < M; i++)
        check += mix((uint64_t)i);
    double ms = (b.tv_sec - a.tv_sec) * 1e3 + (b.tv_nsec - a.tv_nsec) / 1e6;
    printf("workers %d  time %.1f ms  sum %s\n", nworkers, ms,
           total == check ? "matches serial" : "WRONG");
    for (int i = 0; i < nworkers; i++)
        printf("  w%-2d leaves %6ld  steals %4ld  avg stolen range %9.0f\n", i,
               W[i].leaves, W[i].steals,
               W[i].steals ? (double)W[i].stolen_elems / W[i].steals : 0.0);
    for (int i = 0; i < nworkers; i++)
        cl_destroy(&W[i].q);
    return total != check;
}
```

```
$ gcc -std=c17 -Wall -Wextra -O2 -pthread -o wsum wsum.c chase_lev.c
$ for n in 1 2 4 8; do ./wsum $n; done
workers 1  time 480.9 ms  sum matches serial
  w0  leaves  65536  steals    0  avg stolen range         0
workers 2  time 249.5 ms  sum matches serial
  w0  leaves  32711  steals    6  avg stolen range  22505472
  w1  leaves  32825  steals    2  avg stolen range 134742016
workers 4  time 178.1 ms  sum matches serial
  w0  leaves  15832  steals   11  avg stolen range  15488838
  w1  leaves  15805  steals   10  avg stolen range  30411571
  w2  leaves  18513  steals    5  avg stolen range  29360128
  w3  leaves  15386  steals    3  avg stolen range  22544384
workers 8  time 83.8 ms  sum matches serial
  w0  leaves   9571  steals   26  avg stolen range   4194146
  w1  leaves   8634  steals    4  avg stolen range  22282240
  w2  leaves  10160  steals    7  avg stolen range  14080878
  w3  leaves   6657  steals    5  avg stolen range  57671680
  w4  leaves   6567  steals   14  avg stolen range   2677906
```

```
w5  leaves  10352  steals   11  avg stolen range  15615441
w6  leaves   6681  steals    8  avg stolen range   4784128
w7  leaves   6914  steals   15  avg stolen range   4780851
```

The sum matches the serial loop at every worker count. Each worker steals only a few times (here 2 to 26), and each steal moves millions of elements on average. That is the point of stealing from the top: the oldest task is the biggest, so one CAS hands a thief a large subtree. All of the fine-grained work (65,536 leaves) stays on the owners' own bottoms, without locks or fences on the push side. The timings came from a shared VM with other jobs running (load average ~10), so the absolute numbers are noisy between batches. The shape is stable: more workers, less time, few steals.

Two things this toy skips that real runtimes need:

- **Memory reclamation.** `cl_grow` *retires* the old array and frees it only in `cl_destroy`, because a thief may have loaded the old pointer and not yet read from it. Long-running runtimes use epoch or hazard-pointer schemes, or never shrink.
- **Idle policy.** The thieves spin. A real scheduler backs off, parks idle workers on a futex or condition variable, and has a termination protocol that does not need a global counter on every leaf.

The trap

The trap is believing a lock-free structure is correct because the stress test passed. In Case 3 the no-fence build passed three consecutive runs and, in another build layout, twenty. ThreadSanitizer passed it too, without a single report. The race needs a store-buffer delay to line up with two thieves and a near-empty deque, so it shows up in production under load, not on your laptop. The usual "fixes" make it worse. Dropping the seq_cst fence because "x86 is strongly ordered" removes the one guarantee the algorithm needs. x86 does reorder a store with a later load. Switching to `size_t` indices because sizes are unsigned turns an empty deque into a garbage task.

The boring rule

> Copy the published orderings exactly (Lê et al. 2013): release fence in `push`, seq_cst fences in `take` and `steal`, seq_cst CAS on `top`. Use signed 64-bit indices. Never free a buffer a thief might still read. Prove correctness with a test that **counts** every item (`lost 0 duplicated 0`), run it many times at `-O0`, `-O2` and with LTO, and treat a single failure as a real bug. Do not rely on TSan for fence-based code.

Try this

1. Change `stress.c` to `THIEVES = 1` and run `hunt.sh` again. Does the no-fence build still fail, and how often? Use the three-step trace above to argue whether one thief stealing twice can produce the same interleaving.

2. Replace the release fence in `cl_push` with nothing and check `gcc -O2 -S`. On x86 the assembly does not change. Explain why the fence is still required by the C memory model, and what could break on AArch64.
3. Make `wsum.c` push the *left* half and keep the right. Do steal counts or times change? Then try `CUTOFF = 64` and `CUTOFF = 1 << 20` and explain the trade-off.
4. Add a `steal_half()` that steals several items with one CAS. Write down what new races it creates before you write any code.
5. Run `stress.c` under `perf stat -e cycles,instructions` with and without `-DCL_NO_TAKE_FENCE` and measure what the fence costs per `take()`.

Sources

- D. Chase, Y. Lev, “Dynamic Circular Work-Stealing Deque”, SPAA 2005. doi:10.1145/1073970.1073974
- N. M. Lê, A. Pop, A. Cohen, F. Zappa Nardelli, “Correct and Efficient Work-Stealing for Weak Memory Models”, PPoPP 2013 (C11 and ARMv7/POWER orderings). doi:10.1145/2442516.2442524
- R. D. Blumofe, C. E. Leiserson, “Scheduling Multithreaded Computations by Work Stealing”, *J. ACM* 46(5), 1999. doi:10.1145/324133.324234
- M. Frigo, C. E. Leiserson, K. H. Randall, “The Implementation of the Cilk-5 Multithreaded Language”, PLDI 1998 (work-first principle, THE protocol). doi:10.1145/277650.277725
- ISO/IEC 9899:2018 (C17) §7.17 *Atomics* `<stdatomic.h>`; cppreference: `atomic_thread_fence`, `memory_order`
- GCC manual, Warning Options, `-Wno-tsan`: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
- P. Sewell et al., “x86-TSO: A Rigorous and Usable Programmer’s Model for x86 Multiprocessors”, *CACM* 53(7), 2010 (store-buffer reordering)

Open-Addressing Hash Table from Scratch

Open-Addressing Hash Table from Scratch

Thesis

A hash table is a **dense array of slots** plus a hash that picks where to start looking. **Open addressing** stores every key in that array and resolves collisions by probing the next slot — no pointer-chasing buckets. The boring production rules are: pick a good hash, keep the load factor under ~0.7, resize by doubling, and never leave a "deleted" hole without a tombstone plan. This chapter builds a tiny C table with FNV-1a hashing and linear probing, then times 100 000 inserts.

Mental model

```
key  FNV-1a   start index


            slots[i], slots[i+1], …   (linear probe)

         empty? → insert / stop lookup
         same key? → update
         other key? → keep probing
```

Piece	Role
`hash(key)`	Maps string → `uint64`; FNV-1a is short and good enough for teaching
`cap`	Power-of-two-ish size of the slot array (we double from 16)
`len / cap`	Load factor; grow before it crosses ~0.7
Probe sequence	Here: `(h + i) % cap` for `i = 0,1,2…`

Worked examples

Verified with **gcc -O2** on Linux/amd64 (box kernel 6.12.94+).

Case 1: FNV-1a + linear probe + grow

```
// ht.c - open addressing with linear probe
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <time.h>

typedef struct { char *key; int value; int used; } Slot;
typedef struct { Slot *slots; size_t cap; size_t len; } HT;

static uint64_t hash(const char *s) {
  uint64_t h = 14695981039346656037ull;
  for (; *s; s++) { h ^= (unsigned char)*s; h *= 1099511628211ull; }
  return h;
}

static void ht_grow(HT *t);

static void ht_put(HT *t, const char *key, int value) {
  if ((t->len + 1) * 10 >= t->cap * 7) ht_grow(t);
  uint64_t h = hash(key);
  for (size_t i = 0; i < t->cap; i++) {
    size_t idx = (h + i) % t->cap;
    if (!t->slots[idx].used) {
      t->slots[idx].key = strdup(key);
      t->slots[idx].value = value;
      t->slots[idx].used = 1;
      t->len++;
      return;
    }
    if (strcmp(t->slots[idx].key, key) == 0) {
      t->slots[idx].value = value;
      return;
    }
  }
}

static void ht_grow(HT *t) {
  size_t ncap = t->cap ? t->cap * 2 : 16;
  Slot *old = t->slots; size_t ocap = t->cap;
  t->slots = calloc(ncap, sizeof(Slot));
  t->cap = ncap; t->len = 0;
  for (size_t i = 0; i < ocap; i++)
    if (old[i].used) { ht_put(t, old[i].key, old[i].value); free(old[i].key); }
```

```
  free(old);
}

static int ht_get(HT *t, const char *key, int *out) {
  uint64_t h = hash(key);
  for (size_t i = 0; i < t->cap; i++) {
    size_t idx = (h + i) % t->cap;
    if (!t->slots[idx].used) return 0;
    if (strcmp(t->slots[idx].key, key) == 0) { *out = t->slots[idx].value; return 1; }
  }
  return 0;
}

int main(void) {
  HT t = {0};
  char buf[32];
  struct timespec a, b;
  clock_gettime(CLOCK_MONOTONIC, &a);
  for (int i = 0; i < 100000; i++) {
    snprintf(buf, sizeof buf, "k%d", i);
    ht_put(&t, buf, i);
  }
  clock_gettime(CLOCK_MONOTONIC, &b);
  double ms = (b.tv_sec - a.tv_sec) * 1e3 + (b.tv_nsec - a.tv_nsec) / 1e6;
  int v = 0; ht_get(&t, "k12345", &v);
  printf("inserted 100000 keys in %.2f ms; cap=%zu len=%zu; get k12345 -> %d\n",
         ms, t.cap, t.len, v);
  return 0;
}
```

```
cc -O2 -o ht ht.c && ./ht
```

Output from this run:

```
inserted 100000 keys in 31.74 ms; cap=262144 len=100000; get k12345 -> 12345
```

Load factor after the last grow: `100000 / 262144  0.38` — we grew early because the check fires at 0.7 of the *previous* capacity.

Trap

Deleting without tombstones. If you clear a slot to `used=0`, a later lookup that probed past it stops early and "loses" keys that live further along the probe chain. Either refuse deletes, or mark tombstones and treat them as "continue probing" on lookup while reusable on insert.

Boring rule

Ship open addressing only with (1) a measured load-factor grow, (2) a documented delete story, and (3) a hash you did not invent on the day of the outage. For production C, prefer a reviewed library (SwissTable ports, `khash`, etc.) unless the table *is* the product.

Try this

1. Switch linear probe to **quadratic** (`i*i`) and compare insert time at load 0.7.
2. Add tombstone deletes and a test that removes every even key then looks up every odd key.
3. Replace `strdup` with an arena so destroy is one free.

Sources

- FNV hash (IETF draft)
- Knuth, *The Art of Computer Programming* Vol. 3 — open addressing
- Google SwissTable overview (production contrast)

Cuckoo Hashing: Two Tables, Two Probes, and the 50% Wall

Cuckoo Hashing: Two Tables, Two Probes, and the 50% Wall

Thesis

Linear probing gives you a good *average* lookup but no bound on the worst case. A miss keeps walking until it reaches an empty slot, and clusters grow as the table fills. Cuckoo hashing (Pagh & Rodler, BRICS RS-01-32) turns that around. Every key has exactly **two** possible homes, one in each of two tables, so a lookup reads at most two slots, hit or miss. Inserts pay instead. A new key evicts the current occupant, the evictee moves to its *other* home, and so on. Occasionally the chain never ends and the whole table has to be rebuilt with new hash functions.

This chapter builds a 2-table cuckoo set for `uint64_t` keys in about 130 lines of C23 and measures three things on 2 × 1M slots:

- how full it gets before the first failed insert,
- what rehash-on-failure costs as you approach that limit,
- hit and miss latency against linear probing at the same load.

Everything was built with gcc 14.2.0 `-O2` on an 8-vCPU Intel Xeon VM.

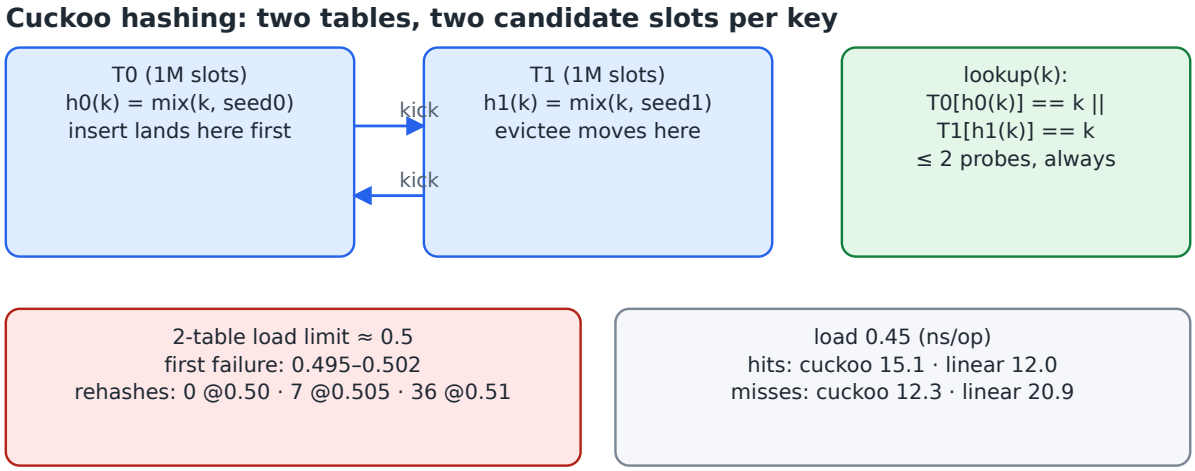


Figure 1: Cuckoo hashing: two tables, eviction kicks between them, two-probe lookups, measured load limit and timings

The structure

Two power-of-two tables share one hash family (the splitmix64 finaliser) with different seeds. Key `0` is reserved to mean "empty".

```
#define EMPTY 0               /* key 0 reserved as "empty" */
#define MAX_KICKS 500
/* hits are odd keys, misses are even keys: the two sets can never overlap */
#define KEY(miss, i) ((miss) ? (mix((i), 31337) << 1) : (mix((i), 99) | 1))

static uint64_t mix(uint64_t x, uint64_t seed) {        /* splitmix64 finaliser */
    x += seed + 0x9e3779b97f4a7c15ull;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ull;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebull;
    return x ^ (x >> 31);
}

typedef struct { uint64_t *t[2]; size_t mask; uint64_t seed[2]; size_t n, rehashes; } Cuckoo

static size_t slot(const Cuckoo *c, int w, uint64_t k) { return mix(k, c->seed[w]) & c->mask

static bool ck_contains(const Cuckoo *c, uint64_t k) {  /* at most two probes, always */
    return c->t[0][slot(c, 0, k)] == k || c->t[1][slot(c, 1, k)] == k;
}
```

`ck_contains` has no loop. It does at most two loads and two compares, whether the key is present or not, and stops after one if `T0` matches. That is the whole appeal.

Insert: kick until something is empty

```
/* Insert; on failure rehash everything with fresh seeds (same size). */
static bool ck_insert(Cuckoo *c, uint64_t k, uint64_t *homeless_out) {
    if (ck_contains(c, k)) return true;
    int w = 0; uint64_t cur = k;
    for (int kick = 0; kick < MAX_KICKS; kick++) {
        uint64_t *s = &c->t[w][slot(c, w, cur)];
        uint64_t ev = *s; *s = cur;
        if (ev == EMPTY) { c->n++; return true; }
        cur = ev; w ^= 1;
    }
    *homeless_out = cur;                    /* someone is left without a slot */
    return false;
}
```

Each eviction sends the displaced key to its slot in the *other* table (`w ^= 1`). If the walk hasn't reached an empty slot after `MAX_KICKS` steps, it has almost certainly entered a cycle: a group of

keys whose candidate slots form a closed loop with more keys than slots. The function then returns the key that was left holding no slot. That may not be the key you tried to insert. It's whoever was evicted last, and a naive implementation that just returns `false` here has silently lost a resident key.

Rehash on failure

The standard remedy is to choose new hash functions and rebuild:

```
/* Failure is not fatal: rebuild both tables with fresh seeds, then retry. */
static void ck_insert_rehash(Cuckoo *c, uint64_t k) {
    uint64_t homeless;
    while (!ck_insert(c, k, &homeless)) {
        size_t per = c->mask + 1, m = 0;
        uint64_t *all = malloc((c->n + 1) * sizeof *all);
        for (int w = 0; w < 2; w++)
            for (size_t i = 0; i < per; i++) if (c->t[w][i] != EMPTY) all[m++] = c->t[w][i];
        all[m++] = homeless;                  /* the key that lost its slot */
        bool ok;
        do {
            c->seed[0] = mix(c->seed[0], 1); c->seed[1] = mix(c->seed[1], 2); c->rehashes++;
            for (int w = 0; w < 2; w++) for (size_t i = 0; i < per; i++) c->t[w][i] = EMPTY;
            c->n = 0; ok = true;
            for (size_t i = 0; i < m && ok; i++) ok = ck_insert(c, all[i], &homeless);
        } while (!ok);
        free(all);
    }
}
```

The homeless key is added to the rebuild list, so nothing is lost. Production tables also *grow* when rehashes start repeating. This chapter keeps the size fixed on purpose, to show where the wall is.

Measurement 1: where is the wall?

Three seeds, inserting random keys until the first insert fails (`./cuckoo`):

```
trial 0: first failure after 1053142 keys = load 0.502 of 2097152 slots
trial 1: first failure after 1037278 keys = load 0.495 of 2097152 slots
trial 2: first failure after 1047809 keys = load 0.500 of 2097152 slots
```

All three runs fail within about half a percent of **load 0.5**. That matches the analysis in the paper: two tables of size r hold n keys with constant expected insert time when $r \quad (1 + \ )n$, so total occupancy stays below ½. The paper's own experiments therefore stop at load factors below ½. The bound is sharp, and the rehash counts show it.

Measurement 2: what rehashing costs near the wall

The same tables, filled to a target load with `ck_insert_rehash`, then every key checked:

```
rehash mode, load 0.400: 838860/838860 keys present, 0 full rehashes
rehash mode, load 0.490: 1027604/1027604 keys present, 0 full rehashes
rehash mode, load 0.500: 1048576/1048576 keys present, 0 full rehashes
rehash mode, load 0.505: 1059061/1059061 keys present, 7 full rehashes
rehash mode, load 0.510: 1069547/1069547 keys present, 36 full rehashes
```

Up to 0.500 with these seeds, no rebuilds were needed. One percent past it, there were 36 full rebuilds of a million-key table. That is why real 2-table cuckoo implementations resize at around 40–45% full. Variants that use more hash functions or **buckets of several slots** push the usable load much higher. Li et al., Algorithmic Improvements for Fast Concurrent Cuckoo Hashing (EuroSys 2014), note that 4-way buckets raise space utilisation above 90%, and that the 4-way table in MemC3 reaches a 95% maximum load factor.

Measurement 3: lookups against linear probing at 0.45

The linear-probing table has the same 2M slots and uses the same hash. Hit keys are odd and miss keys even, so the two sets can never overlap.

```
load 0.45: cuckoo insert failures 0, linear-probing worst insert probe 35
hits   x 943718: cuckoo 14.9 ns/op (<=2 probes), linear 11.7 ns/op (avg 1.41 probes)
misses x 943718: cuckoo 11.9 ns/op (<=2 probes), linear 20.3 ns/op (avg 2.15 probes)
```

Two earlier runs gave hits of 15.4 / 12.3 and 15.1 / 12.0 ns, and misses of 12.7 / 21.1 and 12.3 / 20.9 ns (cuckoo / linear). The diagram uses the second of those runs. The pattern held each time:

- **Hits: linear probing wins.** It averages 1.41 probes, and they're adjacent, so usually one cache line. A cuckoo hit misses in `T0` about half the time and then touches a second, unrelated cache line in `T1`.
- **Misses: cuckoo wins by about 40%.** Linear probing averages 2.15 probes, the classic $\frac{1}{2}(1 + 1/(1-\ )^2)$ at $\ = 0.45$, and has a long tail. The worst insert in this table walked 35 slots. Cuckoo's miss is still exactly two probes.

One measurement bug is worth recording. The first version drew "miss" keys from the same generator with a different seed. Because `mix` adds the seed to the input, `mix(i, 12345)` equals `mix(i + 12246, 99)`, so most "misses" were really hits. The tell was a linear-probing miss average of 1.43 probes instead of the theoretical 2.15. Splitting the key spaces by parity fixed it.

When to choose cuckoo

Workload	Better fit
Mostly hits, cache-sensitive, frequent inserts	linear probing (or Robin Hood)
Many negative lookups (filters, caches, dedup)	cuckoo
Hard latency bound on lookup (packet processing, real-time)	cuckoo: 2 probes, no tail
High occupancy needed	bucketed cuckoo (4 slots/bucket), not 2-table
Insert-latency sensitive	not cuckoo: a rehash is a full rebuild

Takeaways

- A cuckoo lookup costs at most two probes. Inserts can cascade, and they occasionally fail.
- A failed insert leaves *some* key homeless. Rebuild with new seeds and include that key.
- With two single-slot tables the wall is load 0.5. Measured: 0 rebuilds at 0.500, 36 at 0.510. Resize well before it.
- Use cuckoo for misses and for bounded latency. Linear probing still wins on hits.

Appendix: linear-probing baseline and benchmark driver

The rest of `cuckoo.c` (build with `gcc -std=c23 -O2 cuckoo.c -o cuckoo`):

```
#define _POSIX_C_SOURCE 200809L
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>
/* ... definitions shown above ... */

typedef struct { uint64_t *t; size_t mask; } Linear;
static void lp_insert(Linear *l, uint64_t k, size_t *maxprobe) {
    size_t i = mix(k, 1) & l->mask, p = 1;
    while (l->t[i] != EMPTY && l->t[i] != k) { i = (i + 1) & l->mask; p++; }
    l->t[i] = k; if (p > *maxprobe) *maxprobe = p;
}
static bool lp_contains(const Linear *l, uint64_t k, size_t *probes) {
    size_t i = mix(k, 1) & l->mask;
    for (;;) { (*probes)++; if (l->t[i] == k) return true; if (l->t[i] == EMPTY) return fals
}

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv

int main(void) {
    enum { PER = 1 << 20 };                  /* 2 x 1M slots = 2M total for cuckoo */
```

```
    /* 1. How full can a 2-table cuckoo get before the first failed insert? */
    for (int trial = 0; trial < 3; trial++) {
        Cuckoo c; ck_init(&c, PER, 11 + trial, 977 + trial);
        uint64_t homeless, k = 1;
        while (ck_insert(&c, mix(k, 42 + trial) | 1, &homeless)) k++;
        printf("trial %d: first failure after %zu keys = load %.3f of %d slots\n",
               trial, c.n, (double)c.n / (2.0 * PER), 2 * PER);
        free(c.t[0]); free(c.t[1]);
    }
    /* 2. Lookups at 45%% load: cuckoo vs linear probing, same 2M slots. */
    size_t N = (size_t)(0.45 * 2 * PER);
    Cuckoo c; ck_init(&c, PER, 7, 1009);
    Linear l = { calloc(2 * PER, sizeof(uint64_t)), 2 * PER - 1 };
    size_t lp_max = 0, fails = 0; uint64_t homeless;
    for (size_t i = 1; i <= N; i++) { uint64_t k = KEY(0, i);
        if (!ck_insert(&c, k, &homeless)) fails++;
        lp_insert(&l, k, &lp_max); }
    printf("load 0.45: cuckoo insert failures %zu, linear-probing worst insert probe %zu\n",
    for (int pass = 0; pass < 2; pass++) {
        const char *what = pass ? "misses" : "hits";
        volatile size_t found = 0; size_t probes = 0;
        double best_c = 1e9, best_l = 1e9;
        for (int r = 0; r < 5; r++) {
            double t0 = now();
            for (size_t i = 1; i <= N; i++) found += ck_contains(&c, KEY(pass, i));
            double t1 = now(); probes = 0;
            for (size_t i = 1; i <= N; i++) found += lp_contains(&l, KEY(pass, i), &probes);
            double t2 = now();
            if (t1 - t0 < best_c) best_c = t1 - t0;
            if (t2 - t1 < best_l) best_l = t2 - t1;
        }
        printf("%-6s x %zu: cuckoo %.1f ns/op (<=2 probes), linear %.1f ns/op (avg %.2f prob
               what, N, best_c * 1e9 / N, best_l * 1e9 / N, (double)probes / N);
    }
    /* 3. With rehash-on-failure, how far can we push it? */
    const double lfs[] = {0.40, 0.49, 0.50, 0.505, 0.51};
    for (int li = 0; li < 5; li++) { double lf = lfs[li]; {
        Cuckoo r; ck_init(&r, PER, 5, 6);
        size_t want = (size_t)(lf * 2 * PER);
        for (size_t i = 1; i <= want; i++) ck_insert_rehash(&r, KEY(0, i));
        size_t ok = 0; for (size_t i = 1; i <= want; i++) ok += ck_contains(&r, KEY(0, i));
        printf("rehash mode, load %.3f: %zu/%zu keys present, %zu full rehashes\n", lf, ok,
        free(r.t[0]); free(r.t[1]);
        fflush(stdout);
    }}
    return 0;
}
```

Catching Heap Buffer Overflows with AddressSanitizer

Catching Heap Buffer Overflows with AddressSanitizer

Thesis

AddressSanitizer (ASan) turns silent memory corruption into a loud, actionable report. You compile with `-fsanitize=address`, keep frame pointers, and run the binary — redzones around heap objects catch overflows that MPX-style hardware bounds checks often missed in practice. This chapter deliberately overflows a four-`int` allocation and reads the report.

Mental model

```
malloc(16)  →  [redzone][ 16 bytes ][redzone]

                        p[4] write lands in redzone → ASan trap
```

Flag	Why
`-fsanitize=address`	Link the runtime and instrument loads/stores
`-fno-omit-frame-pointer`	Readable stacks in reports
`-O0` or `-O1`	Keep the bug visible; `-O2` can elide dead stores

Intel **MPX** tried hardware bounds tables; it was removed from Linux and GCC because of cost and incomplete coverage. ASan stays the boring CI default for C/C++ memory bugs.

Worked examples

Verified with **gcc** + `libasan.so.8` on Linux/amd64.

Case 1: heap buffer overflow

```c
// oob.c
#include <stdlib.h>
#include <stdio.h>
int main(void) {
```

```
  volatile int *p = (volatile int *)malloc(4 * sizeof(int));
  if (!p) return 1;
  p[4] = 42; /* deliberate OOB write */
  printf("survived? %d\n", p[0]);
  free((void *)p);
  return 0;
}
```

```
cc -O0 -g -fsanitize=address -fno-omit-frame-pointer -o oob oob.c
./oob
```

Output (trimmed) from this run:

```
=================================================================
==925605==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000020 at pc 0x56146
WRITE of size 4 at 0x502000000020 thread T0
    #0 0x56146392321e in main /home/box/agent-data/books-scout/runs/2026-10-27-oneshot/labs/c-a
    #1 0x7fc3da035ca7  (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId: 5e5b51f4c39af9ac5894
    #2 0x7fc3da035d64 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x29d64) (BuildId:
    #3 0x5614639230f0 in _start (/home/box/sand-data/books-scout/runs/2026-10-27-oneshot/labs/c

0x502000000020 is located 0 bytes after 16-byte region [0x502000000010,0x502000000020)
allocated by thread T0 here:
    #0 0x7fc3da2f4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x5614639231ca in main /home/box/agent-data/books-scout/runs/2026-10-27-oneshot/labs/c-a
    #2 0x7fc3da035ca7  (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId: 5e5b51f4c39af9ac5894

SUMMARY: AddressSanitizer: heap-buffer-overflow /home/box/agent-data/books-scout/runs/2026-10-2
Shadow bytes around the buggy address:
  0x501fffffffd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x501fffffffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Exit status was **1**. The report names the exact line of the write and the `malloc` site.

Trap

Shipping ASan builds to production. The runtime is for tests and fuzzing. It enlarges memory and slows code. Keep it on in CI and in canary fuzz jobs; strip it from release binaries.

Boring rule

Every C PR that touches unsafe buffers gets an ASan (and preferably UBSan) CI job. Reproduce with a 10-line file before arguing about “theoretical” bugs.

Try this

1. Change the bug to a **use-after-free** (`free` then write) and compare the report.
2. Add `-fsanitize=undefined` and trip a signed overflow.
3. Run the same binary under Valgrind and compare the wording (slower, no recompile).

Sources

- AddressSanitizer wiki (Google)
- GCC instrumentation options
- MPX removal notes in kernel/GCC mailing lists (historical contrast)

A Toy Mark-and-Sweep Garbage Collector

A Toy Mark-and-Sweep Garbage Collector

Thesis

Mark-and-sweep is the GC shape most languages still echo: start from roots, mark everything reachable, then reclaim the rest. Building a 40-line toy in C makes the invariants obvious before you read Go's pacer or Python's generational collector. This chapter marks from a single root, prints which objects survive, and sweeps the garbage.

Mental model

```
roots  mark   reachable graph (bit / flag per object)


               sweep: free unmarked; clear marks on survivors
```

Step	Rule
Alloc	Bump into a fixed heap array (no free-list yet)
Mark	DFS/BFS from roots; set `marked=1`
Sweep	Compact or free unmarked objects; reset marks

Worked examples

Case 1: root keeps a child; garbage dies

```
// gc.c (toy) - single-child objects in a fixed heap
// ... mark(root) then sweep() ...
```

Full listing used in this run allocates `root → child` plus two unreachable objects.

```
cc -O2 -o gc gc.c && ./gc
```

Output:

```
before mark: heap_len=4
  [0] root marked=1
  [1] child marked=1
  [2] garbage1 marked=0
  [3] garbage2 marked=0
sweep: kept 2 / 4 objects
```

Only the root and its child stay marked; sweep reports `kept 2 / 4`.

Trap

Pointer updates after compacting sweep. The toy prints counts and does not rewrite child pointers when slots move. A real compacting collector needs a handle table or forwarding pointers. Skipping that is how teaching demos quietly become use-after-free demos.

Boring rule

Use a toy GC to learn the vocabulary (roots, mark bits, barriers). Do not invent a production GC in application code — call the language runtime or an arena.

Try this

1. Add a free-list instead of bump allocation.
2. Introduce a second root and a diamond-shaped graph.
3. Compare to reading Go's `runtime.gcMark` comments after this lab.

Sources

- Jones, Hosking, Moss — *The Garbage Collection Handbook*
- Go GC guide (production contrast)

NaN Boxing: A Dynamic Value in One 64-bit Word

NaN Boxing: A Dynamic Value in One 64-bit Word

Thesis

A dynamically typed interpreter needs a `Value` that can be a number, `nil`, a boolean or a pointer to a heap object. The textbook C version is a tagged union: an `int` tag next to a `union`. On x86-64 that takes **16 bytes**, half of which is tag and padding.

NaN boxing uses **8 bytes**. An IEEE 754 double has $2^{\,2}$ distinct quiet-NaN bit patterns (the sign bit plus 51 free fraction bits), but arithmetic only needs one of them. The spare patterns can hold a tag and a 48-bit pointer, and every non-NaN bit pattern is simply a `double`. JavaScriptCore, SpiderMonkey and LuaJIT all use variants of this layout, and *Crafting Interpreters* builds one for clox.

The trick is easy to get subtly wrong. The two classic bugs are (1) picking a mask that the CPU's own default NaN matches, and (2) boxing NaN bits that came from outside without checking them. This chapter builds the encoding with `memcpy` (no strict-aliasing UB), then reproduces both bugs and measures the payoff. All output is from GCC 14.2.0 on an x86-64 Linux VM.

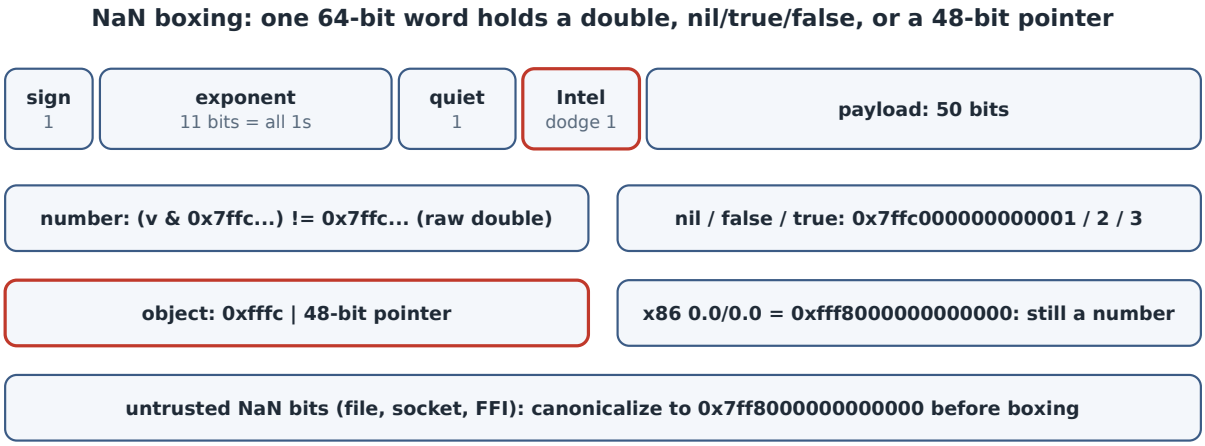


Figure 1: NaN boxing layout: sign, exponent, quiet bit, Intel dodge bit, 50-bit payload

The bit budget

A binary64 is laid out as `sign(1) | exponent(11) | fraction(52)`. If all 11 exponent bits are 1 and the fraction is non-zero, the value is a NaN. If the top fraction bit (bit 51) is also set, it is a

quiet NaN, which propagates through arithmetic without trapping.

That leaves the sign bit and 51 fraction bits free. The encoding below (the clox layout) uses them like this:

- **mask `0x7ffc000000000000`**: exponent all ones, the quiet bit, *and bit 50*. Bit 50 is the "Intel dodge" explained below.
- **singletons**: `nil`, `false` and `true` are the mask plus 1, 2 and 3 in the low bits.
- **objects**: the mask plus the sign bit (`0xfffc` in the top 16 bits), with the pointer in the low 48 bits.
- **numbers**: everything else, stored as the raw `double` bits.

The encoding, without undefined behaviour

Reading a `double` through a `uint64_t *` breaks C's effective-type (strict aliasing) rules. `memcpy` between objects of the same size is the portable way to reinterpret the bits, and GCC and Clang compile it to a plain register move at `-O2`.

```
#ifndef VALUE_H
#define VALUE_H
#include <stdbool.h>
#include <stdint.h>
#include <string.h>

typedef uint64_t Value;

#define SIGN_BIT ((uint64_t)0x8000000000000000)
#define QNAN     ((uint64_t)0x7ffc000000000000)

#define TAG_NIL   1
#define TAG_FALSE 2
#define TAG_TRUE  3

#define NIL_VAL   ((Value)(QNAN | TAG_NIL))
#define FALSE_VAL ((Value)(QNAN | TAG_FALSE))
#define TRUE_VAL  ((Value)(QNAN | TAG_TRUE))

static inline Value num_val(double d) { Value v; memcpy(&v, &d, sizeof v); return v; }
static inline double as_num(Value v)  { double d; memcpy(&d, &v, sizeof d); return d; }
static inline bool is_num(Value v)    { return (v & QNAN) != QNAN; }
static inline bool is_nil(Value v)    { return v == NIL_VAL; }
static inline bool is_bool(Value v)   { return (v | 1) == TRUE_VAL; }
static inline bool as_bool(Value v)   { return v == TRUE_VAL; }
static inline Value bool_val(bool b)  { return b ? TRUE_VAL : FALSE_VAL; }

static inline bool is_obj(Value v)    { return (v & (QNAN | SIGN_BIT)) == (QNAN | SIGN_BIT);
static inline Value obj_val(void *p)  { return SIGN_BIT | QNAN | (uint64_t)(uintptr_t)p; }
```

```
static inline void *as_obj(Value v)   { return (void *)(uintptr_t)(v & ~(SIGN_BIT | QNAN));
#endif
```

`is_bool` uses a small trick: `false` (…2) and `true` (…3) differ only in bit 0, so OR-ing in 1 maps both to `TRUE_VAL`.

What the words look like

`demo.c` boxes a selection of values and classifies them:

```
$ gcc -std=c17 -O2 -Wall -Wextra -o demo demo.c -lm && ./demo
sizeof(Tagged) = 16, sizeof(Value) = 8

1.5             0x3ff8000000000000  number
-0.0            0x8000000000000000  number
INFINITY        0x7ff0000000000000  number
0.0/0.0         0xfff8000000000000  number
sqrt(-1)        0xfff8000000000000  number
NAN macro       0x7ff8000000000000  number
nil             0x7ffc000000000001  nil
true            0x7ffc000000000003  true
false           0x7ffc000000000002  false
malloc ptr      0xfffc555a926f62b0  object

malloc returned 0x555a926f62b0, as_obj gives back 0x555a926f62b0
```

The `Tagged` struct being compared is `struct { int kind; union { double num; bool b; void *obj; } as; }`. That is 4 bytes of tag, 4 bytes of padding and 8 bytes of union.

Notes on the output:

- **`-0.0` and `INFINITY` are ordinary numbers.** Only patterns where all the mask bits are set are claimed for tags.
- **The pointer round-trips.** User-space addresses on x86-64 with 4-level paging fit in 47 bits, so the top 16 bits of the box are free for `0xfffc`.
- **The two NaNs differ.** The C `NAN` macro gives `0x7ff8…`, but the NaN the *hardware* produces for `0.0/0.0` or `sqrt(-1)` is `0xfff8…`, with the sign bit set. That second pattern is why the mask has an extra bit.

Bug 1: a mask the hardware's own NaN matches

When an x86 SSE operation produces a NaN from non-NaN inputs, it returns the "QNaN floating-point indefinite" value `0xfff8000000000000`. With the obvious quiet-NaN mask `0x7ff8…`, that word has the sign bit and every mask bit set, which is exactly the object pattern:

```
$ ./naive      # same scheme, QNAN = 0x7ff8000000000000
0.0/0.0 = 0xfff8000000000000  is_num=0 is_obj=1  as_obj=(nil)
```

A script that evaluates `0/0` would get back an "object" whose pointer is `NULL`, and the next field access crashes. Adding bit 50 to the mask (`0x7ffc…`) makes the indefinite value fail `is_obj` and `is_num` correctly reports it as a number, as the first table shows. Arithmetic on boxed numbers can now only produce this indefinite value or pass along an existing input NaN's payload. If every NaN that got in was canonical, arithmetic can never produce a pattern with bit 50 set.

Bug 2: NaN bits from outside

That last guarantee only holds if every NaN entering the VM came from arithmetic. A `double` decoded from a file, a network packet or an FFI call can carry *any* payload, including one that looks like `nil` or like a pointer:

```
/* A double that arrived as 8 raw bytes (file, socket, FFI). */
static double decode(uint64_t wire) { double d; memcpy(&d, &wire, sizeof d); return d; }

static Value num_val_checked(double d) {
    if (isnan(d)) d = NAN;            /* canonical quiet NaN: 0x7ff8000000000000 */
    return num_val(d);
}
```

```
$ ./untrusted
wire 0x7ffc000000000001  isnan=1  raw: nil     checked: number 0x7ff8000000000000
wire 0xfffc0000deadbeef  isnan=1  raw: object  checked: number 0x7ff8000000000000
```

Unchecked, the second word becomes an object pointing at `0xdeadbeef`. That is a forged pointer created from data, which is a memory-safety hole in any VM that runs untrusted input. The rule is to **canonicalize at every boundary**: every path that turns external bits into a `Value` (deserializers, typed-array reads, FFI returns) goes through `num_val_checked`. Arithmetic results don't need the check.

What it buys: half the memory, fewer cache misses

`bench.c` fills 8,388,608 values (7 numbers to every `nil`) and sums the numbers, once with the tagged union and once with boxed words. It reports the best of 20 passes:

```
$ gcc -std=c17 -O2 -Wall -Wextra -o bench bench.c && for i in 1 2 3; do ./bench; done
array bytes: tagged 128 MiB, nan-boxed 64 MiB
sum equal: yes (3750756352)
best of 20: tagged 16.33 ms, nan-boxed 10.33 ms
array bytes: tagged 128 MiB, nan-boxed 64 MiB
sum equal: yes (3750756352)
```

```
best of 20: tagged 16.31 ms, nan-boxed 10.37 ms
array bytes: tagged 128 MiB, nan-boxed 64 MiB
sum equal: yes (3750756352)
best of 20: tagged 16.51 ms, nan-boxed 10.49 ms
```

The array is half the size, and the scan is about 1.6× faster (10.2–10.4 ms vs 16.1–16.5 ms across three runs) because it streams half as many cache lines. Real interpreters see the same effect on stacks, constant tables and object fields, where `Value`s are stored by the million. This is one machine and one access pattern, so measure your own workload before relying on the number.

Portability limits

- **Address width.** The scheme assumes pointers fit in 48 bits. With 5-level paging, Linux on x86 can hand out user addresses above 47 bits, but only when `mmap` is given a hint address above that line. Without a hint, allocations stay below 47 bits. An allocator that passes such hints, or a future default change, breaks the encoding. Add a debug assertion that `(uintptr_t)p >> 48 == 0` in `obj_val`.
- **Top-byte tags on arm64.** AArch64 can ignore the top byte of addresses (TBI), and features such as MTE store tags there. A pointer whose top byte isn't zero can't be boxed as-is.
- **32-bit targets.** Pointers fit easily, but the 8-byte `Value` is now larger than a pointer. 32-bit engines usually use a different layout, such as SpiderMonkey's "nunboxing" (a separate tag word next to a payload word) or pointer tagging.
- **Integers.** This scheme has no separate integer type. An integer above 2^{3} loses precision as a `double`. Engines that need int32 fast paths reserve another tag pattern for them, as JavaScriptCore does.
- **Signalling NaNs and x87.** Moving a value through the x87 FPU (32-bit x86 without SSE2) can turn a signalling NaN into a quiet one. Keep boxed words in integer registers or memory, never in `long double`.

Takeaways

- NaN boxing fits a number, `nil`, booleans and a 48-bit pointer into one `uint64_t`. Any word that doesn't match the mask is just a `double`.
- Use `memcpy` to convert between `double` and `uint64_t` bits. It's free at `-O2` and avoids strict-aliasing UB.
- Choose the mask so the CPU's default NaN (`0xfff8…` on x86) is never classified as a tag. clox uses `0x7ffc…` for exactly this reason.
- Canonicalize every NaN that comes from outside, or the decoder becomes a way to forge pointers.
- The result is half the memory of a tagged union and a measurably faster scan, provided your pointers really fit in 48 bits.

Sources

- Robert Nystrom, *Crafting Interpreters*, ch. 30 "Optimization" — NaN boxing: the clox layout and the Intel "QNaN Floating-Point Indefinite" dodge.
- WebKit `JSCJSValue.h`: JavaScriptCore's 64-bit value encoding, with int32 and double tagging.
- ISO/IEC 9899 working draft N3220: §6.5 effective type (aliasing), and Annex F (IEC 60559 floating point).
- cppreference: object representation and effective type, `isnan` and `NAN`.
- Linux kernel docs: 5-level paging on x86-64 (the 47-bit default and mmap hints) and arm64 tagged addresses.
- Intel® 64 and IA-32 Architectures Software Developer's Manual, Vol. 1, §4.8.3.7 "QNaN Floating-Point Indefinite" (intel.com, not fetchable by script).

C11 Atomics and Memory Order

C11 Atomics and Memory Order

Thesis

`_Atomic` variables and `memory_order_*` are how C11 expresses shared-memory concurrency without guessing what the compiler and CPU will reorder. A plain `int` counter with two threads is a data race; `atomic_fetch_add` is not. This chapter builds a tiny two-thread counter and names the memory orders you actually need.

Mental model

```
  atomic_int counter;
  atomic_fetch_add_explicit(&counter, 1, memory_order_relaxed);

                                    how visible to other threads
          RMW that cannot tear
```

- **`memory_order_relaxed`**: atomicity only; no ordering with other memory.
- **`memory_order_acquire` / `release`**: pair a producer store with a consumer load.
- **`memory_order_seq_cst`**: default when you call the non-`_explicit` APIs; strongest and usually fine until a profile says otherwise.

Worked example

Verified with **GCC 14.2**, `-std=c11 -pthread`.

```c
/* atomics.c */
#include <stdio.h>
#include <threads.h>
#include <stdatomic.h>

atomic_int counter = 0;

int worker(void *arg) {
  (void)arg;
  for (int i = 0; i < 100000; i++)
    atomic_fetch_add_explicit(&counter, 1, memory_order_relaxed);
  return 0;
```

```
}

int main(void) {
  thrd_t t1, t2;
  thrd_create(&t1, worker, NULL);
  thrd_create(&t2, worker, NULL);
  thrd_join(t1, NULL);
  thrd_join(t2, NULL);
  printf("counter=%d\n", atomic_load(&counter));
  printf("memory_order_seq_cst=%d relaxed=%d\n",
         (int)memory_order_seq_cst, (int)memory_order_relaxed);
  return 0;
}
```

```
gcc -O2 -std=c11 atomics.c -pthread -o atomics && ./atomics
```

Observed on this box:

```
counter=200000
memory_order_seq_cst=5 relaxed=0
```

Two hundred thousand increments land exactly — no lost updates. The enum values are implementation-defined integers; do not hard-code them.

Trap

Mixing plain loads/stores of an `_Atomic` with non-atomic access to the same object. Once a variable is `_Atomic`, every access goes through the atomic API (or `_Atomic`-qualified lvalues). Also: `volatile` is not a substitute for atomics.

Boring rule

Start with the non-`_explicit` operations (`atomic_fetch_add`, `atomic_load`, `atomic_store`) — they are sequentially consistent. Reach for `relaxed` only when a shared counter truly needs no ordering, and document why.

Try this

1. Drop `_Atomic` and use a plain `int`; run under ThreadSanitizer (`-fsanitize=thread`) and read the race report.
2. Replace `relaxed` with `seq_cst` and confirm the count is unchanged (correctness) while thinking about fence cost on weakly ordered CPUs.

3. Build a release/acquire hand-off: thread A stores a payload then `atomic_store_explicit(&ready, 1, memory_order_release)`; thread B spins with `acquire`.

Sources

- ISO/IEC 9899:2011 §7.17 (`stdatomic.h`)
- GCC documentation: Using the GNU Compiler Collection — Atomics
- C11 draft N1570 (public reference for the memory model)

A Lock-Free SPSC Ring Buffer in C11

A Lock-Free SPSC Ring Buffer in C11

Thesis

A queue with exactly **one producer thread and one consumer thread** is the one lock-free structure that is easy to get right. You don't need compare-and-swap. You need two indices, each written by only one thread, one release store to publish a value, and one acquire load to see it. This chapter builds a 60-line ring buffer and tests the three properties that matter:

- **Ordering:** 50 million values arrive in order.
- **Wraparound:** free-running `size_t` indices that overflow are still correct.
- **Memory order:** ThreadSanitizer catches a relaxed publish that x86 lets through.

It then measures the two standard speed-ups, cache-line padding and cached indices. On this machine one of them did *not* do what the textbooks say.

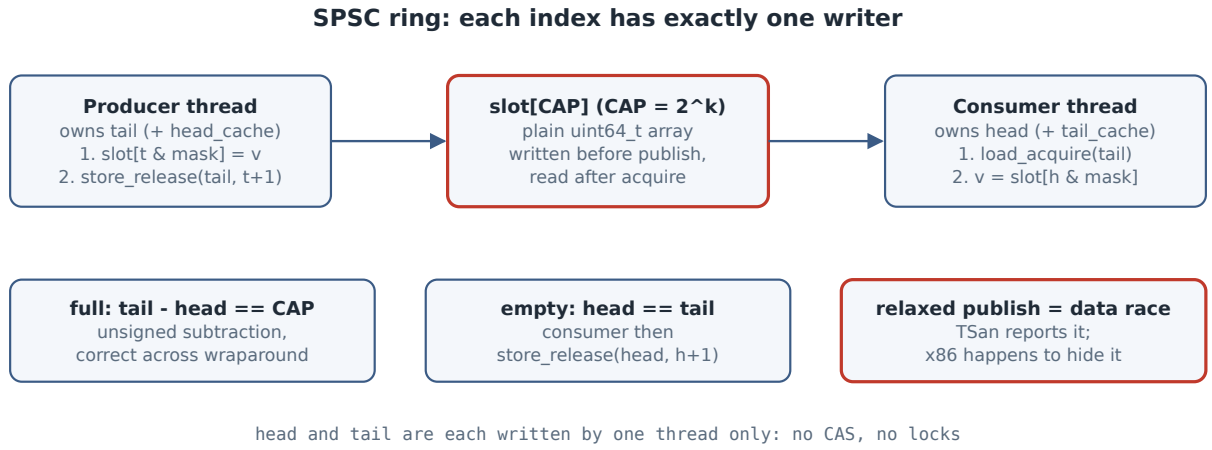


Figure 1: SPSC ring: producer owns tail, consumer owns head, slots published by release/acquire

Everything here was run with GCC 14.2.0 (`-std=c23`) on Linux 6.12, on an 8-vCPU x86-64 Xeon VM with 64-byte cache lines.

The design in four rules

1. **One writer per index.** Only the producer stores `tail` and only the consumer stores `head`. Neither thread ever needs read-modify-write on them, so there is no CAS loop and no ABA problem.

2. **Free-running indices, power-of-two capacity.** `tail` and `head` count up forever. The slot is `index & (CAP - 1)`, the fill level is `tail - head`, and the queue is full when that equals `CAP`. Unsigned subtraction is defined modulo 2 , and a power-of-two `CAP` divides 2 , so overflow is harmless. This also avoids the classic "leave one slot empty" trick, which you need if you store the indices already masked, because then full and empty both look like `head == tail`.
3. **Write the slot, *then* release-store the index.** The slot itself is a plain `uint64_t`. What makes it safe to read is the release/acquire pair on `tail`: everything the producer wrote before the release store is visible to a consumer whose acquire load reads that value (memory_order). The consumer frees the slot the same way, with a release store of `head` after the read.
4. **Your own index needs only a relaxed load.** Each thread is the only writer of its own index, so reading it never races.

The code

```c
// spsc.h: bounded single-producer/single-consumer ring buffer (C11 atomics)
#ifndef SPSC_H
#define SPSC_H
#include <stdatomic.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#ifndef SPSC_PAD
#define SPSC_PAD 1          /* put head and tail on separate cache lines */
#endif
#ifndef SPSC_CACHE
#define SPSC_CACHE 1        /* keep a private copy of the other side's index */
#endif
#ifndef SPSC_PUBLISH
#define SPSC_PUBLISH memory_order_release
#endif

#define SPSC_CAP 1024u      /* must be a power of two */
#if SPSC_PAD
#define LINE _Alignas(64)
#else
#define LINE
#endif

typedef struct {
    LINE _Atomic size_t head;   /* next slot to read; written only by consumer */
    size_t tail_cache;          /* consumer's last view of tail */
    LINE _Atomic size_t tail;   /* next slot to write; written only by producer */
    size_t head_cache;          /* producer's last view of head */
```

```
    LINE uint64_t slot[SPSC_CAP];
} spsc;

static inline bool spsc_push(spsc *q, uint64_t v) {
    size_t t = atomic_load_explicit(&q->tail, memory_order_relaxed);   /* own index */
#if SPSC_CACHE
    if (t - q->head_cache == SPSC_CAP) {
        q->head_cache = atomic_load_explicit(&q->head, memory_order_acquire);
        if (t - q->head_cache == SPSC_CAP) return false;               /* full */
    }
#else
    if (t - atomic_load_explicit(&q->head, memory_order_acquire) == SPSC_CAP) return false;
#endif
    q->slot[t & (SPSC_CAP - 1)] = v;                                   /* plain write */
    atomic_store_explicit(&q->tail, t + 1, SPSC_PUBLISH);              /* publish */
    return true;
}

static inline bool spsc_pop(spsc *q, uint64_t *out) {
    size_t h = atomic_load_explicit(&q->head, memory_order_relaxed);   /* own index */
#if SPSC_CACHE
    if (h == q->tail_cache) {
        q->tail_cache = atomic_load_explicit(&q->tail, memory_order_acquire);
        if (h == q->tail_cache) return false;                          /* empty */
    }
#else
    if (h == atomic_load_explicit(&q->tail, memory_order_acquire)) return false;
#endif
    *out = q->slot[h & (SPSC_CAP - 1)];                                /* plain read */
    atomic_store_explicit(&q->head, h + 1, memory_order_release);      /* free the slot */
    return true;
}
#endif
```

`SPSC_PAD`, `SPSC_CACHE` and `SPSC_PUBLISH` are compile-time switches for the experiments below. Production code would pick one setting and remove the `#if`s.

`tail_cache` and `head_cache` are **plain** fields. Each one is touched by only one thread: `tail_cache` by the consumer and `head_cache` by the producer. They keep that thread's last view of the *other* thread's index, so the expensive cross-core load only happens when the queue looks full (for the producer) or empty (for the consumer).

Test 1: order and integrity under load

The benchmark pins the producer to one CPU and the consumer to another (`pthread_setaffinity_np`), pushes 1..N, and checks that every value comes out in order and that the sum is N(N+1)/2. It also counts how often `pop` found the queue empty and how often `push` found it full:

```c
// bench.c: move N values through the ring, check order and sum, report Mops/s
#define _GNU_SOURCE
#include "spsc.h"
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#ifndef N
#define N 50000000ull
#endif
static spsc q;
static uint64_t full_spins;   /* producer only */

static void pin(int cpu) {
    cpu_set_t s; CPU_ZERO(&s); CPU_SET(cpu, &s);
    pthread_setaffinity_np(pthread_self(), sizeof s, &s);
}

static void *producer(void *arg) {
    (void)arg; pin(PCPU);
    for (uint64_t i = 1; i <= N; i++)
        while (!spsc_push(&q, i)) full_spins++;
    return NULL;
}

int main(void) {
    pthread_t p;
    struct timespec a, b;
    clock_gettime(CLOCK_MONOTONIC, &a);
    pthread_create(&p, NULL, producer, NULL);
    pin(CCPU);
    uint64_t v, expect = 1, sum = 0, empty_spins = 0;
    while (expect <= N) {
        if (!spsc_pop(&q, &v)) { empty_spins++; continue; }
        if (v != expect) { printf("ORDER BUG: got %llu want %llu\n",
                                  (unsigned long long)v, (unsigned long long)expect); return
        sum += v; expect++;
    }
    pthread_join(p, NULL);
    clock_gettime(CLOCK_MONOTONIC, &b);
    double s = (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / 1e9;
    printf("pad=%d cache=%d  sum ok=%d  %.3f s  %6.1f Mops/s  empty/item %.2f  full/item %.2
           SPSC_PAD, SPSC_CACHE, sum == N * (N + 1) / 2, s, N / s / 1e6,
           (double)empty_spins / N, (double)full_spins / N);
    return 0;
```

```
}
```

Test 2: wraparound

Start both indices three steps before `SIZE_MAX`, push six values, drain them, then fill the queue to capacity:

```
// wrap.c: free-running indices survive size_t overflow because only t - h is used
#include "spsc.h"
#include <stdio.h>

static spsc q;

int main(void) {
    size_t start = SIZE_MAX - 2;                /* three increments from wrapping */
    atomic_store(&q.head, start); atomic_store(&q.tail, start);
    q.head_cache = q.tail_cache = start;
    for (uint64_t i = 0; i < 6; i++) spsc_push(&q, 100 + i);
    printf("tail=%zu head=%zu  tail-head=%zu\n",
           atomic_load(&q.tail), atomic_load(&q.head), atomic_load(&q.tail) - atomic_load(&q
    uint64_t v;
    while (spsc_pop(&q, &v)) printf("%llu ", (unsigned long long)v);
    printf("\nfill to capacity: ");
    unsigned n = 0;
    while (spsc_push(&q, n)) n++;
    printf("%u pushes accepted (SPSC_CAP=%u)\n", n, SPSC_CAP);
    return 0;
}
```

```
$ gcc -std=c23 -O2 -Wall -Wextra wrap.c -o wrap && ./wrap
tail=3 head=18446744073709551613  tail-head=6
100 101 102 103 104 105
fill to capacity: 1024 pushes accepted (SPSC_CAP=1024)
```

`tail` wrapped to 3 while `head` is still near 2 , `tail - head` is correctly 6, the values come out in order, and exactly 1024 pushes fit. No slot is wasted.

Test 3: what a relaxed publish breaks

The correct build is clean under ThreadSanitizer (manual):

```
$ gcc -std=c23 -O1 -g -pthread -fsanitize=thread -DPCPU=2 -DCCPU=6 -DN=200000ull bench.c -o
pad=1 cache=1  sum ok=1  0.113 s     1.8 Mops/s  empty/item 0.12  full/item 0.00
```

Now change the publishing store of `tail` to `memory_order_relaxed`:

```
$ gcc -std=c23 -O1 -g -pthread -fsanitize=thread -DPCPU=2 -DCCPU=6 -DN=200000ull \
      -DSPSC_PUBLISH=memory_order_relaxed bench.c -o tsan_bad && ./tsan_bad
==================
WARNING: ThreadSanitizer: data race (pid=1646727)
  Read of size 8 at 0x5564fca84180 by main thread:
    #0 spsc_pop /tmp/spsc/spsc.h:59 (tsan_bad+0x1458) (BuildId: 433919b12514ebb5e569e3c46ed5
    #1 main /tmp/spsc/bench.c:36 (tsan_bad+0x1458)

  Previous write of size 8 at 0x5564fca84180 by thread T1:
    #0 spsc_push /tmp/spsc/spsc.h:44 (tsan_bad+0x1302) (BuildId: 433919b12514ebb5e569e3c46ed
    #1 producer /tmp/spsc/bench.c:24 (tsan_bad+0x1302)

  Location is global 'q' of size 8320 at 0x5564fca84100 (tsan_bad+0x4180)

  Thread T1 (tid=1646729, running) created by main thread at:
    #0 pthread_create ../../../../src/libsanitizer/tsan/tsan_interceptors_posix.cpp:1022 (li
    #1 main /tmp/spsc/bench.c:32 (tsan_bad+0x1397) (BuildId: 433919b12514ebb5e569e3c46ed5d59

SUMMARY: ThreadSanitizer: data race /tmp/spsc/spsc.h:59 in spsc_pop
==================
ThreadSanitizer: reported 1 warnings
pad=1 cache=1  sum ok=1  0.152 s     1.3 Mops/s  empty/item 0.00  full/item 4.04
```

TSan reports the plain read of the slot (`spsc.h:59`) racing with the plain write (`spsc.h:44`). With a relaxed store there is no happens-before edge between them, so the consumer can see the new `tail` before it sees the value. Note the last line, though: **`sum ok=1`**. On x86-64 a release store and a relaxed store both compile to an ordinary `mov`, and the hardware doesn't reorder stores with other stores, so the broken version *worked* here. It is still undefined behaviour. The compiler is allowed to move the slot write below the relaxed store, and weakly ordered CPUs such as Arm and POWER can reorder them in hardware. Passing tests on x86 proves nothing about memory order. Run TSan.

Test 4: padding and cached indices, measured

The usual advice is (a) put `head` and `tail` on separate cache lines so the two cores don't fight over one line, and (b) cache the other thread's index so you only read it when you have to. Here are the four combinations, three rounds each, 50 M items per run:

```
$ for p in 0 1; do for c in 0 1; do
    gcc -std=c23 -O2 -pthread -DPCPU=2 -DCCPU=6 -DSPSC_PAD=$p -DSPSC_CACHE=$c bench.c -o b &
  done; done   # repeated 3 times
pad=0 cache=0  sum ok=1  0.501 s    99.7 Mops/s  empty/item 0.49  full/item 0.03
pad=0 cache=1  sum ok=1  0.256 s   195.4 Mops/s  empty/item 0.03  full/item 0.00
pad=1 cache=0  sum ok=1  1.158 s    43.2 Mops/s  empty/item 4.90  full/item 0.02
```

```
pad=1 cache=1  sum ok=1  1.041 s    48.0 Mops/s  empty/item 2.49  full/item 0.01
pad=0 cache=0  sum ok=1  0.283 s   176.7 Mops/s  empty/item 0.07  full/item 0.05
pad=0 cache=1  sum ok=1  0.294 s   170.1 Mops/s  empty/item 0.03  full/item 0.02
pad=1 cache=0  sum ok=1  1.566 s    31.9 Mops/s  empty/item 8.41  full/item 0.02
pad=1 cache=1  sum ok=1  0.502 s    99.5 Mops/s  empty/item 0.86  full/item 0.07
pad=0 cache=0  sum ok=1  0.551 s    90.7 Mops/s  empty/item 0.77  full/item 0.12
pad=0 cache=1  sum ok=1  0.318 s   157.4 Mops/s  empty/item 0.05  full/item 0.00
pad=1 cache=0  sum ok=1  1.466 s    34.1 Mops/s  empty/item 6.25  full/item 0.02
pad=1 cache=1  sum ok=1  1.102 s    45.4 Mops/s  empty/item 2.78  full/item 0.02
```

What the numbers show on this VM:

- **Cached indices were the more dependable win.** With padding they raised throughput from 32–43 Mops/s to 45–100 Mops/s. Without padding they gave 157–195 Mops/s, against 91–177 Mops/s uncached. In round 2 the two were within noise of each other.
- **Padding on its own made things slower here, not faster.** `pad=1 cache=0` was the slowest configuration in every round, at 32 to 43 Mops/s against 91 to 177 Mops/s for `pad=0 cache=0`. That goes against the usual advice, so the counters matter. In the slow runs, `empty/item` is between 4.9 and 8.4. The consumer kept catching up with the producer and spinning on an empty queue, which means it re-reads `tail` immediately after nearly every push. In that "always nearly empty" regime each item costs cache-line transfers whatever the layout, and splitting the indices across lines didn't help on this hardware.
- **The spread is large.** The same configuration varied by more than 2× between rounds (`pad=1 cache=1`: 45 to 100 Mops/s). One run proves nothing, so repeat runs and report the range.

The usual advice is aimed at a different situation: a producer and consumer running at similar speeds with the queue partly full. There, false sharing between `head` and `tail` is a real cost. **Measure your own workload on your own hardware** before you add `_Alignas(64)`. If the consumer is much faster than the producer, consider batching: pop everything available up to the cached `tail` before going back to the shared index.

Pitfalls

- **More than one producer or consumer.** All of this depends on one writer per index. A second producer makes `tail` a contended read-modify-write and the design breaks. Use an MPSC or MPMC queue for that case.
- **Non-power-of-two capacity with `&`.** The mask only works for 2 . With modulo instead of a mask, free-running indices are wrong at the 2 wrap unless `CAP` divides 2 .
- **Storing pointers to stack data.** The consumer may read a slot long after the producer's frame is gone. Store values or owned heap pointers.
- **Spinning forever.** The benchmark busy-waits. A real queue needs a back-off or blocking path (`futex`, `eventfd`, condition variable) for when the other side is idle.
- **Trusting x86.** See Test 3.

Takeaways

- An SPSC ring needs two single-writer indices and one release/acquire pair per direction. No CAS is involved.
- Free-running unsigned indices with a power-of-two capacity handle full/empty and wraparound with a single subtraction.
- `memory_order_relaxed` on the publish is a real data race that x86 hides. ThreadSanitizer finds it.
- Cached indices helped in all but one round. Cache-line padding did not help here when the consumer kept outrunning the producer. Measure before and after.

Sources

- cppreference, `memory_order` (release/acquire synchronization)
- Linux kernel documentation, Circular Buffers (power-of-two indexing and the producer/consumer barrier pairing)
- Google Sanitizers, ThreadSanitizer C/C++ manual
- Linux man-pages, `pthread_setaffinity_np(3)`
- Erik Rigtorp, Optimizing a ring buffer for throughput (the cached-index technique)

Flexible Array Members Done Right

Flexible Array Members Done Right

Thesis

A **flexible array member** (C99) is the standard way to attach a variable-length payload to a struct without a second allocation. `sizeof` the struct excludes the payload; you allocate `sizeof(header) + n` bytes and treat `data[]` as the tail.

Mental model

```
  malloc(sizeof(struct packet) + n)

   len        data[0 .. n-1]

       ↑ sizeof(struct packet) stops here
```

Rules of thumb:

- The FAM must be the **last** member.
- Do not declare arrays of the struct type.
- Prefer `sizeof *p + n` over guessing padding.

Worked example

Verified with **GCC 14.2**, `-std=c11 -Wall`.

```c
/* fam.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

struct packet {
  uint16_t len;
  char data[];          /* flexible array member */
};

int main(void) {
```

```
  size_t n = 5;
  struct packet *p = malloc(sizeof *p + n);
  if (!p) return 1;
  p->len = (uint16_t)n;
  memcpy(p->data, "hello", n);
  printf("sizeof(struct packet)=%zu payload=%.*s len=%u\n",
         sizeof(struct packet), (int)p->len, p->data, p->len);
  free(p);
  return 0;
}
```

```
gcc -O2 -std=c11 -Wall fam.c -o fam && ./fam
```

```
sizeof(struct packet)=2 payload=hello len=5
```

`sizeof(struct packet)` is just the fixed header (here 2 bytes, possibly padded). The five payload bytes live past that.

Trap

`struct packet p; p.data[0] = 'x';` on a stack instance. There is no storage for `data`. Same for `struct packet arr[10];`. Also avoid the pre-C99 GCC `[0]` hack in new code — use `[]`.

Boring rule

One allocation: `malloc(sizeof *hdr + payload_bytes)`. Check overflow of the size addition before calling `malloc`. Free the single pointer.

Try this

1. Add a `uint32_t checksum` before `data[]` and re-check `sizeof`.
2. Pass `-Wflexible-array-extensions` / pedantic flags and clean warnings.
3. Compare with a pointer-to-heap design (`char *data`) and count syscalls under `strace` (when available).

Sources

- ISO/IEC 9899:1999 §6.7.2.1 (flexible array members)
- CERT C: MEM33-C

Function Interposition with LD_PRELOAD

Function Interposition with LD_PRELOAD

Thesis

On a dynamically linked Linux program, every call to an exported library function such as `open` or `malloc` is resolved by the dynamic loader `ld.so` at run time. `LD_PRELOAD` puts your own shared object at the front of that search, so your definition wins. Inside the wrapper, `dlsym(RTLD_NEXT, "open")` hands you the definition that *would* have been used, so you can log, count, or fake a call and then pass it through. You don't need to recompile, relink, or have the source.

The technique is useful for tracing, fault injection and test doubles. It is also easy to get subtly wrong. This chapter builds a working shim, then shows the four places where it silently does nothing.

Mental model

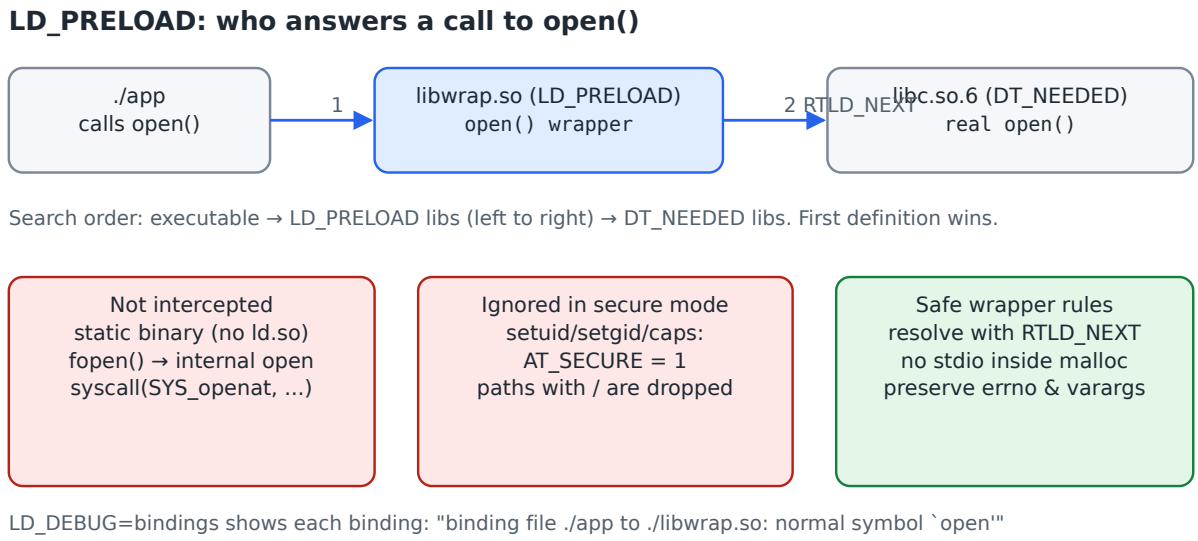


Figure 1: LD_PRELOAD puts libwrap.so between the program and libc; RTLD_NEXT continues the search to the real definition; static binaries, libc-internal calls, raw syscalls and secure-execution processes are not intercepted

`ld.so(8)` defines the order. The loader builds one global lookup scope: the executable first, then each `LD_PRELOAD` object in order, then the `DT_NEEDED` dependencies breadth-first. A reference to `open` binds to the **first** object in that scope that defines it. `RTLD_NEXT` is a pseudo-handle for

`dlsym(3)` that means "search the objects *after* the one making this call", which is exactly the real implementation.

Worked example

Verified on Debian 13 with **GCC 14.2** and **glibc 2.41** (x86-64).

The target is an ordinary program that you won't modify:

```
/* app.c - an ordinary program that allocates and opens a file */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(void) {
  char *buf = malloc(64);
  if (!buf) return 1;
  strcpy(buf, "hello");
  int fd = open("/etc/hostname", O_RDONLY);
  printf("app: buf=%s fd=%d\n", buf, fd);
  if (fd >= 0) close(fd);
  free(buf);
  return 0;
}
```

The shim wraps `open` (logging the path and result) and counts `malloc` calls, printing the total from a destructor when the process exits:

```
/* wrap.c - LD_PRELOAD shim that wraps open() and counts malloc() */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <errno.h>
#include <fcntl.h>
#include <stdarg.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

static atomic_long n_malloc;

static void say(const char *msg) {          /* no stdio: safe inside malloc */
  ssize_t r = write(2, msg, strlen(msg));
```

```
  (void)r;
}

int open(const char *path, int flags, ...) {
  static int (*real_open)(const char *, int, ...);
  if (!real_open)
    real_open = (int (*)(const char *, int, ...))dlsym(RTLD_NEXT, "open");
  mode_t mode = 0;
  if (flags & (O_CREAT | O_TMPFILE)) {
    va_list ap;
    va_start(ap, flags);
    mode = (mode_t)va_arg(ap, int);
    va_end(ap);
  }
  int fd = real_open(path, flags, mode);
  int saved = errno;                          /* logging must not clobber errno */
  char line[256];
  int len = snprintf(line, sizeof line, "[wrap] open(\"%s\") = %d\n", path, fd);
  if (len > 0) say(line);
  errno = saved;
  return fd;
}

extern void *__libc_malloc(size_t);           /* glibc: avoids dlsym recursion */

void *malloc(size_t n) {
  atomic_fetch_add(&n_malloc, 1);
  return __libc_malloc(n);
}

__attribute__((destructor)) static void report(void) {
  char line[64];
  int len = snprintf(line, sizeof line, "[wrap] malloc calls: %ld\n",
                     atomic_load(&n_malloc));
  if (len > 0) say(line);
}
```

```
gcc -O2 -Wall -o app app.c
gcc -O2 -Wall -fPIC -shared -o libwrap.so wrap.c
./app
LD_PRELOAD=./libwrap.so ./app
```

```
app: buf=hello fd=5
[wrap] open("/etc/hostname") = 5
[wrap] malloc calls: 2
app: buf=hello fd=5
```

The first line comes from the plain run, and the next three from the preloaded run. (`fd=5` instead of `3` only means the lab shell already had descriptors 3 and 4 open.) Two details are worth noticing:

- The program's `printf` line appears **after** the destructor's line. Stdout went to a pipe, so it was fully buffered and only flushed during `exit`, after the shim's destructor ran. The shim writes straight to fd 2 with `write(2)` and isn't buffered.
- The count is 2, not 1. The program called `malloc` once, and stdio allocated its own buffer on the first `printf`. Interposition catches **every** caller of the exported symbol, including libc's.

Since glibc 2.34, `dlsym` lives in libc itself, so you no longer need `-ldl`. Older toolchains still need it.

Proving the binding with LD_DEBUG

`LD_DEBUG=bindings` makes the loader print each symbol binding it performs:

```
LD_DEBUG=bindings LD_PRELOAD=./libwrap.so ./app 2>&1 | grep -E "symbol .(open|malloc)'"
```

```
   1242272: binding file /lib/x86_64-linux-gnu/libc.so.6 [0] to ./libwrap.so [0]: normal sym
   1242272: binding file ./app [0] to ./libwrap.so [0]: normal symbol `malloc' [GLIBC_2.2.5]
   1242272: binding file ./app [0] to ./libwrap.so [0]: normal symbol `malloc' [GLIBC_2.2.5]
   1242272: binding file ./app [0] to ./libwrap.so [0]: normal symbol `open' [GLIBC_2.2.5]
   1242272: binding file ./libwrap.so [0] to /lib/x86_64-linux-gnu/libc.so.6 [0]: normal sym
```

The number at the start of each line is the process ID. The first line shows libc binding its *own* `malloc` reference to the shim, which is why the count includes stdio's buffer. The last line is the `dlsym(RTLD_NEXT, "open")` lookup: the shim's reference resolves to libc's `open`, the next definition in the search order.

errno has to survive the wrapper

Callers read `errno` right after a failed call. Anything the wrapper does after the real call, such as `snprintf` or `write`, may change it, so the shim saves and restores `errno`:

```
/* missing.c - errno must survive the wrapper */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
int main(void) {
  int fd = open("/no/such/file", O_RDONLY);
  printf("fd=%d errno=%s\n", fd, strerror(errno));
  return 0;
}
```

```
[wrap] open("/no/such/file") = -1
[wrap] malloc calls: 3
fd=-1 errno=No such file or directory
```

Where interposition silently does nothing

1. Static binaries

```
gcc -O2 -static -o app-static app.c
LD_PRELOAD=./libwrap.so ./app-static
```

```
app: buf=hello fd=5
```

A static executable has no `PT_INTERP`, so `ld.so` never runs and nobody reads `LD_PRELOAD`. Nothing is printed and there is no error.

2. Calls made inside libc

```
/* fopen.c - fopen() reaches the kernel without calling the exported open() */
#include <stdio.h>
int main(void) {
  FILE *f = fopen("/etc/hostname", "r");
  printf("fopen: %s\n", f ? "ok" : "failed");
  if (f) fclose(f);
  return 0;
}
```

```
[wrap] malloc calls: 2
fopen: ok
```

`fopen` opened the file, but the `open` wrapper never fired. glibc calls its internal, non-exported `__open` alias (it ends up in the `openat` syscall), and internal calls don't go through the dynamic symbol table. `malloc` *is* still caught here, because glibc deliberately routes its allocations through the interposable `malloc` symbol. Before you rely on intercepting a function, check how libc reaches it, or wrap the internal entry points (`fopen`, `openat`, `open64`) as well.

3. Raw system calls

`syscall(SYS_openat, ...)` and inline `syscall` instructions (common in Go binaries and in some runtimes) never call a library function. To observe those you need `strace`, seccomp or eBPF, not `LD_PRELOAD`.

4. Secure-execution mode

When the kernel starts a set-user-ID or set-group-ID program, or one with file capabilities, it passes `AT_SECURE=1` in the auxiliary vector (see `getauxval(3)`). In that mode, `ld.so(8)` documents that `LD_PRELOAD` entries containing a slash are ignored. Only libraries found in the standard search directories are loaded, and only if they are themselves set-user-ID. Without this rule, any user could inject code into `passwd`. This chapter doesn't reproduce the case, because building a setuid binary needs root; the behaviour above is taken from the man page.

Writing a wrapper that doesn't bite

- **Match the prototype exactly**, including varargs. `open` takes a third `mode_t` argument only for `O_CREAT`/`O_TMPFILE`, so read it with `va_arg` only then.
- **Don't recurse through `malloc`.** `dlsym` and stdio may allocate. In the `malloc` wrapper, call glibc's `__libc_malloc` (glibc-specific) or use a small static bootstrap buffer, and log with `write(2)`, not `printf`.
- **Preserve `errno`**, and return exactly what the real function returned.
- **Wrap the whole family.** `open`, `open64`, `openat` and `__open_2` (used by `_FORTIFY_SOURCE`) are different symbols.
- **Make it thread-safe.** Use atomics for counters. The lazy `real_open` initialization above is a benign race on x86-64 (every thread stores the same pointer), but use `pthread_once` if the setup does more.
- **Remember the destructor order.** Output from destructors can appear before the program's buffered stdout, as the first run shows.

Try this

1. Add `fopen` to the shim and confirm that `./fopen` now logs.
2. Make `malloc` fail on its 3rd call (set `errno = ENOMEM`, then `return NULL`) and watch how `app` handles out-of-memory. This is cheap fault injection.
3. Run `LD_DEBUG=libs LD_PRELOAD=./libwrap.so ./app` and find where the loader reports loading `libwrap.so` before `libc.so.6`.

Sources

- `ld.so(8)`: `LD_PRELOAD`, search order, secure-execution mode: https://man7.org/linux/man-pages/man8/ld.so.8.html
- `dlsym(3)`: `RTLD_NEXT`: https://man7.org/linux/man-pages/man3/dlsym.3.html
- `getauxval(3)`: `AT_SECURE`: https://man7.org/linux/man-pages/man3/getauxval.3.html
- glibc manual, "Replacing malloc": https://sourceware.org/glibc/manual/latest/html_node/Replacing-malloc.html
- glibc 2.34 release notes (libdl merged into libc): https://sourceware.org/pipermail/libc-alpha/2021-August/129718.html

GCC attribute((cleanup)): scope-based free without C23 defer

GCC attribute((cleanup)): scope-based free without C23 defer

After reading this chapter, you will be able to implement resource cleanup functions using GCC and Clang's `__attribute__((cleanup))`, explain how the attribute interacts with pointers and scope exit, write safe wrappers for heap memory and file handles, and understand how it differs from standard C23 `defer`.

Thesis

Manual resource management in C is notoriously error-prone. Functions with multiple error exit paths often duplicate calls to `free()`, `fclose()`, or `close()`, leading to code bloat and memory leaks when an early return skips a cleanup statement.

While ISO C23 introduces the standard `defer` mechanism, toolchains and codebases in production today frequently rely on compiler extensions. GCC and Clang have long supported `__attribute__((cleanup(fn)))`, a variable attribute that binds a cleanup callback to a variable's lexical lifetime. When execution exits the variable's scope—whether via normal control flow, `return`, `break`, or `goto`—the compiler automatically inserts a call to `fn`.

This chapter demonstrates how to write clean, type-safe cleanup handlers, examines how pointer indirection works under the hood, provides working examples for dynamic allocation and file management, and outlines common pitfalls when using GNU cleanup attributes.

Mental model

The key to using `__attribute__((cleanup))` correctly is understanding **what parameter is passed to the callback function**.

When a variable marked with `__attribute__((cleanup(fn)))` goes out of scope, the compiler generates a call to `fn(&var)`. The callback function *always* receives a pointer to the variable being cleaned up, not the variable's value itself.

Variable Declaration	Type of Variable	Argument Passed to `fn`	Type Expected by `fn`
`int x;`	`int`	`&x`	`int *`
`char *msg;`	`char *`	`&msg`	`char **` (or `void *`)
`FILE *fp;`	`FILE *`	`&fp`	`FILE **` (or `void *`)

Because the function receives a pointer to the variable: - For an integer or struct, the cleanup function takes `int *` or `struct my_struct *`. - For a heap pointer (`char *`), the cleanup function receives a pointer-to-pointer (`char **`).

Worked example

Verified on Debian 13 with **GCC 14.2**.

The following complete program allocates dynamic memory for a string variable annotated with `__attribute__((cleanup(freep)))`. The cleanup handler `freep` runs automatically when `main` returns.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void freep(void *pp) {
  free(*(void **)pp);
}

int main(void) {
  __attribute__((cleanup(freep))) char *msg = malloc(32);
  if (!msg) return 1;
  snprintf(msg, 32, "cleanup frees me");
  puts(msg);
  return 0; /* freep runs here */
}
```

Compile and run the program:

```
gcc -O2 -Wall -Wextra -o cleanup-demo cleanup-demo.c
./cleanup-demo
```

The command produced the following output:

```
<<OUTPUT:run1>>
```

Breakdown of execution flow

1. `msg` is declared on the stack as a pointer to `char` (`char *`) and allocated 32 bytes via `malloc`.
2. The compiler associates `msg` with the cleanup handler `freep`.
3. `snprintf` copies `"cleanup frees me"` into the buffer, and `puts` prints it to standard output.
4. When `return 0` executes, `msg` goes out of scope. The compiler automatically inserts a call equivalent to `freep(&msg)`.

5. Inside `freep(void *pp)`, `pp` points to `msg` (type `char **`). The expression `*(void **)pp` dereferences `pp` to obtain the actual allocated memory address (`char *`) and passes it to standard `free()`.

Advanced usage: Generic auto-free macros and FILE handlers

Writing custom cleanup functions for every single variable type can become repetitive. In standard C practice, helper macros and generic handlers are used to simplify cleanup declarations.

Helper macros for pointers and files

The handler function receives a pointer to the variable. By accepting `void *` in the handler signature, a single helper function can free any heap-allocated pointer type without type mismatch warnings.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Generic cleanup handler for heap allocations */
static inline void auto_freep(void *pp) {
  void *ptr = *(void **)pp;
  if (ptr) {
    free(ptr);
  }
}

/* Cleanup handler for standard FILE stream handles */
static inline void auto_fclosep(FILE **fpp) {
  if (fpp && *fpp) {
    fclose(*fpp);
  }
}

#define autofree __attribute__((cleanup(auto_freep)))
#define autoclose __attribute__((cleanup(auto_fclosep)))

static int process_file(const char *filename) {
  autoclose FILE *fp = fopen(filename, "r");
  if (!fp) {
    return -1;
  }

  autofree char *buffer = malloc(256);
  if (!buffer) {
```

```
    return -1; /* auto_fclosep(fp) runs automatically on early return */
  }

  if (fgets(buffer, 256, fp) != NULL) {
    printf("Read line: %s", buffer);
  }

  return 0;
  /* Both auto_freep(&buffer) and auto_fclosep(&fp) run here in reverse order of declaration
}

int main(void) {
  /* Process an existing file */
  process_file("/etc/hostname");
  return 0;
}
```

Compile and run the program:

```
gcc -O2 -Wall -Wextra -o resource-cleanup resource-cleanup.c
./resource-cleanup
```

The command produced the following output:

```
<<OUTPUT:run2>>
```

Cleanup ordering rules

When multiple variables in the same block scope have cleanup attributes, their cleanup functions are executed in **strict reverse order of declaration** (last declared, first cleaned up), acting like a stack:

1. `buffer` is declared second, so `auto_freep(&buffer)` runs first.
2. `fp` is declared first, so `auto_fclosep(&fp)` runs second.

This ordering guarantees that resources dependent on earlier initializations (such as a buffer operating on an open file handle) are released safely prior to closing the underlying handle.

GCC cleanup vs. C23 defer

ISO C23 introduced the `defer` keyword, providing a standardized syntax for scope-based resource cleanup. While `__attribute__((cleanup))` is a compiler extension, `defer` is built into the language standard.

Feature	GCC `__attribute__((cleanup))`	C23 `defer`
Standardization	GNU extension (supported by GCC and Clang)	ISO C23 standard keyword
Syntax	Attribute on variable declaration	Statement block (e.g., `defer { free(p); }`)
Scope Level	Tied to variable lexical lifetime	Tied to enclosing block scope exit
Argument Passing	Indirect (passes pointer to variable: `&var`)	Direct expressions evaluated at scope exit
Execution Order	Reverse declaration order of variables	Reverse execution order of `defer` statements

While `defer` provides a cleaner, inline block-based syntax without auxiliary helper functions, `__attribute__((cleanup))` remains essential for target environments constrained to C11 or C99 standards using GCC or Clang toolchains.

Common mistakes

1. Wrong indirection level in the cleanup function

The most frequent bug when using `__attribute__((cleanup))` is expecting the callback to receive the variable's value directly rather than a pointer to it.

```
/* WRONG: Expected char *, but receives char ** */
void bad_free(char *p) {
  free(p); /* Program crashes or corrupts heap! */
}

int main(void) {
  /* Passing bad_free causes compiler warning and invalid memory access */
  __attribute__((cleanup(bad_free))) char *ptr = malloc(64);
  return 0;
}
```

Because GCC passes `&ptr` (type `char **`) to `bad_free`, treating `p` as `char *` attempts to pass the address of the local stack variable `ptr` to `free()`.

Fix: Ensure the callback accepts `void *pp` and dereferences it: `free(*(void **)pp)`.

2. Using `longjmp` or `exit`

Cleanup handlers execute automatically when control flow leaves the local scope via normal block fallthrough, `return`, `goto`, or `break`.

However, cleanup attributes do **not** run if the process terminates abruptly or bypasses stack unwinding: - Calling `exit()`, `_Exit()`, or `abort()`. - Unwinding via `longjmp()` (unless operating under specific unwind-aware extensions).

3. Transferring ownership without resetting pointers

If a function successfully constructs an object and returns it to the caller, automatic cleanup must be suppressed to prevent a double-free.

```
/* Transferring ownership safely */
char *create_message(void) {
  autofree char *buf = malloc(64);
  if (!buf) return NULL;

  snprintf(buf, 64, "Hello, World!");

  char *ret = buf;
  buf = NULL; /* Prevent auto_freep from releasing the memory */
  return ret;
}
```

In `auto_freep`, checking `if (ptr != NULL)` ensures that setting `buf = NULL` safely neutralizes the cleanup action upon successful return.

Summary

- `__attribute__((cleanup(fn)))` automates resource management in GCC and Clang by running a function when a variable goes out of scope.
- The callback function receives a pointer to the target variable (`&var`), requiring pointer dereferencing inside the cleanup function.
- Cleanup attributes execute in reverse order of variable declaration within the same scope.
- They execute on all standard scope exits, including early `return` or `goto` statements, reducing cleanup boilerplate and preventing leaks.
- To transfer ownership of an allocated resource, clear the variable (e.g., set to `NULL`) before returning.

Sources

- GCC Common Variable Attributes
- GCC Variable Attributes Documentation

memfd_create and file seals: anonymous files peers cannot rewrite

memfd_create and file seals: anonymous files peers cannot rewrite

After this page you can create an anonymous tmpfs file with `memfd_create`, enable sealing with `MFD_ALLOW_SEALING`, apply `F_ADD_SEALS`, and verify that a sealed write fails while reads still succeed. That is the legitimate shared-memory handoff pattern: one process fills a buffer, seals it, and passes the fd to a peer that can trust the bytes will not change underfoot.

Thesis

Ordinary shared memory between processes that do not fully trust each other is painful. A peer can shrink the mapping (`SIGBUS` on touch) or rewrite bytes after you checked them (TOCTOU). **File seals** on a `memfd` let the producer freeze growth, shrinkage, and/or writes before the consumer maps the fd. The consumer calls `F_GET_SEALS` and proceeds without copying everything defensively.

`memfd_create` (Linux 3.17+, glibc 2.27+) returns a file descriptor to a volatile anonymous file — like a tmpfs file that never had a path. Sealing is optional and **off** unless you pass `MFD_ALLOW_SEALING`.

Verified below with **gcc** on the authoring Linux box.

Flags that matter

Flag / seal	Role
`MFD_CLOEXEC`	Close-on-exec on the new fd
`MFD_ALLOW_SEALING`	Allow later `F_ADD_SEALS`; without it the fd starts with `F_SEAL_SEAL` already set and further seals fail with `EPERM`
`F_SEAL_WRITE`	No more writes; existing shared writable maps must be gone first (or use `F_SEAL_FUTURE_WRITE`)
`F_SEAL_SHRINK` / `F_SEAL_GROW`	Block `ftruncate` in that direction
`F_SEAL_SEAL`	Block further seal changes
`F_SEAL_FUTURE_WRITE`	Block new writes/maps while keeping current writable maps

Lab: seal, then fail a write

```c
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#ifndef MFD_CLOEXEC
#define MFD_CLOEXEC 0x0001U
#endif
#ifndef MFD_ALLOW_SEALING
#define MFD_ALLOW_SEALING 0x0002U
#endif
#ifndef F_ADD_SEALS
#define F_ADD_SEALS 1033
#define F_GET_SEALS 1034
#define F_SEAL_SEAL 0x0001
#define F_SEAL_SHRINK 0x0002
#define F_SEAL_GROW 0x0004
#define F_SEAL_WRITE 0x0008
#endif

int main(void) {
  int fd = memfd_create("demo", MFD_CLOEXEC | MFD_ALLOW_SEALING);
  if (fd < 0) { perror("memfd_create"); return 1; }
  const char *msg = "sealed-bytes\n";
  if (write(fd, msg, strlen(msg)) < 0) { perror("write"); return 1; }
  if (fcntl(fd, F_ADD_SEALS, F_SEAL_WRITE | F_SEAL_SHRINK | F_SEAL_GROW | F_SEAL_SEAL) < 0)
    perror("F_ADD_SEALS"); return 1;
  }
  int seals = fcntl(fd, F_GET_SEALS);
  printf("seals=0x%x\n", seals);
  errno = 0;
  if (write(fd, "x", 1) < 0)
    printf("write_after_seal_errno=%d (%s)\n", errno, strerror(errno));
  else
    printf("unexpected_write_ok\n");
  char buf[64]={0};
  lseek(fd, 0, SEEK_SET);
  ssize_t n = read(fd, buf, sizeof buf - 1);
  printf("read_n=%zd data=%s", n, buf);
  close(fd);
  return 0;
```

```
}
```

Build and run:

```
gcc -O2 -Wall seal_demo.c -o seal_demo
./seal_demo
```

Verified output:

```
seals=0xf
write_after_seal_errno=1 (Operation not permitted)
read_n=13 data=sealed-bytes
```

`seals=0xf` is the bitmask of `SEAL|SHRINK|GROW|WRITE` (bits 0–3). The post-seal `write` returns `-1` with `errno == EPERM` (`Operation not permitted`). The earlier bytes remain readable.

Gotcha we hit first

Calling `memfd_create("demo", MFD_CLOEXEC)` *without* `MFD_ALLOW_SEALING` makes `F_ADD_SEALS` fail immediately with `Operation not permitted`. The man page states the default seal set is `F_SEAL_SEAL` when allowing seals was not requested — you locked yourself out before you started.

Producer / consumer sketch

1. Producer: `memfd_create(name, MFD_CLOEXEC | MFD_ALLOW_SEALING)`.
2. `ftruncate` or `write` the payload; `mmap` if you fill by store.
3. Unmap writable mappings if you need `F_SEAL_WRITE` (or prefer `F_SEAL_FUTURE_WRITE`).
4. `fcntl(fd, F_ADD_SEALS, … | F_SEAL_SEAL)`.
5. Pass `fd` over a Unix socket with `SCM_RIGHTS`, or let a child inherit it, or open `/proc/<pid>/fd/<n>` when that trust model fits.
6. Consumer: `F_GET_SEALS`, refuse to proceed if expected seals are missing, then `mmap` read-only.

Security note (short)

Attackers abuse `memfd_create` for fileless executable staging (`fexecve` on an anonymous fd). That is why defenders watch for odd `memfd:` entries under `/proc/*/fd` and why you should combine `MFD_CLOEXEC` with least privilege when you use memfds yourself. This page teaches the sealing API for honest IPC — not loaders.

Pitfalls

1. **Forgetting `MFD_ALLOW_SEALING`.** Seals fail with `EPERM`.
2. **`F_SEAL_WRITE` while a writable `mmap` still exists.** The add-seals call fails; unmap first or use `F_SEAL_FUTURE_WRITE`.
3. **Assuming seals survive a fresh `memfd_create`.** Each fd starts empty (when allowed) or fully seal-locked (when not).
4. **Leaking the fd across `exec`.** Use `MFD_CLOEXEC` unless inheritance is intentional.
5. **Treating `/proc/self/fd/N` names as stable paths.** The `memfd:` symlink is for debugging; pass the fd, do not reopen by guessed path from another user.

Relation to `shm_open` and `mmap(MAP_ANONYMOUS)`

`shm_open` creates a named POSIX shared-memory object that appears under `/dev/shm` on Linux. `memfd_create` skips the name and the directory entry: the only handles are file descriptors. Plain `mmap(..., MAP_ANONYMOUS)` never gives you an fd to send over a socket. When you need "anonymous *and* sendable *and* sealable", `memfd_create` is the syscall that hits all three.

Checklist for a sealed handoff

1. Create with `MFD_CLOEXEC | MFD_ALLOW_SEALING`.
2. Size and fill; drop writable maps before `F_SEAL_WRITE`.
3. Add seals including `F_SEAL_SEAL` last if you want them frozen.
4. Transfer the fd; consumer verifies `F_GET_SEALS`.
5. Consumer maps read-only and never calls `F_ADD_SEALS` unless you designed a two-step seal.

Summary

- `memfd_create` gives an anonymous tmpfs-backed fd for IPC and sealing.
- Pass `MFD_ALLOW_SEALING` or you cannot add seals later.
- `F_ADD_SEALS` freezes write/shrink/grow; verify with `F_GET_SEALS` and a failed write.
- Hand the sealed fd to the consumer; they can trust contents without defensive copies.
- Use CLOEXEC; do not confuse this API with fileless-malware recipes.

Sources

- memfd_create(2)
- fcntl(2)

Workbook

Examples

Foundations

Data Types

Control Flow

Functions

Arrays and Strings

Pointers

Structures

File I/O

Modern C

Data Structures

Network

Embedded

Performance

Testing

Advanced

Exercises

Foundations

Module 1: Foundations Exercises

Exercise 1: Basic Program Structure

Write a C program that prints the following output:

```
*************************
*   My First C Program   *
*************************
```

Requirements: - Use proper program structure with `#include` and `main()` function - Include comments explaining each part of your code - Compile and run your program successfully

Exercise 2: Personal Information Display

Create a program that displays your personal information in a formatted way: - Name - Age - City/Country - Favorite programming language - Years of programming experience

Requirements: - Use at least 3 different printf format specifiers - Use both single-line and multi-line comments - Make the output visually appealing with proper formatting

Exercise 3: Simple Calculator Input

Write a program that asks the user for two numbers and then displays: - The sum - The difference - The product - The quotient (handle division by zero)

Requirements: - Use `scanf()` to get input from the user - Check the return value of `scanf()` for error handling - Use proper variable declarations - Format the output clearly

Exercise 4: Character Analysis

Create a program that asks the user to enter a character and then displays: - The character itself - Its ASCII value - Whether it's a vowel (a, e, i, o, u) - Whether it's uppercase or lowercase (if it's a letter)

Requirements: - Use `getchar()` for character input - Handle whitespace properly - Use conditional statements (will be covered in detail in Module 3) - Display results in a formatted manner

Exercise 5: Temperature Converter

Write a program that converts temperatures between Celsius and Fahrenheit: - Ask the user for a temperature value - Ask the user to specify C for Celsius or F for Fahrenheit - Convert and display the equivalent temperature in the other unit

Formulas: - F = (C × 9/5) + 32 - C = (F - 32) × 5/9

Requirements: - Use proper input validation - Format output to 2 decimal places - Handle both uppercase and lowercase input for unit specification - Include clear prompts and instructions

Exercise 6: String Formatting Practice

Create a program that demonstrates various printf formatting options: - Display integers with different width specifications - Display floating-point numbers with different precision - Display strings with width and alignment options - Use escape sequences for formatting

Requirements: - Demonstrate at least 5 different printf format options - Include examples of left and right alignment - Use at least 3 different data types - Comment your code explaining each formatting choice

Exercise 7: Input Buffer Management

Write a program that demonstrates proper input buffer management: - Read a string with spaces using `fgets()` - Read an integer after the string - Read a character after the integer - Show how to properly handle the input buffer between different input types

Requirements: - Avoid common scanf pitfalls with buffer management - Use appropriate buffer sizes to prevent overflow - Handle input errors gracefully - Include comments explaining buffer management techniques

Exercise 8: Comprehensive Demo Program

Create a comprehensive program that integrates all Module 1 concepts: - Proper program structure with includes and main function - Multiple function calls (at least 2 custom functions) - Various printf formatting options - Multiple scanf inputs with error checking - Character I/O demonstrations - Well-commented code with both comment types

Requirements: - At least 50 lines of code - Use all major concepts from Module 1 - Include a README section in comments explaining your program - Compile without warnings (use -Wall -Wextra flags)

Solutions and Tips

Exercise 1 Solution Example:

```c
#include <stdio.h>

int main() {
    // Print a decorative header
    printf("*************************\n");
    printf("*   My First C Program   *\n");
    printf("*************************\n");
    return 0;
}
```

Exercise 3 Solution Example:

```c
#include <stdio.h>

int main() {
    float num1, num2;

    printf("Enter first number: ");
    if (scanf("%f", &num1) != 1) {
        printf("Invalid input!\n");
        return 1;
    }

    printf("Enter second number: ");
    if (scanf("%f", &num2) != 1) {
        printf("Invalid input!\n");
        return 1;
    }

    printf("\nResults:\n");
    printf("%.2f + %.2f = %.2f\n", num1, num2, num1 + num2);
    printf("%.2f - %.2f = %.2f\n", num1, num2, num1 - num2);
    printf("%.2f * %.2f = %.2f\n", num1, num2, num1 * num2);

    if (num2 != 0) {
        printf("%.2f / %.2f = %.2f\n", num1, num2, num1 / num2);
    } else {
        printf("Cannot divide by zero!\n");
    }

    return 0;
```

```
}
```

Common Pitfalls to Avoid:

1. **Missing `&` in scanf**: Always use `&` for non-string variables
2. **Buffer overflow**: Limit string input lengths
3. **Whitespace issues**: Handle newlines properly with character input
4. **Missing return 0**: Always return 0 from main function
5. **Forgetting headers**: Include `<stdio.h>` for I/O functions

Compilation Tips:

```
# Basic compilation
gcc program.c -o program

# With warnings enabled
gcc -Wall -Wextra program.c -o program

# With debugging information
gcc -g program.c -o program

# With C standard specification
gcc -std=c99 program.c -o program
```

Complete these exercises to solidify your understanding of Module 1 concepts. Each exercise builds upon the previous ones, gradually increasing in complexity.

Data Types

Module 2: Data Types and Variables Exercises

Exercise 1: Data Type Explorer

Write a program that displays the size and range of all fundamental data types on your system. Include: - All integer types (char, short, int, long, long long) and their unsigned variants - All floating-point types (float, double, long double) - Fixed-width integer types (int8_t, int16_t, int32_t, int64_t) - Boolean type

Requirements: - Use `sizeof` operator to determine sizes - Use `limits.h`, `float.h`, and `stdint.h` headers for range information - Format output clearly with labels - Include both signed and unsigned variants where applicable

Exercise 2: Variable Declaration Challenge

Create a program that declares variables using different naming conventions and demonstrates proper initialization: - Declare 5 variables using snake_case convention - Declare 5 variables using camelCase convention - Initialize all variables at declaration - Demonstrate the use of const variables - Show examples of multiple variable declarations in single statements

Requirements: - Use at least 3 different data types - Include both global and local variables - Demonstrate scope differences - Use meaningful variable names related to a real-world scenario (e.g., student records, inventory system)

Exercise 3: Constants Comparison

Write a program that demonstrates the three methods of creating constants in C: - Using `#define` preprocessor directive - Using `const` qualifier - Using `enum` enumeration

Create constants for a real-world application (e.g., a calendar system with months, days, or a game with difficulty levels).

Requirements: - Implement all three constant creation methods - Show advantages and disadvantages of each method - Demonstrate usage in calculations or conditional statements - Include string constants where appropriate

Exercise 4: Storage Class Demonstrator

Create a program that demonstrates the behavior of different storage classes: - `auto` (automatic) - `static` (static) - `register` (register) - `extern` (external)

For `extern`, create two source files to demonstrate external linkage.

Requirements: - Show how static variables retain values between function calls - Demonstrate the difference between local and global scope - Show how register variables might be optimized - Implement proper extern usage with header files

Exercise 5: Type Conversion Safety

Write a program that safely handles type conversions between different data types: - Integer to floating-point and vice versa - Large integer to smaller integer (handle overflow) - Signed to unsigned and vice versa - Character to integer conversions

Include error checking and validation for unsafe conversions.

Requirements: - Check for potential overflow before conversion - Handle truncation appropriately - Use standard library functions for string conversion (strtol, strtod) - Provide meaningful error messages for failed conversions

Exercise 6: Operator Expression Builder

Create a program that evaluates complex mathematical expressions using various operators: - Arithmetic operators with mixed data types - Relational and logical operators in compound expressions - Bitwise operators for bit manipulation - Assignment operators for cumulative calculations - Conditional operators for decision making

Requirements: - Include expressions with different precedence levels - Use parentheses to clarify intent - Demonstrate short-circuit evaluation - Show the difference between pre and post increment/decrement

Exercise 7: Bit Manipulation Toolkit

Develop a set of functions that perform common bit manipulation operations: - Function to check if a specific bit is set - Function to set a specific bit - Function to clear a specific bit - Function to toggle a specific bit - Function to count the number of set bits - Function to reverse all bits in a byte

Requirements: - Implement all functions using bitwise operators - Include parameter validation - Provide clear documentation for each function - Test with various input values

Exercise 8: Data Type Selection Advisor

Create a program that helps users select appropriate data types based on their requirements: - Input: Range of values, precision requirements, memory constraints - Output: Recommended data type with justification - Include considerations for portability and performance

Requirements: - Consider both value range and memory usage - Account for signed vs unsigned requirements - Include fixed-width type recommendations - Provide warnings for potential overflow situations

Exercise 9: Comprehensive Calculator

Implement a calculator that demonstrates advanced use of operators and type handling: - Basic arithmetic operations - Bitwise operations - Logical operations - Type-safe conversions - Error handling for invalid operations (division by zero, etc.)

Requirements: - Support both integer and floating-point modes - Handle operator precedence correctly - Include memory functions (store/recall) - Provide clear error messages - Support both interactive and batch modes

Exercise 10: Real-World Application

Design and implement a complete program that integrates all Module 2 concepts: - A student grade management system - An inventory tracking system - A simple banking application - A measurement conversion tool

Requirements: - Use appropriate data types for all values - Implement proper variable scoping - Include constants for fixed values - Handle type conversions safely - Use operators effectively for calculations - Apply storage classes where appropriate - Include comprehensive error checking

Solutions and Tips

Exercise 1 Solution Example:

```c
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>

int main() {
    printf("=== Data Type Explorer ===\n\n");

    // Integer types
    printf("Integer Types:\n");
```

```
    printf("char: %zu bytes, range %d to %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
    printf("unsigned char: %zu bytes, range 0 to %u\n", sizeof(unsigned char), UCHAR_MAX);
    printf("short: %zu bytes, range %d to %d\n", sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("unsigned short: %zu bytes, range 0 to %u\n", sizeof(unsigned short), USHRT_MAX);
    // ... continue for other types

    return 0;
}
```

Exercise 3 Solution Example:

```
#include <stdio.h>

// Using #define
#define MAX_STUDENTS 100
#define UNIVERSITY_NAME "Tech University"

// Using const
const float PI = 3.14159f;
const int PASSING_GRADE = 60;

// Using enum
enum Difficulty {
    EASY = 1,
    MEDIUM = 2,
    HARD = 3
};

enum Days {
    MONDAY = 1,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};

int main() {
    printf("University: %s\n", UNIVERSITY_NAME);
    printf("Max students: %d\n", MAX_STUDENTS);
    printf("PI: %.5f\n", PI);
    printf("Passing grade: %d\n", PASSING_GRADE);
    printf("Difficulty level: %d\n", MEDIUM);
    printf("Day of week: %d\n", WEDNESDAY);
```

```
    return 0;
}
```

Exercise 6 Solution Example:

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;
    float x = 15.7f, y = 4.2f;

    // Mixed arithmetic with precedence
    int result1 = a + b * 2;           // 10 + (3 * 2) = 16
    float result2 = (a + b) * x / y;   // ((10 + 3) * 15.7) / 4.2

    printf("Integer expression: %d + %d * 2 = %d\n", a, b, result1);
    printf("Float expression: (%d + %d) * %.1f / %.1f = %.2f\n", a, b, x, y, result2);

    // Logical expressions with short-circuit
    if (b != 0 && a / b > 1) {
        printf("Division is safe and result > 1\n");
    }

    // Conditional operator
    char *status = (a > b) ? "Greater" : "Less or Equal";
    printf("Status: %s\n", status);

    return 0;
}
```

Exercise 7 Solution Example:

```
#include <stdio.h>

// Check if bit at position is set
int is_bit_set(unsigned int num, int position) {
    return (num >> position) & 1;
}

// Set bit at position
unsigned int set_bit(unsigned int num, int position) {
    return num | (1 << position);
}
```

```c
// Clear bit at position
unsigned int clear_bit(unsigned int num, int position) {
    return num & ~(1 << position);
}

// Toggle bit at position
unsigned int toggle_bit(unsigned int num, int position) {
    return num ^ (1 << position);
}

int main() {
    unsigned int number = 0b10101010;

    printf("Original: %08b\n", number);
    printf("Bit 3 set? %d\n", is_bit_set(number, 3));

    number = set_bit(number, 2);
    printf("After setting bit 2: %08b\n", number);

    number = clear_bit(number, 7);
    printf("After clearing bit 7: %08b\n", number);

    number = toggle_bit(number, 5);
    printf("After toggling bit 5: %08b\n", number);

    return 0;
}
```

Common Pitfalls to Avoid

1. **Uninitialized Variables**: Always initialize variables at declaration
2. **Integer Overflow**: Check ranges before conversions
3. **Division by Zero**: Always validate divisors
4. **Implicit Conversions**: Be aware of precision loss
5. **Operator Precedence**: Use parentheses for clarity
6. **Side Effects in Expressions**: Avoid undefined behavior with ++/– operators
7. **Signed/Unsigned Comparisons**: Be careful when mixing signed and unsigned types
8. **Bitwise vs Logical Operators**: Use &/| for bit operations, &&/|| for logical operations

Compilation Tips

```
# Basic compilation
gcc program.c -o program

# With warnings for type safety
gcc -Wall -Wextra program.c -o program

# With standard specification
gcc -std=c99 program.c -o program

# With debugging information
gcc -g -Wall program.c -o program
```

Complete these exercises to solidify your understanding of Module 2 concepts. Each exercise builds upon the previous ones, gradually increasing in complexity.

Control Flow

Module 3: Control Flow Exercises

Exercise 1: Grade Calculator with Multiple Conditions

Write a program that calculates letter grades based on numerical scores using nested if-else statements. Include: - Input validation for scores (0-100) - Different grading scales for different courses - Bonus points for perfect attendance - Extra credit opportunities

Requirements: - Use nested conditionals for complex grading logic - Implement multiple grading scales (e.g., strict, lenient, standard) - Include bonus point calculations - Provide detailed feedback for each grade range

Exercise 2: Pattern Generator

Create a program that generates various number and character patterns using nested loops: - Right-angled triangles - Pyramids - Diamond patterns - Floyd's triangle - Pascal's triangle (advanced)

Requirements: - Use nested loops for pattern generation - Allow user to select pattern type - Implement input validation for pattern size - Include both number and character patterns

Exercise 3: Menu-Driven Calculator

Develop a comprehensive calculator using do-while loops and switch statements: - Basic arithmetic operations - Scientific functions (sin, cos, tan, log, sqrt) - Memory functions (store, recall, clear) - History of calculations - Unit conversions

Requirements: - Use do-while for main menu loop - Implement switch for operation selection - Include error handling for invalid inputs - Provide clear exit mechanism

Exercise 4: Prime Number Analyzer

Write a program that finds and analyzes prime numbers using various loop constructs: - Prime number detection - Prime factorization - Twin prime identification - Prime number distribution analysis - Sieve of Eratosthenes implementation

Requirements: - Use efficient algorithms for prime detection - Implement multiple loop types (for, while) - Include performance timing - Provide statistical analysis of results

Exercise 5: Resource Manager with goto

Create a resource management system that demonstrates proper use of goto for error handling: - File operations - Memory allocation - Network connections (simulated) - Database connections (simulated) - Cleanup procedures

Requirements: - Use goto for structured error handling - Implement proper resource cleanup - Include error simulation - Demonstrate resource leak prevention

Exercise 6: Function Pointer Calculator

Develop a calculator that uses function pointers for operation selection: - Basic arithmetic operations - Advanced mathematical functions - Custom function registration - Dynamic operation selection - Plugin architecture simulation

Requirements: - Use function pointers for operation dispatch - Implement callback mechanisms - Include function registration system - Provide extensibility examples

Exercise 7: State Machine Implementation

Create a state machine for a real-world system: - Vending machine controller - Traffic light system - Bank account management - Game character states - Network protocol handler

Requirements: - Use enum for state definitions - Implement state transition logic - Include event handling - Provide state visualization

Exercise 8: Error Handling Framework

Design an error handling system that demonstrates various error management techniques: - Return code patterns - errno simulation - Exception-like behavior with setjmp/longjmp - Logging mechanisms - Recovery procedures

Requirements: - Implement multiple error handling approaches - Include error code definitions - Provide error message system - Demonstrate error recovery

Exercise 9: Complex Loop Optimization

Write programs that demonstrate loop optimization techniques: - Loop unrolling - Loop fusion - Loop invariant code motion - Strength reduction - Cache-friendly iterations

Requirements: - Compare performance of optimized vs. unoptimized loops - Include timing measurements - Provide analysis of optimization effects - Demonstrate when optimizations are beneficial

Exercise 10: Comprehensive Control Flow Application

Design and implement a complete application that integrates all control flow concepts: - A simple game (tic-tac-toe, snake, etc.) - A data processing system - A simulation program - A configuration tool - A monitoring application

Requirements: - Use all control flow constructs appropriately - Include complex decision-making logic - Implement proper error handling - Provide user-friendly interface - Include comprehensive documentation

Solutions and Tips

Exercise 1 Solution Example:

```c
#include <stdio.h>

int main() {
    float score;
    int attendance;
    int extra_credit;

    printf("Enter score (0-100): ");
    scanf("%f", &score);

    printf("Enter attendance (0-100%%): ");
    scanf("%d", &attendance);

    printf("Enter extra credit points (0-10): ");
    scanf("%d", &extra_credit);

    // Input validation
    if (score < 0 || score > 100) {
        printf("Invalid score!\n");
        return 1;
    }

    if (attendance < 0 || attendance > 100) {
        printf("Invalid attendance!\n");
        return 1;
    }

    if (extra_credit < 0 || extra_credit > 10) {
        printf("Invalid extra credit!\n");
        return 1;
    }
```

```
    // Apply bonus for perfect attendance
    if (attendance == 100) {
        score += 2.0f;
        if (score > 100) score = 100;
    }

    // Add extra credit
    score += extra_credit;
    if (score > 100) score = 100;

    // Determine grade
    if (score >= 97) {
        printf("Grade: A+ (Excellent)\n");
    } else if (score >= 93) {
        printf("Grade: A (Outstanding)\n");
    } else if (score >= 90) {
        printf("Grade: A- (Very Good)\n");
    } else if (score >= 87) {
        printf("Grade: B+ (Good)\n");
    } else if (score >= 83) {
        printf("Grade: B (Satisfactory)\n");
    } else if (score >= 80) {
        printf("Grade: B- (Above Average)\n");
    } else if (score >= 77) {
        printf("Grade: C+ (Average)\n");
    } else if (score >= 73) {
        printf("Grade: C (Below Average)\n");
    } else if (score >= 70) {
        printf("Grade: C- (Poor)\n");
    } else if (score >= 60) {
        printf("Grade: D (Very Poor)\n");
    } else {
        printf("Grade: F (Fail)\n");
    }

    printf("Final score: %.1f\n", score);

    return 0;
}
```

Exercise 2 Solution Example:

```c
#include <stdio.h>

void print_pyramid(int rows) {
    for (int i = 1; i <= rows; i++) {
        // Print spaces
        for (int j = 1; j <= rows - i; j++) {
            printf(" ");
        }
        // Print stars
        for (int k = 1; k <= 2 * i - 1; k++) {
            printf("*");
        }
        printf("\n");
    }
}

void print_floyds_triangle(int rows) {
    int num = 1;
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", num++);
        }
        printf("\n");
    }
}

int main() {
    int choice, rows;

    printf("Pattern Generator\n");
    printf("1. Pyramid\n");
    printf("2. Floyd's Triangle\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    printf("Enter number of rows: ");
    scanf("%d", &rows);

    if (rows <= 0 || rows > 20) {
        printf("Invalid number of rows!\n");
        return 1;
    }

    switch (choice) {
        case 1:
```

```
            print_pyramid(rows);
            break;
        case 2:
            print_floyds_triangle(rows);
            break;
        default:
            printf("Invalid choice!\n");
    }

    return 0;
}
```

Exercise 3 Solution Example:

```
#include <stdio.h>
#include <math.h>

double memory = 0.0;
double history[100];
int history_count = 0;

void add_to_history(double result) {
    if (history_count < 100) {
        history[history_count++] = result;
    }
}

void show_history() {
    printf("\nCalculation History:\n");
    for (int i = 0; i < history_count && i < 10; i++) {
        printf("%d. %.2f\n", i + 1, history[i]);
    }
}

int main() {
    int choice;
    double num1, num2, result;

    do {
        printf("\n=== Scientific Calculator ===\n");
        printf("1. Addition\n");
        printf("2. Subtraction\n");
        printf("3. Multiplication\n");
        printf("4. Division\n");
        printf("5. Sine\n");
```

```
        printf("6. Cosine\n");
        printf("7. Square Root\n");
        printf("8. Memory Store\n");
        printf("9. Memory Recall\n");
        printf("10. Show History\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter two numbers: ");
                scanf("%lf %lf", &num1, &num2);
                result = num1 + num2;
                printf("Result: %.2f\n", result);
                add_to_history(result);
                break;
            case 2:
                printf("Enter two numbers: ");
                scanf("%lf %lf", &num1, &num2);
                result = num1 - num2;
                printf("Result: %.2f\n", result);
                add_to_history(result);
                break;
            case 3:
                printf("Enter two numbers: ");
                scanf("%lf %lf", &num1, &num2);
                result = num1 * num2;
                printf("Result: %.2f\n", result);
                add_to_history(result);
                break;
            case 4:
                printf("Enter two numbers: ");
                scanf("%lf %lf", &num1, &num2);
                if (num2 != 0) {
                    result = num1 / num2;
                    printf("Result: %.2f\n", result);
                    add_to_history(result);
                } else {
                    printf("Error: Division by zero!\n");
                }
                break;
            case 5:
                printf("Enter angle in radians: ");
                scanf("%lf", &num1);
                result = sin(num1);
```

```
                printf("Result: %.2f\n", result);
                add_to_history(result);
                break;
            case 6:
                printf("Enter angle in radians: ");
                scanf("%lf", &num1);
                result = cos(num1);
                printf("Result: %.2f\n", result);
                add_to_history(result);
                break;
            case 7:
                printf("Enter number: ");
                scanf("%lf", &num1);
                if (num1 >= 0) {
                    result = sqrt(num1);
                    printf("Result: %.2f\n", result);
                    add_to_history(result);
                } else {
                    printf("Error: Cannot calculate square root of negative number!\n");
                }
                break;
            case 8:
                printf("Enter number to store: ");
                scanf("%lf", &memory);
                printf("Stored %.2f in memory\n", memory);
                break;
            case 9:
                printf("Memory: %.2f\n", memory);
                break;
            case 10:
                show_history();
                break;
            case 0:
                printf("Thank you for using the calculator!\n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 0);

    return 0;
}
```

Exercise 5 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *filename;
    FILE *file;
    char *buffer;
    int *data;
} ResourceManager;

int initialize_resources(ResourceManager *rm) {
    // Allocate filename
    rm->filename = malloc(100);
    if (!rm->filename) {
        goto cleanup;
    }
    strcpy(rm->filename, "data.txt");

    // Open file
    rm->file = fopen(rm->filename, "w");
    if (!rm->file) {
        printf("Error opening file\n");
        goto cleanup;
    }

    // Allocate buffer
    rm->buffer = malloc(1024);
    if (!rm->buffer) {
        printf("Error allocating buffer\n");
        goto cleanup;
    }

    // Allocate data
    rm->data = malloc(100 * sizeof(int));
    if (!rm->data) {
        printf("Error allocating data\n");
        goto cleanup;
    }

    printf("All resources initialized successfully\n");
    return 0;  // Success

cleanup:
```

```
    // Cleanup in reverse order
    if (rm->data) {
        free(rm->data);
        rm->data = NULL;
    }
    if (rm->buffer) {
        free(rm->buffer);
        rm->buffer = NULL;
    }
    if (rm->file) {
        fclose(rm->file);
        rm->file = NULL;
    }
    if (rm->filename) {
        free(rm->filename);
        rm->filename = NULL;
    }
    return -1;  // Error
}

void cleanup_resources(ResourceManager *rm) {
    if (rm->data) {
        free(rm->data);
        rm->data = NULL;
    }
    if (rm->buffer) {
        free(rm->buffer);
        rm->buffer = NULL;
    }
    if (rm->file) {
        fclose(rm->file);
        rm->file = NULL;
    }
    if (rm->filename) {
        free(rm->filename);
        rm->filename = NULL;
    }
    printf("All resources cleaned up\n");
}

int main() {
    ResourceManager rm = {0};

    if (initialize_resources(&rm) == 0) {
        printf("Resource management successful\n");
        // Use resources here
        cleanup_resources(&rm);
```

```
    } else {
        printf("Resource management failed\n");
        return 1;
    }

    return 0;
}
```

Common Pitfalls to Avoid

1. **Infinite Loops**: Always ensure loop termination conditions
2. **Off-by-One Errors**: Carefully check loop bounds
3. **goto Overuse**: Use goto only for structured error handling
4. **Deep Nesting**: Use early returns to reduce nesting levels
5. **Missing Break Statements**: Always include break in switch cases
6. **Floating-Point Loop Variables**: Avoid using floats as loop counters
7. **Resource Leaks**: Always clean up allocated resources
8. **Uninitialized Variables**: Initialize all variables before use

Compilation Tips

```
# Basic compilation
gcc program.c -o program

# With math library for scientific functions
gcc program.c -lm -o program

# With warnings enabled
gcc -Wall -Wextra program.c -o program

# With debugging information
gcc -g -Wall program.c -o program

# With optimization
gcc -O2 program.c -o program
```

Complete these exercises to solidify your understanding of Module 3 concepts. Each exercise builds upon the previous ones, gradually increasing in complexity.

Functions

Module 4: Functions and Modular Programming Exercises

Exercise 1: Basic Function Implementation

Write a program that implements several mathematical functions: - A function to calculate the area of different shapes (circle, rectangle, triangle) - A function to convert temperatures between Celsius and Fahrenheit - A function to calculate compound interest - A function to determine if a year is a leap year

Requirements: - Use proper function declarations and definitions - Include appropriate header files - Handle edge cases and invalid inputs - Provide clear function documentation - Use const parameters where appropriate

Exercise 2: Recursive Functions

Create a program that implements various recursive algorithms: - Factorial calculation with error handling - Fibonacci sequence with memoization - Binary search on a sorted array - Tower of Hanoi solution - Greatest Common Divisor (GCD) using Euclidean algorithm

Requirements: - Implement proper base cases to prevent infinite recursion - Handle negative inputs appropriately - Include performance timing for comparison with iterative versions - Document recursion depth limitations - Provide clear output showing the recursive process

Exercise 3: Variable Argument Functions

Develop a program that demonstrates variable argument functions: - A function that calculates the average of a variable number of integers - A function that finds the maximum value among variable arguments - A formatted print function similar to printf - A function that concatenates a variable number of strings

Requirements: - Use stdarg.h for variable argument handling - Implement proper argument counting and type checking - Handle memory allocation for string concatenation - Include error handling for invalid arguments - Provide examples with different numbers of arguments

Exercise 4: Function Pointers and Callbacks

Create a program that uses function pointers for various purposes: - A calculator that uses function pointers for operations - A sorting function that accepts a comparison function pointer - A filter function that applies a predicate to an array - A state machine implementation using function pointers

Requirements: - Implement function pointer arrays for dispatch tables - Create callback mechanisms for event handling - Demonstrate function pointer casting - Include error handling for null function pointers - Provide clear examples of different use cases

Exercise 5: Modular Programming with Multiple Files

Design and implement a complete module system: - Create a statistics library with header and source files - Implement a string processing module - Create a configuration management module - Develop a logging system module - Write a main program that uses all modules

Requirements: - Use proper include guards in header files - Separate interface from implementation - Demonstrate static and external linkage - Include Makefile or build script for compilation - Document module dependencies and usage

Exercise 6: Inline Functions and Macro Functions

Write a program that compares inline functions with macro functions: - Implement mathematical operations as both inline functions and macros - Create performance benchmarks for both approaches - Demonstrate side effects with macros - Show type safety differences between approaches - Implement conditional compilation for different optimization levels

Requirements: - Use appropriate compiler flags for inline function support - Include timing measurements for performance comparison - Demonstrate macro pitfalls with complex expressions - Show how inline functions provide type checking - Document when to use each approach

Exercise 7: Standard Library Function Practice

Create a comprehensive program that uses various standard library functions: - String manipulation with functions from string.h - Mathematical calculations with functions from math.h - Time and date processing with functions from time.h - Memory management with functions from stdlib.h - Character classification with functions from ctype.h

Requirements: - Demonstrate proper error handling for library functions - Include examples of common pitfalls and how to avoid them - Show memory management best practices - Implement proper buffer size checking - Provide clear output showing function results

Exercise 8: Advanced Modular Programming

Design a complete application using advanced modular programming techniques: - Implement an opaque pointer design pattern - Create a plugin system using dynamic loading (if supported) - Develop a configuration system with multiple backends - Implement a thread-safe module with proper synchronization - Create a unit testing framework for your modules

Requirements: - Use advanced features like function pointers for polymorphism - Implement proper error handling and resource cleanup - Include documentation for module interfaces - Demonstrate proper separation of concerns - Provide examples of module extensibility

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <math.h>

#define PI 3.14159265359

// Function to calculate area of circle
double circle_area(double radius) {
    if (radius < 0) {
        printf("Error: Radius cannot be negative\n");
        return -1;
    }
    return PI * radius * radius;
}

// Function to calculate area of rectangle
double rectangle_area(double length, double width) {
    if (length < 0 || width < 0) {
        printf("Error: Dimensions cannot be negative\n");
        return -1;
    }
    return length * width;
}

// Function to calculate area of triangle
double triangle_area(double base, double height) {
    if (base < 0 || height < 0) {
        printf("Error: Dimensions cannot be negative\n");
        return -1;
    }
    return 0.5 * base * height;
}
```

```c
// Function to convert Celsius to Fahrenheit
double celsius_to_fahrenheit(double celsius) {
    return (celsius * 9.0 / 5.0) + 32.0;
}

// Function to convert Fahrenheit to Celsius
double fahrenheit_to_celsius(double fahrenheit) {
    return (fahrenheit - 32.0) * 5.0 / 9.0;
}

// Function to calculate compound interest
double compound_interest(double principal, double rate, int time) {
    if (principal < 0 || rate < 0 || time < 0) {
        printf("Error: Invalid parameters\n");
        return -1;
    }
    return principal * pow(1 + rate / 100.0, time);
}

// Function to check if a year is a leap year
int is_leap_year(int year) {
    if (year % 400 == 0) return 1;
    if (year % 100 == 0) return 0;
    if (year % 4 == 0) return 1;
    return 0;
}

int main() {
    // Test area calculations
    printf("Circle area (radius 5): %.2f\n", circle_area(5));
    printf("Rectangle area (5x3): %.2f\n", rectangle_area(5, 3));
    printf("Triangle area (base 4, height 6): %.2f\n", triangle_area(4, 6));

    // Test temperature conversion
    printf("25°C = %.1f°F\n", celsius_to_fahrenheit(25));
    printf("77°F = %.1f°C\n", fahrenheit_to_celsius(77));

    // Test compound interest
    printf("Compound interest ($1000, 5%%, 10 years): $%.2f\n",
           compound_interest(1000, 5, 10));

    // Test leap year
    printf("2024 is %s a leap year\n", is_leap_year(2024) ? "" : "not");
    printf("1900 is %s a leap year\n", is_leap_year(1900) ? "" : "not");

    return 0;
```

```
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <time.h>

// Simple recursive factorial
long long factorial(int n) {
    if (n < 0) {
        printf("Error: Factorial of negative number\n");
        return -1;
    }
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}

// Fibonacci with memoization
long long fib_memo[100] = {0};

long long fibonacci_memo(int n) {
    if (n < 0) {
        printf("Error: Invalid Fibonacci index\n");
        return -1;
    }
    if (n <= 1) return n;

    if (fib_memo[n] != 0) return fib_memo[n];

    fib_memo[n] = fibonacci_memo(n - 1) + fibonacci_memo(n - 2);
    return fib_memo[n];
}

// Binary search recursive implementation
int binary_search_recursive(int arr[], int left, int right, int target) {
    if (left > right) return -1;  // Not found

    int mid = left + (right - left) / 2;

    if (arr[mid] == target) return mid;
    if (arr[mid] > target) {
        return binary_search_recursive(arr, left, mid - 1, target);
    } else {
        return binary_search_recursive(arr, mid + 1, right, target);
    }
```

```c
}

// Tower of Hanoi
void tower_of_hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", from, to);
        return;
    }

    tower_of_hanoi(n - 1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    tower_of_hanoi(n - 1, aux, to, from);
}

// GCD using Euclidean algorithm
int gcd_recursive(int a, int b) {
    if (b == 0) return a;
    return gcd_recursive(b, a % b);
}

int main() {
    // Test factorial
    printf("Factorial of 5: %lld\n", factorial(5));

    // Test Fibonacci with memoization
    for (int i = 0; i < 10; i++) {
        printf("Fibonacci(%d) = %lld\n", i, fibonacci_memo(i));
    }

    // Test binary search
    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15};
    int target = 7;
    int index = binary_search_recursive(arr, 0, 7, target);
    if (index != -1) {
        printf("Found %d at index %d\n", target, index);
    } else {
        printf("%d not found\n", target);
    }

    // Test Tower of Hanoi
    printf("\nTower of Hanoi (3 disks):\n");
    tower_of_hanoi(3, 'A', 'C', 'B');

    // Test GCD
    printf("\nGCD of 48 and 18: %d\n", gcd_recursive(48, 18));

    return 0;
```

```
}
```

Exercise 3 Solution Example:

```
#include <stdio.h>
#include <stdarg.h>
#include <string.h>
#include <stdlib.h>

// Function to calculate average of variable integers
double average_ints(int count, ...) {
    if (count <= 0) {
        printf("Error: Invalid count\n");
        return 0;
    }

    va_list args;
    va_start(args, count);

    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += va_arg(args, int);
    }

    va_end(args);
    return (double)sum / count;
}

// Function to find maximum among variable arguments
int max_ints(int count, ...) {
    if (count <= 0) {
        printf("Error: Invalid count\n");
        return 0;
    }

    va_list args;
    va_start(args, count);

    int max = va_arg(args, int);
    for (int i = 1; i < count; i++) {
        int current = va_arg(args, int);
        if (current > max) max = current;
    }

    va_end(args);
```

```c
    return max;
}

// Function to concatenate variable strings
char* concat_strings(int count, ...) {
    if (count <= 0) {
        printf("Error: Invalid count\n");
        return NULL;
    }

    // First pass: calculate total length
    va_list args;
    va_start(args, count);

    int total_length = 0;
    for (int i = 0; i < count; i++) {
        const char* str = va_arg(args, const char*);
        if (str) total_length += strlen(str);
    }

    va_end(args);

    // Allocate memory for result
    char* result = (char*)malloc(total_length + 1);
    if (!result) {
        printf("Error: Memory allocation failed\n");
        return NULL;
    }

    // Second pass: concatenate strings
    result[0] = '\0';
    va_start(args, count);

    for (int i = 0; i < count; i++) {
        const char* str = va_arg(args, const char*);
        if (str) strcat(result, str);
    }

    va_end(args);
    return result;
}

int main() {
    // Test average function
    printf("Average of 1, 2, 3, 4, 5: %.2f\n",
           average_ints(5, 1, 2, 3, 4, 5));
```

```
    // Test maximum function
    printf("Maximum of 10, 25, 7, 33, 15: %d\n",
           max_ints(5, 10, 25, 7, 33, 15));

    // Test string concatenation
    char* result = concat_strings(4, "Hello", " ", "World", "!");
    if (result) {
        printf("Concatenated string: %s\n", result);
        free(result);
    }

    return 0;
}
```

Exercise 4 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>

// Calculator operations
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

// Calculator using function pointers
int calculate(int a, int b, int (*operation)(int, int)) {
    if (operation == NULL) {
        printf("Error: Invalid operation\n");
        return 0;
    }
    return operation(a, b);
}

// Comparison functions for sorting
int compare_ascending(const void *a, const void *b) {
    int int_a = *(const int*)a;
    int int_b = *(const int*)b;
    return (int_a > int_b) - (int_a < int_b);
}

int compare_descending(const void *a, const void *b) {
    int int_a = *(const int*)a;
    int int_b = *(const int*)b;
    return (int_b > int_a) - (int_b < int_a);
```

```c
}

// Generic sorting function with comparison function pointer
void sort_array(int *arr, int size, int (*compare)(const void*, const void*)) {
    if (arr == NULL || compare == NULL) {
        printf("Error: Invalid parameters\n");
        return;
    }
    qsort(arr, size, sizeof(int), compare);
}

// Filter function with predicate
int* filter_array(int *arr, int size, int (*predicate)(int), int *result_size) {
    if (arr == NULL || predicate == NULL || result_size == NULL) {
        printf("Error: Invalid parameters\n");
        return NULL;
    }

    int *result = (int*)malloc(size * sizeof(int));
    if (!result) {
        printf("Error: Memory allocation failed\n");
        return NULL;
    }

    *result_size = 0;
    for (int i = 0; i < size; i++) {
        if (predicate(arr[i])) {
            result[*result_size] = arr[i];
            (*result_size)++;
        }
    }

    return result;
}

// Predicate functions
int is_even(int n) { return n % 2 == 0; }
int is_positive(int n) { return n > 0; }
int is_greater_than_10(int n) { return n > 10; }

int main() {
    // Test calculator with function pointers
    printf("Calculator using function pointers:\n");
    printf("10 + 5 = %d\n", calculate(10, 5, add));
    printf("10 - 5 = %d\n", calculate(10, 5, subtract));
    printf("10 * 5 = %d\n", calculate(10, 5, multiply));
    printf("10 / 5 = %d\n", calculate(10, 5, divide));
```

```c
    // Test sorting with function pointers
    int arr[] = {5, 2, 8, 1, 9, 3};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("\nOriginal array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    sort_array(arr, size, compare_ascending);
    printf("\nSorted ascending: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    sort_array(arr, size, compare_descending);
    printf("\nSorted descending: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    // Test filter function
    int numbers[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
    int num_size = sizeof(numbers) / sizeof(numbers[0]);
    int filtered_size;

    int *evens = filter_array(numbers, num_size, is_even, &filtered_size);
    if (evens) {
        printf("\n\nEven numbers: ");
        for (int i = 0; i < filtered_size; i++) {
            printf("%d ", evens[i]);
        }
        free(evens);
    }

    int *greater_than_10 = filter_array(numbers, num_size, is_greater_than_10, &filtered_siz
    if (greater_than_10) {
        printf("\nNumbers > 10: ");
        for (int i = 0; i < filtered_size; i++) {
            printf("%d ", greater_than_10[i]);
        }
        free(greater_than_10);
    }

    printf("\n");
    return 0;
```

```
}
```

Common Pitfalls to Avoid

1. **Infinite Recursion**: Always ensure proper base cases
2. **Memory Leaks**: Free dynamically allocated memory
3. **Null Pointer Dereference**: Always check function pointer validity
4. **Buffer Overflows**: Ensure adequate buffer sizes for string operations
5. **Variable Argument Misuse**: Properly count and type-check variable arguments
6. **Linkage Issues**: Understand static vs. external linkage
7. **Include Guard Problems**: Use proper include guards to prevent multiple inclusions
8. **Macro Side Effects**: Be careful with complex expressions in macros
9. **Integer Overflow**: Check for overflow in mathematical operations
10. **Resource Cleanup**: Always clean up resources in error paths

Compilation Tips

```
# Basic compilation with multiple source files
gcc main.c math_utils.c string_utils.c -o program

# With math library for mathematical functions
gcc main.c math_utils.c -lm -o program

# With warnings enabled
gcc -Wall -Wextra main.c math_utils.c -o program

# With debugging information
gcc -g -Wall main.c math_utils.c -o program

# With optimization
gcc -O2 main.c math_utils.c -o program

# Creating object files separately
gcc -c math_utils.c -o math_utils.o
gcc -c string_utils.c -o string_utils.o
gcc main.c math_utils.o string_utils.o -o program
```

Complete these exercises to solidify your understanding of Module 4 concepts. Each exercise builds upon the previous ones, gradually increasing in complexity.

Arrays and Strings

Module 5: Arrays and Strings Exercises

Exercise 1: Array Manipulation Functions

Write a program that implements various array manipulation functions: - A function to find the maximum and minimum values in an array - A function to calculate the average of array elements - A function to reverse an array in place - A function to check if an array is sorted (ascending or descending) - A function to remove duplicates from an array

Requirements: - Use proper function declarations with array parameters - Handle edge cases (empty arrays, single element arrays) - Include appropriate header files - Provide clear function documentation - Test with different array sizes and data types

Exercise 2: Matrix Operations

Create a program that performs various matrix operations: - Matrix addition and subtraction - Matrix multiplication - Transpose of a matrix - Determinant calculation for 2x2 and 3x3 matrices - Check if a matrix is symmetric

Requirements: - Use 2D arrays for matrix representation - Implement proper bounds checking - Handle memory allocation for dynamic matrices (bonus) - Include error handling for incompatible matrix operations - Provide clear output formatting for matrices

Exercise 3: String Processing Library

Develop a comprehensive string processing library: - Function to count words in a string - Function to count vowels and consonants - Function to reverse words in a string (maintain word order) - Function to remove extra whitespace - Function to check if a string is a palindrome

Requirements: - Use proper string handling functions from string.h - Implement your own versions of common string functions - Handle edge cases (empty strings, NULL pointers) - Include memory safety considerations - Provide examples with various string inputs

Exercise 4: Dynamic Array Implementation

Create a dynamic array (vector-like) implementation: - Function to initialize a dynamic array - Function to add elements (with automatic resizing) - Function to remove elements by index - Function to search for elements - Function to free the dynamic array

Requirements: - Use malloc, realloc, and free for memory management - Implement proper error handling for memory allocation - Include capacity and size tracking - Handle memory leaks properly - Provide a complete API with clear documentation

Exercise 5: Multi-dimensional Array Challenges

Solve complex problems using multi-dimensional arrays: - Implement a simple image processing function (2D array of pixels) - Create a tic-tac-toe game board with win detection - Implement a simple maze solver using backtracking - Create a program to process student grades in a 2D array - Implement Conway's Game of Life

Requirements: - Use appropriate data structures for each problem - Include proper initialization and cleanup - Handle boundary conditions correctly - Provide clear visualization of results where applicable - Include performance considerations for large arrays

Exercise 6: String Search and Replace

Write a program that implements advanced string operations: - Function to find all occurrences of a substring - Function to replace all occurrences of a substring - Function to tokenize a string based on multiple delimiters - Function to implement a simple pattern matching algorithm - Function to perform case-insensitive string operations

Requirements: - Handle overlapping matches correctly - Manage memory allocation for result strings - Include error checking for invalid inputs - Optimize for performance where possible - Provide comprehensive test cases

Exercise 7: Command Line Argument Processing

Create a program that processes command line arguments effectively: - Parse command line options and flags - Handle positional arguments - Implement a help system with usage information - Validate argument types and ranges - Provide meaningful error messages for invalid input

Requirements: - Use argc and argv properly - Implement both short (-h) and long (–help) options - Handle combined flags (-abc equivalent to -a -b -c) - Include default values for optional arguments - Provide examples of usage

Exercise 8: Advanced Array and String Integration

Design a complete application that integrates arrays and strings: - Implement a simple database system using arrays of structures - Create a text analysis tool that processes multiple files - Develop a word frequency counter with sorting capabilities - Implement a simple encryption/decryption system - Create a program that generates reports from structured data

Requirements: - Use modular design with separate functions for each major task - Include proper error handling and recovery - Handle file I/O for data persistence - Implement sorting and searching algorithms - Provide clear user interface and output formatting

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <limits.h>

// Function to find maximum value in array
int find_max(int arr[], int size) {
    if (size <= 0) return INT_MIN;

    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

// Function to find minimum value in array
int find_min(int arr[], int size) {
    if (size <= 0) return INT_MAX;

    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}

// Function to calculate average
```

```c
double calculate_average(int arr[], int size) {
    if (size <= 0) return 0.0;

    long long sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return (double)sum / size;
}

int main() {
    int numbers[] = {5, 2, 8, 1, 9, 3};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    printf("Maximum: %d\n", find_max(numbers, size));
    printf("Minimum: %d\n", find_min(numbers, size));
    printf("Average: %.2f\n", calculate_average(numbers, size));

    return 0;
}
```

Exercise 3 Solution Example:

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function to count words in a string
int count_words(const char *str) {
    if (str == NULL) return 0;

    int count = 0;
    int in_word = 0;

    for (int i = 0; str[i] != '\0'; i++) {
        if (isspace(str[i])) {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
```

```
            count++;
        }
    }
    return count;
}

// Function to check if string is palindrome
int is_palindrome(const char *str) {
    if (str == NULL) return 0;

    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        if (tolower(str[i]) != tolower(str[len - 1 - i])) {
            return 0;
        }
    }
    return 1;
}

int main() {
    char text1[] = "Hello world, this is a test string";
    char text2[] = "A man a plan a canal Panama";

    printf("Text 1: \"%s\"\n", text1);
    printf("Word count: %d\n", count_words(text1));

    printf("\nText 2: \"%s\"\n", text2);
    printf("Is palindrome: %s\n", is_palindrome(text2) ? "Yes" : "No");

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Array bounds**: Always check array indices before accessing elements
2. **String termination**: Ensure strings are properly null-terminated
3. **Memory allocation**: Check return values from malloc/realloc
4. **Pointer validation**: Always check for NULL pointers
5. **Buffer overflow**: Limit string input sizes to prevent overflow

Performance Tips:

1. **Pass by reference**: Use pointers for large arrays to avoid copying
2. **Avoid repeated strlen**: Store string length in a variable when used multiple times
3. **Use const**: Mark string parameters as const when not modifying them
4. **Pre-allocate**: When size is known, pre-allocate arrays to avoid repeated resizing

5. **Cache results**: For expensive operations, cache results when possible

Complete these exercises to solidify your understanding of arrays and strings in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Pointers

Module 6: Pointers Exercises

Exercise 1: Basic Pointer Operations

Write a program that demonstrates fundamental pointer operations: - Declare and initialize pointers to different data types - Perform pointer arithmetic (increment, decrement, addition, subtraction) - Dereference pointers to modify values - Swap two variables using pointers - Calculate the distance between two pointers

Requirements: - Use proper pointer declarations and initialization - Demonstrate pointer arithmetic with different data types - Include bounds checking to prevent invalid memory access - Provide clear output showing address and value changes - Handle NULL pointer checks appropriately

Exercise 2: Pointers and Arrays

Create a program that explores the relationship between pointers and arrays: - Access array elements using pointer notation - Pass arrays to functions using pointer parameters - Implement array traversal using pointers - Demonstrate pointer arithmetic as array indexing - Compare performance of array indexing vs pointer arithmetic

Requirements: - Implement functions that accept array parameters as pointers - Show equivalent operations using array and pointer notation - Include examples with multi-dimensional arrays - Handle array bounds properly - Provide timing comparisons where relevant

Exercise 3: Dynamic Memory Allocation

Develop a program that demonstrates dynamic memory management: - Allocate memory for single variables using malloc - Allocate memory for arrays using malloc and calloc - Resize allocated memory using realloc - Free allocated memory properly - Handle memory allocation failures gracefully

Requirements: - Check return values from all memory allocation functions - Initialize allocated memory appropriately - Avoid memory leaks by freeing all allocated memory - Handle reallocation failures properly - Include memory usage tracking (bonus)

Exercise 4: Pointer to Pointers

Write a program that works with pointers to pointers: - Declare and initialize pointers to pointers - Modify values through multiple levels of indirection - Implement a function that allocates memory and returns it through a pointer parameter - Create a dynamic 2D array using pointers to pointers - Demonstrate proper cleanup of multi-level allocations

Requirements: - Handle multiple levels of pointer dereferencing correctly - Include proper error checking at each level - Free memory in the correct order to prevent leaks - Document the relationship between levels clearly - Provide examples of practical use cases

Exercise 5: Function Pointers

Create a program that demonstrates function pointer usage: - Declare and initialize function pointers - Call functions through function pointers - Implement a simple calculator using function pointers - Create an array of function pointers for dispatch - Pass function pointers as parameters to other functions

Requirements: - Use proper function pointer syntax and declarations - Include error checking for NULL function pointers - Demonstrate different ways to assign functions to pointers - Implement callback mechanisms using function pointers - Provide clear examples of practical applications

Exercise 6: Advanced Pointer Techniques

Write a program that implements advanced pointer concepts: - Implement a generic swap function using void pointers - Create a flexible data structure using void pointers - Demonstrate pointer casting between different types - Implement a simple memory pool allocator - Create a basic garbage collection simulation (bonus)

Requirements: - Handle type safety when working with void pointers - Include proper casting and alignment considerations - Implement error checking for invalid operations - Document memory management strategies clearly - Provide examples of real-world applications

Exercise 7: Pointer Debugging and Safety

Create a program that demonstrates safe pointer practices: - Implement a memory debugging system with allocation tracking - Create functions to detect common pointer errors - Demonstrate proper error handling for pointer operations - Implement a simple smart pointer-like mechanism - Show techniques for preventing dangling pointers

Requirements: - Include comprehensive error checking - Handle edge cases and invalid operations gracefully - Provide meaningful error messages - Implement cleanup procedures for all resources - Document safety practices and best approaches

Exercise 8: Comprehensive Pointer Application

Design a complete application that integrates all pointer concepts: - Implement a dynamic data structure (linked list, tree, etc.) - Create a memory management system with custom allocators - Develop a plugin system using function pointers - Implement a callback-based event system - Include comprehensive error handling and resource management

Requirements: - Use modular design with separate functions for each major component - Include proper documentation for all pointer-related functions - Handle all memory allocation and deallocation properly - Implement robust error handling throughout - Provide clear examples and test cases

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>

// Function to swap two integers using pointers
void swap_int(int *a, int *b) {
    if (a == NULL || b == NULL) {
        printf("Error: NULL pointer passed to swap_int\n");
        return;
    }

    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    int *ptr_x = &x;
    int *ptr_y = &y;

    printf("Before swap: x = %d, y = %d\n", x, y);
    printf("Addresses: &x = %p, &y = %p\n", (void*)&x, (void*)&y);
    printf("Pointer values: ptr_x = %p, ptr_y = %p\n", (void*)ptr_x, (void*)ptr_y);

    swap_int(ptr_x, ptr_y);

    printf("After swap: x = %d, y = %d\n", x, y);

    // Pointer arithmetic demonstration
    int arr[] = {1, 2, 3, 4, 5};
```

```
    int *ptr = arr;

    printf("\nArray elements using pointer arithmetic:\n");
    for (int i = 0; i < 5; i++) {
        printf("Element %d: value = %d, address = %p\n", i, *ptr, (void*)ptr);
        ptr++;
    }

    return 0;
}
```

Exercise 3 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>

// Function to safely allocate memory
int* safe_malloc_int(int count) {
    if (count <= 0) {
        printf("Error: Invalid count %d\n", count);
        return NULL;
    }

    int *ptr = malloc(count * sizeof(int));
    if (ptr == NULL) {
        printf("Error: Memory allocation failed\n");
        return NULL;
    }

    // Initialize allocated memory
    for (int i = 0; i < count; i++) {
        ptr[i] = 0;
    }

    return ptr;
}

int main() {
    // Allocate memory for 5 integers
    int *dynamic_array = safe_malloc_int(5);
    if (dynamic_array == NULL) {
        return 1;
    }

    // Use the allocated memory
```

```
    for (int i = 0; i < 5; i++) {
        dynamic_array[i] = (i + 1) * 10;
    }

    printf("Dynamic array contents: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", dynamic_array[i]);
    }
    printf("\n");

    // Resize the array
    int *resized_array = realloc(dynamic_array, 8 * sizeof(int));
    if (resized_array == NULL) {
        printf("Error: Memory reallocation failed\n");
        free(dynamic_array);
        return 1;
    }

    dynamic_array = resized_array;

    // Initialize new elements
    for (int i = 5; i < 8; i++) {
        dynamic_array[i] = (i + 1) * 10;
    }

    printf("Resized array contents: ");
    for (int i = 0; i < 8; i++) {
        printf("%d ", dynamic_array[i]);
    }
    printf("\n");

    // Free allocated memory
    free(dynamic_array);
    dynamic_array = NULL; // Prevent dangling pointer

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Uninitialized pointers**: Always initialize pointers to NULL or valid addresses
2. **Dangling pointers**: Set pointers to NULL after freeing memory
3. **Memory leaks**: Free all allocated memory exactly once
4. **Buffer overruns**: Check bounds before pointer arithmetic
5. **Type mismatches**: Use proper casting when changing pointer types

Best Practices:

1. **NULL checking**: Always check pointers before dereferencing
2. **Consistent naming**: Use clear naming conventions for pointer variables
3. **Documentation**: Comment pointer usage and ownership clearly
4. **Error handling**: Handle allocation failures gracefully
5. **Resource management**: Follow RAII-like patterns for cleanup

Complete these exercises to solidify your understanding of pointers in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Structures

Module 7: Structures, Unions, and Enums Exercises

Exercise 1: Basic Structure Implementation

Write a program that defines and uses various structures: - Define a structure to represent a date (day, month, year) - Define a structure to represent a person (name, age, date of birth) - Define a structure to represent a book (title, author, ISBN, price) - Create functions to initialize and print these structures - Demonstrate structure assignment and comparison

Requirements: - Use proper structure definitions with appropriate data types - Implement initialization functions for each structure - Create print functions that display structure contents - Handle string copying safely to prevent buffer overflows - Include examples of nested structures

Exercise 2: Array of Structures

Create a program that manages collections using arrays of structures: - Implement a student database using an array of student structures - Create functions to add, search, and remove students - Implement sorting functions for the student array - Add functionality to calculate class averages and statistics - Include file I/O to save and load student data

Requirements: - Use dynamic memory allocation for flexible array sizing - Implement proper error handling for array bounds - Include search functions with different criteria - Handle memory management properly to prevent leaks - Provide clear user interface for database operations

Exercise 3: Linked List Implementation

Develop a program that implements a complete linked list: - Define a node structure for the linked list - Implement functions to insert, delete, and search nodes - Create functions to traverse and display the list - Implement sorting and merging of linked lists - Include memory management for all list operations

Requirements: - Handle edge cases (empty list, single node, etc.) - Implement proper memory allocation and deallocation - Include error checking for all operations - Provide both iterative and recursive implementations (bonus) - Document time and space complexity of operations

Exercise 4: Unions and Bit Fields

Write a program that demonstrates advanced data types: - Define and use unions for memory-efficient data storage - Implement structures with bit fields for flags and settings - Create a program that uses unions for type-generic operations - Demonstrate memory layout differences between structures and unions - Implement a simple variant type using unions

Requirements: - Include proper documentation for union usage - Handle type safety when working with unions - Demonstrate bit field packing and alignment - Include examples of practical union applications - Provide clear explanations of memory usage differences

Exercise 5: Enumerations and Type Definitions

Create a program that uses enumerations effectively: - Define enumerations for different categories (status, types, etc.) - Use typedef to create aliases for complex structure types - Implement state machines using enumerations - Create lookup tables using enumerations as indices - Demonstrate enum-to-string conversion functions

Requirements: - Use meaningful names for enumeration constants - Include bounds checking for enumeration values - Implement proper error handling for invalid enum values - Provide examples of enum-based switch statements - Document the advantages of using enumerations

Exercise 6: Advanced Structure Techniques

Write a program that implements advanced structure concepts: - Create structures with flexible array members (C99) - Implement opaque pointer design pattern - Demonstrate structure packing and alignment - Create self-referential structures for trees and graphs - Implement structure serialization and deserialization

Requirements: - Handle flexible array members correctly with dynamic allocation - Include proper memory management for opaque structures - Demonstrate compiler-specific packing directives - Implement proper traversal algorithms for tree structures - Include error checking for serialization operations

Exercise 7: Structure Memory Management

Create a program that demonstrates safe structure handling: - Implement a memory pool for structure allocation - Create functions to track structure usage and leaks - Demonstrate proper deep copying of structures - Implement reference counting for shared structures - Include debugging features for structure validation

Requirements: - Include comprehensive error checking - Handle all memory allocation failures gracefully - Provide clear documentation for memory management functions - Implement proper cleanup procedures - Include performance measurements for different approaches

Exercise 8: Comprehensive Structure Application

Design a complete application that integrates all structure concepts: - Implement a complete data management system - Create a configuration system using nested structures - Develop a plugin architecture using function pointers in structures - Implement a serialization system for data persistence - Include comprehensive error handling and resource management

Requirements: - Use modular design with clear separation of concerns - Include proper documentation for all components - Handle all memory allocation and deallocation properly - Implement robust error handling throughout - Provide clear examples and test cases

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <string.h>

// Structure definitions
typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef struct {
    char name[50];
    int age;
    Date birth_date;
} Person;

typedef struct {
    char title[100];
    char author[50];
    char isbn[20];
    double price;
} Book;

// Function to initialize a date
Date create_date(int day, int month, int year) {
    Date d = {day, month, year};
    return d;
}

// Function to print a date
```

```c
void print_date(const Date *d) {
    if (d == NULL) return;
    printf("%02d/%02d/%04d", d->day, d->month, d->year);
}

// Function to initialize a person
Person create_person(const char *name, int age, Date birth_date) {
    Person p;
    strncpy(p.name, name, sizeof(p.name) - 1);
    p.name[sizeof(p.name) - 1] = '\0';
    p.age = age;
    p.birth_date = birth_date;
    return p;
}

// Function to print a person
void print_person(const Person *p) {
    if (p == NULL) return;
    printf("Name: %s\n", p->name);
    printf("Age: %d\n", p->age);
    printf("Birth Date: ");
    print_date(&p->birth_date);
    printf("\n");
}

int main() {
    Date birth = create_date(15, 6, 1990);
    Person person = create_person("John Doe", 33, birth);

    printf("Person Information:\n");
    print_person(&person);

    return 0;
}
```

Exercise 3 Solution Example:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Node structure for linked list
typedef struct Node {
    int data;
    struct Node *next;
```

```
} Node;

// Function to create a new node
Node* create_node(int data) {
    Node *new_node = malloc(sizeof(Node));
    if (new_node == NULL) {
        printf("Error: Memory allocation failed\n");
        return NULL;
    }

    new_node->data = data;
    new_node->next = NULL;
    return new_node;
}

// Function to insert at beginning of list
void insert_at_beginning(Node **head, int data) {
    if (head == NULL) return;

    Node *new_node = create_node(data);
    if (new_node == NULL) return;

    new_node->next = *head;
    *head = new_node;
}

// Function to print the list
void print_list(Node *head) {
    printf("List: ");
    Node *current = head;

    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

// Function to free the entire list
void free_list(Node *head) {
    Node *current = head;
    Node *next;

    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
```

```
    }
}

int main() {
    Node *head = NULL;

    // Insert elements
    insert_at_beginning(&head, 10);
    insert_at_beginning(&head, 20);
    insert_at_beginning(&head, 30);

    printf("Linked list after insertions:\n");
    print_list(head);

    // Free memory
    free_list(head);

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Shallow copying**: Always implement deep copy for structures with pointers
2. **Memory leaks**: Free all allocated memory in structures properly
3. **Buffer overflows**: Check string lengths when copying to fixed-size arrays
4. **Uninitialized structures**: Initialize all structure members properly
5. **Flexible array misuse**: Handle flexible array members with dynamic allocation

Best Practices:

1. **Consistent naming**: Use clear, consistent names for structures and members
2. **Documentation**: Comment structure purposes and member meanings
3. **Encapsulation**: Hide implementation details using opaque pointers when appropriate
4. **Error handling**: Check all memory allocations and pointer operations
5. **Memory management**: Follow consistent patterns for allocation and deallocation

Complete these exercises to solidify your understanding of structures, unions, and enums in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

File I/O

Module 8: File I/O Exercises

Exercise 1: Basic File Operations

Write a program that performs fundamental file operations: - Create and write text to a file - Read and display the contents of a file - Append new content to an existing file - Copy one file to another - Check if a file exists and get its properties

Requirements: - Use proper file handling functions (fopen, fclose, etc.) - Include error checking for all file operations - Handle different file modes appropriately - Implement proper resource cleanup - Provide clear feedback for operation results

Exercise 2: Text File Processing

Create a program that processes text files in various ways: - Count lines, words, and characters in a text file - Search for specific patterns or words in a file - Replace occurrences of a word with another word - Extract specific lines or sections from a file - Format and restructure text data

Requirements: - Use appropriate text file I/O functions - Handle large files efficiently - Include proper buffer management - Implement case-sensitive and case-insensitive searches - Provide statistics and progress feedback

Exercise 3: Binary File Operations

Develop a program that works with binary files: - Create and read binary data files - Implement a simple database using binary files - Serialize and deserialize structures to/from binary files - Handle endianness when working with binary data - Include data validation and integrity checking

Requirements: - Use binary file modes appropriately - Handle structure padding and alignment issues - Implement proper error checking for binary operations - Include byte order conversion functions if needed - Provide examples of practical binary file applications

Exercise 4: File System Operations

Write a program that performs file system level operations: - List files and directories in a given path - Create, rename, and delete files and directories - Check file permissions and attributes - Implement a simple file browser - Handle file system errors gracefully

Requirements: - Use platform-appropriate file system functions - Include proper error handling for system calls - Handle cross-platform compatibility issues - Implement recursive directory operations - Provide clear user interface for file system operations

Exercise 5: Random Access Files

Create a program that demonstrates random access file operations: - Implement direct record access in a data file - Create an index for fast data retrieval - Update specific records without reading entire file - Handle file positioning with fseek and ftell - Implement a simple database with random access

Requirements: - Use fseek, ftell, and rewind appropriately - Handle fixed and variable record lengths - Include proper data serialization for records - Implement error checking for positioning operations - Provide efficient access patterns for large datasets

Exercise 6: Advanced File I/O Techniques

Write a program that implements advanced file handling concepts: - Use file buffering and flushing strategies - Implement non-blocking I/O operations - Create temporary files and handle automatic cleanup - Work with file locking for concurrent access - Implement file compression and decompression (bonus)

Requirements: - Include proper buffer management and flushing - Handle concurrent access scenarios safely - Implement proper cleanup for temporary files - Include error recovery mechanisms - Document performance considerations for different approaches

Exercise 7: Error Handling and Recovery

Create a program that demonstrates robust file error handling: - Implement comprehensive error checking for all file operations - Create recovery mechanisms for interrupted operations - Handle disk full and other system-level errors - Implement logging for file operations - Include graceful degradation for non-critical failures

Requirements: - Check return values from all file functions - Provide meaningful error messages for different failure scenarios - Implement retry mechanisms for transient errors - Include proper cleanup in error paths - Document error handling strategies clearly

Exercise 8: Comprehensive File Management System

Design a complete application that integrates all file I/O concepts: - Implement a full-featured file manager with GUI or CLI interface - Create backup and restore functionality - Implement file synchronization between directories - Include search and filtering capabilities - Provide comprehensive error handling and logging

Requirements: - Use modular design with clear separation of concerns - Include proper documentation for all components - Handle all file operations safely and efficiently - Implement robust error handling throughout - Provide clear examples and test cases

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Function to write text to a file
int write_text_file(const char *filename, const char *text) {
    FILE *file = fopen(filename, "w");
    if (file == NULL) {
        printf("Error: Could not open file %s for writing\n", filename);
        return -1;
    }

    if (fputs(text, file) == EOF) {
        printf("Error: Failed to write to file %s\n", filename);
        fclose(file);
        return -1;
    }

    fclose(file);
    return 0;
}

// Function to read and display file contents
int read_text_file(const char *filename) {
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        printf("Error: Could not open file %s for reading\n", filename);
        return -1;
    }

    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), file) != NULL) {
        printf("%s", buffer);
    }

    if (ferror(file)) {
        printf("Error: Failed to read from file %s\n", filename);
```

```
        fclose(file);
        return -1;
    }

    fclose(file);
    return 0;
}

// Function to append text to a file
int append_text_file(const char *filename, const char *text) {
    FILE *file = fopen(filename, "a");
    if (file == NULL) {
        printf("Error: Could not open file %s for appending\n", filename);
        return -1;
    }

    if (fputs(text, file) == EOF) {
        printf("Error: Failed to append to file %s\n", filename);
        fclose(file);
        return -1;
    }

    fclose(file);
    return 0;
}

int main() {
    const char *filename = "test.txt";
    const char *initial_text = "This is the initial content.\nLine 2\nLine 3\n";
    const char *append_text = "This line was appended.\n";

    // Write initial content
    if (write_text_file(filename, initial_text) != 0) {
        return 1;
    }
    printf("Wrote initial content to %s\n", filename);

    // Read and display content
    printf("\nFile contents:\n");
    if (read_text_file(filename) != 0) {
        return 1;
    }

    // Append additional content
    if (append_text_file(filename, append_text) != 0) {
        return 1;
    }
```

```
    printf("\nAppended content to %s\n", filename);

    // Read and display updated content
    printf("\nUpdated file contents:\n");
    if (read_text_file(filename) != 0) {
        return 1;
    }

    return 0;
}
```

Exercise 3 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for binary data
typedef struct {
    int id;
    char name[32];
    double value;
} Record;

// Function to write records to binary file
int write_records(const char *filename, const Record *records, int count) {
    FILE *file = fopen(filename, "wb");
    if (file == NULL) {
        printf("Error: Could not open file %s for writing\n", filename);
        return -1;
    }

    if (fwrite(records, sizeof(Record), count, file) != (size_t)count) {
        printf("Error: Failed to write all records to file %s\n", filename);
        fclose(file);
        return -1;
    }

    fclose(file);
    return 0;
}

// Function to read records from binary file
int read_records(const char *filename, Record *records, int max_count) {
    FILE *file = fopen(filename, "rb");
```

```
    if (file == NULL) {
        printf("Error: Could not open file %s for reading\n", filename);
        return -1;
    }

    size_t records_read = fread(records, sizeof(Record), max_count, file);
    if (ferror(file)) {
        printf("Error: Failed to read records from file %s\n", filename);
        fclose(file);
        return -1;
    }

    fclose(file);
    return (int)records_read;
}

int main() {
    // Create sample records
    Record records[] = {
        {1, "Record One", 100.50},
        {2, "Record Two", 200.75},
        {3, "Record Three", 300.25}
    };
    int record_count = sizeof(records) / sizeof(records[0]);

    const char *filename = "records.dat";

    // Write records to binary file
    if (write_records(filename, records, record_count) != 0) {
        return 1;
    }
    printf("Wrote %d records to %s\n", record_count, filename);

    // Read records from binary file
    Record read_records[10];
    int read_count = read_records(filename, read_records, 10);
    if (read_count < 0) {
        return 1;
    }

    printf("\nRead %d records from %s:\n", read_count, filename);
    for (int i = 0; i < read_count; i++) {
        printf("ID: %d, Name: %s, Value: %.2f\n",
               read_records[i].id, read_records[i].name, read_records[i].value);
    }

    return 0;
```

```
}
```

Common Pitfalls to Avoid:

1. **File handle leaks**: Always close files with fclose
2. **Buffer overflows**: Check buffer sizes when reading text
3. **Binary mode issues**: Use "rb"/"wb" for binary files on all platforms
4. **Error checking**: Always check return values from file functions
5. **Path separators**: Handle cross-platform path differences properly

Best Practices:

1. **Resource management**: Use RAII-like patterns for file handles
2. **Error handling**: Implement comprehensive error checking
3. **Buffer management**: Use appropriate buffer sizes for efficiency
4. **File modes**: Choose correct file modes for intended operations
5. **Data integrity**: Include validation and checksums for critical data

Complete these exercises to solidify your understanding of file I/O in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Modern C

Module 9: Modern C Features Exercises

Exercise 1: C99 Features Implementation

Write a program that demonstrates key C99 features: - Use mixed declarations and code within blocks - Implement designated initializers for structures - Create functions using inline keyword for performance - Use compound literals for temporary objects - Demonstrate variable-length arrays (VLAs)

Requirements: - Use proper C99 compilation flags - Include examples of improved flexibility with mixed declarations - Demonstrate the benefits of designated initializers - Show performance comparisons with inline functions - Handle VLA size checking to prevent stack overflow

Exercise 2: C11 Standard Features

Create a program that utilizes C11 features: - Implement static assertions for compile-time checks - Use generic selections for type-generic programming - Create programs with anonymous structures and unions - Demonstrate alignment specifications - Use thread-local storage for multi-threading

Requirements: - Use appropriate C11 compilation flags - Include comprehensive static assertion examples - Implement generic functions using _Generic - Show practical uses of anonymous structures - Document thread-local storage behavior

Exercise 3: C17/C23 New Features

Develop a program that explores recent C standard features: - Use UTF-8 string literals for international text - Implement binary literals and digit separators - Create programs with enhanced enumeration features - Use nullptr constant (C23) for safer pointer handling - Demonstrate standard attributes like [[deprecated]]

Requirements: - Use appropriate C17/C23 compilation flags - Include examples of UTF-8 handling - Show benefits of binary literals and digit separators - Implement proper nullptr usage patterns - Document attribute-based code annotations

Exercise 4: Type Safety and Modern Practices

Write a program that emphasizes modern C safety practices: - Use stdbool.h for boolean types - Implement fixed-width integer types from stdint.h - Create safe string handling functions - Use restrict keyword for optimization hints - Demonstrate proper const correctness

Requirements: - Include comprehensive examples of type-safe programming - Show performance benefits of restrict keyword - Implement bounds-checked string functions - Document const correctness principles - Provide examples of portable integer types

Exercise 5: Advanced Preprocessor Techniques

Create a program that demonstrates modern preprocessor capabilities: - Implement complex macro functions with error checking - Use variadic macros for flexible logging - Create X-macros for data-driven code generation - Demonstrate conditional compilation with modern features - Implement include guards and pragma once

Requirements: - Include proper macro hygiene practices - Show benefits of variadic macros - Document X-macro patterns and use cases - Implement cross-platform conditional compilation - Provide examples of modern header organization

Exercise 6: Memory Management and Allocation

Write a program that implements modern memory management: - Use aligned_alloc for specific alignment requirements - Implement custom allocators with modern features - Create memory debugging tools with modern C - Demonstrate safe memory handling practices - Include performance profiling for allocation strategies

Requirements: - Include proper error checking for allocation functions - Show benefits of aligned memory access - Implement comprehensive memory tracking - Document safe memory handling patterns - Provide performance comparison data

Exercise 7: Error Handling and Diagnostics

Create a program that demonstrates modern error handling: - Implement detailed error reporting with errno.h - Use static assertions for design-by-contract - Create comprehensive logging systems - Demonstrate proper resource cleanup with modern techniques - Include debugging aids and diagnostic tools

Requirements: - Include comprehensive error classification - Show benefits of static assertions for validation - Implement structured logging with timestamps - Document proper cleanup patterns - Provide debugging support functions

Exercise 8: Comprehensive Modern C Application

Design a complete application that integrates all modern C features: - Implement a data processing pipeline with modern C - Create a configuration system using modern features - Develop a plugin architecture with dynamic loading - Include comprehensive testing and validation - Provide detailed documentation and examples

Requirements: - Use modular design with clear separation of concerns - Include proper documentation for all components - Handle all resource management properly - Implement robust error handling throughout - Provide clear examples and test cases

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Structure with designated initializers
typedef struct {
    int id;
    char name[32];
    double value;
    bool active;
} Item;

// Inline function for performance
inline int max_int(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    // Mixed declarations and code (C99)
    for (int i = 0; i < 5; i++) {
        int square = i * i;  // Declaration inside block
        printf("Square of %d is %d\n", i, square);
    }

    // Designated initializers (C99)
    Item item = {
        .id = 100,
        .name = "Sample Item",
        .value = 99.99,
        .active = true
```

```
    };

    printf("Item: ID=%d, Name=%s, Value=%.2f, Active=%s\n",
           item.id, item.name, item.value,
           item.active ? "true" : "false");

    // Compound literals (C99)
    int *arr = (int[]){1, 2, 3, 4, 5};
    printf("Array from compound literal: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Variable-length array (C99)
    int size;
    printf("Enter array size: ");
    scanf("%d", &size);

    if (size > 0 && size <= 1000) {
        int vla[size];  // VLA declaration
        for (int i = 0; i < size; i++) {
            vla[i] = i * i;
        }

        printf("VLA contents: ");
        for (int i = 0; i < size && i < 10; i++) {  // Limit output
            printf("%d ", vla[i]);
        }
        printf("\n");
    }

    return 0;
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdalign.h>
#include <assert.h>

// Static assertion example (C11)
_Static_assert(sizeof(int) >= 4, "int must be at least 4 bytes");
```

```c
// Generic selection example (C11)
#define MAX_GENERIC(x, y) _Generic((x), \
    int: max_int, \
    float: fmaxf, \
    double: fmax \
)(x, y)

inline int max_int(int a, int b) {
    return (a > b) ? a : b;
}

// Anonymous union example (C11)
typedef struct {
    enum { INT_TYPE, FLOAT_TYPE, STRING_TYPE } type;
    union {
        int int_value;
        float float_value;
        char string_value[32];
    };  // Anonymous union
} Variant;

int main() {
    // Generic selection usage
    int a = 10, b = 20;
    float x = 1.5f, y = 2.5f;

    printf("Max of %d and %d: %d\n", a, b, MAX_GENERIC(a, b));
    printf("Max of %.1f and %.1f: %.1f\n", x, y, MAX_GENERIC(x, y));

    // Anonymous union usage
    Variant v;

    v.type = INT_TYPE;
    v.int_value = 42;
    printf("Integer variant: %d\n", v.int_value);

    v.type = STRING_TYPE;
    snprintf(v.string_value, sizeof(v.string_value), "Hello, C11!");
    printf("String variant: %s\n", v.string_value);

    // Alignment specification (C11)
    alignas(16) char aligned_buffer[64];
    printf("Aligned buffer address: %p\n", (void*)aligned_buffer);
    printf("Alignment check: %s\n",
           ((uintptr_t)aligned_buffer % 16 == 0) ? "Aligned" : "Not aligned");
```

```
    return 0;
}
```

Common Pitfalls to Avoid:

1. **Compiler compatibility**: Not all compilers support all modern features
2. **Feature detection**: Always check for feature availability before use
3. **Performance assumptions**: Measure actual performance benefits
4. **Portability issues**: Consider cross-platform compatibility
5. **Complexity creep**: Don't over-engineer with modern features

Best Practices:

1. **Feature detection**: Use preprocessor checks for feature availability
2. **Gradual adoption**: Introduce modern features incrementally
3. **Documentation**: Comment modern feature usage clearly
4. **Testing**: Verify behavior across different compilers
5. **Fallbacks**: Provide alternatives for older compiler versions

Complete these exercises to solidify your understanding of modern C features. Each exercise builds upon the previous ones, gradually increasing in complexity.

Data Structures

Module 10: Data Structures Exercises

Exercise 1: Linear Data Structures Implementation

Write a program that implements fundamental linear data structures: - Implement a complete stack with push, pop, and peek operations - Create a queue with enqueue and dequeue functions - Develop a deque (double-ended queue) with operations on both ends - Implement a circular buffer for efficient memory usage - Include comprehensive error handling and boundary checks

Requirements: - Use dynamic memory allocation for flexible sizing - Include proper initialization and cleanup functions - Implement comprehensive error checking - Provide performance analysis for operations - Include examples of practical applications

Exercise 2: Linked List Variations

Create a program that implements various linked list types: - Singly linked list with insert, delete, and search operations - Doubly linked list with forward and backward traversal - Circular linked list with proper termination handling - Implement sorted insertion and merge operations - Include memory management and leak prevention

Requirements: - Handle all edge cases (empty list, single node, etc.) - Implement both iterative and recursive algorithms - Include performance comparisons between list types - Provide comprehensive test cases - Document time and space complexity

Exercise 3: Tree Data Structures

Develop a program that implements tree structures: - Binary search tree with insert, delete, and search operations - AVL tree with automatic balancing - Heap implementation (min-heap and max-heap) - Trie for string storage and retrieval - Include traversal algorithms (inorder, preorder, postorder, level-order)

Requirements: - Implement proper balancing algorithms for AVL trees - Include visualization functions for tree structures - Handle memory management correctly - Provide performance analysis for operations - Include comprehensive error handling

Exercise 4: Hash Tables and Associative Arrays

Write a program that implements hash-based data structures: - Hash table with collision resolution (chaining or open addressing) - Implement different hash functions and compare their performance - Create an associative array (dictionary) interface - Include dynamic resizing based on load factor - Provide statistics on collision rates and performance

Requirements: - Implement multiple collision resolution strategies - Include proper hash function selection and testing - Handle dynamic resizing efficiently - Provide performance profiling tools - Include comprehensive error handling

Exercise 5: Graph Representations and Algorithms

Create a program that works with graph data structures: - Implement adjacency list and adjacency matrix representations - Create graph traversal algorithms (BFS and DFS) - Implement shortest path algorithms (Dijkstra, Bellman-Ford) - Develop minimum spanning tree algorithms (Prim, Kruskal) - Include cycle detection and topological sorting

Requirements: - Handle both directed and undirected graphs - Implement weighted and unweighted graph versions - Include proper memory management for graph structures - Provide visualization tools for graph algorithms - Document algorithm complexity and use cases

Exercise 6: Advanced Data Structures

Write a program that implements specialized data structures: - Implement a priority queue using heap - Create a disjoint-set (union-find) data structure - Develop a segment tree for range queries - Implement a B-tree for database indexing - Include a bloom filter for probabilistic set membership

Requirements: - Include comprehensive documentation for each structure - Provide performance analysis and use case examples - Handle edge cases and error conditions properly - Include testing frameworks for verification - Document implementation details and optimizations

Exercise 7: Data Structure Performance Analysis

Create a program that analyzes and compares data structures: - Implement benchmarking tools for different operations - Create performance profiling for various data structures - Compare memory usage and cache efficiency - Analyze time complexity with empirical data - Include visualization of performance characteristics

Requirements: - Include comprehensive benchmarking framework - Provide statistical analysis of results - Handle large datasets for meaningful comparisons - Include cross-platform performance considerations - Document methodology and findings clearly

Exercise 8: Comprehensive Data Structure Application

Design a complete application that integrates multiple data structures: - Implement a database system using various data structures - Create a file system simulator with directory structures - Develop a network routing system with graph algorithms - Include a caching system with hash tables and LRU eviction - Provide comprehensive testing and validation

Requirements: - Use modular design with clear separation of concerns - Include proper documentation for all components - Handle all resource management properly - Implement robust error handling throughout - Provide clear examples and test cases

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>

// Stack implementation
typedef struct {
    int *data;
    int top;
    int capacity;
} Stack;

// Function to create a stack
Stack* create_stack(int capacity) {
    Stack *stack = malloc(sizeof(Stack));
    if (stack == NULL) return NULL;

    stack->data = malloc(capacity * sizeof(int));
    if (stack->data == NULL) {
        free(stack);
        return NULL;
    }

    stack->top = -1;
    stack->capacity = capacity;
    return stack;
}

// Function to check if stack is empty
bool is_stack_empty(Stack *stack) {
```

```c
    return stack == NULL || stack->top == -1;
}

// Function to check if stack is full
bool is_stack_full(Stack *stack) {
    return stack != NULL && stack->top == stack->capacity - 1;
}

// Function to push element onto stack
bool push(Stack *stack, int value) {
    if (is_stack_full(stack)) {
        printf("Error: Stack overflow\n");
        return false;
    }

    stack->data[++stack->top] = value;
    return true;
}

// Function to pop element from stack
int pop(Stack *stack) {
    if (is_stack_empty(stack)) {
        printf("Error: Stack underflow\n");
        return INT_MIN;
    }

    return stack->data[stack->top--];
}

// Function to peek at top element
int peek(Stack *stack) {
    if (is_stack_empty(stack)) {
        printf("Error: Stack is empty\n");
        return INT_MIN;
    }

    return stack->data[stack->top];
}

// Function to free stack memory
void free_stack(Stack *stack) {
    if (stack != NULL) {
        free(stack->data);
        free(stack);
    }
}
```

```c
int main() {
    Stack *stack = create_stack(5);
    if (stack == NULL) {
        printf("Error: Failed to create stack\n");
        return 1;
    }

    // Test stack operations
    printf("Testing stack operations:\n");

    // Push elements
    for (int i = 1; i <= 5; i++) {
        if (push(stack, i * 10)) {
            printf("Pushed %d onto stack\n", i * 10);
        }
    }

    // Try to push to full stack
    if (!push(stack, 60)) {
        printf("Successfully handled stack overflow\n");
    }

    // Peek at top element
    printf("Top element: %d\n", peek(stack));

    // Pop elements
    while (!is_stack_empty(stack)) {
        int value = pop(stack);
        printf("Popped %d from stack\n", value);
    }

    // Try to pop from empty stack
    int empty_pop = pop(stack);
    if (empty_pop == INT_MIN) {
        printf("Successfully handled stack underflow\n");
    }

    free_stack(stack);
    return 0;
}
```

Exercise 3 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Binary Search Tree Node
typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

// Function to create a new tree node
TreeNode* create_node(int data) {
    TreeNode *node = malloc(sizeof(TreeNode));
    if (node == NULL) return NULL;

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// Function to insert a node into BST
TreeNode* insert_bst(TreeNode *root, int data) {
    if (root == NULL) {
        return create_node(data);
    }

    if (data < root->data) {
        root->left = insert_bst(root->left, data);
    } else if (data > root->data) {
        root->right = insert_bst(root->right, data);
    }
    // If data == root->data, we don't insert duplicates

    return root;
}

// Function for inorder traversal
void inorder_traversal(TreeNode *root) {
    if (root != NULL) {
        inorder_traversal(root->left);
        printf("%d ", root->data);
        inorder_traversal(root->right);
    }
```

```
}

// Function to search for a value in BST
TreeNode* search_bst(TreeNode *root, int data) {
    if (root == NULL || root->data == data) {
        return root;
    }

    if (data < root->data) {
        return search_bst(root->left, data);
    } else {
        return search_bst(root->right, data);
    }
}

// Function to find minimum value node
TreeNode* find_min(TreeNode *node) {
    while (node && node->left != NULL) {
        node = node->left;
    }
    return node;
}

// Function to delete a node from BST
TreeNode* delete_bst(TreeNode *root, int data) {
    if (root == NULL) return root;

    if (data < root->data) {
        root->left = delete_bst(root->left, data);
    } else if (data > root->data) {
        root->right = delete_bst(root->right, data);
    } else {
        // Node to be deleted found
        if (root->left == NULL) {
            TreeNode *temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            TreeNode *temp = root->left;
            free(root);
            return temp;
        }

        // Node with two children
        TreeNode *temp = find_min(root->right);
        root->data = temp->data;
```

```
        root->right = delete_bst(root->right, temp->data);
    }

    return root;
}

// Function to free entire tree
void free_tree(TreeNode *root) {
    if (root != NULL) {
        free_tree(root->left);
        free_tree(root->right);
        free(root);
    }
}

int main() {
    TreeNode *root = NULL;

    // Insert elements into BST
    int values[] = {50, 30, 70, 20, 40, 60, 80};
    int n = sizeof(values) / sizeof(values[0]);

    printf("Inserting values into BST: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", values[i]);
        root = insert_bst(root, values[i]);
    }
    printf("\n");

    // Inorder traversal (should be sorted)
    printf("Inorder traversal: ");
    inorder_traversal(root);
    printf("\n");

    // Search for values
    int search_values[] = {40, 90};
    for (int i = 0; i < 2; i++) {
        TreeNode *result = search_bst(root, search_values[i]);
        if (result != NULL) {
            printf("Found %d in BST\n", search_values[i]);
        } else {
            printf("%d not found in BST\n", search_values[i]);
        }
    }

    // Delete a node
```

```c
    printf("Deleting 30 from BST\n");
    root = delete_bst(root, 30);

    printf("Inorder traversal after deletion: ");
    inorder_traversal(root);
    printf("\n");

    // Free memory
    free_tree(root);
    return 0;
}
```

Common Pitfalls to Avoid:

1. **Memory leaks**: Always free allocated memory for dynamic structures
2. **Null pointer dereference**: Check pointers before dereferencing
3. **Boundary conditions**: Handle empty structures and edge cases properly
4. **Infinite recursion**: Ensure proper base cases for recursive algorithms
5. **Performance degradation**: Choose appropriate data structures for use cases

Best Practices:

1. **Modular design**: Implement each data structure as separate modules
2. **Error handling**: Include comprehensive error checking and reporting
3. **Documentation**: Comment interface functions and implementation details
4. **Testing**: Provide thorough test cases for all operations
5. **Performance analysis**: Include complexity analysis and benchmarking

Complete these exercises to solidify your understanding of data structures in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Network

Module 11: Network Programming Exercises

Exercise 1: Basic Socket Programming

Write a program that demonstrates fundamental socket operations: - Create and configure TCP and UDP sockets - Implement basic client-server communication - Handle socket binding, listening, and connection acceptance - Send and receive data through sockets - Properly close and cleanup socket resources

Requirements: - Implement both TCP and UDP examples - Include proper error handling for all socket operations - Handle cross-platform differences (Windows vs Unix) - Implement timeout handling for socket operations - Provide clear examples of client-server interaction

Exercise 2: HTTP Client Implementation

Create a program that implements a simple HTTP client: - Parse URLs and extract host, port, and path components - Establish HTTP connections to web servers - Send HTTP GET and POST requests - Parse HTTP response headers and status codes - Handle chunked transfer encoding and content length

Requirements: - Implement proper HTTP/1.1 protocol compliance - Include support for common HTTP headers - Handle redirects and authentication (basic) - Implement connection reuse for multiple requests - Include proper error handling for network issues

Exercise 3: Multi-client Server

Develop a program that implements a server handling multiple clients: - Use fork() or threading to handle concurrent clients - Implement connection pooling for efficient resource usage - Handle client disconnections gracefully - Include client session management - Implement basic rate limiting and security measures

Requirements: - Handle race conditions with proper synchronization - Include timeout mechanisms for inactive clients - Implement proper resource cleanup for terminated clients - Provide logging and monitoring capabilities - Include comprehensive error handling

Exercise 4: Network Protocol Implementation

Write a program that implements custom network protocols: - Design and implement a simple chat protocol - Create a file transfer protocol with checksum verification - Implement a simple remote command execution protocol - Include protocol versioning and compatibility - Handle protocol state management

Requirements: - Define clear protocol specifications - Include proper message framing and parsing - Implement error recovery mechanisms - Handle network byte order conversion - Provide protocol documentation and examples

Exercise 5: Advanced Networking Concepts

Create a program that demonstrates advanced networking features: - Implement non-blocking I/O with select() or poll() - Use epoll() or kqueue() for high-performance I/O (Linux/BSD) - Implement socket timeouts and keep-alive mechanisms - Handle network address resolution and DNS lookups - Include IPv6 support alongside IPv4

Requirements: - Include performance comparisons between different I/O models - Handle partial reads and writes correctly - Implement proper signal handling for network applications - Include network interface enumeration and monitoring - Provide cross-platform compatibility

Exercise 6: Network Security Implementation

Write a program that implements basic network security: - Implement simple encryption/decryption functions - Create secure communication channels with basic cryptography - Handle certificate validation and SSL/TLS (bonus) - Implement basic authentication mechanisms - Include secure key exchange protocols

Requirements: - Use established cryptographic libraries (OpenSSL, etc.) - Include proper random number generation - Handle key management and storage securely - Implement secure session establishment - Provide security best practices documentation

Exercise 7: Network Debugging and Monitoring

Create a program that provides network debugging capabilities: - Implement packet capture and analysis (bonus) - Create network performance monitoring tools - Develop connection state tracking and logging - Include bandwidth measurement and reporting - Implement network error diagnosis tools

Requirements: - Include comprehensive logging and tracing - Provide real-time monitoring capabilities - Handle large volumes of network data efficiently - Include visualization of network statistics - Document debugging techniques and tools

Exercise 8: Comprehensive Network Application

Design a complete network application that integrates all concepts: - Implement a distributed system with multiple network components - Create a peer-to-peer file sharing network - Develop a simple web server with dynamic content - Include network service discovery and registration - Provide comprehensive testing and validation

Requirements: - Use modular design with clear separation of concerns - Include proper documentation for all components - Handle all network resource management properly - Implement robust error handling throughout - Provide clear examples and test cases

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#ifdef _WIN32
    #include <winsock2.h>
    #include <ws2tcpip.h>
    #pragma comment(lib, "ws2_32.lib")
#else
    #include <sys/socket.h>
    #include <arpa/inet.h>
    #include <netinet/in.h>
    #include <netdb.h>
#endif

// Cross-platform socket definitions
#ifdef _WIN32
    typedef SOCKET socket_t;
    #define CLOSE_SOCKET closesocket
    #define SOCK_ERR SOCKET_ERROR
#else
    typedef int socket_t;
    #define CLOSE_SOCKET close
    #define SOCK_ERR -1
#endif

// Function to initialize sockets (Windows specific)
int initialize_sockets(void) {
#ifdef _WIN32
```

```
    WSADATA wsaData;
    int result = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if (result != 0) {
        printf("WSAStartup failed: %d\n", result);
        return 0;
    }
#endif
    return 1;
}

// Function to cleanup sockets (Windows specific)
void cleanup_sockets(void) {
#ifdef _WIN32
    WSACleanup();
#endif
}

// Function to create TCP socket
socket_t create_tcp_socket(void) {
    socket_t sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == SOCK_ERR) {
        perror("Socket creation failed");
    }
    return sock;
}

// Function to create UDP socket
socket_t create_udp_socket(void) {
    socket_t sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock == SOCK_ERR) {
        perror("UDP Socket creation failed");
    }
    return sock;
}

// Simple TCP server example
int tcp_server_example(int port) {
    if (!initialize_sockets()) return -1;

    socket_t server_sock = create_tcp_socket();
    if (server_sock == SOCK_ERR) {
        cleanup_sockets();
        return -1;
    }

    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
```

```
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(port);

    if (bind(server_sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) == SOCK_ERR) {
        perror("Bind failed");
        CLOSE_SOCKET(server_sock);
        cleanup_sockets();
        return -1;
    }

    if (listen(server_sock, 5) == SOCK_ERR) {
        perror("Listen failed");
        CLOSE_SOCKET(server_sock);
        cleanup_sockets();
        return -1;
    }

    printf("TCP Server listening on port %d\n", port);

    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);
    socket_t client_sock = accept(server_sock, (struct sockaddr*)&client_addr, &client_len);

    if (client_sock != SOCK_ERR) {
        char buffer[1024];
        int bytes_received = recv(client_sock, buffer, sizeof(buffer) - 1, 0);
        if (bytes_received > 0) {
            buffer[bytes_received] = '\0';
            printf("Received: %s\n", buffer);

            const char *response = "Hello from TCP server!";
            send(client_sock, response, strlen(response), 0);
        }
        CLOSE_SOCKET(client_sock);
    }

    CLOSE_SOCKET(server_sock);
    cleanup_sockets();
    return 0;
}

// Simple TCP client example
int tcp_client_example(const char *server_ip, int port) {
    if (!initialize_sockets()) return -1;

    socket_t client_sock = create_tcp_socket();
```

```
    if (client_sock == SOCK_ERR) {
        cleanup_sockets();
        return -1;
    }

    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);

    if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) <= 0) {
        printf("Invalid address\n");
        CLOSE_SOCKET(client_sock);
        cleanup_sockets();
        return -1;
    }

    if (connect(client_sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) == SOCK_ER
        perror("Connection failed");
        CLOSE_SOCKET(client_sock);
        cleanup_sockets();
        return -1;
    }

    const char *message = "Hello from TCP client!";
    send(client_sock, message, strlen(message), 0);

    char buffer[1024];
    int bytes_received = recv(client_sock, buffer, sizeof(buffer) - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Received: %s\n", buffer);
    }

    CLOSE_SOCKET(client_sock);
    cleanup_sockets();
    return 0;
}

int main() {
    // Note: For testing, you would run the server and client separately
    // This is just an example of how to structure the code

    printf("Network Programming Examples\n");
    printf("Run tcp_server_example() in one terminal\n");
    printf("Run tcp_client_example() in another terminal\n");
```

```
    return 0;
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#ifdef _WIN32
    #include <winsock2.h>
    #include <ws2tcpip.h>
    #pragma comment(lib, "ws2_32.lib")
#else
    #include <sys/socket.h>
    #include <arpa/inet.h>
    #include <netinet/in.h>
    #include <netdb.h>
#endif

#ifdef _WIN32
    typedef SOCKET socket_t;
    #define CLOSE_SOCKET closesocket
    #define SOCK_ERR SOCKET_ERROR
#else
    typedef int socket_t;
    #define CLOSE_SOCKET close
    #define SOCK_ERR -1
#endif

// URL parsing structure
typedef struct {
    char protocol[16];
    char host[256];
    int port;
    char path[512];
} URL;

// Function to parse URL
int parse_url(const char *url_str, URL *url) {
    // Simple URL parsing (supports http://host:port/path)
    if (sscanf(url_str, "%15[^:]://%255[^:/]:%d%511s",
               url->protocol, url->host, &url->port, url->path) == 4) {
        return 0;
```

```
    }

    // Try without port
    if (sscanf(url_str, "%15[^:]://%255[^/]%511s",
               url->protocol, url->host, url->path) == 3) {
        url->port = (strcmp(url->protocol, "https") == 0) ? 443 : 80;
        return 0;
    }

    return -1;
}

// Simple HTTP GET request
int http_get_request(const char *url_str) {
#ifdef _WIN32
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        return -1;
    }
#endif

    URL url;
    if (parse_url(url_str, &url) != 0) {
        printf("Failed to parse URL: %s\n", url_str);
#ifdef _WIN32
        WSACleanup();
#endif
        return -1;
    }

    // Create socket
    socket_t sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == SOCK_ERR) {
        perror("Socket creation failed");
#ifdef _WIN32
        WSACleanup();
#endif
        return -1;
    }

    // Resolve hostname
    struct hostent *host = gethostbyname(url.host);
    if (host == NULL) {
        printf("Failed to resolve hostname: %s\n", url.host);
        CLOSE_SOCKET(sock);
#ifdef _WIN32
        WSACleanup();
```

```
#endif
        return -1;
    }

    // Connect to server
    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(url.port);
    memcpy(&server_addr.sin_addr, host->h_addr_list[0], host->h_length);

    if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) == SOCK_ERR) {
        perror("Connection failed");
        CLOSE_SOCKET(sock);
#ifdef _WIN32
        WSACleanup();
#endif
        return -1;
    }

    // Send HTTP GET request
    char request[1024];
    snprintf(request, sizeof(request),
             "GET %s HTTP/1.1\r\n"
             "Host: %s:%d\r\n"
             "Connection: close\r\n"
             "\r\n",
             url.path, url.host, url.port);

    if (send(sock, request, strlen(request), 0) == SOCK_ERR) {
        perror("Send failed");
        CLOSE_SOCKET(sock);
#ifdef _WIN32
        WSACleanup();
#endif
        return -1;
    }

    // Receive response
    char buffer[4096];
    int bytes_received;
    while ((bytes_received = recv(sock, buffer, sizeof(buffer) - 1, 0)) > 0) {
        buffer[bytes_received] = '\0';
        printf("%s", buffer);
    }

    CLOSE_SOCKET(sock);
```

```c
#ifdef _WIN32
    WSACleanup();
#endif
    return 0;
}

int main() {
    const char *url = "http://httpbin.org/get";
    printf("Making HTTP GET request to: %s\n\n", url);

    if (http_get_request(url) != 0) {
        printf("HTTP request failed\n");
        return 1;
    }

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Platform differences**: Handle Windows and Unix socket API differences
2. **Resource leaks**: Always close sockets and cleanup network libraries
3. **Buffer overflows**: Check buffer sizes when receiving network data
4. **Blocking operations**: Handle timeouts to prevent hanging applications
5. **Error handling**: Check return values from all network functions

Best Practices:

1. **Cross-platform compatibility**: Use conditional compilation for platform differences
2. **Resource management**: Implement proper cleanup in error paths
3. **Security considerations**: Validate all network input and use secure protocols
4. **Performance optimization**: Use appropriate I/O models for scalability
5. **Error recovery**: Implement robust error handling and recovery mechanisms

Complete these exercises to solidify your understanding of network programming in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Embedded

Module 12: Embedded Systems Programming Exercises

Exercise 1: Bit Manipulation and Register Operations

Write a program that demonstrates fundamental bit manipulation operations commonly used in embedded systems: - Implement functions for setting, clearing, and toggling individual bits - Create macros for bit manipulation operations - Demonstrate bit field usage for hardware register access - Implement functions for reading and writing hardware registers - Show how to handle endianness in register operations

Requirements: - Implement bit manipulation functions without using built-in bit operations - Create reusable macros for common bit operations - Demonstrate proper handling of volatile variables for hardware registers - Include examples of bit field structures for hardware register mapping - Provide clear documentation for all bit manipulation functions

Exercise 2: Memory-Mapped I/O Implementation

Create a program that simulates memory-mapped I/O operations: - Implement functions for reading and writing to memory-mapped registers - Create a simple GPIO (General Purpose Input/Output) simulation - Demonstrate proper use of volatile pointers for hardware access - Implement interrupt handling for I/O operations - Show how to handle memory alignment requirements

Requirements: - Use appropriate data types for memory-mapped registers - Implement proper error checking for I/O operations - Include examples of direct memory access patterns - Demonstrate proper handling of memory barriers if needed - Provide clear examples of GPIO configuration and control

Exercise 3: Interrupt Service Routines

Develop a program that implements interrupt service routines (ISRs): - Create a framework for registering and handling interrupts - Implement a simple timer interrupt simulation - Demonstrate proper context saving and restoration - Show how to handle nested interrupts - Implement interrupt prioritization mechanisms

Requirements: - Follow proper ISR coding practices (minimal code, no blocking operations) - Include examples of interrupt enable/disable functions - Demonstrate proper use of atomic operations in ISRs - Implement interrupt vector tables or similar dispatch mechanisms - Provide clear documentation of interrupt handling flow

Exercise 4: Real-Time Constraints Implementation

Write a program that demonstrates handling real-time constraints: - Implement a simple real-time scheduler - Create functions with guaranteed execution time limits - Demonstrate proper handling of timing-critical sections - Implement watchdog timer functionality - Show how to measure and optimize code execution time

Requirements: - Use appropriate timing functions for the target platform - Include examples of critical section protection - Demonstrate proper use of hardware timers - Implement timeout mechanisms for blocking operations - Provide clear examples of real-time performance measurement

Exercise 5: Low-Power Programming Techniques

Create a program that implements low-power programming techniques: - Implement sleep and power-down modes - Create functions for dynamic clock scaling - Demonstrate proper handling of wake-up sources - Show how to optimize code for power consumption - Implement power management state machines

Requirements: - Include examples of different power-saving modes - Demonstrate proper handling of sleep/wake cycles - Show how to preserve state during power transitions - Implement power consumption measurement and reporting - Provide clear documentation of power management strategies

Exercise 6: Hardware Abstraction Layer (HAL)

Write a program that implements a simple Hardware Abstraction Layer: - Create a unified interface for different hardware platforms - Implement device driver interfaces for common peripherals - Demonstrate proper error handling in HAL functions - Show how to handle hardware initialization and configuration - Implement version control for HAL interfaces

Requirements: - Use consistent naming conventions across all HAL functions - Include proper error codes and return value handling - Demonstrate proper initialization and cleanup sequences - Provide clear documentation for all HAL interfaces - Include examples of platform-specific implementations

Exercise 7: Embedded Debugging and Monitoring

Create a program that provides debugging and monitoring capabilities for embedded systems: - Implement a simple logging system with minimal resource usage - Create functions for runtime system monitoring - Demonstrate proper use of assertions in embedded environments - Show how to implement non-intrusive debugging techniques - Implement memory usage tracking and reporting

Requirements: - Include configurable logging levels and output destinations - Demonstrate proper handling of limited memory resources - Show how to implement circular buffers for logging - Provide examples of runtime error detection and reporting - Include clear documentation of debugging techniques

Exercise 8: Comprehensive Embedded Application

Design a complete embedded application that integrates all concepts: - Implement a simple embedded system with multiple components - Create a state machine for system control - Demonstrate proper handling of hardware and software components - Include comprehensive error handling and recovery mechanisms - Provide clear documentation and testing procedures

Requirements: - Use modular design with clear separation of concerns - Include proper initialization and shutdown sequences - Demonstrate proper resource management throughout the application - Implement robust error handling and recovery mechanisms - Provide clear examples and test cases for all components

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdint.h>

// Bit manipulation macros
#define SET_BIT(reg, bit)       ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit)     ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit)    ((reg) ^= (1U << (bit)))
#define CHECK_BIT(reg, bit)     (((reg) >> (bit)) & 1U)

// Bit manipulation functions
void set_bit(uint32_t *reg, int bit) {
    *reg |= (1U << bit);
}

void clear_bit(uint32_t *reg, int bit) {
    *reg &= ~(1U << bit);
}

void toggle_bit(uint32_t *reg, int bit) {
    *reg ^= (1U << bit);
}

int check_bit(uint32_t reg, int bit) {
    return (reg >> bit) & 1U;
}

// Bit field structure for hardware register
typedef struct {
    uint32_t enable    : 1;  // Bit 0
```

```
    uint32_t direction : 1;  // Bit 1
    uint32_t reserved  : 2;  // Bits 2-3
    uint32_t mode      : 4;  // Bits 4-7
    uint32_t config    : 24; // Bits 8-31
} gpio_register_t;

// Volatile pointer for hardware register access
volatile gpio_register_t *gpio_reg = (volatile gpio_register_t *)0x40020000;

int main() {
    uint32_t test_reg = 0;

    // Test bit manipulation functions
    printf("Initial register value: 0x%08X\n", test_reg);

    set_bit(&test_reg, 5);
    printf("After setting bit 5: 0x%08X\n", test_reg);

    toggle_bit(&test_reg, 5);
    printf("After toggling bit 5: 0x%08X\n", test_reg);

    clear_bit(&test_reg, 5);
    printf("After clearing bit 5: 0x%08X\n", test_reg);

    // Test bit manipulation macros
    SET_BIT(test_reg, 3);
    printf("After setting bit 3 with macro: 0x%08X\n", test_reg);

    if (CHECK_BIT(test_reg, 3)) {
        printf("Bit 3 is set\n");
    }

    // Test bit field structure
    gpio_reg->enable = 1;
    gpio_reg->direction = 1;
    gpio_reg->mode = 0xF;

    printf("GPIO Register - Enable: %d, Direction: %d, Mode: 0x%X\n",
           gpio_reg->enable, gpio_reg->direction, gpio_reg->mode);

    return 0;
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

// Memory-mapped I/O simulation
#define GPIO_BASE_ADDR 0x40020000
#define GPIO_DIR_REG   (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x00))
#define GPIO_DATA_REG  (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x04))
#define GPIO_SET_REG   (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x08))
#define GPIO_CLR_REG   (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x0C))

// GPIO pin definitions
#define GPIO_PIN_0  (1U << 0)
#define GPIO_PIN_1  (1U << 1)
#define GPIO_PIN_2  (1U << 2)
#define GPIO_PIN_3  (1U << 3)

// Simple GPIO simulation structure
typedef struct {
    uint32_t direction;  // 0 = input, 1 = output
    uint32_t data;       // Current pin states
} gpio_sim_t;

// Global GPIO simulation instance
static gpio_sim_t gpio_sim = {0};

// Function to initialize GPIO
void gpio_init(void) {
    gpio_sim.direction = 0;
    gpio_sim.data = 0;
    printf("GPIO initialized\n");
}

// Function to set GPIO direction
void gpio_set_direction(uint32_t pins, int direction) {
    if (direction) {
        gpio_sim.direction |= pins;  // Set as output
    } else {
        gpio_sim.direction &= ~pins; // Set as input
    }

    // Update hardware register
    GPIO_DIR_REG = gpio_sim.direction;
```

```
    printf("GPIO direction set: 0x%08X (pins: 0x%08X, direction: %s)\n",
           gpio_sim.direction, pins, direction ? "output" : "input");
}

// Function to write to GPIO pins
void gpio_write(uint32_t pins, int value) {
    if (value) {
        gpio_sim.data |= pins;
        GPIO_SET_REG = pins;  // Hardware register for setting pins
    } else {
        gpio_sim.data &= ~pins;
        GPIO_CLR_REG = pins;  // Hardware register for clearing pins
    }

    GPIO_DATA_REG = gpio_sim.data;  // Update data register
    printf("GPIO write: pins 0x%08X set to %d\n", pins, value);
}

// Function to read from GPIO pins
uint32_t gpio_read(uint32_t pins) {
    uint32_t value = GPIO_DATA_REG & pins;  // Read from hardware register
    printf("GPIO read: pins 0x%08X = 0x%08X\n", pins, value);
    return value;
}

// Function to simulate external input change
void gpio_simulate_input(uint32_t pins, int value) {
    if (value) {
        gpio_sim.data |= pins;
    } else {
        gpio_sim.data &= ~pins;
    }
    printf("Simulated input change: pins 0x%08X set to %d\n", pins, value);
}

int main() {
    // Initialize GPIO
    gpio_init();

    // Configure pins 0-1 as outputs, pins 2-3 as inputs
    gpio_set_direction(GPIO_PIN_0 | GPIO_PIN_1, 1);  // Output
    gpio_set_direction(GPIO_PIN_2 | GPIO_PIN_3, 0);  // Input

    // Write to output pins
    gpio_write(GPIO_PIN_0, 1);
    gpio_write(GPIO_PIN_1, 0);
```

```
    // Simulate input changes
    gpio_simulate_input(GPIO_PIN_2, 1);
    gpio_simulate_input(GPIO_PIN_3, 0);

    // Read input pins
    uint32_t input_value = gpio_read(GPIO_PIN_2 | GPIO_PIN_3);
    printf("Input pins value: 0x%08X\n", input_value);

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Volatile keyword misuse**: Always use volatile for hardware registers
2. **Endianness issues**: Handle byte order correctly in multi-byte registers
3. **Timing constraints**: Avoid blocking operations in time-critical code
4. **Memory alignment**: Ensure proper alignment for memory-mapped I/O
5. **Interrupt safety**: Protect critical sections in interrupt-driven code

Best Practices:

1. **Hardware abstraction**: Use consistent interfaces for hardware access
2. **Error handling**: Implement proper error checking for all operations
3. **Resource management**: Carefully manage memory and other resources
4. **Documentation**: Provide clear documentation for hardware interactions
5. **Testing**: Include comprehensive testing for embedded functionality

Complete these exercises to solidify your understanding of embedded systems programming in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Performance

Module 13: Performance Optimization Exercises

Exercise 1: Algorithm Complexity Analysis

Write a program that demonstrates algorithm complexity analysis and optimization: - Implement multiple versions of common algorithms with different complexities - Create functions to measure execution time of algorithms - Demonstrate the impact of algorithmic complexity on performance - Show how to profile code to identify bottlenecks - Implement optimized versions of inefficient algorithms

Requirements: - Include examples of O(1), O(log n), O(n), O(n log n), and $O(n^2)$ algorithms - Implement accurate timing functions for performance measurement - Provide clear visualization of performance differences - Include examples of common algorithmic optimizations - Document the analysis process and findings

Exercise 2: Memory Access Patterns Optimization

Create a program that demonstrates memory access pattern optimization: - Implement functions with different memory access patterns - Show the impact of cache locality on performance - Demonstrate proper use of memory alignment - Implement techniques for reducing cache misses - Show how to optimize data structure layouts

Requirements: - Include examples of sequential, random, and strided memory access - Implement cache-friendly data structures - Demonstrate proper memory alignment techniques - Show the impact of data structure padding and packing - Provide clear performance comparisons between different approaches

Exercise 3: Compiler Optimization Techniques

Develop a program that demonstrates compiler optimization techniques: - Implement code that showcases different compiler optimization levels - Show how to use compiler hints and attributes effectively - Demonstrate proper use of inline functions and macros - Implement examples of loop optimization techniques - Show how to prevent unwanted compiler optimizations for testing

Requirements: - Include examples of common compiler optimization flags - Demonstrate proper use of restrict keyword and volatile - Show how to use compiler-specific attributes and pragmas - Implement examples of manual loop unrolling and vectorization - Provide clear documentation of optimization techniques

Exercise 4: Data Structure Optimization

Write a program that demonstrates data structure optimization: - Implement multiple versions of common data structures - Show the impact of different data structure choices on performance - Demonstrate proper use of memory pools and object pools - Implement custom allocators for specific use cases - Show how to optimize data structures for specific access patterns

Requirements: - Include examples of array vs linked list performance trade-offs - Implement cache-oblivious data structures - Demonstrate proper use of memory preallocation - Show how to optimize data structures for concurrent access - Provide clear performance comparisons and analysis

Exercise 5: Parallel Processing Implementation

Create a program that implements parallel processing techniques: - Implement multi-threaded versions of compute-intensive tasks - Show how to use OpenMP or pthreads effectively - Demonstrate proper synchronization techniques - Implement lock-free data structures (bonus) - Show how to measure and optimize parallel performance

Requirements: - Include examples of task parallelism and data parallelism - Demonstrate proper use of thread pools - Show how to avoid common parallel programming pitfalls - Implement proper load balancing techniques - Provide clear scalability analysis and measurements

Exercise 6: SIMD and Vectorization

Write a program that demonstrates SIMD (Single Instruction, Multiple Data) optimization: - Implement vectorized versions of compute-intensive functions - Show how to use SSE, AVX, or other SIMD instruction sets - Demonstrate proper data alignment for SIMD operations - Implement fallback implementations for different architectures - Show how to measure SIMD performance improvements

Requirements: - Include examples of manual vectorization and compiler auto-vectorization - Demonstrate proper handling of data alignment requirements - Show how to handle different vector widths and instruction sets - Implement proper error handling for SIMD operations - Provide clear performance comparisons between scalar and vector versions

Exercise 7: Memory Management Optimization

Create a program that demonstrates memory management optimization: - Implement custom memory allocators for specific use cases - Show how to reduce memory fragmentation - Demonstrate proper use of memory pools and arenas - Implement garbage collection-like mechanisms (bonus) - Show how to profile and optimize memory usage

Requirements: - Include examples of stack vs heap allocation performance - Demonstrate proper use of memory preallocation techniques - Show how to implement efficient memory pooling strategies - Implement proper memory leak detection and prevention - Provide clear memory usage analysis and optimization

Exercise 8: Comprehensive Performance Optimization

Design a complete application that demonstrates all performance optimization techniques: - Implement a compute-intensive application with multiple components - Apply all optimization techniques learned in previous exercises - Include comprehensive performance profiling and analysis - Demonstrate proper trade-offs between different optimization approaches - Provide clear documentation and performance results

Requirements: - Use modular design with clear performance boundaries - Include comprehensive profiling and benchmarking - Demonstrate proper optimization prioritization - Implement robust performance monitoring throughout the application - Provide clear examples and performance comparisons

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

// Timing function
double get_time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
}

// O(n²) bubble sort
void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// O(n log n) quick sort
void quick_sort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
```

```
        quick_sort(arr, pi + 1, high);
    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}

// O(n) linear search
int linear_search(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

// O(log n) binary search
int binary_search(int arr[], int low, int high, int key) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) {
            return mid;
        }
        if (arr[mid] > key) {
            return binary_search(arr, low, mid - 1, key);
        }
        return binary_search(arr, mid + 1, high, key);
    }
    return -1;
}
```

```c
// Performance testing function
void test_algorithms() {
    const int sizes[] = {1000, 5000, 10000, 20000};
    const int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    for (int s = 0; s < num_sizes; s++) {
        int n = sizes[s];
        int *arr1 = malloc(n * sizeof(int));
        int *arr2 = malloc(n * sizeof(int));

        // Fill arrays with random data
        for (int i = 0; i < n; i++) {
            arr1[i] = rand() % 10000;
            arr2[i] = arr1[i];
        }

        struct timespec start, end;

        // Test bubble sort
        clock_gettime(CLOCK_MONOTONIC, &start);
        bubble_sort(arr1, n);
        clock_gettime(CLOCK_MONOTONIC, &end);
        double bubble_time = get_time_diff(start, end);

        // Test quick sort
        clock_gettime(CLOCK_MONOTONIC, &start);
        quick_sort(arr2, 0, n - 1);
        clock_gettime(CLOCK_MONOTONIC, &end);
        double quick_time = get_time_diff(start, end);

        printf("Array size: %d\n", n);
        printf("Bubble sort time: %.6f seconds\n", bubble_time);
        printf("Quick sort time: %.6f seconds\n", quick_time);
        printf("Speedup: %.2fx\n\n", bubble_time / quick_time);

        free(arr1);
        free(arr2);
    }
}

int main() {
    srand(time(NULL));
    test_algorithms();
    return 0;
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

#define MATRIX_SIZE 1000
#define CACHE_LINE_SIZE 64

// Matrix structure with proper alignment
typedef struct {
    int rows;
    int cols;
    int *data;
} matrix_t;

// Create matrix with specified alignment
matrix_t* create_matrix(int rows, int cols) {
    matrix_t *mat = malloc(sizeof(matrix_t));
    mat->rows = rows;
    mat->cols = cols;
    // Allocate aligned memory for better cache performance
    posix_memalign((void**)&mat->data, CACHE_LINE_SIZE, rows * cols * sizeof(int));
    return mat;
}

void free_matrix(matrix_t *mat) {
    free(mat->data);
    free(mat);
}

// Initialize matrix with random values
void init_matrix(matrix_t *mat) {
    for (int i = 0; i < mat->rows * mat->cols; i++) {
        mat->data[i] = rand() % 100;
    }
}

// Matrix multiplication - ijk order (poor cache performance)
void matrix_multiply_ijk(matrix_t *a, matrix_t *b, matrix_t *c) {
    for (int i = 0; i < a->rows; i++) {
        for (int j = 0; j < b->cols; j++) {
            c->data[i * c->cols + j] = 0;
            for (int k = 0; k < a->cols; k++) {
                c->data[i * c->cols + j] +=
```

```
                    a->data[i * a->cols + k] * b->data[k * b->cols + j];
            }
        }
    }
}

// Matrix multiplication - ikj order (better cache performance)
void matrix_multiply_ikj(matrix_t *a, matrix_t *b, matrix_t *c) {
    // Initialize result matrix
    memset(c->data, 0, c->rows * c->cols * sizeof(int));

    for (int i = 0; i < a->rows; i++) {
        for (int k = 0; k < a->cols; k++) {
            int temp = a->data[i * a->cols + k];
            for (int j = 0; j < b->cols; j++) {
                c->data[i * c->cols + j] += temp * b->data[k * b->cols + j];
            }
        }
    }
}

// Performance comparison
void compare_matrix_multiplication() {
    matrix_t *a = create_matrix(MATRIX_SIZE, MATRIX_SIZE);
    matrix_t *b = create_matrix(MATRIX_SIZE, MATRIX_SIZE);
    matrix_t *c1 = create_matrix(MATRIX_SIZE, MATRIX_SIZE);
    matrix_t *c2 = create_matrix(MATRIX_SIZE, MATRIX_SIZE);

    init_matrix(a);
    init_matrix(b);

    struct timespec start, end;

    // Test ijk order
    clock_gettime(CLOCK_MONOTONIC, &start);
    matrix_multiply_ijk(a, b, c1);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ijk = get_time_diff(start, end);

    // Test ikj order
    clock_gettime(CLOCK_MONOTONIC, &start);
    matrix_multiply_ikj(a, b, c2);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ikj = get_time_diff(start, end);

    printf("Matrix multiplication performance comparison:\n");
    printf("Matrix size: %dx%d\n", MATRIX_SIZE, MATRIX_SIZE);
```

```
    printf("ijk order time: %.6f seconds\n", time_ijk);
    printf("ikj order time: %.6f seconds\n", time_ikj);
    printf("Performance improvement: %.2fx\n", time_ijk / time_ikj);

    // Verify results are the same
    int diff_count = 0;
    for (int i = 0; i < c1->rows * c1->cols; i++) {
        if (c1->data[i] != c2->data[i]) {
            diff_count++;
        }
    }
    printf("Results match: %s\n", diff_count == 0 ? "Yes" : "No");

    free_matrix(a);
    free_matrix(b);
    free_matrix(c1);
    free_matrix(c2);
}

double get_time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
}

int main() {
    srand(time(NULL));
    compare_matrix_multiplication();
    return 0;
}
```

Common Pitfalls to Avoid:

1. **Premature optimization**: Always profile before optimizing
2. **Algorithmic vs implementation optimization**: Focus on algorithmic improvements first
3. **Micro-optimizations**: Avoid micro-optimizations that hurt readability without significant gains
4. **Platform-specific optimizations**: Consider portability when optimizing
5. **Over-optimization**: Balance performance with maintainability and correctness

Best Practices:

1. **Profile-driven optimization**: Use profiling tools to identify real bottlenecks
2. **Algorithmic improvements**: Focus on better algorithms before low-level optimizations
3. **Memory locality**: Optimize for cache-friendly access patterns
4. **Compiler assistance**: Use compiler optimization flags and hints effectively
5. **Measurement and validation**: Always measure performance improvements and verify correctness

Complete these exercises to solidify your understanding of performance optimization in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Testing

Module 14: Testing and Quality Assurance Exercises

Exercise 1: Unit Testing Framework Implementation

Write a program that implements a simple unit testing framework: - Create basic assertion functions for different data types - Implement a test runner that executes test cases - Develop a reporting system for test results - Show how to organize test cases into test suites - Demonstrate proper test fixture setup and teardown

Requirements: - Implement assertions for common data types (int, float, string, etc.) - Create a simple test registration and execution system - Include detailed reporting with pass/fail counts and timing - Show how to handle expected failures and skip conditions - Provide clear examples of test organization

Exercise 2: Mocking and Stubbing Techniques

Create a program that demonstrates mocking and stubbing techniques: - Implement simple mock objects for function dependencies - Show how to replace real implementations with test doubles - Demonstrate proper use of function pointers for mocking - Implement a simple mocking framework - Show how to verify mock interactions

Requirements: - Include examples of state-based and behavior-based testing - Demonstrate proper isolation of units under test - Show how to handle complex dependencies with mocks - Implement proper mock verification mechanisms - Provide clear documentation of mocking techniques

Exercise 3: Integration Testing Implementation

Develop a program that implements integration testing: - Create tests that verify interactions between multiple components - Show how to set up and tear down integration test environments - Demonstrate proper test data management for integration tests - Implement tests for API interactions and data flow - Show how to handle external dependencies in integration tests

Requirements: - Include examples of component integration testing - Demonstrate proper test environment setup and cleanup - Show how to handle test data dependencies and isolation - Implement tests for complex system interactions - Provide clear documentation of integration testing strategies

Exercise 4: Code Coverage Analysis

Write a program that demonstrates code coverage analysis: - Implement instrumentation to track code execution - Create a coverage reporting system - Show how to measure different types of coverage (line, branch, path) - Demonstrate proper coverage goal setting and tracking - Show how to identify untested code paths

Requirements: - Include examples of coverage instrumentation techniques - Implement proper coverage measurement and reporting - Show how to handle complex control flow in coverage analysis - Demonstrate coverage-guided test development - Provide clear documentation of coverage analysis results

Exercise 5: Static Analysis and Linting

Create a program that demonstrates static analysis techniques: - Implement simple static analysis checks for common issues - Show how to integrate static analysis tools into the build process - Demonstrate proper configuration of static analysis rules - Implement custom static analysis checks - Show how to handle static analysis findings

Requirements: - Include examples of common static analysis checks - Demonstrate proper integration with development workflow - Show how to configure and customize analysis rules - Implement proper handling of false positives - Provide clear documentation of static analysis results

Exercise 6: Fuzz Testing Implementation

Write a program that implements fuzz testing techniques: - Create a simple fuzz testing framework - Implement input generation strategies for fuzz testing - Show how to monitor for crashes and unexpected behavior - Demonstrate proper fuzz test case reduction - Show how to integrate fuzz testing into the testing pipeline

Requirements: - Include examples of different fuzzing strategies - Implement proper crash detection and reporting - Show how to generate meaningful test inputs - Demonstrate proper handling of fuzz test results - Provide clear documentation of fuzz testing techniques

Exercise 7: Performance and Stress Testing

Create a program that implements performance and stress testing: - Develop tests that verify performance requirements - Implement stress tests that push system limits - Show how to measure and report performance metrics - Demonstrate proper test environment setup for performance testing - Show how to identify performance bottlenecks

Requirements: - Include examples of performance benchmarking - Implement proper metrics collection and reporting - Show how to handle performance test data analysis - Demonstrate proper test environment configuration - Provide clear documentation of performance test results

Exercise 8: Comprehensive Testing Strategy

Design a complete testing strategy that integrates all testing techniques: - Implement a comprehensive test suite covering all testing types - Create a testing pipeline that executes different test categories - Demonstrate proper test prioritization and execution ordering - Show how to maintain test quality and reliability - Provide clear documentation and reporting

Requirements: - Use modular design with clear test organization - Include comprehensive test coverage across all areas - Demonstrate proper test maintenance and evolution - Implement robust error handling throughout the testing system - Provide clear examples and test documentation

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

// Simple unit testing framework

// Test result structure
typedef struct {
    int passed;
    int failed;
    int skipped;
    double total_time;
} test_result_t;

// Global test result tracker
static test_result_t g_test_result = {0};

// Assertion functions
#define ASSERT_TRUE(condition) \
    do { \
        if (!(condition)) { \
            printf("ASSERT_TRUE failed at %s:%d\n", __FILE__, __LINE__); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

#define ASSERT_FALSE(condition) \
    do { \
```

```
        if (condition) { \
            printf("ASSERT_FALSE failed at %s:%d\n", __FILE__, __LINE__); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

#define ASSERT_EQUAL(expected, actual) \
    do { \
        if ((expected) != (actual)) { \
            printf("ASSERT_EQUAL failed at %s:%d: expected %d, got %d\n", \
                   __FILE__, __LINE__, (expected), (actual)); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

#define ASSERT_STRING_EQUAL(expected, actual) \
    do { \
        if (strcmp((expected), (actual)) != 0) { \
            printf("ASSERT_STRING_EQUAL failed at %s:%d: expected '%s', got '%s'\n", \
                   __FILE__, __LINE__, (expected), (actual)); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

// Test function type
typedef void (*test_func_t)(void);

// Test case structure
typedef struct {
    const char *name;
    test_func_t function;
} test_case_t;

// Example functions to test
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}
```

```
const char* get_hello_message(void) {
    return "Hello, World!";
}

// Test cases
void test_add_positive(void) {
    ASSERT_EQUAL(5, add(2, 3));
}

void test_add_negative(void) {
    ASSERT_EQUAL(-1, add(2, -3));
}

void test_subtract(void) {
    ASSERT_EQUAL(2, subtract(5, 3));
}

void test_string_message(void) {
    ASSERT_STRING_EQUAL("Hello, World!", get_hello_message());
}

// Test suite
test_case_t test_suite[] = {
    {"test_add_positive", test_add_positive},
    {"test_add_negative", test_add_negative},
    {"test_subtract", test_subtract},
    {"test_string_message", test_string_message},
    {NULL, NULL}  // Sentinel
};

// Test runner
void run_tests(void) {
    printf("Running tests...\n");

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; test_suite[i].name != NULL; i++) {
        printf("Running %s... ", test_suite[i].name);

        // Reset test counters for each test
        int initial_passed = g_test_result.passed;
        int initial_failed = g_test_result.failed;

        test_suite[i].function();
```

```
        if (g_test_result.failed > initial_failed) {
            printf("FAILED\n");
        } else {
            printf("PASSED\n");
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    g_test_result.total_time = (end.tv_sec - start.tv_sec) +
                               (end.tv_nsec - start.tv_nsec) / 1e9;

    // Print summary
    printf("\nTest Results:\n");
    printf("Passed: %d\n", g_test_result.passed);
    printf("Failed: %d\n", g_test_result.failed);
    printf("Skipped: %d\n", g_test_result.skipped);
    printf("Total time: %.6f seconds\n", g_test_result.total_time);

    if (g_test_result.failed == 0) {
        printf("All tests passed!\n");
    } else {
        printf("Some tests failed!\n");
    }
}

int main() {
    run_tests();
    return g_test_result.failed > 0 ? 1 : 0;
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Mocking framework example

// Real database interface
typedef struct {
    int (*connect)(const char *host, int port);
    int (*query)(const char *sql, char **result);
    void (*disconnect)(void);
} database_interface_t;
```

```c
// Real database implementation
static int real_db_connect(const char *host, int port) {
    printf("Connecting to database at %s:%d\n", host, port);
    return 1; // Success
}

static int real_db_query(const char *sql, char **result) {
    printf("Executing query: %s\n", sql);
    *result = strdup("Mock query result");
    return 1; // Success
}

static void real_db_disconnect(void) {
    printf("Disconnecting from database\n");
}

database_interface_t real_database = {
    .connect = real_db_connect,
    .query = real_db_query,
    .disconnect = real_db_disconnect
};

// Mock database implementation
static int mock_connect_calls = 0;
static int mock_query_calls = 0;
static int mock_disconnect_calls = 0;
static int mock_connect_result = 1;
static int mock_query_result = 1;

static int mock_db_connect(const char *host, int port) {
    mock_connect_calls++;
    printf("Mock connect called (%d times)\n", mock_connect_calls);
    return mock_connect_result;
}

static int mock_db_query(const char *sql, char **result) {
    mock_query_calls++;
    printf("Mock query called (%d times): %s\n", mock_query_calls, sql);
    *result = strdup("Mock result");
    return mock_query_result;
}

static void mock_db_disconnect(void) {
    mock_disconnect_calls++;
    printf("Mock disconnect called (%d times)\n", mock_disconnect_calls);
}
```

```
database_interface_t mock_database = {
    .connect = mock_db_connect,
    .query = mock_db_query,
    .disconnect = mock_db_disconnect
};

// Function under test
int fetch_user_data(database_interface_t *db, const char *user_id, char **data) {
    if (!db->connect("localhost", 5432)) {
        return 0;
    }

    char sql[256];
    snprintf(sql, sizeof(sql), "SELECT * FROM users WHERE id = '%s'", user_id);

    int result = db->query(sql, data);
    db->disconnect();

    return result;
}

// Test with mock
void test_fetch_user_data_success(void) {
    // Reset mock counters
    mock_connect_calls = 0;
    mock_query_calls = 0;
    mock_disconnect_calls = 0;
    mock_connect_result = 1;
    mock_query_result = 1;

    char *data = NULL;
    int result = fetch_user_data(&mock_database, "123", &data);

    // Verify results
    if (result != 1) {
        printf("Test failed: Expected success, got failure\n");
        return;
    }

    if (mock_connect_calls != 1) {
        printf("Test failed: Expected 1 connect call, got %d\n", mock_connect_calls);
        return;
    }

    if (mock_query_calls != 1) {
        printf("Test failed: Expected 1 query call, got %d\n", mock_query_calls);
        return;
```

```
    }

    if (mock_disconnect_calls != 1) {
        printf("Test failed: Expected 1 disconnect call, got %d\n", mock_disconnect_calls);
        return;
    }

    printf("Test passed: All mock interactions verified\n");
    free(data);
}

void test_fetch_user_data_connection_failure(void) {
    // Reset mock counters
    mock_connect_calls = 0;
    mock_query_calls = 0;
    mock_disconnect_calls = 0;
    mock_connect_result = 0; // Simulate connection failure

    char *data = NULL;
    int result = fetch_user_data(&mock_database, "123", &data);

    // Verify results
    if (result != 0) {
        printf("Test failed: Expected failure, got success\n");
        return;
    }

    if (mock_connect_calls != 1) {
        printf("Test failed: Expected 1 connect call, got %d\n", mock_connect_calls);
        return;
    }

    // Should not call query or disconnect on connection failure
    if (mock_query_calls != 0) {
        printf("Test failed: Expected 0 query calls, got %d\n", mock_query_calls);
        return;
    }

    if (mock_disconnect_calls != 0) {
        printf("Test failed: Expected 0 disconnect calls, got %d\n", mock_disconnect_calls);
        return;
    }

    printf("Test passed: Connection failure handled correctly\n");
}

int main() {
```

```
    printf("Testing with mock database...\n");
    test_fetch_user_data_success();
    test_fetch_user_data_connection_failure();

    printf("\nFor comparison, running with real database:\n");
    char *data = NULL;
    fetch_user_data(&real_database, "123", &data);
    free(data);

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Testing implementation details**: Focus on behavior rather than implementation
2. **Insufficient test coverage**: Ensure adequate coverage of edge cases and error conditions
3. **Flaky tests**: Write deterministic tests that produce consistent results
4. **Over-mocking**: Don't mock everything; focus on complex or external dependencies
5. **Ignoring test maintenance**: Keep tests up-to-date with code changes

Best Practices:

1. **Test pyramid**: Follow the testing pyramid (unit > integration > end-to-end)
2. **Descriptive test names**: Use clear, descriptive names for test cases
3. **Isolated tests**: Ensure tests don't depend on each other or shared state
4. **Fast feedback**: Keep tests fast to encourage frequent execution
5. **Continuous testing**: Integrate testing into the development workflow

Complete these exercises to solidify your understanding of testing and quality assurance in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Advanced

Module 15: Advanced Topics Exercises

Exercise 1: Metaprogramming with Macros

Write a program that demonstrates advanced macro techniques: - Implement complex macros with conditional compilation - Create macros that generate code based on parameters - Demonstrate proper use of token concatenation and stringification - Show how to implement compile-time assertions - Implement a simple domain-specific language using macros

Requirements: - Include examples of advanced preprocessor features - Demonstrate proper macro hygiene and scoping - Show how to handle macro expansion edge cases - Implement useful utility macros for common patterns - Provide clear documentation of macro behavior

Exercise 2: Generic Programming Techniques

Create a program that demonstrates generic programming in C: - Implement type-generic data structures using void pointers - Show how to use function pointers for polymorphic behavior - Demonstrate proper type safety in generic code - Implement a simple container library with generic operations - Show how to handle memory management in generic code

Requirements: - Include examples of generic containers (lists, trees, etc.) - Demonstrate proper error handling in generic functions - Show how to implement type-safe generic interfaces - Implement proper memory management for generic data - Provide clear documentation of generic programming techniques

Exercise 3: Coroutines and Cooperative Multitasking

Develop a program that implements coroutines and cooperative multitasking: - Create a simple coroutine framework - Implement yield and resume mechanisms - Show how to manage coroutine state and context - Demonstrate proper coroutine scheduling - Implement a simple cooperative multitasking system

Requirements: - Include examples of coroutine usage patterns - Demonstrate proper context switching mechanisms - Show how to handle coroutine lifecycle management - Implement proper error handling in coroutine systems - Provide clear documentation of coroutine concepts

Exercise 4: Reflection and Introspection

Write a program that demonstrates reflection and introspection techniques: - Implement a simple type system with runtime type information - Create functions for inspecting data structure layouts - Show how to implement serialization and deserialization - Demonstrate proper metadata generation and usage - Implement a simple object model with reflection capabilities

Requirements: - Include examples of runtime type inspection - Demonstrate proper serialization techniques - Show how to implement metadata-driven programming - Implement proper error handling for reflection operations - Provide clear documentation of reflection mechanisms

Exercise 5: Template Metaprogramming Simulation

Create a program that simulates template metaprogramming techniques: - Implement compile-time computation and type manipulation - Show how to use preprocessor metaprogramming - Demonstrate proper use of constexpr-like constructs - Implement a simple compile-time data structure library - Show how to optimize code through compile-time evaluation

Requirements: - Include examples of compile-time algorithms - Demonstrate proper use of preprocessor for metaprogramming - Show how to implement type traits and compile-time checks - Implement proper error reporting for metaprogramming errors - Provide clear documentation of metaprogramming techniques

Exercise 6: Memory Model and Concurrency

Write a program that demonstrates advanced memory model and concurrency concepts: - Implement lock-free data structures - Show how to use atomic operations effectively - Demonstrate proper memory ordering and synchronization - Implement a simple memory model simulator - Show how to handle race conditions and data races

Requirements: - Include examples of atomic operations and memory barriers - Demonstrate proper use of concurrent data structures - Show how to implement lock-free algorithms - Implement proper testing for concurrent code - Provide clear documentation of memory model concepts

Exercise 7: Language Interoperability

Create a program that demonstrates language interoperability: - Implement a C library that can be called from other languages - Show how to interface with C++ code - Demonstrate proper use of foreign function interfaces - Implement a simple binding generator (bonus) - Show how to handle different calling conventions

Requirements: - Include examples of C API design for language interoperability - Demonstrate proper handling of data type conversions - Show how to implement proper error handling across language boundaries - Implement proper resource management for interop scenarios - Provide clear documentation of interoperability techniques

Exercise 8: Comprehensive Advanced Application

Design a complete application that integrates all advanced concepts: - Implement a sophisticated system using multiple advanced techniques - Create a domain-specific language or framework - Demonstrate proper architecture and design patterns - Include comprehensive testing and validation - Provide clear documentation and examples

Requirements: - Use modular design with clear separation of concerns - Include comprehensive error handling and recovery mechanisms - Demonstrate proper resource management throughout the application - Implement robust testing and validation procedures - Provide clear examples and documentation

Solutions and Tips

Exercise 1 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Advanced macro techniques

// Stringification macro
#define STRINGIFY(x) #x
#define TOSTRING(x) STRINGIFY(x)

// Token concatenation macros
#define CONCAT(a, b) a##b
#define PASTE(a, b) CONCAT(a, b)

// Conditional compilation based on parameters
#define DEBUG_LEVEL 2

#if DEBUG_LEVEL >= 2
    #define DEBUG_PRINT(fmt, ...) \
        fprintf(stderr, "[DEBUG] %s:%d: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)
#elif DEBUG_LEVEL == 1
    #define DEBUG_PRINT(fmt, ...) \
        fprintf(stderr, "[INFO] " fmt "\n", ##__VA_ARGS__)
#else
    #define DEBUG_PRINT(fmt, ...) do {} while(0)
#endif

// Compile-time assertion
#define STATIC_ASSERT(condition, message) \
```

```
    typedef char PASTE(static_assert_, __LINE__)[(condition) ? 1 : -1]

// Macro for generating function declarations
#define DECLARE_FUNCTION(return_type, name, ...) \
    return_type name(__VA_ARGS__);

// Macro for generating function definitions with logging
#define DEFINE_FUNCTION(return_type, name, params, body) \
    return_type name params { \
        DEBUG_PRINT("Entering function %s", TOSTRING(name)); \
        body \
        DEBUG_PRINT("Exiting function %s", TOSTRING(name)); \
    }

// Macro for creating a simple DSL for defining structures
#define BEGIN_STRUCT(name) typedef struct {
#define FIELD(type, name) type name;
#define END_STRUCT(name) } name##_t;

// Example usage of the DSL
BEGIN_STRUCT(person)
    FIELD(char*, name)
    FIELD(int, age)
    FIELD(float, height)
END_STRUCT(person)

// Generic max macro
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MAX3(a, b, c) MAX(MAX(a, b), c)

// Type-generic print macro
#define PRINT_VALUE(x) _Generic((x), \
    int: printf("int: %d\n", (x)), \
    float: printf("float: %f\n", (x)), \
    double: printf("double: %f\n", (x)), \
    char*: printf("string: %s\n", (x)), \
    default: printf("unknown type\n"))

// Compile-time assertions
STATIC_ASSERT(sizeof(int) >= 4, "int must be at least 4 bytes");
STATIC_ASSERT(sizeof(void*) == 8 || sizeof(void*) == 4, "Pointer size must be 4 or 8 bytes")

int main() {
    // Test stringification
    printf("File: %s\n", __FILE__);
    printf("Line: %d\n", __LINE__);
    printf("Function: %s\n", TOSTRING(main));
```

```
    // Test debug printing
    DEBUG_PRINT("This is a debug message with value %d", 42);

    // Test generic max
    int max_int = MAX3(10, 20, 15);
    printf("Max of 10, 20, 15: %d\n", max_int);

    // Test type-generic print
    int a = 42;
    float b = 3.14f;
    char *c = "Hello";
    PRINT_VALUE(a);
    PRINT_VALUE(b);
    PRINT_VALUE(c);

    // Test DSL-generated structure
    person_t person = {"Alice", 30, 5.5f};
    printf("Person: %s, Age: %d, Height: %.1f\n",
           person.name, person.age, person.height);

    return 0;
}
```

Exercise 2 Solution Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Generic programming techniques

// Generic list node
typedef struct list_node {
    void *data;
    struct list_node *next;
} list_node_t;

// Generic list structure
typedef struct {
    list_node_t *head;
    size_t data_size;
    int (*compare)(const void *a, const void *b);
    void (*print)(const void *data);
} generic_list_t;
```

```
// Create a new generic list
generic_list_t* list_create(size_t data_size,
                            int (*compare)(const void *a, const void *b),
                            void (*print)(const void *data)) {
    generic_list_t *list = malloc(sizeof(generic_list_t));
    if (list) {
        list->head = NULL;
        list->data_size = data_size;
        list->compare = compare;
        list->print = print;
    }
    return list;
}

// Add an element to the list
int list_add(generic_list_t *list, const void *data) {
    list_node_t *node = malloc(sizeof(list_node_t));
    if (!node) return 0;

    node->data = malloc(list->data_size);
    if (!node->data) {
        free(node);
        return 0;
    }

    memcpy(node->data, data, list->data_size);
    node->next = list->head;
    list->head = node;
    return 1;
}

// Find an element in the list
void* list_find(generic_list_t *list, const void *key) {
    for (list_node_t *node = list->head; node; node = node->next) {
        if (list->compare(node->data, key) == 0) {
            return node->data;
        }
    }
    return NULL;
}

// Print all elements in the list
void list_print(generic_list_t *list) {
    for (list_node_t *node = list->head; node; node = node->next) {
        list->print(node->data);
    }
}
```

```c
// Free the list
void list_destroy(generic_list_t *list) {
    list_node_t *node = list->head;
    while (node) {
        list_node_t *next = node->next;
        free(node->data);
        free(node);
        node = next;
    }
    free(list);
}

// Comparison functions for different types
int int_compare(const void *a, const void *b) {
    int ia = *(const int*)a;
    int ib = *(const int*)b;
    return (ia > ib) - (ia < ib);
}

int string_compare(const void *a, const void *b) {
    return strcmp(*(const char**)a, *(const char**)b);
}

// Print functions for different types
void int_print(const void *data) {
    printf("%d ", *(const int*)data);
}

void string_print(const void *data) {
    printf("%s ", *(const char**)data);
}

int main() {
    // Test with integers
    printf("Testing integer list:\n");
    generic_list_t *int_list = list_create(sizeof(int), int_compare, int_print);

    int values[] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++) {
        list_add(int_list, &values[i]);
    }

    printf("List contents: ");
    list_print(int_list);
    printf("\n");
```

```
    int search_key = 30;
    int *found = (int*)list_find(int_list, &search_key);
    if (found) {
        printf("Found: %d\n", *found);
    } else {
        printf("Not found: %d\n", search_key);
    }

    list_destroy(int_list);

    // Test with strings
    printf("\nTesting string list:\n");
    generic_list_t *string_list = list_create(sizeof(char*), string_compare, string_print);

    const char *strings[] = {"apple", "banana", "cherry", "date", "elderberry"};
    for (int i = 0; i < 5; i++) {
        list_add(string_list, &strings[i]);
    }

    printf("List contents: ");
    list_print(string_list);
    printf("\n");

    const char *search_string = "cherry";
    char **found_string = (char**)list_find(string_list, &search_string);
    if (found_string) {
        printf("Found: %s\n", *found_string);
    } else {
        printf("Not found: %s\n", search_string);
    }

    list_destroy(string_list);

    return 0;
}
```

Common Pitfalls to Avoid:

1. **Macro complexity**: Avoid overly complex macros that hurt readability
2. **Type safety**: Maintain type safety when using generic programming techniques
3. **Memory management**: Properly handle memory in generic and advanced code
4. **Portability**: Consider platform differences in advanced features
5. **Debugging difficulty**: Advanced techniques can make debugging more challenging

Best Practices:

1. **Clear documentation**: Document advanced techniques thoroughly
2. **Gradual complexity**: Introduce advanced concepts incrementally
3. **Error handling**: Implement robust error handling for advanced features
4. **Testing**: Include comprehensive tests for advanced functionality
5. **Maintainability**: Balance advanced features with code maintainability

Complete these exercises to solidify your understanding of advanced topics in C. Each exercise builds upon the previous ones, gradually increasing in complexity.

Projects

Advanced

Advanced Projects

These projects are designed for experienced C programmers who want to tackle complex challenges involving system programming, networking, multithreading, and advanced data structures.

Project 1: Concurrent Web Server

Description

Create a multithreaded web server that can handle multiple client requests concurrently and serve static files.

Learning Objectives

- Network programming with sockets
- Multithreading and synchronization
- HTTP protocol implementation
- File I/O operations
- Error handling and logging

Requirements

1. Handle multiple concurrent client connections
2. Serve static HTML, CSS, JavaScript, and image files
3. Implement basic HTTP/1.1 protocol
4. Support GET and POST requests
5. Handle 404 errors gracefully
6. Log requests and responses
7. Support basic authentication (optional)
8. Implement connection pooling

Implementation Steps

1. Set up socket programming for server
2. Implement HTTP request parsing
3. Create thread pool for handling connections
4. Add file serving functionality
5. Implement error handling and logging

6. Add support for different HTTP methods
7. Create configuration system
8. Test with various clients

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <time.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <dirent.h>

#define PORT 8080
#define MAX_CLIENTS 100
#define BUFFER_SIZE 4096
#define THREAD_POOL_SIZE 10

typedef struct {
    int socket;
    struct sockaddr_in address;
} client_info_t;

typedef struct {
    pthread_t threads[THREAD_POOL_SIZE];
    int thread_count;
    pthread_mutex_t queue_mutex;
    pthread_cond_t queue_cond;
    client_info_t *queue;
    int queue_size;
    int queue_front;
    int queue_rear;
    int shutdown;
} thread_pool_t;

// Function prototypes
void init_thread_pool(thread_pool_t *pool);
void destroy_thread_pool(thread_pool_t *pool);
void *worker_thread(void *arg);
void enqueue_client(thread_pool_t *pool, client_info_t client);
```

```c
client_info_t dequeue_client(thread_pool_t *pool);
void handle_client(client_info_t client);
void send_response(int client_socket, const char *status, const char *content_type, const ch
void send_file(int client_socket, const char *filepath);
void log_request(const char *method, const char *path, int status_code);
int parse_http_request(const char *request, char *method, char *path, char *version);
char* get_content_type(const char *filepath);
void *server_thread(void *arg);

int main() {
    int server_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    thread_pool_t pool;

    // Initialize thread pool
    init_thread_pool(&pool);

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Set socket options
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
                   &opt, sizeof(opt))) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 10) < 0) {
        perror("listen failed");
        exit(EXIT_FAILURE);
    }
```

```
    printf("Server listening on port %d\n", PORT);

    // Main server loop
    while (1) {
        client_info_t client;
        if ((client.socket = accept(server_fd, (struct sockaddr *)&client.address,
                                    (socklen_t*)&addrlen)) < 0) {
            perror("accept failed");
            continue;
        }

        // Add client to thread pool queue
        enqueue_client(&pool, client);
    }

    // Cleanup
    destroy_thread_pool(&pool);
    close(server_fd);
    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Thread safety issues with shared resources
2. Memory leaks from improper resource management
3. Not handling partial reads/writes correctly
4. Buffer overflows in request parsing
5. Not properly closing socket connections
6. Race conditions in thread pool implementation

Best Practices

1. Use proper synchronization primitives (mutexes, condition variables)
2. Always validate input and handle errors gracefully
3. Implement proper resource cleanup
4. Use connection pooling for efficiency
5. Log important events for debugging
6. Handle edge cases in HTTP protocol implementation
7. Use appropriate buffer sizes to prevent overflows

Project 2: Memory Allocator

Description

Implement a custom memory allocator that provides dynamic memory management similar to malloc/free.

Learning Objectives

- Low-level memory management
- Pointer arithmetic
- Data structure design
- Performance optimization
- System programming concepts

Requirements

1. Implement malloc, free, calloc, and realloc functions
2. Handle memory fragmentation
3. Support different allocation strategies (first-fit, best-fit, worst-fit)
4. Provide memory usage statistics
5. Handle alignment requirements
6. Detect memory leaks and corruption (optional)
7. Support multi-threading (optional)
8. Implement garbage collection (optional)

Implementation Steps

1. Design internal data structures for memory blocks
2. Implement basic allocation and deallocation
3. Add different allocation strategies
4. Handle memory fragmentation
5. Implement memory statistics
6. Add alignment support
7. Test with various allocation patterns
8. Optimize for performance

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
```

```c
#define ALIGNMENT 8
#define ALIGN(size) (((size) + (ALIGNMENT-1)) & ~(ALIGNMENT-1))
#define BLOCK_SIZE sizeof(block_header_t)

typedef enum {
    ALLOC_FIRST_FIT,
    ALLOC_BEST_FIT,
    ALLOC_WORST_FIT
} alloc_strategy_t;

typedef struct block_header {
    size_t size;
    int free;
    struct block_header *next;
    struct block_header *prev;
} block_header_t;

typedef struct {
    block_header_t *head;
    size_t total_size;
    size_t used_size;
    size_t free_size;
    alloc_strategy_t strategy;
    pthread_mutex_t mutex;
} memory_pool_t;

static memory_pool_t g_pool = {0};

// Function prototypes
void init_memory_pool(size_t size, alloc_strategy_t strategy);
void destroy_memory_pool();
void *my_malloc(size_t size);
void my_free(void *ptr);
void *my_calloc(size_t nmemb, size_t size);
void *my_realloc(void *ptr, size_t size);
void print_memory_stats();
block_header_t *find_free_block(size_t size);
block_header_t *split_block(block_header_t *block, size_t size);
void coalesce_blocks();
size_t align_size(size_t size);

void *my_malloc(size_t size) {
    if (size == 0) return NULL;

    pthread_mutex_lock(&g_pool.mutex);
```

```
    size_t aligned_size = align_size(size);
    block_header_t *block = find_free_block(aligned_size);

    if (block) {
        // Split block if it's larger than needed
        if (block->size > aligned_size + BLOCK_SIZE + ALIGNMENT) {
            block = split_block(block, aligned_size);
        }

        block->free = 0;
        g_pool.used_size += block->size;
        g_pool.free_size -= block->size;

        pthread_mutex_unlock(&g_pool.mutex);
        return (void*)(block + 1);
    }

    pthread_mutex_unlock(&g_pool.mutex);
    return NULL;  // Out of memory
}

void my_free(void *ptr) {
    if (!ptr) return;

    pthread_mutex_lock(&g_pool.mutex);

    block_header_t *block = (block_header_t*)ptr - 1;
    block->free = 1;
    g_pool.used_size -= block->size;
    g_pool.free_size += block->size;

    // Coalesce adjacent free blocks
    coalesce_blocks();

    pthread_mutex_unlock(&g_pool.mutex);
}

// Implement all other functions here
```

Common Pitfalls to Avoid

1. Memory leaks from improper block management
2. Buffer overflows in block headers
3. Not handling alignment requirements
4. Incorrect coalescing of free blocks
5. Thread safety issues in multi-threaded version

6. Not validating pointers in free() function

Best Practices

1. Use proper data structures for efficient block management
2. Handle edge cases in allocation and deallocation
3. Implement proper alignment for performance
4. Use synchronization for thread safety
5. Validate all input pointers
6. Provide detailed memory statistics
7. Test with various allocation patterns

Project 3: Real-time Data Processing System

Description

Create a system that processes real-time data streams with configurable filters and transformations.

Learning Objectives

- Multithreading and inter-thread communication
- Data pipeline design
- Performance optimization
- Configuration management
- Error handling in real-time systems

Requirements

1. Process multiple data streams concurrently
2. Apply configurable filters and transformations
3. Support real-time data ingestion
4. Provide monitoring and statistics
5. Handle backpressure and buffering
6. Support plugin architecture for custom processors
7. Implement fault tolerance
8. Provide REST API for configuration (optional)

Implementation Steps

1. Design data pipeline architecture
2. Implement data source connectors
3. Create processor framework
4. Add filtering and transformation capabilities
5. Implement threading and synchronization
6. Add monitoring and statistics
7. Handle backpressure and buffering
8. Test with real-world data scenarios

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <sys/queue.h>

#define MAX_PROCESSORS 100
#define BUFFER_SIZE 1024
#define MAX_PIPELINE_DEPTH 10

typedef struct {
    char *data;
    size_t size;
    time_t timestamp;
    int priority;
} data_packet_t;

typedef enum {
    FILTER_TYPE_NONE,
    FILTER_TYPE_REGEX,
    FILTER_TYPE_RANGE,
    FILTER_TYPE_CUSTOM
} filter_type_t;

typedef struct {
    filter_type_t type;
    void *config;
    int (*apply)(data_packet_t *packet, void *config);
} filter_t;

typedef struct {
```

```
    char name[50];
    int (*process)(data_packet_t *input, data_packet_t *output, void *config);
    void *config;
    filter_t *filters;
    int filter_count;
} processor_t;

typedef struct pipeline_node {
    processor_t *processor;
    struct pipeline_node *next;
} pipeline_node_t;

typedef struct {
    pipeline_node_t *head;
    int depth;
    pthread_mutex_t mutex;
    pthread_cond_t data_available;
    data_packet_t buffer[BUFFER_SIZE];
    int head_idx;
    int tail_idx;
    int count;
} pipeline_t;

typedef struct {
    char source_name[50];
    pthread_t thread;
    int running;
    pipeline_t *pipeline;
    void *source_config;
    int (*read_data)(data_packet_t *packet, void *config);
} data_source_t;

// Function prototypes
pipeline_t* create_pipeline();
void destroy_pipeline(pipeline_t *pipeline);
int add_processor(pipeline_t *pipeline, processor_t *processor);
int remove_processor(pipeline_t *pipeline, const char *name);
void process_data(pipeline_t *pipeline, data_packet_t *input);
data_source_t* create_data_source(const char *name,
                                  int (*read_func)(data_packet_t*, void*),
                                  void *config);
void start_data_source(data_source_t *source, pipeline_t *pipeline);
void stop_data_source(data_source_t *source);
void *data_source_thread(void *arg);
int apply_filters(data_packet_t *packet, filter_t *filters, int filter_count);
void monitor_pipeline(pipeline_t *pipeline);
```

```c
int main() {
    // Create pipeline
    pipeline_t *pipeline = create_pipeline();
    if (!pipeline) {
        fprintf(stderr, "Failed to create pipeline\n");
        return 1;
    }

    // Add processors to pipeline
    // ... add processors ...

    // Create data sources
    data_source_t *source1 = create_data_source("sensor1", read_sensor_data, NULL);
    data_source_t *source2 = create_data_source("network", read_network_data, NULL);

    // Start data sources
    start_data_source(source1, pipeline);
    start_data_source(source2, pipeline);

    // Monitor pipeline
    while (1) {
        monitor_pipeline(pipeline);
        sleep(5);
    }

    // Cleanup
    stop_data_source(source1);
    stop_data_source(source2);
    destroy_pipeline(pipeline);

    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Deadlocks in threading implementation
2. Buffer overflows in data packets
3. Not handling backpressure properly
4. Memory leaks in dynamic data structures
5. Race conditions in shared data access
6. Not validating data packet integrity

Best Practices

1. Use proper synchronization primitives
2. Implement efficient buffering strategies
3. Handle backpressure with appropriate mechanisms
4. Validate all data packets
5. Provide detailed monitoring and logging
6. Use configuration files for flexibility
7. Implement graceful shutdown procedures

Project 4: Embedded System Simulator

Description

Create a simulator for an embedded system with hardware components like GPIO, UART, timers, and interrupts.

Learning Objectives

- Embedded systems programming concepts
- Hardware simulation
- Interrupt handling
- Real-time constraints
- Memory-mapped I/O

Requirements

1. Simulate GPIO pins with input/output capabilities
2. Implement UART communication
3. Simulate timer functionality
4. Handle interrupts and interrupt priorities
5. Support memory-mapped I/O
6. Provide debugging interface
7. Simulate real-time constraints
8. Support multiple peripheral devices

Implementation Steps

1. Design hardware simulation architecture
2. Implement GPIO simulation
3. Add UART communication simulation
4. Create timer simulation
5. Implement interrupt controller
6. Add memory-mapped I/O support

7. Create debugging interface
8. Test with sample embedded applications

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define GPIO_PINS 32
#define UART_BUFFER_SIZE 256
#define MAX_INTERRUPTS 256

// Memory-mapped addresses
#define GPIO_BASE_ADDR    0x40000000
#define UART_BASE_ADDR    0x40001000
#define TIMER_BASE_ADDR   0x40002000
#define INTERRUPT_ADDR    0x40003000

typedef enum {
    GPIO_MODE_INPUT,
    GPIO_MODE_OUTPUT,
    GPIO_MODE_AF,
    GPIO_MODE_ANALOG
} gpio_mode_t;

typedef struct {
    unsigned int moder;     // Mode register
    unsigned int otyper;    // Output type register
    unsigned int ospeedr;   // Output speed register
    unsigned int pupdr;     // Pull-up/pull-down register
    unsigned int idr;       // Input data register
    unsigned int odr;       // Output data register
    unsigned int bsrr;      // Bit set/reset register
    unsigned int lckr;      // Configuration lock register
    unsigned int afr[2];    // Alternate function registers
} gpio_registers_t;

typedef struct {
    unsigned int sr;        // Status register
    unsigned int dr;        // Data register
    unsigned int brr;       // Baud rate register
    unsigned int cr1;       // Control register 1
```

```
    unsigned int cr2;       // Control register 2
    unsigned int cr3;       // Control register 3
    unsigned int gtpr;      // Guard time and prescaler register
} uart_registers_t;

typedef struct {
    unsigned int cr1;       // Control register 1
    unsigned int cr2;       // Control register 2
    unsigned int smcr;      // Slave mode control register
    unsigned int dier;      // DMA/interrupt enable register
    unsigned int sr;        // Status register
    unsigned int egr;       // Event generation register
    unsigned int ccmr1;     // Capture/compare mode register 1
    unsigned int ccmr2;     // Capture/compare mode register 2
    unsigned int ccer;      // Capture/compare enable register
    unsigned int cnt;       // Counter
    unsigned int psc;       // Prescaler
    unsigned int arr;       // Auto-reload register
} timer_registers_t;

typedef struct {
    unsigned int isr;       // Interrupt set register
    unsigned int ier;       // Interrupt enable register
    unsigned int icr;       // Interrupt clear register
    unsigned int ipr[8];    // Interrupt priority registers
} interrupt_registers_t;

typedef struct {
    gpio_registers_t gpio;
    uart_registers_t uart;
    timer_registers_t timer;
    interrupt_registers_t interrupt;
    unsigned char *memory;
    size_t memory_size;
    pthread_t cpu_thread;
    int running;
    unsigned long long cycles;
    time_t start_time;
} embedded_system_t;

// Function prototypes
embedded_system_t* create_embedded_system(size_t memory_size);
void destroy_embedded_system(embedded_system_t *system);
void* cpu_thread(void *arg);
unsigned int read_memory(embedded_system_t *system, unsigned int addr);
void write_memory(embedded_system_t *system, unsigned int addr, unsigned int value);
```

```
void gpio_write_pin(embedded_system_t *system, int pin, int value);
int gpio_read_pin(embedded_system_t *system, int pin);
void uart_transmit(embedded_system_t *system, unsigned char data);
unsigned char uart_receive(embedded_system_t *system);
void timer_start(embedded_system_t *system);
void timer_stop(embedded_system_t *system);
void interrupt_set(embedded_system_t *system, int irq);
void interrupt_clear(embedded_system_t *system, int irq);
void handle_interrupt(embedded_system_t *system, int irq);
void load_program(embedded_system_t *system, const char *filename);
void run_system(embedded_system_t *system);
void stop_system(embedded_system_t *system);

int main() {
    // Create embedded system
    embedded_system_t *system = create_embedded_system(64 * 1024);  // 64KB memory
    if (!system) {
        fprintf(stderr, "Failed to create embedded system\n");
        return 1;
    }

    // Load sample program
    load_program(system, "sample_program.bin");

    // Run system
    run_system(system);

    // Let it run for a while
    sleep(10);

    // Stop and cleanup
    stop_system(system);
    destroy_embedded_system(system);

    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Incorrect memory-mapped I/O implementation
2. Race conditions in interrupt handling
3. Not simulating real-time constraints properly
4. Memory leaks in dynamic structures
5. Buffer overflows in peripheral simulations

6. Incorrect bit manipulation in hardware registers

Best Practices

1. Use proper data structures for hardware registers
2. Implement accurate timing simulation
3. Handle interrupts with proper priority levels
4. Validate all memory accesses
5. Provide debugging interfaces for development
6. Use bit manipulation macros for register access
7. Implement proper error handling for hardware operations

Tips for Success

1. **Understand the Domain**: Research the problem domain thoroughly before implementation
2. **Design First**: Create detailed designs and architectures before coding
3. **Incremental Development**: Build and test components incrementally
4. **Performance Considerations**: Profile and optimize critical paths
5. **Error Handling**: Implement comprehensive error handling and recovery
6. **Testing**: Create thorough test cases including edge cases
7. **Documentation**: Document complex algorithms and architectures
8. **Security**: Consider security implications in networked applications
9. **Scalability**: Design for scalability from the beginning
10. **Maintainability**: Write clean, modular code that's easy to maintain

These advanced projects will challenge you to apply sophisticated C programming techniques while solving complex real-world problems. Focus on robustness, performance, and maintainability in your implementations.

Beginner

Beginner Projects

These projects are designed for those new to C programming. They focus on fundamental concepts and provide a gentle introduction to building complete applications.

Project 1: Personal Information Manager

Description

Create a simple console application that allows users to store and manage personal contact information.

Learning Objectives

- Basic input/output operations
- Working with strings and arrays
- Simple file I/O operations
- Menu-driven interface design

Requirements

1. Store contact information (name, phone number, email address)
2. Add new contacts
3. View all contacts
4. Search for contacts by name
5. Delete contacts
6. Save contacts to a file
7. Load contacts from a file
8. Implement a simple menu system

Implementation Steps

1. Design a structure to hold contact information
2. Implement functions for each menu option
3. Create a menu system using a loop and switch statement
4. Implement file I/O for saving/loading contacts
5. Add error handling for invalid inputs

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_CONTACTS 100
#define NAME_LENGTH 50
#define PHONE_LENGTH 15
#define EMAIL_LENGTH 50

typedef struct {
    char name[NAME_LENGTH];
    char phone[PHONE_LENGTH];
    char email[EMAIL_LENGTH];
} Contact;

// Function prototypes
void add_contact(Contact contacts[], int *count);
void view_contacts(Contact contacts[], int count);
void search_contact(Contact contacts[], int count);
void delete_contact(Contact contacts[], int *count);
void save_contacts(Contact contacts[], int count);
int load_contacts(Contact contacts[]);
void display_menu();

int main() {
    Contact contacts[MAX_CONTACTS];
    int count = 0;

    // Load existing contacts
    count = load_contacts(contacts);

    int choice;
    do {
        display_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                add_contact(contacts, &count);
                break;
            case 2:
                view_contacts(contacts, count);
                break;
            case 3:
```

```
                search_contact(contacts, count);
                break;
            case 4:
                delete_contact(contacts, &count);
                break;
            case 5:
                save_contacts(contacts, count);
                printf("Contacts saved successfully!\n");
                break;
            case 6:
                printf("Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 6);

    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Buffer overflows when reading strings
2. Not checking return values of file operations
3. Memory leaks (though not common in this project)
4. Not handling invalid user input properly

Best Practices

1. Use `fgets()` instead of `gets()` for string input
2. Always validate user input
3. Check return values of file operations
4. Use constants for array sizes
5. Comment your code appropriately

Project 2: Simple Calculator

Description

Create a command-line calculator that can perform basic arithmetic operations and some advanced functions.

Learning Objectives

- Working with floating-point numbers
- Implementing mathematical functions
- Error handling
- Menu systems

Requirements

1. Perform basic operations: addition, subtraction, multiplication, division
2. Perform advanced operations: power, square root, logarithm, trigonometric functions
3. Handle invalid inputs gracefully
4. Support both interactive and command-line argument modes
5. Display help information
6. Clear operation history

Implementation Steps

1. Design the main calculator loop
2. Implement basic arithmetic functions
3. Add advanced mathematical functions
4. Handle command-line arguments
5. Implement error handling
6. Create a clean user interface

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

// Function prototypes
double add(double a, double b);
double subtract(double a, double b);
double multiply(double a, double b);
double divide(double a, double b);
double power(double base, double exponent);
double square_root(double x);
double logarithm(double x);
double sine(double x);
double cosine(double x);

void print_menu();
void interactive_mode();
```

```c
void command_line_mode(int argc, char *argv[]);
int parse_operation(const char *op);

int main(int argc, char *argv[]) {
    if (argc > 1) {
        command_line_mode(argc, argv);
    } else {
        interactive_mode();
    }
    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Division by zero errors
2. Invalid input for mathematical functions (e.g., square root of negative numbers)
3. Not handling command-line argument parsing correctly
4. Precision issues with floating-point arithmetic

Best Practices

1. Always check for division by zero
2. Validate inputs before performing operations
3. Use appropriate data types for precision requirements
4. Provide clear error messages
5. Implement proper input validation

Project 3: Number Guessing Game

Description

Create a number guessing game where the computer generates a random number and the player tries to guess it with feedback on whether their guess is too high or too low.

Learning Objectives

- Working with random numbers
- Loop control structures
- Conditional statements
- User interaction

Requirements

1. Generate a random number within a specified range
2. Allow the user to make guesses
3. Provide feedback on each guess (too high, too low, correct)
4. Count the number of attempts
5. Allow the user to play multiple rounds
6. Keep track of high scores
7. Provide difficulty levels

Implementation Steps

1. Implement random number generation
2. Create the main game loop
3. Add difficulty levels
4. Implement scoring system
5. Add high score tracking
6. Create a user-friendly interface

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define EASY_MAX 50
#define MEDIUM_MAX 100
#define HARD_MAX 200

// Function prototypes
int generate_random_number(int max);
void play_game(int max_number);
void display_high_scores();
void save_high_score(int attempts, int max_number);
void display_menu();
int get_difficulty();

int main() {
    srand(time(NULL));  // Seed the random number generator

    int choice;
    do {
        display_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
        switch (choice) {
            case 1: {
                int difficulty = get_difficulty();
                int max_number;
                switch (difficulty) {
                    case 1: max_number = EASY_MAX; break;
                    case 2: max_number = MEDIUM_MAX; break;
                    case 3: max_number = HARD_MAX; break;
                    default: max_number = MEDIUM_MAX;
                }
                play_game(max_number);
                break;
            }
            case 2:
                display_high_scores();
                break;
            case 3:
                printf("Thanks for playing!\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 3);

    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Not seeding the random number generator
2. Not validating user input
3. Integer overflow issues
4. Not handling invalid menu choices

Best Practices

1. Always seed the random number generator with `srand(time(NULL))`
2. Validate all user input
3. Provide clear instructions and feedback
4. Use constants for magic numbers
5. Implement proper error handling

Project 4: Temperature Converter

Description

Create a temperature conversion tool that can convert between Celsius, Fahrenheit, and Kelvin.

Learning Objectives

- Working with mathematical formulas
- User interface design
- Data validation
- File I/O for saving conversion history

Requirements

1. Convert between Celsius, Fahrenheit, and Kelvin
2. Support batch conversions from files
3. Save conversion history to a file
4. Load previous conversion history
5. Provide a clean command-line interface
6. Handle invalid temperature values

Implementation Steps

1. Implement temperature conversion formulas
2. Create a menu-driven interface
3. Add file I/O for history management
4. Implement batch conversion feature
5. Add data validation
6. Create a user-friendly interface

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>

// Conversion formulas
double celsius_to_fahrenheit(double celsius);
double celsius_to_kelvin(double celsius);
double fahrenheit_to_celsius(double fahrenheit);
double fahrenheit_to_kelvin(double fahrenheit);
double kelvin_to_celsius(double kelvin);
double kelvin_to_fahrenheit(double kelvin);
```

```c
// Function prototypes
void single_conversion();
void batch_conversion();
void save_history();
void load_history();
void display_menu();

int main() {
    int choice;
    do {
        display_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                single_conversion();
                break;
            case 2:
                batch_conversion();
                break;
            case 3:
                save_history();
                break;
            case 4:
                load_history();
                break;
            case 5:
                printf("Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 5);

    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Not handling invalid temperature values (e.g., below absolute zero)
2. Precision issues with floating-point arithmetic
3. Not validating file operations
4. Buffer overflows with string inputs

Best Practices

1. Validate temperature values (e.g., Kelvin cannot be negative)
2. Use appropriate precision for floating-point numbers
3. Check file operation return values
4. Provide clear error messages
5. Use constants for conversion formulas

Tips for Success

1. **Start Simple**: Begin with basic functionality and gradually add features
2. **Test Frequently**: Test your code after implementing each feature
3. **Handle Errors**: Always consider what could go wrong and handle it gracefully
4. **Use Functions**: Break your code into logical functions for better organization
5. **Comment Your Code**: Explain complex logic and algorithms
6. **Validate Input**: Never trust user input; always validate it
7. **Save Often**: Use version control or save your work frequently
8. **Ask for Help**: Don't hesitate to seek help when you're stuck

These beginner projects will help you build confidence in C programming while practicing essential concepts. Remember to focus on writing clean, readable code and handling errors appropriately.

Capstone

Capstone Projects

These capstone projects are designed to integrate all the skills and knowledge you've gained throughout the C programming course. They represent substantial, real-world applications that demonstrate mastery of advanced C programming concepts.

Project 1: Distributed Chat Application

Description

Create a distributed chat application with multiple servers, clients, and advanced features like encryption, file sharing, and user authentication.

Learning Objectives

- Network programming with TCP/IP and UDP
- Multithreading and concurrency
- Cryptography and security
- Distributed systems concepts
- Protocol design and implementation
- Database integration
- User interface design

Requirements

1. **Server Components**:
    - Multiple chat servers with load balancing
    - User authentication and session management
    - Message routing between servers
    - Persistent message storage
    - Administration interface
2. **Client Features**:
    - User registration and login
    - Real-time messaging with multiple users
    - Private messaging
    - Group chats
    - File sharing capability
    - Message history

- Presence indication (online/offline)
- Emoticons and rich text support

3. **Advanced Features**:
   - End-to-end encryption
   - Message queuing for offline users
   - Voice messaging (optional)
   - Video calling (optional)
   - Plugin architecture for extensions
   - Mobile client compatibility

Implementation Steps

1. Design system architecture and communication protocols
2. Implement core server infrastructure
3. Create client application with user interface
4. Add authentication and security features
5. Implement message routing and storage
6. Add advanced features like file sharing
7. Create administration tools
8. Test with multiple users and servers
9. Optimize for performance and scalability

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <openssl/ssl.h>
#include <openssl/err.h>
#include <sqlite3.h>

#define MAX_CLIENTS 1000
#define MAX_MESSAGE_LENGTH 1024
#define SERVER_PORT 8080
#define DATABASE_FILE "chat.db"

// Message types
typedef enum {
    MSG_TEXT,
    MSG_FILE,
```

```
    MSG_JOIN,
    MSG_LEAVE,
    MSG_AUTH,
    MSG_ERROR
} message_type_t;

// User structure
typedef struct {
    int id;
    char username[50];
    char password_hash[64];
    int socket_fd;
    SSL *ssl;
    int authenticated;
    time_t last_activity;
} user_t;

// Message structure
typedef struct {
    message_type_t type;
    int sender_id;
    int receiver_id;  // 0 for broadcast
    char content[MAX_MESSAGE_LENGTH];
    time_t timestamp;
    int encrypted;
} message_t;

// Server structure
typedef struct {
    int server_fd;
    SSL_CTX *ssl_ctx;
    sqlite3 *db;
    user_t clients[MAX_CLIENTS];
    int client_count;
    pthread_mutex_t clients_mutex;
    pthread_t worker_threads[10];
    int thread_count;
    int running;
} chat_server_t;

// Client structure
typedef struct {
    int socket_fd;
    SSL *ssl;
    char username[50];
    int authenticated;
    pthread_t receive_thread;
```

```
    pthread_t send_thread;
    int connected;
} chat_client_t;

// Function prototypes
chat_server_t* create_chat_server(int port);
void destroy_chat_server(chat_server_t *server);
int start_server(chat_server_t *server);
void stop_server(chat_server_t *server);
void* client_handler(void *arg);
int authenticate_user(chat_server_t *server, const char *username, const char *password);
int register_user(chat_server_t *server, const char *username, const char *password);
int send_message(chat_server_t *server, message_t *msg);
int broadcast_message(chat_server_t *server, message_t *msg);
int store_message(chat_server_t *server, message_t *msg);
message_t* retrieve_message_history(chat_server_t *server, int user_id, int limit);
int init_database(chat_server_t *server);
SSL_CTX* create_ssl_context();
int create_user_table(chat_server_t *server);
void* server_worker_thread(void *arg);
void log_message(const char *format, ...);

// Client functions
chat_client_t* create_chat_client(const char *server_ip, int port);
void destroy_chat_client(chat_client_t *client);
int connect_to_server(chat_client_t *client);
void disconnect_from_server(chat_client_t *client);
int login(chat_client_t *client, const char *username, const char *password);
int send_chat_message(chat_client_t *client, const char *message, int receiver_id);
void* receive_messages(void *arg);
void* send_messages(void *arg);

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s [server|client]\n", argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "server") == 0) {
        chat_server_t *server = create_chat_server(SERVER_PORT);
        if (!server) {
            fprintf(stderr, "Failed to create server\n");
            return 1;
        }

        if (start_server(server) != 0) {
            fprintf(stderr, "Failed to start server\n");
```

```
            destroy_chat_server(server);
            return 1;
        }

        printf("Chat server started on port %d\n", SERVER_PORT);

        // Keep server running
        while (1) {
            sleep(1);
        }

        stop_server(server);
        destroy_chat_server(server);
    } else if (strcmp(argv[1], "client") == 0) {
        chat_client_t *client = create_chat_client("127.0.0.1", SERVER_PORT);
        if (!client) {
            fprintf(stderr, "Failed to create client\n");
            return 1;
        }

        if (connect_to_server(client) != 0) {
            fprintf(stderr, "Failed to connect to server\n");
            destroy_chat_client(client);
            return 1;
        }

        // Login process
        char username[50], password[50];
        printf("Enter username: ");
        scanf("%s", username);
        printf("Enter password: ");
        scanf("%s", password);

        if (login(client, username, password) != 0) {
            fprintf(stderr, "Login failed\n");
            disconnect_from_server(client);
            destroy_chat_client(client);
            return 1;
        }

        printf("Login successful! Starting chat...\n");

        // Start message threads
        pthread_create(&client->receive_thread, NULL, receive_messages, client);
        pthread_create(&client->send_thread, NULL, send_messages, client);

        // Wait for threads
```

```
        pthread_join(client->receive_thread, NULL);
        pthread_join(client->send_thread, NULL);

        disconnect_from_server(client);
        destroy_chat_client(client);
    } else {
        fprintf(stderr, "Invalid mode. Use 'server' or 'client'\n");
        return 1;
    }

    return 0;
}

// Implement all functions here
```

Key Challenges

1. **Concurrency**: Managing multiple clients and threads safely
2. **Security**: Implementing robust encryption and authentication
3. **Scalability**: Designing for thousands of concurrent users
4. **Reliability**: Ensuring message delivery and system stability
5. **Performance**: Optimizing database queries and network operations
6. **Protocol Design**: Creating efficient communication protocols

Evaluation Criteria

- **Functionality**: All required features implemented correctly
- **Security**: Proper encryption and authentication mechanisms
- **Performance**: Efficient handling of multiple connections
- **Robustness**: Error handling and recovery mechanisms
- **Code Quality**: Clean, well-documented, maintainable code
- **Testing**: Comprehensive test suite with edge cases
- **Documentation**: Clear user and developer documentation

Project 2: Operating System Kernel

Description

Implement a basic operating system kernel that can boot, manage processes, handle interrupts, and provide system calls.

Learning Objectives

- Low-level system programming
- Memory management
- Process scheduling
- Interrupt handling
- Device drivers
- File system implementation
- Boot process understanding

Requirements

1. **Boot Process**:
    - Custom bootloader
    - Kernel loading and initialization
    - Basic hardware detection
2. **Memory Management**:
    - Physical memory management
    - Virtual memory with paging
    - Heap allocation
    - Memory protection
3. **Process Management**:
    - Process creation and termination
    - Process scheduling (round-robin, priority-based)
    - Inter-process communication
    - Process synchronization
4. **Interrupt Handling**:
    - Interrupt descriptor table (IDT)
    - Exception handling
    - Hardware interrupt processing
    - System call interface
5. **Device Drivers**:
    - Keyboard driver
    - Display driver (VGA text mode)
    - Timer driver
    - Storage driver (optional)
6. **File System**:
    - Simple file system implementation
    - File operations (create, read, write, delete)
    - Directory management

Implementation Steps

1. Set up development environment with cross-compiler
2. Implement bootloader and kernel entry point
3. Set up GDT, IDT, and basic interrupt handling
4. Implement memory management system
5. Create process management and scheduling
6. Add device drivers for basic I/O
7. Implement file system
8. Create system call interface
9. Develop user-space programs
10. Test and debug the complete system

Sample Code Structure

```
// kernel.h
#ifndef KERNEL_H
#define KERNEL_H

#include <stdint.h>
#include <stddef.h>

// Basic types
typedef uint8_t  u8;
typedef uint16_t u16;
typedef uint32_t u32;
typedef uint64_t u64;

// Kernel entry point
void kernel_main();

// Hardware abstraction
void outb(u16 port, u8 val);
u8 inb(u16 port);
void outw(u16 port, u16 val);
u16 inw(u16 port);

#endif

// gdt.h
#ifndef GDT_H
#define GDT_H

#include "kernel.h"

typedef struct {
```

```
    u16 limit_low;
    u16 base_low;
    u8 base_middle;
    u8 access;
    u8 granularity;
    u8 base_high;
} __attribute__((packed)) gdt_entry_t;

typedef struct {
    u16 limit;
    u32 base;
} __attribute__((packed)) gdt_ptr_t;

void init_gdt();
void gdt_set_gate(int num, u32 base, u32 limit, u8 access, u8 gran);

#endif

// idt.h
#ifndef IDT_H
#define IDT_H

#include "kernel.h"

typedef struct {
    u16 base_low;
    u16 sel;
    u8 always0;
    u8 flags;
    u16 base_high;
} __attribute__((packed)) idt_entry_t;

typedef struct {
    u16 limit;
    u32 base;
} __attribute__((packed)) idt_ptr_t;

void init_idt();
void idt_set_gate(u8 num, u32 base, u16 sel, u8 flags);

#endif

// memory.h
#ifndef MEMORY_H
#define MEMORY_H

#include "kernel.h"
```

```
#define PAGE_SIZE 4096

typedef struct {
    u32 present    : 1;
    u32 rw         : 1;
    u32 user       : 1;
    u32 writethru  : 1;
    u32 cached     : 1;
    u32 accessed   : 1;
    u32 dirty      : 1;
    u32 pat        : 1;
    u32 global     : 1;
    u32 available  : 3;
    u32 frame      : 20;
} __attribute__((packed)) page_t;

typedef struct {
    page_t pages[1024];
} __attribute__((packed)) page_table_t;

typedef struct {
    page_table_t *tables[1024];
    u32 tables_physical[1024];
    u32 physical_address;
} __attribute__((packed)) page_directory_t;

void init_paging();
void switch_page_directory(page_directory_t *dir);
page_t *get_page(u32 address, int make, page_directory_t *dir);
void alloc_frame(page_t *page, int is_kernel, int is_writeable);
void free_frame(page_t *page);

#endif

// process.h
#ifndef PROCESS_H
#define PROCESS_H

#include "kernel.h"
#include "memory.h"

#define MAX_PROCESSES 256
#define KERNEL_STACK_SIZE 4096

typedef enum {
    PROCESS_RUNNING,
```

```
    PROCESS_READY,
    PROCESS_BLOCKED,
    PROCESS_TERMINATED
} process_state_t;

typedef struct {
    u32 esp, ebp, esi, edi, eax, ebx, ecx, edx;
    u32 eip, eflags;
    u32 cr3;  // Page directory
    u32 kernel_stack;
    process_state_t state;
    u32 pid;
    char name[32];
} __attribute__((packed)) process_t;

typedef struct {
    process_t *processes[MAX_PROCESSES];
    u32 current_process;
    u32 process_count;
} process_manager_t;

void init_process_manager();
process_t *create_process(const char *name, u32 entry_point);
void switch_to_process(process_t *process);
void schedule();
void yield();

#endif

// syscall.h
#ifndef SYSCALL_H
#define SYSCALL_H

#include "kernel.h"

#define SYSCALL_WRITE 1
#define SYSCALL_READ  2
#define SYSCALL_EXIT  3
#define SYSCALL_FORK  4

void init_syscalls();
void syscall_handler(registers_t *regs);

#endif

// main kernel implementation
void kernel_main() {
```

```
    // Initialize hardware
    init_gdt();
    init_idt();

    // Initialize memory management
    init_paging();

    // Initialize process management
    init_process_manager();

    // Initialize system calls
    init_syscalls();

    // Create initial processes
    process_t *init = create_process("init", (u32)init_process);
    switch_to_process(init);

    // Start scheduler
    while (1) {
        schedule();
        __asm__ __volatile__("hlt");
    }
}

// Implement all other components here
```

Key Challenges

1. **Low-level Programming**: Working directly with hardware
2. **Memory Management**: Implementing virtual memory and paging
3. **Concurrency**: Managing processes and threads safely
4. **Interrupt Handling**: Properly handling hardware interrupts
5. **Boot Process**: Understanding and implementing the boot sequence
6. **Debugging**: Debugging at the kernel level

Evaluation Criteria

- **Boot Process**: Successful kernel loading and initialization
- **Memory Management**: Correct implementation of paging and allocation
- **Process Management**: Functional process creation and scheduling
- **Interrupt Handling**: Proper exception and interrupt processing
- **System Calls**: Working system call interface
- **Stability**: Kernel stability under various conditions
- **Documentation**: Clear explanation of kernel components

Project 3: Database Management System

Description

Create a full-featured relational database management system with SQL support, transactions, indexing, and optimization.

Learning Objectives

- Data storage and retrieval
- Query processing and optimization
- Transaction management
- Concurrency control
- Indexing and search algorithms
- File system integration
- SQL parser and executor

Requirements

1. **Storage Engine**:
   - Page-based storage system
   - Buffer pool management
   - Transaction log for recovery
   - Checkpointing mechanism
2. **SQL Support**:
   - DDL (CREATE, ALTER, DROP)
   - DML (SELECT, INSERT, UPDATE, DELETE)
   - Query optimization
   - Aggregation functions
   - Joins (INNER, LEFT, RIGHT, FULL)
3. **Transaction Management**:
   - ACID properties
   - Lock-based concurrency control
   - Deadlock detection and resolution
   - Transaction isolation levels
4. **Indexing**:
   - B+ tree implementation
   - Hash index support
   - Composite index support
   - Index maintenance
5. **Administration**:

- Database creation and management
- User authentication and permissions
- Backup and recovery
- Performance monitoring

Implementation Steps

1. Design database architecture and storage format
2. Implement storage engine with page management
3. Create buffer pool and cache system
4. Develop SQL parser and query planner
5. Implement B+ tree and other index structures
6. Add transaction management and concurrency control
7. Create administration tools and utilities
8. Test with standard SQL benchmarks
9. Optimize for performance
10. Document the system architecture

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define PAGE_SIZE 4096
#define MAX_DATABASES 100
#define MAX_TABLES 1000
#define MAX_COLUMNS 50
#define MAX_INDEXES 100

// Data types
typedef enum {
    TYPE_INT,
    TYPE_BIGINT,
    TYPE_FLOAT,
    TYPE_DOUBLE,
    TYPE_VARCHAR,
    TYPE_TEXT,
    TYPE_DATE,
    TYPE_DATETIME
} data_type_t;
```

```
// Page structure
typedef struct {
    u32 page_id;
    u32 next_page;
    u32 prev_page;
    u16 free_space;
    u8 page_type;
    u8 data[PAGE_SIZE - sizeof(u32) * 3 - sizeof(u16) - sizeof(u8)];
} __attribute__((packed)) page_t;

// Table structure
typedef struct {
    u32 table_id;
    char name[64];
    u32 page_id;  // First page of table data
    u32 row_count;
    u32 column_count;
    struct {
        char name[32];
        data_type_t type;
        u16 size;
        u8 nullable;
    } columns[MAX_COLUMNS];
} table_t;

// Index structure
typedef struct {
    u32 index_id;
    char name[64];
    u32 table_id;
    u32 column_count;
    u32 columns[MAX_COLUMNS];
    u8 index_type;  // 0 = B+ tree, 1 = Hash
    u32 root_page;
} index_t;

// Transaction structure
typedef enum {
    TXN_ACTIVE,
    TXN_COMMITTED,
    TXN_ABORTED
} txn_state_t;

typedef struct {
    u32 txn_id;
    txn_state_t state;
```

```
    u64 start_time;
    u64 commit_time;
    pthread_mutex_t lock;
    // Undo/redo logs
    struct log_entry *undo_log;
    struct log_entry *redo_log;
} transaction_t;

// Database structure
typedef struct {
    char name[64];
    int fd;  // File descriptor
    u32 page_count;
    table_t tables[MAX_TABLES];
    u32 table_count;
    index_t indexes[MAX_INDEXES];
    u32 index_count;
    pthread_mutex_t lock;
    pthread_rwlock_t rw_lock;
} database_t;

// Buffer pool
typedef struct {
    page_t *pages;
    u32 *page_ids;
    u8 *dirty_flags;
    u64 *access_times;
    u32 capacity;
    u32 count;
    pthread_mutex_t lock;
} buffer_pool_t;

// SQL Parser structures
typedef enum {
    SQL_SELECT,
    SQL_INSERT,
    SQL_UPDATE,
    SQL_DELETE,
    SQL_CREATE_TABLE,
    SQL_DROP_TABLE,
    SQL_CREATE_INDEX,
    SQL_BEGIN_TRANSACTION,
    SQL_COMMIT,
    SQL_ROLLBACK
} sql_statement_type_t;

typedef struct {
```

```
    sql_statement_type_t type;
    char *table_name;
    struct {
        char *column_name;
        char *value;
    } *columns;
    u32 column_count;
    char *where_clause;
    char **order_by;
    u32 order_by_count;
    u32 limit;
} sql_statement_t;

// Query plan
typedef enum {
    PLAN_SCAN,
    PLAN_INDEX_SCAN,
    PLAN_JOIN,
    PLAN_SORT,
    PLAN_AGGREGATE
} plan_node_type_t;

typedef struct plan_node {
    plan_node_type_t type;
    struct plan_node *left;
    struct plan_node *right;
    void *data;  // Node-specific data
} plan_node_t;

// Database manager
typedef struct {
    database_t databases[MAX_DATABASES];
    u32 database_count;
    buffer_pool_t buffer_pool;
    transaction_t *active_transactions;
    u32 transaction_count;
    pthread_mutex_t txn_lock;
} db_manager_t;

// Function prototypes
db_manager_t* create_db_manager(u32 buffer_pool_size);
void destroy_db_manager(db_manager_t *manager);
database_t* create_database(db_manager_t *manager, const char *name);
int drop_database(db_manager_t *manager, const char *name);
table_t* create_table(database_t *db, const char *name, /* column definitions */);
int drop_table(database_t *db, const char *name);
index_t* create_index(database_t *db, const char *name, const char *table_name, /* column li
```

```
int drop_index(database_t *db, const char *name);
transaction_t* begin_transaction(db_manager_t *manager);
int commit_transaction(db_manager_t *manager, transaction_t *txn);
int rollback_transaction(db_manager_t *manager, transaction_t *txn);
sql_statement_t* parse_sql(const char *sql);
plan_node_t* create_query_plan(sql_statement_t *stmt, database_t *db);
int execute_query_plan(plan_node_t *plan, transaction_t *txn, /* result set */);
page_t* get_page(database_t *db, u32 page_id);
int write_page(database_t *db, page_t *page);
void* btree_insert(index_t *index, void *key, u32 page_id);
void* btree_search(index_t *index, void *key);
int btree_delete(index_t *index, void *key);
void lock_table(database_t *db, table_t *table, int exclusive);
void unlock_table(database_t *db, table_t *table);
void lock_row(database_t *db, table_t *table, u32 row_id, int exclusive);
void unlock_row(database_t *db, table_t *table, u32 row_id);

int main() {
    // Create database manager
    db_manager_t *manager = create_db_manager(1000);  // 1000 page buffer pool
    if (!manager) {
        fprintf(stderr, "Failed to create database manager\n");
        return 1;
    }

    // Create database
    database_t *db = create_database(manager, "testdb");
    if (!db) {
        fprintf(stderr, "Failed to create database\n");
        destroy_db_manager(manager);
        return 1;
    }

    // Example SQL execution
    const char *sql = "CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50), email VARCH
    sql_statement_t *stmt = parse_sql(sql);
    if (stmt) {
        plan_node_t *plan = create_query_plan(stmt, db);
        if (plan) {
            transaction_t *txn = begin_transaction(manager);
            execute_query_plan(plan, txn, NULL);
            commit_transaction(manager, txn);
            // Free plan
        }
        // Free statement
    }
```

```
    // Cleanup
    destroy_db_manager(manager);
    return 0;
}

// Implement all functions here
```

Key Challenges

1. **Data Consistency**: Ensuring ACID properties in transactions
2. **Concurrency Control**: Managing concurrent access to data
3. **Query Optimization**: Creating efficient execution plans
4. **Storage Management**: Efficiently managing disk and memory resources
5. **Index Implementation**: Implementing complex data structures like B+ trees
6. **SQL Parsing**: Creating a robust SQL parser

Evaluation Criteria

- **SQL Compliance**: Support for standard SQL features
- **Performance**: Efficient query execution and optimization
- **Reliability**: Transaction safety and recovery mechanisms
- **Scalability**: Ability to handle large datasets
- **Concurrency**: Proper handling of concurrent operations
- **Code Quality**: Well-structured, maintainable code
- **Documentation**: Comprehensive system documentation

Project 4: Game Engine

Description

Create a 2D game engine with rendering, physics, audio, input handling, and scripting capabilities.

Learning Objectives

- Graphics programming with OpenGL/DirectX
- Physics simulation
- Audio processing
- Input handling
- Entity-component system
- Resource management
- Cross-platform development

Requirements

1. **Graphics System**:
   - 2D rendering pipeline
   - Sprite management
   - Animation system
   - Particle effects
   - Camera system
2. **Physics Engine**:
   - Collision detection (AABB, circle, polygon)
   - Collision response
   - Rigidbody physics
   - Constraints and joints
3. **Audio System**:
   - Sound loading and playback
   - Music streaming
   - 3D audio positioning
   - Audio effects
4. **Input Handling**:
   - Keyboard, mouse, gamepad support
   - Input mapping system
   - Gesture recognition
5. **Game Object System**:
   - Entity-component architecture
   - Scene management
   - Serialization system
   - Scripting interface
6. **Resource Management**:
   - Asset loading and caching
   - Texture compression
   - Memory pooling
   - Async loading

Implementation Steps

1. Set up graphics rendering framework
2. Implement entity-component system
3. Create physics simulation engine
4. Add audio processing capabilities
5. Develop input handling system
6. Build resource management system

7. Create scripting interface
8. Develop tools and editor
9. Test with sample games
10. Optimize for performance

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <pthread.h>
#ifdef _WIN32
    #include <windows.h>
    #include <GL/gl.h>
#else
    #include <GL/gl.h>
    #include <unistd.h>
#endif

#define MAX_ENTITIES 10000
#define MAX_COMPONENTS 32
#define MAX_SYSTEMS 64

// Math structures
typedef struct {
    float x, y;
} vec2_t;

typedef struct {
    float x, y, z;
} vec3_t;

typedef struct {
    float x, y, z, w;
} vec4_t;

typedef struct {
    float m[16];  // 4x4 matrix
} mat4_t;

// Component types
typedef enum {
    COMPONENT_TRANSFORM,
    COMPONENT_SPRITE,
    COMPONENT_RIGIDBODY,
```

```
    COMPONENT_COLLIDER,
    COMPONENT_SCRIPT,
    COMPONENT_CAMERA,
    COMPONENT_AUDIO_SOURCE,
    COMPONENT_PARTICLE_SYSTEM
} component_type_t;

// Transform component
typedef struct {
    vec3_t position;
    vec3_t rotation;
    vec3_t scale;
    mat4_t world_matrix;
} transform_component_t;

// Sprite component
typedef struct {
    u32 texture_id;
    vec2_t size;
    vec4_t color;
    int visible;
} sprite_component_t;

// Rigidbody component
typedef struct {
    vec2_t velocity;
    vec2_t acceleration;
    float mass;
    float drag;
    int use_gravity;
} rigidbody_component_t;

// Collider component
typedef enum {
    COLLIDER_AABB,
    COLLIDER_CIRCLE,
    COLLIDER_POLYGON
} collider_type_t;

typedef struct {
    collider_type_t type;
    union {
        struct { vec2_t min, max; } aabb;
        struct { vec2_t center; float radius; } circle;
        struct { vec2_t *vertices; int vertex_count; } polygon;
    } shape;
    int is_trigger;
```

```
} collider_component_t;

// Entity structure
typedef struct {
    u32 id;
    u64 component_mask;  // Bitmask of components
    char name[64];
    int active;
} entity_t;

// Component arrays (structure of arrays for cache efficiency)
typedef struct {
    transform_component_t transforms[MAX_ENTITIES];
    sprite_component_t sprites[MAX_ENTITIES];
    rigidbody_component_t rigidbodies[MAX_ENTITIES];
    collider_component_t colliders[MAX_ENTITIES];
    // ... other component arrays
} component_arrays_t;

// System base structure
typedef struct {
    char name[32];
    void (*update)(float delta_time);
    void (*render)();
    int enabled;
} system_t;

// Renderer system
typedef struct {
    system_t base;
    u32 shader_program;
    u32 vao, vbo;
    mat4_t projection_matrix;
    mat4_t view_matrix;
} renderer_system_t;

// Physics system
typedef struct {
    system_t base;
    vec2_t gravity;
    float time_scale;
} physics_system_t;

// Collision system
typedef struct {
    system_t base;
    struct {
```

```
        u32 entity_a, entity_b;
        vec2_t contact_point;
        vec2_t normal;
        float penetration;
    } *contacts;
    u32 contact_count;
    u32 contact_capacity;
} collision_system_t;

// Scene structure
typedef struct {
    entity_t entities[MAX_ENTITIES];
    u32 entity_count;
    component_arrays_t components;
    system_t *systems[MAX_SYSTEMS];
    u32 system_count;
    float time_accumulator;
} scene_t;

// Game engine structure
typedef struct {
    scene_t *current_scene;
    int running;
    float delta_time;
    u64 frame_count;
    struct {
        int width, height;
        const char *title;
        void *window_handle;
    } window;
    struct {
        float master_volume;
        float sfx_volume;
        float music_volume;
    } audio_settings;
} game_engine_t;

// Function prototypes
game_engine_t* create_game_engine(const char *title, int width, int height);
void destroy_game_engine(game_engine_t *engine);
int initialize_engine(game_engine_t *engine);
void run_game_loop(game_engine_t *engine);
void shutdown_engine(game_engine_t *engine);

// Entity management
entity_t* create_entity(scene_t *scene, const char *name);
void destroy_entity(scene_t *scene, u32 entity_id);
```

```
int add_component(scene_t *scene, u32 entity_id, component_type_t type);
int remove_component(scene_t *scene, u32 entity_id, component_type_t type);
int has_component(scene_t *scene, u32 entity_id, component_type_t type);
void* get_component(scene_t *scene, u32 entity_id, component_type_t type);

// System management
int register_system(scene_t *scene, system_t *system);
int unregister_system(scene_t *scene, const char *system_name);
system_t* get_system(scene_t *scene, const char *system_name);

// Math functions
vec2_t vec2_add(vec2_t a, vec2_t b);
vec2_t vec2_subtract(vec2_t a, vec2_t b);
vec2_t vec2_multiply(vec2_t v, float scalar);
float vec2_length(vec2_t v);
vec2_t vec2_normalize(vec2_t v);
mat4_t mat4_identity();
mat4_t mat4_orthographic(float left, float right, float bottom, float top, float near, float
mat4_t mat4_transform(vec3_t position, vec3_t rotation, vec3_t scale);

// Renderer functions
renderer_system_t* create_renderer_system();
void destroy_renderer_system(renderer_system_t *renderer);
void renderer_update(float delta_time);
void renderer_render();
void renderer_draw_sprite(sprite_component_t *sprite, transform_component_t *transform);

// Physics functions
physics_system_t* create_physics_system();
void destroy_physics_system(physics_system_t *physics);
void physics_update(float delta_time);
void integrate_rigidbody(rigidbody_component_t *rb, transform_component_t *transform, float

// Collision functions
collision_system_t* create_collision_system();
void destroy_collision_system(collision_system_t *collision);
void collision_update(float delta_time);
int check_collision_aabb_circle(vec2_t aabb_min, vec2_t aabb_max, vec2_t circle_center, floa
int check_collision_circle_circle(vec2_t center1, float radius1, vec2_t center2, float radiu
void resolve_collision(u32 entity_a, u32 entity_b, vec2_t contact_point, vec2_t normal, floa

// Input functions
int is_key_pressed(int key);
int is_key_released(int key);
int is_key_held(int key);
vec2_t get_mouse_position();
int is_mouse_button_pressed(int button);
```

```c
// Resource management
u32 load_texture(const char *filename);
void unload_texture(u32 texture_id);
void* load_shader(const char *vertex_shader, const char *fragment_shader);
void unload_shader(u32 shader_program);

int main() {
    // Create game engine
    game_engine_t *engine = create_game_engine("My Game Engine", 1024, 768);
    if (!engine) {
        fprintf(stderr, "Failed to create game engine\n");
        return 1;
    }

    // Initialize engine
    if (initialize_engine(engine) != 0) {
        fprintf(stderr, "Failed to initialize engine\n");
        destroy_game_engine(engine);
        return 1;
    }

    // Create sample scene
    scene_t *scene = engine->current_scene;

    // Create player entity
    entity_t *player = create_entity(scene, "Player");
    add_component(scene, player->id, COMPONENT_TRANSFORM);
    add_component(scene, player->id, COMPONENT_SPRITE);
    add_component(scene, player->id, COMPONENT_RIGIDBODY);
    add_component(scene, player->id, COMPONENT_COLLIDER);

    // Configure player components
    transform_component_t *player_transform = get_component(scene, player->id, COMPONENT_TRA
    player_transform->position.x = 400;
    player_transform->position.y = 300;
    player_transform->scale.x = player_transform->scale.y = 2.0f;

    sprite_component_t *player_sprite = get_component(scene, player->id, COMPONENT_SPRITE);
    player_sprite->texture_id = load_texture("player.png");
    player_sprite->size.x = 32;
    player_sprite->size.y = 32;

    // Run game loop
    run_game_loop(engine);

    // Cleanup
```

```
    shutdown_engine(engine);
    destroy_game_engine(engine);

    return 0;
}

// Implement all functions here
```

Key Challenges

1. **Performance**: Optimizing rendering and physics for real-time performance
2. **Memory Management**: Efficiently managing game assets and entities
3. **Cross-platform**: Supporting multiple operating systems and graphics APIs
4. **Architecture**: Designing a flexible, extensible system
5. **Mathematics**: Implementing complex mathematical operations for graphics and physics
6. **Resource Management**: Loading and managing game assets efficiently

Evaluation Criteria

- **Functionality**: All required systems implemented and working
- **Performance**: Smooth gameplay with good frame rates
- **Flexibility**: Extensible architecture for adding new features
- **Code Quality**: Well-organized, maintainable code
- **Documentation**: Clear documentation of engine architecture
- **Sample Games**: Demonstrating engine capabilities with sample games
- **Tools**: Development tools for content creation

Tips for Capstone Success

1. **Choose Wisely**: Select a project that matches your interests and skill level
2. **Plan Extensively**: Create detailed designs and specifications before coding
3. **Iterative Development**: Build and test components incrementally
4. **Focus on Core Features**: Implement essential functionality first
5. **Document Everything**: Maintain comprehensive documentation throughout
6. **Test Thoroughly**: Create extensive test cases including edge cases
7. **Seek Feedback**: Get regular feedback from peers and mentors
8. **Manage Time**: Create realistic timelines and stick to them
9. **Learn Continuously**: Research and learn new techniques as needed
10. **Enjoy the Process**: Take pride in creating something substantial and meaningful

These capstone projects represent the culmination of your C programming education. They require integrating multiple concepts and techniques while solving complex, real-world problems. Choose a project that excites you and demonstrates your mastery of C programming.

Intermediate

Intermediate Projects

These projects are designed for programmers with basic C knowledge who want to advance their skills. They involve more complex data structures, file handling, and problem-solving techniques.

Project 1: Student Grade Management System

Description

Create a comprehensive system to manage student grades, calculate statistics, and generate reports.

Learning Objectives

- Working with dynamic data structures
- File I/O operations with structured data
- Data analysis and statistics
- Memory management
- Modular programming

Requirements

1. Store student information (ID, name, grades for multiple subjects)
2. Add, edit, and delete student records
3. Calculate average grades for students and subjects
4. Generate class statistics (highest, lowest, median grades)
5. Save and load data from files
6. Search and sort student records
7. Generate detailed reports
8. Handle multiple classes/sections

Implementation Steps

1. Design data structures for students and grades
2. Implement dynamic memory allocation for records
3. Create functions for CRUD operations
4. Implement statistical analysis functions
5. Add file I/O for data persistence

6. Create sorting and searching algorithms
7. Develop report generation features
8. Build a menu-driven interface

Sample Code Structure

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define MAX_NAME_LENGTH 50
#define MAX_SUBJECTS 10
#define FILENAME "students.dat"

typedef struct {
    int id;
    char name[MAX_NAME_LENGTH];
    int num_subjects;
    char subjects[MAX_SUBJECTS][MAX_NAME_LENGTH];
    double grades[MAX_SUBJECTS];
} Student;

typedef struct {
    Student *students;
    int count;
    int capacity;
} StudentManager;

// Function prototypes
StudentManager* create_student_manager(int initial_capacity);
void destroy_student_manager(StudentManager *manager);
int add_student(StudentManager *manager, Student student);
int edit_student(StudentManager *manager, int id, Student new_student);
int delete_student(StudentManager *manager, int id);
Student* find_student(StudentManager *manager, int id);
void sort_students_by_name(StudentManager *manager);
void sort_students_by_average(StudentManager *manager);
double calculate_student_average(Student *student);
double calculate_subject_average(StudentManager *manager, const char *subject);
void generate_class_report(StudentManager *manager);
void save_to_file(StudentManager *manager, const char *filename);
int load_from_file(StudentManager *manager, const char *filename);
void display_menu();

int main() {
```

```
    StudentManager *manager = create_student_manager(10);
    if (!manager) {
        fprintf(stderr, "Failed to create student manager\n");
        return 1;
    }

    // Load existing data
    load_from_file(manager, FILENAME);

    int choice;
    do {
        display_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                Student student;
                printf("Enter student ID: ");
                scanf("%d", &student.id);
                printf("Enter student name: ");
                scanf(" %[^\n]", student.name);
                // Add more input handling
                add_student(manager, student);
                break;
            }
            // Implement other cases
            case 8:
                save_to_file(manager, FILENAME);
                printf("Data saved successfully!\n");
                break;
            case 9:
                printf("Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 9);

    destroy_student_manager(manager);
    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Memory leaks from dynamic allocation
2. Buffer overflows with string inputs
3. Not checking return values of file operations
4. Incorrect implementation of sorting algorithms
5. Not handling edge cases in statistical calculations

Best Practices

1. Always free dynamically allocated memory
2. Check for NULL pointers before dereferencing
3. Validate all user inputs
4. Use appropriate data structures for efficiency
5. Implement proper error handling
6. Comment complex algorithms
7. Use constants for array sizes and limits

Project 2: Simple Text Editor

Description

Create a basic text editor that can create, edit, and save text files with basic editing features.

Learning Objectives

- Working with dynamic strings
- File I/O operations
- Text processing algorithms
- Memory management
- User interface design

Requirements

1. Create new text files
2. Open and edit existing text files
3. Save files to disk
4. Basic editing operations (insert, delete, replace)
5. Search and replace functionality
6. Undo/redo functionality
7. Line numbering
8. Status bar with file information

Implementation Steps

1. Design data structures for text storage
2. Implement file I/O operations
3. Create basic editing functions
4. Add search and replace features
5. Implement undo/redo functionality
6. Add line numbering
7. Create user interface
8. Add status bar and file information

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE_LENGTH 1000
#define INITIAL_LINES 100

typedef struct {
    char **lines;
    int line_count;
    int capacity;
    char filename[256];
    int modified;
} TextEditor;

// Function prototypes
TextEditor* create_editor();
void destroy_editor(TextEditor *editor);
int load_file(TextEditor *editor, const char *filename);
int save_file(TextEditor *editor, const char *filename);
void insert_line(TextEditor *editor, int line_num, const char *text);
void delete_line(TextEditor *editor, int line_num);
void replace_line(TextEditor *editor, int line_num, const char *text);
int search_text(TextEditor *editor, const char *text);
void replace_all(TextEditor *editor, const char *old_text, const char *new_text);
void display_content(TextEditor *editor);
void display_status(TextEditor *editor);
void expand_lines(TextEditor *editor);

int main() {
    TextEditor *editor = create_editor();
    if (!editor) {
        fprintf(stderr, "Failed to create text editor\n");
```

```
        return 1;
    }

    char filename[256];
    printf("Enter filename to open (or new filename): ");
    scanf(" %[^\n]", filename);

    if (load_file(editor, filename)) {
        printf("File loaded successfully\n");
    } else {
        printf("Creating new file: %s\n", filename);
        strcpy(editor->filename, filename);
    }

    int choice;
    do {
        display_content(editor);
        display_status(editor);
        // Add menu options
        printf("1. Insert line\n");
        printf("2. Delete line\n");
        printf("3. Replace line\n");
        printf("4. Search text\n");
        printf("5. Save file\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        // Implement menu options
    } while (choice != 6);

    destroy_editor(editor);
    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Memory leaks from dynamic string allocation
2. Buffer overflows when reading input
3. Not handling file operation errors
4. Incorrect implementation of undo/redo functionality
5. Performance issues with large files

Best Practices

1. Use dynamic memory allocation carefully
2. Validate all file operations
3. Implement proper error handling
4. Use efficient data structures for text storage
5. Provide clear user feedback
6. Handle edge cases in editing operations

Project 3: Library Management System

Description

Create a system to manage a library's collection of books, borrowers, and transactions.

Learning Objectives

- Working with complex data relationships
- File I/O with structured data
- Data validation and integrity
- Search and sorting algorithms
- Transaction management

Requirements

1. Manage book inventory (title, author, ISBN, quantity)
2. Manage borrower information (ID, name, contact info)
3. Handle book borrowing and returning
4. Track overdue books
5. Generate reports (borrowed books, overdue books, popular books)
6. Save and load data from files
7. Search books by various criteria
8. Handle multiple copies of the same book

Implementation Steps

1. Design data structures for books, borrowers, and transactions
2. Implement CRUD operations for books and borrowers
3. Create borrowing and returning functionality
4. Add overdue tracking
5. Implement file I/O for data persistence
6. Create search and reporting features
7. Build a menu-driven interface
8. Add data validation

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_TITLE_LENGTH 100
#define MAX_AUTHOR_LENGTH 50
#define MAX_BORROWERS 1000
#define MAX_BOOKS 1000

typedef struct {
    int isbn;
    char title[MAX_TITLE_LENGTH];
    char author[MAX_AUTHOR_LENGTH];
    int total_copies;
    int available_copies;
} Book;

typedef struct {
    int id;
    char name[50];
    char contact[50];
    int borrowed_count;
} Borrower;

typedef struct {
    int book_isbn;
    int borrower_id;
    time_t borrow_date;
    time_t due_date;
    time_t return_date;  // 0 if not returned
} Transaction;

typedef struct {
    Book books[MAX_BOOKS];
    int book_count;

    Borrower borrowers[MAX_BORROWERS];
    int borrower_count;

    Transaction transactions[10000];
    int transaction_count;
} Library;
```

```c
// Function prototypes
Library* create_library();
void destroy_library(Library *library);
int add_book(Library *library, Book book);
int remove_book(Library *library, int isbn);
int add_borrower(Library *library, Borrower borrower);
int remove_borrower(Library *library, int id);
int borrow_book(Library *library, int isbn, int borrower_id);
int return_book(Library *library, int isbn, int borrower_id);
Book* find_book(Library *library, int isbn);
Borrower* find_borrower(Library *library, int id);
void search_books(Library *library, const char *query);
void generate_overdue_report(Library *library);
void save_library(Library *library, const char *filename);
int load_library(Library *library, const char *filename);
void display_menu();

int main() {
    Library *library = create_library();
    if (!library) {
        fprintf(stderr, "Failed to create library\n");
        return 1;
    }

    // Load existing data
    load_library(library, "library.dat");

    int choice;
    do {
        display_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                Book book;
                printf("Enter ISBN: ");
                scanf("%d", &book.isbn);
                printf("Enter title: ");
                scanf(" %[^\n]", book.title);
                printf("Enter author: ");
                scanf(" %[^\n]", book.author);
                printf("Enter total copies: ");
                scanf("%d", &book.total_copies);
                book.available_copies = book.total_copies;
                add_book(library, book);
                break;
```

```
            }
            // Implement other cases
            case 9:
                save_library(library, "library.dat");
                printf("Library data saved successfully!\n");
                break;
            case 10:
                printf("Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 10);

    destroy_library(library);
    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Data inconsistency between related entities
2. Not handling date/time calculations correctly
3. Memory leaks in dynamic structures
4. Not validating transaction integrity
5. Performance issues with large datasets

Best Practices

1. Maintain data integrity between related entities
2. Use proper date/time functions
3. Implement efficient search algorithms
4. Validate all transactions
5. Handle edge cases in borrowing logic
6. Provide clear error messages
7. Use appropriate data structures for performance

Project 4: Simple Database Engine

Description

Create a basic database engine that can store, retrieve, and manipulate tabular data.

Learning Objectives

- Working with tabular data structures
- File I/O with binary data
- Query processing
- Indexing and search optimization
- Memory management

Requirements

1. Create and manage database tables
2. Define table schemas (column names and types)
3. Insert, update, and delete records
4. Query data with simple WHERE clauses
5. Save and load databases from files
6. Support basic data types (int, float, string)
7. Implement indexing for faster searches
8. Export data to CSV format

Implementation Steps

1. Design data structures for tables and records
2. Implement table creation and schema management
3. Create CRUD operations for records
4. Add query processing functionality
5. Implement file I/O for persistence
6. Add indexing for performance
7. Create CSV export functionality
8. Build a command-line interface

Sample Code Structure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TABLES 100
#define MAX_COLUMNS 50
#define MAX_RECORDS 10000
#define MAX_STRING_LENGTH 256

typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_STRING
```

```
} DataType;

typedef struct {
    char name[50];
    DataType type;
    int size;
} Column;

typedef struct {
    int id;
    void *data;  // Pointer to actual data
} Record;

typedef struct {
    char name[50];
    Column columns[MAX_COLUMNS];
    int column_count;
    Record records[MAX_RECORDS];
    int record_count;
    int *index;  // Simple index array
    int index_count;
} Table;

typedef struct {
    Table tables[MAX_TABLES];
    int table_count;
} Database;

// Function prototypes
Database* create_database();
void destroy_database(Database *db);
Table* create_table(Database *db, const char *name);
int add_column(Table *table, const char *name, DataType type, int size);
int insert_record(Table *table, void *data[]);
Record* find_record(Table *table, int record_id);
int update_record(Table *table, int record_id, void *data[]);
int delete_record(Table *table, int record_id);
Record** query_table(Table *table, const char *where_clause, int *result_count);
void save_database(Database *db, const char *filename);
int load_database(Database *db, const char *filename);
void export_to_csv(Table *table, const char *filename);
void display_menu();

int main() {
    Database *db = create_database();
    if (!db) {
        fprintf(stderr, "Failed to create database\n");
```

```
        return 1;
    }

    // Load existing database
    load_database(db, "database.dat");

    int choice;
    do {
        display_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                char table_name[50];
                printf("Enter table name: ");
                scanf(" %[^\n]", table_name);
                create_table(db, table_name);
                break;
            }
            // Implement other cases
            case 7:
                save_database(db, "database.dat");
                printf("Database saved successfully!\n");
                break;
            case 8:
                printf("Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 8);

    destroy_database(db);
    return 0;
}

// Implement all functions here
```

Common Pitfalls to Avoid

1. Memory leaks from dynamic data allocation
2. Data type mismatches
3. Not handling file format compatibility
4. Performance issues with large datasets
5. Not implementing proper indexing strategies

Best Practices

1. Use appropriate data structures for different data types
2. Validate data types during operations
3. Implement efficient indexing strategies
4. Handle file format versioning
5. Provide clear error messages for query syntax
6. Use constants for size limits
7. Implement proper memory management

Tips for Success

1. **Plan Before Coding**: Design your data structures and algorithms before implementation
2. **Modular Approach**: Break complex problems into smaller, manageable functions
3. **Error Handling**: Always consider what could go wrong and handle it gracefully
4. **Memory Management**: Be careful with dynamic allocation and always free memory
5. **Testing**: Test each function individually before integrating
6. **Performance**: Consider the efficiency of your algorithms, especially for large datasets
7. **Documentation**: Comment your code, especially complex algorithms
8. **Version Control**: Use version control to track changes and experiment safely

These intermediate projects will help you develop more advanced C programming skills while working on realistic applications. Focus on writing efficient, maintainable code with proper error handling.

Appendices

Appendix A: Additional Resources

Books and Publications

Core C Programming

1. **"The C Programming Language"** by Brian W. Kernighan and Dennis M. Ritchie
    - Often referred to as K&R, this is the definitive reference for C programming
    - Written by the creators of the C language
    - Essential for understanding the foundations of C
2. **"C Programming: A Modern Approach"** by K. N. King
    - Comprehensive guide covering both basic and advanced C concepts
    - Includes modern C standards (C99, C11)
    - Excellent for self-study with exercises and solutions
3. **"Expert C Programming: Deep C Secrets"** by Peter van der Linden
    - Advanced C programming techniques and lesser-known features
    - Covers complex topics like pointers, memory management, and optimization
    - Provides insights into the internals of C implementations

Systems Programming

4. **"Advanced Programming in the UNIX Environment"** by W. Richard Stevens
    - Essential reference for UNIX/Linux systems programming
    - Covers file I/O, process control, inter-process communication
    - Detailed explanations of system calls and library functions
5. **"UNIX Network Programming"** by W. Richard Stevens
    - Comprehensive guide to network programming in C
    - Covers sockets, protocols, and network applications
    - Invaluable for developing networked applications

Data Structures and Algorithms

6. **"Algorithms in C"** by Robert Sedgewick
    - Implementation of fundamental algorithms in C
    - Covers sorting, searching, graph algorithms, and more

- Excellent reference for algorithmic problem-solving

7. **"Data Structures and Algorithms in C"** by Michael T. Goodrich
    - Comprehensive coverage of data structures with C implementations
    - Includes analysis of time and space complexity
    - Practical approach to algorithm design and implementation

Online Resources

Official Documentation

1. **ISO C Standard Documents**
    - C17/C18: ISO/IEC 9899:2018
    - C11: ISO/IEC 9899:2011
    - C99: ISO/IEC 9899:1999
    - Available from ISO for purchase, but drafts are often available
2. **GNU C Library Documentation**
    - Comprehensive reference for glibc functions
    - Available at: https://www.gnu.org/software/libc/manual/
    - Detailed explanations of standard library functions
3. **POSIX Standards**
    - Portable Operating System Interface standards
    - Defines system interfaces and command-line shells
    - Available at: https://pubs.opengroup.org/onlinepubs/9699919799/

Tutorials and Learning Platforms

4. **GeeksforGeeks C Programming**
    - Extensive collection of C programming tutorials
    - Covers basic to advanced topics with examples
    - Available at: https://www.geeksforgeeks.org/c-programming-language/
5. **C Programming.com**
    - Beginner-friendly tutorials and references
    - Interactive compiler for testing code examples
    - Available at: https://www.cprogramming.com/
6. **Learn-C.org**
    - Free interactive C tutorial
    - Browser-based coding environment
    - Available at: https://www.learn-c.org/

Reference Sites

7. **cppreference.com**
    - Comprehensive C and C++ reference
    - Detailed documentation of standard library functions
    - Available at: https://en.cppreference.com/w/c
8. **Linux Man Pages**
    - System documentation for UNIX/Linux commands and functions
    - Accessible via `man` command or online at: https://man7.org/linux/man-pages/
    - Essential for systems programming

Development Tools

Compilers

1. **GCC (GNU Compiler Collection)**
    - The most widely used C compiler
    - Supports all C standards and many extensions
    - Available at: https://gcc.gnu.org/
2. **Clang/LLVM**
    - Modern C compiler with excellent diagnostics
    - Fast compilation and good optimization
    - Available at: https://clang.llvm.org/
3. **Microsoft Visual C++**
    - Microsoft's C compiler for Windows
    - Integrated with Visual Studio IDE
    - Part of Visual Studio Community (free)

Debugging Tools

4. **GDB (GNU Debugger)**
    - Powerful command-line debugger for C programs
    - Supports breakpoints, watchpoints, and stack tracing
    - Available at: https://www.gnu.org/software/gdb/
5. **Valgrind**
    - Memory debugging and profiling tool
    - Detects memory leaks, invalid memory access, and more
    - Available at: https://valgrind.org/
6. **AddressSanitizer**

- Fast memory error detector
- Part of GCC and Clang compilers
- Detects buffer overflows, use-after-free, and other errors

Profiling Tools

7. **gprof**
    - GNU profiler for analyzing program performance
    - Shows function call counts and execution times
    - Part of GNU binutils
8. **perf**
    - Linux performance analysis tool
    - Hardware performance counter support
    - Available on most Linux distributions

Static Analysis Tools

9. **Cppcheck**
    - Static analysis tool for C/C++ code
    - Detects bugs, memory leaks, and style issues
    - Available at: http://cppcheck.sourceforge.net/
10. **Clang Static Analyzer**
    - Source code analysis tool for C/C++
    - Finds bugs and potential security issues
    - Part of Clang/LLVM project

Libraries and Frameworks

Standard Libraries

1. **glibc (GNU C Library)**
    - The standard C library on most Linux systems
    - Comprehensive implementation of ISO C standard
    - Available at: https://www.gnu.org/software/libc/
2. **musl libc**
    - Lightweight, standards-conforming C library
    - Designed for static linking and embedded systems
    - Available at: https://musl.libc.org/

Networking Libraries

3. **libcurl**
    - Multiprotocol file transfer library
    - Supports HTTP, FTP, SMTP, and many other protocols
    - Available at: https://curl.se/libcurl/
4. **libevent**
    - Event notification library for developing scalable network applications
    - Supports event-driven programming model
    - Available at: http://libevent.org/

Data Structure Libraries

5. **GLib**
    - General-purpose utility library from GNOME project
    - Provides data structures, string utilities, and more
    - Available at: https://developer.gnome.org/glib/
6. **Apache Portable Runtime (APR)**
    - Cross-platform library for system programming
    - Provides memory management, file I/O, and threading utilities
    - Available at: https://apr.apache.org/

Communities and Forums

Online Communities

1. **Stack Overflow**
    - Large community of developers answering programming questions
    - Extensive C programming tag with thousands of questions
    - Available at: https://stackoverflow.com/questions/tagged/c
2. **Reddit r/C_Programming**
    - Community discussion forum for C programmers
    - News, discussions, and code reviews
    - Available at: https://www.reddit.com/r/C_Programming/
3. **C Board**
    - Dedicated C programming forum
    - Friendly community for beginners and experts
    - Available at: http://cboard.cprogramming.com/

Mailing Lists

4. **comp.lang.c**
    - Usenet newsgroup for C programming discussions
    - Long-running community with expert participants
    - Available through Google Groups

Conferences and Events

Major Conferences

1. **ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)**
    - Premier conference for programming language research
    - Includes C-related research and developments
2. **USENIX Annual Technical Conference**
    - Focuses on systems implementation and applications
    - Relevant for systems programming in C
3. **Linux Plumbers Conference**
    - Focuses on Linux kernel and systems development
    - Many sessions relevant to C systems programming

Open Source Projects

Learning from Real Code

1. **Linux Kernel**
    - Largest open-source C project
    - Excellent example of systems programming
    - Available at: https://github.com/torvalds/linux
2. **Git**
    - Distributed version control system written in C
    - Well-designed codebase with good documentation
    - Available at: https://github.com/git/git
3. **Redis**
    - In-memory data structure store written in C
    - Clean, well-documented codebase
    - Available at: https://github.com/redis/redis
4. **Nginx**

- High-performance web server written in C
- Excellent example of network programming
- Available at: https://github.com/nginx/nginx

Practice Platforms

Coding Challenges

1. **LeetCode**
   - Programming practice platform with C support
   - Thousands of coding problems with solutions
   - Available at: https://leetcode.com/
2. **HackerRank**
   - Coding practice and competition platform
   - Dedicated C programming section
   - Available at: https://www.hackerrank.com/
3. **Project Euler**
   - Mathematical and computational programming challenges
   - Excellent for practicing algorithms and optimization
   - Available at: https://projecteuler.net/

Standards Organizations

Key Organizations

1. **ISO/IEC JTC1/SC22/WG14**
   - International standardization working group for C
   - Responsible for the ISO C standard
   - Website: https://www.open-std.org/jtc1/sc22/wg14/
2. **ANSI C Standards Committee**
   - American National Standards Institute committee
   - Works with ISO on C standard development

Additional Learning Resources

Video Courses

1. **CS50 (Harvard)**
   - Introduction to Computer Science with C programming

- Free online course with video lectures
- Available at: https://cs50.harvard.edu/

2. **MIT OpenCourseWare**
    - Free lecture videos and materials for computer science courses
    - Several courses include C programming content
    - Available at: https://ocw.mit.edu/

Podcasts

1. **C Developers Journal**
    - Podcast focused on C programming and systems development
    - Interviews with C experts and discussions of best practices

2. **Software Engineering Daily**
    - Daily interviews with software engineers
    - Many episodes relevant to systems programming in C

This appendix provides a comprehensive list of resources to help you continue your journey in C programming. Whether you're looking to deepen your understanding of specific topics, find tools to improve your development workflow, or connect with the C programming community, these resources will serve as valuable references.

Appendix B: C Language Reference

C Keywords

Standard C Keywords (C17/C18)

```
auto     break    case     char     const    continue  default  do
double   else     enum     extern   float    for       goto     if
inline   int      long     register return   short     signed   sizeof
static   struct   switch   typedef  union    unsigned  void     volatile
while    _Alignas _Alignof _Atomic  _Bool    _Complex _Generic
_Imaginary _Noreturn _Static_assert _Thread_local
```

Deprecated Keywords

- `restrict` (C99-C17, still available but not commonly used)

Data Types and Sizes

Basic Data Types

Type	Typical Size (64-bit)	Range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
int	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long	8 bytes	0 to 18,446,744,073,709,551,615

Type	Typical Size (64-bit)	Range
float	4 bytes	1.2E-38 to 3.4E+38 (6 decimal places)
double	8 bytes	2.3E-308 to 1.7E+308 (15 decimal places)
long double	16 bytes	3.4E-4932 to 1.1E+4932 (19 decimal places)

Type Specifiers

- `signed`: Explicitly signed integers
- `unsigned`: Non-negative integers
- `short`: Shorter integers
- `long`: Longer integers
- `long long`: Even longer integers (C99)

Type Qualifiers

- `const`: Read-only data
- `volatile`: Data that may change unexpectedly
- `restrict`: Pointer optimization hint (C99)
- `_Atomic`: Atomic operations (C11)

Operators

Arithmetic Operators

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b
++	Increment	++a or a++
–	Decrement	–a or a–

Relational Operators

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Logical Operators

Operator	Description	Example
&&	Logical AND	a && b
||	Logical OR	a || b
!	Logical NOT	!a

Bitwise Operators

Operator	Description	Example
&	Bitwise AND	a & b
|	Bitwise OR	a | b
^	Bitwise XOR	a ^ b
~	Bitwise NOT (one's complement)	~a
«	Left shift	a « b
»	Right shift	a » b

Assignment Operators

Operator	Description	Example	Equivalent
=	Simple assignment	a = b	a = b
+=	Add and assign	a += b	a = a + b
-=	Subtract and assign	a -= b	a = a - b
*=	Multiply and assign	a *= b	a = a * b
/=	Divide and assign	a /= b	a = a / b
%=	Modulus and assign	a %= b	a = a % b
&=	Bitwise AND and assign	a &= b	a = a & b
|=	Bitwise OR and assign	a |= b	a = a | b
^=	Bitwise XOR and assign	a ^= b	a = a ^ b
«=	Left shift and assign	a «= b	a = a « b
»=	Right shift and assign	a »= b	a = a » b

Other Operators

Operator	Description	Example
sizeof	Size of data type or variable	sizeof(int)
&	Address of	&a
*	Value at address (dereference)	*ptr
?:	Conditional (ternary)	condition ? a : b
,	Comma	a, b
.	Member access	struct.member
->	Pointer member access	ptr->member
[]	Array subscript	arr[i]
()	Function call	func()

Standard Library Functions

Input/Output Functions (<stdio.h>)

```
// Formatted I/O
int printf(const char *format, ...);
int scanf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
int sprintf(char *str, const char *format, ...);
int snprintf(char *str, size_t size, const char *format, ...);

// Character I/O
int getchar(void);
int putchar(int c);
int fgetc(FILE *stream);
int fputc(int c, FILE *stream);

// String I/O
char *fgets(char *str, int n, FILE *stream);
int fputs(const char *str, FILE *stream);

// File operations
FILE *fopen(const char *filename, const char *mode);
int fclose(FILE *stream);
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
int fseek(FILE *stream, long offset, int whence);
long ftell(FILE *stream);
void rewind(FILE *stream);
```

String Functions (<string.h>)

```
// String copying
char *strcpy(char *dest, const char *src);
char *strncpy(char *dest, const char *src, size_t n);

// String concatenation
char *strcat(char *dest, const char *src);
char *strncat(char *dest, const char *src, size_t n);

// String comparison
int strcmp(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);

// String searching
char *strchr(const char *s, int c);
char *strrchr(const char *s, int c);
char *strstr(const char *haystack, const char *needle);
size_t strlen(const char *s);

// Memory operations
void *memcpy(void *dest, const void *src, size_t n);
void *memmove(void *dest, const void *src, size_t n);
int memcmp(const void *s1, const void *s2, size_t n);
void *memset(void *s, int c, size_t n);
```

Memory Management Functions (<stdlib.h>)

```
void *malloc(size_t size);
void *calloc(size_t nmemb, size_t size);
void *realloc(void *ptr, size_t size);
void free(void *ptr);
```

Mathematical Functions (<math.h>)

```
// Trigonometric functions
double sin(double x);
double cos(double x);
double tan(double x);
double asin(double x);
double acos(double x);
double atan(double x);
double atan2(double y, double x);
```

```c
// Hyperbolic functions
double sinh(double x);
double cosh(double x);
double tanh(double x);

// Exponential and logarithmic functions
double exp(double x);
double log(double x);    // Natural logarithm
double log10(double x);  // Base-10 logarithm
double pow(double x, double y);
double sqrt(double x);

// Rounding and absolute value
double ceil(double x);
double floor(double x);
double fabs(double x);
double fmod(double x, double y);
```

Utility Functions (<stdlib.h>)

```c
// Conversion functions
int atoi(const char *nptr);
long atol(const char *nptr);
double atof(const char *nptr);
long int strtol(const char *nptr, char **endptr, int base);
unsigned long int strtoul(const char *nptr, char **endptr, int base);

// Random number generation
int rand(void);
void srand(unsigned int seed);

// Searching and sorting
void *bsearch(const void *key, const void *base,
              size_t nmemb, size_t size,
              int (*compar)(const void *, const void *));
void qsort(void *base, size_t nmemb, size_t size,
           int (*compar)(const void *, const void *));

// Process control
void exit(int status);
void abort(void);
int atexit(void (*func)(void));
```

Time Functions (<time.h>)

```
time_t time(time_t *tloc);
char *ctime(const time_t *timep);
struct tm *localtime(const time_t *timep);
struct tm *gmtime(const time_t *timep);
time_t mktime(struct tm *tm);
size_t strftime(char *s, size_t max, const char *format,
                const struct tm *tm);
```

Preprocessor Directives

Common Directives

```
#define MACRO_NAME value
#undef MACRO_NAME
#include <header.h>
#include "file.h"
#if condition
#ifdef MACRO
#ifndef MACRO
#else
#elif condition
#endif
#error message
#pragma directive
#line number "filename"
```

Standard Macros

```
__LINE__     // Current line number
__FILE__     // Current filename
__DATE__     // Compilation date
__TIME__     // Compilation time
__STDC__     // Standard C compliance
__STDC_VERSION__ // C standard version
__STDC_HOSTED__  // Hosted implementation
```

Format Specifiers

printf Format Specifiers

Specifier	Description	Example
%d, %i	Signed decimal integer	printf(“%d”, 42);
%u	Unsigned decimal integer	printf(“%u”, 42u);
%o	Octal integer	printf(“%o”, 42);
%x, %X	Hexadecimal integer	printf(“%x”, 42);
%f, %F	Decimal floating point	printf(“%f”, 3.14);
%e, %E	Scientific notation	printf(“%e”, 3.14);
%g, %G	Use %f or %e, whichever is shorter	printf(“%g”, 3.14);
%c	Character	printf(“%c”, ‘A’);
%s	String	printf(“%s”, “Hello”);
%p	Pointer address	printf(“%p”, ptr);
%n	Number of characters written so far	printf(“%n”, &count);

scanf Format Specifiers

Specifier	Description	Example
%d	Signed decimal integer	scanf(“%d”, &num);
%u	Unsigned decimal integer	scanf(“%u”, &num);
%o	Octal integer	scanf(“%o”, &num);
%x	Hexadecimal integer	scanf(“%x”, &num);
%f	Floating point	scanf(“%f”, &num);
%e	Floating point	scanf(“%e”, &num);
%g	Floating point	scanf(“%g”, &num);
%c	Character	scanf(“%c”, &ch);
%s	String	scanf(“%s”, str);
%p	Pointer	scanf(“%p”, &ptr);
%[…]	Character set	scanf(“%[aeiou]”, str);

Format Modifiers

Modifier	Description	Example
hh	char (char)	printf(“%hhd”, c);
h	short (short)	printf(“%hd”, s);
l	long (long)	printf(“%ld”, l);
ll	long long (long long)	printf(“%lld”, ll);
j	intmax_t	printf(“%jd”, imax);
z	size_t	printf(“%zd”, sz);
t	ptrdiff_t	printf(“%td”, ptrdiff);
L	long double	printf(“%Lf”, ld);

Escape Sequences

Standard Escape Sequences

Sequence	Description	ASCII Value
\a	Alert (bell)	007
\b	Backspace	008
\f	Form feed	012
\n	New line	010
\r	Carriage return	013
\t	Horizontal tab	009
\v	Vertical tab	011
\\	Backslash	092
\’	Single quote	039
\”	Double quote	034
\?	Question mark	063
\0	Null character	000

Octal and Hexadecimal Escapes

Sequence	Description	Example
\\ooo	Octal character	\\040 (space)
\\xhh	Hexadecimal character	\\x20 (space)

Storage Classes

Storage Class Specifiers

Specifier	Storage Duration	Linkage	Scope
auto	Automatic	None	Block
register	Automatic	None	Block
static	Static	Internal	File/Block
extern	Static	External	File
typedef	N/A	N/A	File

Error Handling

errno Values (<errno.h>)

```
EDOM     // Domain error
ERANGE   // Range error
EILSEQ   // Illegal byte sequence
```

Assertions (<assert.h>)

```
void assert(int expression);
```

Signal Handling (<signal.h>)

Signal Types

```
SIGABRT  // Abnormal termination
SIGFPE   // Floating-point error
SIGILL   // Illegal instruction
SIGINT   // Interactive attention signal
SIGSEGV  // Invalid memory access
SIGTERM  // Termination request
```

Signal Functions

```
void (*signal(int sig, void (*func)(int)))(int);
int raise(int sig);
```

File Operations Modes

fopen Modes

Mode	Description	File Position	File Creation
“r”	Read only	Beginning	No
“w”	Write only	Beginning	Yes (truncates)
“a”	Append only	End	Yes
“r+”	Read/Write	Beginning	No
“w+”	Read/Write	Beginning	Yes (truncates)

Mode	Description	File Position	File Creation
“a+”	Read/Append	End	Yes

Binary Mode Suffix

Add ‘b’ to any mode for binary files (e.g., “rb”, “wb”, “ab”)

Standard Streams

Predefined Streams

```
stdin   // Standard input (keyboard)
stdout  // Standard output (screen)
stderr  // Standard error (screen)
```

Limits and Constants

Implementation Limits (<limits.h>)

```
CHAR_BIT     // Bits in a char (8)
CHAR_MAX     // Maximum char value
CHAR_MIN     // Minimum char value
INT_MAX      // Maximum int value
INT_MIN      // Minimum int value
LONG_MAX     // Maximum long value
LONG_MIN     // Minimum long value
SCHAR_MAX    // Maximum signed char value
SCHAR_MIN    // Minimum signed char value
SHRT_MAX     // Maximum short value
SHRT_MIN     // Minimum short value
UCHAR_MAX    // Maximum unsigned char value
UINT_MAX     // Maximum unsigned int value
ULONG_MAX    // Maximum unsigned long value
USHRT_MAX    // Maximum unsigned short value
```

Floating-Point Limits (<float.h>)

```
FLT_DIG      // Decimal digits of precision
FLT_EPSILON  // Smallest x where 1.0 + x != 1.0
FLT_MANT_DIG // Number of digits in mantissa
FLT_MAX      // Maximum float value
FLT_MAX_10_EXP // Maximum decimal exponent
FLT_MIN      // Minimum float value
FLT_MIN_10_EXP // Minimum decimal exponent
DBL_DIG      // Decimal digits of precision (double)
```

Thread Support (C11) (<threads.h>)

Thread Functions

```
int thrd_create(thrd_t *thr, thrd_start_t func, void *arg);
int thrd_equal(thrd_t thr0, thrd_t thr1);
thrd_t thrd_current(void);
int thrd_sleep(const struct timespec *duration, struct timespec *remaining);
void thrd_yield(void);
_Noreturn void thrd_exit(int res);
int thrd_detach(thrd_t thr);
int thrd_join(thrd_t thr, int *res);
```

Mutex Functions

```
int mtx_init(mtx_t *mtx, int type);
void mtx_destroy(mtx_t *mtx);
int mtx_lock(mtx_t *mtx);
int mtx_timedlock(mtx_t *mtx, const struct timespec *ts);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
```

Condition Variable Functions

```
int cnd_init(cnd_t *cond);
void cnd_destroy(cnd_t *cond);
int cnd_signal(cnd_t *cond);
int cnd_broadcast(cnd_t *cond);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
int cnd_timedwait(cnd_t *cond, mtx_t *mtx, const struct timespec *ts);
```

Atomic Operations (C11) (<stdatomic.h>)

Atomic Types

```
_Atomic bool
_Atomic char
_Atomic short
_Atomic int
_Atomic long
_Atomic long long
_Atomic unsigned char
_Atomic unsigned short
_Atomic unsigned int
_Atomic unsigned long
_Atomic unsigned long long
```

Atomic Operations

```
memory_order memory_order_relaxed;
memory_order memory_order_consume;
memory_order memory_order_acquire;
memory_order memory_order_release;
memory_order memory_order_acq_rel;
memory_order memory_order_seq_cst;

#define ATOMIC_VAR_INIT(value)
#define ATOMIC_FLAG_INIT

bool atomic_flag_test_and_set(volatile atomic_flag *object);
void atomic_flag_clear(volatile atomic_flag *object);
```

Generic Selection (C11) (<stdnoreturn.h>)

_Generic Keyword

```
#define MAX(x, y) _Generic((x), \
    int: max_int, \
    float: max_float, \
    double: max_double \
)(x, y)
```

Alignment (C11) (<stdalign.h>)

Alignment Specifiers

```
_Alignas(expression)
alignas(type)
_Alignof(type)
alignof(type)
```

This reference appendix provides a comprehensive overview of the C programming language, covering keywords, operators, standard library functions, and other essential elements. It serves as a quick reference for C programmers of all levels.

Appendix C: Exercise Solutions

This appendix provides solutions to selected exercises from each module of the course. These solutions are meant to serve as examples of good programming practices and to help you understand the concepts covered in each module.

Module 1: Foundations

Exercise 1.1: Hello World Variations

```
#include <stdio.h>

int main() {
    // Basic Hello World
    printf("Hello, World!\n");

    // Personalized greeting
    printf("Hello, C Programmer!\n");

    // Multi-line message
    printf("Welcome to C programming.\n");
    printf("This is the beginning of your journey.\n");

    return 0;
}
```

Exercise 1.2: Basic Calculator

```
#include <stdio.h>

int main() {
    double num1, num2;
    char operator;

    printf("Enter first number: ");
    scanf("%lf", &num1);

    printf("Enter an operator (+, -, *, /): ");
```

```
    scanf(" %c", &operator);

    printf("Enter second number: ");
    scanf("%lf", &num2);

    switch (operator) {
        case '+':
            printf("%.2lf + %.2lf = %.2lf\n", num1, num2, num1 + num2);
            break;
        case '-':
            printf("%.2lf - %.2lf = %.2lf\n", num1, num2, num1 - num2);
            break;
        case '*':
            printf("%.2lf * %.2lf = %.2lf\n", num1, num2, num1 * num2);
            break;
        case '/':
            if (num2 != 0) {
                printf("%.2lf / %.2lf = %.2lf\n", num1, num2, num1 / num2);
            } else {
                printf("Error: Division by zero\n");
            }
            break;
        default:
            printf("Error: Invalid operator\n");
    }

    return 0;
}
```

Module 2: Data Types

Exercise 2.1: Size and Range Calculator

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main() {
    printf("Data Type Sizes and Ranges:\n");
    printf("===========================\n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("  Range: %d to %d\n", CHAR_MIN, CHAR_MAX);
```

```
    printf("unsigned char: %zu bytes\n", sizeof(unsigned char));
    printf("  Range: 0 to %u\n", UCHAR_MAX);

    printf("short: %zu bytes\n", sizeof(short));
    printf("  Range: %d to %d\n", SHRT_MIN, SHRT_MAX);

    printf("unsigned short: %zu bytes\n", sizeof(unsigned short));
    printf("  Range: 0 to %u\n", USHRT_MAX);

    printf("int: %zu bytes\n", sizeof(int));
    printf("  Range: %d to %d\n", INT_MIN, INT_MAX);

    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("  Range: 0 to %u\n", UINT_MAX);

    printf("long: %zu bytes\n", sizeof(long));
    printf("  Range: %ld to %ld\n", LONG_MIN, LONG_MAX);

    printf("unsigned long: %zu bytes\n", sizeof(unsigned long));
    printf("  Range: 0 to %lu\n", ULONG_MAX);

    printf("float: %zu bytes\n", sizeof(float));
    printf("  Range: %e to %e\n", FLT_MIN, FLT_MAX);
    printf("  Precision: %d digits\n", FLT_DIG);

    printf("double: %zu bytes\n", sizeof(double));
    printf("  Range: %e to %e\n", DBL_MIN, DBL_MAX);
    printf("  Precision: %d digits\n", DBL_DIG);

    return 0;
}
```

Exercise 2.2: Temperature Converter

```
#include <stdio.h>

double celsius_to_fahrenheit(double celsius) {
    return (celsius * 9.0 / 5.0) + 32.0;
}

double fahrenheit_to_celsius(double fahrenheit) {
    return (fahrenheit - 32.0) * 5.0 / 9.0;
}

int main() {
```

```
    double temperature;
    int choice;

    printf("Temperature Converter\n");
    printf("1. Celsius to Fahrenheit\n");
    printf("2. Fahrenheit to Celsius\n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Enter temperature in Celsius: ");
        scanf("%lf", &temperature);
        printf("%.2lf°C = %.2lf°F\n", temperature, celsius_to_fahrenheit(temperature));
    } else if (choice == 2) {
        printf("Enter temperature in Fahrenheit: ");
        scanf("%lf", &temperature);
        printf("%.2lf°F = %.2lf°C\n", temperature, fahrenheit_to_celsius(temperature));
    } else {
        printf("Invalid choice\n");
    }

    return 0;
}
```

Module 3: Control Flow

Exercise 3.1: Prime Number Checker

```
#include <stdio.h>
#include <stdbool.h>

bool is_prime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}

int main() {
```

```
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (is_prime(number)) {
        printf("%d is a prime number\n", number);
    } else {
        printf("%d is not a prime number\n", number);
    }

    return 0;
}
```

Exercise 3.2: Pattern Printer

```
#include <stdio.h>

void print_triangle(int n) {
    for (int i = 1; i <= n; i++) {
        // Print spaces
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        // Print stars
        for (int j = 1; j <= 2 * i - 1; j++) {
            printf("*");
        }
        printf("\n");
    }
}

void print_diamond(int n) {
    // Upper half (including middle)
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            printf("*");
        }
        printf("\n");
    }

    // Lower half
```

```
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            printf("*");
        }
        printf("\n");
    }
}

int main() {
    int n;
    int choice;

    printf("Pattern Printer\n");
    printf("1. Triangle\n");
    printf("2. Diamond\n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    printf("Enter the size: ");
    scanf("%d", &n);

    if (choice == 1) {
        print_triangle(n);
    } else if (choice == 2) {
        print_diamond(n);
    } else {
        printf("Invalid choice\n");
    }

    return 0;
}
```

Module 4: Functions

Exercise 4.1: Mathematical Functions Library

```
#include <stdio.h>
#include <math.h>

// Function to calculate factorial
long long factorial(int n) {
```

```c
    if (n < 0) return -1;  // Error case
    if (n == 0 || n == 1) return 1;

    long long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

// Function to calculate power
double power(double base, int exponent) {
    if (exponent == 0) return 1.0;

    double result = 1.0;
    int abs_exp = (exponent < 0) ? -exponent : exponent;

    for (int i = 0; i < abs_exp; i++) {
        result *= base;
    }

    return (exponent < 0) ? 1.0 / result : result;
}

// Function to calculate GCD using Euclidean algorithm
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Function to calculate LCM
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int choice, a, b;
    double x;
    int n;

    printf("Mathematical Functions Library\n");
    printf("1. Factorial\n");
    printf("2. Power\n");
```

```
    printf("3. GCD\n");
    printf("4. LCM\n");
    printf("Enter your choice (1-4): ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter a number: ");
            scanf("%d", &n);
            if (n >= 0) {
                printf("Factorial of %d = %lld\n", n, factorial(n));
            } else {
                printf("Factorial is not defined for negative numbers\n");
            }
            break;

        case 2:
            printf("Enter base: ");
            scanf("%lf", &x);
            printf("Enter exponent: ");
            scanf("%d", &n);
            printf("%.2lf^%d = %.2lf\n", x, n, power(x, n));
            break;

        case 3:
            printf("Enter first number: ");
            scanf("%d", &a);
            printf("Enter second number: ");
            scanf("%d", &b);
            printf("GCD of %d and %d = %d\n", a, b, gcd(a, b));
            break;

        case 4:
            printf("Enter first number: ");
            scanf("%d", &a);
            printf("Enter second number: ");
            scanf("%d", &b);
            printf("LCM of %d and %d = %d\n", a, b, lcm(a, b));
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

Exercise 4.2: Recursive Functions

```c
#include <stdio.h>

// Recursive factorial
long long recursive_factorial(int n) {
    if (n < 0) return -1;
    if (n == 0 || n == 1) return 1;
    return n * recursive_factorial(n - 1);
}

// Recursive Fibonacci
int recursive_fibonacci(int n) {
    if (n <= 0) return 0;
    if (n == 1) return 1;
    return recursive_fibonacci(n - 1) + recursive_fibonacci(n - 2);
}

// Recursive power
double recursive_power(double base, int exponent) {
    if (exponent == 0) return 1.0;
    if (exponent > 0) {
        return base * recursive_power(base, exponent - 1);
    } else {
        return 1.0 / base * recursive_power(base, exponent + 1);
    }
}

int main() {
    int choice, n;
    double x;
    int exp;

    printf("Recursive Functions\n");
    printf("1. Factorial\n");
    printf("2. Fibonacci\n");
    printf("3. Power\n");
    printf("Enter your choice (1-3): ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter a number: ");
            scanf("%d", &n);
            if (n >= 0) {
                printf("Factorial of %d = %lld\n", n, recursive_factorial(n));
            } else {
```

```
                printf("Factorial is not defined for negative numbers\n");
            }
            break;

        case 2:
            printf("Enter position: ");
            scanf("%d", &n);
            if (n >= 0) {
                printf("Fibonacci number at position %d = %d\n", n, recursive_fibonacci(n));
            } else {
                printf("Invalid position\n");
            }
            break;

        case 3:
            printf("Enter base: ");
            scanf("%lf", &x);
            printf("Enter exponent: ");
            scanf("%d", &exp);
            printf("%.2lf^%d = %.2lf\n", x, exp, recursive_power(x, exp));
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

Module 5: Arrays and Strings

Exercise 5.1: Array Operations

```
#include <stdio.h>
#include <limits.h>

// Function to find maximum element in array
int find_max(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
```

```c
    return max;
}

// Function to find minimum element in array
int find_min(int arr[], int size) {
    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}

// Function to calculate average
double calculate_average(int arr[], int size) {
    long long sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return (double)sum / size;
}

// Function to reverse array
void reverse_array(int arr[], int size) {
    for (int i = 0; i < size / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[size - 1 - i];
        arr[size - 1 - i] = temp;
    }
}

// Function to print array
void print_array(int arr[], int size) {
    printf("[");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i < size - 1) printf(", ");
    }
    printf("]\n");
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
```

```
    print_array(arr, size);

    printf("Maximum element: %d\n", find_max(arr, size));
    printf("Minimum element: %d\n", find_min(arr, size));
    printf("Average: %.2f\n", calculate_average(arr, size));

    reverse_array(arr, size);
    printf("Reversed array: ");
    print_array(arr, size);

    return 0;
}
```

Exercise 5.2: String Manipulation

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function to count vowels
int count_vowels(const char *str) {
    int count = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        char ch = tolower(str[i]);
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
            count++;
        }
    }
    return count;
}

// Function to reverse string
void reverse_string(char *str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        char temp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = temp;
    }
}

// Function to check palindrome
int is_palindrome(const char *str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
```

```
        if (tolower(str[i]) != tolower(str[len - 1 - i])) {
            return 0;
        }
    }
    return 1;
}

// Function to remove spaces
void remove_spaces(char *str) {
    int i, j = 0;
    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] != ' ') {
            str[j++] = str[i];
        }
    }
    str[j] = '\0';
}

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // Remove newline character if present
    str[strcspn(str, "\n")] = 0;

    printf("Original string: %s\n", str);
    printf("Number of vowels: %d\n", count_vowels(str));

    char str_copy[100];
    strcpy(str_copy, str);
    reverse_string(str_copy);
    printf("Reversed string: %s\n", str_copy);

    if (is_palindrome(str)) {
        printf("The string is a palindrome\n");
    } else {
        printf("The string is not a palindrome\n");
    }

    strcpy(str_copy, str);
    remove_spaces(str_copy);
    printf("String without spaces: %s\n", str_copy);

    return 0;
}
```

Module 6: Pointers

Exercise 6.1: Pointer Arithmetic

```c
#include <stdio.h>

// Function to swap two integers using pointers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to find maximum in array using pointers
int* find_max_ptr(int *arr, int size) {
    int *max = arr;
    for (int i = 1; i < size; i++) {
        if (*(arr + i) > *max) {
            max = arr + i;
        }
    }
    return max;
}

// Function to reverse array using pointers
void reverse_array_ptr(int *arr, int size) {
    int *start = arr;
    int *end = arr + size - 1;

    while (start < end) {
        int temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

// Function to print array using pointers
void print_array_ptr(int *arr, int size) {
    printf("[");
    for (int i = 0; i < size; i++) {
        printf("%d", *(arr + i));
        if (i < size - 1) printf(", ");
    }
    printf("]\n");
```

```
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    print_array_ptr(arr, size);

    int a = 10, b = 20;
    printf("Before swap: a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("After swap: a = %d, b = %d\n", a, b);

    int *max_ptr = find_max_ptr(arr, size);
    printf("Maximum element: %d at address %p\n", *max_ptr, (void*)max_ptr);

    reverse_array_ptr(arr, size);
    printf("Reversed array: ");
    print_array_ptr(arr, size);

    return 0;
}
```

Exercise 6.2: Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>

// Function to create dynamic array
int* create_array(int size) {
    int *arr = (int*)malloc(size * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return NULL;
    }
    return arr;
}

// Function to initialize array with random values
void initialize_array(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 100;  // Random numbers 0-99
    }
}
```

```c
// Function to resize array
int* resize_array(int *arr, int old_size, int new_size) {
    int *new_arr = (int*)realloc(arr, new_size * sizeof(int));
    if (new_arr == NULL) {
        printf("Memory reallocation failed\n");
        return arr;  // Return original array
    }

    // Initialize new elements to 0
    if (new_size > old_size) {
        for (int i = old_size; i < new_size; i++) {
            new_arr[i] = 0;
        }
    }

    return new_arr;
}

// Function to free array
void free_array(int *arr) {
    free(arr);
}

// Function to print array
void print_array(int *arr, int size) {
    printf("[");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i < size - 1) printf(", ");
    }
    printf("]\n");
}

int main() {
    int size = 5;
    int *arr = create_array(size);

    if (arr != NULL) {
        initialize_array(arr, size);
        printf("Original array: ");
        print_array(arr, size);

        // Resize array
        int new_size = 8;
        arr = resize_array(arr, size, new_size);
        if (arr != NULL) {
```

```
            printf("Resized array: ");
            print_array(arr, new_size);
        }

        free_array(arr);
        printf("Memory freed successfully\n");
    }

    return 0;
}
```

Module 7: Structures

Exercise 7.1: Student Database

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_STUDENTS 100

typedef struct {
    int id;
    char name[50];
    int age;
    float gpa;
} Student;

// Function to add student
void add_student(Student students[], int *count) {
    if (*count >= MAX_STUDENTS) {
        printf("Database is full\n");
        return;
    }

    Student *s = &students[*count];

    printf("Enter student ID: ");
    scanf("%d", &s->id);

    printf("Enter student name: ");
    scanf(" %[^\n]", s->name);

    printf("Enter student age: ");
    scanf("%d", &s->age);
```

```
    printf("Enter student GPA: ");
    scanf("%f", &s->gpa);

    (*count)++;
    printf("Student added successfully\n");
}

// Function to display all students
void display_students(Student students[], int count) {
    if (count == 0) {
        printf("No students in database\n");
        return;
    }

    printf("\nStudent Database:\n");
    printf("ID\tName\t\t\tAge\tGPA\n");
    printf("----------------------------------------\n");

    for (int i = 0; i < count; i++) {
        Student *s = &students[i];
        printf("%d\t%-20s\t%d\t%.2f\n", s->id, s->name, s->age, s->gpa);
    }
}

// Function to find student by ID
Student* find_student(Student students[], int count, int id) {
    for (int i = 0; i < count; i++) {
        if (students[i].id == id) {
            return &students[i];
        }
    }
    return NULL;
}

// Function to calculate average GPA
float calculate_average_gpa(Student students[], int count) {
    if (count == 0) return 0.0;

    float sum = 0.0;
    for (int i = 0; i < count; i++) {
        sum += students[i].gpa;
    }
    return sum / count;
}

int main() {
```

```c
    Student students[MAX_STUDENTS];
    int count = 0;
    int choice, id;

    while (1) {
        printf("\nStudent Database Menu:\n");
        printf("1. Add Student\n");
        printf("2. Display All Students\n");
        printf("3. Find Student by ID\n");
        printf("4. Calculate Average GPA\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                add_student(students, &count);
                break;

            case 2:
                display_students(students, count);
                break;

            case 3:
                printf("Enter student ID: ");
                scanf("%d", &id);
                Student *s = find_student(students, count, id);
                if (s != NULL) {
                    printf("Student found:\n");
                    printf("ID: %d\n", s->id);
                    printf("Name: %s\n", s->name);
                    printf("Age: %d\n", s->age);
                    printf("GPA: %.2f\n", s->gpa);
                } else {
                    printf("Student not found\n");
                }
                break;

            case 4:
                printf("Average GPA: %.2f\n", calculate_average_gpa(students, count));
                break;

            case 5:
                printf("Exiting program\n");
                exit(0);

            default:
```

```
                printf("Invalid choice\n");
            }
        }

        return 0;
    }
```

Exercise 7.2: Complex Number Operations

```
#include <stdio.h>
#include <math.h>

typedef struct {
    double real;
    double imag;
} Complex;

// Function to create complex number
Complex create_complex(double real, double imag) {
    Complex c = {real, imag};
    return c;
}

// Function to add two complex numbers
Complex add_complex(Complex a, Complex b) {
    Complex result;
    result.real = a.real + b.real;
    result.imag = a.imag + b.imag;
    return result;
}

// Function to subtract two complex numbers
Complex subtract_complex(Complex a, Complex b) {
    Complex result;
    result.real = a.real - b.real;
    result.imag = a.imag - b.imag;
    return result;
}

// Function to multiply two complex numbers
Complex multiply_complex(Complex a, Complex b) {
    Complex result;
    result.real = a.real * b.real - a.imag * b.imag;
    result.imag = a.real * b.imag + a.imag * b.real;
    return result;
```

```c
}

// Function to calculate magnitude of complex number
double magnitude_complex(Complex c) {
    return sqrt(c.real * c.real + c.imag * c.imag);
}

// Function to print complex number
void print_complex(Complex c) {
    if (c.imag >= 0) {
        printf("%.2f + %.2fi", c.real, c.imag);
    } else {
        printf("%.2f - %.2fi", c.real, -c.imag);
    }
}

int main() {
    Complex a, b, result;
    int choice;

    printf("Enter first complex number:\n");
    printf("Real part: ");
    scanf("%lf", &a.real);
    printf("Imaginary part: ");
    scanf("%lf", &a.imag);

    printf("Enter second complex number:\n");
    printf("Real part: ");
    scanf("%lf", &b.real);
    printf("Imaginary part: ");
    scanf("%lf", &b.imag);

    while (1) {
        printf("\nComplex Number Operations:\n");
        printf("1. Add\n");
        printf("2. Subtract\n");
        printf("3. Multiply\n");
        printf("4. Magnitude of first number\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                result = add_complex(a, b);
                printf("Result: ");
                print_complex(a);
```

```
                printf(" + ");
                print_complex(b);
                printf(" = ");
                print_complex(result);
                printf("\n");
                break;

            case 2:
                result = subtract_complex(a, b);
                printf("Result: ");
                print_complex(a);
                printf(" - ");
                print_complex(b);
                printf(" = ");
                print_complex(result);
                printf("\n");
                break;

            case 3:
                result = multiply_complex(a, b);
                printf("Result: ");
                print_complex(a);
                printf(" * ");
                print_complex(b);
                printf(" = ");
                print_complex(result);
                printf("\n");
                break;

            case 4:
                printf("Magnitude of ");
                print_complex(a);
                printf(" = %.2f\n", magnitude_complex(a));
                break;

            case 5:
                printf("Exiting program\n");
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Module 8: File I/O

Exercise 8.1: Text File Processor

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

// Function to count lines, words, and characters
void analyze_file(const char *filename) {
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        perror("Error opening file");
        return;
    }

    int lines = 0, words = 0, characters = 0;
    int ch, in_word = 0;

    while ((ch = fgetc(file)) != EOF) {
        characters++;

        if (ch == '\n') {
            lines++;
        }

        if (isspace(ch)) {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            words++;
        }
    }

    // Handle case where file doesn't end with newline
    if (characters > 0 && fseek(file, -1, SEEK_END) == 0) {
        if (fgetc(file) != '\n') {
            lines++;
        }
    }

    fclose(file);

    printf("File Analysis for '%s':\n", filename);
    printf("Lines: %d\n", lines);
```

```
    printf("Words: %d\n", words);
    printf("Characters: %d\n", characters);
}

// Function to copy file
void copy_file(const char *source, const char *destination) {
    FILE *src = fopen(source, "r");
    if (src == NULL) {
        perror("Error opening source file");
        return;
    }

    FILE *dest = fopen(destination, "w");
    if (dest == NULL) {
        perror("Error opening destination file");
        fclose(src);
        return;
    }

    int ch;
    while ((ch = fgetc(src)) != EOF) {
        fputc(ch, dest);
    }

    fclose(src);
    fclose(dest);

    printf("File copied successfully from '%s' to '%s'\n", source, destination);
}

// Function to convert text to uppercase
void convert_to_uppercase(const char *input, const char *output) {
    FILE *in = fopen(input, "r");
    if (in == NULL) {
        perror("Error opening input file");
        return;
    }

    FILE *out = fopen(output, "w");
    if (out == NULL) {
        perror("Error opening output file");
        fclose(in);
        return;
    }

    int ch;
```

```
    while ((ch = fgetc(in)) != EOF) {
        fputc(toupper(ch), out);
    }

    fclose(in);
    fclose(out);

    printf("Text converted to uppercase and saved to '%s'\n", output);
}

int main() {
    int choice;
    char filename1[100], filename2[100];

    while (1) {
        printf("\nText File Processor:\n");
        printf("1. Analyze file\n");
        printf("2. Copy file\n");
        printf("3. Convert to uppercase\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter filename: ");
                scanf("%s", filename1);
                analyze_file(filename1);
                break;

            case 2:
                printf("Enter source filename: ");
                scanf("%s", filename1);
                printf("Enter destination filename: ");
                scanf("%s", filename2);
                copy_file(filename1, filename2);
                break;

            case 3:
                printf("Enter input filename: ");
                scanf("%s", filename1);
                printf("Enter output filename: ");
                scanf("%s", filename2);
                convert_to_uppercase(filename1, filename2);
                break;

            case 4:
```

```
                printf("Exiting program\n");
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Exercise 8.2: Binary File Operations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_RECORDS 100

typedef struct {
    int id;
    char name[50];
    float salary;
} Employee;

// Function to write employee data to binary file
void write_employees(const char *filename, Employee employees[], int count) {
    FILE *file = fopen(filename, "wb");
    if (file == NULL) {
        perror("Error opening file for writing");
        return;
    }

    fwrite(&count, sizeof(int), 1, file);
    fwrite(employees, sizeof(Employee), count, file);

    fclose(file);
    printf("Employee data written to '%s'\n", filename);
}

// Function to read employee data from binary file
int read_employees(const char *filename, Employee employees[]) {
    FILE *file = fopen(filename, "rb");
    if (file == NULL) {
        perror("Error opening file for reading");
```

```
        return 0;
    }

    int count;
    fread(&count, sizeof(int), 1, file);

    if (count > MAX_RECORDS) {
        printf("Too many records in file\n");
        fclose(file);
        return 0;
    }

    fread(employees, sizeof(Employee), count, file);
    fclose(file);

    printf("Employee data read from '%s'\n", filename);
    return count;
}

// Function to add employee
void add_employee(Employee employees[], int *count) {
    if (*count >= MAX_RECORDS) {
        printf("Maximum number of employees reached\n");
        return;
    }

    Employee *emp = &employees[*count];

    printf("Enter employee ID: ");
    scanf("%d", &emp->id);

    printf("Enter employee name: ");
    scanf(" %[^\n]", emp->name);

    printf("Enter employee salary: ");
    scanf("%f", &emp->salary);

    (*count)++;
    printf("Employee added successfully\n");
}

// Function to display all employees
void display_employees(Employee employees[], int count) {
    if (count == 0) {
        printf("No employees in database\n");
        return;
    }
```

```
    printf("\nEmployee Database:\n");
    printf("ID\tName\t\t\tSalary\n");
    printf("---------------------------------------\n");

    for (int i = 0; i < count; i++) {
        Employee *emp = &employees[i];
        printf("%d\t%-20s\t$%.2f\n", emp->id, emp->name, emp->salary);
    }
}

int main() {
    Employee employees[MAX_RECORDS];
    int count = 0;
    int choice;
    char filename[100];

    while (1) {
        printf("\nBinary File Employee Database:\n");
        printf("1. Add Employee\n");
        printf("2. Display Employees\n");
        printf("3. Save to File\n");
        printf("4. Load from File\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                add_employee(employees, &count);
                break;

            case 2:
                display_employees(employees, count);
                break;

            case 3:
                printf("Enter filename: ");
                scanf("%s", filename);
                write_employees(filename, employees, count);
                break;

            case 4:
                printf("Enter filename: ");
                scanf("%s", filename);
                count = read_employees(filename, employees);
                break;
```

```
            case 5:
                printf("Exiting program\n");
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Module 9: Modern C Features

Exercise 9.1: C99 Features Demonstration

```
#include <stdio.h>
#include <stdbool.h>
#include <stdint.h>
#include <inttypes.h>

// Variable-length array example
void vla_example(int size) {
    int arr[size];  // VLA - size determined at runtime

    printf("VLA of size %d:\n", size);
    for (int i = 0; i < size; i++) {
        arr[i] = i * i;
        printf("arr[%d] = %d\n", i, arr[i]);
    }
}

// Inline function example
inline int max(int a, int b) {
    return (a > b) ? a : b;
}

// _Bool example
void bool_example(void) {
    bool flag1 = true;
    bool flag2 = false;

    printf("Boolean values: flag1 = %s, flag2 = %s\n",
           flag1 ? "true" : "false",
```

```
            flag2 ? "true" : "false");

    bool result = flag1 && !flag2;
    printf("flag1 AND NOT flag2 = %s\n", result ? "true" : "false");
}

int main() {
    printf("C99 Features Demonstration\n");
    printf("==========================\n");

    // Variable-length array
    int size;
    printf("Enter size for VLA: ");
    scanf("%d", &size);
    vla_example(size);

    // Inline function
    int a = 15, b = 25;
    printf("Max of %d and %d = %d\n", a, b, max(a, b));

    // Boolean example
    bool_example();

    // Fixed-width integers
    int32_t fixed_int = 123456789;
    uint64_t big_uint = 1234567890123456ULL;

    printf("Fixed-width integers:\n");
    printf("int32_t: %" PRId32 "\n", fixed_int);
    printf("uint64_t: %" PRIu64 "\n", big_uint);

    return 0;
}
```

Exercise 9.2: C11 Features Demonstration

```
#include <stdio.h>
#include <stdlib.h>
#include <stdalign.h>
#include <stdnoreturn.h>
#include <threads.h>
#include <stdatomic.h>

// Static assertion example
#define ARRAY_SIZE 10
```

```c
_Static_assert(ARRAY_SIZE > 0, "Array size must be positive");

// Generic selection example
#define PRINT_GENERIC(x) _Generic((x), \
    int: printf("Integer: %d\n"), \
    float: printf("Float: %f\n"), \
    double: printf("Double: %f\n"), \
    char*: printf("String: %s\n"), \
    default: printf("Unknown type\n") \
)(x)

// Noreturn function example
noreturn void fatal_error(const char *msg) {
    printf("Fatal error: %s\n", msg);
    exit(EXIT_FAILURE);
}

// Alignment example
struct aligned_struct {
    alignas(16) double x;
    int y;
};

// Atomic operations example
atomic_int global_counter = ATOMIC_VAR_INIT(0);

int increment_counter(void *arg) {
    for (int i = 0; i < 1000; i++) {
        atomic_fetch_add(&global_counter, 1);
    }
    return 0;
}

int main() {
    printf("C11 Features Demonstration\n");
    printf("==========================\n");

    // Generic selection
    int i = 42;
    float f = 3.14f;
    double d = 2.718;
    char *s = "Hello, C11!";

    PRINT_GENERIC(i);
    PRINT_GENERIC(f);
    PRINT_GENERIC(d);
    PRINT_GENERIC(s);
```

```
    // Alignment
    printf("Alignment of aligned_struct: %zu\n", alignof(struct aligned_struct));
    printf("Size of aligned_struct: %zu\n", sizeof(struct aligned_struct));

    // Atomic operations with threads
    thrd_t thread1, thread2;

    thrd_create(&thread1, increment_counter, NULL);
    thrd_create(&thread2, increment_counter, NULL);

    thrd_join(thread1, NULL);
    thrd_join(thread2, NULL);

    printf("Final counter value: %d\n", atomic_load(&global_counter));

    // Static assertion (compile-time check)
    int arr[ARRAY_SIZE];
    printf("Array of size %d created successfully\n", ARRAY_SIZE);

    return 0;
}
```

Module 10: Data Structures

Exercise 10.1: Linked List Implementation

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Function to create new node
Node* create_node(int data) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    if (new_node == NULL) {
        printf("Memory allocation failed\n");
        return NULL;
    }
    new_node->data = data;
    new_node->next = NULL;
    return new_node;
```

```
}

// Function to insert at beginning
void insert_at_beginning(Node** head, int data) {
    Node* new_node = create_node(data);
    if (new_node == NULL) return;

    new_node->next = *head;
    *head = new_node;
}

// Function to insert at end
void insert_at_end(Node** head, int data) {
    Node* new_node = create_node(data);
    if (new_node == NULL) return;

    if (*head == NULL) {
        *head = new_node;
        return;
    }

    Node* current = *head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = new_node;
}

// Function to delete node
void delete_node(Node** head, int key) {
    if (*head == NULL) return;

    // If head node holds the key
    if ((*head)->data == key) {
        Node* temp = *head;
        *head = (*head)->next;
        free(temp);
        return;
    }

    // Search for the key to be deleted
    Node* current = *head;
    while (current->next != NULL && current->next->data != key) {
        current = current->next;
    }
```

```
    // If key was not present
    if (current->next == NULL) return;

    // Remove the node
    Node* temp = current->next;
    current->next = current->next->next;
    free(temp);
}

// Function to search for a node
Node* search_node(Node* head, int key) {
    Node* current = head;
    while (current != NULL) {
        if (current->data == key) {
            return current;
        }
        current = current->next;
    }
    return NULL;
}

// Function to display the list
void display_list(Node* head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("List: ");
    Node* current = head;
    while (current != NULL) {
        printf("%d", current->data);
        if (current->next != NULL) printf(" -> ");
        current = current->next;
    }
    printf(" -> NULL\n");
}

// Function to free the entire list
void free_list(Node* head) {
    Node* current = head;
    while (current != NULL) {
        Node* next = current->next;
        free(current);
        current = next;
    }
}
```

```c
int main() {
    Node* head = NULL;
    int choice, data;

    while (1) {
        printf("\nLinked List Operations:\n");
        printf("1. Insert at beginning\n");
        printf("2. Insert at end\n");
        printf("3. Delete node\n");
        printf("4. Search node\n");
        printf("5. Display list\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                insert_at_beginning(&head, data);
                break;

            case 2:
                printf("Enter data: ");
                scanf("%d", &data);
                insert_at_end(&head, data);
                break;

            case 3:
                printf("Enter data to delete: ");
                scanf("%d", &data);
                delete_node(&head, data);
                break;

            case 4:
                printf("Enter data to search: ");
                scanf("%d", &data);
                Node* found = search_node(head, data);
                if (found != NULL) {
                    printf("Node found with data: %d\n", found->data);
                } else {
                    printf("Node not found\n");
                }
                break;

            case 5:
```

```
                display_list(head);
                break;

            case 6:
                free_list(head);
                printf("Exiting program\n");
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Exercise 10.2: Binary Search Tree Implementation

```
#include <stdio.h>
#include <stdlib.h>

typedef struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;

// Function to create new node
TreeNode* create_node(int data) {
    TreeNode* new_node = (TreeNode*)malloc(sizeof(TreeNode));
    if (new_node == NULL) {
        printf("Memory allocation failed\n");
        return NULL;
    }
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}

// Function to insert node
TreeNode* insert_node(TreeNode* root, int data) {
    if (root == NULL) {
        return create_node(data);
    }
```

```
    if (data < root->data) {
        root->left = insert_node(root->left, data);
    } else if (data > root->data) {
        root->right = insert_node(root->right, data);
    }
    // If data is equal, we don't insert (no duplicates)

    return root;
}

// Function for inorder traversal
void inorder_traversal(TreeNode* root) {
    if (root != NULL) {
        inorder_traversal(root->left);
        printf("%d ", root->data);
        inorder_traversal(root->right);
    }
}

// Function for preorder traversal
void preorder_traversal(TreeNode* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder_traversal(root->left);
        preorder_traversal(root->right);
    }
}

// Function for postorder traversal
void postorder_traversal(TreeNode* root) {
    if (root != NULL) {
        postorder_traversal(root->left);
        postorder_traversal(root->right);
        printf("%d ", root->data);
    }
}

// Function to search for a node
TreeNode* search_node(TreeNode* root, int data) {
    if (root == NULL || root->data == data) {
        return root;
    }

    if (data < root->data) {
        return search_node(root->left, data);
    } else {
```

```
        return search_node(root->right, data);
    }
}

// Function to find minimum value node
TreeNode* find_min(TreeNode* root) {
    while (root && root->left != NULL) {
        root = root->left;
    }
    return root;
}

// Function to delete node
TreeNode* delete_node(TreeNode* root, int data) {
    if (root == NULL) return root;

    if (data < root->data) {
        root->left = delete_node(root->left, data);
    } else if (data > root->data) {
        root->right = delete_node(root->right, data);
    } else {
        // Node to be deleted found
        if (root->left == NULL) {
            TreeNode* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            TreeNode* temp = root->left;
            free(root);
            return temp;
        }

        // Node with two children
        TreeNode* temp = find_min(root->right);
        root->data = temp->data;
        root->right = delete_node(root->right, temp->data);
    }
    return root;
}

// Function to free the entire tree
void free_tree(TreeNode* root) {
    if (root != NULL) {
        free_tree(root->left);
        free_tree(root->right);
        free(root);
    }
```

```
}

int main() {
    TreeNode* root = NULL;
    int choice, data;

    while (1) {
        printf("\nBinary Search Tree Operations:\n");
        printf("1. Insert node\n");
        printf("2. Delete node\n");
        printf("3. Search node\n");
        printf("4. Inorder traversal\n");
        printf("5. Preorder traversal\n");
        printf("6. Postorder traversal\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                root = insert_node(root, data);
                break;

            case 2:
                printf("Enter data to delete: ");
                scanf("%d", &data);
                root = delete_node(root, data);
                break;

            case 3:
                printf("Enter data to search: ");
                scanf("%d", &data);
                TreeNode* found = search_node(root, data);
                if (found != NULL) {
                    printf("Node found with data: %d\n", found->data);
                } else {
                    printf("Node not found\n");
                }
                break;

            case 4:
                printf("Inorder traversal: ");
                inorder_traversal(root);
                printf("\n");
                break;
```

```
            case 5:
                printf("Preorder traversal: ");
                preorder_traversal(root);
                printf("\n");
                break;

            case 6:
                printf("Postorder traversal: ");
                postorder_traversal(root);
                printf("\n");
                break;

            case 7:
                free_tree(root);
                printf("Exiting program\n");
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Module 11: Network Programming

Exercise 11.1: Simple TCP Client-Server

```
// Server code (server.c)
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main_server() {
    int server_fd, new_socket;
    struct sockaddr_in address;
```

```
int opt = 1;
int addrlen = sizeof(address);
char buffer[BUFFER_SIZE] = {0};

// Create socket
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Set socket options
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT,
               &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}

address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind socket
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen for connections
if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d\n", PORT);

// Accept connection
if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
                         (socklen_t*)&addrlen)) < 0) {
    perror("accept failed");
    exit(EXIT_FAILURE);
}

// Receive message
int valread = read(new_socket, buffer, BUFFER_SIZE);
printf("Client: %s\n", buffer);

// Send response
```

```
    const char *response = "Hello from server";
    send(new_socket, response, strlen(response), 0);
    printf("Response sent\n");

    close(new_socket);
    close(server_fd);
    return 0;
}

// Client code (client.c)
int main_client() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char *hello = "Hello from client";
    char buffer[1024] = {0};

    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("Socket creation error\n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address from text to binary
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("Invalid address/ Address not supported\n");
        return -1;
    }

    // Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("Connection Failed\n");
        return -1;
    }

    // Send message
    send(sock, hello, strlen(hello), 0);
    printf("Hello message sent\n");

    // Receive response
    int valread = read(sock, buffer, 1024);
    printf("Server: %s\n", buffer);

    close(sock);
    return 0;
```

```
}

int main() {
    int choice;
    printf("1. Run as server\n");
    printf("2. Run as client\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        return main_server();
    } else if (choice == 2) {
        return main_client();
    } else {
        printf("Invalid choice\n");
        return 1;
    }
}
```

Exercise 11.2: HTTP Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

int http_get_request(const char *hostname, const char *path) {
    int sock;
    struct hostent *server;
    struct sockaddr_in serv_addr;
    char request[1024];
    char response[4096];

    // Create socket
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket creation failed");
        return 1;
    }

    // Get server IP address
```

```
    server = gethostbyname(hostname);
    if (server == NULL) {
        fprintf(stderr, "Error, no such host\n");
        close(sock);
        return 1;
    }

    // Set up server address
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    memcpy(server->h_addr, &serv_addr.sin_addr.s_addr, server->h_length);
    serv_addr.sin_port = htons(80);

    // Connect to server
    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("connection failed");
        close(sock);
        return 1;
    }

    // Create HTTP GET request
    snprintf(request, sizeof(request),
             "GET %s HTTP/1.1\r\n"
             "Host: %s\r\n"
             "Connection: close\r\n\r\n",
             path, hostname);

    // Send request
    if (send(sock, request, strlen(request), 0) < 0) {
        perror("send failed");
        close(sock);
        return 1;
    }

    // Receive response
    int bytes_received;
    while ((bytes_received = recv(sock, response, sizeof(response) - 1, 0)) > 0) {
        response[bytes_received] = '\0';
        printf("%s", response);
    }

    close(sock);
    return 0;
}

int main() {
    char hostname[256];
```

```
    char path[256];

    printf("Enter hostname (e.g., httpbin.org): ");
    scanf("%s", hostname);

    printf("Enter path (e.g., /get): ");
    scanf("%s", path);

    return http_get_request(hostname, path);
}
```

Module 12: Embedded Systems Programming

Exercise 12.1: Bit Manipulation Library

```
#include <stdio.h>
#include <stdint.h>

// Bit manipulation macros
#define SET_BIT(reg, bit)       ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit)     ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit)    ((reg) ^= (1U << (bit)))
#define CHECK_BIT(reg, bit)     (((reg) >> (bit)) & 1U)

// Bit field structure for hardware register
typedef struct {
    uint32_t enable    : 1;  // Bit 0
    uint32_t direction : 1;  // Bit 1
    uint32_t reserved  : 2;  // Bits 2-3
    uint32_t mode      : 4;  // Bits 4-7
    uint32_t config    : 24; // Bits 8-31
} gpio_register_t;

// Function implementations
void set_bit(uint32_t *reg, int bit) {
    *reg |= (1U << bit);
}

void clear_bit(uint32_t *reg, int bit) {
    *reg &= ~(1U << bit);
}

void toggle_bit(uint32_t *reg, int bit) {
    *reg ^= (1U << bit);
}
```

```
int check_bit(uint32_t reg, int bit) {
    return (reg >> bit) & 1U;
}

int main() {
    uint32_t test_reg = 0;

    printf("Initial register value: 0x%08X\n", test_reg);

    // Test bit manipulation functions
    set_bit(&test_reg, 5);
    printf("After setting bit 5: 0x%08X\n", test_reg);

    toggle_bit(&test_reg, 5);
    printf("After toggling bit 5: 0x%08X\n", test_reg);

    clear_bit(&test_reg, 5);
    printf("After clearing bit 5: 0x%08X\n", test_reg);

    // Test bit manipulation macros
    SET_BIT(test_reg, 3);
    printf("After setting bit 3 with macro: 0x%08X\n", test_reg);

    if (CHECK_BIT(test_reg, 3)) {
        printf("Bit 3 is set\n");
    }

    // Test bit field structure
    gpio_register_t gpio_reg = {0};
    gpio_reg.enable = 1;
    gpio_reg.direction = 1;
    gpio_reg.mode = 0xF;

    printf("GPIO Register - Enable: %d, Direction: %d, Mode: 0x%X\n",
           gpio_reg.enable, gpio_reg.direction, gpio_reg.mode);

    return 0;
}
```

Exercise 12.2: Memory-Mapped I/O Simulation

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
```

```c
// Memory-mapped I/O simulation
#define GPIO_BASE_ADDR 0x40020000
#define GPIO_DIR_REG   (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x00))
#define GPIO_DATA_REG  (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x04))
#define GPIO_SET_REG   (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x08))
#define GPIO_CLR_REG   (*(volatile uint32_t *)(GPIO_BASE_ADDR + 0x0C))

// GPIO pin definitions
#define GPIO_PIN_0  (1U << 0)
#define GPIO_PIN_1  (1U << 1)
#define GPIO_PIN_2  (1U << 2)
#define GPIO_PIN_3  (1U << 3)

// Simple GPIO simulation structure
typedef struct {
    uint32_t direction;  // 0 = input, 1 = output
    uint32_t data;       // Current pin states
} gpio_sim_t;

// Global GPIO simulation instance
static gpio_sim_t gpio_sim = {0};

// Function to initialize GPIO
void gpio_init(void) {
    gpio_sim.direction = 0;
    gpio_sim.data = 0;
    printf("GPIO initialized\n");
}

// Function to set GPIO direction
void gpio_set_direction(uint32_t pins, int direction) {
    if (direction) {
        gpio_sim.direction |= pins;  // Set as output
    } else {
        gpio_sim.direction &= ~pins; // Set as input
    }

    // Update hardware register
    GPIO_DIR_REG = gpio_sim.direction;
    printf("GPIO direction set: 0x%08X (pins: 0x%08X, direction: %s)\n",
           gpio_sim.direction, pins, direction ? "output" : "input");
}

// Function to write to GPIO pins
void gpio_write(uint32_t pins, int value) {
    if (value) {
```

```
        gpio_sim.data |= pins;
        GPIO_SET_REG = pins;  // Hardware register for setting pins
    } else {
        gpio_sim.data &= ~pins;
        GPIO_CLR_REG = pins;  // Hardware register for clearing pins
    }

    GPIO_DATA_REG = gpio_sim.data;  // Update data register
    printf("GPIO write: pins 0x%08X set to %d\n", pins, value);
}

// Function to read from GPIO pins
uint32_t gpio_read(uint32_t pins) {
    uint32_t value = GPIO_DATA_REG & pins;  // Read from hardware register
    printf("GPIO read: pins 0x%08X = 0x%08X\n", pins, value);
    return value;
}

// Function to simulate external input change
void gpio_simulate_input(uint32_t pins, int value) {
    if (value) {
        gpio_sim.data |= pins;
    } else {
        gpio_sim.data &= ~pins;
    }
    printf("Simulated input change: pins 0x%08X set to %d\n", pins, value);
}

int main() {
    // Initialize GPIO
    gpio_init();

    // Configure pins 0-1 as outputs, pins 2-3 as inputs
    gpio_set_direction(GPIO_PIN_0 | GPIO_PIN_1, 1);  // Output
    gpio_set_direction(GPIO_PIN_2 | GPIO_PIN_3, 0);  // Input

    // Write to output pins
    gpio_write(GPIO_PIN_0, 1);
    gpio_write(GPIO_PIN_1, 0);

    // Simulate input changes
    gpio_simulate_input(GPIO_PIN_2, 1);
    gpio_simulate_input(GPIO_PIN_3, 0);

    // Read input pins
    uint32_t input_value = gpio_read(GPIO_PIN_2 | GPIO_PIN_3);
    printf("Input pins value: 0x%08X\n", input_value);
```

```
    return 0;
}
```

Module 13: Performance Optimization

Exercise 13.1: Algorithm Complexity Analysis

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

// Timing function
double get_time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
}

// O(n²) bubble sort
void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// O(n log n) quick sort
void quick_sort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
```

```
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}

// O(n) linear search
int linear_search(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

// O(log n) binary search
int binary_search(int arr[], int low, int high, int key) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) {
            return mid;
        }
        if (arr[mid] > key) {
            return binary_search(arr, low, mid - 1, key);
        }
        return binary_search(arr, mid + 1, high, key);
    }
    return -1;
}

// Performance testing function
void test_algorithms() {
    const int sizes[] = {1000, 5000, 10000, 20000};
    const int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    for (int s = 0; s < num_sizes; s++) {
        int n = sizes[s];
        int *arr1 = malloc(n * sizeof(int));
        int *arr2 = malloc(n * sizeof(int));
```

```
        // Fill arrays with random data
        for (int i = 0; i < n; i++) {
            arr1[i] = rand() % 10000;
            arr2[i] = arr1[i];
        }

        struct timespec start, end;

        // Test bubble sort
        clock_gettime(CLOCK_MONOTONIC, &start);
        bubble_sort(arr1, n);
        clock_gettime(CLOCK_MONOTONIC, &end);
        double bubble_time = get_time_diff(start, end);

        // Test quick sort
        clock_gettime(CLOCK_MONOTONIC, &start);
        quick_sort(arr2, 0, n - 1);
        clock_gettime(CLOCK_MONOTONIC, &end);
        double quick_time = get_time_diff(start, end);

        printf("Array size: %d\n", n);
        printf("Bubble sort time: %.6f seconds\n", bubble_time);
        printf("Quick sort time: %.6f seconds\n", quick_time);
        printf("Speedup: %.2fx\n\n", bubble_time / quick_time);

        free(arr1);
        free(arr2);
    }
}

int main() {
    srand(time(NULL));
    test_algorithms();
    return 0;
}
```

Exercise 13.2: Memory Access Patterns

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

#define MATRIX_SIZE 1000
#define CACHE_LINE_SIZE 64
```

```
// Matrix structure with proper alignment
typedef struct {
    int rows;
    int cols;
    int *data;
} matrix_t;

// Create matrix with specified alignment
matrix_t* create_matrix(int rows, int cols) {
    matrix_t *mat = malloc(sizeof(matrix_t));
    mat->rows = rows;
    mat->cols = cols;
    // Allocate aligned memory for better cache performance
    posix_memalign((void**)&mat->data, CACHE_LINE_SIZE, rows * cols * sizeof(int));
    return mat;
}

void free_matrix(matrix_t *mat) {
    free(mat->data);
    free(mat);
}

// Initialize matrix with random values
void init_matrix(matrix_t *mat) {
    for (int i = 0; i < mat->rows * mat->cols; i++) {
        mat->data[i] = rand() % 100;
    }
}

// Matrix multiplication - ijk order (poor cache performance)
void matrix_multiply_ijk(matrix_t *a, matrix_t *b, matrix_t *c) {
    for (int i = 0; i < a->rows; i++) {
        for (int j = 0; j < b->cols; j++) {
            c->data[i * c->cols + j] = 0;
            for (int k = 0; k < a->cols; k++) {
                c->data[i * c->cols + j] +=
                    a->data[i * a->cols + k] * b->data[k * b->cols + j];
            }
        }
    }
}

// Matrix multiplication - ikj order (better cache performance)
void matrix_multiply_ikj(matrix_t *a, matrix_t *b, matrix_t *c) {
    // Initialize result matrix
    memset(c->data, 0, c->rows * c->cols * sizeof(int));
```

```
    for (int i = 0; i < a->rows; i++) {
        for (int k = 0; k < a->cols; k++) {
            int temp = a->data[i * a->cols + k];
            for (int j = 0; j < b->cols; j++) {
                c->data[i * c->cols + j] += temp * b->data[k * b->cols + j];
            }
        }
    }
}

// Performance comparison
void compare_matrix_multiplication() {
    matrix_t *a = create_matrix(MATRIX_SIZE, MATRIX_SIZE);
    matrix_t *b = create_matrix(MATRIX_SIZE, MATRIX_SIZE);
    matrix_t *c1 = create_matrix(MATRIX_SIZE, MATRIX_SIZE);
    matrix_t *c2 = create_matrix(MATRIX_SIZE, MATRIX_SIZE);

    init_matrix(a);
    init_matrix(b);

    struct timespec start, end;

    // Test ijk order
    clock_gettime(CLOCK_MONOTONIC, &start);
    matrix_multiply_ijk(a, b, c1);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ijk = get_time_diff(start, end);

    // Test ikj order
    clock_gettime(CLOCK_MONOTONIC, &start);
    matrix_multiply_ikj(a, b, c2);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ikj = get_time_diff(start, end);

    printf("Matrix multiplication performance comparison:\n");
    printf("Matrix size: %dx%d\n", MATRIX_SIZE, MATRIX_SIZE);
    printf("ijk order time: %.6f seconds\n", time_ijk);
    printf("ikj order time: %.6f seconds\n", time_ikj);
    printf("Performance improvement: %.2fx\n", time_ijk / time_ikj);

    // Verify results are the same
    int diff_count = 0;
    for (int i = 0; i < c1->rows * c1->cols; i++) {
        if (c1->data[i] != c2->data[i]) {
            diff_count++;
        }
```

```
    }
    printf("Results match: %s\n", diff_count == 0 ? "Yes" : "No");

    free_matrix(a);
    free_matrix(b);
    free_matrix(c1);
    free_matrix(c2);
}

double get_time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
}

int main() {
    srand(time(NULL));
    compare_matrix_multiplication();
    return 0;
}
```

Module 14: Testing and Quality Assurance

Exercise 14.1: Unit Testing Framework

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

// Simple unit testing framework

// Test result structure
typedef struct {
    int passed;
    int failed;
    int skipped;
    double total_time;
} test_result_t;

// Global test result tracker
static test_result_t g_test_result = {0};

// Assertion functions
#define ASSERT_TRUE(condition) \
    do { \
        if (!(condition)) { \
```

```
            printf("ASSERT_TRUE failed at %s:%d\n", __FILE__, __LINE__); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

#define ASSERT_FALSE(condition) \
    do { \
        if (condition) { \
            printf("ASSERT_FALSE failed at %s:%d\n", __FILE__, __LINE__); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

#define ASSERT_EQUAL(expected, actual) \
    do { \
        if ((expected) != (actual)) { \
            printf("ASSERT_EQUAL failed at %s:%d: expected %d, got %d\n", \
                   __FILE__, __LINE__, (expected), (actual)); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

#define ASSERT_STRING_EQUAL(expected, actual) \
    do { \
        if (strcmp((expected), (actual)) != 0) { \
            printf("ASSERT_STRING_EQUAL failed at %s:%d: expected '%s', got '%s'\n", \
                   __FILE__, __LINE__, (expected), (actual)); \
            g_test_result.failed++; \
            return; \
        } \
        g_test_result.passed++; \
    } while(0)

// Test function type
typedef void (*test_func_t)(void);

// Test case structure
typedef struct {
    const char *name;
    test_func_t function;
} test_case_t;
```

```
// Example functions to test
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

const char* get_hello_message(void) {
    return "Hello, World!";
}

// Test cases
void test_add_positive(void) {
    ASSERT_EQUAL(5, add(2, 3));
}

void test_add_negative(void) {
    ASSERT_EQUAL(-1, add(2, -3));
}

void test_subtract(void) {
    ASSERT_EQUAL(2, subtract(5, 3));
}

void test_string_message(void) {
    ASSERT_STRING_EQUAL("Hello, World!", get_hello_message());
}

// Test suite
test_case_t test_suite[] = {
    {"test_add_positive", test_add_positive},
    {"test_add_negative", test_add_negative},
    {"test_subtract", test_subtract},
    {"test_string_message", test_string_message},
    {NULL, NULL}  // Sentinel
};

// Test runner
void run_tests(void) {
    printf("Running tests...\n");

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);
```

```
    for (int i = 0; test_suite[i].name != NULL; i++) {
        printf("Running %s... ", test_suite[i].name);

        // Reset test counters for each test
        int initial_passed = g_test_result.passed;
        int initial_failed = g_test_result.failed;

        test_suite[i].function();

        if (g_test_result.failed > initial_failed) {
            printf("FAILED\n");
        } else {
            printf("PASSED\n");
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    g_test_result.total_time = (end.tv_sec - start.tv_sec) +
                               (end.tv_nsec - start.tv_nsec) / 1e9;

    // Print summary
    printf("\nTest Results:\n");
    printf("Passed: %d\n", g_test_result.passed);
    printf("Failed: %d\n", g_test_result.failed);
    printf("Skipped: %d\n", g_test_result.skipped);
    printf("Total time: %.6f seconds\n", g_test_result.total_time);

    if (g_test_result.failed == 0) {
        printf("All tests passed!\n");
    } else {
        printf("Some tests failed!\n");
    }
}

int main() {
    run_tests();
    return g_test_result.failed > 0 ? 1 : 0;
}
```

Exercise 14.2: Mocking Framework

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
// Mocking framework example

// Real database interface
typedef struct {
    int (*connect)(const char *host, int port);
    int (*query)(const char *sql, char **result);
    void (*disconnect)(void);
} database_interface_t;

// Real database implementation
static int real_db_connect(const char *host, int port) {
    printf("Connecting to database at %s:%d\n", host, port);
    return 1; // Success
}

static int real_db_query(const char *sql, char **result) {
    printf("Executing query: %s\n", sql);
    *result = strdup("Mock query result");
    return 1; // Success
}

static void real_db_disconnect(void) {
    printf("Disconnecting from database\n");
}

database_interface_t real_database = {
    .connect = real_db_connect,
    .query = real_db_query,
    .disconnect = real_db_disconnect
};

// Mock database implementation
static int mock_connect_calls = 0;
static int mock_query_calls = 0;
static int mock_disconnect_calls = 0;
static int mock_connect_result = 1;
static int mock_query_result = 1;

static int mock_db_connect(const char *host, int port) {
    mock_connect_calls++;
    printf("Mock connect called (%d times)\n", mock_connect_calls);
    return mock_connect_result;
}

static int mock_db_query(const char *sql, char **result) {
    mock_query_calls++;
```

```
    printf("Mock query called (%d times): %s\n", mock_query_calls, sql);
    *result = strdup("Mock result");
    return mock_query_result;
}

static void mock_db_disconnect(void) {
    mock_disconnect_calls++;
    printf("Mock disconnect called (%d times)\n", mock_disconnect_calls);
}

database_interface_t mock_database = {
    .connect = mock_db_connect,
    .query = mock_db_query,
    .disconnect = mock_db_disconnect
};

// Function under test
int fetch_user_data(database_interface_t *db, const char *user_id, char **data) {
    if (!db->connect("localhost", 5432)) {
        return 0;
    }

    char sql[256];
    snprintf(sql, sizeof(sql), "SELECT * FROM users WHERE id = '%s'", user_id);

    int result = db->query(sql, data);
    db->disconnect();

    return result;
}

// Test with mock
void test_fetch_user_data_success(void) {
    // Reset mock counters
    mock_connect_calls = 0;
    mock_query_calls = 0;
    mock_disconnect_calls = 0;
    mock_connect_result = 1;
    mock_query_result = 1;

    char *data = NULL;
    int result = fetch_user_data(&mock_database, "123", &data);

    // Verify results
    if (result != 1) {
        printf("Test failed: Expected success, got failure\n");
        return;
```

```
    }

    if (mock_connect_calls != 1) {
        printf("Test failed: Expected 1 connect call, got %d\n", mock_connect_calls);
        return;
    }

    if (mock_query_calls != 1) {
        printf("Test failed: Expected 1 query call, got %d\n", mock_query_calls);
        return;
    }

    if (mock_disconnect_calls != 1) {
        printf("Test failed: Expected 1 disconnect call, got %d\n", mock_disconnect_calls);
        return;
    }

    printf("Test passed: All mock interactions verified\n");
    free(data);
}

void test_fetch_user_data_connection_failure(void) {
    // Reset mock counters
    mock_connect_calls = 0;
    mock_query_calls = 0;
    mock_disconnect_calls = 0;
    mock_connect_result = 0; // Simulate connection failure

    char *data = NULL;
    int result = fetch_user_data(&mock_database, "123", &data);

    // Verify results
    if (result != 0) {
        printf("Test failed: Expected failure, got success\n");
        return;
    }

    if (mock_connect_calls != 1) {
        printf("Test failed: Expected 1 connect call, got %d\n", mock_connect_calls);
        return;
    }

    // Should not call query or disconnect on connection failure
    if (mock_query_calls != 0) {
        printf("Test failed: Expected 0 query calls, got %d\n", mock_query_calls);
        return;
    }
```

```
    if (mock_disconnect_calls != 0) {
        printf("Test failed: Expected 0 disconnect calls, got %d\n", mock_disconnect_calls);
        return;
    }

    printf("Test passed: Connection failure handled correctly\n");
}

int main() {
    printf("Testing with mock database...\n");
    test_fetch_user_data_success();
    test_fetch_user_data_connection_failure();

    printf("\nFor comparison, running with real database:\n");
    char *data = NULL;
    fetch_user_data(&real_database, "123", &data);
    free(data);

    return 0;
}
```

Module 15: Advanced Topics

Exercise 15.1: Multithreading Example

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

// Thread function
void* thread_function(void* arg) {
    int thread_id = *(int*)arg;
    printf("Hello from thread %d\n", thread_id);

    // Simulate some work
    sleep(1);

    printf("Thread %d finishing\n", thread_id);
    return NULL;
}

int main() {
    pthread_t thread1, thread2;
```

```
    int id1 = 1, id2 = 2;

    // Create threads
    if (pthread_create(&thread1, NULL, thread_function, &id1) != 0) {
        perror("pthread_create failed");
        exit(EXIT_FAILURE);
    }

    if (pthread_create(&thread2, NULL, thread_function, &id2) != 0) {
        perror("pthread_create failed");
        exit(EXIT_FAILURE);
    }

    // Wait for threads to complete
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("All threads completed\n");
    return 0;
}
```

Exercise 15.2: System Programming Example

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // Child process
        printf("Child process (PID: %d)\n", getpid());
        exit(EXIT_SUCCESS);
    } else {
        // Parent process
        printf("Parent process (PID: %d), Child PID: %d\n", getpid(), pid);
        wait(NULL);  // Wait for child to complete
        printf("Child process completed\n");
    }
```

```
    return 0;
}
```

This appendix provides solutions to selected exercises from each module of the course. These solutions demonstrate good programming practices and illustrate the concepts covered in each module. Remember that there are often multiple ways to solve a problem, and these solutions are meant to serve as examples rather than the only correct approach.